

AlgoMaster 600

**Priority-Grouped · Category-Ordered · 2×1
Landscape**

**Python Only · Priority-Grouped ·
Difficulty-First · 2×1 Landscape**

**600 questions · 9 rounds · Inline Quick Review
· Chapter 2 Summary**

**High Easy → High Med → High Hard → Mid Easy
→ Mid Med → Mid Hard → Low Easy → Low Med
→ Low Hard**

Contents

★ = LeetCode Premium (Premium 98 list)

Round 1 | High | E Easy (68)

Arrays (9)

- ☐ ● #26 Remove Duplicates from Sorted Array ↗
- ☐ ● #27 Remove Element ↗
- ☐ ● #121 Best Time to Buy and Sell Stock ↗
- ☐ ● ★ #163 Missing Ranges ↗
- ☐ ● #169 Majority Element ↗
- ☐ ● #283 Move Zeroes ↗
- ☐ ● #414 Third Maximum Number ↗
- ☐ ● #485 Max Consecutive Ones ↗
- ☐ ● #1470 Shuffle the Array ↗

Strings (10)

- ☐ ● #14 Longest Common Prefix ↗
- ☐ ● #28 Find the Index of the First Occurrence in a String ↗
- ☐ ● #58 Length of Last Word ↗

p.3

- ☐ ● #125 Valid Palindrome ↗

- ☐ ● #242 Valid Anagram ↗

- ☐ ● #344 Reverse String ↗

- ☐ ● #392 Is Subsequence ↗

- ☐ ● #409 Longest Palindrome ↗

- ☐ ● #680 Valid Palindrome II ↗

- ☐ ● #796 Rotate String ↗

Hash Tables (11)

- ☐ ● #205 Isomorphic Strings ↗

- ☐ ● #217 Contains Duplicate ↗

- ☐ ● #219 Contains Duplicate II ↗

- ☐ ● #290 Word Pattern ↗

- ☐ ● #350 Intersection of Two Arrays II ↗

- ☐ ● #383 Ransom Note ↗

- ☐ ● #387 First Unique Character in a String ↗

- ☐ ● #448 Find All Numbers Disappeared in an Array ↗

- ☐ ● #706 Design HashMap ↗

p.4

☐ ● #1189 Maximum Number of Balloons ↗

☐ ● #1512 Number of Good Pairs ↗

Two Pointers (7) ↗

☐ ● #1 Two Sum ↗

☐ ● #88 Merge Sorted Array ↗

☐ ● ★ #408 Valid Word Abbreviation ↗

☐ ● #696 Count Binary Substrings ↗

☐ ● #844 Backspace String Compare ↗

☐ ● #977 Squares of a Sorted Array ↗

☐ ● #1768 Merge Strings Alternately ↗

Sliding Window – Fixed Size (1) ↗

☐ ● #643 Maximum Average Subarray I ↗

Binary Search (6) ↗

☐ ● #35 Search Insert Position ↗

☐ ● #278 First Bad Version ↗

☐ ● #367 Valid Perfect Square ↗

☐ ● #374 Guess Number Higher or Lower ↗

☐ ● #704 Binary Search ↗

p.5

☐ ● #744 Find Smallest Letter Greater Than Target ↗

Stacks (4) ↗

☐ ● #20 Valid Parentheses ↗

☐ ● #682 Baseball Game ↗

☐ ● #1047 Remove All Adjacent Duplicates In String ↗

☐ ● #1614 Maximum Nesting Depth of the Parentheses ↗

Tree Traversal – Level Order (2) ↗

☐ ● #104 Maximum Depth of Binary Tree ↗

☐ ● #637 Average of Levels in Binary Tree ↗

Tree Traversal – Pre Order (8) ↗

☐ ● #100 Same Tree ↗

☐ ● #101 Symmetric Tree ↗

☐ ● #108 Convert Sorted Array to Binary Search Tree ↗

☐ ● #144 Binary Tree Preorder Traversal ↗

☐ ● #222 Count Complete Tree Nodes ↗

p.6

☐ ● #257 Binary Tree Paths ↗

☐ ● #404 Sum of Left Leaves ↗

☐ ● #572 Subtree of Another Tree ↗

Tree Traversal – In Order (3) ↗

☐ ● #94 Binary Tree Inorder Traversal ↗

☐ ● #530 Minimum Absolute Difference in BST ↗

☐ ● #783 Minimum Distance Between BST Nodes ↗

Tree Traversal – Post-Order (4) ↗

☐ ● #110 Balanced Binary Tree ↗

☐ ● #145 Binary Tree Postorder Traversal ↗

☐ ● #226 Invert Binary Tree ↗

☐ ● #543 Diameter of Binary Tree ↗

Depth First Search (DFS) (1) ↗

☐ ● #733 Flood Fill ↗

Linked List (2) ↗

☐ ● #83 Remove Duplicates from Sorted List ↗

☐ ● #160 Intersection of Two Linked Lists ↗

p.7

Round 2 | High | M Medium (136) ↗

Arrays (8) ↗

☐ ● #31 Next Permutation ↗

☐ ● #80 Remove Duplicates from Sorted Array II ↗

☐ ● #122 Best Time to Buy and Sell Stock II ↗

☐ ● #189 Rotate Array ↗

☐ ● #229 Majority Element II ↗

☐ ● #238 Product of Array Except Self ↗

☐ ● #334 Increasing Triplet Subsequence ↗

☐ ● #2348 Number of Zero-Filled Subarrays ↗

Strings (6) ↗

☐ ● #6 Zigzag Conversion ↗

☐ ● #38 Count and Say ↗

☐ ● #151 Reverse Words in a String ↗

☐ ● ★ #161 One Edit Distance ↗

☐ ● ★ #616 Add Bold Tag in String ↗

☐ ● #1657 Determine if Two Strings Are Close ↗

p.8

Hash Tables (9) ↗

- ☐ ● #49 Group Anagrams ↗
- ☐ ● #128 Longest Consecutive Sequence ↗
- ☐ ● ★ #249 Group Shifted Strings ↗
- ☐ ● #535 Encode and Decode TinyURL ↗
- ☐ ● #659 Split Array into Consecutive Subsequences ↗
- ☐ ● #767 Reorganize String ↗
- ☐ ● #792 Number of Matching Subsequences ↗
- ☐ ● #1525 Number of Good Ways to Split a String ↗
- ☐ ● #1647 Minimum Deletions to Make Character Frequencies Unique ↗

Two Pointers (7) ↗

- ☐ ● #5 Longest Palindromic Substring ↗
- ☐ ● #11 Container With Most Water ↗
- ☐ ● #15 3Sum ↗
- ☐ ● #18 4Sum ↗
- ☐ ● #167 Two Sum II - Input Array Is Sorted ↗

p.9

- ☐ ● #443 String Compression ↗

- ☐ ● #861 Boats to Save People ↗

Sliding Window - Fixed Size (4) ↗

- ☐ ● #438 Find All Anagrams in a String ↗
- ☐ ● #567 Permutation in String ↗
- ☐ ● #1456 Maximum Number of Vowels in a Substring of Given Length ↗

- ☐ ● #2461 Maximum Sum of Distinct Subarrays With Length K ↗

Sliding Window - Dynamic Size (12) ↗

- ☐ ● #3 Longest Substring Without Repeating Characters ↗
- ☐ ● ★ #159 Longest Substring with At Most Two Distinct Characters ↗
- ☐ ● #209 Minimum Size Subarray Sum ↗
- ☐ ● ★ #340 Longest Substring with At Most K Distinct Characters ↗
- ☐ ● #395 Longest Substring with At Least K Repeating Characters ↗

p.10

- ☐ ● #424 Longest Repeating Character Replacement ↗
- ☐ ● #713 Subarray Product Less Than K ↗
- ☐ ● #904 Fruit Into Baskets ↗
- ☐ ● #1004 Max Consecutive Ones III ↗
- ☐ ● #1248 Count Number of Nice Subarrays ↗
- ☐ ● #1423 Maximum Points You Can Obtain from Cards ↗
- ☐ ● #1838 Frequency of the Most Frequent Element ↗
- Binary Search (14) ↗
- ☐ ● #33 Search in Rotated Sorted Array ↗
- ☐ ● #34 Find First and Last Position of Element in Sorted Array ↗
- ☐ ● #74 Search a 2D Matrix ↗
- ☐ ● #81 Search in Rotated Sorted Array II ↗
- ☐ ● #153 Find Minimum in Rotated Sorted Array ↗
- ☐ ● #162 Find Peak Element ↗

p.11

- ☐ ● #240 Search a 2D Matrix II ↗
- ☐ ● #475 Heaters ↗
- ☐ ● #528 Random Pick with Weight ↗
- ☐ ● #658 Find K Closest Elements ↗
- ☐ ● #875 Koko Eating Bananas ↗
- ☐ ● #1011 Capacity to Ship Packages Within D Days ↗
- ☐ ● #1482 Minimum Number of Days to Make m Bouquets ↗
- ☐ ● #1552 Magnetic Force Between Two Balls ↗
- Stacks (12) ↗
- ☐ ● #71 Simplify Path ↗
- ☐ ● #150 Evaluate Reverse Polish Notation ↗
- ☐ ● #155 Min Stack ↗
- ☐ ● #227 Basic Calculator II ↗
- ☐ ● #316 Remove Duplicate Letters ↗
- ☐ ● #636 Exclusive Time of Functions ↗
- ☐ ● #678 Valid Parenthesis String ↗

p.12

- ☐ ● #735 Asteroid Collision ↗
- ☐ ● #853 Car Fleet ↗
- ☐ ● #946 Validate Stack Sequences ↗
- ☐ ● #1249 Minimum Remove to Make Valid Parentheses ↗
- ☐ ● #2390 Removing Stars From a String ↗
- Tree Traversal – Level Order (8) ↗**
- ☐ ● #102 Binary Tree Level Order Traversal ↗
- ☐ ● #103 Binary Tree Zigzag Level Order Traversal ↗
- ☐ ● #116 Populating Next Right Pointers in Each Node ↗
- ☐ ● #117 Populating Next Right Pointers in Each Node II ↗
- ☐ ● #199 Binary Tree Right Side View ↗
- ☐ ● ★ #314 Binary Tree Vertical Order Traversal ↗
- ☐ ● #662 Maximum Width of Binary Tree ↗
- ☐ ● #958 Check Completeness of a Binary Tree ↗

p.13

- Tree Traversal – Pre Order (6) ↗**
- ☐ ● #105 Construct Binary Tree from Preorder and Inorder Traversal ↗
- ☐ ● #106 Construct Binary Tree from Inorder and Postorder Traversal ↗
- ☐ ● #113 Path Sum II ↗
- ☐ ● #437 Path Sum III ↗
- ☐ ● #687 Longest Univalue Path ↗
- ☐ ● #1026 Maximum Difference Between Node and Ancestor ↗
- Tree Traversal – In Order (4) ↗**
- ☐ ● #98 Validate Binary Search Tree ↗
- ☐ ● #173 Binary Search Tree Iterator ↗
- ☐ ● #230 Kth Smallest Element in a BST ↗
- ☐ ● #1382 Balance a Binary Search Tree ↗
- Tree Traversal – Post-Order (9) ↗**
- ☐ ● #114 Flatten Binary Tree to Linked List ↗
- ☐ ● #129 Sum Root to Leaf Numbers ↗

p.14

☐ ● #236 Lowest Common Ancestor of a Binary Tree ↗

☐ ● ★ #545 Boundary of Binary Tree ↗

☐ ● #652 Find Duplicate Subtrees ↗

☐ ● #979 Distribute Coins in Binary Tree ↗

☐ ● #1110 Delete Nodes And Return Forest ↗

☐ ● #1448 Count Good Nodes in Binary Tree ↗

☐ ● #2096 Step-By-Step Directions From a Binary Tree Node to Another

Depth First Search (DFS) (12) ↗

☐ ● #79 Word Search ↗

☐ ● #130 Surrounded Regions ↗

☐ ● #133 Clone Graph ↗

☐ ● #200 Number of Islands ↗

☐ ● #417 Pacific Atlantic Water Flow ↗

☐ ● #690 Employee Importance ↗

☐ ● ★ #694 Number of Distinct Islands ↗

☐ ● #785 Is Graph Bipartite? ↗

p.15

☐ ● #797 All Paths From Source to Target ↗

☐ ● #863 All Nodes Distance K in Binary Tree ↗

☐ ● #1020 Number of Enclaves ↗

☐ ● #1376 Time Needed to Inform All Employees

Breadth First Search (BFS) (9) ↗

☐ ● ★ #286 Walls and Gates ↗

☐ ● #433 Minimum Genetic Mutation ↗

☐ ● #542 01 Matrix ↗

☐ ● #752 Open the Lock ↗

☐ ● #909 Snakes and Ladders ↗

☐ ● #994 Rotting Oranges ↗

☐ ● #1091 Shortest Path in Binary Matrix ↗

☐ ● #1102 Path With Maximum Minimum Value ↗

☐ ● ★ #1197 Minimum Knight Moves ↗

Linked List (16) ↗

☐ ● #2 Add Two Numbers ↗

☐ ● #19 Remove Nth Node From End of List ↗

p.16

- ☐ ● #24 Swap Nodes in Pairs ↗
- ☐ ● #61 Rotate List ↗
- ☐ ● #82 Remove Duplicates from Sorted List II ↗
- ☐ ● #86 Partition List ↗
- ☐ ● #138 Copy List with Random Pointer ↗
- ☐ ● #143 Reorder List ↗
- ☐ ● #237 Delete Node in a Linked List ↗
- ☐ ● #328 Odd Even Linked List ↗
- ☐ ● ★ #430 Flatten a Multilevel Doubly Linked List ↗
- ☐ ● #445 Add Two Numbers II ↗
- ☐ ● #707 Design Linked List ↗
- ☐ ● ★ #708 Insert into a Sorted Circular Linked List ↗
- ☐ ● #1669 Merge In Between Linked Lists ↗
- ☐ ● #2095 Delete the Middle Node of a Linked List ↗

Round 3 | High | H Hard (24) ↗

Arrays (1) ↗

p.17

- ☐ ● #41 First Missing Positive ↗
- Strings (2) ↗**
- ☐ ● #68 Text Justification ↗
- ☐ ● #843 Guess the Word ↗
- Two Pointers (2) ↗**
- ☐ ● #42 Trapping Rain Water ↗
- ☐ ● #2444 Count Subarrays With Fixed Bounds ↗
- Sliding Window – Fixed Size (1) ↗**
- ☐ ● #30 Substring with Concatenation of All Words ↗
- Sliding Window – Dynamic Size (3) ↗**
- ☐ ● #76 Minimum Window Substring ↗
- ☐ ● #727 Minimum Window Subsequence ↗
- ☐ ● #992 Subarrays with K Different Integers ↗
- Binary Search (4) ↗**
- ☐ ● #4 Median of Two Sorted Arrays ↗
- ☐ ● #410 Split Array Largest Sum ↗
- ☐ ● #719 Find K-th Smallest Pair Distance ↗
- ☐ ● #1095 Find in Mountain Array ↗

p.18

Stacks (2) ↗

☐ ● #32 Longest Valid Parentheses ↗

☐ ● #224 Basic Calculator ↗

Tree Traversal – Pre Order (1) ↗

☐ ● #297 Serialize and Deserialize Binary Tree ↗

Tree Traversal – Post-Order (1) ↗

☐ ● #124 Binary Tree Maximum Path Sum ↗

Depth First Search (DFS) (3) ↗

☐ ● #827 Making A Large Island ↗

☐ ● #1192 Critical Connections in a Network ↗

☐ ● #2246 Longest Path With Different Adjacent Characters ↗

Breadth First Search (BFS) (4) ↗

☐ ● #127 Word Ladder ↗

☐ ● #815 Bus Routes ↗

☐ ● #1293 Shortest Path in a Grid with Obstacles Elimination ↗

☐ ● #2290 Minimum Obstacle Removal to Reach Corner ↗

p.19

Round 4 | Mid | E Easy (38) ↗

Bit Manipulation (8) ↗

☐ ● #136 Single Number ↗

☐ ● #190 Reverse Bits ↗

☐ ● #191 Number of 1 Bits ↗

☐ ● #231 Power of Two ↗

☐ ● #268 Missing Number ↗

☐ ● #338 Counting Bits ↗

☐ ● #461 Hamming Distance ↗

☐ ● #645 Set Mismatch ↗

Prefix Sum (4) ↗

☐ ● #303 Range Sum Query – Immutable ↗

☐ ● #724 Find Pivot Index ↗

☐ ● #1480 Running Sum of 1d Array ↗

☐ ● #1854 Maximum Population Year ↗

Matrix (2D Array) (1) ↗

☐ ● #867 Transpose Matrix ↗

LinkedList In-place Reversal (2) ↗

p.20

- ☐ ● #206 Reverse Linked List ↗
- ☐ ● #234 Palindrome Linked List ↗
- Fast and Slow Pointers (3) ↗
- ☐ ● #141 Linked List Cycle ↗
- ☐ ● #202 Happy Number ↗
- ☐ ● #876 Middle of the Linked List ↗
- Monotonic Stack (2) ↗
- ☐ ● #496 Next Greater Element I ↗
- ☐ ● #1475 Final Prices With a Special Discount in a Shop ↗
- Queues (3) ↗
- ☐ ● ★ #346 Moving Average from Data Stream ↗
- ☐ ● #933 Number of Recent Calls ↗
- ☐ ● #2073 Time Needed to Buy Tickets ↗
- Recursion (1) ↗
- ☐ ● #21 Merge Two Sorted Lists ↗
- Divide and Conquer (1) ↗
- ☐ ● #1763 Longest Nice Substring ↗
- BST / Ordered Set (4) ↗

p.21

- ☐ ● ★ #270 Closest Binary Search Tree Value ↗
- ☐ ● #653 Two Sum IV – Input is a BST ↗
- ☐ ● #700 Search in a Binary Search Tree ↗
- ☐ ● #938 Range Sum of BST ↗
- Heaps (1) ↗
- ☐ ● #1046 Last Stone Weight ↗
- Top K Elements (1) ↗
- ☐ ● #703 Kth Largest Element in a Stream ↗
- Intervals (2) ↗
- ☐ ● #228 Summary Ranges ↗
- ☐ ● ★ #252 Meeting Rooms ↗
- Data Structure Design (3) ↗
- ☐ ● #225 Implement Stack using Queues ↗
- ☐ ● #232 Implement Queue using Stacks ↗
- ☐ ● #359 Logger Rate Limiter ↗
- Greedy (2) ↗
- ☐ ● #455 Assign Cookies ↗
- ☐ ● #3074 Apple Redistribution into Boxes ↗

p.22

Round 5 | Mid | M Medium (160)

Bit Manipulation (4)

- ☐ #137 Single Number II
- ☐ #201 Bitwise AND of Numbers Range
- ☐ #260 Single Number III
- ☐ #371 Sum of Two Integers

Prefix Sum (8)

- ☐ #304 Range Sum Query 2D - Immutable
- ☐ #370 Range Addition
- ☐ #523 Continuous Subarray Sum
- ☐ #525 Contiguous Array
- ☐ #560 Subarray Sum Equals K
- ☐ #974 Subarray Sums Divisible by K
- ☐ #1314 Matrix Block Sum
- ☐ #2536 Increment Submatrices by One

Kadane's Algorithm (7)

- ☐ #53 Maximum Subarray
- ☐ #152 Maximum Product Subarray

p.23

- ☐ #918 Maximum Sum Circular Subarray
- ☐ #978 Longest Turbulent Subarray
- ☐ #1014 Best Sightseeing Pair
- ☐ #1186 Maximum Subarray Sum with One Deletion
- ☐ #1749 Maximum Absolute Sum of Any Subarray

Matrix (2D Array) (8)

- ☐ #36 Valid Sudoku
- ☐ #48 Rotate Image
- ☐ #54 Spiral Matrix
- ☐ #59 Spiral Matrix II
- ☐ #73 Set Matrix Zeroes
- ☐ #289 Game of Life
- ☐ #498 Diagonal Traverse
- ☐ ★ #723 Candy Crush

LinkedList In-place Reversal (1)

- ☐ #92 Reverse Linked List II

Monotonic Stack (12)

p.24

- ☐ ● #402 Remove K Digits ↗
- ☐ ● #456 132 Pattern ↗
- ☐ ● #503 Next Greater Element II ↗
- ☐ ● #581 Shortest Unsorted Continuous Subarray ↗
- ☐ ● #739 Daily Temperatures ↗
- ☐ ● #769 Max Chunks To Make Sorted ↗
- ☐ ● #901 Online Stock Span ↗
- ☐ ● #907 Sum of Subarray Minimums ↗
- ☐ ● #962 Maximum Width Ramp ↗
- ☐ ● #1019 Next Greater Node In Linked List ↗
- ☐ ● ★ #1762 Buildings With an Ocean View ↗
- ☐ ● #2104 Sum of Subarray Ranges ↗
- Queues (4) ↗**
- ☐ ● #950 Reveal Cards In Increasing Order ↗
- ☐ ● ★ #1429 First Unique Number ↗
- ☐ ● #1823 Find the Winner of the Circular Game ↗

p.25

- ☐ ● #2327 Number of People Aware of a Secret ↗
- Monotonic Queue (5) ↗**
- ☐ ● #1438 Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit ↗
- ☐ ● #1673 Find the Most Competitive Subsequence ↗
- ☐ ● #1696 Jump Game VI ↗
- ☐ ● #2762 Continuous Subarrays ↗
- ☐ ● #3578 Count Partitions With Max-Min Difference at Most K ↗
- Recursion (2) ↗**
- ☐ ● #50 Pow(x, n) ↗
- ☐ ● #394 Decode String ↗
- Divide and Conquer (3) ↗**
- ☐ ● #109 Convert Sorted List to Binary Search Tree ↗
- ☐ ● #427 Construct Quad Tree ↗
- ☐ ● #654 Maximum Binary Tree ↗
- Merge Sort (2) ↗**

p.26

- ☐ **#148 Sort List** ↗
- ☐ **#912 Sort an Array** ↗
- QuickSort / QuickSelect (3)** ↗
- ☐ **#75 Sort Colors** ↗
- ☐ **#215 Kth Largest Element in an Array** ↗
- ☐ **#1985 Find the Kth Largest Integer in the Array** ↗
- Backtracking (12)** ↗
- ☐ **#17 Letter Combinations of a Phone Number** ↗
- ☐ **#22 Generate Parentheses** ↗
- ☐ **#39 Combination Sum** ↗
- ☐ **#40 Combination Sum II** ↗
- ☐ **#46 Permutations** ↗
- ☐ **#47 Permutations II** ↗
- ☐ **#77 Combinations** ↗
- ☐ **#78 Subsets** ↗
- ☐ **#90 Subsets II** ↗
- ☐ **#93 Restore IP Addresses** ↗

p.27

- ☐ **#131 Palindrome Partitioning** ↗
- ☐ **#216 Combination Sum III** ↗
- BST / Ordered Set (13)** ↗
- ☐ **#99 Recover Binary Search Tree** ↗
- ☐ **#235 Lowest Common Ancestor of a Binary Search Tree** ↗
- ☐ **#285 Inorder Successor in BST** ↗
- ☐ **★ #333 Largest BST Subtree** ↗
- ☐ **#426 Convert Binary Search Tree to Sorted Doubly Linked List** ↗
- ☐ **#450 Delete Node in a BST** ↗
- ☐ **#510 Inorder Successor in BST II** ↗
- ☐ **#669 Trim a Binary Search Tree** ↗
- ☐ **#701 Insert into a Binary Search Tree** ↗
- ☐ **#729 My Calendar I** ↗
- ☐ **#731 My Calendar II** ↗
- ☐ **#1008 Construct Binary Search Tree from Preorder Traversal** ↗
- ☐ **#2034 Stock Price Fluctuation** ↗

p.28

Tries (9) ↗

- ☐ ● #208 Implement Trie (Prefix Tree) ↗
- ☐ ● #211 Design Add and Search Words Data Structure ↗
- ☐ ● #421 Maximum XOR of Two Numbers in an Array ↗
- ☐ ● #648 Replace Words ↗
- ☐ ● #720 Longest Word in Dictionary ↗
- ☐ ● #1233 Remove Sub-Folders from the Filesystem ↗
- ☐ ● #1268 Search Suggestions System ↗
- ☐ ● #2452 Words Within Two Edits of Dictionary ↗
- ☐ ● #3043 Find the Length of the Longest Common Prefix ↗

Heaps (5) ↗

- ☐ ● ★ #1167 Minimum Cost to Connect Sticks ↗
- ☐ ● #1405 Longest Happy String ↗
- ☐ ● #1642 Furthest Building You Can Reach ↗

p.29

- ☐ ● #1834 Single-Threaded CPU ↗

- ☐ ● #1882 Process Tasks Using Servers ↗

Top K Elements (2) ↗

- ☐ ● #347 Top K Frequent Elements ↗
- ☐ ● #973 K Closest Points to Origin ↗

Intervals (12) ↗

- ☐ ● #56 Merge Intervals ↗
- ☐ ● #57 Insert Interval ↗
- ☐ ● ★ #253 Meeting Rooms II ↗
- ☐ ● #435 Non-overlapping Intervals ↗
- ☐ ● #452 Minimum Number of Arrows to Burst Balloons ↗
- ☐ ● #763 Partition Labels ↗
- ☐ ● #986 Interval List Intersections ↗
- ☐ ● #1094 Car Pooling ↗
- ☐ ● #1229 Meeting Scheduler ↗
- ☐ ● #1288 Remove Covered Intervals ↗
- ☐ ● #1353 Maximum Number of Events That Can Be Attended ↗

p.30

☐ ● #2054 Two Best Non-Overlapping Events ↗

K-Way Merge (2) ↗

☐ ● #373 Find K Pairs with Smallest Sums ↗

☐ ● #378 Kth Smallest Element in a Sorted Matrix ↗

Data Structure Design (11) ↗

☐ ● #146 LRU Cache ↗

☐ ● ★ #271 Encode and Decode Strings ↗

☐ ● #355 Design Twitter ↗

☐ ● #362 Design Hit Counter ↗

☐ ● #380 Insert Delete GetRandom O(1) ↗

☐ ● #622 Design Circular Queue ↗

☐ ● #641 Design Circular Deque ↗

☐ ● #981 Time Based Key-Value Store ↗

☐ ● #1146 Snapshot Array ↗

☐ ● #1472 Design Browser History ↗

☐ ● #2353 Design a Food Rating System ↗

Greedy (12) ↗

p.31

☐ ● #45 Jump Game II ↗

☐ ● #55 Jump Game ↗

☐ ● #134 Gas Station ↗

☐ ● #406 Queue Reconstruction by Height ↗

☐ ● #621 Task Scheduler ↗

☐ ● #670 Maximum Swap ↗

☐ ● #826 Most Profit Assigning Work ↗

☐ ● #921 Minimum Add to Make Parentheses Valid ↗

☐ ● #1488 Avoid Flood in The City ↗

☐ ● #1717 Maximum Score From Removing Substrings ↗

☐ ● #1871 Jump Game VII ↗

☐ ● #1975 Maximum Matrix Sum ↗

Topological Sort (6) ↗

☐ ● #210 Course Schedule II ↗

☐ ● #310 Minimum Height Trees ↗

☐ ● #802 Find Eventual Safe States ↗

p.32

- ☐ ● ★ #1136 Parallel Courses ↗
- ☐ ● #1462 Course Schedule IV ↗
- ☐ ● #2115 Find All Possible Recipes from Given Supplies ↗

Union Find (7) ↗

- ☐ ● ★ #261 Graph Valid Tree ↗
- ☐ ● ★ #323 Number of Connected Components in an Undirected Graph ↗
- ☐ ● #399 Evaluate Division ↗
- ☐ ● #547 Number of Provinces ↗
- ☐ ● #684 Redundant Connection ↗
- ☐ ● #721 Accounts Merge ↗
- ☐ ● #947 Most Stones Removed with Same Row or Column ↗

Minimum Spanning Tree (2) ↗

- ☐ ● #1135 Connecting Cities With Minimum Cost ↗
- ☐ ● #1584 Min Cost to Connect All Points ↗

Shortest Path (8) ↗

p.33

- ☐ ● ★ #505 The Maze II ↗
- ☐ ● #743 Network Delay Time ↗
- ☐ ● #787 Cheapest Flights Within K Stops ↗
- ☐ ● #1514 Path with Maximum Probability ↗
- ☐ ● #1631 Path With Minimum Effort ↗
- ☐ ● #2976 Minimum Cost to Convert String I ↗
- ☐ ● #3341 Find Minimum Time to Reach Last Room I ↗
- ☐ ● #3650 Minimum Cost Path with Edge Reversals ↗

Round 6 | Mid | H Hard (50) ↗

LinkedList In-place Reversal (1) ↗

- ☐ ● #25 Reverse Nodes in k-Group ↗

Monotonic Stack (3) ↗

- ☐ ● #84 Largest Rectangle in Histogram ↗
- ☐ ● #321 Create Maximum Number ↗

- ☐ ● #1944 Number of Visible People in a Queue ↗

Monotonic Queue (4) ↗

- ☐ ● #239 Sliding Window Maximum ↗

p.34

- ☐ ● #862 Shortest Subarray with Sum at Least K ↗
- ☐ ● #1425 Constrained Subsequence Sum ↗
- ☐ ● #1499 Max Value of Equation ↗
 Recursion (2) ↗
- ☐ ● #273 Integer to English Words ↗
- ☐ ● #761 Special Binary String ↗
 Merge Sort (2) ↗
- ☐ ● #327 Count of Range Sum ↗
- ☐ ● #493 Reverse Pairs ↗
 Backtracking (4) ↗
- ☐ ● #37 Sudoku Solver ↗
- ☐ ● #51 N-Queens ↗
- ☐ ● #301 Remove Invalid Parentheses ↗
- ☐ ● #980 Unique Paths III ↗
 Tries (5) ↗
- ☐ ● #212 Word Search II ↗
- ☐ ● #336 Palindrome Pairs ↗
- ☐ ● #425 Word Squares ↗

p.35

- ☐ ● #472 Concatenated Words ↗
- ☐ ● #1032 Stream of Characters ↗
 Heaps (1) ↗
- ☐ ● #1383 Maximum Performance of a Team ↗
 Two Heaps (3) ↗
- ☐ ● #295 Find Median from Data Stream ↗
- ☐ ● #480 Sliding Window Median ↗
- ☐ ● #502 IPO ↗
 Intervals (1) ↗
- ☐ ● #2402 Meeting Rooms III ↗
 K-Way Merge (2) ↗
- ☐ ● #23 Merge k Sorted Lists ↗
- ☐ ● #632 Smallest Range Covering Elements from K Lists ↗
 Data Structure Design (8) ↗
- ☐ ● #432 All O'one Data Structure ↗
- ☐ ● #460 LFU Cache ↗
- ☐ ● ★ #588 Design In-Memory File System ↗

p.36

- ☐ ●★ #642 Design Search Autocomplete System ↗
- ☐ ● #715 Range Module ↗
- ☐ ● #716 Max Stack ↗
- ☐ ● #895 Maximum Frequency Stack ↗
- ☐ ● #2296 Design a Text Editor ↗
- Greedy (4) ↗
- ☐ ● #135 Candy ↗
- ☐ ● #757 Set Intersection Size At Least Two ↗
- ☐ ● #857 Minimum Cost to Hire K Workers ↗
- ☐ ● #871 Minimum Number of Refueling Stops ↗
- Topological Sort (2) ↗
- ☐ ●★ #269 Alien Dictionary ↗
- ☐ ● #1203 Sort Items by Groups Respecting Dependencies ↗
- Union Find (2) ↗
- ☐ ●★ #305 Number of Islands II ↗
- ☐ ● #924 Minimize Malware Spread ↗
- Minimum Spanning Tree (3) ↗

p.37

- ☐ ● #1168 Optimize Water Distribution in a Village ↗
- ☐ ● #1489 Find Critical and Pseudo-Critical Edges in Minimum Spanning Tree ↗
- ☐ ● #3600 Maximize Spanning Tree Stability with Upgrades ↗
- Shortest Path (3) ↗
- ☐ ● #778 Swim in Rising Water ↗
- ☐ ● #1368 Minimum Cost to Make at Least One Valid Path in a Grid ↗
- ☐ ● #2203 Minimum Weighted Subgraph With the Required Paths ↗
- Round 7 | Low | E Easy (11) ↗
- 1-D DP (3) ↗
- ☐ ● #70 Climbing Stairs ↗
- ☐ ● #509 Fibonacci Number ↗
- ☐ ● #746 Min Cost Climbing Stairs ↗
- 2D Grid DP (1) ↗
- ☐ ● #118 Pascal's Triangle ↗
- Maths / Geometry (6) ↗

p.38

- ☐ ● #9 Palindrome Number ↗
- ☐ ● #13 Roman to Integer ↗
- ☐ ● #66 Plus One ↗
- ☐ ● #168 Excel Sheet Column Title ↗
- ☐ ● #263 Ugly Number ↗
- ☐ ● #1071 Greatest Common Divisor of Strings ↗
- String Matching (1) ↗
- ☐ ● #459 Repeated Substring Pattern ↗
- Round 8 | Low | M Medium (65) ↗
- Bucket Sort (3) ↗
- ☐ ● #164 Maximum Gap ↗
- ☐ ● #451 Sort Characters By Frequency ↗
- ☐ ● #692 Top K Frequent Words ↗
- 1-D DP (6) ↗
- ☐ ● #198 House Robber ↗
- ☐ ● #213 House Robber II ↗
- ☐ ● #343 Integer Break ↗
- ☐ ● #646 Maximum Length of Pair Chain ↗

p.39

- ☐ ● #740 Delete and Earn ↗
- ☐ ● #1262 Greatest Sum Divisible by Three ↗
- 0/1 Knapsack (4) ↗
- ☐ ● #416 Partition Equal Subset Sum ↗
- ☐ ● #474 Ones and Zeroes ↗
- ☐ ● #494 Target Sum ↗
- ☐ ● #1049 Last Stone Weight II ↗
- Unbounded Knapsack (4) ↗
- ☐ ● #279 Perfect Squares ↗
- ☐ ● #322 Coin Change ↗
- ☐ ● #518 Coin Change II ↗
- ☐ ● #983 Minimum Cost For Tickets ↗
- Longest Increasing Subsequence (LIS) (4) ↗
- ☐ ● #300 Longest Increasing Subsequence ↗
- ☐ ● #673 Number of Longest Increasing Subsequence ↗
- ☐ ● #1027 Longest Arithmetic Subsequence ↗
- ☐ ● #1048 Longest String Chain ↗

p.40

2D Grid DP (9) ↗

- ☐ ● #62 Unique Paths ↗
- ☐ ● #63 Unique Paths II ↗
- ☐ ● #64 Minimum Path Sum ↗
- ☐ ● #120 Triangle ↗
- ☐ ● #221 Maximal Square ↗
- ☐ ● ★ #256 Paint House ↗
- ☐ ● #931 Minimum Falling Path Sum ↗
- ☐ ● #1277 Count Square Submatrices with All Ones ↗
- ☐ ● #1937 Maximum Number of Points with Cost ↗

String DP (7) ↗

- ☐ ● #72 Edit Distance ↗
- ☐ ● #91 Decode Ways ↗
- ☐ ● #97 Interleaving String ↗
- ☐ ● #139 Word Break ↗
- ☐ ● #516 Longest Palindromic Subsequence ↗
- ☐ ● #1143 Longest Common Subsequence ↗

p.41

- ☐ ● #1653 Minimum Deletions to Make String Balanced ↗

Tree / Graph DP (3) ↗

- ☐ ● #95 Unique Binary Search Trees II ↗
- ☐ ● #337 House Robber III ↗
- ☐ ● #1976 Number of Ways to Arrive at Destination ↗

Bitmask DP (4) ↗

- ☐ ● #698 Partition to K Equal Sum Subsets ↗
- ☐ ● #1066 Campus Bikes II ↗
- ☐ ● #1986 Minimum Number of Work Sessions to Finish the Tasks ↗
- ☐ ● #2305 Fair Distribution of Cookies ↗

Digit DP (1) ↗

- ☐ ● #357 Count Numbers with Unique Digits ↗

Probability DP (3) ↗

- ☐ ● #688 Knight Probability in Chessboard ↗
- ☐ ● #808 Soup Servings ↗
- ☐ ● #837 New 21 Game ↗

p.42

State Machine DP (2) ↗

- ☐ ● #309 Best Time to Buy and Sell Stock with Cooldown ↗
- ☐ ● #714 Best Time to Buy and Sell Stock with Transaction Fee ↗

Maths / Geometry (11) ↗

- ☐ ● #7 Reverse Integer ↗
- ☐ ● #43 Multiply Strings ↗
- ☐ ● #172 Factorial Trailing Zeroes ↗
- ☐ ● #204 Count Primes ↗
- ☐ ● #264 Ugly Number II ↗
- ☐ ● #287 Find the Duplicate Number ↗
- ☐ ● #593 Valid Square ↗
- ☐ ● #611 Valid Triangle Number ↗
- ☐ ● #939 Minimum Area Rectangle ↗
- ☐ ● #963 Minimum Area Rectangle II ↗
- ☐ ● #1922 Count Good Numbers ↗

String Matching (2) ↗

- ☐ ● #686 Repeated String Match ↗

p.43

- ☐ ● #1698 Number of Distinct Substrings in a String ↗

Binary Indexed Tree / Segment Tree (2) ↗

- ☐ ● #307 Range Sum Query – Mutable ↗
- ☐ ● #308 Range Sum Query 2D – Mutable ↗

Round 9 | Low | H Hard (48) ↗

Bucket Sort (1) ↗

- ☐ ● #220 Contains Duplicate III ↗

Eulerian Circuit (3) ↗

- ☐ ● #332 Reconstruct Itinerary ↗
- ☐ ● #753 Cracking the Safe ↗
- ☐ ● #2097 Valid Arrangement of Pairs ↗

1-D DP (1) ↗

- ☐ ● #1411 Number of Ways to Paint $N \times 3$ Grid ↗

0/1 Knapsack (1) ↗

- ☐ ● #879 Profitable Schemes ↗

Longest Increasing Subsequence (LIS) (1) ↗

- ☐ ● #354 Russian Doll Envelopes ↗

2D Grid DP (13) ↗

p.44

- ☐ ● #85 Maximal Rectangle ↗
- ☐ ● #174 Dungeon Game ↗
- ☐ ● #312 Burst Balloons ↗
- ☐ ● #329 Longest Increasing Path in a Matrix ↗
- ☐ ● #403 Frog Jump ↗
- ☐ ● #465 Optimal Account Balancing ↗
- ☐ ● #546 Remove Boxes ↗
- ☐ ● #741 Cherry Pickup ↗
- ☐ ● #1235 Maximum Profit in Job Scheduling ↗
- ☐ ● #1335 Minimum Difficulty of a Job Schedule ↗
- ☐ ● #1547 Minimum Cost to Cut a Stick ↗
- ☐ ● #1931 Painting a Grid With Three Different Colors ↗
- ☐ ● #3651 Minimum Cost Path with Teleportations ↗
- String DP (7) ↗**
- ☐ ● #10 Regular Expression Matching ↗
- ☐ ● #44 Wildcard Matching ↗

p.45

- ☐ ● #115 Distinct Subsequences ↗
- ☐ ● #132 Palindrome Partitioning II ↗
- ☐ ● #140 Word Break II ↗
- ☐ ● #664 Strange Printer ↗
- ☐ ● #1092 Shortest Common Supersequence ↗
- Tree / Graph DP (2) ↗**
- ☐ ● #834 Sum of Distances in Tree ↗
- ☐ ● #968 Binary Tree Cameras ↗
- Bitmask DP (1) ↗**
- ☐ ● #847 Shortest Path Visiting All Nodes ↗
- Digit DP (2) ↗**
- ☐ ● #233 Number of Digit One ↗
- ☐ ● #902 Numbers At Most N Given Digit Set ↗
- State Machine DP (2) ↗**
- ☐ ● #123 Best Time to Buy and Sell Stock III ↗
- ☐ ● #188 Best Time to Buy and Sell Stock IV ↗
- Maths / Geometry (1) ↗**
- ☐ ● #149 Max Points on a Line ↗

p.46

String Matching (3) ↗

☐ ● #214 Shortest Palindrome ↗

☐ ● #1044 Longest Duplicate Substring ↗

☐ ● #1392 Longest Happy Prefix ↗

Binary Indexed Tree / Segment Tree (5) ↗

☐ ● #315 Count of Smaller Numbers After Self ↗

☐ ● #699 Falling Squares ↗

☐ ● #2426 Number of Pairs Satisfying Inequality ↗

☐ ● #3161 Block Placement Queries ↗

☐ ● #3721 Longest Balanced Subarray II ↗

Line Sweep (5) ↗

☐ ● #218 The Skyline Problem ↗

☐ ● #391 Perfect Rectangle ↗

☐ ● #759 Employee Free Time ↗

☐ ● #850 Rectangle Area II ↗

☐ ● #1851 Minimum Interval to Include Each Query ↗

p.47

Round 1 · High · Easy

Round 1

High Priority · Easy

68 questions · 13 patterns

Arrays #26 #27 #121 #163 #169 #283 #414
#485 #1470

Strings #14 #28 #58 #125 #242 #344 #392
#409 #680 #796

Hash Tables #205 #217 #219 #290 #350 #383
#387 #448 #706 #1189 #1512

Two Pointers #1 #88 #408 #696 #844 #977
#1768

Sliding Window – Fixed Size #643

Binary Search #35 #278 #367 #374 #704 #744

Stacks #20 #682 #1047 #1614

Tree Traversal – Level Order #104 #637

Tree Traversal – Pre Order #100 #101 #108
#144 #222 #257 #404 #572

p.48

Tree Traversal – In Order #94 #530 #783
Tree Traversal – Post-Order #110 #145 #226 #543
Depth First Search (DFS) #733
Linked List #83 #160

Arrays

Round 1 · High · Easy · 9 questions

Round 1 · High · Easy

p.49

Round 1 · High · Easy

p.50

Arrays

☐ #26 Remove Duplicates from Sorted Array

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Two Pointers
 Lists: AlgoMaster 600

Problem

Given an integer array `nums` sorted in non-decreasing order, remove the duplicates in-place such that each unique element appears only once. The relative order of the elements should be kept the same.

Consider the number of unique elements in `nums` to be `k`. After removing duplicates, return the number of unique elements `k`.

The first `k` elements of `nums` should contain the unique numbers in sorted order. The remaining elements beyond index `k - 1` can be ignored.

Custom Judge:

The judge will test your solution with the following code:

Round 1 · High · Easy

p.51

```
int[] nums = [...]; // Input array
int[] expectedNums = [...]; // The expected answer with correct length

int k = removeDuplicates(nums); // Calls your implementation

assert k == expectedNums.length;
for (int i = 0; i < k; i++) {
    assert nums[i] == expectedNums[i];
}
```

If all assertions pass, then your solution will be accepted.

Example 1:

```
Input: nums = [1,1,2]
Output: 2, nums = [1,2,_]
Explanation: Your function should return k = 2, with the first two elements of
nums being 1 and 2 respectively.
It does not matter what you leave beyond the returned k (hence they are
underscores).
```

Example 2:

```
Input: nums = [0,0,1,1,1,2,2,3,3,4]
Output: 5, nums = [0,1,2,3,4,_,_,_,_,_]
Explanation: Your function should return k = 5, with the first five elements of
nums being 0, 1, 2, 3, and 4 respectively.
It does not matter what you leave beyond the returned k (hence they are
underscores).
```

Constraints:

- $1 \leq \text{nums.length} \leq 3 \times 10^4$
- $-100 \leq \text{nums}[i] \leq 100$
- `nums` is sorted in non-decreasing order.

Community Solutions (Python)

- WalkCC

Round 1 · High · Easy

p.52

```
class Solution:
    def removeDuplicates(self, nums: list[int]) -> int:
        i = 0

        for num in nums:
            if i < 1 or num > nums[i - 1]:
                nums[i] = num
                i += 1

        return i
```

• LeetCode

```
class Solution:
    def removeDuplicates(self, nums: List[int]) -> int:
        k = 0
        for x in nums:
            if k == 0 or x != nums[k - 1]:
                nums[k] = x
                k += 1
        return k
```

• SimplyLeet

```
# Function to remove duplicates and return count of unique elements.
def removeDuplicates(nums):
    if not nums:
        return 0

    unique_index = 1 # Pointer for the next unique element.
    # Iterate over the array starting from the second element.
    for i in range(1, len(nums)):
        # If the current element is different from the previous.
        if nums[i] != nums[i - 1]:
            nums[unique_index] = nums[i] # Place the unique element at the
            unique_index.
```

```
        unique_index += 1
    return unique_index

# Example usage:
# nums = [0,0,1,1,1,2,2,3,3,4]
# k = removeDuplicates(nums)
# print(k, nums[:k])
```

Arrays

□ #27 Remove Element

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode

Array · Two Pointers

Lists: AlgoMaster 600

Problem

Given an integer array `nums` and an integer `val`, remove all occurrences of `val` in `nums` in-place.

The order of the elements may be changed.

Then return the number of elements in `nums` which are not equal to `val`.

Consider the number of elements in `nums` which are not equal to `val` be `k`, to get accepted, you need to do the following things:

- Change the array `nums` such that the first `k` elements of `nums` contain the elements which are not equal to `val`. The remaining elements of `nums` are not important as well as the size of `nums`.
- Return `k`.

Custom Judge:

Round 1 · High · Easy

p.55

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int val = ...; // Value to remove
int[] expectedNums = [...]; // The expected answer with correct length.
// It is sorted with no values equaling val.

int k = removeElement(nums, val); // Calls your implementation

assert k == expectedNums.length;
```

If all assertions pass, then your solution will be accepted.

Example 1:

```
Input: nums = [3,2,2,3], val = 3
Output: 2, nums = [2,2,_,_]
Explanation: Your function should return k = 2, with the first two elements of
nums being 2.
It does not matter what you leave beyond the returned k (hence they are
underscores).
```

Example 2:

```
Input: nums = [0,1,2,2,3,0,4,2], val = 2
Output: 5, nums = [0,1,4,0,3,_,_]
Explanation: Your function should return k = 5, with the first five elements of
nums containing 0, 0, 1, 3, and 4.
Note that the five elements can be returned in any order.
It does not matter what you leave beyond the returned k (hence they are
underscores).
```

Constraints:

- $0 \leq \text{nums.length} \leq 100$
- $0 \leq \text{nums}[i] \leq 50$
- $0 \leq \text{val} \leq 100$

Community Solutions (Python)

Round 1 · High · Easy

p.56

• WalkCC

```
class Solution:
    def removeElement(self, nums: list[int], val: int) -> int:
        i = 0

        for num in nums:
            if num != val:
                nums[i] = num
                i += 1

        return i
```

• LeetCode

```
class Solution:
    def removeElement(self, nums: List[int], val: int) -> int:
        k = 0
        for x in nums:
            if x != val:
                nums[k] = x
                k += 1
        return k
```

• SimplyLeet

```
def removeElement(nums, val):
    # Pointer for the position of the next valid element
    valid_index = 0
    # Iterate through each element in the array
    for num in nums:
        # If the current element is not equal to val, it is valid
        if num != val:
            # Write it to the position indicated by valid_index
            nums[valid_index] = num
            # Move the valid_index to the next position
```

```
        valid_index += 1
        # Return the count of valid elements
        return valid_index

# Example usage
nums = [3,2,2,3]
val = 3
k = removeElement(nums, val)
print(k, nums[:k])
```

Arrays

□ #121 Best Time to Buy and Sell Stock

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Dynamic Programming
 Lists: Grind 169

Problem

You are given an array `prices` where `prices[i]` is the price of a given stock on the `i`th day.

You want to maximize your profit by choosing a single day to buy one stock and choosing a different day in the future to sell that stock.

Return the maximum profit you can achieve from this transaction. If you cannot achieve any profit, return 0.

Example 1:

Input: `prices = [7,1,5,3,6,4]`
 Output: 5
 Explanation: Buy on day 2 (price = 1) and sell on day 5 (price = 6), profit = 6-1 = 5.
 Note that buying on day 2 and selling on day 1 is not allowed because you must buy before you sell.

Example 2:

Round 1 · High · Easy

p.59

Input: `prices = [7,6,4,3,1]`
 Output: 0
 Explanation: In this case, no transactions are done and the max profit = 0.

Constraints:

- $1 \leq \text{prices.length} \leq 105$
- $0 \leq \text{prices}[i] \leq 104$

Community Solutions (Python)

□ • WalkCC

```
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        sellOne = 0
        holdOne = -math.inf

        for price in prices:
            sellOne = max(sellOne, holdOne + price)
            holdOne = max(holdOne, -price)

        return sellOne
```

□ • LeetDoocs

```
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        ans, mi = 0, inf
        for v in prices:
            ans = max(ans, v - mi)
            mi = min(mi, v)
        return ans
```

□ • SimplyLeet

Round 1 · High · Easy

p.60

```
def maxProfit(prices):
    # Initialize min_price as infinity and max_profit as 0
    min_price = float('inf')
    max_profit = 0
    # Iterate through each price in the list
    for price in prices:
        # Update the minimum price if current price is lower
        if price < min_price:
            min_price = price
        else:
            # Calculate current profit and update max_profit if it is higher
            max_profit = max(max_profit, price - min_price)
    return max_profit

# Example usage:
# result = maxProfit([7,1,5,3,6,4])
# print(result) # Output should be 5
```

• LC.ca

```
'''
>>> import math
>>> a = math.inf
>>> a
inf

>>> x = float('-inf')
>>> y = float('inf')
>>> print(x < y) # Output: True
True

>>>
>>> z = -math.inf
>>> x==z
True

'''

class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        ans, mi = 0, inf
        for v in prices:
```

Round 1 · High · Easy

p.61

```
        mi = min(mi, v)
        ans = max(ans, v - mi)
    return ans

#####

class Solution(object):
    def maxProfit(self, prices):
        """
        :type prices: List[int]

        :rtype: int
        """
        if not prices:
            return 0
        ans = 0
        pre = prices[0]
        for i in range(1, len(prices)):
            pre = min(pre, prices[i])
            ans = max(prices[i] - pre, ans)
        return ans
```

◆ Quick Review

Key Insights

- Traverse the list of prices once while keeping track of the minimum price encountered so far.
- At each day, calculate the profit if you sold the stock on that day.
- Update the maximum profit when the current profit exceeds the previous maximum.
- The optimal solution makes a single pass through the array with constant space usage.

Round 1 · High · Easy

p.62

Space & Time

Time Complexity: $O(n)$ – requires one pass through the prices array.

Space Complexity: $O(1)$ – only a few extra variables are used.

Solution

We solve the problem using a simple linear scan. We maintain two key variables: one for the minimum price seen so far and another for the maximum profit calculated so far. As we iterate, we continuously update the minimum price if a lower price is found, and then compute the potential profit at the current price. If this profit exceeds the current maximum profit, we update the maximum profit. The final maximum profit is returned after iterating through all prices.

Round 1 · High · Easy

p.63

Arrays

★ #163 Missing Ranges ★

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array

Lists: ★ Premium | Premium 98

Problem

You are given an inclusive range [lower, upper] and a sorted unique integer array nums, where all elements are within the inclusive range.

A number x is considered missing if x is in the range [lower, upper] and x is not in nums.

Return the shortest sorted list of ranges that exactly covers all the missing numbers. That is, no element of nums is included in any of the ranges, and each missing number is covered by one of the ranges.

Example 1:

```
Input: nums = [0,1,3,50,75], lower = 0, upper = 99
Output: [[2,2],[4,49],[51,74],[76,99]]
Explanation: The ranges are:
[2,2]
[4,49]
[51,74]
[76,99]
```

Round 1 · High · Easy

p.64

Example 2:

Input: nums = [-1], lower = -1, upper = -1

Output: []

Explanation: There are no missing ranges since there are no missing numbers.

Constraints:

- $-109 \leq \text{lower} \leq \text{upper} \leq 109$
- $0 \leq \text{nums.length} \leq 100$
- $\text{lower} \leq \text{nums}[i] \leq \text{upper}$
- All the values of nums are unique.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findMissingRanges(
        self,
        nums: List[int],
        lower: int,
        upper: int,
    ) -> List[List[int]]:
        def getRange(lo: int, hi: int) -> List[int]:
            if lo == hi:
                return [lo, lo]

            return [lo, hi]

        if not nums:
            return [getRange(lower, upper)]

        ans = []

        if nums[0] > lower:
            ans.append(getRange(lower, nums[0] - 1))
```

```
for prev, curr in zip(nums, nums[1:]):
    if curr > prev + 1:
        ans.append(getRange(prev + 1, curr - 1))

if nums[-1] < upper:
    ans.append(getRange(nums[-1] + 1, upper))

return ans
```

• LeetCode

```
class Solution:
    def findMissingRanges(
        self, nums: List[int], lower: int, upper: int
    ) -> List[List[int]]:
        n = len(nums)
        if n == 0:
            return [[lower, upper]]
        ans = []
        if nums[0] > lower:
            ans.append([lower, nums[0] - 1])
```

```
for a, b in pairwise(nums):
    if b - a > 1:
        ans.append([a + 1, b - 1])
if nums[-1] < upper:
    ans.append([nums[-1] + 1, upper])
return ans
```

• SimplyLeet

```
# Python solution for Missing Ranges problem

def findMissingRanges(nums, lower, upper):
    missing_ranges = [] # List to store the missing range intervals
    prev = lower - 1 # Initialize previous value to one less than lower

    # Append a sentinel at the end for easier gap handling
    nums.append(upper + 1)

    # Loop through each number, including the sentinel
```

```

for num in nums:
    # If there is a gap between previous and current number
    if num - prev >= 2:
        # If gap is of single number, add that number as a range
        if num - prev == 2:
            missing_ranges.append([prev + 1, prev + 1])
        else:
            missing_ranges.append([prev + 1, num - 1])
    prev = num # Update previous to current
return missing_ranges

```

```

# Example usage:
print(findMissingRanges([0,1,3,50,75], 0, 99))

```

• LC.ca

```

class Solution:
    def findMissingRanges(
        self, nums: List[int], lower: int, upper: int
    ) -> List[List[int]]:
        n = len(nums)
        if n == 0:
            return [[lower, upper]]
        ans = []
        if nums[0] > lower:
            ans.append([lower, nums[0] - 1])

        for a, b in pairwise(nums):
            if b - a > 1:
                ans.append([a + 1, b - 1])
        if nums[-1] < upper:
            ans.append([nums[-1] + 1, upper])
        return ans

```

◆ Quick Review

Key Insights

- Use a pointer (or iterate index) to traverse the nums array while keeping track of the previous number considered.
- Check for missing numbers before the first element, between consecutive elements, and after the last element.
- When a gap is found, convert it to a range string that is either a single number (if the gap is exactly one number) or an interval.
- Handle edge cases when the nums array is empty.

Space & Time

Time Complexity: $O(n)$, where n is the length of the nums array, since we iterate through the array once.

Space Complexity: $O(1)$ additional space (ignoring the output list), as we only use a few extra variables.

Solution

The solution leverages a single pass through the input array and calculates the missing ranges by comparing the expected number with the current number in nums. First, initialize a variable "prev" to one less than the

lower bound so that missing numbers from the very start of the range can be handled uniformly. For each number in the array, if there is a gap between prev and the current number (i.e., current number is at least 2 more than prev), then add the missing range (prev+1 to current number-1) to the output list. Update "prev" to the current number and finally handle any gap between the last number and the upper bound. The primary data structures used are the list for the output; the approach is iterative.

Round 1 · High · Easy

p.69

Arrays

□ #169 Majority Element

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table · Divide and Conquer ·
 Sorting · Counting

Lists: Grind 169

Problem

Given an array nums of size n, return the majority element.

The majority element is the element that appears more than $n / 2$ times. You may assume that the majority element always exists in the array.

Example 1:

Input: nums = [3,2,3]
 Output: 3

Example 2:

Input: nums = [2,2,1,1,1,2,2]
 Output: 2

Constraints:

- $n == \text{nums.length}$
- $1 \leq n \leq 5 * 10^4$

Round 1 · High · Easy

p.70

- $-109 \leq \text{nums}[i] \leq 109$
- The input is generated such that a majority element will exist in the array.

Follow-up: Could you solve the problem in linear time and in $O(1)$ space?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def majorityElement(self, nums: List[int]) -> int:
        ans = None
        count = 0

        for num in nums:
            if count == 0:
                ans = num
            count += (1 if num == ans else -1)

        return ans
```

• LeetCode

```
class Solution:
    def majorityElement(self, nums: List[int]) -> int:
        cnt = m = 0
        for x in nums:
            if cnt == 0:
                m, cnt = x, 1
            else:
                cnt += 1 if m == x else -1
        return m
```

• SimplyLeet

Round 1 · High · Easy

p.71

```
# Python implementation of the Boyer-Moore Voting Algorithm

def majorityElement(nums):
    # Initialize candidate and counter
    candidate = None
    count = 0

    # Iterate over each number in the input list
    for num in nums:
        # If count is 0, choose the current number as the candidate

        if count == 0:
            candidate = num
        # Increment count if the current number equals the candidate, otherwise
        # decrement
        count += 1 if num == candidate else -1
    # Return the candidate, which is the majority element
    return candidate

# Example usage
print(majorityElement([2,2,1,1,1,2,2])) # Output: 2
```

• LC.ca

```
class Solution:
    def majorityElement(self, nums: List[int]) -> int:
        cnt = m = 0
        for x in nums:
            if cnt == 0:
                m, cnt = x, 1
            else:
                cnt += 1 if m == x else -1
        return m
```

◆ Quick Review

Key Insights

- The majority element appears more than half the length of the array.

Round 1 · High · Easy

p.72

- A simple hash table approach can count occurrences, but a more efficient solution exists.
- The Boyer–Moore Voting Algorithm finds the majority in $O(n)$ time and $O(1)$ space.
- The algorithm maintains a candidate and a counter, adjusting the candidate when the count drops to zero.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Solution

We use the Boyer–Moore Voting Algorithm to identify the majority element. The algorithm iterates through the array using two variables: one for the candidate and one for the count. Initially, we set the candidate to an arbitrary value and count to 0. For each element, if the count is 0, we update the candidate to the current element. Then, if the current element is the same as the candidate, we increment the count; otherwise, we decrement it. Since the majority element appears more than $n/2$ times, it will remain as the candidate by the

Round 1 · High · Easy

p.73

end of the iteration.

Round 1 · High · Easy

p.74

Arrays

#283 Move Zeroes

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode

Array · Two Pointers

Lists: Grind 169

Problem

Given an integer array `nums`, move all 0's to the end of it while maintaining the relative order of the non-zero elements.

Note that you must do this in-place without making a copy of the array.

Example 1:

Input: `nums = [0,1,0,3,12]`
Output: `[1,3,12,0,0]`

Example 2:

Input: `nums = [0]`
Output: `[0]`

Constraints:

- $1 \leq \text{nums.length} \leq 104$
- $-231 \leq \text{nums}[i] \leq 231 - 1$

Follow up: Could you minimize the total number of operations done?

Round 1 · High · Easy

p.75

Community Solutions (Python)

• WalkCC

```
class Solution:
    def moveZeroes(self, nums: List[int]) -> None:
        j = 0
        for num in nums:
            if num != 0:
                nums[j] = num
                j += 1

        for i in range(j, len(nums)):
            nums[i] = 0
```

• LeetDoocs

```
class Solution:
    def moveZeroes(self, nums: List[int]) -> None:
        k = 0
        for i, x in enumerate(nums):
            if x:
                nums[k], nums[i] = nums[i], nums[k]
                k += 1
```

• SimplyLeet

```
# Moves all zeros in the list 'nums' to the end in-place.
def moveZeroes(nums):
    # 'j' is the pointer where the next non-zero element should be placed.
    j = 0
    # Loop through all the elements with index 'i'
    for i in range(len(nums)):
        # Check if the current element is non-zero.
        if nums[i] != 0:
            # Swap only if i is not equal to j to avoid unnecessary operations.
            nums[j], nums[i] = nums[i], nums[j]
```

Round 1 · High · Easy

p.76

```
# Increment j to move to the next insertion index.
j += 1
```

```
# Example usage:
nums = [0, 1, 0, 3, 12]
moveZeroes(nums)
print(nums) # Expected output: [1, 3, 12, 0, 0]
```

• LC.ca

```
class Solution:
    def moveZeroes(self, nums: List[int]) -> None:
        i = -1
        for j, x in enumerate(nums):
            if x:
                i += 1
                nums[i], nums[j] = nums[j], nums[i]
```

◆ Quick Review

Key Insights

- Use a two-pointer technique: one pointer to iterate through the array and another pointer to track the position for the next non-zero element.
- For each non-zero element encountered, swap it with the element at the tracking pointer.
- This strategy minimizes operations by ensuring each non-zero element is moved at most once.

Round 1 · High · Easy

p.77

- After repositioning all non-zero elements, zeros will naturally occupy the remaining positions.
- The in-place requirement ensures space complexity remains $O(1)$.

Space & Time

Time Complexity: $O(n)$, where n is the number of elements in the array, since only one pass (or at most two simple passes) is required.

Space Complexity: $O(1)$ extra space, as the reordering is accomplished in-place.

Solution

The solution leverages a two-pointer approach. The left pointer (j) keeps track of the index where the next non-zero element should be placed, while the right pointer (i) iterates through the array. Whenever a non-zero element is encountered, it is swapped with the element at index j , and j is then incremented. This effectively gathers all non-zero elements at the beginning while moving zeros to the end. A key optimization is to perform the swap only when necessary (i.e., when i and j are not the same) to reduce the

Round 1 · High · Easy

p.78

number of write operations.

Round 1 · High · Easy

p.79

Arrays

□ #414 Third Maximum Number

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Sorting
Lists: AlgoMaster 600

Problem

Given an integer array `nums`, return the third distinct maximum number in this array. If the third maximum does not exist, return the maximum number.

Example 1:

```
Input: nums = [3,2,1]
Output: 1
Explanation:
The first distinct maximum is 3.
The second distinct maximum is 2.
The third distinct maximum is 1.
```

Example 2:

```
Input: nums = [1,2]
Output: 2
Explanation:
The first distinct maximum is 2.
The second distinct maximum is 1.
The third distinct maximum does not exist, so the maximum (2) is returned instead.
```

Example 3:

Round 1 · High · Easy

p.80

Input: nums = [2,2,3,1]
 Output: 1
 Explanation:
 The first distinct maximum is 3.
 The second distinct maximum is 2 (both 2's are counted together since they have the same value).
 The third distinct maximum is 1.

Constraints:

- $1 \leq \text{nums.length} \leq 104$
- $-231 \leq \text{nums}[i] \leq 231 - 1$

Follow up: Can you find an $O(n)$ solution?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def thirdMax(self, nums: list[int]) -> int:
        max1 = -math.inf # the maximum
        max2 = -math.inf # the second maximum
        max3 = -math.inf # the third maximum

        for num in nums:
            if num > max1:
                max3 = max2
                max2 = max1

            max1 = num
            elif max1 > num and num > max2:
                max3 = max2
                max2 = num
            elif max2 > num and num > max3:
                max3 = num

        return max1 if max3 == -math.inf else max3
```

```
class Solution:
    def thirdMax(self, nums: list[int]) -> int:
        minHeap = []
        seen = set()

        for num in nums:
            if num not in seen:
                seen.add(num)
                heapq.heappush(minHeap, num)
                if len(minHeap) > 3:

                    heapq.heappop(minHeap)

        if len(minHeap) == 2:
            heapq.heappop(minHeap)

        return minHeap[0]
```

• LeetDoocs

```
class Solution:
    def thirdMax(self, nums: List[int]) -> int:
        m1 = m2 = m3 = -inf
        for num in nums:
            if num in [m1, m2, m3]:
                continue
            if num > m1:
                m3, m2, m1 = m2, m1, num
            elif num > m2:
                m3, m2 = m2, num

            elif num > m3:
                m3 = num
        return m3 if m3 != -inf else m1
```

• SimplyLeet

```
# Define function thirdMax that returns the third distinct maximum number in
the array.
def thirdMax(nums):
    # Initialize three maximum variables to None to represent their absence.
    first = second = third = None

    # Iterate over each number in the nums array.
    for num in nums:
        # Skip the current number if it is already one of the three maximums.
        if num == first or num == second or num == third:
            continue

        # Update the maximums based on the current number's value.
        if first is None or num > first:
            # Shift down the values: first becomes second, second becomes
            third, second, first = second, first, num
        elif second is None or num > second:
            # Update second and shift second to third.
            third, second = second, num
        elif third is None or num > third:
            # Update third maximum.

            third = num

    # If there is no third distinct maximum, return the first maximum.
    return first if third is None else third

# Example usage:
print(thirdMax([3,2,1])) # Output: 1
print(thirdMax([1,2]))  # Output: 2
print(thirdMax([2,2,3,1])) # Output: 1
```

Arrays

☐ #485 Max Consecutive Ones

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array

Lists: AlgoMaster 600

Problem

Given a binary array `nums`, return the maximum number of consecutive 1's in the array.

Example 1:

Input: `nums = [1,1,0,1,1,1]`
 Output: 3
 Explanation: The first two digits or the last three digits are consecutive 1s.
 The maximum number of consecutive 1s is 3.

Example 2:

Input: `nums = [1,0,1,1,0,1]`
 Output: 2

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- `nums[i]` is either 0 or 1.

Community Solutions (Python)

- WalkCC

```
class Solution:
    def findMaxConsecutiveOnes(self, nums: list[int]) -> int:
        ans = 0
        summ = 0

        for num in nums:
            if num == 0:
                summ = 0
            else:
                summ += num

        ans = max(ans, summ)

        return ans
```

• LeetCode

```
class Solution:
    def findMaxConsecutiveOnes(self, nums: List[int]) -> int:
        ans = cnt = 0
        for x in nums:
            if x:
                cnt += 1
                ans = max(ans, cnt)
            else:
                cnt = 0
        return ans
```

• SimplyLeet

```
# Function to return maximum consecutive 1s in an array
def findMaxConsecutiveOnes(nums):
    # Initialize counters for consecutive ones
    max_consecutive = 0
    current_count = 0
    # Iterate through each number in the list
    for num in nums:
        if num == 1:
            # Increase the count when the current number is 1
            current_count += 1
```

```
        # Update maximum if the current count is higher
        max_consecutive = max(max_consecutive, current_count)
    else:
        # Reset current count when a 0 is encountered
        current_count = 0
    return max_consecutive

# Example usage:
if __name__ == "__main__":
    example_nums = [1, 1, 0, 1, 1, 1]

    print(findMaxConsecutiveOnes(example_nums)) # Expected output: 3
```

Arrays

#1470 Shuffle the Array

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array

Lists: AlgoMaster 600

Problem

Given the array `nums` consisting of $2n$ elements in the form `[x1,x2,...,xn,y1,y2,...,yn]`.

Return the array in the form `[x1,y1,x2,y2,...,xn,yn]`.

Example 1:

Input: `nums = [2,5,1,3,4,7]`, `n = 3`

Output: `[2,3,5,4,1,7]`

Explanation: Since `x1=2`, `x2=5`, `x3=1`, `y1=3`, `y2=4`, `y3=7` then the answer is `[2,3,5,4,1,7]`.

Example 2:

Input: `nums = [1,2,3,4,4,3,2,1]`, `n = 4`

Output: `[1,4,2,3,3,2,4,1]`

Example 3:

Input: `nums = [1,1,2,2]`, `n = 2`

Output: `[1,2,1,2]`

Round 1 · High · Easy

p.87

Constraints:

- $1 \leq n \leq 500$
- `nums.length == 2n`
- $1 \leq \text{nums}[i] \leq 10^3$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def shuffle(self, nums: List[int], n: int) -> List[int]:
        ans = []
        for a, b in zip(nums[:n], nums[n:]):
            ans.append(a)
            ans.append(b)
        return ans
```

• LeetDoocs

```
class Solution:
    def shuffle(self, nums: List[int], n: int) -> List[int]:
        return [x for pair in zip(nums[:n], nums[n:]) for x in pair]
```

• SimplyLeet

```
def shuffle(nums, n):
    # Create an empty list to store the result
    result = []
    # Loop through the first half of the array
    for i in range(n):
        # Append the element from the first half
        result.append(nums[i])
        # Append the corresponding element from the second half
        result.append(nums[i + n])
    return result
```

Round 1 · High · Easy

p.88


```
# Example usage:  
# nums = [2,5,1,3,4,7]  
# print(shuffle(nums, 3))
```

Round 1 · High · Easy**p.89**

Strings

Round 1 · High · Easy · 10 questions**Round 1 · High · Easy****p.90**

Strings

#14 Longest Common Prefix

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Trie
Lists: Grind 169

Problem

Write a function to find the longest common prefix string amongst an array of strings.
If there is no common prefix, return an empty string "".

Example 1:

Input: strs = ["flower","flow","flight"]
Output: "fl"

Example 2:

Input: strs = ["dog","racecar","car"]
Output: ""
Explanation: There is no common prefix among the input strings.

Constraints:

- $1 \leq \text{strs.length} \leq 200$
- $0 \leq \text{strs}[i].\text{length} \leq 200$
- $\text{strs}[i]$ consists of only lowercase English letters if it is non-empty.

Round 1 · High · Easy

p.91

Community Solutions (Python)

• WalkCC

```
class Solution:
    def longestCommonPrefix(self, strs: list[str]) -> str:
        if not strs:
            return ''

        for i in range(len(strs[0])):
            for j in range(1, len(strs)):
                if i == len(strs[j]) or strs[j][i] != strs[0][i]:
                    return strs[0][:i]

        return strs[0]
```

• LeetDoocs

```
class Solution:
    def longestCommonPrefix(self, strs: List[str]) -> str:
        for i in range(len(strs[0])):
            for s in strs[1:]:
                if len(s) <= i or s[i] != strs[0][i]:
                    return s[:i]
        return strs[0]
```

• SimplyLeet

```
def longestCommonPrefix(strs):
    # If no strings exist, return empty
    if not strs:
        return ""
    # Iterate over each index of the first string
    for i in range(len(strs[0])):
        current_char = strs[0][i]
        # Compare the current character with the corresponding one in the rest
        # of the strings
        for s in strs[1:]:
            # If index exceeds length or characters differ, return the prefix
            # found so far
```

Round 1 · High · Easy

p.92

```

        if i >= len(s) or s[i] != current_char:
            return strs[0][:i]
        # If no mismatch is found, return the complete first string
        return strs[0]

# Example usage:
print(longestCommonPrefix(["flower", "flow", "flight"]))

```

• LC.ca

```

class Solution:
    def longestCommonPrefix(self, strs: List[str]) -> str:
        for i in range(len(strs[0])):
            for s in strs[1:]:
                if len(s) <= i or s[i] != strs[0][i]:
                    return s[:i]
        return strs[0]

```

◆ Quick Review

Key Insights

- Compare characters of the strings one column at a time (vertical scanning).
- The longest possible prefix is limited by the length of the shortest string in the array.
- As soon as a mismatch is found, the prefix up to that point is the answer.
- Handle edge cases such as an empty input array or strings with no common prefix.

Space & Time

Round 1 · High · Easy

p.93

Time Complexity: $O(N \cdot M)$ where N is the number of strings and M is the length of the shortest string.

Space Complexity: $O(1)$ additional space (ignoring output space).

Solution

We use the vertical scanning approach:

– If the input array is empty, return an empty string. – Iterate character by character over the first string. – For each character, compare it with the corresponding character in each of the other strings. – If a mismatch or end of any string is encountered, return the substring from the start up to the current index. – If no mismatch is found, the entire first string is the common prefix.

This solution ensures that we only traverse the necessary part of the strings and stops early if a discrepancy is discovered.

Round 1 · High · Easy

p.94

Strings

#28 Find the Index of the First Occurrence in a String

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Two Pointers · String · String Matching
Lists: AlgoMaster 600

Problem

Given two strings `needle` and `haystack`, return the index of the first occurrence of `needle` in `haystack`, or `-1` if `needle` is not part of `haystack`.

Example 1:

Input: `haystack = "sadbutsad"`, `needle = "sad"`
Output: `0`
Explanation: "sad" occurs at index 0 and 6.
The first occurrence is at index 0, so we return 0.

Example 2:

Input: `haystack = "leetcode"`, `needle = "leeto"`
Output: `-1`
Explanation: "leeto" did not occur in "leetcode", so we return -1.

Constraints:

- $1 \leq \text{haystack.length}, \text{needle.length} \leq 104$
- `haystack` and `needle` consist of only lowercase English characters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def strStr(self, haystack: str, needle: str) -> int:
        m = len(haystack)
        n = len(needle)

        for i in range(m - n + 1):
            if haystack[i:i + n] == needle:
                return i

        return -1
```

• LeetDoocs

```
class Solution:
    def strStr(self, haystack: str, needle: str) -> int:
        n, m = len(haystack), len(needle)
        for i in range(n - m + 1):
            if haystack[i : i + m] == needle:
                return i
        return -1
```

• SimplyLeet

```
# Function to find the first occurrence of needle in haystack
def strStr(haystack: str, needle: str) -> int:
    haystack_length = len(haystack)
    needle_length = len(needle)
    # Edge: if needle is longer than haystack, it cannot be a substring
    if needle_length > haystack_length:
        return -1

    # Loop through each index in haystack where needle could fit
    for i in range(haystack_length - needle_length + 1):
```

```
# Check if the substring of haystack starting at i matches needle
match = True
for j in range(needle_length):
    if haystack[i + j] != needle[j]:
        match = False
        break # break early if mismatch found
if match:
    return i # return the starting index of the match
return -1 # return -1 if needle is not found in haystack
```

```
# Example usage:
# print(strStr("sadbutsad", "sad")) # Expected output: 0
```

Strings

□ #58 Length of Last Word

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode String

Lists: AlgoMaster 600

Problem

Given a string *s* consisting of words and spaces, return the length of the last word in the string.

A word is a maximal substring consisting of non-space characters only.

Example 1:

Input: *s* = "Hello World"
Output: 5
Explanation: The last word is "World" with length 5.

Example 2:

Input: *s* = " fly me to the moon "
Output: 4
Explanation: The last word is "moon" with length 4.

Example 3:

Input: *s* = "luffy is still joyboy"
Output: 6
Explanation: The last word is "joyboy" with length 6.

Constraints:

- $1 \leq s.length \leq 104$

- s consists of only English letters and spaces ' '.
- There will be at least one word in s.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def lengthOfLastWord(self, s: str) -> int:
        i = len(s) - 1

        while i >= 0 and s[i] == ' ':
            i -= 1
        lastIndex = i
        while i >= 0 and s[i] != ' ':
            i -= 1

        return lastIndex - i
```

• LeetDoocs

```
class Solution:
    def lengthOfLastWord(self, s: str) -> int:
        i = len(s) - 1
        while i >= 0 and s[i] == ' ':
            i -= 1
        j = i
        while j >= 0 and s[j] != ' ':
            j -= 1
        return i - j
```

• SimplyLeet

```
# Function to compute the length of the last word in a string
def length_of_last_word(s):
    # Remove trailing spaces from the string
    s = s.rstrip()
    # Find the index of the last space in the string
    last_space_index = s.rfind(' ')
    # The length of the last word is the total length minus the position of
    # last space - 1
    return len(s) - last_space_index - 1

# Test example

print(length_of_last_word("Hello World")) # Expected output: 5
```

Strings

#125 Valid Palindrome

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Two Pointers · String
Lists: Grind 169

Problem

A phrase is a palindrome if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward. Alphanumeric characters include letters and numbers.

Given a string *s*, return true if it is a palindrome, or false otherwise.

Example 1:

Input: *s* = "A man, a plan, a canal: Panama"
Output: true
Explanation: "amanaplanacanalpanama" is a palindrome.

Example 2:

Input: *s* = "race a car"
Output: false
Explanation: "raceacar" is not a palindrome.

Example 3:

Round 1 · High · Easy

p.101

Input: *s* = " "
Output: true
Explanation: *s* is an empty string "" after removing non-alphanumeric characters.
Since an empty string reads the same forward and backward, it is a palindrome.

Constraints:

- $1 \leq s.length \leq 2 * 10^5$
- *s* consists only of printable ASCII characters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isPalindrome(self, s: str) -> bool:
        l = 0
        r = len(s) - 1

        while l < r:
            while l < r and not s[l].isalnum():
                l += 1
            while l < r and not s[r].isalnum():
                r -= 1

            if s[l].lower() != s[r].lower():
                return False
            l += 1
            r -= 1

        return True
```

• LeetDoocs

Round 1 · High · Easy

p.102

```
class Solution:
    def isPalindrome(self, s: str) -> bool:
        i, j = 0, len(s) - 1
        while i < j:
            if not s[i].isalnum():
                i += 1
            elif not s[j].isalnum():
                j -= 1
            elif s[i].lower() != s[j].lower():
                return False
        else:
            i, j = i + 1, j - 1
        return True
```

• SimplyLeet

```
class Solution:
    def isPalindrome(self, s: str) -> bool:
        # Initialize two pointers at the beginning and end of the string
        left, right = 0, len(s) - 1
        while left < right:
            # Skip non-alphanumeric characters from the left
            while left < right and not s[left].isalnum():
                left += 1
            # Skip non-alphanumeric characters from the right
            while left < right and not s[right].isalnum():
                right -= 1
            # Compare the characters in lowercase
            if s[left].lower() != s[right].lower():
                return False
            left += 1
            right -= 1
        return True
```

• LC.ca

Round 1 · High · Easy

p.103

```
class Solution:
    def isPalindrome(self, s: str) -> bool:
        i, j = 0, len(s) - 1
        while i < j:
            if not s[i].isalnum():
                i += 1
            elif not s[j].isalnum():
                j -= 1
            elif s[i].lower() != s[j].lower():
                return False
        else:
            i, j = i + 1, j - 1
        return True
```

◆ Quick Review

Key Insights

- Use two pointers (one starting at the beginning and one at the end) to compare characters.
- Skip characters that are not alphanumeric.
- Compare characters in a case-insensitive manner.
- Continue until the pointers meet or cross.

Space & Time

Time Complexity: $O(n)$, where n is the length of the string, as each character is processed at most once.

Round 1 · High · Easy

p.104

Space Complexity: $O(1)$, using only a few extra variables for pointers and temporary comparisons.

Solution

The solution employs a two-pointer approach:

- Initialize two pointers: one at the start (left) and one at the end (right) of the string.
- Skip any non-alphanumeric characters using these pointers.
- Convert the remaining characters at each pointer to lowercase and compare them.
- If any mismatch is found, return false; otherwise, continue until the pointers cross.
- Return true if all corresponding characters match. This method efficiently checks for a palindrome without needing additional space for a filtered string.

Strings

□ #242 Valid Anagram

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Sorting
Lists: Grind 169

Problem

Given two strings s and t , return true if t is an anagram of s , and false otherwise.

Example 1:

Input: $s = \text{"anagram"}, t = \text{"nagaram"}$

Output: true

Example 2:

Input: $s = \text{"rat"}, t = \text{"car"}$

Output: false

Constraints:

- $1 \leq s.length, t.length \leq 5 * 10^4$
- s and t consist of lowercase English letters.

Follow up: What if the inputs contain Unicode characters? How would you adapt your solution to such a case?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isAnagram(self, s: str, t: str) -> bool:
        if len(s) != len(t):
            return False

        count = collections.Counter(s)
        count.subtract(collections.Counter(t))
        return all(freq == 0 for freq in count.values())
```

• LeetDoocs

```
class Solution:
    def isAnagram(self, s: str, t: str) -> bool:
        if len(s) != len(t):
            return False
        cnt = Counter(s)
        for c in t:
            cnt[c] -= 1
            if cnt[c] < 0:
                return False
        return True

class Solution:
    def isAnagram(self, s: str, t: str) -> bool:
        return Counter(s) == Counter(t)
```

• SimplyLeet

```
# Python solution for Valid Anagram

def isAnagram(s: str, t: str) -> bool:
    # Check if lengths differ; if so, they can't be anagrams
    if len(s) != len(t):
        return False

    # Create a dictionary to count frequency of each character in s
    char_count = {}
    for char in s:
```

Round 1 · High · Easy

p.107

```
# Increase the count of the character
char_count[char] = char_count.get(char, 0) + 1

# Decrease the count for each character in t
for char in t:
    if char not in char_count or char_count[char] == 0:
        # Character not found or count mismatch, not an anagram
        return False
    char_count[char] -= 1

# If all counts are zero, it's a valid anagram
return True

# Example usage:
print(isAnagram("anagram", "nagaram")) # Output: True
print(isAnagram("rat", "car"))          # Output: False
```

• LC.ca

```
class Solution:
    def isAnagram(self, s: str, t: str) -> bool:
        if len(s) != len(t):
            return False
        cnt = Counter(s)
        for c in t:
            cnt[c] -= 1
            if cnt[c] < 0:
                return False
        return True
```

◆ Quick Review

Key Insights

- If *s* and *t* have different lengths, they cannot be anagrams.

Round 1 · High · Easy

p.108

- An anagram requires exactly the same character frequency for both strings.
- Using a hash table (or frequency array when limited to 26 lowercase letters) provides an efficient solution.
- An alternative approach is to sort both strings and compare them.
- For Unicode inputs, a hash table is more adaptable than a fixed-size frequency array.

Space & Time

Time Complexity: $O(n)$, where n is the length of the strings (using a hash table for counting).

Space Complexity: $O(1)$ if we use a fixed-size array (for 26 letters), otherwise $O(n)$ for a hash table with a larger character set.

Solution

We use a hash table (dictionary) to count the frequency of each character in the first string s . Then, we iterate over string t and decrement the count for each character. If at any point a character count goes negative or a character is missing in the hash table, t cannot be an anagram of s . Finally, if all counts return to zero, the strings are anagrams. This approach

is efficient and handles the problem in linear time.

Strings

#344 Reverse String

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode

Two Pointers · String

Lists: AlgoMaster 600

Problem

Write a function that reverses a string. The input string is given as an array of characters *s*.

You must do this by modifying the input array in-place with $O(1)$ extra memory.

Example 1:

Input: *s* = ["h","e","l","l","o"]
Output: ["o","l","l","e","h"]

Example 2:

Input: *s* = ["H","a","n","n","a","h"]
Output: ["h","a","n","n","a","H"]

Constraints:

- $1 \leq s.length \leq 105$
- *s*[*i*] is a printable ascii character.

Community Solutions (Python)

- WalkCC

Round 1 · High · Easy

p.111

```
class Solution:
    def reverseString(self, s: list[str]) -> None:
        l = 0
        r = len(s) - 1

        while l < r:
            s[l], s[r] = s[r], s[l]
            l += 1
            r -= 1
```

• LeetDoocs

```
class Solution:
    def reverseString(self, s: List[str]) -> None:
        i, j = 0, len(s) - 1
        while i < j:
            s[i], s[j] = s[j], s[i]
            i, j = i + 1, j - 1
```

• SimplyLeet

```
def reverseString(s):
    # Initialize two pointers: one at the beginning (left) and one at the end (right)
    left, right = 0, len(s) - 1
    # Swap characters until the two pointers meet or cross
    while left < right:
        s[left], s[right] = s[right], s[left] # Swap the characters at the left and right positions
        left += 1 # Move the left pointer to the right
        right -= 1 # Move the right pointer to the left
```

Round 1 · High · Easy

p.112

Strings

☐ #392 Is Subsequence

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Two Pointers · String · Dynamic Programming
Lists: AlgoMaster 600

Problem

Given two strings *s* and *t*, return true if *s* is a subsequence of *t*, or false otherwise.

A subsequence of a string is a new string that is formed from the original string by deleting some (can be none) of the characters without disturbing the relative positions of the remaining characters. (i.e., "ace" is a subsequence of "abcde" while "aec" is not).

Example 1:

Input: *s* = "abc", *t* = "ahbgdc"
Output: true

Example 2:

Input: *s* = "axc", *t* = "ahbgdc"
Output: false

Constraints:

- $0 \leq s.length \leq 100$

Round 1 · High · Easy

p.113

- $0 \leq t.length \leq 104$
- *s* and *t* consist only of lowercase English letters.

Follow up: Suppose there are lots of incoming *s*, say *s*₁, *s*₂, ..., *s*_k where $k \geq 109$, and you want to check one by one to see if *t* has its subsequence. In this scenario, how would you change your code?

Community Solutions (Python)

☐ • WalkCC

```
class Solution:
    def isSubsequence(self, s: str, t: str) -> bool:
        if not s:
            return True

        i = 0
        for c in t:
            if s[i] == c:
                i += 1
            if i == len(s):
                return True

        return False
```

☐ • LeetDoocs

Round 1 · High · Easy

p.114

```
class Solution:
    def isSubsequence(self, s: str, t: str) -> bool:
        i = j = 0
        while i < len(s) and j < len(t):
            if s[i] == t[j]:
                i += 1
            j += 1
        return i == len(s)
```

• SimplyLeet

```
# Python solution with two pointers

def isSubsequence(s, t):
    # Initialize pointer for s
    s_index = 0
    # Iterate over each character in t
    for char in t:
        # If the pointer hasn't reached the end of s and the character matches
        if s_index < len(s) and s[s_index] == char:
            s_index += 1 # move pointer in s to next character

    # If all characters in s are found, return True
    if s_index == len(s):
        return True
    # Return whether the pointer reached the end of s after traversing t
    return s_index == len(s)

# Example usage:
print(isSubsequence("abc", "ahbgdc")) # Output: True
print(isSubsequence("axc", "ahbgdc")) # Output: False
```

Strings

□ #409 Longest Palindrome

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Greedy
Lists: Grind 169

Problem

Given a string *s* which consists of lowercase or uppercase letters, return the length of the longest palindrome that can be built with those letters.

Letters are case sensitive, for example, "Aa" is not considered a palindrome.

Example 1:

```
Input: s = "abcccccdd"
Output: 7
Explanation: One longest palindrome that can be built is "dccaccd", whose length is 7.
```

Example 2:

```
Input: s = "a"
Output: 1
Explanation: The longest palindrome that can be built is "a", whose length is 1.
```

Constraints:

- $1 \leq s.length \leq 2000$

- **s consists of lowercase and/or uppercase English letters only.**

Community Solutions (Python)

• WalkCC

```
class Solution:
    def longestPalindrome(self, s: str) -> int:
        ans = 0
        count = collections.Counter(s)

        for c in count.values():
            ans += c if c % 2 == 0 else c - 1

        hasOddCount = any(c % 2 == 1 for c in count.values())
        return ans + hasOddCount
```

• LeetDoocs

```
class Solution:
    def longestPalindrome(self, s: str) -> int:
        cnt = Counter(s)
        ans = sum(v // 2 * 2 for v in cnt.values())
        ans += int(ans < len(s))
        return ans
```

```
class Solution:
    def longestPalindrome(self, s: str) -> int:
        odd = defaultdict(int)
        cnt = 0
        for c in s:
            odd[c] ^= 1
            cnt += 1 if odd[c] else -1
        return len(s) - cnt + 1 if cnt else len(s)
```

• SimplyLeet

Python solution for the Longest Palindrome problem

```
def longestPalindrome(s: str) -> int:
    # Create a dictionary to store the frequency of each character
    char_counts = {}
    for char in s:
        # Increment the count for the character
        char_counts[char] = char_counts.get(char, 0) + 1
```

Variable to store the length of the palindrome

```
palindrome_length = 0
# Flag to indicate whether we can add one more character in the middle for
an odd count
center_added = False
```

Iterate through the character counts

```
for count in char_counts.values():
    # Add the largest even number from count to palindrome_length
    palindrome_length += (count // 2) * 2
    # If count is odd and we haven't added the center character yet
    if count % 2 == 1 and not center_added:
```

```
    palindrome_length += 1
    center_added = True
```

```
return palindrome_length
```

Example usage:

```
print(longestPalindrome("abcccd"))
```

• LC.ca

```
class Solution:
    def longestPalindrome(self, s: str) -> int:
        cnt = Counter(s)
        ans = 0
        for v in cnt.values():
            ans += v - (v & 1)
            ans += (ans & 1 ^ 1) and (v & 1)
        return ans
```

◆ Quick Review

Key Insights

- Count the frequency of each character.
- For each character, you can use pairs (even count or odd count minus one) to form the palindrome.
- If any character has an odd frequency, you can add one extra character as the center of the palindrome.

Space & Time

Time Complexity: $O(n)$, where n is the length of the string.

Space Complexity: $O(1)$ since the frequency counter will have at most 52 keys (considering both uppercase and lowercase letters).

Solution

The solution uses a hash table to count the frequency of each character. For every character, the maximum number of characters that can be used in a palindrome is determined by taking the largest even number that is less than or equal to its frequency (which is $\text{frequency} - (\text{frequency} \% 2)$). If there exists

any character with an odd frequency, one of these can be used as the center of the palindrome, adding an extra character to the total length. This greedy approach efficiently computes the length of the largest possible palindrome from the available characters.

Strings

#680 Valid Palindrome II

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Two Pointers · String · Greedy
Lists: AlgoMaster 600

Problem

Given a string s , return true if the s can be palindrome after deleting at most one character from it.

Example 1:

Input: $s = \text{"aba"}$
Output: true

Example 2:

Input: $s = \text{"abca"}$
Output: true
Explanation: You could delete the character 'c'.

Example 3:

Input: $s = \text{"abc"}$
Output: false

Constraints:

- $1 \leq s.length \leq 105$
- s consists of lowercase English letters.

Round 1 · High · Easy

p.121

Community Solutions (Python)

• WalkCC

```
class Solution:
    def validPalindrome(self, s: str) -> bool:
        def validPalindrome(l: int, r: int) -> bool:
            return all(s[i] == s[r - i + l] for i in range(l, (l + r) // 2 + 1))

        n = len(s)

        for i in range(n // 2):
            if s[i] != s[~i]:
                return validPalindrome(i + 1, n - 1 - i) or validPalindrome(i, n - 2 - i)

        return True
```

• LeetDoocs

```
class Solution:
    def validPalindrome(self, s: str) -> bool:
        def check(i, j):
            while i < j:
                if s[i] != s[j]:
                    return False
                i, j = i + 1, j - 1
            return True

        i, j = 0, len(s) - 1

        while i < j:
            if s[i] != s[j]:
                return check(i, j - 1) or check(i + 1, j)
            i, j = i + 1, j - 1
        return True
```

• SimplyLeet

Round 1 · High · Easy

p.122

```
# Define the Solution class
class Solution:
    def validPalindrome(self, s: str) -> bool:
        # Helper function to check if the substring s[left:right+1] is a
        # palindrome.
        def isPalindromeRange(left, right):
            while left < right:
                if s[left] != s[right]:
                    return False
                left += 1
                right -= 1
            return True

        # Initialize two pointers.
        left, right = 0, len(s) - 1

        # Use two pointers to compare characters from both ends.
        while left < right:
            # If characters don't match, try skipping one character
            if s[left] != s[right]:
                # Check if either skipping the left or the right character
                # results in a palindrome.
                return isPalindromeRange(left + 1, right) or isPalindromeRange(
                    left, right - 1)
            left += 1
            right -= 1
        return True
```

Strings

□ #796 Rotate String

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · String Matching
Lists: AlgoMaster 600

Problem

Given two strings s and $goal$, return true if and only if s can become $goal$ after some number of shifts on s .

A shift on s consists of moving the leftmost character of s to the rightmost position.

- For example, if $s = "abcde"$, then it will be $"bcdea"$ after one shift.

Example 1:

Input: $s = "abcde"$, $goal = "cdeab"$
Output: true

Example 2:

Input: $s = "abcde"$, $goal = "abcd"$
Output: false

Constraints:

- $1 \leq s.length, goal.length \leq 100$

- s and goal consist of lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def rotateString(self, s: str, goal: str) -> bool:
        return len(s) == len(goal) and goal in s + s
```

• LeetCode

```
class Solution:
    def rotateString(self, s: str, goal: str) -> bool:
        return len(s) == len(goal) and goal in s + s
```

• SimplyLeet

```
# Function to determine if s can become goal after some number of shifts
def rotateString(s, goal):
    # If lengths are different, s cannot be rotated to form goal
    if len(s) != len(goal):
        return False

    # Concatenate s with itself to cover all possible rotations
    doubled = s + s
    # Check if goal is a substring of the concatenated string
    return goal in doubled
```

```
# Example usage:
s = "abcde"
goal = "cdeab"
print(rotateString(s, goal)) # Output: True
```

Hash Tables

Round 1 · High · Easy · 11 questions

Round 1 · High · Easy

p.125

Round 1 · High · Easy

p.126

Hash Tables

☐ #205 Isomorphic Strings

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode

Hash Table · String

Lists: AlgoMaster 600

Problem

Given two strings *s* and *t*, determine if they are isomorphic.

Two strings *s* and *t* are isomorphic if the characters in *s* can be replaced to get *t*.

All occurrences of a character must be replaced with another character while preserving the order of characters. No two characters may map to the same character, but a character may map to itself.

Example 1:

Input: *s* = "egg", *t* = "add"

Output: true

Explanation:

The strings *s* and *t* can be made identical by:

- Mapping 'e' to 'a'.

Round 1 · High · Easy

p.127

- Mapping 'g' to 'd'.

Example 2:

Input: *s* = "f11", *t* = "b23"

Output: false

Explanation:

The strings *s* and *t* can not be made identical as '1' needs to be mapped to both '2' and '3'.

Example 3:

Input: *s* = "paper", *t* = "title"

Output: true

Constraints:

- $1 \leq s.length \leq 5 * 10^4$
- $t.length == s.length$
- *s* and *t* consist of any valid ascii character.

Community Solutions (Python)

☐ • WalkCC

```
class Solution:
    def isIsomorphic(self, s: str, t: str) -> bool:
        return [*map(s.index, s)] == [*map(t.index, t)]
```

☐ • LeetDoocs

Round 1 · High · Easy

p.128

```
class Solution:
    def isIsomorphic(self, s: str, t: str) -> bool:
        d1 = {}
        d2 = {}
        for a, b in zip(s, t):
            if (a in d1 and d1[a] != b) or (b in d2 and d2[b] != a):
                return False
            d1[a] = b
            d2[b] = a
        return True
```

```
class Solution:
    def isIsomorphic(self, s: str, t: str) -> bool:
        d1, d2 = [0] * 256, [0] * 256
        for i, (a, b) in enumerate(zip(s, t), 1):
            a, b = ord(a), ord(b)
            if d1[a] != d2[b]:
                return False
            d1[a] = d2[b] = i
        return True
```

• SimplyLeet

```
# Function to check if s and t are isomorphic
def is_isomorphic(s, t):
    # Dictionary to store mapping of characters from s to t
    mapping_st = {}
    # Dictionary to store mapping of characters from t to s
    mapping_ts = {}

    # Iterate over characters in s and t concurrently
    for char_s, char_t in zip(s, t):
        # Check mapping from s to t
```

```
if char_s in mapping_st:
    if mapping_st[char_s] != char_t:
        return False
else:
    mapping_st[char_s] = char_t
```

```
# Check mapping from t to s
if char_t in mapping_ts:
    if mapping_ts[char_t] != char_s:
        return False
```

```
else:
    mapping_ts[char_t] = char_s
```

```
return True
```

```
# Example usage:
print(is_isomorphic("egg", "add")) # Output: True
print(is_isomorphic("foo", "bar")) # Output: False
print(is_isomorphic("paper", "title")) # Output: True
```

Hash Tables

☐ #217 Contains Duplicate

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table · Sorting
 Lists: Grind 169

Problem

Given an integer array `nums`, return `true` if any value appears at least twice in the array, and return `false` if every element is distinct.

Example 1:

Input: `nums = [1,2,3,1]`

Output: `true`

Explanation:

The element 1 occurs at the indices 0 and 3.

Example 2:

Input: `nums = [1,2,3,4]`

Output: `false`

Explanation:

All elements are distinct.

Example 3:

Input: `nums = [1,1,1,3,3,4,3,2,4,2]`

Round 1 · High · Easy

p.131

Output: `true`

Constraints:

- `1 <= nums.length <= 105`
- `-109 <= nums[i] <= 109`

Community Solutions (Python)

☐ • WalkCC

```
class Solution:
    def containsDuplicate(self, nums: List[int]) -> bool:
        return len(nums) != len(set(nums))
```

☐ • LeetDoocs

```
class Solution:
    def containsDuplicate(self, nums: List[int]) -> bool:
        return any(a == b for a, b in pairwise(sorted(nums)))
```

☐ • SimplyLeet

```
# Function to determine if there are any duplicates in the list.
def contains_duplicate(nums):
    # Create a set to keep track of numbers seen so far.
    seen = set()
    # Iterate over each number in the list.
    for num in nums:
        # If num is already in seen, duplicate found.
        if num in seen:
            return True
        # Add num to the set.
```

Round 1 · High · Easy

p.132

```

    seen.add(num)
    # No duplicates found.
    return False

# Example usage:
if __name__ == "__main__":
    nums = [1, 2, 3, 1]
    print(contains_duplicate(nums)) # Expected output: True

```

• LC.ca

```

class Solution:
    def containsDuplicate(self, nums: List[int]) -> bool:
        return len(set(nums)) < len(nums)

#####

class Solution(object):
    def containsDuplicate(self, nums):
        """
        :type nums: List[int]

        :rtype: bool
        """
        nums.sort()
        for i in range(0, len(nums) - 1):
            if nums[i] == nums[i + 1]:
                return True
        return False

```

◆ Quick Review

Key Insights

- A hash set can be used to keep track of seen numbers.
- Iterating once through the array provides an efficient solution.

- If a number is already in the set, a duplicate is found.
- Alternative approaches like sorting have a higher time complexity ($O(n \log n)$).

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Solution

The solution uses a hash set to track the numbers seen while iterating through the array. For each number in the array:

- Check if it exists in the hash set. – If it does, return true immediately as a duplicate is found. – Otherwise, add the number to the hash set. If the iteration completes without finding any duplicates, return false. This approach efficiently determines if any duplicates exist in a single pass.

Hash Tables

#219 Contains Duplicate II

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table · Sliding Window
 Lists: AlgoMaster 600

Problem

Given an integer array `nums` and an integer `k`, return true if there are two distinct indices `i` and `j` in the array such that `nums[i] == nums[j]` and `abs(i - j) <= k`.

Example 1:

Input: `nums = [1,2,3,1]`, `k = 3`
 Output: true

Example 2:

Input: `nums = [1,0,1,1]`, `k = 1`
 Output: true

Example 3:

Input: `nums = [1,2,3,1,2,3]`, `k = 2`
 Output: false

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $-109 \leq \text{nums}[i] \leq 109$

Round 1 · High · Easy

p.135

- $0 \leq k \leq 105$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def containsNearbyDuplicate(self, nums: list[int], k: int) -> bool:
        seen = set()

        for i, num in enumerate(nums):
            if i > k:
                seen.remove(nums[i - k - 1])
            if num in seen:
                return True
            seen.add(num)

        return False
```

• LeetDoocs

```
class Solution:
    def containsNearbyDuplicate(self, nums: List[int], k: int) -> bool:
        d = {}
        for i, x in enumerate(nums):
            if x in d and i - d[x] <= k:
                return True
            d[x] = i
        return False
```

• SimplyLeet

Round 1 · High · Easy

p.136


```
# Python solution with detailed comments
def containsNearbyDuplicate(nums, k):
    # Create a dictionary to store the number and its most recent index
    index_map = {}
    # Loop over every index and corresponding number in nums
    for i, num in enumerate(nums):
        # Check if the number is already in the dictionary
        if num in index_map:
            # If the difference between the current index and the stored index
            # is <= k, return True
            if i - index_map[num] <= k:
                return True
        # Update the dictionary with the current index for the number
        index_map[num] = i
    # If no such duplicate pair is found, return False
    return False

# Example usage:
# print(containsNearbyDuplicate([1,2,3,1], 3))
```

Hash Tables

□ #290 Word Pattern

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String
Lists: AlgoMaster 600

Problem

Given a pattern and a string *s*, find if *s* follows the same pattern.

Here follow means a full match, such that there is a bijection between a letter in pattern and a non-empty word in *s*. Specifically:

- Each letter in pattern maps to exactly one unique word in *s*.
- Each unique word in *s* maps to exactly one letter in pattern.
- No two letters map to the same word, and no two words map to the same letter.

Example 1:

Input: pattern = "abba", s = "dog cat cat dog"

Output: true

Explanation:

The bijection can be established as:

- 'a' maps to "dog".
- 'b' maps to "cat".

Example 2:

Input: pattern = "abba", s = "dog cat cat fish"

Output: false

Example 3:

Input: pattern = "aaaa", s = "dog cat cat dog"

Output: false

Constraints:

- $1 \leq \text{pattern.length} \leq 300$
- pattern contains only lower-case English letters.
- $1 \leq \text{s.length} \leq 3000$
- s contains only lowercase English letters and spaces ' '.
- s does not contain any leading or trailing spaces.
- All the words in s are separated by a single space.

Community Solutions (Python)

- WalkCC

Round 1 · High · Easy

p.139

```
class Solution:
    def wordPattern(self, pattern: str, str: str) -> bool:
        t = str.split()
        return [*map(pattern.index, pattern)] == [*map(t.index, t)]
```

• LeetCode

```
class Solution:
    def wordPattern(self, pattern: str, s: str) -> bool:
        ws = s.split()
        if len(pattern) != len(ws):
            return False
        d1 = {}
        d2 = {}
        for a, b in zip(pattern, ws):
            if (a in d1 and d1[a] != b) or (b in d2 and d2[b] != a):
                return False
            d1[a] = b
            d2[b] = a
        return True
```

• SimplyLeet

```
# Python solution with line-by-line comments

def wordPattern(pattern: str, s: str) -> bool:
    # Split the string s based on spaces to get the list of words
    words = s.split()

    # If the number of words is not equal to the length of the pattern, the
    # mapping is impossible
    if len(pattern) != len(words):
        return False
```

Round 1 · High · Easy

p.140

```

# Initialize two dictionaries to maintain the mapping
char_to_word = {}
word_to_char = {}

# Traverse each character from the pattern along with the corresponding word
for ch, word in zip(pattern, words):
    # If the mapping for character doesn't exist, check and assign
    if ch not in char_to_word:
        if word in word_to_char:
            # Word is already mapped to a different character, hence return
            False

        return False
    # Create mapping in both dictionaries
    char_to_word[ch] = word
    word_to_char[word] = ch
else:
    # If the character already has a mapping, validate it against the current word
    if char_to_word[ch] != word:
        return False
    # If all mappings are consistent, return True
    return True

# Example usage:
print(wordPattern("abba", "dog cat cat dog")) # Expected output: True
print(wordPattern("abba", "dog cat cat fish")) # Expected output: False

```

Hash Tables

□ #350 Intersection of Two Arrays II

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table · Two Pointers · Binary Search · Sorting
 Lists: AlgoMaster 600

Problem

Given two integer arrays `nums1` and `nums2`, return an array of their intersection. Each element in the result must appear as many times as it shows in both arrays and you may return the result in any order.

Example 1:

Input: `nums1 = [1,2,2,1]`, `nums2 = [2,2]`
 Output: `[2,2]`

Example 2:

Input: `nums1 = [4,9,5]`, `nums2 = [9,4,9,8,4]`
 Output: `[4,9]`
 Explanation: `[9,4]` is also accepted.

Constraints:

- `1 <= nums1.length, nums2.length <= 1000`
- `0 <= nums1[i], nums2[i] <= 1000`

Follow up:

- What if the given array is already sorted? How would you optimize your algorithm?
- What if nums1's size is small compared to nums2's size? Which algorithm is better?
- What if elements of nums2 are stored on disk, and the memory is limited such that you cannot load all elements into the memory at once?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def intersect(self, nums1: List[int], nums2: List[int]) -> List[int]:
        if len(nums1) > len(nums2):
            return self.intersect(nums2, nums1)

        ans = []
        count = collections.Counter(nums1)

        for num in nums2:
            if count[num] > 0:

                ans.append(num)
                count[num] -= 1

        return ans
```

• LeetCode

```
class Solution:
    def intersect(self, nums1: List[int], nums2: List[int]) -> List[int]:
        cnt = Counter(nums1)
        ans = []
        for x in nums2:
            if cnt[x]:
                ans.append(x)
                cnt[x] -= 1
        return ans
```

• SimplyLeet

```
# Function to compute the intersection of two arrays using a hash table
def intersect(nums1, nums2):
    # Create a dictionary to store frequency of elements in nums1
    frequency = {}
    for num in nums1:
        frequency[num] = frequency.get(num, 0) + 1

    # List to store the intersection result
    result = []
    # Iterate through nums2, and check if the element is in frequency with a
    # positive count

    for num in nums2:
        if frequency.get(num, 0) > 0:
            result.append(num)
            frequency[num] -= 1
    return result

# Example usage:
# print(intersect([1,2,2,1], [2,2]))
```

Hash Tables

#383 Ransom Note

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Counting
Lists: Grind 169

Problem

Given two strings ransomNote and magazine, return true if ransomNote can be constructed by using the letters from magazine and false otherwise.

Each letter in magazine can only be used once in ransomNote.

Example 1:

Input: ransomNote = "a", magazine = "b"
Output: false

Example 2:

Input: ransomNote = "aa", magazine = "ab"
Output: false

Example 3:

Input: ransomNote = "aa", magazine = "aab"
Output: true

Constraints:

Round 1 · High · Easy

p.145

- 1 <= ransomNote.length, magazine.length <= 105
- ransomNote and magazine consist of lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def canConstruct(self, ransomNote: str, magazine: str) -> bool:
        count1 = collections.Counter(ransomNote)
        count2 = collections.Counter(magazine)
        return all(count1[c] <= count2[c] for c in string.ascii_lowercase)
```

• LeetDoocs

```
class Solution:
    def canConstruct(self, ransomNote: str, magazine: str) -> bool:
        cnt = Counter(magazine)
        for c in ransomNote:
            cnt[c] -= 1
            if cnt[c] < 0:
                return False
        return True
```

• SimplyLeet

```
# Function to check if ransom note can be constructed from magazine
def canConstruct(ransomNote: str, magazine: str) -> bool:
    # Create a dictionary to hold the frequency of each character in magazine
    char_frequency = {}
    for char in magazine:
        char_frequency[char] = char_frequency.get(char, 0) + 1

    # Check each character in ransomNote against the frequency map
    for char in ransomNote:
        if char_frequency.get(char, 0) == 0:
```

Round 1 · High · Easy

p.146

```
    return False # Character not available or used up
    char_frequency[char] -= 1 # Use the character
```

```
    return True
```

```
# Example usage:
```

```
if __name__ == "__main__":
    print(canConstruct("aa", "aab")) # Expected output: True
```

• LC.ca

```
class Solution:
    def canConstruct(self, ransomNote: str, magazine: str) -> bool:
        cnt = Counter(magazine)
        for c in ransomNote:
            cnt[c] -= 1
            if cnt[c] < 0:
                return False
        return True
```

◆ Quick Review

Key Insights

- Use a hash table (or dictionary) to store the frequency of each character present in magazine.
- Iterate over every character in ransomNote and check if the corresponding count in the hash table is available and sufficient.
- Decrease the character count for every match; if any character count falls below zero, return false.

Round 1 · High · Easy

p.147

- If all characters in ransomNote can be accounted for, return true.

Space & Time

Time Complexity: $O(n + m)$, where n is the length of magazine and m is the length of ransomNote.

Space Complexity: $O(1)$, since the extra space required for the hash table is limited to at most 26 entries for lowercase English letters.

Solution

We construct a frequency map for the magazine string using a hash table. This frequency map records how many times each letter appears in magazine. Then we iterate through the ransomNote string, and for every letter, we check the corresponding frequency in the map. If a letter is not found or its frequency is zero, we immediately return false as it means the letter cannot be used to build the note. Otherwise, we decrement the frequency count by one. If we successfully process all characters in ransomNote, then it can be constructed from magazine, and we return true.

Round 1 · High · Easy

p.148

Hash Tables

☐ #387 First Unique Character in a String EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Queue · Counting
Lists: AlgoMaster 600

Problem

Given a string *s*, find the first non-repeating character in it and return its index. If it does not exist, return -1.

Example 1:

Input: *s* = "leetcode"

Output: 0

Explanation:

The character 'l' at index 0 is the first character that does not occur at any other index.

Example 2:

Input: *s* = "loveleetcode"

Output: 2

Example 3:

Input: *s* = "aabb"

Output: -1

Round 1 · High · Easy

p.149

Constraints:

- $1 \leq s.length \leq 105$
- *s* consists of only lowercase English letters.

Community Solutions (Python)

☐ • WalkCC

```
class Solution:
    def firstUniqChar(self, s: str) -> int:
        count = collections.Counter(s)

        for i, c in enumerate(s):
            if count[c] == 1:
                return i

        return -1
```

☐ • LeetDoocs

```
class Solution:
    def firstUniqChar(self, s: str) -> int:
        cnt = Counter(s)
        for i, c in enumerate(s):
            if cnt[c] == 1:
                return i
        return -1
```

☐ • SimplyLeet

```
def firstUniqChar(s):
    # Create a frequency dictionary to count occurrences of each character
    char_count = {}
    # First pass: count every character in the string
    for char in s:
        if char in char_count:
            char_count[char] += 1
        else:
            char_count[char] = 1
    # Second pass: find the first character that occurs exactly once
```

Round 1 · High · Easy

p.150

```

for index, char in enumerate(s):
    if char_count[char] == 1:
        return index
# Return -1 if no unique character is found
return -1

# Example usage:
print(firstUniqChar("leetcode")) # Expected output: 0
print(firstUniqChar("loveleetcode")) # Expected output: 2

```

Hash Tables

□ #448 Find All Numbers Disappeared in an Array

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table
 Lists: AlgoMaster 600

Problem

Given an array `nums` of n integers where `nums[i]` is in the range $[1, n]$, return an array of all the integers in the range $[1, n]$ that do not appear in `nums`.

Example 1:

Input: `nums = [4,3,2,7,8,2,3,1]`
 Output: `[5,6]`

Example 2:

Input: `nums = [1,1]`
 Output: `[2]`

Constraints:

- $n == \text{nums.length}$
- $1 \leq n \leq 105$
- $1 \leq \text{nums}[i] \leq n$

Follow up: Could you do it without extra space and in $O(n)$ runtime? You may assume the returned list does not count as extra space.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findDisappearedNumbers(self, nums: List[int]) -> List[int]:
        for num in nums:
            index = abs(num) - 1
            nums[index] = -abs(nums[index])

        return [i + 1 for i, num in enumerate(nums) if num > 0]
```

• LeetCode

```
class Solution:
    def findDisappearedNumbers(self, nums: List[int]) -> List[int]:
        s = set(nums)
        return [x for x in range(1, len(nums) + 1) if x not in s]
```

```
class Solution:
    def findDisappearedNumbers(self, nums: List[int]) -> List[int]:
        for x in nums:
            i = abs(x) - 1
            if nums[i] > 0:
                nums[i] *= -1
        return [i + 1 for i in range(len(nums)) if nums[i] > 0]
```

• SimplyLeet

Round 1 · High · Easy

p.153

```
# Function to find all disappeared numbers in an array
def findDisappearedNumbers(nums):
    # Iterate over each number in the list
    for i in range(len(nums)):
        # Get the index corresponding to the current number (adjusting for
        # 0-indexed array)
        index = abs(nums[i]) - 1
        # Mark the number at this index as negative if it is not already marked
        if nums[index] > 0:
            nums[index] = -nums[index]

    # Initialize the result list
    result = []
    # Iterate over the array to collect indices of numbers that did not appear
    for i, num in enumerate(nums):
        # If the number is positive, then index + 1 is missing in the original
        # array
        if num > 0:
            result.append(i + 1)

    # Return the list of missing numbers
    return result

# Example usage:
# print(findDisappearedNumbers([4,3,2,7,8,2,3,1])) # Output: [5,6]
```

Round 1 · High · Easy

p.154

Hash Tables

#706 Design HashMap

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Linked List · Design ·
Hash Function

Lists: Denny Zhang

Problem

Design a HashMap without using any built-in hash table libraries.

Implement the MyHashMap class:

- MyHashMap() initializes the object with an empty map.
- void put(int key, int value) inserts a (key, value) pair into the HashMap. If the key already exists in the map, update the corresponding value.
- int get(int key) returns the value to which the specified key is mapped, or -1 if this map contains no mapping for the key.
- void remove(key) removes the key and its corresponding value if the map contains the

Round 1 · High · Easy

p.155

mapping for the key.

Example 1:

```
Input
["MyHashMap", "put", "put", "get", "get", "put", "get", "remove", "get"]
[[], [1, 1], [2, 2], [1], [3], [2, 1], [2], [2], [2]]
Output
[null, null, null, 1, -1, null, 1, null, -1]
```

```
Explanation
MyHashMap myHashMap = new MyHashMap();
```

Constraints:

- $0 \leq \text{key}, \text{value} \leq 106$
- At most 104 calls will be made to put, get, and remove.

Community Solutions (Python)

• LeetDoocs

```
class MyHashMap:
    def __init__(self):
        self.data = [-1] * 1000001

    def put(self, key: int, value: int) -> None:
        self.data[key] = value

    def get(self, key: int) -> int:
        return self.data[key]
```

Round 1 · High · Easy

p.156

```
def remove(self, key: int) -> None:
    self.data[key] = -1
```

```
# Your MyHashMap object will be instantiated and called as such:
# obj = MyHashMap()
# obj.put(key,value)
# param_2 = obj.get(key)
# obj.remove(key)
```

• SimplyLeet

```
# ListNode class for handling collisions in each bucket
```

```
class ListNode:
    def __init__(self, key, value, next=None):
        self.key = key        # Store key
        self.value = value    # Store value
        self.next = next      # Pointer to the next node
```

```
class MyHashMap:
    def __init__(self):
        self.size = 1000      # Define the bucket size
```

```
        self.buckets = [None] * self.size    # Initialize buckets as an array
of None
```

```
def hash(self, key):
    # Simple modulo based hash function to map keys to bucket indices
    return key % self.size
```

```
def put(self, key: int, value: int) -> None:
    index = self.hash(key)
    if self.buckets[index] is None:
        # If the bucket is empty, create a new node and place it here
```

```
        self.buckets[index] = ListNode(key, value)
    else:
        current = self.buckets[index]
        while True:
            if current.key == key:
                # If key exists, update its value
                current.value = value
                return
            if current.next is None:
                break
```

```
        current = current.next
    # Append a new node at the end of the list if key not found
    current.next = ListNode(key, value)
```

```
def get(self, key: int) -> int:
    index = self.hash(key)
    current = self.buckets[index]
    while current:
        if current.key == key:
            # Key found; return its value
```

```
            return current.value
        current = current.next
    # Key not found; return -1
    return -1
```

```
def remove(self, key: int) -> None:
    index = self.hash(key)
    current = self.buckets[index]
    if current is None:
        return
```

```
    if current.key == key:
        # Remove head node if it contains the key
        self.buckets[index] = current.next
    else:
        prev = current
        current = current.next
        while current:
            if current.key == key:
                # Bypass the node to remove it from the list
                prev.next = current.next
```

```

        return
    prev, current = current, current.next

```

• LC.ca

```

class MyHashMap:
    def __init__(self):
        self.data = [-1] * 1000001

    def put(self, key: int, value: int) -> None:
        self.data[key] = value

    def get(self, key: int) -> int:
        return self.data[key]

```

```

    def remove(self, key: int) -> None:
        self.data[key] = -1

```

```

# Your MyHashMap object will be instantiated and called as such:
# obj = MyHashMap()
# obj.put(key,value)
# param_2 = obj.get(key)
# obj.remove(key)

```

```

#####

```

```

class MyHashMap: # hash to bucket
    def __init__(self):
        """
        Initialize your data structure here.
        """
        self.size = 1000
        self.buckets = [[] for _ in range(self.size)]

```

```

def _hash(self, key: int) -> int:
    """
    Generate a hash for a given key.
    """
    return key % self.size

def put(self, key: int, value: int) -> None:
    """
    Value will always be non-negative.
    """

```

```

    hash_key = self._hash(key)
    for i, (k, v) in enumerate(self.buckets[hash_key]):
        if k == key:
            self.buckets[hash_key][i] = (key, value)
            return
    self.buckets[hash_key].append((key, value))

```

```

def get(self, key: int) -> int:
    """
    Returns the value to which the specified key is mapped, or -1 if this
    map contains no mapping for the key.

```

```

    """
    hash_key = self._hash(key)
    for k, v in self.buckets[hash_key]:
        if k == key:
            return v
    return -1

```

```

def remove(self, key: int) -> None:
    """
    Removes the mapping of the specified value key if this map contains a
    mapping for the key.

```

```

    """
    hash_key = self._hash(key)
    for i, (k, v) in enumerate(self.buckets[hash_key]):
        if k == key:
            del self.buckets[hash_key][i]
            return

#####

class MyHashMap: # open addressing with linear probing

    def __init__(self):
        self.capacity = 10000
        self.data = [None] * self.capacity
        self.DELETED = (None, None) # Marker for deleted entries

    def _hash(self, key):
        return key % self.capacity

    def put(self, key: int, value: int) -> None:
        idx = self._hash(key)

        while self.data[idx] not in (None, self.DELETED, key):
            idx = (idx + 1) % self.capacity
        self.data[idx] = (key, value)

    def get(self, key: int) -> int:
        idx, found_key = self._find(key)
        if found_key:
            return self.data[idx][1]
        return -1

    def remove(self, key: int) -> None:
        idx, found_key = self._find(key)
        if found_key:
            self.data[idx] = self.DELETED

    def _find(self, key):
        original_idx = idx = self._hash(key)
        while self.data[idx] is not None:
            if self.data[idx] != self.DELETED and self.data[idx][0] == key:
                return idx, True

```

```

        idx = (idx + 1) % self.capacity
        if idx == original_idx: # Came full circle
            break
        return original_idx, False

```

◆ Quick Review

Key Insights

- Use a fixed-size array of buckets where each bucket uses a linked list to handle collisions.
- Choose a simple hash function (e.g., modulo operation) to map a key to its corresponding bucket.
- For each bucket, traverse the linked list to update an existing key or append a new node if the key does not exist.
- Removal requires updating pointers in the linked list to bypass the removed node.

Space & Time

Time Complexity: Average $O(1)$ for put, get, and remove; worst-case $O(n)$ when many keys hash to the same bucket.

Space Complexity: $O(n)$, where n is the number of key-value pairs stored.

Solution

We implement the HashMap using an array of fixed size (e.g., 1000). Each array element is the head of a linked list that manages collisions using chaining. The key is hashed using a modulo operation. For the put operation, if a key already exists in the chain, its value is updated; otherwise, a new node is appended. The get operation traverses the chain at the computed bucket index to find and return the corresponding value, or returns -1 if not found. The remove operation also traverses the list, carefully reconnecting the links to remove the target node while handling edge cases such as removal at the head.

Hash Tables

□ #1189 Maximum Number of Balloons

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Counting
Lists: AlgoMaster 600

Problem

Given a string text, you want to use the characters of text to form as many instances of the word "balloon" as possible.

You can use each character in text at most once. Return the maximum number of instances that can be formed.

Example 1:

Input: text = "nlaebolko"
Output: 1

Example 2:

Input: text = "loonbalxballpoon"
Output: 2

Example 3:

Input: text = "leetcode"
Output: 0

Constraints:

- $1 \leq \text{text.length} \leq 104$
- text consists of lower case English letters only.

Note: This question is the same as 2287: Rearrange Characters to Make Target String.

Community Solutions (Python)

• LeetCode

```
class Solution:
    def maxNumberOfBalloons(self, text: str) -> int:
        cnt = Counter(text)
        cnt['o'] >= 1
        cnt['l'] >= 1
        return min(cnt[c] for c in 'balon')
```

• SimplyLeet

```
from collections import Counter

def maxNumberOfBalloons(text: str) -> int:
    # Count frequency of each character in the text.
    char_count = Counter(text)
    # Calculate the number of possible "balloon" words for each character,
    # noting that 'l' and 'o' are needed twice.
    count_b = char_count.get('b', 0)
    count_a = char_count.get('a', 0)
    count_l = char_count.get('l', 0) // 2
    count_o = char_count.get('o', 0) // 2

    count_n = char_count.get('n', 0)
    # The maximum number of balloons is limited by the smallest count from
    # required characters.
    return min(count_b, count_a, count_l, count_o, count_n)

# Example usage:
print(maxNumberOfBalloons("nlaebolko")) # Output: 1
```

Hash Tables

#1512 Number of Good Pairs

EAS

LeetCode · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table · Math · Counting
 Lists: AlgoMaster 600

Problem

Given an array of integers nums, return the number of good pairs.

A pair (i, j) is called good if $\text{nums}[i] == \text{nums}[j]$ and $i < j$.

Example 1:

Input: nums = [1,2,3,1,1,3]
 Output: 4
 Explanation: There are 4 good pairs (0,3), (0,4), (3,4), (2,5) 0-indexed.

Example 2:

Input: nums = [1,1,1,1]
 Output: 6
 Explanation: Each pair in the array are good.

Example 3:

Input: nums = [1,2,3]
 Output: 0

Constraints:

- $1 \leq \text{nums.length} \leq 100$

- $1 \leq \text{nums}[i] \leq 100$

Community Solutions (Python)

• LeetCode

```
class Solution:
    def numIdenticalPairs(self, nums: List[int]) -> int:
        ans = 0
        cnt = Counter()
        for x in nums:
            ans += cnt[x]
            cnt[x] += 1
        return ans
```

• SimplyLeet

```
# Python solution to count the number of good pairs in an array

def num_identical_pairs(nums):
    # Dictionary to store the frequency of each number
    frequency = {}
    count = 0
    # Iterate over each number in the list
    for num in nums:
        # If the number is already seen, add its current frequency to count
        if num in frequency:
            count += frequency[num]
            frequency[num] += 1 # Increment the frequency
        else:
            frequency[num] = 1 # Initialize frequency for new number
    return count

# Example usage:
nums = [1, 2, 3, 1, 1, 3]
print(num_identical_pairs(nums)) # Expected output: 4
```

Two Pointers

Round 1 · High · Easy · 7 questions

Round 1 · High · Easy

p.167

Round 1 · High · Easy

p.168

Two Pointers

#1 Two Sum

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table
 Lists: Grind 169

Problem

Given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`.

You may assume that each input would have exactly one solution, and you may not use the same element twice.

You can return the answer in any order.

Example 1:

Input: `nums = [2,7,11,15]`, `target = 9`
 Output: `[0,1]`
 Explanation: Because `nums[0] + nums[1] == 9`, we return `[0, 1]`.

Example 2:

Input: `nums = [3,2,4]`, `target = 6`
 Output: `[1,2]`

Example 3:

Input: `nums = [3,3]`, `target = 6`
 Output: `[0,1]`

Round 1 · High · Easy

p.169

Constraints:

- $2 \leq \text{nums.length} \leq 104$
- $-109 \leq \text{nums}[i] \leq 109$
- $-109 \leq \text{target} \leq 109$
- Only one valid answer exists.

Follow-up: Can you come up with an algorithm that is less than $O(n^2)$ time complexity?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        numToIndex = {}

        for i, num in enumerate(nums):
            if target - num in numToIndex:
                return numToIndex[target - num], i
            numToIndex[num] = i
```

• LeetDoocs

```
class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        d = {}
        for i, x in enumerate(nums):
            if (y := target - x) in d:
                return [d[y], i]
            d[x] = i
```

• SimplyLeet

Round 1 · High · Easy

p.170

```
def two_sum(nums, target):
    # Create a dictionary to store the number and its index
    num_to_index = {}
    # Iterate over the list with index
    for index, num in enumerate(nums):
        # Calculate the complement needed to reach the target
        complement = target - num
        # Check if the complement is in the dictionary
        if complement in num_to_index:
            # Return the index of the complement and the current index

            return [num_to_index[complement], index]
        # Store the number with its index
        num_to_index[num] = index
    # Since the problem guarantees one solution, we don't need a return
    # statement here.

# Example usage:
# print(two_sum([2, 7, 11, 15], 9)) # Output: [0, 1]
```

• LC.ca

```
class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        m = {}
        for i, x in enumerate(nums):
            y = target - x
            if y in m:
                return [m[y], i]
            m[x] = i
```

◆ Quick Review

Key Insights

- Use a hash table to store the elements and their indices for fast lookup.

- For each element, calculate its complement: target minus the current element.
- Check if the complement exists in the hash table.
- If found, return the indices of the current element and its complement.
- This approach only requires a single pass through the array.

Space & Time

Time Complexity: $O(n)$ – We traverse the list only once.

Space Complexity: $O(n)$ – In the worst-case, we store all elements in the hash table.

Solution

We use a hash table (dictionary) to map each array element to its index as we iterate. For each number, we compute the complement (target – number) and check if it already exists in our hash table. If the complement is found, we return the indices. This one-pass solution ensures linear time complexity and avoids the need for nested loops.

Two Pointers

☐ #88 Merge Sorted Array

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Two Pointers · Sorting
 Lists: Denny Zhang

Problem

You are given two integer arrays `nums1` and `nums2`, sorted in non-decreasing order, and two integers `m` and `n`, representing the number of elements in `nums1` and `nums2` respectively.

Merge `nums1` and `nums2` into a single array sorted in non-decreasing order.

The final sorted array should not be returned by the function, but instead be stored inside the array `nums1`. To accommodate this, `nums1` has a length of `m + n`, where the first `m` elements denote the elements that should be merged, and the last `n` elements are set to 0 and should be ignored. `nums2` has a length of `n`.

Example 1:

Input: `nums1 = [1,2,3,0,0,0]`, `m = 3`, `nums2 = [2,5,6]`, `n = 3`
 Output: `[1,2,2,3,5,6]`
 Explanation: The arrays we are merging are `[1,2,3]` and `[2,5,6]`.
 The result of the merge is `[1,2,2,3,5,6]` with the underlined elements coming from `nums1`.

Example 2:

Input: `nums1 = [1]`, `m = 1`, `nums2 = []`, `n = 0`
 Output: `[1]`
 Explanation: The arrays we are merging are `[1]` and `[]`.
 The result of the merge is `[1]`.

Example 3:

Input: `nums1 = [0]`, `m = 0`, `nums2 = [1]`, `n = 1`
 Output: `[1]`
 Explanation: The arrays we are merging are `[]` and `[1]`.
 The result of the merge is `[1]`.
 Note that because `m = 0`, there are no elements in `nums1`. The 0 is only there to ensure the merge result can fit in `nums1`.

Constraints:

- `nums1.length == m + n`
- `nums2.length == n`
- `0 <= m, n <= 200`
- `1 <= m + n <= 200`
- `-109 <= nums1[i], nums2[j] <= 109`

Follow up: Can you come up with an algorithm that runs in $O(m + n)$ time?

Community Solutions (Python)

- WalkCC

```
class Solution:
    def merge(self, nums1: list[int], m: int, nums2: list[int], n: int) -> None:
        i = m - 1 # nums1's index (the actual nums)
        j = n - 1 # nums2's index
        k = m + n - 1 # nums1's index (the next filled position)

        while j >= 0:
            if i >= 0 and nums1[i] > nums2[j]:
                nums1[k] = nums1[i]
                k -= 1
            else:
                nums1[k] = nums2[j]
                k -= 1
                j -= 1
            i -= 1
```

• LeetCode

```
class Solution:
    def merge(self, nums1: List[int], m: int, nums2: List[int], n: int) -> None:
        k = m + n - 1
        i, j = m - 1, n - 1
        while j >= 0:
            if i >= 0 and nums1[i] > nums2[j]:
                nums1[k] = nums1[i]
                i -= 1
            else:
                nums1[k] = nums2[j]
                j -= 1
            k -= 1
```

• SimplyLeet

Round 1 · High · Easy

p.175

```
def merge(nums1, m, nums2, n):
    # Initialize pointers for nums1, nums2 and the tail index
    p1 = m - 1
    p2 = n - 1
    tail = m + n - 1

    # Merge arrays from the end by choosing the larger element
    while p1 >= 0 and p2 >= 0:
        if nums1[p1] > nums2[p2]:
            nums1[tail] = nums1[p1]
            p1 -= 1 # Move pointer in nums1
        else:
            nums1[tail] = nums2[p2]
            p2 -= 1 # Move pointer in nums2
            tail -= 1 # Move tail pointer

    # Copy any remaining elements from nums2 into nums1
    while p2 >= 0:
        nums1[tail] = nums2[p2]
        p2 -= 1
        tail -= 1
```

• LC.ca

```
class Solution:
    def merge(self, nums1: List[int], m: int, nums2: List[int], n: int) -> None:
        """
        Do not return anything, modify nums1 in-place instead.
        """
        i, j, k = m - 1, n - 1, m + n - 1
        # what if j=-1 already but k is not 0 yet? e.g. m=[1,2,0], n=[3]
        # => then no more ops needed, rest of m[] already sorted, so this while
        # is good enough
        while j >= 0:
            # i could be -1, eg. [7,8,9, ] and [1]
```

Round 1 · High · Easy

p.176

```

if i >= 0 and nums1[i] > nums2[j]:
    nums1[k] = nums1[i]
    i -= 1
else:
    nums1[k] = nums2[j]
    j -= 1
k += 1

```

◆ Quick Review

Key Insights

- Both arrays are already sorted in non-decreasing order.
- The merge should be done in-place into nums1.
- Start merging from the end to avoid overwriting elements in nums1.
- Use two pointers, one for nums1 and one for nums2, and a tail pointer for placement.

Space & Time

Time Complexity: $O(m + n)$

Space Complexity: $O(1)$

Solution

We use a two-pointer approach starting from the end of both arrays. Initialize three pointers:

- p1 pointing to the last valid element in nums1.
- p2 pointing to the last element in nums2.
- tail pointing to the last index of nums1.

While both pointers are valid, compare the elements pointed by p1 and p2, placing the larger element at the tail position. Decrement the corresponding pointer and the tail pointer. Finally, if any elements remain in nums2 (i.e., $p2 \geq 0$), copy them over into nums1. This solution leverages the extra space at the end of nums1 and ensures the merge is done in-place in linear time.

Two Pointers

★ #408 Valid Word Abbreviation ★

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode

Two Pointers · String

Lists: ★ Premium | Premium 98

Problem

A string can be abbreviated by replacing any number of non-adjacent, non-empty substrings with their lengths. The lengths should not have leading zeros.

For example, a string such as "substitution" could be abbreviated as (but not limited to):

- "s10n" ("s ubstitutio n")
- "sub4u4" ("sub stit u tion")
- "12" ("substitution")
- "su3i1u2on" ("su bst i t u ti on")
- "substitution" (no substrings replaced)

The following are not valid abbreviations:

- "s55n" ("s ubsti tutio n", the replaced substrings are adjacent)
- "s010n" (has leading zeros)

Round 1 · High · Easy

p.179

- "s0ubstitution" (replaces an empty substring)

Given a string word and an abbreviation abbr, return whether the string matches the given abbreviation.

A substring is a contiguous non-empty sequence of characters within a string.

Example 1:

```
Input: word = "internationalization", abbr = "i12iz4n"
Output: true
Explanation: The word "internationalization" can be abbreviated as "i12iz4n"
("i nternational iz atio n").
```

Example 2:

```
Input: word = "apple", abbr = "a2e"
Output: false
Explanation: The word "apple" cannot be abbreviated as "a2e".
```

Constraints:

- $1 \leq \text{word.length} \leq 20$
- word consists of only lowercase English letters.
- $1 \leq \text{abbr.length} \leq 10$
- abbr consists of lowercase English letters and digits.
- All the integers in abbr will fit in a 32-bit integer.

Round 1 · High · Easy

p.180

Community Solutions (Python)

• WalkCC

```
class Solution:
    def validWordAbbreviation(self, word: str, abbr: str) -> bool:
        i = 0 # word's index
        j = 0 # abbr's index

        while i < len(word) and j < len(abbr):
            if word[i] == abbr[j]:
                i += 1
                j += 1
                continue

            if not abbr[j].isdigit() or abbr[j] == '0':
                return False
            num = 0
            while j < len(abbr) and abbr[j].isdigit():
                num = num * 10 + int(abbr[j])
                j += 1
            i += num

        return i == len(word) and j == len(abbr)
```

• LeetDoocs

```
class Solution:
    def validWordAbbreviation(self, word: str, abbr: str) -> bool:
        m, n = len(word), len(abbr)
        i = j = x = 0
        while i < m and j < n:
            if abbr[j].isdigit():
                if abbr[j] == "0" and x == 0:
                    return False
                x = x * 10 + int(abbr[j])
            else:
                if abbr[j] != word[i]:
                    return False
                i += 1
                j += 1
        return i == m and j == n
```

Round 1 · High · Easy

p.181

```
i += x
x = 0
if i >= m or word[i] != abbr[j]:
    return False
i += 1
j += 1
return i + x == m and j == n
```

• SimplyLeet

```
# Python solution for valid word abbreviation

def validWordAbbreviation(word: str, abbr: str) -> bool:
    word_index = 0 # pointer for the word
    abbr_index = 0 # pointer for the abbreviation

    while abbr_index < len(abbr):
        # if current abbrev char is a digit
        if abbr[abbr_index].isdigit():
            # leading zero check: if the digit is '0', it is invalid

            if abbr[abbr_index] == '0':
                return False
            # build the entire number
            num = 0
            while abbr_index < len(abbr) and abbr[abbr_index].isdigit():
                num = num * 10 + int(abbr[abbr_index])
                abbr_index += 1
            # skip num characters in word
            word_index += num
        else:
            # if letter doesn't match or word index is out of bounds, invalid
            if word_index >= len(word) or word[word_index] != abbr[abbr_index]:
                return False
            word_index += 1
            abbr_index += 1

    # Valid if and only if the entire word was covered
    return word_index == len(word)

# Example usage:
```

Round 1 · High · Easy

p.182

```
#print(validWordAbbreviation("internationalization", "i12iz4n")) # Expected
output: True
#print(validWordAbbreviation("apple", "a2e")) # Expected output: False
```

• LC.ca

```
class Solution:
    def validWordAbbreviation(self, word: str, abbr: str) -> bool:
        m, n = len(word), len(abbr)
        i = j = x = 0
        while i < m and j < n:
            if abbr[j].isdigit():
                if abbr[j] == "0" and x == 0:
                    return False
                x = x * 10 + int(abbr[j])
            else:
                i += x
                x = 0
                if i >= m or word[i] != abbr[j]:
                    return False
                i += 1
                j += 1
        return i + x == m and j == n
```

◆ Quick Review

Key Insights

- Use two pointers: one for looping through the word and one for the abbreviation.
- When encountering a digit in abbr, accumulate the number and skip that many characters in the word.
- Check for invalid cases such as numbers with leading zeros or consecutive digit

segments that might imply adjacent skipped substrings.

- Compare letters directly when they are not digits.

Space & Time

Time Complexity: $O(n + m)$ where n is the length of the word and m is the length of the abbreviation.

Space Complexity: $O(1)$ (constant extra space)

Solution

The approach uses two pointers to iterate over the word and abbreviation. For each character in the abbreviation:

- If it is a letter, compare it with the current character in the word.
- If it is a digit, first ensure that it does not start with '0'. Then, convert the sequence of digits into a number which represents how many characters in the word to skip. After processing, both pointers should have reached the end of their respective strings for the abbreviation to be valid. This approach is efficient due to its linear pass with constant extra space.

Two Pointers

□ #696 Count Binary Substrings

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode

Two Pointers · String

Lists: AlgoMaster 600

Problem

Given a binary string s , return the number of non-empty substrings that have the same number of 0's and 1's, and all the 0's and all the 1's in these substrings are grouped consecutively.

Substrings that occur multiple times are counted the number of times they occur.

Example 1:

Input: $s = "00110011"$
 Output: 6
 Explanation: There are 6 substrings that have equal number of consecutive 1's and 0's: "0011", "01", "1100", "10", "0011", and "01".
 Notice that some of these substrings repeat and are counted the number of times they occur.
 Also, "00110011" is not a valid substring because all the 0's (and 1's) are not grouped together.

Example 2:

Round 1 · High · Easy

p.185

Input: $s = "10101"$
 Output: 4
 Explanation: There are 4 substrings: "10", "01", "10", "01" that have equal number of consecutive 1's and 0's.

Constraints:

- $1 \leq s.length \leq 105$
- $s[i]$ is either '0' or '1'.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def countBinarySubstrings(self, s: str) -> int:
        ans = 0
        prevCount = 0
        equals = 1

        for i in range(len(s) - 1):
            if s[i] == s[i + 1]:
                equals += 1
            else:
                ans += min(prevCount, equals)
                prevCount = equals
                equals = 1

        return ans + min(prevCount, equals)
```

• LeetDoocs

Round 1 · High · Easy

p.186

```
class Solution:
    def countBinarySubstrings(self, s: str) -> int:
        n = len(s)
        ans = i = 0
        pre = 0
        while i < n:
            j = i + 1
            while j < n and s[j] == s[i]:
                j += 1
            cur = j - i

            ans += min(pre, cur)
            pre = cur
            i = j
        return ans
```

• SimplyLeet

```
# Python solution with clear comments
def countBinarySubstrings(s: str) -> int:
    # List to store counts of consecutive groups
    groups = []
    count = 1 # Start count for the first character

    # Iterate through the string starting from the second character
    for i in range(1, len(s)):
        if s[i] == s[i - 1]:
            # Same character as previous, increase count

            count += 1
        else:
            # Character changed, append the count of the previous group and
            # reset count
            groups.append(count)
            count = 1

    # Append the count for the last group
    groups.append(count)

    # Now count valid substrings using adjacent group counts
    validSubstrings = 0
```

```
for i in range(1, len(groups)):
    # The number of valid substrings is the minimum of current and previous
    # group counts
    validSubstrings += min(groups[i - 1], groups[i])
return validSubstrings

# Example usage:
s = "00110011"
print(countBinarySubstrings(s)) # Output: 6
```

Two Pointers

#844 Backspace String Compare

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Two Pointers · String · Stack · Simulation
Lists: Grind 169

Problem

Given two strings *s* and *t*, return true if they are equal when both are typed into empty text editors. '#' means a backspace character.

Note that after backspacing an empty text, the text will continue empty.

Example 1:

Input: *s* = "ab#c", *t* = "ad#c"
Output: true
Explanation: Both *s* and *t* become "ac".

Example 2:

Input: *s* = "ab##", *t* = "c#d#"
Output: true
Explanation: Both *s* and *t* become "".

Example 3:

Input: *s* = "a#c", *t* = "b"
Output: false
Explanation: *s* becomes "c" while *t* becomes "b".

Constraints:

Round 1 · High · Easy

p.189

- $1 \leq s.length, t.length \leq 200$
- *s* and *t* only contain lowercase letters and '#' characters.

Follow up: Can you solve it in $O(n)$ time and $O(1)$ space?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def backspaceCompare(self, s: str, t: str) -> bool:
        def backspace(s: str) -> str:
            stack = []
            for c in s:
                if c != '#':
                    stack.append(c)
                elif stack:
                    stack.pop()
            return stack
```

```
return backspace(s) == backspace(t)
```

```
class Solution:
    def backspaceCompare(self, s: str, t: str) -> bool:
        i = len(s) - 1 # s' index
        j = len(t) - 1 # t's index

        while True:
            # Delete characters of s if needed.
            backspace = 0
            while i >= 0 and (s[i] == '#' or backspace > 0):
                backspace += 1 if s[i] == '#' else -1
```

Round 1 · High · Easy

p.190

```

    i -= 1
    # Delete characters of t if needed.
    backspace = 0
    while j >= 0 and (t[j] == '#' or backspace > 0):
        backspace += 1 if t[j] == '#' else -1
        j -= 1
    if i >= 0 and j >= 0 and s[i] == t[j]:
        i -= 1
        j -= 1
    else:
        break

    return i == -1 and j == -1

```

• LeetCode

```

class Solution:
    def backspaceCompare(self, s: str, t: str) -> bool:
        i, j, skip1, skip2 = len(s) - 1, len(t) - 1, 0, 0
        while i >= 0 or j >= 0:
            while i >= 0:
                if s[i] == '#':
                    skip1 += 1
                    i -= 1
                elif skip1:
                    skip1 -= 1
                    i -= 1
            else:
                break
            while j >= 0:
                if t[j] == '#':
                    skip2 += 1
                    j -= 1
                elif skip2:
                    skip2 -= 1
                    j -= 1

```

```

        else:
            break
        if i >= 0 and j >= 0:
            if s[i] != t[j]:
                return False
            elif i >= 0 or j >= 0:
                return False
            i, j = i - 1, j - 1
        return True

```

• SimplyLeet

```

# Function to compare two strings with backspace processing
def backspaceCompare(s: str, t: str) -> bool:
    # Pointers for both strings starting at the end
    i, j = len(s) - 1, len(t) - 1

    # Process both strings until all characters are compared
    while i >= 0 or j >= 0:
        # Process string s: skip backspaced characters
        skip = 0
        while i >= 0:
            if s[i] == '#':
                skip += 1
                i -= 1
            elif skip > 0:
                skip -= 1
                i -= 1
            else:
                break

        # Process string t: skip backspaced characters

```

```

skip = 0
while j >= 0:
    if t[j] == '#':
        skip += 1
        j -= 1
    elif skip > 0:
        skip -= 1
        j -= 1
    else:
        break

# Compare the current valid characters from s and t
if i >= 0 and j >= 0:
    if s[i] != t[j]:
        return False
elif i >= 0 or j >= 0:
    return False

i -= 1
j -= 1

```

```
return True
```

```

# Example test cases
print(backspaceCompare("ab#c", "ad#c")) # Expected output: True
print(backspaceCompare("ab##", "c#d#")) # Expected output: True
print(backspaceCompare("a#c", "b"))     # Expected output: False

```

• LC.ca

```

class Solution:
    def backspaceCompare(self, s: str, t: str) -> bool:
        i, j, skip1, skip2 = len(s) - 1, len(t) - 1, 0, 0
        while i >= 0 or j >= 0:
            while i >= 0:
                if s[i] == '#':
                    skip1 += 1
                    i -= 1
                elif skip1:
                    skip1 -= 1

```

```

        i -= 1
    else:
        break
    while j >= 0:
        if t[j] == '#':
            skip2 += 1
            j -= 1
        elif skip2:
            skip2 -= 1
            j -= 1
    else:
        break
    if i >= 0 and j >= 0:
        if s[i] != t[j]:
            return False
    elif i >= 0 or j >= 0:
        return False
    i, j = i - 1, j - 1
    return True

```

◆ Quick Review

Key Insights

- '#' represents a backspace operation that deletes the previous character.
- A direct simulation using a stack can reconstruct the final string.
- A two-pointer approach allows comparison from the end of the strings while skipping backspaced characters.
- The two-pointer technique can achieve $O(n)$ time and $O(1)$ space by processing the strings

in reverse.

Space & Time

Time Complexity: $O(n)$ where n is the length of the longer string.

Space Complexity: $O(1)$ using the two-pointer strategy ($O(n)$ if using stacks).

Solution

The solution uses two pointers starting from the end of each string. For each string, maintain a skip counter to account for backspace characters. As you iterate backwards:

- If a '#' is encountered, increment the skip counter.
- If a normal character is encountered while the skip counter is positive, skip that character by decrementing the counter.
- Once a valid character is found, compare the characters from both strings. If all corresponding characters match and both pointers finish processing at the same time, the strings are considered equal.

Two Pointers

□ #977 Squares of a Sorted Array

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Sorting
Lists: Grind 169

Problem

Given an integer array `nums` sorted in non-decreasing order, return an array of the squares of each number sorted in non-decreasing order.

Example 1:

```
Input: nums = [-4,-1,0,3,10]
Output: [0,1,9,16,100]
Explanation: After squaring, the array becomes [16,1,0,9,100].
After sorting, it becomes [0,1,9,16,100].
```

Example 2:

```
Input: nums = [-7,-3,2,3,11]
Output: [4,9,9,49,121]
```

Constraints:

- $1 \leq \text{nums.length} \leq 104$
- $-104 \leq \text{nums}[i] \leq 104$
- `nums` is sorted in non-decreasing order.

Follow up: Squaring each element and sorting the new array is very trivial, could you find an O(n) solution using a different approach?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def sortedSquares(self, nums: List[int]) -> List[int]:
        n = len(nums)
        l = 0
        r = n - 1
        ans = [0] * n

        while n:
            n -= 1
            if abs(nums[l]) > abs(nums[r]):
                ans[n] = nums[l] * nums[l]
                l += 1
            else:
                ans[n] = nums[r] * nums[r]
                r -= 1

        return ans
```

• LeetCode

```
class Solution:
    def sortedSquares(self, nums: List[int]) -> List[int]:
        ans = []
        i, j = 0, len(nums) - 1
        while i <= j:
            a = nums[i] * nums[i]
            b = nums[j] * nums[j]
            if a > b:
                ans.append(a)
                i += 1
            else:
                ans.append(b)
                j -= 1
        return ans
```

Round 1 · High · Easy

p.197

```
else:
    ans.append(b)
    j -= 1
return ans[::-1]
```

• SimplyLeet

```
# Function to return squares of a sorted array in non-decreasing order.
def sortedSquares(nums):
    # Initialize two pointers at the start and end of the array.
    left, right = 0, len(nums) - 1
    # Create a result array of the same length as nums.
    result = [0] * len(nums)
    # Index to insert the largest square at the current position.
    position = len(nums) - 1

    # Loop until the left and right pointers cross.
```

```
while left <= right:
    # Compare absolute values at the left and right pointers.
    if abs(nums[left]) > abs(nums[right]):
        result[position] = nums[left] * nums[left]
        left += 1 # Move left pointer rightward.
    else:
        result[position] = nums[right] * nums[right]
        right -= 1 # Move right pointer leftward.
    position -= 1 # Move to the next insertion position.
return result
```

```
# Example usage:
print(sortedSquares([-4, -1, 0, 3, 10])) # Output: [0, 1, 9, 16, 100]
```

• LC.ca

Round 1 · High · Easy

p.198

```

class Solution:
    def sortedSquares(self, nums: List[int]) -> List[int]:
        n = len(nums)
        res = [0] * n
        i, j, k = 0, n - 1, n - 1
        while i <= j:
            if nums[i] * nums[i] > nums[j] * nums[j]:
                res[k] = nums[i] * nums[i]
                i += 1
            else:
                res[k] = nums[j] * nums[j]
                j -= 1
            k -= 1
        return res

```

◆ Quick Review

Key Insights

- Squaring numbers can disrupt the original order since negative numbers become positive.
- A two-pointer approach can efficiently retrieve the largest square from either end of the array.
- By comparing the absolute values at the two pointers, the largest square is inserted from the back of the result array.
- This method achieves an $O(n)$ time complexity without needing to sort the squared values afterward.

Space & Time

Time Complexity: $O(n)$ – We traverse the array only once.

Space Complexity: $O(n)$ – We use an extra array to store the output.

Solution

We use a two-pointer technique to traverse the array from both ends. Initialize pointers (left and right) at the beginning and end of the array, respectively, and an index pointer starting at the end of a new results array. At each step, compare the absolute values of the elements at the left and right pointers. The larger absolute value, when squared, is placed in the result array at the current index, and the corresponding pointer is adjusted. This continues until the left pointer exceeds the right pointer. Filling the result array from the back ensures that the array remains sorted in non-decreasing order.

Two Pointers

☐ #1768 Merge Strings Alternately

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Two Pointers · String
Lists: AlgoMaster 600

Problem

You are given two strings `word1` and `word2`. Merge the strings by adding letters in alternating order, starting with `word1`. If a string is longer than the other, append the additional letters onto the end of the merged string. Return the merged string.

Example 1:

Input: `word1 = "abc", word2 = "pqr"`

Output: `"apbqcr"`

Explanation: The merged string will be merged as so:

`word1: a b c`

Example 2:

Round 1 · High · Easy

p.201

Input: `word1 = "ab", word2 = "pqrs"`

Output: `"apbqrs"`

Explanation: Notice that as `word2` is longer, `"rs"` is appended to the end.

`word1: a b`

Example 3:

Input: `word1 = "abcd", word2 = "pq"`

Output: `"apbqcd"`

Explanation: Notice that as `word1` is longer, `"cd"` is appended to the end.

`word1: a b c d`

Constraints:

- $1 \leq \text{word1.length}, \text{word2.length} \leq 100$
- `word1` and `word2` consist of lowercase English letters.

Community Solutions (Python)

☐ • WalkCC

```
class Solution:
    def mergeAlternately(self, word1: str, word2: str) -> str:
        return ''.join(a + b for a, b in zip_longest(word1, word2, fillvalue=''))
```

☐ • LeetDoocs

```
class Solution:
    def mergeAlternately(self, word1: str, word2: str) -> str:
        return ''.join(a + b for a, b in zip_longest(word1, word2, fillvalue=''))
```

☐ • SimplyLeet

Round 1 · High · Easy

p.202

```
# Define function to merge two strings alternately
def merge_alternately(word1: str, word2: str) -> str:
    # Initialize an empty list to collect merged characters
    merged = []
    # Determine the minimum length to iterate for both strings
    min_length = min(len(word1), len(word2))

    # Iterate over both strings up to the minimum length
    for i in range(min_length):
        # Append character from word1
        merged.append(word1[i])
        # Append character from word2
        merged.append(word2[i])

    # After the loop, append the remaining part of word1 if any
    if min_length < len(word1):
        merged.append(word1[min_length:])
    # Append the remaining part of word2 if any
    if min_length < len(word2):
        merged.append(word2[min_length:])

    # Join all parts into a final string and return
    return ''.join(merged)

# Example usage:
print(merge_alternately("abc", "pqr")) # Output: "apbqcr"
```

Round 1 · High · Easy

p.203

Sliding Window – Fixed Size

Round 1 · High · Easy · 1 question

Round 1 · High · Easy

p.204

Sliding Window – Fixed Size

#643 Maximum Average Subarray I

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Sliding Window
 Lists: AlgoMaster 600

Problem

You are given an integer array `nums` consisting of `n` elements, and an integer `k`.

Find a contiguous subarray whose length is equal to `k` that has the maximum average value and return this value. Any answer with a calculation error less than 10^{-5} will be accepted.

Example 1:

Input: `nums = [1,12,-5,-6,50,3]`, `k = 4`
 Output: `12.75000`
 Explanation: Maximum average is $(12 - 5 - 6 + 50) / 4 = 51 / 4 = 12.75$

Example 2:

Input: `nums = [5]`, `k = 1`
 Output: `5.00000`

Constraints:

- `n == nums.length`

Round 1 · High · Easy

p.205

- `1 <= k <= n <= 105`
- `-104 <= nums[i] <= 104`

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findMaxAverage(self, nums: List[int], k: int) -> float:
        summ = sum(nums[:k])
        ans = summ

        for i in range(k, len(nums)):
            summ += nums[i] - nums[i - k]
            ans = max(ans, summ)

        return ans / k
```

• LeetDoocs

```
class Solution:
    def findMaxAverage(self, nums: List[int], k: int) -> float:
        ans = s = sum(nums[:k])
        for i in range(k, len(nums)):
            s += nums[i] - nums[i - k]
            ans = max(ans, s)
        return ans / k
```

• SimplyLeet

```
# Python solution using the sliding window technique

def findMaxAverage(nums, k):
    # Calculate the initial sum of the first window of size k
    window_sum = sum(nums[:k])
    max_sum = window_sum

    # Slide the window from index k to the end of the array
    for i in range(k, len(nums)):
        # Update the sum by subtracting the element that is sliding out
```

Round 1 · High · Easy

p.206

```
# and adding the new element
window_sum += nums[i] - nums[i - k]
# Update max_sum if the new window_sum is higher
max_sum = max(max_sum, window_sum)

# Return the maximum average found
return max_sum / k

# Example usage:
# nums = [1, 12, -5, -6, 50, 3]

# k = 4
# print(findMaxAverage(nums, k)) # Output: 12.75
```

Round 1 · High · Easy**p.207**

Binary Search

Round 1 · High · Easy · 6 questions**Round 1 · High · Easy****p.208**

Binary Search

☐ #35 Search Insert Position

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Binary Search
 Lists: Denny Zhang

Problem

Given a sorted array of distinct integers and a target value, return the index if the target is found. If not, return the index where it would be if it were inserted in order.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: nums = [1,3,5,6], target = 5
 Output: 2

Example 2:

Input: nums = [1,3,5,6], target = 2
 Output: 1

Example 3:

Input: nums = [1,3,5,6], target = 7
 Output: 4

Constraints:

Round 1 · High · Easy

p.209

- $1 \leq \text{nums.length} \leq 104$
- $-104 \leq \text{nums}[i] \leq 104$
- nums contains distinct values sorted in ascending order.
- $-104 \leq \text{target} \leq 104$

Community Solutions (Python)

☐ • WalkCC

```
class Solution:
    def searchInsert(self, nums: list[int], target: int) -> int:
        l = 0
        r = len(nums)

        while l < r:
            m = (l + r) // 2
            if nums[m] == target:
                return m
            if nums[m] < target:
                l = m + 1
            else:
                r = m

        return l
```

☐ • LeetDoocs

Round 1 · High · Easy

p.210

```
class Solution:
    def searchInsert(self, nums: List[int], target: int) -> int:
        l, r = 0, len(nums)
        while l < r:
            mid = (l + r) >> 1
            if nums[mid] >= target:
                r = mid
            else:
                l = mid + 1
        return l
```

• SimplyLeet

```
# Python solution using binary search
def searchInsert(nums, target):
    # Initialize low and high pointers for binary search
    low, high = 0, len(nums) - 1
    # Perform binary search until low pointer exceeds high pointer
    while low <= high:
        mid = (low + high) // 2 # Calculate the middle index
        if nums[mid] == target:
            return mid # Return mid if target is found
        elif target < nums[mid]:
            high = mid - 1 # Narrow search on the left side
        else:
            low = mid + 1 # Narrow search on the right side
    # Return low pointer as the insertion index when target is not found
    return low

# Example usage
print(searchInsert([1, 3, 5, 6], 5)) # Output: 2
```

• LC.ca

Round 1 · High · Easy

p.211

```
class Solution:
    def searchInsert(self, nums: List[int], target: int) -> int:
        left, right = 0, len(nums)
        while left < right:
            mid = (left + right) >> 1
            if nums[mid] >= target:
                right = mid
            else:
                left = mid + 1
        return left
```

◆ Quick Review

Key Insights

- The array is sorted, enabling the use of binary search.
- Binary search achieves a time complexity of $O(\log n)$ which meets the problem constraints.
- When the target is not found during the search, the final low pointer represents the correct insertion index.

Space & Time

Time Complexity: $O(\log n)$ due to binary search.

Space Complexity: $O(1)$ as no extra space is used.

Solution

Round 1 · High · Easy

p.212

We use binary search to find the target in the sorted array. Initialize two pointers, low and high. At each iteration, compute the mid index. If the value at mid matches the target, return mid. If the target is less than the mid value, adjust the high pointer; if greater, adjust the low pointer. When the iterations finish without finding the target, return the low pointer, which represents the correct insertion index.

Binary Search

☐ #278 First Bad Version

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Binary Search · Interactive
Lists: Grind 169

Problem

You are a product manager and currently leading a team to develop a new product. Unfortunately, the latest version of your product fails the quality check. Since each version is developed based on the previous version, all the versions after a bad version are also bad.

Suppose you have n versions $[1, 2, \dots, n]$ and you want to find out the first bad one, which causes all the following ones to be bad.

You are given an API `bool isBadVersion(version)` which returns whether version is bad.

Implement a function to find the first bad version. You should minimize the number of calls to the API.

Example 1:

Input: n = 5, bad = 4
 Output: 4
 Explanation:
 call isBadVersion(3) -> false
 call isBadVersion(5) -> true
 call isBadVersion(4) -> true
 Then 4 is the first bad version.

Example 2:

Input: n = 1, bad = 1
 Output: 1

Constraints:

- $1 \leq \text{bad} \leq n \leq 2^{31} - 1$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def firstBadVersion(self, n: int) -> int:
        l = 1
        r = n

        while l < r:
            m = (l + r) >> 1
            if isBadVersion(m):
                r = m
            else:
                l = m + 1

        return l
```

• LeetDoocs

The isBadVersion API is already defined for you.
 # def isBadVersion(version: int) -> bool:

```
class Solution:
    def firstBadVersion(self, n: int) -> int:
        l, r = 1, n
        while l < r:
            mid = (l + r) >> 1
            if isBadVersion(mid):
                r = mid
            else:
                l = mid + 1
        return l
```

• SimplyLeet

```
# Python implementation of First Bad Version
class Solution:
    def firstBadVersion(self, n: int) -> int:
        # Initialize the left pointer to the first version and right pointer to
        # the last version
        left, right = 1, n
        while left < right:
            # Calculate the mid point to avoid potential overflow
            mid = left + (right - left) // 2
            # Call the API isBadVersion to check if mid version is bad
            if isBadVersion(mid):
                # If mid is bad, search the left half including mid
                right = mid
            else:
                # If mid is not bad, search the right half excluding mid
                left = mid + 1
        # When left equals right, we found the first bad version
        return left
```

• LC.ca


```
# The isBadVersion API is already defined for you.
# @param version, an integer
# @return an integer
# def isBadVersion(version):
```

```
class Solution:
    def firstBadVersion(self, n):
        """
        :type n: int

        :rtype: int
        """
        left, right = 1, n
        while left < right:
            mid = (left + right) >> 1
            if isBadVersion(mid):
                right = mid
            else:
                left = mid + 1
        return left
```

◆ Quick Review

Key Insights

- The API returns true for bad versions and all versions after a bad version.
- The problem is essentially about finding a transition point in a sorted sequence which calls for a binary search approach.
- The aim is to minimize the number of calls to `isBadVersion` by leveraging the ordered nature of the versions.

Round 1 · High · Easy

p.217

Space & Time

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

Solution

We use binary search to efficiently locate the first bad version. Initialize two pointers, `left` and `right`, at 1 and `n` respectively. At each step, compute the mid point and call `isBadVersion(mid)`. If `mid` is bad, it indicates that the first bad version may be `mid` or to its left, so adjust `right` to `mid`. Otherwise, adjust `left` to `mid + 1`. Continue the process until the pointers converge. The converged pointer will be the first bad version. Key structures include simple integer variables for `left`, `right`, and `mid`, with binary search being the fundamental algorithm to reduce the search space logarithmically.

Round 1 · High · Easy

p.218

Binary Search

#367 Valid Perfect Square

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · Binary Search
Lists: AlgoMaster 600

Problem

Given a positive integer num, return true if num is a perfect square or false otherwise.

A perfect square is an integer that is the square of an integer. In other words, it is the product of some integer with itself.

You must not use any built-in library function, such as sqrt.

Example 1:

Input: num = 16
Output: true
Explanation: We return true because $4 * 4 = 16$ and 4 is an integer.

Example 2:

Input: num = 14
Output: false
Explanation: We return false because $3.742 * 3.742 = 14$ and 3.742 is not an integer.

Constraints:

Round 1 · High · Easy

p.219

- $1 \leq \text{num} \leq 231 - 1$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isPerfectSquare(self, num: int) -> bool:
        l = bisect.bisect_left(range(num), num, key=lambda m: m * m)
        return l * 2 == num
```

• LeetDoocs

```
class Solution:
    def isPerfectSquare(self, num: int) -> bool:
        l = bisect.bisect_left(range(1, num + 1), num, key=lambda x: x * x) + 1
        return l * l == num
```

```
class Solution:
    def isPerfectSquare(self, num: int) -> bool:
        i = 1
        while num > 0:
            num -= i
            i += 2
        return num == 0
```

• SimplyLeet

```
# Function to determine if a number is a perfect square using binary search.
def isPerfectSquare(num):
    # If num is less than 2, it is a perfect square (e.g., 1).
    if num < 2:
        return True
    # Set initial boundaries for binary search.
    left, right = 2, num // 2
    while left <= right:
        # Calculate mid point.
        mid = left + (right - left) // 2
```

Round 1 · High · Easy

p.220

```

guessed_square = mid * mid
# Check if the mid value squares to num.
if guessed_square == num:
    return True
# If guessed square is too high, discard right half.
elif guessed_square > num:
    right = mid - 1
# Otherwise, discard left half.
else:
    left = mid + 1

return False

# Example usage:
print(isPerfectSquare(16)) # True
print(isPerfectSquare(14)) # False

```

Binary Search

□ #374 Guess Number Higher or Lower

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Math · Binary Search · Interactive
 Lists: Denny Zhang

Problem

We are playing the Guess Game. The game is as follows:

I pick a number from 1 to n. You have to guess which number I picked (the number I picked stays the same throughout the game).

Every time you guess wrong, I will tell you whether the number I picked is higher or lower than your guess.

You call a pre-defined API `int guess(int num)`, which returns three possible results:

- -1: Your guess is higher than the number I picked (i.e. `num > pick`).
- 1: Your guess is lower than the number I picked (i.e. `num < pick`).

- 0: your guess is equal to the number I picked (i.e. `num == pick`).

Return the number that I picked.

Example 1:

Input: `n = 10, pick = 6`
Output: 6

Example 2:

Input: `n = 1, pick = 1`
Output: 1

Example 3:

Input: `n = 2, pick = 1`
Output: 1

Constraints:

- $1 \leq n \leq 231 - 1$
- $1 \leq \text{pick} \leq n$

Community Solutions (Python)

• WalkCC

```
# The guess API is already defined for you.
# @param num, your guess
# @return -1 if num is higher than the picked number
# 1 if num is lower than the picked number
# otherwise return 0
# def guess(num: int) -> int:

class Solution:
    def guessNumber(self, n: int) -> int:
        l = 1
```

Round 1 · High · Easy

p.223

```
r = n

# Find the first guess number that >= the target number
while l < r:
    m = (l + r) // 2
    if guess(m) <= 0: # -1, 0
        r = m
    else:
        l = m + 1

return l
```

• LeetCode

```
# The guess API is already defined for you.
# @param num, your guess
# @return -1 if num is higher than the picked number
# 1 if num is lower than the picked number
# otherwise return 0
# def guess(num: int) -> int:

class Solution:
    def guessNumber(self, n: int) -> int:

        return bisect.bisect(range(1, n + 1), 0, key=lambda x: -guess(x))
```

• SimplyLeet

```
# Python solution using binary search
class Solution:
    def guessNumber(self, n: int) -> int:
        # Initialize low and high pointers
        low, high = 1, n
        # Continue binary search until low exceeds high
        while low <= high:
            # Calculate midpoint safely to prevent overflow
            mid = low + (high - low) // 2
            # Call the guess API with mid
```

Round 1 · High · Easy

p.224

```

result = guess(mid)
# If the guess is correct, return mid
if result == 0:
    return mid
# If guess was too high, adjust the high pointer
elif result == -1:
    high = mid - 1
# If guess was too low, adjust the low pointer
else:
    low = mid + 1

# Return low (ideally should never reach here if pick exists in [1, n])
return low

```

• LC.ca

```

# The guess API is already defined for you.
# @param num, your guess
# @return -1 if num is higher than the picked number
# 1 if num is lower than the picked number
# otherwise return 0
# def guess(num: int) -> int:

class Solution:
    def guessNumber(self, n: int) -> int:

        return bisect.bisect(range(1, n + 1), 0, key=lambda x: -guess(x))

```

◆ Quick Review

Key Insights

- The problem can be effectively solved using binary search since the search space is sorted by nature (1 to n).
- Each API call helps to halve the search space, leading to an efficient solution.

- Be cautious with potential overflow when calculating the mid-point by using $\text{low} + (\text{high} - \text{low}) // 2$.

Space & Time

Time Complexity: $O(\log n)$ since we binary search the range.

Space Complexity: $O(1)$ as we use a constant amount of extra space.

Solution

We use a binary search algorithm to efficiently locate the target number. We initialize two pointers: low (1) and high (n). In each iteration, we compute the mid-point, then call the guess API. Based on the response:

- If `guess(mid)` returns 0, we have found the number.
- If it returns -1, it indicates that our guess is higher than the pick, so we update the high pointer to `mid - 1`.
- If it returns 1, it indicates that our guess is lower than the pick, so we update the low pointer to `mid + 1`. This approach guarantees a logarithmic time solution.

Binary Search

#704 Binary Search

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Binary Search
 Lists: Grind 169

Problem

Given an array of integers `nums` which is sorted in ascending order, and an integer `target`, write a function to search `target` in `nums`. If `target` exists, then return its index. Otherwise, return `-1`.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: `nums = [-1,0,3,5,9,12]`, `target = 9`
 Output: `4`
 Explanation: 9 exists in `nums` and its index is 4

Example 2:

Input: `nums = [-1,0,3,5,9,12]`, `target = 2`
 Output: `-1`
 Explanation: 2 does not exist in `nums` so return `-1`

Constraints:

- $1 \leq \text{nums.length} \leq 104$

Round 1 · High · Easy

p.227

- $-104 < \text{nums}[i], \text{target} < 104$
- All the integers in `nums` are unique.
- `nums` is sorted in ascending order.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def search(self, nums: List[int], target: int) -> int:
        i = bisect.bisect_left(nums, target)
        return -1 if i == len(nums) or nums[i] != target else i
```

• LeetDoocs

```
class Solution:
    def search(self, nums: List[int], target: int) -> int:
        l, r = 0, len(nums) - 1
        while l < r:
            mid = (l + r) >> 1
            if nums[mid] >= target:
                r = mid
            else:
                l = mid + 1
        return l if nums[l] == target else -1
```

• SimplyLeet

```
# Python code for binary search

def search(nums, target):
    # Set initial boundaries for the search
    left, right = 0, len(nums) - 1

    # Loop until the search space is empty
    while left <= right:
        # Calculate middle index to avoid overflow
        mid = left + (right - left) // 2
```

Round 1 · High · Easy

p.228

```
# Check if the mid element matches the target
if nums[mid] == target:
    return mid
# If target is smaller than mid element, narrow search to left half
elif target < nums[mid]:
    right = mid - 1
# Else, narrow search to right half
else:
    left = mid + 1

# Target not found, return -1
return -1

# Example usage:
print(search([-1,0,3,5,9,12], 9)) # Expected output: 4
```

• LC.ca

```
class Solution:
    def search(self, nums: List[int], target: int) -> int:
        left, right = 0, len(nums) - 1
        while left < right:
            mid = (left + right) >> 1
            if nums[mid] >= target:
                right = mid
            else:
                left = mid + 1
        return left if nums[left] == target else -1
```

◆ Quick Review

Key Insights

- Utilize binary search to achieve $O(\log n)$ runtime on a sorted array.

Round 1 · High · Easy

p.229

- Use two pointers to maintain the current search region.
- Narrow down the search range by comparing the middle element with the target.
- Return the index when the target is found; otherwise, return -1 if not found.

Space & Time

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

Solution

We use an iterative binary search algorithm. We initialize two pointers, left and right, to represent the current segment of the array that could contain the target. At each iteration, we compute the middle index and compare the element there with the target. If they match, we return the index. Otherwise, based on whether the target is smaller or larger than the middle element, we adjust our pointers to narrow the search to either the left or right half of the array. This process continues until the target is found or until the pointers cross, indicating that the target is not present. No extra data structures are used, making the

Round 1 · High · Easy

p.230

space complexity $O(1)$.

Round 1 · High · Easy

p.231

Binary Search

□ #744 Find Smallest Letter Greater Than Target

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search
Lists: AlgoMaster 600

Problem

You are given an array of characters letters that is sorted in non-decreasing order, and a character target. There are at least two different characters in letters.

Return the smallest character in letters that is lexicographically greater than target. If such a character does not exist, return the first character in letters.

Example 1:

Input: letters = ["c","f","j"], target = "a"
Output: "c"
Explanation: The smallest character that is lexicographically greater than 'a' in letters is 'c'.

Example 2:

Input: letters = ["c","f","j"], target = "c"
Output: "f"
Explanation: The smallest character that is lexicographically greater than 'c' in letters is 'f'.

Round 1 · High · Easy

p.232

Example 3:

Input: letters = ["x","x","y","y"], target = "z"

Output: "x"

Explanation: There are no characters in letters that is lexicographically greater than 'z' so we return letters[0].

Constraints:

- $2 \leq \text{letters.length} \leq 104$
- letters[i] is a lowercase English letter.
- letters is sorted in non-decreasing order.
- letters contains at least two different characters.
- target is a lowercase English letter.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def nextGreatestLetter(self, letters: list[str], target: str) -> str:
        l = bisect.bisect_right(range(len(letters)), target,
                                key=lambda m: letters[m])
        return letters[l % len(letters)]
```

• LeetDoocs

```
class Solution:
    def nextGreatestLetter(self, letters: List[str], target: str) -> str:
        i = bisect_right(letters, ord(target), key=lambda c: ord(c))
        return letters[i % len(letters)]
```

• SimplyLeet

```
# Function to find the smallest letter greater than target using binary search.
def nextGreatestLetter(letters, target):
    # Initialize left and right pointers.
    left, right = 0, len(letters) - 1

    # Perform binary search.
    while left <= right:
        # Calculate the middle index.
        mid = left + (right - left) // 2

        # If the middle element is less than or equal to target, search in the
        # right half.
        if letters[mid] <= target:
            left = mid + 1
        else:
            # Otherwise, search in the left half.
            right = mid - 1

    # Use modulo to handle wrap-around and return the next greatest letter.
    return letters[left % len(letters)]

# Example usage:
# letters = ['c', 'f', 'j']
# target = 'a'
# print(nextGreatestLetter(letters, target)) # Output: 'c'
```

Stacks

Round 1 · High · Easy · 4 questions

Round 1 · High · Easy

p.235

Stacks

□ #20 Valid Parentheses

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Stack

Lists: Grind 169

Problem

Given a string *s* containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.

An input string is valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.
3. Every close bracket has a corresponding open bracket of the same type.

Example 1:

Input: *s* = "()"

Output: true

Example 2:

Input: *s* = "()[]{}"

Round 1 · High · Easy

p.236

Output: true

Example 3:

Input: s = "()"

Output: false

Example 4:

Input: s = "([])"

Output: true

Example 5:

Input: s = "([)]"

Output: false

Constraints:

- $1 \leq s.length \leq 104$
- s consists of parentheses only '()[]{'.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isValid(self, s: str) -> bool:
        stack = []

        for c in s:
            if c == '(':
                stack.append(')')
            elif c == '{':
                stack.append('}')
            elif c == '[':
```

Round 1 · High · Easy

p.237

```
        stack.append(']')
        elif not stack or stack.pop() != c:
            return False

        return not stack
```

• LeetDoocs

```
class Solution:
    def isValid(self, s: str) -> bool:
        stk = []
        d = {'(': ')', '[': ']', '{': '}'
        for c in s:
            if c in '([{':
                stk.append(c)
            elif not stk or stk.pop() + c not in d:
                return False
        return not stk
```

• SimplyLeet

```
def isValid(s: str) -> bool:
    # Mapping from closing brackets to their corresponding opening brackets.
    paren_map = {')': '(', '}': '{', ']': '['}
    # Initialize an empty list to use as a stack.
    stack = []
    # Process each character in the string.
    for char in s:
        # If the character is a closing bracket.
        if char in paren_map:
            # Pop the top element from the stack if available, else use a
            # placeholder.
```

Round 1 · High · Easy

p.238

```

    top_element = stack.pop() if stack else '#'
    # Check if the mapping does not match the top element.
    if paren_map[char] != top_element:
        return False
    else:
        # For opening brackets, push onto the stack.
        stack.append(char)
    # Return True if all brackets are matched (i.e., the stack is empty).
    return not stack

```

```

# Example usage
print(isValid("()[]{}")) # Expected output: True

```

• LC.ca

```

class Solution:
    def isValid(self, s: str) -> bool:
        stk = []
        d = {'(': ')', '[': ']', '{': '}' }
        for c in s:
            if c in '({[':
                stk.append(c)
            elif not stk or stk.pop() + c not in d:
                return False
        return not stk

```

◆ Quick Review

Key Insights

- The problem can be solved using a stack data structure.
- When encountering an opening bracket, push it onto the stack.
- When encountering a closing bracket, check if the top of the stack is its matching opening

bracket.

- If the stack is empty before finding a match or if there is a mismatch, the string is invalid.
- At the end, the string is valid only if the stack is empty, meaning all brackets were properly matched.

Space & Time

Time Complexity: $O(n)$, where n is the length of the string, since we process each character once.

Space Complexity: $O(n)$ in the worst case for the stack when all characters are opening brackets.

Solution

We use a stack to check for valid parentheses. The algorithm iterates through the string, pushing opening brackets onto the stack and popping them when a corresponding closing bracket is encountered. A mapping of closing to opening brackets helps in validating the matching pairs. By ensuring that the stack is empty at the end, we confirm that every opening bracket was properly closed.

Stacks

□ #682 Baseball Game

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Stack · Simulation
 Lists: AlgoMaster 600

Problem

You are keeping the scores for a baseball game with strange rules. At the beginning of the game, you start with an empty record.

You are given a list of strings operations, where operations[i] is the ith operation you must apply to the record and is one of the following:

- An integer x. Record a new score of x.
- Record a new score that is the sum of the previous two scores.
- Record a new score that is the double of the previous score.
- Invalidate the previous score, removing it from the record.

Return the sum of all the scores on the record after applying all the operations.

Round 1 · High · Easy

p.241

The test cases are generated such that the answer and all intermediate calculations fit in a 32-bit integer and that all operations are valid.

Example 1:

```
Input: ops = ["5","2","C","D","+"]
Output: 30
Explanation:
"5" - Add 5 to the record, record is now [5].
"2" - Add 2 to the record, record is now [5, 2].
"C" - Invalidate and remove the previous score, record is now [5].
"D" - Add 2 * 5 = 10 to the record, record is now [5, 10].
"+" - Add 5 + 10 = 15 to the record, record is now [5, 10, 15].
```

Example 2:

```
Input: ops = ["5","-2","4","C","D","9","+","+"]
Output: 27
Explanation:
"5" - Add 5 to the record, record is now [5].
"-2" - Add -2 to the record, record is now [5, -2].
"4" - Add 4 to the record, record is now [5, -2, 4].
"C" - Invalidate and remove the previous score, record is now [5, -2].
"D" - Add 2 * -2 = -4 to the record, record is now [5, -2, -4].
```

Example 3:

```
Input: ops = ["1","C"]
Output: 0
Explanation:
"1" - Add 1 to the record, record is now [1].
"C" - Invalidate and remove the previous score, record is now [].
Since the record is empty, the total sum is 0.
```

Constraints:

- $1 \leq \text{operations.length} \leq 1000$
- operations[i] is "C", "D", "+", or a string representing an integer in the range $[-3 * 10^4, 3 * 10^4]$.

Round 1 · High · Easy

p.242

- For operation "+", there will always be at least two previous scores on the record.
- For operations "C" and "D", there will always be at least one previous score on the record.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def calPoints(self, operations: List[str]) -> int:
        scores = []

        for operation in operations:
            match operation:
                case '+':
                    scores.append(scores[-1] + scores[-2])
                case 'D':
                    scores.append(scores[-1] * 2)

                case 'C':
                    scores.pop()
                case default:
                    scores.append(int(operation))

        return sum(scores)
```

• LeetCode

```
class Solution:
    def calPoints(self, operations: List[str]) -> int:
        stk = []
        for op in operations:
            if op == "+":
                stk.append(stk[-1] + stk[-2])
            elif op == "D":
                stk.append(stk[-1] << 1)
            elif op == "C":
                stk.pop()
```

Round 1 · High · Easy

p.243

• SimplyLeet

```
else:
    stk.append(int(op))
return sum(stk)
```

```
# Define the function to calculate total score for the baseball game
def calPoints(ops):
    # Initialize a list to be used as a stack for scores
    score_stack = []

    # Iterate over each operation in the list
    for op in ops:
        # If the operation is "C", remove the last score from the stack
        if op == "C":
            score_stack.pop()

        # If the operation is "D", double the last score and append it
        elif op == "D":
            score_stack.append(2 * score_stack[-1])

        # If the operation is "+", sum the last two scores and append the
        result
        elif op == "+":
            score_stack.append(score_stack[-1] + score_stack[-2])

        # Otherwise, the operation is a number in string form, convert to
        integer and append
        else:
            score_stack.append(int(op))

    # Return the total sum of all scores in the stack
    return sum(score_stack)

# Example usage:
print(calPoints(["5", "2", "C", "D", "+"])) # Expected output: 30
```

Round 1 · High · Easy

p.244

Stacks

☐ #1047 Remove All Adjacent Duplicates In String

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode String · Stack

Lists: AlgoMaster 600

Problem

You are given a string s consisting of lowercase English letters. A duplicate removal consists of choosing two adjacent and equal letters and removing them.

We repeatedly make duplicate removals on s until we no longer can.

Return the final string after all such duplicate removals have been made. It can be proven that the answer is unique.

Example 1:

Input: $s = \text{"abbaca"}$
 Output: "ca"
 Explanation:
 For example, in "abbaca" we could remove "bb" since the letters are adjacent and equal, and this is the only possible move. The result of this move is that the string is "aaca", of which only "aa" is possible, so the final string is "ca".

Example 2:

Round 1 · High · Easy

p.245

Input: $s = \text{"azxxzy"}$
 Output: "ay"

Constraints:

- $1 \leq s.length \leq 105$
- s consists of lowercase English letters.

Community Solutions (Python)

• LeetDoocs

```
class Solution:
    def removeDuplicates(self, s: str) -> str:
        stk = []
        for c in s:
            if stk and stk[-1] == c:
                stk.pop()
            else:
                stk.append(c)
        return ''.join(stk)
```

• SimplyLeet

```
# Function to remove all adjacent duplicates in string
def remove_duplicates(s: str) -> str:
    # Initialize an empty list to use as a stack
    stack = []
    # Iterate through each character in the input string
    for char in s:
        # If stack is not empty and the top element equals the current
        # character, pop the top element
        if stack and stack[-1] == char:
            stack.pop()
        else:
```

Round 1 · High · Easy

p.246

```
# Otherwise, push the current character onto the stack
stack.append(char)
# Join the stack elements to form the final string result and return it
return ''.join(stack)

# Example usage:
# result = remove_duplicates("abbaca")
# print(result) # Output: "ca"
```

Stacks

☐ #1614 Maximum Nesting Depth of the Parentheses

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Stack

Lists: AlgoMaster 600

Problem

Given a valid parentheses string s , return the nesting depth of s . The nesting depth is the maximum number of nested parentheses.

Example 1:

Input: $s = "(1+(2*3)+((8)/4))+1"$

Output: 3

Explanation:

Digit 8 is inside of 3 nested parentheses in the string.

Example 2:

Input: $s = "(1)+((2))+(((3)))"$

Output: 3

Explanation:

Digit 3 is inside of 3 nested parentheses in the string.

Example 3:

Input: $s = "()(())((()()))"$

Output: 3

Constraints:

- $1 \leq s.length \leq 100$
- s consists of digits 0–9 and characters '+', '-', '*', '/', '(', and ')'. It is guaranteed that parentheses expression s is a VPS.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxDepth(self, s: str) -> int:
        ans = 0
        opened = 0

        for c in s:
            if c == '(':
                opened += 1
                ans = max(ans, opened)
            elif c == ')':
                opened -= 1

        return ans
```

• LeetDoocs

Round 1 · High · Easy

p.249

```
class Solution:
    def maxDepth(self, s: str) -> int:
        ans = d = 0
        for c in s:
            if c == '(':
                d += 1
                ans = max(ans, d)
            elif c == ')':
                d -= 1
        return ans
```

• SimplyLeet

```
# Function to calculate the maximum nesting depth of the parentheses in the string
def maxDepth(s: str) -> int:
    currentDepth = 0 # To keep track of current nesting level
    maxDepth = 0 # To record the maximum nesting depth encountered

    # Iterate over each character in the string s
    for char in s:
        if char == '(':
            currentDepth += 1 # Increase nesting level for '('
            maxDepth = max(maxDepth, currentDepth) # Update maxDepth if needed

        elif char == ')':
            currentDepth -= 1 # Decrease nesting level for ')'

    return maxDepth

# Example usage
s = "(1+(2*3)+((8)/4))+1"
print(maxDepth(s)) # Output: 3
```

Round 1 · High · Easy

p.250

Tree Traversal – Level Order

Round 1 · High · Easy · 2 questions

Round 1 · High · Easy

p.251

Tree Traversal – Level Order

□ #104 Maximum Depth of Binary Tree

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · BFS

Lists: Grind 169

Problem

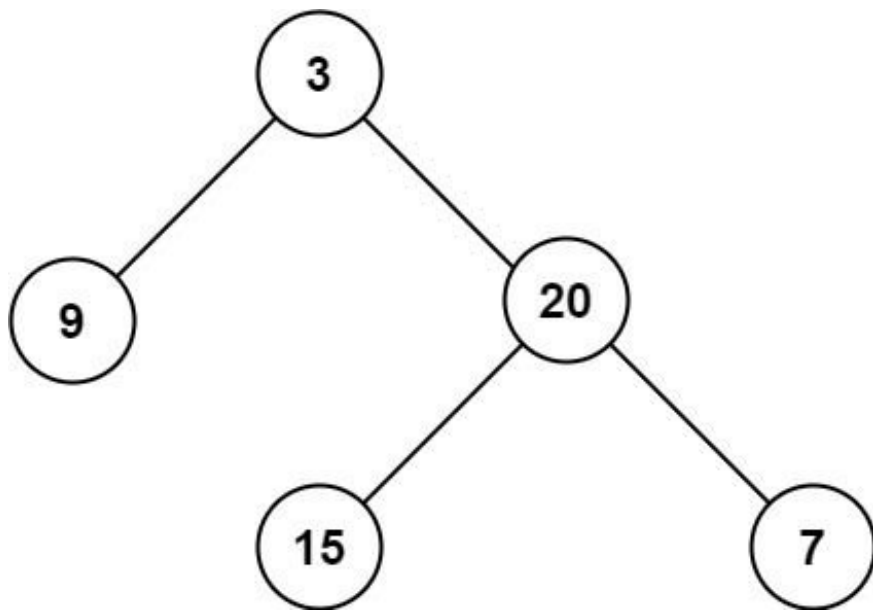
Given the root of a binary tree, return its maximum depth.

A binary tree's maximum depth is the number of nodes along the longest path from the root node down to the farthest leaf node.

Example 1:

Round 1 · High · Easy

p.252



Input: root = [3,9,20,null,null,15,7]
Output: 3

Example 2:

Input: root = [1,null,2]
Output: 2

Constraints:

- The number of nodes in the tree is in the range [0, 104].
- $-100 \leq \text{Node.val} \leq 100$

Community Solutions (Python)

- WalkCC

Round 1 · High · Easy

p.253

```

class Solution:
    def maxDepth(self, root: TreeNode | None) -> int:
        if not root:
            return 0
        return 1 + max(self.maxDepth(root.left), self.maxDepth(root.right))
  
```

• LeetCode

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def maxDepth(self, root: TreeNode) -> int:
        if root is None:
            return 0

        l, r = self.maxDepth(root.left), self.maxDepth(root.right)
        return 1 + max(l, r)
  
```

• SimplyLeet

```

# Python implementation using DFS

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
  
```

Round 1 · High · Easy

p.254

```
def maxDepth(self, root: Optional[TreeNode]) -> int:
    # Base case: if the node is null, return 0.
    if not root:
        return 0

    # Recursively compute the maximum depth of the left subtree.
    left_depth = self.maxDepth(root.left)
    # Recursively compute the maximum depth of the right subtree.
    right_depth = self.maxDepth(root.right)

    # Return the greater depth plus one for the current node.
    return 1 + max(left_depth, right_depth)
```

• LC.ca

```
# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):
# self.val = val
# self.left = left
# self.right = right
class Solution:
    def maxDepth(self, root: TreeNode) -> int:
        if root is None:
            return 0

        l, r = self.maxDepth(root.left), self.maxDepth(root.right)
        return 1 + max(l, r)
```

◆ Quick Review

Key Insights

- Use a recursive strategy (DFS) to traverse the binary tree.
- At each node, compute the maximum depth of its left and right subtree.

Round 1 · High · Easy

p.255

- The maximum depth at the current node is 1 (current node) plus the maximum of the left and right subtree depths.
- Alternatively, an iterative approach using BFS (level-order traversal) can be used.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes, since every node is visited once.

Space Complexity: $O(h)$ for recursion stack in DFS, where h is the height of the tree (worst-case $O(n)$ for a skewed tree); $O(n)$ for BFS in the worst-case scenario if the tree is balanced.

Solution

The problem is approached using Depth-First Search (DFS) with recursion. The idea is to visit every node and calculate the maximum depth of the left and right subtrees, then add 1 for the current node. This method naturally handles the base case when a null node (empty tree) is reached by returning 0. Alternatively, a Breadth-First Search (BFS) approach can determine the tree level by level, with the total number of levels equaling the

Round 1 · High · Easy

p.256

maximum depth. Key data structures include recursion call stack for DFS or a queue for BFS.

Round 1 · High · Easy

p.257

Tree Traversal – Level Order

□ #637 Average of Levels in Binary Tree

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · Depth-First Search · Breadth-First
Search · Binary Tree
Lists: AlgoMaster 600

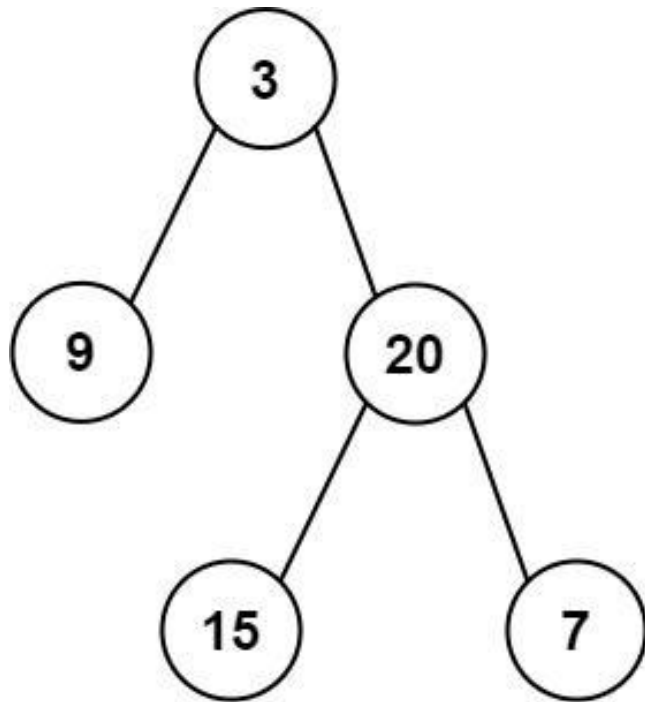
Problem

Given the root of a binary tree, return the average value of the nodes on each level in the form of an array. Answers within 10^{-5} of the actual answer will be accepted.

Example 1:

Round 1 · High · Easy

p.258

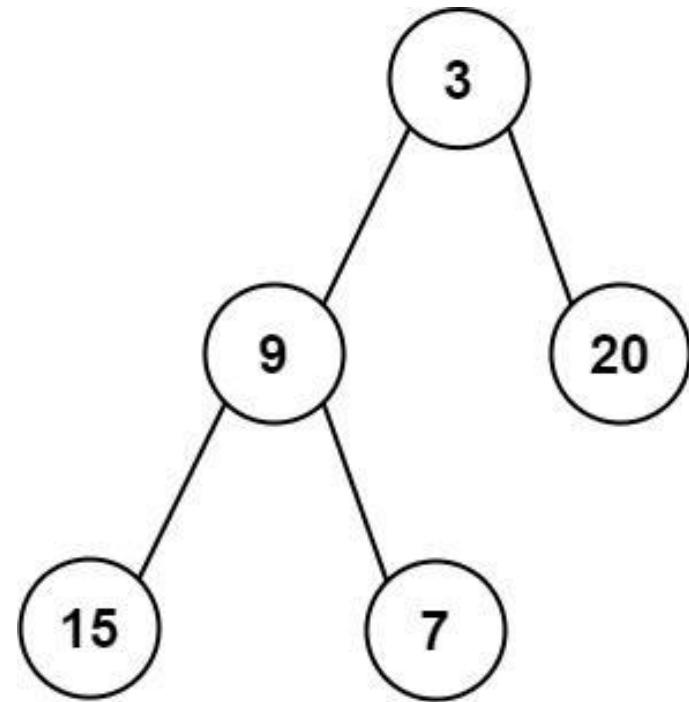


Input: root = [3,9,20,null,null,15,7]
 Output: [3.00000,14.50000,11.00000]
 Explanation: The average value of nodes on level 0 is 3, on level 1 is 14.5, and on level 2 is 11.
 Hence return [3, 14.5, 11].

Example 2:

Round 1 · High · Easy

p.259



Input: root = [3,9,20,15,7]
 Output: [3.00000,14.50000,11.00000]

Constraints:

- The number of nodes in the tree is in the range [1, 104].
- $-231 \leq \text{Node.val} \leq 231 - 1$

Community Solutions (Python)

- [LeetDoocs](#)

Round 1 · High · Easy

p.260

```
# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):
# self.val = val
# self.left = left
# self.right = right
class Solution:
    def averageOfLevels(self, root: Optional[TreeNode]) -> List[float]:
        q = deque([root])
        ans = []
```

```
        while q:
            s, n = 0, len(q)
            for _ in range(n):
                root = q.popleft()
                s += root.val
                if root.left:
                    q.append(root.left)
                if root.right:
                    q.append(root.right)
            ans.append(s / n)
```

```
        return ans
```

• SimplyLeet

```
# Python solution using BFS (level-order traversal)
from collections import deque

def averageOfLevels(root):
    # List to store the average of each level
    result = []
    # Deque for efficient popping from the left
    queue = deque([root])

    # Process until no more nodes exist in the current level
```

```
while queue:
    level_size = len(queue)
    level_sum = 0
    # Iterate through nodes on the current level
    for _ in range(level_size):
        node = queue.popleft() # Remove node from the queue
        level_sum += node.val # Add node's value to current level sum
        # Enqueue left and right children if they exist
        if node.left:
            queue.append(node.left)

        if node.right:
            queue.append(node.right)
    # Compute the average and add it to the result list
    result.append(level_sum / level_size)
return result
```

Tree Traversal – Pre Order

Round 1 · High · Easy · 8 questions

Round 1 · High · Easy

p.263

Tree Traversal – Pre Order

□ #100 Same Tree

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · BFS

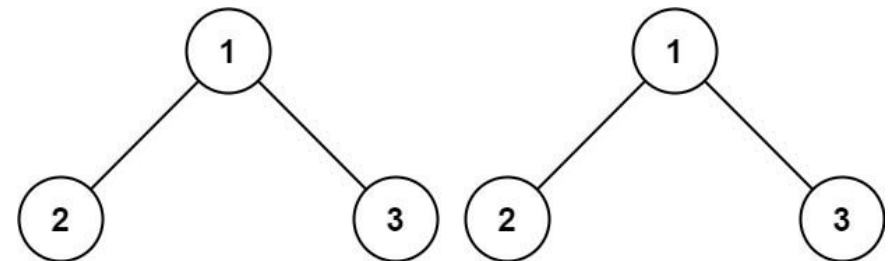
Lists: Grind 169

Problem

Given the roots of two binary trees p and q , write a function to check if they are the same or not.

Two binary trees are considered the same if they are structurally identical, and the nodes have the same value.

Example 1:

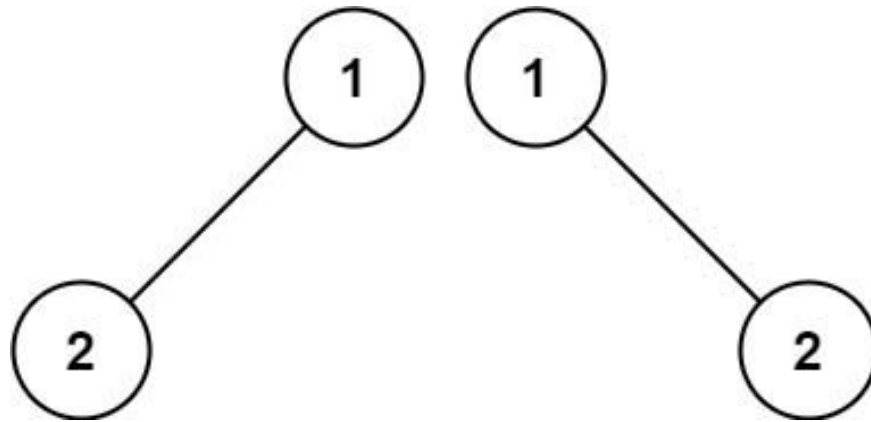


Input: $p = [1,2,3]$, $q = [1,2,3]$
Output: true

Example 2:

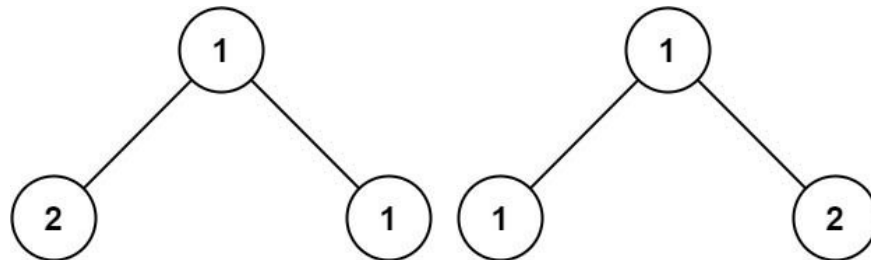
Round 1 · High · Easy

p.264



Input: p = [1,2], q = [1,null,2]
Output: false

Example 3:



Input: p = [1,2,1], q = [1,1,2]
Output: false

Constraints:

- The number of nodes in both trees is in the range [0, 100].
- $-104 \leq \text{Node.val} \leq 104$

Round 1 · High · Easy

p.265

Community Solutions (Python)

• WalkCC

```

class Solution:
    def isSameTree(self, p: TreeNode | None, q: TreeNode | None) -> bool:
        if not p or not q:
            return p == q
        return (p.val == q.val and
                self.isSameTree(p.left, q.left) and
                self.isSameTree(p.right, q.right))

```

• LeetCode

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isSameTree(self, p: Optional[TreeNode], q: Optional[TreeNode]) -> bool:
        :
            if p == q:
                return True

            if p is None or q is None or p.val != q.val:
                return False
            return self.isSameTree(p.left, q.left) and self.isSameTree(p.right, q.right)

```

• SimplyLeet

Round 1 · High · Easy

p.266

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

def isSameTree(p, q):
    # If both nodes are None, trees are identical at this branch.
    if not p and not q:
        return True
    # If one of the nodes is None, trees are not identical.
    if not p or not q:
        return False
    # Check if current nodes have the same value.
    if p.val != q.val:
        return False
    # Recursively compare the left subtrees and the right subtrees.
    return isSameTree(p.left, q.left) and isSameTree(p.right, q.right)
```

• LC.ca

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
# Definition for a binary tree node.
# class TreeNode(object):
#     def __init__(self, x):
#         self.val = x
```

Round 1 · High · Easy

p.267

```
# self.left = None
# self.right = None

class Solution(object):
    def isSameTree(self, p, q):
        """
        :type p: TreeNode
        :type q: TreeNode
        :rtype: bool
        """

        if not p or not q:
            return p == q # covers: p=None and q!=None, q=None and p!=None, both None
            return p.val == q.val and self.isSameTree(p.left, q.left) and self.isSameTree(p.right, q.right)

# iteration
class Solution:
    def isSameTree(self, p: TreeNode, q: TreeNode) -> bool:
        if p is None:
            return q is None

        if q is None:
            return p is None

        stack1 = [p]
        stack2 = [q]

        while stack1 and stack2:
            current1 = stack1.pop()
            current2 = stack2.pop()
```

Round 1 · High · Easy

p.268

```

if current1 is None and current2 is None:
    continue
elif current1 is None or current2 is None:
    return False
elif current1.val != current2.val:
    return False

stack1.append(current1.left)
stack2.append(current2.left)

stack1.append(current1.right)
stack2.append(current2.right)

return not stack1 and not stack2

```

◆ Quick Review

Key Insights

- Use recursion to compare nodes in both trees.
- If both nodes being compared are null, they are the same.
- If one node is null while the other is not, the trees differ.
- Check if current nodes' values match, then recursively verify the left and right subtrees.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes in the trees, since every node is

visited once.

Space Complexity: $O(n)$ in the worst-case (unbalanced tree), due to recursion call stack usage. In a balanced tree, the space would be $O(\log n)$.

Solution

The problem can be solved using a recursive approach (Depth-First Search) where we compare the current nodes from both trees. If the current nodes are both null, they match; if one is null, then the trees are not identical. For non-null nodes, check if the values are equal. Then, recursively apply the same logic on the left and right children of the nodes. This approach ensures that both the structure and the node values are identical across the two trees.

Tree Traversal – Pre Order

□ #101 Symmetric Tree

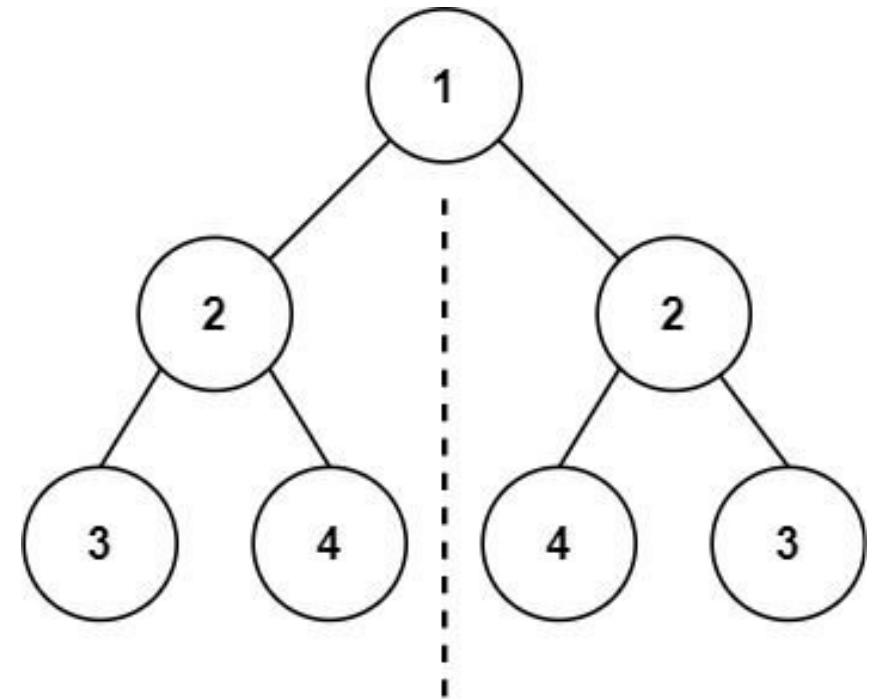
EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Tree · DFS · BFS
 Lists: Grind 169

Problem

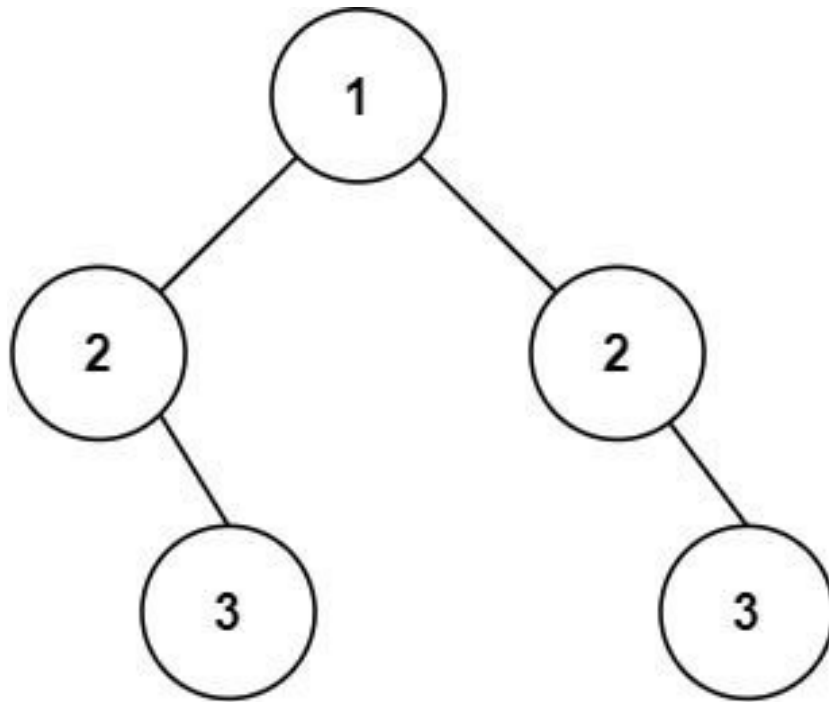
Given the root of a binary tree, check whether it is a mirror of itself (i.e., symmetric around its center).

Example 1:



Input: root = [1,2,2,3,4,4,3]
 Output: true

Example 2:



Input: root = [1,2,2,null,3,null,3]
Output: false

Constraints:

- The number of nodes in the tree is in the range [1, 1000].
- $-100 \leq \text{Node.val} \leq 100$

Follow up: Could you solve it both recursively and iteratively?

Community Solutions (Python)

Round 1 · High · Easy

p.273

• WalkCC

```

class Solution:
    def isSymmetric(self, root: TreeNode | None) -> bool:
        def isSymmetric(p: TreeNode | None, q: TreeNode | None) -> bool:
            if not p or not q:
                return p == q
            return (p.val == q.val and
                    isSymmetric(p.left, q.right) and
                    isSymmetric(p.right, q.left))

        return isSymmetric(root, root)
  
```

• LeetCode

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isSymmetric(self, root: Optional[TreeNode]) -> bool:
        def dfs(root1: Optional[TreeNode], root2: Optional[TreeNode]) -> bool:
            if root1 == root2:
                return True
            if root1 is None or root2 is None or root1.val != root2.val:
                return False
            return dfs(root1.left, root2.right) and dfs(root1.right, root2.left)

        return dfs(root.left, root.right)
  
```

• SimplyLeet

Round 1 · High · Easy

p.274

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val      # current node's value
        self.left = left    # pointer to left child
        self.right = right  # pointer to right child

class Solution:
    def isSymmetric(self, root: TreeNode) -> bool:
        # Helper function to check mirror symmetry between two trees.

        def isMirror(t1: TreeNode, t2: TreeNode) -> bool:
            # Both nodes are null, so they are mirrors.
            if t1 is None and t2 is None:
                return True
            # If one is null or the values do not match, symmetry fails.
            if t1 is None or t2 is None or t1.val != t2.val:
                return False
            # Recursively check mirror symmetry for children:
            # t1's left with t2's right and t1's right with t2's left.
            return isMirror(t1.left, t2.right) and isMirror(t1.right, t2.left)

        # Start recursion comparing the tree with itself.
        return isMirror(root, root)
```

• LC.ca

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isSymmetric(self, root: Optional[TreeNode]) -> bool:
        def dfs(root1, root2):
            if root1 is None and root2 is None:
```

Round 1 · High · Easy

p.275

```
        return True
        if root1 is None or root2 is None or root1.val != root2.val:
            return False
        return dfs(root1.left, root2.right) and dfs(root1.right, root2.left)

    def dfs(self, root1, root2):
        return dfs(root1, root2)
```

◆ Quick Review

Key Insights

- Use recursion (or iteration) to compare pairs of nodes.
- For two trees (or two nodes) to be mirror images:
 - Their root values must be the same.
 - The left subtree of one must match the right subtree of the other.
 - The right subtree of one must match the left subtree of the other.
- Alternatively, iterative approaches using a queue can be applied to compare the nodes in pairs.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes in the tree, since we traverse each node once.

Round 1 · High · Easy

p.276

Space Complexity: $O(h)$ for recursive call stack (worst-case $O(n)$ for a skewed tree) or $O(n)$ for the iterative approach using a queue.

Solution

We solve the problem using recursion by defining a helper function that takes two nodes and checks if they are mirrors of each other. The helper function:

- Returns true if both nodes are null.
- Returns false if one node is null or if their values differ.
- Recursively compares the left child of the first node with the right child of the second node, and vice-versa. This method ensures each comparison verifies symmetry at every level. A similar logic can be applied iteratively by using a queue to compare pairs of nodes.

Round 1 · High · Easy

p.277

Tree Traversal – Pre Order

☐ #108 Convert Sorted Array to Binary Search Tree

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Divide and Conquer · Tree
Lists: Grind 169

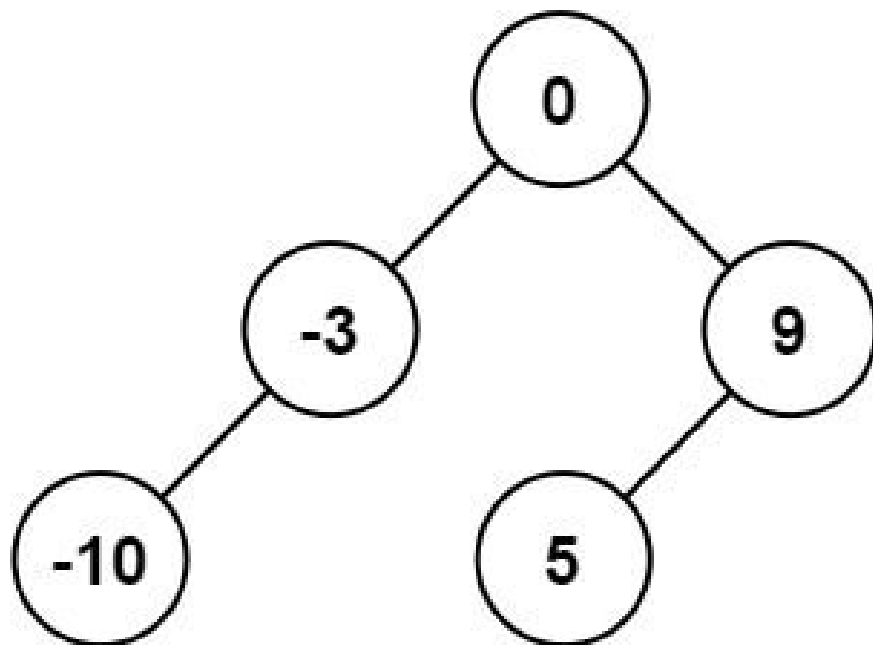
Problem

Given an integer array `nums` where the elements are sorted in ascending order, convert it to a height-balanced binary search tree.

Example 1:

Round 1 · High · Easy

p.278

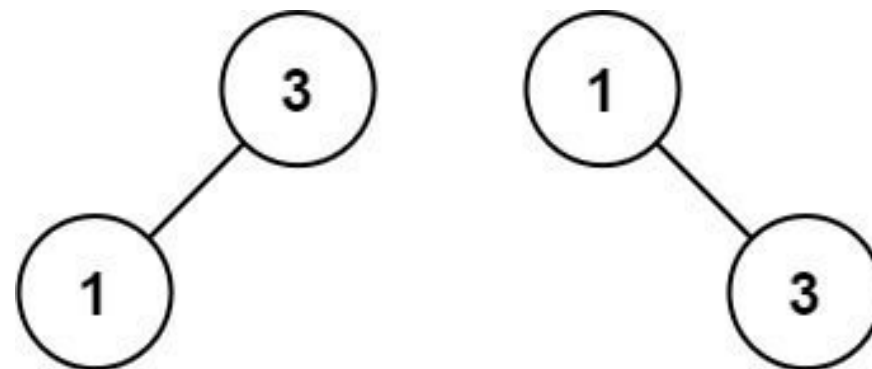


Input: nums = [-10,-3,0,5,9]
 Output: [0,-3,9,-10,null,5]
 Explanation: [0,-10,5,null,-3,null,9] is also accepted:

Example 2:

Round 1 · High · Easy

p.279



Input: nums = [1,3]
 Output: [3,1]
 Explanation: [1,null,3] and [3,1] are both height-balanced BSTs.

Constraints:

- $1 \leq \text{nums.length} \leq 104$
- $-104 \leq \text{nums}[i] \leq 104$
- nums is sorted in a strictly increasing order.

Community Solutions (Python)

• WalkCC

```

class Solution:
    def sortedArrayToBST(self, nums: list[int]) -> TreeNode | None:
        def build(l: int, r: int) -> TreeNode | None:
            if l > r:
                return None
            m = (l + r) // 2
            return TreeNode(nums[m],
                            build(l, m - 1),
                            build(m + 1, r))

        return build(0, len(nums) - 1)
  
```

Round 1 · High · Easy

p.280

• LeetCode

```
# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):
# self.val = val
# self.left = left
# self.right = right
class Solution:
    def sortedArrayToBST(self, nums: List[int]) -> Optional[TreeNode]:
        def dfs(l: int, r: int) -> Optional[TreeNode]:
            if l > r:

                return None
            mid = (l + r) >> 1
            return TreeNode(nums[mid], dfs(l, mid - 1), dfs(mid + 1, r))

        return dfs(0, len(nums) - 1)
```

• SimplyLeet

```
# Definition for a binary tree node.
class TreeNode:
    # Initializes a tree node with a value and optional left/right children.
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def sortedArrayToBST(self, nums):

        # Helper function to build BST from subarray indices.
        def buildBST(left, right):
            # Base condition: if no elements to process, return None.
            if left > right:
                return None
            # Choose the middle element as the root for balance.
            mid = (left + right) // 2
            node = TreeNode(nums[mid])
            # Recursively build the left subtree.
            node.left = buildBST(left, mid - 1)
```

• LC.ca

```
# Recursively build the right subtree.
node.right = buildBST(mid + 1, right)
return node

# Build and return the BST from the entire array.
return buildBST(0, len(nums) - 1)
```

```
# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):
# self.val = val
# self.left = left
# self.right = right
class Solution:
    def sortedArrayToBST(self, nums: List[int]) -> Optional[TreeNode]:
        def dfs(l, r):
            if l > r:

                return None
            mid = (l + r) >> 1
            left = dfs(l, mid - 1)
            right = dfs(mid + 1, r)
            return TreeNode(nums[mid], left, right)

        return dfs(0, len(nums) - 1)
```

```
#####
```

```
# Definition for a binary tree node.
# class TreeNode(object):
# def __init__(self, x):
# self.val = x
# self.left = None
# self.right = None
```

```
class Solution(object):
    def sortedArrayToBST(self, nums):
        ""
```

```

:type nums: List[int]
:rtype: TreeNode
"""
if nums:
    midPos = len(nums) // 2
    mid = nums[midPos]
    root = TreeNode(mid)
    root.left = self.sortedArrayToBST(nums[:midPos])
    root.right = self.sortedArrayToBST(nums[midPos + 1:])
    return root

```

◆ Quick Review

Key Insights

- The array is sorted in ascending order, which allows the use of a divide and conquer strategy.
- Choosing the middle element of the array as the root ensures that the left and right subtrees are roughly equal in size, yielding a balanced BST.
- A recursive approach is natural for building the tree: the base case is when the subarray is empty.
- The algorithm splits the array into two halves to recursively build the left and right subtrees.

Space & Time

Round 1 · High · Easy

p.283

Time Complexity: $O(n)$ – Every element is processed exactly once.

Space Complexity: $O(n)$ – $O(n)$ space is used to store the BST, and additional $O(\log n)$ space may be used for the recursion stack.

Solution

We use a divide and conquer approach with recursion. The main idea is to select the middle element of the current subarray as the root node so that the left subarray elements form the left subtree and the right subarray elements form the right subtree, ensuring the tree is balanced. The recursion continues until the subarray is empty, which serves as the base case. This strategy naturally maintains the BST property since all elements in the left subarray are smaller than the root, and all in the right subarray are larger.

Round 1 · High · Easy

p.284

Tree Traversal – Pre Order

□ #144 Binary Tree Preorder Traversal

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Stack · Tree · Depth-First Search · Binary Tree
Lists: AlgoMaster 600

Problem

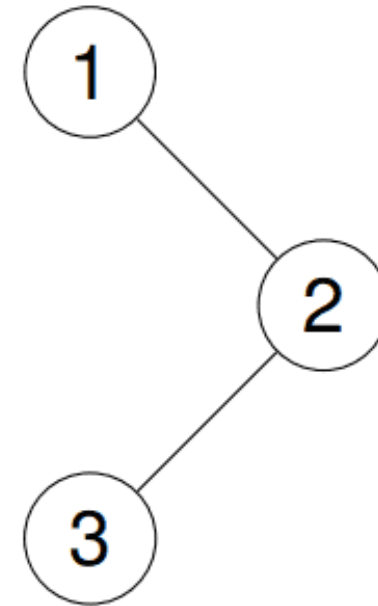
Given the root of a binary tree, return the preorder traversal of its nodes' values.

Example 1:

Input: root = [1,null,2,3]

Output: [1,2,3]

Explanation:

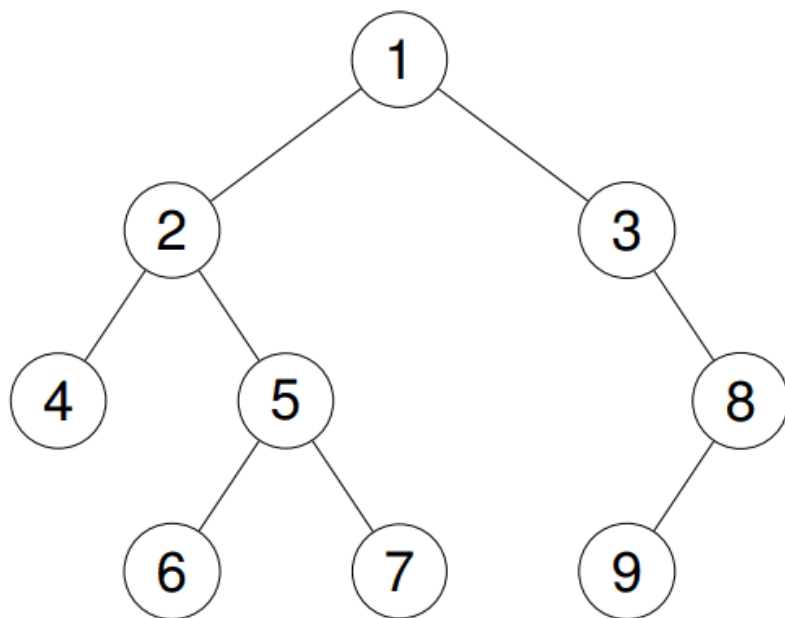


Example 2:

Input: root = [1,2,3,4,5,null,8,null,null,6,7,9]

Output: [1,2,4,5,6,7,3,8,9]

Explanation:



Example 3:

Input: root = []

Output: []

Example 4:

Input: root = [1]

Output: [1]

Constraints:

- The number of nodes in the tree is in the range [0, 100].

Round 1 · High · Easy

p.287

- $-100 \leq \text{Node.val} \leq 100$

Follow up: Recursive solution is trivial, could you do it iteratively?

Community Solutions (Python)

• WalkCC

```

class Solution:
    def preorderTraversal(self, root: TreeNode | None) -> list[int]:
        ans = []

        def preorder(root: TreeNode | None) -> None:
            if not root:
                return

            ans.append(root.val)
            preorder(root.left)

            preorder(root.right)

        preorder(root)
        return ans
  
```

```

class Solution:
    def preorderTraversal(self, root: TreeNode | None) -> list[int]:
        if not root:
            return []

        ans = []
        stack = [root]

        while stack:
            node = stack.pop()
  
```

Round 1 · High · Easy

p.288

```

ans.append(node.val)
if node.right:
    stack.append(node.right)
if node.left:
    stack.append(node.left)

return ans

```

• LeetCode

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def preorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
        def dfs(root):
            if root is None:
                return
            ans.append(root.val)
            dfs(root.left)
            dfs(root.right)

        ans = []
        dfs(root)
        return ans

```

• SimplyLeet

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

def preorderTraversal(root):
    # List to store the preorder traversal
    result = []

```

Round 1 · High · Easy

p.289

```

# Edge case: if the tree is empty, return an empty list
if not root:
    return result

# Initialize stack with the root node
stack = [root]

```

```

# Iterate while the stack is not empty
while stack:

```

```

    # Pop the top node from the stack
    node = stack.pop()
    # Add the node's value to the result list
    result.append(node.val)

```

```

    # Push the right child first if it exists (stack LIFO ensures left is
    # processed next)
    if node.right:
        stack.append(node.right)
    # Push the left child if it exists
    if node.left:

```

```

        stack.append(node.left)

```

```

# Return the result list containing preorder traversal
return result

```

Round 1 · High · Easy

p.290

Tree Traversal – Pre Order

□ #222 Count Complete Tree Nodes

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Binary Search · Bit Manipulation · Tree · Binary Tree

Lists: AlgoMaster 600

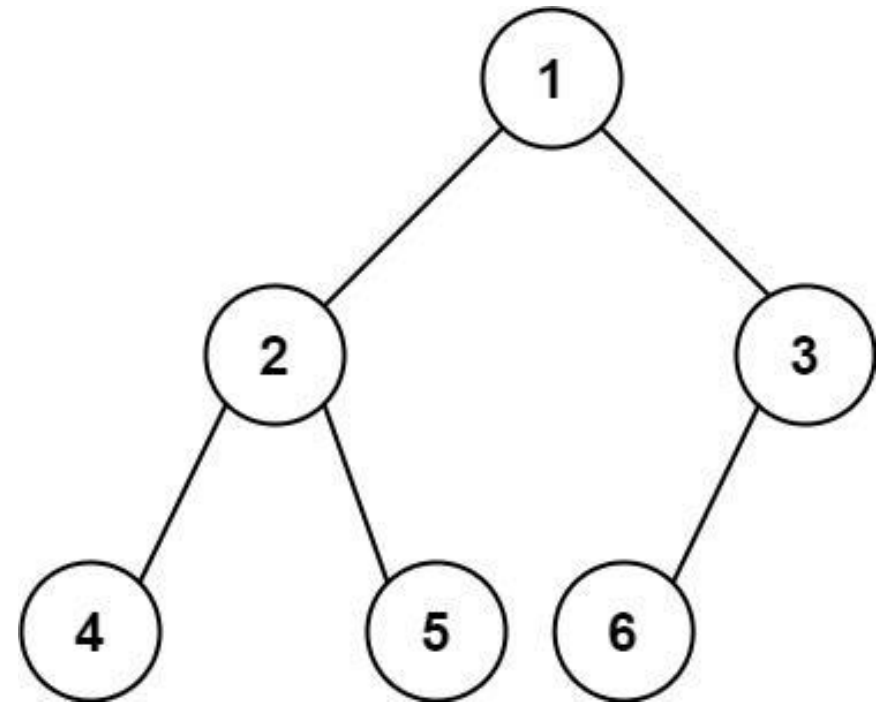
Problem

Given the root of a complete binary tree, return the number of the nodes in the tree.

According to Wikipedia, every level, except possibly the last, is completely filled in a complete binary tree, and all nodes in the last level are as far left as possible. It can have between 1 and 2^h nodes inclusive at the last level h .

Design an algorithm that runs in less than $O(n)$ time complexity.

Example 1:



Input: root = [1,2,3,4,5,6]
Output: 6

Example 2:

Input: root = []
Output: 0

Example 3:

Input: root = [1]
Output: 1

Constraints:

- The number of nodes in the tree is in the range $[0, 5 * 10^4]$.
- $0 \leq \text{Node.val} \leq 5 * 10^4$
- The tree is guaranteed to be complete.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def countNodes(self, root: TreeNode | None) -> int:
        if not root:
            return 0
        return 1 + self.countNodes(root.left) + self.countNodes(root.right)
```

• LeetCode

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def countNodes(self, root: Optional[TreeNode]) -> int:
        if root is None:
            return 0

        return 1 + self.countNodes(root.left) + self.countNodes(root.right)
```

• SimplyLeet

Round 1 · High · Easy

p.293

```
# Python Solution

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def countNodes(root: TreeNode) -> int:

    # Helper function to compute the left depth of the tree.
    def leftDepth(node: TreeNode) -> int:
        depth = 0
        while node:
            depth += 1
            node = node.left
        return depth

    # Helper function to compute the right depth of the tree.
    def rightDepth(node: TreeNode) -> int:
        depth = 0
        while node:
            depth += 1
            node = node.right
        return depth

    if not root:
        return 0

    left_depth = leftDepth(root)

    right_depth = rightDepth(root)

    # If left and right depth are the same, tree is perfect.
    if left_depth == right_depth:
        return (1 << left_depth) - 1 # equivalent to 2^left_depth - 1
    else:
        # Recursively count nodes in left and right subtree plus one for root.
        return 1 + countNodes(root.left) + countNodes(root.right)

# Example usage:
```

Round 1 · High · Easy

p.294

```
# root = TreeNode(1, TreeNode(2), TreeNode(3))  
# print(countNodes(root))
```

Round 1 · High · Easy

p.295

Tree Traversal – Pre Order

□ #257 Binary Tree Paths

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Backtracking · Tree · Depth-First
Search · Binary Tree
Lists: AlgoMaster 600

Problem

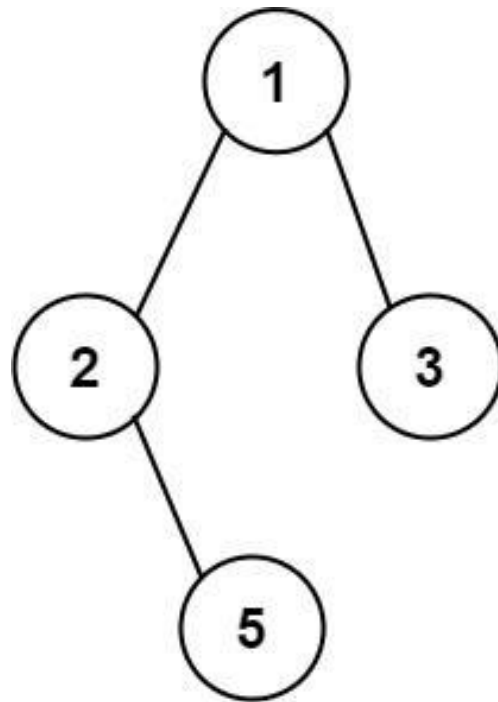
Given the root of a binary tree, return all
root-to-leaf paths in any order.

A leaf is a node with no children.

Example 1:

Round 1 · High · Easy

p.296



Input: root = [1,2,3,null,5]
 Output: ["1->2->5","1->3"]

Example 2:

Input: root = [1]
 Output: ["1"]

Constraints:

- The number of nodes in the tree is in the range [1, 100].
- $-100 \leq \text{Node.val} \leq 100$

Community Solutions (Python)

• WalkCC

```

class Solution:
    def binaryTreePaths(self, root: TreeNode | None) -> list[str]:
        ans = []

        def dfs(root: TreeNode | None, path: list[str]) -> None:
            if not root:
                return
            if not root.left and not root.right:
                ans.append(''.join(path) + str(root.val))
                return

            path.append(str(root.val) + '->')
            dfs(root.left, path)
            dfs(root.right, path)
            path.pop()

        dfs(root, [])
        return ans
  
```

• LeetCode

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def binaryTreePaths(self, root: Optional[TreeNode]) -> List[str]:
        def dfs(root: Optional[TreeNode]):
            if root is None:
  
```

```

        return
    t.append(str(root.val))
    if root.left is None and root.right is None:
        ans.append("->".join(t))
    else:
        dfs(root.left)
        dfs(root.right)
    t.pop()

ans = []

t = []
dfs(root)
return ans

```

• SimplyLeet

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def binaryTreePaths(self, root: TreeNode) -> List[str]:
        # List to store the final root-to-leaf paths.

        result = []

        # Helper function to perform DFS on the tree.
        def dfs(node, current_path):
            if not node:
                return
            # Append current node's value to the path.
            if current_path:
                current_path += "->" + str(node.val)
            else:

```

```

        current_path = str(node.val)
        # If the node is a leaf, add the path to result.
        if not node.left and not node.right:
            result.append(current_path)
            return
        # Continue DFS on the left and right children.
        if node.left:
            dfs(node.left, current_path)
        if node.right:
            dfs(node.right, current_path)

# Initiate DFS from the root.
dfs(root, "")
return result

```

Tree Traversal – Pre Order

□ #404 Sum of Left Leaves

EAS

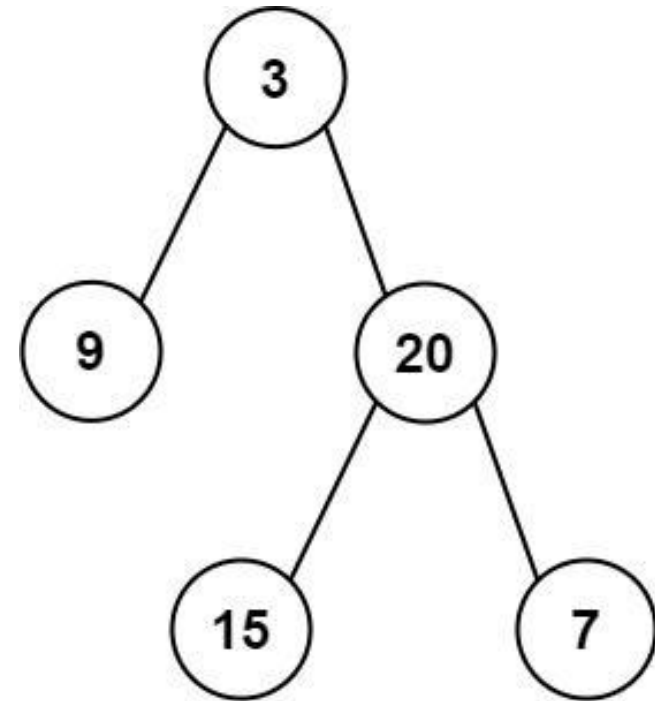
LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Tree · Depth-First Search · Breadth-First
 Search · Binary Tree
 Lists: AlgoMaster 600

Problem

Given the root of a binary tree, return the sum of all left leaves.

A leaf is a node with no children. A left leaf is a leaf that is the left child of another node.

Example 1:



Input: root = [3,9,20,null,null,15,7]

Output: 24

Explanation: There are two left leaves in the binary tree, with values 9 and 15 respectively.

Example 2:

Input: root = [1]

Output: 0

Constraints:

- The number of nodes in the tree is in the range [1, 1000].
- $-1000 \leq \text{Node.val} \leq 1000$

Round 1 · High · Easy

p.301

Round 1 · High · Easy

p.302

Community Solutions (Python)

• WalkCC

```
class Solution:
    def sumOfLeftLeaves(self, root: TreeNode | None) -> int:
        if not root:
            return 0

        ans = 0

        if root.left:
            if not root.left.left and not root.left.right:
                ans += root.left.val

        else:
            ans += self.sumOfLeftLeaves(root.left)
            ans += self.sumOfLeftLeaves(root.right)

        return ans
```

• LeetCode

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def sumOfLeftLeaves(self, root: Optional[TreeNode]) -> int:
        if root is None:
            return 0

        ans = self.sumOfLeftLeaves(root.right)
        if root.left:
            if root.left.left == root.left.right:
                ans += root.left.val
            else:
                ans += self.sumOfLeftLeaves(root.left)
        return ans
```

Round 1 · High · Easy

p.303

• SimplyLeet

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

def sumOfLeftLeaves(root):
    # If the node is None, return 0 as there is nothing to add.
    if not root:
        return 0
    total = 0
    # Check if the left child exists and is a leaf.
    if root.left:
        if not root.left.left and not root.left.right:
            # If it's a left leaf, add its value.
            total += root.left.val
        else:
            # Otherwise, recursively check the left subtree.
            total += sumOfLeftLeaves(root.left)

    # Always traverse the right subtree.
    total += sumOfLeftLeaves(root.right)
    return total
```

Round 1 · High · Easy

p.304

Tree Traversal – Pre Order

□ #572 Subtree of Another Tree

EAS

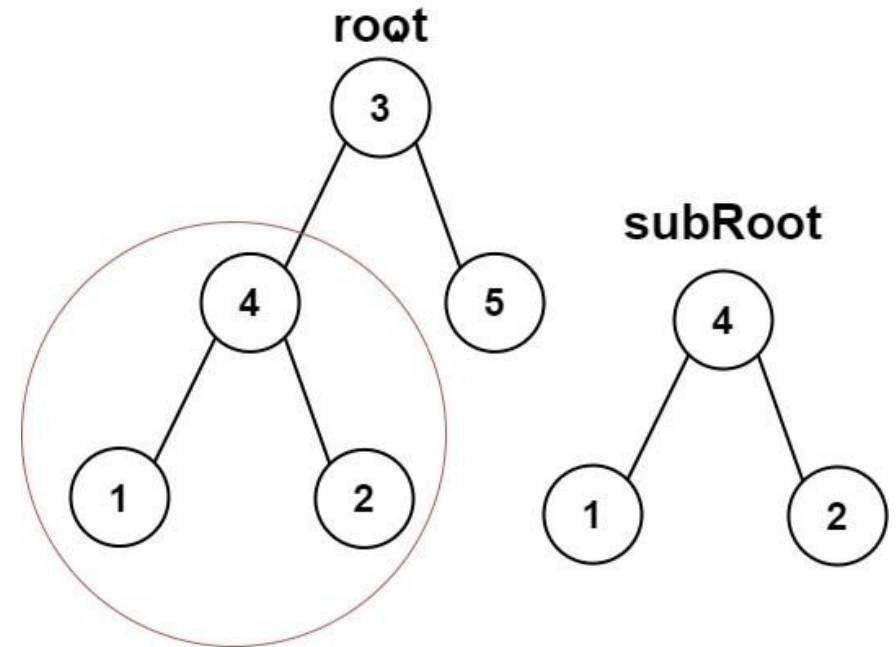
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · String Matching
Lists: Grind 169

Problem

Given the roots of two binary trees `root` and `subRoot`, return `true` if there is a subtree of `root` with the same structure and node values of `subRoot` and `false` otherwise.

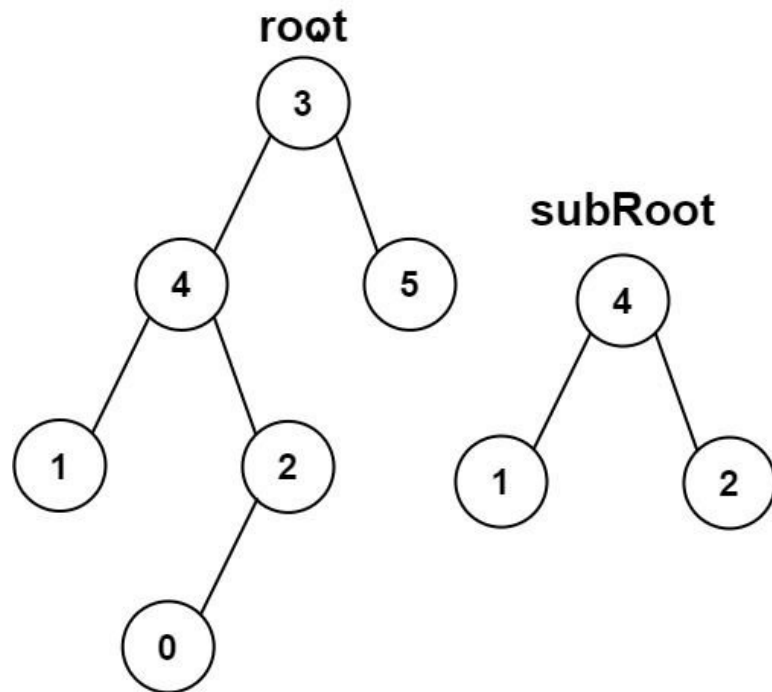
A subtree of a binary tree `tree` is a tree that consists of a node in `tree` and all of this node's descendants. The tree `tree` could also be considered as a subtree of itself.

Example 1:



Input: `root = [3,4,5,1,2]`, `subRoot = [4,1,2]`
Output: `true`

Example 2:



Input: root = [3,4,5,1,2,null,null,null,null,0], subRoot = [4,1,2]
 Output: false

Constraints:

- The number of nodes in the root tree is in the range [1, 2000].
- The number of nodes in the subRoot tree is in the range [1, 1000].
- $-104 \leq \text{root.val} \leq 104$
- $-104 \leq \text{subRoot.val} \leq 104$

Round 1 · High · Easy

p.307

Community Solutions (Python)

• LeetCode

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isSubtree(self, root: Optional[TreeNode], subRoot: Optional[TreeNode]) -> bool:
        def same(p: Optional[TreeNode], q: Optional[TreeNode]) -> bool:
            if p is None or q is None:
                return p is q
            return p.val == q.val and same(p.left, q.left) and same(p.right, q.right)

        if root is None:
            return False
        return (
            same(root, subRoot)
            or self.isSubtree(root.left, subRoot)
            or self.isSubtree(root.right, subRoot)
        )
  
```

• SimplyLeet

```

# Python solution for checking if subRoot is a subtree of root.

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
  
```

Round 1 · High · Easy

p.308

```
def isSubtree(self, root: TreeNode, subRoot: TreeNode) -> bool:
    # Helper function to check if two trees are identical.
    def isSameTree(s, t):
        # Both nodes are None, trees match.
        if not s and not t:
            return True
        # One node is None or values differ.
        if not s or not t or s.val != t.val:
            return False
        # Recurse on left and right subtrees.

        return isSameTree(s.left, t.left) and isSameTree(s.right, t.right)

    # Base case: if root is None, then no subtree exists.
    if not root:
        return False
    # Check if current subtree is identical to subRoot.
    if isSameTree(root, subRoot):
        return True
    # Recursively check left and right subtrees.
    return self.isSubtree(root.left, subRoot) or self.isSubtree(root.right, subRoot)
```

• LC.ca

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isSubtree(self, root: TreeNode, subRoot: TreeNode) -> bool:
        def dfs(root1, root2):
            if root1 is None and root2 is None:
```

```
        return True
    if root1 is None or root2 is None:
        return False
    return (
        root1.val == root2.val
        and dfs(root1.left, root2.left)
        and dfs(root1.right, root2.right)
    )

    if root is None:

        return False
    return (
        dfs(root, subRoot)
        or self.isSubtree(root.left, subRoot)
        or self.isSubtree(root.right, subRoot)
    )
```

◆ Quick Review

Key Insights

- Use Depth-First Search (DFS) to visit each node in the main tree.
- At each node, check if the subtree starting from that node is identical to subRoot.
- A helper function is used to compare the structure and node values of two trees.
- Consider recursion as the primary approach to traverse and compare trees.
- Early exit when a mismatch is found can optimize average-case performance.

Space & Time

Time Complexity: $O(m * n)$ in the worst case, where m is the number of nodes in the main tree and n is the number of nodes in the subRoot tree.

Space Complexity: $O(\max(\text{depth of main tree}, \text{depth of subRoot}))$ due to recursive call stack.

Solution

We solve the problem by performing a DFS traversal on the main tree. For each visited node, a helper function checks whether the subtree starting at that node is identical to subRoot by comparing node values and recursively checking both left and right subtrees. This method leverages recursion effectively and provides early termination if mismatches occur, allowing for efficient traversal and comparison.

Round 1 · High · Easy

p.311

Tree Traversal – In Order

Round 1 · High · Easy · 3 questions

Round 1 · High · Easy

p.312

Tree Traversal – In Order

□ #94 Binary Tree Inorder Traversal

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Stack · Tree · Depth-First Search · Binary Tree
Lists: AlgoMaster 600

Problem

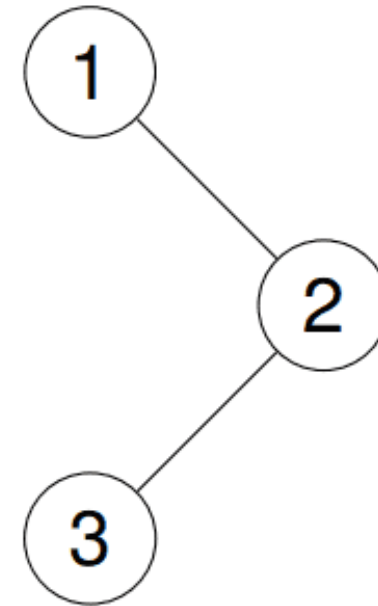
Given the root of a binary tree, return the
inorder traversal of its nodes' values.

Example 1:

Input: root = [1,null,2,3]

Output: [1,3,2]

Explanation:

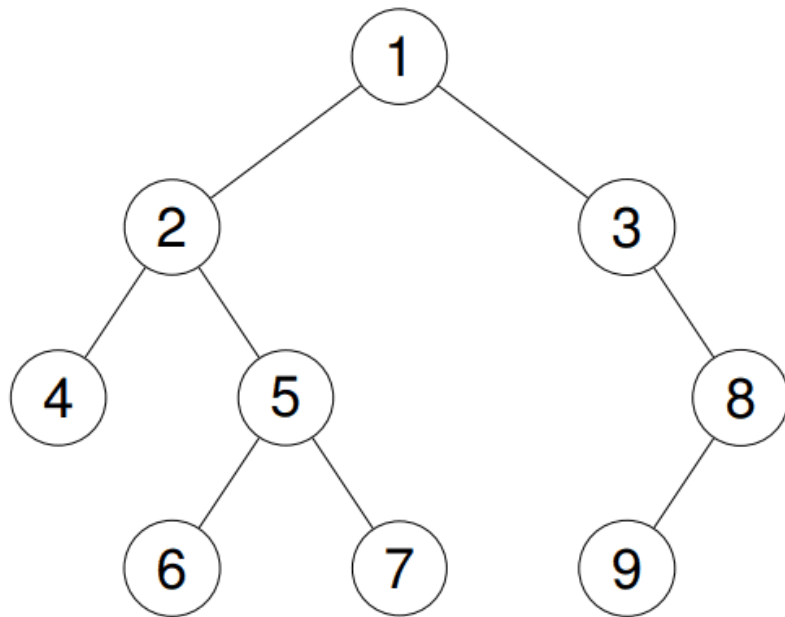


Example 2:

Input: root = [1,2,3,4,5,null,8,null,null,6,7,9]

Output: [4,2,6,5,7,1,3,9,8]

Explanation:



Example 3:

Input: root = []

Output: []

Example 4:

Input: root = [1]

Output: [1]

Constraints:

- The number of nodes in the tree is in the range [0, 100].

Round 1 · High · Easy

p.315

- $-100 \leq \text{Node.val} \leq 100$

Follow up: Recursive solution is trivial, could you do it iteratively?

Community Solutions (Python)

• WalkCC

```

class Solution:
    def inorderTraversal(self, root: TreeNode | None) -> List[int]:
        ans = []
        stack = []

        while root or stack:
            while root:
                stack.append(root)
                root = root.left
            root = stack.pop()

            ans.append(root.val)
            root = root.right

        return ans
  
```

• LeetCode

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def inorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
        def dfs(root):
            if root is None:
  
```

Round 1 · High · Easy

p.316

```

        return
    dfs(root.left)
    ans.append(root.val)
    dfs(root.right)

ans = []
dfs(root)
return ans

```

• SimplyLeet

```

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val      # Node value
        self.left = left    # Left child
        self.right = right   # Right child

class Solution:
    def inorderTraversal(self, root: TreeNode) -> list:
        result = []         # List to store inorder traversal

        stack = []           # Stack to simulate recursion
        current = root       # Start with the root node

        # Iterate while there are nodes to visit or process
        while current or stack:
            # Reach the left most node of the current subtree
            while current:
                stack.append(current)
                current = current.left

            # Current is null; backtrack via the stack
            current = stack.pop()
            result.append(current.val)  # Process current node
            current = current.right     # Consider right subtree

        return result

```

Round 1 · High · Easy

p.317

Tree Traversal – In Order

□ #530 Minimum Absolute Difference in BST

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Tree · Depth-First Search · Breadth-First
 Search · Binary Search Tree · Binary Tree
 Lists: AlgoMaster 600

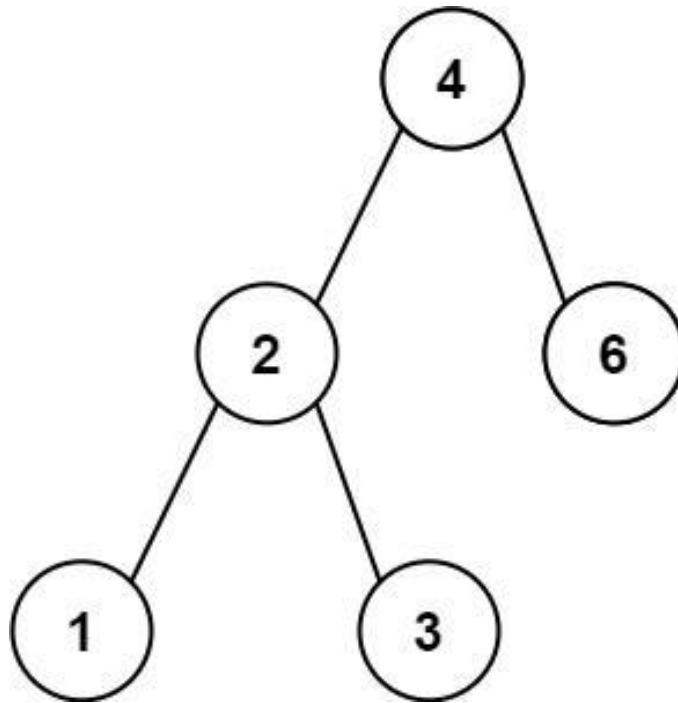
Problem

Given the root of a Binary Search Tree (BST),
 return the minimum absolute difference
 between the values of any two different nodes
 in the tree.

Example 1:

Round 1 · High · Easy

p.318

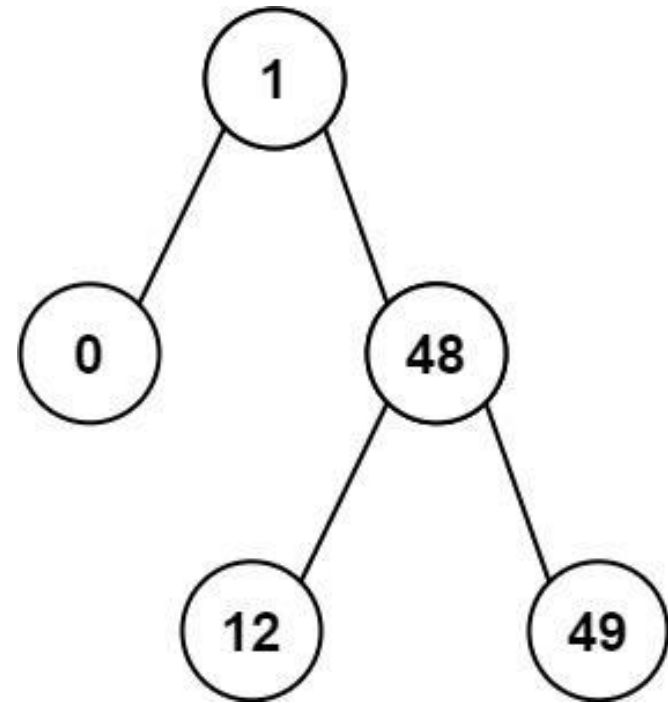


Input: root = [4,2,6,1,3]
Output: 1

Example 2:

Round 1 · High · Easy

p.319



Input: root = [1,0,48,null,null,12,49]
Output: 1

Constraints:

- The number of nodes in the tree is in the range [2, 104].
- $0 \leq \text{Node.val} \leq 105$

Note: This question is the same as 783: <https://leetcode.com/problems/minimum-distance-between-bst-nodes/>

Round 1 · High · Easy

p.320

Community Solutions (Python)

• LeetCode

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def getMinimumDifference(self, root: Optional[TreeNode]) -> int:
        def dfs(root: Optional[TreeNode]):
            if root is None:
                return
            dfs(root.left)
            nonlocal pre, ans
            ans = min(ans, root.val - pre)
            pre = root.val
            dfs(root.right)

        pre = -inf
        ans = inf
        dfs(root)

        return ans
```

• SimplyLeet

```
# Define the TreeNode class for clarity
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def getMinimumDifference(self, root: TreeNode) -> int:
        # Initialize previous node value (None initially) and the minimum
        # difference as infinity
```

Round 1 · High · Easy

p.321

```
self.prev = None
self.min_diff = float('inf')

# Inorder traversal function to process nodes in sorted order
def inorder(node):
    if not node:
        return
    # Process left subtree
    inorder(node.left)
    # If there is a previous node, update the minimum difference

    if self.prev is not None:
        self.min_diff = min(self.min_diff, node.val - self.prev)
    # Update the previous node value to the current node's value
    self.prev = node.val
    # Process right subtree
    inorder(node.right)

inorder(root)
return self.min_diff
```

Round 1 · High · Easy

p.322

Tree Traversal – In Order

□ #783 Minimum Distance Between BST Nodes

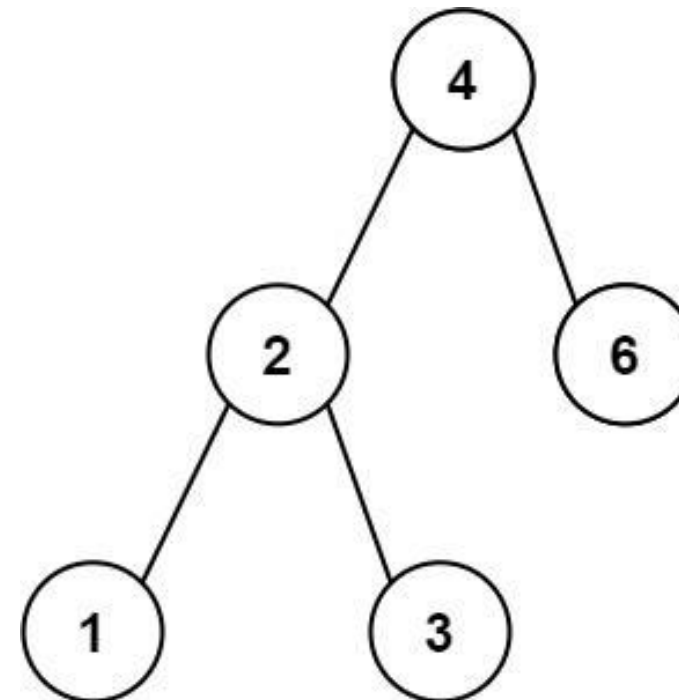
EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · Depth-First Search · Breadth-First
Search · Binary Search Tree · Binary Tree
Lists: AlgoMaster 600

Problem

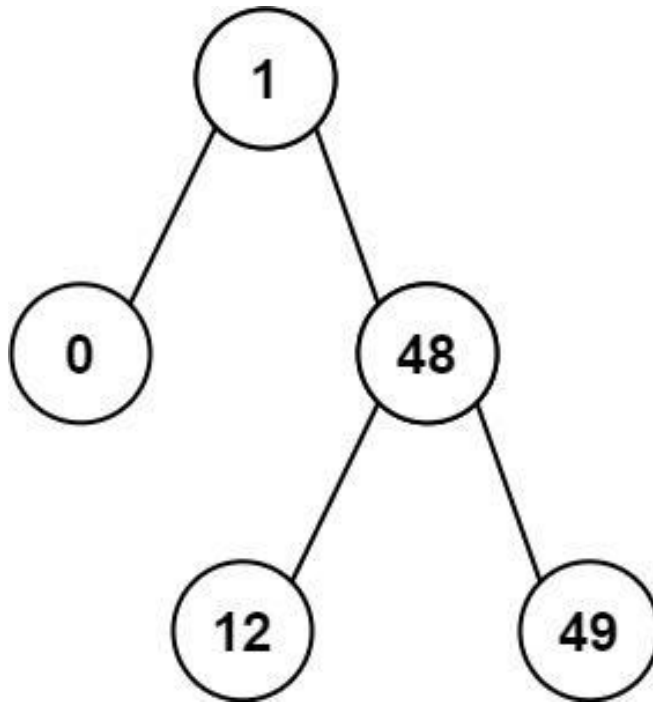
Given the root of a Binary Search Tree (BST),
return the minimum difference between the
values of any two different nodes in the tree.

Example 1:



Input: root = [4,2,6,1,3]
Output: 1

Example 2:



Input: root = [1,0,48,null,null,12,49]
Output: 1

Constraints:

- The number of nodes in the tree is in the range [2, 100].
- $0 \leq \text{Node.val} \leq 105$

Note: This question is the same as 530: <https://leetcode.com/problems/minimum-absolute-difference-in-bst/>

Community Solutions (Python)

• LeetCode

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def minDiffInBST(self, root: Optional[TreeNode]) -> int:
        def dfs(root: Optional[TreeNode]):
            if root is None:
                return

            dfs(root.left)
            nonlocal pre, ans
            ans = min(ans, root.val - pre)
            pre = root.val
            dfs(root.right)

        pre = -inf
        ans = inf
        dfs(root)

        return ans

```

• SimplyLeet

```

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def minDiffInBST(self, root: TreeNode) -> int:
        # Initialize variables:

```

```
# prev_val keeps track of the previous node's value.
# min_diff stores the minimum difference found so far, initialized to a
large number.
self.prev_val = None
self.min_diff = float('inf')

# Define the in-order traversal function.
def in_order(node):
    if not node:
        return
    # Traverse left subtree.

    in_order(node.left)

    # Process current node:
    if self.prev_val is not None:
        # Update min_diff using the difference with previous node.
        self.min_diff = min(self.min_diff, node.val - self.prev_val)
    # Update the previous value to current node's value.
    self.prev_val = node.val

    # Traverse right subtree.

    in_order(node.right)

# Start in-order traversal from the root.
in_order(root)
return self.min_diff
```

Round 1 · High · Easy

p.327

Tree Traversal – Post-Order

Round 1 · High · Easy · 4 questions

Round 1 · High · Easy

p.328

Tree Traversal – Post-Order

□ #110 Balanced Binary Tree

EAS

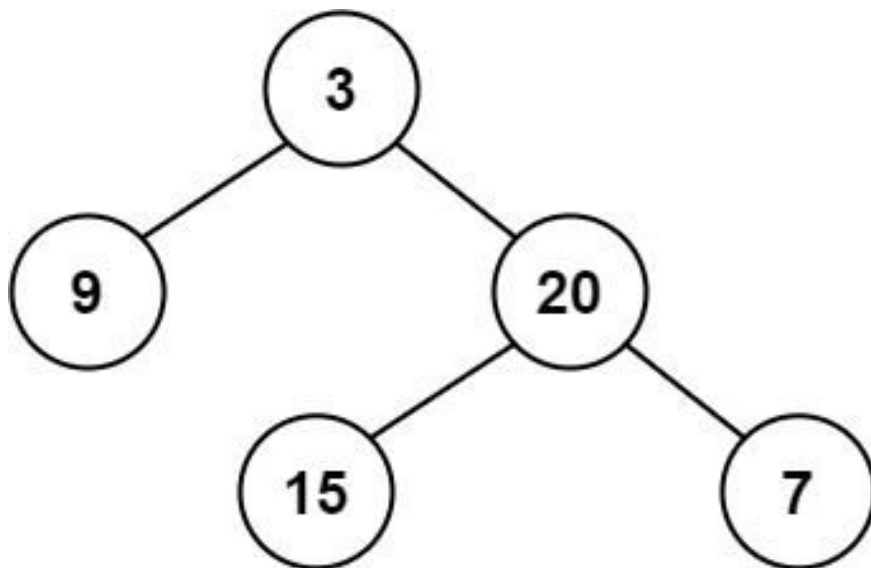
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS

Lists: Grind 169

Problem

Given a binary tree, determine if it is height-balanced.

Example 1:

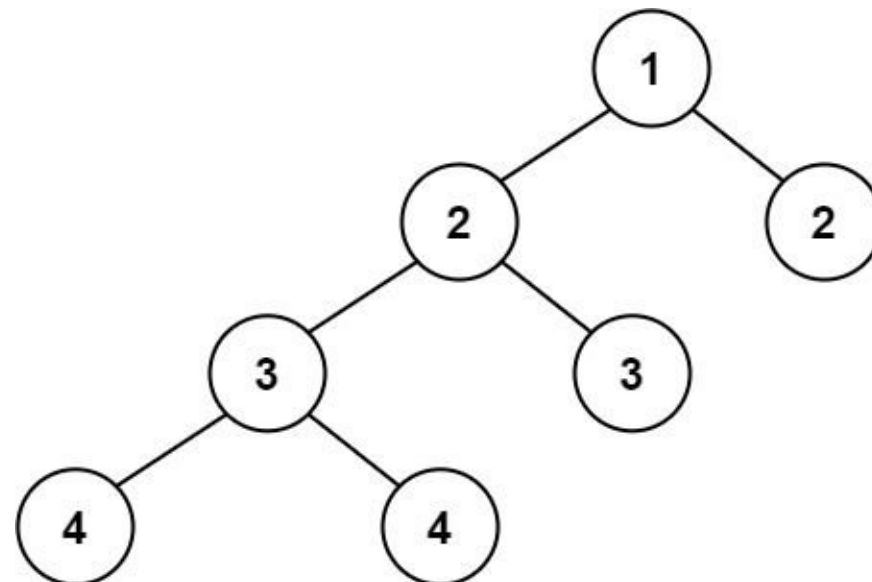


Round 1 · High · Easy

p.329

Input: root = [3,9,20,null,null,15,7]
Output: true

Example 2:



Input: root = [1,2,2,3,3,null,null,4,4]
Output: false

Example 3:

Input: root = []
Output: true

Constraints:

- The number of nodes in the tree is in the range [0, 5000].
- $-104 \leq \text{Node.val} \leq 104$

Round 1 · High · Easy

p.330

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isBalanced(self, root: TreeNode | None) -> bool:
        if not root:
            return True

        def maxDepth(root: TreeNode | None) -> int:
            if not root:
                return 0
            return 1 + max(maxDepth(root.left), maxDepth(root.right))

        return (abs(maxDepth(root.left) - maxDepth(root.right)) <= 1 and
                self.isBalanced(root.left) and
                self.isBalanced(root.right))
```

```
class Solution:
    def isBalanced(self, root: TreeNode | None) -> bool:
        ans = True

        def maxDepth(root: TreeNode | None) -> int:
            """Returns the height of root and sets ans to false if root
            unbalanced."""
            nonlocal ans
            if not root or not ans:
                return 0
            left = maxDepth(root.left)

            right = maxDepth(root.right)
            if abs(left - right) > 1:
                ans = False
            return max(left, right) + 1

        maxDepth(root)
        return ans
```

Round 1 · High · Easy

p.331

```
class Solution:
    def isBalanced(self, root: TreeNode | None) -> bool:
        def maxDepth(root: TreeNode | None) -> int:
            """Returns the height of root if root is balanced; otherwise, returns
            -1."""
            if not root:
                return 0

            left = maxDepth(root.left)
            if left == -1:
                return -1

            right = maxDepth(root.right)
            if right == -1:
                return -1
            if abs(left - right) > 1:
                return -1

            return 1 + max(maxDepth(root.left), maxDepth(root.right))

        return maxDepth(root) != -1
```

• LeetDoocs

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isBalanced(self, root: Optional[TreeNode]) -> bool:
        def height(root):
            if root is None:
                return 0
            l, r = height(root.left), height(root.right)
            if l == -1 or r == -1 or abs(l - r) > 1:
                return -1
            return 1 + max(l, r)

        return height(root) >= 0
```

Round 1 · High · Easy

p.332

• SimplyLeet

```
# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):
# self.val = val
# self.left = left
# self.right = right

class Solution:
    def isBalanced(self, root: TreeNode) -> bool:
        # Helper function to return the height of the subtree or -1 if
        # unbalanced.
```

```
    def checkHeight(node: TreeNode) -> int:
        # An empty tree has height 0 and is balanced.
        if not node:
            return 0

        # Recursively get the height of the left subtree.
        leftHeight = checkHeight(node.left)
        # If left subtree is unbalanced, return -1.
        if leftHeight == -1:
            return -1
```

```
        # Recursively get the height of the right subtree.
        rightHeight = checkHeight(node.right)
        # If right subtree is unbalanced, return -1.
        if rightHeight == -1:
            return -1

        # If the current node's subtrees differ in height by more than 1,
        # it's unbalanced.
        if abs(leftHeight - rightHeight) > 1:
            return -1
```

```
        # Return the height of the current node.
        return max(leftHeight, rightHeight) + 1

        # The tree is balanced if checkHeight does not return -1.
        return checkHeight(root) != -1
```

Round 1 · High · Easy

p.333

• LC.ca

```
# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):
# self.val = val
# self.left = left
# self.right = right

class Solution:
    def isBalanced(self, root: Optional[TreeNode]) -> bool:
        def height(root):
            if root is None:
                return 0
            l, r = height(root.left), height(root.right)
            if l == -1 or r == -1 or abs(l - r) > 1:
                return -1
            return 1 + max(l, r)

        return height(root) >= 0
```

◆ Quick Review

Key Insights

- Use a bottom-up DFS (Depth-First Search) traversal to compute the height of each subtree.
- If any subtree is found unbalanced, propagate an error indicator (e.g., -1) upward, allowing early termination.
- An empty tree is considered balanced.
- The key check is to ensure that for each node, the absolute difference between the

Round 1 · High · Easy

p.334

heights of its left and right subtrees is no more than 1.

Space & Time

Time Complexity: $O(n)$ where n is the number of nodes in the tree, since we visit each node once.

Space Complexity: $O(h)$ where h is the height of the tree, which corresponds to the recursion stack ($O(n)$ in the worst-case of a skewed tree).

Solution

The solution uses a recursive DFS approach. A helper function recursively computes the height of a subtree. If a subtree is found to be unbalanced (the height difference between its left and right subtrees exceeds 1), the helper returns a special value (e.g., -1) to indicate that the subtree is unbalanced. This value is then propagated up the recursion to terminate further unnecessary computations. The main function checks the result of the helper function and returns true if the tree is balanced (i.e., the helper did not return -1) and false otherwise.

Round 1 · High · Easy

p.335

Tree Traversal – Post-Order

□ #145 Binary Tree Postorder Traversal

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Stack · Tree · Depth-First Search · Binary Tree
Lists: AlgoMaster 600

Problem

Given the root of a binary tree, return the postorder traversal of its nodes' values.

Example 1:

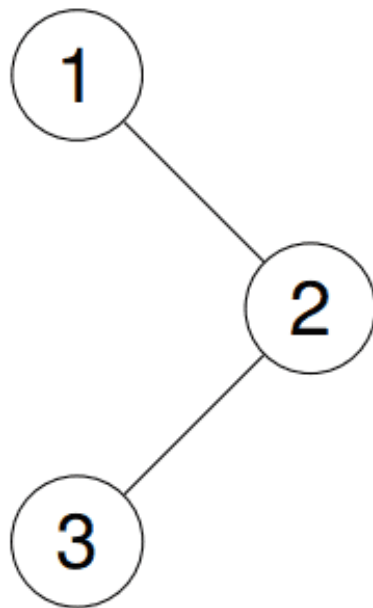
Input: root = [1,null,2,3]

Output: [3,2,1]

Explanation:

Round 1 · High · Easy

p.336



Example 2:

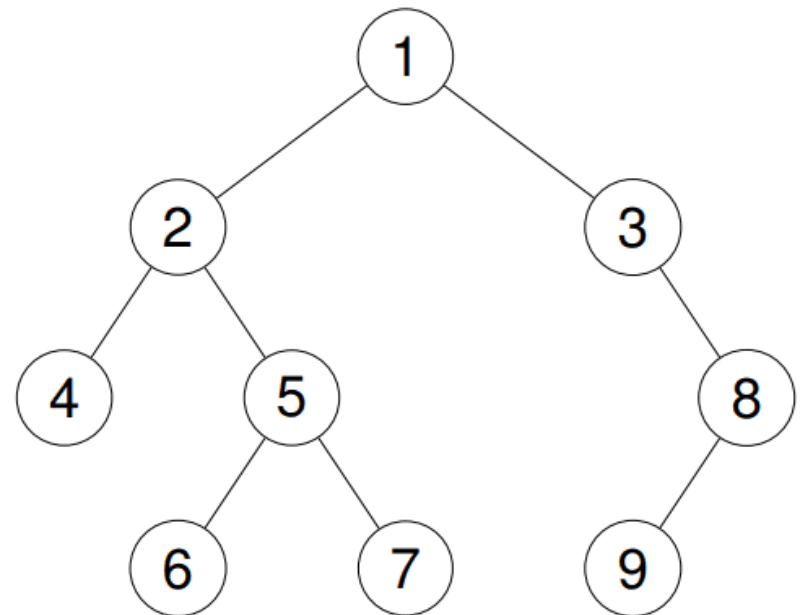
Input: root = [1,2,3,4,5,null,8,null,null,6,7,9]

Output: [4,6,7,5,2,9,8,3,1]

Explanation:

Round 1 · High · Easy

p.337



Example 3:

Input: root = []

Output: []

Example 4:

Input: root = [1]

Output: [1]

Constraints:

- The number of the nodes in the tree is in the range [0, 100].

Round 1 · High · Easy

p.338

- $-100 \leq \text{Node.val} \leq 100$

Follow up: Recursive solution is trivial, could you do it iteratively?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def postorderTraversal(self, root: TreeNode | None) -> list[int]:
        ans = []

        def postorder(root: TreeNode | None) -> None:
            if not root:
                return

            postorder(root.left)
            postorder(root.right)

            ans.append(root.val)

        postorder(root)
        return ans
```

```
class Solution:
    def postorderTraversal(self, root: TreeNode | None) -> list[int]:
        if not root:
            return []

        ans = []
        stack = [root]

        while stack:
            node = stack.pop()
```

```
ans.append(node.val)
if node.left:
    stack.append(node.left)
if node.right:
    stack.append(node.right)

return ans[::-1]
```

• LeetCode

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def postorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
        def dfs(root):
            if root is None:
                return

            dfs(root.left)
            dfs(root.right)
            ans.append(root.val)

        ans = []
        dfs(root)
        return ans
```

• SimplyLeet

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def postorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
        # If the tree is empty, return an empty list.
```

```

if not root:
    return []

# Stack for processing nodes.
stack1 = [root]
# Stack to collect nodes in reverse postorder.
stack2 = []
# Loop until all nodes are processed.
while stack1:
    # Pop a node from the first stack.

    node = stack1.pop()
    # Push the node onto the second stack.
    stack2.append(node)
    # Push the left child first (if exists).
    if node.left:
        stack1.append(node.left)
    # Push the right child (if exists).
    if node.right:
        stack1.append(node.right)

# The second stack now contains nodes in reverse of postorder.
# Extract the node values in proper postorder.
result = []
while stack2:
    result.append(stack2.pop().val)
return result

```

Tree Traversal – Post-Order

□ #226 Invert Binary Tree

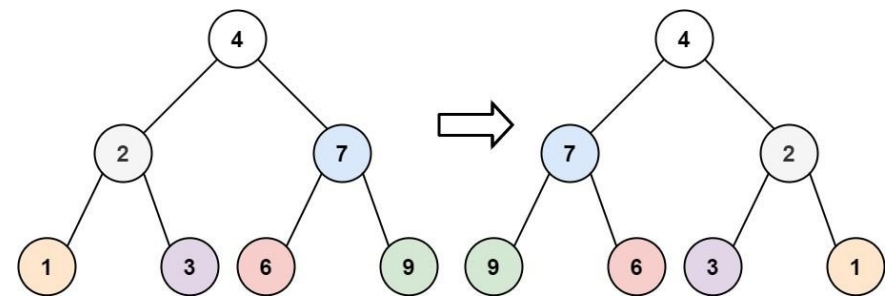
EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Tree · DFS · BFS
 Lists: Grind 169

Problem

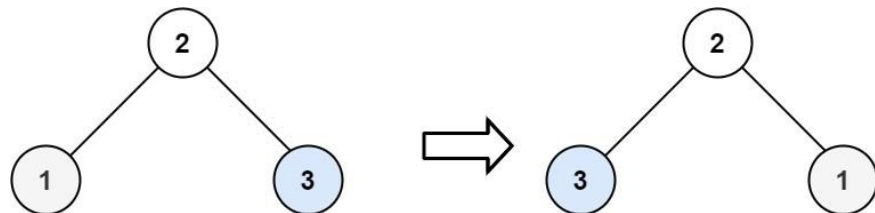
Given the root of a binary tree, invert the tree, and return its root.

Example 1:



Input: root = [4,2,7,1,3,6,9]
 Output: [4,7,2,9,6,3,1]

Example 2:



Input: root = [2,1,3]
Output: [2,3,1]

Example 3:

Input: root = []
Output: []

Constraints:

- The number of nodes in the tree is in the range [0, 100].
- $-100 \leq \text{Node.val} \leq 100$

Community Solutions (Python)

• WalkCC

```

class Solution:
    def invertTree(self, root: TreeNode | None) -> TreeNode | None:
        if not root:
            return None

        left = root.left
        right = root.right
        root.left = self.invertTree(right)
        root.right = self.invertTree(left)
        return root
  
```

• LeetCode

Round 1 · High · Easy

p.343

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def invertTree(self, root: Optional[TreeNode]) -> Optional[TreeNode]:
        if root is None:
            return None

        l, r = self.invertTree(root.left), self.invertTree(root.right)
        root.left, root.right = r, l
        return root
  
```

• SimplyLeet

```

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val # Initialize node value
        self.left = left # Pointer to left child
        self.right = right # Pointer to right child

class Solution:
    def invertTree(self, root: TreeNode) -> TreeNode:
        # Base case: if the node is None, return None.

        if not root:
            return None
        # Recursively invert the left and right subtrees.
        left_inverted = self.invertTree(root.left)
        right_inverted = self.invertTree(root.right)
        # Swap the left and right children.
        root.left, root.right = right_inverted, left_inverted
        return root
  
```

• LC.ca

Round 1 · High · Easy

p.344


```
# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):
# self.val = val
# self.left = left
# self.right = right
class Solution:
    def invertTree(self, root: Optional[TreeNode]) -> Optional[TreeNode]:
        def dfs(root):
            if root is None:
                return
            root.left, root.right = root.right, root.left
            dfs(root.left)
            dfs(root.right)
        dfs(root)
        return root
```

◆ Quick Review

Key Insights

- The problem can be solved using recursion (depth-first search) by swapping the children of every node.
- Alternatively, an iterative approach (using a queue for breadth-first search) can be used.
- The recursion terminates when a null node is encountered.
- The operation is performed in-place, modifying the original tree structure.

Space & Time

Round 1 · High · Easy

p.345

Time Complexity: $O(n)$ where n is the number of nodes, since each node is visited once.

Space Complexity: $O(h)$ where h is the height of the tree, which is the recursion stack space in the recursive approach. In worst-case (skewed tree), this could be $O(n)$.

Solution

We solve the problem using recursion (depth-first search). The approach is to:

- Check the base case: if the node is null, simply return null.
- Recursively invert the left subtree and the right subtree.
- Swap the left and right children of the current node.
- Return the current node after the swap. The recursive approach elegantly handles all nodes and ensures the inversion process covers the complete tree.

Round 1 · High · Easy

p.346

Tree Traversal – Post-Order

□ #543 Diameter of Binary Tree

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS

Lists: Grind 169

Problem

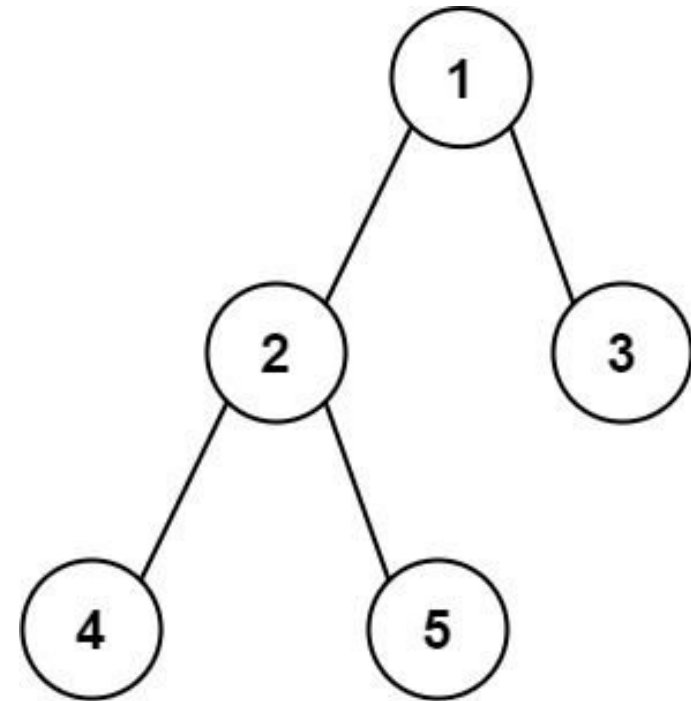
Given the root of a binary tree, return the length of the diameter of the tree.

The diameter of a binary tree is the length of the longest path between any two nodes in a tree.

This path may or may not pass through the root.

The length of a path between two nodes is represented by the number of edges between them.

Example 1:



Input: root = [1,2,3,4,5]

Output: 3

Explanation: 3 is the length of the path [4,2,1,3] or [5,2,1,3].

Example 2:

Input: root = [1,2]

Output: 1

Constraints:

- The number of nodes in the tree is in the range [1, 104].
- $-100 \leq \text{Node.val} \leq 100$

Round 1 · High · Easy

p.347

Round 1 · High · Easy

p.348

Community Solutions (Python)

• WalkCC

```
class Solution:
    def diameterOfBinaryTree(self, root: TreeNode | None) -> int:
        ans = 0

        def maxDepth(root: TreeNode | None) -> int:
            nonlocal ans
            if not root:
                return 0

            l = maxDepth(root.left)

            r = maxDepth(root.right)
            ans = max(ans, l + r)
            return 1 + max(l, r)

        maxDepth(root)
        return ans
```

• LeetDoocs

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def diameterOfBinaryTree(self, root: TreeNode) -> int:
        def dfs(root):
            if root is None:
```

```
        return 0
        nonlocal ans
        left, right = dfs(root.left), dfs(root.right)
        ans = max(ans, left + right)
        return 1 + max(left, right)

        ans = 0
        dfs(root)
        return ans
```

• SimplyLeet

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def diameterOfBinaryTree(self, root: TreeNode) -> int:
        # Global variable to track the maximum diameter found
```

```
        self.max_diameter = 0
```

```
    def dfs(node):
        # Base case: if node is None, depth is 0.
        if not node:
            return 0
        # Compute the depth of left and right subtrees.
        left_depth = dfs(node.left)
        right_depth = dfs(node.right)
        # Update the global diameter: the path through this node
```

```
        self.max_diameter = max(self.max_diameter, left_depth + right_depth)

    # Return the depth of the tree rooted at this node.
    return max(left_depth, right_depth) + 1

    dfs(root)
    return self.max_diameter
```

• LC.ca

```
# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):
# self.val = val
# self.left = left
# self.right = right
class Solution:
    def diameterOfBinaryTree(self, root: TreeNode) -> int:
        def dfs(root):
            if root is None:
                return 0
            nonlocal ans
            left, right = dfs(root.left), dfs(root.right)
            ans = max(ans, left + right)
            return 1 + max(left, right)

        ans = 0
        dfs(root)
        return ans
```

◆ Quick Review

Key Insights

- Use depth-first search (DFS) to explore the tree.
- The diameter at a node is the sum of the depths of its left and right subtrees.
- Track the maximum diameter found during the DFS traversal.
- Recursively calculating the depth of subtrees provides an efficient solution.

Round 1 · High · Easy

p.351

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes in the tree.

Space Complexity: $O(n)$, in the worst-case scenario due to the recursion stack (skewed tree).

Solution

The solution employs a DFS traversal of the binary tree. At every node in the tree, compute the depth of the left and right subtrees. The sum of these depths gives the potential diameter (longest path through that node). Update a global maximum diameter variable if this sum exceeds the current maximum. Finally, return the maximum diameter after completing the DFS.

Round 1 · High · Easy

p.352

Depth First Search (DFS)

Round 1 · High · Easy · 1 question

Round 1 · High · Easy

p.353

Depth First Search (DFS)

□ #733 Flood Fill

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · DFS · BFS · Matrix

Lists: Grind 169

Problem

You are given an image represented by an $m \times n$ grid of integers `image`, where `image[i][j]` represents the pixel value of the image. You are also given three integers `sr`, `sc`, and `color`. Your task is to perform a flood fill on the image starting from the pixel `image[sr][sc]`.

To perform a flood fill:

1. Begin with the starting pixel and change its color to `color`.
2. Perform the same process for each pixel that is directly adjacent (pixels that share a side with the original pixel, either horizontally or vertically) and shares the same color as the starting pixel.

Round 1 · High · Easy

p.354

3. Keep repeating this process by checking neighboring pixels of the updated pixels and modifying their color if it matches the original color of the starting pixel.

4. The process stops when there are no more adjacent pixels of the original color to update.

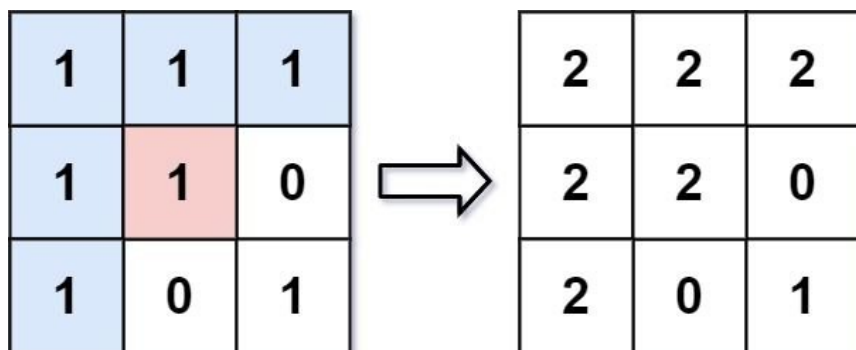
Return the modified image after performing the flood fill.

Example 1:

Input: image = [[1,1,1],[1,1,0],[1,0,1]], sr = 1, sc = 1, color = 2

Output: [[2,2,2],[2,2,0],[2,0,1]]

Explanation:



From the center of the image with position (sr, sc) = (1, 1) (i.e., the red pixel), all pixels connected by a path of the same color as the

starting pixel (i.e., the blue pixels) are colored with the new color.

Note the bottom corner is not colored 2, because it is not horizontally or vertically connected to the starting pixel.

Example 2:

Input: image = [[0,0,0],[0,0,0]], sr = 0, sc = 0, color = 0

Output: [[0,0,0],[0,0,0]]

Explanation:

The starting pixel is already colored with 0, which is the same as the target color. Therefore, no changes are made to the image.

Constraints:

- $m == \text{image.length}$
- $n == \text{image}[i].\text{length}$
- $1 \leq m, n \leq 50$
- $0 \leq \text{image}[i][j], \text{color} < 216$
- $0 \leq \text{sr} < m$
- $0 \leq \text{sc} < n$

Community Solutions (Python)

- WalkCC

```
class Solution:
    def floodFill(self, image: List[List[int]],
                  sr: int, sc: int, newColor: int) -> List[List[int]]:
        startColor = image[sr][sc]
        seen = set()

        def dfs(i: int, j: int) -> None:
            if i < 0 or i == len(image) or j < 0 or j == len(image[0]):
                return
            if image[i][j] != startColor or (i, j) in seen:
                return

            image[i][j] = newColor
            seen.add((i, j))

            dfs(i + 1, j)
            dfs(i - 1, j)
            dfs(i, j + 1)
            dfs(i, j - 1)

        dfs(sr, sc)
        return image
```

• LeetCode

```
class Solution:
    def floodFill(
        self, image: List[List[int]], sr: int, sc: int, color: int
    ) -> List[List[int]]:
        def dfs(i: int, j: int):
            image[i][j] = color
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                if 0 <= x < len(image) and 0 <= y < len(image[0]) and image[x][y] == oc:
                    dfs(x, y)
```

Round 1 · High · Easy

p.357

```
oc = image[sr][sc]
if oc != color:
    dirs = (-1, 0, 1, 0, -1)
    dfs(sr, sc)
return image
```

• SimplyLeet

```
def floodFill(image, sr, sc, color):
    # Get the original color of the starting pixel
    orig_color = image[sr][sc]
    # If the new color is the same as the original, return the image as is
    if orig_color == color:
        return image
    rows, cols = len(image), len(image[0])

    # Recursive DFS helper function to fill the image
    def dfs(r, c):

        # Check boundaries and if the current pixel is the same as the original
        color
        if r < 0 or r >= rows or c < 0 or c >= cols or image[r][c] != orig_color:
            return
        # Update the current pixel's color
        image[r][c] = color
        # Recursively call DFS in all four directions
        dfs(r - 1, c) # up
        dfs(r + 1, c) # down
        dfs(r, c - 1) # left
        dfs(r, c + 1) # right

    dfs(sr, sc)
    return image
```

• LC.ca

Round 1 · High · Easy

p.358

```

class Solution:
    def floodFill(
        self, image: List[List[int]], sr: int, sc: int, color: int
    ) -> List[List[int]]:
        def dfs(i, j):
            if (
                not 0 <= i < m
                or not 0 <= j < n
                or image[i][j] != oc
                or image[i][j] == color
            ):
                return
            image[i][j] = color
            for a, b in pairwise(dirs):
                dfs(i + a, j + b)

        dirs = (-1, 0, 1, 0, -1)
        m, n = len(image), len(image[0])
        oc = image[sr][sc]
        dfs(sr, sc)

        return image

```

◆ Quick Review

Key Insights

- The flood fill operation can be executed using either DFS (Depth First Search) or BFS (Breadth First Search).
- A check is needed at the beginning to determine if the new color is the same as the original; if so, no changes are needed.
- The algorithm recursively or iteratively visits each pixel that is directly adjacent and

Round 1 · High · Easy

p.359

matches the original color, then updates its color.

- The problem has a worst-case scenario where all pixels are the same color, leading to a full traversal of the grid.

Space & Time

Time Complexity: $O(m * n)$, where m and n are the dimensions of the image grid, because in the worst-case all cells are visited.

Space Complexity: $O(m * n)$ in the worst-case due to the recursion stack (or BFS queue) if the entire image needs to be filled.

Solution

We use a DFS approach to perform the flood fill. The algorithm starts at the given starting pixel and uses recursion to fill connected pixels with the same original color. The following steps are key:

- Check if the starting pixel's color is already the target color; if so, return the image immediately.
- Define a recursive helper function that: Checks boundary conditions. Updates the current pixel's color. Recursively calls itself on the four adjacent directions (up,

Round 1 · High · Easy

p.360

down, left, right). – Return the modified image after the recursion completes.

Data structures used:

– Recursion call stack for the DFS implementation. – Variables to store the image dimensions and original color.

Round 1 · High · Easy

p.361

Linked List

Round 1 · High · Easy · 2 questions

Round 1 · High · Easy

p.362

Linked List

□ #83 Remove Duplicates from Sorted List **EAS**

LeetDoocs · SimplyLeet · WalkCC · LeetCode

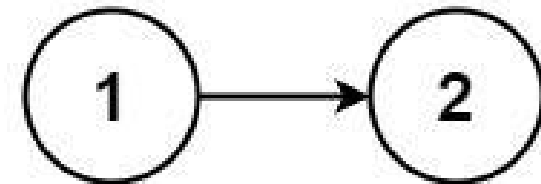
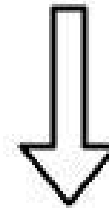
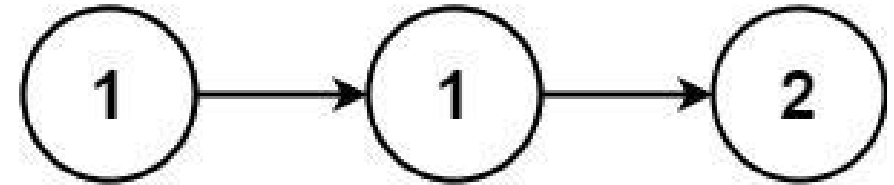
Linked List

Lists: AlgoMaster 600

Problem

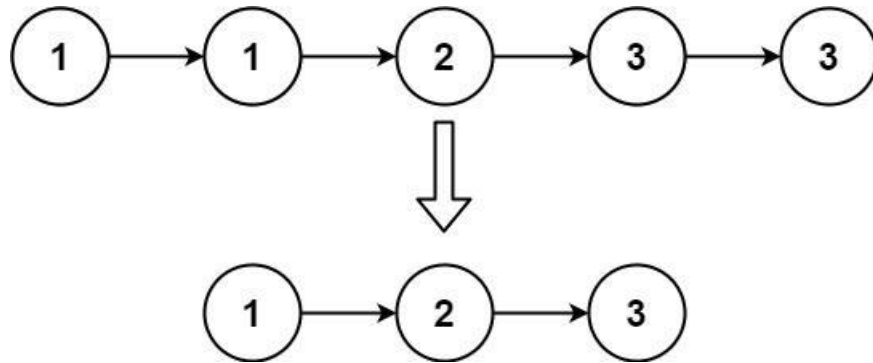
Given the head of a sorted linked list, delete all duplicates such that each element appears only once. Return the linked list sorted as well.

Example 1:



Input: head = [1,1,2]
Output: [1,2]

Example 2:



Input: head = [1,1,2,3,3]
Output: [1,2,3]

Constraints:

- The number of nodes in the list is in the range [0, 300].
- $-100 \leq \text{Node.val} \leq 100$
- The list is guaranteed to be sorted in ascending order.

Community Solutions (Python)

• WalkCC

```

class Solution:
    def deleteDuplicates(self, head: ListNode) -> ListNode:
        curr = head

        while curr:
            while curr.next and curr.val == curr.next.val:
                curr.next = curr.next.next
            curr = curr.next

        return head
  
```

Round 1 · High · Easy

p.365

• LeetCode

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def deleteDuplicates(self, head: Optional[ListNode]) -> Optional[ListNode]:
        cur = head
        while cur and cur.next:
            if cur.val == cur.next.val:
                cur.next = cur.next.next
            else:
                cur = cur.next
        return head
  
```

• SimplyLeet

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def deleteDuplicates(self, head: ListNode) -> ListNode:
        # Initialize current pointer to head of the list
        current = head

        # Traverse until we reach the end of the list
        while current and current.next:
            # If duplicate is found, skip the next node
            if current.val == current.next.val:
                current.next = current.next.next # Remove duplicate by
                linking to the next distinct node
            else:
                current = current.next # Move to next node if current and next
                values are distinct
        return head
  
```

Round 1 · High · Easy

p.366

Linked List

#160 Intersection of Two Linked Lists

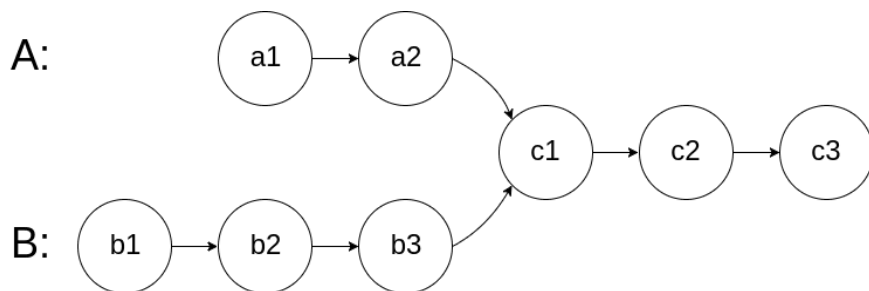
EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Linked List · Two Pointers
Lists: AlgoMaster 600

Problem

Given the heads of two singly linked-lists headA and headB, return the node at which the two lists intersect. If the two linked lists have no intersection at all, return null.

For example, the following two linked lists begin to intersect at node c1:



The test cases are generated such that there are no cycles anywhere in the entire linked structure.

Round 1 · High · Easy

p.367

Note that the linked lists must retain their original structure after the function returns.

Custom Judge:

The inputs to the judge are given as follows (your program is not given these inputs):

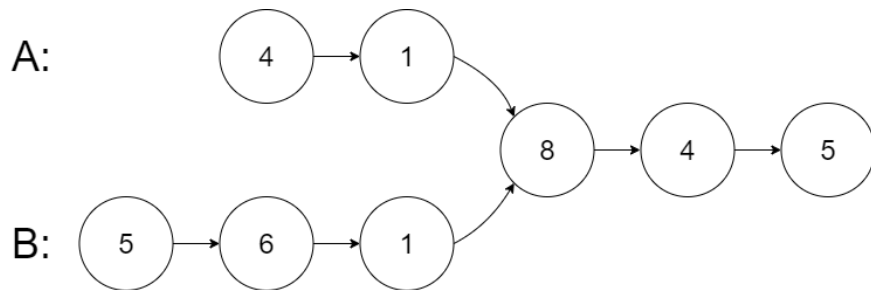
- intersectVal – The value of the node where the intersection occurs. This is 0 if there is no intersected node.
- listA – The first linked list.
- listB – The second linked list.
- skipA – The number of nodes to skip ahead in listA (starting from the head) to get to the intersected node.
- skipB – The number of nodes to skip ahead in listB (starting from the head) to get to the intersected node.

The judge will then create the linked structure based on these inputs and pass the two heads, headA and headB to your program. If you correctly return the intersected node, then your solution will be accepted.

Example 1:

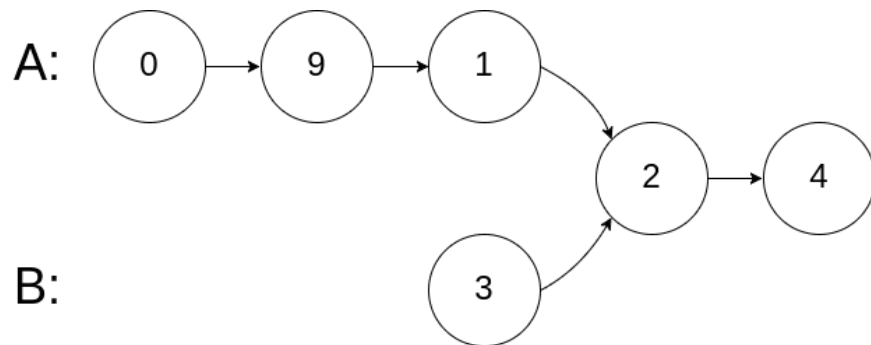
Round 1 · High · Easy

p.368



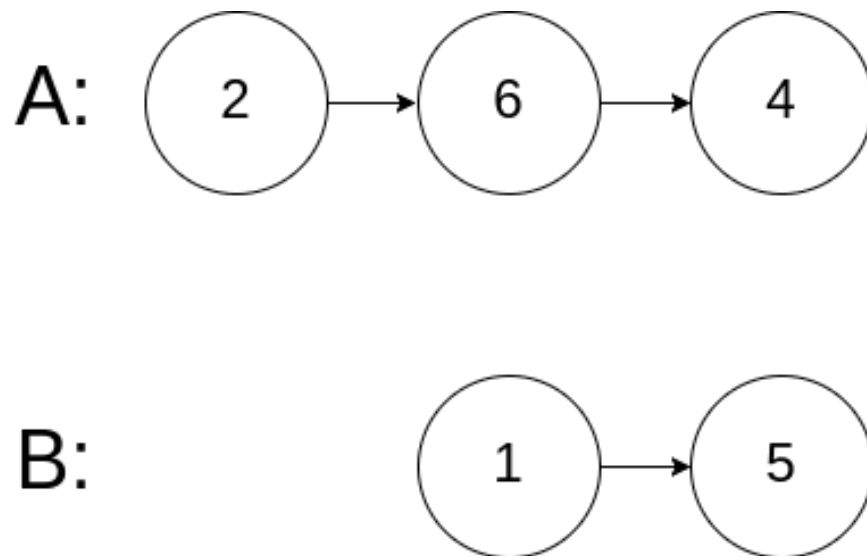
Input: intersectVal = 8, listA = [4,1,8,4,5], listB = [5,6,1,8,4,5], skipA = 2, skipB = 3
 Output: Intersected at '8'
 Explanation: The intersected node's value is 8 (note that this must not be 0 if the two lists intersect).
 From the head of A, it reads as [4,1,8,4,5]. From the head of B, it reads as [5,6,1,8,4,5]. There are 2 nodes before the intersected node in A; There are 3 nodes before the intersected node in B.
 - Note that the intersected node's value is not 1 because the nodes with value 1 in A and B (2nd node in A and 3rd node in B) are different node references. In other words, they point to two different locations in memory, while the nodes with value 8 in A and B (3rd node in A and 4th node in B) point to the same location in memory.

Example 2:



Input: intersectVal = 2, listA = [1,9,1,2,4], listB = [3,2,4], skipA = 3, skipB = 1
 Output: Intersected at '2'
 Explanation: The intersected node's value is 2 (note that this must not be 0 if the two lists intersect).
 From the head of A, it reads as [1,9,1,2,4]. From the head of B, it reads as [3,2,4]. There are 3 nodes before the intersected node in A; There are 1 node before the intersected node in B.

Example 3:



Input: intersectVal = 0, listA = [2,6,4], listB = [1,5], skipA = 3, skipB = 2
 Output: No intersection
 Explanation: From the head of A, it reads as [2,6,4]. From the head of B, it reads as [1,5]. Since the two lists do not intersect, intersectVal must be 0, while skipA and skipB can be arbitrary values.
 Explanation: The two lists do not intersect, so return null.

Constraints:

- The number of nodes of listA is in the m.
- The number of nodes of listB is in the n.

- $1 \leq m, n \leq 3 * 10^4$
- $1 \leq \text{Node.val} \leq 10^5$
- $0 \leq \text{skipA} \leq m$
- $0 \leq \text{skipB} \leq n$
- `intersectVal` is 0 if listA and listB do not intersect.
- `intersectVal == listA[skipA] == listB[skipB]` if listA and listB intersect.

Follow up: Could you write a solution that runs in $O(m + n)$ time and use only $O(1)$ memory?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def getIntersectionNode(
        self,
        headA: ListNode,
        headB: ListNode,
    ) -> ListNode | None:
        a = headA
        b = headB

        while a != b:

            a = a.next if a else headB
            b = b.next if b else headA

        return a
```

• LeetCode

Round 1 · High · Easy

p.371

• SimplyLeet

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, x):
#         self.val = x
#         self.next = None

class Solution:
    def getIntersectionNode(self, headA: ListNode, headB: ListNode) -> ListNod
e:
        a, b = headA, headB

        while a != b:
            a = a.next if a else headB
            b = b.next if b else headA
        return a
```

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, x):
#         self.val = x
#         self.next = None

class Solution:
    def getIntersectionNode(self, headA: ListNode, headB: ListNode) -> ListNod
e:
        # If either list is empty, there is no intersection.
        if not headA or not headB:

            return None

        ptrA = headA # Pointer for list A
        ptrB = headB # Pointer for list B

        # Loop until the two pointers are the same.
        while ptrA != ptrB:
            # If pointer A reaches the end, switch to headB; otherwise move to
            the next node.
            ptrA = headB if ptrA is None else ptrA.next
            # If pointer B reaches the end, switch to headA; otherwise move to
            the next node.
```

Round 1 · High · Easy

p.372

```
ptrB = headA if ptrB is None else ptrB.next

# Either both pointers point to the intersection node or are both None.
return ptrA
```

Round 1 · High · Easy

p.373

Round 2 · High · Medium

Round 2

High Priority · Medium

136 questions · 15 patterns

Arrays #31 #80 #122 #189 #229 #238 #334
#2348

Strings #6 #38 #151 #161 #616 #1657

Hash Tables #49 #128 #249 #535 #659 #767
#792 #1525 #1647

Two Pointers #5 #11 #15 #18 #167 #443 #861

Sliding Window – Fixed Size #438 #567 #1456
#2461

Sliding Window – Dynamic Size #3 #159 #209
#340 #395 #424 #713 #904 #1004 #1248
#1423 #1838

Binary Search #33 #34 #74 #81 #153 #162 #240
#475 #528 #658 #875 #1011 #1482 #1552

Round 1 · High · Easy

p.374

**Stacks #71 #150 #155 #227 #316 #636 #678
#735 #853 #946 #1249 #2390**

**Tree Traversal – Level Order #102 #103 #116
#117 #199 #314 #662 #958**

**Tree Traversal – Pre Order #105 #106 #113
#437 #687 #1026**

Tree Traversal – In Order #98 #173 #230 #1382

**Tree Traversal – Post-Order #114 #129 #236
#545 #652 #979 #1110 #1448 #2096**

**Depth First Search (DFS) #79 #130 #133 #200
#417 #690 #694 #785 #797 #863 #1020 #1376**

**Breadth First Search (BFS) #286 #433 #542 #752
#909 #994 #1091 #1102 #1197**

**Linked List #2 #19 #24 #61 #82 #86 #138 #143
#237 #328 #430 #445 #707 #708 #1669 #2095**

Round 2 · High · Medium

p.375

Arrays

Round 2 · High · Medium · 8 questions

Round 2 · High · Medium

p.376

Arrays

#31 Next Permutation

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Two Pointers
 Lists: Grind 169

Problem

A permutation of an array of integers is an arrangement of its members into a sequence or linear order.

- For example, for $arr = [1,2,3]$, the following are all the permutations of arr : $[1,2,3]$, $[1,3,2]$, $[2, 1, 3]$, $[2, 3, 1]$, $[3,1,2]$, $[3,2,1]$.

The next permutation of an array of integers is the next lexicographically greater permutation of its integer. More formally, if all the permutations of the array are sorted in one container according to their lexicographical order, then the next permutation of that array is the permutation that follows it in the sorted container. If such arrangement is not possible, the array must be rearranged as the lowest possible order (i.e., sorted in ascending order).

Round 2 · High · Medium

p.377

- For example, the next permutation of $arr = [1,2,3]$ is $[1,3,2]$.
- Similarly, the next permutation of $arr = [2,3,1]$ is $[3,1,2]$.
- While the next permutation of $arr = [3,2,1]$ is $[1,2,3]$ because $[3,2,1]$ does not have a lexicographical larger rearrangement.

Given an array of integers $nums$, find the next permutation of $nums$.

The replacement must be in place and use only constant extra memory.

Example 1:

Input: $nums = [1,2,3]$
 Output: $[1,3,2]$

Example 2:

Input: $nums = [3,2,1]$
 Output: $[1,2,3]$

Example 3:

Input: $nums = [1,1,5]$
 Output: $[1,5,1]$

Constraints:

- $1 \leq nums.length \leq 100$
- $0 \leq nums[i] \leq 100$

Community Solutions (Python)

Round 2 · High · Medium

p.378

• WalkCC

```
class Solution:
    def nextPermutation(self, nums: list[int]) -> None:
        n = len(nums)

        # From back to front, find the first number < nums[i + 1].
        i = n - 2
        while i >= 0:
            if nums[i] < nums[i + 1]:
                break
            i -= 1

        # From back to front, find the first number > nums[i], swap it with
        # nums[i].
        if i >= 0:
            for j in range(n - 1, i, -1):
                if nums[j] > nums[i]:
                    nums[i], nums[j] = nums[j], nums[i]
                    break

        def reverse(nums: list[int], l: int, r: int) -> None:
            while l < r:
                nums[l], nums[r] = nums[r], nums[l]
                l += 1
                r -= 1

        # Reverse nums[i + 1..n - 1].
        reverse(nums, i + 1, len(nums) - 1)
```

• LeetCode

```
class Solution:
    def nextPermutation(self, nums: List[int]) -> None:
        n = len(nums)
        i = next((i for i in range(n - 2, -1, -1) if nums[i] < nums[i + 1]), -1)

        if ~i:
            j = next((j for j in range(n - 1, i, -1) if nums[j] > nums[i]))
            nums[i], nums[j] = nums[j], nums[i]
            nums[i + 1:] = nums[i + 1:][::-1]
```

Round 2 · High · Medium

p.379

• SimplyLeet

```
def nextPermutation(nums):
    # Length of the array
    n = len(nums)
    # Find the first index from the right where the current number is less than
    # the next number
    i = n - 2
    while i >= 0 and nums[i] >= nums[i + 1]:
        i -= 1
    # If such an index is found, find the number just larger than nums[i] from
    # the right side to swap
    if i >= 0:
        j = n - 1
        while nums[j] <= nums[i]:
            j -= 1
        nums[i], nums[j] = nums[j], nums[i] # Swap the two numbers
    # Reverse the subarray from i+1 to the end to get the smallest sequence
    left, right = i + 1, n - 1
    while left < right:
        nums[left], nums[right] = nums[right], nums[left]
        left += 1
        right -= 1

    # Example usage:
    nums = [1, 2, 3]
    nextPermutation(nums)
    print(nums) # Output: [1, 3, 2]
```

• LC.ca

```
'''
>>> i = 3
>>> ~i
-4
>>> bool(~i)
True
#####

>>> j = -1
>>> ~j
```

Round 2 · High · Medium

p.380

```

0
>>> bool(~j)
False
#####

>>> a = (i for i in range(10, -1, -1) if i < 6)
>>> a
<generator object <genexpr> at 0x10a17eeb0>
>>> next(a)
5

```

```

>>>
>>>
>>> b = (i for i in range(10, -1, -1) if i < 0)
>>> next(b)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
StopIteration
>>> next(b, -1)
-1
>>>

```

```

'''
class Solution:
    def nextPermutation(self, nums: List[int]) -> None:
        """
        Do not return anything, modify nums in-place instead.
        """
        n = len(nums)
        # next(func, -1) => default value -1
        i = next((i for i in range(n - 2, -1, -1) if nums[i] < nums[i + 1]), -1)
        if i != -1:

```

Round 2 · High · Medium

p.381

```

        j = next((j for j in range(n - 1, i, -1) if nums[j] > nums[i]))
        nums[i], nums[j] = nums[j], nums[i]
        nums[i + 1:] = nums[i + 1:][::-1]
        # wrong reverse: nums[i + 1:] = nums[i + 1::-1]

```

```

#####

'''
>>> a=[1,2,3]
>>> reversed(a)

```

```

<list_reverseiterator object at 0x108458be0>
>>> list(reversed(a))
[3, 2, 1]

# but, use reversed(a) to directly assign values is ok
>>> b=[4,5,6,7,8]
>>> b[:3] = reversed(a)
>>> b
[3, 2, 1, 7, 8]
'''

```

```

class Solution:
    def nextPermutation(self, nums: List[int]) -> None:
        if not nums:
            return

        # #####next
        for i in range(len(nums)-2, -1, -1): # 3, 4, 5, 2, 1
            if nums[i] < nums[i+1]: # ##### => 4
                j = next(j for j in range(len(nums)-1, i, -1) if nums[j] > num
s[i]) # #####nums-i#### => 5
                nums[i], nums[j] = nums[j], nums[i] # 3,5,4,2,1

```

Round 2 · High · Medium

p.382

```

# reverse []
# []nums-i[]insert position
nums[i+1:] = reversed(nums[i+1:]) # [4,2,1] reverse to
[1,2,4] => 3, 5, 1, 2, 4
return

nums.reverse() # for[]return[]

#####

class Solution:
    def nextPermutation(self, nums: List[int]) -> None:
        n = len(nums)
        to_left = [i for i in range(n - 2, -1, -1) if nums[i] < nums[i + 1]]

        if not to_left:
            nums = nums[::-1]
            return

        i = max(to_left)
        if i >= 0:
            to_right = [j for j in range(n - 1, i, -1) if nums[j] > nums[i]]
            j = to_right[0]
            nums[i], nums[j] = nums[j], nums[i]
            nums[i + 1:] = nums[i + 1::-1]

```

◆ Quick Review

Key Insights

- The problem is solved by identifying the first pair from the right where the left element is less than the right element.

- Once this pivot is found, find the rightmost element that is greater than this pivot.
- Swap these two elements.
- Reverse the subarray that comes after the pivot to ensure the next permutation is the next lexicographically larger sequence.
- If no such pivot exists, the array is in descending order and should be reversed to get the smallest permutation.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Solution

We start by scanning the array from the right to find the first index (i) such that $\text{nums}[i]$ is less than $\text{nums}[i+1]$. This index 'i' represents the pivot point where the next permutation can be formed. If we find such an index, we then scan from the right again to locate the first number that is greater than $\text{nums}[i]$ (let this index be j), and swap these two numbers. Finally, we reverse the subarray starting from index $i+1$ to the end of the array, which gives us the next permutation in lexicographical

order. This approach leverages in-place modifications using basic operations like swapping and reversing, thus meeting the constant space requirement.

Round 2 · High · Medium

p.385

Arrays

☐ #80 Remove Duplicates from Sorted Array II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers
Lists: AlgoMaster 600

Problem

Given an integer array `nums` sorted in non-decreasing order, remove some duplicates in-place such that each unique element appears at most twice. The relative order of the elements should be kept the same.

Since it is impossible to change the length of the array in some languages, you must instead have the result be placed in the first part of the array `nums`. More formally, if there are `k` elements after removing the duplicates, then the first `k` elements of `nums` should hold the final result. It does not matter what you leave beyond the first `k` elements.

Return `k` after placing the final result in the first `k` slots of `nums`.

Round 2 · High · Medium

p.386

Do not allocate extra space for another array. You must do this by modifying the input array in-place with $O(1)$ extra memory.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int[] expectedNums = [...]; // The expected answer with correct length

int k = removeDuplicates(nums); // Calls your implementation

assert k == expectedNums.length;
for (int i = 0; i < k; i++) {
    assert nums[i] == expectedNums[i];
}
```

If all assertions pass, then your solution will be accepted.

Example 1:

```
Input: nums = [1,1,1,2,2,3]
Output: 5, nums = [1,1,2,2,3,_]
Explanation: Your function should return k = 5, with the first five elements of
nums being 1, 1, 2, 2 and 3 respectively.
It does not matter what you leave beyond the returned k (hence they are
underscores).
```

Example 2:

```
Input: nums = [0,0,1,1,1,2,3,3]
Output: 7, nums = [0,0,1,1,2,3,3,_]
Explanation: Your function should return k = 7, with the first seven elements
of nums being 0, 0, 1, 1, 2, 3 and 3 respectively.
It does not matter what you leave beyond the returned k (hence they are
underscores).
```

Constraints:

- $1 \leq \text{nums.length} \leq 3 * 104$

- $-104 \leq \text{nums}[i] \leq 104$
- nums is sorted in non-decreasing order.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def removeDuplicates(self, nums: List[int]) -> int:
        i = 0

        for num in nums:
            if i < 2 or num != nums[i - 2]:
                nums[i] = num
                i += 1

        return i
```

• LeetDoocs

```
class Solution:
    def removeDuplicates(self, nums: List[int]) -> int:
        k = 0
        for x in nums:
            if k < 2 or x != nums[k - 2]:
                nums[k] = x
                k += 1
        return k
```

• SimplyLeet

```
# Python solution for removing duplicates allowing at most two occurrences.
def removeDuplicates(nums):
    # slow pointer for the placement of the element
    slow = 0
    # iterate through each element using fast pointer
    for num in nums:
        # if less than two elements have been placed or the current number
        # is greater than the element at slow-2 (which ensures it is allowed)
        if slow < 2 or num > nums[slow - 2]:
            nums[slow] = num # place the current number at the slow pointer
            slow += 1 # increment the slow pointer
    return slow # slow is the length of the valid array

# Example usage:
# nums = [1,1,1,2,2,3]
# new_length = removeDuplicates(nums)
# print(new_length, nums[:new_length])
```

Arrays

□ #122 Best Time to Buy and Sell Stock II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Dynamic Programming · Greedy
 Lists: AlgoMaster 600

Problem

You are given an integer array prices where prices[i] is the price of a given stock on the ith day.

On each day, you may decide to buy and/or sell the stock. You can only hold at most one share of the stock at any time. However, you can sell and buy the stock multiple times on the same day, ensuring you never hold more than one share of the stock.

Find and return the maximum profit you can achieve.

Example 1:

Input: prices = [7,1,5,3,6,4]
 Output: 7
 Explanation: Buy on day 2 (price = 1) and sell on day 3 (price = 5), profit = 5-1 = 4.
 Then buy on day 4 (price = 3) and sell on day 5 (price = 6), profit = 6-3 = 3.
 Total profit is 4 + 3 = 7.

Example 2:

Input: prices = [1,2,3,4,5]
 Output: 4
 Explanation: Buy on day 1 (price = 1) and sell on day 5 (price = 5), profit = 5-1 = 4.
 Total profit is 4.

Example 3:

Input: prices = [7,6,4,3,1]
 Output: 0
 Explanation: There is no way to make a positive profit, so we never buy the stock to achieve the maximum profit of 0.

Constraints:

- $1 \leq \text{prices.length} \leq 3 \times 10^4$
- $0 \leq \text{prices}[i] \leq 10^4$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        sell = 0
        hold = -math.inf

        for price in prices:
            sell = max(sell, hold + price)
            hold = max(hold, sell - price)

        return sell
```

• LeetDoocs

```
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        return sum(max(0, b - a) for a, b in pairwise(prices))
```

```
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        n = len(prices)
        f = [[0] * 2 for _ in range(n)]
        f[0][0] = -prices[0]
        for i in range(1, n):
            f[i][0] = max(f[i - 1][0], f[i - 1][1] - prices[i])
            f[i][1] = max(f[i - 1][1], f[i - 1][0] + prices[i])
        return f[n - 1][1]
```

• SimplyLeet

```
# Function to calculate maximum profit
def maxProfit(prices):
    profit = 0
    # Iterate over the price list starting from the first day
    for i in range(1, len(prices)):
        # If there is a rise in price from previous day, add up the profit
        if prices[i] > prices[i - 1]:
            profit += prices[i] - prices[i - 1]
    return profit
```

```
# Example usage:
prices = [7,1,5,3,6,4]
print(maxProfit(prices)) # Output: 7
```


Arrays

#189 Rotate Array

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Math · Two Pointers

Lists: Grind 169

Problem

Given an integer array `nums`, rotate the array to the right by `k` steps, where `k` is non-negative.

Example 1:

Input: `nums = [1,2,3,4,5,6,7]`, `k = 3`
Output: `[5,6,7,1,2,3,4]`
Explanation:
rotate 1 steps to the right: `[7,1,2,3,4,5,6]`
rotate 2 steps to the right: `[6,7,1,2,3,4,5]`
rotate 3 steps to the right: `[5,6,7,1,2,3,4]`

Example 2:

Input: `nums = [-1,-100,3,99]`, `k = 2`
Output: `[3,99,-1,-100]`
Explanation:
rotate 1 steps to the right: `[99,-1,-100,3]`
rotate 2 steps to the right: `[3,99,-1,-100]`

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $-231 \leq \text{nums}[i] \leq 231 - 1$
- $0 \leq k \leq 105$

Round 2 · High · Medium

p.393

Follow up:

- Try to come up with as many solutions as you can. There are at least three different ways to solve this problem.
- Could you do it in-place with $O(1)$ extra space?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def rotate(self, nums: List[int], k: int) -> None:
        k %= len(nums)
        self.reverse(nums, 0, len(nums) - 1)
        self.reverse(nums, 0, k - 1)
        self.reverse(nums, k, len(nums) - 1)

    def reverse(self, nums: List[int], l: int, r: int) -> None:
        while l < r:
            nums[l], nums[r] = nums[r], nums[l]

            l += 1
            r -= 1
```

• LeetDoocs

```
class Solution:
    def rotate(self, nums: List[int], k: int) -> None:
        def reverse(i: int, j: int):
            while i < j:
                nums[i], nums[j] = nums[j], nums[i]
                i, j = i + 1, j - 1

        n = len(nums)
        k %= n
        reverse(0, n - 1)
```

Round 2 · High · Medium

p.394

```
reverse(0, k - 1)
reverse(k, n - 1)
```

• SimplyLeet

```
# Python solution using the reverse method
```

```
def rotate(nums, k):
    # Calculate the effective rotations needed
    n = len(nums)
    k = k % n # in case k is greater than the length of the array
```

```
# Helper function to reverse a portion of the list in place
```

```
def reverse(left, right):
    while left < right:
```

```
        # Swap the elements at indices left and right
        nums[left], nums[right] = nums[right], nums[left]
        left += 1
        right -= 1
```

```
# Reverse the entire array
```

```
reverse(0, n - 1)
```

```
# Reverse the first k elements to restore their order
```

```
reverse(0, k - 1)
```

```
# Reverse the remaining n - k elements
```

```
reverse(k, n - 1)
```

```
# Example usage:
```

```
# nums = [1,2,3,4,5,6,7]
```

```
# rotate(nums, 3)
```

```
# print(nums) # Output: [5,6,7,1,2,3,4]
```

• LC.ca

```
class Solution:
    def rotate(self, nums: List[int], k: int) -> None:
        k %= len(nums)
        nums[:] = nums[-k:] + nums[:-k]
```

```
#####
```

```
class Solution:
    def rotate(self, nums: List[int], k: int) -> None:
        """
```

```
        Do not return anything, modify nums in-place instead.
        """
```

```
        n = len(nums)
```

```
        k %= n
```

```
        if n < 2 or k == 0:
```

```
            return
```

```
        nums[:] = nums[::-1]
```

```
        nums[:k] = nums[:k][::-1]
```

```
        nums[k:] = nums[k:][::-1]
```

```
#####
```

```
class Solution(object):
    def rotate(self, nums, k):
        """
```

```
:type nums: List[int]
```

```
:type k: int
```

```
:rtype: void Do not return anything, modify nums in-place instead.
```

```
        """
```

```
        if len(nums) == 0 or k == 0:
```

```
            return
```

```
        def reverse(start, end, s):
```

```
            while start < end:
```

```
                s[start], s[end] = s[end], s[start]
```

```
                start += 1
```

```
                end -= 1
```

```
        n = len(nums) - 1
```

```
        k = k % len(nums)
```

```
reverse(0, n - k, nums)
reverse(n - k + 1, n, nums)
reverse(0, n, nums)
```

◆ Quick Review

Key Insights

- The rotation amount k might be greater than the length of the array, so use $k \% n$ (where n is the array length) to compute the effective rotation.
- A smart in-place solution involves reversing sections of the array:
 - Reverse the entire array.
 - Reverse the first k elements.
 - Reverse the remaining $n-k$ elements.
- Alternative approaches include using extra space or performing k single-step rotations (which is less optimal).

Space & Time

Time Complexity: $O(n)$ where n is the number of elements in the array.

Space Complexity: $O(1)$ if done in-place.

Solution

Round 2 · High · Medium

p.397

We solve the problem using the "reverse" technique to perform the rotation in-place. First, we adjust k by taking k modulo the length of the array to handle cases where k exceeds the array length. Then, we reverse the entire array. Next, we reverse the first k elements to restore their correct order. Finally, we reverse the remaining $n-k$ elements. This sequence yields the array rotated to the right by k positions while strictly using $O(1)$ extra space.

Round 2 · High · Medium

p.398

Arrays

#229 Majority Element II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table · Sorting · Counting
 Lists: AlgoMaster 600

Problem

Given an integer array of size n , find all elements that appear more than $n/3$ times.

Example 1:

Input: `nums = [3,2,3]`
 Output: `[3]`

Example 2:

Input: `nums = [1]`
 Output: `[1]`

Example 3:

Input: `nums = [1,2]`
 Output: `[1,2]`

Constraints:

- $1 \leq \text{nums.length} \leq 5 \times 10^4$
- $-109 \leq \text{nums}[i] \leq 109$

Follow up: Could you solve the problem in linear time and in $O(1)$ space?

Round 2 · High · Medium

p.399

Community Solutions (Python)

• WalkCC

```
class Solution:
    def majorityElement(self, nums: List[int]) -> List[int]:
        ans1 = 0
        ans2 = 1
        count1 = 0
        count2 = 0

        for num in nums:
            if num == ans1:
                count1 += 1

            elif num == ans2:
                count2 += 1
            elif count1 == 0:
                ans1 = num
                count1 = 1
            elif count2 == 0:
                ans2 = num
                count2 = 1
            else:
                count1 -= 1
                count2 -= 1

        return [ans for ans in (ans1, ans2) if nums.count(ans) > len(nums) // 3]
```

• LeetDoocs

```
class Solution:
    def majorityElement(self, nums: List[int]) -> List[int]:
        n1 = n2 = 0
        m1, m2 = 0, 1
        for m in nums:
            if m == m1:
                n1 += 1
            elif m == m2:
                n2 += 1
            elif n1 == 0:
```

Round 2 · High · Medium

p.400

```

        m1, n1 = m, 1
    elif n2 == 0:
        m2, n2 = m, 1
    else:
        n1, n2 = n1 - 1, n2 - 1
    return [m for m in [m1, m2] if nums.count(m) > len(nums) // 3]

```

• SimplyLeet

```

# Function to find all majority elements appearing more than n/3 times
def majorityElement(nums):
    # Initialize candidate and counters for two possible majority elements
    candidate1, candidate2 = None, None
    count1, count2 = 0, 0

    # First pass: find potential candidates
    for num in nums:
        if candidate1 == num:
            count1 += 1
        elif candidate2 == num:
            count2 += 1
        elif count1 == 0:
            candidate1 = num
            count1 = 1
        elif count2 == 0:
            candidate2 = num
            count2 = 1
        else:
            count1 -= 1
            count2 -= 1

    # Second pass: validate the candidates
    result = []
    count1 = count2 = 0
    for num in nums:
        if num == candidate1:
            count1 += 1
        elif num == candidate2:
            count2 += 1

```

Round 2 · High · Medium

p.401

```

n = len(nums)
if count1 > n // 3:
    result.append(candidate1)
if count2 > n // 3:
    result.append(candidate2)
return result

# Example usage:
print(majorityElement([3,2,3])) # Output: [3]

```

Round 2 · High · Medium

p.402

Arrays

#238 Product of Array Except Self

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Prefix Sum
 Lists: Grind 169

Problem

Given an integer array `nums`, return an array `answer` such that `answer[i]` is equal to the product of all the elements of `nums` except `nums[i]`.

The product of any prefix or suffix of `nums` is guaranteed to fit in a 32-bit integer.

You must write an algorithm that runs in $O(n)$ time and without using the division operation.

Example 1:

Input: `nums = [1,2,3,4]`
 Output: `[24,12,8,6]`

Example 2:

Input: `nums = [-1,1,0,-3,3]`
 Output: `[0,0,9,0,0]`

Constraints:

Round 2 · High · Medium

p.403

- $2 \leq \text{nums.length} \leq 105$
- $-30 \leq \text{nums}[i] \leq 30$
- The input is generated such that `answer[i]` is guaranteed to fit in a 32-bit integer.

Follow up: Can you solve the problem in $O(1)$ extra space complexity? (The output array does not count as extra space for space complexity analysis.)

Community Solutions (Python)

• WalkCC

```
class Solution:
    def productExceptSelf(self, nums: list[int]) -> list[int]:
        n = len(nums)
        prefix = [1] * n # prefix product
        suffix = [1] * n # suffix product

        for i in range(1, n):
            prefix[i] = prefix[i - 1] * nums[i - 1]

        for i in reversed(range(n - 1)):
            suffix[i] = suffix[i + 1] * nums[i + 1]

        return [prefix[i] * suffix[i] for i in range(n)]
```

Round 2 · High · Medium

p.404

```
class Solution:
    def productExceptSelf(self, nums: list[int]) -> list[int]:
        n = len(nums)
        ans = [1] * n

        # Use ans as the prefix product array.
        for i in range(1, n):
            ans[i] = ans[i - 1] * nums[i - 1]

        suffix = 1 # suffix product

        for i, num in reversed(list(enumerate(nums))):
            ans[i] *= suffix
            suffix *= num

        return ans
```

• LeetCode

```
class Solution:
    def productExceptSelf(self, nums: List[int]) -> List[int]:
        n = len(nums)
        ans = [0] * n
        left = right = 1
        for i, x in enumerate(nums):
            ans[i] = left
            left *= x
        for i in range(n - 1, -1, -1):
            ans[i] *= right

            right *= nums[i]
        return ans
```

• SimplyLeet

Round 2 · High · Medium

p.405

```
# Function to compute product of array except self
def productExceptSelf(nums):
    n = len(nums)
    # Initialize the output array with 1s for prefix multiplication
    answer = [1] * n

    # Compute prefix products: for each element answer[i] holds the product of
    # all numbers left of i
    prefix_product = 1
    for i in range(n):
        answer[i] = prefix_product # store prefix product so far

        prefix_product *= nums[i] # update the prefix product

    # Compute suffix products and multiply with prefix products in answer array
    suffix_product = 1
    for i in range(n - 1, -1, -1):
        answer[i] *= suffix_product # multiply current answer with suffix
        # product
        suffix_product *= nums[i] # update the suffix product

    return answer

# Example usage
nums = [1, 2, 3, 4]
print(productExceptSelf(nums)) # Output: [24, 12, 8, 6]
```

• LC.ca

```
class Solution: # 2 extra arrays
    def productExceptSelf(self, nums):
        if nums is None or len(nums) == 0:
            return nums

        # Product from index=0 to index=i-1
        from_left = [1] * len(nums)
        for i in range(1, len(nums)):
            from_left[i] = nums[i - 1] * from_left[i - 1]
```

Round 2 · High · Medium

p.406

```
# Product from index=n-1 to current position
from_right = [1] * len(nums)
for i in range(len(nums) - 2, -1, -1):
    from_right[i] = nums[i + 1] * from_right[i + 1]

# Calculate result
result = [0] * len(nums)
for i in range(len(nums)):
    result[i] = from_left[i] * from_right[i]
```

```
return result
```

```
#####
```

```
class Solution: # follow up, no extra space
    def productExceptSelf(self, nums: List[int]) -> List[int]:
        n = len(nums)
        ans = [0] * n
        left = right = 1
        for i, v in enumerate(nums):
```

```
    ans[i] = left
    left *= v
    for i in range(n - 1, -1, -1):
        ans[i] *= right
        right *= nums[i]
    return ans
```

```
#####
```

```
class Solution(object):
    def productExceptSelf(self, nums):
        """
        :type nums: List[int]
        :rtype: List[int]
        """
        dp = [1] * len(nums)
        for i in range(1, len(nums)):
            dp[i] = dp[i - 1] * nums[i - 1]
```

```
prod = 1
for i in reversed(range(0, len(nums))):
    dp[i] = dp[i] * prod
    prod *= nums[i]
return dp
```

◆ Quick Review

Key Insights

- The product for each index can be computed by multiplying the product of all numbers to the left and the product of all numbers to the right of that index.
- Division is not allowed, so we must compute the cumulative products explicitly.
- Create two passes: one forward pass for prefix products and one backward pass for suffix products.
- Use the output array to store the cumulative product values, achieving $O(1)$ extra space by not using additional arrays.

Space & Time

Time Complexity: $O(n)$ because we iterate through the array twice.

Space Complexity: $O(1)$ extra space (excluding the output array).

Solution

The approach uses two passes over the input array:

- Initialize the answer array and fill it with prefix products. For each index i , $\text{answer}[i]$ contains the product of all elements before i .
- Traverse the array from the end using a running suffix product. Update the answer array by multiplying the current suffix product with the prefix product already stored. This method leverages an output array to store intermediate results and uses a temporary variable for the suffix product, ensuring constant extra space.

Arrays

□ #334 Increasing Triplet Subsequence

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Greedy
 Lists: AlgoMaster 600

Problem

Given an integer array `nums`, return `true` if there exists a triple of indices (i, j, k) such that $i < j < k$ and $\text{nums}[i] < \text{nums}[j] < \text{nums}[k]$. If no such indices exists, return `false`.

Example 1:

Input: `nums = [1,2,3,4,5]`
 Output: `true`
 Explanation: Any triplet where $i < j < k$ is valid.

Example 2:

Input: `nums = [5,4,3,2,1]`
 Output: `false`
 Explanation: No triplet exists.

Example 3:

Input: `nums = [2,1,5,0,4,6]`
 Output: `true`
 Explanation: One of the valid triplet is $(1, 4, 5)$, because $\text{nums}[1] == 1 < \text{nums}[4] == 4 < \text{nums}[5] == 6$.

Constraints:

- $1 \leq \text{nums.length} \leq 5 * 10^5$
- $-231 \leq \text{nums}[i] \leq 231 - 1$

Follow up: Could you implement a solution that runs in $O(n)$ time complexity and $O(1)$ space complexity?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def increasingTriplet(self, nums: List[int]) -> bool:
        first = math.inf
        second = math.inf

        for num in nums:
            if num <= first:
                first = num
            elif num <= second: # first < num <= second
                second = num

        else:
            return True # first < second < num (third)

        return False
```

• LeetCode

```
class Solution:
    def increasingTriplet(self, nums: List[int]) -> bool:
        mi, mid = inf, inf
        for num in nums:
            if num > mid:
                return True
            if num <= mi:
                mi = num
            else:
                mid = num
```

Round 2 · High · Medium

p.411

• SimplyLeet

```
def increasingTriplet(nums):
    # Initialize first and second to be infinity
    first = float('inf')
    second = float('inf')

    # Loop through each number in the list
    for num in nums:
        # Update first if current num is smaller than or equal first
        if num <= first:
            first = num

        # Update second if current num is smaller than or equal second but
        # greater than first
        elif num <= second:
            second = num

        # If current num is greater than both first and second, a valid triplet
        # is found
        else:
            return True

    return False

# Sample Test
print(increasingTriplet([2,1,5,0,4,6])) # Expected output: True
```

Round 2 · High · Medium

p.412

Arrays

#2348 Number of Zero-Filled Subarrays **MED**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Math

Lists: AlgoMaster 600

Problem

Given an integer array `nums`, return the number of subarrays filled with 0.

A subarray is a contiguous non-empty sequence of elements within an array.

Example 1:

Input: `nums = [1,3,0,0,2,0,0,4]`
Output: 6
Explanation:
There are 4 occurrences of `[0]` as a subarray.
There are 2 occurrences of `[0,0]` as a subarray.
There is no occurrence of a subarray with a size more than 2 filled with 0.
Therefore, we return 6.

Example 2:

Input: `nums = [0,0,0,2,0,0]`
Output: 9
Explanation:
There are 5 occurrences of `[0]` as a subarray.
There are 3 occurrences of `[0,0]` as a subarray.
There is 1 occurrence of `[0,0,0]` as a subarray.
There is no occurrence of a subarray with a size more than 3 filled with 0.
Therefore, we return 9.

Round 2 · High · Medium

p.413

Example 3:

Input: `nums = [2,10,2019]`
Output: 0
Explanation: There is no subarray filled with 0. Therefore, we return 0.

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $-109 \leq \text{nums}[i] \leq 109$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def zeroFilledSubarray(self, nums: List[int]) -> int:
        ans = 0
        indexBeforeZero = -1

        for i, num in enumerate(nums):
            if num:
                indexBeforeZero = i
            else:
                ans += i - indexBeforeZero

        return ans
```

• LeetDoocs

```
class Solution:
    def zeroFilledSubarray(self, nums: List[int]) -> int:
        ans = cnt = 0
        for x in nums:
            if x == 0:
                cnt += 1
                ans += cnt
            else:
                cnt = 0
        return ans
```

Round 2 · High · Medium

p.414

• SimplyLeet

```
# Python solution for counting zero-filled subarrays.
def zeroFilledSubarray(nums):
    total_subarrays = 0 # To hold the total count of zero-filled subarrays.
    consecutive_zeros = 0 # To count the current consecutive zeros.

    # Iterate through each number in the array.
    for num in nums:
        if num == 0:
            consecutive_zeros += 1 # Increase count if the current element is
0.
        else:
            # When the streak of zeros ends, add the number of subarrays from
the streak.
            total_subarrays += consecutive_zeros * (consecutive_zeros + 1) //
2
            consecutive_zeros = 0 # Reset the counter.

    # If the array ends with a streak of zeros, count them as well.
    total_subarrays += consecutive_zeros * (consecutive_zeros + 1) // 2
    return total_subarrays

# Example usage:
print(zeroFilledSubarray([1,3,0,0,2,0,0,4])) # Output should be 6
```

Strings

Round 2 · High · Medium · 6 questions

Round 2 · High · Medium

p.415

Round 2 · High · Medium

p.416

Strings

#6 Zigzag Conversion

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode String

Lists: AlgoMaster 600

Problem

The string "PAYPALISHIRING" is written in a zigzag pattern on a given number of rows like this: (you may want to display this pattern in a fixed font for better legibility)

```
P A H N
A P L S I I G
Y I R
```

And then read line by line: "PAHNAPLSIIGYIR"

Write the code that will take a string and make this conversion given a number of rows:

```
string convert(string s, int numRows);
```

Example 1:

```
Input: s = "PAYPALISHIRING", numRows = 3
Output: "PAHNAPLSIIGYIR"
```

Example 2:

Round 2 · High · Medium

p.417

```
Input: s = "PAYPALISHIRING", numRows = 4
Output: "PINALSIGYAHRPI"
Explanation:
P I N
A L S I G
Y A H R
P I
```

Example 3:

```
Input: s = "A", numRows = 1
Output: "A"
```

Constraints:

- $1 \leq s.length \leq 1000$
- s consists of English letters (lower-case and upper-case), ',' and '.'.
- $1 \leq numRows \leq 1000$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def convert(self, s: str, numRows: int) -> str:
        rows = [''] * numRows
        k = 0
        direction = (numRows == 1) - 1

        for c in s:
            rows[k] += c
            if k == 0 or k == numRows - 1:
                direction *= -1
            k += direction

        return ''.join(rows)
```

• LeetDoocs

Round 2 · High · Medium

p.418

```
class Solution:
    def convert(self, s: str, numRows: int) -> str:
        if numRows == 1:
            return s
        g = [[] for _ in range(numRows)]
        i, k = 0, -1
        for c in s:
            g[i].append(c)
            if i == 0 or i == numRows - 1:
                k = -k
            i += k
        return ''.join(chain(*g))
```

• SimplyLeet

```
def convert(s: str, numRows: int) -> str:
    # Handle the edge case when the zigzag pattern is not applicable.
    if numRows == 1 or numRows >= len(s):
        return s

    # Initialize a list to hold strings for each row.
    rows = [''] * numRows
    current_row = 0
    going_down = False

    # Loop over each character in the input string.
    for char in s:
        rows[current_row] += char
        # Reverse direction at the first or last row.
        if current_row == 0 or current_row == numRows - 1:
            going_down = not going_down
        current_row += 1 if going_down else -1

    # Concatenate all row strings to form the final result.
    return ''.join(rows)

# Example usage:
# print(convert("PAYPALISHIRING", 3))
```

Strings

□ #38 Count and Say

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode String

Lists: AlgoMaster 600

Problem

The count-and-say sequence is a sequence of digit strings defined by the recursive formula:

- $\text{countAndSay}(1) = "1"$
- $\text{countAndSay}(n)$ is the run-length encoding of $\text{countAndSay}(n - 1)$.

Run-length encoding (RLE) is a string compression method that works by replacing consecutive identical characters (repeated 2 or more times) with the concatenation of the character and the number marking the count of the characters (length of the run). For example, to compress the string "3322251" we replace "33" with "23", replace "222" with "32", replace "5" with "15" and replace "1" with "11". Thus the compressed string becomes "23321511".

Given a positive integer n , return the n th element of the count-and-say sequence.

Example 1:

Input: $n = 4$

Output: "1211"

Explanation:

```
countAndSay(1) = "1"
countAndSay(2) = RLE of "1" = "11"
countAndSay(3) = RLE of "11" = "21"
countAndSay(4) = RLE of "21" = "1211"
```

Example 2:

Input: $n = 1$

Output: "1"

Explanation:

This is the base case.

Constraints:

- $1 \leq n \leq 30$

Follow up: Could you solve it iteratively?

Community Solutions (Python)

- WalkCC

Round 2 · High · Medium

p.421

```
class Solution:
    def countAndSay(self, n: int) -> str:
        ans = '1'

        for _ in range(n - 1):
            nxt = ''
            i = 0
            while i < len(ans):
                count = 1
                while i + 1 < len(ans) and ans[i] == ans[i + 1]:
                    count += 1
                    i += 1
                nxt += str(count) + ans[i]
                i += 1
            ans = nxt

        return ans
```

• LeetCode

```
class Solution:
    def countAndSay(self, n: int) -> str:
        s = '1'
        for _ in range(n - 1):
            i = 0
            t = []
            while i < len(s):
                j = i
                while j < len(s) and s[j] == s[i]:
                    j += 1
                t.append(str(j - i))
                t.append(str(s[i]))
                i = j
            s = ''.join(t)
        return s
```

• SimplyLeet

Round 2 · High · Medium

p.422

```
# Function to generate the nth count-and-say sequence element
def countAndSay(n: int) -> str:
    # Base case: if n is 1, return "1"
    current_sequence = "1"

    # Iterate from 2 to n, building each sequence iteratively
    for _ in range(2, n + 1):
        next_sequence = "" # Initialize the next sequence
        i = 0
        # Process the current sequence using run-length encoding

        while i < len(current_sequence):
            count = 1 # We have at least one occurrence of current digit
            # Count the number of same consecutive digits
            while i + 1 < len(current_sequence) and current_sequence[i] == current_sequence[i + 1]:
                count += 1
                i += 1
            # Append the count and the digit to the next sequence
            next_sequence += str(count) + current_sequence[i]
            i += 1 # Move to the next distinct digit group
        current_sequence = next_sequence # Update current sequence for next iteration

    return current_sequence

# Example Usage:
print(countAndSay(4)) # Expected output: "1211"
```

Strings

□ #151 Reverse Words in a String

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Two Pointers · String
Lists: AlgoMaster 600

Problem

Given an input string *s*, reverse the order of the words.

A word is defined as a sequence of non-space characters. The words in *s* will be separated by at least one space.

Return a string of the words in reverse order concatenated by a single space.

Note that *s* may contain leading or trailing spaces or multiple spaces between two words. The returned string should only have a single space separating the words. Do not include any extra spaces.

Example 1:

Input: *s* = "the sky is blue"
Output: "blue is sky the"

Example 2:

Input: s = " hello world "
 Output: "world hello"
 Explanation: Your reversed string should not contain leading or trailing spaces.

Example 3:

Input: s = "a good example"
 Output: "example good a"
 Explanation: You need to reduce multiple spaces between two words to a single space in the reversed string.

Constraints:

- $1 \leq s.length \leq 104$
- s contains English letters (upper-case and lower-case), digits, and spaces ' '.
- There is at least one word in s.

Follow-up: If the string data type is mutable in your language, can you solve it in-place with $O(1)$ extra space?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def reverseWords(self, s: str) -> str:
        return ' '.join(reversed(s.split()))
```

• LeetCode

```
class Solution:
    def reverseWords(self, s: str) -> str:
        words = []
        i, n = 0, len(s)
        while i < n:
            while i < n and s[i] == " ":
                i += 1
            if i < n:
                j = i
                while j < n and s[j] != " ":
                    j += 1
                words.append(s[i:j])
                i = j
        return " ".join(words[::-1])
```

```
class Solution:
    def reverseWords(self, s: str) -> str:
        return " ".join(reversed(s.split()))
```

• SimplyLeet

```
# Python solution
def reverseWords(s: str) -> str:
    # Split the string by spaces; this automatically handles multiple spaces
    # and trimming
    words = s.split()
    # Reverse the list of words
    words.reverse()
    # Join the reversed words with a single space and return the result
    return " ".join(words)
```

Example usage

```
if __name__ == "__main__":
    s = " hello world "
    print(reverseWords(s)) # Expected output: "world hello"
```

Strings

★ #161 One Edit Distance ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Two Pointers · String

Lists: ★ Premium | Premium 98

Problem

Given two strings s and t , return true if they are both one edit distance apart, otherwise return false.

A string s is said to be one distance apart from a string t if you can:

- Insert exactly one character into s to get t .
- Delete exactly one character from s to get t .
- Replace exactly one character of s with a different character to get t .

Example 1:

Input: $s = "ab"$, $t = "acb"$
Output: true
Explanation: We can insert 'c' into s to get t .

Example 2:

Input: $s = ""$, $t = ""$
Output: false
Explanation: We cannot get t from s by only one step.

Round 2 · High · Medium

p.427

Constraints:

- $0 \leq s.length, t.length \leq 104$
- s and t consist of lowercase letters, uppercase letters, and digits.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isOneEditDistance(self, s: str, t: str) -> bool:
        m = len(s)
        n = len(t)
        if m > n: # Make sure that |s| <= |t|.
            return self.isOneEditDistance(t, s)

        for i in range(m):
            if s[i] != t[i]:
                if m == n:

                    return s[i + 1:] == t[i + 1:] # Replace s[i] with t[i].
                    return s[i:] == t[i + 1:] # Delete t[i].

                return m + 1 == n # Delete t[-1].
```

• LeetDoocs

```
class Solution:
    def isOneEditDistance(self, s: str, t: str) -> bool:
        if len(s) < len(t):
            return self.isOneEditDistance(t, s)
        m, n = len(s), len(t)
        if m - n > 1:
            return False
        for i, c in enumerate(t):
            if c != s[i]:
                return s[i + 1:] == t[i + 1:] if m == n else s[i + 1:] == t[i:]
```

Round 2 · High · Medium

p.428

```
return m == n + 1
```

• SimplyLeet

```
# Function to check if two strings are one edit distance apart
def isOneEditDistance(s: str, t: str) -> bool:
    # Get lengths of both strings
    m, n = len(s), len(t)

    # The difference in length must not be greater than 1
    if abs(m - n) > 1:
        return False

    # Ensure s is the shorter string, or they are equal in length
    if m > n:
        return isOneEditDistance(t, s)

    # Initialize variables; found difference flag and indices for both strings
    found_difference = False
    i = j = 0

    while i < m and j < n:
        if s[i] != t[j]:
            # If a difference has already been found, return False

            if found_difference:
                return False
            found_difference = True
            # If lengths are equal, move both pointers
            if m == n:
                i += 1
            else:
                # If characters are the same, move the pointer in the shorter
                string
                i += 1
            # Always move pointer for longer string
```

Round 2 · High · Medium

p.429

```
j += 1
```

```
# If no difference was found during iteration, then one extra character at
the end is the edit
return found_difference or (j - i == 1)

# Example usage:
#print(isOneEditDistance("ab", "acb")) # Output: True
```

• LC.ca

```
class Solution:
    def isOneEditDistance(self, s: str, t: str) -> bool:
        if len(s) < len(t): # goal is to ensure t is always the shorter one
            return self.isOneEditDistance(t, s)
        m, n = len(s), len(t)
        if m - n > 1:
            # case: abc, abcdefg, return false
            return False
        for i, c in enumerate(t):
            if c != s[i]:
                return s[i + 1 :] == t[i + 1 :] if m == n else s[i + 1 :] == t
[i:]

        # case: abc, abcd. return true
        # case: abc, abc. the same 2 strings should return false
        # case: abc, abcdefg, return false
        return m == n + 1

#####

class Solution(object):
    def isOneEditDistance(self, s, t):
```

Round 2 · High · Medium

p.430

```

"""
:type s: str
:type t: str
:rtype: bool
"""
if len(s) >= len(t):
    i = 0
    while i < len(t) and s[i] == t[i]:
        i += 1
    return s != t and (s[i + 1:] == t[i:] if len(s) != len(t) else s[i + 1:]
== t[i + 1:])
else:
    return self.isOneEditDistance(t, s)

```

◆ Quick Review

Key Insights

- The difference in length between s and t must be at most 1.
- When lengths are equal, the only possibility is that exactly one character is different (replacement).
- When one string is longer by one character, the difference can be fixed by one insertion (or deletion in the other direction).
- Use two pointers to compare corresponding characters and carefully handle the first difference.

Space & Time

Round 2 · High · Medium

p.431

Time Complexity: $O(n)$, where n is the length of the shorter string, as we traverse both strings at most once.

Space Complexity: $O(1)$, using only constant extra space.

Solution

The solution uses a two-pointer approach. First, we check if the length difference is greater than 1, in which case they cannot be one edit apart. For the two primary cases:

- When strings are of equal length, iterate through both strings and count the number of differing characters. There should be exactly one difference.
- When lengths differ by one, use pointers for both strings. Upon encountering a difference, increment the pointer for the longer string only and ensure that no previous difference has been encountered. If a second discrepancy is found, return false. The key is to handle the edge case where the difference is at the end of the string.

Round 2 · High · Medium

p.432

Strings

★ #616 Add Bold Tag in String ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode

Array · Hash Table · String · Trie

Lists: ★ Premium | Premium 98

Problem

You are given a string *s* and an array of strings *words*.

You should add a closed pair of bold tag **** and **** to wrap the substrings in *s* that exist in *words*.

- If two such substrings overlap, you should wrap them together with only one pair of closed bold-tag.
- If two substrings wrapped by bold tags are consecutive, you should combine them.

Return *s* after adding the bold tags.

Example 1:

```
Input: s = "abcxyz123", words = ["abc","123"]
Output: "<b>abc</b>xyz<b>123</b>"
Explanation: The two strings of words are substrings of s as following:
"abcxyz123".
We add <b> before each substring and </b> after each substring.
```

Round 2 · High · Medium

p.433

Example 2:

```
Input: s = "aaabbb", words = ["aa","b"]
Output: "<b>aaabbb</b>"
Explanation:
"aa" appears as a substring two times: "aaabbb" and "aaabbb".
"b" appears as a substring three times: "aaabbb", "aaabbb", and "aaabbb".
We add <b> before each substring and </b> after each substring:
"<b>a<b>a</b>a</b>b</b>b</b>b</b>b</b>b</b>".
Since the first two <b>'s overlap, we merge them:
"<b>aaa</b>b</b>b</b>b</b>b</b>b</b>".
Since now the four <b>'s are consecutive, we merge them: "<b>aaabbb</b>".
```

Constraints:

- $1 \leq s.length \leq 1000$
- $0 \leq words.length \leq 100$
- $1 \leq words[i].length \leq 1000$
- *s* and *words[i]* consist of English letters and digits.
- All the values of *words* are unique.

Note: This question is the same as 758. Bold Words in String.

Community Solutions (Python)

- WalkCC

Round 2 · High · Medium

p.434

```

class Solution:
    def addBoldTag(self, s: str, words: list[str]) -> str:
        n = len(s)
        ans = []
        # bold[i] := True if s[i] should be bolded
        bold = [0] * n

        boldEnd = -1 # s[i:boldEnd] should be bolded
        for i in range(n):
            for word in words:

                if s[i:].startswith(word):
                    boldEnd = max(boldEnd, i + len(word))
                bold[i] = boldEnd > i

        # Construct the with bold tags
        i = 0
        while i < n:
            if bold[i]:
                j = i
                while j < n and bold[j]:
                    j += 1
                    # `s[i..j)` should be bolded.
                    ans.append('<b>' + s[i:j] + '</b>')
                    i = j
            else:
                ans.append(s[i])
                i += 1

        return ''.join(ans)

class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.isWord = False

class Solution:
    def addBoldTag(self, s: str, words: list[str]) -> str:
        n = len(s)
        ans = []

```

Round 2 · High · Medium

p.435

```

# bold[i] := True if s[i] should be bolded
bold = [0] * n
root = TrieNode()

def insert(word: str) -> None:
    node = root
    for c in word:
        node = node.children.setdefault(c, TrieNode())
    node.isWord = True

def find(s: str, i: int) -> int:
    node = root
    ans = -1
    for j in range(i, len(s)):
        if s[j] not in node.children:
            node.children[s[j]] = TrieNode()
        node = node.children[s[j]]
        if node.isWord:
            ans = j
    return ans

for word in words:
    insert(word)

boldEnd = -1 # `s[i..boldEnd)` should be bolded.
for i in range(n):
    boldEnd = max(boldEnd, find(s, i))
    bold[i] = boldEnd >= i

# Construct the with bold tags

i = 0
while i < n:
    if bold[i]:
        j = i
        while j < n and bold[j]:
            j += 1
            # `s[i..j)` should be bolded.
            ans.append('<b>' + s[i:j] + '</b>')
            i = j
    else:

```

Round 2 · High · Medium

p.436

```

        ans.append(s[i])
        i += 1

    return ''.join(ans)

```

• LeetCode

```

class Trie:
    def __init__(self):
        self.children = [None] * 128
        self.is_end = False

    def insert(self, word):
        node = self
        for c in word:
            idx = ord(c)
            if node.children[idx] is None:
                node.children[idx] = Trie()
                node = node.children[idx]
            node.is_end = True

class Solution:
    def addBoldTag(self, s: str, words: List[str]) -> str:
        trie = Trie()
        for w in words:
            trie.insert(w)

        n = len(s)
        pairs = []
        for i in range(n):
            node = trie
            for j in range(i, n):
                idx = ord(s[j])
                if node.children[idx] is None:
                    break
                node = node.children[idx]
                if node.is_end:

```

Round 2 · High · Medium

p.437

```

        pairs.append([i, j])

    if not pairs:
        return s
    st, ed = pairs[0]
    t = []
    for a, b in pairs[1:]:
        if ed + 1 < a:
            t.append([st, ed])
            st, ed = a, b
        else:
            ed = max(ed, b)
            t.append([st, ed])

    ans = []
    i = j = 0
    while i < n:
        if j == len(t):
            ans.append(s[i:])
            break
        st, ed = t[j]

        if i < st:
            ans.append(s[i:st])
        ans.append('<b>')
        ans.append(s[st : ed + 1])
        ans.append('</b>')
        j += 1
        i = ed + 1

    return ''.join(ans)

```

• SimplyLeet

Round 2 · High · Medium

p.438

```
# Python Code Solution
def addBoldTag(s, words):
    n = len(s)
    # Mark positions to be bolded
    bold = [False] * n
    # Iterate each word and mark its occurrences
    for word in words:
        start = s.find(word)
        while start != -1:
            for i in range(start, start + len(word)):
                bold[i] = True
            start = s.find(word, start + 1)

    # Build the final result with bold tags
    result = []
    i = 0
    while i < n:
        if not bold[i]:
            result.append(s[i])
            i += 1
        else:
            result.append("<b>")
            while i < n and bold[i]:
                result.append(s[i])
                i += 1
            result.append("</b>")
    return "".join(result)

# Example usage:
# s = "abcxyz123"
```

```
# words = ["abc", "123"]
# print(addBoldTag(s, words)) # "<b>abc</b>xyz<b>123</b>"

# Line-by-line:
# 1. Initialize an array "bold" of length n to track which characters need bold formatting.
# 2. For each word, use string.find to locate every occurrence and mark the corresponding indices.
# 3. Traverse the string s. If the current character is marked bold and it is the start of a bold segment,
# insert the opening tag before appending characters until the end of that bold segment.
# 4. Append the closing tag after the bold segment and continue the process.
```

• LC.ca

```
class Trie:
    def __init__(self):
        self.children = [None] * 128
        self.is_end = False

    def insert(self, word):
        node = self
        for c in word:
            idx = ord(c)
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
        node.is_end = True

class Solution:
    def addBoldTag(self, s: str, words: List[str]) -> str:
        trie = Trie()
        for w in words:
            trie.insert(w)
```



```

n = len(s)
pairs = []
for i in range(n):
    node = trie
    for j in range(i, n):
        idx = ord(s[j])
        if node.children[idx] is None:
            break
        node = node.children[idx]
        if node.is_end:

```

```

            pairs.append([i, j])
if not pairs:
    return s
st, ed = pairs[0]
t = []
for a, b in pairs[1:]:
    if ed + 1 < a:
        t.append([st, ed])
        st, ed = a, b
    else:

```

```

        ed = max(ed, b)
t.append([st, ed])

```

```

ans = []
i = j = 0
while i < n:
    if j == len(t):
        ans.append(s[i:])
        break
    st, ed = t[j]

```

```

    if i < st:
        ans.append(s[i:st])
    ans.append('<b>')
    ans.append(s[st : ed + 1])
    ans.append('</b>')
    j += 1
    i = ed + 1

```

```

return ''.join(ans)

```

◆ Quick Review

Key Insights

- We need to identify every occurrence of each word in s.
- Overlapping or consecutive intervals should be merged to form one continuous bold region.
- Use an auxiliary data structure (e.g., a boolean array) to mark positions that should be bolded.
- Once the positions are marked, build the final string by inserting bold tags at the appropriate boundaries.

Space & Time

Time Complexity: $O(n * m * k)$ where n is the length of s , m is the number of words, and k is the average length of the words (due to substring matching).

Space Complexity: $O(n)$ for the auxiliary boolean array used to mark bold positions.

Solution

The solution involves two main steps:

- Identify and mark the indices in the string *s* that are part of any matching word. We do this by iterating over each word and marking every occurrence's start and end in a boolean array.
- Construct the final string by traversing the boolean array. When encountering a segment of consecutive true values, wrap that segment with **and** tags.

The key idea is to merge intervals by marking every index that should be bold and then using a single pass to insert tags only when transitioning from non-bold to bold and vice versa.

Strings

☐ #1657 Determine if Two Strings Are Close

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Sorting · Counting
Lists: AlgoMaster 600

Problem

Two strings are considered close if you can attain one from the other using the following operations:

- Operation 1: Swap any two existing characters. For example, *abcde* → *aecdb*
- For example, *aacabb* → *bbcbaa* (all *a*'s turn into *b*'s, and all *b*'s turn into *a*'s)

You can use the operations on either string as many times as necessary.

Given two strings, *word1* and *word2*, return true if *word1* and *word2* are close, and false otherwise.

Example 1:

Input: word1 = "abc", word2 = "bca"
 Output: true
 Explanation: You can attain word2 from word1 in 2 operations.
 Apply Operation 1: "abc" -> "acb"
 Apply Operation 1: "acb" -> "bca"

Example 2:

Input: word1 = "a", word2 = "aa"
 Output: false
 Explanation: It is impossible to attain word2 from word1, or vice versa, in any number of operations.

Example 3:

Input: word1 = "cabbba", word2 = "abbccc"
 Output: true
 Explanation: You can attain word2 from word1 in 3 operations.
 Apply Operation 1: "cabbba" -> "caabbb"
 Apply Operation 2: "caabbb" -> "baaccc"
 Apply Operation 2: "baaccc" -> "abbccc"

Constraints:

- $1 \leq \text{word1.length}, \text{word2.length} \leq 105$
- word1 and word2 contain only lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def closeStrings(self, word1: str, word2: str) -> bool:
        if len(word1) != len(word2):
            return False

        count1 = collections.Counter(word1)
        count2 = collections.Counter(word2)
        if count1.keys() != count2.keys():
            return False
```

```
return sorted(count1.values()) == sorted(count2.values())
```

• LeetDoocs

```
class Solution:
    def closeStrings(self, word1: str, word2: str) -> bool:
        cnt1, cnt2 = Counter(word1), Counter(word2)
        return sorted(cnt1.values()) == sorted(cnt2.values()) and set(
            cnt1.keys()
        ) == set(cnt2.keys())
```

• SimplyLeet

```
# Python solution with line-by-line comments
from collections import Counter

def closeStrings(word1, word2):
    # If the lengths differ, they cannot be close.
    if len(word1) != len(word2):
        return False

    # Count frequency of each character in word1 and word2.
    counter1 = Counter(word1)

    counter2 = Counter(word2)

    # Check if both strings have the same set of characters.
    if set(counter1.keys()) != set(counter2.keys()):
        return False

    # Compare the sorted frequency distributions.
    if sorted(counter1.values()) != sorted(counter2.values()):
        return False

    return True

# Example usage:
print(closeStrings("cabbba", "abbccc")) # Expected output: True
```

Hash Tables

Round 2 · High · Medium · 9 questions

Round 2 · High · Medium

p.447

Hash Tables

□ #49 Group Anagrams

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · String · Sorting
Lists: Grind 169

Problem

Given an array of strings `strs`, group the anagrams together. You can return the answer in any order.

Example 1:

Input: `strs = ["eat","tea","tan","ate","nat","bat"]`

Output: `[["bat"],["nat","tan"],["ate","eat","tea"]]`

Explanation:

- There is no string in `strs` that can be rearranged to form "bat".
- The strings "nat" and "tan" are anagrams as they can be rearranged to form each other.
- The strings "ate", "eat", and "tea" are anagrams as they can be rearranged to form each other.

Example 2:

Round 2 · High · Medium

p.448

Input: strs = [""]

Output: [[""]]

Example 3:

Input: strs = ["a"]

Output: [["a"]]

Constraints:

- $1 \leq \text{strs.length} \leq 104$
- $0 \leq \text{strs}[i].\text{length} \leq 100$
- $\text{strs}[i]$ consists of lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def groupAnagrams(self, strs: list[str]) -> list[list[str]]:
        dict = collections.defaultdict(list)

        for str in strs:
            key = ''.join(sorted(str))
            dict[key].append(str)

        return dict.values()
```

• LeetCode

```
class Solution:
    def groupAnagrams(self, strs: List[str]) -> List[List[str]]:
        d = defaultdict(list)
        for s in strs:
            k = ''.join(sorted(s))
            d[k].append(s)
        return list(d.values())
```

Round 2 · High · Medium

p.449

• SimplyLeet

```
# Python solution for grouping anagrams
from collections import defaultdict

def groupAnagrams(strs):
    # Create a dictionary to hold lists of anagrams; default to an empty list
    # for new keys
    anagrams = defaultdict(list)
    # Iterate over each string in the input list
    for s in strs:
        # Sort the string to use as a key; sorted returns a list so join back
        # to string
        key = ''.join(sorted(s))

        # Append the original string under the sorted key
        anagrams[key].append(s)
    # Return the grouped anagrams as a list of lists
    return list(anagrams.values())

# Example usage:
if __name__ == "__main__":
    strs = ["eat", "tea", "tan", "ate", "nat", "bat"]
    result = groupAnagrams(strs)
    print(result)
```

• LC.ca

```
'''
>>> from collections import defaultdict
>>> d = defaultdict(list)
>>> d["a"]=1
>>> d["b"]=2
>>> d
defaultdict(<class 'list'>, {'a': 1, 'b': 2})
>>> d.values()
dict_values([1, 2])
>>> list(d.values())
```

Round 2 · High · Medium

p.450

```
[1, 2]

### sorted(str) will return a list of chars
>>> a = "sdfddxyz"
>>> sorted(a)
['d', 'd', 'd', 'f', 's', 'x', 'y', 'z']
>>> "".join(sorted(a))
'dddfsxyz'
```

```
class Solution:
    def groupAnagrams(self, strs: List[str]) -> List[List[str]]:
        d = defaultdict(list)
        for s in strs:
            k = "".join(sorted(s))
            d[k].append(s)
        return list(d.values())
```

◆ Quick Review

Key Insights

- Anagrams have the same characters when sorted alphabetically.
- Sorting each string provides a unique key for grouping.
- Use a hash table (dictionary) to map each sorted key to a list of its original strings.
- Finally, gather all the lists from the dictionary to form the result.

Space & Time

Round 2 · High · Medium

p.451

Time Complexity: $O(n * k \log k)$, where n is the number of strings and k is the maximum length of a string (due to sorting each string).
Space Complexity: $O(n * k)$ for storing the grouped strings in the hash table.

Solution

The primary idea is to iterate through each string, sort it to obtain a key that represents its character composition, and then group the original string under this key in a hash table. After processing all strings, the values of the hash table will be the groups of anagrams. This approach efficiently groups anagrams by leveraging sorting and a dictionary for constant-time lookups.

Round 2 · High · Medium

p.452

Hash Tables

#128 Longest Consecutive Sequence

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table · Union Find
 Lists: Grind 169 | CodeSignal

Problem

Given an unsorted array of integers `nums`, return the length of the longest consecutive elements sequence.

You must write an algorithm that runs in $O(n)$ time.

Example 1:

Input: `nums = [100,4,200,1,3,2]`
 Output: 4
 Explanation: The longest consecutive elements sequence is [1, 2, 3, 4]. Therefore its length is 4.

Example 2:

Input: `nums = [0,3,7,2,5,8,4,6,0,1]`
 Output: 9

Example 3:

Input: `nums = [1,0,1,2]`
 Output: 3

Constraints:

- $0 \leq \text{nums.length} \leq 105$
- $-109 \leq \text{nums}[i] \leq 109$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def longestConsecutive(self, nums: List[int]) -> int:
        ans = 0
        seen = set(nums)

        for num in seen:
            # `num` is the start of a sequence.
            if num - 1 in seen:
                continue
            length = 0

            while num in seen:
                num += 1
                length += 1
            ans = max(ans, length)

        return ans
```

• LeetDoocs

```
class Solution:
    def longestConsecutive(self, nums: List[int]) -> int:
        s = set(nums)
        ans = 0
        d = defaultdict(int)
        for x in nums:
            y = x
            while y in s:
                s.remove(y)
                y += 1

            d[x] = d[y] + y - x
            ans = max(ans, d[x])
        return ans
```

• SimplyLeet

```
# Python solution for Longest Consecutive Sequence

def longestConsecutive(nums):
    # Create a set from the list for O(1) look-up times
    num_set = set(nums)
    longest_streak = 0

    # Iterate through each number in the set
    for num in num_set:
        # Only start counting if 'num-1' is not present in the set

        if num - 1 not in num_set:
            current_num = num
            current_streak = 1

            # Count consecutive numbers starting from current_num
            while current_num + 1 in num_set:
                current_num += 1
                current_streak += 1

            # Update the longest streak if current one is longer

            longest_streak = max(longest_streak, current_streak)

    return longest_streak

# Example usage:
print(longestConsecutive([100, 4, 200, 1, 3, 2])) # Output: 4
```

• LC.ca

```
class Solution:
    def longestConsecutive(self, nums: List[int]) -> int:
        s = set(nums)
        ans = 0
        for x in nums:
            if x - 1 not in s:
                y = x + 1
                while y in s:
                    y += 1
                ans = max(ans, y - x)
```

Round 2 · High · Medium

p.455

```
return ans
```

◆ Quick Review

Key Insights

- Utilize a set (or hash table) for constant time look-ups.
- Only start counting a sequence from a number that is the beginning (its previous number is not in the set).
- Each number is visited only once, ensuring an $O(n)$ time complexity.
- Update the maximum sequence length as you discover longer consecutive sequences.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Solution

We first insert all the numbers into a hash set to enable $O(1)$ membership checks. For each number in the set, we check if it is the start of a sequence by ensuring that the number minus one is not in the set. If it is the beginning, we then count all consecutive numbers following

Round 2 · High · Medium

p.456

it, updating the maximum sequence length when necessary. This method guarantees that we only count each number once, satisfying the $O(n)$ time requirement.

Hash Tables

□ ★ #249 Group Shifted Strings ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · String
Lists: ★ Premium | Premium 98

Problem

Perform the following shift operations on a string:

- Right shift: Replace every letter with the successive letter of the English alphabet, where 'z' is replaced by 'a'. For example, "abc" can be right-shifted to "bcd" or "xyz" can be right-shifted to "yza".
- Left shift: Replace every letter with the preceding letter of the English alphabet, where 'a' is replaced by 'z'. For example, "bcd" can be left-shifted to "abc" or "yza" can be left-shifted to "xyz".

We can keep shifting the string in both directions to form an endless shifting sequence.

- For example, shift "abc" to form the sequence: ... <-> "abc" <-> "bcd" <-> ... <-> "xyz" <-> "yza" <-> <-> "zab" <-> "abc" <-> ...

You are given an array of strings `strings`, group together all `strings[i]` that belong to the same shifting sequence. You may return the answer in any order.

Example 1:

Input: `strings =`

`["abc","bcd","acef","xyz","az","ba","a","z"]`

Output:

`[["acef"],["a","z"],["abc","bcd","xyz"],["az","ba"]]`

Example 2:

Input: `strings = ["a"]`

Output: `[["a"]]`

Constraints:

- $1 \leq \text{strings.length} \leq 200$
- $1 \leq \text{strings}[i].\text{length} \leq 50$
- `strings[i]` consists of lowercase English letters.

Community Solutions (Python)

- WalkCC

Round 2 · High · Medium

p.459

```
class Solution:
    def groupStrings(self, strings: list[str]) -> list[list[str]]:
        keyToStrings = collections.defaultdict(list)

        def getKey(s: str) -> str:
            """
            Returns the key of 's' by pairwise calculation of differences.
            e.g. getKey("abc") -> "1,1" because diff(a, b) = 1 and diff(b, c) = 1.
            """
            diffs = []

            for i in range(1, len(s)):
                diff = (ord(s[i]) - ord(s[i - 1]) + 26) % 26
                diffs.append(str(diff))

            return ','.join(diffs)

        for s in strings:
            keyToStrings[getKey(s)].append(s)

        return keyToStrings.values()
```

• LeetCode

```
class Solution:
    def groupStrings(self, strings: List[str]) -> List[List[str]]:
        g = defaultdict(list)
        for s in strings:
            diff = ord(s[0]) - ord("a")
            t = []
            for c in s:
                c = ord(c) - diff
                if c < ord("a"):
                    c += 26
                t.append(chr(c))
            g["".join(t)].append(s)
        return list(g.values())
```

• SimplyLeet

Round 2 · High · Medium

p.460

```
# Python solution for grouping shifted strings

def group_shifted_strings(strings):
    # Helper function to generate a unique key for each string based on
    # differences
    def get_key(s):
        # Single character strings return a fixed key
        if len(s) == 1:
            return "0"
        # Compute differences modulo 26 for each adjacent characters
        key = []

        for i in range(1, len(s)):
            diff = (ord(s[i]) - ord(s[i-1]) + 26) % 26
            key.append(str(diff))

        # Use tuple interleaving or join to form a string key
        return ','.join(key)

    groups = {}
    # Group strings by their computed key
    for s in strings:
        key = get_key(s)

        if key not in groups:
            groups[key] = []
        groups[key].append(s)

    # Return grouped strings as a list of lists
    return list(groups.values())

# Example usage:
print(group_shifted_strings(["abc","bcd","acef","xyz","az","ba","a","z"]))
```

• LC.ca

Round 2 · High · Medium

p.461

```
'''
>>> a = defaultdict(list)
>>> b = defaultdict([])
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: first argument must be callable or None

>>> ord('c')
99

>>> ord('c') - ord('a')
2
>>> ord('d') - 2
98
>>> chr(98)
'b'

>>> a = {'s':1, 't':2, 'x':3}
>>> a

{'s': 1, 't': 2, 'x': 3}
>>> a.items()
dict_items([('s', 1), ('t', 2), ('x', 3)])
>>> a.values()
dict_values([1, 2, 3])
>>> list(a.values())
[1, 2, 3]
'''

from collections import defaultdict
```

```
class Solution:
    def groupStrings(self, strings: List[str]) -> List[List[str]]:
        mp = defaultdict(list)
        for s in strings:
            t = []
            diff = ord(s[0]) - ord('a')
            for c in s:
                d = ord(c) - diff
                if d < ord('a'):
                    continue
```

Round 2 · High · Medium

p.462

```

        d += 26
        t.append(chr(d))
    k = ''.join(t)
    mp[k].append(s)
    return list(mp.values())

```

◆ Quick Review

Key Insights

- All strings in the same group have the same relative differences between adjacent characters.
- Normalize each string's pattern by converting it to a key representation based on the differences between consecutive letters.
- A single character string forms its own group as there are no differences to compute.

Space & Time

Time Complexity: $O(n * m)$ where n is the number of strings and m is the average length of the strings (to compute each string's key).

Space Complexity: $O(n * m)$ to store the keys and grouping in the hash table.

Solution

The solution involves iterating over each string and generating a unique key that

Round 2 · High · Medium

p.463

represents the relative differences between adjacent characters. This key is calculated by computing the difference (mod 26) between every consecutive pair in the string. Strings that share the same key can be grouped together since they can be shifted into one another. A hash table (or dictionary) is used to store and group the strings by their computed key. This approach handles edge cases like single-character strings separately.

Round 2 · High · Medium

p.464

Hash Tables

#535 Encode and Decode TinyURL

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Design · Hash Function
Lists: AlgoMaster 600

Problem

TinyURL is a URL shortening service where you enter a URL such as `https://leetcode.com/problems/design-tinyurl` and it returns a short URL such as `http://tinyurl.com/4e9iAk`. Design a class to encode a URL and decode a tiny URL.

There is no restriction on how your encode/decode algorithm should work. You just need to ensure that a URL can be encoded to a tiny URL and the tiny URL can be decoded to the original URL.

Implement the Solution class:

- `Solution()` Initializes the object of the system.

Round 2 · High · Medium

p.465

- `String encode(String longUrl)` Returns a tiny URL for the given `longUrl`.
- `String decode(String shortUrl)` Returns the original long URL for the given `shortUrl`. It is guaranteed that the given `shortUrl` was encoded by the same object.

Example 1:

```
Input: url = "https://leetcode.com/problems/design-tinyurl"
Output: "https://leetcode.com/problems/design-tinyurl"
```

Explanation:

```
Solution obj = new Solution();
string tiny = obj.encode(url); // returns the encoded tiny url.
string ans = obj.decode(tiny); // returns the original url after decoding it.
```

Constraints:

- $1 \leq \text{url.length} \leq 104$
- `url` is guaranteed to be a valid URL.

Community Solutions (Python)

• WalkCC

```
class Codec:
    alphabets = string.ascii_letters + '0123456789'
    urlToCode = {}
    codeToUrl = {}

    def encode(self, longUrl: str) -> str:
        while longUrl not in self.urlToCode:
            code = ''.join(random.choice(self.alphabets) for _ in range(6))
            if code not in self.codeToUrl:
                self.codeToUrl[code] = longUrl
```

Round 2 · High · Medium

p.466

```

        self.urlToCode[longUrl] = code
    return 'http://tinyurl.com/' + self.urlToCode[longUrl]

def decode(self, shortUrl: str) -> str:
    return self.codeToUrl[shortUrl[-6:]]

```

• LeetCode

```

class Codec:
    def __init__(self):
        self.m = defaultdict()
        self.idx = 0
        self.domain = 'https://tinyurl.com/'

    def encode(self, longUrl: str) -> str:
        """Encodes a URL to a shortened URL."""
        self.idx += 1
        self.m[str(self.idx)] = longUrl

    def decode(self, shortUrl: str) -> str:
        """Decodes a shortened URL to its original URL."""
        idx = shortUrl.split('/')[1]
        return self.m[idx]

# Your Codec object will be instantiated and called as such:
# codec = Codec()

# codec.decode(codec.encode(url))

```

• SimplyLeet

```

import string
import random

class Solution:
    def __init__(self):
        # Dictionary to map short code to long URL
        self.url_map = {}
        # Base URL for the tiny URL
        self.base_url = "http://tinyurl.com/"

    def encode(self, longUrl: str) -> str:
        # Generate a random 6-character string
        code = ''.join(random.choices(string.ascii_letters + string.digits, k=6))

        # Ensure the code is unique
        while code in self.url_map:
            code = ''.join(random.choices(string.ascii_letters + string.digits, k=6))

        # Map the code to the long URL
        self.url_map[code] = longUrl
        # Return the complete tiny URL
        return self.base_url + code

    def decode(self, shortUrl: str) -> str:
        # Extract code from the short URL by removing the base URL prefix
        code = shortUrl.split(self.base_url)[-1]
        # Return the original long URL
        return self.url_map.get(code, "")

```

Hash Tables

☐ #659 Split Array into Consecutive Subsequences

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode Array · Hash Table · Greedy · Heap (Priority Queue)

Lists: AlgoMaster 600

Problem

You are given an integer array `nums` that is sorted in non-decreasing order.

Determine if it is possible to split `nums` into one or more subsequences such that both of the following conditions are true:

- Each subsequence is a consecutive increasing sequence (i.e. each integer is exactly one more than the previous integer).
- All subsequences have a length of 3 or more.

Return `true` if you can split `nums` according to the above conditions, or `false` otherwise.

A subsequence of an array is a new array that is formed from the original array by deleting some

Round 2 · High · Medium

p.469

(can be none) of the elements without disturbing the relative positions of the remaining elements. (i.e., `[1,3,5]` is a subsequence of `[1,2,3,4,5]` while `[1,3,2]` is not).

Example 1:

```
Input: nums = [1,2,3,3,4,5]
Output: true
Explanation: nums can be split into the following subsequences:
[1,2,3,3,4,5] --> 1, 2, 3
[1,2,3,3,4,5] --> 3, 4, 5
```

Example 2:

```
Input: nums = [1,2,3,3,4,4,5,5]
Output: true
Explanation: nums can be split into the following subsequences:
[1,2,3,3,4,4,5,5] --> 1, 2, 3, 4, 5
[1,2,3,3,4,4,5,5] --> 3, 4, 5
```

Example 3:

```
Input: nums = [1,2,3,4,4,5]
Output: false
Explanation: It is impossible to split nums into consecutive increasing subsequences of length 3 or more.
```

Constraints:

- $1 \leq \text{nums.length} \leq 104$
- $-1000 \leq \text{nums}[i] \leq 1000$
- `nums` is sorted in non-decreasing order.

Community Solutions (Python)

- LeetDoocs

Round 2 · High · Medium

p.470

```
class Solution:
    def isPossible(self, nums: List[int]) -> bool:
        d = defaultdict(list)
        for v in nums:
            if h := d[v - 1]:
                heappush(d[v], heappop(h) + 1)
            else:
                heappush(d[v], 1)
        return all(not v or v and v[0] > 2 for v in d.values())
```

• SimplyLeet

```
# Python solution for splitting array into consecutive subsequences

from collections import Counter

def isPossible(nums):
    # Frequency map for all numbers in nums
    freq = Counter(nums)
    # Map to track the number of subsequences that can be extended (ending with
    a number)
    appendfreq = Counter()

    for num in nums:
        if freq[num] == 0:
            # This number has already been used completely, skip it
            continue

        # Decrement the frequency of current num as we are going to use it
        freq[num] -= 1

        if appendfreq[num] > 0:
            # If there's a subsequence ending with num-1, extend this
            subsequence with num
```

```
        appendfreq[num] -= 1
        # Now, this subsequence can be extended with num+1 in the future
        appendfreq[num + 1] += 1
    elif freq[num + 1] > 0 and freq[num + 2] > 0:
        # If num cannot extend a subsequence, try to create a new
        subsequence of length at least 3
        freq[num + 1] -= 1
        freq[num + 2] -= 1
        # The new subsequence now can be extended with num+3
        appendfreq[num + 3] += 1
    else:
        # Neither extending nor starting a new valid subsequence is
        possible
        return False
    return True

# Example usage:
print(isPossible([1,2,3,3,4,5])) # Expected output: True
```


Hash Tables

#767 Reorganize String

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Hash Table · String · Greedy · Sorting · Heap
 (Priority Queue) · Counting
 Lists: AlgoMaster 600

Problem

Given a string s , rearrange the characters of s so that any two adjacent characters are not the same.

Return any possible rearrangement of s or return "" if not possible.

Example 1:

Input: $s = \text{"aab"}$
 Output: "aba"

Example 2:

Input: $s = \text{"aaab"}$
 Output: ""

Constraints:

- $1 \leq s.length \leq 500$
- s consists of lowercase English letters.

Round 2 · High · Medium

p.473

Community Solutions (Python)

• WalkCC

```
class Solution:
    def reorganizeString(self, s: str) -> str:
        count = collections.Counter(s)
        if max(count.values()) > (len(s) + 1) // 2:
            return ''

        ans = []
        maxHeap = [(-freq, c) for c, freq in count.items()]
        heapq.heapify(maxHeap)
        prevFreq = 0

        prevChar = '@'

        while maxHeap:
            # Get the letter with the maximum frequency.
            freq, c = heapq.heappop(maxHeap)
            ans.append(c)
            # Add the previous letter back s.t. any two adjacent characters are not
            # the same.
            if prevFreq < 0:
                heapq.heappush(maxHeap, (prevFreq, prevChar))

            prevFreq = freq + 1
            prevChar = c

        return ''.join(ans)

class Solution:
    def reorganizeString(self, s: str) -> str:
        n = len(s)
        count = collections.Counter(s)
        maxCount = max(count.values())

        if maxCount > (n + 1) // 2:
            return ''

        if maxCount == (n + 1) // 2:
```

Round 2 · High · Medium

p.474

```

maxLetter = max(count, key=count.get)
ans = [maxLetter if i % 2 == 0 else '' for i in range(n)]
del count[maxLetter]
i = 1
else:
    ans = [''] * n
    i = 0

for c, freq in count.items():
    for _ in range(freq):

        ans[i] = c
        i += 2
        if i >= n:
            i = 1

return ''.join(ans)

```

• LeetCode

```

class Solution:
    def reorganizeString(self, s: str) -> str:
        n = len(s)
        cnt = Counter(s)
        mx = max(cnt.values())
        if mx > (n + 1) // 2:
            return ''

        i = 0
        ans = [None] * n
        for k, v in cnt.most_common():

            while v:
                ans[i] = k
                v -= 1
                i += 2
                if i >= n:
                    i = 1
            return ''.join(ans)

```

Round 2 · High · Medium

p.475

```

class Solution:
    def reorganizeString(self, s: str) -> str:
        return self.rearrangeString(s, 2)

    def rearrangeString(self, s: str, k: int) -> str:
        h = [(-v, c) for c, v in Counter(s).items()]
        heapify(h)
        q = deque()
        ans = []
        while h:
            v, c = heappop(h)
            v *= -1
            ans.append(c)
            q.append((-v - 1, c))
            if len(q) >= k:
                w, c = q.popleft()
                if w:
                    heappush(h, (-w, c))
        return "" if len(ans) != len(s) else "".join(ans)

```

• SimplyLeet

```

# Python solution using heapq. The heap is simulated as a max heap by storing
negative frequencies.
import heapq

def reorganizeString(s: str) -> str:
    # Count frequency of each character.
    frequency = {}
    for char in s:
        frequency[char] = frequency.get(char, 0) + 1

    # Check if reorganization is possible.

```

Round 2 · High · Medium

p.476

```

max_allowed = (len(s) + 1) // 2
for count in frequency.values():
    if count > max_allowed:
        return ""

# Build a max heap based on frequency.
max_heap = [(-count, char) for char, count in frequency.items()]
heapq.heapify(max_heap)

result = []

prev_count, prev_char = 0, ''

# Process the heap.
while max_heap:
    count, char = heapq.heappop(max_heap)
    # Append the current character to the result.
    result.append(char)

    # If there is a previous character leftover, push it back into the
    heap.
    if prev_count < 0:
        heapq.heappush(max_heap, (prev_count, prev_char))

    # Update previous character info: decrement the count.
    count += 1 # since count is negative, incrementing moves it towards
    zero.
    prev_count, prev_char = count, char

return "".join(result)

# Example usage:
print(reorganizeString("aab")) # Expected Output: "aba"

print(reorganizeString("aaab")) # Expected Output: ""

```

Round 2 · High · Medium

p.477

Hash Tables

□ #792 Number of Matching Subsequences **MED**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table · String · Binary Search ·
 Dynamic Programming · Trie · Sorting
 Lists: AlgoMaster 600

Problem

Given a string *s* and an array of strings *words*, return the number of *words[i]* that is a subsequence of *s*.

A subsequence of a string is a new string generated from the original string with some characters (can be none) deleted without changing the relative order of the remaining characters.

- For example, "ace" is a subsequence of "abcde".

Example 1:

Input: *s* = "abcde", *words* = ["a","bb","acd","ace"]
 Output: 3
 Explanation: There are three strings in *words* that are a subsequence of *s*: "a", "acd", "ace".

Example 2:

Round 2 · High · Medium

p.478

Input: s = "dsahjppjauf", words = ["ahjppjau","ja","ahbwzggqnuk","tnmlanowax"]
Output: 2

Constraints:

- $1 \leq s.length \leq 5 * 10^4$
- $1 \leq words.length \leq 5000$
- $1 \leq words[i].length \leq 50$
- s and words[i] consist of only lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def numMatchingSubseq(self, s: str, words: list[str]) -> int:
        root = {}

        def insert(word: str) -> None:
            node = root
            for c in word:
                if c not in node:
                    node[c] = {'count': 0}
                node = node[c]

            node['count'] += 1

        for word in words:
            insert(word)

        def dfs(s: str, i: int, node: dict) -> int:
            ans = node['count'] if 'count' in node else 0

            if i >= len(s):
                return ans
```

```
for c in string.ascii_lowercase:
    if c in node:
        try:
            index = s.index(c, i)
            ans += dfs(s, index + 1, node[c])
        except ValueError:
            continue
```

```
return ans
```

```
return dfs(s, 0, root)
```

```
class Solution:
    def numMatchingSubseq(self, s: str, words: list[str]) -> int:
        ans = 0
        # [(i, j)] := words[i] and the letter words[i][j] is waiting for
        bucket = [[] for _ in range(26)]

        # For each word, it's waiting for word[0].
        for i, word in enumerate(words):
            bucket[ord(word[0]) - ord('a')].append((i, 0))
```

```
for c in s:
    # Let prevBucket = bucket[c] and clear bucket[c].
    index = ord(c) - ord('a')
    prevBucket = bucket[index]
    bucket[index] = []
    for i, j in prevBucket:
        j += 1
        if j == len(words[i]): # All the letters in words[i] are matched.
            ans += 1
        else:
```

```
        bucket[ord(words[i][j]) - ord('a')].append((i, j))
```

```
return ans
```

• LeetCode

Round 2 · High · Medium

p.479

Round 2 · High · Medium

p.480

```
class Solution:
    def numMatchingSubseq(self, s: str, words: List[str]) -> int:
        d = defaultdict(deque)
        for w in words:
            d[w[0]].append(w)
        ans = 0
        for c in s:
            for _ in range(len(d[c])):
                t = d[c].popleft()
                if len(t) == 1:
                    ans += 1
            else:
                d[t[1]].append(t[1:])
        return ans
```

```
class Solution:
    def numMatchingSubseq(self, s: str, words: List[str]) -> int:
        def check(w):
            i = -1
            for c in w:
                j = bisect_right(d[c], i)
                if j == len(d[c]):
                    return False
                i = d[c][j]
            return True
        d = defaultdict(list)
        for i, c in enumerate(s):
            d[c].append(i)
        return sum(check(w) for w in words)
```

• SimplyLeet

Round 2 · High · Medium

p.481

```
# Python solution with line-by-line comments
from collections import defaultdict, deque

def numMatchingSubseq(s, words):
    # Dictionary mapping from character to a deque of iterators over words
    waiting = defaultdict(deque)

    # Initialize waiting lists with an iterator for each word starting with its
    # first character
    for word in words:
        # Append the word itself as the sequence, waiting for its first
        # character (or next character pointer)
        waiting[word[0]].append(word)

    count = 0 # To count the number of words that are subsequences
    # Iterate over each character in s
    for char in s:
        # Access the list of words waiting for the current character
        current_bucket = waiting[char]
        # Clear the bucket as we are going to process all the current waiting
        # words
        waiting[char] = deque()

        # Process each word that was waiting for the current character
        while current_bucket:
            word = current_bucket.popleft()
            # If the word length is 1, then current character was the final
            # needed character
            if len(word) == 1:
                count += 1 # Word is fully matched
            else:
                # Move to the next character in the word and add it to the
                # appropriate bucket
                waiting[word[1]].append(word[1:])
```

Round 2 · High · Medium

p.482

```
    return count

# Example usage:
s = "abcde"
words = ["a", "bb", "acd", "ace"]
print(numMatchingSubseq(s, words)) # Expected output: 3
```

Hash Tables

☐ #1525 Number of Good Ways to Split a String

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Dynamic Programming ·
Bit Manipulation · Prefix Sum

Lists: AlgoMaster 600

Problem

You are given a string s .

A split is called good if you can split s into two non-empty strings s_{left} and s_{right} where their concatenation is equal to s (i.e., $s_{left} + s_{right} = s$) and the number of distinct letters in s_{left} and s_{right} is the same.

Return the number of good splits you can make in s .

Example 1:

Input: s = "aacaba"
 Output: 2
 Explanation: There are 5 ways to split "aacaba" and 2 of them are good.
 ("a", "acaba") Left string and right string contains 1 and 3 different letters respectively.
 ("aa", "caba") Left string and right string contains 1 and 3 different letters respectively.
 ("aac", "aba") Left string and right string contains 2 and 2 different letters respectively (good split).
 ("aaca", "ba") Left string and right string contains 2 and 2 different letters respectively (good split).
 ("aacab", "a") Left string and right string contains 3 and 1 different letters respectively.

Example 2:

Input: s = "abcd"
 Output: 1
 Explanation: Split the string as follows ("ab", "cd").

Constraints:

- $1 \leq s.length \leq 105$
- s consists of only lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def numSplits(self, s: str) -> int:
        n = len(s)
        ans = 0
        seen = set()
        prefix = [0] * n
        suffix = [0] * n

        for i in range(n):
            seen.add(s[i])
```

Round 2 · High · Medium

p.485

```
        prefix[i] = len(seen)

        seen.clear()

        for i in reversed(range(n)):
            seen.add(s[i])
            suffix[i] = len(seen)

        for i in range(n - 1):
            if prefix[i] == suffix[i + 1]:
                ans += 1

        return ans
```

• LeetCode

```
class Solution:
    def numSplits(self, s: str) -> int:
        cnt = Counter(s)
        vis = set()
        ans = 0
        for c in s:
            vis.add(c)
            cnt[c] -= 1
            if cnt[c] == 0:
                cnt.pop(c)

        ans += len(vis) == len(cnt)
        return ans
```

• SimplyLeet

Round 2 · High · Medium

p.486

```

# Python solution
def numSplits(s: str) -> int:
    n = len(s)

    # leftDistinct[i] represents the number of unique characters in s[:i+1]
    leftDistinct = [0] * n
    seen_left = set()

    for i in range(n):
        seen_left.add(s[i])

        leftDistinct[i] = len(seen_left)

    # rightDistinct[i] represents the number of unique characters in s[i:]
    rightDistinct = [0] * n
    seen_right = set()

    for i in range(n-1, -1, -1):
        seen_right.add(s[i])
        rightDistinct[i] = len(seen_right)

    goodSplits = 0
    # Iterate through all possible split points
    for i in range(n-1):
        if leftDistinct[i] == rightDistinct[i+1]:
            goodSplits += 1

    return goodSplits

# For testing
if __name__ == "__main__":

    test_string = "aacaba"
    print(numSplits(test_string)) # Expected output: 2

```

Hash Tables

☐ #1647 Minimum Deletions to Make Character Frequencies Unique

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode Hash Table · String · Greedy · Sorting Lists: AlgoMaster 600

Problem

A string s is called good if there are no two different characters in s that have the same frequency.

Given a string s , return the minimum number of characters you need to delete to make s good.

The frequency of a character in a string is the number of times it appears in the string. For example, in the string "aab", the frequency of 'a' is 2, while the frequency of 'b' is 1.

Example 1:

```

Input: s = "aab"
Output: 0
Explanation: s is already good.

```

Example 2:

Input: s = "aaabbbcc"

Output: 2

Explanation: You can delete two 'b's resulting in the good string "aaabcc".
Another way it to delete one 'b' and one 'c' resulting in the good string "aaabbc".

Example 3:

Input: s = "ceabaacb"

Output: 2

Explanation: You can delete both 'c's resulting in the good string "eabaab".
Note that we only care about characters that are still in the string at the end (i.e. frequency of 0 is ignored).

Constraints:

- $1 \leq s.length \leq 105$
- s contains only lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minDeletions(self, s: str) -> int:
        ans = 0
        count = collections.Counter(s)
        usedFreq = set()

        for freq in count.values():
            while freq > 0 and freq in usedFreq:
                freq -= 1 # Delete ('a' + i).
                ans += 1

            usedFreq.add(freq)

        return ans
```

• LeetDoocs

```
class Solution:
    def minDeletions(self, s: str) -> int:
        cnt = Counter(s)
        ans, pre = 0, inf
        for v in sorted(cnt.values(), reverse=True):
            if pre == 0:
                ans += v
            elif v >= pre:
                ans += v - pre + 1
                pre -= 1
            else:
                pre = v
        return ans
```

```
class Solution:
    def minDeletions(self, s: str) -> int:
        cnt = Counter(s)
        vals = sorted(cnt.values(), reverse=True)
        ans = 0
        for i in range(1, len(vals)):
            while vals[i] >= vals[i - 1] and vals[i] > 0:
                vals[i] -= 1
                ans += 1
        return ans
```

• SimplyLeet

```
def minDeletions(s: str) -> int:
    # Count the frequency of each character in the string.
    from collections import Counter
    freq_counter = Counter(s)

    # Initialize a set to track frequencies that have been used.
    used_frequencies = set()
    deletions = 0

    # Iterate over each character's frequency.
```

```
for char, freq in freq_counter.items():
    # Adjust the frequency until it becomes unique (or zero)
    while freq > 0 and freq in used_frequencies:
        freq -= 1 # simulate deletion of a character
        deletions += 1
    # Add the (now unique) frequency to the set if it is positive.
    used_frequencies.add(freq)

return deletions
```

```
# Example usage:
print(minDeletions("aaabbbcc")) # Output: 2
```

Round 2 · High · Medium

p.491

Two Pointers

Round 2 · High · Medium · 7 questions

Round 2 · High · Medium

p.492

Two Pointers

#5 Longest Palindromic Substring

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming
Lists: Grind 169

Problem

Given a string *s*, return the longest palindromic substring in *s*.

Example 1:

Input: *s* = "babad"
Output: "bab"
Explanation: "aba" is also a valid answer.

Example 2:

Input: *s* = "cbabd"
Output: "bb"

Constraints:

- $1 \leq s.length \leq 1000$
- *s* consist of only digits and English letters.

Community Solutions (Python)

- WalkCC

Round 2 · High · Medium

p.493

```
class Solution:
    def longestPalindrome(self, s: str) -> str:
        if not s:
            return ''

        # (start, end) indices of the longest palindrome in s
        indices = [0, 0]

        def extend(s: str, i: int, j: int) -> tuple[int, int]:
            while i >= 0 and j < len(s):
                if s[i] != s[j]:
                    break
                i -= 1
                j += 1
            return i + 1, j - 1

        Returns the (start, end) indices of the longest palindrome extended from
        the substring s[i..j].
        while i >= 0 and j < len(s):
            if s[i] != s[j]:
                break
            i -= 1
            j += 1
            return i + 1, j - 1

        for i in range(len(s)):
            l1, r1 = extend(s, i, i)
            if r1 - l1 > indices[1] - indices[0]:
                indices = l1, r1
            if i + 1 < len(s) and s[i] == s[i + 1]:
                l2, r2 = extend(s, i, i + 1)
                if r2 - l2 > indices[1] - indices[0]:
                    indices = l2, r2

        return s[indices[0]:indices[1] + 1]

class Solution:
    def longestPalindrome(self, s: str) -> str:
        t = '#'.join('@' + s + '$')
        p = self._manacher(t)
        maxPalindromeLength, bestCenter = max((extend, i)
                                              for i, extend in enumerate(p))

        l = (bestCenter - maxPalindromeLength) // 2
        r = (bestCenter + maxPalindromeLength) // 2
        return s[l:r]
```

Round 2 · High · Medium

p.494

```
def _manacher(self, t: str) -> list[int]:
    """
    Returns an array `p` s.t. `p[i]` is the length of the longest palindrome
    centered at `t[i]`, where `t` is a string with delimiters and sentinels.
    """
    p = [0] * len(t)
    center = 0
    for i in range(1, len(t) - 1):
        rightBoundary = center + p[center]
        mirrorIndex = center - (i - center)

        if rightBoundary > i:
            p[i] = min(rightBoundary - i, p[mirrorIndex])
            # Try to expand the palindrome centered at i.
            while t[i + 1 + p[i]] == t[i - 1 - p[i]]:
                p[i] += 1
            # If a palindrome centered at i expands past `rightBoundary`, adjust
            # the center based on the expanded palindrome.
            if i + p[i] > rightBoundary:
                center = i
    return p
```

• LeetCode

```
class Solution:
    def longestPalindrome(self, s: str) -> str:
        n = len(s)
        f = [[True] * n for _ in range(n)]
        k, mx = 0, 1
        for i in range(n - 2, -1, -1):
            for j in range(i + 1, n):
                f[i][j] = False
                if s[i] == s[j]:
                    f[i][j] = f[i + 1][j - 1]

            if f[i][j] and mx < j - i + 1:
                k, mx = i, j - i + 1
        return s[k : k + mx]
```

Round 2 · High · Medium

p.495

```
class Solution:
    def longestPalindrome(self, s: str) -> str:
        def f(l, r):
            while l >= 0 and r < n and s[l] == s[r]:
                l, r = l - 1, r + 1
            return r - l - 1

        n = len(s)
        start, mx = 0, 1
        for i in range(n):
            a = f(i, i)
            b = f(i, i + 1)
            t = max(a, b)
            if mx < t:
                mx = t
                start = i - ((t - 1) >> 1)
        return s[start : start + mx]
```

• SimplyLeet

```
# Function to expand from the center and return the palindrome substring
def expand_from_center(s, left, right):
    # Expand while the characters on both sides are the same
    while left >= 0 and right < len(s) and s[left] == s[right]:
        left -= 1
        right += 1
    # Return the palindromic substring found after expansion
    return s[left + 1:right]

def longest_palindromic_substring(s):
    if not s:
        return ""
    longest = ""
    # Iterate over each index in string as potential center
    for i in range(len(s)):
        # Check for odd-length palindrome
        odd_palindrome = expand_from_center(s, i, i)
        if len(odd_palindrome) > len(longest):
            longest = odd_palindrome
        # Check for even-length palindrome
```

Round 2 · High · Medium

p.496

```

even_palindrome = expand_from_center(s, i, i + 1)
if len(even_palindrome) > len(longest):
    longest = even_palindrome
return longest

```

Example usage

```
print(longest_palindromic_substring("babad"))
```

• LC.ca

```

class Solution:
    def longestPalindrome(self, s: str) -> str:
        mlen = 0
        start = end = 0
        n = len(s)
        dp = [ [False] * n for i in range(n) ]

        for j in range(n):
            for i in range(j + 1):
                dp[i][j] = (i == j) or (s[i] == s[j] and j - i == 1) or (s[i]
== s[j] and dp[i + 1][j - 1])

```

```

        if dp[i][j] is True and j - i + 1 > mlen:
            mlen = j - i + 1
            start = i
            end = j

```

```
return s[start: end + 1]
```

#####

```
class Solution:
```

```

def longestPalindrome(self, s: str) -> str:
    n = len(s)
    f = [[True] * n for _ in range(n)]
    k, mx = 0, 1
    for i in range(n - 2, -1, -1):
        for j in range(i + 1, n):
            f[i][j] = False
            if s[i] == s[j]:
                f[i][j] = f[i + 1][j - 1]
                if f[i][j] and mx < j - i + 1:

```

Round 2 · High · Medium

p.497

```

        k, mx = i, j - i + 1
    return s[k : k + mx]

```

◆ Quick Review

Key Insights

- Every palindrome is centered around a character (for odd-length) or a pair of characters (for even-length).
- Expand from each center until the substring is no longer a palindrome.
- Update the longest palindrome when a longer one is found during the expansion.
- The solution employs a two-pointer technique without extra storage for dynamic programming tables.

Space & Time

Time Complexity: $O(n^2)$ in the worst case as we expand around each center.

Space Complexity: $O(1)$, excluding the space required for the output substring.

Solution

The approach is based on expanding around potential centers of palindromes. For each

Round 2 · High · Medium

p.498

character in the string, consider it as the center for an odd-length palindrome and the gap between it and the next character for an even-length palindrome. Using two pointers, expand outward until the characters differ. Track the longest palindrome found during these expansions. This method is efficient given the input constraints, and no extra data structures are needed besides variables to store indices or substrings.

Two Pointers

☐ #11 Container With Most Water

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Two Pointers · Greedy
 Lists: Grind 169

Problem

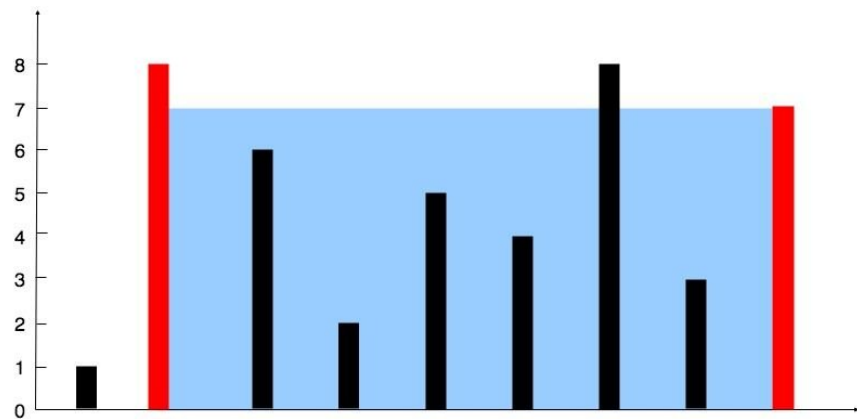
You are given an integer array `height` of length `n`. There are `n` vertical lines drawn such that the two endpoints of the `i`th line are $(i, 0)$ and $(i, \text{height}[i])$.

Find two lines that together with the `x`-axis form a container, such that the container contains the most water.

Return the maximum amount of water a container can store.

Notice that you may not slant the container.

Example 1:



Input: height = [1,8,6,2,5,4,8,3,7]

Output: 49

Explanation: The above vertical lines are represented by array [1,8,6,2,5,4,8,3,7]. In this case, the max area of water (blue section) the container can contain is 49.

Example 2:

Input: height = [1,1]

Output: 1

Constraints:

- $n == \text{height.length}$
- $2 \leq n \leq 105$
- $0 \leq \text{height}[i] \leq 104$

Community Solutions (Python)

- WalkCC

```
class Solution:
    def maxArea(self, height: list[int]) -> int:
        ans = 0
        l = 0
        r = len(height) - 1

        while l < r:
            minHeight = min(height[l], height[r])
            ans = max(ans, minHeight * (r - l))
            if height[l] < height[r]:
                l += 1
            else:
                r -= 1

        return ans
```

• LeetCode

```
class Solution:
    def maxArea(self, height: List[int]) -> int:
        l, r = 0, len(height) - 1
        ans = 0
        while l < r:
            t = min(height[l], height[r]) * (r - l)
            ans = max(ans, t)
            if height[l] < height[r]:
                l += 1
            else:
                r -= 1

        return ans
```

• SimplyLeet

```
# Function to calculate the maximum water container area
def maxArea(height):
    left, right = 0, len(height) - 1 # Initialize two pointers
    max_water = 0 # Variable to store the maximum area

    # Loop until pointers meet
    while left < right:
        # Calculate current area based on smaller height and distance between
        # pointers
        current_area = min(height[left], height[right]) * (right - left)
        max_water = max(max_water, current_area) # Update maximum area if
        # current is larger

        # Move the pointer corresponding to the smaller height
        if height[left] < height[right]:
            left += 1
        else:
            right -= 1
    return max_water

# Test the function with an example
if __name__ == "__main__":
    input_heights = [1,8,6,2,5,4,8,3,7]
    print(maxArea(input_heights)) # Expected output: 49
```

• LC.ca

```
class Solution:
    def maxArea(self, height: List[int]) -> int:
        i, j = 0, len(height) - 1
        ans = 0
        while i < j:
            t = (j - i) * min(height[i], height[j])
            ans = max(ans, t)
            if height[i] < height[j]:
                i += 1
            else:
                j -= 1
        return ans
```

Round 2 · High · Medium

p.503

◆ Quick Review

Key Insights

- The container's area is determined by the distance between two lines multiplied by the shorter line's height.
- A brute-force approach checking all possible pairs would result in $O(n^2)$ time complexity, which is inefficient for large inputs.
- A two-pointer approach allows us to solve the problem in $O(n)$ time by initializing pointers at both ends and gradually moving them inward.
- By always moving the pointer corresponding to the shorter line, we are more likely to find a taller line that could potentially form a container with a larger capacity.

Space & Time

Time Complexity: $O(n)$ – The algorithm makes a single pass through the array with two pointers.

Space Complexity: $O(1)$ – Only a few extra variables are used, regardless of the input size.

Round 2 · High · Medium

p.504

Solution

The optimal approach uses a two-pointer strategy:

- Start with two pointers at each end of the array.
- Calculate the area formed between the two lines at these pointers.
- Update the maximum area if the calculated value is larger.
- Move the pointer pointing to the shorter line inward, as this is the only possible way to potentially increase the height of the shorter line and thus the area.
- Continue until the two pointers meet.

This method effectively reduces the number of comparisons and ensures that every possible pair that can yield a larger area is considered.

Round 2 · High · Medium

p.505

Two Pointers

□ #15 3Sum

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Sorting
Lists: Grind 169

Problem

Given an integer array `nums`, return all the triplets `[nums[i], nums[j], nums[k]]` such that $i \neq j$, $i \neq k$, and $j \neq k$, and $nums[i] + nums[j] + nums[k] == 0$.

Notice that the solution set must not contain duplicate triplets.

Example 1:

```
Input: nums = [-1,0,1,2,-1,-4]
Output: [[-1,-1,2],[-1,0,1]]
Explanation:
nums[0] + nums[1] + nums[2] = (-1) + 0 + 1 = 0.
nums[1] + nums[2] + nums[4] = 0 + 1 + (-1) = 0.
nums[0] + nums[3] + nums[4] = (-1) + 2 + (-1) = 0.
The distinct triplets are [-1,0,1] and [-1,-1,2].
Notice that the order of the output and the order of the triplets does not matter.
```

Example 2:

```
Input: nums = [0,1,1]
Output: []
Explanation: The only possible triplet does not sum up to 0.
```

Round 2 · High · Medium

p.506

Example 3:

Input: nums = [0,0,0]

Output: [[0,0,0]]

Explanation: The only possible triplet sums up to 0.

Constraints:

- $3 \leq \text{nums.length} \leq 3000$
- $-105 \leq \text{nums}[i] \leq 105$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def threeSum(self, nums: List[int]) -> List[List[int]]:
        if len(nums) < 3:
            return []

        ans = []

        nums.sort()

        for i in range(len(nums) - 2):

            if i > 0 and nums[i] == nums[i - 1]:
                continue
            # Choose nums[i] as the first number in the triplet, then search the
            # remaining numbers in [i + 1, n - 1].
            l = i + 1
            r = len(nums) - 1
            while l < r:
                summ = nums[i] + nums[l] + nums[r]
                if summ == 0:
                    ans.append((nums[i], nums[l], nums[r]))
```

Round 2 · High · Medium

p.507

```
l += 1
r -= 1
while nums[l] == nums[l - 1] and l < r:
    l += 1
while nums[r] == nums[r + 1] and l < r:
    r -= 1
elif summ < 0:
    l += 1
else:
    r -= 1
```

return ans

• LeetCode

```
class Solution:
    def threeSum(self, nums: List[int]) -> List[List[int]]:
        nums.sort()
        n = len(nums)
        ans = []
        for i in range(n - 2):
            if nums[i] > 0:
                break
            if i and nums[i] == nums[i - 1]:
                continue

            j, k = i + 1, n - 1
            while j < k:
                x = nums[i] + nums[j] + nums[k]
                if x < 0:
                    j += 1
                elif x > 0:
                    k -= 1
                else:
                    ans.append([nums[i], nums[j], nums[k]])
                    j, k = j + 1, k - 1

            while j < k and nums[j] == nums[j - 1]:
                j += 1
            while j < k and nums[k] == nums[k + 1]:
                k -= 1

        return ans
```

Round 2 · High · Medium

p.508

• SimplyLeet

```
# Function to find all unique triplets that sum to zero.
def threeSum(nums):
    # Sort the array to facilitate finding duplicates and using two pointers.
    nums.sort()
    result = []
    n = len(nums)

    # Iterate through the array, fixing one element at a time.
    for i in range(n - 2):
        # Skip duplicate elements for the first element.

        if i > 0 and nums[i] == nums[i - 1]:
            continue

        left, right = i + 1, n - 1
        while left < right:
            current_sum = nums[i] + nums[left] + nums[right]
            # Check if the triplet adds up to zero.
            if current_sum == 0:
                result.append([nums[i], nums[left], nums[right]])
                # Skip duplicates for the second element.

                while left < right and nums[left] == nums[left + 1]:
                    left += 1
                # Skip duplicates for the third element.
                while left < right and nums[right] == nums[right - 1]:
                    right -= 1
                left += 1
                right -= 1
            elif current_sum < 0:
                left += 1
            else:
                right -= 1

        right -= 1

    return result

# Example usage:
#print(threeSum([-1,0,1,2,-1,-4]))
```

• LC.ca

Round 2 · High · Medium

p.509

```
class Solution:
    def threeSum(self, nums: List[int]) -> List[List[int]]:
        nums.sort()
        n = len(nums)
        ans = []
        for i in range(n - 2):
            if nums[i] > 0:
                break
            if i and nums[i] == nums[i - 1]:
                continue

            j, k = i + 1, n - 1
            while j < k:
                x = nums[i] + nums[j] + nums[k]
                if x < 0:
                    j += 1
                elif x > 0:
                    k -= 1
                else:
                    ans.append([nums[i], nums[j], nums[k]])
                    j, k = j + 1, k - 1

                    while j < k and nums[j] == nums[j - 1]:
                        j += 1
                    while j < k and nums[k] == nums[k + 1]:
                        k -= 1

            return ans
```

◆ Quick Review

Key Insights

- Sort the array to efficiently skip duplicates and facilitate the two pointers approach.
- Use a fixed pointer for the first element, and then use two additional pointers (left and right) to find pairs that sum to the negation of

Round 2 · High · Medium

p.510

the fixed element.

- Skip duplicate elements to ensure unique triplets.

Space & Time

Time Complexity: $O(n^2)$ where n is the number of elements in the array.

Space Complexity: $O(1)$ extra space (ignoring the space required for the output).

Solution

We first sort the array to bring duplicates together and allow the two-pointer technique to work efficiently. Then, iterate over the array and for each element, use two pointers (one at the beginning right after the current element and one at the end of the array) to find a pair that sums with the current element to zero. Skip duplicates for the current element and within the inner loop to avoid duplicate triplets. This ensures that each valid triplet is unique and the overall runtime remains efficient.

Two Pointers

#18 4Sum

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Two Pointers · Sorting
 Lists: AlgoMaster 600

Problem

Given an array `nums` of n integers, return an array of all the unique quadruplets `[nums[a], nums[b], nums[c], nums[d]]` such that:

- $0 \leq a, b, c, d < n$
- $a, b, c,$ and d are distinct.
- $nums[a] + nums[b] + nums[c] + nums[d] == target$

You may return the answer in any order.

Example 1:

Input: `nums = [1,0,-1,0,-2,2]`, `target = 0`
 Output: `[[-2,-1,1,2], [-2,0,0,2], [-1,0,0,1]]`

Example 2:

Input: `nums = [2,2,2,2,2]`, `target = 8`
 Output: `[[2,2,2,2]]`

Constraints:

- $1 \leq nums.length \leq 200$

- $-109 \leq \text{nums}[i] \leq 109$
- $-109 \leq \text{target} \leq 109$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def fourSum(self, nums: list[int], target: int):
        ans = []

        def nSum(
            l: int, r: int, target: int, n: int, path: list[int],
            ans: list[list[int]]) -> None:
            """Finds n numbers that add up to the target in [l, r]."""
            if r - l + 1 < n or n < 2 or target < nums[l] * n or target > nums[r] *
n:
                return

            if n == 2:
                while l < r:
                    summ = nums[l] + nums[r]
                    if summ == target:
                        ans.append(path + [nums[l], nums[r]])
                        l += 1
                        while nums[l] == nums[l - 1] and l < r:
                            l += 1
                    elif summ < target:
                        l += 1
                    else:
                        r -= 1
                return

            for i in range(l, r + 1):
                if i > l and nums[i] == nums[i - 1]:
                    continue

                nSum(i + 1, r, target - nums[i], n - 1, path + [nums[i]], ans)
```

Round 2 · High · Medium

p.513

```
nums.sort()
nSum(0, len(nums) - 1, target, 4, [], ans)
return ans
```

• LeetCode

```
class Solution:
    def fourSum(self, nums: List[int], target: int) -> List[List[int]]:
        n = len(nums)
        ans = []
        if n < 4:
            return ans
        nums.sort()
        for i in range(n - 3):
            if i and nums[i] == nums[i - 1]:
                continue

            for j in range(i + 1, n - 2):
                if j > i + 1 and nums[j] == nums[j - 1]:
                    continue
                k, l = j + 1, n - 1
                while k < l:
                    x = nums[i] + nums[j] + nums[k] + nums[l]
                    if x < target:
                        k += 1
                    elif x > target:
                        l -= 1
                    else:
                        ans.append([nums[i], nums[j], nums[k], nums[l]])
                        k, l = k + 1, l - 1
                        while k < l and nums[k] == nums[k - 1]:
                            k += 1
                        while k < l and nums[l] == nums[l + 1]:
                            l -= 1

            return ans
```

• SimplyLeet

Round 2 · High · Medium

p.514

```

# Define a function to find all quadruplets that sum to the target.
def fourSum(nums, target):
    # Sort the input list to facilitate the two-pointer technique and duplicate
    # handling.
    nums.sort()
    quadruplets = []
    n = len(nums)

    # First loop: iterate through the list for the first element.
    for i in range(n - 3):
        # Skip duplicate values for the first element.

        if i > 0 and nums[i] == nums[i - 1]:
            continue
        # Second loop: iterate for the second element.
        for j in range(i + 1, n - 2):
            # Skip duplicate values for the second element.
            if j > i + 1 and nums[j] == nums[j - 1]:
                continue
            # Two pointers for the third and fourth element.
            left = j + 1
            right = n - 1

            while left < right:
                current_sum = nums[i] + nums[j] + nums[left] + nums[right]
                # If quadruplet sum matches target, add to result.
                if current_sum == target:
                    quadruplets.append([nums[i], nums[j], nums[left], nums[righ
ht]])

                    # Skip duplicates for the third element.
                    while left < right and nums[left] == nums[left + 1]:
                        left += 1
                    # Skip duplicates for the fourth element.
                    while left < right and nums[right] == nums[right - 1]:

```

Round 2 · High · Medium

p.515

```

                right -= 1
                left += 1
                right -= 1
            # Move left pointer if sum is less.
            elif current_sum < target:
                left += 1
            # Move right pointer if sum is more.
            else:
                right -= 1
        return quadruplets

# Example usage:
print(fourSum([1, 0, -1, 0, -2, 2], 0))

```

Round 2 · High · Medium

p.516

Two Pointers

☐ #167 Two Sum II – Input Array Is Sorted **MED**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Two Pointers · Binary Search
 Lists: Denny Zhang

Problem

Given a 1-indexed array of integers `numbers` that is already sorted in non-decreasing order, find two numbers such that they add up to a specific target number. Let these two numbers be `numbers[index1]` and `numbers[index2]` where $1 \leq \text{index1} < \text{index2} \leq \text{numbers.length}$.

Return the indices of the two numbers `index1` and `index2`, each incremented by one, as an integer array `[index1, index2]` of length 2.

The tests are generated such that there is exactly one solution. You may not use the same element twice.

Your solution must use only constant extra space.

Example 1:

Round 2 · High · Medium

p.517

Input: `numbers = [2,7,11,15]`, `target = 9`
 Output: `[1,2]`
 Explanation: The sum of 2 and 7 is 9. Therefore, `index1 = 1`, `index2 = 2`. We return `[1, 2]`.

Example 2:

Input: `numbers = [2,3,4]`, `target = 6`
 Output: `[1,3]`
 Explanation: The sum of 2 and 4 is 6. Therefore `index1 = 1`, `index2 = 3`. We return `[1, 3]`.

Example 3:

Input: `numbers = [-1,0]`, `target = -1`
 Output: `[1,2]`
 Explanation: The sum of `-1` and `0` is `-1`. Therefore `index1 = 1`, `index2 = 2`. We return `[1, 2]`.

Constraints:

- $2 \leq \text{numbers.length} \leq 3 \times 10^4$
- $-1000 \leq \text{numbers}[i] \leq 1000$
- `numbers` is sorted in non-decreasing order.
- $-1000 \leq \text{target} \leq 1000$
- The tests are generated such that there is exactly one solution.

Community Solutions (Python)

- WalkCC

Round 2 · High · Medium

p.518

```
class Solution:
    def twoSum(self, numbers: List[int], target: int) -> List[int]:
        l = 0
        r = len(numbers) - 1

        while l < r:
            summ = numbers[l] + numbers[r]
            if summ == target:
                return [l + 1, r + 1]
            if summ < target:
                l += 1
            else:
                r -= 1
```

• LeetCode

```
class Solution:
    def twoSum(self, numbers: List[int], target: int) -> List[int]:
        n = len(numbers)
        for i in range(n - 1):
            x = target - numbers[i]
            j = bisect_left(numbers, x, lo=i + 1)
            if j < n and numbers[j] == x:
                return [i + 1, j + 1]
```

```
class Solution:
    def twoSum(self, numbers: List[int], target: int) -> List[int]:
        i, j = 0, len(numbers) - 1
        while i < j:
            x = numbers[i] + numbers[j]
            if x == target:
                return [i + 1, j + 1]
            if x < target:
                i += 1
            else:
                j -= 1
```

• SimplyLeet

Round 2 · High · Medium

p.519

```
# Python solution for Two Sum II - Input Array Is Sorted
def twoSum(numbers, target):
    # Initialize two pointers at the beginning and end of the array
    left = 0
    right = len(numbers) - 1

    # Continue until the two pointers meet
    while left < right:
        # Calculate the sum of the two pointed values
        current_sum = numbers[left] + numbers[right]

        # If the sum matches the target, return the 1-indexed results
        if current_sum == target:
            return [left + 1, right + 1]
        # If the current sum is less than the target, move the left pointer to
        # the right
        elif current_sum < target:
            left += 1
        # Otherwise, move the right pointer to the left since the current sum
        # is too high
        else:
            right -= 1

# Example call
print(twoSum([2, 7, 11, 15], 9)) # Output: [1, 2]
```

• LC.ca

```
class Solution:
    def twoSum(self, numbers: List[int], target: int) -> List[int]:
        i, j = 0, len(numbers) - 1
        while i < j:
            x = numbers[i] + numbers[j]
            if x == target:
                return [i + 1, j + 1]
            if x < target:
                i += 1
            else:
                j -= 1
```

Round 2 · High · Medium

p.520

◆ Quick Review

Key Insights

- The array is sorted in non-decreasing order, which makes the two pointers technique very effective.
- Using two pointers (one at the start and one at the end) allows checking pairs and adjusting the pointers based on the sum.
- Since the problem guarantees exactly one solution and requires constant space, using two pointers satisfies both constraints.

Space & Time

Time Complexity: $O(n)$, where n is the number of elements in the array, because in the worst-case scenario each element is visited at most once.

Space Complexity: $O(1)$, as no additional data structure is used that scales with the input size.

Solution

We use the two pointers approach:

- Initialize two pointers, left at the beginning (index 0) and right at the end (last index) of

the array. – While left is less than right, compute the sum of the elements at the two pointers. – If the sum equals the target, return the indices as $[left+1, right+1]$ (to adjust for the 1-indexed array). – If the sum is less than the target, move the left pointer to the right to increase the sum. – If the sum is greater than the target, move the right pointer to the left to decrease the sum. This approach leverages the sorted property and uses constant extra space.

Two Pointers

☐ #443 String Compression

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode

Two Pointers · String

Lists: AlgoMaster 600

Problem

Given an array of characters `chars`, compress it using the following algorithm:

Begin with an empty string `s`. For each group of consecutive repeating characters in `chars`:

- If the group's length is 1, append the character to `s`.
- Otherwise, append the character followed by the group's length.

The compressed string `s` should not be returned separately, but instead, be stored in the input character array `chars`. Note that group lengths that are 10 or longer will be split into multiple characters in `chars`.

After you are done modifying the input array, return the new length of the array.

Round 2 · High · Medium

p.523

You must write an algorithm that uses only constant extra space.

Note: The characters in the array beyond the returned length do not matter and should be ignored.

Example 1:

Input: `chars = ["a","a","b","b","c","c","c"]`
 Output: 6
 Explanation: The groups are "aa", "bb", and "ccc". This compresses to "a2b2c3".

Example 2:

Input: `chars = ["a"]`
 Output: 1
 Explanation: The only group is "a", which remains uncompressed since it's a single character.

Example 3:

Input: `chars = ["a","b","b","b","b","b","b","b","b","b","b","b","b"]`
 Output: 4
 Explanation: The groups are "a" and "bbbbbbbbbbb". This compresses to "ab12".

Constraints:

- $1 \leq \text{chars.length} \leq 2000$
- `chars[i]` is a lowercase English letter, uppercase English letter, digit, or symbol.

Community Solutions (Python)

- WalkCC

Round 2 · High · Medium

p.524

```
class Solution:
    def compress(self, chars: list[str]) -> int:
        ans = 0
        i = 0

        while i < len(chars):
            letter = chars[i]
            count = 0
            while i < len(chars) and chars[i] == letter:
                count += 1

            i += 1
            chars[ans] = letter
            ans += 1
            if count > 1:
                for c in str(count):
                    chars[ans] = c
                    ans += 1

        return ans
```

• LeetCode

```
class Solution:
    def compress(self, chars: List[str]) -> int:
        i, k, n = 0, 0, len(chars)
        while i < n:
            j = i + 1
            while j < n and chars[j] == chars[i]:
                j += 1
            chars[k] = chars[i]
            k += 1
            if j - i > 1:
                cnt = str(j - i)
                for c in cnt:
                    chars[k] = c
                    k += 1
            i = j
        return k
```

• SimplyLeet

Round 2 · High · Medium

p.525

```
# Function to compress the list of characters in-place.
def compress(chars):
    # Initialize read and write pointers.
    read, write = 0, 0
    # Loop until the read pointer reaches the end of the array.
    while read < len(chars):
        # Current character to be processed.
        current_char = chars[read]
        count = 0
        # Count duplicates of current_char.

        while read < len(chars) and chars[read] == current_char:
            read += 1
            count += 1
        # Write the current character to the result.
        chars[write] = current_char
        write += 1
        # If count is greater than 1, write each digit of the count.
        if count > 1:
            for digit in str(count):
                chars[write] = digit
                write += 1

    # Return the new length of the array.
    return write

# Example usage:
# chars = ["a","a","b","b","c","c","c"]
# new_length = compress(chars)
# print(new_length, chars[:new_length])
```

Round 2 · High · Medium

p.526

Two Pointers

#861 Boats to Save People

MED

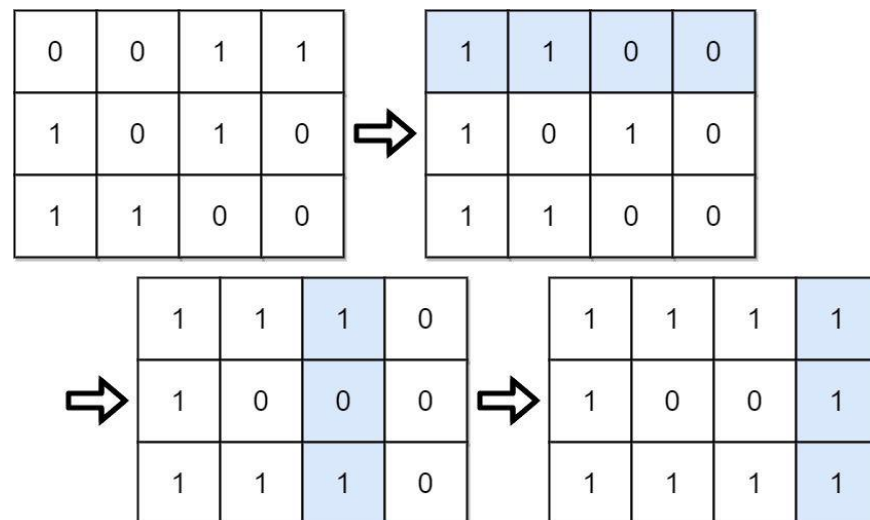
LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Two Pointers · Greedy · Sorting
 Lists: AlgoMaster 600

Problem

You are given an $m \times n$ binary matrix grid. A move consists of choosing any row or column and toggling each value in that row or column (i.e., changing all 0's to 1's, and all 1's to 0's). Every row of the matrix is interpreted as a binary number, and the score of the matrix is the sum of these numbers.

Return the highest possible score after making any number of moves (including zero moves).

Example 1:



Input: grid = [[0,0,1,1],[1,0,1,0],[1,1,0,0]]
 Output: 39
 Explanation: 0b1111 + 0b1001 + 0b1111 = 15 + 9 + 15 = 39

Example 2:

Input: grid = [[0]]
 Output: 1

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq m, n \leq 20$
- $\text{grid}[i][j]$ is either 0 or 1.

Community Solutions (Python)

- WalkCC

```
class Solution:
    def matrixScore(self, grid: list[list[int]]) -> int:
        # Flip the rows with a leading 0.
        for row in grid:
            if row[0] == 0:
                self._flip(row)

        # Flip the columns with 1s < 0s.
        for j, col in enumerate(list(zip(*grid))):
            if sum(col) * 2 < len(grid):

                self._flipCol(grid, j)

        # Add a binary number for each row.
        return sum(self._binary(row) for row in grid)

    def _flip(self, row: list[int]) -> None:
        for i in range(len(row)):
            row[i] ^= 1

    def _flipCol(self, grid: list[list[int]], j: int) -> None:
```

```
        for i in range(len(grid)):
            grid[i][j] ^= 1
```

```
    def _binary(self, row: list[int]) -> int:
        res = row[0]
        for j in range(1, len(row)):
            res = res * 2 + row[j]
        return res
```

```
class Solution:
    def matrixScore(self, grid: list[list[int]]) -> int:
        m = len(grid)
        n = len(grid[0])
        ans = m # All the cells in the first column are 1.

        for j in range(1, n):
            # The best strategy is flipping the rows with a leading 0..
            onesCount = sum(grid[i][j] == grid[i][0] for i in range(m))
            ans = ans * 2 + max(onesCount, m - onesCount)
```

Round 2 · High · Medium

p.529

```
        return ans
```

• LeetCode

```
class Solution:
    def matrixScore(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        for i in range(m):
            if grid[i][0] == 0:
                for j in range(n):
                    grid[i][j] ^= 1

        ans = 0
        for j in range(n):
            cnt = sum(grid[i][j] for i in range(m))

            ans += max(cnt, m - cnt) * (1 << (n - j - 1))
        return ans
```

• SimplyLeet

```
# Function to calculate the minimum number of boats needed
def numRescueBoats(people, limit):
    # Sort the list of people's weights
    people.sort()
    # Initialize two pointers; left points to the lightest, right to the
    heaviest
    left, right = 0, len(people) - 1
    boats = 0 # Count of boats used

    # Iterate until all persons are assigned to a boat
    while left <= right:
```

Round 2 · High · Medium

p.530

```
# If the lightest and heaviest person can share a boat, do so
if people[left] + people[right] <= limit:
    left += 1 # Move left pointer to the next lightest person
# Always move the right pointer as the heaviest person occupies a boat
now
right -= 1
boats += 1 # Increment boat count for each allocation

return boats

# Example usage:

# people = [3,2,2,1], limit = 3
# print(numRescueBoats(people, limit)) # Expected output: 3
```

Round 2 · High · Medium

p.531

Sliding Window – Fixed Size

Round 2 · High · Medium · 4 questions

Round 2 · High · Medium

p.532

Sliding Window – Fixed Size

#438 Find All Anagrams in a String

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Sliding Window
Lists: Grind 169

Problem

Given two strings s and p , return an array of all the start indices of p 's anagrams in s . You may return the answer in any order.

Example 1:

Input: $s = \text{"cbaebabacd"}, p = \text{"abc"}$
Output: $[0,6]$
Explanation:
The substring with start index = 0 is "cba", which is an anagram of "abc".
The substring with start index = 6 is "bac", which is an anagram of "abc".

Example 2:

Input: $s = \text{"abab"}, p = \text{"ab"}$
Output: $[0,1,2]$
Explanation:
The substring with start index = 0 is "ab", which is an anagram of "ab".
The substring with start index = 1 is "ba", which is an anagram of "ab".
The substring with start index = 2 is "ab", which is an anagram of "ab".

Constraints:

- $1 \leq s.length, p.length \leq 3 * 10^4$
- s and p consist of lowercase English letters.

Round 2 · High · Medium

p.533

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findAnagrams(self, s: str, p: str) -> list[int]:
        ans = []
        count = collections.Counter(p)
        required = len(p)

        for r, c in enumerate(s):
            count[c] -= 1
            if count[c] >= 0:
                required -= 1

            if r >= len(p):
                count[s[r - len(p)]] += 1
                if count[s[r - len(p)]] > 0:
                    required += 1
            if required == 0:
                ans.append(r - len(p) + 1)

        return ans
```

• LeetDoocs

```
class Solution:
    def findAnagrams(self, s: str, p: str) -> List[int]:
        m, n = len(s), len(p)
        ans = []
        if m < n:
            return ans
        cnt1 = Counter(p)
        cnt2 = Counter(s[:n-1])
        for i in range(n-1, m):
            cnt2[s[i]] += 1

            if cnt1 == cnt2:
                ans.append(i - n + 1)
            cnt2[s[i - n + 1]] -= 1
        return ans
```

Round 2 · High · Medium

p.534

• SimplyLeet

```
# Define the function to find all anagram indices in the string
def findAnagrams(s: str, p: str):
    # Import Counter for frequency counting
    from collections import Counter
    # List to store the starting indices of all anagrams
    result = []
    # Length of string p
    p_length = len(p)
    # Frequency count for characters in p
    p_count = Counter(p)

    # Frequency count for the current window in s
    s_count = Counter()

    # Iterate over the string s
    for i in range(len(s)):
        # Add current character to the window count
        s_count[s[i]] += 1

        # Remove the leftmost character from window if window size exceeds
        # p_length
        if i >= p_length:
            if s_count[s[i - p_length]] == 1:
                del s_count[s[i - p_length]] # Remove key completely if count
            # becomes 0
            else:
                s_count[s[i - p_length]] -= 1

        # Compare window count with p_count when window size is equal to
        # p_length
        if s_count == p_count:
            result.append(i - p_length + 1)

    return result

# Example usage:
print(findAnagrams("cbaebabacd", "abc")) # Expected output: [0, 6]
```

• LC.ca

Round 2 · High · Medium

p.535

```
class Solution:
    def findAnagrams(self, s: str, p: str) -> List[int]:
        m, n = len(s), len(p)
        ans = []
        if m < n:
            return ans
        cnt1 = Counter(p)
        cnt2 = Counter(s[: n - 1])
        for i in range(n - 1, m):
            cnt2[s[i]] += 1

            if cnt1 == cnt2:
                ans.append(i - n + 1)
            cnt2[s[i - n + 1]] -= 1
        return ans
```

◆ Quick Review

Key Insights

- Use a sliding window of length equal to p over s .
- Maintain frequency counts of characters in p and within the current window in s .
- Compare frequency counts to determine if the current window is an anagram.
- Update the window efficiently by adding one character and removing the character that goes out of the window.

Space & Time

Time Complexity: $O(n)$ where n is the length of s .

Round 2 · High · Medium

p.536

Space Complexity: $O(1)$ since we use fixed-size frequency arrays or hash maps.

Solution

We use the sliding window technique to iterate over s with a window size equal to the length of p . First, count the frequencies of characters in p . Then, iterate over s and update a frequency count for the current window. When the window size equals the length of p , compare the two frequency counts. If they match, record the start index. To slide the window efficiently, increment the count for the incoming character and decrement the count for the outgoing character. This way, we achieve an $O(n)$ solution that is optimal for the problem constraints.

Round 2 · High · Medium

p.537

Sliding Window – Fixed Size

□ #567 Permutation in String

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode Hash Table · Two Pointers · String · Sliding Window

Lists: NeetCode 150

Problem

Given two strings $s1$ and $s2$, return true if $s2$ contains a permutation of $s1$, or false otherwise. In other words, return true if one of $s1$'s permutations is the substring of $s2$.

Example 1:

Input: $s1 = "ab"$, $s2 = "eidbaooo"$
 Output: true
 Explanation: $s2$ contains one permutation of $s1$ ("ba").

Example 2:

Input: $s1 = "ab"$, $s2 = "eidboaoo"$
 Output: false

Constraints:

- $1 \leq s1.length, s2.length \leq 104$
- $s1$ and $s2$ consist of lowercase English letters.

Round 2 · High · Medium

p.538

Community Solutions (Python)

• WalkCC

```
class Solution:
    def checkInclusion(self, s1: str, s2: str) -> bool:
        count = collections.Counter(s1)
        required = len(s1)

        for r, c in enumerate(s2):
            count[c] -= 1
            if count[c] >= 0:
                required -= 1
            if r >= len(s1): # The window is oversized.

                count[s2[r - len(s1)]] += 1
                if count[s2[r - len(s1)]] > 0:
                    required += 1
                if required == 0:
                    return True

        return False
```

• LeetDoocs

```
class Solution:
    def checkInclusion(self, s1: str, s2: str) -> bool:
        cnt = Counter(s1)
        need = len(cnt)
        m = len(s1)
        for i, c in enumerate(s2):
            cnt[c] -= 1
            if cnt[c] == 0:
                need -= 1
            if i >= m:

                cnt[s2[i - m]] += 1
                if cnt[s2[i - m]] == 1:
                    need += 1

                if need == 0:
                    return True
        return False
```

Round 2 · High · Medium

p.539

• SimplyLeet

```
# Python solution with line-by-line comments

def checkInclusion(s1: str, s2: str) -> bool:
    # Get lengths of the input strings
    len1, len2 = len(s1), len(s2)
    # Edge case: s1 cannot be a substring if longer than s2
    if len1 > len2:
        return False

    # Initialize frequency arrays for s1 and for the first window of s2

    s1_count = [0] * 26
    window_count = [0] * 26

    # Build frequency counts for s1 and the initial window in s2
    for i in range(len1):
        s1_count[ord(s1[i]) - ord('a')] += 1
        window_count[ord(s2[i]) - ord('a')] += 1

    # Check if initial window is a permutation of s1
    if s1_count == window_count:

        return True

    # Slide the window across s2, updating the counts on the fly
    for i in range(len1, len2):
        # Add the new character from the right end of the window
        window_count[ord(s2[i]) - ord('a')] += 1
        # Remove the character that is no longer in the window
        window_count[ord(s2[i - len1]) - ord('a')] -= 1

        # Check if current window matches s1's frequency distribution

        if window_count == s1_count:
            return True

    return False

# Example usage:
# print(checkInclusion("ab", "eidbaooo")) # Expected output: True
```

Round 2 · High · Medium

p.540

Sliding Window – Fixed Size

☐ #1456 Maximum Number of Vowels in a Substring of Given Length MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode String · Sliding Window
Lists: AlgoMaster 600

Problem

Given a string *s* and an integer *k*, return the maximum number of vowel letters in any substring of *s* with length *k*.

Vowel letters in English are 'a', 'e', 'i', 'o', and 'u'.

Example 1:

Input: *s* = "abciidef", *k* = 3
Output: 3
Explanation: The substring "iii" contains 3 vowel letters.

Example 2:

Input: *s* = "aeiou", *k* = 2
Output: 2
Explanation: Any substring of length 2 contains 2 vowels.

Example 3:

Input: *s* = "leetcode", *k* = 3
Output: 2
Explanation: "lee", "eet" and "ode" contain 2 vowels.

Constraints:

- $1 \leq s.length \leq 105$
- *s* consists of lowercase English letters.
- $1 \leq k \leq s.length$

Community Solutions (Python)

☐ • WalkCC

```
class Solution:
    def maxVowels(self, s: str, k: int) -> int:
        ans = 0
        mx = 0
        VOWELS = 'aeiou'

        for i, c in enumerate(s):
            if c in VOWELS:
                mx += 1
            if i >= k and s[i - k] in VOWELS:

                mx -= 1
            ans = max(ans, mx)

        return ans
```

☐ • LeetDoocs

```
class Solution:
    def maxVowels(self, s: str, k: int) -> int:
        vowels = set("aeiou")
        ans = cnt = sum(c in vowels for c in s[:k])
        for i in range(k, len(s)):
            cnt += int(s[i] in vowels) - int(s[i - k] in vowels)
            ans = max(ans, cnt)
        return ans
```

☐ • SimplyLeet

```
# Python solution using sliding window algorithm

def maxVowels(s: str, k: int) -> int:
    vowels = set("aeiou") # Set for quick vowel lookup
    current_count = 0

    # Count vowels in the initial window of size k
    for i in range(k):
        if s[i] in vowels:
            current_count += 1

    max_count = current_count # Initialize max_count with the count of the
    first window

    # Slide the window through the string
    for i in range(k, len(s)):
        if s[i - k] in vowels: # Remove letter going out of the window
            current_count -= 1
        if s[i] in vowels: # Add incoming letter
            current_count += 1
        max_count = max(max_count, current_count) # Update maximum found so
    far

    return max_count

# Example usage:
# print(maxVowels("abcciiidef", 3))
```

Sliding Window – Fixed Size

□ #2461 Maximum Sum of Distinct Subarrays With Length K

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table · Sliding Window
 Lists: AlgoMaster 600

Problem

You are given an integer array `nums` and an integer `k`. Find the maximum subarray sum of all the subarrays of `nums` that meet the following conditions:

- The length of the subarray is `k`, and
- All the elements of the subarray are distinct.

Return the maximum subarray sum of all the subarrays that meet the conditions. If no subarray meets the conditions, return 0.

A subarray is a contiguous non-empty sequence of elements within an array.

Example 1:

Input: nums = [1,5,4,2,9,9,9], k = 3

Output: 15

Explanation: The subarrays of nums with length 3 are:

- [1,5,4] which meets the requirements and has a sum of 10.
- [5,4,2] which meets the requirements and has a sum of 11.
- [4,2,9] which meets the requirements and has a sum of 15.
- [2,9,9] which does not meet the requirements because the element 9 is repeated.
- [9,9,9] which does not meet the requirements because the element 9 is repeated.

Example 2:

Input: nums = [4,4,4], k = 3

Output: 0

Explanation: The subarrays of nums with length 3 are:

- [4,4,4] which does not meet the requirements because the element 4 is repeated.

We return 0 because no subarrays meet the conditions.

Constraints:

- $1 \leq k \leq \text{nums.length} \leq 105$
- $1 \leq \text{nums}[i] \leq 105$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maximumSubarraySum(self, nums: List[int], k: int) -> int:
        ans = 0
        summ = 0
        distinct = 0
        count = collections.Counter()

        for i, num in enumerate(nums):
            summ += num
            count[num] += 1
```

```
if count[num] == 1:
    distinct += 1
if i >= k:
    count[nums[i - k]] -= 1
    if count[nums[i - k]] == 0:
        distinct -= 1
    summ -= nums[i - k]
if i >= k - 1 and distinct == k:
    ans = max(ans, summ)
```

```
return ans
```

• LeetDoocs

```
class Solution:
    def maximumSubarraySum(self, nums: List[int], k: int) -> int:
        cnt = Counter(nums[:k])
        s = sum(nums[:k])
        ans = s if len(cnt) == k else 0
        for i in range(k, len(nums)):
            cnt[nums[i]] += 1
            cnt[nums[i - k]] -= 1
            if cnt[nums[i - k]] == 0:
                cnt.pop(nums[i - k])

            s += nums[i] - nums[i - k]
            if len(cnt) == k:
                ans = max(ans, s)
        return ans
```

• SimplyLeet

```
# Python solution for Maximum Sum of Distinct Subarrays With Length K

def maximumSubarraySum(nums, k):
    # Dictionary to store frequencies of elements in current window
    freq = {}
    current_sum = 0
    max_sum = 0
    left = 0 # window left pointer

    # Loop through each element in the array using a sliding window approach

    for right in range(len(nums)):
        # Add the new element at the 'right' end of the window
        current = nums[right]
        freq[current] = freq.get(current, 0) + 1
        current_sum += current

        # Slide window if its size is greater than k
        if right - left + 1 > k:
            remove_val = nums[left]
            freq[remove_val] -= 1

            # Remove element from dictionary if its count becomes zero
            if freq[remove_val] == 0:
                del freq[remove_val]
            current_sum -= remove_val
            left += 1

        # Check the window only if it has exactly k elements and all are
        distinct
        if right - left + 1 == k and len(freq) == k:
            max_sum = max(max_sum, current_sum)

    return max_sum

# Example usage:
print(maximumSubarraySum([1,5,4,2,9,9,9], 3)) # Output: 15
```

Round 2 · High · Medium

p.547

Sliding Window – Dynamic Size

Round 2 · High · Medium · 12 questions

Round 2 · High · Medium

p.548

Sliding Window – Dynamic Size

#3 Longest Substring Without Repeating Characters MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Sliding Window
Lists: Grind 169

Problem

Given a string s , find the length of the longest substring without duplicate characters.

Example 1:

Input: $s = \text{"abcabcbb"}$
Output: 3
Explanation: The answer is "abc", with the length of 3. Note that "bca" and "cab" are also correct answers.

Example 2:

Input: $s = \text{"bbbbbb"}$
Output: 1
Explanation: The answer is "b", with the length of 1.

Example 3:

Input: $s = \text{"pwwkew"}$
Output: 3
Explanation: The answer is "wke", with the length of 3.
Notice that the answer must be a substring, "pwke" is a subsequence and not a substring.

Constraints:

- $0 \leq s.length \leq 5 * 10^4$

Round 2 · High · Medium

p.549

- s consists of English letters, digits, symbols and spaces.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        ans = 0
        count = collections.Counter()

        l = 0
        for r, c in enumerate(s):
            count[c] += 1
            while count[c] > 1:
                count[s[l]] -= 1
                l += 1
            ans = max(ans, r - l + 1)

        return ans
```

```
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        ans = 0
        # The substring s[j + 1..i] has no repeating characters.
        j = -1
        # lastSeen[c] := the index of the last time c appeared
        lastSeen = {}

        for i, c in enumerate(s):
            # Update j to lastSeen[c], so the window must start from j + 1.

            j = max(j, lastSeen.get(c, -1))
            ans = max(ans, i - j)
            lastSeen[c] = i

        return ans
```

• LeetDoocs

Round 2 · High · Medium

p.550

```
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        cnt = Counter()
        ans = l = 0
        for r, c in enumerate(s):
            cnt[c] += 1
            while cnt[c] > 1:
                cnt[s[l]] -= 1
                l += 1
            ans = max(ans, r - l + 1)

        return ans
```

• SimplyLeet

```
# Function to find the length of the longest substring without repeating
characters
def length_of_longest_substring(s):
    # Dictionary to store the most recent index of each character
    char_index = {}
    # Initialize starting index of current window and max length
    left = 0
    max_length = 0

    # Iterate over the string with the right pointer
    for right, char in enumerate(s):

        # If char is in dictionary and its index is within the current window
        if char in char_index and char_index[char] >= left:
            # Move the left pointer to one position after the last occurrence
            left = char_index[char] + 1
        # Update the latest index of the char
        char_index[char] = right
        # Update max_length with the current window size if larger
        max_length = max(max_length, right - left + 1)

    return max_length

# Example usage:
# print(length_of_longest_substring("abcabcbb")) # Expected output: 3
```

Round 2 · High · Medium

p.551

• LC.ca

```
class Solution:
    def lengthOfLongestSubstring(self, s): # map for index
        d = {} # value => its index
        i = 0
        ans = 0
        for j, c in enumerate(s):
            if c in d:
                # must max check i, example "abba"
                i = max(i, d[c] + 1)
            d[c] = j

            ans = max(ans, j - i + 1)
        return ans

class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:

        if not s:
            return 0

        l = 0 # left
        r = 0 # right

        result = 0
        isFoundInWindow = [False] * 256

        while r < len(s):

            while isFoundInWindow[ord(s[r])]:
                isFoundInWindow[ord(s[l])] = False
```

Round 2 · High · Medium

p.552


```

        l += 1

        if foundInWindow[ord(s[r])] == True:

            # check before shrink
            result = max(result, r - l + 1)

            r += 1

        return result

```

```

class Solution: # extra space for set()
    def lengthOfLongestSubstring(self, s: str) -> int:

```

```

        d = collections.defaultdict(int)
        start = 0
        ans = 0
        for i, c in enumerate(s):
            # shrink
            while d[c] > 0:

```

```

                d[s[start]] -= 1 # not s[start], not start
                start += 1
            ans = max(ans, i - start + 1)
            d[c] = 1
        return ans

```

```

#####

```

```

class Solution(object):
    def _lengthOfLongestSubstring(self, s): # no extra data structure

```

```

        """
        :type s: str
        :rtype: int
        """
        d = collections.defaultdict(int)
        l = ans = 0
        for i, c in enumerate(s):
            while l > 0 and d[c] > 0:
                d[s[i - l]] -= 1
                l -= 1

```

Round 2 · High · Medium

p.553

```

        d[c] += 1
        l += 1
        ans = max(ans, l)
        return ans

```

```

#####

```

```

class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        ss = set()

```

```

        i = ans = 0
        for j, c in enumerate(s):
            while c in ss:
                ss.remove(s[i])
                i += 1
            ss.add(c)
            ans = max(ans, j - i + 1)
        return ans

```

◆ Quick Review

Key Insights

- Use the sliding window technique to maintain a window of unique characters.
- Use a hash table (or dictionary) to quickly check for duplicate characters and update window boundaries.
- The window is expanded by moving a pointer (right) and contracted by moving another pointer (left) when duplicates are encountered.

Round 2 · High · Medium

p.554

- Time complexity can be maintained at $O(n)$ by ensuring each character is processed a limited number of times.

Space & Time

Time Complexity: $O(n)$, where n is the length of the input string.

Space Complexity: $O(\min(n, m))$, where m is the size of the character set (constant for fixed character sets).

Solution

The solution uses a sliding window technique with two pointers, left and right, to keep track of the current substring without repeating characters. A hash table (dictionary or map) is used to store the most recent index of each character encountered. As the right pointer moves forward, if a duplicate character is found within the current window, the left pointer is advanced to the position right after where the duplicate was last seen. This ensures that the window always contains unique characters. The maximum length is updated throughout the process.

Round 2 · High · Medium

p.555

Sliding Window – Dynamic Size

★ #159 Longest Substring with At Most Two Distinct Characters ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Sliding Window
Lists: ★ Premium | Premium 98

Problem

Given a string s , return the length of the longest substring that contains at most two distinct characters.

Example 1:

Input: $s = \text{"eceba"}$
Output: 3
Explanation: The substring is "ece" which its length is 3.

Example 2:

Input: $s = \text{"ccaabbb"}$
Output: 5
Explanation: The substring is "aabbb" which its length is 5.

Constraints:

- $1 \leq s.length \leq 105$
- s consists of English letters.

Community Solutions (Python)

- WalkCC

Round 2 · High · Medium

p.556

```
class Solution:
    def lengthOfLongestSubstringTwoDistinct(self, s: str) -> int:
        ans = 0
        distinct = 0
        count = [0] * 128

        l = 0
        for r, c in enumerate(s):
            count[ord(c)] += 1
            if count[ord(c)] == 1:
                distinct += 1

            while distinct == 3:
                count[ord(s[l])] -= 1
                if count[ord(s[l])] == 0:
                    distinct -= 1
                l += 1
            ans = max(ans, r - l + 1)

        return ans
```

• LeetCode

```
class Solution:
    def lengthOfLongestSubstringTwoDistinct(self, s: str) -> int:
        cnt = Counter()
        ans = j = 0
        for i, c in enumerate(s):
            cnt[c] += 1
            while len(cnt) > 2:
                cnt[s[j]] -= 1
                if cnt[s[j]] == 0:
                    cnt.pop(s[j])
                j += 1
            ans = max(ans, i - j + 1)
        return ans
```

• SimplyLeet

Round 2 · High · Medium

p.557

```
# Python solution for Longest Substring with At Most Two Distinct Characters
class Solution:
    def lengthOfLongestSubstringTwoDistinct(self, s: str) -> int:
        left = 0 # Initialize left pointer of the window
        max_length = 0 # Variable to store the maximum window size found
        char_count = {} # Dictionary to count characters in the current window

        for right, char in enumerate(s):
            # Add the current character to the window (increase count)
            char_count[char] = char_count.get(char, 0) + 1

            # If there are more than two distinct characters, shrink the window
            # from the left
            while len(char_count) > 2:
                left_char = s[left]
                char_count[left_char] -= 1
                if char_count[left_char] == 0:
                    del char_count[left_char]
                left += 1

            # Update max_length if current window is larger
            max_length = max(max_length, right - left + 1)

        return max_length
```

• LC.ca

```
'''
>>> from collections import Counter
>>> c = Counter()
>>> c
Counter()
>>> c['a'] += 1
>>> c
Counter({'a': 1})
'''
```

Round 2 · High · Medium

p.558

```

>>> c
Counter({'a': 1, 'c': 1, 'b': 1})
>>> c.pop(2)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 2
>>>
>>> c.pop('c')
1
>>> c
Counter({'a': 1, 'b': 1})
'''

from collections import Counter

class Solution:
    def lengthOfLongestSubstringTwoDistinct(self, s: str) -> int:
        cnt = Counter() # or, cnt = defaultdict(int)
        ans = i = 0
        for j, c in enumerate(s):
            cnt[c] += 1
            while len(cnt) > 2: # shrink
                cnt[s[i]] -= 1
                if cnt[s[i]] == 0:
                    cnt.pop(s[i])
                i += 1
            ans = max(ans, j - i + 1)
        return ans

```

◆ Quick Review

Key Insights

- Use a sliding window to maintain a contiguous substring.
- Keep track of character frequencies within the current window using a hash map or

Round 2 · High · Medium

p.559

dictionary.

- When the window contains more than two distinct characters, shrink the window from the left until only two remain.
- Update the maximum length each time the window satisfies the condition.

Space & Time

Time Complexity: $O(n)$ where n is the length of the string, since each character is processed at most twice.

Space Complexity: $O(1)$ because the hash map holds at most 3 entries (usually up to 2 distinct characters).

Solution

We use the sliding window technique with two pointers (left and right) to represent the current window. A hash table/dictionary records the count of each character inside the window. As we extend the right pointer, we update the count. If the hash table contains more than two distinct characters, we start moving the left pointer to reduce the number of distinct characters until we have at most two distinct keys. The length of the current

Round 2 · High · Medium

p.560

valid window is then used to update our maximum length answer.

Round 2 · High · Medium

p.561

Sliding Window – Dynamic Size

□ #209 Minimum Size Subarray Sum

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Sliding Window · Prefix Sum

Lists: AlgoMaster 600

Problem

Given an array of positive integers `nums` and a positive integer `target`, return the minimal length of a subarray whose sum is greater than or equal to `target`. If there is no such subarray, return 0 instead.

Example 1:

Input: `target = 7, nums = [2,3,1,2,4,3]`
Output: 2
Explanation: The subarray `[4,3]` has the minimal length under the problem constraint.

Example 2:

Input: `target = 4, nums = [1,4,4]`
Output: 1

Example 3:

Input: `target = 11, nums = [1,1,1,1,1,1,1,1]`
Output: 0

Round 2 · High · Medium

p.562

Constraints:

- $1 \leq \text{target} \leq 109$
- $1 \leq \text{nums.length} \leq 105$
- $1 \leq \text{nums}[i] \leq 104$

Follow up: If you have figured out the $O(n)$ solution, try coding another solution of which the time complexity is $O(n \log(n))$.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minSubArrayLen(self, target: int, nums: list[int]) -> int:
        ans = math.inf
        summ = 0
        j = 0

        for i, num in enumerate(nums):
            summ += num
            while summ >= target:
                ans = min(ans, i - j + 1)

            summ -= nums[j]
            j += 1

        return 0 if ans == math.inf else ans
```

• LeetCode

```
class Solution:
    def minSubArrayLen(self, target: int, nums: List[int]) -> int:
        n = len(nums)
        s = list(accumulate(nums, initial=0))
        ans = n + 1
        for i, x in enumerate(s):
            j = bisect_left(s, x + target)
            if j <= n:
                ans = min(ans, j - i)
        return ans if ans <= n else 0

class Solution:
    def minSubArrayLen(self, target: int, nums: List[int]) -> int:
        l = s = 0
        ans = inf
        for r, x in enumerate(nums):
            s += x
            while s >= target:
                ans = min(ans, r - l + 1)
                s -= nums[l]
                l += 1

        return 0 if ans == inf else ans
```

• SimplyLeet

```
# Python implementation with detailed comments

def minSubArrayLen(target, nums):
    # Initialize the window pointers and variables
    n = len(nums)
    left = 0           # Left pointer of the sliding window
    window_sum = 0     # Sum of the current window
    min_len = float('inf') # Track minimum length, start with infinity

    # Iterate over the array with right pointer
```

```

for right in range(n):
    window_sum += nums[right] # Expand the window by adding the current
    element

    # Contract the window as long as the current sum is large enough
    while window_sum >= target:
        # Update the minimum length found so far
        current_window_length = right - left + 1
        min_len = min(min_len, current_window_length)

        # Shrink the window from the left and subtract the value

    window_sum -= nums[left]
    left += 1

# If the min_len wasn't updated, no valid subarray was found. Return 0.
return 0 if min_len == float('inf') else min_len

# Example usage:
print(minSubArrayLen(7, [2,3,1,2,4,3])) # Expected output: 2

```

Sliding Window – Dynamic Size

☐ ★ #340 Longest Substring with At Most K Distinct Characters ★ **MED**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Sliding Window
Lists: ★ Premium | Premium 98

Problem

Given a string *s* and an integer *k*, return the length of the longest substring of *s* that contains at most *k* distinct characters.

Example 1:

Input: *s* = "eceba", *k* = 2
Output: 3
Explanation: The substring is "ece" with length 3.

Example 2:

Input: *s* = "aa", *k* = 1
Output: 2
Explanation: The substring is "aa" with length 2.

Constraints:

- $1 \leq s.length \leq 5 * 10^4$
- $0 \leq k \leq 50$

Community Solutions (Python)

- WalkCC

```
class Solution:
    def lengthOfLongestSubstringKDistinct(self, s: str, k: int) -> int:
        ans = 0
        distinct = 0
        count = collections.Counter()

        l = 0
        for r, c in enumerate(s):
            count[c] += 1
            if count[c] == 1:

                distinct += 1
                while distinct == k + 1:
                    count[s[l]] -= 1
                    if count[s[l]] == 0:
                        distinct -= 1
                    l += 1
                ans = max(ans, r - l + 1)

        return ans
```

• LeetCode

```
class Solution:
    def lengthOfLongestSubstringKDistinct(self, s: str, k: int) -> int:
        l = 0
        cnt = Counter()
        for c in s:
            cnt[c] += 1
            if len(cnt) > k:
                cnt[s[l]] -= 1
                if cnt[s[l]] == 0:
                    del cnt[s[l]]

                l += 1
        return len(s) - l
```

• SimplyLeet

Round 2 · High · Medium

p.567

```
# Python solution for Longest Substring with At Most K Distinct Characters

def longest_substring_with_k_distinct(s: str, k: int) -> int:
    # Handle edge case: if k is 0, no valid substring exists
    if k == 0:
        return 0

    # Dictionary to store frequency of characters in the current window
    char_count = {}
    left = 0 # Left pointer for the window

    max_length = 0 # Variable to track the maximum length found

    # Iterate through the string using the right pointer
    for right in range(len(s)):
        # Add/update the count of the current character in the dictionary
        current_char = s[right]
        char_count[current_char] = char_count.get(current_char, 0) + 1

        # If window contains more than k distinct characters, shrink from the
        left

        while len(char_count) > k:
            left_char = s[left]
            char_count[left_char] -= 1
            # Remove the character count if it becomes 0
            if char_count[left_char] == 0:
                del char_count[left_char]
            left += 1

        # Update the maximum length of the valid window found so far
        max_length = max(max_length, right - left + 1)

    return max_length

# Example usage:
# s = "eceba", k = 2
# print(longest_substring_with_k_distinct(s, 2)) # Output: 3
```

• LC.ca

Round 2 · High · Medium

p.568


```

class Solution:
    def lengthOfLongestSubstringTwoDistinct(self, s: str) -> int:
        cnt = Counter() # or, cnt = defaultdict(int)
        ans = i = 0
        for j, c in enumerate(s):
            cnt[c] += 1
            while len(cnt) > k:
                cnt[s[i]] -= 1
                if cnt[s[i]] == 0:
                    cnt.pop(s[i])
                i += 1
            ans = max(ans, j - i + 1)
        return ans

```

◆ Quick Review

Key Insights

- Use the sliding window technique to efficiently scan through the string.
- Maintain a hash table (or dictionary) that keeps track of the frequency of characters in the current window.
- When the number of distinct characters exceeds k , increment the left pointer to shrink the window until the condition is met.
- Update the maximum length of valid substrings throughout the process.

Space & Time

Round 2 · High · Medium

p.569

Time Complexity: $O(n)$, where n is the length of the string, as each character is processed at most twice (once when expanding the window and once when contracting it).

Space Complexity: $O(\min(m, k))$, where m is the size of the charset (or number of unique characters in s), since at most $k+1$ keys are stored.

Solution

This problem is solved using the sliding window technique. The algorithm keeps track of a window defined by two indices (left and right pointers). A hash map is used to count the frequency of each character in the current window. As the right pointer extends the window, the count of the character is incremented. When the count of distinct keys in the map exceeds k , the left pointer is moved forward while decrementing the count of the leftmost character, until the condition is restored. The maximum window length encountered is recorded and returned as the result.

Round 2 · High · Medium

p.570

Sliding Window – Dynamic Size

#395 Longest Substring with At Least K Repeating Characters

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode Hash Table · String · Divide and Conquer · Sliding Window

Lists: AlgoMaster 600

Problem

Given a string s and an integer k , return the length of the longest substring of s such that the frequency of each character in this substring is greater than or equal to k .

if no such substring exists, return 0.

Example 1:

Input: $s = \text{"aaabb"}, k = 3$
 Output: 3
 Explanation: The longest substring is "aaa", as 'a' is repeated 3 times.

Example 2:

Input: $s = \text{"ababbc"}, k = 2$
 Output: 5
 Explanation: The longest substring is "ababb", as 'a' is repeated 2 times and 'b' is repeated 3 times.

Constraints:

- $1 \leq s.length \leq 104$

Round 2 · High · Medium

p.571

- s consists of only lowercase English letters.
- $1 \leq k \leq 105$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def longestSubstring(self, s: str, k: int) -> int:
        def longestSubstringWithNUniqueLetters(n: int) -> int:
            res = 0
            uniqueLetters = 0 # the number of unique letters
            lettersHavingKFreq = 0 # the number of letters having frequency >= k
            count = collections.Counter()

            l = 0
            for r, c in enumerate(s):

                count[c] += 1
                if count[c] == 1:
                    uniqueLetters += 1
                if count[c] == k:
                    lettersHavingKFreq += 1
                while uniqueLetters > n:
                    if count[s[l]] == k:
                        lettersHavingKFreq -= 1
                    count[s[l]] -= 1
                    if count[s[l]] == 0:

                        uniqueLetters -= 1
                        l += 1
                # Since both the number of unique letters and the number of letters
                # having frequency >= k are equal to n, this is a valid window.
                if lettersHavingKFreq == n: # Implicit: uniqueLetters == n
                    res = max(res, r - l + 1)

            return res

        return max(longestSubstringWithNUniqueLetters(n)

                    for n in range(1, 27))
```

Round 2 · High · Medium

p.572

• LeetCode

```
class Solution:
    def longestSubstring(self, s: str, k: int) -> int:
        def dfs(l, r):
            cnt = Counter(s[l : r + 1])
            split = next((c for c, v in cnt.items() if v < k), '')
            if not split:
                return r - l + 1
            i = l
            ans = 0
            while i <= r:
                while i <= r and s[i] == split:
                    i += 1
                if i >= r:
                    break
                j = i
                while j <= r and s[j] != split:
                    j += 1
                t = dfs(i, j - 1)
                ans = max(ans, t)
                i = j

            return ans

        return dfs(0, len(s) - 1)
```

• SimplyLeet

```
# Python solution with line-by-line comments

def longestSubstring(s: str, k: int) -> int:
    max_len = 0
    n = len(s)
    # Check for each potential number of unique characters in the substring
    for target_unique in range(1, 27): # There are at most 26 unique
        characters
        count = [0] * 26 # Frequency counter for current window
        start, end = 0, 0 # Sliding window pointers
        unique_count = 0 # Current number of unique characters in window
```

Round 2 · High · Medium

p.573

```
count_at_least_k = 0 # Number of characters meeting the k threshold
```

```
while end < n:
    # Add character at end pointer
    if count[ord(s[end]) - ord('a')] == 0:
        unique_count += 1 # New unique character encountered
    count[ord(s[end]) - ord('a')] += 1
    if count[ord(s[end]) - ord('a')] == k:
        count_at_least_k += 1 # Character frequency reached k
    end += 1
```

```
# Shrink window from left if unique characters exceed target
while unique_count > target_unique:
    if count[ord(s[start]) - ord('a')] == k:
        count_at_least_k -= 1
    count[ord(s[start]) - ord('a')] -= 1
    if count[ord(s[start]) - ord('a')] == 0:
        unique_count -= 1
    start += 1
```

```
# If window is valid update max_len
if unique_count == target_unique and unique_count == count_at_least_k:
    t_k:
        max_len = max(max_len, end - start)
    return max_len
```

```
# Example test cases
if __name__ == "__main__":
    print(longestSubstring("aaabb", 3)) # Expected output: 3
    print(longestSubstring("ababb", 2)) # Expected output: 5
```

Round 2 · High · Medium

p.574

Sliding Window – Dynamic Size

#424 Longest Repeating Character Replacement

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Sliding Window
Lists: Grind 169

Problem

You are given a string s and an integer k . You can choose any character of the string and change it to any other uppercase English character. You can perform this operation at most k times.

Return the length of the longest substring containing the same letter you can get after performing the above operations.

Example 1:

Input: $s = \text{"ABAB"}, k = 2$
Output: 4
Explanation: Replace the two 'A's with two 'B's or vice versa.

Example 2:

Input: $s = \text{"AABABBA"}, k = 1$
Output: 4
Explanation: Replace the one 'A' in the middle with 'B' and form "AABBBBA". The substring "BBBB" has the longest repeating letters, which is 4. There may exists other ways to achieve this answer too.

Round 2 · High · Medium

p.575

Constraints:

- $1 \leq s.length \leq 105$
- s consists of only uppercase English letters.
- $0 \leq k \leq s.length$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def characterReplacement(self, s: str, k: int) -> int:
        ans = 0
        maxCount = 0
        count = collections.Counter()

        l = 0
        for r, c in enumerate(s):
            count[c] += 1
            maxCount = max(maxCount, count[c])

            while maxCount + k < r - l + 1:
                count[s[l]] -= 1
                l += 1
            ans = max(ans, r - l + 1)

        return ans
```

```
class Solution:
    def characterReplacement(self, s: str, k: int) -> int:
        maxCount = 0
        count = collections.Counter()

        # l and r track the maximum window instead of the valid window.
        l = 0
        for r, c in enumerate(s):
            count[c] += 1
            maxCount = max(maxCount, count[c])
```

Round 2 · High · Medium

p.576

```

while maxCount + k < r - l + 1:
    count[s[l]] -= 1
    l += 1

return r - l + 1

```

• LeetCode

```

class Solution:
    def characterReplacement(self, s: str, k: int) -> int:
        cnt = Counter()
        l = mx = 0
        for r, c in enumerate(s):
            cnt[c] += 1
            mx = max(mx, cnt[c])
            if r - l + 1 - mx > k:
                cnt[s[l]] -= 1
                l += 1

        return len(s) - l

```

• SimplyLeet

```

# Python solution using the sliding window technique
def characterReplacement(s: str, k: int) -> int:
    # Dictionary to store frequency of characters in the current window
    char_counts = {}
    max_freq = 0 # Track the maximum frequency of a single character in the
    window
    max_length = 0
    left = 0 # Left pointer of the sliding window

    # Expand the sliding window with the right pointer
    for right in range(len(s)):

```

```

# Count the current character
char_counts[s[right]] = char_counts.get(s[right], 0) + 1
# Update the max frequency in the current window
max_freq = max(max_freq, char_counts[s[right]])

# Check if the number of replacements needed exceeds k
# (current window size) - (count of most frequent character) > k
if (right - left + 1) - max_freq > k:
    # If condition fails, shrink the window by moving left pointer
    char_counts[s[left]] -= 1

    left += 1

# Update the maximum length found so far
max_length = max(max_length, right - left + 1)

return max_length

# Example usage:
# print(characterReplacement("AABABBA", 1)) # Expected output: 4

```

• LC.ca

```

class Solution:
    def characterReplacement(self, s: str, k: int) -> int:
        counter = [0] * 26
        i = j = maxCnt = 0
        while i < len(s):
            counter[ord(s[i]) - ord('A')] += 1
            maxCnt = max(maxCnt, counter[ord(s[i]) - ord('A')])
            if i - j + 1 > maxCnt + k:
                counter[ord(s[j]) - ord('A')] -= 1
                j += 1

            i += 1
        return i - j

```

◆ Quick Review

Key Insights

- Use a Sliding Window algorithm to keep track of a contiguous substring.
- Maintain a frequency count of letters in the current window.
- Keep track of the count of the most frequent character in the window.
- If the current window size minus the maximum frequency is greater than k , shrink the window.
- The maximum window size seen during the process is the answer.

Space & Time

Time Complexity: $O(n)$ where n is the length of the string, since the window traverses the string in linear time.

Space Complexity: $O(1)$ because the frequency count for letters (A–Z) has a constant size.

Solution

The problem is solved using the sliding window technique. We initialize two pointers representing the left and right bounds of the current window. As we expand the window by moving the right pointer, we update the frequency count of the letter at the right

Round 2 · High · Medium

p.579

pointer and track the letter that appears most frequently. The key observation is that the number of characters that need to be replaced in the current window is the window's size minus the count of the most frequently occurring letter. If this number exceeds k , we increment the left pointer to shrink the window until the condition is satisfied again. The maximum size of the window during the process gives the answer.

Round 2 · High · Medium

p.580

Sliding Window – Dynamic Size

#713 Subarray Product Less Than K

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Sliding Window · Prefix Sum

Lists: AlgoMaster 600

Problem

Given an array of integers `nums` and an integer `k`, return the number of contiguous subarrays where the product of all the elements in the subarray is strictly less than `k`.

Example 1:

Input: `nums = [10,5,2,6]`, `k = 100`
Output: 8
Explanation: The 8 subarrays that have product less than 100 are:
[10], [5], [2], [6], [10, 5], [5, 2], [2, 6], [5, 2, 6]
Note that [10, 5, 2] is not included as the product of 100 is not strictly less than `k`.

Example 2:

Input: `nums = [1,2,3]`, `k = 0`
Output: 0

Constraints:

- $1 \leq \text{nums.length} \leq 3 \cdot 10^4$
- $1 \leq \text{nums}[i] \leq 1000$

Round 2 · High · Medium

p.581

- $0 \leq k \leq 10^6$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def numSubarrayProductLessThanK(self, nums: List[int], k: int) -> int:
        if k <= 1:
            return 0

        ans = 0
        prod = 1

        j = 0
        for i, num in enumerate(nums):

            prod *= num
            while prod >= k:
                prod /= nums[j]
                j += 1
            ans += i - j + 1

        return ans
```

• LeetDoocs

```
class Solution:
    def numSubarrayProductLessThanK(self, nums: List[int], k: int) -> int:
        ans = l = 0
        p = 1
        for r, x in enumerate(nums):
            p *= x
            while l <= r and p >= k:
                p //= nums[l]
                l += 1
            ans += r - l + 1

        return ans
```

• SimplyLeet

Round 2 · High · Medium

p.582

```
def numSubarrayProductLessThanK(nums, k):
    # If k is less than or equal to 1, no valid subarray exists
    if k <= 1:
        return 0
    prod = 1 # Running product of the current window
    ans = 0 # Counter for valid subarrays
    left = 0 # Left pointer of the sliding window
    # Iterate over the array using the right pointer
    for right, value in enumerate(nums):
        prod *= value # Multiply the current value into the running product

        # Shrink the window from the left until the product is less than k
        while prod >= k:
            prod //= nums[left]
            left += 1

        # All subarrays ending at 'right' with start indices between 'left' and
        # 'right' are valid
        ans += right - left + 1
    return ans

# Example usage
print(numSubarrayProductLessThanK([10, 5, 2, 6], 100)) # Expected output: 8
```

Sliding Window – Dynamic Size

□ #904 Fruit Into Baskets

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table · Sliding Window
 Lists: AlgoMaster 600

Problem

You are visiting a farm that has a single row of fruit trees arranged from left to right. The trees are represented by an integer array `fruits` where `fruits[i]` is the type of fruit the *i*th tree produces. You want to collect as much fruit as possible. However, the owner has some strict rules that you must follow:

- You only have two baskets, and each basket can only hold a single type of fruit. There is no limit on the amount of fruit each basket can hold.
- Starting from any tree of your choice, you must pick exactly one fruit from every tree (including the start tree) while moving to the right. The picked fruits must fit in one of your baskets.

- Once you reach a tree with fruit that cannot fit in your baskets, you must stop.

Given the integer array `fruits`, return the maximum number of fruits you can pick.

Example 1:

Input: `fruits = [1,2,1]`
 Output: 3
 Explanation: We can pick from all 3 trees.

Example 2:

Input: `fruits = [0,1,2,2]`
 Output: 3
 Explanation: We can pick from trees [1,2,2].
 If we had started at the first tree, we would only pick from trees [0,1].

Example 3:

Input: `fruits = [1,2,3,2,2]`
 Output: 4
 Explanation: We can pick from trees [2,3,2,2].
 If we had started at the first tree, we would only pick from trees [1,2].

Constraints:

- $1 \leq \text{fruits.length} \leq 105$
- $0 \leq \text{fruits}[i] < \text{fruits.length}$

Community Solutions (Python)

- WalkCC

Round 2 · High · Medium

p.585

```
class Solution:
    def totalFruit(self, fruits: list[int]) -> int:
        ans = 0
        count = collections.defaultdict(int)

        l = 0
        for r, fruit in enumerate(fruits):
            count[fruit] += 1
            while len(count) > 2:
                count[fruits[l]] -= 1
                if count[fruits[l]] == 0:
                    del count[fruits[l]]
                l += 1
            ans = max(ans, r - l + 1)

        return ans
```

• LeetDoocs

```
class Solution:
    def totalFruit(self, fruits: List[int]) -> int:
        cnt = Counter()
        ans = j = 0
        for i, x in enumerate(fruits):
            cnt[x] += 1
            while len(cnt) > 2:
                y = fruits[j]
                cnt[y] -= 1
                if cnt[y] == 0:
                    cnt.pop(y)
                j += 1
            ans = max(ans, i - j + 1)
        return ans
```

• SimplyLeet

Round 2 · High · Medium

p.586

Python solution for Fruit Into Baskets

```
def totalFruit(fruits):
    # Dictionary to track the count of each fruit type in the current window
    fruit_count = {}
    left = 0 # left index of our sliding window
    max_fruits = 0 # maximum number of fruits collected so far

    # Iterate over fruits with the index 'right'
    for right, fruit in enumerate(fruits):

        # Add/update the current fruit in the window
        fruit_count[fruit] = fruit_count.get(fruit, 0) + 1

        # If the window contains more than 2 distinct fruit types, shrink it
        # from the left
        while len(fruit_count) > 2:
            fruit_count[fruits[left]] -= 1
            # If the count drops to 0, remove that fruit type from the
            # dictionary
            if fruit_count[fruits[left]] == 0:
                del fruit_count[fruits[left]]
            left += 1

        # Update the maximum fruits picked
        max_fruits = max(max_fruits, right - left + 1)

    return max_fruits

# Example usage:
print(totalFruit([1, 2, 1])) # Expected Output: 3
```

Sliding Window – Dynamic Size

□ #1004 Max Consecutive Ones III

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Binary Search · Sliding Window · Prefix Sum

Lists: AlgoMaster 600

Problem

Given a binary array `nums` and an integer `k`, return the maximum number of consecutive 1's in the array if you can flip at most `k` 0's.

Example 1:

Input: `nums = [1,1,1,0,0,0,1,1,1,1,0]`, `k = 2`
 Output: 6
 Explanation: `[1,1,1,0,0,0,1,1,1,1,1]`
 Bolded numbers were flipped from 0 to 1. The longest subarray is underlined.

Example 2:

Input: `nums = [0,0,1,1,0,0,1,1,1,0,1,1,0,0,0,1,1,1,1]`, `k = 3`
 Output: 10
 Explanation: `[0,0,1,1,1,1,1,1,1,1,0,0,0,1,1,1,1]`
 Bolded numbers were flipped from 0 to 1. The longest subarray is underlined.

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- `nums[i]` is either 0 or 1.
- $0 \leq k \leq \text{nums.length}$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def longestOnes(self, nums: List[int], k: int) -> int:
        ans = 0

        l = 0
        for r, num in enumerate(nums):
            if num == 0:
                k -= 1
            while k < 0:
                if nums[l] == 0:
                    k += 1
                l += 1
            ans = max(ans, r - l + 1)

        return ans
```

• LeetCode

```
class Solution:
    def longestOnes(self, nums: List[int], k: int) -> int:
        l = cnt = 0
        for x in nums:
            cnt += x ^ 1
            if cnt > k:
                cnt -= nums[l] ^ 1
                l += 1
        return len(nums) - l
```

• SimplyLeet

Round 2 · High · Medium

p.589

```
# Python solution with line-by-line comments
def longestOnes(nums, k):
    # Initialize two pointers for the sliding window and zero_count to count
    # zeros in the window.
    left = 0
    max_length = 0
    zero_count = 0

    # Expand the window using the right pointer
    for right in range(len(nums)):
        # If the right element is 0, increment the zero counter
        if nums[right] == 0:
            zero_count += 1

        # If there are more than k zeros, shrink the window from the left
        while zero_count > k:
            if nums[left] == 0:
                zero_count -= 1
            left += 1

        # Update the maximum length with the current window size
        max_length = max(max_length, right - left + 1)

    return max_length

# Example usage:
# print(longestOnes([1,1,1,0,0,0,1,1,1,0], 2))
```

Round 2 · High · Medium

p.590

Sliding Window – Dynamic Size

☐ #1248 Count Number of Nice Subarrays **MED**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Math · Sliding Window ·
Prefix Sum

Lists: CodeSignal

Problem

Given an array of integers `nums` and an integer `k`. A continuous subarray is called nice if there are `k` odd numbers on it.

Return the number of nice sub-arrays.

Example 1:

Input: `nums = [1,1,2,1,1]`, `k = 3`
Output: 2
Explanation: The only sub-arrays with 3 odd numbers are `[1,1,2,1]` and `[1,2,1,1]`.

Example 2:

Input: `nums = [2,4,6]`, `k = 1`
Output: 0
Explanation: There are no odd numbers in the array.

Example 3:

Input: `nums = [2,2,2,1,2,2,1,2,2,2]`, `k = 2`
Output: 16

Constraints:

Round 2 · High · Medium

p.591

- `1 <= nums.length <= 50000`
- `1 <= nums[i] <= 10^5`
- `1 <= k <= nums.length`

Community Solutions (Python)

☐ • WalkCC

```
class Solution:
    def numberOfSubarrays(self, nums: list[int], k: int) -> int:
        def numberOfSubarraysAtMost(k: int) -> int:
            ans = 0
            l = 0
            r = 0

            while r <= len(nums):
                if k >= 0:
                    ans += r - l

                if r == len(nums):
                    break
                if nums[r] & 1:
                    k -= 1
                    r += 1
                else:
                    if nums[l] & 1:
                        k += 1
                        l += 1
                    return ans

            return numberOfSubarraysAtMost(k) - numberOfSubarraysAtMost(k - 1)
```

☐ • LeetDoocs

Round 2 · High · Medium

p.592

```
class Solution:
    def numberOfSubarrays(self, nums: List[int], k: int) -> int:
        cnt = Counter({0: 1})
        ans = t = 0
        for v in nums:
            t += v & 1
            ans += cnt[t - k]
            cnt[t] += 1
        return ans
```

• SimplyLeet

Python solution with line-by-line comments

```
def numberOfSubarrays(nums, k):
    # Initialize dictionary to store frequency of prefix sums
    prefix_count = {0: 1} # There is one way to have zero odd numbers
    initially.
    count = 0 # This will store the final count of nice subarrays
    odd_sum = 0 # Prefix sum representing the count of odd numbers so far

    # Iterate through each number in the array
    for num in nums:
```

```
    # If the number is odd, increment the odd_count prefix sum
    if num % 2 == 1:
        odd_sum += 1
```

```
    # If there is a prefix such that (current odd_sum - prefix) equals k,
    # then we found subarrays ending here with k odd numbers
    count += prefix_count.get(odd_sum - k, 0)
```

```
    # Update the frequency for the current odd_sum in the hash table
    prefix_count[odd_sum] = prefix_count.get(odd_sum, 0) + 1
```

```
    # Return the total number of nice subarrays
    return count
```

```
# Example usage:
# nums = [1,1,2,1,1]
# k = 3
# print(numberOfSubarrays(nums, k)) # Expected output is 2
```

• LC.ca

```
class Solution:
    def numberOfSubarrays(self, nums: List[int], k: int) -> int:
        cnt = Counter({0: 1})
        ans = t = 0
        for v in nums:
            t += v & 1
            ans += cnt[t - k]
            cnt[t] += 1
        return ans
```

◆ Quick Review

Key Insights

- Convert the problem to counting subarrays with a specific "prefix sum" value where the sum represents the count of odd numbers.
- Use a hash table (or dictionary) to store the frequency of prefix sums encountered.
- The number of nice subarrays ending at the current index equals the frequency of (current prefix sum - k) seen so far.
- This approach allows you to find the answer in one pass over the array.

Space & Time

Time Complexity: $O(n)$, where n is the number of elements in `nums`, because we traverse the array once.

Space Complexity: $O(n)$, in the worst case where each prefix sum is unique and stored in the hash table.

Solution

We solve the problem using a prefix sum approach. First, we iterate through the array and convert it into a prefix sum representing the cumulative count of odd numbers encountered. As we compute the prefix sums, we use a hash table to record how many times each prefix sum appears. For every current prefix sum value, we check how many times we have seen a prefix sum equal to (current prefix sum - k). The idea is that if $\text{prefix}[j] - \text{prefix}[i]$ equals k, then between indices $i+1$ and j , there are exactly k odd numbers. This gives us the count of valid subarrays ending at the current index.

This method works efficiently as it requires only a single pass through the array and maintains a running frequency count using the hash table.

Sliding Window – Dynamic Size

□ #1423 Maximum Points You Can Obtain from Cards

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Sliding Window · Prefix Sum
Lists: AlgoMaster 600

Problem

There are several cards arranged in a row, and each card has an associated number of points. The points are given in the integer array `cardPoints`.

In one step, you can take one card from the beginning or from the end of the row. You have to take exactly k cards.

Your score is the sum of the points of the cards you have taken.

Given the integer array `cardPoints` and the integer k, return the maximum score you can obtain.

Example 1:

Input: cardPoints = [1,2,3,4,5,6,1], k = 3

Output: 12

Explanation: After the first step, your score will always be 1. However, choosing the rightmost card first will maximize your total score. The optimal strategy is to take the three cards on the right, giving a final score of 1 + 6 + 5 = 12.

Example 2:

Input: cardPoints = [2,2,2], k = 2

Output: 4

Explanation: Regardless of which two cards you take, your score will always be 4.

Example 3:

Input: cardPoints = [9,7,7,9,7,7,9], k = 7

Output: 55

Explanation: You have to take all the cards. Your score is the sum of points of all cards.

Constraints:

- $1 \leq \text{cardPoints.length} \leq 105$
- $1 \leq \text{cardPoints}[i] \leq 104$
- $1 \leq k \leq \text{cardPoints.length}$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxScore(self, cardPoints: list[int], k: int) -> int:
        n = len(cardPoints)
        summ = sum(cardPoints)
        windowSum = sum(cardPoints[:n - k])
        ans = summ - windowSum

        for i in range(k):
            windowSum -= cardPoints[i]
            windowSum += cardPoints[i + n - k]
```

```
ans = max(ans, summ - windowSum)
```

```
return ans
```

• LeetCode

```
class Solution:
    def maxScore(self, cardPoints: List[int], k: int) -> int:
        ans = s = sum(cardPoints[-k:])
        for i, x in enumerate(cardPoints[:k]):
            s += x - cardPoints[-k + i]
            ans = max(ans, s)
        return ans
```

```
# @param {Integer[]} card_points
# @param {Integer} k
# @return {Integer}
def max_score(card_points, k)
  n = card_points.length
  s = card_points[-k..].sum
  ans = s
  k.times do |i|
    s += card_points[i] - card_points[n - k + i]
    ans = [ans, s].max
  end
end
```

• SimplyLeet

```
# Function to calculate the maximum points obtainable
def maxScore(cardPoints, k):
    n = len(cardPoints)
    # If k equals the total number of cards, return the total sum of cardPoints
    if k == n:
        return sum(cardPoints)

    total_sum = sum(cardPoints)
    window_size = n - k # Size of the subarray we are leaving behind
```

```

# Compute the sum of the first window
current_window_sum = sum(cardPoints[:window_size])
min_window_sum = current_window_sum

# Slide the window from index 1 to n - window_size
for i in range(window_size, n):
    # Subtract the element going out of the window and add the element
    # coming into the window
    current_window_sum += cardPoints[i] - cardPoints[i - window_size]
    # Keep track of the minimum sum we've seen in the window
    if current_window_sum < min_window_sum:

        min_window_sum = current_window_sum

# Maximum points is the total sum minus the minimum window sum
return total_sum - min_window_sum

# Example usage:
print(maxScore([1,2,3,4,5,6,1], 3)) # Output: 12

```

Sliding Window – Dynamic Size

#1838 Frequency of the Most Frequent Element

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Binary Search · Greedy · Sliding Window
 · Sorting · Prefix Sum

Lists: AlgoMaster 600

Problem

The frequency of an element is the number of times it occurs in an array.

You are given an integer array `nums` and an integer `k`. In one operation, you can choose an index of `nums` and increment the element at that index by 1.

Return the maximum possible frequency of an element after performing at most `k` operations.

Example 1:

Input: `nums = [1,2,4]`, `k = 5`
 Output: 3
 Explanation: Increment the first element three times and the second element two times to make `nums = [4,4,4]`.
 4 has a frequency of 3.

Example 2:

Input: nums = [1,4,8,13], k = 5

Output: 2

Explanation: There are multiple optimal solutions:

- Increment the first element three times to make nums = [4,4,8,13]. 4 has a frequency of 2.
- Increment the second element four times to make nums = [1,8,8,13]. 8 has a frequency of 2.
- Increment the third element five times to make nums = [1,4,13,13]. 13 has a frequency of 2.

Example 3:

Input: nums = [3,9,6], k = 2

Output: 1

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $1 \leq \text{nums}[i] \leq 105$
- $1 \leq k \leq 105$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxFrequency(self, nums: list[int], k: int) -> int:
        ans = 0
        summ = 0

        nums.sort()

        l = 0
        for r, num in enumerate(nums):
            summ += num
```

```
while summ + k < num * (r - l + 1):
    summ -= nums[l]
    l += 1
ans = max(ans, r - l + 1)

return ans
```

• LeetDoocs

```
class Solution:
    def maxFrequency(self, nums: List[int], k: int) -> int:
        def check(m: int) -> bool:
            for i in range(m, n + 1):
                if nums[i - 1] * m - (s[i] - s[i - m]) <= k:
                    return True
            return False

        n = len(nums)
        nums.sort()

        s = list(accumulate(nums, initial=0))
        l, r = 1, n
        while l < r:
            mid = (l + r + 1) >> 1
            if check(mid):
                l = mid
            else:
                r = mid - 1
        return l
```

```
class Solution:
    def maxFrequency(self, nums: List[int], k: int) -> int:
        nums.sort()
        ans = 1
        s = j = 0
        for i in range(1, len(nums)):
            s += (nums[i] - nums[i - 1]) * (i - j)
            while s > k:
                s -= nums[i] - nums[j]
                j += 1

        ans = max(ans, i - j + 1)
        return ans
```

• SimplyLeet

```
# Python solution with line-by-line comments
class Solution:
    def maxFrequency(self, nums: List[int], k: int) -> int:
        # Sort the array to group similar values together
        nums.sort()
        left = 0 # left pointer of the sliding window
        total_sum = 0 # sum of elements in the current window
        max_freq = 0 # the maximum frequency found

        # Iterate through the array using right pointer

        for right in range(len(nums)):
            # Add the current element to the running sum
            total_sum += nums[right]
            # Calculate cost to make all elements in window equal to
            nums[right]
            # If cost exceeds k, move left pointer to shrink the window
            while nums[right] * (right - left + 1) - total_sum > k:
                total_sum -= nums[left]
                left += 1
            # Update maximum frequency if the current window is larger
            max_freq = max(max_freq, right - left + 1)

        return max_freq
```

Binary Search

Round 2 · High · Medium · 14 questions

Round 2 · High · Medium

p.603

Round 2 · High · Medium

p.604

Binary Search

#33 Search in Rotated Sorted Array

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Binary Search
 Lists: Grind 169

Problem

There is an integer array `nums` sorted in ascending order (with distinct values).

Prior to being passed to your function, `nums` is possibly left rotated at an unknown index `k` ($1 \leq k < \text{nums.length}$) such that the resulting array is `[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]` (0-indexed). For example, `[0,1,2,4,5,6,7]` might be left rotated by 3 indices and become `[4,5,6,7,0,1,2]`.

Given the array `nums` after the possible rotation and an integer `target`, return the index of `target` if it is in `nums`, or `-1` if it is not in `nums`.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Round 2 · High · Medium

p.605

Input: `nums = [4,5,6,7,0,1,2]`, `target = 0`
 Output: 4

Example 2:

Input: `nums = [4,5,6,7,0,1,2]`, `target = 3`
 Output: -1

Example 3:

Input: `nums = [1]`, `target = 0`
 Output: -1

Constraints:

- $1 \leq \text{nums.length} \leq 5000$
- $-104 \leq \text{nums}[i] \leq 104$
- All values of `nums` are unique.
- `nums` is an ascending array that is possibly rotated.
- $-104 \leq \text{target} \leq 104$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def search(self, nums: list[int], target: int) -> int:
        l = 0
        r = len(nums) - 1

        while l <= r:
            m = (l + r) // 2
            if nums[m] == target:
                return m
            if nums[l] <= nums[m]: # nums[l..m] are sorted.
```

Round 2 · High · Medium

p.606

```

    if nums[l] <= target < nums[m]:
        r = m - 1
    else:
        l = m + 1
else: # nums[m..n - 1] are sorted.
    if nums[m] < target <= nums[r]:
        l = m + 1
    else:
        r = m - 1

```

```

return -1

```

• LeetCode

```

class Solution:
    def search(self, nums: List[int], target: int) -> int:
        n = len(nums)
        left, right = 0, n - 1
        while left < right:
            mid = (left + right) >> 1
            if nums[0] <= nums[mid]:
                if nums[0] <= target <= nums[mid]:
                    right = mid
                else:
                    left = mid + 1
            else:
                if nums[mid] < target <= nums[n - 1]:
                    left = mid + 1
                else:
                    right = mid
        return left if nums[left] == target else -1

```

• SimplyLeet

Round 2 · High · Medium

p.607

```

# Define a function to search for the target in a rotated sorted array.
def search(nums, target):
    # Initialize pointers for binary search.
    left, right = 0, len(nums) - 1

    # Loop until the search space is exhausted.
    while left <= right:
        # Compute the middle index.
        mid = (left + right) // 2

        # If the mid element is the target, return its index.
        if nums[mid] == target:
            return mid

        # Check if the left part of the array is sorted.
        if nums[left] <= nums[mid]:
            # If the target is within the sorted left half, adjust the right
            # pointer.
            if nums[left] <= target < nums[mid]:
                right = mid - 1
            else:
                # Otherwise, search the right half.
                left = mid + 1
        else:
            # The right half must be sorted.
            if nums[mid] < target <= nums[right]:
                left = mid + 1
            else:
                right = mid - 1

        # If the target is not found, return -1.

    return -1

# Example usage:
# result = search([4,5,6,7,0,1,2], 0)
# print(result) # Expected output: 4

```

• LC.ca

Round 2 · High · Medium

p.608

```
# below 2 solutions, diff is while condition: left ('<' or '<=') right
```

```
# I like this better
```

```
class Solution:
    def search(self, nums: List[int], target: int) -> int:
        n = len(nums)
        left, right = 0, n - 1
        while left <= right:
            mid = (left + right) >> 1
            if nums[mid] == target:
```

```
                return mid
            elif nums[0] <= nums[mid]: # left half sorted
                if nums[0] <= target <= nums[mid]:
                    right = mid - 1
                else:
                    left = mid + 1
            else: # right half sorted
                if nums[mid] < target <= nums[n - 1]:
                    left = mid + 1
                else:
```

```
                    right = mid - 1
        return -1
```

```
#####
```

```
class Solution:
    def search(self, nums: List[int], target: int) -> int:
        n = len(nums)
        left, right = 0, n - 1
        while left < right:
```

```
            mid = (left + right) >> 1
            if nums[0] <= nums[mid]: # left half sorted
                if nums[0] <= target <= nums[mid]:
                    right = mid
                else:
                    left = mid + 1
            else: # right half sorted
                if nums[mid] < target <= nums[n - 1]:
                    left = mid + 1
                else:
```

Round 2 · High · Medium

p.609

```
        right = mid
        return left if nums[left] == target else -1
```

◆ Quick Review

Key Insights

- The array is originally sorted and then rotated at an unknown pivot.
- Use a modified binary search to accommodate the rotation.
- At every step, determine which half of the array is properly sorted.
- Narrow down the search based on the sorted half and the target's value.

Space & Time

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

Solution

We perform a binary search while accounting for the possibility that the array is rotated. At each iteration, we check if the left-to-mid portion is sorted. If it is, and the target falls within that range, we search the left half; otherwise, we search the right half. If the left

Round 2 · High · Medium

p.610

half is not sorted, then the right half must be sorted, and similarly, we check if the target is in that sorted half. This approach ensures logarithmic time complexity while handling the rotation correctly.

Round 2 · High · Medium

p.611

Binary Search

□ #34 Find First and Last Position of Element in Sorted Array

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search
Lists: Denny Zhang

Problem

Given an array of integers `nums` sorted in non-decreasing order, find the starting and ending position of a given target value.

If target is not found in the array, return `[-1, -1]`.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: `nums = [5,7,7,8,8,10]`, `target = 8`
Output: `[3,4]`

Example 2:

Input: `nums = [5,7,7,8,8,10]`, `target = 6`
Output: `[-1,-1]`

Example 3:

Input: `nums = []`, `target = 0`
Output: `[-1,-1]`

Round 2 · High · Medium

p.612

Constraints:

- $0 \leq \text{nums.length} \leq 105$
- $-109 \leq \text{nums}[i] \leq 109$
- **nums** is a non-decreasing array.
- $-109 \leq \text{target} \leq 109$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def searchRange(self, nums: List[int], target: int) -> List[int]:
        l = bisect_left(nums, target)
        if l == len(nums) or nums[l] != target:
            return -1, -1
        r = bisect_right(nums, target) - 1
        return l, r
```

• LeetDoocs

```
class Solution:
    def searchRange(self, nums: List[int], target: int) -> List[int]:
        l = bisect_left(nums, target)
        r = bisect_left(nums, target + 1)
        return [-1, -1] if l == r else [l, r - 1]
```

• SimplyLeet

Round 2 · High · Medium

p.613

```
def searchRange(nums, target):
    # Helper function to find the boundary index (leftmost or rightmost) for
    # target.
    def findBoundary(isLeft):
        left, right = 0, len(nums) - 1
        boundary = -1
        while left <= right:
            mid = (left + right) // 2
            # If the target is found, update boundary and continue to search on
            # one side.
            if nums[mid] == target:
                boundary = mid

        if isLeft:
            right = mid - 1 # Continue search on left half
        else:
            left = mid + 1 # Continue search on right half
        elif nums[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
        return boundary

    # Find the leftmost index
    leftIndex = findBoundary(True)
    if leftIndex == -1:
        return [-1, -1]
    # Find the rightmost index
    rightIndex = findBoundary(False)
    return [leftIndex, rightIndex]

# Example usage:
# nums = [5,7,7,8,8,10]

# target = 8
# print(searchRange(nums, target)) # Output: [3,4]
```

• LC.ca

Round 2 · High · Medium

p.614

```
'''
>>> bisect.bisect_left([1,1,1,2,2,2,7,7,7], 2)
3
>>> bisect.bisect_left([1,1,1,2,2,2,7,7,7], 2+1)
6

>>> bisect.bisect_right([1,1,1,2,2,2,7,7,7], 2)
6
>>> bisect.bisect_right([1,1,1,2,2,2,7,7,7], 2+1)

6

>>> bisect.bisect_left([1,1,1,2,2,2,7,7,7], 0)
0
>>> bisect.bisect_left([1,1,1,2,2,2,7,7,7], 1)
0

# below, even single value for 2, after +1 the index will be different
>>> bisect.bisect_left([1,2,3,4,5], 2)

1
>>> bisect.bisect_left([1,2,3,4,5], 2+1)
2
'''

import bisect

class Solution:
    def searchRange(self, nums: List[int], target: int) -> List[int]:
        l = bisect_left(nums, target)

        r = bisect_left(nums, target + 1)
        return [-1, -1] if l == r else [l, r - 1]

#####

class Solution:
    def searchRange(self, nums: List[int], target: int) -> List[int]:
        def findRange(to_left):
            l, r = 0, len(nums) - 1
            m = 0
```

Round 2 · High · Medium

p.615

```
while l <= r:
    m = l + (r - l) // 2
    if nums[m] < target:
        l = m + 1
    elif nums[m] > target:
        r = m - 1
    elif to_left and m > 0 and nums[m - 1] == nums[m]: # so now
nums[m] == target, but maybe not the leftmost or rightmost
        r = m - 1
    elif not to_left and m + 1 < len(nums) and nums[m + 1] == nums
[m]: # so now nums[m] == target, but maybe not the leftmost or rightmost
        l = m + 1

else:
    return m
if m < len(nums) and nums[m] == target:
    # 1. cover not-found,
    # or 2. cover input=[3] and target=3, first and last both 0, return
[0,0]

    return m
else:
    return -1
return [findRange(True), findRange(False)]
```

◆ Quick Review

Key Insights

- The array is sorted, allowing the use of binary search.
- Two binary searches can efficiently find the leftmost (first) and rightmost (last) indices of the target.
- Handling edge cases where the target is not present by checking bounds.

Round 2 · High · Medium

p.616

Space & Time

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

Solution

The approach is to use binary search twice:

- The first binary search finds the leftmost occurrence of the target by continuing to search in the left half even after finding the target.
 - The second binary search finds the rightmost occurrence by searching in the right half.
- The algorithm maintains pointers for the current search bounds and narrows the search space using comparisons. If the target is not found during the first binary search, the solution immediately returns $[-1, -1]$. This method leverages the sorted property of the array, ensuring $O(\log n)$ time complexity with constant extra space.

Round 2 · High · Medium

p.617

Binary Search

□ #74 Search a 2D Matrix

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Binary Search · Matrix
 Lists: Grind 169

Problem

You are given an $m \times n$ integer matrix `matrix` with the following two properties:

- Each row is sorted in non-decreasing order.
- The first integer of each row is greater than the last integer of the previous row.

Given an integer `target`, return `true` if `target` is in `matrix` or `false` otherwise.

You must write a solution in $O(\log(m * n))$ time complexity.

Example 1:

Round 2 · High · Medium

p.618

1	3	5	7
10	11	16	20
23	30	34	60

Input: matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 3
Output: true

Example 2:

Round 2 · High · Medium

p.619

1	3	5	7
10	11	16	20
23	30	34	60

Input: matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 13
Output: false

Constraints:

- $m == \text{matrix.length}$
- $n == \text{matrix}[i].\text{length}$
- $1 \leq m, n \leq 100$
- $-104 \leq \text{matrix}[i][j], \text{target} \leq 104$

Community Solutions (Python)

- WalkCC

Round 2 · High · Medium

p.620

```
class Solution:
    def searchMatrix(self, matrix: list[list[int]], target: int) -> bool:
        if not matrix:
            return False

        m = len(matrix)
        n = len(matrix[0])
        l = 0
        r = m * n
```

```
while l < r:
    mid = (l + r) // 2
    i = mid // n
    j = mid % n
    if matrix[i][j] == target:
        return True
    if matrix[i][j] < target:
        l = mid + 1
    else:
        r = mid
```

```
return False
```

• LeetCode

```
class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        m, n = len(matrix), len(matrix[0])
        left, right = 0, m * n - 1
        while left < right:
            mid = (left + right) >> 1
            x, y = divmod(mid, n)
            if matrix[x][y] >= target:
                right = mid
            else:
```

```
        left = mid + 1
        return matrix[left // n][left % n] == target
```

• SimplyLeet

Round 2 · High · Medium

p.621

```
def searchMatrix(matrix, target):
    # Get number of rows and columns
    if not matrix or not matrix[0]:
        return False

    rows, cols = len(matrix), len(matrix[0])
    left, right = 0, rows * cols - 1

    # Perform binary search over the virtual flattened array
    while left <= right:

        # Find the middle index
        mid = (left + right) // 2
        # Map the middle index to row and column in the matrix
        row = mid // cols
        col = mid % cols
        mid_value = matrix[row][col]

        # Check if the mid_value equals the target
        if mid_value == target:
            return True

        elif mid_value < target:
            left = mid + 1 # Move to the right subarray
        else:
            right = mid - 1 # Move to the left subarray

    # Target not found
    return False

# Example usage:
# matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]]

# target = 3
# print(searchMatrix(matrix, target))
```

• LC.ca

Round 2 · High · Medium

p.622

```

class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        m, n = len(matrix), len(matrix[0])
        left, right = 0, m * n - 1
        while left <= right: # must be <=, not <, for matrix=[[1]],target=1
            mid = (left + right) >> 1
            x, y = divmod(mid, n) # note: divide column count
            if matrix[x][y] == target:
                return True
            elif matrix[x][y] > target:
                right = mid - 1
            else:
                left = mid + 1
        return False

#####

class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        m, n = len(matrix), len(matrix[0])

        left, right = 0, m * n - 1
        while left < right:
            mid = (left + right) >> 1
            x, y = divmod(mid, n)
            if matrix[x][y] >= target:
                right = mid
            else:
                left = mid + 1
        return matrix[left // n][left % n] == target

```

◆ Quick Review

Key Insights

- The matrix is essentially a flattened sorted array due to its properties.

Round 2 · High · Medium

p.623

- Use binary search to efficiently determine if the target exists.
- Map the 1D binary search index to the corresponding 2D matrix coordinates using division and modulus operations.
- Achieve $O(\log(m*n))$ time complexity by navigating the matrix as if it were a single sorted list.

Space & Time

Time Complexity: $O(\log(m * n))$

Space Complexity: $O(1)$

Solution

We can treat the matrix as a flattened sorted list without physically copying the elements. Given the total number of elements = $m * n$, perform binary search on indices ranging from 0 to $(m*n - 1)$. For any index mid , calculate its corresponding row as $mid // n$ and column as $mid \% n$. Compare the element at $matrix[row][col]$ with the target. If it equals target, return true; if it is less than target, search the right half; otherwise, search the left half.

Round 2 · High · Medium

p.624

Binary Search

☐ #81 Search in Rotated Sorted Array II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Binary Search
 Lists: AlgoMaster 600

Problem

There is an integer array `nums` sorted in non-decreasing order (not necessarily with distinct values).

Before being passed to your function, `nums` is rotated at an unknown pivot index k ($0 \leq k < \text{nums.length}$) such that the resulting array is `[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]` (0-indexed). For example, `[0,1,2,4,4,4,5,6,6,7]` might be rotated at pivot index 5 and become `[4,5,6,6,7,0,1,2,4,4]`.

Given the array `nums` after the rotation and an integer `target`, return `true` if `target` is in `nums`, or `false` if it is not in `nums`.

You must decrease the overall operation steps as much as possible.

Round 2 · High · Medium

p.625

Example 1:

Input: `nums = [2,5,6,0,0,1,2]`, `target = 0`
 Output: `true`

Example 2:

Input: `nums = [2,5,6,0,0,1,2]`, `target = 3`
 Output: `false`

Constraints:

- $1 \leq \text{nums.length} \leq 5000$
- $-104 \leq \text{nums}[i] \leq 104$
- `nums` is guaranteed to be rotated at some pivot.
- $-104 \leq \text{target} \leq 104$

Follow up: This problem is similar to Search in Rotated Sorted Array, but `nums` may contain duplicates. Would this affect the runtime complexity? How and why?

Community Solutions (Python)

☐ • WalkCC

```
class Solution:
    def search(self, nums: List[int], target: int) -> bool:
        l = 0
        r = len(nums) - 1

        while l <= r:
            m = (l + r) // 2
            if nums[m] == target:
                return True
            if nums[l] == nums[m] == nums[r]:
```

Round 2 · High · Medium

p.626

```

    l += 1
    r -= 1
elif nums[l] <= nums[m]: # nums[l..m] are sorted
    if nums[l] <= target < nums[m]:
        r = m - 1
    else:
        l = m + 1
else: # nums[m..n - 1] are sorted
    if nums[m] < target <= nums[r]:
        l = m + 1

    else:
        r = m - 1

return False

```

• LeetCode

```

class Solution:
    def search(self, nums: List[int], target: int) -> bool:
        n = len(nums)
        l, r = 0, n - 1
        while l < r:
            mid = (l + r) >> 1
            if nums[mid] > nums[r]:
                if nums[l] <= target <= nums[mid]:
                    r = mid
                else:
                    l = mid + 1
            elif nums[mid] < nums[r]:
                if nums[mid] < target <= nums[r]:
                    l = mid + 1
                else:
                    r = mid
            else:
                r -= 1
        return nums[l] == target

```

• SimplyLeet

Round 2 · High · Medium

p.627

```

# Function to search for the target in a rotated sorted array that may contain
duplicates.
class Solution:
    def search(self, nums: List[int], target: int) -> bool:
        low, high = 0, len(nums) - 1 # Initialize two pointers.

        while low <= high:
            mid = (low + high) // 2 # Find the mid-point.
            if nums[mid] == target: # If the mid element is the target, return
                True.

                return True

        # If the values at low, mid, and high are equal, we cannot decide
        which half is sorted.
        if nums[low] == nums[mid] == nums[high]:
            low += 1 # Shrink the search window.
            high -= 1
        # If the left half is sorted.
        elif nums[low] <= nums[mid]:
            if nums[low] <= target < nums[mid]: # Check if target is in
                the left half.

                high = mid - 1
            else:
                low = mid + 1

        # Otherwise, the right half must be sorted.
        else:
            if nums[mid] < target <= nums[high]: # Check if target is in
                the right half.

                low = mid + 1
            else:
                high = mid - 1

        return False # Target not found.

```

Round 2 · High · Medium

p.628

Binary Search

☐ #153 Find Minimum in Rotated Sorted Array

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search
Lists: Grind 169

Problem

Suppose an array of length n sorted in ascending order is rotated between 1 and n times. For example, the array `nums = [0,1,2,4,5,6,7]` might become:

- `[4,5,6,7,0,1,2]` if it was rotated 4 times.
- `[0,1,2,4,5,6,7]` if it was rotated 7 times.

Notice that rotating an array `[a[0], a[1], a[2], ..., a[n-1]]` 1 time results in the array `[a[n-1], a[0], a[1], a[2], ..., a[n-2]]`.

Given the sorted rotated array `nums` of unique elements, return the minimum element of this array.

You must write an algorithm that runs in $O(\log n)$ time.

Example 1:

Round 2 · High · Medium

p.629

Input: `nums = [3,4,5,1,2]`

Output: 1

Explanation: The original array was `[1,2,3,4,5]` rotated 3 times.

Example 2:

Input: `nums = [4,5,6,7,0,1,2]`

Output: 0

Explanation: The original array was `[0,1,2,4,5,6,7]` and it was rotated 4 times.

Example 3:

Input: `nums = [11,13,15,17]`

Output: 11

Explanation: The original array was `[11,13,15,17]` and it was rotated 4 times.

Constraints:

- $n == \text{nums.length}$
- $1 \leq n \leq 5000$
- $-5000 \leq \text{nums}[i] \leq 5000$
- All the integers of `nums` are unique.
- `nums` is sorted and rotated between 1 and n times.

Community Solutions (Python)

- WalkCC

Round 2 · High · Medium

p.630

```
class Solution:
    def findMin(self, nums: list[int]) -> int:
        l = 0
        r = len(nums) - 1

        while l < r:
            m = (l + r) // 2
            if nums[m] < nums[r]:
                r = m
            else:
                l = m + 1

        return nums[l]
```

• LeetCode

```
class Solution:
    def findMin(self, nums: List[int]) -> int:
        l, r = 0, len(nums) - 1
        while l < r:
            mid = (l + r) >> 1
            if nums[mid] > nums[-1]:
                l = mid + 1
            else:
                r = mid
        return nums[l]
```

• SimplyLeet

```
def findMin(nums):
    # Initialize left and right pointers
    left, right = 0, len(nums) - 1
    # Continue while the search space is valid
    while left < right:
        mid = (left + right) // 2 # Compute middle index
        # If middle element is greater than the rightmost element,
        # the minimum must be in the right half
        if nums[mid] > nums[right]:
            left = mid + 1
```

```
else:
    # Otherwise, the minimum is in the left half (including mid)
    right = mid
# When loop ends, left == right, pointing to the minimum element
return nums[left]
```

• LC.ca

```
class Solution:
    def findMin(self, nums: List[int]) -> int:
        if nums[0] <= nums[-1]:
            return nums[0]
        left, right = 0, len(nums) - 1
        while left < right:
            mid = (left + right) >> 1
            if nums[0] <= nums[mid]:
                left = mid + 1
            else:
                right = mid
        return nums[left]
```

◆ Quick Review

Key Insights

- The rotated sorted array consists of two sorted subarrays.
- A modified binary search can be used to locate the pivot point where the rotation occurs.
- By comparing the middle element to the rightmost element, we can decide which half of the array contains the minimum.

Space & Time

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

Solution

Use a binary search approach. Initialize two pointers, left and right, at the start and end of the array. While left is less than right, calculate the middle index. If the middle element is greater than the right element, then the minimum is in the right half, so move the left pointer to $\text{mid} + 1$; otherwise, the minimum must be in the left half including mid, so adjust the right pointer to mid. When the loop terminates, left equals right and points to the minimum element. This approach leverages the properties of the rotated sorted array to achieve logarithmic time complexity.

Round 2 · High · Medium

p.633

Binary Search

#162 Find Peak Element

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search
Lists: AlgoMaster 600

Problem

A peak element is an element that is strictly greater than its neighbors.

Given a 0-indexed integer array `nums`, find a peak element, and return its index. If the array contains multiple peaks, return the index to any of the peaks.

You may imagine that $\text{nums}[-1] = \text{nums}[n] = -\infty$. In other words, an element is always considered to be strictly greater than a neighbor that is outside the array.

You must write an algorithm that runs in $O(\log n)$ time.

Example 1:

Input: `nums = [1,2,3,1]`
Output: 2
Explanation: 3 is a peak element and your function should return the index number 2.

Round 2 · High · Medium

p.634

Example 2:

Input: nums = [1,2,1,3,5,6,4]

Output: 5

Explanation: Your function can return either index number 1 where the peak element is 2, or index number 5 where the peak element is 6.

Constraints:

- $1 \leq \text{nums.length} \leq 1000$
- $-231 \leq \text{nums}[i] \leq 231 - 1$
- $\text{nums}[i] \neq \text{nums}[i + 1]$ for all valid i .

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findPeakElement(self, nums: list[int]) -> int:
        l = 0
        r = len(nums) - 1

        while l < r:
            m = (l + r) // 2
            if nums[m] >= nums[m + 1]:
                r = m
            else:
                l = m + 1

        return l
```

• LeetCode

```
class Solution:
    def findPeakElement(self, nums: List[int]) -> int:
        left, right = 0, len(nums) - 1
        while left < right:
            mid = (left + right) >> 1
            if nums[mid] > nums[mid + 1]:
                right = mid
            else:
                left = mid + 1
        return left
```

• SimplyLeet

```
# Python solution for Find Peak Element

def findPeakElement(nums):
    # Initialize pointers for binary search
    low, high = 0, len(nums) - 1

    # Continue binary search until low meets high
    while low < high:
        mid = (low + high) // 2 # Find the middle index

        # If the element at mid is less than its right neighbor,
        # then a peak exists in the right half.
        if nums[mid] < nums[mid + 1]:
            low = mid + 1
        else:
            # Otherwise, a peak exists in the left half,
            # including possibly mid itself.
            high = mid

    # The low pointer eventually points to a peak element.
    return low

# Example usage:
# result = findPeakElement([1,2,3,1])
# print(result)
```

Binary Search

□ #240 Search a 2D Matrix II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Binary Search · Divide and Conquer ·
 Matrix

Lists: AlgoMaster 600

Problem

Write an efficient algorithm that searches for a value target in an m x n integer matrix matrix. This matrix has the following properties:

This matrix has the following properties:

- Integers in each row are sorted in ascending from left to right.
- Integers in each column are sorted in ascending from top to bottom.

Example 1:

1	4	7	11	15
2	5	8	12	19
3	6	9	16	22
10	13	14	17	24
18	21	23	26	30

Input: matrix =
 [[1,4,7,11,15],[2,5,8,12,19],[3,6,9,16,22],[10,13,14,17,24],[18,21,23,26,30]],
 target = 5
 Output: true

Example 2:

1	4	7	11	15
2	5	8	12	19
3	6	9	16	22
10	13	14	17	24
18	21	23	26	30

Input: matrix =
 [[1,4,7,11,15],[2,5,8,12,19],[3,6,9,16,22],[10,13,14,17,24],[18,21,23,26,30]],
 target = 20
 Output: false

Constraints:

- $m == \text{matrix.length}$
- $n == \text{matrix}[i].\text{length}$
- $1 \leq n, m \leq 300$
- $-109 \leq \text{matrix}[i][j] \leq 109$

Round 2 · High · Medium

p.639

- All the integers in each row are sorted in ascending order.
- All the integers in each column are sorted in ascending order.
- $-109 \leq \text{target} \leq 109$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        r = 0
        c = len(matrix[0]) - 1

        while r < len(matrix) and c >= 0:
            if matrix[r][c] == target:
                return True
            if target < matrix[r][c]:
                c -= 1
            else:
                r += 1

        return False
```

• LeetCode

```
class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        for row in matrix:
            j = bisect_left(row, target)
            if j < len(matrix[0]) and row[j] == target:
                return True
        return False
```

Round 2 · High · Medium

p.640

```
class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        m, n = len(matrix), len(matrix[0])
        i, j = m - 1, 0
        while i >= 0 and j < n:
            if matrix[i][j] == target:
                return True
            if matrix[i][j] > target:
                i -= 1
            else:
                j += 1
        return False
```

• SimplyLeet

```
# Python implementation with detailed comments

def searchMatrix(matrix, target):
    # If the matrix is empty, return False
    if not matrix or not matrix[0]:
        return False

    # Set initial position to top-right corner of the matrix
    row, col = 0, len(matrix[0]) - 1

    # Iterate until we either find the target or exhaust the matrix bounds
    while row < len(matrix) and col >= 0:
        # Store the current element
        current = matrix[row][col]
        # Check if the current element is equal to target
        if current == target:
            return True
        # If the current element is greater, move left
        elif current > target:
            col -= 1
```

Round 2 · High · Medium

p.641

```
# If smaller, move down
else:
    row += 1

# Target was not found
return False
```

Round 2 · High · Medium

p.642

Binary Search

#475 Heaters

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Two Pointers · Binary Search · Sorting
 Lists: AlgoMaster 600

Problem

Winter is coming! During the contest, your first job is to design a standard heater with a fixed warm radius to warm all the houses.

Every house can be warmed, as long as the house is within the heater's warm radius range. Given the positions of houses and heaters on a horizontal line, return the minimum radius standard of heaters so that those heaters could cover all houses.

Notice that all the heaters follow your radius standard, and the warm radius will be the same.

Example 1:

Input: houses = [1,2,3], heaters = [2]
 Output: 1
 Explanation: The only heater was placed in the position 2, and if we use the radius 1 standard, then all the houses can be warmed.

Example 2:

Round 2 · High · Medium

p.643

Input: houses = [1,2,3,4], heaters = [1,4]

Output: 1

Explanation: The two heaters were placed at positions 1 and 4. We need to use a radius 1 standard, then all the houses can be warmed.

Example 3:

Input: houses = [1,5], heaters = [2]

Output: 3

Constraints:

- $1 \leq \text{houses.length}, \text{heaters.length} \leq 3 * 10^4$
- $1 \leq \text{houses}[i], \text{heaters}[i] \leq 10^9$

Community Solutions (Python)

• LeetDoocs

```
class Solution:
    def findRadius(self, houses: List[int], heaters: List[int]) -> int:
        houses.sort()
        heaters.sort()

        def check(r):
            m, n = len(houses), len(heaters)
            i = j = 0
            while i < m:
                if j >= n:
                    return False
                mi = heaters[j] - r
                mx = heaters[j] + r
                if houses[i] < mi:
                    return False
                if houses[i] > mx:
                    j += 1
            else:
                i += 1
            return True
```

Round 2 · High · Medium

p.644

```

left, right = 0, int(1e9)
while left < right:
    mid = (left + right) >> 1
    if check(mid):
        right = mid
    else:
        left = mid + 1
return left

```

• SimplyLeet

```

# Function to compute the minimum radius required for heaters to warm all
houses
def findRadius(houses, heaters):
    # Sort both arrays for efficient searching
    houses.sort() # O(n log n)
    heaters.sort() # O(m log m)

    radius = 0
    # Iterate over each house and find the nearest heater using binary search
    for house in houses:
        low, high = 0, len(heaters) - 1

        # Binary search to find the right position in heaters array
        while low <= high:
            mid = (low + high) // 2
            if heaters[mid] < house:
                low = mid + 1
            else:
                high = mid - 1
        # low is the index of the first heater >= house (if exists)
        # high is the index of the last heater <= house (if exists)
        left_dist = abs(house - heaters[high]) if high >= 0 else float('inf')

```

```

        right_dist = abs(heaters[low] - house) if low < len(heaters) else float('inf')
        nearest = min(left_dist, right_dist)
        radius = max(radius, nearest)
    return radius

# Example usage:
houses = [1,5]
heaters = [2]
print(findRadius(houses, heaters)) # Expected Output: 3

```

Binary Search

#528 Random Pick with Weight

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · Binary Search · Prefix Sum ·
Randomized

Lists: Grind 169

Problem

You are given a 0-indexed array of positive integers w where $w[i]$ describes the weight of the i th index.

You need to implement the function `pickIndex()`, which randomly picks an index in the range $[0, w.length - 1]$ (inclusive) and returns it. The probability of picking an index i is $w[i] / \text{sum}(w)$.

- For example, if $w = [1, 3]$, the probability of picking index 0 is $1 / (1 + 3) = 0.25$ (i.e., 25%), and the probability of picking index 1 is $3 / (1 + 3) = 0.75$ (i.e., 75%).

Example 1:

Round 2 · High · Medium

p.647

```
Input
["Solution","pickIndex"]
[[[1]],[]]
Output
[null,0]
```

```
Explanation
Solution solution = new Solution([1]);
```

Example 2:

```
Input
["Solution","pickIndex","pickIndex","pickIndex","pickIndex","pickIndex"]
[[[1,3]],[],[],[],[],[]]
Output
[null,1,1,1,1,0]
```

```
Explanation
Solution solution = new Solution([1, 3]);
```

Constraints:

- $1 \leq w.length \leq 104$
- $1 \leq w[i] \leq 105$
- `pickIndex` will be called at most 104 times.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def __init__(self, w: List[int]):
        self.prefix = list(itertools.accumulate(w))

    def pickIndex(self) -> int:
        target = random.randint(0, self.prefix[-1] - 1)
        return bisect.bisect_right(range(len(self.prefix)), target,
                                   key=lambda m: self.prefix[m])
```

• LeetDoocs

Round 2 · High · Medium

p.648


```
class Solution:
    def __init__(self, w: List[int]):
        self.s = [0]
        for c in w:
            self.s.append(self.s[-1] + c)
```

```
def pickIndex(self) -> int:
    x = random.randint(1, self.s[-1])
    left, right = 1, len(self.s) - 1
    while left < right:
```

```
        mid = (left + right) >> 1
        if self.s[mid] >= x:
            right = mid
        else:
            left = mid + 1
    return left - 1
```

```
# Your Solution object will be instantiated and called as such:
# obj = Solution(w)
```

```
# param_1 = obj.pickIndex()
```

• SimplyLeet

```
import random
import bisect
```

```
class Solution:
    def __init__(self, w):
        # Build the prefix sum array.
        self.prefix_sums = []
        current_sum = 0
        for weight in w:
            current_sum += weight
```

```
        self.prefix_sums.append(current_sum)
        # Store the total sum of weights.
        self.total_sum = current_sum
```

```
def pickIndex(self):
    # Get a random target in the range [1, total_sum]
    target = random.randint(1, self.total_sum)
    # Use bisect_left to find the proper index.
    index = bisect.bisect_left(self.prefix_sums, target)
    return index
```

```
# Example usage:
# solution = Solution([1, 3])
# print(solution.pickIndex())
```

• LC.ca

```
class Solution:
    def __init__(self, w: List[int]):
        self.s = [0]
        for c in w:
            self.s.append(self.s[-1] + c)
```

```
def pickIndex(self) -> int:
    x = random.randint(1, self.s[-1])
    left, right = 1, len(self.s) - 1
    while left < right:
```

```
        mid = (left + right) >> 1
        if self.s[mid] >= x:
            right = mid
        else:
            left = mid + 1
    return left - 1
```

```
# Your Solution object will be instantiated and called as such:
# obj = Solution(w)
```

```
# param_1 = obj.pickIndex()
```

◆ Quick Review

Key Insights

- Build a prefix sum array from the weights.
- Calculate the total sum of the weights.
- Generate a random number in the range [1, total sum] and use binary search to find the corresponding index in the prefix sum array.
- Using binary search ensures efficient selection in $O(\log n)$ time for each call of `pickIndex()`.

Space & Time

Time Complexity: $O(n)$ for constructing the prefix sum array, and $O(\log n)$ per `pickIndex()` call.

Space Complexity: $O(n)$ for storing the prefix sum array.

Solution

The solution leverages a prefix sum array where each entry at index i represents the cumulative sum of weights up to i . After computing the total weight, a random integer is generated between 1 and the total sum (inclusive). A binary search is then used to find

the first index where the prefix sum is greater than or equal to the random number. This approach ensures that the index is selected with a probability proportional to its weight.

Binary Search

#658 Find K Closest Elements

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Sorting · Heap · Two
Pointers

Lists: Grind 169

Problem

Given a sorted integer array `arr`, two integers `k` and `x`, return the `k` closest integers to `x` in the array. The result should also be sorted in ascending order.

An integer `a` is closer to `x` than an integer `b` if:

- $|a - x| < |b - x|$, or
- $|a - x| == |b - x|$ and $a < b$

Example 1:

Input: `arr = [1,2,3,4,5]`, `k = 4`, `x = 3`

Output: `[1,2,3,4]`

Example 2:

Input: `arr = [1,1,2,3,4,5]`, `k = 4`, `x = -1`

Output: `[1,1,2,3]`

Constraints:

Round 2 · High · Medium

p.653

- $1 \leq k \leq \text{arr.length}$
- $1 \leq \text{arr.length} \leq 104$
- `arr` is sorted in ascending order.
- $-104 \leq \text{arr}[i], x \leq 104$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findClosestElements(self, arr: List[int], k: int, x: int) -> List[int]:
        l = 0
        r = len(arr) - k

        while l < r:
            m = (l + r) // 2
            if x - arr[m] <= arr[m + k] - x:
                r = m
            else:
                l = m + 1

        return arr[l:l + k]
```

• LeetDoocs

```
class Solution:
    def findClosestElements(self, arr: List[int], k: int, x: int) -> List[int]:
        arr.sort(key=lambda v: abs(v - x))
        return sorted(arr[:k])
```

• SimplyLeet

Round 2 · High · Medium

p.654

```
# Function to find k closest elements to x in a sorted array
def findClosestElements(arr, k, x):
    n = len(arr)
    # Initialize the binary search bounds for the starting index of the window
    left, right = 0, n - k
    # Binary search to find the best starting index
    while left < right:
        mid = left + (right - left) // 2
        # Compare the distance from x to arr[mid] and arr[mid+k]
        # If arr[mid] is farther from x than arr[mid+k], move window to the
        right

        if x - arr[mid] > arr[mid + k] - x:
            left = mid + 1
        else:
            # Otherwise, continue searching towards the left
            right = mid
    # Return the k elements starting from index `left`
    return arr[left:left + k]

# Example usage:
# arr = [1,2,3,4,5], k = 4, x = 3

# print(findClosestElements(arr, 4, 3)) # Output: [1,2,3,4]
```

• LC.ca

```
class Solution:
    def findClosestElements(self, arr: List[int], k: int, x: int) -> List[int]:
        left, right = 0, len(arr) - k
        while left < right:
            mid = (left + right) >> 1
            if x - arr[mid] <= arr[mid + k] - x:
                right = mid
            else:
                left = mid + 1
        return arr[left : left + k]
```

◆ Quick Review

Round 2 · High · Medium

p.655

Key Insights

- The input array is already sorted.
- A binary search can be used to efficiently narrow down the potential starting position.
- A two-pointers or sliding window approach is effective for selecting k consecutive elements.
- The comparison of absolute differences (with a tie-breaker on values) is crucial.
- Maintaining ascending order simplifies the final output.

Space & Time

Time Complexity: $O(\log n + k)$, where $O(\log n)$ for binary search and $O(k)$ for collecting results.

Space Complexity: $O(k)$, for storing the output.

Solution

We can solve this problem by first using binary search to identify the left-most window where the k closest elements might start. The binary search operates over the possible starting indices for windows of size k (from 0 to n-k). For each middle index mid, we compare the

Round 2 · High · Medium

p.656

distances between x and $\text{arr}[\text{mid}]$ and $\text{arr}[\text{mid} + k]$ to decide whether to move the window to the right or left. Once the correct starting position is found, return the k elements starting from that index. This approach leverages the sorted property of the array to efficiently converge on the optimal window.

Round 2 · High · Medium

p.657

Binary Search

□ #875 Koko Eating Bananas

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search
Lists: NeetCode 150

Problem

Koko loves to eat bananas. There are n piles of bananas, the i th pile has $\text{piles}[i]$ bananas. The guards have gone and will come back in h hours. Koko can decide her bananas-per-hour eating speed of k . Each hour, she chooses some pile of bananas and eats k bananas from that pile. If the pile has less than k bananas, she eats all of them instead and will not eat any more bananas during this hour.

Koko likes to eat slowly but still wants to finish eating all the bananas before the guards return. Return the minimum integer k such that she can eat all the bananas within h hours.

Example 1:

Input: $\text{piles} = [3, 6, 7, 11]$, $h = 8$
Output: 4

Round 2 · High · Medium

p.658

Example 2:

Input: piles = [30,11,23,4,20], h = 5
Output: 30

Example 3:

Input: piles = [30,11,23,4,20], h = 6
Output: 23

Constraints:

- $1 \leq \text{piles.length} \leq 104$
- $\text{piles.length} \leq h \leq 109$
- $1 \leq \text{piles}[i] \leq 109$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minEatingSpeed(self, piles: list[int], h: int) -> int:
        def eatHours(m: int) -> bool:
            """Returns the hours to eat all the piles with speed m."""
            return sum((pile - 1) // m + 1 for pile in piles)
        l = 1
        r = max(piles)
        return bisect.bisect_left(range(l, r), True,
                                key=lambda m: eatHours(m) <= h) + 1
```

• LeetDoocs

```
class Solution:
    def minEatingSpeed(self, piles: List[int], h: int) -> int:
        def check(k: int) -> bool:
            return sum((x + k - 1) // k for x in piles) <= h

        return 1 + bisect_left(range(1, max(piles) + 1), True, key=check)
```

• SimplyLeet

Round 2 · High · Medium

p.659

```
def minEatingSpeed(piles, h):
    # Helper function to determine if Koko can finish with speed k
    def canFinish(k):
        hours = 0
        for pile in piles:
            # Calculate hours by using ceiling division for each pile
            hours += (pile + k - 1) // k
        return hours <= h

    left, right = 1, max(piles) # Speed is between 1 and the max pile element

    while left < right:
        mid = (left + right) // 2 # Candidate eating speed
        if canFinish(mid):
            right = mid # Try finding a smaller valid speed
        else:
            left = mid + 1 # Increase the speed because mid is too slow
    return left

# Example usage:
piles = [3, 6, 7, 11]

h = 8
print(minEatingSpeed(piles, h)) # Expected output: 4
```

Round 2 · High · Medium

p.660

Binary Search

☐ #1011 Capacity to Ship Packages Within D Days MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search
Lists: Denny Zhang

Problem

A conveyor belt has packages that must be shipped from one port to another within *days* days.

The *i*th package on the conveyor belt has a weight of *weights[i]*. Each day, we load the ship with packages on the conveyor belt (in the order given by *weights*). We may not load more weight than the maximum weight capacity of the ship. Return the least weight capacity of the ship that will result in all the packages on the conveyor belt being shipped within *days* days.

Example 1:

```
Input: weights = [1,2,3,4,5,6,7,8,9,10], days = 5
Output: 15
Explanation: A ship capacity of 15 is the minimum to ship all the packages in 5
days like this:
1st day: 1, 2, 3, 4, 5
2nd day: 6, 7
3rd day: 8
4th day: 9
5th day: 10
```

Example 2:

```
Input: weights = [3,2,2,4,1,4], days = 3
Output: 6
Explanation: A ship capacity of 6 is the minimum to ship all the packages in 3
days like this:
1st day: 3, 2
2nd day: 2, 4
3rd day: 1, 4
```

Example 3:

```
Input: weights = [1,2,3,1,1], days = 4
Output: 3
Explanation:
1st day: 1
2nd day: 2
3rd day: 3
4th day: 1, 1
```

Constraints:

- $1 \leq \text{days} \leq \text{weights.length} \leq 5 * 10^4$
- $1 \leq \text{weights}[i] \leq 500$

Community Solutions (Python)

- WalkCC

```
class Solution:
    def shipWithinDays(self, weights: list[int], days: int) -> int:
        def shipDays(shipCapacity: int) -> int:
            shipDays = 1
            capacity = 0
            for weight in weights:
                if capacity + weight > shipCapacity:
                    shipDays += 1
                    capacity = weight
                else:
                    capacity += weight
            return shipDays

        l = max(weights)
        r = sum(weights)
        return bisect.bisect_left(range(l, r), True,
                                   key=lambda m: shipDays(m) <= days) + 1
```

• LeetDoocs

```
class Solution:
    def shipWithinDays(self, weights: List[int], days: int) -> int:
        def check(mx):
            ws, cnt = 0, 1
            for w in weights:
                ws += w
                if ws > mx:
                    cnt += 1
                    ws = w
            return cnt <= days

        left, right = max(weights), sum(weights) + 1
        return left + bisect_left(range(left, right), True, key=check)
```

• SimplyLeet

Round 2 · High · Medium

p.663

```
def shipWithinDays(weights, days):
    # Helper function to determine if a given capacity can ship packages within
    # 'days' days.
    def canShip(capacity):
        current_load = 0 # Current total weight on the ship for the current
        day.
        required_days = 1 # Start with the first day.
        for weight in weights:
            # If adding the current weight exceeds capacity, increment the day
            counter.
            if current_load + weight > capacity:
                required_days += 1
                current_load = 0

            current_load += weight
        return required_days <= days

    # Establish lower and upper bounds for binary search.
    low, high = max(weights), sum(weights)
    while low < high:
        mid = (low + high) // 2
        # If mid capacity can ship all packages within the allowed days, try a
        lower capacity.
        if canShip(mid):
            high = mid
        else:
            low = mid + 1
    return low

# Example usage:
weights = [1,2,3,4,5,6,7,8,9,10]
days = 5
print(shipWithinDays(weights, days))
```

• LC.ca

Round 2 · High · Medium

p.664


```
class Solution:
    def shipWithinDays(self, weights: List[int], days: int) -> int:
        def check(mx):
            ws, cnt = 0, 1
            for w in weights:
                ws += w
                if ws > mx:
                    cnt += 1
                    ws = w
            return cnt <= days

        left, right = max(weights), sum(weights) + 1
        return left + bisect_left(range(left, right), True, key=check)
```

◆ Quick Review

Key Insights

- The minimal possible capacity is the maximum value in weights; no ship can carry a package heavier than its capacity.
- The maximum possible capacity is the sum of all weights; this allows shipping all packages in one day.
- The problem is monotonic: if a capacity C works, any capacity greater than C will also work.
- Use binary search on the capacity range and a greedy approach to simulate shipping for each candidate capacity.

Space & Time

Time Complexity: $O(n * \log(\text{sum}(\text{weights})))$
 where n is the number of packages.

Space Complexity: $O(1)$ additional space.

Solution

The solution employs binary search to find the minimum ship capacity. Start by setting the search bounds: the lower bound is the maximum weight (since each package must be shipped individually if needed) and the upper bound is the sum of all weights (shipping everything in one go). For each midpoint capacity in the binary search, use a greedy method to simulate the shipping process—accumulating weights until adding another package would exceed the candidate ship capacity, at which point you increment the day count. If the number of days required exceeds D , the capacity is insufficient and the lower bound is increased; otherwise, adjust the upper bound. Continue until the bounds converge.

Binary Search

#1482 Minimum Number of Days to Make m Bouquets

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Binary Search
 Lists: AlgoMaster 600

Problem

You are given an integer array `bloomDay`, an integer `m` and an integer `k`.

You want to make `m` bouquets. To make a bouquet, you need to use `k` adjacent flowers from the garden.

The garden consists of `n` flowers, the `i`th flower will bloom in the `bloomDay[i]` and then can be used in exactly one bouquet.

Return the minimum number of days you need to wait to be able to make `m` bouquets from the garden. If it is impossible to make `m` bouquets return `-1`.

Example 1:

Round 2 · High · Medium

p.667

Input: `bloomDay = [1,10,3,10,2]`, `m = 3`, `k = 1`

Output: 3

Explanation: Let us see what happened in the first three days. `x` means flower bloomed and `_` means flower did not bloom in the garden.

We need 3 bouquets each should contain 1 flower.

After day 1: `[x, _, _, _, _]` // we can only make one bouquet.

After day 2: `[x, _, _, _, x]` // we can only make two bouquets.

After day 3: `[x, _, x, _, x]` // we can make 3 bouquets. The answer is 3.

Example 2:

Input: `bloomDay = [1,10,3,10,2]`, `m = 3`, `k = 2`

Output: -1

Explanation: We need 3 bouquets each has 2 flowers, that means we need 6 flowers. We only have 5 flowers so it is impossible to get the needed bouquets and we return -1.

Example 3:

Input: `bloomDay = [7,7,7,7,12,7,7]`, `m = 2`, `k = 3`

Output: 12

Explanation: We need 2 bouquets each should have 3 flowers.

Here is the garden after the 7 and 12 days:

After day 7: `[x, x, x, x, _, x, x]`

We can make one bouquet of the first three flowers that bloomed. We cannot make another bouquet from the last three flowers that bloomed because they are not adjacent.

After day 12: `[x, x, x, x, x, x, x]`

It is obvious that we can make two bouquets in different ways.

Constraints:

- `bloomDay.length == n`
- `1 <= n <= 105`
- `1 <= bloomDay[i] <= 109`
- `1 <= m <= 106`
- `1 <= k <= n`

Community Solutions (Python)

- WalkCC

Round 2 · High · Medium

p.668

```

class Solution:
    def minDays(self, bloomDay: list[int], m: int, k: int) -> int:
        if len(bloomDay) < m * k:
            return -1

        def getBouquetCount(waitingDays: int) -> int:
            """
            Returns the number of bouquets (k flowers needed) can be made after the
            `waitingDays`.
            """

            bouquetCount = 0
            requiredFlowers = k
            for day in bloomDay:
                if day > waitingDays:
                    # Reset `requiredFlowers` since there was not enough adjacent
                    # flowers.
                    requiredFlowers = k
                else:
                    requiredFlowers -= 1
                    if requiredFlowers == 0:
                        # Use k adjacent flowers to make a bouquet.

                        bouquetCount += 1
                        requiredFlowers = k
            return bouquetCount

            l = min(bloomDay)
            r = max(bloomDay)

            while l < r:
                mid = (l + r) // 2
                if getBouquetCount(mid) >= m:

                    r = mid
                else:
                    l = mid + 1

            return l

```

• LeetCode

Round 2 · High · Medium

p.669

```

class Solution:
    def minDays(self, bloomDay: List[int], m: int, k: int) -> int:
        def check(days: int) -> int:
            cnt = cur = 0
            for x in bloomDay:
                cur = cur + 1 if x <= days else 0
                if cur == k:
                    cnt += 1
                    cur = 0
            return cnt >= m

            mx = max(bloomDay)
            l = bisect_left(range(mx + 2), True, key=check)
            return -1 if l > mx else l

```

• SimplyLeet

```

# Python solution for the problem

def minDays(bloomDay, m, k):
    # If total flowers are less than required, return -1 early
    if m * k > len(bloomDay):
        return -1

    # Helper function to check if we can make required bouquets by day "day"
    def canMakeBouquets(day):
        bouquets = 0 # Number of bouquets made so far

        consecutive = 0 # Count of consecutive blooming flowers
        for bloom in bloomDay:
            # If the flower blooms by the given day, increase consecutive count
            if bloom <= day:
                consecutive += 1
                # If we reached k consecutive flowers, one bouquet is formed
                if consecutive == k:
                    bouquets += 1
                    consecutive = 0 # Reset for the next bouquet
            else:

```

Round 2 · High · Medium

p.670

```

        consecutive = 0 # Reset count if flower not bloomed
        # Early return if required bouquets have been made
        if bouquets >= m:
            return True
        return False

    low, high = min(bloomDay), max(bloomDay)
    result = -1

    # Binary search on the day range

    while low <= high:
        mid = (low + high) // 2 # middle day value
        if canMakeBouquets(mid):
            result = mid # candidate answer
            high = mid - 1 # try to find a smaller day
        else:
            low = mid + 1 # need more days
    return result

# Example Test Cases

print(minDays([1,10,3,10,2], 3, 1)) # Expected output: 3
print(minDays([1,10,3,10,2], 3, 2)) # Expected output: -1
print(minDays([7,7,7,7,12,7,7], 2, 3)) # Expected output: 12

```

Round 2 · High · Medium

p.671

Binary Search

#1552 Magnetic Force Between Two Balls

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Binary Search · Sorting
 Lists: AlgoMaster 600

Problem

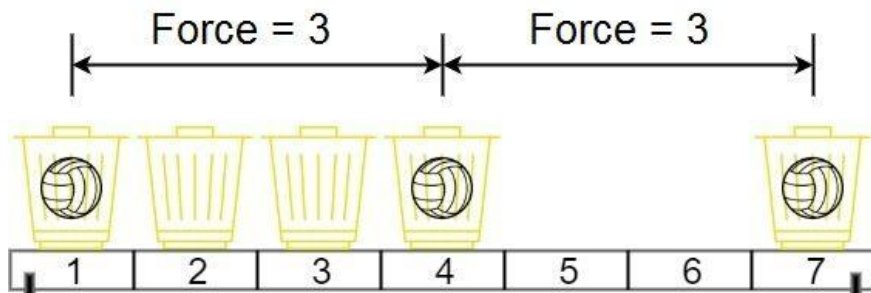
In the universe Earth C-137, Rick discovered a special form of magnetic force between two balls if they are put in his new invented basket. Rick has n empty baskets, the i th basket is at position $[i]$, Morty has m balls and needs to distribute the balls into the baskets such that the minimum magnetic force between any two balls is maximum.

Rick stated that magnetic force between two different balls at positions x and y is $|x - y|$. Given the integer array position and the integer m . Return the required force.

Example 1:

Round 2 · High · Medium

p.672



Input: position = [1,2,3,4,7], m = 3

Output: 3

Explanation: Distributing the 3 balls into baskets 1, 4 and 7 will make the magnetic force between ball pairs [3, 3, 6]. The minimum magnetic force is 3. We cannot achieve a larger minimum magnetic force than 3.

Example 2:

Input: position = [5,4,3,2,1,1000000000], m = 2

Output: 999999999

Explanation: We can use baskets 1 and 1000000000.

Constraints:

- $n == \text{position.length}$
- $2 \leq n \leq 105$
- $1 \leq \text{position}[i] \leq 10^9$
- All integers in position are distinct.
- $2 \leq m \leq \text{position.length}$

Community Solutions (Python)

- WalkCC

Round 2 · High · Medium

p.673

```
class Solution:
    def maxDistance(self, position: list[int], m: int) -> int:
        position.sort()

        l = 1
        r = position[-1] - position[0]

        def numBalls(force: int) -> int:
            balls = 0
            prevPosition = -force

            for pos in position:
                if pos - prevPosition >= force:
                    balls += 1
                    prevPosition = pos
            return balls

        while l < r:
            mid = r - (r - l) // 2
            if numBalls(mid) >= m:
                l = mid
            else:
                r = mid - 1

        return l
```

• LeetCode

```
class Solution:
    def maxDistance(self, position: List[int], m: int) -> int:
        def check(f: int) -> bool:
            prev = -inf
            cnt = 0
            for curr in position:
                if curr - prev >= f:
                    prev = curr
                    cnt += 1
            return cnt <= m
```

Round 2 · High · Medium

p.674

```
position.sort()
l, r = 1, position[-1]
return bisect_left(range(l, r + 1), True, key=check)
```

• SimplyLeet

```
# Python solution for Magnetic Force Between Two Balls
def maxDistance(position, m):
    # Sort the basket positions
    position.sort()

    # Function to check if we can place m balls with at least min_force between
    # them
    def canPlace(min_force):
        count = 1 # First ball placed in the first basket
        last_position = position[0]
        # Try to place remaining m-1 balls

        for pos in position[1:]:
            if pos - last_position >= min_force:
                count += 1
                last_position = pos
            if count == m: # All m balls placed successfully
                return True
        return False

    # Binary search boundaries
    low, high = 1, position[-1] - position[0]

    best_force = 0

    while low <= high:
        mid = (low + high) // 2 # Candidate minimum force
        if canPlace(mid):
            best_force = mid # mid is a possible answer, try to find a larger
            distance
            low = mid + 1
        else:
            high = mid - 1
    return best_force
```

Round 2 · High · Medium

p.675

```
# Example usage:
print(maxDistance([1,2,3,4,7], 3)) # Output should be 3
```

Round 2 · High · Medium

p.676

Stacks

Round 2 · High · Medium · 12 questions

Round 2 · High · Medium

p.677

Stacks

□ #71 Simplify Path

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Stack

Lists: AlgoMaster 600

Problem

You are given an absolute path for a Unix-style file system, which always begins with a slash '/'. Your task is to transform this absolute path into its simplified canonical path.

The rules of a Unix-style file system are as follows:

- A single period '.' represents the current directory.
- A double period '..' represents the previous/parent directory.
- Multiple consecutive slashes such as '/' and '/' are treated as a single slash '/'.
- Any sequence of periods that does not match the rules above should be treated as a valid directory or file name. For example, '...'

Round 2 · High · Medium

p.678

and '...' are valid directory or file names.

The simplified canonical path should follow these rules:

- The path must start with a single slash '/'.
- Directories within the path must be separated by exactly one slash '/'.
- The path must not end with a slash '/', unless it is the root directory.
- The path must not have any single or double periods ('.' and '..') used to denote current or parent directories.

Return the simplified canonical path.

Example 1:

Input: path = "/home/"

Output: "/home"

Explanation:

The trailing slash should be removed.

Example 2:

Input: path = "/home//foo/"

Output: "/home/foo"

Explanation:

Multiple consecutive slashes are replaced by a single one.

Example 3:

Input: path =

"/home/user/Documents/../Pictures"

Output: "/home/user/Pictures"

Explanation:

A double period ".." refers to the directory up a level (the parent directory).

Example 4:

Input: path = "../"

Output: "/"

Explanation:

Going one level up from the root directory is not possible.

Example 5:

Input: path = "/.../a/../b/c/../d/./"

Output: "/.../b/d"

Explanation:

"..." is a valid name for a directory in this problem.

Constraints:

- $1 \leq \text{path.length} \leq 3000$
- path consists of English letters, digits, period '.', slash '/' or '_'.

- path is a valid absolute Unix path.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def simplifyPath(self, path: str) -> str:
        stack = []

        for str in path.split('/'):
            if str in ('', '.'):
                continue
            if str == '..':
                if stack:
                    stack.pop()
            else:
                stack.append(str)

        return '/' + '/'.join(stack)
```

• LeetCode

```
class Solution:
    def simplifyPath(self, path: str) -> str:
        stk = []
        for s in path.split('/'):
            if not s or s == '.':
                continue
            if s == '..':
                if stk:
                    stk.pop()
            else:
                stk.append(s)

        return '/' + '/'.join(stk)
```

• SimplyLeet

Round 2 · High · Medium

p.681

```
def simplifyPath(path):
    # Split the path by '/'
    parts = path.split('/')
    # Initialize a stack to store valid directory names
    stack = []
    # Process each part from the split operation
    for part in parts:
        # Skip empty parts or the current directory symbol
        if not part or part == '.':
            continue

        # If encountering a parent directory symbol, pop if possible
        if part == '..':
            if stack:
                stack.pop()
        else:
            # Otherwise, push the valid directory name onto the stack
            stack.append(part)

    # Construct the canonical path by joining the stack with '/'
    return '/' + '/'.join(stack)

# Example usage
print(simplifyPath("/home//foo/"))
```

Round 2 · High · Medium

p.682

Stacks

□ #150 Evaluate Reverse Polish Notation

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Math · Stack
 Lists: Grind 169

Problem

You are given an array of strings `tokens` that represents an arithmetic expression in a Reverse Polish Notation.

Evaluate the expression. Return an integer that represents the value of the expression.

Note that:

- The valid operators are '+', '-', '*', and '/'.
- Each operand may be an integer or another expression.
- The division between two integers always truncates toward zero.
- There will not be any division by zero.
- The input represents a valid arithmetic expression in a reverse polish notation.

Round 2 · High · Medium

p.683

- The answer and all the intermediate calculations can be represented in a 32-bit integer.

Example 1:

```
Input: tokens = ["2","1","+","3","*"]
Output: 9
Explanation: ((2 + 1) * 3) = 9
```

Example 2:

```
Input: tokens = ["4","13","5","/","+"]
Output: 6
Explanation: (4 + (13 / 5)) = 6
```

Example 3:

```
Input: tokens = ["10","6","9","3","+","-11","*","/", "*", "17","+","5","+"]
Output: 22
Explanation: ((10 * (6 / ((9 + 3) * -11))) + 17) + 5
= ((10 * (6 / (12 * -11))) + 17) + 5
= ((10 * (6 / -132)) + 17) + 5
= ((10 * 0) + 17) + 5
= (0 + 17) + 5
= 17 + 5
```

Constraints:

- $1 \leq \text{tokens.length} \leq 104$
- `tokens[i]` is either an operator: "+", "-", "*", or "/", or an integer in the range $[-200, 200]$.

Community Solutions (Python)

- WalkCC

Round 2 · High · Medium

p.684

```
class Solution:
    def evalRPN(self, tokens: list[str]) -> int:
        stack = []
        op = {
            '+': lambda a, b: a + b,
            '-': lambda a, b: a - b,
            '*': lambda a, b: a * b,
            '/': lambda a, b: int(a / b),
        }

        for token in tokens:
            if token in op:
                b = stack.pop()
                a = stack.pop()
                stack.append(op[token](a, b))
            else:
                stack.append(int(token))

        return stack.pop()
```

• LeetCode

```
import operator

class Solution:
    def evalRPN(self, tokens: List[str]) -> int:
        opt = {
            "+": operator.add,
            "-": operator.sub,
            "*": operator.mul,
            "/": operator.truediv,
        }
        s = []
        for token in tokens:
            if token in opt:
                s.append(int(opt[token](s.pop(-2), s.pop(-1))))
            else:
                s.append(int(token))
        return s[0]
```

Round 2 · High · Medium

p.685

```
class Solution:
    def evalRPN(self, tokens: List[str]) -> int:
        nums = []
        for t in tokens:
            if len(t) > 1 or t.isdigit():
                nums.append(int(t))
            else:
                if t == "+":
                    nums[-2] += nums[-1]
                elif t == "-":
                    nums[-2] -= nums[-1]
                elif t == "*":
                    nums[-2] *= nums[-1]
                else:
                    nums[-2] = int(nums[-2] / nums[-1])
                nums.pop()
        return nums[0]
```

• SimplyLeet

```
def evalRPN(tokens):
    # Initialize a stack to hold numbers
    stack = []
    # Loop through each token in the input list
    for token in tokens:
        if token in {"+", "-", "*", "/"}:
            # Pop the two topmost values; note the order is important
            b = stack.pop()
            a = stack.pop()
            # Apply the operation based on the token

            if token == "+":
                result = a + b
            elif token == "-":
                result = a - b
            elif token == "*":
                result = a * b
            else:
                # Use int division to truncate towards zero
                result = int(a / b)
            # Push the result back onto the stack
```

Round 2 · High · Medium

p.686

```

        stack.append(result)
    else:
        # Token is a number; push its integer value onto the stack
        stack.append(int(token))
    # The final result is the only element left in the stack
    return stack.pop()

# Example usage:
# tokens = ["2", "1", "+", "3", "*"]
# print(evalRPN(tokens)) # Output: 9

```

• LC.ca

```

class Solution:
    def evalRPN(self, tokens: List[str]) -> int:
        nums = []
        for t in tokens:
            if len(t) > 1 or t.isdigit():
                nums.append(int(t))
            else:
                if t == "+":
                    nums[-2] += nums[-1]
                elif t == "-":
                    nums[-2] -= nums[-1]
                elif t == "*":
                    nums[-2] *= nums[-1]
                else:
                    nums[-2] = int(nums[-2] / nums[-1])
                nums.pop()
        return nums[0]

```

◆ Quick Review

Key Insights

- Use a stack to process the tokens.
- Push numbers onto the stack.

- When an operator is encountered, pop the top two numbers, perform the operation, and push the result back.
- Handle division carefully to ensure it truncates toward zero.
- The final result remains as the only value in the stack.

Space & Time

Time Complexity: $O(n)$, where n is the number of tokens (each token is processed once).

Space Complexity: $O(n)$, in the worst-case scenario where all tokens are numbers and are stored in the stack.

Solution

We use a stack data structure to evaluate the RPN expression. Iterate through each token:

- If the token is a number, convert and push it onto the stack.
- If the token is an operator, pop the top two numbers (be careful about order), perform the corresponding arithmetic operation, and push the result back onto the stack.
- At the end of processing, the stack contains the result of the expression. Special attention is needed for division to ensure it

truncates toward zero (using language-specific functions when necessary).

Round 2 · High · Medium

p.689

Stacks

□ #155 Min Stack

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Stack · Design
Lists: Grind 169

Problem

Design a stack that supports push, pop, top, and retrieving the minimum element in constant time.

Implement the MinStack class:

- MinStack() initializes the stack object.
- void push(int val) pushes the element val onto the stack.
- void pop() removes the element on the top of the stack.
- int top() gets the top element of the stack.
- int getMin() retrieves the minimum element in the stack.

You must implement a solution with $O(1)$ time complexity for each function.

Example 1:

Round 2 · High · Medium

p.690

Input
 ["MinStack","push","push","push","getMin","pop","top","getMin"]
 [[],[{-2},{0},{-3}],[],[],[],[]]

Output
 [null,null,null,null,-3,null,0,-2]

Explanation

Constraints:

- $-231 \leq \text{val} \leq 231 - 1$
- Methods pop, top and getMin operations will always be called on non-empty stacks.
- At most $3 * 10^4$ calls will be made to push, pop, top, and getMin.

Community Solutions (Python)

• WalkCC

```
class MinStack:
    def __init__(self):
        self.stack = []

    def push(self, x: int) -> None:
        mn = x if not self.stack else min(self.stack[-1][1], x)
        self.stack.append([x, mn])

    def pop(self) -> None:
        self.stack.pop()

    def top(self) -> int:
        return self.stack[-1][0]

    def getMin(self) -> int:
        return self.stack[-1][1]
```

Round 2 · High · Medium

p.691

• LeetCode

```
class MinStack:
    def __init__(self):
        self.stk1 = []
        self.stk2 = [inf]

    def push(self, val: int) -> None:
        self.stk1.append(val)
        self.stk2.append(min(val, self.stk2[-1]))

    def pop(self) -> None:
        self.stk1.pop()
        self.stk2.pop()

    def top(self) -> int:
        return self.stk1[-1]

    def getMin(self) -> int:
        return self.stk2[-1]
```

```
# Your MinStack object will be instantiated and called as such:
# obj = MinStack()
# obj.push(val)
# obj.pop()
# param_3 = obj.top()
# param_4 = obj.getMin()
```

• SimplyLeet

```
# Python implementation of MinStack with detailed comments

class MinStack:
    def __init__(self):
        # Main stack to store all values
        self.stack = []
        # Auxiliary stack to store minimum values
        self.min_stack = []

    def push(self, val: int) -> None:
```

Round 2 · High · Medium

p.692

```

        # Push value onto main stack
        self.stack.append(val)
        # If min_stack is empty or the new value is less than or equal to
        current minimum, push it onto min_stack
        if not self.min_stack or val <= self.min_stack[-1]:
            self.min_stack.append(val)

    def pop(self) -> None:
        # Pop the top value from main stack
        popped_value = self.stack.pop()
        # If the popped value is the current minimum, pop it from min_stack as
        well

        if popped_value == self.min_stack[-1]:
            self.min_stack.pop()

    def top(self) -> int:
        # Return the top value from main stack without removing it
        return self.stack[-1]

    def getMin(self) -> int:
        # Return the minimum value from min_stack
        return self.min_stack[-1]

# Example usage:
# minStack = MinStack()
# minStack.push(-2)
# minStack.push(0)
# minStack.push(-3)
# print(minStack.getMin()) # -> -3
# minStack.pop()
# print(minStack.top()) # -> 0
# print(minStack.getMin()) # -> -2

```

☐ • LC.ca

Round 2 · High · Medium

p.693

```

class MinStack:
    def __init__(self):
        self.sk = []
        self.minsk = [inf] # trick, avoid later empty-check for min-stack

    def push(self, x: int) -> None:
        self.sk.append(x)
        self.minsk.append(min(x, self.minsk[-1]))

    def pop(self) -> None:

if not self.sk:
    return
    self.sk.pop()
    self.minsk.pop()

    def top(self) -> int:
        return self.sk[-1]

    def getMin(self) -> int:
        return self.minsk[-1]

# Your MinStack object will be instantiated and called as such:
# obj = MinStack()
# obj.push(x)
# obj.pop()
# param_3 = obj.top()
# param_4 = obj.getMin()

#####

class MinStack:

    def __init__(self):
        self.sk = []
        self.minsk = []

    def push(self, val: int) -> None:
        self.sk.append(val)

```

Round 2 · High · Medium

p.694

```

        # empty check
self.minsk.append(val if not self.minsk else min(val, self.minsk[-1]))

def pop(self) -> None:
    if not self.sk:
        return

    self.sk.pop()
    self.minsk.pop()

def top(self) -> int:
    return self.sk[-1]

def getMin(self) -> int:
    return self.minsk[-1]

# optimize above, using only 1 stack, with stack element as tuple: (val, its
associated min)
class MinStack:

    def __init__(self):

        self._stack = []

    def push(self, x: int) -> None:
        cur_min = self.getMin()
        if x < cur_min:
            cur_min = x
        self._stack.append((x, cur_min))

    def pop(self) -> None:
        self._stack.pop()

```

Round 2 · High · Medium

p.695

```

def top(self) -> int:
    if not self._stack:
        return None
    else:
        return self._stack[-1][0]

def getMin(self) -> int:
    if not self._stack:
        return float('inf')

    else:
        return self._stack[-1][1]

```

◆ Quick Review

Key Insights

- Use an auxiliary data structure to track the current minimum element.
- A common approach is to use two stacks: one regular stack for all elements and a second stack to keep track of the minimum values.
- When pushing a new element, push it onto the main stack and also update the min stack if the element is less than or equal to the current minimum.
- When popping, if the popped element is equal to the top of the min stack, pop from the min stack as well.

Round 2 · High · Medium

p.696

- This design guarantees that all operations (push, pop, top, getMin) run in constant time, $O(1)$.

Space & Time

Time Complexity: $O(1)$ per operation (push, pop, top, getMin)

Space Complexity: $O(n)$, where n is the number of elements in the stack, since an additional stack is used to keep track of minima.

Solution

The solution involves using two stacks:

- Main Stack: Stores all the elements.
- Min Stack: Stores the minimum elements encountered so far.

When an element is pushed, we compare it with the top of the min stack. If it is smaller than or equal to the current minimum, we also push it onto the min stack. For the pop operation, if the element being popped is the same as the top of the min stack, we pop from the min stack as well. This approach ensures that the getMin operation always returns the top element of the min stack, which is the current minimum in constant time.

Stacks

□ #227 Basic Calculator II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · String · Stack

Lists: Grind 169

Problem

Given a string s which represents an expression, evaluate this expression and return its value.

The integer division should truncate toward zero.

You may assume that the given expression is always valid. All intermediate results will be in the range of $[-2^{31}, 2^{31} - 1]$.

Note: You are not allowed to use any built-in function which evaluates strings as mathematical expressions, such as `eval()`.

Example 1:

Input: $s = "3+2*2"$
Output: 7

Example 2:

Input: $s = " 3/2 "$
Output: 1

Example 3:

Input: s = " 3+5 / 2 "
Output: 5

Constraints:

- $1 \leq s.length \leq 3 \times 10^5$
- s consists of integers and operators ('+', '-', '*', '/') separated by some number of spaces.
- s represents a valid expression.
- All the integers in the expression are non-negative integers in the range [0, 2³¹ - 1].
- The answer is guaranteed to fit in a 32-bit integer.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def calculate(self, s: str) -> int:
        ans = 0
        prevNum = 0
        currNum = 0
        op = '+'

        for i, c in enumerate(s):
            if c.isdigit():
                currNum = currNum * 10 + int(c)
```

```
if not c.isdigit() and c != ' ' or i == len(s) - 1:
    if op == '+' or op == '-':
        ans += prevNum
        prevNum = currNum if op == '+' else -currNum
    elif op == '*':
        prevNum = prevNum * currNum
    elif op == '/':
        if prevNum < 0:
            prevNum = math.ceil(prevNum / currNum)
        else:
            prevNum = prevNum // currNum
    op = c
    currNum = 0

return ans + prevNum
```

• LeetDoocs

```
class Solution:
    def calculate(self, s: str) -> int:
        v, n = 0, len(s)
        sign = '+'
        stk = []
        for i, c in enumerate(s):
            if c.isdigit():
                v = v * 10 + int(c)
            if i == n - 1 or c in '+-*/':
                match sign:
                    case '+':
                        stk.append(v)
                    case '-':
                        stk.append(-v)
                    case '*':
                        stk.append(stk.pop() * v)
                    case '/':
                        stk.append(int(stk.pop() / v))
                sign = c
                v = 0

        return sum(stk)
```

• SimplyLeet

Python solution for Basic Calculator II

```
def calculate(s: str) -> int:
    # Initialize the stack, current number, and the last seen operator.
    stack = []
    current_number = 0
    previous_operator = '+'

    # Loop over each character in the string along with one extra iteration to
    # handle the end.
    for i, char in enumerate(s):

        if char.isdigit():
            # Build the current number by shifting previous digits.
            current_number = current_number * 10 + int(char)

            # If the character is an operator (or we're at the end), process the
            # previous operator.
            if char in "+-*/" or i == len(s) - 1:
                if previous_operator == '+':
                    stack.append(current_number)
                elif previous_operator == '-':
                    stack.append(-current_number)
                elif previous_operator == '*':

                    last_number = stack.pop()
                    stack.append(last_number * current_number)
                elif previous_operator == '/':
                    last_number = stack.pop()
                    # Python division truncates toward negative infinity by
                    # default, so we use int() to truncate toward zero.
                    stack.append(int(last_number / current_number))
                # Reset for the next number and update the operator.
                previous_operator = char
                current_number = 0

    # Sum up all numbers in the stack which gives the final result.
```

Round 2 · High · Medium

p.701

```
return sum(stack)

# Example usage:
print(calculate("3+2*2")) # Output: 7
print(calculate(" 3/2 ")) # Output: 1
print(calculate(" 3+5 / 2 ")) # Output: 5
```

• LC.ca

```
class Solution:
    def calculate(self, s: str) -> int:
        v, n = 0, len(s)
        sign = '+'
        stk = []
        for i, c in enumerate(s):
            if c.isdigit():
                v = v * 10 + int(c)
            if i == n - 1 or c in '+-*/':
                if sign == '+':
                    stk.append(v)
                # for "10-2*5": when '-' encountered, var 'sign' is still '+'
                # so '10' will be pushed to stk before setting sign to '-'
                elif sign == '-':
                    stk.append(-v)
                elif sign == '*':
                    stk.append(stk.pop() * v)
                elif sign == '/':
                    stk.append(int(stk.pop() / v))
                else:
                    print("operator not supported")

                sign = c # reset inside 'if'
                v = 0
        return sum(stk)
```

◆ Quick Review

Key Insights

Round 2 · High · Medium

p.702

- The expression can be processed in one pass while using a stack to handle operator precedence.
- When encountering '+' or '-' operators, the current number is directly added or subtracted (as a positive or negative value).
- For '*' and '/' operators, compute immediately with the last number from the stack and update it.
- Handle spaces and multi-digit numbers by building the number while iterating over the string.
- Division truncation toward zero must be carefully applied, which aligns with typical integer division in most languages.

Space & Time

Time Complexity: $O(n)$ where n is the length of the expression, since the algorithm processes each character once.

Space Complexity: $O(n)$ in the worst-case scenario when all numbers are stored in the stack.

Solution

Round 2 · High · Medium

p.703

We use a stack-based approach. Iterate over each character of the string while maintaining a variable for the current number and a previous operator (initialized to '+'). When an operator or the end of the string is reached, perform an action based on the previous operator:

- For '+' add the current number to the stack.
 - For '-' add the negative of the current number.
 - For '*' and '/' pop the last number from the stack, compute the result by performing multiplication or division, and then push the result back to the stack.
- After processing the entire string, sum the elements in the stack to get the final result. This approach ensures that multiplication and division are handled before addition and subtraction.

Round 2 · High · Medium

p.704

Stacks

#316 Remove Duplicate Letters

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Stack · Greedy · Monotonic Stack
Lists: AlgoMaster 600

Problem

Given a string s , remove duplicate letters so that every letter appears once and only once. You must make sure your result is the smallest in lexicographical order among all possible results.

Example 1:

Input: $s = "bcabc"$
Output: $"abc"$

Example 2:

Input: $s = "cbacdcbc"$
Output: $"acdb"$

Constraints:

- $1 \leq s.length \leq 104$
- s consists of lowercase English letters.

Note: This question is the same as 1081: <https://leetcode.com/problems/smallest-subsequence-of-distinct-characters/>

Round 2 · High · Medium

p.705

Community Solutions (Python)

• WalkCC

```
class Solution:
    def removeDuplicateLetters(self, s: str) -> str:
        ans = []
        count = collections.Counter(s)
        used = [False] * 26

        for c in s:
            count[c] -= 1
            if used[ord(c) - ord('a')]:
                continue

            while ans and ans[-1] > c and count[ans[-1]] > 0:
                used[ord(ans[-1]) - ord('a')] = False
                ans.pop()
            ans.append(c)
            used[ord(ans[-1]) - ord('a')] = True

        return ''.join(ans)
```

• LeetDoocs

```
class Solution:
    def removeDuplicateLetters(self, s: str) -> str:
        last = {c: i for i, c in enumerate(s)}
        stk = []
        vis = set()
        for i, c in enumerate(s):
            if c in vis:
                continue
            while stk and stk[-1] > c and last[stk[-1]] > i:
                vis.remove(stk.pop())

            stk.append(c)
            vis.add(c)
        return ''.join(stk)
```

Round 2 · High · Medium

p.706

```
class Solution:
    def removeDuplicateLetters(self, s: str) -> str:
        count, in_stack = [0] * 128, [False] * 128
        stack = []
        for c in s:
            count[ord(c)] += 1
        for c in s:
            count[ord(c)] -= 1
            if in_stack[ord(c)]:
                continue

            while len(stack) and stack[-1] > c:
                peek = stack[-1]
                if count[ord(peek)] < 1:
                    break
                in_stack[ord(peek)] = False
                stack.pop()
            stack.append(c)
            in_stack[ord(c)] = True
        return ''.join(stack)
```

• SimplyLeet

```
class Solution:
    def removeDuplicateLetters(self, s: str) -> str:
        # Map each character to its last occurrence index
        last_occurrence = {c: i for i, c in enumerate(s)}
        stack = [] # Monotonic stack to build the result
        seen = set() # Set to record characters already in the stack

        # Iterate over each character and its index
        for i, char in enumerate(s):
            # Skip if character is already in the stack
```

```
        if char in seen:
            continue
        # Remove characters that are larger than current character and
        appear later
        while stack and char < stack[-1] and last_occurrence[stack[-1]] >
i:
            removed_char = stack.pop()
            seen.remove(removed_char)
        stack.append(char)
        seen.add(char)
        # Join the stack to form the final result
        return ''.join(stack)
```

Stacks

#636 Exclusive Time of Functions

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Stack

Lists: AlgoMaster 600

Problem

On a single-threaded CPU, we execute a program containing n functions. Each function has a unique ID between 0 and $n-1$.

Function calls are stored in a call stack: when a function call starts, its ID is pushed onto the stack, and when a function call ends, its ID is popped off the stack. The function whose ID is at the top of the stack is the current function being executed. Each time a function starts or ends, we write a log with the ID, whether it started or ended, and the timestamp.

You are given a list `logs`, where `logs[i]` represents the i th log message formatted as a string `"{function_id}:{start | end}:{timestamp}"`. For example, `"0:start:3"` means a function call with function ID 0 started

Round 2 · High · Medium

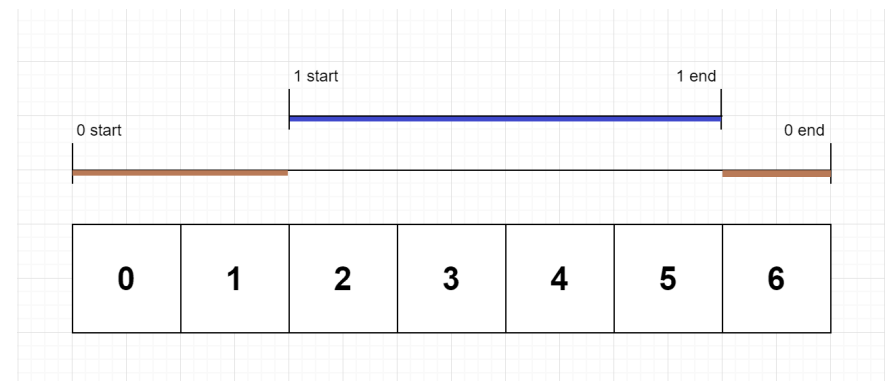
p.709

at the beginning of timestamp 3, and `"1:end:2"` means a function call with function ID 1 ended at the end of timestamp 2. Note that a function can be called multiple times, possibly recursively.

A function's exclusive time is the sum of execution times for all function calls in the program. For example, if a function is called twice, one call executing for 2 time units and another call executing for 1 time unit, the exclusive time is $2 + 1 = 3$.

Return the exclusive time of each function in an array, where the value at the i th index represents the exclusive time for the function with ID i .

Example 1:



Round 2 · High · Medium

p.710

Input: $n = 2$, $\text{logs} = ["0:\text{start}:0", "1:\text{start}:2", "1:\text{end}:5", "0:\text{end}:6"]$
 Output: $[3, 4]$
 Explanation:
 Function 0 starts at the beginning of time 0, then it executes 2 for units of time and reaches the end of time 1.
 Function 1 starts at the beginning of time 2, executes for 4 units of time, and ends at the end of time 5.
 Function 0 resumes execution at the beginning of time 6 and executes for 1 unit of time.
 So function 0 spends $2 + 1 = 3$ units of total time executing, and function 1 spends 4 units of total time executing.

Example 2:

Input: $n = 1$, $\text{logs} = ["0:\text{start}:0", "0:\text{start}:2", "0:\text{end}:5", "0:\text{start}:6", "0:\text{end}:6", "0:\text{end}:7"]$
 Output: $[8]$
 Explanation:
 Function 0 starts at the beginning of time 0, executes for 2 units of time, and recursively calls itself.
 Function 0 (recursive call) starts at the beginning of time 2 and executes for 4 units of time.
 Function 0 (initial call) resumes execution then immediately calls itself again.
 Function 0 (2nd recursive call) starts at the beginning of time 6 and executes for 1 unit of time.
 Function 0 (initial call) resumes execution at the beginning of time 7 and executes for 1 unit of time.

Example 3:

Input: $n = 2$, $\text{logs} = ["0:\text{start}:0", "0:\text{start}:2", "0:\text{end}:5", "1:\text{start}:6", "1:\text{end}:6", "0:\text{end}:7"]$
 Output: $[7, 1]$
 Explanation:
 Function 0 starts at the beginning of time 0, executes for 2 units of time, and recursively calls itself.
 Function 0 (recursive call) starts at the beginning of time 2 and executes for 4 units of time.
 Function 0 (initial call) resumes execution then immediately calls function 1.
 Function 1 starts at the beginning of time 6, executes 1 unit of time, and ends at the end of time 6.
 Function 0 resumes execution at the beginning of time 6 and executes for 2 units of time.

Constraints:

- $1 \leq n \leq 100$
- $2 \leq \text{logs.length} \leq 500$
- $0 \leq \text{function_id} < n$
- $0 \leq \text{timestamp} \leq 10^9$
- No two start events will happen at the same timestamp.
- No two end events will happen at the same timestamp.
- Each function has an "end" log for each "start" log.

Community Solutions (Python)

• LeetCode

```
class Solution:
    def exclusiveTime(self, n: int, logs: List[str]) -> List[int]:
        stk = []
        ans = [0] * n
        pre = 0
        for log in logs:
            i, op, t = log.split(":")
            i, cur = int(i), int(t)
            if op[0] == "s":
                if stk:
                    ans[stk[-1]] += cur - pre
                    stk.append(i)
                    pre = cur
            else:
                ans[stk.pop()] += cur - pre + 1
                pre = cur + 1
        return ans
```


• SimplyLeet

```
# Python solution for Exclusive Time of Functions

def exclusiveTime(n, logs):
    # Initialize result array with zeros for each function
    result = [0] * n
    # Stack to keep track of the function ids
    stack = []
    # Previous timestamp to calculate elapsed time
    prev_time = 0

    # Process each log entry
    for log in logs:
        # Split the log string into parts
        fn_id, typ, time_str = log.split(":")
        curr_time = int(time_str)
        fn_id = int(fn_id)

        if typ == "start":
            # If there is a function currently running, update its exclusive
            time
            if stack:
                result[stack[-1]] += curr_time - prev_time
            # Push the new function id onto the stack
            stack.append(fn_id)
            # Update previous time to current timestamp
            prev_time = curr_time
        else: # 'end' log
            # Pop the current function from the stack
            completed_fn = stack.pop()
            # Add the elapsed time including the current time unit
            result[completed_fn] += curr_time - prev_time + 1
```

Round 2 · High · Medium

p.713

```
        # Update previous time to one unit after the current timestamp
        prev_time = curr_time + 1

    return result

# Example usage:
n = 2
logs = ["0:start:0", "1:start:2", "1:end:5", "0:end:6"]
print(exclusiveTime(n, logs)) # Output: [3, 4]
```

Round 2 · High · Medium

p.714

Stacks

□ #678 Valid Parenthesis String

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming · Stack · Greedy
Lists: NeetCode 150

Problem

Given a string *s* containing only three types of characters: '(', ')' and '*', return true if *s* is valid.

The following rules define a valid string:

- Any left parenthesis '(' must have a corresponding right parenthesis ')'.
• Any right parenthesis ')' must have a corresponding left parenthesis '('.
- Left parenthesis '(' must go before the corresponding right parenthesis ')'.
• '*' could be treated as a single right parenthesis ')' or a single left parenthesis '(' or an empty string "".

Example 1:

Input: *s* = "()"
Output: true

Round 2 · High · Medium

p.715

Example 2:

Input: *s* = "(*)"
Output: true

Example 3:

Input: *s* = "(*)"
Output: true

Constraints:

- $1 \leq s.length \leq 100$
- *s*[*i*] is '(', ')' or '*'.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def checkValidString(self, s: str) -> bool:
        low = 0
        high = 0

        for c in s:
            if c == '(':
                low += 1
                high += 1
            elif c == ')':
                if low > 0:
                    low -= 1
                high -= 1
            else:
                if low > 0:
                    low -= 1
                high += 1
                if high < 0:
                    return False
        return low == 0
```

Round 2 · High · Medium

p.716

```
return low == 0
```

• LeetCode

```
class Solution:
    def checkValidString(self, s: str) -> bool:
        n = len(s)
        dp = [[False] * n for _ in range(n)]
        for i, c in enumerate(s):
            dp[i][i] = c == '*'
            for i in range(n - 2, -1, -1):
                for j in range(i + 1, n):
                    dp[i][j] = (
                        s[i] in '(*' and s[j] in '*)' and (i + 1 == j or dp[i + 1]
[j - 1])
```

```
                    )
                    dp[i][j] = dp[i][j] or any(
                        dp[i][k] and dp[k + 1][j] for k in range(i, j)
                    )
        return dp[0][-1]
```

```
class Solution:
    def checkValidString(self, s: str) -> bool:
        x = 0
        for c in s:
            if c in '(':
                x += 1
            elif x:
                x -= 1
            else:
                return False
```

```
x = 0
for c in s[::-1]:
    if c in ')':
        x += 1
    elif x:
        x -= 1
    else:
        return False
return True
```

Round 2 · High · Medium

p.717

• SimplyLeet

```
# Python implementation with detailed inline comments
```

```
def checkValidString(s):
    # Initialize the minimum and maximum possible count of open parentheses
    minOpen = 0
    maxOpen = 0

    # Iterate over each character in the string
    for char in s:
        if char == '(':
            # For an open parenthesis, both min and max increase by 1
            minOpen += 1
            maxOpen += 1
        elif char == ')':
            # For a close parenthesis, both min and max decrease by 1
            minOpen -= 1
            maxOpen -= 1
        else: # char == '*'
            # '*' can act as a ')' (decrease min) or '(' (increase max)
            minOpen -= 1
            maxOpen += 1

            # Ensure minOpen does not drop below 0
            if minOpen < 0:
                minOpen = 0

            # If maxOpen is negative, there are too many ')' to balance
            if maxOpen < 0:
                return False

    # The string is valid if there is a possible balance where minOpen equals 0
    return minOpen == 0

# Example usage:
print(checkValidString("(*)")) # Expected output: True
```

Round 2 · High · Medium

p.718

Stacks

☐ #735 Asteroid Collision

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Stack · Simulation
 Lists: Grind 169

Problem

We are given an array `asteroids` of integers representing asteroids in a row. The indices of the asteroid in the array represent their relative position in space.

For each asteroid, the absolute value represents its size, and the sign represents its direction (positive meaning right, negative meaning left). Each asteroid moves at the same speed.

Find out the state of the asteroids after all collisions. If two asteroids meet, the smaller one will explode. If both are the same size, both will explode. Two asteroids moving in the same direction will never meet.

Example 1:

Round 2 · High · Medium

p.719

Input: `asteroids = [5,10,-5]`
 Output: `[5,10]`
 Explanation: The 10 and -5 collide resulting in 10. The 5 and 10 never collide.

Example 2:

Input: `asteroids = [8,-8]`
 Output: `[]`
 Explanation: The 8 and -8 collide exploding each other.

Example 3:

Input: `asteroids = [10,2,-5]`
 Output: `[10]`
 Explanation: The 2 and -5 collide resulting in -5. The 10 and -5 collide resulting in 10.

Example 4:

Input: `asteroids = [3,5,-6,2,-1,4]`
 Output: `[-6,2,4]`
 Explanation: The asteroid -6 makes the asteroid 3 and 5 explode, and then continues going left. On the other side, the asteroid 2 makes the asteroid -1 explode and then continues going right, without reaching asteroid 4.

Constraints:

- $2 \leq \text{asteroids.length} \leq 104$
- $-1000 \leq \text{asteroids}[i] \leq 1000$
- $\text{asteroids}[i] \neq 0$

Community Solutions (Python)

- WalkCC

Round 2 · High · Medium

p.720

```
class Solution:
    def asteroidCollision(self, asteroids: list[int]) -> list[int]:
        stack = []

        for a in asteroids:
            if a > 0:
                stack.append(a)
            else: # a < 0
                # Destroy the previous positive one(s).
                while stack and stack[-1] > 0 and stack[-1] < -a:
                    stack.pop()

                if not stack or stack[-1] < 0:
                    stack.append(a)
                elif stack[-1] == -a:
                    stack.pop() # Both asteroids explode.
                else: # stack[-1] > the current asteroid.
                    pass # Destroy the current asteroid, so do nothing.

        return stack
```

• LeetCode

```
class Solution:
    def asteroidCollision(self, asteroids: List[int]) -> List[int]:
        stk = []
        for x in asteroids:
            if x > 0:
                stk.append(x)
            else:
                while stk and stk[-1] > 0 and stk[-1] < -x:
                    stk.pop()
                if stk and stk[-1] == -x:
                    stk.pop()

                elif not stk or stk[-1] < 0:
                    stk.append(x)

        return stk
```

• SimplyLeet

Round 2 · High · Medium

p.721

```
# Function to simulate asteroid collisions
def asteroidCollision(asteroids):
    # Initialize an empty list as a stack to hold surviving asteroids
    stack = []
    # Iterate through each asteroid in the input list
    for ast in asteroids:
        # Flag to check if the current asteroid has exploded
        exploded = False
        # Check for potential collision while there is an asteroid in stack and
        # a collision condition holds (right-moving on stack and left-moving current)
        while stack and ast < 0 and stack[-1] > 0:
            # If the incoming left-moving asteroid is larger than the
            # right-moving one on stack, pop the top and continue checking
            if abs(ast) > abs(stack[-1]):
                stack.pop()
                continue # Continue the loop to check next potential collision
            # If they are equal, both explode: pop the stack and mark
            # explosion, break the loop
            elif abs(ast) == abs(stack[-1]):
                stack.pop()
                exploded = True
                break
            # Mark the incoming asteroid as exploded and break out of the loop
            else:
                exploded = True
                break

        # If the asteroid did not explode during collisions, push it onto the
        # stack
        if not exploded:
            stack.append(ast)
    # Return the surviving asteroids
    return stack

# Example usage:
# print(asteroidCollision([5,10,-5])) # Output: [5,10]
```

• LC.ca

Round 2 · High · Medium

p.722

```
class Solution:
    def asteroidCollision(self, asteroids: List[int]) -> List[int]:
        stk = []
        for x in asteroids:
            if x > 0:
                stk.append(x)
            else:
                while stk and stk[-1] > 0 and stk[-1] < -x:
                    stk.pop()
                if stk and stk[-1] == -x:
                    stk.pop()
                elif not stk or stk[-1] < 0:
                    stk.append(x)
        return stk
```

◆ Quick Review

Key Insights

- Use a stack to keep track of the surviving asteroids.
- Only a collision happens when a right-moving asteroid (positive) is on the stack and a left-moving asteroid (negative) is encountered.
- Compare the absolute values to decide if one or both asteroids explode.
- Continue processing collisions until no further collisions are possible with the current asteroid.

Space & Time

Round 2 · High · Medium

p.723

Time Complexity: $O(n)$ where n is the number of asteroids, since each asteroid is pushed and popped at most once.

Space Complexity: $O(n)$ in the worst case, when no collisions occur and all asteroids remain on the stack.

Solution

The solution uses a stack to simulate the asteroid collision process. For each asteroid in the input:

- If it is moving to the right or the stack is empty, simply add it to the stack.
- If it is moving to the left, check the asteroid on top of the stack: If the top asteroid is moving to the right, they might collide. Compare sizes: If the left-moving asteroid is larger (in absolute terms), pop the stack and continue checking. If both are equal, pop the stack and do not add the current asteroid. If the right-moving asteroid is larger, the current asteroid explodes.
- If no collision occurs for the current asteroid, push it onto the stack. Finally, the stack represents the surviving asteroids order.

Round 2 · High · Medium

p.724

Stacks

#853 Car Fleet

MED

LeetCode · SimplyLeet · WalkCC · LeetCode
 Array · Stack · Sorting · Monotonic Stack
 Lists: NeetCode 150

Problem

There are n cars at given miles away from the starting mile 0, traveling to reach the mile target.

You are given two integer arrays `position` and `speed`, both of length n , where `position[i]` is the starting mile of the i th car and `speed[i]` is the speed of the i th car in miles per hour.

A car cannot pass another car, but it can catch up and then travel next to it at the speed of the slower car.

A car fleet is a single car or a group of cars driving next to each other. The speed of the car fleet is the minimum speed of any car in the fleet.

Round 2 · High · Medium

p.725

If a car catches up to a car fleet at the mile target, it will still be considered as part of the car fleet.

Return the number of car fleets that will arrive at the destination.

Example 1:

Input: `target = 12`, `position = [10,8,0,5,3]`, `speed = [2,4,1,1,3]`

Output: 3

Explanation:

- The cars starting at 10 (speed 2) and 8 (speed 4) become a fleet, meeting each other at 12. The fleet forms at target.
- The car starting at 0 (speed 1) does not catch up to any other car, so it is a fleet by itself.
- The cars starting at 5 (speed 1) and 3 (speed 3) become a fleet, meeting each other at 6. The fleet moves at speed 1 until it reaches target.

Example 2:

Input: `target = 10`, `position = [3]`, `speed = [3]`

Output: 1

Explanation:

Round 2 · High · Medium

p.726

Example 3:

Input: target = 100, position = [0,2,4], speed = [4,2,1]

Output: 1

Explanation:

- The cars starting at 0 (speed 4) and 2 (speed 2) become a fleet, meeting each other at 4. The car starting at 4 (speed 1) travels to 5.
- Then, the fleet at 4 (speed 2) and the car at position 5 (speed 1) become one fleet, meeting each other at 6. The fleet moves at speed 1 until it reaches target.

Constraints:

- $n == \text{position.length} == \text{speed.length}$
- $1 \leq n \leq 105$
- $0 < \text{target} \leq 106$
- $0 \leq \text{position}[i] < \text{target}$
- All the values of position are unique.
- $0 < \text{speed}[i] \leq 106$

Community Solutions (Python)

• WalkCC

Round 2 · High · Medium

p.727

```
class Solution:
    def carFleet(self, target: int, position: list[int], speed: list[int]) -> int:
        ans = 0
        times = [
            float(target - p) / s for p, s in sorted(zip(position, speed),
                                                    reverse=True)]

        maxTime = 0 # the time of the slowest car to reach the target

        for time in times:
            # A car needs more time to reach the target, so it becomes the slowest.

            if time > maxTime:
                maxTime = time
                ans += 1

        return ans
```

• LeetCode

```
class Solution:
    def carFleet(self, target: int, position: List[int], speed: List[int]) -> int:
        idx = sorted(range(len(position)), key=lambda i: position[i])
        ans = pre = 0
        for i in idx[::-1]:
            t = (target - position[i]) / speed[i]
            if t > pre:
                ans += 1
                pre = t
        return ans
```

• SimplyLeet

Round 2 · High · Medium

p.728


```
# Python implementation of Car Fleet
def carFleet(target, position, speed):
    # Pair up position and speed and sort by starting position in descending
    # order.
    cars = sorted(zip(position, speed), key=lambda car: car[0], reverse=True)

    fleets = 0        # Count of car fleets
    last_time = 0.0    # Time taken by the last fleet to reach the target

    # Process each car from closest to farthest from the target.
    for pos, spd in cars:

        # Calculate time for this car to reach the target
        time = (target - pos) / spd

        # If this car takes longer than the last fleet time, it starts a new
        # fleet.
        if time > last_time:
            fleets += 1
            last_time = time # Update the last fleet time with this car's time

    return fleets

# Example usage:
# target = 12, position = [10,8,0,5,3], speed = [2,4,1,1,3]
# print(carFleet(12, [10,8,0,5,3], [2,4,1,1,3])) # Output: 3
```

Stacks

□ #946 Validate Stack Sequences

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Stack · Simulation
 Lists: AlgoMaster 600

Problem

Given two integer arrays pushed and popped each with distinct values, return true if this could have been the result of a sequence of push and pop operations on an initially empty stack, or false otherwise.

Example 1:

```
Input: pushed = [1,2,3,4,5], popped = [4,5,3,2,1]
Output: true
Explanation: We might do the following sequence:
push(1), push(2), push(3), push(4),
pop() -> 4,
push(5),
pop() -> 5, pop() -> 3, pop() -> 2, pop() -> 1
```

Example 2:

```
Input: pushed = [1,2,3,4,5], popped = [4,3,5,1,2]
Output: false
Explanation: 1 cannot be popped before 2.
```

Constraints:

- $1 \leq \text{pushed.length} \leq 1000$

- $0 \leq \text{pushed}[i] \leq 1000$
- All the elements of pushed are unique.
- $\text{popped.length} == \text{pushed.length}$
- popped is a permutation of pushed.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def validateStackSequences(
        self,
        pushed: list[int],
        popped: list[int],
    ) -> bool:
        stack = []
        i = 0 # popped's index

        for x in pushed:

            stack.append(x)
            while stack and stack[-1] == popped[i]:
                stack.pop()
                i += 1

        return not stack
```

• LeetCode

```
class Solution:
    def validateStackSequences(self, pushed: List[int], popped: List[int]) ->
bool:
        stk = []
        i = 0
        for x in pushed:
            stk.append(x)
            while stk and stk[-1] == popped[i]:
                stk.pop()
                i += 1
        return i == len(popped)
```

• SimplyLeet

```
# Function to validate stack sequences using simulation with a stack.
def validateStackSequences(pushed, popped):
    # Initialize an empty stack.
    stack = []
    # Initialize pointer for popped array.
    popped_index = 0

    # Iterate through each element in the pushed array.
    for num in pushed:
        stack.append(num) # Push the current element onto the stack.

        # Check if the current stack top matches the expected popped element.
        while stack and stack[-1] == popped[popped_index]:
            stack.pop() # Pop the element from the stack.
            popped_index += 1 # Move to the next element in the popped
sequence.

        # If the stack is empty, the sequence is valid.
        return not stack

# Example Usage
pushed = [1,2,3,4,5]

popped = [4,5,3,2,1]
print(validateStackSequences(pushed, popped)) # Expected output: True
```

Stacks

#1249 Minimum Remove to Make Valid Parentheses

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode String · Stack

Lists: AlgoMaster 600

Problem

Given a string s of '(' , ')' and lowercase English characters.

Your task is to remove the minimum number of parentheses ('(' or ')', in any positions) so that the resulting parentheses string is valid and return any valid string.

Formally, a parentheses string is valid if and only if:

- It is the empty string, contains only lowercase characters, or
- It can be written as AB (A concatenated with B), where A and B are valid strings, or
- It can be written as (A) , where A is a valid string.

Example 1:

Round 2 · High · Medium

p.733

Input: $s = \text{"lee(t(c)o)de}"$
 Output: $\text{"lee(t(c)o)de}"$
 Explanation: "lee(t(co)de)" , "lee(t(c)ode)" would also be accepted.

Example 2:

Input: $s = \text{"a)b(c)d}"$
 Output: $\text{"ab(c)d}"$

Example 3:

Input: $s = \text{"")((("}$
 Output: $\text{"}"$
 Explanation: An empty string is also valid.

Constraints:

- $1 \leq s.length \leq 105$
- $s[i]$ is either '(', ')', or lowercase English letter.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minRemoveToMakeValid(self, s: str) -> str:
        stack = [] # unpaired '(' indices
        chars = list(s)

        for i, c in enumerate(chars):
            if c == '(':
                stack.append(i) # Record the unpaired '(' index.
            elif c == ')':
                if stack:
```

Round 2 · High · Medium

p.734

```

        stack.pop() # Find a pair
    else:
        chars[i] = '*' # Mark the unpaired ')' as '*'.

# Mark the unpaired '(' as '*'.
while stack:
    chars[stack.pop()] = '*'

return ''.join(chars).replace('*', '')

```

• LeetCode

```

class Solution:
    def minRemoveToMakeValid(self, s: str) -> str:
        stk = []
        x = 0
        for c in s:
            if c == ')' and x == 0:
                continue
            if c == '(':
                x += 1
            elif c == ')':
                x -= 1
                stk.append(c)
        x = 0
        ans = []
        for c in stk[::-1]:
            if c == '(' and x == 0:
                continue
            if c == ')':
                x += 1
            elif c == '(':
                x -= 1
                ans.append(c)
        return ''.join(ans[::-1])

```

• SimplyLeet

Round 2 · High · Medium

p.735

```

# Define a function to remove invalid parentheses and return a valid string.
def minRemoveToMakeValid(s: str) -> str:
    # Initialize a list to store the indices of '(' that haven't been matched yet.
    open_stack = []
    # Use a set to record indices to remove.
    indices_to_remove = set()

    # Loop over every character and its index in the string.
    for i, char in enumerate(s):
        # If the character is an open parenthesis, store its index.

        if char == '(':
            open_stack.append(i)
        # If the character is a closing parenthesis.
        elif char == ')':
            # If there is a matching open parenthesis, pop from the stack.
            if open_stack:
                open_stack.pop()
            # Otherwise, mark the index of this unmatched ')' for removal.
            else:
                indices_to_remove.add(i)

    # Add any remaining unmatched '(' indices from the stack to the removal set.
    indices_to_remove = indices_to_remove.union(set(open_stack))

    # Build and return the valid string by skipping characters at indices to be removed.
    result = []
    for i, char in enumerate(s):
        if i not in indices_to_remove:
            result.append(char)
    return ''.join(result)

# Example usage:
print(minRemoveToMakeValid("lee(t(c)o)de"))

```

Round 2 · High · Medium

p.736

Stacks

☐ #2390 Removing Stars From a String

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Stack · Simulation
Lists: AlgoMaster 600

Problem

You are given a string *s*, which contains stars *.
In one operation, you can:

- Choose a star in *s*.
- Remove the closest non-star character to its left, as well as remove the star itself.

Return the string after all stars have been removed.

Note:

- The input will be generated such that the operation is always possible.
- It can be shown that the resulting string will always be unique.

Example 1:

Round 2 · High · Medium

p.737

Input: *s* = "leet**cod*e"

Output: "lecoe"

Explanation: Performing the removals from left to right:

- The closest character to the 1st star is 't' in "leet**cod*e". *s* becomes "lee*cod*e".
 - The closest character to the 2nd star is 'e' in "lee*cod*e". *s* becomes "lecod*e".
 - The closest character to the 3rd star is 'd' in "lecod*e". *s* becomes "lecoe".
- There are no more stars, so we return "lecoe".

Example 2:

Input: *s* = "erase*****"

Output: ""

Explanation: The entire string is removed, so we return an empty string.

Constraints:

- $1 \leq s.length \leq 105$
- *s* consists of lowercase English letters and stars *.
- The operation above can be performed on *s*.

Community Solutions (Python)

☐ • WalkCC

```
class Solution:
    def removeStars(self, s: str) -> str:
        ans = []
        for c in s:
            if c == '*':
                ans.pop()
            else:
                ans.append(c)
        return ''.join(ans)
```

☐ • LeetDoocs

Round 2 · High · Medium

p.738

```
class Solution:
    def removeStars(self, s: str) -> str:
        ans = []
        for c in s:
            if c == '*':
                ans.pop()
            else:
                ans.append(c)
        return ''.join(ans)
```

• SimplyLeet

```
# Python solution for Removing Stars From a String
class Solution:
    def removeStars(self, s: str) -> str:
        result_stack = [] # Initialize a stack to hold characters
        # Iterate through each character in the string
        for char in s:
            if char == '*':
                # When a star is encountered, pop the last character off the
                stack

                if result_stack:
                    result_stack.pop()

            else:
                # For any normal character, push it onto the stack
                result_stack.append(char)

        # Join the remaining characters to form the final result
        return ''.join(result_stack)

# Example usage
# sol = Solution()
# print(sol.removeStars("leet**cod*e"))
```

Round 2 · High · Medium

p.739

Tree Traversal – Level Order

Round 2 · High · Medium · 8 questions

Round 2 · High · Medium

p.740

Tree Traversal – Level Order

□ #102 Binary Tree Level Order Traversal **MED**

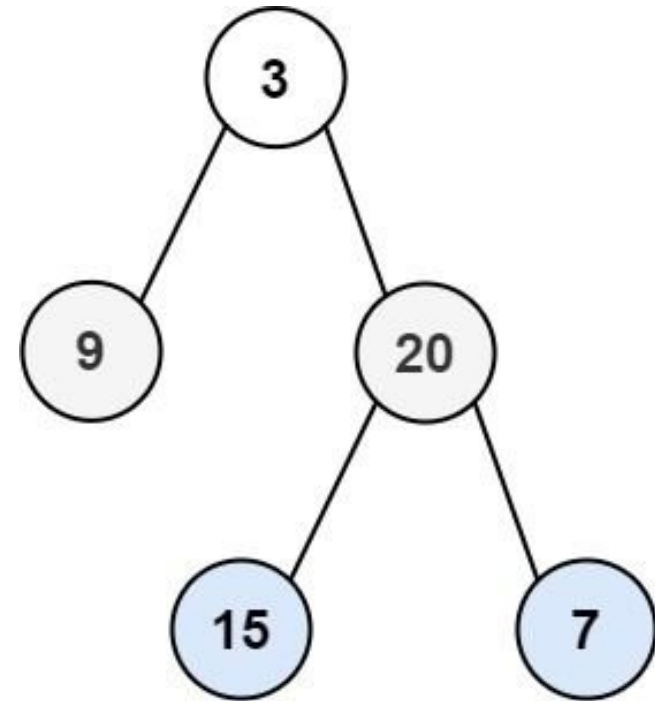
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · BFS

Lists: Grind 169

Problem

Given the root of a binary tree, return the level order traversal of its nodes' values. (i.e., from left to right, level by level).

Example 1:



Input: root = [3,9,20,null,null,15,7]
Output: [[3],[9,20],[15,7]]

Example 2:

Input: root = [1]
Output: [[1]]

Example 3:

Input: root = []
Output: []

Constraints:

Round 2 · High · Medium

p.741

Round 2 · High · Medium

p.742

- The number of nodes in the tree is in the range [0, 2000].
- $-1000 \leq \text{Node.val} \leq 1000$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def levelOrder(self, root: TreeNode | None) -> list[list[int]]:
        if not root:
            return []

        ans = []
        q = collections.deque([root])

        while q:
            currLevel = []

            for _ in range(len(q)):
                node = q.popleft()
                currLevel.append(node.val)
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)
            ans.append(currLevel)

        return ans
```

• LeetCode

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def levelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        ans = []
        if root is None:
            return ans

        q = deque([root])
        while q:
            t = []
            for _ in range(len(q)):
                node = q.popleft()
                t.append(node.val)
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)
            ans.append(t)
        return ans
```

• SimplyLeet

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

from collections import deque

class Solution:
```



```
def levelOrder(self, root: TreeNode) -> List[List[int]]:
    # Return an empty list if the tree is empty.
    if not root:
        return []

    result = [] # Final result list containing values of each level.
    queue = deque([root]) # Initialize the queue with the root node.

    # Process nodes level by level.
    while queue:

        level_size = len(queue) # Number of nodes at the current level.
        level_nodes = [] # List to hold node values at the current level.

        for _ in range(level_size):
            node = queue.popleft() # Dequeue the next node.
            level_nodes.append(node.val)
            # Enqueue left child if it exists.
            if node.left:
                queue.append(node.left)
            # Enqueue right child if it exists.
            if node.right:
                queue.append(node.right)

        # Append the current level's values to the result.
        result.append(level_nodes)
    return result
```

• LC.ca

```
# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):
# self.val = val
# self.left = left
# self.right = right
class Solution:
    def levelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        ans = []
        if root is None:
```

Round 2 · High · Medium

p.745

```
        return ans
    q = deque([root])
    while q:
        t = []
        for _ in range(len(q)):
            node = q.popleft()
            t.append(node.val)
            if node.left:
                q.append(node.left)
            if node.right:
                q.append(node.right)
        ans.append(t)
    return ans
```

◆ Quick Review

Key Insights

- Use Breadth-First Search (BFS) to traverse the tree.
- A queue data structure is ideal for maintaining the order of nodes at each level.
- For every level, process all nodes currently in the queue and add their children for the next level.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes, because each node is processed once.

Round 2 · High · Medium

p.746

Space Complexity: $O(n)$, which accounts for the space used by the queue in the worst-case scenario.

Solution

This solution leverages a BFS approach using a queue. The algorithm starts by enqueueing the root node. Then, it enters a loop that continues until the queue is empty. For each iteration—representing one level of the tree—it computes the number of nodes at that level (`levelSize`), processes these nodes by dequeuing them, and collects their values. Their non-null children are then enqueued. The list of values for each level is added to the final result list. Key data structures include the queue (for managing nodes) and a list (to store level values).

Tree Traversal – Level Order

□ #103 Binary Tree Zigzag Level Order Traversal

MED

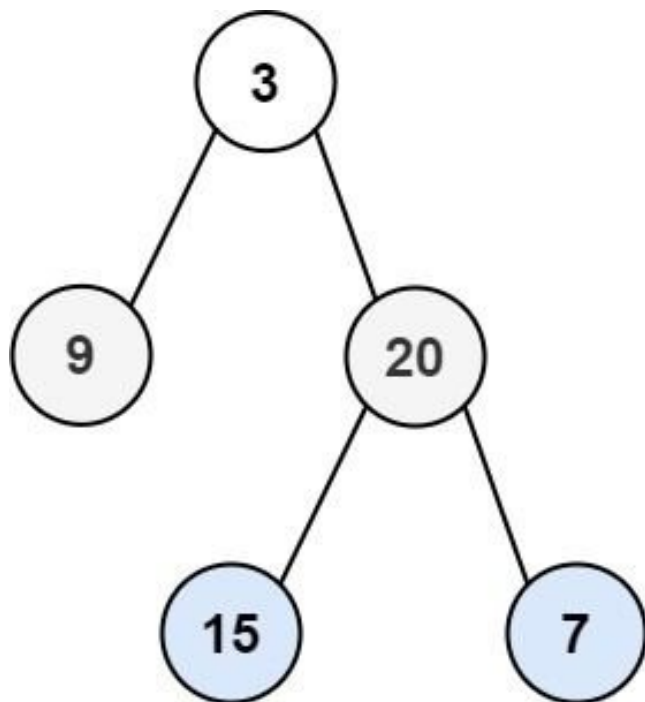
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · BFS

Lists: Grind 169

Problem

Given the root of a binary tree, return the zigzag level order traversal of its nodes' values. (i.e., from left to right, then right to left for the next level and alternate between).

Example 1:



Input: root = [3,9,20,null,null,15,7]
 Output: [[3],[20,9],[15,7]]

Example 2:

Input: root = [1]
 Output: [[1]]

Example 3:

Input: root = []
 Output: []

Constraints:

Round 2 · High · Medium

p.749

- The number of nodes in the tree is in the range [0, 2000].
- $-100 \leq \text{Node.val} \leq 100$

Community Solutions (Python)

• WalkCC

```

class Solution:
    def zigzagLevelOrder(self, root: TreeNode | None) -> list[list[int]]:
        if not root:
            return []

        ans = []
        dq = collections.deque([root])
        isLeftToRight = True

        while dq:
            currLevel = []
            for _ in range(len(dq)):
                if isLeftToRight:
                    node = dq.popleft()
                    currLevel.append(node.val)
                    if node.left:
                        dq.append(node.left)
                    if node.right:
                        dq.append(node.right)
                else:
                    node = dq.pop()
                    currLevel.append(node.val)
                    if node.right:
                        dq.appendleft(node.right)
                    if node.left:
                        dq.appendleft(node.left)
            ans.append(currLevel)
            isLeftToRight = not isLeftToRight

        return ans

```

Round 2 · High · Medium

p.750

• LeetCode

```
# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):
# self.val = val
# self.left = left
# self.right = right
class Solution:
    def zigzagLevelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        ans = []
        if root is None:
            return ans
        q = deque([root])
        ans = []
        left = 1
        while q:
            t = []
            for _ in range(len(q)):
                node = q.popleft()
                t.append(node.val)
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)
            ans.append(t if left else t[::-1])
            left ^= 1
        return ans
```

• SimplyLeet

```
# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):
# self.val = val
# self.left = left
# self.right = right

def zigzagLevelOrder(root):
    # If the tree is empty, return an empty list
    if not root:
```

Round 2 · High · Medium

p.751

```
return []

# Initialize the result list and the queue with the root node.
result = []
queue = [root]
left_to_right = True # Flag to indicate the current direction of traversal

# Process the tree level by level.
while queue:
    level_size = len(queue)

    current_level = []

    # Process all nodes for the current level.
    for i in range(level_size):
        node = queue.pop(0) # Dequeue the node from the front of the queue
        current_level.append(node.val) # Append the node's value

        # Enqueue the left child if it exists.
        if node.left:
            queue.append(node.left)

        # Enqueue the right child if it exists.
        if node.right:
            queue.append(node.right)

    # If the current level's order is right-to-left, reverse the list.
    if not left_to_right:
        current_level.reverse()

    # Append the current level's values to the result.
    result.append(current_level)

    # Toggle the direction for the next level.
    left_to_right = not left_to_right

return result
```

• LC.ca

Round 2 · High · Medium

p.752

```
# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):
# self.val = val
# self.left = left
# self.right = right

'''
can also use list.insert()
```

```
>>> my_list = [2, 3, 4]
>>> my_list.insert(0, 1) # Insert 1 at the head of the list
>>> print(my_list) # Output: [1, 2, 3, 4]
```

```
>>> a = deque([])
>>> a
deque([])
>>> a.append(1)
>>> a.append(2)
```

```
>>> a.append(3)
>>> a
deque([1, 2, 3])
>>>
>>> a.append(0, 555)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: deque.append() takes exactly one argument (2 given)
>>> a.insert(0, 555)
>>> a
```

```
deque([555, 1, 2, 3])
'''
```

```
from collections import deque
```

```
class Solution:
    def zigzagLevelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        ans = []
        if root is None:
            return ans
```

```
q = deque([root])
ans = []
left = True
while q:
    t = []
    for _ in range(len(q)):
        node = q.popleft()
        t.append(node.val)
        if node.left:
            q.append(node.left)
```

```
        if node.right:
            q.append(node.right)
        ans.append(t if left else t[::-1])
        left = (not left)
    return ans
```

```
#####
```

```
# Definition for a binary tree node.
# class TreeNode(object):
```

```
# def __init__(self, x):
# self.val = x
# self.left = None
# self.right = None
from collections import deque
```

```
class Solution(object):
    def zigzagLevelOrder(self, root):
        '''
```

```
:type root: TreeNode
:rtype: List[List[int]]
'''
```

```
stack = deque([root])
ans = []
odd = True
while stack:
    level = []
    for k in range(0, len(stack)):
        top = stack.popleft()
```

```

    if top is None:
        continue
    level.append(top.val)
    stack.append(top.left)
    stack.append(top.right)
    if level:
        if odd:
            ans.append(level)
        else:
            ans.append(level[::-1])

```

```

    odd = not odd
    return ans

```

```
#####
```

```

# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):
# self.val = val
# self.left = left

```

```

# self.right = right
class Solution:
    def zigzagLevelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        ans = []
        if root is None:
            return ans
        q = deque([root])
        ans = []
        left = 1
        while q:

```

```

            t = []
            for _ in range(len(q)):
                node = q.popleft()
                t.append(node.val)
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)
            ans.append(t if left else t[::-1])
            left ^= 1

```

Round 2 · High · Medium

p.755

```
return ans
```

◆ Quick Review

Key Insights

- Use a breadth-first search (BFS) approach to traverse the tree level by level.
- Alternate the direction of traversal on each level to achieve the zigzag pattern.
- A flag or counter can be used to determine the order of nodes for the current level.
- Using a double-ended data structure or simply reversing the list at every alternate level is an effective approach.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes, because each node is visited exactly once.

Space Complexity: $O(n)$, as in the worst-case scenario (e.g., a complete binary tree) the queue used for BFS may hold $O(n)$ nodes at once.

Solution

Round 2 · High · Medium

p.756

We can solve this problem using a breadth-first search (BFS). Start by initializing a queue to hold nodes for each level. For each level:

- Determine the number of nodes at the current level.
 - Iterate over these nodes, appending their children for the next level.
 - Depending on a flag (or level count), either append node values in the natural order (left to right) or reverse the order (right to left) before adding to the result.
- Key data structures:
- A queue for BFS traversal.
 - A list (or similar container) to store current level values. The main trick is toggling the order of insertion or reversing the list at alternate levels to achieve the "zigzag" pattern.

Tree Traversal – Level Order

☐ #116 Populating Next Right Pointers in Each Node

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Linked List · Tree · Depth-First Search ·
 Breadth-First Search · Binary Tree
 Lists: AlgoMaster 600

Problem

You are given a perfect binary tree where all leaves are on the same level, and every parent has two children. The binary tree has the following definition:

```
struct Node {
    int val;
    Node *left;
    Node *right;
    Node *next;
}
```

Populate each next pointer to point to its next right node. If there is no next right node, the next pointer should be set to NULL.

Initially, all next pointers are set to NULL.

Example 1:

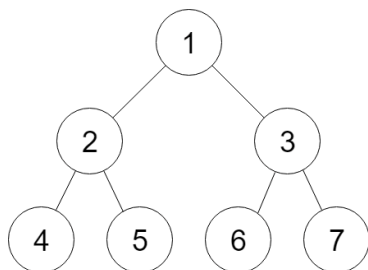


Figure A

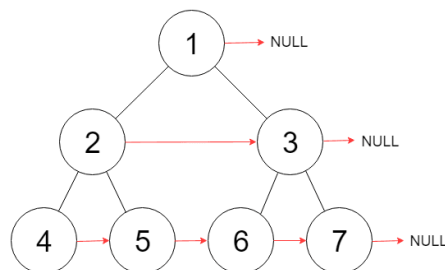


Figure B

Input: root = [1,2,3,4,5,6,7]

Output: [1,#,2,3,#,4,5,6,7,#]

Explanation: Given the above perfect binary tree (Figure A), your function should populate each next pointer to point to its next right node, just like in Figure B. The serialized output is in level order as connected by the next pointers, with '#' signifying the end of each level.

Example 2:

Input: root = []

Output: []

Constraints:

- The number of nodes in the tree is in the range [0, 2¹² - 1].
- -1000 ≤ Node.val ≤ 1000

Follow-up:

- You may only use constant extra space.
- The recursive approach is fine. You may assume implicit stack space does not count as extra space for this problem.

Community Solutions (Python)

Round 2 · High · Medium

p.759

• WalkCC

```

class Solution:
    def connect(self, root: 'Node | None') -> 'Node | None':
        if not root:
            return None

```

```

    def connectTwoNodes(p, q) -> None:
        if not p:
            return
        p.next = q
        connectTwoNodes(p.left, p.right)

```

```

        connectTwoNodes(q.left, q.right)
        connectTwoNodes(p.right, q.left)

```

```

        connectTwoNodes(root.left, root.right)
        return root

```

```

class Solution:
    def connect(self, root: 'Node') -> 'Node':
        node = root # the node that is above the current needling

        while node and node.left:
            dummy = Node(0) # a dummy node before needling
            # Needle the children of the node.
            needle = dummy
            while node:
                needle.next = node.left

                needle = needle.next
                needle.next = node.right
                needle = needle.next
                node = node.next
            node = dummy.next # Move the node to the next level.

        return root

```

• LeetCode

Round 2 · High · Medium

p.760


```

"""
# Definition for a Node.
class Node:
    def __init__(self, val: int = 0, left: 'Node' = None, right: 'Node' = None,
next: 'Node' = None):
        self.val = val
        self.left = left
        self.right = right
        self.next = next
"""

```

```

class Solution:
    def connect(self, root: "Optional[Node]") -> "Optional[Node]":
        if root is None:
            return root
        q = deque([root])
        while q:
            p = None
            for _ in range(len(q)):
                node = q.popleft()

```

```

                if p:
                    p.next = node
                p = node
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)
            return root

```

• SimplyLeet

Round 2 · High · Medium

p.761

```

# Definition for a Node.
class Node:
    def __init__(self, val: int = 0, left: 'Node' = None, right: 'Node' = None
, next: 'Node' = None):
        self.val = val          # Node's value
        self.left = left        # Left child
        self.right = right      # Right child
        self.next = next       # Next right pointer

```

```

def connect(root: 'Node') -> 'Node':
    if not root:

```

```

        return root # If tree is empty, return null

```

```

# Start with the root node
level_start = root
while level_start.left:
    # Traverse nodes at the current level
    current = level_start
    while current:
        # Connect left child to right child
        current.left.next = current.right

```

```

        # Connect right child to the next node's left child if available
        if current.next:
            current.right.next = current.next.left
        # Move to the next node on the same level using next pointer
        current = current.next
    # Move to the leftmost node of the next level
    level_start = level_start.left

```

```

    return root # Return the modified tree

```

Round 2 · High · Medium

p.762

Tree Traversal – Level Order

#117 Populating Next Right Pointers in Each Node II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Linked List · Tree · Depth-First Search ·
 Breadth-First Search · Binary Tree

Lists: AlgoMaster 600

Problem

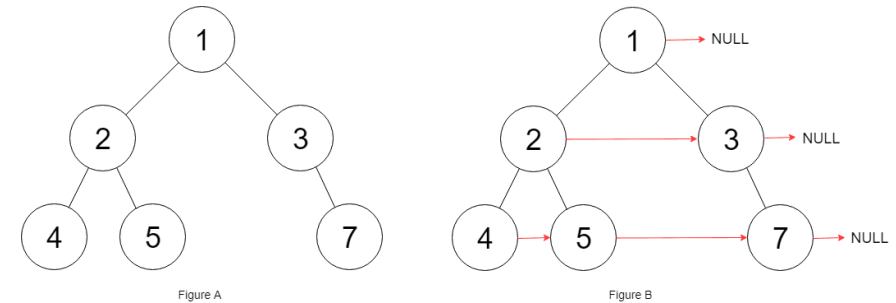
Given a binary tree

```
struct Node {
  int val;
  Node *left;
  Node *right;
  Node *next;
}
```

Populate each next pointer to point to its next right node. If there is no next right node, the next pointer should be set to NULL.

Initially, all next pointers are set to NULL.

Example 1:



Input: root = [1,2,3,4,5,null,7]

Output: [1,#,2,3,#,4,5,7,#]

Explanation: Given the above binary tree (Figure A), your function should populate each next pointer to point to its next right node, just like in Figure B. The serialized output is in level order as connected by the next pointers, with '#' signifying the end of each level.

Example 2:

Input: root = []

Output: []

Constraints:

- The number of nodes in the tree is in the range [0, 6000].
- $-100 \leq \text{Node.val} \leq 100$

Follow-up:

- You may only use constant extra space.
- The recursive approach is fine. You may assume implicit stack space does not count as extra space for this problem.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def connect(self, root: 'Node') -> 'Node':
        node = root # the node that is above the current needling

        while node:
            dummy = Node(0) # a dummy node before needling
            # Needle the children of the node.
            needle = dummy
            while node:
                if node.left: # Needle the left child.

                    needle.next = node.left
                    needle = needle.next
                if node.right: # Needle the right child.
                    needle.next = node.right
                    needle = needle.next
                node = node.next
            node = dummy.next # Move the node to the next level.

        return root
```

• LeetDoocs

```
"""
# Definition for a Node.
class Node:
    def __init__(self, val: int = 0, left: 'Node' = None, right: 'Node' = None,
next: 'Node' = None):
        self.val = val
        self.left = left
        self.right = right
        self.next = next
"""
```

Round 2 · High · Medium

p.765

```
class Solution:
    def connect(self, root: "Node") -> "Node":
        if root is None:
            return root
        q = deque([root])
        while q:
            p = None
            for _ in range(len(q)):
                node = q.popleft()

                if p:
                    p.next = node
                p = node
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)
            return root
```

• SimplyLeet

```
# Definition for a Node.
# class Node:
#     def __init__(self, val: int = 0, left: 'Node' = None, right: 'Node' = None,
next: 'Node' = None):
#         self.val = val
#         self.left = left
#         self.right = right
#         self.next = next

class Solution:
    def connect(self, root: 'Node') -> 'Node':
```

Round 2 · High · Medium

p.766

```

# Start with the root node
current = root
# Process level by level until no nodes remain
while current:
    # Initialize pointers for the next level
    nextHead = None
    nextTail = None
    # Traverse nodes in the current level using next pointers
    while current:
        # Process current node's left child

        if current.left:
            if not nextHead:
                nextHead = current.left
                nextTail = current.left
            else:
                nextTail.next = current.left
                nextTail = nextTail.next
        # Process current node's right child
        if current.right:
            if not nextHead:
                nextHead = current.right
                nextTail = current.right
            else:
                nextTail.next = current.right
                nextTail = nextTail.next
        # Move to the next node in current level
        current = current.next
    # Move to next level
    current = nextHead
return root

```

Round 2 · High · Medium

p.767

Tree Traversal – Level Order

□ #199 Binary Tree Right Side View

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · BFS
Lists: Grind 169

Problem

Given the root of a binary tree, imagine yourself standing on the right side of it, return the values of the nodes you can see ordered from top to bottom.

Example 1:

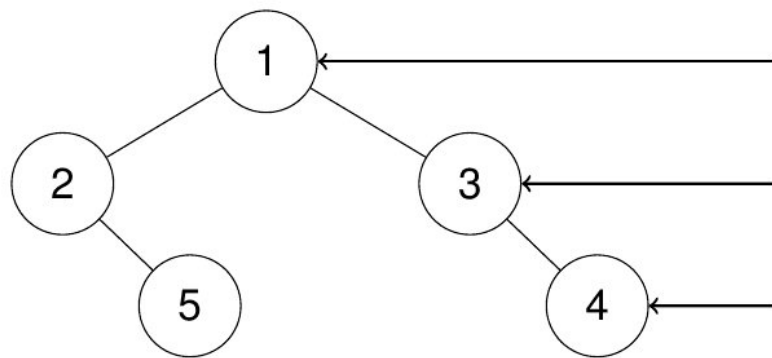
Input: root = [1,2,3,null,5,null,4]

Output: [1,3,4]

Explanation:

Round 2 · High · Medium

p.768

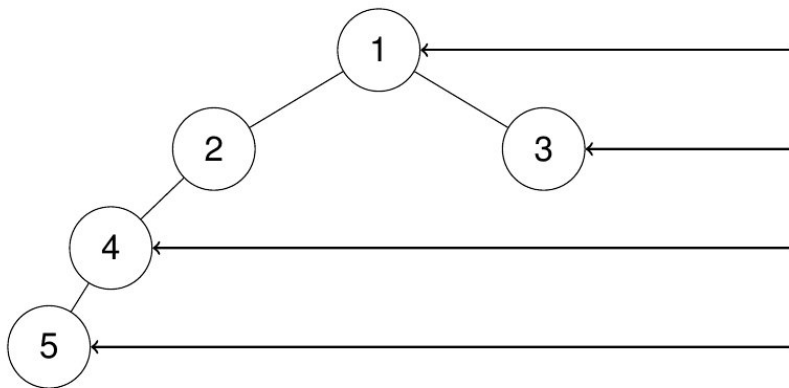


Example 2:

Input: root = [1,2,3,4,null,null,null,5]

Output: [1,3,4,5]

Explanation:



Round 2 · High · Medium

p.769

Example 3:

Input: root = [1,null,3]

Output: [1,3]

Example 4:

Input: root = []

Output: []

Constraints:

- The number of nodes in the tree is in the range [0, 100].
- $-100 \leq \text{Node.val} \leq 100$

Community Solutions (Python)

• WalkCC

```

class Solution:
    def rightSideView(self, root: TreeNode | None) -> list[int]:
        if not root:
            return []

        ans = []
        q = collections.deque([root])

        while q:
            size = len(q)
  
```

Round 2 · High · Medium

p.770

```

for i in range(size):
    root = q.popleft()
    if i == size - 1:
        ans.append(root.val)
    if root.left:
        q.append(root.left)
    if root.right:
        q.append(root.right)

```

```

return ans

```

```

class Solution:
    def rightSideView(self, root: TreeNode | None) -> List[int]:
        ans = []

```

```

    def dfs(self, root: TreeNode | None, depth: int) -> None:
        if not root:
            return

```

```

        if depth == len(ans):
            ans.append(root.val)

```

```

        dfs(self, root.right, depth + 1)
        dfs(self, root.left, depth + 1)

```

```

    def dfs(self, root: TreeNode | None, depth: int) -> None:
        dfs(self, root, 0)
        return ans

```

• LeetCode

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def rightSideView(self, root: Optional[TreeNode]) -> List[int]:
        ans = []
        if root is None:

```

```

        return ans
        q = deque([root])
        while q:
            ans.append(q[-1].val)
            for _ in range(len(q)):
                node = q.popleft()
                if node.right:
                    q.append(node.right)
                if node.left:
                    q.append(node.left)

```

```

        return ans

```

• SimplyLeet

```

# Python solution using BFS
from collections import deque

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val          # Node value
        self.left = left        # Left child
        self.right = right      # Right child

    def rightSideView(self):

```

```

        # If the tree is empty, return an empty list
        if not root:
            return []

        right_view = []          # List to store right side view
        queue = deque([root])    # Initialize queue with the root node

        while queue:
            level_size = len(queue)  # Number of nodes in current level
            for i in range(level_size):

```

```

node = queue.popleft() # Get the front node from the queue
# if it's the last node in the current level, add it.
if i == level_size - 1:
    right_view.append(node.val)
# add left child if exists
if node.left:
    queue.append(node.left)
# add right child if exists
if node.right:
    queue.append(node.right)

```

```

return right_view

```

• LC.ca

```

# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):
# self.val = val
# self.left = left
# self.right = right
class Solution:
    def rightSideView(self, root: Optional[TreeNode]) -> List[int]:
        ans = []
        if root is None:

```

```

            return ans
        q = deque([root])
        while q:
            ans.append(q[-1].val) # add last node of previous level traversal
            results
            for _ in range(len(q)):
                node = q.popleft()
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)

```

Round 2 · High · Medium

p.773

```

return ans

#####

# Definition for a binary tree node.
# class TreeNode(object):
# def __init__(self, x):
# self.val = x
# self.left = None
# self.right = None

```

```

from collections import deque

```

```

class Solution(object):
    def rightSideView(self, root):
        """
        :type root: TreeNode
        :rtype: List[int]
        """

```

```

    def dfs(root, h):
        if root:
            if h == len(ans):
                ans.append(root.val)
            # pre-order, all the way to the right
            dfs(root.right, h + 1)
            dfs(root.left, h + 1)

```

```

ans = []
dfs(root, 0)

```

```

return ans

```

◆ Quick Review

Key Insights

Round 2 · High · Medium

p.774

- The problem is essentially about capturing the last node at each level (or the rightmost node) in a binary tree.
- A Breadth-First Search (BFS) approach naturally works by processing nodes level by level.
- Alternatively, Depth-First Search (DFS) can be adapted by prioritizing the right subtree first, ensuring that the first node encountered at each depth is the visible one.
- The number of nodes is capped at 100, so both approaches are efficient.

Space & Time

Time Complexity: $O(n)$ – we visit every node in the tree.

Space Complexity: $O(n)$ – in the worst case, the queue (or call stack for DFS) may hold all nodes of the tree.

Solution

We can solve this problem using BFS (level-order traversal). The idea is to traverse the tree level by level using a queue. For each level, the last element added to the level represents the rightmost view from that level.

Round 2 · High · Medium

p.775

We then append that element's value to our result list.

Alternatively, a DFS approach can be used where we visit the right subtree first. By keeping track of the current depth and checking if it's the first time we've encountered this depth, we can add the first node visited at that depth (which will be the rightmost one) to the result.

In both approaches, appropriate data structures (a queue for BFS or recursion with a list for DFS) are used to store nodes and track the traversal state.

Round 2 · High · Medium

p.776

Tree Traversal – Level Order

★ #314 Binary Tree Vertical Order Traversal ★

MED

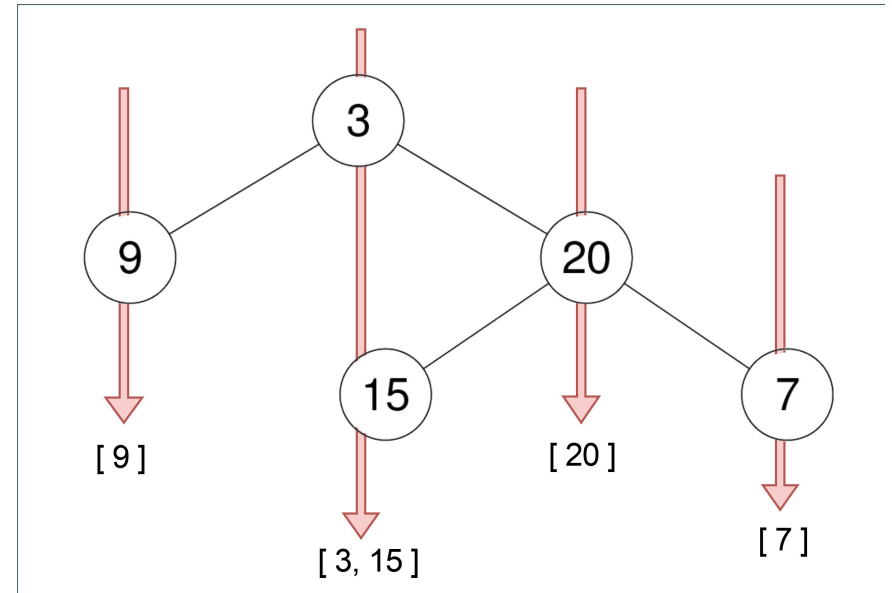
LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Hash Table · Tree · BFS · Sorting
 Lists: ★ Premium | Premium 98

Problem

Given the root of a binary tree, return the vertical order traversal of its nodes' values. (i.e., from top to bottom, column by column).

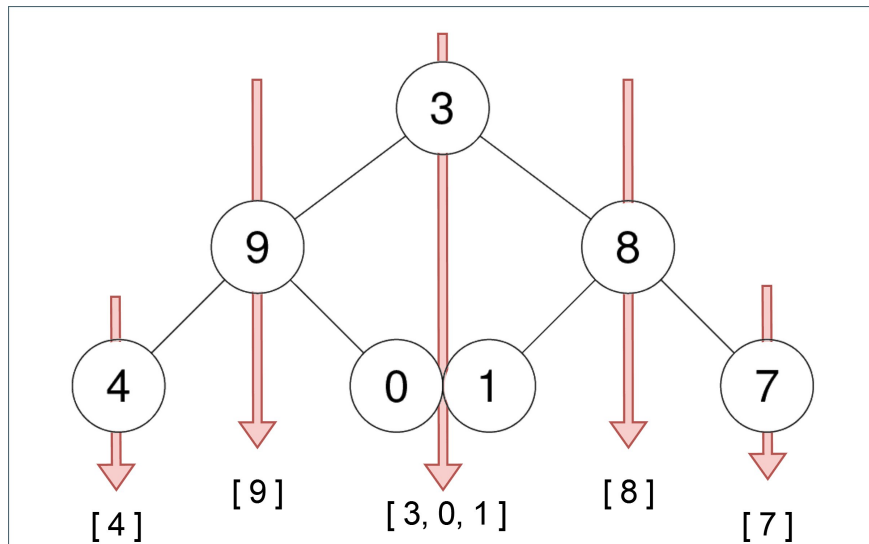
If two nodes are in the same row and column, the order should be from left to right.

Example 1:



Input: root = [3,9,20,null,null,15,7]
 Output: [[9],[3,15],[20],[7]]

Example 2:

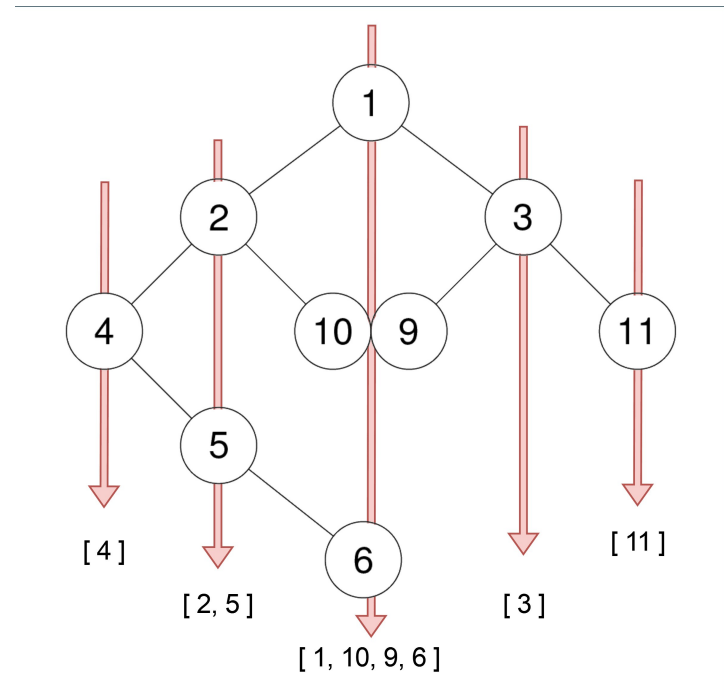


Input: root = [3,9,8,4,0,1,7]
 Output: [[4],[9],[3,0,1],[8],[7]]

Example 3:

Round 2 · High · Medium

p.779



Input: root = [1,2,3,4,10,9,11,null,5,null,null,null,null,null,null,6]
 Output: [[4],[2,5],[1,10,9,6],[3],[11]]

Constraints:

- The number of nodes in the tree is in the range [0, 100].
- $-100 \leq \text{Node.val} \leq 100$

Community Solutions (Python)

- WalkCC

Round 2 · High · Medium

p.780

```
class Solution:
    def verticalOrder(self, root: TreeNode | None) -> list[list[int]]:
        if not root:
            return []

        range_ = [0] * 2

        def getRange(root: TreeNode | None, x: int) -> None:
            if not root:
                return

            range_[0] = min(range_[0], x)
            range_[1] = max(range_[1], x)

            getRange(root.left, x - 1)
            getRange(root.right, x + 1)

        getRange(root, 0) # Get the leftmost and the rightmost x index.

        ans = [[] for _ in range(range_[1] - range_[0] + 1)]

        q = collections.deque([(root, -range_[0])]) # (TreeNode, x)

        while q:
            node, x = q.popleft()
            ans[x].append(node.val)
            if node.left:
                q.append((node.left, x - 1))
            if node.right:
                q.append((node.right, x + 1))

        return ans
```

• LeetCode

Round 2 · High · Medium

p.781

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def verticalOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        def dfs(root, depth, offset):
            if root is None:
                return
            d[offset].append((depth, root.val))
            dfs(root.left, depth + 1, offset - 1)
            dfs(root.right, depth + 1, offset + 1)

        d = defaultdict(list)
        dfs(root, 0, 0)
        ans = []
        for _, v in sorted(d.items()):
            v.sort(key=lambda x: x[0])

            ans.append([x[1] for x in v])
        return ans
```

• SimplyLeet

```
from collections import deque, defaultdict

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, x):
        self.val = x
        self.left = None
        self.right = None

class Solution:
```

Round 2 · High · Medium

p.782

```
def verticalOrder(self, root: TreeNode):
    # Return empty list if the tree is empty
    if not root:
        return []

    # Dictionary to hold nodes for each column index
    col_table = defaultdict(list)
    # Use deque for efficient FIFO queue for BFS; store tuple (node,
    column)
    queue = deque([(root, 0)])

    while queue:
        node, col = queue.popleft()
        # Append current node value to the list of its column
        col_table[col].append(node.val)
        # If there is a left child, assign column col-1
        if node.left:
            queue.append((node.left, col - 1))
        # If there is a right child, assign column col+1
        if node.right:
            queue.append((node.right, col + 1))

    # Extract the columns in sorted order to build the final result
    sorted_cols = sorted(col_table.keys())
    result = [col_table[col] for col in sorted_cols]
    return result
```

• LC.ca

```
# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):
# self.val = val
# self.left = left
# self.right = right

'''
>>> d = {1:'a', -1:'b', 10:'c', -100:'d'}
>>>
```

Round 2 · High · Medium

p.783

```
>>> d.items()
{1: 'a', 10: 'c', -100: 'd', -1: 'b'}

>>> ds = sorted(d.items())
>>> ds
[(-100, 'd'), (-1, 'b'), (1, 'a'), (10, 'c')]

>>> sorted(d.items(), key=lambda l: l[1])
[(1, 'a'), (-1, 'b'), (10, 'c'), (-100, 'd')]
>>> dict(sorted(d.items(), key=lambda l: l[1]))

{1: 'a', 10: 'c', -100: 'd', -1: 'b'}

>>> [v for i,v in sorted(d.items(), key=lambda l: l[0])]
['d', 'b', 'a', 'c']
'''

class Solution:
    def verticalOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        if root is None:
            return []

        q = deque([(root, 0)])
        d = defaultdict(list)
        while q:
            for _ in range(len(q)):
                root, offset = q.popleft()
                d[offset].append(root.val)
                if root.left:
                    q.append((root.left, offset - 1))
                if root.right:
                    q.append((root.right, offset + 1))

        return [v for _, v in sorted(d.items())]

#####

from sortedcontainers import SortedDict
from typing import List
from collections import deque

# similar to above, but use existing data structure
class Solution:
```

Round 2 · High · Medium

p.784

```
def verticalOrder(self, root: TreeNode) -> List[List[int]]:
    if not root:
        return []

    # Use SortedDict to automatically sort by column index
    sorted_dict = SortedDict()

    # Queue for breadth-first search; stores pairs of (node, column_index)
    queue = deque([(0, root)])

    while queue:
        node, column = queue.popleft()

        if column not in sorted_dict:
            sorted_dict[column] = [node.val]
        else:
            sorted_dict[column].append(node.val)

        # Add left and right children to the queue
        if node.left:
            queue.append((node.left, column - 1))
        if node.right:
            queue.append((node.right, column + 1))

    # Return the values from sorted_dict as a list of lists
    return list(sorted_dict.values())

#####

# another SortedDict, but different handling on None node, before enqueue vs
after enqueue
```

Round 2 · High · Medium

p.785

```
class Solution:
    def verticalOrder(self, root: TreeNode):
        if not root:
            return []

        # Initialize a SortedDict to maintain columns in sorted order.
        column_table = SortedDict()
        queue = deque([(root, 0)]) # (node, column_index)

        while queue:
            node, column = queue.popleft()

            if node is not None:
                if column not in column_table:
                    column_table[column] = []
                column_table[column].append(node.val)

            # Add left and right children to the queue.
            queue.append((node.left, column - 1))
            queue.append((node.right, column + 1))

        # Extract the values from the sorted dictionary and return them.
        return list(column_table.values())

#####

from collections import defaultdict

class Solution(object):
    def verticalOrder(self, root):
        """
        :type root: TreeNode
        :rtype: List[List[int]]
        """

        def dfs(p, i, j, res): # i -> depth, j -> vertical-shift
            if p:
                res[j].append((p.val, i))
```

Round 2 · High · Medium

p.786

```

        self.leftMost = min(j, self.leftMost)
        dfs(p.left, i + 1, j - 1, res)
        dfs(p.right, i + 1, j + 1, res)

    self.leftMost = float("inf")
    ans = []
    res = defaultdict(list)
    dfs(root, 0, 0, res)
    i = self.leftMost
    while True:
        if not res[i]:
            break
        ans.append([item[0] for item in sorted(res[i], key=lambda a: a[1])])
        i += 1
    return ans

```

◆ Quick Review

Key Insights

- Use Breadth-First Search (BFS) to traverse the tree level by level.
- Track the column index for each node where root is column 0, left child is column-1, and right child is column+1.
- Group nodes by their column indices using a hash table (or dictionary).
- BFS traversal naturally handles left-to-right ordering within the same level.
- Finally, sort the keys (column indices) to generate results from leftmost column to

rightmost column.

Space & Time

Time Complexity: $O(n \log n)$ in the worst-case due to sorting the column indices (n nodes in the worst-case if each node is on a unique column).

Space Complexity: $O(n)$ for storing the node information and the resultant structure.

Solution

The solution uses BFS with a queue that stores nodes along with their corresponding column index. For each visited node, add its value to a dictionary keyed by its column index. When processing children, update the column index appropriately (left child: $col-1$, right child: $col+1$). After the traversal, sort the keys of the dictionary and compile the node values in order from smallest to largest column index. This approach guarantees that nodes at the same level will be processed in left-to-right order, matching the required output.

Tree Traversal – Level Order

#662 Maximum Width of Binary Tree

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · BFS

Lists: Grind 169

Problem

Given the root of a binary tree, return the maximum width of the given tree.

The maximum width of a tree is the maximum width among all levels.

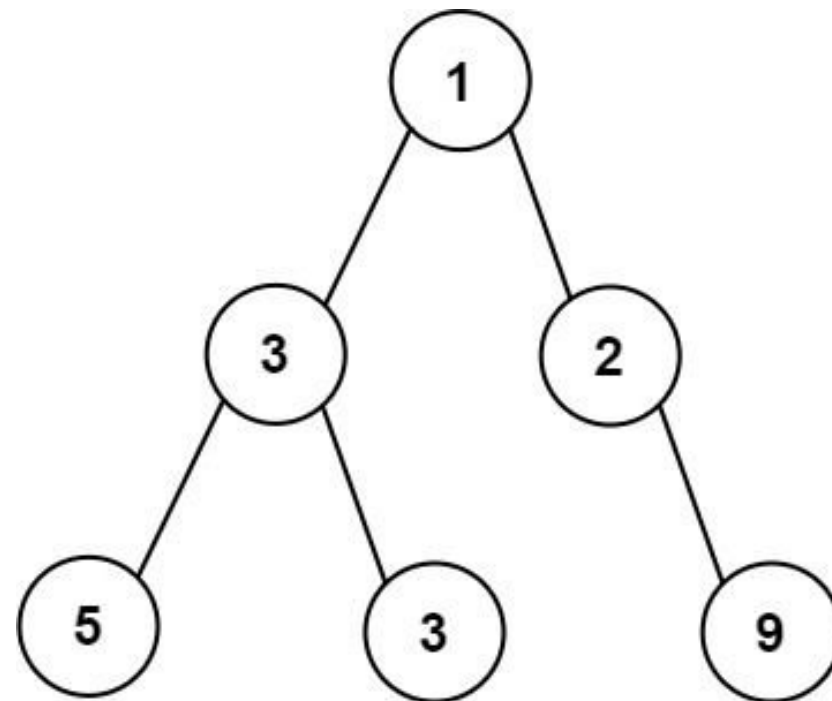
The width of one level is defined as the length between the end-nodes (the leftmost and rightmost non-null nodes), where the null nodes between the end-nodes that would be present in a complete binary tree extending down to that level are also counted into the length calculation.

It is guaranteed that the answer will in the range of a 32-bit signed integer.

Example 1:

Round 2 · High · Medium

p.789



Input: root = [1,3,2,5,3,null,9]

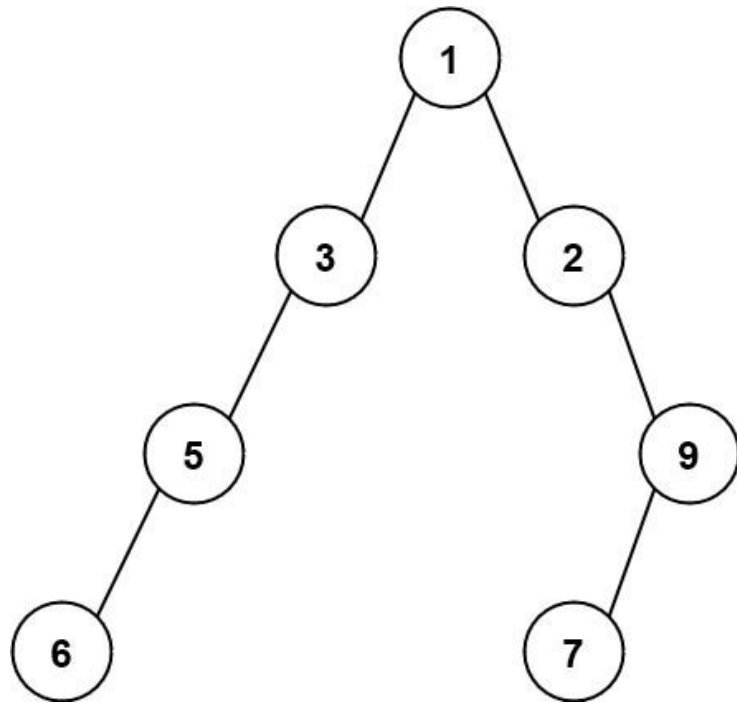
Output: 4

Explanation: The maximum width exists in the third level with length 4 (5,3,null,9).

Example 2:

Round 2 · High · Medium

p.790



Input: root = [1,3,2,5,null,null,9,6,null,7]

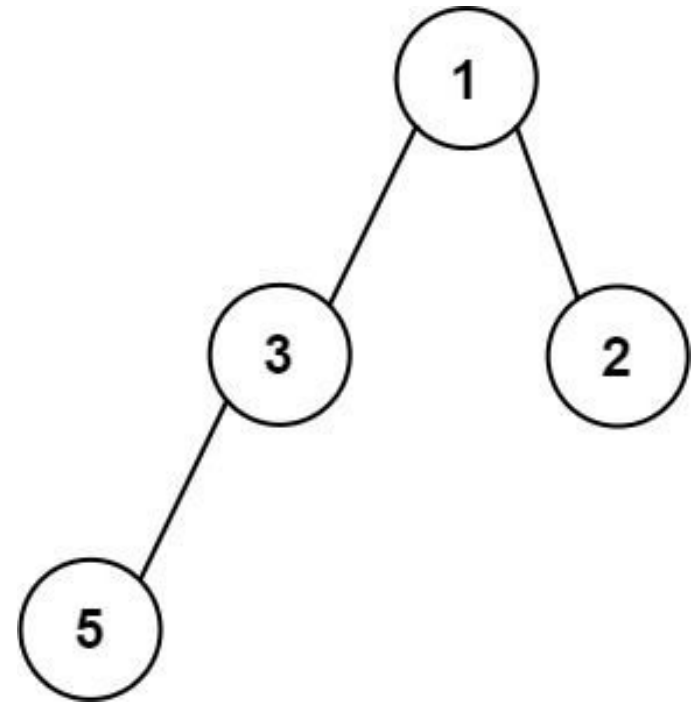
Output: 7

Explanation: The maximum width exists in the fourth level with length 7 (6,null,null,null,null,null,7).

Example 3:

Round 2 · High · Medium

p.791



Input: root = [1,3,2,5]

Output: 2

Explanation: The maximum width exists in the second level with length 2 (3,2).

Constraints:

- The number of nodes in the tree is in the range [1, 3000].
- $-100 \leq \text{Node.val} \leq 100$

Community Solutions (Python)

- [LeetDoocs](#)

Round 2 · High · Medium

p.792


```
# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):
# self.val = val
# self.left = left
# self.right = right
class Solution:
    def widthOfBinaryTree(self, root: Optional[TreeNode]) -> int:
        ans = 0
        q = deque([(root, 1)])

        while q:
            ans = max(ans, q[-1][1] - q[0][1] + 1)
            for _ in range(len(q)):
                root, i = q.popleft()
                if root.left:
                    q.append((root.left, i << 1))
                if root.right:
                    q.append((root.right, i << 1 | 1))
        return ans
```

• SimplyLeet

```
# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):
# self.val = val
# self.left = left
# self.right = right

from collections import deque

class Solution:
```

Round 2 · High · Medium

p.793

```
def widthOfBinaryTree(self, root: TreeNode) -> int:
    if not root:
        return 0

    max_width = 0
    # Initialize queue with a tuple of (node, index)
    queue = deque([(root, 1)])

    while queue:
        level_length = len(queue)

        # Get the index of first node at current level
        _, first_index = queue[0]
        # Process nodes at current level
        for i in range(level_length):
            node, index = queue.popleft()
            # Check and add left child with index 2 * index
            if node.left:
                queue.append((node.left, 2 * index))
            # Check and add right child with index 2 * index + 1
            if node.right:
                queue.append((node.right, 2 * index + 1))
        # For the last node, calculate width of the level
        if i == level_length - 1:
            current_width = index - first_index + 1
            max_width = max(max_width, current_width)
    return max_width
```

• LC.ca

```
# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):
# self.val = val
# self.left = left
# self.right = right
class Solution:
    def widthOfBinaryTree(self, root: Optional[TreeNode]) -> int:
        ans = 0
        q = deque([(root, 1)])
```

Round 2 · High · Medium

p.794

```

while q:
    ans = max(ans, q[-1][1] - q[0][1] + 1)
    for _ in range(len(q)):
        root, i = q.popleft()
        if root.left:
            q.append((root.left, i << 1))
        if root.right:
            q.append((root.right, i << 1 | 1))
return ans

```

◆ Quick Review

Key Insights

- Perform a level order traversal (BFS) of the tree.
- Assign an index to each node as if the tree were complete (e.g., root is index 1, left child is 2
- i, right child is 2
- i+1).
- For each level, compute the width as (last_index - first_index + 1).
- Using indices allows capturing gaps between nodes due to nulls in between.
- Edge cases include handling very skewed trees.

Space & Time

Round 2 · High · Medium

p.795

Time Complexity: $O(N)$, where N is the number of nodes in the tree.

Space Complexity: $O(N)$, due to the queue used for BFS which in the worst case holds nodes of the last level.

Solution

We use a Breadth-First Search (BFS) approach with a queue to traverse the tree level by level. At each level, we pair each node with an index representing its position assuming a complete binary tree. This allows us to compute the width of each level using the formula: $\text{width} = \text{last_index} - \text{first_index} + 1$. We then track the maximum width seen throughout the traversal. The use of indices handles cases where there are null gaps between nodes.

Round 2 · High · Medium

p.796

Tree Traversal – Level Order

□ #958 Check Completeness of a Binary Tree

MED

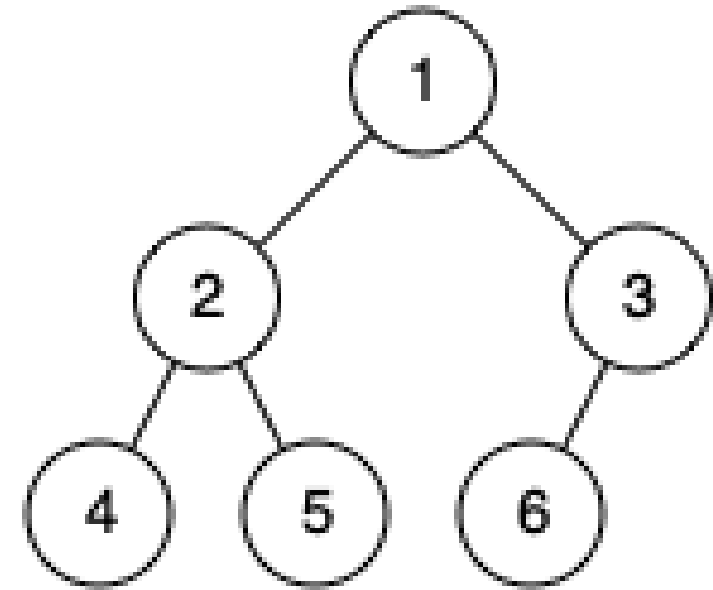
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · Breadth-First Search · Binary Tree
Lists: AlgoMaster 600

Problem

Given the root of a binary tree, determine if it is a complete binary tree.

In a complete binary tree, every level, except possibly the last, is completely filled, and all nodes in the last level are as far left as possible. It can have between 1 and 2^h nodes inclusive at the last level h .

Example 1:



Input: root = [1,2,3,4,5,6]

Output: true

Explanation: Every level before the last is full (ie. levels with node-values {1} and {2, 3}), and all nodes in the last level ({4, 5, 6}) are as far left as possible.

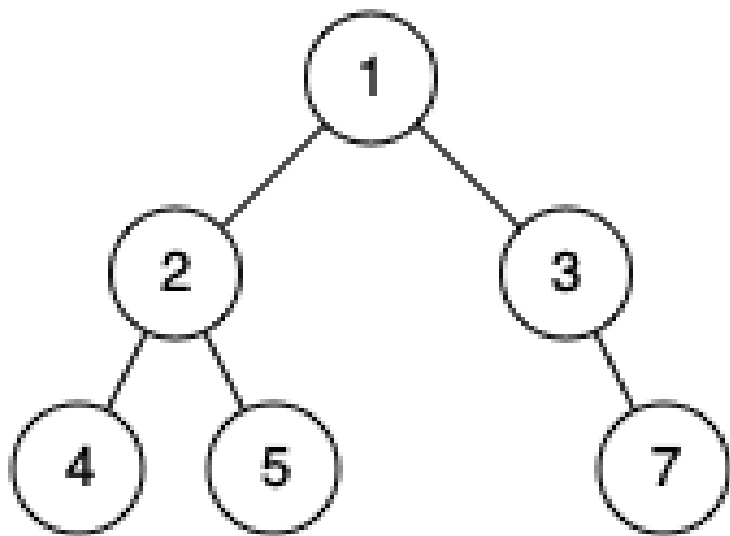
Example 2:

Round 2 · High · Medium

p.797

Round 2 · High · Medium

p.798



Input: root = [1,2,3,4,5,null,7]

Output: false

Explanation: The node with value 7 isn't as far left as possible.

Constraints:

- The number of nodes in the tree is in the range [1, 100].
- $1 \leq \text{Node.val} \leq 1000$

Community Solutions (Python)

- LeetCode

Round 2 · High · Medium

p.799

```

# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):
# self.val = val
# self.left = left
# self.right = right
class Solution:
    def isCompleteTree(self, root: TreeNode) -> bool:
        q = deque([root])
        while q:
            node = q.popleft()
            if node is None:
                break
            q.append(node.left)
            q.append(node.right)
        return all(node is None for node in q)
  
```

SimplyLeet

```

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

# Function to check completeness of binary tree
def isCompleteTree(root: TreeNode) -> bool:
    if not root:
        return True

    # Initialize queue for BFS traversal
    queue = [root]
    # Flag to mark when a missing child is seen
    end = False

    while queue:
        node = queue.pop(0)
  
```

Round 2 · High · Medium

p.800

```
# Check left child
if node.left:
    if end:
        return False # Node with left child found after a missing
child has been encountered
    queue.append(node.left)
else:
    end = True # Future nodes must be leaves

# Check right child
if node.right:
    if end:
        return False # Node with right child found after a missing
child has been encountered
    queue.append(node.right)
else:
    end = True # Future nodes must be leaves

return True
```

Round 2 · High · Medium**p.801**

Tree Traversal – Pre Order

Round 2 · High · Medium · 6 questions**Round 2 · High · Medium****p.802**

Tree Traversal – Pre Order

□ #105 Construct Binary Tree from Preorder and Inorder Traversal

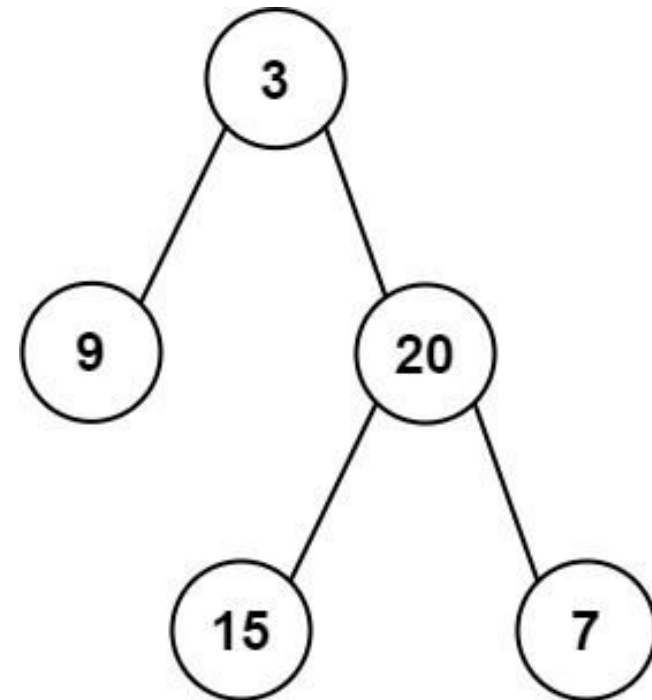
MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table · Tree · DFS
 Lists: Grind 169

Problem

Given two integer arrays preorder and inorder where preorder is the preorder traversal of a binary tree and inorder is the inorder traversal of the same tree, construct and return the binary tree.

Example 1:



Input: preorder = [3,9,20,15,7], inorder = [9,3,15,20,7]
 Output: [3,9,20,null,null,15,7]

Example 2:

Input: preorder = [-1], inorder = [-1]
 Output: [-1]

Constraints:

- $1 \leq \text{preorder.length} \leq 3000$
- $\text{inorder.length} == \text{preorder.length}$
- $-3000 \leq \text{preorder}[i], \text{inorder}[i] \leq 3000$

- preorder and inorder consist of unique values.
- Each value of inorder also appears in preorder.
- preorder is guaranteed to be the preorder traversal of the tree.
- inorder is guaranteed to be the inorder traversal of the tree.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def buildTree(
        self,
        preorder: List[int],
        inorder: List[int],
    ) -> TreeNode | None:
        inToIndex = {num: i for i, num in enumerate(inorder)}

        def build(
            preStart: int,
            preEnd: int,
            inStart: int,
            inEnd: int,
        ) -> TreeNode | None:
            if preStart > preEnd:
                return None

            rootVal = preorder[preStart]
            rootInIndex = inToIndex[rootVal]
            leftSize = rootInIndex - inStart
```

```
root = TreeNode(rootVal)
root.left = build(preStart + 1, preStart + leftSize,
                  inStart, rootInIndex - 1)
root.right = build(preStart + leftSize + 1,
                  preEnd, rootInIndex + 1, inEnd)

return root

return build(0, len(preorder) - 1, 0, len(inorder) - 1)
```

• LeetDoocs

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def buildTree(self, preorder: List[int], inorder: List[int]) -> Optional[TreeNode]:
        def dfs(i: int, j: int, n: int) -> Optional[TreeNode]:
            if n <= 0:
                return None
            v = preorder[i]
            k = d[v]
            l = dfs(i + 1, j, k - j)
            r = dfs(i + 1 + k - j, k + 1, n - k + j - 1)
            return TreeNode(v, l, r)

            d = {v: i for i, v in enumerate(inorder)}
            return dfs(0, 0, len(preorder))
```

```
class Solution:
    def getBinaryTrees(self, preOrder: List[int], inOrder: List[int]) -> List[TreeNode]:
        def dfs(i: int, j: int, n: int) -> List[TreeNode]:
            if n <= 0:
                return [None]
            v = preOrder[i]
            ans = []
            for k in d[v]:
                if j <= k < j + n:
                    for l in dfs(i + 1, j, k - j):
                        for r in dfs(i + 1 + k - j, k + 1, n - 1 - (k - j)):
                            ans.append(TreeNode(v, l, r))
            return ans

            d = defaultdict(list)
            for i, x in enumerate(inOrder):
                d[x].append(i)
            return dfs(0, 0, len(preOrder))
```

• SimplyLeet

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def buildTree(self, preorder: list[int], inorder: list[int]) -> TreeNode:
        # Create a hashmap to store value->index for inorder traversal
```

Round 2 · High · Medium

p.807

```
inorder_index = {value: idx for idx, value in enumerate(inorder)}

# Recursive helper function to build the tree
def helper(pre_start, in_left, in_right):
    # Base case: if there are no elements to construct subtree
    if in_left > in_right:
        return None

    # Pick up pre_start element as a root
    root_val = preorder[pre_start]

    root = TreeNode(root_val)

    # Get the inorder index of the root
    in_index = inorder_index[root_val]

    # Number of nodes in left subtree
    left_subtree_size = in_index - in_left

    # Recursively build the left subtree
    root.left = helper(pre_start + 1, in_left, in_index - 1)

    # Recursively build the right subtree
    root.right = helper(pre_start + left_subtree_size + 1, in_index + 1, in_right)

    return root

# Initiate recursion from the start of preorder and full inorder range
return helper(0, 0, len(inorder) - 1)
```

• LC.ca

Round 2 · High · Medium

p.808


```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

# best
class Solution:
    def buildTree(self, preorder: List[int], inorder: List[int]) -> Optional[TreeNode]:

        if not preorder: # or, not inorder
            return None
        v = preorder[0]
        i = inorder.index(v)
        root = TreeNode(v)
        root.left = self.buildTree(preorder[1: i+1], inorder[:i])
        root.right = self.buildTree(preorder[i+1:], inorder[i+1:])
        return root

# inorder.index(preorder_val)
class Solution:
    def buildTree(self, preorder: List[int], inorder: List[int]) -> Optional[TreeNode]:
        def dfs(pleft, pright, ileft,  iright):
            if pleft > pright or ileft > iright:
                return None
            k = inorder.index(preorder[pleft])
            root = TreeNode(preorder[pleft])
            root.left = dfs(pleft + 1, pleft + (k - ileft), ileft, k - 1)
            root.right = dfs(pleft + 1 + (k - ileft), pright, k + 1, iright)
```

Round 2 · High · Medium

p.809

```
        return root

        n = len(preorder)
        return dfs(0, n - 1, 0, n - 1)

# build dict for val => index
class Solution:
    def buildTree(self, preorder: List[int], inorder: List[int]) -> Optional[TreeNode]:
        def dfs(pleft, pright, ileft,  iright):

            if pleft > pright or ileft > iright:
                return None
            v = preorder[pleft]
            k = d[v]
            root = TreeNode(v)
            root.left = dfs(pleft + 1, pleft + (k - ileft), ileft, k - 1)
            root.right = dfs(pleft + 1 + (k - ileft), pright, k + 1, iright)
            return root

            d = {v: i for i, v in enumerate(inorder)}

        n = len(preorder)
        return dfs(0, n - 1, 0, n - 1)

#####

'''
search for index, also use .index(val)
>>> a = [1,2,3,4,5]
>>> a.index(3)
2
```

Round 2 · High · Medium

p.810

```

'''
class Solution(object):
    def buildTree(self, preorder, inorder):
        """
        :type preorder: List[int]
        :type inorder: List[int]
        :rtype: TreeNode
        """

        self.preindex = 0

        ind = {v: i for i, v in enumerate(inorder)}
        head = self.dc(0, len(preorder) - 1, preorder, inorder, ind)
        return head

    def dc(self, start, end, preorder, inorder, ind):
        if start <= end:
            mid = ind[preorder[self.preindex]]
            self.preindex += 1
            root = TreeNode(inorder[mid])
            root.left = self.dc(start, mid - 1, preorder, inorder, ind)

            root.right = self.dc(mid + 1, end, preorder, inorder, ind)
            return root

#####

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left

```

Round 2 · High · Medium

p.811

```

# self.right = right
class Solution:
    def buildTree(self, preorder: List[int], inorder: List[int]) -> Optional[TreeNode]:
        def dfs(i: int, j: int, n: int):
            if n <= 0:
                return None
            v = preorder[i]
            k = d[v]
            l = dfs(i + 1, j, k - j)
            r = dfs(i + 1 + k - j, k + 1, n - k + j - 1)

            return TreeNode(v, l, r)

        d = {v: i for i, v in enumerate(inorder)}
        return dfs(0, 0, len(preorder))

```

◆ Quick Review

Key Insights

- Preorder traversal always starts with the root node.
- Inorder traversal splits the tree into left subtree and right subtree based on the root.
- A hash table (map/dictionary) can be used to quickly locate the index of the root in the inorder array.
- Use recursion to construct left and right subtrees.

Space & Time

Round 2 · High · Medium

p.812

Time Complexity: $O(n)$ since we process every node exactly once.

Space Complexity: $O(n)$ due to the recursion stack and the hash map used for inorder indices.

Solution

We solve the problem using a recursive divide-and-conquer approach. First, create a hash map to store each value's index in the inorder array for quick lookups. The preorder array always provides the current root. Use the inorder array to divide the tree into left and right subtrees. Recursively construct the subtrees by adjusting the preorder index and the boundaries of the inorder array. The algorithm leverages recursion and a hash map to achieve $O(n)$ time complexity.

Tree Traversal – Pre Order

□ #106 Construct Binary Tree from Inorder and Postorder Traversal **MED**

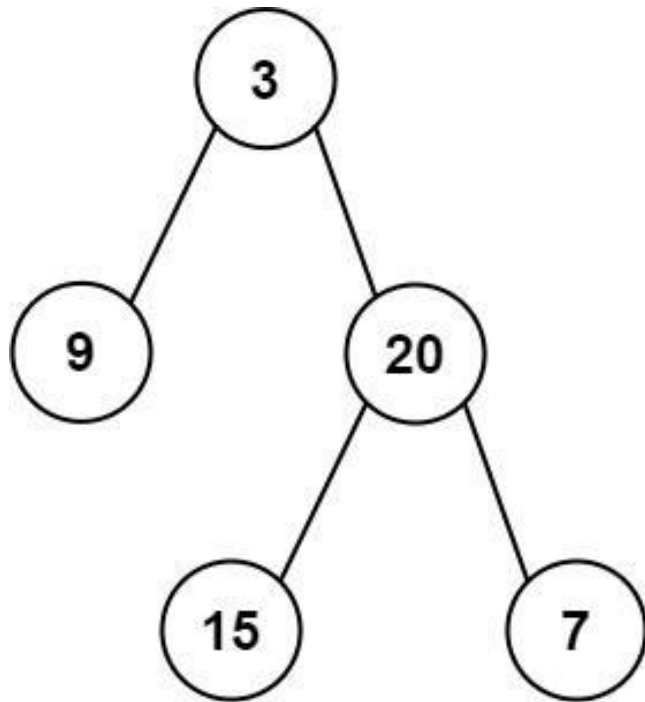
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Divide and Conquer · Tree
· Binary Tree

Lists: AlgoMaster 600

Problem

Given two integer arrays inorder and postorder where inorder is the inorder traversal of a binary tree and postorder is the postorder traversal of the same tree, construct and return the binary tree.

Example 1:



Input: inorder = [9,3,15,20,7], postorder = [9,15,7,20,3]
 Output: [3,9,20,null,null,15,7]

Example 2:

Input: inorder = [-1], postorder = [-1]
 Output: [-1]

Constraints:

- $1 \leq \text{inorder.length} \leq 3000$
- $\text{postorder.length} == \text{inorder.length}$
- $-3000 \leq \text{inorder}[i], \text{postorder}[i] \leq 3000$

- inorder and postorder consist of unique values.
- Each value of postorder also appears in inorder.
- inorder is guaranteed to be the inorder traversal of the tree.
- postorder is guaranteed to be the postorder traversal of the tree.

Community Solutions (Python)

• WalkCC

```

class Solution:
    def buildTree(
        self,
        inorder: list[int],
        postorder: list[int],
    ) -> TreeNode | None:
        inToIndex = {num: i for i, num in enumerate(inorder)}

        def build(
            inStart: int,

            inEnd: int,
            postStart: int,
            postEnd: int,
        ) -> TreeNode | None:
            if inStart > inEnd:
                return None

            rootVal = postorder[postEnd]
            rootInIndex = inToIndex[rootVal]
            leftSize = rootInIndex - inStart
  
```

```

root = TreeNode(rootVal)
root.left = build(inStart, rootInIndex - 1, postStart,
                  postStart + leftSize - 1)
root.right = build(rootInIndex + 1, inEnd, postStart + leftSize,
                  postEnd - 1)

return root

return build(0, len(inorder) - 1, 0, len(postorder) - 1)

```

• LeetCode

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def buildTree(self, inorder: List[int], postorder: List[int]) -> Optional[TreeNode]:
        def dfs(i: int, j: int, n: int) -> Optional[TreeNode]:
            if n <= 0:
                return None
            v = postorder[j + n - 1]
            k = d[v]
            l = dfs(i, j, k - i)
            r = dfs(k + 1, j + k - i, n - k + i - 1)
            return TreeNode(v, l, r)

            d = {v: i for i, v in enumerate(inorder)}
            return dfs(0, 0, len(inorder))

```

• SimplyLeet

Round 2 · High · Medium

p.817

```

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val # Node's value
        self.left = left # Left child
        self.right = right # Right child

class Solution:
    def buildTree(self, inorder: List[int], postorder: List[int]) -> TreeNode:
        # Create a dictionary to map value -> index in the inorder array for
        # O(1) lookup
        inorder_index_map = {value: idx for idx, value in enumerate(inorder)}

        # Recursive helper function that builds the tree
        def build(left_in, right_in, left_post, right_post):
            if left_in > right_in or left_post > right_post:
                return None

            # Last element in current postorder range is root
            root_val = postorder[right_post]
            root = TreeNode(root_val)

            # Find the index of the root in inorder traversal
            index = inorder_index_map[root_val]
            # Number of nodes in left subtree
            left_count = index - left_in

            # Construct left subtree using the corresponding subarrays
            root.left = build(left_in, index - 1, left_post, left_post + left_
count - 1)

            # Construct right subtree using the corresponding subarrays
            root.right = build(index + 1, right_in, left_post + left_count, ri
ght_post - 1)

```

Round 2 · High · Medium

p.818

```

    return root

    # Initiate the recursive function with proper indices
    return build(0, len(inorder) - 1, 0, len(postorder) - 1)

# Example usage:
# sol = Solution()
# tree = sol.buildTree([9,3,15,20,7], [9,15,7,20,3])

```

Tree Traversal – Pre Order

□ #113 Path Sum II

MED

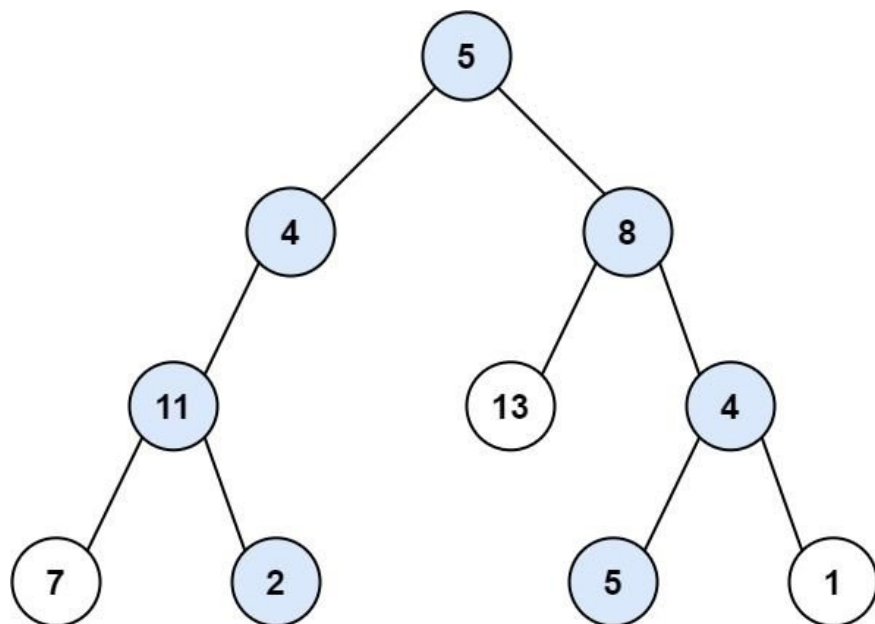
LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Backtracking · Tree · DFS
 Lists: Grind 169

Problem

Given the root of a binary tree and an integer targetSum, return all root-to-leaf paths where the sum of the node values in the path equals targetSum. Each path should be returned as a list of the node values, not node references.

A root-to-leaf path is a path starting from the root and ending at any leaf node. A leaf is a node with no children.

Example 1:



Input: root = [5,4,8,11,null,13,4,7,2,null,null,5,1], targetSum = 22

Output: [[5,4,11,2],[5,8,4,5]]

Explanation: There are two paths whose sum equals targetSum:

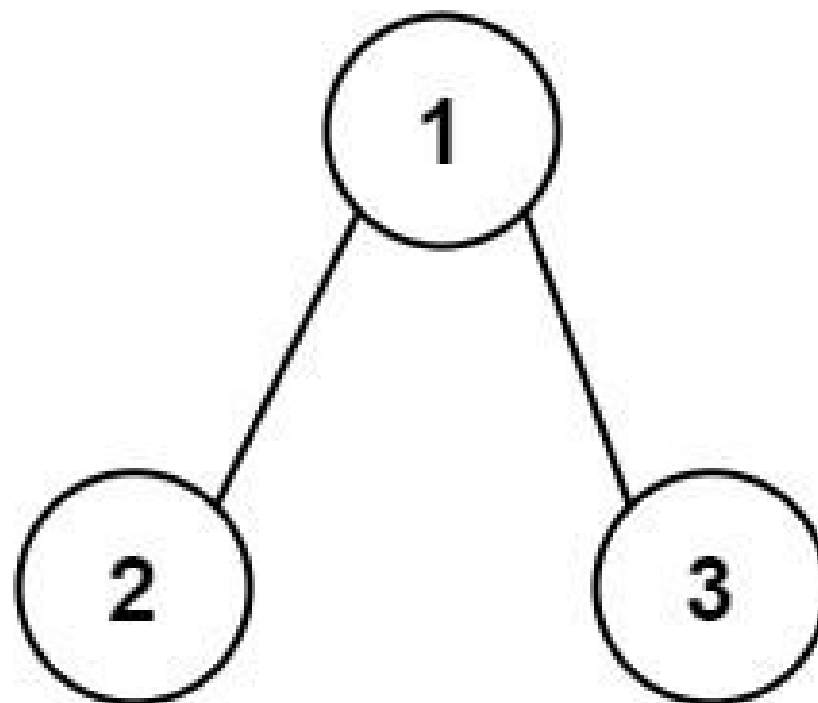
$5 + 4 + 11 + 2 = 22$

$5 + 8 + 4 + 5 = 22$

Example 2:

Round 2 · High · Medium

p.821



Input: root = [1,2,3], targetSum = 5

Output: []

Example 3:

Input: root = [1,2], targetSum = 0

Output: []

Constraints:

- The number of nodes in the tree is in the range [0, 5000].
- $-1000 \leq \text{Node.val} \leq 1000$
- $-1000 \leq \text{targetSum} \leq 1000$

Round 2 · High · Medium

p.822

Community Solutions (Python)

• WalkCC

```
class Solution:
    def pathSum(self, root: TreeNode, summ: int) -> list[list[int]]:
        ans = []

        def dfs(root: TreeNode, summ: int, path: list[int]) -> None:
            if not root:
                return
            if root.val == summ and not root.left and not root.right:
                ans.append(path + [root.val])
                return

            dfs(root.left, summ - root.val, path + [root.val])
            dfs(root.right, summ - root.val, path + [root.val])

        dfs(root, summ, [])
        return ans
```

• LeetCode

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def pathSum(self, root: Optional[TreeNode], targetSum: int) -> List[List[int]]:
        def dfs(root, s):
            if root is None:
```

```
        return
        s += root.val
        t.append(root.val)
        if root.left is None and root.right is None and s == targetSum:
            ans.append(t[:])
        dfs(root.left, s)
        dfs(root.right, s)
        t.pop()

        ans = []

        t = []
        dfs(root, 0)
        return ans
```

• SimplyLeet

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

def pathSum(root, targetSum):
    result = [] # This will store all valid paths

    def dfs(node, current_path, remaining_sum):
        if not node:
            return # Base case: reached the end of a path

        # Add the current node's value to the path
        current_path.append(node.val)

        # Check if it's a leaf and if the path sum equals targetSum
        if not node.left and not node.right and node.val == remaining_sum:
            result.append(list(current_path)) # Add a copy of the current path
```



```

else:
    # Continue the search in the left and right subtrees
    dfs(node.left, current_path, remaining_sum - node.val)
    dfs(node.right, current_path, remaining_sum - node.val)

    # Backtrack: remove the last element before returning to the caller
    current_path.pop()

dfs(root, [], targetSum)
return result

```

• LC.ca

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def pathSum(self, root: Optional[TreeNode], targetSum: int) -> List[List[int]]:
        def dfs(root, s):
            if root is None:
                return
            s += root.val
            t.append(root.val)
            if root.left is None and root.right is None and s == targetSum:
                ans.append(t[:])
            dfs(root.left, s)
            dfs(root.right, s)
            t.pop()

        ans = []

        t = []
        dfs(root, 0)
        return ans

```

◆ Quick Review

Round 2 · High · Medium

p.825

Key Insights

- Use Depth-First Search (DFS) to explore all paths from the root to the leaves.
- Backtracking is essential: add the current node value to the path before the recursive calls and remove it (backtrack) after exploring both subtrees.
- At each leaf, check if the accumulated sum equals targetSum.
- Maintain a running sum or subtract the current node's value from the target sum as you traverse.

Space & Time

Time Complexity: $O(N^2)$ in the worst-case scenario (where N is the number of nodes) when most nodes are leaf nodes, since each potential path copy operation can be $O(N)$.

Space Complexity: $O(N)$ for the recursion stack and the path storage, where N is the height of the tree.

Solution

We solve the problem using a recursive DFS approach combined with backtracking. The idea is to traverse the tree while maintaining

Round 2 · High · Medium

p.826

the current path and the remaining sum needed to reach targetSum. When a leaf node is reached (node with no children), we check if the remaining sum equals the leaf's value – if yes, we record a copy of the current path.

Data structures and algorithmic approaches used:

- Recursion for DFS: to traverse to each leaf.
- List (or Array) for storing the current path.
- Backtracking: to remove the last added element when stepping back up the recursion stack.
- Copying the path when a valid leaf is reached to store it in the final result.

A common pitfall is to forget to backtrack, which would result in incorrect paths being recorded.

Tree Traversal – Pre Order

□ #437 Path Sum III

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · Prefix Sum

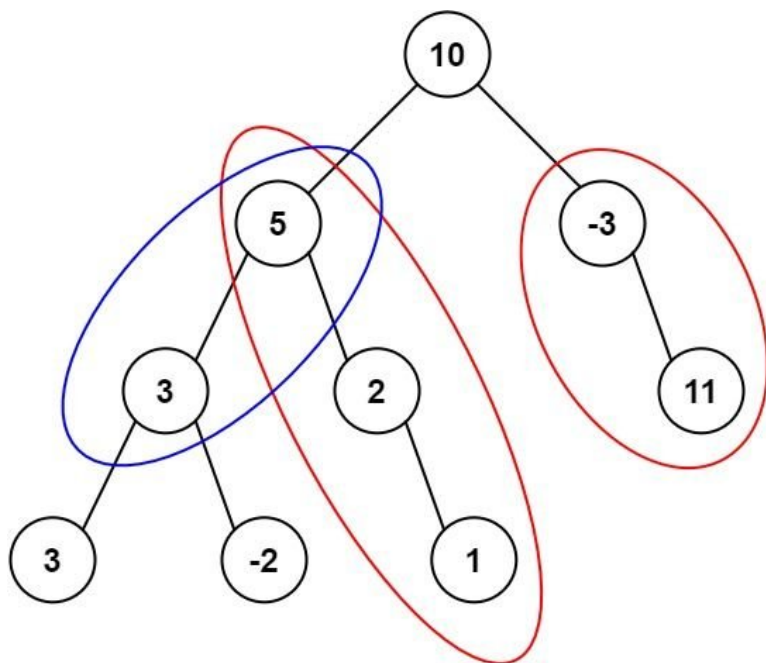
Lists: Grind 169

Problem

Given the root of a binary tree and an integer targetSum, return the number of paths where the sum of the values along the path equals targetSum.

The path does not need to start or end at the root or a leaf, but it must go downwards (i.e., traveling only from parent nodes to child nodes).

Example 1:



Input: root = [10,5,-3,3,2,null,11,3,-2,null,1], targetSum = 8
 Output: 3
 Explanation: The paths that sum to 8 are shown.

Example 2:

Input: root = [5,4,8,11,null,13,4,7,2,null,null,5,1], targetSum = 22
 Output: 3

Constraints:

- The number of nodes in the tree is in the range [0, 1000].
- $-109 \leq \text{Node.val} \leq 109$

Round 2 · High · Medium

p.829

- $-1000 \leq \text{targetSum} \leq 1000$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def pathSum(self, root: TreeNode | None, summ: int) -> int:
        if not root:
            return 0

        def dfs(root: TreeNode, summ: int) -> int:
            if not root:
                return 0
            return (int(summ == root.val) +
                    dfs(root.left, summ - root.val) +
                    dfs(root.right, summ - root.val))

        return (dfs(root, summ) +
                self.pathSum(root.left, summ) +
                self.pathSum(root.right, summ))
```

• LeetDoocs

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def pathSum(self, root: Optional[TreeNode], targetSum: int) -> int:
        def dfs(node, s):
            if node is None:
```

Round 2 · High · Medium

p.830

```

        return 0
    s += node.val
    ans = cnt[s - targetSum]
    cnt[s] += 1
    ans += dfs(node.left, s)
    ans += dfs(node.right, s)
    cnt[s] -= 1
    return ans

```

```
cnt = Counter({0: 1})
```

```
return dfs(root, 0)
```

• SimplyLeet

Python solution for Path Sum III

```
class TreeNode:
```

```

    def __init__(self, val=0, left=None, right=None):
        self.val = val          # value of the node
        self.left = left        # left child
        self.right = right      # right child

```

```
class Solution:
```

```

    def pathSum(self, root: TreeNode, targetSum: int) -> int:
        # Helper function to perform DFS traversal

```

```

    def dfs(node, current_sum, prefix_sums):
        if not node:
            return 0

```

```

        # Update current sum by including the current node's value
        current_sum += node.val
        # Check the number of times (current_sum - targetSum) has occurred
        num_paths = prefix_sums.get(current_sum - targetSum, 0)

```

```
        # Update the prefix sum hash map with the current sum
```

```

        prefix_sums[current_sum] = prefix_sums.get(current_sum, 0) + 1

        # Recursively search in the left and right subtree
        num_paths += dfs(node.left, current_sum, prefix_sums)
        num_paths += dfs(node.right, current_sum, prefix_sums)

        # Backtrack: remove the current sum count before returning to the
        parent node
        prefix_sums[current_sum] -= 1
        return num_paths

    # Initial prefix sum hashmap with 0 sum seen once (empty path)
    prefix_sums = {0: 1}
    return dfs(root, 0, prefix_sums)

```

• LC.ca

Definition for a binary tree node.

```
# class TreeNode:
```

```

    # def __init__(self, val=0, left=None, right=None):
    #     self.val = val
    #     self.left = left
    #     self.right = right

```

```
# self.val = val
```

```
# self.left = left
```

```
# self.right = right
```

```
class Solution:
```

```

    def pathSum(self, root: Optional[TreeNode], targetSum: int) -> int:
        def dfs(node, s):
            if node is None:

```

```

                return 0
            s += node.val
            ans = cnt[s - targetSum]
            cnt[s] += 1
            ans += dfs(node.left, s)
            ans += dfs(node.right, s)
            cnt[s] -= 1
            return ans

```

```
cnt = Counter({0: 1})
```

```
return dfs(root, 0)
```

◆ Quick Review

Key Insights

- A path can start and end at any nodes, as long as it goes downwards.
- Using DFS helps traverse the tree and explore all possible paths.
- A prefix sum technique with a hash map allows quick calculation of the sum of any subpath.
- Backtracking is used to remove a path's prefix sum when moving back up the recursion tree.
- Although a brute-force approach exists, it is inefficient compared to the prefix sum method.

Space & Time

Time Complexity: $O(n)$ on average (where n is the number of nodes), though worst-case can be $O(n^2)$ in a skewed tree.

Space Complexity: $O(n)$ due to recursion stack and the hash map that stores prefix sums.

Solution

The solution employs a DFS traversal of the tree along with a hash map that records the

frequency of prefix sums encountered so far. At each node, the current prefix sum is updated by adding the node's value. We then check the hash map for how many times (current prefix sum – targetSum) has occurred, since this value indicates the existence of a subpath summing to targetSum. We update the hash map with the current prefix sum, continue the DFS to traverse child nodes, and then backtrack by decrementing the current prefix sum's count before retreating to the parent node. This ensures that sums from other branches are not incorrectly combined.

Tree Traversal – Pre Order

#687 Longest Univalue Path

MED

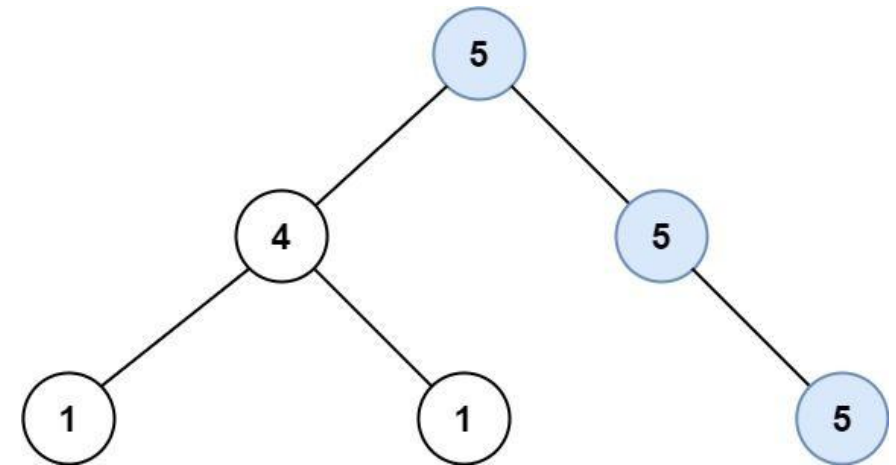
LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Tree · Depth-First Search · Binary Tree
 Lists: AlgoMaster 600

Problem

Given the root of a binary tree, return the length of the longest path, where each node in the path has the same value. This path may or may not pass through the root.

The length of the path between two nodes is represented by the number of edges between them.

Example 1:

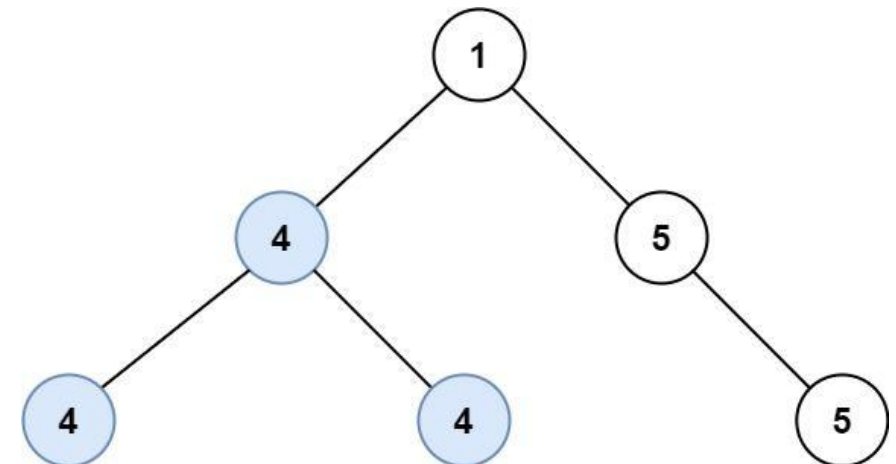


Input: root = [5,4,5,1,1,null,5]

Output: 2

Explanation: The shown image shows that the longest path of the same value (i.e. 5).

Example 2:



Round 2 · High · Medium

p.835

Round 2 · High · Medium

p.836

Input: root = [1,4,5,4,4,null,5]

Output: 2

Explanation: The shown image shows that the longest path of the same value (i.e. 4).

Constraints:

- The number of nodes in the tree is in the range [0, 104].
- $-1000 \leq \text{Node.val} \leq 1000$
- The depth of the tree will not exceed 1000.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def longestUnivaluePath(self, root: TreeNode | None) -> int:
        ans = 0

        def longestUnivaluePathDownFrom(root: TreeNode | None) -> int:
            nonlocal ans
            if not root:
                return 0

            l = longestUnivaluePathDownFrom(root.left)

            r = longestUnivaluePathDownFrom(root.right)
            arrowLeft = l + 1 if root.left and root.left.val == root.val else 0
            arrowRight = r + 1 if root.right and root.right.val == root.val else 0
            ans = max(ans, arrowLeft + arrowRight)
            return max(arrowLeft, arrowRight)

        longestUnivaluePathDownFrom(root)
        return ans
```

• LeetCode

Round 2 · High · Medium

p.837

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def longestUnivaluePath(self, root: Optional[TreeNode]) -> int:
        def dfs(root: Optional[TreeNode]) -> int:
            if root is None:
                return 0
            l, r = dfs(root.left), dfs(root.right)
            l = l + 1 if root.left and root.left.val == root.val else 0
            r = r + 1 if root.right and root.right.val == root.val else 0
            nonlocal ans
            ans = max(ans, l + r)
            return max(l, r)

        ans = 0
        dfs(root)

        return ans
```

• SimplyLeet

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val # The value of the node
        self.left = left # Pointer to the left child
        self.right = right # Pointer to the right child

class Solution:
    def longestUnivaluePath(self, root: TreeNode) -> int:
        self.max_path = 0 # Global variable to track the longest univalue path
```

Round 2 · High · Medium

p.838

```

def dfs(node: TreeNode) -> int:
    if not node:
        return 0 # Base case: no node means 0 path length
    # Recursively find the longest univalue paths in the left and right
    subtrees
    left_length = dfs(node.left)
    right_length = dfs(node.right)

    left_path = right_path = 0 # Initialize paths from left and right
    # If left child exists and value matches, extend the left path

    if node.left and node.left.val == node.val:
        left_path = left_length + 1
    # If right child exists and value matches, extend the right path
    if node.right and node.right.val == node.val:
        right_path = right_length + 1

    # Update the global maximum path: path may use both left and right
    extensions
    self.max_path = max(self.max_path, left_path + right_path)
    # Return the longer of one side which could be extended to parent's
    path
    return max(left_path, right_path)

dfs(root) # Start DFS from the root
return self.max_path # Return the longest univalue path

```

Tree Traversal – Pre Order

#1026 Maximum Difference Between Node and Ancestor

MED

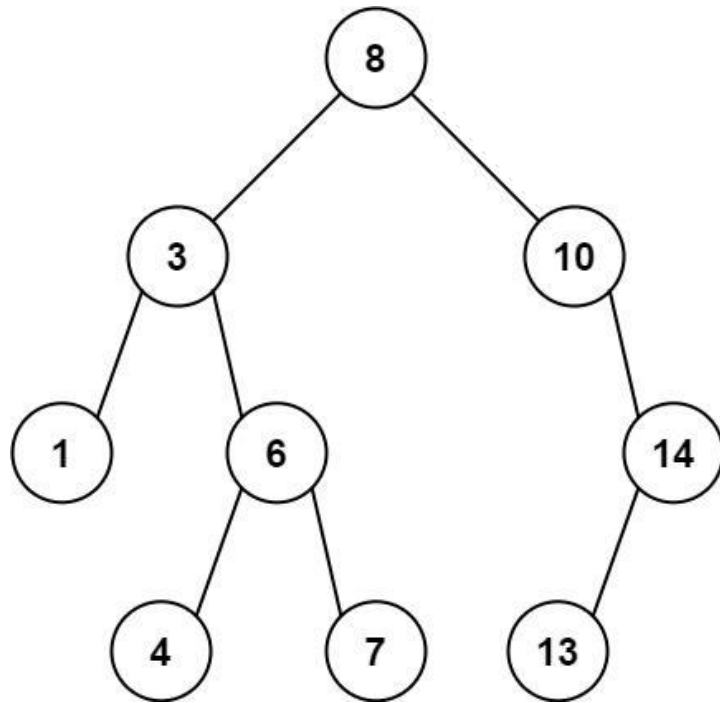
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · Depth-First Search · Binary Tree
Lists: AlgoMaster 600

Problem

Given the root of a binary tree, find the maximum value v for which there exist different nodes a and b where $v = |a.val - b.val|$ and a is an ancestor of b .

A node a is an ancestor of b if either: any child of a is equal to b or any child of a is an ancestor of b .

Example 1:



Input: root = [8,3,10,1,6,null,14,null,null,4,7,13]

Output: 7

Explanation: We have various ancestor-node differences, some of which are given below :

$$|8 - 3| = 5$$

$$|3 - 7| = 4$$

$$|8 - 1| = 7$$

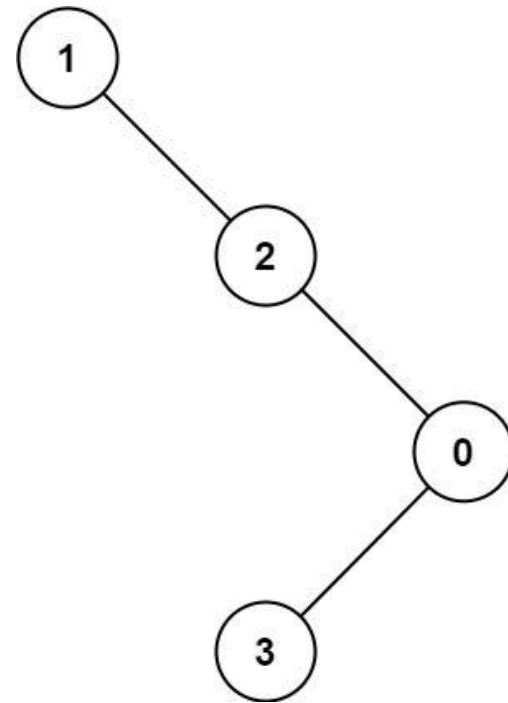
$$|10 - 13| = 3$$

Among all possible differences, the maximum value of 7 is obtained by $|8 - 1| = 7$.

Example 2:

Round 2 · High · Medium

p.841



Input: root = [1,null,2,null,0,3]

Output: 3

Constraints:

- The number of nodes in the tree is in the range [2, 5000].
- $0 \leq \text{Node.val} \leq 105$

Community Solutions (Python)

- [LeetDoocs](#)

Round 2 · High · Medium

p.842

```
# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):
# self.val = val
# self.left = left
# self.right = right
class Solution:
    def maxAncestorDiff(self, root: Optional[TreeNode]) -> int:
        def dfs(root: Optional[TreeNode], mi: int, mx: int):
            if root is None:
                return
            nonlocal ans
            ans = max(ans, abs(mi - root.val), abs(mx - root.val))
            mi = min(mi, root.val)
            mx = max(mx, root.val)
            dfs(root.left, mi, mx)
            dfs(root.right, mi, mx)

        ans = 0
        dfs(root, root.val, root.val)

        return ans
```

• SimplyLeet

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def maxAncestorDiff(self, root: TreeNode) -> int:
        # Helper function for DFS traversal with current min and max values.
```

Round 2 · High · Medium

p.843

```
def dfs(node, cur_min, cur_max):
    if not node:
        # At a null node, return the difference encountered so far.
        return cur_max - cur_min
    # Update the current path's min and max values.
    cur_min = min(cur_min, node.val)
    cur_max = max(cur_max, node.val)
    # Recursively get the maximum difference in both subtrees.
    left_diff = dfs(node.left, cur_min, cur_max)
    right_diff = dfs(node.right, cur_min, cur_max)

    # Return the maximum difference found.
    return max(left_diff, right_diff)

# Start DFS with the root value as both current min and max.
return dfs(root, root.val, root.val)
```

Round 2 · High · Medium

p.844

Tree Traversal – In Order

Round 2 · High · Medium · 4 questions

Round 2 · High · Medium

p.845

Tree Traversal – In Order

□ #98 Validate Binary Search Tree

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · BST

Lists: Grind 169

Problem

Given the root of a binary tree, determine if it is a valid binary search tree (BST).

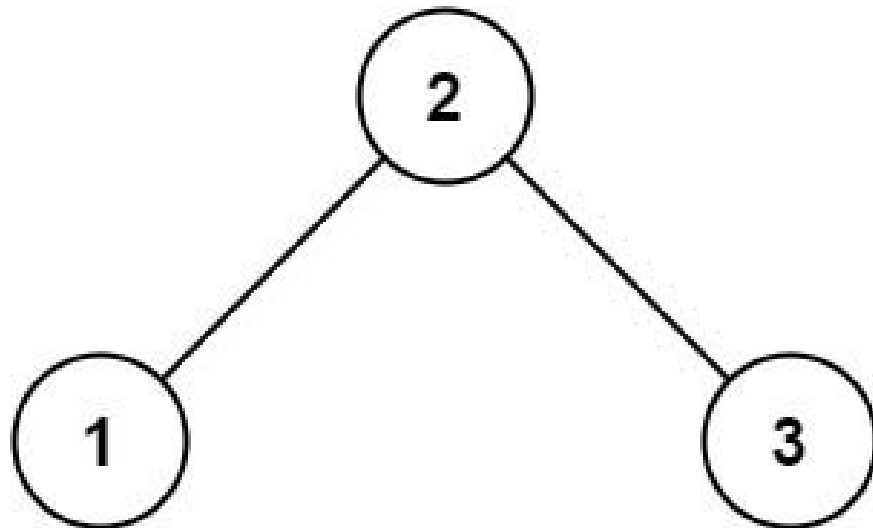
A valid BST is defined as follows:

- The left subtree of a node contains only nodes with keys strictly less than the node's key.
- The right subtree of a node contains only nodes with keys strictly greater than the node's key.
- Both the left and right subtrees must also be binary search trees.

Example 1:

Round 2 · High · Medium

p.846

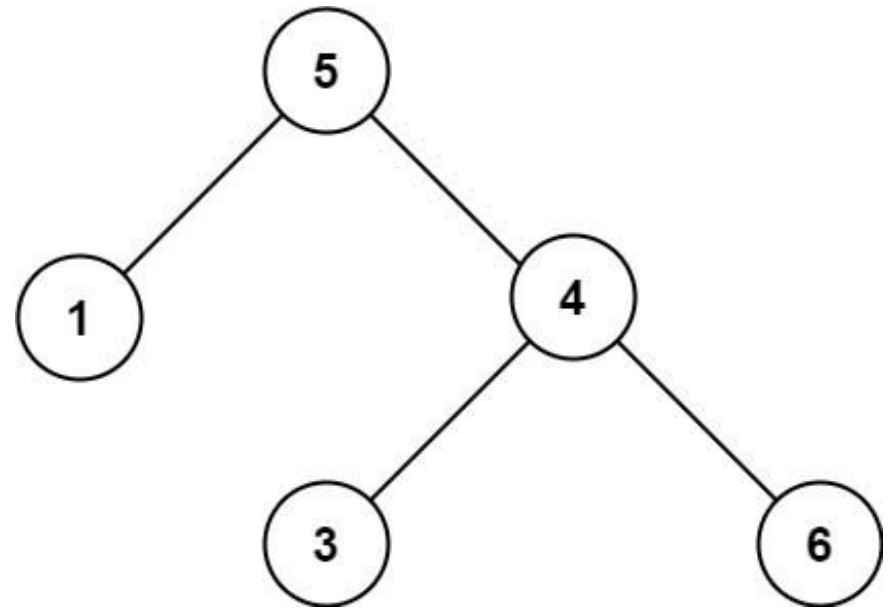


Input: root = [2,1,3]
Output: true

Example 2:

Round 2 · High · Medium

p.847



Input: root = [5,1,4,null,null,3,6]
Output: false
Explanation: The root node's value is 5 but its right child's value is 4.

Constraints:

- The number of nodes in the tree is in the range [1, 104].
- $-231 \leq \text{Node.val} \leq 231 - 1$

Community Solutions (Python)

- WalkCC

Round 2 · High · Medium

p.848

```
class Solution:
    def isValidBST(self, root: TreeNode | None) -> bool:
        def isValidBST(root: TreeNode | None,
                        minNode: TreeNode | None, maxNode: TreeNode | None) -> bool:
            :
            if not root:
                return True
            if minNode and root.val <= minNode.val:
                return False
            if maxNode and root.val >= maxNode.val:
                return False

            return (isValidBST(root.left, minNode, root) and
                    isValidBST(root.right, root, maxNode))

        return isValidBST(root, None, None)

class Solution:
    def isValidBST(self, root: TreeNode | None) -> bool:
        stack = []
        pred = None

        while root or stack:
            while root:
                stack.append(root)
                root = root.left
            root = stack.pop()

            if pred and pred.val >= root.val:
                return False
            pred = root
            root = root.right

        return True
```

• LeetCode

Round 2 · High · Medium

p.849

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isValidBST(self, root: Optional[TreeNode]) -> bool:
        def dfs(root: Optional[TreeNode]) -> bool:
            if root is None:
                return True
            if not dfs(root.left):
                return False
            nonlocal prev
            if prev >= root.val:
                return False
            prev = root.val
            return dfs(root.right)

        prev = -inf

        return dfs(root)
```

• SimplyLeet

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val # Node's value
        self.left = left # Left child
        self.right = right # Right child

class Solution:
    def isValidBST(self, root: TreeNode) -> bool:
        # Helper function to recursively validate nodes
```

Round 2 · High · Medium

p.850

```
def validate(node, low, high):
    if not node:
        return True # An empty node/subtree is valid
    # If current node's value is out of the allowed range, return False
    if not (low < node.val < high):
        return False
    # Recursively check the left subtree and right subtree
    return validate(node.left, low, node.val) and validate(node.right,
node.val, high)

return validate(root, float("-inf"), float("inf"))
```

• LC.ca

```
# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):
# self.val = val
# self.left = left
# self.right = right
# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):
# self.val = val

# self.left = left
# self.right = right

class Solution: # recursive
    def isValidBST(self, root: Optional[TreeNode]) -> bool:
        def dfs(root, mi, ma):
            if not root:
                return True
            return mi < root.val < ma and dfs(root.left, mi, root.val) and dfs
(root.right, root.val, ma)
```

```
return dfs(root, -math.inf, math.inf)
```

```
class Solution: # iterative
    def isValidBST(self, root: TreeNode) -> bool:
        # store (node, lower, upper) tuples in a single queue
        queue = [(root, None, None)]

        # another option:
        # queue = [(root, -math.inf, math.inf)]
```

```
while queue:
    node, lower, upper = queue.pop(0)
```

```
if node is None:
    continue
```

```
val = node.val
if lower is not None and val <= lower:
    return False
```

```
if upper is not None and val >= upper:
    return False
```

```
queue.append((node.right, val, upper))
queue.append((node.left, lower, val))
```

```
return True
```

```
#####
```

```

class Solution:
    def isValidBST(self, root: Optional[TreeNode]) -> bool:
        def isValidBST(root: Optional[TreeNode],
                        minNode: Optional[TreeNode], maxNode: Optional[TreeNode]) -
> bool:
            if not root:
                return True
            if minNode and root.val <= minNode.val:
                return False
            if maxNode and root.val >= maxNode.val:
                return False

            return isValidBST(root.left, minNode, root) and isValidBST(root.right, r
oot, maxNode)

        return isValidBST(root, None, None)

#####

# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):

# self.val = val
# self.left = left
# self.right = right
class Solution:
    def isValidBST(self, root: TreeNode) -> bool:
        def dfs(root):
            nonlocal prev
            if root is None:
                return True
            if not dfs(root.left):

```

Round 2 · High · Medium

p.853

```

        return False
    if prev >= root.val:
        return False
    prev = root.val
    if not dfs(root.right):
        return False
    return True

```

```

prev = -inf
return dfs(root)

```

#####

```

# Definition for a binary tree node.
# class TreeNode(object):
# def __init__(self, x):
# self.val = x
# self.left = None
# self.right = None

```

```

class Solution(object):
    def isValidBST(self, root):
        """
        :type root: TreeNode
        :rtype: bool
        """
        prev = -float("inf")
        stack = [(1, root)]
        while stack:
            p = stack.pop()

            if not p[1]:
                continue
            if p[0] == 0:
                if p[1].val <= prev:
                    return False
                prev = p[1].val
            else:
                stack.append((1, p[1].right))
                stack.append((0, p[1]))
                stack.append((1, p[1].left))

```

Round 2 · High · Medium

p.854

```
return True
```

◆ Quick Review

Key Insights

- Every node must adhere to a range: all nodes in the left subtree should be less than the current node, and all nodes in the right subtree should be greater.
- A recursive depth-first search approach works well by passing down the valid range (lower and upper bounds) for each node.
- Using the limits (negative and positive infinity) simplifies handling edge cases at the tree root.
- An in-order traversal would alternatively yield a sorted order for node values, but updating ranges directly is more intuitive for recursive implementation.

Space & Time

Time Complexity: $O(N)$, where N is the number of nodes, as each node is visited once.

Space Complexity: $O(H)$, where H is the height of the tree, due to the recursion stack.

Round 2 · High · Medium

p.855

Solution

We use a recursive approach to validate the BST property. We maintain two variables, low and high, representing the valid range for the current node's value. For the left child, the valid range becomes (low, node.val) and for the right child, it becomes (node.val, high). By checking if each node's value falls within its respective range, we can ensure that the tree satisfies the BST properties.

Round 2 · High · Medium

p.856

Tree Traversal – In Order

□ #173 Binary Search Tree Iterator

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Stack · Tree · Design · Binary Search Tree ·
Iterator

Lists: Denny Zhang

Problem

Implement the BSTIterator class that represents an iterator over the in-order traversal of a binary search tree (BST):

- **BSTIterator(TreeNode root)** Initializes an object of the BSTIterator class. The root of the BST is given as part of the constructor. The pointer should be initialized to a non-existent number smaller than any element in the BST.
- **boolean hasNext()** Returns true if there exists a number in the traversal to the right of the pointer, otherwise returns false.
- **int next()** Moves the pointer to the right, then returns the number at the pointer.

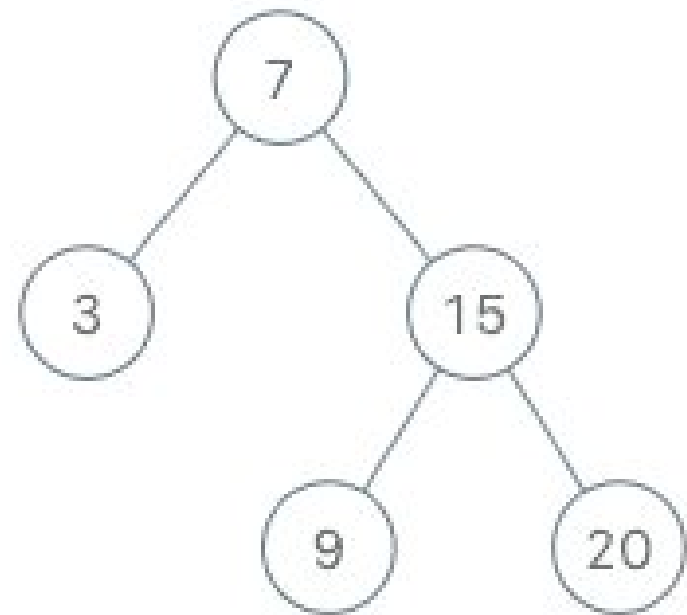
Round 2 · High · Medium

p.857

Notice that by initializing the pointer to a non-existent smallest number, the first call to next() will return the smallest element in the BST.

You may assume that next() calls will always be valid. That is, there will be at least a next number in the in-order traversal when next() is called.

Example 1:



Round 2 · High · Medium

p.858

Input
 ["BSTIterator", "next", "next", "hasNext", "next", "hasNext", "next",
 "hasNext", "next", "hasNext"]
 [[7, 3, 15, null, null, 9, 20], [], [], [], [], [], [], [], []]
 Output
 [null, 3, 7, true, 9, true, 15, true, 20, false]

Explanation
 BSTIterator bSTIterator = new BSTIterator([7, 3, 15, null, null, 9, 20]);

Constraints:

- The number of nodes in the tree is in the range [1, 105].
- $0 \leq \text{Node.val} \leq 106$
- At most 105 calls will be made to hasNext, and next.

Follow up:

- Could you implement next() and hasNext() to run in average $O(1)$ time and use $O(h)$ memory, where h is the height of the tree?

Community Solutions (Python)

• WalkCC

```
class BSTIterator:
    def __init__(self, root: TreeNode | None):
        self.i = 0
        self.vals = []
        self._inorder(root)

    def next(self) -> int:
        self.i += 1
        return self.vals[self.i - 1]
```

Round 2 · High · Medium

p.859

```
def hasNext(self) -> bool:
    return self.i < len(self.vals)

def _inorder(self, root: TreeNode | None) -> None:
    if not root:
        return
    self._inorder(root.left)
    self.vals.append(root.val)
    self._inorder(root.right)

class BSTIterator:
    def __init__(self, root: TreeNode | None):
        self.stack = []
        self._pushLeftsUntilNull(root)

    def next(self) -> int:
        root = self.stack.pop()
        self._pushLeftsUntilNull(root.right)
        return root.val

    def hasNext(self) -> bool:
        return self.stack

    def _pushLeftsUntilNull(self, root: TreeNode | None) -> None:
        while root:
            self.stack.append(root)
            root = root.left
```

• LeetDoocs

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class BSTIterator:
    def __init__(self, root: TreeNode):
        def inorder(root):
            if root:
```

Round 2 · High · Medium

p.860

```

        inorder(root.left)
        self.vals.append(root.val)
        inorder(root.right)

```

```

self.cur = 0
self.vals = []
inorder(root)

```

```

def next(self) -> int:
    res = self.vals[self.cur]

```

```

self.cur += 1
return res

```

```

def hasNext(self) -> bool:
    return self.cur < len(self.vals)

```

```

# Your BSTIterator object will be instantiated and called as such:
# obj = BSTIterator(root)
# param_1 = obj.next()

```

```

# param_2 = obj.hasNext()

```

• SimplyLeet

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class BSTIterator:
    def __init__(self, root):
        # Initialize an empty stack to simulate in-order traversal

```

Round 2 · High · Medium

p.861

```

self.stack = []
# Push all left nodes starting from the root
self._push_left_branch(root)

```

```

def _push_left_branch(self, node):
    # Push node and all its left children to the stack
    while node:
        self.stack.append(node)
        node = node.left

```

```

def next(self):
    # Pop the top node which represents the next smallest element
    node = self.stack.pop()
    # If the node has a right child, push its left branch onto the stack
    if node.right:
        self._push_left_branch(node.right)
    # Return the node's value
    return node.val

```

```

def hasNext(self):

```

```

# The iterator has a next element if the stack is not empty
return len(self.stack) > 0

```

• LC.ca

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class BSTIterator:
    def __init__(self, root: TreeNode):
        self.stack = []
        self._leftmost_inorder(root)

```

Round 2 · High · Medium

p.862

```
def _leftmost_inorder(self, node):
    while node:
        self.stack.append(node)
        node = node.left

def next(self) -> int:
    cur = self.stack.pop()
    node = cur.right
    self._leftmost_inorder(node)

    return cur.val

def hasNext(self) -> bool:
    return len(self.stack) > 0

# Your BSTIterator object will be instantiated and called as such:
# obj = BSTIterator(root)
# param_1 = obj.next()
# param_2 = obj.hasNext()
```

#####

```
# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):
# self.val = val
# self.left = left
# self.right = right
class BSTIterator:
    def __init__(self, root: TreeNode):
```

```
        self.stack = []
        while root:
            self.stack.append(root)
            root = root.left

    def next(self) -> int:
        cur = self.stack.pop()
        node = cur.right
        while node: # duplicated while block
            self.stack.append(node)
```

Round 2 · High · Medium

p.863

```
        node = node.left
        return cur.val

def hasNext(self) -> bool:
    return len(self.stack) > 0

# Your BSTIterator object will be instantiated and called as such:
# obj = BSTIterator(root)
# param_1 = obj.next()

# param_2 = obj.hasNext()
```

◆ Quick Review

Key Insights

- Use an iterative in-order traversal strategy with a stack.
- In-order traversal for BST yields sorted order.
- Pre-load the stack with all left descendants from the root.
- After returning a node, process its right subtree by pushing its left branch.
- This method maintains an average time complexity of $O(1)$ per operation and uses $O(h)$ memory, where h is the tree height.

Space & Time

Round 2 · High · Medium

p.864

Time Complexity: $O(1)$ on average per operation (amortized)

Space Complexity: $O(h)$, where h is the height of the tree

Solution

We use a stack to simulate an in-order traversal. The process involves initially pushing all left-child nodes from the root into the stack. For each call to `next()`, we pop the top element (the current smallest node) and then push all the left descendents of the node's right child. This ensures that the next smallest element is always at the top of the stack. The `hasNext()` function then simply checks if the stack is non-empty.

Round 2 · High · Medium

p.865

Tree Traversal – In Order

□ #230 Kth Smallest Element in a BST

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · BST

Lists: Grind 169

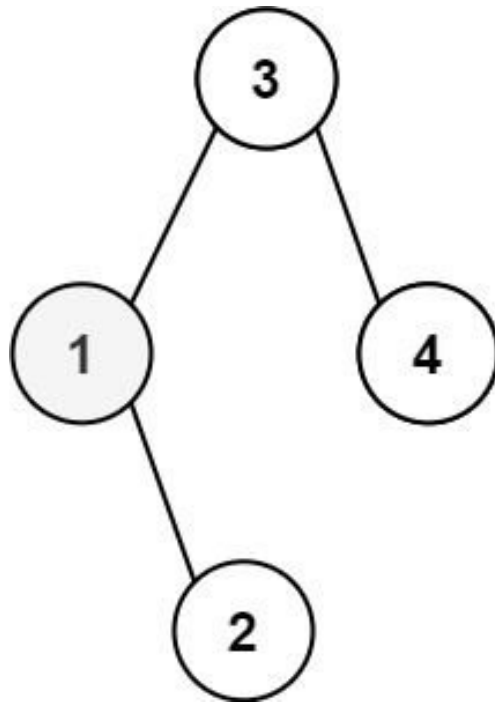
Problem

Given the root of a binary search tree, and an integer k , return the k th smallest value (1-indexed) of all the values of the nodes in the tree.

Example 1:

Round 2 · High · Medium

p.866

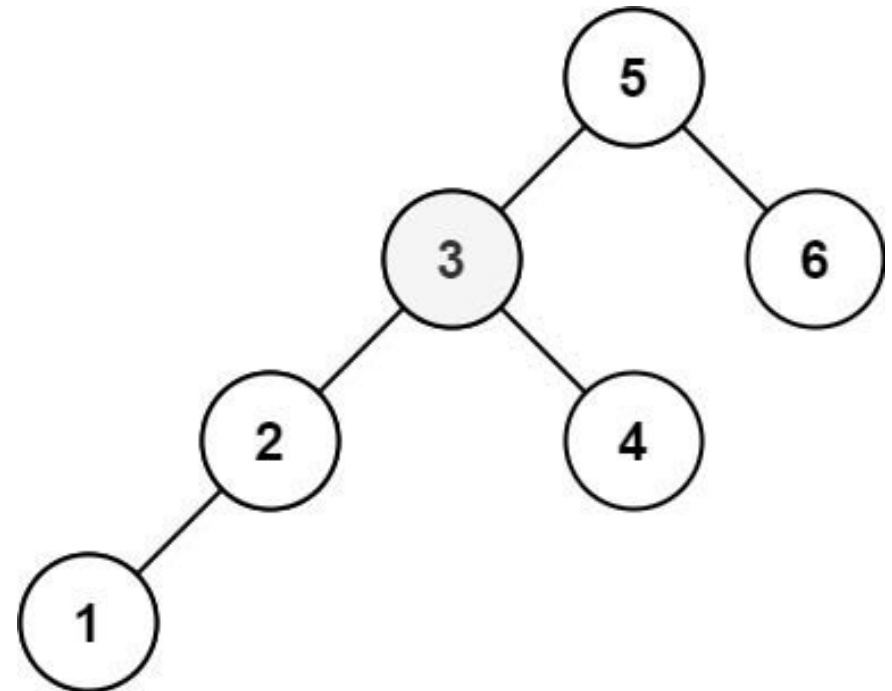


Input: root = [3,1,4,null,2], k = 1
Output: 1

Example 2:

Round 2 · High · Medium

p.867



Input: root = [5,3,6,2,4,null,null,1], k = 3
Output: 3

Constraints:

- The number of nodes in the tree is n.
- $1 \leq k \leq n \leq 104$
- $0 \leq \text{Node.val} \leq 104$

Follow up: If the BST is modified often (i.e., we can do insert and delete operations) and you need to find the kth smallest frequently, how would you optimize?

Round 2 · High · Medium

p.868

Community Solutions (Python)

• WalkCC

```
class Solution:
    def kthSmallest(self, root: TreeNode | None, k: int) -> int:
        def countNodes(root: TreeNode | None) -> int:
            if not root:
                return 0
            return 1 + countNodes(root.left) + countNodes(root.right)

        leftCount = countNodes(root.left)

        if leftCount == k - 1:

            return root.val
        if leftCount >= k:
            return self.kthSmallest(root.left, k)
        return self.kthSmallest(root.right, k - 1 - leftCount) # leftCount < k

class Solution:
    def kthSmallest(self, root: TreeNode | None, k: int) -> int:
        rank = 0
        ans = 0

        def traverse(root: TreeNode | None) -> None:
            nonlocal rank
            nonlocal ans
            if not root:
                return
            traverse(root.left)
            rank += 1
            if rank == k:
                ans = root.val
                return
            traverse(root.right)

        traverse(root)
        return ans
```

Round 2 · High · Medium

p.869

```
class Solution:
    def kthSmallest(self, root: TreeNode | None, k: int) -> int:
        stack = []

        while root:
            stack.append(root)
            root = root.left

        for _ in range(k - 1):
            root = stack.pop()

            root = root.right
            while root:
                stack.append(root)
                root = root.left

        return stack[-1].val
```

• LeetDoocs

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def kthSmallest(self, root: Optional[TreeNode], k: int) -> int:
        stk = []
        while root or stk:

            if root:
                stk.append(root)
                root = root.left
            else:
                root = stk.pop()
                k -= 1
                if k == 0:
                    return root.val
                root = root.right
```

• SimplyLeet

Round 2 · High · Medium

p.870

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
```

```
class Solution:
    def kthSmallest(self, root: TreeNode, k: int) -> int:
        # Initialize counter and result
```

```
self.k = k
self.ans = None
```

```
# In-order traversal: left -> node -> right
def inorder(node):
    if not node:
        return
    # Traverse left subtree
    inorder(node.left)
    # Process current node
```

```
self.k -= 1
if self.k == 0:
    self.ans = node.val
    return
# Traverse right subtree if kth element not found
inorder(node.right)
```

```
inorder(root)
return self.ans
```

• LC.ca

Round 2 · High · Medium

p.871

```
# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):
# self.val = val
# self.left = left
# self.right = right
class Solution:
    def kthSmallest(self, root: Optional[TreeNode], k: int) -> int:
        stk = []
        while root or stk:
```

```
if root:
    stk.append(root)
    root = root.left
else:
    root = stk.pop()
    k -= 1
    if k == 0:
        return root.val
    root = root.right
```

```
#####
```

```
# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):
# self.val = val
# self.left = left
# self.right = right
class Solution_followUp:
    def kthSmallest(self, root: TreeNode, k: int) -> int:
```

```
node = self.build(root)
return self.dfs(node, k)
```

```
class MyTreeNode:
    def __init__(self, x):
        self.val = x
        self.count = 1 # default 1
        self.left = None
        self.right = None
```

Round 2 · High · Medium

p.872


```
def build(self, root: TreeNode) -> MyTreeNode:
    if not root:
        return None
    node = self.MyTreeNode(root.val) # default count already is 1
    node.left = self.build(root.left)
    node.right = self.build(root.right)
    if node.left:
        node.count += node.left.count
    if node.right:
        node.count += node.right.count

    return node

def dfs(self, node: MyTreeNode, k: int) -> int:
    if node.left:
        cnt = node.left.count
        if k < cnt + 1:
            return self.dfs(node.left, k)
        elif k > cnt + 1:
            return self.dfs(node.right, k - 1 - cnt)
        else: # k == cnt+1
            return node.val
    else:
        if k == 1: # cannot move to beginning of dfs()
            return node.val
        return self.dfs(node.right, k - 1)
```

◆ Quick Review

Key Insights

- An in-order traversal of a BST visits nodes in increasing order.
- Use a counter to keep track of the number of nodes visited.

- As soon as the counter reaches k, the current node's value is the kth smallest.
- For frequent modifications (insertions/deletions) and queries, consider augmenting the BST with subtree counts.

Space & Time

Time Complexity: $O(n)$ in the worst-case scenario (when $k \approx n$), but on average the traversal may stop early.

Space Complexity: $O(h)$ where h is the height of the tree ($O(n)$ in the worst-case for a skewed tree).

Solution

We perform an in-order traversal of the BST which visits nodes in ascending order. During the traversal, we maintain a counter (or decrement k) to check when we have encountered the kth smallest element. When the counter reaches 0 (or k becomes 0), we capture the current node's value as our answer and return it. This approach is both simple and efficient for the given constraints.

Tree Traversal – In Order

#1382 Balance a Binary Search Tree

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Divide and Conquer · Greedy · Tree ·
Depth-First Search · Binary Search Tree · Binary
Tree

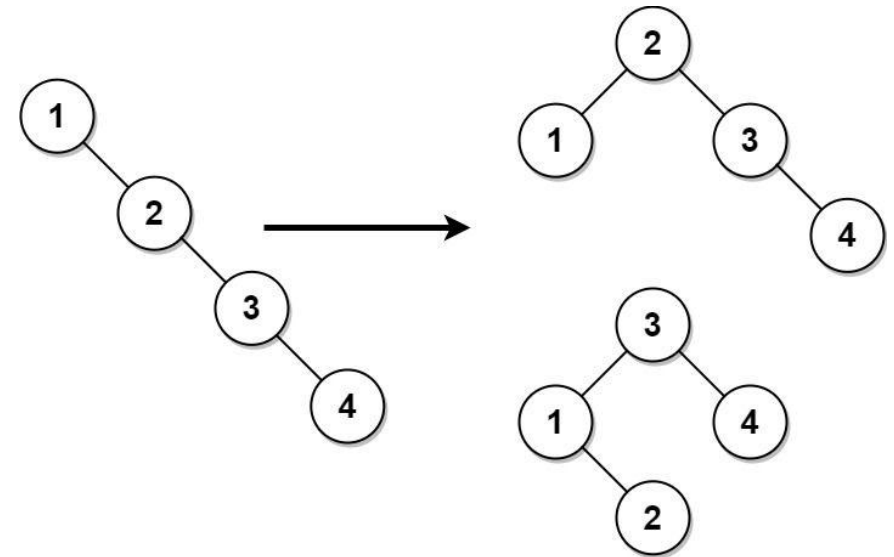
Lists: AlgoMaster 600

Problem

Given the root of a binary search tree, return a balanced binary search tree with the same node values. If there is more than one answer, return any of them.

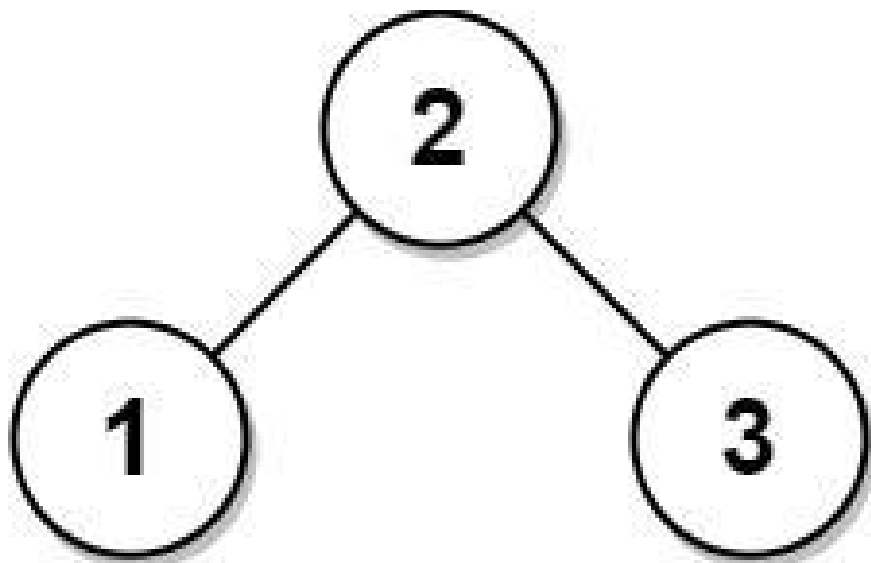
A binary search tree is balanced if the depth of the two subtrees of every node never differs by more than 1.

Example 1:



Input: root = [1,null,2,null,3,null,4,null,null]
Output: [2,1,3,null,null,4]
Explanation: This is not the only correct answer, [3,1,4,null,2] is also correct.

Example 2:



Input: root = [2,1,3]
Output: [2,1,3]

Constraints:

- The number of nodes in the tree is in the range [1, 104].
- $1 \leq \text{Node.val} \leq 105$

Community Solutions (Python)

- WalkCC

Round 2 · High · Medium

p.877

```

class Solution:
    def balanceBST(self, root: TreeNode | None) -> TreeNode | None:
        nums = []

        def inorder(root: TreeNode | None) -> None:
            if not root:
                return
            inorder(root.left)
            nums.append(root.val)
            inorder(root.right)

        inorder(root)

        # Same as 108. Convert Sorted Array to Binary Search Tree
        def build(l: int, r: int) -> TreeNode | None:
            if l > r:
                return None
            m = (l + r) // 2
            return TreeNode(nums[m],
                            build(l, m - 1),
                            build(m + 1, r))

        return build(0, len(nums) - 1)
  
```

• LeetCode

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def balanceBST(self, root: TreeNode) -> TreeNode:
        def dfs(root: TreeNode):
            if root is None:
  
```

Round 2 · High · Medium

p.878

```

        return
    dfs(root.left)
    nums.append(root.val)
    dfs(root.right)

def build(i: int, j: int) -> TreeNode:
    if i > j:
        return None
    mid = (i + j) >> 1
    left = build(i, mid - 1)

    right = build(mid + 1, j)
    return TreeNode(nums[mid], left, right)

nums = []
dfs(root)
return build(0, len(nums) - 1)

```

• SimplyLeet

```

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val      # Node's value.
        self.left = left    # Left child.
        self.right = right  # Right child.

class Solution:
    def balanceBST(self, root: TreeNode) -> TreeNode:
        # In-order traversal to collect nodes in sorted order.

        def inorder(node):
            if not node:
                return []
            return inorder(node.left) + [node.val] + inorder(node.right)

        sorted_vals = inorder(root)  # Sorted list of node values.

        # Recursively build balanced BST from the sorted list.
        def buildBST(left, right):
            if left > right:

```

Round 2 · High · Medium

p.879

```

        return None
    mid = (left + right) // 2
    node = TreeNode(sorted_vals[mid])
    node.left = buildBST(left, mid - 1)
    node.right = buildBST(mid + 1, right)
    return node

return buildBST(0, len(sorted_vals) - 1)

```

Round 2 · High · Medium

p.880

Tree Traversal – Post-Order

Round 2 · High · Medium · 9 questions

Round 2 · High · Medium

p.881

Tree Traversal – Post-Order

□ #114 Flatten Binary Tree to Linked List

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · Stack · Tree · Depth-First Search
· Binary Tree

Lists: AlgoMaster 600

Problem

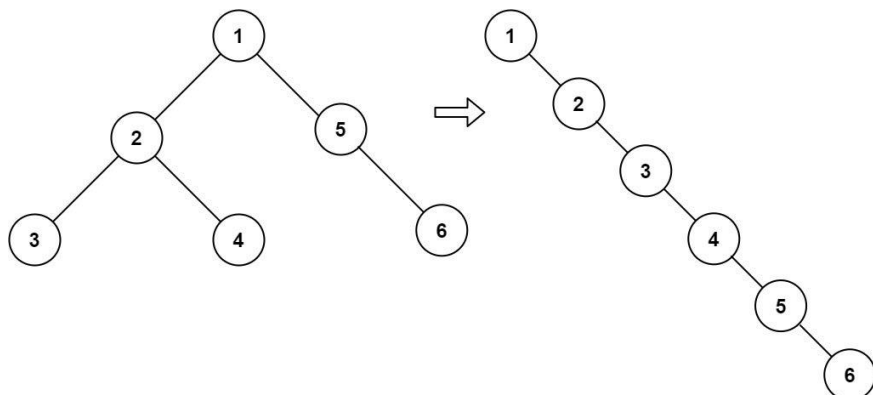
Given the root of a binary tree, flatten the tree into a "linked list":

- The "linked list" should use the same `TreeNode` class where the right child pointer points to the next node in the list and the left child pointer is always null.
- The "linked list" should be in the same order as a pre-order traversal of the binary tree.

Example 1:

Round 2 · High · Medium

p.882



Input: root = [1,2,5,3,4,null,6]
 Output: [1,null,2,null,3,null,4,null,5,null,6]

Example 2:

Input: root = []
 Output: []

Example 3:

Input: root = [0]
 Output: [0]

Constraints:

- The number of nodes in the tree is in the range [0, 2000].
- $-100 \leq \text{Node.val} \leq 100$

Follow up: Can you flatten the tree in-place (with $O(1)$ extra space)?

Community Solutions (Python)

- WalkCC

Round 2 · High · Medium

p.883

```

class Solution:
    def flatten(self, root: TreeNode | None) -> None:
        if not root:
            return

        self.flatten(root.left)
        self.flatten(root.right)

        left = root.left # flattened left
        right = root.right # flattened right

        root.left = None
        root.right = left

        # Connect the original right subtree to the end of the new right subtree.
        rightmost = root
        while rightmost.right:
            rightmost = rightmost.right
        rightmost.right = right
  
```

• LeetCode

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def flatten(self, root: Optional[TreeNode]) -> None:
        """
        Do not return anything, modify root in-place instead.
  
```

Round 2 · High · Medium

p.884

```

    """
    while root:
        if root.left:
            pre = root.left
            while pre.right:
                pre = pre.right
            pre.right = root.right
            root.right = root.left
            root.left = None
        root = root.right

```

• SimplyLeet

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

def flatten(root):
    # Start from the root node
    current = root

    # Iterate until we have processed all nodes
    while current:
        # If current node has a left subtree
        if current.left:
            # Find the rightmost node in the left subtree
            rightmost = current.left
            while rightmost.right:
                rightmost = rightmost.right
            # Connect the original right subtree to the rightmost node
            rightmost.right = current.right

        # Move the left subtree to become the right subtree
        current.right = current.left
        # Set the left child to None as per linked list requirement
        current.left = None
        # Move to the next right node in the flattened list
        current = current.right

```

Round 2 · High · Medium

p.885

Tree Traversal – Post-Order

□ #129 Sum Root to Leaf Numbers

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · Depth-First Search · Binary Tree
Lists: AlgoMaster 600

Problem

You are given the root of a binary tree containing digits from 0 to 9 only.

Each root-to-leaf path in the tree represents a number.

- For example, the root-to-leaf path 1 -> 2 -> 3 represents the number 123.

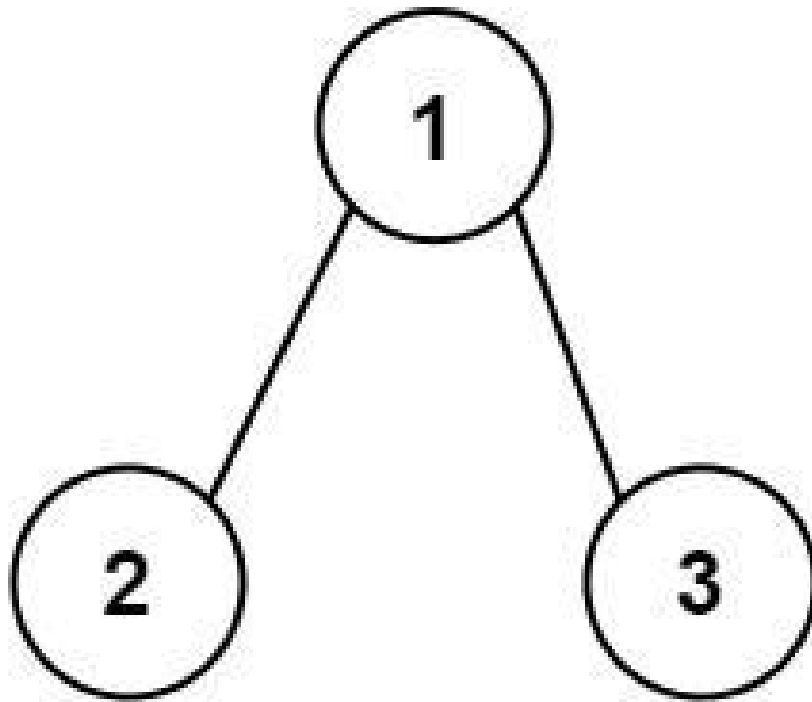
Return the total sum of all root-to-leaf numbers. Test cases are generated so that the answer will fit in a 32-bit integer.

A leaf node is a node with no children.

Example 1:

Round 2 · High · Medium

p.886



Input: root = [1,2,3]

Output: 25

Explanation:

The root-to-leaf path 1->2 represents the number 12.

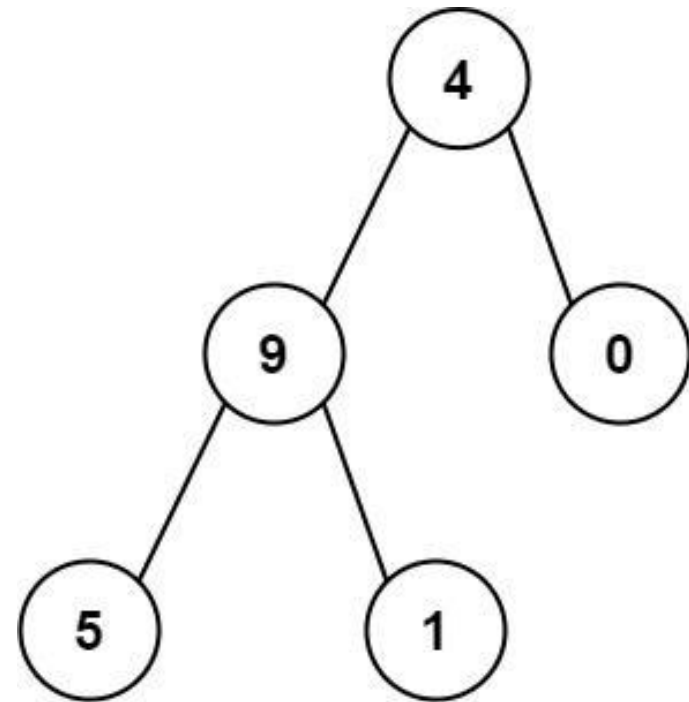
The root-to-leaf path 1->3 represents the number 13.

Therefore, sum = 12 + 13 = 25.

Example 2:

Round 2 · High · Medium

p.887



Input: root = [4,9,0,5,1]

Output: 1026

Explanation:

The root-to-leaf path 4->9->5 represents the number 495.

The root-to-leaf path 4->9->1 represents the number 491.

The root-to-leaf path 4->0 represents the number 40.

Therefore, sum = 495 + 491 + 40 = 1026.

Constraints:

- The number of nodes in the tree is in the range [1, 1000].
- $0 \leq \text{Node.val} \leq 9$
- The depth of the tree will not exceed 10.

Round 2 · High · Medium

p.888

Community Solutions (Python)

• WalkCC

```
class Solution:
    def sumNumbers(self, root: TreeNode | None) -> int:
        ans = 0

        def dfs(root: TreeNode | None, path: int) -> None:
            nonlocal ans
            if not root:
                return
            if not root.left and not root.right:
                ans += path * 10 + root.val

            dfs(root.left, path * 10 + root.val)
            dfs(root.right, path * 10 + root.val)

        dfs(root, 0)
        return ans
```

• LeetDoocs

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def sumNumbers(self, root: Optional[TreeNode]) -> int:
        def dfs(root, s):
            if root is None:
```

```
        return 0
        s = s * 10 + root.val
        if root.left is None and root.right is None:
            return s
        return dfs(root.left, s) + dfs(root.right, s)

    return dfs(root, 0)
```

• SimplyLeet

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None

class Solution:
    def sumNumbers(self, root: TreeNode) -> int:
        # Define a helper function for DFS traversal.

        def dfs(node, current_sum):
            if not node:
                return 0
            # Update the current sum (forming the number by appending current
            # node's value).
            current_sum = current_sum * 10 + node.val
            # If the node is a leaf, return the current sum.
            if not node.left and not node.right:
                return current_sum
            # Recursively compute the sum for left and right subtrees.
            left_sum = dfs(node.left, current_sum)

            right_sum = dfs(node.right, current_sum)
            return left_sum + right_sum

        # Start DFS from the root with an initial sum of 0.
        return dfs(root, 0)
```

Tree Traversal – Post-Order

□ #236 Lowest Common Ancestor of a Binary Tree

MED

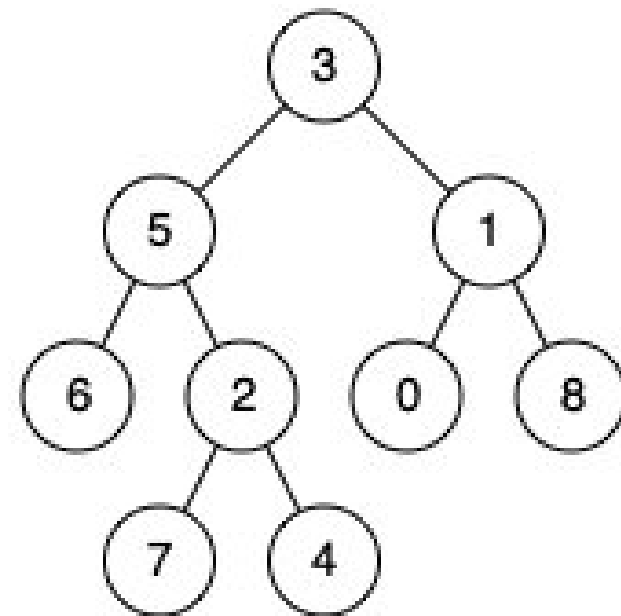
LeetDoocs · SimplyLeet · WalkCC · LeetCode Tree · DFS

Lists: Grind 169

Problem

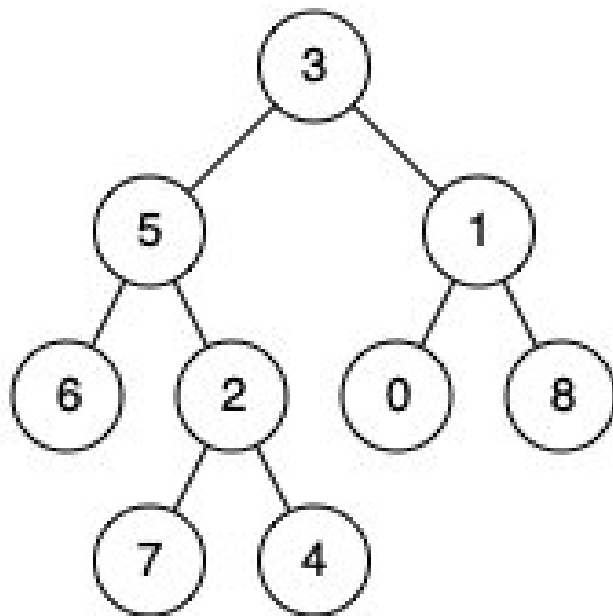
Given a binary tree, find the lowest common ancestor (LCA) of two given nodes in the tree. According to the definition of LCA on Wikipedia: “The lowest common ancestor is defined between two nodes p and q as the lowest node in T that has both p and q as descendants (where we allow a node to be a descendant of itself).”

Example 1:



Input: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1
 Output: 3
 Explanation: The LCA of nodes 5 and 1 is 3.

Example 2:



Input: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 4

Output: 5

Explanation: The LCA of nodes 5 and 4 is 5, since a node can be a descendant of itself according to the LCA definition.

Example 3:

Input: root = [1,2], p = 1, q = 2

Output: 1

Constraints:

- The number of nodes in the tree is in the range [2, 105].
- $-109 \leq \text{Node.val} \leq 109$

Round 2 · High · Medium

p.893

- All Node.val are unique.
- $p \neq q$
- p and q will exist in the tree.

Community Solutions (Python)

• WalkCC

```

class Solution:
    def lowestCommonAncestor(
        self,
        root: 'TreeNode',
        p: 'TreeNode',
        q: 'TreeNode',
    ) -> 'TreeNode':
        if not root or root == p or root == q:
            return root
        left = self.lowestCommonAncestor(root.left, p, q)
        right = self.lowestCommonAncestor(root.right, p, q)
        if left and right:
            return root
        return left or right

```

```

class Solution:
    def lowestCommonAncestor(
        self,
        root: TreeNode | None,
        p: TreeNode | None,
        q: TreeNode | None,
    ) -> TreeNode | None:
        q_ = collections.deque([root])
        parent = {root: None}
        ancestors = set() # p's ancestors

```

Round 2 · High · Medium

p.894

```
# Iterate until we find both p and q.
while p not in parent or q not in parent:
    root = q_.popleft()
    if root.left:
        parent[root.left] = root
        q_.append(root.left)
    if root.right:
        parent[root.right] = root
        q_.append(root.right)
```

```
# Insert all the p's ancestors.
while p:
    ancestors.add(p)
    p = parent[p] # `p` becomes None in the end.

# Go up from q until we meet any of p's ancestors.
while q not in ancestors:
    q = parent[q]
```

```
return q
```

• LeetCode

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None
```

```
class Solution:
    def lowestCommonAncestor(
```

Round 2 · High · Medium

p.895

```
self, root: "TreeNode", p: "TreeNode", q: "TreeNode"
) -> "TreeNode":
    if root in (None, p, q):
        return root
    left = self.lowestCommonAncestor(root.left, p, q)
    right = self.lowestCommonAncestor(root.right, p, q)
    return root if left and right else (left or right)
```

• SimplyLeet

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def lowestCommonAncestor(self, root: 'TreeNode', p: 'TreeNode', q: 'TreeNode') -> 'TreeNode':
        # Base case: if the current node is None, return None.
```

```
if root is None:
    return None
```

```
# If the current node is either p or q, return the current node.
if root == p or root == q:
    return root
```

```
# Search for p and q in the left and right subtrees.
left = self.lowestCommonAncestor(root.left, p, q)
right = self.lowestCommonAncestor(root.right, p, q)
```

```
# If both left and right are non-null, the current node is the LCA.
if left and right:
    return root
```

```
# Otherwise, return the non-null node.
return left if left is not None else right
```

• LC.ca

Round 2 · High · Medium

p.896

```
# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, x):
# self.val = x
# self.left = None
# self.right = None

class Solution:
    def lowestCommonAncestor(
        self, root: 'TreeNode', p: 'TreeNode', q: 'TreeNode'

    ) -> 'TreeNode':
        if root is None or root == p or root == q:
            return root
        left = self.lowestCommonAncestor(root.left, p, q)
        right = self.lowestCommonAncestor(root.right, p, q)
        return root if left and right else (left or right)

#####

# Definition for a binary tree node.

# class TreeNode(object):
# def __init__(self, x):
# self.val = x
# self.left = None
# self.right = None

class Solution(object):
    def lowestCommonAncestor(self, root, p, q):
        """
        :type root: TreeNode
        :type p: TreeNode
        :type q: TreeNode
        :rtype: TreeNode
        """

        if not root:
            return root

        left = self.lowestCommonAncestor(root.left, p, q)
        right = self.lowestCommonAncestor(root.right, p, q)
```

Round 2 · High · Medium

p.897

```
if left and right:
    return root

if root == p or root == q:
    return root

if left:
    return left
if right:

    return right
return None
```

◆ Quick Review

Key Insights

- Use a depth-first search (DFS) traversal to explore the tree.
- If the current node is either p or q, it could be the LCA.
- Recursively find p and q in the left and right subtrees.
- If both left and right recursive calls return a non-null value, then the current node is the LCA.
- If only one side returns a non-null value, propagate that value up the recursion.

Space & Time

Round 2 · High · Medium

p.898

Time Complexity: $O(n)$, where n is the number of nodes in the tree since each node is visited once.

Space Complexity: $O(h)$, where h is the height of the tree due to the recursion stack ($O(n)$ in the worst case of a skewed tree).

Solution

We solve the problem using recursion with depth-first search. The algorithm checks if the current node matches p or q . It then recursively searches the left and right subtrees. If both subtrees contain one of p or q , the current node is the LCA. Otherwise, the algorithm forwards the non-null result up the recursion. This approach naturally handles cases where one node is an ancestor of the other.

Tree Traversal – Post-Order

★ #545 Boundary of Binary Tree ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode Tree · DFS

Lists: ★ Premium | Premium 98

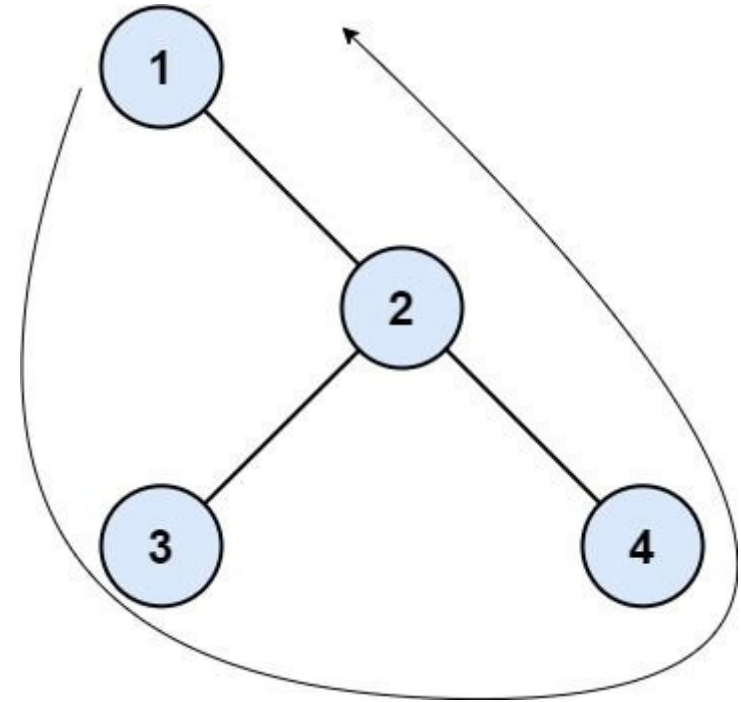
Problem

The boundary of a binary tree is the concatenation of the root, the left boundary, the leaves ordered from left-to-right, and the reverse order of the right boundary.

The left boundary is the set of nodes defined by the following:

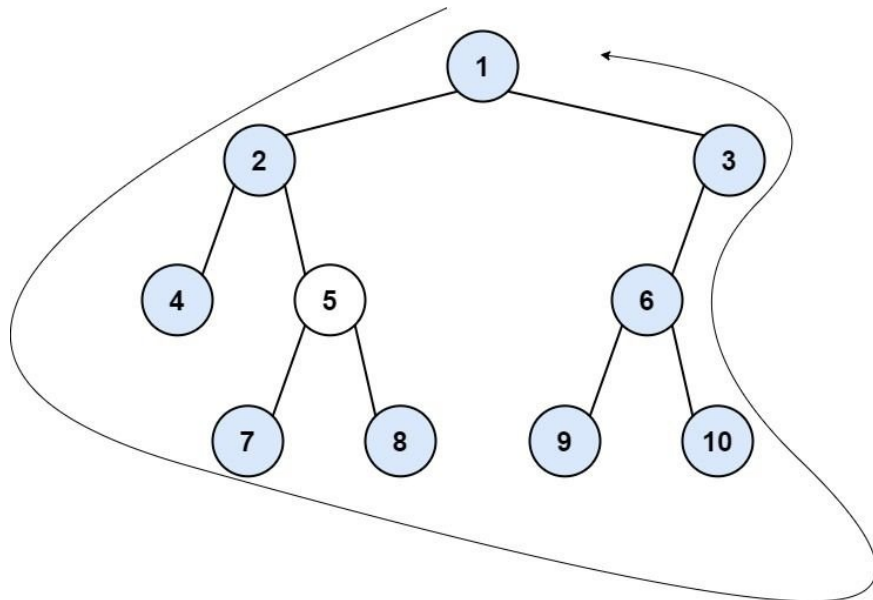
- The root node's left child is in the left boundary. If the root does not have a left child, then the left boundary is empty.
- If a node is in the left boundary and has a left child, then the left child is in the left boundary.
- If a node is in the left boundary, has no left child, but has a right child, then the right child is in the left boundary.

- The leftmost leaf is not in the left boundary. The right boundary is similar to the left boundary, except it is the right side of the root's right subtree. Again, the leaf is not part of the right boundary, and the right boundary is empty if the root does not have a right child. The leaves are nodes that do not have any children. For this problem, the root is not a leaf. Given the root of a binary tree, return the values of its boundary.
- Example 1:



Input: root = [1,null,2,3,4]
 Output: [1,3,4,2]
 Explanation:
 - The left boundary is empty because the root does not have a left child.
 - The right boundary follows the path starting from the root's right child 2 -> 4.
 4 is a leaf, so the right boundary is [2].
 - The leaves from left to right are [3,4].
 Concatenating everything results in [1] + [] + [3,4] + [2] = [1,3,4,2].

Example 2:



Input: root = [1,2,3,4,5,6,null,null,null,7,8,9,10]

Output: [1,2,4,7,8,9,10,6,3]

Explanation:

- The left boundary follows the path starting from the root's left child 2 -> 4. 4 is a leaf, so the left boundary is [2].
- The right boundary follows the path starting from the root's right child 3 -> 6 -> 10. 10 is a leaf, so the right boundary is [3,6], and in reverse order is [6,3].
- The leaves from left to right are [4,7,8,9,10].

Constraints:

- The number of nodes in the tree is in the range [1, 104].
- $-1000 \leq \text{Node.val} \leq 1000$

Community Solutions (Python)

Round 2 · High · Medium

p.903

• WalkCC

```
class Solution:
    def boundaryOfBinaryTree(self, root: TreeNode | None) -> list[int]:
        if not root:
            return []

        ans = [root.val]

        def dfs(root: TreeNode | None, lb: bool, rb: bool):
            """
            1. root.left is left boundary if root is left boundary.

            root.right if left boundary if root.left is None.
            2. Same applies for right boundary.
            3. If root is left boundary, add it before 2 children - preorder.
               If root is right boundary, add it after 2 children - postorder.
            4. A leaf that is neither left/right boundary belongs to the bottom.
            """
            if not root:
                return
            if lb:
                ans.append(root.val)

            if not lb and not rb and not root.left and not root.right:
                ans.append(root.val)

            dfs(root.left, lb, rb and not root.right)
            dfs(root.right, lb and not root.left, rb)
            if rb:
                ans.append(root.val)

            dfs(root.left, True, False)
            dfs(root.right, False, True)

        return ans
```

• LeetCode

Round 2 · High · Medium

p.904


```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def boundaryOfBinaryTree(self, root: Optional[TreeNode]) -> List[int]:
        def dfs(nums: List[int], root: Optional[TreeNode], i: int):
            if root is None:
                return
            if i == 0:
                if root.left != root.right:
                    nums.append(root.val)
                    if root.left:
                        dfs(nums, root.left, i)
                    else:
                        dfs(nums, root.right, i)
            elif i == 1:
                if root.left == root.right:
                    nums.append(root.val)
            else:
                dfs(nums, root.left, i)
                dfs(nums, root.right, i)

        ans = [root.val]
        if root.left == root.right:
            return ans
        left, leaves, right = [], [], []
        dfs(left, root.left, 0)
        dfs(leaves, root, 1)
        dfs(right, root.right, 2)
        ans += left + leaves + right[::-1]

```

Round 2 · High · Medium

p.905

```

return ans

```

• SimplyLeet

```

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def boundaryOfBinaryTree(self, root: TreeNode) -> list:
        if not root:
            return []

        # Check if a node is a leaf.
        def isLeaf(node):
            return node and not node.left and not node.right

        # Add left boundary nodes (excluding leaves).
        def addLeftBoundary(node, boundary):
            while node:
                if not isLeaf(node):
                    boundary.append(node.val)
                # Prefer left child, if not present then right child.
                if node.left:
                    node = node.left
                else:
                    node = node.right

        # Add leaf nodes using DFS.
        def addLeaves(node, leaves):
            if not node:

```

Round 2 · High · Medium

p.906

```

        return
    if isLeaf(node):
        leaves.append(node.val)
    else:
        addLeaves(node.left, leaves)
        addLeaves(node.right, leaves)

```

```

# Add right boundary nodes in reverse order (excluding leaves).
def addRightBoundary(node, boundary):
    temp = []

```

```

    while node:
        if not isLeaf(node):
            temp.append(node.val)
        if node.right:
            node = node.right
        else:
            node = node.left
    # Append in reverse order.
    while temp:
        boundary.append(temp.pop())

```

```

boundary = []
# Add root value if it is not a leaf.
if not isLeaf(root):
    boundary.append(root.val)

addLeftBoundary(root.left, boundary)
addLeaves(root, boundary)
addRightBoundary(root.right, boundary)

```

```

return boundary

```

```

# Example usage:
# Construct a binary tree and call boundaryOfBinaryTree(root)

```

• LC.ca

Round 2 · High · Medium

p.907

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def boundaryOfBinaryTree(self, root: TreeNode) -> List[int]:
        self.res = []
        if not root:

```

```

            return self.res
        # root
        if not self.is_leaf(root):
            self.res.append(root.val)

        # left boundary
        t = root.left
        while t:
            if not self.is_leaf(t):
                self.res.append(t.val)

```

```

            t = t.left if t.left else t.right

```

```

        # leaves
        self.add_leaves(root)

        # right boundary(reverse order)
        s = []
        t = root.right
        while t:
            if not self.is_leaf(t):

```

```

                s.append(t.val)
                t = t.right if t.right else t.left
            while s:
                self.res.append(s.pop())

```

```

        # output
        return self.res

```

```

def add_leaves(self, root):
    if self.is_leaf(root):

```

Round 2 · High · Medium

p.908

```

        self.res.append(root.val)
        return
    if root.left:
        self.add_leaves(root.left)
    if root.right:
        self.add_leaves(root.right)

def is_leaf(self, node) -> bool:
    return node and node.left is None and node.right is None

```

◆ Quick Review

Key Insights

- Break down the problem into three parts:
- Left boundary (excluding leaves).
- All leaves (do a DFS to find leaves).
- Right boundary (excluding leaves, then reverse the collected list).
- Special handling is needed when the tree doesn't have a left or right subtree.
- The root is always included and is never considered as a leaf, even if it has no children.
- Avoid duplication of leaf nodes in the left/right boundaries.

Space & Time

Time Complexity: $O(n)$ – Every node is visited at most once.

Space Complexity: $O(n)$ – $O(n)$ space is used for the output list and recursion stack (in worst case tree is skewed).

Solution

The solution uses a Depth-First Search (DFS) strategy to traverse the tree and collect the boundary nodes in three steps:

- For the left boundary, start at `root.left` and traverse down, adding nodes (skipping leaves).
 - For the leaves, perform a DFS from the root and add any node that has no children.
 - For the right boundary, start at `root.right` and traverse down, adding nodes (skipping leaves).
- Since the right boundary must be added in reverse order, we reverse the collected list after traversal. – Finally, concatenate these lists in the order: root, left boundary, leaves, reversed right boundary.

The approach makes use of helper functions to clearly separate the logic for gathering left boundary, leaves, and right boundary. This modular approach simplifies reasoning and avoids edge case mistakes such as including duplicate leaf nodes.

Tree Traversal – Post-Order

#652 Find Duplicate Subtrees

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Tree · Depth-First Search · Binary Tree

Lists: AlgoMaster 600

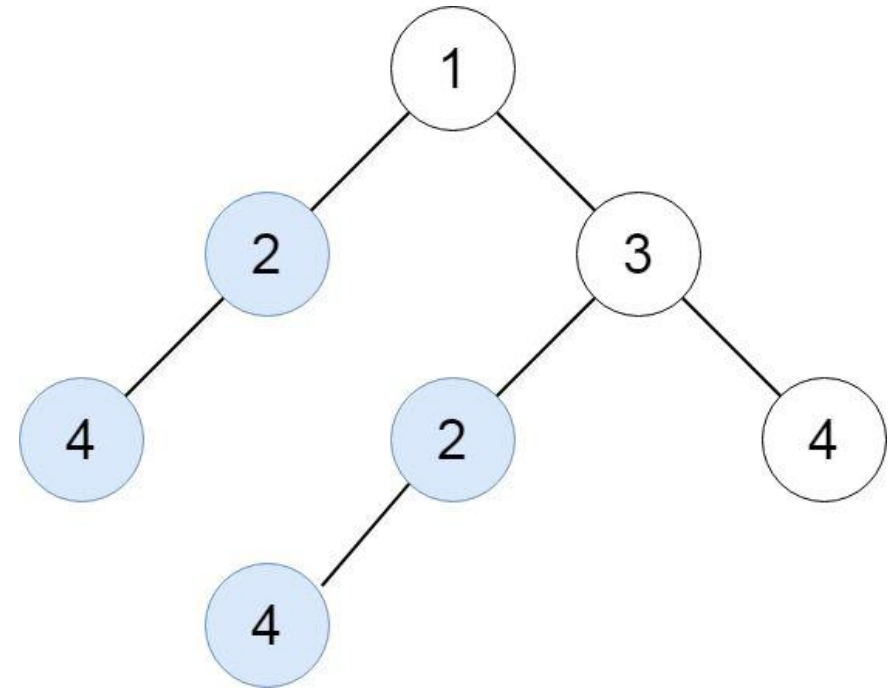
Problem

Given the root of a binary tree, return all duplicate subtrees.

For each kind of duplicate subtrees, you only need to return the root node of any one of them.

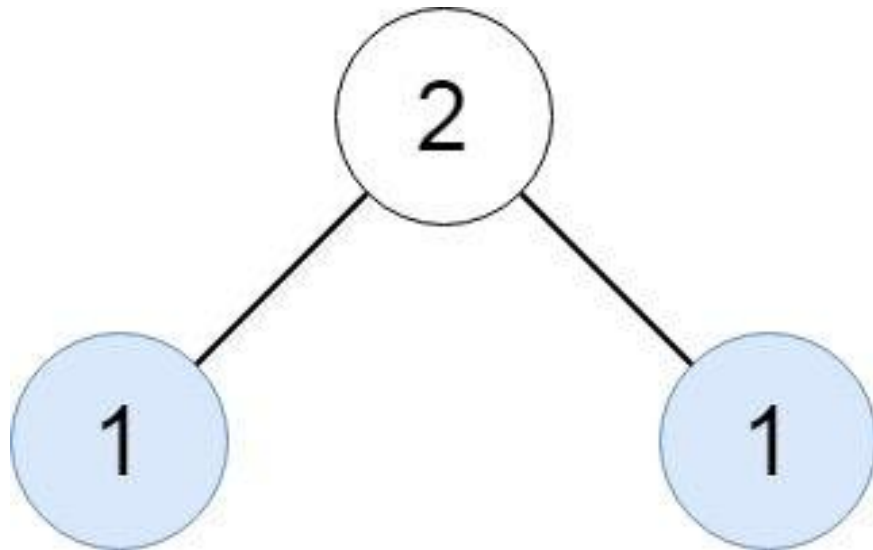
Two trees are duplicate if they have the same structure with the same node values.

Example 1:



Input: root = [1,2,3,4,null,2,4,null,null,4]
Output: [[2,4],[4]]

Example 2:

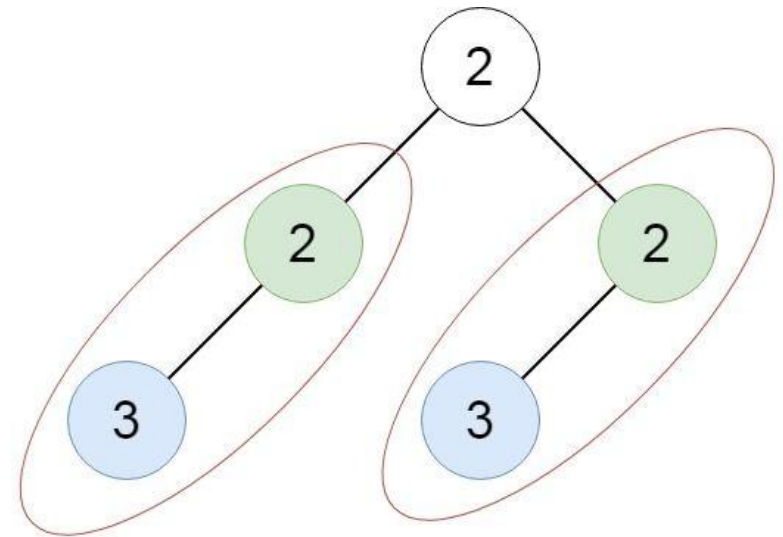


Input: root = [2,1,1]
Output: [[1]]

Example 3:

Round 2 · High · Medium

p.913



Input: root = [2,2,2,3,null,3,null]
Output: [[2,3],[3]]

Constraints:

- The number of the nodes in the tree will be in the range [1, 5000]
- $-200 \leq \text{Node.val} \leq 200$

Community Solutions (Python)

- WalkCC

Round 2 · High · Medium

p.914

```
class Solution:
    def findDuplicateSubtrees(self, root: TreeNode | None) -> list[TreeNode | None]:
        ans = []
        count = collections.Counter()

        def encode(root: TreeNode | None) -> str:
            if not root:
                return ''

            encoded = (str(root.val) + '#' +
                      encode(root.left) + '#' +
                      encode(root.right))

            count[encoded] += 1
            if count[encoded] == 2:
                ans.append(root)
            return encoded

        encode(root)
        return ans
```

• LeetCode

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def findDuplicateSubtrees(
        self, root: Optional[TreeNode]
    ) -> List[Optional[TreeNode]]:
```

Round 2 · High · Medium

p.915

```
def dfs(root):
    if root is None:
        return '#'
    v = f'{root.val},{dfs(root.left)},{dfs(root.right)}'
    counter[v] += 1
    if counter[v] == 2:
        ans.append(root)
    return v

ans = []

counter = Counter()
dfs(root)
return ans
```

• SimplyLeet

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def findDuplicateSubtrees(self, root: TreeNode) -> List[TreeNode]:
        # Dictionary to store the frequency of each subtree serialization.

        subtree_count = {}
        # List to store the root nodes of duplicate subtrees.
        duplicates = []

        # Define a helper function for postorder traversal and serialization.
        def serialize(node):
            # If the current node is None, use a special marker.
            if not node:
                return "#"
            # Recursively serialize the left and right subtree.
```

Round 2 · High · Medium

p.916

```

    left_serial = serialize(node.left)
    right_serial = serialize(node.right)
    # Create a unique serialization for the subtree rooted at the
    current node.
    serial = str(node.val) + "," + left_serial + "," + right_serial
    # Update the count of this serialization in our dictionary.
    subtree_count[serial] = subtree_count.get(serial, 0) + 1
    # When a duplicate is found for the first time, add the node to the
    duplicates list.
    if subtree_count[serial] == 2:
        duplicates.append(node)
    # Return the serialization to the parent caller.

    return serial

serialize(root)
return duplicates

```

Tree Traversal – Post-Order

#979 Distribute Coins in Binary Tree

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Tree · Depth-First Search · Binary Tree
 Lists: AlgoMaster 600

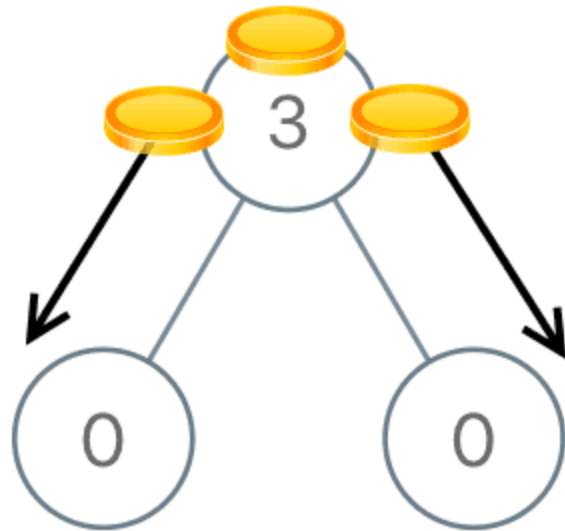
Problem

You are given the root of a binary tree with n nodes where each node in the tree has `node.val` coins. There are n coins in total throughout the whole tree.

In one move, we may choose two adjacent nodes and move one coin from one node to another. A move may be from parent to child, or from child to parent.

Return the minimum number of moves required to make every node have exactly one coin.

Example 1:



Input: root = [3,0,0]

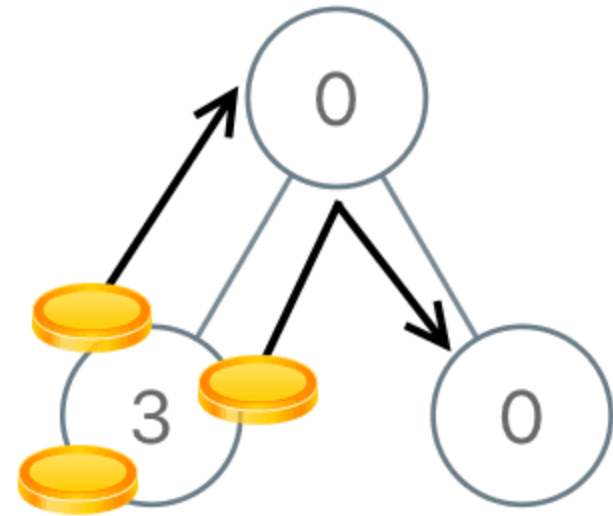
Output: 2

Explanation: From the root of the tree, we move one coin to its left child, and one coin to its right child.

Example 2:

Round 2 · High · Medium

p.919



Input: root = [0,3,0]

Output: 3

Explanation: From the left child of the root, we move two coins to the root [taking two moves]. Then, we move one coin from the root of the tree to the right child.

Constraints:

- The number of nodes in the tree is n .
- $1 \leq n \leq 100$
- $0 \leq \text{Node.val} \leq n$
- The sum of all Node.val is n .

Round 2 · High · Medium

p.920

Community Solutions (Python)

• LeetCode

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def distributeCoins(self, root: Optional[TreeNode]) -> int:
        def dfs(root):
            if root is None:
                return 0
            left, right = dfs(root.left), dfs(root.right)
            nonlocal ans
            ans += abs(left) + abs(right)
            return left + right + root.val - 1

        ans = 0
        dfs(root)
        return ans
```

• SimplyLeet

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val # number of coins at the node
        self.left = left # left child
        self.right = right # right child

class Solution:
    def distributeCoins(self, root: TreeNode) -> int:
        self.moves = 0 # initialize the move counter
```

```
def dfs(node):
    if not node:
        return 0 # base case: null nodes contribute 0 balance
    # Recursively process left and right children
    left_balance = dfs(node.left)
    right_balance = dfs(node.right)
    # Accumulate moves: absolute coin imbalances from both children
    self.moves += abs(left_balance) + abs(right_balance)
    # Return the current node's net balance

    return node.val + left_balance + right_balance - 1

dfs(root)
return self.moves
```

Tree Traversal – Post-Order

□ #1110 Delete Nodes And Return Forest

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table · Tree · Depth-First Search
 · Binary Tree

Lists: AlgoMaster 600

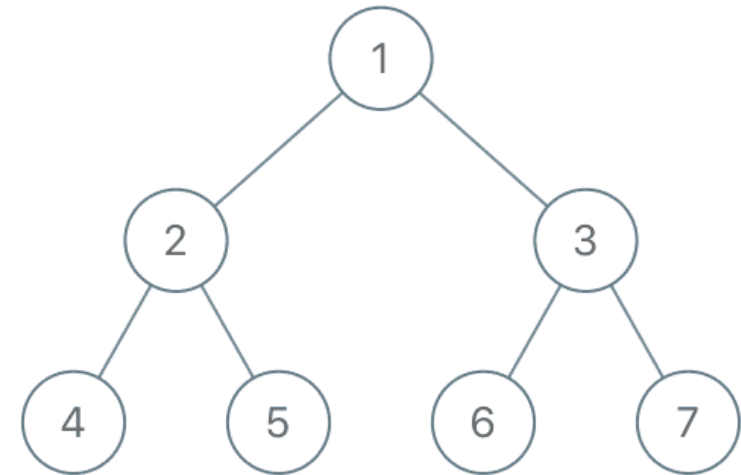
Problem

Given the root of a binary tree, each node in the tree has a distinct value.

After deleting all nodes with a value in `to_delete`, we are left with a forest (a disjoint union of trees).

Return the roots of the trees in the remaining forest. You may return the result in any order.

Example 1:



Input: root = [1,2,3,4,5,6,7], to_delete = [3,5]
 Output: [[1,2,null,4],[6],[7]]

Example 2:

Input: root = [1,2,4,null,3], to_delete = [3]
 Output: [[1,2,4]]

Constraints:

- The number of nodes in the given tree is at most 1000.
- Each node has a distinct value between 1 and 1000.
- `to_delete.length` $\leq$ 1000
- `to_delete` contains distinct values between 1 and 1000.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def delNodes(self, root: TreeNode, to_delete: list[int]) -> list[TreeNode]:
        ans = []
        toDeleteSet = set(to_delete)

        def dfs(root: TreeNode, isRoot: bool) -> TreeNode:
            if not root:
                return None

            deleted = root.val in toDeleteSet

            if isRoot and not deleted:
                ans.append(root)

            # If root is deleted, both children have the possibility to be a new root
            root.left = dfs(root.left, deleted)
            root.right = dfs(root.right, deleted)
            return None if deleted else root

        dfs(root, True)
        return ans
```

• LeetCode

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def delNodes(
        self, root: Optional[TreeNode], to_delete: List[int]
    ) -> List[TreeNode]:
```

```
def dfs(root: Optional[TreeNode]) -> Optional[TreeNode]:
    if root is None:
        return None
    root.left, root.right = dfs(root.left), dfs(root.right)
    if root.val not in s:
        return root
    if root.left:
        ans.append(root.left)
    if root.right:
        ans.append(root.right)
```

```
return None
```

```
s = set(to_delete)
ans = []
if dfs(root):
    ans.append(root)
return ans
```

• SimplyLeet

```
# Python solution with line-by-line comments
```

```
# Definition for a binary tree node.
```

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
```

```
class Solution:
```

```
def delNodes(self, root: TreeNode, to_delete: list) -> list:
    # Convert the to_delete list to a set for faster lookup
    to_delete_set = set(to_delete)
    # Result list to hold roots of the remaining forest
    remaining_forest = []

    def dfs(node, is_root):
        # Base condition: if the current node is null, return None
        if not node:
            return None
```

```

# Determine if current node needs deletion
node_deleted = node.val in to_delete_set

# If this node is a root (or newly added) and is not deleted,
# add it to the remaining forest.
if is_root and not node_deleted:
    remaining_forest.append(node)

# Recursively process the left and right subtrees

node.left = dfs(node.left, node_deleted)
node.right = dfs(node.right, node_deleted)

# Return None if node is deleted, else return the node itself.
return None if node_deleted else node

# Start DFS from the root, the root is considered as a starting root.
dfs(root, True)
return remaining_forest

```

Tree Traversal – Post-Order

□ #1448 Count Good Nodes in Binary Tree **MED**

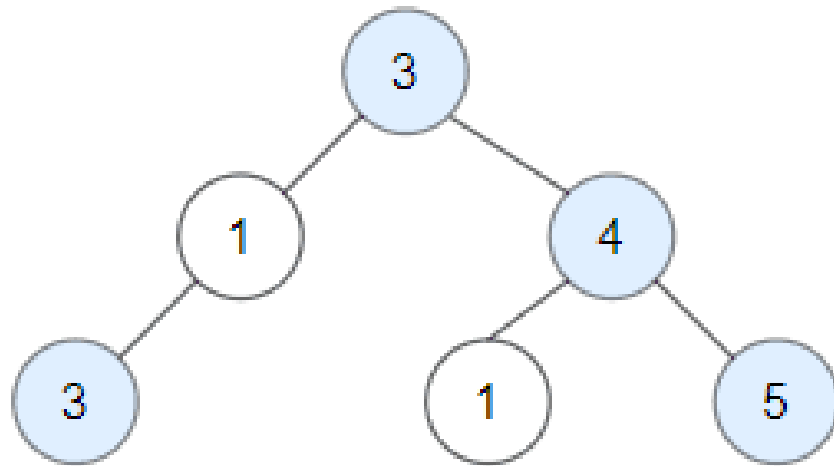
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · Depth-First Search · Breadth-First
Search · Binary Tree

Lists: NeetCode 150

Problem

Given a binary tree root, a node X in the tree is named good if in the path from root to X there are no nodes with a value greater than X. Return the number of good nodes in the binary tree.

Example 1:



Input: root = [3,1,4,3,null,1,5]

Output: 4

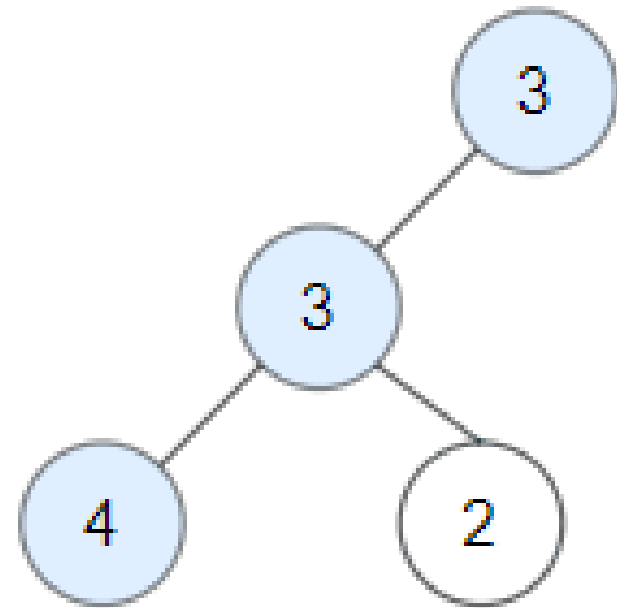
Explanation: Nodes in blue are good.

Root Node (3) is always a good node.

Example 2:

Round 2 · High · Medium

p.929



Input: root = [3,3,null,4,2]

Output: 3

Explanation: Node 2 -> (3, 3, 2) is not good, because "3" is higher than it.

Example 3:

Input: root = [1]

Output: 1

Explanation: Root is considered as good.

Constraints:

Round 2 · High · Medium

p.930

- The number of nodes in the binary tree is in the range $[1, 10^5]$.
- Each node's value is between $[-10^4, 10^4]$.

Community Solutions (Python)

• LeetCode

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def goodNodes(self, root: TreeNode) -> int:
        def dfs(root: TreeNode, mx: int):
            if root is None:
                return
            nonlocal ans
            if mx <= root.val:
                ans += 1
                mx = root.val
            dfs(root.left, mx)
            dfs(root.right, mx)

        ans = 0
        dfs(root, -1000000)

        return ans
```

• SimplyLeet

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
```

```
class Solution:
    def goodNodes(self, root: TreeNode) -> int:
        # Helper function to perform DFS traversal.
```

```
def dfs(node, currentMax):
    # If we reach a null node, there are no further nodes to count.
    if not node:
        return 0
```

```
    # Check if current node is good.
    count = 1 if node.val >= currentMax else 0
    # Update the current maximum value.
    newMax = max(currentMax, node.val)
```

```
    # Recursively count good nodes in the left and right subtrees.
    count += dfs(node.left, newMax)
    count += dfs(node.right, newMax)
    return count

    # Start DFS traversal from root with initial maximum value as root's
    value.
    return dfs(root, root.val)
```

Tree Traversal – Post-Order

□ #2096 Step-By-Step Directions From a Binary Tree Node to Another

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode String · Tree · Depth-First Search · Binary Tree Lists: AlgoMaster 600

Problem

You are given the root of a binary tree with n nodes. Each node is uniquely assigned a value from 1 to n . You are also given an integer `startValue` representing the value of the start node s , and a different integer `destValue` representing the value of the destination node t . Find the shortest path starting from node s and ending at node t . Generate step-by-step directions of such path as a string consisting of only the uppercase letters 'L', 'R', and 'U'. Each letter indicates a specific direction:

- 'L' means to go from a node to its left child node.
- 'R' means to go from a node to its right child node.

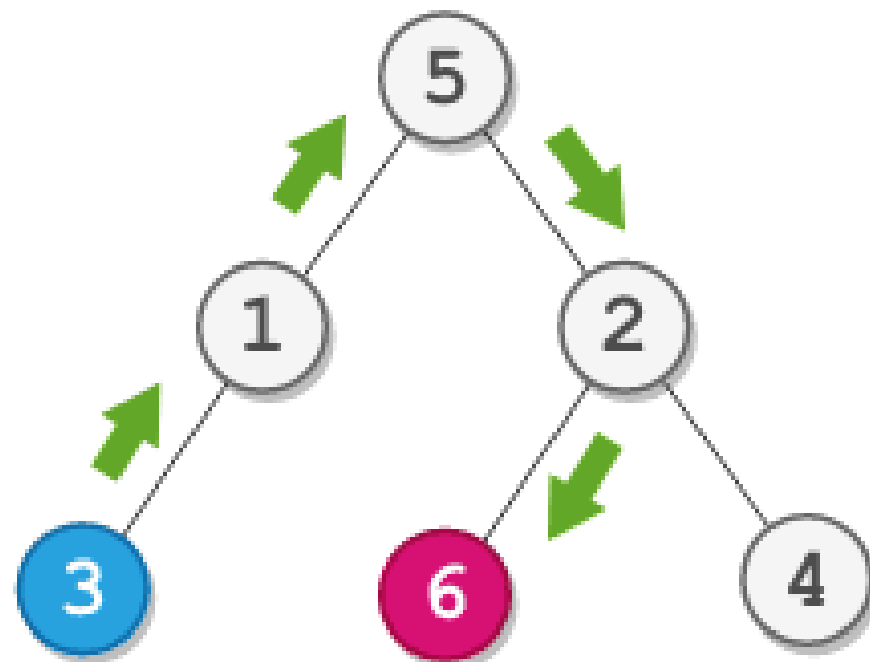
Round 2 · High · Medium

p.933

- 'U' means to go from a node to its parent node.

Return the step-by-step directions of the shortest path from node s to node t .

Example 1:

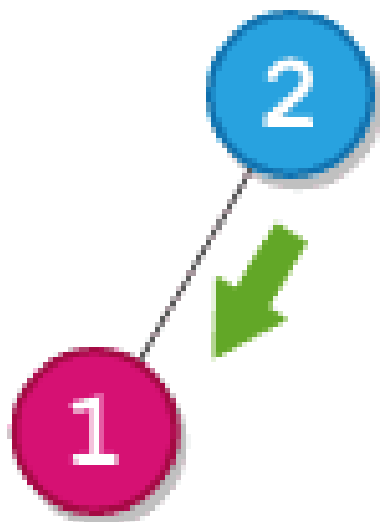


Input: `root = [5,1,2,3,null,6,4]`, `startValue = 3`, `destValue = 6`
 Output: "UURL"
 Explanation: The shortest path is: 3 → 1 → 5 → 2 → 6.

Example 2:

Round 2 · High · Medium

p.934



Input: root = [2,1], startValue = 2, destValue = 1

Output: "L"

Explanation: The shortest path is: 2 → 1.

Constraints:

- The number of nodes in the tree is n .
- $2 \leq n \leq 105$
- $1 \leq \text{Node.val} \leq n$
- All the values in the tree are unique.
- $1 \leq \text{startValue}, \text{destValue} \leq n$
- $\text{startValue} \neq \text{destValue}$

Community Solutions (Python)

- WalkCC

Round 2 · High · Medium

p.935

```

class Solution:
    def getDirections(
        self,
        root: TreeNode | None,
        startValue: int,
        destValue: int,
    ) -> str:
        def lca(root: TreeNode | None) -> TreeNode | None:
            if not root or root.val in (startValue, destValue):
                return root

            left = lca(root.left)
            right = lca(root.right)
            if left and right:
                return root
            return left or right

        def dfs(root: TreeNode | None, path: list[str]) -> None:
            if not root:
                return
            if root.val == startValue:
                self.pathToStart = ''.join(path)
            if root.val == destValue:
                self.pathToDest = ''.join(path)
            path.append('L')
            dfs(root.left, path)
            path.pop()
            path.append('R')
            dfs(root.right, path)
            path.pop()

        dfs(lca(root), []) # Only this subtree matters.
        return 'U' * len(self.pathToStart) + ''.join(self.pathToDest)
  
```

- LeetCode

Round 2 · High · Medium

p.936


```
# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):
# self.val = val
# self.left = left
# self.right = right

class Solution:
    def getDirections(
        self, root: Optional[TreeNode], startValue: int, destValue: int
    ) -> str:
        def lca(node: Optional[TreeNode], p: int, q: int):
            if node is None or node.val in (p, q):
                return node
            left = lca(node.left, p, q)
            right = lca(node.right, p, q)
            if left and right:
                return node
            return left or right

        def dfs(node: Optional[TreeNode], x: int, path: List[str]):
            if node is None:
                return False
            if node.val == x:
                return True
            path.append("L")
            if dfs(node.left, x, path):
                return True
            path[-1] = "R"

            if dfs(node.right, x, path):
                return True
            path.pop()
            return False

        node = lca(root, startValue, destValue)

        path_to_start = []
        path_to_dest = []
```

Round 2 · High · Medium

p.937

```
dfs(node, startValue, path_to_start)
dfs(node, destValue, path_to_dest)

return "U" * len(path_to_start) + "".join(path_to_dest)
```

• SimplyLeet

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

# Function to perform DFS and find the path from the root to a target node.
# The path list will record 'L' or 'R' for each move from the current node.
def dfs(node, target, path):

    if node is None:
        return False
    if node.val == target:
        return True

    # Try left subtree
    path.append('L')
    if dfs(node.left, target, path):
        return True
    path.pop()

    # Try right subtree

    path.append('R')
    if dfs(node.right, target, path):
        return True
    path.pop()
    return False

def getDirections(root, startValue, destValue):
    # Get the paths from root to start and destination
    path_to_start = []
    path_to_dest = []
```

Round 2 · High · Medium

p.938

```
dfs(root, startValue, path_to_start)
dfs(root, destValue, path_to_dest)

# Find the common prefix length (i.e. the LCA)
i = 0
while i < len(path_to_start) and i < len(path_to_dest) and path_to_start[i] == path_to_dest[i]:
    i += 1

# Steps to move up from start to the LCA

upward_moves = "U" * (len(path_to_start) - i)
# Steps to move down from the LCA to the destination
downward_moves = "".join(path_to_dest[i:])

return upward_moves + downward_moves

# Example usage:
# Suppose we have a tree constructed and we need to call getDirections(root, startValue, destValue)
```

Round 2 · High · Medium

p.939

Depth First Search (DFS)

Round 2 · High · Medium · 12 questions

Round 2 · High · Medium

p.940

Depth First Search (DFS)

#79 Word Search

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · String · Backtracking · Matrix
 Lists: Grind 169

Problem

Given an $m \times n$ grid of characters `board` and a string `word`, return `true` if word exists in the grid.

The word can be constructed from letters of sequentially adjacent cells, where adjacent cells are horizontally or vertically neighboring. The same letter cell may not be used more than once.

Example 1:

A	B	C	E
S	F	C	S
A	D	E	E

Input: `board = [ ["A","B","C","E"], ["S","F","C","S"], ["A","D","E","E"] ]`, `word = "ABCCED"`
 Output: `true`

Example 2:

A	B	C	E
S	F	C	S
A	D	E	E

Input: board = [["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]], word = "SEE"
Output: true

Example 3:

Round 2 · High · Medium

p.943

A	B	C	E
S	F	C	S
A	D	E	E

Input: board = [["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]], word = "ABCB"
Output: false

Constraints:

- $m == \text{board.length}$
- $n = \text{board}[i].\text{length}$
- $1 \leq m, n \leq 6$
- $1 \leq \text{word.length} \leq 15$
- board and word consists of only lowercase and uppercase English letters.

Round 2 · High · Medium

p.944

Follow up: Could you use search pruning to make your solution faster with a larger board?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def exist(self, board: List[List[str]], word: str) -> bool:
        m = len(board)
        n = len(board[0])

        def dfs(i: int, j: int, s: int) -> bool:
            if i < 0 or i == m or j < 0 or j == n:
                return False
            if board[i][j] != word[s] or board[i][j] == '*':
                return False

            if s == len(word) - 1:
                return True

            cache = board[i][j]
            board[i][j] = '*'
            isExist = (dfs(i + 1, j, s + 1) or
                      dfs(i - 1, j, s + 1) or
                      dfs(i, j + 1, s + 1) or
                      dfs(i, j - 1, s + 1))
            board[i][j] = cache

            return isExist

        return any(dfs(i, j, 0)
                   for i in range(m)
                   for j in range(n))
```

• LeetDoocs

Round 2 · High · Medium

p.945

```
class Solution:
    def exist(self, board: List[List[str]], word: str) -> bool:
        def dfs(i: int, j: int, k: int) -> bool:
            if k == len(word) - 1:
                return board[i][j] == word[k]
            if board[i][j] != word[k]:
                return False
            c = board[i][j]
            board[i][j] = "0"
            for a, b in pairwise((-1, 0, 1, 0, -1)):
```

```
                x, y = i + a, j + b
                ok = 0 <= x < m and 0 <= y < n and board[x][y] != "0"
                if ok and dfs(x, y, k + 1):
                    return True
            board[i][j] = c
            return False

        m, n = len(board), len(board[0])
        return any(dfs(i, j, 0) for i in range(m) for j in range(n))
```

• SimplyLeet

```
# Define the function to solve the Word Search problem.
def exist(board, word):
    # Get dimensions of the board.
    rows, cols = len(board), len(board[0])

    # Helper function for DFS search.
    def dfs(r, c, index):
        # If the entire word is found, return True.
        if index == len(word):
            return True
```

Round 2 · High · Medium

p.946

```

        # Check boundaries and if current cell already used or not matching.
        if r < 0 or c < 0 or r >= rows or c >= cols or board[r][c] != word[index]:
            return False

        # Temporarily mark the cell as visited.
        temp = board[r][c]
        board[r][c] = "#"

        # Explore all four directions.

        found = (dfs(r + 1, c, index + 1) or
                 dfs(r - 1, c, index + 1) or
                 dfs(r, c + 1, index + 1) or
                 dfs(r, c - 1, index + 1))

        # Backtrack: restore the original cell value.
        board[r][c] = temp
        return found

# Initiate DFS from each cell in the grid.

for i in range(rows):
    for j in range(cols):
        if dfs(i, j, 0):
            return True
    return False

# Example test case
if __name__ == "__main__":
    board = [
        ["A", "B", "C", "E"],
        ["S", "F", "C", "S"],
        ["A", "D", "E", "E"]
    ]
    word = "ABCCED"
    print(exist(board, word)) # Expected output: True

```

☐ • LC.ca

Round 2 · High · Medium

p.947

```

class Solution:
    def exist(self, board: List[List[str]], word: str) -> bool:
        def dfs(i, j, cur): # cur: current char count
            if cur == len(word):
                return True
            if (
                i < 0
                or i >= m
                or j < 0
                or j >= n
                or board[i][j] == '0'
                or word[cur] != board[i][j]
            ):
                return False
            t = board[i][j]
            board[i][j] = '0' # mark as visited
            for a, b in [[0, 1], [0, -1], [-1, 0], [1, 0]]:
                x, y = i + a, j + b
                if dfs(x, y, cur + 1):
                    return True

            board[i][j] = t
            return False

        m, n = len(board), len(board[0])
        return any(dfs(i, j, 0) for i in range(m) for j in range(n))

```

◆ Quick Review

Key Insights

- Use Depth-First Search (DFS) to explore each cell in the board as a potential starting point.
- Employ backtracking to mark cells as visited when exploring a path and revert them back when backtracking.

Round 2 · High · Medium

p.948

- Check boundary conditions and ensure the current cell's character matches the corresponding character in the word.
- Given the small grid and word sizes, a recursive solution is both feasible and straightforward.
- Search pruning can be applied by stopping a DFS branch as soon as the current path does not match the prefix of the target word.

Space & Time

Time Complexity: $O(m * n * 3^L)$, where L is the length of the word (each DFS may branch to at most 3 further cells, excluding the coming cell).

Space Complexity: $O(L)$ due to recursion call stack (where L is the word length).

Solution

The solution uses DFS with backtracking to traverse the grid. For each cell, we:

- Check if the cell matches the first character of the word.
- Recursively explore the adjacent cells (up, down, left, right) while marking the current cell as visited.
- If the path does not lead to a complete match, backtrack by

Round 2 · High · Medium

p.949

resetting the visited cell. – The DFS stops if all characters in the word have been found in sequence.

The algorithm leverages in-place modifications (or an auxiliary visited array) to avoid revisiting cells while exploring a candidate path.

Round 2 · High · Medium

p.950

Depth First Search (DFS)

#130 Surrounded Regions

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · DFS · BFS · Union Find · Matrix
 Lists: Denny Zhang

Problem

You are given an $m \times n$ matrix board containing letters 'X' and 'O', capture regions that are surrounded:

- **Connect:** A cell is connected to adjacent cells horizontally or vertically.
- **Region:** To form a region connect every 'O' cell.
- **Surround:** A region is surrounded if none of the 'O' cells in that region are on the edge of the board. Such regions are completely enclosed by 'X' cells.

To capture a surrounded region, replace all 'O's with 'X's in-place within the original board. You do not need to return anything.

Example 1:

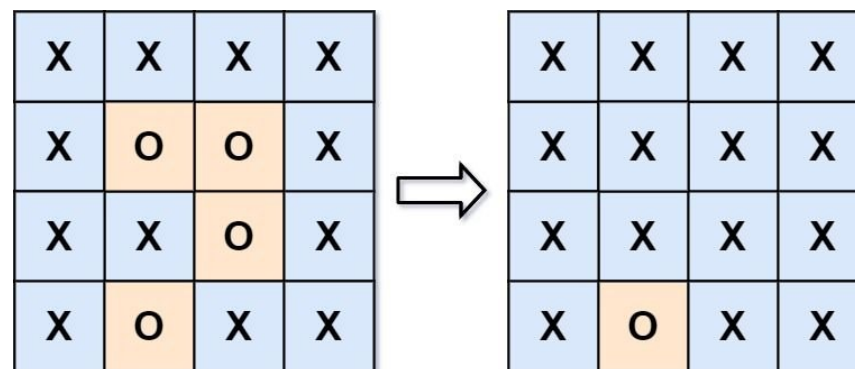
Round 2 · High · Medium

p.951

Input: board = [
 ["X","X","X","X"],
 ["X","O","O","X"],
 ["X","X","O","X"],
 ["X","O","X","X"]
]

Output: [
 ["X","X","X","X"],
 ["X","X","X","X"],
 ["X","X","X","X"],
 ["X","O","X","X"]
]

Explanation:



In the above diagram, the bottom region is not captured because it is on the edge of the board and cannot be surrounded.

Example 2:

Input: board = [
 ["X"]
]

Output: [
 ["X"]
]

Constraints:

- $m == \text{board.length}$
- $n == \text{board}[i].\text{length}$
- $1 \leq m, n \leq 200$

Round 2 · High · Medium

p.952

- `board[i][j]` is 'X' or 'O'.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def solve(self, board: List[List[str]]) -> None:
        if not board:
            return

        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(board)
        n = len(board[0])
        q = collections.deque()

        for i in range(m):
            for j in range(n):
                if i * j == 0 or i == m - 1 or j == n - 1:
                    if board[i][j] == 'O':
                        q.append((i, j))
                        board[i][j] = '*'

        # Mark the grids that stretch from the four sides with '*'.
        while q:
            i, j = q.popleft()

            for dx, dy in DIRS:
                x = i + dx
                y = j + dy
                if x < 0 or x == m or y < 0 or y == n:
                    continue
                if board[x][y] != 'O':
                    continue
                q.append((x, y))
                board[x][y] = '*'

        for row in board:
            for i, c in enumerate(row):
                row[i] = 'O' if c == '*' else 'X'
```

Round 2 · High · Medium

p.953

• LeetCode

```
class Solution:
    def solve(self, board: List[List[str]]) -> None:
        def dfs(i: int, j: int):
            if not (0 <= i < m and 0 <= j < n and board[i][j] == "O"):
                return
            board[i][j] = "."
            for a, b in pairwise((-1, 0, 1, 0, -1)):
                dfs(i + a, j + b)

        m, n = len(board), len(board[0])

        for i in range(m):
            dfs(i, 0)
            dfs(i, n - 1)
        for j in range(n):
            dfs(0, j)
            dfs(m - 1, j)
        for i in range(m):
            for j in range(n):
                if board[i][j] == ".":
                    board[i][j] = "O"

        elif board[i][j] == "O":
            board[i][j] = "X"

        class Solution:
            def solve(self, board: List[List[str]]) -> None:
                def find(x: int) -> int:
                    if p[x] != x:
                        p[x] = find(p[x])
                    return p[x]

                m, n = len(board), len(board[0])
                p = list(range(m * n + 1))
                for i in range(m):
```

Round 2 · High · Medium

p.954

```

for j in range(n):
    if board[i][j] == "0":
        if i in (0, m - 1) or j in (0, n - 1):
            p[find(i * n + j)] = find(m * n)
        else:
            for a, b in pairwise((-1, 0, 1, 0, -1)):
                x, y = i + a, j + b
                if board[x][y] == "0":
                    p[find(x * n + y)] = find(i * n + j)
for i in range(m):
    for j in range(n):
        if board[i][j] == "0" and find(i * n + j) != find(m * n):
            board[i][j] = "X"

```

• SimplyLeet

```

def solve(board):
    if not board or not board[0]:
        return

    rows, cols = len(board), len(board[0])

    def dfs(r, c):
        # Return if out of bounds or current cell is not '0'
        if r < 0 or c < 0 or r >= rows or c >= cols or board[r][c] != '0':
            return

        # Mark the cell as safe
        board[r][c] = '*'
        # Explore four possible directions
        dfs(r + 1, c)
        dfs(r - 1, c)
        dfs(r, c + 1)
        dfs(r, c - 1)

    # Traverse the board borders
    for i in range(rows):

```

Round 2 · High · Medium

p.955

```

if board[i][0] == '0':
    dfs(i, 0)
if board[i][cols - 1] == '0':
    dfs(i, cols - 1)
for j in range(cols):
    if board[0][j] == '0':
        dfs(0, j)
    if board[rows - 1][j] == '0':
        dfs(rows - 1, j)

```

```

# Flip surrounded '0's and restore safe cells
for i in range(rows):
    for j in range(cols):
        if board[i][j] == '0':
            board[i][j] = 'X'
        elif board[i][j] == '*':
            board[i][j] = '0'

```

• LC.ca

```

class Solution:
    def solve(self, board: List[List[str]]) -> None:
        """
        Do not return anything, modify board in-place instead.
        """

        def dfs(i, j):
            board[i][j] = '.'
            for a, b in [[0, -1], [0, 1], [1, 0], [-1, 0]]:
                x, y = i + a, j + b

                if 0 <= x < m and 0 <= y < n and board[x][y] == '0':
                    dfs(x, y)

        m, n = len(board), len(board[0])
        for i in range(m):
            for j in range(n):
                if board[i][j] == '0' and (
                    i == 0 or i == m - 1 or j == 0 or j == n - 1
                ):
                    dfs(i, j)

```

Round 2 · High · Medium

p.956

```

for i in range(m):
    for j in range(n):
        if board[i][j] == 'O':
            board[i][j] = 'X'
        elif board[i][j] == '.':
            board[i][j] = 'O'

```

◆ Quick Review

Key Insights

- Regions connected to the border are not captured.
- Use DFS/BFS from border 'O's to mark safe cells.
- After marking, flip all remaining 'O's to 'X' and then revert the marked cells back to 'O'.

Space & Time

Time Complexity: $O(m * n)$

Space Complexity: $O(m * n)$ in the worst case due to recursion stack or queue storage for DFS/BFS

Solution

The solution involves two main steps. First, traverse the board's borders and run a DFS or BFS from any encountered 'O', marking it (e.g., using ' ') to indicate that it should not be

flipped. Next, go through each cell in the board: flip any remaining 'O' (i.e., not marked safe) to 'X' and change the temporary marker ' ' back to 'O'. This approach ensures only the regions completely surrounded by 'X' are captured.

Depth First Search (DFS)

#133 Clone Graph

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · DFS · BFS · Graph
Lists: Grind 169

Problem

Given a reference of a node in a connected undirected graph.

Return a deep copy (clone) of the graph.

Each node in the graph contains a value (int) and a list (List[Node]) of its neighbors.

```
class Node {
public int val;
public List<Node> neighbors;
}
```

Test case format:

For simplicity, each node's value is the same as the node's index (1-indexed). For example, the first node with `val == 1`, the second node with `val == 2`, and so on. The graph is represented in the test case using an adjacency list.

An adjacency list is a collection of unordered lists used to represent a finite graph. Each list

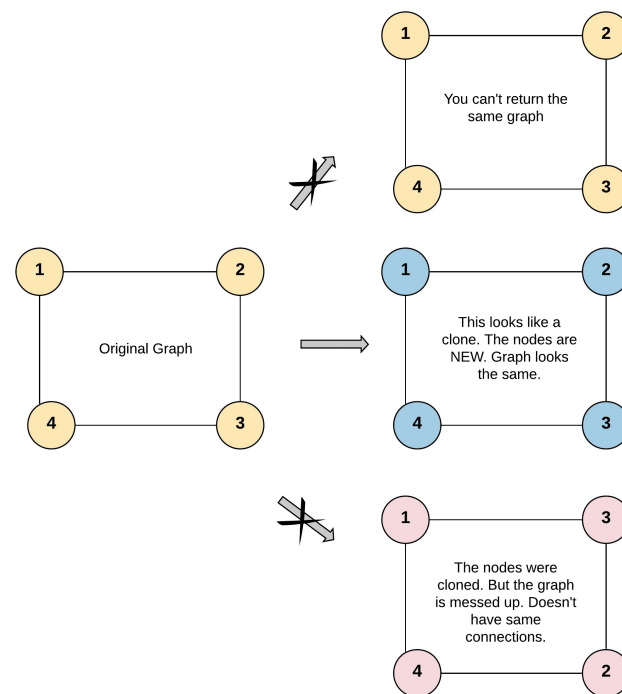
Round 2 · High · Medium

p.959

describes the set of neighbors of a node in the graph.

The given node will always be the first node with `val == 1`. You must return the copy of the given node as a reference to the cloned graph.

Example 1:



Round 2 · High · Medium

p.960

Input: adjList = [[2,4],[1,3],[2,4],[1,3]]

Output: [[2,4],[1,3],[2,4],[1,3]]

Explanation: There are 4 nodes in the graph.

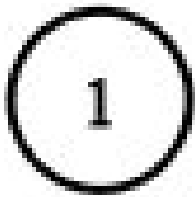
1st node (val = 1)'s neighbors are 2nd node (val = 2) and 4th node (val = 4).

2nd node (val = 2)'s neighbors are 1st node (val = 1) and 3rd node (val = 3).

3rd node (val = 3)'s neighbors are 2nd node (val = 2) and 4th node (val = 4).

4th node (val = 4)'s neighbors are 1st node (val = 1) and 3rd node (val = 3).

Example 2:



Input: adjList = [[]]

Output: [[]]

Explanation: Note that the input contains one empty list. The graph consists of only one node with val = 1 and it does not have any neighbors.

Example 3:

Round 2 · High · Medium

p.961

Input: adjList = []

Output: []

Explanation: This an empty graph, it does not have any nodes.

Constraints:

- The number of nodes in the graph is in the range [0, 100].
- $1 \leq \text{Node.val} \leq 100$
- Node.val is unique for each node.
- There are no repeated edges and no self-loops in the graph.
- The Graph is connected and all nodes can be visited starting from the given node.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def cloneGraph(self, node: 'Node') -> 'Node':
        if not node:
            return None

        q = collections.deque([node])
        map = {node: Node(node.val)}

        while q:
            u = q.popleft()
```

Round 2 · High · Medium

p.962

```

        for v in u.neighbors:
            if v not in map:
                map[v] = Node(v.val)
                q.append(v)
            map[u].neighbors.append(map[v])

        return map[node]

class Solution:
    def cloneGraph(self, node: 'Node') -> 'Node':
        if not node:
            return None
        if node in self.map:
            return self.map[node]

        newNode = Node(node.val, [])
        self.map[node] = newNode

        for neighbor in node.neighbors:
            self.map[node].neighbors.append(self.cloneGraph(neighbor))

        return newNode

map = {}

```

• LeetCode

```

"""
# Definition for a Node.
class Node:
    def __init__(self, val = 0, neighbors = None):
        self.val = val
        self.neighbors = neighbors if neighbors is not None else []
"""

from typing import Optional

```

Round 2 · High · Medium

p.963

```

class Solution:
    def cloneGraph(self, node: Optional["Node"]) -> Optional["Node"]:
        def dfs(node):
            if node is None:
                return None
            if node in g:
                return g[node]
            cloned = Node(node.val)
            g[node] = cloned

            for nxt in node.neighbors:
                cloned.neighbors.append(dfs(nxt))
            return cloned

        g = defaultdict()
        return dfs(node)

```

• SimplyLeet

```

# Definition for a Node.
class Node:
    def __init__(self, val, neighbors=None):
        self.val = val
        # Initialize neighbors with empty list if None is provided
        self.neighbors = neighbors if neighbors is not None else []

    def cloneGraph(node: 'Node') -> 'Node':
        # Hash map to track original nodes and their corresponding clones
        old_to_new = {}

        def dfs(current):
            # Base case: if the current node is None, return None
            if current is None:
                return None

            # If the node has already been cloned, return the clone to avoid
            # duplication and cycle issues
            if current in old_to_new:
                return old_to_new[current]

            # Create a clone for the current node
            copy = Node(current.val)

            for neighbor in current.neighbors:
                copy.neighbors.append(dfs(neighbor))

            old_to_new[current] = copy
            return copy

        return dfs(node)

```

Round 2 · High · Medium

p.964

```

old_to_new[current] = copy
# Recursively clone all neighbors
for neighbor in current.neighbors:
    copy.neighbors.append(dfs(neighbor))
return copy

return dfs(node)

```

• LC.ca

```

"""
# Definition for a Node.
class Node:
    def __init__(self, val = 0, neighbors = None):
        self.val = val
        self.neighbors = neighbors if neighbors is not None else []
"""

class Solution:

    def cloneGraph(self, node: 'Node') -> 'Node':
        visited = defaultdict()

        def clone(node):
            if node is None:
                return None
            if node in visited:
                return visited[node]
            c = Node(node.val)
            visited[node] = c

            for e in node.neighbors:
                c.neighbors.append(clone(e))
            return c

        return clone(node)

```

◆ Quick Review

Key Insights

Round 2 · High · Medium

p.965

- The graph is undirected and connected.
- Each node must be cloned exactly once to prevent cycles and repetitions.
- Use a mapping/dictionary to keep track of cloned nodes.
- A depth-first search (DFS) or breadth-first search (BFS) can be used to traverse and clone the graph.
- Handle edge cases where the graph might be empty.

Space & Time

Time Complexity: $O(N + E)$, where N is the number of nodes and E is the number of edges, because each node and each edge are traversed once.

Space Complexity: $O(N)$, to store the mapping from original nodes to their clones, and for the recursion/queue used in traversal.

Solution

We employ a depth-first search (DFS) approach to clone the graph. The main idea is to create a mapping (hash table) that links each original node to its corresponding cloned node. During the DFS traversal, if a node is

Round 2 · High · Medium

p.966

encountered for the first time, a new node is created and stored in the mapping; then, the algorithm recursively clones all of its neighbors. For an already visited node, the cloned node is directly returned from the mapping. This ensures that every node is cloned only once and correctly handles cycles in the graph.

Depth First Search (DFS)

□ #200 Number of Islands

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · DFS · BFS · Union Find · Matrix
 Lists: Grind 169

Problem

Given an $m \times n$ 2D binary grid which represents a map of '1's (land) and '0's (water), return the number of islands.

An island is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are all surrounded by water.

Example 1:

```
Input: grid = [
  ["1","1","1","1","0"],
  ["1","1","0","1","0"],
  ["1","1","0","0","0"],
  ["0","0","0","0","0"]
]
Output: 1
```

Example 2:


```

Input: grid = [
  ["1","1","0","0","0"],
  ["1","1","0","0","0"],
  ["0","0","1","0","0"],
  ["0","0","0","1","1"]
]
Output: 3

```

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq m, n \leq 300$
- $\text{grid}[i][j]$ is '0' or '1'.

Community Solutions (Python)

• WalkCC

```

class Solution:
    def numIslands(self, grid: list[list[str]]) -> int:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(grid)
        n = len(grid[0])

        def bfs(r, c):
            q = collections.deque([(r, c)])
            grid[r][c] = '2' # Mark '2' as visited.
            while q:

                i, j = q.popleft()
                for dx, dy in DIRS:
                    x = i + dx
                    y = j + dy
                    if x < 0 or x == m or y < 0 or y == n:
                        continue
                    if grid[x][y] != '1':
                        continue
                    q.append((x, y))
                    grid[x][y] = '2' # Mark '2' as visited.

```

Round 2 · High · Medium

p.969

```

ans = 0

for i in range(m):
    for j in range(n):
        if grid[i][j] == '1':
            bfs(i, j)
            ans += 1

return ans

class Solution:
    def numIslands(self, grid: list[list[str]]) -> int:
        m = len(grid)
        n = len(grid[0])

        def dfs(i: int, j: int) -> None:
            if i < 0 or i == m or j < 0 or j == n:
                return
            if grid[i][j] != '1':
                return

            grid[i][j] = '2' # Mark '2' as visited.
            dfs(i + 1, j)
            dfs(i - 1, j)
            dfs(i, j + 1)
            dfs(i, j - 1)

ans = 0

for i in range(m):

    for j in range(n):
        if grid[i][j] == '1':
            dfs(i, j)
            ans += 1

return ans

```

• LeetCode

Round 2 · High · Medium

p.970

```

class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        def dfs(i, j):
            grid[i][j] = '0'
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                if 0 <= x < m and 0 <= y < n and grid[x][y] == '1':
                    dfs(x, y)

        ans = 0

        dirs = (-1, 0, 1, 0, -1)
        m, n = len(grid), len(grid[0])
        for i in range(m):
            for j in range(n):
                if grid[i][j] == '1':
                    dfs(i, j)
                    ans += 1
        return ans

class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        def bfs(i, j):
            grid[i][j] = '0'
            q = deque([(i, j)])
            while q:
                i, j = q.popleft()
                for a, b in pairwise(dirs):
                    x, y = i + a, j + b
                    if 0 <= x < m and 0 <= y < n and grid[x][y] == '1':
                        q.append((x, y))
                        grid[x][y] = 0

        ans = 0
        dirs = (-1, 0, 1, 0, -1)
        m, n = len(grid), len(grid[0])
        for i in range(m):
            for j in range(n):
                if grid[i][j] == '1':
                    bfs(i, j)

```

Round 2 · High · Medium

p.971

```

        ans += 1
        return ans

class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        def find(x):
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        dirs = (0, 1, 0)
        m, n = len(grid), len(grid[0])
        p = list(range(m * n))

        for i in range(m):
            for j in range(n):
                if grid[i][j] == '1':
                    for a, b in pairwise(dirs):
                        x, y = i + a, j + b
                        if x < m and y < n and grid[x][y] == '1':
                            p[find(i * n + j)] = find(x * n + y)

        return sum(
            grid[i][j] == '1' and i * n + j == find(i * n + j)
            for i in range(m)
            for j in range(n)
        )

```

• SimplyLeet

```

# Function to count the number of islands in the grid
def numIslands(grid):
    # If the grid is empty, return 0
    if not grid:
        return 0
    rows, cols = len(grid), len(grid[0])
    islands = 0

    # Depth-First Search helper to mark the entire island as visited
    def dfs(r, c):

```

Round 2 · High · Medium

p.972

```
# Check if the current cell is out of bounds or water
if r < 0 or r >= rows or c < 0 or c >= cols or grid[r][c] == "0":
    return

# Mark the cell as visited by setting it to water
grid[r][c] = "0"
# Explore all four adjacent cells (up, down, left, right)
dfs(r + 1, c)
dfs(r - 1, c)
dfs(r, c + 1)
dfs(r, c - 1)
```

```
# Iterate over every cell in the grid
for r in range(rows):
    for c in range(cols):
        # If a land cell is found, conduct DFS to visit the entire island
        if grid[r][c] == "1":
            islands += 1
            dfs(r, c)

return islands
```

• LC.ca

```
# dfs
class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        def dfs(i, j):
            if not (0 <= i < m and 0 <= j < n and grid[i][j] == '1'):
                return
            grid[i][j] = '0'
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                dfs(x, y)
```

Round 2 · High · Medium

p.973

```
ans = 0
dirs = (-1, 0, 1, 0, -1)
m, n = len(grid), len(grid[0])
for i in range(m):
    for j in range(n):
        if grid[i][j] == '1':
            dfs(i, j)
            ans += 1
return ans
```

```
#####
```

```
# bfs
from collections import deque

class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        if not grid:
            return 0
```

```
m, n = len(grid), len(grid[0])
directions = [(0, 1), (0, -1), (1, 0), (-1, 0)]
islands = 0

for i in range(m):
    for j in range(n):
        if grid[i][j] == "1":
            islands += 1
            q = deque([(i, j)]) # no need to reset q, q already drained
            from previous bfs
```

Round 2 · High · Medium

p.974

```

        while q:
            x, y = q.popleft()
            for dx, dy in directions:
                nx, ny = x + dx, y + dy
                if 0 <= nx < m and 0 <= ny < n and grid[nx][ny] ==
"1":
                    q.append((nx, ny))
                    grid[nx][ny] = "0" # mark as visited

        return islands

#####

# union find
# similar to UF in https://leetcode.ca/2016-09-30-305-Number-of-Islands-II/
class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        def check(i, j):
            return 0 <= i < m and 0 <= j < n and grid[i][j] == "1" # Check
for "1" instead of 1

        def find(x):
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

m, n = len(grid), len(grid[0])
p = list(range(m * n))
cur = 0 # Initialize cur as 0 instead of n
for i in range(m):
    for j in range(n):
        if grid[i][j] == "1":
            cur += 1 # Increment cur when encountering "1"

```

Round 2 · High · Medium

p.975

```

        for x, y in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
            if check(i + x, j + y) and find(i * n + j) != find((i
+ x) * n + j + y):
                p[find(i * n + j)] = find((i + x) * n + j + y)
                cur -= 1

        return cur

#####

'''
>>> a = set()
>>>
>>> a |= {(1, 1)}
>>> a
{(1, 1)}
>>>
>>> a |= {(2, 2)}
>>> a
{(1, 1), (2, 2)}

>>>
>>> a.add((3, 3))
>>> a
{(1, 1), (3, 3), (2, 2)}
>>>
>>> (1,1) in a
True
>>> (10,10) in a
False
'''

```

Round 2 · High · Medium

p.976

```

class Solution(object):
    def numIslands(self, grid):
        """
        :type grid: List[List[str]]
        :rtype: int
        """
        visited = set()
        ans = 0

        def dfs(grid, i, j, visited):

            if i < 0 or i >= len(grid) or j < 0 or j >= len(grid[0]) or grid[i][j] =
            = "0" or (i, j) in visited:
                return False
            visited |= {(i, j)}
            for di, dj in [(-1, 0), (1, 0), (0, 1), (0, -1)]:
                newi, newj = i + di, j + dj
                dfs(grid, newi, newj, visited)
            return True

        for i in range(0, len(grid)):
            for j in range(0, len(grid[0])):

                if dfs(grid, i, j, visited):
                    ans += 1
        return ans

```

◆ Quick Review

Key Insights

- Use graph traversal (DFS/BFS) to explore connected components of land.
- Iterate over the matrix and start a traversal (e.g., DFS) when encountering an unvisited "1".
- During traversal, mark visited cells to avoid duplicate counting.

Round 2 · High · Medium

p.977

- Each DFS/BFS call completely explores one island.
- Alternative approach can use Union Find to group connected lands.

Space & Time

Time Complexity: $O(m * n)$, as each cell is visited at most once.

Space Complexity: $O(m * n)$ in the worst-case scenario due to recursion stack (DFS) or auxiliary data structures.

Solution

We solve the problem using a DFS-based approach. The algorithm iterates over each cell in the grid; when a "1" (land) is encountered, the DFS is initiated to mark the entire island (all connected "1"s) as visited (by setting them to "0"). This prevents recounting the same island. The island count is incremented each time a new island is discovered and fully explored.

Round 2 · High · Medium

p.978

Depth First Search (DFS)

#417 Pacific Atlantic Water Flow

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · DFS · BFS · Matrix
 Lists: Grind 169

Problem

There is an $m \times n$ rectangular island that borders both the Pacific Ocean and Atlantic Ocean. The Pacific Ocean touches the island's left and top edges, and the Atlantic Ocean touches the island's right and bottom edges. The island is partitioned into a grid of square cells. You are given an $m \times n$ integer matrix heights where heights[r][c] represents the height above sea level of the cell at coordinate (r, c).

The island receives a lot of rain, and the rain water can flow to neighboring cells directly north, south, east, and west if the neighboring cell's height is less than or equal to the current cell's height. Water can flow from any cell adjacent to an ocean into the ocean.

Round 2 · High · Medium

p.979

Return a 2D list of grid coordinates result where result[i] = [ri, ci] denotes that rain water can flow from cell (ri, ci) to both the Pacific and Atlantic oceans.

Example 1:

Pacific Ocean					
Pacific Ocean	1	2	2	3	5
	3	2	3	4	4
	2	4	5	3	1
	6	7	1	4	5
	5	1	1	2	4
Atlantic Ocean					

Round 2 · High · Medium

p.980

Input: heights = [[1,2,2,3,5],[3,2,3,4,4],[2,4,5,3,1],[6,7,1,4,5],[5,1,1,2,4]]
 Output: [[0,4],[1,3],[1,4],[2,2],[3,0],[3,1],[4,0]]
 Explanation: The following cells can flow to the Pacific and Atlantic oceans, as shown below:
 [0,4]: [0,4] -> Pacific Ocean
 [0,4] -> Atlantic Ocean
 [1,3]: [1,3] -> [0,3] -> Pacific Ocean
 [1,3] -> [1,4] -> Atlantic Ocean
 [1,4]: [1,4] -> [1,3] -> [0,3] -> Pacific Ocean

Example 2:

Input: heights = [[1]]
 Output: [[0,0]]
 Explanation: The water can flow from the only cell to the Pacific and Atlantic oceans.

Constraints:

- $m == \text{heights.length}$
- $n == \text{heights[r].length}$
- $1 \leq m, n \leq 200$
- $0 \leq \text{heights[r][c]} \leq 105$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def pacificAtlantic(self, heights: list[list[int]]) -> list[list[int]]:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(heights)
        n = len(heights[0])
        qP = collections.deque()
        qA = collections.deque()
        seenP = [[False] * n for _ in range(m)]
        seenA = [[False] * n for _ in range(m)]
```

```
for i in range(m):
    qP.append((i, 0))
    qA.append((i, n - 1))
    seenP[i][0] = True
    seenA[i][n - 1] = True
```

```
for j in range(n):
    qP.append((0, j))
    qA.append((m - 1, j))
    seenP[0][j] = True
```

```
seenA[m - 1][j] = True
```

```
def bfs(q: deque, seen: list[list[bool]]):
    while q:
        i, j = q.popleft()
        h = heights[i][j]
        for dx, dy in DIRS:
            x = i + dx
            y = j + dy
            if x < 0 or x == m or y < 0 or y == n:
```

```
                continue
            if seen[x][y] or heights[x][y] < h:
                continue
            q.append((x, y))
            seen[x][y] = True
```

```
bfs(qP, seenP)
bfs(qA, seenA)
```

```
return [[i, j] for i in range(m) for j in range(n) if seenP[i][j] and seenA[i][j]]
```

```

class Solution:
    def pacificAtlantic(self, heights: list[list[int]]) -> list[list[int]]:
        m = len(heights)
        n = len(heights[0])
        seenP = [[False] * n for _ in range(m)]
        seenA = [[False] * n for _ in range(m)]

        def dfs(i: int, j: int, h: int, seen: list[list[bool]]) -> None:
            if i < 0 or i == m or j < 0 or j == n:
                return

            if seen[i][j] or heights[i][j] < h:
                return

            seen[i][j] = True
            dfs(i + 1, j, heights[i][j], seen)
            dfs(i - 1, j, heights[i][j], seen)
            dfs(i, j + 1, heights[i][j], seen)
            dfs(i, j - 1, heights[i][j], seen)

        for i in range(m):
            dfs(i, 0, 0, seenP)
            dfs(i, n - 1, 0, seenA)

        for j in range(n):
            dfs(0, j, 0, seenP)
            dfs(m - 1, j, 0, seenA)

        return [[i, j]
                for i in range(m)
                for j in range(n)

                if seenP[i][j] and seenA[i][j]]

```

• LeetDoocs

Round 2 · High · Medium

p.983

```

class Solution:
    def pacificAtlantic(self, heights: List[List[int]]) -> List[List[int]]:
        def bfs(q: Deque[Tuple[int, int]], vis: List[List[bool]]) -> None:
            while q:
                x, y = q.popleft()
                for dx, dy in pairwise(dirs):
                    nx, ny = x + dx, y + dy
                    if (
                        0 <= nx < m
                        and 0 <= ny < n
                        and not vis[nx][ny]
                        and heights[nx][ny] >= heights[x][y]
                    ):
                        vis[nx][ny] = True
                        q.append((nx, ny))

        m, n = len(heights), len(heights[0])
        vis1 = [[False] * n for _ in range(m)]
        vis2 = [[False] * n for _ in range(m)]
        q1: Deque[Tuple[int, int]] = deque()

        q2: Deque[Tuple[int, int]] = deque()
        dirs = (-1, 0, 1, 0, -1)

        for i in range(m):
            q1.append((i, 0))
            vis1[i][0] = True
            q2.append((i, n - 1))
            vis2[i][n - 1] = True

        for j in range(n):
            q1.append((0, j))
            vis1[0][j] = True
            q2.append((m - 1, j))
            vis2[m - 1][j] = True

        bfs(q1, vis1)
        bfs(q2, vis2)

        return [(i, j) for i in range(m) for j in range(n) if vis1[i][j] and vis2[i][j]]

```

Round 2 · High · Medium

p.984

• SimplyLeet

```
# Python solution for Pacific Atlantic Water Flow

class Solution:
    def pacificAtlantic(self, heights: List[List[int]]) -> List[List[int]]:
        if not heights or not heights[0]:
            return []

        rows, cols = len(heights), len(heights[0])
        pacific_reachable = [[False for _ in range(cols)] for _ in range(rows)]
        atlantic_reachable = [[False for _ in range(cols)] for _ in range(rows)]

        # Directions: up, down, left, right
        directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

        # DFS function to mark reachable cells
        def dfs(r, c, reachable):
            reachable[r][c] = True
            for dr, dc in directions:
                new_r, new_c = r + dr, c + dc
                # Check bounds and conditions to flow water
                if (0 <= new_r < rows and 0 <= new_c < cols and
                    not reachable[new_r][new_c] and
                    heights[new_r][new_c] >= heights[r][c]):
                    dfs(new_r, new_c, reachable)

        # DFS from Pacific ocean borders (top row and left column)
        for i in range(rows):
            dfs(i, 0, pacific_reachable)
        for j in range(cols):
            dfs(0, j, pacific_reachable)
```

Round 2 · High · Medium

p.985

• LC.ca

```
class Solution:
    def pacificAtlantic(self, heights: List[List[int]]) -> List[List[int]]:
        def bfs(q, vis):
            while q:
                for _ in range(len(q)):
                    i, j = q.popleft()
                    for a, b in [[0, -1], [0, 1], [1, 0], [-1, 0]]:
                        x, y = i + a, j + b
                        if (
                            0 <= x < m
                            and 0 <= y < n
                            and (x, y) not in vis
                            and heights[x][y] >= heights[i][j]
                        ):
                            vis.add((x, y))
                            q.append((x, y))

        m, n = len(heights), len(heights[0])
        vis1, vis2 = set(), set()
        q1 = deque()
```

Round 2 · High · Medium

p.986

```

q2 = deque()
for i in range(m):
    for j in range(n):
        if i == 0 or j == 0:
            vis1.add((i, j))
            q1.append((i, j))
        if i == m - 1 or j == n - 1:
            vis2.add((i, j))
            q2.append((i, j))
bfs(q1, vis1)

bfs(q2, vis2)
return [
    (i, j)
    for i in range(m)
    for j in range(n)
    if (i, j) in vis1 and (i, j) in vis2
]

```

◆ Quick Review

Key Insights

- Reverse the typical DFS/BFS perspective: start from the oceans and work inward.
- Maintain two matrices (or sets) to keep track of cells reachable from the Pacific and Atlantic edges.
- A cell can be added to the result if it is reached in both traversals.
- The height constraint ensures water flows only in non-increasing order.

Space & Time

Round 2 · High · Medium

p.987

Time Complexity: $O(m * n)$ where m is the number of rows and n is the number of columns. Each cell is processed up to twice (once from each ocean).

Space Complexity: $O(m * n)$ for the visited matrices and the recursion/queue used in DFS/BFS.

Solution

The solution uses depth-first search (DFS) from the ocean boundaries. We initialize two boolean matrices, one for each ocean. For the Pacific, we start DFS from the leftmost column and top row; for the Atlantic, we start from the rightmost column and bottom row. In each DFS, if the next cell has a height equal to or greater than the current cell and hasn't been visited, we continue the search. Finally, we scan through the grid to collect cells that are reachable from both oceans. This reverse search simplifies the path validations and leverages the allowed water flow conditions.

Round 2 · High · Medium

p.988

Depth First Search (DFS)

#690 Employee Importance

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table · Tree · Depth-First Search
 · Breadth-First Search

Lists: AlgoMaster 600

Problem

You have a data structure of employee information, including the employee's unique ID, importance value, and direct subordinates' IDs. You are given an array of employees `employees` where:

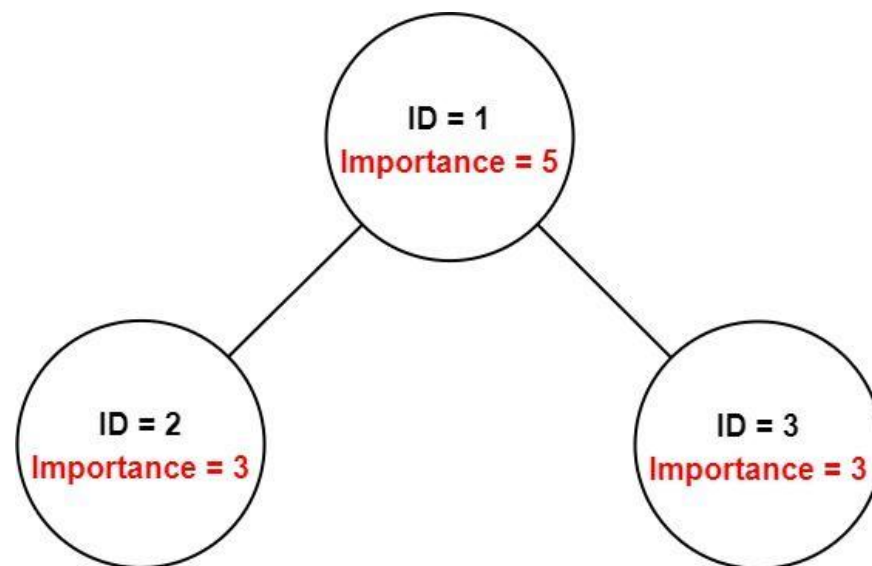
- `employees[i].id` is the ID of the *i*th employee.
- `employees[i].importance` is the importance value of the *i*th employee.
- `employees[i].subordinates` is a list of the IDs of the direct subordinates of the *i*th employee.

Given an integer `id` that represents an employee's ID, return the total importance value of this employee and all their direct and indirect subordinates.

Round 2 · High · Medium

p.989

Example 1:

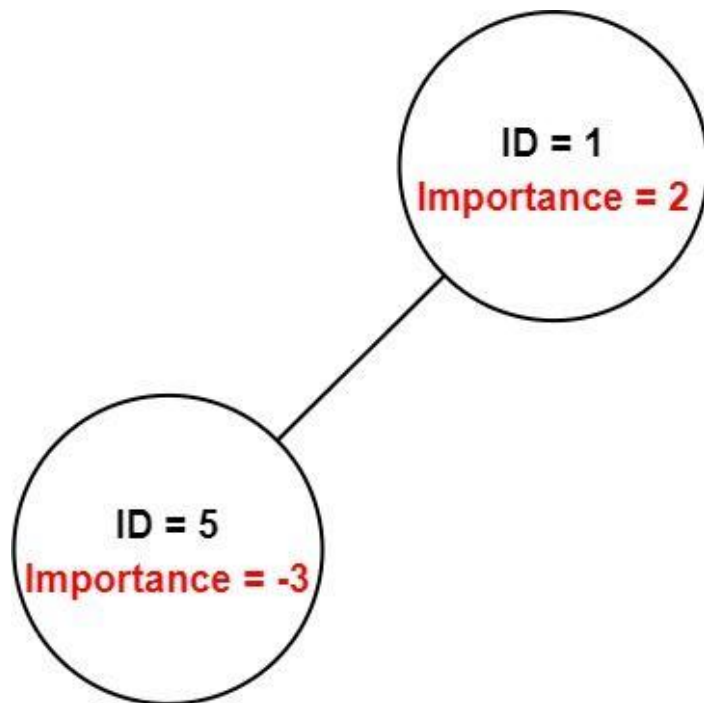


Input: `employees = [[1,5,[2,3]],[2,3,[]],[3,3,[]]]`, `id = 1`
 Output: 11
 Explanation: Employee 1 has an importance value of 5 and has two direct subordinates: employee 2 and employee 3. They both have an importance value of 3. Thus, the total importance value of employee 1 is $5 + 3 + 3 = 11$.

Example 2:

Round 2 · High · Medium

p.990



Input: employees = [[1,2,[5]], [5,-3,[]]], id = 5
 Output: -3
 Explanation: Employee 5 has an importance value of -3 and has no direct subordinates.
 Thus, the total importance value of employee 5 is -3.

Constraints:

- $1 \leq \text{employees.length} \leq 2000$
- $1 \leq \text{employees}[i].\text{id} \leq 2000$
- All $\text{employees}[i].\text{id}$ are unique.
- $-100 \leq \text{employees}[i].\text{importance} \leq 100$

- One employee has at most one direct leader and may have several subordinates.
- The IDs in $\text{employees}[i].\text{subordinates}$ are valid IDs.

Community Solutions (Python)

• WalkCC

```

class Solution:
    def getImportance(self, employees: List['Employee'], id: int) -> int:
        idToEmployee = {employee.id: employee for employee in employees}

        def dfs(id: int) -> int:
            values = idToEmployee[id].importance
            for subId in idToEmployee[id].subordinates:
                values += dfs(subId)
            return values

        return dfs(id)
  
```

• LeetDoocs

```

"""
# Definition for Employee.
class Employee:
    def __init__(self, id: int, importance: int, subordinates: List[int]):
        self.id = id
        self.importance = importance
        self.subordinates = subordinates
"""
  
```

```
class Solution:
    def getImportance(self, employees: List["Employee"], id: int) -> int:
        def dfs(i: int) -> int:
            return d[i].importance + sum(dfs(j) for j in d[i].subordinates)

        d = {e.id: e for e in employees}
        return dfs(id)
```

• SimplyLeet

```
# Define the Employee class (for context)
class Employee:
    def __init__(self, id, importance, subordinates):
        self.id = id # Unique employee ID
        self.importance = importance # Importance value of the employee
        self.subordinates = subordinates # List of subordinate IDs

def getImportance(employees, id):
    # Create a mapping from employee id to the employee object for quick lookup
    employee_map = {employee.id: employee for employee in employees}

    # Define a recursive DFS function
    def dfs(emp_id):
        current = employee_map[emp_id] # Get current employee object using
        employee_map
        total = current.importance # Start with current employee's
        importance

        # Recursively add importance from all direct subordinates
        for sub_id in current.subordinates:
            total += dfs(sub_id) # Add the importance from subordinate
        subtree

        return total

    return dfs(id) # Initiate DFS from the given employee id

# Example usage:
# employees = [Employee(1, 5, [2,3]), Employee(2, 3, []), Employee(3, 3, [])]
# print(getImportance(employees, 1)) # Output: 11
```

Depth First Search (DFS)

★ #694 Number of Distinct Islands ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · DFS · BFS · Union Find · Hash
Function

Lists: ★ Premium | Premium 98

Problem

You are given an $m \times n$ binary matrix grid. An island is a group of 1's (representing land) connected 4-directionally (horizontal or vertical.) You may assume all four edges of the grid are surrounded by water.

An island is considered to be the same as another if and only if one island can be translated (and not rotated or reflected) to equal the other.

Return the number of distinct islands.

Example 1:

1	1	0	0	0
1	1	0	0	0
0	0	0	1	1
0	0	0	1	1

Input: grid = [[1,1,0,0,0],[1,1,0,0,0],[0,0,0,1,1],[0,0,0,1,1]]
Output: 1

Example 2:

Round 2 · High · Medium

p.995

1	1	0	1	1
1	0	0	0	0
0	0	0	0	1
1	1	0	1	1

Input: grid = [[1,1,0,1,1],[1,0,0,0,0],[0,0,0,0,1],[1,1,0,1,1]]
Output: 3

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq m, n \leq 50$
- $\text{grid}[i][j]$ is either 0 or 1.

Community Solutions (Python)

- WalkCC

Round 2 · High · Medium

p.996

```
class Solution:
    def numDistinctIslands(self, grid: list[list[int]]) -> int:
        seen = set()

        def dfs(i: int, j: int, i0: int, j0: int):
            if i < 0 or i == len(grid) or j < 0 or j == len(grid[0]):
                return
            if grid[i][j] == 0 or (i, j) in seen:
                return

            seen.add((i, j))
            island.append((i - i0, j - j0))
            dfs(i + 1, j, i0, j0)
            dfs(i - 1, j, i0, j0)
            dfs(i, j + 1, i0, j0)
            dfs(i, j - 1, i0, j0)

        islands = set() # all the different islands

        for i in range(len(grid)):

            for j in range(len(grid[0])):
                island = []
                dfs(i, j, i, j)
                if island:
                    islands.add(frozenset(island))

        return len(islands)
```

• LeetCode

```
class Solution:
    def numDistinctIslands(self, grid: List[List[int]]) -> int:
        def dfs(i: int, j: int, k: int):
            grid[i][j] = 0
            path.append(str(k))
            dirs = (-1, 0, 1, 0, -1)
            for h in range(1, 5):
                x, y = i + dirs[h - 1], j + dirs[h]
                if 0 <= x < m and 0 <= y < n and grid[x][y]:
                    dfs(x, y, h)
```

Round 2 · High · Medium

p.997

```
        path.append(str(-k))

        paths = set()
        path = []
        m, n = len(grid), len(grid[0])
        for i, row in enumerate(grid):
            for j, x in enumerate(row):
                if x:
                    dfs(i, j, 0)
                    paths.add("".join(path))

        path.clear()
        return len(paths)
```

• SimplyLeet

```
def numDistinctIslands(grid):
    # Get the number of rows and columns
    rows, cols = len(grid), len(grid[0])
    # Set to store unique island shapes
    distinct_shapes = set()

    # DFS function to explore the island and record its relative shape
    def dfs(r, c, origin_r, origin_c, shape):
        # Check boundaries and if the cell is unvisited land
        if r < 0 or c < 0 or r >= rows or c >= cols or grid[r][c] == 0:

            return

        # Mark the cell as visited by setting it to 0
        grid[r][c] = 0
        # Record the relative position from the origin of this island
        shape.append((r - origin_r, c - origin_c))
        # Explore all 4-directionally adjacent cells
        dfs(r + 1, c, origin_r, origin_c, shape)
        dfs(r - 1, c, origin_r, origin_c, shape)
        dfs(r, c + 1, origin_r, origin_c, shape)
        dfs(r, c - 1, origin_r, origin_c, shape)
```

Round 2 · High · Medium

p.998

```

# Iterate over every cell in the grid
for i in range(rows):
    for j in range(cols):
        if grid[i][j] == 1:
            shape = []
            dfs(i, j, i, j, shape)
            # Convert the shape list to a tuple of sorted coordinates for
            canonical form
            distinct_shapes.add(tuple(sorted(shape)))

# The number of distinct islands is the size of the set
return len(distinct_shapes)

# Example usage:
grid = [[1,1,0,0,0],
        [1,1,0,0,0],
        [0,0,0,1,1],
        [0,0,0,1,1]]
print(numDistinctIslands(grid)) # Output: 1

```

• LC.ca

```

class Solution:
    def numDistinctIslands(self, grid: List[List[int]]) -> int:
        def dfs(i: int, j: int, k: int):
            grid[i][j] = 0
            path.append(str(k))
            dirs = (-1, 0, 1, 0, -1)
            for h in range(1, 5):
                x, y = i + dirs[h - 1], j + dirs[h]
                if 0 <= x < m and 0 <= y < n and grid[x][y]:
                    dfs(x, y, h)

```

Round 2 · High · Medium

p.999

```

        path.append(str(-k))

        paths = set()
        path = []
        m, n = len(grid), len(grid[0])
        for i, row in enumerate(grid):
            for j, x in enumerate(row):
                if x:
                    dfs(i, j, 0)
                    paths.add("".join(path))

        path.clear()
        return len(paths)

```

◆ Quick Review

Key Insights

- Use DFS/BFS to traverse and mark each island.
- For each island, record its shape relative to a starting coordinate to allow translation invariance.
- Represent the island's shape as a unique signature (e.g., tuple or string of relative positions).
- Use a set (or hash table) to store unique shapes.

Space & Time

Time Complexity: $O(m * n)$ as each cell is visited once.

Round 2 · High · Medium

p.1000

Space Complexity: $O(m * n)$ in the worst-case due to recursion stack and storing island shapes.

Solution

Use depth-first search (DFS) to explore the grid. When an unvisited land cell (1) is encountered, initiate a DFS to traverse the entire island. Record each land cell's coordinates relative to the starting position of the island. This relative representation is invariant under translation, making it possible to compare island shapes. Store the canonical shape signature (for example, as a tuple or string) in a set to ensure uniqueness. Finally, return the size of the set as the number of distinct islands.

Round 2 · High · Medium

p.1001

Depth First Search (DFS)

□ #785 Is Graph Bipartite?

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
DFS · BFS · Union Find · Graph
Lists: Denny Zhang

Problem

There is an undirected graph with n nodes, where each node is numbered between 0 and $n - 1$. You are given a 2D array `graph`, where `graph[u]` is an array of nodes that node u is adjacent to. More formally, for each v in `graph[u]`, there is an undirected edge between node u and node v . The graph has the following properties:

- There are no self-edges (`graph[u]` does not contain u).
- There are no parallel edges (`graph[u]` does not contain duplicate values).
- If v is in `graph[u]`, then u is in `graph[v]` (the graph is undirected).

Round 2 · High · Medium

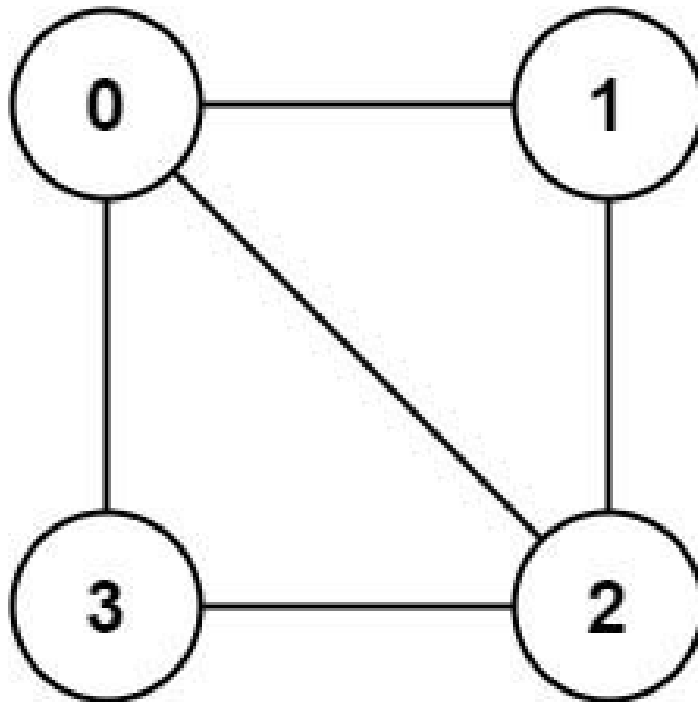
p.1002

- The graph may not be connected, meaning there may be two nodes u and v such that there is no path between them.

A graph is bipartite if the nodes can be partitioned into two independent sets A and B such that every edge in the graph connects a node in set A and a node in set B .

Return true if and only if it is bipartite.

Example 1:



Round 2 · High · Medium

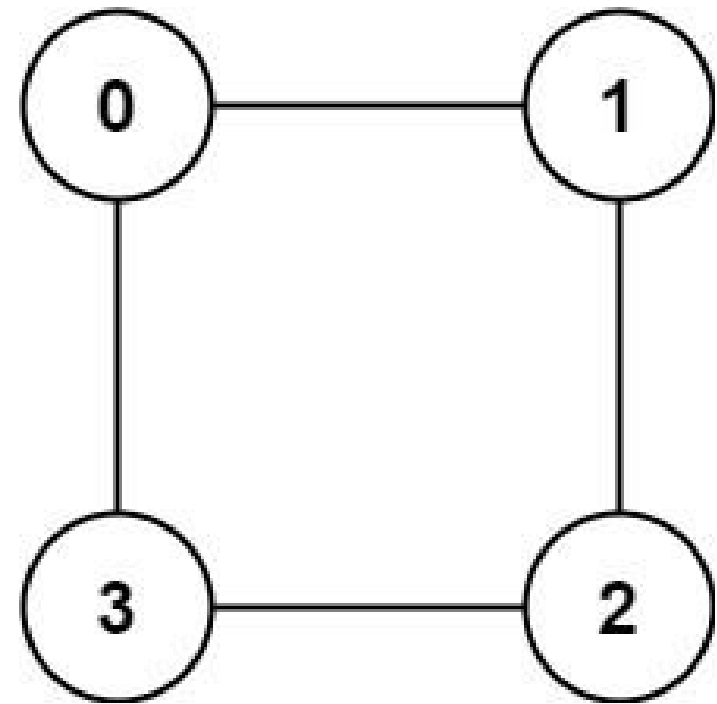
p.1003

Input: graph = [[1,2,3],[0,2],[0,1,3],[0,2]]

Output: false

Explanation: There is no way to partition the nodes into two independent sets such that every edge connects a node in one and a node in the other.

Example 2:



Input: graph = [[1,3],[0,2],[1,3],[0,2]]

Output: true

Explanation: We can partition the nodes into two sets: {0, 2} and {1, 3}.

Constraints:

- graph.length == n
- $1 \leq n \leq 100$

Round 2 · High · Medium

p.1004

- $0 \leq \text{graph}[u].\text{length} < n$
- $0 \leq \text{graph}[u][i] < n - 1$
- $\text{graph}[u]$ does not contain u .
- All the values of $\text{graph}[u]$ are unique.
- If $\text{graph}[u]$ contains v , then $\text{graph}[v]$ contains u .

Community Solutions (Python)

• WalkCC

```
from enum import Enum

class Color(Enum):
    WHITE = 0
    RED = 1
    GREEN = 2

class Solution:

    def isBipartite(self, graph: List[List[int]]) -> bool:
        colors = [Color.WHITE] * len(graph)

        for i in range(len(graph)):
            # This node has been colored, so do nothing.
            if colors[i] != Color.WHITE:
                continue
            # Always paint red for a white node.
            colors[i] = Color.RED
            # BFS.
```

Round 2 · High · Medium

p.1005

```
q = collections.deque([i])
while q:
    for _ in range(len(q)):
        u = q.popleft()
        for v in graph[u]:
            if colors[v] == colors[u]:
                return False
            if colors[v] == Color.WHITE:
                colors[v] = Color.RED if colors[u] == Color.GREEN else Color.GRE
        q.append(v)

return True
```

• LeetCode

```
class Solution:
    def isBipartite(self, graph: List[List[int]]) -> bool:
        def dfs(a: int, c: int) -> bool:
            color[a] = c
            for b in graph[a]:
                if color[b] == c or (color[b] == 0 and not dfs(b, -c)):
                    return False
            return True

        n = len(graph)

        color = [0] * n
        for i in range(n):
            if color[i] == 0 and not dfs(i, 1):
                return False
        return True
```

Round 2 · High · Medium

p.1006

```
class Solution:
    def isBipartite(self, graph: List[List[int]]) -> bool:
        def find(x: int) -> int:
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]
```

```
p = list(range(len(graph)))
for a, bs in enumerate(graph):
    for b in bs:
```

```
        pa, pb = find(a), find(b)
        if pa == pb:
            return False
        p[pb] = find(bs[0])
    return True
```

• SimplyLeet

```
# Using Breadth-First Search (BFS)
from collections import deque
```

```
def isBipartite(graph):
    n = len(graph)
    # color array: -1 means uncolored, 0 and 1 are the two colors.
    colors = [-1] * n
```

```
# Iterate through each node to handle disconnected components
for i in range(n):
```

```
    if colors[i] == -1:
        # Start BFS from node i with color 0
        queue = deque([i])
        colors[i] = 0
        while queue:
            node = queue.popleft()
            # Check each neighbor of the current node
            for neighbor in graph[node]:
                if colors[neighbor] == -1:
                    # Assign the opposite color to the neighbor
```

Round 2 · High · Medium

p.1007

```
        colors[neighbor] = 1 - colors[node]
        queue.append(neighbor)
    elif colors[neighbor] == colors[node]:
        # Found a neighbor with the same color: not bipartite
        return False
```

```
    return True
```

```
# Example usage:
if __name__ == '__main__':
    graph_example = [[1,3],[0,2],[1,3],[0,2]]
```

```
print(isBipartite(graph_example)) # Expected output: True
```

• LC.ca

```
class Solution:
    def isBipartite(self, graph: List[List[int]]) -> bool:
        def dfs(u, c):
            color[u] = c
            for v in graph[u]:
                if not color[v]:
                    if not dfs(v, 3 - c):
                        return False
                elif color[v] == c:
                    return False
```

```
        return True
```

```
n = len(graph)
color = [0] * n
for i in range(n):
    if not color[i] and not dfs(i, 1):
        return False
return True
```

◆ Quick Review

Key Insights

Round 2 · High · Medium

p.1008

- A graph is bipartite if you can assign two colors to each node such that no two adjacent nodes share the same color.
- The challenge may require exploring disconnected components.
- BFS or DFS can be used to attempt the two-coloring, and a conflict during coloring indicates the graph is not bipartite.
- Union Find is an alternative approach, but BFS/DFS is more intuitive for graph coloring problems.

Space & Time

Time Complexity: $O(V + E)$ where V is the number of nodes and E is the number of edges, since every node and edge is traversed at most once.

Space Complexity: $O(V)$, for the color array and the queue (or recursion stack) used in BFS (or DFS).

Solution

We use a BFS/DFS approach to assign colors (0 and 1) to nodes. A color array, initially set to -1 for all nodes, is maintained. For each unvisited node (color -1), we start a BFS (or

Round 2 · High · Medium

p.1009

DFS) and assign it an initial color (e.g., 0). For every visited neighbor, if it has not been colored, assign it the opposite color; if it is already colored with the same color as the current node, a conflict has occurred, and the graph is not bipartite. This method gracefully handles disconnected components by iterating over all nodes.

Round 2 · High · Medium

p.1010

Depth First Search (DFS)

□ #797 All Paths From Source to Target

MED

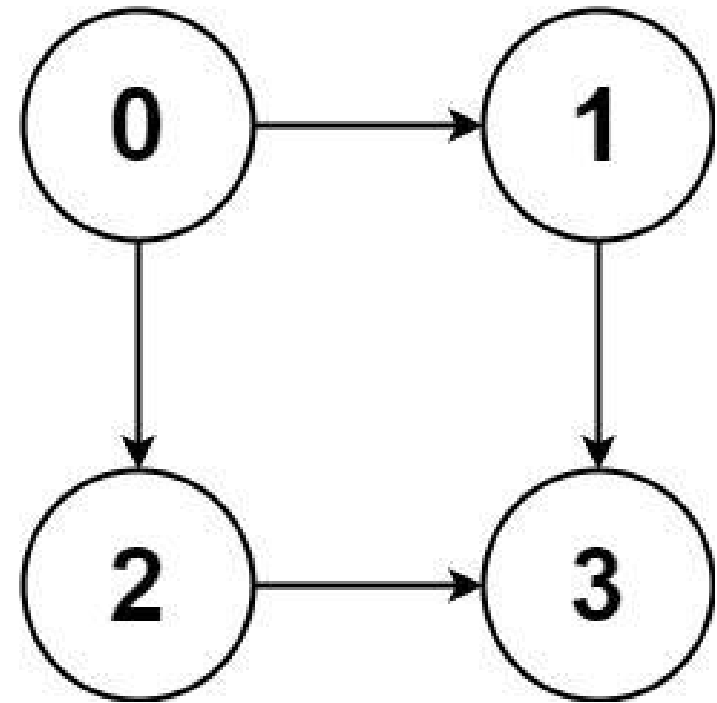
LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Backtracking · Depth-First Search ·
 Breadth-First Search · Graph Theory
 Lists: AlgoMaster 600

Problem

Given a directed acyclic graph (DAG) of n nodes labeled from 0 to $n - 1$, find all possible paths from node 0 to node $n - 1$ and return them in any order.

The graph is given as follows: $\text{graph}[i]$ is a list of all nodes you can visit from node i (i.e., there is a directed edge from node i to node $\text{graph}[i][j]$).

Example 1:

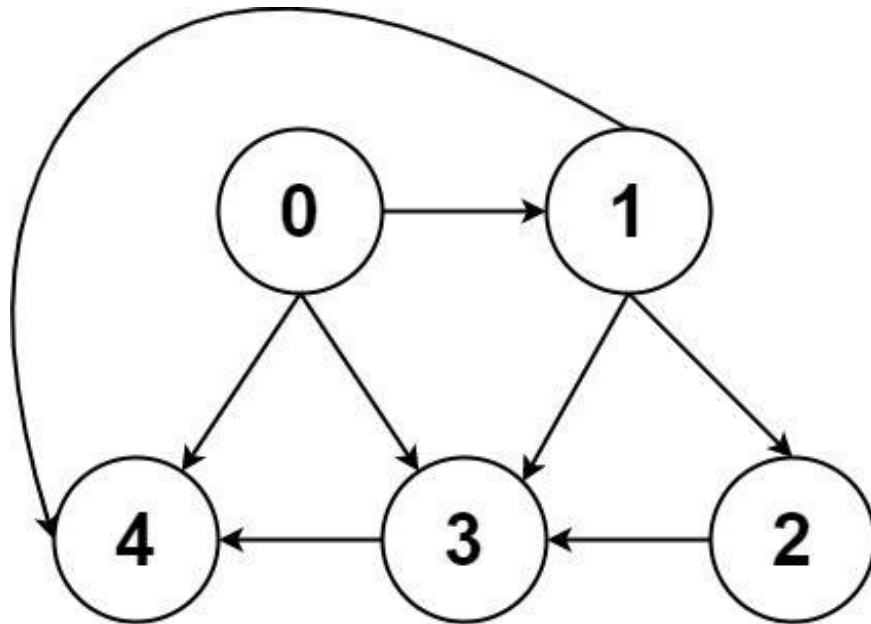


Input: $\text{graph} = [[1,2],[3],[3],[]]$

Output: $[[0,1,3],[0,2,3]]$

Explanation: There are two paths: $0 \rightarrow 1 \rightarrow 3$ and $0 \rightarrow 2 \rightarrow 3$.

Example 2:



Input: graph = [[4,3,1],[3,2,4],[3],[4],[]]
 Output: [[0,4],[0,3,4],[0,1,3,4],[0,1,2,3,4],[0,1,4]]

Constraints:

- $n == \text{graph.length}$
- $2 \leq n \leq 15$
- $0 \leq \text{graph}[i][j] < n$
- $\text{graph}[i][j] \neq i$ (i.e., there will be no self-loops).
- All the elements of $\text{graph}[i]$ are unique.
- The input graph is guaranteed to be a DAG.

Round 2 · High · Medium

p.1013

Community Solutions (Python)

• WalkCC

```
class Solution:
    def allPathsSourceTarget(self, graph: List[List[int]]) -> List[List[int]]:
        ans = []

        def dfs(u: int, path: List[int]) -> None:
            if u == len(graph) - 1:
                ans.append(path)
                return

            for v in graph[u]:
                dfs(v, path + [v])

        dfs(0, [0])
        return ans
```

• LeetCode

```
class Solution:
    def allPathsSourceTarget(self, graph: List[List[int]]) -> List[List[int]]:
        n = len(graph)
        q = deque([[0]])
        ans = []
        while q:
            path = q.popleft()
            u = path[-1]
            if u == n - 1:
                ans.append(path)
                continue
            for v in graph[u]:
                q.append(path + [v])
        return ans
```

• SimplyLeet

Round 2 · High · Medium

p.1014

```
def allPathsSourceTarget(graph):
    # List to store all valid paths from source to target
    result = []
    # Current path tracker
    path = []

    def dfs(node):
        # Add current node to the path
        path.append(node)
        # If target node is reached, add a copy of the current path to result

        if node == len(graph) - 1:
            result.append(list(path))
        else:
            # Explore all neighbors of the current node
            for neighbor in graph[node]:
                dfs(neighbor)
            # Backtrack: remove the current node before exploring a new branch
            path.pop()

    # Start DFS from node 0

    dfs(0)
    return result

# Example usage:
# print(allPathsSourceTarget([[1,2],[3],[3],[1]]))
```

Depth First Search (DFS)

□ #863 All Nodes Distance K in Binary Tree **MED**

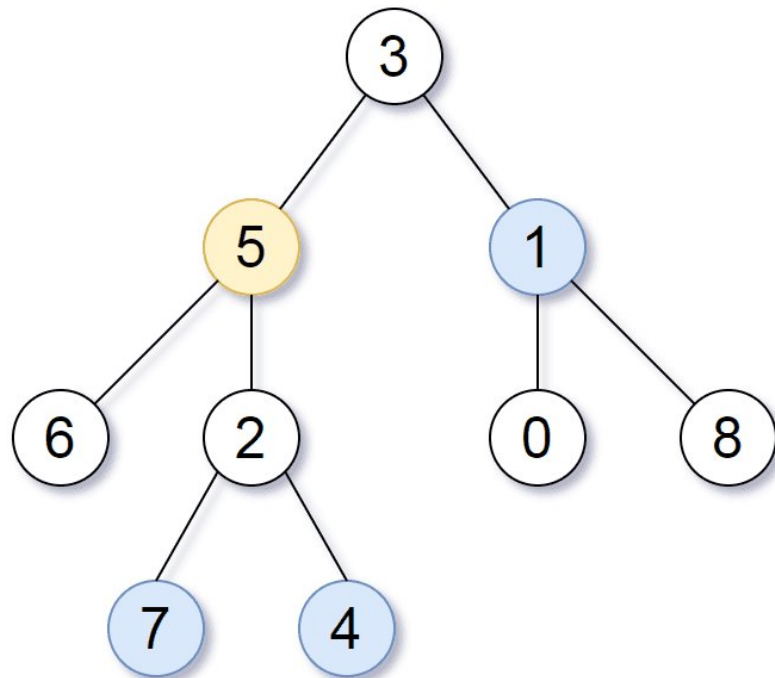
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Tree · DFS · BFS
Lists: Grind 169

Problem

Given the root of a binary tree, the value of a target node target, and an integer k, return an array of the values of all nodes that have a distance k from the target node.

You can return the answer in any order.

Example 1:



Input: root = [3,5,1,6,2,0,8,null,null,7,4], target = 5, k = 2
 Output: [7,4,1]
 Explanation: The nodes that are a distance 2 from the target node (with value 5) have values 7, 4, and 1.

Example 2:

Input: root = [1], target = 1, k = 3
 Output: []

Constraints:

- The number of nodes in the tree is in the range [1, 500].
- $0 \leq \text{Node.val} \leq 500$

Round 2 · High · Medium

p.1017

- All the values `Node.val` are unique.
- target is the value of one of the nodes in the tree.
- $0 \leq k \leq 1000$

Community Solutions (Python)

• LeetCode

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None

class Solution:
    def distanceK(self, root: TreeNode, target: TreeNode, k: int) -> List[int]:
        :

        def dfs(root, fa):
            if root is None:
                return
            g[root] = fa
            dfs(root.left, root)
            dfs(root.right, root)

        def dfs2(root, fa, k):
            if root is None:
                return

```

Round 2 · High · Medium

p.1018

```

    if k == 0:
        ans.append(root.val)
        return
    for nxt in (root.left, root.right, g[root]):
        if nxt != fa:
            dfs2(nxt, root, k - 1)

g = {}
dfs(root, None)
ans = []

```

```

dfs2(target, None, k)
return ans

```

• SimplyLeet

```

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def distanceK(root: TreeNode, target: int, k: int):
    from collections import defaultdict, deque

    # Build an adjacency list to represent the graph
    graph = defaultdict(list)

    # Helper function to build the graph using DFS
    def buildGraph(node, parent):
        if node is None:
            return
        if parent is not None:
            # Add bidirectional connection between node and parent
            graph[node.val].append(parent.val)

```

Round 2 · High · Medium

p.1019

```

        graph[parent.val].append(node.val)
        buildGraph(node.left, node)
        buildGraph(node.right, node)

```

```

buildGraph(root, None)

```

```

# BFS from target node
visited = set([target])
queue = deque([target])
current_distance = 0

```

```

while queue:
    if current_distance == k:
        # Return all nodes at distance k
        return list(queue)
    size = len(queue)
    for _ in range(size):
        current = queue.popleft()
        for neighbor in graph[current]:
            if neighbor not in visited:

```

```

                visited.add(neighbor)
                queue.append(neighbor)
            current_distance += 1

```

```

return [] # If no nodes at distance k

```

```

# Example usage:
# Construct tree or use a tree builder before calling distanceK function.

```

• LC.ca

Round 2 · High · Medium

p.1020

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None

class Solution:
    def distanceK(self, root: TreeNode, target: TreeNode, k: int) -> List[int]
    :

    def parents(root, prev):
        nonlocal p
        if root is None:
            return
        p[root] = prev
        parents(root.left, root)
        parents(root.right, root)

    def dfs(root, k):
        nonlocal ans, vis

        if root is None or root.val in vis:
            return
        vis.add(root.val)
        if k == 0:
            ans.append(root.val)
            return
        dfs(root.left, k - 1)
        dfs(root.right, k - 1)
        dfs(p[root], k - 1)

    p = {}
    parents(root, None)
    ans = []
    vis = set()
    dfs(target, k)
    return ans

```

◆ Quick Review

Round 2 · High · Medium

p.1021

Key Insights

- Convert the binary tree into an undirected graph so that we can easily traverse both down to the children and up to the parent.
- Use a DFS or simple recursive traversal to create an adjacency list representing the graph.
- Perform a BFS starting from the target node to explore nodes level-by-level until reaching distance k.
- Use a visited set (or equivalent) to avoid revisiting nodes and to prevent cycles.

Space & Time

Time Complexity: $O(N)$, where N is the number of nodes in the tree. Each node is visited once when constructing the graph and once in the BFS.

Space Complexity: $O(N)$, for storing the graph and the additional data structures used in BFS and DFS.

Solution

The solution involves two main steps:

Round 2 · High · Medium

p.1022

– Build an undirected graph representation of the binary tree using a DFS. For each node, add the node's value as a key in an adjacency list and connect it with its children (if any). This creates bidirectional links between parent and child. – Use a BFS starting from the target node and explore nodes level-by-level. Keep a counter for the current distance, and once the counter reaches k, return all node values found at that level. To avoid processing nodes more than once, maintain a set of visited nodes.

Depth First Search (DFS)

□ #1020 Number of Enclaves

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Depth-First Search · Breadth-First
 Search · Union-Find · Matrix
 Lists: AlgoMaster 600

Problem

You are given an $m \times n$ binary matrix grid, where 0 represents a sea cell and 1 represents a land cell.

A move consists of walking from one land cell to another adjacent (4-directionally) land cell or walking off the boundary of the grid.

Return the number of land cells in grid for which we cannot walk off the boundary of the grid in any number of moves.

Example 1:

0	0	0	0
1	0	1	0
0	1	1	0
0	0	0	0

Input: grid = [[0,0,0,0],[1,0,1,0],[0,1,1,0],[0,0,0,0]]

Output: 3

Explanation: There are three 1s that are enclosed by 0s, and one 1 that is not enclosed because its on the boundary.

Example 2:

Round 2 · High · Medium

p.1025

0	1	1	0
0	0	1	0
0	0	1	0
0	0	0	0

Input: grid = [[0,1,1,0],[0,0,1,0],[0,0,1,0],[0,0,0,0]]

Output: 0

Explanation: All 1s are either on the boundary or can reach the boundary.

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq m, n \leq 500$
- $\text{grid}[i][j]$ is either 0 or 1.

Community Solutions (Python)

Round 2 · High · Medium

p.1026

• LeetCode

```
class Solution:
    def numEnclaves(self, grid: List[List[int]]) -> int:
        def dfs(i: int, j: int):
            grid[i][j] = 0
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                if 0 <= x < m and 0 <= y < n and grid[x][y]:
                    dfs(x, y)
```

```
m, n = len(grid), len(grid[0])
```

```
dirs = (-1, 0, 1, 0, -1)
for j in range(n):
    if grid[0][j]:
        dfs(0, j)
    if grid[m - 1][j]:
        dfs(m - 1, j)
for i in range(m):
    if grid[i][0]:
        dfs(i, 0)
    if grid[i][n - 1]:
```

```
        dfs(i, n - 1)
    return sum(sum(row) for row in grid)
```

```
class Solution:
    def numEnclaves(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        q = deque()
        for i in range(m):
            for j in range(n):
                if grid[i][j] and (i == 0 or i == m - 1 or j == 0 or j == n -
1):
                    q.append((i, j))
                    grid[i][j] = 0
        dirs = (-1, 0, 1, 0, -1)
```

```
while q:
    i, j = q.popleft()
    for a, b in pairwise(dirs):
        x, y = i + a, j + b
        if x >= 0 and x < m and y >= 0 and y < n and grid[x][y]:
            q.append((x, y))
            grid[x][y] = 0
    return sum(v for row in grid for v in row)
```

```
class UnionFind:
    def __init__(self, n):
        self.p = list(range(n))
        self.size = [1] * n

    def find(self, x):
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]
```

```
    def union(self, a, b):
        pa, pb = self.find(a), self.find(b)
        if pa != pb:
            if self.size[pa] > self.size[pb]:
                self.p[pb] = pa
                self.size[pa] += self.size[pb]
            else:
                self.p[pa] = pb
                self.size[pb] += self.size[pa]
```

```
class Solution:
    def numEnclaves(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        uf = UnionFind(m * n + 1)
        dirs = (-1, 0, 1, 0, -1)
        for i, row in enumerate(grid):
            for j, v in enumerate(row):
                if v:
                    if i == 0 or i == m - 1 or j == 0 or j == n - 1:
```

```

        uf.union(i * n + j, m * n)
    else:
        for a, b in pairwise(dirs):
            x, y = i + a, j + b
            if x >= 0 and x < m and y >= 0 and y < n and grid[
x][y]:
                uf.union(i * n + j, x * n + y)

    return sum(
        grid[i][j] == 1 and uf.find(i * n + j) != uf.find(m * n)
        for i in range(m)
        for j in range(n)
    )

```

• SimplyLeet

```

class Solution:
    def numEnclaves(self, grid: List[List[int]]) -> int:
        # Calculate number of rows and columns
        m, n = len(grid), len(grid[0])

        # Helper function using DFS to mark connected land as sea.
        def dfs(r, c):
            # Base case: if out of bounds or already water, return
            if r < 0 or r >= m or c < 0 or c >= n or grid[r][c] == 0:
                return

            # Mark the current cell as water to avoid revisiting it
            grid[r][c] = 0
            # Explore all 4 directions (up, down, left, right)
            dfs(r + 1, c)
            dfs(r - 1, c)
            dfs(r, c + 1)
            dfs(r, c - 1)

        # Remove boundary-connected land cells by traversing first and last
        # columns for each row.
        for r in range(m):

```

Round 2 · High · Medium

p.1029

```

        dfs(r, 0)
        dfs(r, n - 1)
        # Traverse first and last rows for each column.
        for c in range(n):
            dfs(0, c)
            dfs(m - 1, c)

        # Count remaining land cells that are enclaves.
        enclaveCount = sum(grid[r][c] for r in range(m) for c in range(n))
        return enclaveCount

```

Round 2 · High · Medium

p.1030

Depth First Search (DFS)

□ #1376 Time Needed to Inform All Employees

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode Tree · Depth-First Search · Breadth-First Search

Lists: AlgoMaster 600

Problem

A company has n employees with a unique ID for each employee from 0 to $n - 1$. The head of the company is the one with `headID`.

Each employee has one direct manager given in the manager array where `manager[i]` is the direct manager of the i -th employee, `manager[headID] = -1`. Also, it is guaranteed that the subordination relationships have a tree structure.

The head of the company wants to inform all the company employees of an urgent piece of news. He will inform his direct subordinates, and they will inform their subordinates, and so on until all employees know about the urgent news.

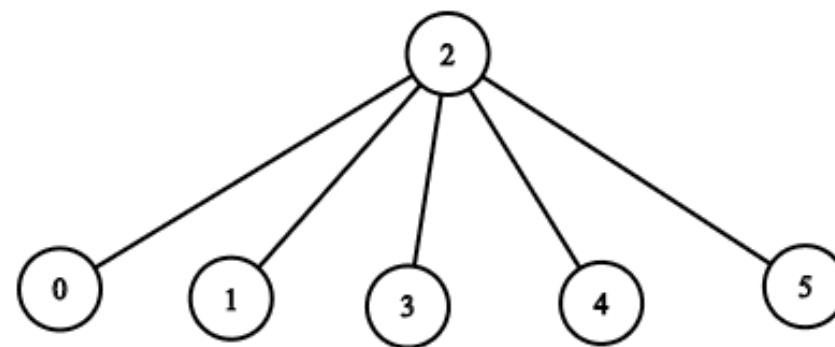
The i -th employee needs `informTime[i]` minutes to inform all of his direct subordinates (i.e., After `informTime[i]` minutes, all his direct subordinates can start spreading the news).

Return the number of minutes needed to inform all the employees about the urgent news.

Example 1:

Input: $n = 1$, `headID = 0`, `manager = [-1]`, `informTime = [0]`
 Output: 0
 Explanation: The head of the company is the only employee in the company.

Example 2:



Input: $n = 6$, `headID = 2`, `manager = [2,2,-1,2,2,2]`, `informTime = [0,0,1,0,0,0]`
 Output: 1
 Explanation: The head of the company with id = 2 is the direct manager of all the employees in the company and needs 1 minute to inform them all.
 The tree structure of the employees in the company is shown.

Constraints:

- $1 \leq n \leq 105$

Round 2 · High · Medium

p.1031

Round 2 · High · Medium

p.1032

- $0 \leq \text{headID} < n$
- $\text{manager.length} == n$
- $0 \leq \text{manager}[i] < n$
- $\text{manager}[\text{headID}] == -1$
- $\text{informTime.length} == n$
- $0 \leq \text{informTime}[i] \leq 1000$
- $\text{informTime}[i] == 0$ if employee i has no subordinates.
- It is guaranteed that all the employees can be informed.

Community Solutions (Python)

• LeetCode

```
class Solution:
    def numOfMinutes(
        self, n: int, headID: int, manager: List[int], informTime: List[int]
    ) -> int:
        def dfs(i: int) -> int:
            ans = 0
            for j in g[i]:
                ans = max(ans, dfs(j) + informTime[i])
            return ans

        g = defaultdict(list)
        for i, x in enumerate(manager):
            g[x].append(i)
        return dfs(headID)
```

• SimplyLeet

```
# Python solution using DFS and an adjacency list to represent the tree.
from collections import defaultdict

def numOfMinutes(n, headID, manager, informTime):
    # Build the tree: map each manager to its direct subordinates.
    tree = defaultdict(list)
    for employee, mgr in enumerate(manager):
        if mgr != -1:
            tree[mgr].append(employee)

    # DFS function to compute the time to inform all subordinates under 'emp'
    def dfs(emp):
        max_subordinate_time = 0
        # Traverse all direct subordinates of the employee
        for subordinate in tree[emp]:
            max_subordinate_time = max(max_subordinate_time, dfs(subordinate))
        # Total time is the current employee's inform time plus max time
        # required by subordinates.
        return informTime[emp] + max_subordinate_time

    return dfs(headID)

# Example usage:
# n = 6, headID = 2, manager = [2,2,-1,2,2,2], informTime = [0,0,1,0,0,0]
# print(numOfMinutes(6, 2, [2,2,-1,2,2,2], [0,0,1,0,0,0]))
```

Breadth First Search (BFS)

Round 2 · High · Medium · 9 questions

Round 2 · High · Medium

p.1035

Breadth First Search (BFS)

□ ★ #286 Walls and Gates ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · BFS · Matrix

Lists: ★ Premium | Premium 98

Problem

You are given an $m \times n$ grid rooms initialized with these three possible values.

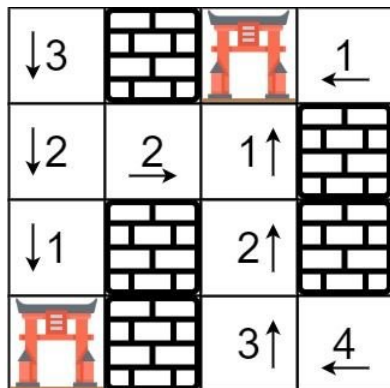
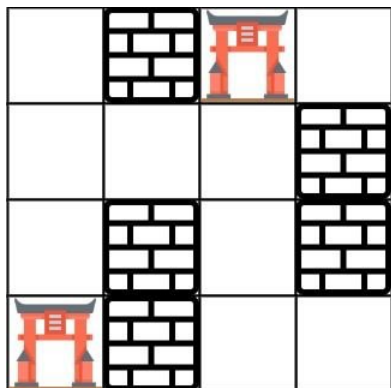
- -1 A wall or an obstacle.
- 0 A gate.
- INF Infinity means an empty room. We use the value $2^{31} - 1 = 2147483647$ to represent INF as you may assume that the distance to a gate is less than 2147483647.

Fill each empty room with the distance to its nearest gate. If it is impossible to reach a gate, it should be filled with INF.

Example 1:

Round 2 · High · Medium

p.1036



Input: rooms = [[2147483647,-1,0,2147483647],[2147483647,2147483647,2147483647,-1],[2147483647,-1,2147483647,-1],[0,-1,2147483647,2147483647]]
 Output: [[3,-1,0,1],[2,2,1,-1],[1,-1,2,-1],[0,-1,3,4]]

Example 2:

Input: rooms = [[-1]]
 Output: [[-1]]

Constraints:

- $m == \text{rooms.length}$
- $n == \text{rooms}[i].\text{length}$
- $1 \leq m, n \leq 250$
- $\text{rooms}[i][j]$ is $-1, 0$, or $231 - 1$.

Community Solutions (Python)

- WalkCC

```
class Solution:
    def wallsAndGates(self, rooms: List[List[int]]) -> None:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        INF = 2**31 - 1
        m = len(rooms)
        n = len(rooms[0])
        q = collections.deque((i, j)
                               for i in range(m)
                               for j in range(n)
                               if rooms[i][j] == 0)
```

```
while q:
    i, j = q.popleft()
    for dx, dy in DIRS:
        x = i + dx
        y = j + dy
        if x < 0 or x == m or y < 0 or y == n:
            continue
        if rooms[x][y] != INF:
            continue
```

```
rooms[x][y] = rooms[i][j] + 1
q.append((x, y))
```

• LeetDoocs

```
class Solution:
    def wallsAndGates(self, rooms: List[List[int]]) -> None:
        """
        Do not return anything, modify rooms in-place instead.
        """
        m, n = len(rooms), len(rooms[0])
        inf = 2**31 - 1
        q = deque([(i, j) for i in range(m) for j in range(n) if rooms[i][j] == 0])
        d = 0
        while q:
```

```

d += 1
for _ in range(len(q)):
    i, j = q.popleft()
    for a, b in [[0, 1], [0, -1], [1, 0], [-1, 0]]:
        x, y = i + a, j + b
        if 0 <= x < m and 0 <= y < n and rooms[x][y] == inf:
            rooms[x][y] = d
            q.append((x, y))

```

• SimplyLeet

```
from collections import deque
```

```
def wallsAndGates(rooms):
    if not rooms or not rooms[0]:
        return

```

```

rows, cols = len(rooms), len(rooms[0])
# Directions: up, down, left, right.
directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
queue = deque()

```

```

# Enqueue all gates (cells with 0).
for r in range(rows):
    for c in range(cols):
        if rooms[r][c] == 0:
            queue.append((r, c))

```

```

# Perform multi-source BFS.
while queue:
    r, c = queue.popleft()

```

```

for dr, dc in directions:
    nr, nc = r + dr, c + dc
    # Process neighbor if it is within bounds and is an empty room.
    if 0 <= nr < rows and 0 <= nc < cols and rooms[nr][nc] == 21474836
47:
        rooms[nr][nc] = rooms[r][c] + 1 # Update distance.
        queue.append((nr, nc))

```

• LC.ca

Round 2 · High · Medium

p.1039

```

class Solution:
    def wallsAndGates(self, rooms: List[List[int]]) -> None:
        """
        Do not return anything, modify rooms in-place instead.
        """
        m, n = len(rooms), len(rooms[0])
        inf = 2**31 - 1
        q = deque([(i, j) for i in range(m) for j in range(n) if rooms[i][j] =
= 0])
        d = 0
        while q:

```

```

d += 1
for _ in range(len(q)):
    i, j = q.popleft()
    for a, b in [[0, 1], [0, -1], [1, 0], [-1, 0]]:
        x, y = i + a, j + b
        # bfs so first reaching inf is always the shortest
        # set its value from inf to d, same as marking as visited
        if 0 <= x < m and 0 <= y < n and rooms[x][y] == inf:
            rooms[x][y] = d
            q.append((x, y))

```

```
#####
```

```

class Solution: # dfs
    def wallsAndGates(self, rooms: List[List[int]]) -> None:
        if not rooms or not rooms[0]:
            return

        m, n = len(rooms), len(rooms[0])
        isVisited = [[False] * n for _ in range(m)] # isVisited map shared
        among all gates

```

Round 2 · High · Medium

p.1040

```
def fill(i: int, j: int, distance: int) -> None:
    nonlocal m, n

    # rooms[i][j] <= 0 meaning == -1 or == 0, both stop
    if i < 0 or i >= m or j < 0 or j >= n or rooms[i][j] <= 0 or isVis
ited[i][j]:
        return

    rooms[i][j] = min(rooms[i][j], distance)
    isVisited[i][j] = True

    fill(i - 1, j, distance + 1)
    fill(i, j + 1, distance + 1)
    fill(i + 1, j, distance + 1)
    fill(i, j - 1, distance + 1)

    isVisited[i][j] = False

for i in range(m):
    for j in range(n):

        if rooms[i][j] == 0: # start from every gate
            fill(i, j, 0)

#####

from collections import deque

INF = 2147483647
```

Round 2 · High · Medium

p.1041

```
class Solution(object):
    def wallsAndGates(self, rooms):
        """
        :type rooms: List[List[int]]
        :rtype: void Do not return anything, modify rooms in-place instead.
        """
        queue = deque([])
        directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]
        for i in range(0, len(rooms)):
            for j in range(0, len(rooms[0])):

                if rooms[i][j] == 0:
                    queue.append((i, j))

        while queue:
            i, j = queue.popleft()
            for di, dj in directions:
                p, q = i + di, j + dj
                if 0 <= p < len(rooms) and 0 <= q < len(rooms[0]) and rooms[p][q] == I
NF:
                    rooms[p][q] = rooms[i][j] + 1
                    queue.append((p, q))
```

◆ Quick Review

Key Insights

- Use a multi-source Breadth-First Search (BFS) starting from all the gates, as this allows spreading the shortest distance to each empty room.
- Instead of starting from every empty room separately, starting from all gates simultaneously helps update all distances in one pass.

Round 2 · High · Medium

p.1042

- Only update a room if the new calculated distance is smaller than its current value to prevent unnecessary processing.
- The problem is essentially a multi-source shortest path problem on a grid.

Space & Time

Time Complexity: $O(m * n)$ where m and n are the dimensions of the grid, as each cell is processed at most once.

Space Complexity: $O(m * n)$ in the worst-case scenario used by the BFS queue.

Solution

The solution uses a multi-source BFS algorithm by initially enqueueing all the gate positions (cells with value 0). Then, for each cell obtained from the queue, check its four possible neighbors (up, down, left, right). If a neighbor is an empty room (INF), update its distance to be the current cell's distance plus one, and then enqueue the neighbor to process further. This ensures that each empty room gets the minimum distance from any gate.

Breadth First Search (BFS)

□ #433 Minimum Genetic Mutation

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Breadth-First Search
Lists: AlgoMaster 600

Problem

A gene string can be represented by an 8-character long string, with choices from 'A', 'C', 'G', and 'T'.

Suppose we need to investigate a mutation from a gene string `startGene` to a gene string `endGene` where one mutation is defined as one single character changed in the gene string.

- For example, "AACCGGTT" --> "AACCGGTA" is one mutation.

There is also a gene bank `bank` that records all the valid gene mutations. A gene must be in `bank` to make it a valid gene string.

Given the two gene strings `startGene` and `endGene` and the gene bank `bank`, return the minimum number of mutations needed to

mutate from startGene to endGene. If there is no such a mutation, return -1.

Note that the starting point is assumed to be valid, so it might not be included in the bank.

Example 1:

Input: startGene = "AACCGGTT", endGene = "AACCGGTA", bank = ["AACCGGTA"]
Output: 1

Example 2:

Input: startGene = "AACCGGTT", endGene = "AAACGGTA", bank = ["AACCGGTA", "AACCGCTA", "AAACGGTA"]
Output: 2

Constraints:

- $0 \leq \text{bank.length} \leq 10$
- $\text{startGene.length} == \text{endGene.length} == \text{bank}[i].\text{length} == 8$
- startGene, endGene, and bank[i] consist of only the characters ['A', 'C', 'G', 'T'].

Community Solutions (Python)

- WalkCC

Round 2 · High · Medium

p.1045

```
class Solution:
    def minMutation(self, startGene: str, endGene: str, bank: list[str]) -> int:
        bankSet = set(bank)
        if endGene not in bankSet:
            return -1

        GENES = 'ACGT'
        q = collections.deque([startGene])

        step = 1

        while q:
            for _ in range(len(q)):
                wordList = list(q.popleft())
                for j, cache in enumerate(wordList):
                    for c in GENES:
                        wordList[j] = c
                        word = ''.join(wordList)
                        if word == endGene:
                            return step
                        if word in bankSet:
                            bankSet.remove(word)
                            q.append(word)
                            wordList[j] = cache
            step += 1

        return -1
```

• LeetCode

```
class Solution:
    def minMutation(self, startGene: str, endGene: str, bank: List[str]) -> int:
        q = deque([(startGene, 0)])
        vis = {startGene}
        while q:
            gene, depth = q.popleft()
            if gene == endGene:
                return depth
            for nxt in bank:
                diff = sum(a != b for a, b in zip(gene, nxt))
```

Round 2 · High · Medium

p.1046

```

        if diff == 1 and nxt not in vis:
            q.append((nxt, depth + 1))
            vis.add(nxt)

    return -1

```

• SimplyLeet

```

# Python implementation for Minimum Genetic Mutation
from collections import deque

```

```

def minMutation(startGene, endGene, bank):
    # Convert bank list to set for quick lookup
    bankSet = set(bank)

```

```

    # Early exit if endGene is not in the bank
    if endGene not in bankSet:
        return -1

```

```

    # Queue holds tuples (current_gene, mutations_count)
    queue = deque([(startGene, 0)])
    # Valid characters for mutation
    validGenes = ['A', 'C', 'G', 'T']

```

```

    while queue:
        current, mutations = queue.popleft()
        # If the current gene equals the endGene, we found the mutation path
        if current == endGene:

```

```

            return mutations

```

```

    # Try all possible single-character mutations
    for i in range(len(current)):
        for char in validGenes:
            if char == current[i]:
                continue
            # Create a new gene string mutation
            mutated = current[:i] + char + current[i+1:]
            # If this mutation is valid and in the bank

```

```

        if mutated in bankSet:
            # Remove to avoid revisiting and add to the queue
            bankSet.remove(mutated)
            queue.append((mutated, mutations + 1))

```

```

    # No valid mutation sequence was found
    return -1

```

```

# Example usage:
# print(minMutation("AACCGGTT", "AAACGGTA",
# ["AACCGGTA", "AACCGCTA", "AAACGGTA"]))

```


Breadth First Search (BFS)

□ #542 01 Matrix

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · BFS · Matrix

Lists: Grind 169 | Denny Zhang

Problem

Given an $m \times n$ binary matrix `mat`, return the distance of the nearest 0 for each cell.

The distance between two cells sharing a common edge is 1.

Example 1:

0	0	0
0	1	0
0	0	0

Input: `mat = [[0,0,0],[0,1,0],[0,0,0]]`
Output: `[[0,0,0],[0,1,0],[0,0,0]]`

Example 2:

0	0	0
0	1	0
1	1	1

Input: mat = [[0,0,0],[0,1,0],[1,1,1]]
Output: [[0,0,0],[0,1,0],[1,2,1]]

Constraints:

- $m == \text{mat.length}$
- $n == \text{mat}[i].\text{length}$
- $1 \leq m, n \leq 104$
- $1 \leq m * n \leq 104$
- $\text{mat}[i][j]$ is either 0 or 1.
- There is at least one 0 in mat.

Round 2 · High · Medium

p.1051

Note: This question is the same as 1765: <https://leetcode.com/problems/map-of-highest-peak/>

Community Solutions (Python)

• WalkCC

```
class Solution:
    def updateMatrix(self, mat: list[list[int]]) -> list[list[int]]:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(mat)
        n = len(mat[0])
        q = collections.deque()
        seen = [[False] * n for _ in range(m)]

        for i in range(m):
            for j in range(n):
                if mat[i][j] == 0:
                    q.append((i, j))
                    seen[i][j] = True

        while q:
            i, j = q.popleft()
            for dx, dy in DIRS:
                x = i + dx
                y = j + dy
                if x < 0 or x == m or y < 0 or y == n:
                    continue
                if seen[x][y]:
                    continue
                mat[x][y] = mat[i][j] + 1
                q.append((x, y))
                seen[x][y] = True

        return mat
```

• LeetCode

Round 2 · High · Medium

p.1052

```
class Solution:
    def updateMatrix(self, mat: List[List[int]]) -> List[List[int]]:
        m, n = len(mat), len(mat[0])
        ans = [[-1] * n for _ in range(m)]
        q = deque()
        for i, row in enumerate(mat):
            for j, x in enumerate(row):
                if x == 0:
                    ans[i][j] = 0
                    q.append((i, j))

        dirs = (-1, 0, 1, 0, -1)
        while q:
            i, j = q.popleft()
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                if 0 <= x < m and 0 <= y < n and ans[x][y] == -1:
                    ans[x][y] = ans[i][j] + 1
                    q.append((x, y))

        return ans
```

• SimplyLeet

```
from collections import deque

def updateMatrix(mat):
    # Get the dimensions of the matrix
    rows, cols = len(mat), len(mat[0])
    # Initialize a queue for multi-source BFS
    queue = deque()

    # Initialize the matrix: use -1 to indicate unvisited cells
    for r in range(rows):
```

Round 2 · High · Medium

p.1053

```
for c in range(cols):
    if mat[r][c] == 0:
        # Enqueue all 0 cells as starting points
        queue.append((r, c))
    else:
        # Mark 1 cells as unvisited (or infinite distance)
        mat[r][c] = -1

# Directions for neighbors (up, down, left, right)
directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]

# BFS traversal to update distances
while queue:
    r, c = queue.popleft()
    for dr, dc in directions:
        nr, nc = r + dr, c + dc
        # Check boundaries and if neighbor has not been visited
        if 0 <= nr < rows and 0 <= nc < cols and mat[nr][nc] == -1:
            # Update neighbor's distance
            mat[nr][nc] = mat[r][c] + 1

# Enqueue the neighbor for further BFS processing
queue.append((nr, nc))

return mat

# Example usage:
# print(updateMatrix([[0,0,0],[0,1,0],[1,1,1]]))
```

• LC.ca

```
class Solution:
    def updateMatrix(self, mat: List[List[int]]) -> List[List[int]]:
        m, n = len(mat), len(mat[0])
        ans = [[-1] * n for _ in range(m)]
        q = deque()
        for i, row in enumerate(mat):
            for j, x in enumerate(row):
                if x == 0:
                    ans[i][j] = 0
                    q.append((i, j))
```

Round 2 · High · Medium

p.1054

```

dirs = (-1, 0, 1, 0, -1)
while q:
    i, j = q.popleft()
    for a, b in pairwise(dirs):
        x, y = i + a, j + b
        if 0 <= x < m and 0 <= y < n and ans[x][y] == -1:
            ans[x][y] = ans[i][j] + 1
            q.append((x, y))
return ans

```

◆ Quick Review

Key Insights

- Treat all 0s as starting points for a multi-source BFS.
- Update each neighboring cell's distance if a shorter distance is found.
- Utilize a queue to process cells in order of increasing distance.
- Ensure that each cell is processed only once by marking visited cells.

Space & Time

Time Complexity: $O(mn)$ since each cell is processed once.

Space Complexity: $O(mn)$ for the queue and the answer matrix.

Solution

Round 2 · High · Medium

p.1055

We use a multi-source Breadth-First Search (BFS) where every 0 in the matrix is initially added to the BFS queue. Each neighboring cell that is a 1 will have its distance updated to be one plus the distance of the current cell if it hasn't been visited yet (or if a smaller distance is found). This ensures that the first time a 1 is reached, it is reached by the shortest path. Data structures used include a queue (or deque) for BFS and the matrix itself for storing distances.

Round 2 · High · Medium

p.1056

Breadth First Search (BFS)

□ #752 Open the Lock

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · String · Breadth-First Search

Lists: AlgoMaster 600

Problem

You have a lock in front of you with 4 circular wheels. Each wheel has 10 slots: '0', '1', '2', '3', '4', '5', '6', '7', '8', '9'. The wheels can rotate freely and wrap around: for example we can turn '9' to be '0', or '0' to be '9'. Each move consists of turning one wheel one slot.

The lock initially starts at '0000', a string representing the state of the 4 wheels.

You are given a list of deadends dead ends, meaning if the lock displays any of these codes, the wheels of the lock will stop turning and you will be unable to open it.

Given a target representing the value of the wheels that will unlock the lock, return the minimum total number of turns required to

Round 2 · High · Medium

p.1057

open the lock, or -1 if it is impossible.

Example 1:

```
Input: deadends = ["0201","0101","0102","1212","2002"], target = "0202"
Output: 6
Explanation:
A sequence of valid moves would be "0000" -> "1000" -> "1100" -> "1200" ->
"1201" -> "1202" -> "0202".
Note that a sequence like "0000" -> "0001" -> "0002" -> "0102" -> "0202" would
be invalid,
because the wheels of the lock become stuck after the display becomes the dead
end "0102".
```

Example 2:

```
Input: deadends = ["8888"], target = "0009"
Output: 1
Explanation: We can turn the last wheel in reverse to move from "0000" ->
"0009".
```

Example 3:

```
Input: deadends = ["8887","8889","8878","8898","8788","8988","7888","9888"],
target = "8888"
Output: -1
Explanation: We cannot reach the target without getting stuck.
```

Constraints:

- $1 \leq \text{deadends.length} \leq 500$
- $\text{deadends}[i].\text{length} == 4$
- $\text{target.length} == 4$
- target will not be in the list deadends.
- target and deadends[i] consist of digits only.

Community Solutions (Python)

- LeetDoocs

Round 2 · High · Medium

p.1058

```

class Solution:
    def openLock(self, deadends: List[str], target: str) -> int:
        def next(s):
            res = []
            s = list(s)
            for i in range(4):
                c = s[i]
                s[i] = '9' if c == '0' else str(int(c) - 1)
                res.append(''.join(s))
                s[i] = '0' if c == '9' else str(int(c) + 1)

            res.append(''.join(s))
            s[i] = c
            return res

        if target == '0000':
            return 0
        s = set(deadends)
        if '0000' in s:
            return -1
        q = deque(['0000'])

        s.add('0000')
        ans = 0
        while q:
            ans += 1
            for _ in range(len(q)):
                p = q.popleft()
                for t in next(p):
                    if t == target:
                        return ans
                    if t not in s:
                        q.append(t)
                        s.add(t)

        return -1

```

Round 2 · High · Medium

p.1059

```

class Solution {
public:
    string target;

    int openLock(vector<string>& deadends, string target) {
        if (target == "0000") return 0;
        unordered_set<string> s(deadends.begin(), deadends.end());
        if (s.count("0000")) return -1;
        string start = "0000";
        this->target = target;

        typedef pair<int, string> PIS;
        priority_queue<PIS, vector<PIS>, greater<PIS>> q;
        unordered_map<string, int> dist;
        dist[start] = 0;
        q.push({f(start), start});
        while (!q.empty()) {
            PIS t = q.top();
            q.pop();
            string state = t.second;
            int step = dist[state];

            if (state == target) return step;
            for (string& t : next(state)) {
                if (s.count(t)) continue;
                if (!dist.count(t) || dist[t] > step + 1) {
                    dist[t] = step + 1;
                    q.push({step + 1 + f(t), t});
                }
            }
        }
        return -1;
    }

    int f(string s) {
        int ans = 0;
        for (int i = 0; i < 4; ++i) {
            int a = s[i] - '0';
            int b = target[i] - '0';
            if (a < b) {
                int t = a;
                a = b;
            }
        }
    }
}

```

Round 2 · High · Medium

p.1060

```

        b = t;
    }
    ans += min(b - a, a + 10 - b);
}
return ans;
}

```

```

vector<string> next(string& t) {
    vector<string> res;
    for (int i = 0; i < 4; ++i) {

```

```

        char c = t[i];
        t[i] = c == '0' ? '9' : (char) (c - 1);
        res.push_back(t);
        t[i] = c == '9' ? '0' : (char) (c + 1);
        res.push_back(t);
        t[i] = c;
    }
    return res;
}
};

```

• SimplyLeet

```

# Python implementation
from collections import deque

def openLock(deadends, target):
    dead_set = set(deadends)
    # Early check: if starting point is a deadend, no solution exists.
    if "0000" in dead_set:
        return -1

    queue = deque(["0000", 0]) # (state, moves)

```

Round 2 · High · Medium

p.1061

```

visited = {"0000"}

# Function to generate neighbors (all possible states from current state)
def get_neighbors(state):
    neighbors = []
    for i in range(4):
        digit = int(state[i])
        # Two possible moves: increment or decrement with wrap-around
        for move in (-1, 1):
            new_digit = (digit + move) % 10

```

```

            new_state = state[:i] + str(new_digit) + state[i+1:]
            neighbors.append(new_state)
    return neighbors

```

```

while queue:
    state, moves = queue.popleft()
    # Check if we reached the target combination
    if state == target:
        return moves

```

```

    for neighbor in get_neighbors(state):
        if neighbor not in visited and neighbor not in dead_set:
            visited.add(neighbor)
            queue.append((neighbor, moves + 1))

```

```

# If target is not reached, return -1
return -1

```

```

# Example test case
print(openLock(["0201", "0101", "0102", "1212", "2002"], "0202"))

```

Round 2 · High · Medium

p.1062

Breadth First Search (BFS)

#909 Snakes and Ladders

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Breadth-First Search · Matrix
Lists: AlgoMaster 600

Problem

You are given an $n \times n$ integer matrix `board` where the cells are labeled from 1 to n^2 in a Boustrophedon style starting from the bottom left of the board (i.e. `board[n - 1][0]`) and alternating direction each row.

You start on square 1 of the board. In each move, starting from square `curr`, do the following:

- Choose a destination square next with a label in the range `[curr + 1, min(curr + 6,  $n^2$ )]`. This choice simulates the result of a standard 6-sided die roll: i.e., there are always at most 6 destinations, regardless of the size of the board.

A board square on row `r` and column `c` has a snake or ladder if `board[r][c] != -1`. The

destination of that snake or ladder is `board[r][c]`. Squares 1 and n^2 are not the starting points of any snake or ladder.

Note that you only take a snake or ladder at most once per dice roll. If the destination to a snake or ladder is the start of another snake or ladder, you do not follow the subsequent snake or ladder.

- For example, suppose the board is `[[-1,4],[-1,3]]`, and on the first move, your destination square is 2. You follow the ladder to square 3, but do not follow the subsequent ladder to 4.

Return the least number of dice rolls required to reach the square n^2 . If it is not possible to reach the square, return `-1`.

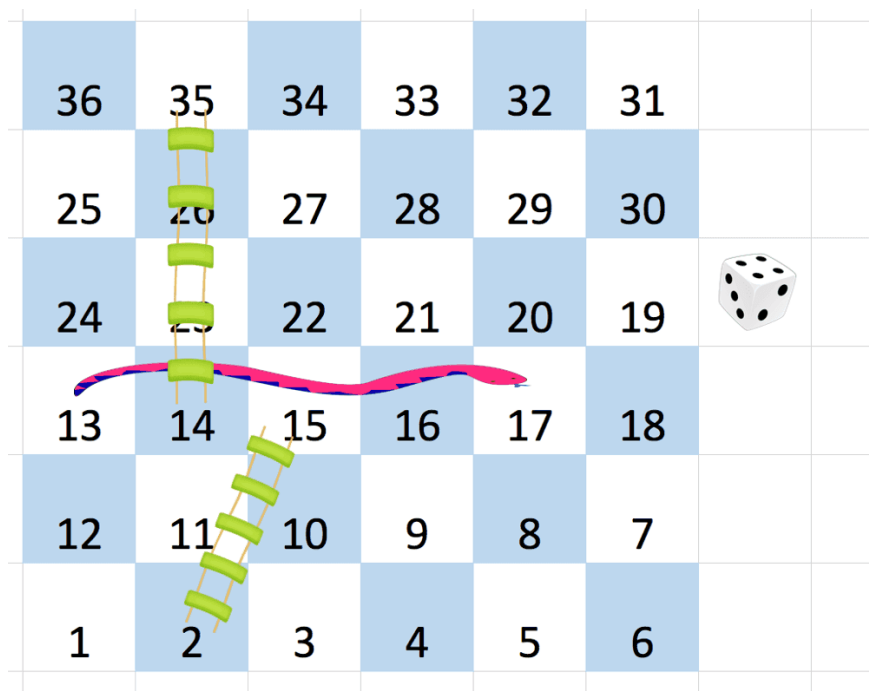
Example 1:

Round 2 · High · Medium

p.1063

Round 2 · High · Medium

p.1064



Input: board = [[-1,-1,-1,-1,-1,-1],[-1,-1,-1,-1,-1,-1],[-1,-1,-1,-1,-1,-1],[1,35,-1,-1,13,-1],[-1,-1,-1,-1,-1,-1],[-1,15,-1,-1,-1,-1]]

Output: 4

Explanation:

In the beginning, you start at square 1 (at row 5, column 0).

You decide to move to square 2 and must take the ladder to square 15.

You then decide to move to square 17 and must take the snake to square 13.

You then decide to move to square 14 and must take the ladder to square 35.

You then decide to move to square 36, ending the game.

Example 2:

Input: board = [[-1,-1],[-1,3]]

Output: 1

Constraints:

- $n == \text{board.length} == \text{board}[i].\text{length}$

Round 2 · High · Medium

p.1065

- $2 \leq n \leq 20$
- $\text{board}[i][j]$ is either -1 or in the range $[1, n^2]$.
- The squares labeled 1 and n^2 are not the starting points of any snake or ladder.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def snakesAndLadders(self, board: list[list[int]]) -> int:
        n = len(board)
        q = collections.deque([1])
        seen = set()
        arr = [0] * (1 + n * n)  # 2D -> 1D

        for i in range(n):
            for j in range(n):
                arr[(n - 1 - i) * n + (n - j if (n - i) % 2 == 0 else j + 1)] = board[i][j]

        step = 1
        while q:
            for _ in range(len(q)):
                curr = q.popleft()
                for next in range(curr + 1, min(curr + 6, n * n) + 1):
                    dest = arr[next] if arr[next] > 0 else next
                    if dest == n * n:
                        return step
                    if dest in seen:
                        continue
                    q.append(dest)
                    seen.add(dest)
            step += 1

        return -1
```

Round 2 · High · Medium

p.1066

• LeetCode

```
class Solution:
    def snakesAndLadders(self, board: List[List[int]]) -> int:
        n = len(board)
        q = deque([1])
        vis = {1}
        ans = 0
        m = n * n
        while q:
            for _ in range(len(q)):
                x = q.popleft()
```

```
                if x == m:
                    return ans
            for y in range(x + 1, min(x + 6, m) + 1):
                i, j = divmod(y - 1, n)
                if i & 1:
                    j = n - j - 1
                i = n - i - 1
                z = y if board[i][j] == -1 else board[i][j]
                if z not in vis:
                    vis.add(z)
```

```
                    q.append(z)
            ans += 1
        return -1
```

• SimplyLeet

```
from collections import deque

def snakesAndLadders(board):
    n = len(board)

    # Helper function to convert a square number to board coordinates
    def get_position(square):
        # Determine row index from bottom to top
        quot, rem = divmod(square - 1, n)
        row = n - 1 - quot
```

```
        # For even rows (from bottom), direction is left->right; for odd rows,
        right->left.
```

```
        if quot % 2 == 0:
            col = rem
        else:
            col = n - 1 - rem
        return row, col
```

```
        visited = set()
        queue = deque([(1, 0)]) # tuple: (current square, moves)
        visited.add(1)
```

```
        while queue:
            curr, moves = queue.popleft()
            if curr == n * n:
                return moves
            for i in range(1, 7): # simulate die roll
                next_sq = curr + i
                if next_sq > n * n:
                    continue
                r, c = get_position(next_sq)
```

```
        # if there's a snake or ladder, take the jump
        if board[r][c] != -1:
            next_sq = board[r][c]
        if next_sq not in visited:
            visited.add(next_sq)
            queue.append((next_sq, moves + 1))
```

```
        return -1
```

```
# Example usage:
#print(snakesAndLadders([[-1,-1,-1,-1,-1,-1],
```

```
# [-1,-1,-1,-1,-1,-1],
# [-1,-1,-1,-1,-1,-1],
# [-1,35,-1,-1,13,-1],
# [-1,-1,-1,-1,-1,-1],
# [-1,15,-1,-1,-1,-1]]))
```

Breadth First Search (BFS)

#994 Rotting Oranges

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode

Array · BFS · Matrix

Lists: Grind 169

Problem

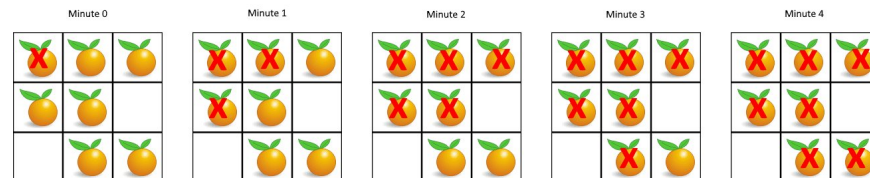
You are given an $m \times n$ grid where each cell can have one of three values:

- 0 representing an empty cell,
- 1 representing a fresh orange, or
- 2 representing a rotten orange.

Every minute, any fresh orange that is 4-directionally adjacent to a rotten orange becomes rotten.

Return the minimum number of minutes that must elapse until no cell has a fresh orange. If this is impossible, return -1.

Example 1:



Input: grid = [[2,1,1],[1,1,0],[0,1,1]]
Output: 4

Example 2:

Input: grid = [[2,1,1],[0,1,1],[1,0,1]]
Output: -1
Explanation: The orange in the bottom left corner (row 2, column 0) is never rotten, because rotting only happens 4-directionally.

Example 3:

Input: grid = [[0,2]]
Output: 0
Explanation: Since there are already no fresh oranges at minute 0, the answer is just 0.

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq m, n \leq 10$
- $\text{grid}[i][j]$ is 0, 1, or 2.

Community Solutions (Python)

- LeetDoocs

```

class Solution:
    def orangesRotting(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        cnt = 0
        q = deque()
        for i, row in enumerate(grid):
            for j, x in enumerate(row):
                if x == 2:
                    q.append((i, j))
                elif x == 1:
                    cnt += 1

        ans = 0
        dirs = (-1, 0, 1, 0, -1)
        while q and cnt:
            ans += 1
            for _ in range(len(q)):
                i, j = q.popleft()
                for a, b in pairwise(dirs):
                    x, y = i + a, j + b
                    if 0 <= x < m and 0 <= y < n and grid[x][y] == 1:
                        grid[x][y] = 2
                        q.append((x, y))
                        cnt -= 1
            if cnt == 0:
                return ans
        return -1 if cnt else 0

```

• SimplyLeet

```

from collections import deque

def orangesRotting(grid):
    # Dimensions of the grid
    rows, cols = len(grid), len(grid[0])
    # Queue to perform BFS, containing tuples of (row, col)
    rotten_queue = deque()
    fresh_count = 0

    # Step 1: Initialize the queue with all initially rotten oranges and count
    # fresh oranges.

```

Round 2 · High · Medium

p.1071

```

for r in range(rows):
    for c in range(cols):
        if grid[r][c] == 2:
            rotten_queue.append((r, c))
        elif grid[r][c] == 1:
            fresh_count += 1

# If there are no fresh oranges, return 0 immediately.
if fresh_count == 0:
    return 0

```

```

minutes_passed = 0
# Directions for adjacent cells: up, down, left, right.
directions = [(-1,0), (1,0), (0,-1), (0,1)]

# Step 2: BFS to rot adjacent fresh oranges.
while rotten_queue:
    # Process oranges that are rotten in the current minute.
    for _ in range(len(rotten_queue)):
        x, y = rotten_queue.popleft()

```

```

        for dx, dy in directions:
            nx, ny = x + dx, y + dy
            # Check if the neighbor is inside the grid and is a fresh
            orange.
            if 0 <= nx < rows and 0 <= ny < cols and grid[nx][ny] == 1:
                # Mark the fresh orange as rotten.
                grid[nx][ny] = 2
                fresh_count -= 1
                # Add the new rotten orange into the queue.
                rotten_queue.append((nx, ny))
            # Increment minutes after processing one level.

```

```
minutes_passed += 1
```

```

# If there are still fresh oranges left, return -1.
return minutes_passed - 1 if fresh_count == 0 else -1

```

```

# Example usage:
# grid = [[2,1,1],[1,1,0],[0,1,1]]
# print(orangesRotting(grid))

```

Round 2 · High · Medium

p.1072

• LC.ca

```
class Solution:
    def orangesRotting(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        q = deque()
        cnt = 0
        for i in range(m):
            for j in range(n):
                if grid[i][j] == 2:
                    q.append((i, j))
                elif grid[i][j] == 1:
                    cnt += 1

        ans = 0
        while q and cnt:
            ans += 1
            for _ in range(len(q)):
                i, j = q.popleft()
                for a, b in [[0, 1], [0, -1], [1, 0], [-1, 0]]:
                    x, y = i + a, j + b
                    if 0 <= x < m and 0 <= y < n and grid[x][y] == 1:
                        cnt -= 1
                        grid[x][y] = 2
                        q.append((x, y))
        return ans if cnt == 0 else -1
```

◆ Quick Review

Key Insights

- Use Breadth-First Search (BFS) starting from all initially rotten oranges.
- Process the grid level by level, where each level represents one minute.

Round 2 · High · Medium

p.1073

- Keep a count of fresh oranges and decrement it as they become rotten.
- If after the BFS there are still fresh oranges, then it is impossible to rot every orange.

Space & Time

Time Complexity: $O(m * n)$ as every cell is processed at most once.

Space Complexity: $O(m * n)$ in the worst case for the queue and auxiliary storage.

Solution

We begin by scanning the grid to locate all rotten oranges and count the fresh ones. These rotten oranges are added to a queue for a BFS. For each minute (BFS level), we process all the current rotten oranges and try to infect their adjacent fresh oranges. If an adjacent cell contains a fresh orange, we mark it rotten, add it to the queue, and decrement our fresh count. We continue this process level by level (minute by minute) until there are no fresh oranges left. If, after processing, there are still fresh oranges remaining, we return -1, indicating that not all oranges can become rotten.

Round 2 · High · Medium

p.1074

Breadth First Search (BFS)

#1091 Shortest Path in Binary Matrix

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Breadth-First Search · Matrix
 Lists: AlgoMaster 600

Problem

Given an $n \times n$ binary matrix grid, return the length of the shortest clear path in the matrix. If there is no clear path, return -1 .

A clear path in a binary matrix is a path from the top-left cell (i.e., $(0, 0)$) to the bottom-right cell (i.e., $(n - 1, n - 1)$) such that:

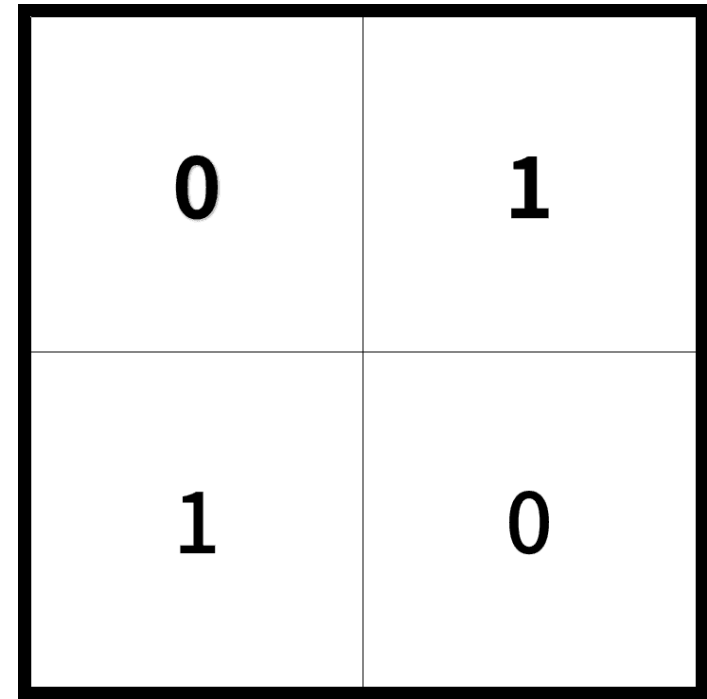
- All the visited cells of the path are 0.
- All the adjacent cells of the path are 8-directionally connected (i.e., they are different and they share an edge or a corner).

The length of a clear path is the number of visited cells of this path.

Example 1:

Round 2 · High · Medium

p.1075



Input: grid = `[[0,1],[1,0]]`
 Output: 2

Example 2:

Round 2 · High · Medium

p.1076

0	0	0
1	1	0
1	1	0

Input: grid = [[0,0,0],[1,1,0],[1,1,0]]
Output: 4

Example 3:

Input: grid = [[1,0,0],[1,1,0],[1,1,0]]
Output: -1

Constraints:

- $n == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq n \leq 100$
- $\text{grid}[i][j]$ is 0 or 1

Round 2 · High · Medium

p.1077

Community Solutions (Python)

• LeetCode

```
class Solution:
    def shortestPathBinaryMatrix(self, grid: List[List[int]]) -> int:
        if grid[0][0]:
            return -1
        n = len(grid)
        grid[0][0] = 1
        q = deque([(0, 0)])
        ans = 1
        while q:
            for _ in range(len(q)):
                i, j = q.popleft()
                if i == j == n - 1:
                    return ans
                for x in range(i - 1, i + 2):
                    for y in range(j - 1, j + 2):
                        if 0 <= x < n and 0 <= y < n and grid[x][y] == 0:
                            grid[x][y] = 1
                            q.append((x, y))
            ans += 1
        return -1
```

• SimplyLeet

```
from collections import deque

def shortestPathBinaryMatrix(grid):
    n = len(grid)
    if grid[0][0] != 0 or grid[n-1][n-1] != 0:
        return -1

    # Directions: 8 directions (vertical, horizontal, diagonal)
    directions = [(-1, -1), (-1, 0), (-1, 1), (0, -1), (0, 1), (1, -1), (1, 0), (1, 1)]
```

Round 2 · High · Medium

p.1078

```

# BFS initialization
queue = deque([(0, 0, 1)]) # (row, column, path_length)
grid[0][0] = 1 # mark as visited

while queue:
    row, col, path_length = queue.popleft()
    if row == n - 1 and col == n - 1:
        return path_length

    # Explore all neighbors

    for dr, dc in directions:
        new_row, new_col = row + dr, col + dc
        if 0 <= new_row < n and 0 <= new_col < n and grid[new_row][new_col] == 0:
            queue.append((new_row, new_col, path_length + 1))
            grid[new_row][new_col] = 1 # mark as visited

    return -1

# Example usage:
# grid = [[0,1],[1,0]]

# print(shortestPathBinaryMatrix(grid))

```

Breadth First Search (BFS)

#1102 Path With Maximum Minimum Value

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Binary Search · Depth-First Search ·
 Breadth-First Search · Union-Find · Heap
 (Priority Queue) · Matrix
 Lists: AlgoMaster 600

Problem

Given an $m \times n$ integer matrix `grid`, return the maximum score of a path starting at $(0, 0)$ and ending at $(m - 1, n - 1)$ moving in the 4 cardinal directions.

The score of a path is the minimum value in that path.

- For example, the score of the path $8 \rightarrow 4 \rightarrow 5 \rightarrow 9$ is 4.

Example 1:

5	4	5
1	2	6
7	4	6

Input: grid = [[5,4,5],[1,2,6],[7,4,6]]

Output: 4

Explanation: The path with the maximum score is highlighted in yellow.

Example 2:

2	2	1	2	2	2
1	2	2	2	1	2

Round 2 · High · Medium

p.1081

Input: grid = [[2,2,1,2,2,2],[1,2,2,2,1,2]]

Output: 2

Example 3:

3	4	6	3	4
0	2	1	1	7
8	8	3	2	7
3	2	4	9	8
4	1	2	0	0
4	6	5	4	3

Input: grid =

[[3,4,6,3,4],[0,2,1,1,7],[8,8,3,2,7],[3,2,4,9,8],[4,1,2,0,0],[4,6,5,4,3]]

Output: 3

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq m, n \leq 100$

Round 2 · High · Medium

p.1082

- $0 \leq \text{grid}[i][j] \leq 109$

Community Solutions (Python)

• LeetDoocs

```
class Solution:
    def maximumMinimumPath(self, grid: List[List[int]]) -> int:
        def find(x: int) -> int:
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        m, n = len(grid), len(grid[0])
        p = list(range(m * n))
        q = [(v, i, j) for i, row in enumerate(grid) for j, v in enumerate(row)
            ]

        q.sort()
        ans = 0
        dirs = (-1, 0, 1, 0, -1)
        vis = set()
        while find(0) != find(m * n - 1):
            v, i, j = q.pop()
            ans = v
            vis.add((i, j))
            for a, b in pairwise(dirs):
                x, y = i + a, j + b

                if (x, y) in vis:
                    p[find(i * n + j)] = find(x * n + y)
        return ans
```

```
class UnionFind:
    __slots__ = ("p", "size")

    def __init__(self, n):
        self.p = list(range(n))
        self.size = [1] * n

    def find(self, x: int) -> int:
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])

        return self.p[x]

    def union(self, a: int, b: int) -> bool:
        pa, pb = self.find(a), self.find(b)
        if pa == pb:
            return False
        if self.size[pa] > self.size[pb]:
            self.p[pb] = pa
            self.size[pa] += self.size[pb]
        else:
            self.p[pa] = pb
            self.size[pb] += self.size[pa]
        return True
```

```
class Solution:
    def maximumMinimumPath(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        uf = UnionFind(m * n)
        q = [(v, i, j) for i, row in enumerate(grid) for j, v in enumerate(row)
            ]
```

Round 2 · High · Medium

p.1083

Round 2 · High · Medium

p.1084

```

q.sort()
ans = 0
vis = set()
dirs = (-1, 0, 1, 0, -1)
while uf.find(0) != uf.find(m * n - 1):
    v, i, j = q.pop()
    ans = v
    vis.add((i, j))
    for a, b in pairwise(dirs):
        x, y = i + a, j + b

        if (x, y) in vis:
            uf.union(x * n + y, i * n + j)
return ans

```

• SimplyLeet

```

import heapq

def maximumMinimumPath(grid):
    m, n = len(grid), len(grid[0])
    # Max-heap: use negative values because Python's heapq is a min-heap.
    heap = [(-grid[0][0], 0, 0)]
    visited = [[False] * n for _ in range(m)]
    visited[0][0] = True

    # Directions: up, down, left, right.
    directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]
    # Current answer is the start value.
    current_min = grid[0][0]

    while heap:
        # Get the cell with the maximum value (using negative because of
        # min-heap).
        val, i, j = heapq.heappop(heap)
        # Reverse the sign to get the actual value.
        val = -val
        # Update the current path min.

```

Round 2 · High · Medium

p.1085

```

current_min = min(current_min, val)
# If destination is reached, return current path's min value.
if i == m - 1 and j == n - 1:
    return current_min

for di, dj in directions:
    ni, nj = i + di, j + dj
    if 0 <= ni < m and 0 <= nj < n and not visited[ni][nj]:
        visited[ni][nj] = True
        # Push neighbor cell into heap.

        heapq.heappush(heap, (-grid[ni][nj], ni, nj))

return -1 # Should never reach here if input grid is valid.

# Example test
print(maximumMinimumPath([[5,4,5],[1,2,6],[7,4,6]]))

```

Round 2 · High · Medium

p.1086

Breadth First Search (BFS)

★ #1197 Minimum Knight Moves ★

MED

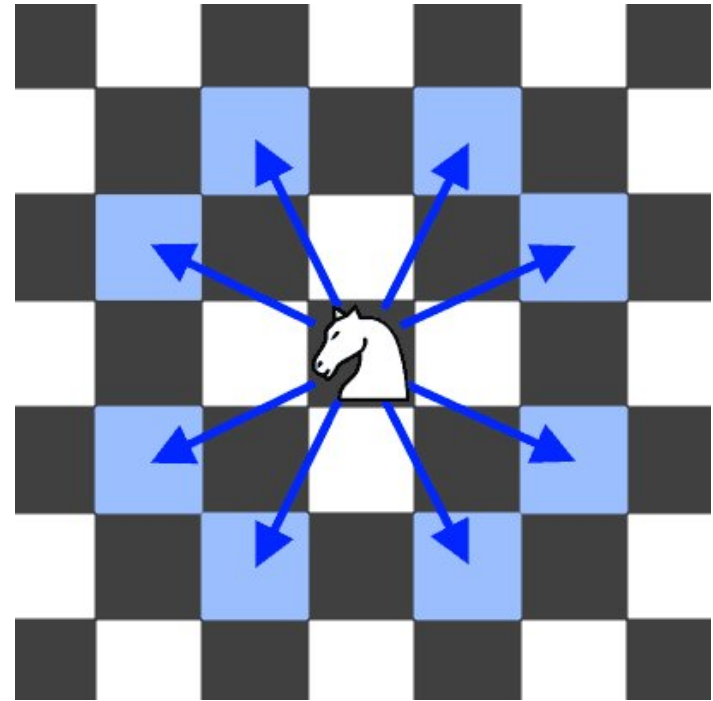
LeetDoocs · SimplyLeet · WalkCC · LeetCode
BFS

Lists: ★ Premium | Grind 169 | Premium 98

Problem

In an infinite chess board with coordinates from $-\infty$ to $+\infty$, you have a knight at square $[0, 0]$.

A knight has 8 possible moves it can make, as illustrated below. Each move is two squares in a cardinal direction, then one square in an orthogonal direction.



Return the minimum number of steps needed to move the knight to the square $[x, y]$. It is guaranteed the answer exists.

Example 1:

Input: $x = 2, y = 1$
Output: 1
Explanation: $[0, 0] \rightarrow [2, 1]$

Example 2:

Input: $x = 5, y = 5$
Output: 4
Explanation: $[0, 0] \rightarrow [2, 1] \rightarrow [4, 2] \rightarrow [3, 4] \rightarrow [5, 5]$

Constraints:

- $-300 \leq x, y \leq 300$
- $0 \leq |x| + |y| \leq 300$

Community Solutions (Python)**• LeetCode**

```
class Solution:
    def minKnightMoves(self, x: int, y: int) -> int:
        q = deque([(0, 0)])
        ans = 0
        vis = {(0, 0)}
        dirs = ((-2, 1), (-1, 2), (1, 2), (2, 1), (2, -1), (1, -2), (-1, -2),
                (-2, -1))
        while q:
            for _ in range(len(q)):
                i, j = q.popleft()
                if (i, j) == (x, y):
                    return ans
            for a, b in dirs:
                c, d = i + a, j + b
                if (c, d) not in vis:
                    vis.add((c, d))
                    q.append((c, d))
            ans += 1
        return -1
```

```
class Solution:
    def minKnightMoves(self, x: int, y: int) -> int:
        def extend(m1, m2, q):
            for _ in range(len(q)):
                i, j = q.popleft()
                step = m1[(i, j)]
                for a, b in (
                    (-2, 1),
                    (-1, 2),
                    (1, 2),
```

Round 2 · High · Medium

p.1089

```
(2, 1),
(2, -1),
(1, -2),
(-1, -2),
(-2, -1),
):
    x, y = i + a, j + b
    if (x, y) in m1:
        continue
    if (x, y) in m2:
```

```
        return step + 1 + m2[(x, y)]
    q.append((x, y))
    m1[(x, y)] = step + 1
    return -1
```

```
if (x, y) == (0, 0):
    return 0
q1, q2 = deque([(0, 0)]), deque([(x, y)])
m1, m2 = {(0, 0): 0}, {(x, y): 0}
while q1 and q2:
```

```
    t = extend(m1, m2, q1) if len(q1) <= len(q2) else extend(m2, m1, q
2)
    if t != -1:
        return t
    return -1
```

• SimplyLeet

```
# Python solution using BFS:
from collections import deque

def minKnightMoves(x, y):
    # Normalize target coordinates using symmetry
    x, y = abs(x), abs(y)

    # Possible knight moves (dx, dy)
    moves = [(2, 1), (1, 2), (-1, 2), (-2, 1), (-2, -1), (-1, -2), (1, -2), (2, -1)]
```

Round 2 · High · Medium

p.1090

```

# BFS queue stores tuples of the form (current_x, current_y, move_count)
queue = deque([(0, 0, 0)])

# Set to track visited positions to prevent cycles
visited = {(0, 0)}

# Process BFS
while queue:
    curr_x, curr_y, move_count = queue.popleft()

    # Return the current move count if target is reached
    if curr_x == x and curr_y == y:
        return move_count

    # Explore all knight moves
    for dx, dy in moves:
        next_x, next_y = curr_x + dx, curr_y + dy

        # Prune search space: only process positions with valid bounds
        if (next_x, next_y) not in visited and next_x >= -1 and next_y >= -1:

            visited.add((next_x, next_y))
            queue.append((next_x, next_y, move_count + 1))

    return -1 # Unreachable because a solution always exists

# Example test
#print(minKnightMoves(5, 5))

```

• LC.ca

Round 2 · High · Medium

p.1091

```

class Solution:
    def minKnightMoves(self, x: int, y: int) -> int:
        q = deque([(0, 0)])
        ans = 0
        vis = {(0, 0)}
        dirs = ((-2, 1), (-1, 2), (1, 2), (2, 1), (2, -1), (1, -2), (-1, -2), (-2, -1))

        while q:
            for _ in range(len(q)):
                i, j = q.popleft()
                if (i, j) == (x, y):
                    return ans

            for a, b in dirs:
                c, d = i + a, j + b
                if (c, d) not in vis:
                    vis.add((c, d))
                    q.append((c, d))

            ans += 1
        return -1

```

◆ Quick Review

Key Insights

- Leverage the symmetry of the chessboard by converting target coordinates to their absolute values.
- Use a Breadth-First Search (BFS) because all moves have equal weight, ensuring the first encounter with the target is the minimal path.
- Prune the search space by only considering positions that are likely beneficial (e.g., positions having coordinates not far less than

Round 2 · High · Medium

p.1092

-1).

- Track visited states to avoid redundant processing.
- Optimize by bounding the search region relative to the target distance.

Space & Time

Time Complexity: $O(n)$ where n is the number of positions processed during the BFS

Space Complexity: $O(n)$ due to the storage requirements for the BFS queue and the visited set

Solution

We solve the problem using a BFS starting from the origin. First, normalize the target by taking the absolute value of x and y . BFS is then applied by exploring all eight possible knight moves at each step. Each move generates a new position that is enqueued if it hasn't been visited and falls within a reasonable bound (e.g., coordinates ≥ -1). The search terminates as soon as the target position is reached.

Round 2 · High · Medium

p.1093

Linked List

Round 2 · High · Medium · 16 questions

Round 2 · High · Medium

p.1094

Linked List

#2 Add Two Numbers

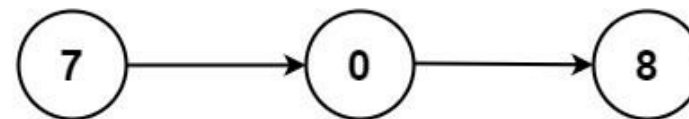
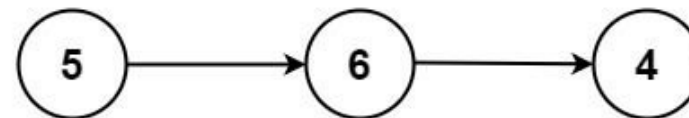
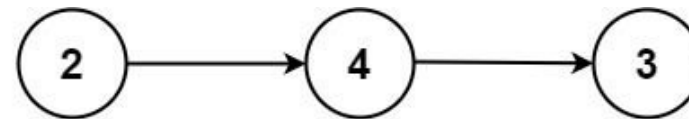
MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Linked List · Math · Recursion
 Lists: Grind 169

Problem

You are given two non-empty linked lists representing two non-negative integers. The digits are stored in reverse order, and each of their nodes contains a single digit. Add the two numbers and return the sum as a linked list. You may assume the two numbers do not contain any leading zero, except the number 0 itself.

Example 1:



Input: l1 = [2,4,3], l2 = [5,6,4]
 Output: [7,0,8]
 Explanation: 342 + 465 = 807.

Example 2:

Input: l1 = [0], l2 = [0]
 Output: [0]

Example 3:

Input: l1 = [9,9,9,9,9,9,9], l2 = [9,9,9,9]
 Output: [8,9,9,9,0,0,0,1]

Constraints:

- The number of nodes in each linked list is in the range [1, 100].

- $0 \leq \text{Node.val} \leq 9$
- It is guaranteed that the list represents a number that does not have leading zeros.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def addTwoNumbers(self, l1: ListNode, l2: ListNode) -> ListNode:
        dummy = ListNode(0)
        curr = dummy
        carry = 0

        while carry or l1 or l2:
            if l1:
                carry += l1.val
                l1 = l1.next

            if l2:
                carry += l2.val
                l2 = l2.next
            curr.next = ListNode(carry % 10)
            carry //= 10
            curr = curr.next

        return dummy.next
```

• LeetCode

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def addTwoNumbers(
        self, l1: Optional[ListNode], l2: Optional[ListNode]
    ) -> Optional[ListNode]:
        dummy = ListNode()
```

Round 2 · High · Medium

p.1097

```
carry, curr = 0, dummy
while l1 or l2 or carry:
    s = (l1.val if l1 else 0) + (l2.val if l2 else 0) + carry
    carry, val = divmod(s, 10)
    curr.next = ListNode(val)
    curr = curr.next
    l1 = l1.next if l1 else None
    l2 = l2.next if l2 else None
return dummy.next
```

• SimplyLeet

```
# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def addTwoNumbers(l1, l2):
    # Initialize a dummy head node to simplify the result list construction.
    dummy = ListNode()
    current = dummy

    carry = 0

    # Traverse through both linked lists until all nodes and any carry are
    # processed.
    while l1 or l2 or carry:
        # Get the current values; if a node is missing, use 0.
        val1 = l1.val if l1 else 0
        val2 = l2.val if l2 else 0

        # Calculate the sum of current digits and carry.
        total = val1 + val2 + carry
```

Round 2 · High · Medium

p.1098

```

carry = total // 10 # Update carry for next iteration.
digit = total % 10 # Compute current digit.

# Append the digit as a new node in the result list.
current.next = ListNode(digit)
current = current.next

# Move to the next nodes if available.
if l1:
    l1 = l1.next

if l2:
    l2 = l2.next

# Return the head of the newly formed linked list.
return dummy.next

```

• LC.ca

```

# Definition for singly-linked list.
# class ListNode:
# def __init__(self, val=0, next=None):
# self.val = val
# self.next = next
class Solution:
    def addTwoNumbers(
        self, l1: Optional[ListNode], l2: Optional[ListNode]
    ) -> Optional[ListNode]:
        dummy = ListNode()

        carry, curr = 0, dummy
        while l1 or l2 or carry:
            s = (l1.val if l1 else 0) + (l2.val if l2 else 0) + carry
            carry, val = divmod(s, 10)
            curr.next = ListNode(val)
            curr = curr.next
            l1 = l1.next if l1 else None
            l2 = l2.next if l2 else None
        return dummy.next

```

◆ Quick Review

Round 2 · High · Medium

p.1099

Key Insights

- The digits are stored in reverse order, meaning the first node is the least significant digit.
- A carry may need to propagate through subsequent nodes.
- The two linked lists might be of different lengths.
- Continue processing until both lists are fully traversed and any remaining carry is handled.

Space & Time

Time Complexity: $O(\max(n, m))$, where n and m are the lengths of the two linked lists.

Space Complexity: $O(\max(n, m) + 1)$ for the result list in the worst case.

Solution

The solution iterates through both linked lists node by node. For each pair of corresponding nodes, their values and any carry from the previous operation are summed. The result is decomposed into a current digit (modulo 10) and an updated carry (total divided by 10). A new node is created for the current digit and appended to the resulting linked list. This

Round 2 · High · Medium

p.1100

process continues until all nodes are processed and any leftover carry is added as a final node.

Linked List

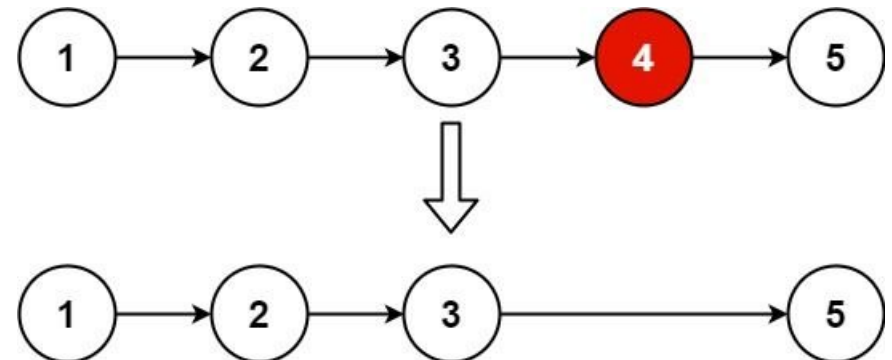
□ #19 Remove Nth Node From End of List **MED**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Linked List · Two Pointers
 Lists: Grind 169

Problem

Given the head of a linked list, remove the *n*th node from the end of the list and return its head.

Example 1:



Input: head = [1,2,3,4,5], n = 2
 Output: [1,2,3,5]

Example 2:

Input: head = [1], n = 1
Output: []

Example 3:

Input: head = [1,2], n = 1
Output: [1]

Constraints:

- The number of nodes in the list is sz.
- $1 \leq sz \leq 30$
- $0 \leq \text{Node.val} \leq 100$
- $1 \leq n \leq sz$

Follow up: Could you do this in one pass?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def removeNthFromEnd(self, head: ListNode, n: int) -> ListNode:
        slow = head
        fast = head

        for _ in range(n):
            fast = fast.next
        if not fast:
            return head.next

        while fast.next:
            slow = slow.next
            fast = fast.next
            slow.next = slow.next.next

        return head
```

Round 2 · High · Medium

p.1103

• LeetCode

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def removeNthFromEnd(self, head: Optional[ListNode], n: int) -> Optional[ListNode]:
        dummy = ListNode(next=head)
        fast = slow = dummy
        for _ in range(n):
            fast = fast.next
        while fast.next:
            slow, fast = slow.next, fast.next
        slow.next = slow.next.next
        return dummy.next
```

• SimplyLeet

```
# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val # Holds the value of the node
        self.next = next # Points to the next node in the list

class Solution:
    def removeNthFromEnd(self, head: ListNode, n: int) -> ListNode:
        # Create a dummy node to simplify edge cases (like removing the head)
        dummy = ListNode(0)

        dummy.next = head

        # Initialize two pointers starting at the dummy node
        first = dummy
        second = dummy

        # Advance first pointer n+1 steps ahead to maintain a gap of n nodes
        for _ in range(n + 1):
            first = first.next
```

Round 2 · High · Medium

p.1104

```
# Move both pointers until first reaches the end of the list
while first:
    first = first.next
    second = second.next

# Skip the node to remove it from the list
second.next = second.next.next

# Return the adjusted list starting from dummy.next
return dummy.next
```

• LC.ca

```
# Definition for singly-linked list.
# class ListNode:
# def __init__(self, val=0, next=None):
# self.val = val
# self.next = next
class Solution:
    def removeNthFromEnd(self, head: Optional[ListNode], n: int) -> Optional[ListNode]:
        dummy = ListNode(next=head)
        fast = slow = dummy
        for _ in range(n):
            fast = fast.next
        while fast.next:
            slow, fast = slow.next, fast.next
        slow.next = slow.next.next
        return dummy.next
```

◆ Quick Review

Key Insights

- Use a dummy node at the start to simplify edge cases, especially when the head is removed.

Round 2 · High · Medium

p.1105

- Utilize two pointers (fast and slow) and maintain a gap of $n+1$ nodes between them.
- By moving both pointers simultaneously, the slow pointer will point to the node immediately before the target node when the fast pointer reaches the end.

Space & Time

Time Complexity: $O(L)$, where L is the length of the list

Space Complexity: $O(1)$

Solution

We use a two-pointer approach to achieve a one-pass solution. A dummy node is created and linked to the head to handle edge cases gracefully (such as needing to remove the head). The fast pointer is advanced $n+1$ steps ahead of the slow pointer to maintain the required gap. Then, both pointers are moved simultaneously until the fast pointer reaches the end of the list. At that point, the slow pointer is immediately before the node to be deleted. We then adjust the pointers to remove the target node from the list.

Round 2 · High · Medium

p.1106

Linked List

#24 Swap Nodes in Pairs

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Linked List · Recursion
 Lists: Grind 169

Problem

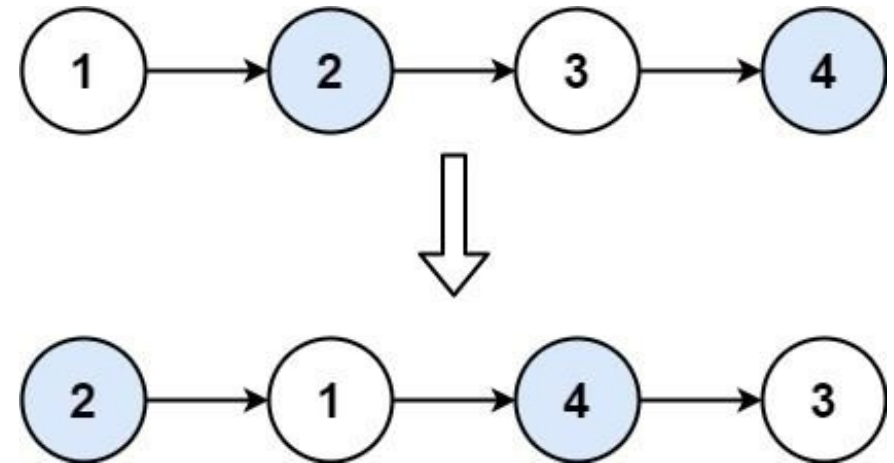
Given a linked list, swap every two adjacent nodes and return its head. You must solve the problem without modifying the values in the list's nodes (i.e., only nodes themselves may be changed.)

Example 1:

Input: head = [1,2,3,4]

Output: [2,1,4,3]

Explanation:



Example 2:

Input: head = []

Output: []

Example 3:

Input: head = [1]

Output: [1]

Example 4:

Input: head = [1,2,3]

Output: [2,1,3]

Constraints:

- The number of nodes in the list is in the range [0, 100].
- $0 \leq \text{Node.val} \leq 100$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def swapPairs(self, head: ListNode) -> ListNode:
        def getLength(head: ListNode) -> int:
            length = 0
            while head:
                length += 1
                head = head.next
            return length

        length = getLength(head)

        dummy = ListNode(0, head)
        prev = dummy
        curr = head

        for _ in range(length // 2):
            next = curr.next
            curr.next = next.next
            next.next = prev.next
            prev.next = next
            prev = curr

            curr = curr.next

        return dummy.next
```

• LeetDoocs

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def swapPairs(self, head: Optional[ListNode]) -> Optional[ListNode]:
        if head is None or head.next is None:
            return head
        t = self.swapPairs(head.next.next)
```

Round 2 · High · Medium

p.1109

```
p = head.next
p.next = head
head.next = t
return p
```

• SimplyLeet

```
# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def swapPairs(self, head: ListNode) -> ListNode:
        # Create a dummy node and point it to the head of the list.
        dummy = ListNode(0)

        dummy.next = head
        # Initialize the pointer 'prev' to dummy.
        prev = dummy

        # Traverse the list while at least two nodes remain.
        while head and head.next:
            # Identify the two nodes to swap.
            first_node = head
            second_node = head.next
```

Round 2 · High · Medium

p.1110

```

    # Swapping the nodes by reassigning pointers.
    prev.next = second_node      # Link previous node to second
node.
    first_node.next = second_node.next # Link first node to node after
second.
    second_node.next = first_node    # Link second node to first to
complete swap.

    # Update the pointers for next pair processing.
    prev = first_node             # Move 'prev' pointer to the end
of the swapped pair.
    head = first_node.next        # Move 'head' pointer to the next
pair.

    # Return the new head of the list.

return dummy.next

```

• LC.ca

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def swapPairs(self, head: Optional[ListNode]) -> Optional[ListNode]:
        dummy = ListNode(next=head)
        pre, cur = dummy, head
        while cur and cur.next:
            t = cur.next
            cur.next = t.next
            t.next = cur
            pre.next = t
            pre, cur = cur, cur.next
        return dummy.next

```

◆ Quick Review

Key Insights

Round 2 · High · Medium

p.1111

- Use a dummy node to simplify edge cases, such as when the list is empty or has only one node.
- Process nodes in pairs and update the pointers accordingly.
- Ensure that the swapping does not lose the connection between the swapped pair and the rest of the list.
- The solution can be implemented iteratively or recursively.

Space & Time

Time Complexity: $O(n)$ – Each node is processed once.

Space Complexity: $O(1)$ – Only constant extra space is used.

Solution

We use an iterative approach with a dummy node. The dummy node points to the head of the list. We maintain a pointer that traverses the list in pairs. For each pair, we adjust the next pointers to swap the two nodes. By the end, the dummy's next pointer will be the new head of the modified list. The main idea is to keep track of the previous node (prev) before

Round 2 · High · Medium

p.1112

the pair, and then adjust pointers accordingly using temporary variables to hold references to the nodes being swapped.

Linked List

□ #61 Rotate List

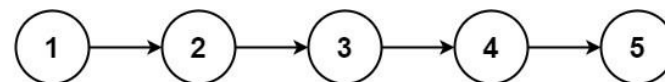
MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Linked List · Two Pointers
 Lists: Grind 169

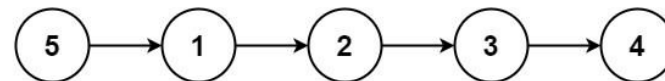
Problem

Given the head of a linked list, rotate the list to the right by k places.

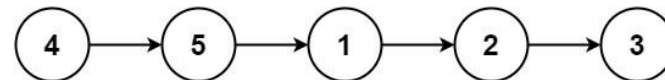
Example 1:



rotate 1

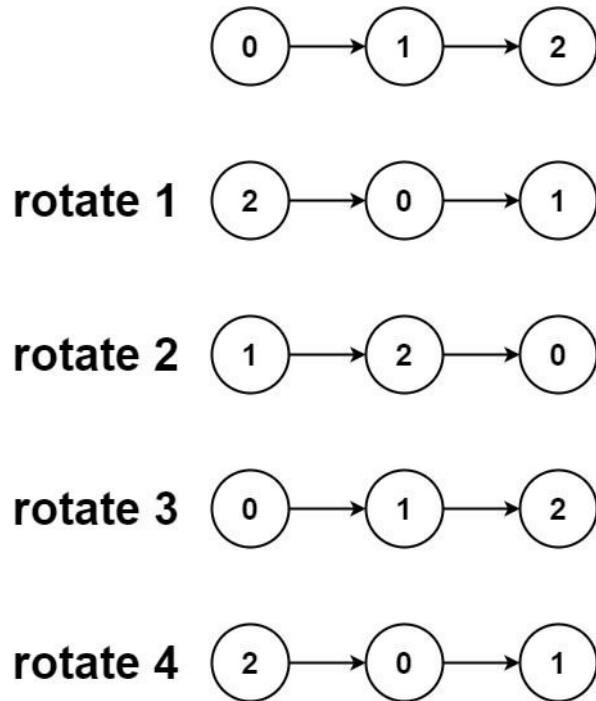


rotate 2



Input: head = [1,2,3,4,5], k = 2
 Output: [4,5,1,2,3]

Example 2:



Input: head = [0,1,2], k = 4
Output: [2,0,1]

Constraints:

- The number of nodes in the list is in the range [0, 500].
- $-100 \leq \text{Node.val} \leq 100$
- $0 \leq k \leq 2 * 10^9$

Community Solutions (Python)

- WalkCC

Round 2 · High · Medium

p.1115

```

class Solution:
    def rotateRight(self, head: ListNode, k: int) -> ListNode:
        if not head or not head.next or k == 0:
            return head

        tail = head
        length = 1
        while tail.next:
            tail = tail.next
            length += 1

        tail.next = head # Circle the list.

        t = length - k % length
        for _ in range(t):
            tail = tail.next
        newHead = tail.next
        tail.next = None

        return newHead
  
```

• LeetCode

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def rotateRight(self, head: Optional[ListNode], k: int) -> Optional[ListNode]:
        if head is None or head.next is None:
            return head
        cur, n = head, 0
  
```

Round 2 · High · Medium

p.1116

```

while cur:
    n += 1
    cur = cur.next
k %= n
if k == 0:
    return head
fast = slow = head
for _ in range(k):
    fast = fast.next
while fast.next:

    fast, slow = fast.next, slow.next

ans = slow.next
slow.next = None
fast.next = head
return ans

```

• SimplyLeet

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, x):
#         self.val = x
#         self.next = None

class Solution:
    def rotateRight(self, head: ListNode, k: int) -> ListNode:
        if not head or not head.next or k == 0: # Check if rotation is
necessary
            return head

        # Compute the length of the list and locate the tail node
        length = 1
        tail = head
        while tail.next:
            tail = tail.next
            length += 1

        # Calculate the effective number of rotations
        k = k % length

```

Round 2 · High · Medium

p.1117

```

if k == 0:
    return head

# Find the new tail: (length - k - 1) steps from the head
stepsToNewTail = length - k - 1
newTail = head
for _ in range(stepsToNewTail):
    newTail = newTail.next

# Set the new head and break the list

newHead = newTail.next
newTail.next = None

# Connect the old tail to the original head
tail.next = head

return newHead

```

• LC.ca

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def rotateRight(self, head: Optional[ListNode], k: int) -> Optional[ListNo
del]:
        if head is None or head.next is None:
            return head
        cur, n = head, 0

        while cur:
            n += 1
            cur = cur.next
        k %= n
        if k == 0:
            return head
        fast = slow = head
        for _ in range(k):
            fast = fast.next
        while fast.next:

```

Round 2 · High · Medium

p.1118

```
fast, slow = fast.next, slow.next
```

```
ans = slow.next
slow.next = None
fast.next = head
return ans
```

◆ Quick Review

Key Insights

- Calculate the length of the list and locate the tail.
- Use k modulo length to handle rotations greater than the list size.
- Find the new tail of the list at position $(\text{length} - k - 1)$ and update pointers accordingly.
- Link the original tail to the original head to form a circular list, then break the circle at the new tail.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes in the list, since we traverse the list a few times.

Space Complexity: $O(1)$, as the rotation is done in place without extra data structures.

Round 2 · High · Medium

p.1119

Solution

We first traverse the list to determine its length and identify the tail node. The effective rotation is calculated by taking k modulo the length of the list. If the result is zero, the list remains unchanged. Otherwise, we locate the new tail node by moving $(\text{length} - k - 1)$ steps from the head. The node after the new tail becomes the new head. Finally, we break the link after the new tail and connect the old tail to the original head to finalize the rotated list.

Round 2 · High · Medium

p.1120

Linked List

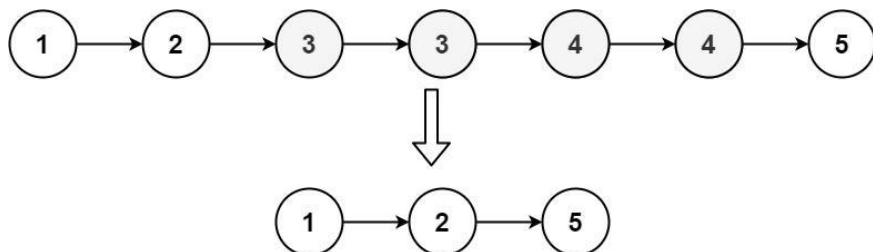
#82 Remove Duplicates from Sorted List II MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Linked List · Two Pointers
 Lists: AlgoMaster 600

Problem

Given the head of a sorted linked list, delete all nodes that have duplicate numbers, leaving only distinct numbers from the original list. Return the linked list sorted as well.

Example 1:

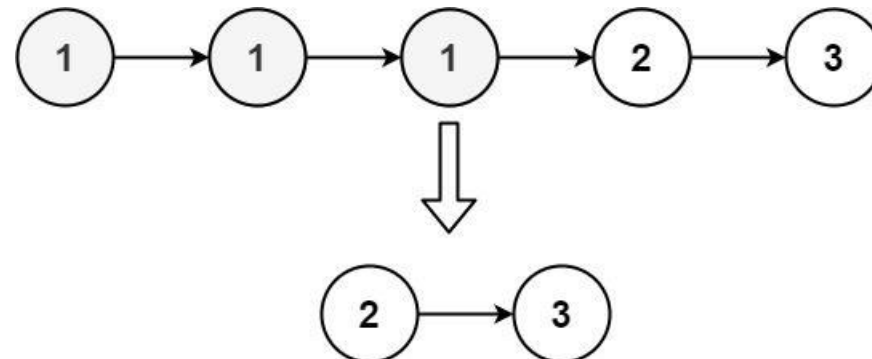


Input: head = [1,2,3,3,4,4,5]
 Output: [1,2,5]

Example 2:

Round 2 · High · Medium

p.1121



Input: head = [1,1,1,2,3]
 Output: [2,3]

Constraints:

- The number of nodes in the list is in the range [0, 300].
- $-100 \leq \text{Node.val} \leq 100$
- The list is guaranteed to be sorted in ascending order.

Community Solutions (Python)

• WalkCC

```

class Solution:
    def deleteDuplicates(self, head: ListNode) -> ListNode:
        dummy = ListNode(0, head)
        prev = dummy

        while head:
            while head.next and head.val == head.next.val:
                head = head.next
            if prev.next == head:
                prev = prev.next
  
```

Round 2 · High · Medium

p.1122

```

else:
    prev.next = head.next
    head = head.next

return dummy.next

```

• LeetCode

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def deleteDuplicates(self, head: Optional[ListNode]) -> Optional[ListNode]:
        :
        dummy = pre = ListNode(next=head)
        cur = head
        while cur:

            while cur.next and cur.next.val == cur.val:
                cur = cur.next
            if pre.next == cur:
                pre = cur
            else:
                pre.next = cur.next
                cur = cur.next
            return dummy.next

```

• SimplyLeet

```

# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def deleteDuplicates(self, head: ListNode) -> ListNode:
        # Create a dummy node to handle edge cases.
        dummy = ListNode(0)

```

Round 2 · High · Medium

p.1123

```

dummy.next = head

# Initialize pointers: prev for the last confirmed non-duplicate node,
current for traversal.
prev = dummy
current = head

while current:
    # If the current node is the start of duplicates.
    if current.next and current.val == current.next.val:
        duplicate_val = current.val

        # Skip all nodes with this duplicate value.
        while current and current.val == duplicate_val:
            current = current.next
        # Link the last non-duplicate node to the next distinct node.
        prev.next = current
    else:
        # No duplicate: move prev pointer to current.
        prev = current
        current = current.next

# Return the modified linked list.
return dummy.next

```

Round 2 · High · Medium

p.1124

Linked List

#86 Partition List

MED

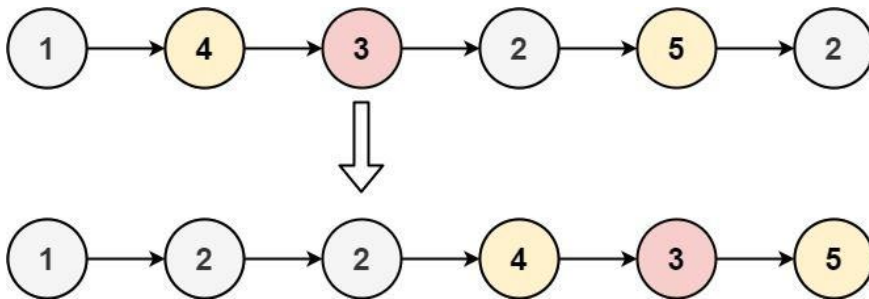
LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Linked List · Two Pointers
 Lists: AlgoMaster 600

Problem

Given the head of a linked list and a value x , partition it such that all nodes less than x come before nodes greater than or equal to x .

You should preserve the original relative order of the nodes in each of the two partitions.

Example 1:



Input: head = [1,4,3,2,5,2], x = 3
 Output: [1,2,2,4,3,5]

Example 2:

Round 2 · High · Medium

p.1125

Input: head = [2,1], x = 2
 Output: [1,2]

Constraints:

- The number of nodes in the list is in the range $[0, 200]$.
- $-100 \leq \text{Node.val} \leq 100$
- $-200 \leq x \leq 200$

Community Solutions (Python)

• WalkCC

```

class Solution:
    def partition(self, head: ListNode, x: int) -> ListNode:
        beforeHead = ListNode(0)
        afterHead = ListNode(0)
        before = beforeHead
        after = afterHead

        while head:
            if head.val < x:
                before.next = head
                before = before.next
            else:
                after.next = head
                after = after.next
            head = head.next

        after.next = None
        beforeHead.next = afterHead.next
        return beforeHead.next
  
```

• LeetDoocs

Round 2 · High · Medium

p.1126

```
# Definition for singly-linked list.
# class ListNode:
# def __init__(self, val=0, next=None):
# self.val = val
# self.next = next
class Solution:
    def partition(self, head: Optional[ListNode], x: int) -> Optional[ListNode]:
        l = ListNode()
        r = ListNode()
        tl, tr = l, r

        while head:
            if head.val < x:
                tl.next = head
                tl = tl.next
            else:
                tr.next = head
                tr = tr.next
            head = head.next
        tr.next = None
        tl.next = r.next

        return l.next
```

• SimplyLeet

```
# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def partition(self, head: ListNode, x: int) -> ListNode:
        # Initialize two dummy nodes for less and greater lists
        less_dummy = ListNode(0)
```

Round 2 · High · Medium

p.1127

```
greater_dummy = ListNode(0)
less = less_dummy # pointer for less-than list
greater = greater_dummy # pointer for greater or equal list

# Traverse through original list and partition nodes
while head:
    if head.val < x:
        less.next = head # add node to less list
        less = less.next # move pointer forward
    else:
        greater.next = head # add node to greater list
        greater = greater.next # move pointer forward
    head = head.next # advance to the next node in the list

# Terminate the greater list to avoid cycles
greater.next = None
# Link the less list with the greater list
less.next = greater_dummy.next

# Return the start of the new list

return less_dummy.next
```

Round 2 · High · Medium

p.1128

Linked List

#138 Copy List with Random Pointer

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Linked List
Lists: NeetCode 150

Problem

A linked list of length n is given such that each node contains an additional random pointer, which could point to any node in the list, or null. Construct a deep copy of the list. The deep copy should consist of exactly n brand new nodes, where each new node has its value set to the value of its corresponding original node. Both the next and random pointer of the new nodes should point to new nodes in the copied list such that the pointers in the original list and copied list represent the same list state. None of the pointers in the new list should point to nodes in the original list.

For example, if there are two nodes X and Y in the original list, where $X.random \rightarrow Y$, then for the corresponding two nodes x and y in the

Round 2 · High · Medium

p.1129

copied list, $x.random \rightarrow y$.

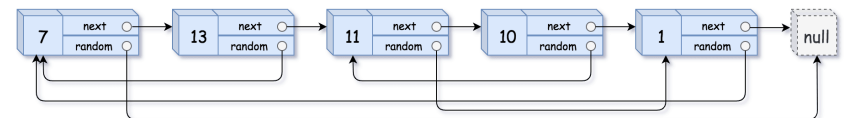
Return the head of the copied linked list.

The linked list is represented in the input/output as a list of n nodes. Each node is represented as a pair of $[val, random_index]$ where:

- **val**: an integer representing `Node.val`
- **random_index**: the index of the node (range from 0 to $n-1$) that the random pointer points to, or null if it does not point to any node.

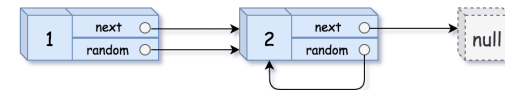
Your code will only be given the head of the original linked list.

Example 1:



Input: head = [[7,null],[13,0],[11,4],[10,2],[1,0]]
Output: [[7,null],[13,0],[11,4],[10,2],[1,0]]

Example 2:

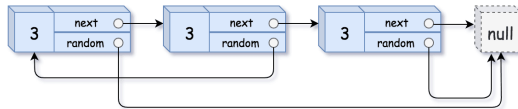


Round 2 · High · Medium

p.1130

Input: head = [[1,1],[2,1]]
Output: [[1,1],[2,1]]

Example 3:



Input: head = [[3,null],[3,0],[3,null]]
Output: [[3,null],[3,0],[3,null]]

Constraints:

- $0 \leq n \leq 1000$
- $-104 \leq \text{Node.val} \leq 104$
- **Node.random** is null or is pointing to some node in the linked list.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def copyRandomList(self, head: 'Node') -> 'Node':
        if not head:
            return None
        if head in self.map:
            return self.map[head]

        newNode = Node(head.val)
        self.map[head] = newNode
        newNode.next = self.copyRandomList(head.next)
```

Round 2 · High · Medium

p.1131

```
newNode.random = self.copyRandomList(head.random)
return newNode
```

```
map = {}
```

• LeetDoocs

```
#####
# Definition for a Node.
class Node:
    def __init__(self, x: int, next: 'Node' = None, random: 'Node' = None):
        self.val = int(x)
        self.next = next
        self.random = random
#####

class Solution:
    def copyRandomList(self, head: "Optional[Node]") -> "Optional[Node]":
        d = {}
        dummy = tail = Node(0)
        cur = head
        while cur:
            node = Node(cur.val)
            tail.next = node
            tail = tail.next
            d[cur] = node

            cur = cur.next
        cur = head
        while cur:
            d[cur].random = d[cur.random] if cur.random else None
            cur = cur.next
        return dummy.next
```

• SimplyLeet

Round 2 · High · Medium

p.1132

```
# Definition for a Node.
class Node:
    def __init__(self, val, next=None, random=None):
        self.val = val      # value of the node.
        self.next = next    # pointer to the next node.
        self.random = random # pointer to a random node.
```

```
class Solution:
    def copyRandomList(self, head: 'Node') -> 'Node':
        # Return None if the head is None.
```

```
    if not head:
        return None
```

```
    # Dictionary to hold old node as key and new node as value.
    old_to_new = {}
```

```
    # First pass: create a copy of each node.
    current = head
    while current:
        old_to_new[current] = Node(current.val)
```

```
        current = current.next
```

```
    # Second pass: assign next and random pointers for the copied nodes.
    current = head
    while current:
        # Using dict.get ensures proper assignment or returns None.
        old_to_new[current].next = old_to_new.get(current.next)
        old_to_new[current].random = old_to_new.get(current.random)
        current = current.next
```

```
    # Return the head of the new copied linked list.
    return old_to_new[head]
```

Linked List

□ #143 Reorder List

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Linked List · Two Pointers · Stack · Recursion
 Lists: Grind 169

Problem

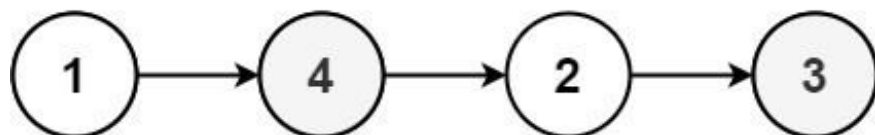
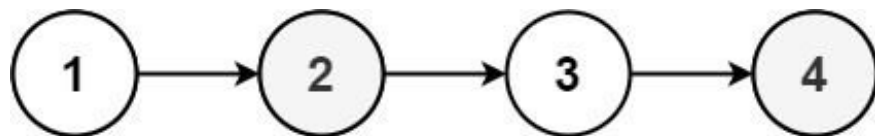
You are given the head of a singly linked-list.
 The list can be represented as:

$$L_0 \rightarrow L_1 \rightarrow \dots \rightarrow L_{n-1} \rightarrow L_n$$

Reorder the list to be on the following form:

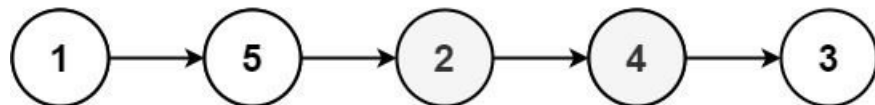
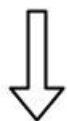
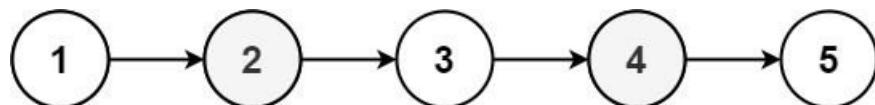
$$L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow L_2 \rightarrow L_{n-2} \rightarrow \dots$$

You may not modify the values in the list's
 nodes. Only nodes themselves may be changed.
 Example 1:



Input: head = [1,2,3,4]
Output: [1,4,2,3]

Example 2:



Input: head = [1,2,3,4,5]
Output: [1,5,2,4,3]

Constraints:

Round 2 · High · Medium

p.1135

- The number of nodes in the list is in the range [1, 5 * 10⁴].
- 1 ≤ Node.val ≤ 1000

Community Solutions (Python)

• WalkCC

```

class Solution:
    def reorderList(self, head: ListNode) -> None:
        def findMid(head: ListNode):
            prev = None
            slow = head
            fast = head

            while fast and fast.next:
                prev = slow
                slow = slow.next
                fast = fast.next.next

            prev.next = None

            return slow

        def reverse(head: ListNode) -> ListNode:
            prev = None
            curr = head

            while curr:
                next = curr.next
                curr.next = prev
                prev = curr
                curr = next

            return prev

        def merge(l1: ListNode, l2: ListNode) -> None:
            while l2:
                next = l1.next
                l1.next = l2
                l2 = l2.next
  
```

Round 2 · High · Medium

p.1136

```

l1.next = l2
l1 = l2
l2 = next

if not head or not head.next:
    return

mid = findMid(head)
reversed = reverse(mid)
merge(head, reversed)

```

• LeetCode

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def reorderList(self, head: Optional[ListNode]) -> None:
        fast = slow = head
        while fast.next and fast.next.next:
            slow = slow.next

```

```

        fast = fast.next.next

```

```

        cur = slow.next
        slow.next = None

```

```

        pre = None
        while cur:
            t = cur.next
            cur.next = pre
            pre, cur = cur, t

```

```

        cur = head
        while pre:
            t = pre.next
            pre.next = cur.next
            cur.next = pre
            cur, pre = pre.next, t

```

Round 2 · High · Medium

p.1137

• SimplyLeet

```

# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def reorderList(self, head: ListNode) -> None:
        if not head or not head.next:
            return

```

```

        # Step 1: Find the middle of the linked list using slow and fast
        pointers.

```

```

        slow, fast = head, head
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next

```

```

        # Step 2: Reverse the second half of the list.
        prev, curr = None, slow.next
        slow.next = None # Split the list into two halves.

```

```

        while curr:
            next_temp = curr.next # Save next node.
            curr.next = prev      # Reverse current pointer.
            prev = curr           # Move prev to current.
            curr = next_temp      # Move to the next node.

```

```

        # Step 3: Merge the two halves, alternating nodes.
        first, second = head, prev
        while second:
            temp1 = first.next # Save next node from first half.

```

```

            temp2 = second.next # Save next node from reversed second half.
            first.next = second # Link first node to second list node.
            second.next = temp1 # Link second node to the next node in first
            half.

```

```

            first = temp1 # Move pointer in first half.
            second = temp2 # Move pointer in second half.

```

Round 2 · High · Medium

p.1138

• LC.ca

```
# Definition for singly-linked list.
# class ListNode:
# def __init__(self, val=0, next=None):
# self.val = val
# self.next = next
class Solution:
    def reorderList(self, head: Optional[ListNode]) -> None:
        fast = slow = head
        while fast.next and fast.next.next:
            slow = slow.next

        fast = fast.next.next

        cur = slow.next
        slow.next = None

        pre = None
        while cur:
            t = cur.next
            cur.next = pre
            pre, cur = cur, t

        cur = head

        while pre:
            t = pre.next
            pre.next = cur.next
            cur.next = pre
            cur, pre = pre.next, t
```

◆ Quick Review

Key Insights

- Find the middle of the linked list using the fast and slow pointer technique.
- Reverse the second half of the list.

Round 2 · High · Medium

p.1139

- Merge the two halves, alternating nodes from the first half and the reversed second half.
- The challenge is done in-place, ensuring a linear time solution with constant extra space.

Space & Time

Time Complexity: $O(n)$ – Each of the three main steps (finding the middle, reversing, merging) takes $O(n)$ time.

Space Complexity: $O(1)$ – The algorithm is done in-place with only a few pointers.

Solution

The solution involves three key steps:

- Use two pointers (slow and fast) to identify the middle of the list.
- Reverse the second half of the list. This can be done iteratively by adjusting the next pointers.
- Merge the two lists: One starting from the head and the other from the head of the reversed second half. Interleave the nodes by alternating between the two lists until complete. This approach uses in-place pointer manipulation, thus ensuring no extra space is used aside from a few temporary variables.

Round 2 · High · Medium

p.1140

Linked List

#237 Delete Node in a Linked List

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode

Linked List

Lists: AlgoMaster 600

Problem

There is a singly-linked list head and we want to delete a node node in it.

You are given the node to be deleted node. You will not be given access to the first node of head.

All the values of the linked list are unique, and it is guaranteed that the given node node is not the last node in the linked list.

Delete the given node. Note that by deleting the node, we do not mean removing it from memory.

We mean:

- The value of the given node should not exist in the linked list.
- The number of nodes in the linked list should decrease by one.

Round 2 · High · Medium

p.1141

- All the values before node should be in the same order.
- All the values after node should be in the same order.

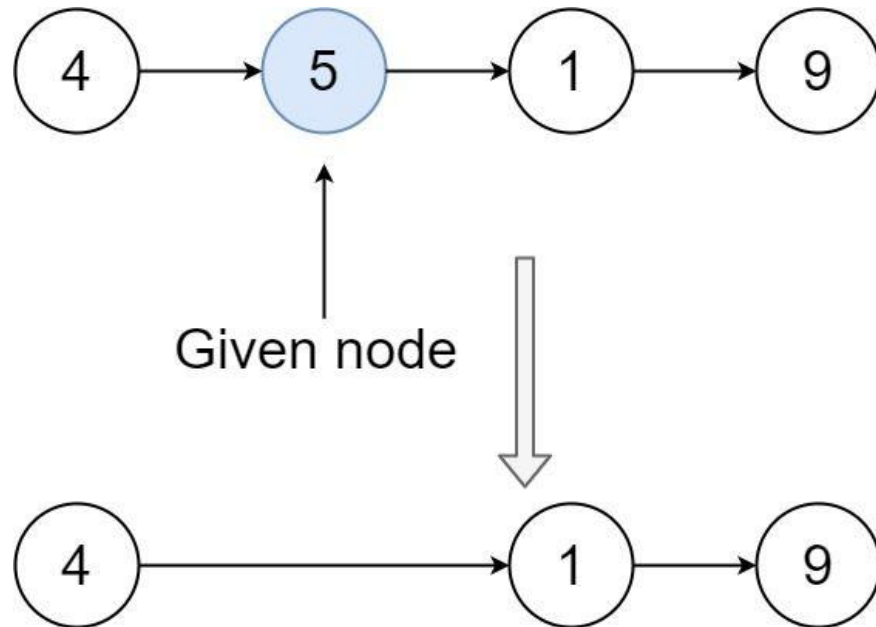
Custom testing:

- For the input, you should provide the entire linked list head and the node to be given node. node should not be the last node of the list and should be an actual node in the list.
- We will build the linked list and pass the node to your function.
- The output will be the entire list after calling your function.

Example 1:

Round 2 · High · Medium

p.1142



Input: head = [4,5,1,9], node = 5

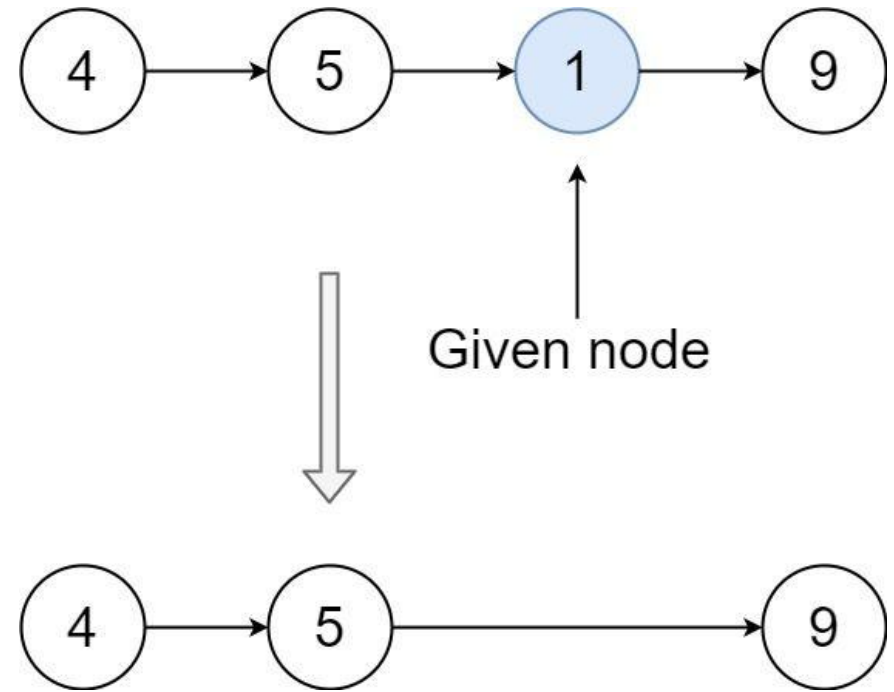
Output: [4,1,9]

Explanation: You are given the second node with value 5, the linked list should become 4 → 1 → 9 after calling your function.

Example 2:

Round 2 · High · Medium

p.1143



Input: head = [4,5,1,9], node = 1

Output: [4,5,9]

Explanation: You are given the third node with value 1, the linked list should become 4 → 5 → 9 after calling your function.

Constraints:

- The number of the nodes in the given list is in the range [2, 1000].
- $-1000 \leq \text{Node.val} \leq 1000$
- The value of each node in the list is unique.
- The node to be deleted is in the list and is not a tail node.

Round 2 · High · Medium

p.1144

Community Solutions (Python)

• WalkCC

```
class Solution:
    def deleteNode(self, node):
        node.val = node.next.val
        node.next = node.next.next
```

• LeetCode

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, x):
#         self.val = x
#         self.next = None

class Solution:
    def deleteNode(self, node):
        """
        :type node: ListNode
        :rtype: void Do not return anything, modify node in-place instead.
        """
        node.val = node.next.val
        node.next = node.next.next
```

• SimplyLeet

```
# Function to delete a node in a linked list.
def deleteNode(node):
    # Copy the next node's value into the current node.
    node.val = node.next.val
    # Bypass the next node by pointing to the next node's next.
    node.next = node.next.next
```

Linked List

□ #328 Odd Even Linked List

MED

LeetCode · SimplyLeet · WalkCC · LeetCode
Linked List

Lists: Grind 169

Problem

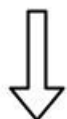
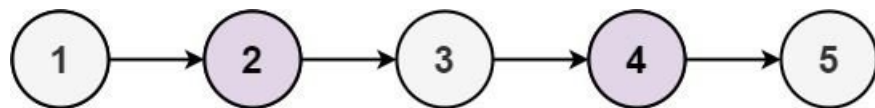
Given the head of a singly linked list, group all the nodes with odd indices together followed by the nodes with even indices, and return the reordered list.

The first node is considered odd, and the second node is even, and so on.

Note that the relative order inside both the even and odd groups should remain as it was in the input.

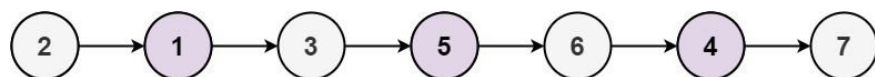
You must solve the problem in $O(1)$ extra space complexity and $O(n)$ time complexity.

Example 1:



Input: head = [1,2,3,4,5]
Output: [1,3,5,2,4]

Example 2:



Input: head = [2,1,3,5,6,4,7]
Output: [2,3,6,7,1,5,4]

Constraints:

- The number of nodes in the linked list is in the range [0, 104].
- $-106 \leq \text{Node.val} \leq 106$

Community Solutions (Python)

Round 2 · High · Medium

p.1147

• WalkCC

```

class Solution:
    def oddEvenList(self, head: ListNode) -> ListNode:
        oddHead = ListNode(0)
        evenHead = ListNode(0)
        odd = oddHead
        even = evenHead
        isOdd = True

        while head:
            if isOdd:
                odd.next = head
                odd = head
            else:
                even.next = head
                even = head
            head = head.next
            isOdd = not isOdd

        even.next = None
        odd.next = evenHead.next

        return oddHead.next
  
```

• LeetCode

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def oddEvenList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        if head is None:
            return None
        a = head
  
```

Round 2 · High · Medium

p.1148

```

b = c = head.next
while b and b.next:
    a.next = b.next
    a = a.next
    b.next = a.next
    b = b.next
a.next = c
return head

```

• SimplyLeet

```

# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def oddEvenList(self, head: ListNode) -> ListNode:
        # Check for empty list or list with one node.
        if not head or not head.next:

```

```

        return head

```

```

odd = head          # Point to the first (odd indexed) node.
even = head.next    # Point to the second (even indexed) node.
even_head = even    # Save the head of the even list.

```

```

# Reorder the list by rearranging next pointers.
while even and even.next:
    odd.next = even.next # Connect current odd to next odd.
    odd = odd.next      # Move odd pointer.

```

```

even.next = odd.next # Connect current even to next even.
even = even.next     # Move even pointer.

```

```

# Append the even list after the odd list.
odd.next = even_head
return head

```

• LC.ca

Round 2 · High · Medium

p.1149

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def oddEvenList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        if head is None:
            return None
        a = head

```

```

b = c = head.next
while b and b.next:
    a.next = b.next
    a = a.next
    b.next = a.next
    b = b.next
a.next = c
return head

```

◆ Quick Review

Key Insights

- Use two pointers: one for odd-indexed nodes and one for even-indexed nodes.
- The first node is considered odd, and the second node is even.
- Traverse the list once, reassigning the next pointers to segregate odd and even nodes.
- Finally, attach the even list at the end of the odd list.
- The solution must use $O(1)$ extra space and $O(n)$ time complexity.

Round 2 · High · Medium

p.1150

Space & Time

Time Complexity: $O(n)$ since the list is traversed once.

Space Complexity: $O(1)$ as only a few pointers are used.

Solution

Maintain two pointers, odd and even, starting from the head and head.next, respectively. As you iterate, update odd.next to point to the next odd element (skipping an even node) and even.next to point to the next even element. Once the end is reached, attach the even list to the end of the odd list. This in-place rearrangement preserves the initial ordering within odd and even groups while meeting the space and time constraints.

Round 2 · High · Medium

p.1151

Linked List

□ ★ #430 Flatten a Multilevel Doubly Linked List ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · DFS · Doubly Linked List
Lists: ★ Premium | Premium 98

Problem

You are given a doubly linked list, which contains nodes that have a next pointer, a previous pointer, and an additional child pointer. This child pointer may or may not point to a separate doubly linked list, also containing these special nodes. These child lists may have one or more children of their own, and so on, to produce a multilevel data structure as shown in the example below.

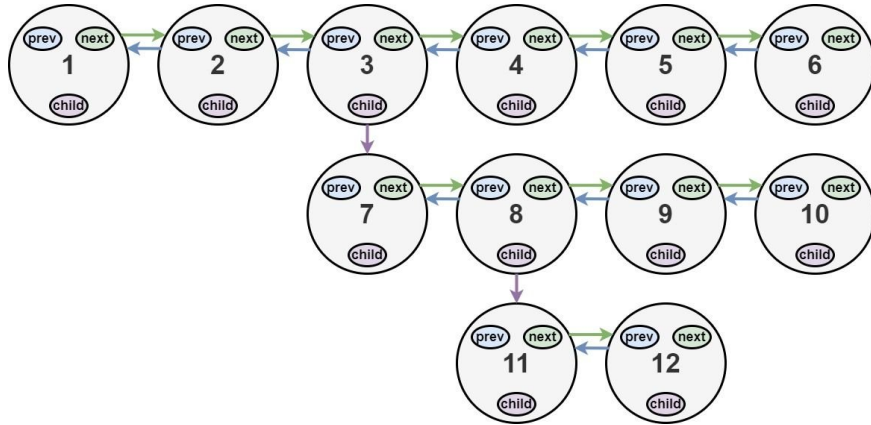
Given the head of the first level of the list, flatten the list so that all the nodes appear in a single-level, doubly linked list. Let curr be a node with a child list. The nodes in the child list should appear after curr and before curr.next in the flattened list.

Round 2 · High · Medium

p.1152

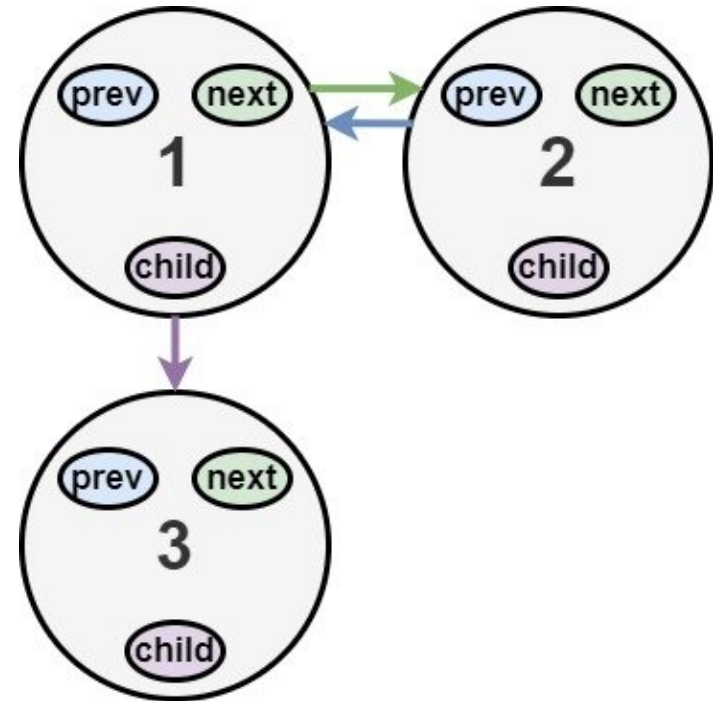
Return the head of the flattened list. The nodes in the list must have all of their child pointers set to null.

Example 1:



Input: head = [1,2,3,4,5,6,null,null,null,7,8,9,10,null,null,11,12]
 Output: [1,2,3,7,8,11,12,9,10,4,5,6]
 Explanation: The multilevel linked list in the input is shown.
 After flattening the multilevel linked list it becomes:

Example 2:



Input: head = [1,2,null,3]
 Output: [1,3,2]
 Explanation: The multilevel linked list in the input is shown.
 After flattening the multilevel linked list it becomes:

Example 3:

Input: head = []
 Output: []
 Explanation: There could be empty list in the input.

Constraints:

- The number of Nodes will not exceed 1000.
- $1 \leq \text{Node.val} \leq 105$

How the multilevel linked list is represented in test cases:

We use the multilevel linked list from Example 1 above:

```
1---2---3---4---5---6--NULL
|
7---8---9---10--NULL
|
11--12--NULL
```

The serialization of each level is as follows:

```
[1,2,3,4,5,6,null]
[7,8,9,10,null]
[11,12,null]
```

To serialize all levels together, we will add nulls in each level to signify no node connects to the upper node of the previous level. The serialization becomes:

```
[1, 2, 3, 4, 5, 6, null]
|
[null, null, 7, 8, 9, 10, null]
|
[ null, 11, 12, null]
```

Merging the serialization of each level and removing trailing nulls we obtain:

```
[1,2,3,4,5,6,null,null,null,7,8,9,10,null,null,11,12]
```

Community Solutions (Python)

• WalkCC

Round 2 · High · Medium

p.1155

```
class Solution:
    def flatten(self, head: 'Node') -> 'Node':
        def flatten(head: 'Node', rest: 'Node') -> 'Node':
            if not head:
                return rest

            head.next = flatten(head.child, flatten(head.next, rest))
            if head.next:
                head.next.prev = head
            head.child = None

            return head

        return flatten(head, None)

class Solution:
    def flatten(self, head: 'Node') -> 'Node':
        curr = head

        while curr:
            if curr.child:
                cachedNext = curr.next
                curr.next = curr.child
                curr.child.prev = curr
                curr.child = None

                tail = curr.next
                while tail.next:
                    tail = tail.next
                tail.next = cachedNext
                if cachedNext:
                    cachedNext.prev = tail
                curr = curr.next

        return head
```

• LeetDoocs

Round 2 · High · Medium

p.1156

```

"""
# Definition for a Node.
class Node:
    def __init__(self, val, prev, next, child):
        self.val = val
        self.prev = prev
        self.next = next
        self.child = child
"""

```

```

class Solution:
    def flatten(self, head: 'Node') -> 'Node':
        def preorder(pre, cur):
            if cur is None:
                return pre
            cur.prev = pre
            pre.next = cur

            t = cur.next

```

```

            tail = preorder(cur, cur.child)
            cur.child = None
            return preorder(tail, t)

```

```

        if head is None:
            return None
        dummy = Node(0, None, head, None)
        preorder(dummy, head)
        dummy.next.prev = None
        return dummy.next

```

• SimplyLeet

Round 2 · High · Medium

p.1157

```

# Definition for a Node.
class Node:
    def __init__(self, val, prev=None, next=None, child=None):
        self.val = val
        self.prev = prev
        self.next = next
        self.child = child

    def flatten(head):
        # If the list is empty, simply return None.

        if not head:
            return head

        # Helper function to perform DFS and return the tail of the flattened list.
        def dfs(node):
            current = node
            last = None
            while current:
                next_node = current.next # Store current.next before
                modifications
                # If current node has a child, we flatten the child list
                recursively.

                if current.child:
                    # Recursively flatten the child list.
                    child_tail = dfs(current.child)
                    # Connect current node with the start of the child list.
                    current.next = current.child
                    current.child.prev = current
                    # If there was a next node, attach it after the child list.
                    if next_node:
                        child_tail.next = next_node
                        next_node.prev = child_tail

```

Round 2 · High · Medium

p.1158

```

        # Set child pointer to None as required.
        current.child = None
        # Update the tail pointer.
        last = child_tail
    else:
        last = current
    # Move to the next node (which might have been updated after
    flattening a child).
    current = next_node
    return last

```

```

dfs(head) # Begin DFS traversal from the head.
return head

```

```

# Example to illustrate usage:
# head = Node(1)
# head.next = Node(2)
# head.child = Node(3)
# flatten(head)

```

• LC.ca

```

"""
# Definition for a Node.
class Node:
    def __init__(self, val, prev, next, child):
        self.val = val
        self.prev = prev
        self.next = next
        self.child = child
"""

```

Round 2 · High · Medium

p.1159

```

class Solution:
    def flatten(self, head: 'Node') -> 'Node':
        def preorder(pre, cur):
            if cur is None:
                return pre
            cur.prev = pre
            pre.next = cur

            t = cur.next

            tail = preorder(cur, cur.child)
            cur.child = None
            return preorder(tail, t)

        if head is None:
            return None
        dummy = Node(0, None, head, None)
        preorder(dummy, head)
        dummy.next.prev = None
        return dummy.next

```

◆ Quick Review

Key Insights

- Use a depth-first traversal to process child lists before proceeding to the next pointer.
- When a node with a child is found, recursively flatten the child list.
- Splice the flattened child list in between the current node and the next node.
- Maintain previous and next pointers correctly to preserve the doubly linked list properties.

Round 2 · High · Medium

p.1160

- Ensure that all child pointers are set to null after flattening.

Space & Time

Time Complexity: $O(n)$, where n is the total number of nodes, since each node is processed exactly once.

Space Complexity: $O(n)$ in the worst-case due to recursion stack in case of a highly nested list.

Solution

We can solve this problem using a recursive depth-first search (DFS) approach. The main idea is to traverse the list and, whenever a node with a child pointer is encountered, recursively flatten its child list. Once the child list is flattened, it can be spliced into the main list between the current node and the node originally following it. We then continue processing the next pointer. This ensures that the entire list is flattened in depth-first order. The recursive helper function returns the tail of the flattened segment, which is used to connect the subsequent parts of the list correctly.

Round 2 · High · Medium

p.1161

Linked List

□ #445 Add Two Numbers II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Linked List · Math · Stack
 Lists: AlgoMaster 600

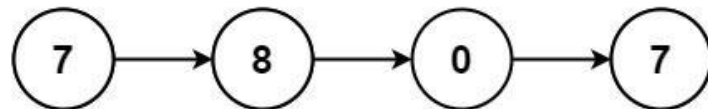
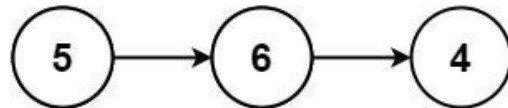
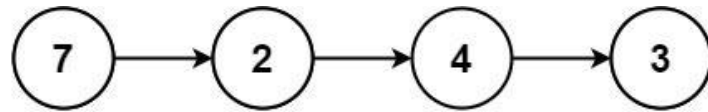
Problem

You are given two non-empty linked lists representing two non-negative integers. The most significant digit comes first and each of their nodes contains a single digit. Add the two numbers and return the sum as a linked list. You may assume the two numbers do not contain any leading zero, except the number 0 itself.

Example 1:

Round 2 · High · Medium

p.1162



Input: l1 = [7,2,4,3], l2 = [5,6,4]
Output: [7,8,0,7]

Example 2:

Input: l1 = [2,4,3], l2 = [5,6,4]
Output: [8,0,7]

Example 3:

Input: l1 = [0], l2 = [0]
Output: [0]

Constraints:

- The number of nodes in each linked list is in the range [1, 100].
- $0 \leq \text{Node.val} \leq 9$

- It is guaranteed that the list represents a number that does not have leading zeros.

Follow up: Could you solve it without reversing the input lists?

Community Solutions (Python)

• WalkCC

```

class Solution:
    def addTwoNumbers(self, l1: ListNode, l2: ListNode) -> ListNode:
        stack1 = []
        stack2 = []

        while l1:
            stack1.append(l1.val)
            l1 = l1.next

        while l2:
            stack2.append(l2.val)
            l2 = l2.next

        head = None
        carry = 0

        while carry or stack1 or stack2:
            if stack1:
                carry += stack1.pop().val
            if stack2:
                carry += stack2.pop().val
            node = ListNode(carry % 10)
            node.next = head
            head = node
            carry //= 10

        return head
  
```

• LeetCode

```
# Definition for singly-linked list.
# class ListNode:
# def __init__(self, val=0, next=None):
# self.val = val
# self.next = next
class Solution:
    def addTwoNumbers(
        self, l1: Optional[ListNode], l2: Optional[ListNode]
    ) -> Optional[ListNode]:
        s1, s2 = [], []

        while l1:
            s1.append(l1.val)
            l1 = l1.next
        while l2:
            s2.append(l2.val)
            l2 = l2.next
        dummy = ListNode()
        carry = 0
        while s1 or s2 or carry:
            s = (0 if not s1 else s1.pop()) + (0 if not s2 else s2.pop()) + ca
            rry

            carry, val = divmod(s, 10)
            # node = ListNode(val, dummy.next)
            # dummy.next = node
            dummy.next = ListNode(val, dummy.next)
        return dummy.next
```

• SimplyLeet

```
# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def addTwoNumbers(l1: ListNode, l2: ListNode) -> ListNode:
    # Initialize two stacks for the two numbers.
    stack1 = []
    stack2 = []
```

Round 2 · High · Medium

p.1165

```
# Push all nodes' values of l1 onto stack1.
while l1:
    stack1.append(l1.val)
    l1 = l1.next

# Push all nodes' values of l2 onto stack2.
while l2:
    stack2.append(l2.val)
    l2 = l2.next

carry = 0
result = None # This will be the head of the resulting linked list.

# Process stacks until both are empty.
while stack1 or stack2 or carry:
    # Get the top values from stacks if available.
    val1 = stack1.pop() if stack1 else 0
    val2 = stack2.pop() if stack2 else 0

    # Calculate the total sum of values and carry.
    total = val1 + val2 + carry
    carry = total // 10
    newNode = ListNode(total % 10)

    # Prepend the new node to the result list.
    newNode.next = result
    result = newNode

return result
```

```
# Example usage:
# l1 = ListNode(7, ListNode(2, ListNode(4, ListNode(3))))
# l2 = ListNode(5, ListNode(6, ListNode(4)))
# head = addTwoNumbers(l1, l2)
```

Round 2 · High · Medium

p.1166

Linked List

#707 Design Linked List

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Linked List · Design
 Lists: Denny Zhang

Problem

Design your implementation of the linked list. You can choose to use a singly or doubly linked list. A node in a singly linked list should have two attributes: `val` and `next`. `val` is the value of the current node, and `next` is a pointer/reference to the next node. If you want to use the doubly linked list, you will need one more attribute `prev` to indicate the previous node in the linked list. Assume all nodes in the linked list are 0-indexed.

Implement the `MyLinkedList` class:

- `MyLinkedList()` Initializes the `MyLinkedList` object.
- `int get(int index)` Get the value of the `index`th node in the linked list. If the `index` is invalid, return `-1`.

Round 2 · High · Medium

p.1167

- `void addAtHead(int val)` Add a node of value `val` before the first element of the linked list. After the insertion, the new node will be the first node of the linked list.
- `void addAtTail(int val)` Append a node of value `val` as the last element of the linked list.
- `void addAtIndex(int index, int val)` Add a node of value `val` before the `index`th node in the linked list. If `index` equals the length of the linked list, the node will be appended to the end of the linked list. If `index` is greater than the length, the node will not be inserted.
- `void deleteAtIndex(int index)` Delete the `index`th node in the linked list, if the `index` is valid.

Example 1:

```
Input
["MyLinkedList", "addAtHead", "addAtTail", "addAtIndex", "get",
 "deleteAtIndex", "get"]
[[], [1], [3], [1, 2], [1], [1], [1]]
Output
[null, null, null, null, 2, null, 3]

Explanation
MyLinkedList myLinkedList = new MyLinkedList();
```

Constraints:

- $0 \leq \text{index}, \text{val} \leq 1000$

Round 2 · High · Medium

p.1168

- Please do not use the built-in LinkedList library.
- At most 2000 calls will be made to get, addAtHead, addAtTail, addAtIndex and deleteAtIndex.

Community Solutions (Python)

• WalkCC

```
from dataclasses import dataclass

@dataclass
class ListNode:
    val: int
    next: ListNode | None = None

class MyLinkedList:

    def __init__(self):
        self.length = 0
        self.dummy = ListNode(0)

    def get(self, index: int) -> int:
        if index < 0 or index >= self.length:
            return -1
        curr = self.dummy.next
        for _ in range(index):
            curr = curr.next
```

```
        return curr.val

    def addAtHead(self, val: int) -> None:
        curr = self.dummy.next
        self.dummy.next = ListNode(val)
        self.dummy.next.next = curr
        self.length += 1

    def addAtTail(self, val: int) -> None:
        curr = self.dummy

        while curr.next:
            curr = curr.next
        curr.next = ListNode(val)
        self.length += 1

    def addAtIndex(self, index: int, val: int) -> None:
        if index > self.length:
            return
        curr = self.dummy
        for _ in range(index):

            curr = curr.next
            temp = curr.next
            curr.next = ListNode(val)
            curr.next.next = temp
            self.length += 1

    def deleteAtIndex(self, index: int) -> None:
        if index < 0 or index >= self.length:
            return
        curr = self.dummy

        for _ in range(index):
            curr = curr.next
            temp = curr.next
            curr.next = temp.next
            self.length -= 1
```

• LeetCode

Round 2 · High · Medium

p.1169

Round 2 · High · Medium

p.1170

```

class MyLinkedList:
    def __init__(self):
        self.dummy = ListNode()
        self.cnt = 0

    def get(self, index: int) -> int:
        if index < 0 or index >= self.cnt:
            return -1
        cur = self.dummy.next
        for _ in range(index):

            cur = cur.next
        return cur.val

    def addAtHead(self, val: int) -> None:
        self.addAtIndex(0, val)

    def addAtTail(self, val: int) -> None:
        self.addAtIndex(self.cnt, val)

    def addAtIndex(self, index: int, val: int) -> None:

        if index > self.cnt:
            return
        pre = self.dummy
        for _ in range(index):
            pre = pre.next
        pre.next = ListNode(val, pre.next)
        self.cnt += 1

    def deleteAtIndex(self, index: int) -> None:
        if index >= self.cnt:

            return
        pre = self.dummy
        for _ in range(index):
            pre = pre.next
        t = pre.next
        pre.next = t.next
        t.next = None
        self.cnt -= 1

```

Round 2 · High · Medium

p.1171

```

# Your MyLinkedList object will be instantiated and called as such:
# obj = MyLinkedList()
# param_1 = obj.get(index)
# obj.addAtHead(val)
# obj.addAtTail(val)
# obj.addAtIndex(index, val)
# obj.deleteAtIndex(index)

```

```

class MyLinkedList:
    def __init__(self):
        self.e = [0] * 1010
        self.ne = [0] * 1010
        self.idx = 0
        self.head = -1
        self.cnt = 0

    def get(self, index: int) -> int:
        if index < 0 or index >= self.cnt:

            return -1
        i = self.head
        for _ in range(index):
            i = self.ne[i]
        return self.e[i]

    def addAtHead(self, val: int) -> None:
        self.e[self.idx] = val
        self.ne[self.idx] = self.head
        self.head = self.idx

        self.idx += 1
        self.cnt += 1

    def addAtTail(self, val: int) -> None:
        self.addAtIndex(self.cnt, val)

    def addAtIndex(self, index: int, val: int) -> None:
        if index > self.cnt:
            return
        if index <= 0:

```

Round 2 · High · Medium

p.1172

```

        self.addAtHead(val)
        return
    i = self.head
    for _ in range(index - 1):
        i = self.ne[i]
    self.e[self.idx] = val
    self.ne[self.idx] = self.ne[i]
    self.ne[i] = self.idx
    self.idx += 1
    self.cnt += 1

```

```

def deleteAtIndex(self, index: int) -> None:
    if index < 0 or index >= self.cnt:
        return -1
    self.cnt -= 1
    if index == 0:
        self.head = self.ne[self.head]
        return
    i = self.head
    for _ in range(index - 1):

```

```

        i = self.ne[i]
        self.ne[i] = self.ne[self.ne[i]]

```

```

# Your MyLinkedList object will be instantiated and called as such:
# obj = MyLinkedList()
# param_1 = obj.get(index)
# obj.addAtHead(val)
# obj.addAtTail(val)
# obj.addAtIndex(index, val)

```

```

# obj.deleteAtIndex(index)

```

• SimplyLeet

Round 2 · High · Medium

p.1173

```

# Define a node for the singly linked list
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val # The value of the node
        self.next = next # Pointer to the next node

class MyLinkedList:
    def __init__(self):
        # Initialize size and a dummy head node to simplify operations
        self.size = 0

```

```

        self.head = ListNode(0) # dummy head

```

```

def get(self, index: int) -> int:
    # If index is out of bounds, return -1
    if index < 0 or index >= self.size:
        return -1
    current = self.head.next # start from the actual head
    for _ in range(index):
        current = current.next
    return current.val

```

```

def addAtHead(self, val: int) -> None:
    self.addAtIndex(0, val) # add at index 0

def addAtTail(self, val: int) -> None:
    self.addAtIndex(self.size, val) # add at the end

```

```

def addAtIndex(self, index: int, val: int) -> None:
    # Normalize negative index to 0 (insert at head)
    if index < 0:

```

```

        index = 0
    # Only insert if index is valid (<= size)
    if index > self.size:
        return
    self.size += 1
    # Start from the dummy head node
    pred = self.head
    for _ in range(index):
        pred = pred.next
    # Insert new node

```

Round 2 · High · Medium

p.1174

```

new_node = ListNode(val)
new_node.next = pred.next
pred.next = new_node

```

```

def deleteAtIndex(self, index: int) -> None:
    # If index is out of bounds, do nothing
    if index < 0 or index >= self.size:
        return
    self.size -= 1
    pred = self.head

```

```

for _ in range(index):
    pred = pred.next
# Remove the node by skipping it
pred.next = pred.next.next

```

• LC.ca

```

# doubly linked node
class ListNode:
    def __init__(self, value=0, prev=None, next=None):
        self.value = value
        self.prev = prev
        self.next = next

class MyLinkedList:
    def __init__(self):
        self.head = ListNode(0) # Sentinel node as pseudo-head

```

```

self.tail = ListNode(0) # Sentinel node as pseudo-tail
self.head.next = self.tail
self.tail.prev = self.head
self.size = 0

```

```

def get(self, index: int) -> int:
    if index < 0 or index >= self.size:
        return -1
    if index + 1 < self.size - index: # decide start from head or tail
        curr = self.head

```

Round 2 · High · Medium

p.1175

```

for _ in range(index + 1):
    curr = curr.next
else:
    curr = self.tail
    for _ in range(self.size - index):
        curr = curr.prev
    return curr.value

```

```

def addAtHead(self, val: int) -> None:
    pred, succ = self.head, self.head.next

```

```

self.size += 1
to_add = ListNode(val, pred, succ)
pred.next = to_add
succ.prev = to_add

```

```

def addAtTail(self, val: int) -> None:
    succ, pred = self.tail, self.tail.prev
    self.size += 1
    to_add = ListNode(val, pred, succ)
    pred.next = to_add

```

```

succ.prev = to_add

```

```

def addAtIndex(self, index: int, val: int) -> None:
    if index > self.size:
        return
    if index < 0:
        index = 0
    if index < self.size - index: # decide start from head or tail
        pred = self.head
        for _ in range(index):

```

```

            pred = pred.next
            succ = pred.next
        else:
            succ = self.tail
            for _ in range(self.size - index):
                succ = succ.prev
            pred = succ.prev
        self.size += 1
        to_add = ListNode(val, pred, succ)
        pred.next = to_add

```

Round 2 · High · Medium

p.1176


```

succ.prev = to_add

def deleteAtIndex(self, index: int) -> None:
    if index < 0 or index >= self.size:
        return
    if index < self.size - index:
        pred = self.head
        for _ in range(index):
            pred = pred.next
        succ = pred.next.next

    else:
        succ = self.tail
        for _ in range(self.size - index - 1):
            succ = succ.prev
        pred = succ.prev.prev
    self.size -= 1
    pred.next = succ
    succ.prev = pred

# Your MyLinkedList object will be instantiated and called as such:

# obj = MyLinkedList()
# param_1 = obj.get(index)
# obj.addAtHead(val)
# obj.addAtTail(val)
# obj.addAtIndex(index, val)
# obj.deleteAtIndex(index)

#####

# singly linked node

```

```

class MyLinkedList:
    def __init__(self):
        self.dummy = ListNode()
        self.cnt = 0

    def get(self, index: int) -> int:
        if index < 0 or index >= self.cnt:
            return -1
        cur = self.dummy.next
        for _ in range(index):

```

Round 2 · High · Medium

p.1177

```

        cur = cur.next
        return cur.val

    def addAtHead(self, val: int) -> None:
        self.addAtIndex(0, val)

    def addAtTail(self, val: int) -> None:
        self.addAtIndex(self.cnt, val)

    def addAtIndex(self, index: int, val: int) -> None:

        if index > self.cnt:
            return
        pre = self.dummy
        for _ in range(index):
            pre = pre.next
        pre.next = ListNode(val, pre.next) # insert in-between
        self.cnt += 1

    def deleteAtIndex(self, index: int) -> None:
        if index >= self.cnt:

            return
        pre = self.dummy
        for _ in range(index):
            pre = pre.next
        t = pre.next
        pre.next = t.next
        t.next = None # don't forget to cutoff
        self.cnt -= 1

# Your MyLinkedList object will be instantiated and called as such:
# obj = MyLinkedList()
# param_1 = obj.get(index)
# obj.addAtHead(val)
# obj.addAtTail(val)
# obj.addAtIndex(index, val)
# obj.deleteAtIndex(index)

```

◆ Quick Review

Round 2 · High · Medium

p.1178

Key Insights

- Use a dummy head node to simplify insertion and deletion at the head.
- Maintain a size counter to quickly validate index-based operations.
- For every operation, traverse the list up to the required point since random access is not available.
- Ensure robustness by handling edge cases like negative indices and indices larger than the current length.

Space & Time

Time Complexity:

- get, addAtIndex, and deleteAtIndex operations require $O(n)$ time due to the need to traverse the list.
- addAtHead and addAtTail (if tail pointer is not maintained) are $O(n)$ in worst case; however, these can be optimized further if a tail pointer is added.

Space Complexity:

- $O(n)$ space for storing n nodes.

Solution

Round 2 · High · Medium

p.1179

The solution employs a singly linked list with a dummy head node to ease edge-case handling. A size counter is maintained for quick index validations. Each node holds a value and a pointer to the next node. Operations operate by traversing the list to the correct insertion or deletion point. This design makes it simple to add the functionality of getting values, inserting at specified indices, and removing nodes.

Round 2 · High · Medium

p.1180

Linked List

★ #708 Insert into a Sorted Circular Linked List ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List

Lists: ★ Premium | Premium 98

Problem

Given a Circular Linked List node, which is sorted in non-descending order, write a function to insert a value `insertVal` into the list such that it remains a sorted circular list. The given node can be a reference to any single node in the list and may not necessarily be the smallest value in the circular list.

If there are multiple suitable places for insertion, you may choose any place to insert the new value. After the insertion, the circular list should remain sorted.

If the list is empty (i.e., the given node is null), you should create a new single circular list and return the reference to that single node.

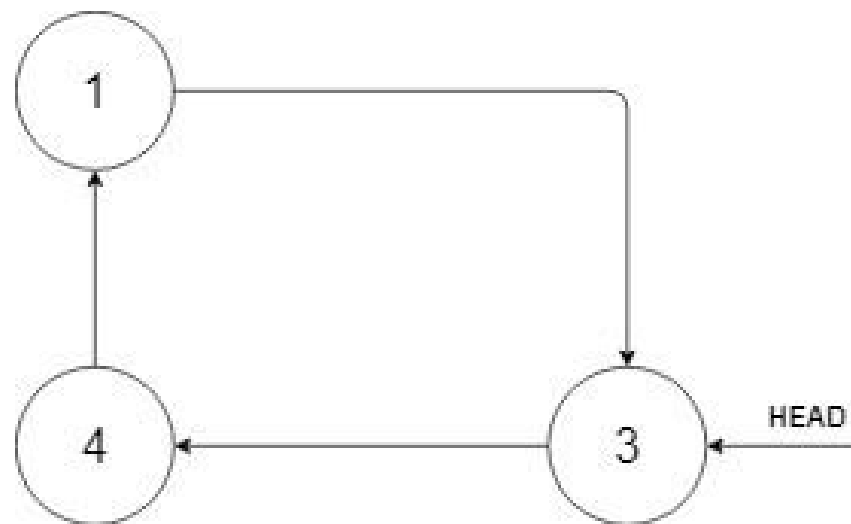
Otherwise, you should return the originally

Round 2 · High · Medium

p.1181

given node.

Example 1:



Input: head = [3,4,1], insertVal = 2

Output: [3,4,1,2]

Explanation: In the figure above, there is a sorted circular list of three elements. You are given a reference to the node with value 3, and we need to insert 2 into the list. The new node should be inserted between node 1 and node 3. After the insertion, the list should look like this, and we should still return node 3.

Example 2:

Input: head = [], insertVal = 1

Output: [1]

Explanation: The list is empty (given head is null). We create a new single circular list and return the reference to that single node.

Example 3:

Input: head = [1], insertVal = 0

Output: [1,0]

Round 2 · High · Medium

p.1182

Constraints:

- The number of nodes in the list is in the range $[0, 5 * 10^4]$.
- $-106 \leq \text{Node.val}, \text{insertVal} \leq 106$

Community Solutions (Python)

• LeetCode

```

"""
# Definition for a Node.
class Node:
    def __init__(self, val=None, next=None):
        self.val = val
        self.next = next
"""

class Solution:

    def insert(self, head: 'Optional[Node]', insertVal: int) -> 'Node':
        node = Node(insertVal)
        if head is None:
            node.next = node
            return node
        prev, curr = head, head.next
        while curr != head:
            if prev.val <= insertVal <= curr.val or (
                prev.val > curr.val and (insertVal >= prev.val or insertVal <=
curr.val)
            ):
                break
            prev, curr = curr, curr.next
        prev.next = node
        node.next = curr
        return head

```

• SimplyLeet

Round 2 · High · Medium

p.1183

```

# Definition for a Node.
class Node:
    def __init__(self, val, next=None):
        self.val = val
        self.next = next

class Solution:
    def insert(self, head: 'Node', insertVal: int) -> 'Node':
        # If the list is empty, create a new node pointing to itself and return
        it.

        if not head:

            newNode = Node(insertVal)
            newNode.next = newNode
            return newNode

        curr = head
        inserted = False

        while True:
            # Case 1: insert between two nodes where insertVal is between
            curr.val and curr.next.val.
            if curr.val <= insertVal <= curr.next.val:

                inserted = True
                # Case 2: curr.val > curr.next.val, i.e., we are at the turning
                point.
                # In this case, insert if insertVal is greater than curr.val
                (largest)
                # or insertVal is less than curr.next.val (smallest).
                elif curr.val > curr.next.val:
                    if insertVal >= curr.val or insertVal <= curr.next.val:
                        inserted = True

                if inserted:
                    newNode = Node(insertVal, curr.next)

```

Round 2 · High · Medium

p.1184

```

        curr.next = newNode
        return head

    curr = curr.next
    # If we have traversed the entire circle and came back to head,
    break.

    if curr == head:
        break

    # Case 3: All values are the same or no valid index was found, insert
    arbitrarily.
    newNode = Node(insertVal, curr.next)

    curr.next = newNode
    return head

```

• LC.ca

```

"""
# Definition for a Node.
class Node:
    def __init__(self, val=None, next=None):
        self.val = val
        self.next = next
"""

class Solution:

    def insert(self, head: 'Optional[Node]', insertVal: int) -> 'Node':
        node = Node(insertVal)
        if head is None:
            node.next = node
            return node
        prev, curr = head, head.next
        while curr != head:
            if prev.val <= insertVal <= curr.val or (
                prev.val > curr.val and (insertVal >= prev.val or insertVal <=
curr.val)
            ):

```

Round 2 · High · Medium

p.1185

```

        break
        prev, curr = curr, curr.next
    prev.next = node
    node.next = curr
    return head

```

◆ Quick Review

Key Insights

- Handle the empty list edge-case by creating a new node that points to itself.
- Traverse the list until the proper insertion point is found.
- Consider the "turning point" where the maximum value is followed by the minimum value.
- If no proper place is found in one full rotation, insert the new node arbitrarily (e.g., after the head).

Space & Time

Time Complexity: $O(n)$ in the worst-case when a full loop is required.

Space Complexity: $O(1)$ since we only use a few extra pointers.

Solution

Round 2 · High · Medium

p.1186

We traverse the circular linked list starting from the arbitrary given node. For each pair of consecutive nodes (curr and next), we check if the new value can be inserted between them by confirming that:

- The new value lies between curr and next ($\text{curr.val} \leq \text{insertVal} \leq \text{next.val}$).
- Or if we are at the pivot point where the maximum value meets the minimum value (i.e., $\text{curr.val} > \text{next.val}$) and then either the new value is larger than or equal to curr.val (should be appended after the maximum) or less than or equal to next.val (should be inserted before the minimum).

If no proper spot is found after one complete loop, we insert anywhere (all values in the list are the same). A pointer reference is updated accordingly to maintain the circular structure.

Linked List

□ #1669 Merge In Between Linked Lists

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List

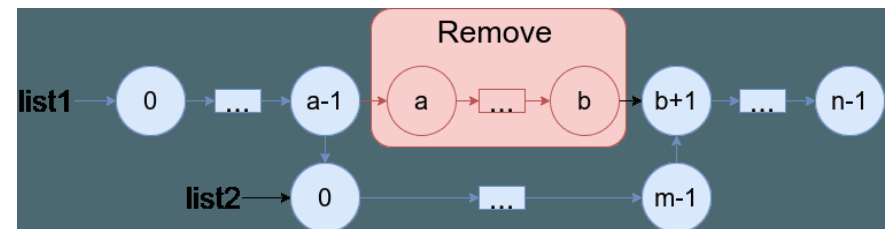
Lists: AlgoMaster 600

Problem

You are given two linked lists: list1 and list2 of sizes n and m respectively.

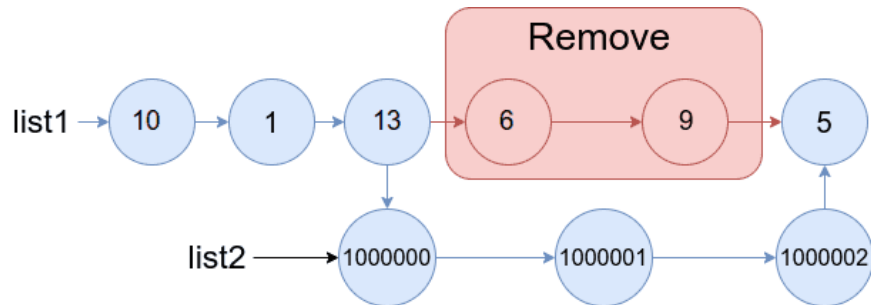
Remove list1's nodes from the ath node to the bth node, and put list2 in their place.

The blue edges and nodes in the following figure indicate the result:



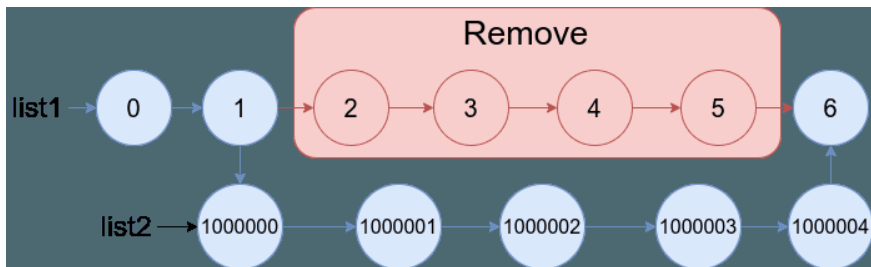
Build the result list and return its head.

Example 1:



Input: list1 = [10,1,13,6,9,5], a = 3, b = 4, list2 = [1000000,1000001,1000002]
 Output: [10,1,13,1000000,1000001,1000002,5]
 Explanation: We remove the nodes 3 and 4 and put the entire list2 in their place. The blue edges and nodes in the above figure indicate the result.

Example 2:



Input: list1 = [0,1,2,3,4,5,6], a = 2, b = 5, list2 = [1000000,1000001,1000002,1000003,1000004]
 Output: [0,1,1000000,1000001,1000002,1000003,1000004,6]
 Explanation: The blue edges and nodes in the above figure indicate the result.

Constraints:

- $3 \leq \text{list1.length} \leq 104$
- $1 \leq a \leq b < \text{list1.length} - 1$
- $1 \leq \text{list2.length} \leq 104$

Round 2 · High · Medium

p.1189

Community Solutions (Python)

• WalkCC

```
class Solution:
    def mergeInBetween(
        self,
        list1: ListNode,
        a: int,
        b: int,
        list2: ListNode,
    ) -> ListNode:
        nodeBeforeA = list1
        for i in range(a - 1):

            nodeBeforeA = nodeBeforeA.next

        nodeB = nodeBeforeA.next
        for i in range(b - a):
            nodeB = nodeB.next

        nodeBeforeA.next = list2
        lastNodeInList2 = list2

        while lastNodeInList2.next:

            lastNodeInList2 = lastNodeInList2.next

        lastNodeInList2.next = nodeB.next
        nodeB.next = None
        return list1
```

• LeetDoocs

Round 2 · High · Medium

p.1190

```
# Definition for singly-linked list.
# class ListNode:
# def __init__(self, val=0, next=None):
# self.val = val
# self.next = next
class Solution:
    def mergeInBetween(
        self, list1: ListNode, a: int, b: int, list2: ListNode
    ) -> ListNode:
        p = q = list1
```

```
        for _ in range(a - 1):
            p = p.next
        for _ in range(b):
            q = q.next
        p.next = list2
        while p.next:
            p = p.next
        p.next = q.next
        q.next = None
        return list1
```

• SimplyLeet

```
# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def mergeInBetween(list1: ListNode, a: int, b: int, list2: ListNode) -> ListNo
de:
    # Step 1: Traverse list1 to reach node just before index a (prevA)
    current = list1
    for i in range(a - 1):
```

```
        current = current.next
    prevA = current

    # Step 2: Continue traversing to reach node after index b (postB)
    current = current.next
    for i in range(b - a + 1):
        current = current.next
    postB = current
```

```
    # Step 3: Find the tail of list2
```

```
    tail2 = list2
    while tail2.next:
        tail2 = tail2.next
```

```
    # Step 4: Connect list2 between list1 parts
    prevA.next = list2      # Connect node before index a to head of list2
    tail2.next = postB      # Connect tail of list2 to node after index b

    return list1            # Return the modified list1
```


Linked List

#2095 Delete the Middle Node of a Linked List

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Linked List · Two Pointers
 Lists: AlgoMaster 600

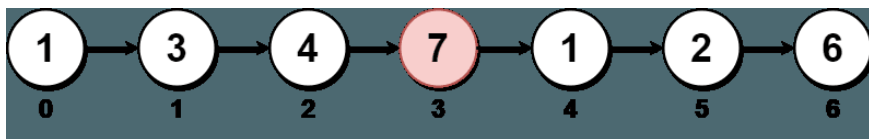
Problem

You are given the head of a linked list. Delete the middle node, and return the head of the modified linked list.

The middle node of a linked list of size n is the $n / 2$ th node from the start using 0-based indexing, where x denotes the largest integer less than or equal to x .

- For $n = 1, 2, 3, 4$, and 5 , the middle nodes are $0, 1, 1, 2$, and 2 , respectively.

Example 1:



Round 2 · High · Medium

p.1193

Input: head = [1,3,4,7,1,2,6]

Output: [1,3,4,1,2,6]

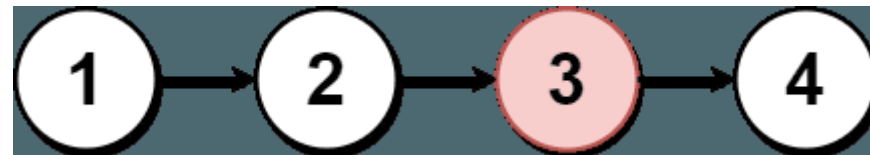
Explanation:

The above figure represents the given linked list. The indices of the nodes are written below.

Since $n = 7$, node 3 with value 7 is the middle node, which is marked in red.

We return the new list after removing this node.

Example 2:



Input: head = [1,2,3,4]

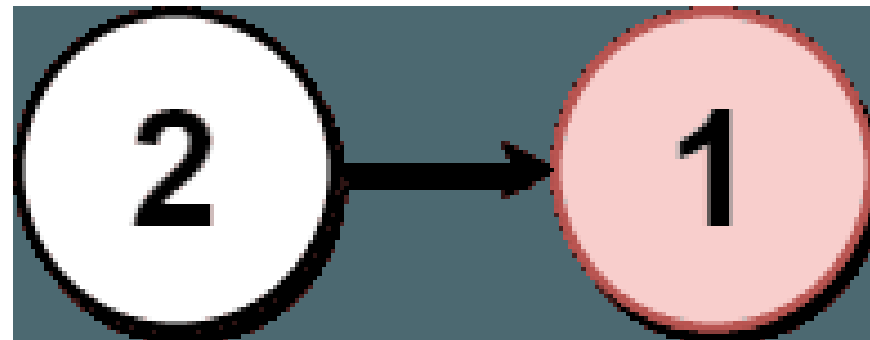
Output: [1,2,4]

Explanation:

The above figure represents the given linked list.

For $n = 4$, node 2 with value 3 is the middle node, which is marked in red.

Example 3:



Input: head = [2,1]

Output: [2]

Explanation:

The above figure represents the given linked list.

For $n = 2$, node 1 with value 1 is the middle node, which is marked in red.

Node 0 with value 2 is the only node remaining after removing node 1.

Round 2 · High · Medium

p.1194

Constraints:

- The number of nodes in the list is in the range [1, 105].
- $1 \leq \text{Node.val} \leq 105$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def deleteMiddle(self, head: ListNode | None) -> ListNode | None:
        dummy = ListNode(0, head)
        slow = dummy
        fast = dummy

        while fast.next and fast.next.next:
            slow = slow.next
            fast = fast.next.next

        # Delete the middle node.
        slow.next = slow.next.next
        return dummy.next
```

• LeetCode

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def deleteMiddle(self, head: Optional[ListNode]) -> Optional[ListNode]:
        dummy = ListNode(next=head)
        slow, fast = dummy, head
        while fast and fast.next:
```

```
slow = slow.next
fast = fast.next.next
slow.next = slow.next.next
return dummy.next
```

• SimplyLeet

```
# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def deleteMiddle(self, head: ListNode) -> ListNode:
        # If the list is empty or contains only one node, return None
        if not head or not head.next:
            return None

        # Initialize two pointers for the two-pointer approach
        slow = head          # slow pointer starts at head
        fast = head          # fast pointer starts at head
        prev = None         # previous pointer to track node before slow

        # Traverse the list until fast reaches the end
        while fast and fast.next:
            prev = slow      # keep track of the node before slow
            slow = slow.next  # move slow by one step
            fast = fast.next.next  # move fast by two steps

        # Now, slow is at the middle node.
        # Remove the middle node by linking the previous node to slow.next.
        prev.next = slow.next

        return head
```

Round 3 · High · Hard

Round 3

High Priority · Hard
24 questions · 11 patterns

- ☐ ● Arrays #41 ↗
- ☐ ● Strings #68 #843 ↗
- ☐ ● Two Pointers #42 #2444 ↗
- ☐ ● Sliding Window – Fixed Size #30 ↗
- ☐ ● Sliding Window – Dynamic Size #76 #727 #992 ↗
- ☐ ● Binary Search #4 #410 #719 #1095 ↗
- ☐ ● Stacks #32 #224 ↗
- ☐ ● Tree Traversal – Pre Order #297 ↗
- ☐ ● Tree Traversal – Post-Order #124 ↗
- ☐ ● Depth First Search (DFS) #827 #1192 #2246 ↗
- ☐ ● Breadth First Search (BFS) #127 #815 #1293 ↗
- ☐ ● #2290 ↗

Round 2 · High · Medium

p.1197

Arrays

Round 3 · High · Hard · 1 question

Round 3 · High · Hard

p.1198

Arrays

#41 First Missing Positive

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table
 Lists: Grind 169

Problem

Given an unsorted integer array `nums`. Return the smallest positive integer that is not present in `nums`.

You must implement an algorithm that runs in $O(n)$ time and uses $O(1)$ auxiliary space.

Example 1:

Input: `nums = [1,2,0]`
 Output: 3
 Explanation: The numbers in the range [1,2] are all in the array.

Example 2:

Input: `nums = [3,4,-1,1]`
 Output: 2
 Explanation: 1 is in the array but 2 is missing.

Example 3:

Input: `nums = [7,8,9,11,12]`
 Output: 1
 Explanation: The smallest positive integer 1 is missing.

Constraints:

Round 3 · High · Hard

p.1199

- $1 \leq \text{nums.length} \leq 105$
- $-231 \leq \text{nums}[i] \leq 231 - 1$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def firstMissingPositive(self, nums: List[int]) -> int:
        n = len(nums)

        # Correct slot:
        # nums[i] = i + 1
        # nums[i] - 1 = i
        # nums[nums[i] - 1] = nums[i]
        for i in range(n):
            while nums[i] > 0 and nums[i] <= n and nums[nums[i] - 1] != nums[i]:
                nums[nums[i] - 1], nums[i] = nums[i], nums[nums[i] - 1]

        for i, num in enumerate(nums):
            if num != i + 1:
                return i + 1

        return n + 1
```

• LeetDoocs

```
class Solution:
    def firstMissingPositive(self, nums: List[int]) -> int:
        n = len(nums)
        for i in range(n):
            while 1 <= nums[i] <= n and nums[i] != nums[nums[i] - 1]:
                j = nums[i] - 1
                nums[i], nums[j] = nums[j], nums[i]
        for i in range(n):
            if nums[i] != i + 1:
                return i + 1

        return n + 1
```

Round 3 · High · Hard

p.1200

• SimplyLeet

```
# Python implementation of First Missing Positive

def firstMissingPositive(nums):
    n = len(nums)
    # Rearrange numbers to their correct positions
    for i in range(n):
        while 1 <= nums[i] <= n and nums[nums[i] - 1] != nums[i]:
            correct_index = nums[i] - 1
            nums[i], nums[correct_index] = nums[correct_index], nums[i]

    # Identify the first missing positive integer
    for i in range(n):
        if nums[i] != i + 1:
            return i + 1
    return n + 1

# Example usage:
print(firstMissingPositive([3, 4, -1, 1])) # Output should be 2
```

• LC.ca

```
class Solution:
    def firstMissingPositive(self, nums: List[int]) -> int:
        def swap(i, j):
            nums[i], nums[j] = nums[j], nums[i]

        n = len(nums)
        for i in range(n):
            while 1 <= nums[i] <= n and nums[i] != nums[nums[i] - 1]:
                swap(i, nums[i] - 1)
        for i in range(n):
            if i + 1 != nums[i]:
                return i + 1
        return n + 1
```

◆ Quick Review

Round 3 · High · Hard

p.1201

Key Insights

- The missing positive number must be in the range $[1, n+1]$ where n is the length of the array.
- Use the array indices to place numbers in their correct positions (i.e., number i should be at index $i-1$) with in-place swapping.
- After rearrangement, scan the array: the first index where the number is not equal to $\text{index}+1$ indicates the missing positive integer.

Space & Time

Time Complexity: $O(n)$ as each number is swapped at most once.

Space Complexity: $O(1)$ since the rearrangement occurs in-place.

Solution

The approach involves reordering the array so that each positive integer in the range $[1, n]$ is placed at the index corresponding to its value minus one. This means for any valid number i , $\text{nums}[i-1]$ should be i . We iterate through the array and perform in-place swaps to achieve this order. After this process, another pass through the array reveals the first position

Round 3 · High · Hard

p.1202

where the number does not match the expected value ($i+1$), which is the smallest missing positive. If every position is correct, then the missing value is $n+1$.

Round 3 · High · Hard

p.1203

Strings

Round 3 · High · Hard · 2 questions

Round 3 · High · Hard

p.1204

Strings

□ #68 Text Justification

HAR

LeetCode · SimplyLeet · WalkCC · LeetCode
 Array · String · Simulation
 Lists: CodeSignal

Problem

Given an array of strings `words` and a width `maxWidth`, format the text such that each line has exactly `maxWidth` characters and is fully (left and right) justified.

You should pack your words in a greedy approach; that is, pack as many words as you can in each line. Pad extra spaces ' ' when necessary so that each line has exactly `maxWidth` characters.

Extra spaces between words should be distributed as evenly as possible. If the number of spaces on a line does not divide evenly between words, the empty slots on the left will be assigned more spaces than the slots on the right.

Round 3 · High · Hard

p.1205

For the last line of text, it should be left-justified, and no extra space is inserted between words.

Note:

- A word is defined as a character sequence consisting of non-space characters only.
- Each word's length is guaranteed to be greater than 0 and not exceed `maxWidth`.
- The input array `words` contains at least one word.

Example 1:

```
Input: words = ["This", "is", "an", "example", "of", "text", "justification."],
       maxWidth = 16
Output:
[
  "This    is    an",
  "example of text",
  "justification. "
]
```

Example 2:

```
Input: words = ["What", "must", "be", "acknowledgment", "shall", "be"], maxWidth = 16
Output:
[
  "What  must  be",
  "acknowledgment ",
  "shall be      "
]
Explanation: Note that the last line is "shall be " instead of "shall be",
because the last line must be left-justified instead of fully-justified.
```

Example 3:

Round 3 · High · Hard

p.1206

```
Input: words = ["Science","is","what","we","understand","well","enough","to","
explain","to","a","computer.","Art","is","everything","else","we","do"],
maxWidth = 20
Output:
[
  "Science is what we",
  "understand well",
  "enough to explain to",
  "a computer. Art is",
  "everything else we",
```

Constraints:

- $1 \leq \text{words.length} \leq 300$
- $1 \leq \text{words}[i].\text{length} \leq 20$
- $\text{words}[i]$ consists of only English letters and symbols.
- $1 \leq \text{maxWidth} \leq 100$
- $\text{words}[i].\text{length} \leq \text{maxWidth}$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def fullJustify(self, words: list[str], maxWidth: int) -> list[str]:
        ans = []
        row = []
        rowLetters = 0

        for word in words:
            # If we place the word in this row, it will exceed the maximum width.
            # Therefore, we cannot put the word in this row and have to pad spaces
            # for each word in this row.
```

```
if rowLetters + len(word) + len(row) > maxWidth:
    for i in range(maxWidth - rowLetters):
        row[i % (len(row) - 1 or 1)] += ' '
    ans.append(''.join(row))
    row = []
    rowLetters = 0
    row.append(word)
    rowLetters += len(word)

return ans + [' '.join(row).ljust(maxWidth)]
```

• LeetDoocs

```
class Solution:
    def fullJustify(self, words: List[str], maxWidth: int) -> List[str]:
        ans = []
        i, n = 0, len(words)
        while i < n:
            t = []
            cnt = len(words[i])
            t.append(words[i])
            i += 1
            while i < n and cnt + 1 + len(words[i]) <= maxWidth:
                cnt += 1 + len(words[i])
                t.append(words[i])
                i += 1
            if i == n or len(t) == 1:
                left = ''.join(t)
                right = ' ' * (maxWidth - len(left))
                ans.append(left + right)
                continue
            space_width = maxWidth - (cnt - len(t) + 1)
            w, m = divmod(space_width, len(t) - 1)

            row = []
            for j, s in enumerate(t[:-1]):
                row.append(s)
                row.append(' ' * (w + (1 if j < m else 0)))
            row.append(t[-1])
            ans.append(''.join(row))
        return ans
```


• SimplyLeet

```
def fullJustify(words, maxWidth):
    result = [] # Final list of justified text lines.
    current_words = [] # Current line's words.
    current_len = 0 # Total length of current line's words (excluding
spaces).

    # Process all words.
    for word in words:
        # Check if adding the next word would exceed the maxWidth when
including minimum one space between words.
        if current_words and (current_len + len(word) + len(current_words)) >
maxWidth:
            total_spaces = maxWidth - current_len # Total spaces to
distribute.

            gaps = len(current_words) - 1 # Number of gaps
between words.

            # If there is only one word, left justify.
            if gaps == 0:
                line = current_words[0] + ' ' * total_spaces
            else:
                # Calculate even spaces and extras to add from the left.
                space_per_gap, extra = divmod(total_spaces, gaps)
                line = ''
                for i in range(gaps):

                    # Distribute an extra space to the leftmost gaps.
                    spaces = space_per_gap + (1 if i < extra else 0)
                    line += current_words[i] + ' ' * spaces

                line += current_words[-1] # Append the
last word.

            result.append(line)
            # Reset the current line to start with the current word.
            current_words = []
            current_len = 0

    # Add the word to the current line.
```

Round 3 · High · Hard

p.1209

```
current_words.append(word)
current_len += len(word)

# Process the last line, which is left-justified.
line = ' '.join(current_words)
line += ' ' * (maxWidth - len(line))
result.append(line)
return result
```

Example usage:

```
words = ["This", "is", "an", "example", "of", "text", "justification."]
maxWidth = 16
print(fullJustify(words, maxWidth))
```

• LC.ca

```
'''
Explanation:

- The function `fullJustify` takes a list of words and a maximum width
`maxWidth` as inputs.
- It iterates over each word, deciding whether to add the current word to the
current line (`cur`) or to start a new line.
- If adding the current word to the line would exceed `maxWidth`, it justifies
the current line by adding extra spaces between words as needed, then starts a
new line.
- Once all words are processed, it left-justifies the last line by joining the
remaining words in `cur` with a single space and then using `.ljust(maxWidth)`
to ensure the line is of maximum width.
- The result is a list of strings, where each string represents a justified
line of text.
```

This solution carefully handles edge cases, such as when there's only one word
in the line (avoiding division by zero) and ensuring the last line is
left-justified instead of fully justified.

Round 3 · High · Hard

p.1210

```
'''
'''
>>> cur = [1]
>>> len(cur) - 1
0
>>> ( len(cur) - 1 or 1 )
1
'''
```

```
'''
# api: string.ljust(width[, fillchar])

>>> "Hello".ljust(10)
'Hello '
>>> "Hello".ljust(10, '-')
'Hello-----'
>>> "Hello".ljust(2)
'Hello'
```

```
>>> "Hello World".ljust(20)
'Hello World '
>>> "Hello World".ljust(20, '-')
'Hello World-----'
```

```
# api: string.rjust(width[, fillchar])
>>> "Hello".rjust(10)
' Hello'
>>> "Hello".rjust(10, '-')
'------Hello'
```

```
>>> "Hello".rjust(2)
'Hello'
```

```
>>> "Hello World".rjust(20)
' Hello World'
>>> "Hello World".rjust(20, '-')
'------Hello World'
```

```
'''
```

Round 3 · High · Hard

p.1211

```
class Solution:
    def fullJustify(self, words: List[str], maxWidth: int) -> List[str]:
        res, cur, num_of_letters = [], [], 0

        for w in words:
            # len(cur) is the space count, from last round, right one less
            # after adding 'w'
            if num_of_letters + len(cur) + len(w) > maxWidth:
                for i in range(maxWidth - num_of_letters):
                    # The "or 1" part is for dealing with the edge case
                    'len(cur) == 1'
```

```
                cur[i % ( len(cur) - 1 or 1 )] += ' '
                res.append(''.join(cur))
                cur, num_of_letters = [], 0
            cur += [w]
            num_of_letters += len(w)

        return res + [' '.join(cur).ljust(maxWidth)]
```

```
#####
```

```
'''
>>> 17 % 3
2
>>> divmod(17,3)
(5, 2)
'''
```

```
'''
>>> ['This', 'is', 'an']
```

Round 3 · High · Hard

p.1212

```

['This', 'is', 'an']
>>> t = ['This', 'is', 'an']
>>> spaces = [' ', ' ']

# note: here j starts from 0
>>> for j, word in enumerate(t[1:]):
...     print(j)
...     print(word)
...
0

is
1
an
'''

class Solution:
    def fullJustify(self, words: List[str], maxWidth: int) -> List[str]:
        def partition(n, cnt):
            base, mod = divmod(n, cnt)
            res = [f"{' ' * (base + (i < mod))}" for i in range(cnt)]

            return res

        ans = []
        i, n = 0, len(words)
        while i < n: # one line per iteration
            t = [words[i]]
            cnt = len(words[i])
            i += 1 # move to next word
            while i < n and cnt + 1 + len(words[i]) <= maxWidth: # greedy
                search for one line
                cnt += 1 + len(words[i])

```

Round 3 · High · Hard

p.1213

```

        t.append(words[i])
        i += 1
    if i == n or len(t) == 1:
        # this is the last line or only one (super-long) word in a line
        left = ' '.join(t)
        right = ' ' * (maxWidth - len(left))
        ans.append(f"{left}{right}")
        continue # so i != n, so only one word, no need to add spaces.
                  # e.g. a word with the same length as maxWidth,
                  # or, e.g. the 2nd word is super long and cannot fit
in current line

        # if i==n, then will not enter next while
        words_width = cnt - (len(t) - 1) # pure words total length, with no
spaces

                                     # spaces in-between these words in
t[: len(t) - 1]
        space_width = maxWidth - words_width
        spaces = partition(space_width, len(t) - 1)
        sb = [t[0]] # 1st word of this line, no space before it
        for j, word in enumerate(t[1:]): # last word has no following space
            sb.append(spaces[j]) # j starts from 0
            sb.append(word)
        ans.append(''.join(sb))

    return ans

#####

class Solution:
    def fullJustify(self, words: List[str], maxWidth: int) -> List[str]:
        left = 0
        result = []

        while left < len(words):

```

Round 3 · High · Hard

p.1214

```

        right = self.findRight(left, words, maxWidth)
        result.append(self.justify(left, right, words, maxWidth))
        left = right + 1

    return result

def findRight(self, left, words, maxWidth):
    right = left
    word_length = len(words[right])

    while (right + 1) < len(words) and (word_length + 1 + len(words[right+
1])) <= maxWidth:
        right += 1
        word_length += 1 + len(words[right])

    return right

def justify(self, left, right, words, maxWidth):
    if right - left == 0:
        return self.padResult(words[left], maxWidth)

    is_last_line = (right == len(words) - 1)
    num_spaces = right - left
    total_space = maxWidth - self.wordsLength(left, right, words)

    space = " " if is_last_line else " " * (total_space // num_spaces)
    remainder = 0 if is_last_line else total_space % num_spaces

    result = ""
    for i in range(left, right):
        result += words[i]

```

Round 3 · High · Hard

p.1215

```

        result += space
        if remainder > 0:
            result += " "
            remainder -= 1

    result += words[right]
    result += " " * (maxWidth - len(result))

    return result

def wordsLength(self, left, right, words):
    words_length = 0
    for i in range(left, right+1):
        words_length += len(words[i])
    return words_length

def padResult(self, result, maxWidth):
    return result + " " * (maxWidth - len(result))

```

◆ Quick Review

Key Insights

- Use a greedy approach to fill each line with as many words as possible without exceeding `maxWidth`.
- Calculate total space to distribute by subtracting the combined words length from `maxWidth`.
- If the line has more than one word, distribute extra spaces evenly; if not, pad the line with spaces at the end.

Round 3 · High · Hard

p.1216

- Handle the last line separately by left-justifying the text (i.e., adding a single space between words and padding the end).

Space & Time

Time Complexity: $O(n)$ where n is the number of words, since each word is processed once.

Space Complexity: $O(n)$ for storing the result and temporary buffers.

Solution

The solution iterates over the list of words and groups them into lines such that the combined length of the words and minimum required spaces does not exceed `maxWidth`. For each completed group (line):

- If it's not the last line, calculate extra spaces required.
- If the line has multiple words, distribute the spaces evenly, adding any extra spaces from left to right.
- If the line contains a single word, append spaces to the right.

For the last line, join words with a single space and pad the remaining width with spaces.

Data structures used include lists (or arrays) for current words accumulation and the final result. The logic is implemented using a

Round 3 · High · Hard

p.1217

simulation approach to mimic text formatting.

Round 3 · High · Hard

p.1218

Strings

#843 Guess the Word

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Math · String · Interactive · Game
Theory

Lists: AlgoMaster 600

Problem

You are given an array of unique strings `words` where `words[i]` is six letters long. One word of `words` was chosen as a secret word.

You are also given the helper object `Master`. You may call `Master.guess(word)` where `word` is a six-letter-long string, and it must be from `words`. `Master.guess(word)` returns:

- `-1` if `word` is not from `words`, or
- an integer representing the number of exact matches (value and position) of your guess to the secret word.

There is a parameter `allowedGuesses` for each test case where `allowedGuesses` is the maximum number of times you can call

Round 3 · High · Hard

p.1219

Master.guess(word).

For each test case, you should call `Master.guess` with the secret word without exceeding the maximum number of allowed guesses. You will get:

- "Either you took too many guesses, or you did not find the secret word." if you called `Master.guess` more than `allowedGuesses` times or if you did not call `Master.guess` with the secret word, or
- "You guessed the secret word correctly." if you called `Master.guess` with the secret word with the number of calls to `Master.guess` less than or equal to `allowedGuesses`.

The test cases are generated such that you can guess the secret word with a reasonable strategy (other than using the brute force method).

Example 1:

```
Input: secret = "acckzz", words = ["acckzz","ccbazz","eiowzz","abcczz"],
       allowedGuesses = 10
Output: You guessed the secret word correctly.
Explanation:
master.guess("aaaaaa") returns -1, because "aaaaaa" is not in words.
master.guess("acckzz") returns 6, because "acckzz" is secret and has all 6
matches.
master.guess("ccbazz") returns 3, because "ccbazz" has 3 matches.
master.guess("eiowzz") returns 2, because "eiowzz" has 2 matches.
master.guess("abcczz") returns 4, because "abcczz" has 4 matches.
```

Round 3 · High · Hard

p.1220

Example 2:

Input: secret = "hamada", words = ["hamada", "khaled"], allowedGuesses = 10
 Output: You guessed the secret word correctly.
 Explanation: Since there are two words, you can guess both.

Constraints:

- $1 \leq \text{words.length} \leq 100$
- $\text{words}[i].\text{length} == 6$
- $\text{words}[i]$ consist of lowercase English letters.
- All the strings of words are unique.
- secret exists in words.
- $10 \leq \text{allowedGuesses} \leq 30$

Community Solutions (Python)

• WalkCC

```
# """
# This is Master's API interface.
# You should not implement it, or speculate about its implementation
# """
# Class Master:
# def guess(self, word: str) -> int:

class Solution:
    def findSecretWord(self, words: List[str], master: 'Master') -> None:
        for _ in range(10):

            guessedWord = words[random.randint(0, len(words) - 1)]
            matches = master.guess(guessedWord)
            if matches == 6:
                break
            words = [
                word for word in words
                if sum(c1 == c2 for c1, c2 in zip(guessedWord, word)) == matches]
```

Round 3 · High · Hard

p.1221

• SimplyLeet

```
# Python solution for "Guess the Word" problem

class Solution:
    def findSecretWord(self, wordlist, master):
        # Helper function to count matching positions between two words
        def countMatches(word1, word2):
            matches = 0
            for c1, c2 in zip(word1, word2):
                if c1 == c2:
                    matches += 1

            return matches

        # Initially, all words are candidates
        candidates = wordlist[:]

        # Iterate until we find the secret word or run out of allowed guesses
        for _ in range(10): # allowed guesses is at most 10-30, using 10 as a
            # safe upper bound
            # pick a candidate word - a simple strategy is to select the first
            # word
            guess_word = candidates[0]

            # Make a guess using Master.guess; it returns the count of matching
            # characters
            matchCount = master.guess(guess_word)

            # If the secret word is found (6 matches for a 6-letter word),
            # return immediately
            if matchCount == 6:
                return

            # Otherwise filter the candidates based on the match count
            new_candidates = []
            for word in candidates:

                if countMatches(guess_word, word) == matchCount:
                    new_candidates.append(word)
            candidates = new_candidates
```

Round 3 · High · Hard

p.1222

Two Pointers

Round 3 · High · Hard · 2 questions

Round 3 · High · Hard

p.1223

Two Pointers

□ #42 Trapping Rain Water

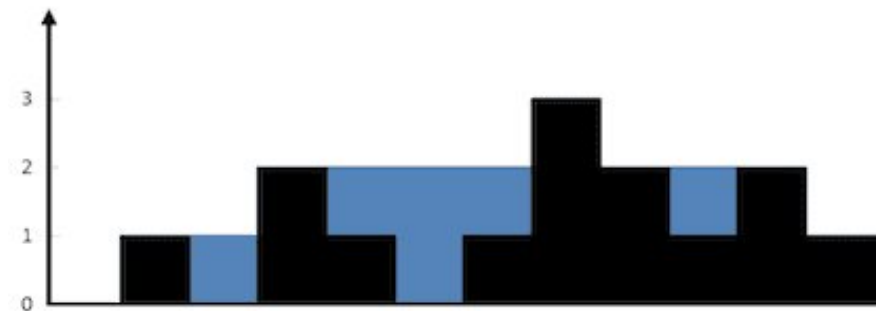
HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Two Pointers · Dynamic Programming ·
 Stack · Monotonic Stack
 Lists: Grind 169

Problem

Given n non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining.

Example 1:



Round 3 · High · Hard

p.1224

Input: height = [0,1,0,2,1,0,1,3,2,1,2,1]

Output: 6

Explanation: The above elevation map (black section) is represented by array [0,1,0,2,1,0,1,3,2,1,2,1]. In this case, 6 units of rain water (blue section) are being trapped.

Example 2:

Input: height = [4,2,0,3,2,5]

Output: 9

Constraints:

- $n == \text{height.length}$
- $1 \leq n \leq 2 * 10^4$
- $0 \leq \text{height}[i] \leq 10^5$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def trap(self, height: list[int]) -> int:
        n = len(height)
        l = [0] * n # l[i] := max(height[0..i])
        r = [0] * n # r[i] := max(height[i..n))

        for i, h in enumerate(height):
            l[i] = h if i == 0 else max(h, l[i - 1])

        for i, h in reversed(list(enumerate(height))):
            r[i] = h if i == n - 1 else max(h, r[i + 1])

        return sum(min(l[i], r[i]) - h
                    for i, h in enumerate(height))
```

```
class Solution:
    def trap(self, height: list[int]) -> int:
        if not height:
            return 0
```

```
ans = 0
l = 0
r = len(height) - 1
maxL = height[l]
maxR = height[r]
```

```
while l < r:
    if maxL < maxR:
        ans += maxL - height[l]
        l += 1
        maxL = max(maxL, height[l])
    else:
        ans += maxR - height[r]
        r -= 1
        maxR = max(maxR, height[r])
```

```
return ans
```

• LeetDoocs

```
class Solution:
    def trap(self, height: List[int]) -> int:
        n = len(height)
        left = [height[0]] * n
        right = [height[-1]] * n
        for i in range(1, n):
            left[i] = max(left[i - 1], height[i])
            right[n - i - 1] = max(right[n - i], height[n - i - 1])
        return sum(min(l, r) - h for l, r, h in zip(left, right, height))
```

• SimplyLeet

```
# Python solution using the two pointers approach
def trap(height):
    # Initialize left and right pointers and water accumulator
    left, right = 0, len(height) - 1
    left_max, right_max = 0, 0
    water = 0

    # Traverse the elevation map from both ends
    while left < right:
        # Update left_max and right_max as we encounter new bars

        if height[left] < height[right]:
            # left side is the determining side
            if height[left] >= left_max:
                left_max = height[left] # update left max since current height
            is higher
            else:
                water += left_max - height[left] # water trapped at left
            position
            left += 1 # move left pointer to the right
        else:
            # right side is the determining side
            if height[right] >= right_max:
                right_max = height[right] # update right max since current
            height is higher
            else:
                water += right_max - height[right] # water trapped at right
            position
            right -= 1 # move right pointer to the left
        return water

# Example usage:
print(trap([0,1,0,2,1,0,1,3,2,1,2,1])) # Expected output: 6
```

☐ • LC.ca

Round 3 · High · Hard

p.1227

```
class Solution:
    def trap(self, height: List[int]) -> int:
        n = len(height)
        if n < 3:
            return 0

        lmx = [height[0]] * n
        rmx = [height[n - 1]] * n
        for i in range(1, n):
            lmx[i] = max(lmx[i - 1], height[i]) # i self compare

            rmx[n - 1 - i] = max(rmx[n - i], height[n - 1 - i])

        # no negative, worst is, min(left,right) is itself
        return sum( min(lmx[i], rmx[i]) - height[i] for i in range(n) )

#####

class Solution: # zip
    def trap(self, height: List[int]) -> int:
        n = len(height)

        left = [height[0]] * n
        right = [height[-1]] * n
        for i in range(1, n):
            left[i] = max(left[i - 1], height[i])
            right[n - i - 1] = max(right[n - i], height[n - i - 1])
        return sum(min(l, r) - h for l, r, h in zip(left, right, height))

#####

class Solution: # find the highest bar first

    def trap(self, height: List[int]) -> int:
        if not height:
            return 0

        total_water = 0
        max_index = 0
        max_height = float('-inf')

        # Find the index of the maximum height
        for i in range(len(height)):
```

Round 3 · High · Hard

p.1228

```

    if height[i] > max_height:
        max_height = height[i]
        max_index = i

    # Calculate water trapped on the left side of the maximum height
    left_max = height[0]
    for i in range(1, max_index):
        if left_max < height[i]:
            left_max = height[i]
        total_water += left_max - height[i]

    # Calculate water trapped on the right side of the maximum height
    right_max = height[-1]
    for i in range(len(height) - 2, max_index, -1):
        if right_max < height[i]:
            right_max = height[i]
        total_water += right_max - height[i]

    return total_water

```

◆ Quick Review

Key Insights

- The water trapped at any index is determined by the minimum of the maximum height on its left and right minus the height at that index.
- A two-pointers approach can efficiently solve the problem in one pass by maintaining left and right boundaries.
- Alternative approaches include dynamic programming with pre-computed left max and

right max arrays, or using a monotonic stack to determine boundaries.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(1)$ when using the two-pointer approach

Solution

We use a two-pointers strategy by initializing pointers at the beginning and the end of the array. Keep track of the maximum height seen so far from the left (left_max) and the right (right_max). At each step, move the pointer with the lesser height because the water trapped is determined by the shorter boundary. Compute the water at that position as the difference between the boundary (left_max or right_max) and the current height, accumulating it into the total. This approach ensures linear time scanning with constant extra space.

Two Pointers

☐ #2444 Count Subarrays With Fixed Bounds

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Queue · Sliding Window · Monotonic
Queue

Lists: AlgoMaster 600

Problem

You are given an integer array `nums` and two integers `minK` and `maxK`.

A fixed-bound subarray of `nums` is a subarray that satisfies the following conditions:

- The minimum value in the subarray is equal to `minK`.
- The maximum value in the subarray is equal to `maxK`.

Return the number of fixed-bound subarrays.

A subarray is a contiguous part of an array.

Example 1:

Input: `nums = [1,3,5,2,7,5]`, `minK = 1`, `maxK = 5`
Output: 2
Explanation: The fixed-bound subarrays are `[1,3,5]` and `[1,3,5,2]`.

Round 3 · High · Hard

p.1231

Example 2:

Input: `nums = [1,1,1,1]`, `minK = 1`, `maxK = 1`
Output: 10
Explanation: Every subarray of `nums` is a fixed-bound subarray. There are 10 possible subarrays.

Constraints:

- $2 \leq \text{nums.length} \leq 105$
- $1 \leq \text{nums}[i], \text{minK}, \text{maxK} \leq 106$

Community Solutions (Python)

☐ • WalkCC

```
class Solution:
    def countSubarrays(self, nums: list[int], minK: int, maxK: int) -> int:
        ans = 0
        j = -1
        prevMinKIndex = -1
        prevMaxKIndex = -1

        for i, num in enumerate(nums):
            if num < minK or num > maxK:
                j = i

            if num == minK:
                prevMinKIndex = i
            if num == maxK:
                prevMaxKIndex = i
            # Any index k in [j + 1, min(prevMinKIndex, prevMaxKIndex)] can be the
            # start of the subarray s.t. nums[k..i] satisfies the conditions.
            ans += max(0, min(prevMinKIndex, prevMaxKIndex) - j)

        return ans
```

☐ • LeetDoocs

Round 3 · High · Hard

p.1232

```
class Solution:
    def countSubarrays(self, nums: List[int], minK: int, maxK: int) -> int:
        j1 = j2 = k = -1
        ans = 0
        for i, v in enumerate(nums):
            if v < minK or v > maxK:
                k = i
            if v == minK:
                j1 = i
            if v == maxK:
                j2 = i
            ans += max(0, min(j1, j2) - k)
        return ans
```

• SimplyLeet

```
# Python implementation with line-by-line comments

def countSubarrays(nums, minK, maxK):
    # Initialize pointers for last indices of minK, maxK, and invalid element
    last_invalid = -1
    last_min = -1
    last_max = -1
    count = 0

    # Loop through all indices in the nums array

    for index, num in enumerate(nums):
        # Check if current number is out of valid range
        if num < minK or num > maxK:
            last_invalid = index # Reset the last invalid position to current
            index

        # Update last position where minK is found
        if num == minK:
            last_min = index

        # Update last position where maxK is found
```

Round 3 · High · Hard

p.1233

```
if num == maxK:
    last_max = index

    # Calculate valid subarray count ending at the current index
    # Only consider subarray if both minK and maxK have been seen after the
    last invalid index
    valid_start = min(last_min, last_max)
    if valid_start > last_invalid:
        count += valid_start - last_invalid

return count

# Example usage:
nums = [1, 3, 5, 2, 7, 5]
minK = 1
maxK = 5
print(countSubarrays(nums, minK, maxK)) # Expected output: 2
```

Round 3 · High · Hard

p.1234

Sliding Window – Fixed Size

Round 3 · High · Hard · 1 question

Round 3 · High · Hard

p.1235

Sliding Window – Fixed Size

□ #30 Substring with Concatenation of All Words

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Hash Table · String · Sliding Window
 Lists: AlgoMaster 600

Problem

You are given a string *s* and an array of strings *words*. All the strings of *words* are of the same length.

A concatenated string is a string that exactly contains all the strings of any permutation of words concatenated.

- For example, if *words* = ["ab","cd","ef"], then "abcdef", "abefcd", "cdabef", "cdefab", "efabcd", and "efcdab" are all concatenated strings. "acdbef" is not a concatenated string because it is not the concatenation of any permutation of words.

Return an array of the starting indices of all the concatenated substrings in *s*. You can return the answer in any order.

Round 3 · High · Hard

p.1236

Example 1:

Input: `s = "barfoothefoobarman"`, `words = ["foo","bar"]`

Output: `[0,9]`

Explanation:

The substring starting at 0 is "barfoo". It is the concatenation of ["bar","foo"] which is a permutation of words. The substring starting at 9 is "foobar". It is the concatenation of ["foo","bar"] which is a permutation of words.

Example 2:

Input: `s = "wordgoodgoodgoodbestword"`, `words = ["word","good","best","word"]`

Output: `[]`

Explanation:

There is no concatenated substring.

Example 3:

Input: `s = "barfoofoobarthefoobarman"`, `words = ["bar","foo","the"]`

Output: `[6,9,12]`

Explanation:

The substring starting at 6 is "foobarthe". It is the concatenation of ["foo","bar","the"]. The substring starting at 9 is "barthefoo". It is the

concatenation of ["bar","the","foo"]. The substring starting at 12 is "thefoobar". It is the concatenation of ["the","foo","bar"].

Constraints:

- `1 <= s.length <= 104`
- `1 <= words.length <= 5000`
- `1 <= words[i].length <= 30`
- `s` and `words[i]` consist of lowercase English letters.

Community Solutions (Python)
☐ • WalkCC

```
class Solution:
    def findSubstring(self, s: str, words: list[str]) -> list[int]:
        if len(s) == 0 or words == []:
            return []

        k = len(words)
        n = len(words[0])
        ans = []
        count = collections.Counter(words)

        for i in range(len(s) - k * n + 1):
            seen = collections.defaultdict(int)
            j = 0
            while j < k:
                word = s[i + j * n: i + (j + 1) * n]
                seen[word] += 1
                if seen[word] > count[word]:
                    break
                j += 1
            if j == k:
                ans.append(i)
```

```
ans.append(i)
```

```
return ans
```

• LeetCode

```
class Solution:
    def findSubstring(self, s: str, words: List[str]) -> List[int]:
        cnt = Counter(words)
        m, n = len(s), len(words)
        k = len(words[0])
        ans = []
        for i in range(k):
            l = r = i
            cnt1 = Counter()
            while r + k <= m:
```

```
        t = s[r : r + k]
        r += k
        if cnt[t] == 0:
            l = r
            cnt1.clear()
            continue
        cnt1[t] += 1
        while cnt1[t] > cnt[t]:
            rem = s[l : l + k]
            l += k
```

```
            cnt1[rem] -= 1
            if r - l == n * k:
                ans.append(l)
        return ans
```

• SimplyLeet

```
# Python solution with detailed comments
class Solution:
    def findSubstring(self, s: str, words: [str]) -> [int]:
        if not s or not words:
            return []
        word_length = len(words[0])
        total_words = len(words)
        window_size = word_length * total_words
        if len(s) < window_size:
            return []
```

```
# Build the frequency map of words
word_freq = {}
for word in words:
    word_freq[word] = word_freq.get(word, 0) + 1

results = []
# Check each offset to account for alignment
for i in range(word_length):
    left = i
```

```
    current_count = {}
    count = 0 # current number of words matched
    # Slide the window in increments of word_length
    for right in range(i, len(s) - word_length + 1, word_length):
        curr_word = s[right:right+word_length]
        if curr_word in word_freq:
            current_count[curr_word] = current_count.get(curr_word, 0)
            + 1

            count += 1
            # Shrink window if frequency is exceeded
            while current_count[curr_word] > word_freq[curr_word]:
```



```
        left_word = s[left:left+word_length]
        current_count[left_word] -= 1
        left += word_length
        count -= 1
    # If window matches all words, record the index
    if count == total_words:
        results.append(left)
    else:
        # Reset the window if the word is not in the words list
        current_count.clear()

    count = 0
    left = right + word_length
    return results
```

Round 3 · High · Hard

p.1241

Sliding Window – Dynamic Size

Round 3 · High · Hard · 3 questions

Round 3 · High · Hard

p.1242

Sliding Window – Dynamic Size

#76 Minimum Window Substring

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Sliding Window
Lists: Grind 169

Problem

Given two strings s and t of lengths m and n respectively, return the minimum window substring of s such that every character in t (including duplicates) is included in the window. If there is no such substring, return the empty string "".

The testcases will be generated such that the answer is unique.

Example 1:

Input: $s = \text{"ADOBECODEBANC"}, t = \text{"ABC"}$
Output: "BANC"
Explanation: The minimum window substring "BANC" includes 'A', 'B', and 'C' from string t .

Example 2:

Input: $s = \text{"a"}, t = \text{"a"}$
Output: "a"
Explanation: The entire string s is the minimum window.

Example 3:

Round 3 · High · Hard

p.1243

Input: $s = \text{"a"}, t = \text{"aa"}$
Output: ""
Explanation: Both 'a's from t must be included in the window. Since the largest window of s only has one 'a', return empty string.

Constraints:

- $m == s.length$
- $n == t.length$
- $1 \leq m, n \leq 105$
- s and t consist of uppercase and lowercase English letters.

Follow up: Could you find an algorithm that runs in $O(m + n)$ time?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minWindow(self, s: str, t: str) -> str:
        count = collections.Counter(t)
        required = len(t)
        bestLeft = -1
        minLength = len(s) + 1

        l = 0
        for r, c in enumerate(s):
            count[c] -= 1
```

Round 3 · High · Hard

p.1244

```

if count[c] >= 0:
    required -= 1
while required == 0:
    if r - l + 1 < minLength:
        bestLeft = l
        minLength = r - l + 1
    count[s[l]] += 1
    if count[s[l]] > 0:
        required += 1
    l += 1

return '' if bestLeft == -1 else s[bestLeft: bestLeft + minLength]

```

• LeetCode

```

class Solution:
    def minWindow(self, s: str, t: str) -> str:
        need = Counter(t)
        window = Counter()
        cnt = l = 0
        k, mi = -1, inf
        for r, c in enumerate(s):
            window[c] += 1
            if need[c] >= window[c]:
                cnt += 1

        while cnt == len(t):
            if r - l + 1 < mi:
                mi = r - l + 1
                k = l
            if need[s[l]] >= window[s[l]]:
                cnt -= 1
            window[s[l]] -= 1
            l += 1
        return '' if k < 0 else s[k : k + mi]

```

• SimplyLeet

Round 3 · High · Hard

p.1245

```

# Python implementation of Minimum Window Substring solution

def minWindow(s: str, t: str) -> str:
    # If t is longer than s, no solution exists.
    if len(t) > len(s):
        return ""

    # Dictionary to store the frequency of each character in t.
    t_freq = {}
    for char in t:
        t_freq[char] = t_freq.get(char, 0) + 1

    # Dictionary to keep track of the characters in the current window.
    window_freq = {}

    required = len(t_freq) # Unique characters in t to be matched
    formed = 0 # Number of unique characters in the current window that meet
    required frequency

    # (window_length, left, right)
    answer = (float('inf'), None, None)

    l = 0 # Left pointer
    # Start sliding the window with the right pointer.
    for r, char in enumerate(s):
        # Add the current character to the window counter.
        window_freq[char] = window_freq.get(char, 0) + 1

        # Check if the current character's frequency in the window matches that
        in t.
        if char in t_freq and window_freq[char] == t_freq[char]:
            formed += 1

```

Round 3 · High · Hard

p.1246

```

# Try and contract the window if all characters are matched.
while l <= r and formed == required:
    character = s[l]

    # Update answer if the current window is smaller than the previous
    best.

    if r - l + 1 < answer[0]:
        answer = (r - l + 1, l, r)

    # The character at the left is going to be removed from the window.

    window_freq[character] -= 1
    if character in t_freq and window_freq[character] < t_freq[character]:
        formed -= 1

    # Move the left pointer ahead.
    l += 1

# Return the smallest window, if found.
return "" if answer[0] == float('inf') else s[answer[1]:answer[2]+1]

# Example usage:
if __name__ == "__main__":
    s = "ADOBECODEBANC"
    t = "ABC"
    print(minWindow(s, t)) # Expected output: "BANC"

```

• LC.ca

```

'''
>>> deq = collections.deque([])
>>> deq.append(11)
>>> deq.append(22)
>>> deq.append(33)
>>>
>>> deq[0]
11
'''

```

Round 3 · High · Hard

p.1247

```

import collections

class Solution:
    def minWindow(self, s: str, t: str) -> str:
        cnt = 0
        need = collections.Counter(t)
        start, end = len(s), 3 * len(s) # Arbitrary 3, just make sure end-start
        # is larger than input s length, for later min-check
        d = {}
        # Using queue to store indexes, good for a large amount of API calls
        deq = collections.deque([])

        for i, c in enumerate(s):
            if c in need:
                deq.append(i)
                d[c] = d.get(c, 0) + 1
                # '=' also +1, because it's increased already one line above :)
                if d[c] <= need[c]:
                    cnt += 1
                while deq and d[s[deq[0]]] > need[s[deq[0]]]:
                    d[s[deq.popleft()]] -= 1
                if cnt == len(t) and deq[-1] - deq[0] < end - start:
                    start, end = deq[0], deq[-1]

        return s[start:end + 1]

#####

from collections import Counter

class Solution:
    def minWindow(self, s: str, t: str) -> str:
        ans = ''

```

Round 3 · High · Hard

p.1248

```

m, n = len(s), len(t)
if m < n:
    return ans
need = Counter(t)
window = Counter()
i, cnt, mi = 0, 0, inf
for j, c in enumerate(s):
    window[c] += 1
    if need[c] >= window[c]: # >= , because 1 line above already +=1
        cnt += 1

    while cnt == n:
        if j - i + 1 < mi: # in while
            mi = j - i + 1
            ans = s[i : j + 1]
        c = s[i]
        if need[c] >= window[c]: # char in window but not in need,
            need[c]=0, window[c]=1..2..
            cnt -= 1
            window[c] -= 1
            i += 1
    return ans

```

◆ Quick Review

Key Insights

- Use a sliding window approach to dynamically adjust the window boundaries.
- Utilize a hash table (or dictionary) to keep track of the frequency of characters required (from t) and the frequency within the current window.
- Expand the window until it satisfies the requirements, then contract to minimize the

Round 3 · High · Hard

p.1249

window.

- Efficiently update counts and check valid window with two pointers (left and right).

Space & Time

Time Complexity: $O(m + n)$, where m is the length of s and n is the length of t .

Space Complexity: $O(m + n)$ in the worst case due to the storage for the hash maps.

Solution

We use a sliding window technique with two pointers representing the window's boundaries. First, create a frequency map of characters from t . Then, expand the right pointer to include characters in the window until the window contains all required characters (meeting or exceeding the counts in t). Once the requirement is met, try contracting the window by moving the left pointer to reduce its size while still maintaining a valid window. Throughout the process, if a valid window is found that is smaller than the previously recorded minimum window, update the minimum. The solution leans heavily on hash tables to track counts and compares

Round 3 · High · Hard

p.1250

them against the required counts from t.

Round 3 · High · Hard

p.1251

Sliding Window – Dynamic Size

□ #727 Minimum Window Subsequence

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming · Sliding
Window

Lists: AlgoMaster 600

Problem

Given strings s1 and s2, return the minimum contiguous substring part of s1, so that s2 is a subsequence of the part.

If there is no such window in s1 that covers all characters in s2, return the empty string "". If there are multiple such minimum-length windows, return the one with the left-most starting index.

Example 1:

Input: s1 = "abcdebddde", s2 = "bde"
Output: "bcde"
Explanation:
"bcde" is the answer because it occurs before "bdde" which has the same length.
"deb" is not a smaller window because the elements of s2 in the window must occur in order.

Example 2:

Round 3 · High · Hard

p.1252

Input: s1 = "jmeqksfrsdcmsiwvaovztaqenprpvbstl", s2 = "u"
Output: ""

Constraints:

- $1 \leq s1.length \leq 2 * 10^4$
- $1 \leq s2.length \leq 100$
- s1 and s2 consist of lowercase English letters.

Community Solutions (Python)

• LeetCode

```
class Solution:
    def minWindow(self, s1: str, s2: str) -> str:
        m, n = len(s1), len(s2)
        f = [[0] * (n + 1) for _ in range(m + 1)]
        for i, a in enumerate(s1, 1):
            for j, b in enumerate(s2, 1):
                if a == b:
                    f[i][j] = i if j == 1 else f[i - 1][j - 1]
                else:
                    f[i][j] = f[i - 1][j]

        p, k = 0, m + 1
        for i, a in enumerate(s1, 1):
            if a == s2[n - 1] and f[i][n]:
                j = f[i][n] - 1
                if i - j < k:
                    k = i - j
                    p = j
        return "" if k > m else s1[p : p + k]
```

• SimplyLeet

```
# Python solution for Minimum Window Subsequence

def minWindow(s1: str, s2: str) -> str:
    s1_len, s2_len = len(s1), len(s2)
    min_len = float('inf')
    start_index = -1

    # Iterate through s1
    for i in range(s1_len):
        # When the current character matches the first character of s2, try to
        # find a valid window

        if s1[i] == s2[0]:
            j = i
            k = 0
            # Move forward as long as characters match s2
            while j < s1_len and k < s2_len:
                if s1[j] == s2[k]:
                    k += 1
                j += 1
            # If entire s2 is matched, perform a backward scan
            if k == s2_len:
                end = j - 1 # end index of the valid window
                k -= 1
                # Move backward to find the start of the window
                while k >= 0:
                    if s1[j - 1] == s2[k]:
                        k -= 1
                    j -= 1
                j += 1 # adjust j to the actual start index
                # Update result if the current window is smaller
                if end - j + 1 < min_len:
                    min_len = end - j + 1
                    start_index = j
            return "" if start_index == -1 else s1[start_index:start_index+min_len]

# Example usage:
# result = minWindow("abcdebddde", "bde")
# print(result) # Expected "bcde"
```

Sliding Window – Dynamic Size

☐ #992 Subarrays with K Different Integers **HAR**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Sliding Window · Counting
Lists: AlgoMaster 600

Problem

Given an integer array `nums` and an integer `k`, return the number of good subarrays of `nums`.

A good array is an array where the number of different integers in that array is exactly `k`.

- For example, `[1,2,3,1,2]` has 3 different integers: 1, 2, and 3.

A subarray is a contiguous part of an array.

Example 1:

Input: `nums = [1,2,1,2,3]`, `k = 2`
Output: 7
Explanation: Subarrays formed with exactly 2 different integers: `[1,2]`, `[2,1]`, `[1,2]`, `[2,3]`, `[1,2,1]`, `[2,1,2]`, `[1,2,1,2]`

Example 2:

Input: `nums = [1,2,1,3,4]`, `k = 3`
Output: 3
Explanation: Subarrays formed with exactly 3 different integers: `[1,2,1,3]`, `[2,1,3]`, `[1,3,4]`.

Constraints:

Round 3 · High · Hard

p.1255

- `1 <= nums.length <= 2 * 104`
- `1 <= nums[i], k <= nums.length`

Community Solutions (Python)

☐ • WalkCC

```
class Solution:
    def subarraysWithKDistinct(self, nums: list[int], k: int) -> int:
        def subarraysWithAtMostKDistinct(k: int) -> int:
            res = 0
            count = collections.Counter()

            l = 0
            for r, num in enumerate(nums):
                count[num] += 1
                if count[num] == 1:
                    k -= 1

                while k < 0:
                    count[nums[l]] -= 1
                    if count[nums[l]] == 0:
                        k += 1
                    l += 1
                res += r - l + 1 # nums[l..r], nums[l + 1..r], ..., nums[r]

            return res

        return subarraysWithAtMostKDistinct(k) - subarraysWithAtMostKDistinct(k - 1)
```

☐ • LeetDoocs

Round 3 · High · Hard

p.1256


```
class Solution:
    def subarraysWithKDistinct(self, nums: List[int], k: int) -> int:
        def f(k):
            pos = [0] * len(nums)
            cnt = Counter()
            j = 0
            for i, x in enumerate(nums):
                cnt[x] += 1
                while len(cnt) > k:
                    cnt[nums[j]] -= 1
                    if cnt[nums[j]] == 0:
                        cnt.pop(nums[j])
                    j += 1
                pos[i] = j
            return pos

        return sum(a - b for a, b in zip(f(k - 1), f(k)))
```

• SimplyLeet

```
# Python code solution with line-by-line comments
def subarraysWithKDistinct(nums, k):
    # Helper function to count subarrays with at most K distinct integers
    def atMostKDistinct(K):
        count = {} # dictionary to hold frequency of elements
        left = result = 0
        # Iterate over nums using the right pointer
        for right, value in enumerate(nums):
            count[value] = count.get(value, 0) + 1 # update frequency
            # If distinct integers exceed K, move left pointer to adjust window

            while len(count) > K:
                count[nums[left]] -= 1 # reduce frequency for element at left
                if count[nums[left]] == 0:
                    del count[nums[left]] # remove element if frequency is
zero
                left += 1 # shrink window from left
            # All subarrays ending at right with at most K distinct integers
            result += right - left + 1
        return result

    # The number of subarrays with exactly k distinct integers
```

Round 3 · High · Hard

p.1257

```
return atMostKDistinct(k) - atMostKDistinct(k - 1)

# Example usage:
#print(subarraysWithKDistinct([1,2,1,2,3], 2)) # Output: 7
```

Round 3 · High · Hard

p.1258

Binary Search

Round 3 · High · Hard · 4 questions

Round 3 · High · Hard

p.1259

Binary Search

□ #4 Median of Two Sorted Arrays

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Divide and Conquer
Lists: Grind 169

Problem

Given two sorted arrays `nums1` and `nums2` of size `m` and `n` respectively, return the median of the two sorted arrays.

The overall run time complexity should be $O(\log(m+n))$.

Example 1:

Input: `nums1 = [1,3]`, `nums2 = [2]`
Output: `2.00000`
Explanation: merged array = `[1,2,3]` and median is 2.

Example 2:

Input: `nums1 = [1,2]`, `nums2 = [3,4]`
Output: `2.50000`
Explanation: merged array = `[1,2,3,4]` and median is $(2 + 3) / 2 = 2.5$.

Constraints:

- `nums1.length == m`
- `nums2.length == n`
- $0 \leq m \leq 1000$

Round 3 · High · Hard

p.1260

- $0 \leq n \leq 1000$
- $1 \leq m + n \leq 2000$
- $-106 \leq \text{nums1}[i], \text{nums2}[i] \leq 106$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findMedianSortedArrays(self, nums1: list[int], nums2: list[int]) -> float:
        n1 = len(nums1)
        n2 = len(nums2)
        if n1 > n2:
            return self.findMedianSortedArrays(nums2, nums1)

        l = 0
        r = n1
```

```
while l <= r:
    partition1 = (l + r) // 2
    partition2 = (n1 + n2 + 1) // 2 - partition1
    maxLeft1 = -2**31 if partition1 == 0 else nums1[partition1 - 1]
    maxLeft2 = -2**31 if partition2 == 0 else nums2[partition2 - 1]
    minRight1 = 2**31 - 1 if partition1 == n1 else nums1[partition1]
    minRight2 = 2**31 - 1 if partition2 == n2 else nums2[partition2]
    if maxLeft1 <= minRight2 and maxLeft2 <= minRight1:
        return (max(maxLeft1, maxLeft2) + min(minRight1, minRight2)) * 0.5 if
(n1 + n2) % 2 == 0 else max(maxLeft1, maxLeft2)
    elif maxLeft1 > minRight2:
```

```
        r = partition1 - 1
    else:
        l = partition1 + 1
```

• LeetCode

Round 3 · High · Hard

p.1261

```
class Solution:
    def findMedianSortedArrays(self, nums1: List[int], nums2: List[int]) -> float:
        def f(i: int, j: int, k: int) -> int:
            if i >= m:
                return nums2[j + k - 1]
            if j >= n:
                return nums1[i + k - 1]
            if k == 1:
                return min(nums1[i], nums2[j])
            p = k // 2

            x = nums1[i + p - 1] if i + p - 1 < m else inf
            y = nums2[j + p - 1] if j + p - 1 < n else inf
            return f(i + p, j, k - p) if x < y else f(i, j + p, k - p)

        m, n = len(nums1), len(nums2)
        a = f(0, 0, (m + n + 1) // 2)
        b = f(0, 0, (m + n + 2) // 2)
        return (a + b) / 2
```

• SimplyLeet

```
# Function to find the median of two sorted arrays
def findMedianSortedArrays(nums1, nums2):
    # Ensure nums1 is the smaller array for faster binary search
    if len(nums1) > len(nums2):
        nums1, nums2 = nums2, nums1

    m, n = len(nums1), len(nums2)
    low, high = 0, m

    while low <= high:
```

Round 3 · High · Hard

p.1262

```

# Partition indexes for nums1 and nums2
partition_nums1 = (low + high) // 2
partition_nums2 = (m + n + 1) // 2 - partition_nums1

# If partition is 0 then there are no elements on left side. Use
-infinity for comparison.
maxLeftNums1 = float('-inf') if partition_nums1 == 0 else nums1[partition_nums1 - 1]
# If partition is length then there are no elements on right side. Use
infinity for comparison.
minRightNums1 = float('inf') if partition_nums1 == m else nums1[partition_nums1]

# Similarly for nums2

maxLeftNums2 = float('-inf') if partition_nums2 == 0 else nums2[partition_nums2 - 1]
minRightNums2 = float('inf') if partition_nums2 == n else nums2[partition_nums2]

# Check if we have found the correct partition
if maxLeftNums1 <= minRightNums2 and maxLeftNums2 <= minRightNums1:
    # If total number of elements is even
    if (m + n) % 2 == 0:
        return (max(maxLeftNums1, maxLeftNums2) + min(minRightNums1, minRightNums2)) / 2.0
    else:
        return float(max(maxLeftNums1, maxLeftNums2))

# If not, adjust binary search range
elif maxLeftNums1 > minRightNums2:
    # Move partition in nums1 to left
    high = partition_nums1 - 1
else:
    # Move partition in nums1 to right
    low = partition_nums1 + 1

# Example usage:
print(findMedianSortedArrays([1,3], [2])) # Output: 2.0

print(findMedianSortedArrays([1,2], [3,4])) # Output: 2.5

```

 • LC.ca

Round 3 · High · Hard

p.1263

```

class Solution:
    def findMedianSortedArrays(self, nums1: List[int], nums2: List[int]) -> float:

        def findKth(i: int, j: int, k: int) -> float:
            if i >= m:
                return nums2[j + k - 1]
            if j >= n:
                return nums1[i + k - 1]
            if k == 1:
                return min(nums1[i], nums2[j])

        # Division a // b : floordiv(a, b)
        midVal1 = nums1[i + k // 2 - 1] if i + k // 2 - 1 < m else math.inf
        midVal2 = nums2[j + k // 2 - 1] if j + k // 2 - 1 < n else math.inf

        if midVal1 < midVal2:
            return findKth(i + k // 2, j, k - k // 2) # '+' or '-' k//2
        else:
            return findKth(i, j + k // 2, k - k // 2)

        m = len(nums1)
        n = len(nums2)
        # Division a // b : floordiv(a, b)
        left = (m + n + 1) // 2
        right = (m + n + 2) // 2
        return (findKth(0, 0, left) + findKth(0, 0, right)) / 2.0

#####

# iteration version

```

Round 3 · High · Hard

p.1264

```
class Solution(object):
    def findMedianSortedArrays(self, nums1, nums2):
        a, b = sorted((nums1, nums2), key=len)
        m, n = len(a), len(b)
        after = (m + n - 1) / 2
        lo, hi = 0, m
        while lo < hi:
            i = (lo + hi) / 2
            if after - i - 1 < 0 or a[i] >= b[after - i - 1]:
                hi = i
            else:
                lo = i + 1
        i = lo
        nextfew = sorted(a[i:i + 2] + b[after - i:after - i + 2])
        return (nextfew[0] + nextfew[1 - (m + n) % 2]) / 2.0
```

◆ Quick Review

Key Insights

- The median divides the sorted array into two equal halves.
- We can perform binary search on the smaller array to partition both arrays correctly.
- We need to ensure that every element in the left partition is less than or equal to every element in the right partition.
- Edge cases must be handled when partitions are at the boundaries of either array.

Space & Time

Time Complexity: $O(\log(\min(m, n)))$

Round 3 · High · Hard

p.1265

Space Complexity: $O(1)$

Solution

The solution uses a binary search technique on the smaller array. We define a partition index i for `nums1` and compute the corresponding index j for `nums2` such that $i + j$ equals half of the total number of elements. The algorithm checks if the largest element on the left side of `nums1` (if any) is less than or equal to the smallest element on the right side of `nums2`, and vice versa for `nums2`. If the condition holds, we have found the correct partition and compute the median. If not, adjust the binary search range until the correct partition is found. This partitioning strategy allows us to find the median in logarithmic time.

Round 3 · High · Hard

p.1266

Binary Search

☐ #410 Split Array Largest Sum

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Dynamic Programming · Greedy · Prefix Sum

Lists: Denny Zhang

Problem

Given an integer array `nums` and an integer `k`, split `nums` into `k` non-empty subarrays such that the largest sum of any subarray is minimized.

Return the minimized largest sum of the split.

A subarray is a contiguous part of the array.

Example 1:

Input: `nums = [7,2,5,10,8]`, `k = 2`
Output: 18
Explanation: There are four ways to split `nums` into two subarrays.
The best way is to split it into `[7,2,5]` and `[10,8]`, where the largest sum among the two subarrays is only 18.

Example 2:

Input: `nums = [1,2,3,4,5]`, `k = 2`
Output: 9
Explanation: There are four ways to split `nums` into two subarrays.
The best way is to split it into `[1,2,3]` and `[4,5]`, where the largest sum among the two subarrays is only 9.

Round 3 · High · Hard

p.1267

Constraints:

- $1 \leq \text{nums.length} \leq 1000$
- $0 \leq \text{nums}[i] \leq 106$
- $1 \leq k \leq \min(50, \text{nums.length})$

Community Solutions (Python)

☐ • WalkCC

```
class Solution:
    def splitArray(self, nums: list[int], k: int) -> int:
        prefix = list(itertools.accumulate(nums, initial=0))

        @functools.lru_cache(None)
        def dp(i: int, k: int) -> int:
            """
            Returns the minimum of the maximum sum to split the first i numbers into
            k groups.
            """

            if k == 1:
                return prefix[i]
            return min(max(dp(j, k - 1), prefix[i] - prefix[j])
                       for j in range(k - 1, i))

        return dp(len(nums), k)

class Solution:
    def splitArray(self, nums: list[int], k: int) -> int:
        n = len(nums)
        # dp[i][k] := the minimum of the maximum sum to split the first i numbers
        # into k groups
        dp = [[math.inf] * (k + 1) for _ in range(n + 1)]
        prefix = list(itertools.accumulate(nums, initial=0))

        for i in range(1, n + 1):
            dp[i][1] = prefix[i]
```

Round 3 · High · Hard

p.1268

```

for l in range(2, k + 1):
    for i in range(l, n + 1):
        for j in range(l - 1, i):
            dp[i][l] = min(dp[i][l], max(dp[j][l - 1], prefix[i] - prefix[j]))

```

```

return dp[n][k]

```

```

class Solution:
    def splitArray(self, nums: list[int], k: int) -> int:
        l = max(nums)
        r = sum(nums) + 1

```

```

    def numGroups(maxSumInGroup: int) -> int:
        groupCount = 1
        sumInGroup = 0

```

```

    for num in nums:

```

```

        if sumInGroup + num <= maxSumInGroup:
            sumInGroup += num
        else:
            groupCount += 1
            sumInGroup = num

```

```

    return groupCount

```

```

while l < r:
    m = (l + r) // 2

```

```

    if numGroups(m) > k:
        l = m + 1
    else:
        r = m

```

```

return l

```

• LeetCode

Round 3 · High · Hard

p.1269

```

class Solution:
    def splitArray(self, nums: List[int], k: int) -> int:
        def check(mx):
            s, cnt = inf, 0
            for x in nums:
                s += x
                if s > mx:
                    s = x
                    cnt += 1
            return cnt <= k

```

```

left, right = max(nums), sum(nums)
return left + bisect_left(range(left, right + 1), True, key=check)

```

• SimplyLeet

Python solution for Split Array Largest Sum

```

def splitArray(nums, k):
    # Helper function to determine if we can split array into <= k subarrays
    # with max subarray sum <= max_sum
    def canSplit(max_sum):
        subarray_count = 1 # start with one subarray
        current_sum = 0
        for num in nums:
            if current_sum + num > max_sum:
                subarray_count += 1 # start a new subarray

```

```

                current_sum = num # reset the sum with the current number
            else:
                current_sum += num # add num to current subarray
        return subarray_count <= k # valid if we used k or fewer subarrays

```

```

low, high = max(nums), sum(nums) # boundaries for binary search
while low < high:
    mid = (low + high) // 2 # candidate maximum subarray sum
    if canSplit(mid):
        high = mid # candidate is valid, try to find a smaller one

```

Round 3 · High · Hard

p.1270

```

    else:
        low = mid + 1 # candidate too small, increase it
    return low

# Example usage:
nums = [7,2,5,10,8]
k = 2
print(splitArray(nums, k)) # Expected output: 18

```

• LC.ca

```

class Solution:
    def splitArray(self, nums: List[int], k: int) -> int:
        def check(mx):
            s, cnt = inf, 0
            for x in nums:
                s += x
                if s > mx:
                    s = x
                    cnt += 1
            return cnt <= k

        left, right = max(nums), sum(nums)
        return left + bisect_left(range(left, right + 1), True, key=check)

```

◆ Quick Review

Key Insights

- Use binary search over the answer: the candidate value is the maximum subarray sum.
- The lower bound of the search is $\max(\text{nums})$ and the upper bound is $\text{sum}(\text{nums})$.
- A greedy approach is used to check if a candidate maximum sum allows splitting

Round 3 · High · Hard

p.1271

nums into at most k subarrays.

- Adjust the binary search bounds based on the feasibility check.

Space & Time

Time Complexity: $O(n * \log(\text{sum}(\text{nums})))$ – n elements are processed for each binary search iteration.

Space Complexity: $O(1)$ – Constant extra space is used.

Solution

The solution employs binary search to determine the smallest maximum subarray sum possible. We set the initial lower bound to the maximum element and the upper bound to the total sum of the array. For each candidate value (mid), we check if the array can be split into k or fewer subarrays without any subarray sum exceeding mid using a greedy approach. If possible, we try to lower the candidate; if not, we increase the candidate. This continues until the optimal solution is found.

Round 3 · High · Hard

p.1272

Binary Search

#719 Find K-th Smallest Pair Distance

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Binary Search · Sorting
Lists: AlgoMaster 600

Problem

The distance of a pair of integers a and b is defined as the absolute difference between a and b .

Given an integer array `nums` and an integer k , return the k th smallest distance among all the pairs `nums[i]` and `nums[j]` where $0 \leq i < j < \text{nums.length}$.

Example 1:

Input: `nums = [1,3,1]`, $k = 1$
Output: 0
Explanation: Here are all the pairs:
(1,3) -> 2
(1,1) -> 0
(3,1) -> 2
Then the 1st smallest distance pair is (1,1), and its distance is 0.

Example 2:

Input: `nums = [1,1,1]`, $k = 2$
Output: 0

Example 3:

Round 3 · High · Hard

p.1273

Input: `nums = [1,6,1]`, $k = 3$
Output: 5

Constraints:

- $n == \text{nums.length}$
- $2 \leq n \leq 104$
- $0 \leq \text{nums}[i] \leq 106$
- $1 \leq k \leq n * (n - 1) / 2$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def smallestDistancePair(self, nums: list[int], k: int) -> int:
        nums.sort()

        def numPairDistancesNoGreaterThan(m: int) -> int:
            count = 0
            j = 1
            # For each index i, find the first index j s.t. nums[j] > nums[i] + m,
            # so numPairDistancesNoGreaterThan for the index i will be j - i - 1.
            for i, num in enumerate(nums):
                while j < len(nums) and nums[j] <= num + m:
                    j += 1
                count += j - i - 1
            return count

        return bisect.bisect_left(range(nums[-1] - nums[0]), k,
                                key=numPairDistancesNoGreaterThan)
```

• LeetDoocs

Round 3 · High · Hard

p.1274

```
class Solution:
    def smallestDistancePair(self, nums: List[int], k: int) -> int:
        def count(dist):
            cnt = 0
            for i, b in enumerate(nums):
                a = b - dist
                j = bisect_left(nums, a, 0, i)
                cnt += i - j
            return cnt
```

```
nums.sort()
return bisect_left(range(nums[-1] - nums[0]), k, key=count)
```

• SimplyLeet

```
# Function to find the kth smallest pair distance using binary search and two pointers
```

```
def smallestDistancePair(nums, k):
    # Sort the array to simplify finding pair distances.
    nums.sort()
```

```
# Define the search range for possible distances
low, high = 0, nums[-1] - nums[0]
```

```
# Helper function: count number of pairs with difference <= mid.
```

```
def count_pairs(mid):
```

```
    count, left = 0, 0
```

```
    # Use two pointers to count valid pairs.
```

```
    for right in range(len(nums)):
```

```
        while nums[right] - nums[left] > mid:
```

```
            left += 1
```

```
        count += right - left
```

```
    return count
```

```
# Binary search over the possible distances.
```

```
while low < high:
```

```
mid = (low + high) // 2 # Midpoint of current distance range.
if count_pairs(mid) < k:
    low = mid + 1 # Not enough pairs, increase low bound.
else:
    high = mid # Enough pairs, try a smaller distance.
return low
```

```
# Example usage:
```

```
print(smallestDistancePair([1, 3, 1], 1)) # Expected Output: 0
```

Binary Search

☐ #1095 Find in Mountain Array

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Interactive
Lists: AlgoMaster 600

Problem

(This problem is an interactive problem.)

You may recall that an array `arr` is a mountain array if and only if:

- `arr.length >= 3`
- There exists some `i` with `0 < i < arr.length - 1` such that: `arr[0] < arr[1] < ... < arr[i - 1] < arr[i]`
- `arr[i] > arr[i + 1] > ... > arr[arr.length - 1]`

Given a mountain array `mountainArr`, return the minimum index such that `mountainArr.get(index) == target`. If such an index does not exist, return `-1`.

You cannot access the mountain array directly. You may only access the array using a `MountainArray` interface:

Round 3 · High · Hard

p.1277

- `MountainArray.get(k)` returns the element of the array at index `k` (0-indexed).
- `MountainArray.length()` returns the length of the array.

Submissions making more than 100 calls to `MountainArray.get` will be judged Wrong Answer. Also, any solutions that attempt to circumvent the judge will result in disqualification.

Example 1:

```
Input: mountainArr = [1,2,3,4,5,3,1], target = 3
Output: 2
Explanation: 3 exists in the array, at index=2 and index=5. Return the minimum index, which is 2.
```

Example 2:

```
Input: mountainArr = [0,1,2,4,2,1], target = 3
Output: -1
Explanation: 3 does not exist in the array, so we return -1.
```

Constraints:

- `3 <= mountainArr.length() <= 104`
- `0 <= target <= 109`
- `0 <= mountainArr.get(index) <= 109`

Community Solutions (Python)

- WalkCC

Round 3 · High · Hard

p.1278

```
# """
# This is MountainArray's API interface.
# You should not implement it, or speculate about its implementation
# """
# Class MountainArray:
# def get(self, index: int) -> int:
# def length(self) -> int:

class Solution:
    def findInMountainArray(
        self,
        target: int,
        mountain_arr: 'MountainArray',
    ) -> int:
        n = mountain_arr.length()
        peakIndex = self.peakIndexInMountainArray(mountain_arr, 0, n - 1)

        leftIndex = self.searchLeft(mountain_arr, target, 0, peakIndex)
        if mountain_arr.get(leftIndex) == target:
            return leftIndex

        rightIndex = self.searchRight(mountain_arr, target, peakIndex + 1, n - 1)
        if mountain_arr.get(rightIndex) == target:
            return rightIndex

        return -1

# 852. Peak Index in a Mountain Array
def peakIndexInMountainArray(self, A: 'MountainArray', l: int, r: int) -> int:
    while l < r:
```

Round 3 · High · Hard

p.1279

```
        m = (l + r) // 2
        if A.get(m) < A.get(m + 1):
            l = m + 1
        else:
            r = m
        return l

    def searchLeft(self, A: 'MountainArray', target: int, l: int, r: int) -> int:
        while l < r:
            m = (l + r) // 2

            if A.get(m) < target:
                l = m + 1
            else:
                r = m
            return l

    def searchRight(self, A: 'MountainArray', target: int, l: int, r: int) -> int:
        while l < r:
            m = (l + r) // 2
            if A.get(m) > target:

                l = m + 1
            else:
                r = m
            return l
```

• LeetDoocs

```
# """
# This is MountainArray's API interface.
# You should not implement it, or speculate about its implementation
# """
# class MountainArray:
# def get(self, index: int) -> int:
# def length(self) -> int:

class Solution:
```

Round 3 · High · Hard

p.1280

```
def findInMountainArray(self, target: int, mountain_arr: 'MountainArray')
-> int:
    def search(l: int, r: int, k: int) -> int:
        while l < r:
            mid = (l + r) >> 1
            if k * mountain_arr.get(mid) >= k * target:
                r = mid
            else:
                l = mid + 1
        return -1 if mountain_arr.get(l) != target else l

    n = mountain_arr.length()
    l, r = 0, n - 1
    while l < r:
        mid = (l + r) >> 1
        if mountain_arr.get(mid) > mountain_arr.get(mid + 1):
            r = mid
        else:
            l = mid + 1
    ans = search(0, l, 1)
    return search(l + 1, n - 1, -1) if ans == -1 else ans
```

• SimplyLeet

```
# Python solution with detailed comments.
class Solution:
    def findInMountainArray(self, target: int, mountain_arr: 'MountainArray')
-> int:
    # Find the peak of the mountain array using binary search.
    n = mountain_arr.length()
    left, right = 0, n - 1
    while left < right:
        mid = (left + right) // 2
        # Compare current value with the next value to decide direction.
        if mountain_arr.get(mid) < mountain_arr.get(mid + 1):
```

Round 3 · High · Hard

p.1281

```
        left = mid + 1
    else:
        right = mid
    peak = left # The index of the peak

    # Binary search on the ascending part of the array.
    def binarySearchAsc(low, high):
        while low <= high:
            mid = (low + high) // 2
            value = mountain_arr.get(mid)

            if value == target:
                return mid
            elif value < target:
                low = mid + 1
            else:
                high = mid - 1
        return -1

    # Binary search on the descending part of the array.
    def binarySearchDesc(low, high):
        while low <= high:
            mid = (low + high) // 2
            value = mountain_arr.get(mid)
            if value == target:
                return mid
            elif value > target:
                low = mid + 1
            else:
                high = mid - 1
        return -1

    # Try searching in the ascending part.
    index = binarySearchAsc(0, peak)
    if index != -1:
        return index
    # Otherwise, search in the descending part.
    return binarySearchDesc(peak + 1, n - 1)
```

Round 3 · High · Hard

p.1282

Stacks

Round 3 · High · Hard · 2 questions

Round 3 · High · Hard

p.1283

Stacks

□ #32 Longest Valid Parentheses

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 String · Dynamic Programming · Stack
 Lists: Grind 169

Problem

Given a string containing just the characters '(' and ')', return the length of the longest valid (well-formed) parentheses substring.

Example 1:

Input: s = "()"
 Output: 2
 Explanation: The longest valid parentheses substring is "()".

Example 2:

Input: s = "()()())"
 Output: 4
 Explanation: The longest valid parentheses substring is "()()".

Example 3:

Input: s = ""
 Output: 0

Constraints:

- $0 \leq s.length \leq 3 \times 10^4$
- $s[i]$ is '(', or ')'.

Round 3 · High · Hard

p.1284

Community Solutions (Python)

• WalkCC

```
class Solution:
    def longestValidParentheses(self, s: str) -> int:
        s2 = ')' + s
        # dp[i] := the length of the longest valid parentheses in the substring
        # s2[1..i]
        dp = [0] * len(s2)

        for i in range(1, len(s2)):
            if s2[i] == ')' and s2[i - dp[i - 1] - 1] == '(':
                dp[i] = dp[i - 1] + dp[i - dp[i - 1] - 2] + 2

        return max(dp)
```

• LeetCode

```
class Solution:
    def longestValidParentheses(self, s: str) -> int:
        n = len(s)
        f = [0] * (n + 1)
        for i, c in enumerate(s, 1):
            if c == ")":
                if i > 1 and s[i - 2] == "(":
                    f[i] = f[i - 2] + 2
                else:
                    j = i - f[i - 1] - 1
                    if j and s[j - 1] == "(":
                        f[i] = f[i - 1] + 2 + f[j - 1]
        return max(f)
```

Round 3 · High · Hard

p.1285

```
class Solution:
    def longestValidParentheses(self, s: str) -> int:
        stack = [-1]
        ans = 0
        for i in range(len(s)):
            if s[i] == '(':
                stack.append(i)
            else:
                stack.pop()
                if not stack:
                    stack.append(i)
                else:
                    ans = max(ans, i - stack[-1])
        return ans
```

• SimplyLeet

```
# Python solution using a stack
def longestValidParentheses(s: str) -> int:
    # Initialize the stack with the base index -1
    stack = [-1]
    max_length = 0

    # Iterate over the string with index
    for i, char in enumerate(s):
        if char == '(':
            # Push the index of '(' onto the stack

            stack.append(i)
        else:
            # Pop the last index for ')'
            stack.pop()
            if not stack:
                # If stack is empty, push current index as the new base
                stack.append(i)
            else:
                # Calculate the length of the current valid substring
                current_length = i - stack[-1]
```

Round 3 · High · Hard

p.1286

```
max_length = max(max_length, current_length)
```

```
return max_length
```

```
# Example usage:
```

```
print(longestValidParentheses("()()")) # Output: 4
```

• LC.ca

```
class Solution:
```

```
def longestValidParentheses(self, s: str) -> int:
```

```
    left = right = 0
```

```
    res = 0
```

```
    for c in s: # from left to right, '()' => will never hit left==right
```

```
        if c == '(':
```

```
            left += 1
```

```
        else:
```

```
            right += 1
```

```
        if left == right:
```

```
            res = max(res, 2 * left)
```

```
        if left < right:
```

```
            left = right = 0
```

```
    left = right = 0 # dont forget to reset
```

```
    for c in reversed(s): # from right to left, '()' => will never hit left==right
```

```
        if c == '(':
```

```
            left += 1
```

```
        else:
```

```
            right += 1
```

```
        if left == right:
```

```
            res = max(res, 2 * left)
```

```
        if left > right: # reverse '<' to '>'
```

```
            left = right = 0
```

```
    return res
```

```
#####
```

```
class Solution:
```

```
def longestValidParentheses(self, s: str) -> int:
```

```
    n = len(s)
```

```
    if n < 2:
```

```
        return 0
```

```
    dp = [0] * n
```

```
    for i in range(1, n):
```

```
        if s[i] == ')':
```

```
            if s[i - 1] == '(':
```

```
                dp[i] = 2 + (dp[i - 2] if i > 1 else 0)
```

```
            else:
```

```
                j = i - dp[i - 1] - 1
```

```
                if j >= 0 and s[j] == '(':
```

```
                    dp[i] = 2 + dp[i - 1] + dp[j - 1]
```

```
    return max(dp)
```

```
#####
```

```
'''
```

```
>>> s="abcdefg"
```

```
>>> [print(i,",",c) for i, c in enumerate(s, 1)]
```

```
1 , a
```

```
2 , b
```

```
3 , c
```

```
4 , d
```

```
5 , e
```

```
6 , f
```

```
7 , g
```

```
[None, None, None, None, None, None, None]
```

```
>>>
```

```
'''
```

```
class Solution:
```

```
def longestValidParentheses(self, s: str) -> int:
```

```
    n = len(s)
```

```
    f = [0] * (n + 1)
```

```
    for i, c in enumerate(s, 1): # starting i from 1, not 0
```

```
        if c == "(":
```

```
            if i > 1 and s[i - 2] == "(":
```

```
                f[i] = f[i - 2] + 2
```

```
            else:
```

```
                j = i - f[i - 1] - 1
```



```

        if j and s[j - 1] == "(":
            f[i] = f[i - 1] + 2 + f[j - 1]
    return max(f)

```

◆ Quick Review

Key Insights

- Use a stack to keep track of indices where potential valid substrings begin.
- Start by pushing a dummy index (-1) to handle edge cases.
- For every '(', push its index onto the stack.
- For every ')', pop from the stack. If the stack becomes empty, push the current index as a new base. Otherwise, compute the length of the valid substring using the current index minus the top index of the stack.
- An alternative is the dynamic programming approach where $dp[i]$ represents the length of the longest valid substring ending at index i .

Space & Time

Time Complexity: $O(n)$ where n is the length of the string.

Space Complexity: $O(n)$ for the stack data structure.

Round 3 · High · Hard

p.1289

Solution

The stack-based solution works by maintaining indices for unmatched parentheses. Initially, a dummy index (-1) is pushed onto the stack to serve as the base for calculating subsequent valid substring lengths. As we iterate through the string:

- If the character is '(', we push its index onto the stack.
- If the character is ')', we pop from the stack. If the stack is empty afterwards, that means there is no valid starting position, so we push the current index to reset the base. Otherwise, we calculate the length of the current valid substring by subtracting the current index with the new top index of the stack and update the maximum length accordingly.

Round 3 · High · Hard

p.1290

Stacks

☐ #224 Basic Calculator

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · String · Stack · Recursion
Lists: Grind 169

Problem

Given a string *s* representing a valid expression, implement a basic calculator to evaluate it, and return the result of the evaluation.

Note: You are not allowed to use any built-in function which evaluates strings as mathematical expressions, such as `eval()`.

Example 1:

Input: *s* = "1 + 1"
Output: 2

Example 2:

Input: *s* = " 2-1 + 2 "
Output: 3

Example 3:

Input: *s* = "(1+(4+5+2)-3)+(6+8)"
Output: 23

Constraints:

Round 3 · High · Hard

p.1291

- $1 \leq s.length \leq 3 \times 10^5$
- *s* consists of digits, '+', '-', '(', ')', and ' '.
- *s* represents a valid expression.
- '+' is not used as a unary operation (i.e., "+1" and "(2 + 3)" is invalid).
- '-' could be used as a unary operation (i.e., "-1" and "-(2 + 3)" is valid).
- There will be no two consecutive operators in the input.
- Every number and running calculation will fit in a signed 32-bit integer.

Community Solutions (Python)

☐ • WalkCC

```
class Solution:
    def calculate(self, s: str) -> int:
        ans = 0
        num = 0
        sign = 1
        stack = [sign] # stack[-1]: the current environment's sign

        for c in s:
            if c.isdigit():
                num = num * 10 + int(c)
```

Round 3 · High · Hard

p.1292

```

elif c == '(':
    stack.append(sign)
elif c == ')':
    stack.pop()
elif c == '+' or c == '-':
    ans += sign * num
    sign = (1 if c == '+' else -1) * stack[-1]
    num = 0

return ans + sign * num

```

• LeetCode

```

class Solution:
    def calculate(self, s: str) -> int:
        stk = []
        ans, sign = 0, 1
        i, n = 0, len(s)
        while i < n:
            if s[i].isdigit():
                x = 0
                j = i
                while j < n and s[j].isdigit():
                    x = x * 10 + int(s[j])
                    j += 1
                ans += sign * x
                i = j - 1
            elif s[i] == "+":
                sign = 1
            elif s[i] == "-":
                sign = -1
            elif s[i] == "(":
                stk.append(ans)

                stk.append(sign)
                ans, sign = 0, 1
            elif s[i] == ")":
                ans = stk.pop() * ans + stk.pop()
                i += 1
        return ans

```

Round 3 · High · Hard

p.1293

• SimplyLeet

```

# Python solution for the Basic Calculator problem

def calculate(s: str) -> int:
    # Stack to store pairs of (current result, current sign) before a '('
    stack = []
    result = 0 # Running total
    number = 0 # Current number being processed
    sign = 1 # Current sign (1 or -1)

    # Iterate through each character in the string

    for char in s:
        if char.isdigit():
            # Build number digit by digit
            number = number * 10 + int(char)
        elif char in "+-":
            # Add the complete number with its sign to result
            result += sign * number
            # Reset number for the next potential number
            number = 0
            # Update the sign based on the operator
            sign = 1 if char == '+' else -1
        elif char == '(':
            # Push the current result and sign onto the stack
            stack.append(result)
            stack.append(sign)
            # Reset result and sign for the new sub-expression
            result = 0
            sign = 1
        elif char == ')':
            # Complete the number before the parentheses close

```

Round 3 · High · Hard

p.1294

```

        result += sign * number
        number = 0
        # The top of the stack is the sign before the parenthesis
        prev_sign = stack.pop()
        # The next value on the stack is the result before the parenthesis
        prev_result = stack.pop()
        # Combine the result of the previous expression with the
sub-expression evaluated
        result = prev_result + prev_sign * result
        # Skip spaces implicitly when no if condition is met

# Add any number left after the loop ends
result += sign * number
return result

```

```

# Example usage:
print(calculate("1 + 1")) # Expected output: 2
print(calculate(" 2-1 + 2 ")) # Expected output: 3
print(calculate("(1+(4+5+2)-3)+(6+8)")) # Expected output: 23

```

• LC.ca

```

# only + and - , no * , no /
class Solution:
    def calculate(self, s: str) -> int:
        stk = []
        ans, sign = 0, 1
        i, n = 0, len(s)
        while i < n:
            if s[i].isdigit():
                x = 0
                j = i

```

Round 3 · High · Hard

p.1295

```

# with this while, no need to do final calculation like below
solution

while j < n and s[j].isdigit():
    x = x * 10 + int(s[j])
    j += 1
    ans += sign * x
    i = j - 1
elif s[i] == "+":
    sign = 1
elif s[i] == "-":
    sign = -1

elif s[i] == "(":
    stk.append(ans)
    stk.append(sign)
    ans, sign = 0, 1
elif s[i] == ")":
    ans = stk.pop() * ans + stk.pop()
    i += 1
return ans

```

```
#####
```

```

class Solution:
    def calculate(self, s: str) -> int:
        ans = 0
        num = 0
        sign = 1
        stack = [sign] # stack[-1]: current env's sign

        for c in s:
            if c.isdigit():

```

Round 3 · High · Hard

p.1296

```

num = num * 10 + int(c)
# num = num * 10 + (ord(c) - ord('0')) => also works
elif c == '(':
    stack.append(sign)
elif c == ')':
    stack.pop() # pop pairing sign for this () pair
elif c == '+' or c == '-':
    ans += sign * num
    sign = (1 if c == '+' else -1) * stack[-1] # after all + or -, the sign
for current num
    num = 0

return ans + sign * num # final calculation

```

◆ Quick Review

Key Insights

- Use a stack to save previous results and signs when encountering parentheses.
- Process the string left-to-right by accumulating multi-digit numbers and applying the current sign.
- When encountering a '(', push the current result and sign onto the stack and reset them.
- When encountering a ')', complete the current sub-expression then pop the previous state to incorporate the computed value.
- Handle whitespaces by simply skipping them.

Round 3 · High · Hard

p.1297

Space & Time

Time Complexity: $O(n)$ where n is the length of the input string, since we process each character once.

Space Complexity: $O(n)$ in the worst-case scenario due to the use of a stack for nested parentheses.

Solution

The solution uses an iterative approach with a stack:

- Initialize variables: result to keep the running total, sign to handle addition/subtraction, and index to iterate over the string.
- As you iterate, build multi-digit numbers by checking for consecutive numerical characters.
- Adjust the total by adding the product of the current number and its sign.
- When encountering a '(', push the current result and sign onto the stack, then reset results and sign for the new sub-expression.
- When encountering a ')', complete the sub-expression by popping the previous result and sign, then combine.
- Skip over spaces.
- The final result is the computed

Round 3 · High · Hard

p.1298

value after the end of the string.

This approach effectively simulates recursion and correctly handles nested expressions and negation.

Round 3 · High · Hard

p.1299

Tree Traversal – Pre Order

Round 3 · High · Hard · 1 question

Round 3 · High · Hard

p.1300

Tree Traversal – Pre Order

☐ #297 Serialize and Deserialize Binary Tree

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Tree · DFS · BFS · Design
Lists: Grind 169

Problem

Serialization is the process of converting a data structure or object into a sequence of bits so that it can be stored in a file or memory buffer, or transmitted across a network connection link to be reconstructed later in the same or another computer environment.

Design an algorithm to serialize and deserialize a binary tree. There is no restriction on how your serialization/deserialization algorithm should work. You just need to ensure that a binary tree can be serialized to a string and this string can be deserialized to the original tree structure.

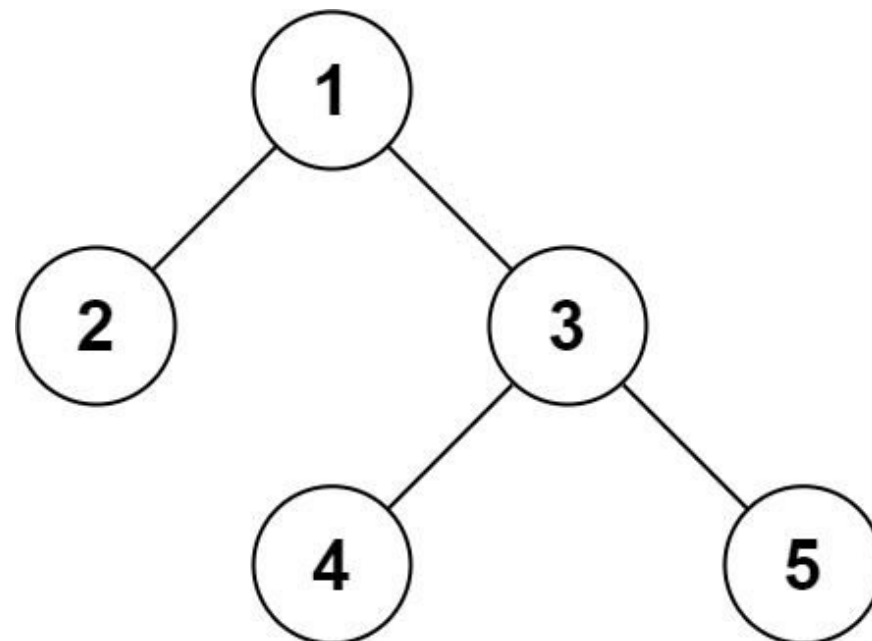
Clarification: The input/output format is the same as how LeetCode serializes a binary tree.

Round 3 · High · Hard

p.1301

You do not necessarily need to follow this format, so please be creative and come up with different approaches yourself.

Example 1:



Input: root = [1,2,3,null,null,4,5]
Output: [1,2,3,null,null,4,5]

Example 2:

Input: root = []
Output: []

Constraints:

Round 3 · High · Hard

p.1302

- The number of nodes in the tree is in the range [0, 104].
- $-1000 \leq \text{Node.val} \leq 1000$

Community Solutions (Python)

• WalkCC

```
class Codec:
    def serialize(self, root: 'TreeNode') -> str:
        """Encodes a tree to a single string."""
        if not root:
            return ''

        s = ''
        q = collections.deque([root])

        while q:

            node = q.popleft()
            if node:
                s += str(node.val) + ' '
                q.append(node.left)
                q.append(node.right)
            else:
                s += 'n '

        return s

    def deserialize(self, data: str) -> 'TreeNode':
        """Decodes your encoded data to tree."""
        if not data:
            return None

        vals = data.split()
        root = TreeNode(vals[0])
        q = collections.deque([root])

        for i in range(1, len(vals), 2):
```

```
node = q.popleft()
if vals[i] != 'n':
    node.left = TreeNode(vals[i])
    q.append(node.left)
if vals[i + 1] != 'n':
    node.right = TreeNode(vals[i + 1])
    q.append(node.right)

return root
```

• LeetCode

```
# Definition for a binary tree node.
# class TreeNode(object):
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None

class Codec:
    def serialize(self, root):

        """Encodes a tree to a single string.

        :type root: TreeNode
        :rtype: str
        """

        if root is None:
            return ""
        q = deque([root])
        ans = []
        while q:
```



```

        node = q.popleft()
        if node:
            ans.append(str(node.val))
            q.append(node.left)
            q.append(node.right)
        else:
            ans.append("#")
    return ",".join(ans)

```

```

def deserialize(self, data):

```

```

    """Decodes your encoded data to tree.

```

```

:type data: str
:rtype: TreeNode
    """

```

```

    if not data:
        return None
    vals = data.split(",")
    root = TreeNode(int(vals[0]))
    q = deque([root])

```

```

    i = 1
    while q:
        node = q.popleft()
        if vals[i] != "#":
            node.left = TreeNode(int(vals[i]))
            q.append(node.left)
        i += 1
        if vals[i] != "#":
            node.right = TreeNode(int(vals[i]))
            q.append(node.right)

```

```

        i += 1
    return root

```

```

# Your Codec object will be instantiated and called as such:
# codec = Codec()
# codec.deserialize(codec.serialize(root))

```

• SimplyLeet

Round 3 · High · Hard

p.1305

```

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val # value of the node
        self.left = left # left child
        self.right = right # right child

```

```

class Codec:
    # Encodes a tree to a single string using BFS.
    def serialize(self, root):

```

```

        if not root:
            return "" # Return empty string for an empty tree
        result = []
        queue = [root] # Initialize the queue with the root node
        while queue:
            node = queue.pop(0)
            if node:
                result.append(str(node.val)) # Append the value of the node
                queue.append(node.left) # Enqueue left child
                queue.append(node.right) # Enqueue right child

```

```

        else:
            result.append("null") # Placeholder for null node
        return ",".join(result) # Join list with commas to form the string

```

```

# Decodes the encoded data to reconstruct the tree using BFS.

```

```

def deserialize(self, data):
    if not data:
        return None # Return None for empty string
    nodes = data.split(",")
    root = TreeNode(int(nodes[0])) # Create root node from first element

```

```

    queue = [root]
    i = 1 # Index for traversing nodes list
    while queue:
        current = queue.pop(0)
        if i < len(nodes) and nodes[i] != "null":
            current.left = TreeNode(int(nodes[i]))
            queue.append(current.left)
        i += 1
        if i < len(nodes) and nodes[i] != "null":
            current.right = TreeNode(int(nodes[i]))

```

Round 3 · High · Hard

p.1306

```

        queue.append(current.right)
    i += 1
    return root

```

• LC.ca

```

# Definition for a binary tree node.
# class TreeNode(object):
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None

```

```

class Codec:
    def serialize(self, root):

```

```

        """Encodes a tree to a single string.

```

```

        :type root: TreeNode
        :rtype: str
        """

```

```

        if root is None:
            return ''
        res = []

```

```

        def preorder(root):
            if root is None:
                res.append("#,")
                return
            res.append(str(root.val) + ",")
            preorder(root.left)
            preorder(root.right)

```

```

        preorder(root)
        return ''.join(res)

```

Round 3 · High · Hard

p.1307

```

def deserialize(self, data):
    """Decodes your encoded data to tree.

```

```

    :type data: str
    :rtype: TreeNode
    """

```

```

    if not data:
        return None
    vals = data.split(',')

```

```

    def inner():
        first = vals.pop(0)
        if first == '#':
            return None
        return TreeNode(int(first), inner(), inner())
        # seems using a constructor __init__(val, left, right)

```

```

    return inner()

```

```

# Your Codec object will be instantiated and called as such:
# ser = Codec()
# deser = Codec()
# ans = deser.deserialize(ser.serialize(root))

```

```

#####

```

```

from collections import deque

```

```

# bfs, each level based
class Codec:
    def serialize(self, root):
        """Encodes a tree to a single string.

```

```

        :type root: TreeNode
        :rtype: str
        """

```

```

        ret = []
        queue = deque([root])

```

Round 3 · High · Hard

p.1308

```

while queue:
    top = queue.popleft()
    if not top:
        ret.append("None")
        continue
    else:
        ret.append(str(top.val))
        queue.append(top.left)
        queue.append(top.right)
return ",".join(ret)

def deserialize(self, data):
    """Decodes your encoded data to tree.

    :type data: str
    :rtype: TreeNode
    """
    data = data.split(",")
    if data[0] == "None":
        return None

    root = TreeNode(int(data[0]))
    queue = deque([root])
    i = 0
    while queue and i < len(data):
        top = queue.popleft()
        i += 1
        left = right = None
        if i < len(data) and data[i] != "None":
            left = TreeNode(int(data[i]))
            queue.append(left)

        i += 1
        if i < len(data) and data[i] != "None":
            right = TreeNode(int(data[i]))
            queue.append(right)

        top.left = left
        top.right = right

    return root

```

Round 3 · High · Hard

p.1309

```

# Your Codec object will be instantiated and called as such:
# codec = Codec()
# codec.deserialize(codec.serialize(root))

```

◆ Quick Review

Key Insights

- The tree can be traversed using either depth-first search (DFS) or breadth-first search (BFS).
- Using BFS (level-order traversal) simplifies reconstruction since nodes are processed level-by-level.
- Placeholder markers (e.g., "null") are used to indicate missing children.
- Both serialization and deserialization processes should handle empty trees and edge cases gracefully.
- Ensuring a one-to-one mapping between the tree structure and the serialized string is critical.

Space & Time

Time Complexity: $O(n)$ for both serialization and deserialization, where n is the number of nodes in the tree.

Round 3 · High · Hard

p.1310

Space Complexity: $O(n)$ due to the storage used for the serialized string and additional data structures (queue) during processing.

Solution

This solution uses level-order traversal (BFS) for both serialization and deserialization. During serialization, each node is processed sequentially with missing children recorded as "null". For deserialization, the string is split by commas, and a queue is used to rebuild the tree level by level. The root is created from the first token, then subsequent tokens are assigned as left and right children accordingly, ensuring an accurate reconstruction of the original tree.

Round 3 · High · Hard

p.1311

Tree Traversal – Post-Order

Round 3 · High · Hard · 1 question

Round 3 · High · Hard

p.1312

Tree Traversal – Post-Order

#124 Binary Tree Maximum Path Sum

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Dynamic Programming · Tree · DFS
 Lists: Grind 169

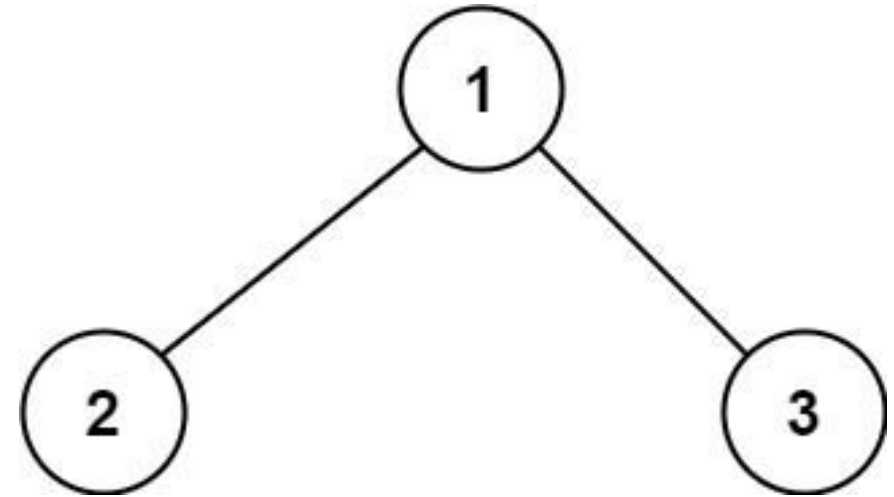
Problem

A path in a binary tree is a sequence of nodes where each pair of adjacent nodes in the sequence has an edge connecting them. A node can only appear in the sequence at most once. Note that the path does not need to pass through the root.

The path sum of a path is the sum of the node's values in the path.

Given the root of a binary tree, return the maximum path sum of any non-empty path.

Example 1:



Input: root = [1,2,3]

Output: 6

Explanation: The optimal path is 2 -> 1 -> 3 with a path sum of 2 + 1 + 3 = 6.

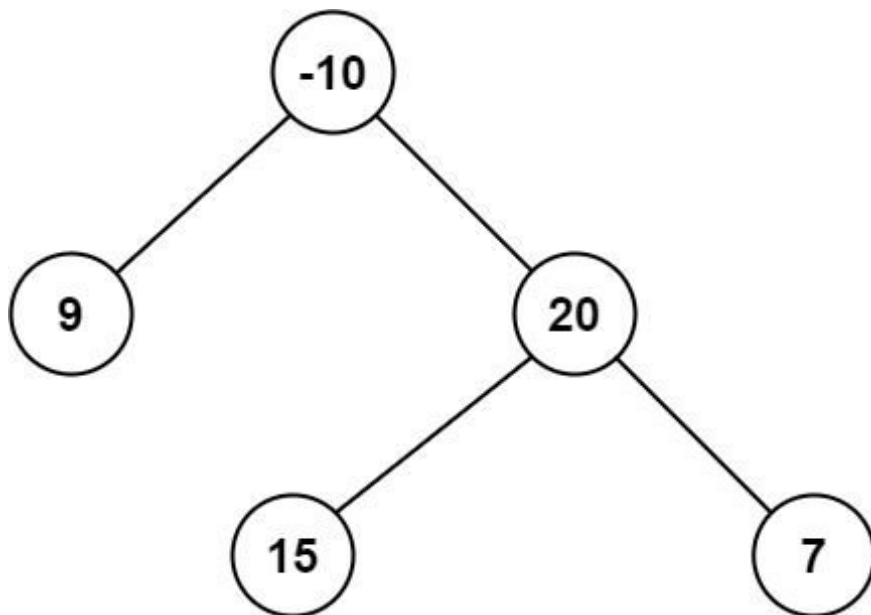
Example 2:

Round 3 · High · Hard

p.1313

Round 3 · High · Hard

p.1314



Input: root = [-10,9,20,null,null,15,7]

Output: 42

Explanation: The optimal path is 15 → 20 → 7 with a path sum of 15 + 20 + 7 = 42.

Constraints:

- The number of nodes in the tree is in the range [1, 3 * 10⁴].
- -1000 ≤ Node.val ≤ 1000

Community Solutions (Python)

- WalkCC

Round 3 · High · Hard

p.1315

```

class Solution:
    def maxPathSum(self, root: TreeNode | None) -> int:
        ans = -math.inf

        def maxPathSumDownFrom(root: TreeNode | None) -> int:
            """
            Returns the maximum path sum starting from the current root, where
            root.val is always included.
            """
            nonlocal ans

            if not root:
                return 0

            l = max(0, maxPathSumDownFrom(root.left))
            r = max(0, maxPathSumDownFrom(root.right))
            ans = max(ans, root.val + l + r)
            return root.val + max(l, r)

        maxPathSumDownFrom(root)
        return ans
  
```

• LeetDoocs

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def maxPathSum(self, root: Optional[TreeNode]) -> int:
        def dfs(root: Optional[TreeNode]) -> int:
            if root is None:
  
```

Round 3 · High · Hard

p.1316

- Use Depth-First Search (DFS) to explore the tree in a post-order fashion.
- For each node, calculate the maximum gain including that node and optionally one of its subtrees.
- A node's best contribution to its parent can be either just its own value or its value plus the maximum gain from either the left or right child.
- Update a global maximum path sum that considers the possibility of combining both left and right gains with the current node.
- Handle negative values by comparing gains against 0 to avoid decreasing the path sum.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes in the tree, since each node is visited exactly once.

Space Complexity: $O(n)$ in the worst-case scenario due to the recursion stack ($O(H)$ where H is the height of the tree, which can be $O(n)$ for skewed trees).

Solution

Round 3 · High · Hard

p.1319

We use a recursive DFS approach (post-order traversal) to compute the maximum gain from each subtree. For every node:

- Recursively compute the maximum gain from the left and right child, ensuring that negative gains are treated as 0.
- The maximum gain that can be provided to the node's parent is the node's value plus the larger gain from its left or right child.
- Simultaneously, the potential maximum path sum including the current node as the highest (or root) node is the node's value plus both left and right gains.
- Update a global variable that holds the maximum path sum found so far.
- Return the maximum gain (node value plus one best child) to be used by the node's parent.

This strategy ensures that every possible path sum is considered while avoiding double-counting nodes.

Round 3 · High · Hard

p.1320

Depth First Search (DFS)

Round 3 · High · Hard · 3 questions

Round 3 · High · Hard

p.1321

Depth First Search (DFS)

□ #827 Making A Large Island

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Depth-First Search · Breadth-First
Search · Union-Find · Matrix
Lists: AlgoMaster 600

Problem

You are given an $n \times n$ binary matrix grid. You are allowed to change at most one 0 to be 1. Return the size of the largest island in grid after applying this operation.

An island is a 4-directionally connected group of 1s.

Example 1:

Input: grid = [[1,0],[0,1]]
Output: 3
Explanation: Change one 0 to 1 and connect two 1s, then we get an island with area = 3.

Example 2:

Input: grid = [[1,1],[1,0]]
Output: 4
Explanation: Change the 0 to 1 and make the island bigger, only one island with area = 4.

Example 3:

Round 3 · High · Hard

p.1322

Input: grid = [[1,1],[1,1]]

Output: 4

Explanation: Can't change any 0 to 1, only one island with area = 4.

Constraints:

- $n == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq n \leq 500$
- $\text{grid}[i][j]$ is either 0 or 1.

Community Solutions (Python)

• LeetCode

```
class Solution:
    def largestIsland(self, grid: List[List[int]]) -> int:
        def dfs(i: int, j: int):
            p[i][j] = root
            cnt[root] += 1
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                if 0 <= x < n and 0 <= y < n and grid[x][y] and p[x][y] == 0:
                    dfs(x, y)
```

```
n = len(grid)
cnt = Counter()
p = [[0] * n for _ in range(n)]
dirs = (-1, 0, 1, 0, -1)
root = 0
for i, row in enumerate(grid):
    for j, x in enumerate(row):
        if x and p[i][j] == 0:
            root += 1
            dfs(i, j)
```

Round 3 · High · Hard

p.1323

```
ans = max(cnt.values() or [0])
for i, row in enumerate(grid):
    for j, x in enumerate(row):
        if x == 0:
            s = set()
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                if 0 <= x < n and 0 <= y < n:
                    s.add(p[x][y])
            ans = max(ans, sum(cnt[root] for root in s) + 1)

return ans
```

• SimplyLeet

Python solution for Making A Large Island

class Solution:

```
def largestIsland(self, grid: List[List[int]]) -> int:
    n = len(grid)
    island_size = {} # Map island id to island area
    island_id = 2 # Start island ids from 2
```

Helper function: DFS to label the islands

```
def dfs(i, j, id):
    if i < 0 or i >= n or j < 0 or j >= n or grid[i][j] != 1:
```

```
        return 0
    grid[i][j] = id # Label the cell with the island id
    size = 1
    # Visit neighbors (4-directionally)
    size += dfs(i + 1, j, id)
    size += dfs(i - 1, j, id)
    size += dfs(i, j + 1, id)
    size += dfs(i, j - 1, id)
    return size
```

Round 3 · High · Hard

p.1324

```

# First pass: identify each island and record its size.
for i in range(n):
    for j in range(n):
        if grid[i][j] == 1: # Found an unlabeled island cell
            size = dfs(i, j, island_id)
            island_size[island_id] = size
            island_id += 1

# If no 0 cell exists, then the entire grid is already one island.
max_area = max(island_size.values()) if island_size else 0

# Second pass: Try changing each 0 to 1 and calculate the connected
island area.
for i in range(n):
    for j in range(n):
        if grid[i][j] == 0:
            seen_islands = set()
            current_area = 1 # Count the flip (0 -> 1)
            for di, dj in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
                ni, nj = i + di, j + dj
                if 0 <= ni < n and 0 <= nj < n:
                    id = grid[ni][nj]
                    if id > 1 and id not in seen_islands:
                        seen_islands.add(id)
                        current_area += island_size[id]
            max_area = max(max_area, current_area)

return max_area

```

Round 3 · High · Hard

p.1325

Depth First Search (DFS)

#1192 Critical Connections in a Network **HAR**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 DFS · Graph · Biconnected Component
 Lists: Denny Zhang

Problem

There are n servers numbered from 0 to $n - 1$ connected by undirected server-to-server connections forming a network where $\text{connections}[i] = [a_i, b_i]$ represents a connection between servers a_i and b_i . Any server can reach other servers directly or indirectly through the network.

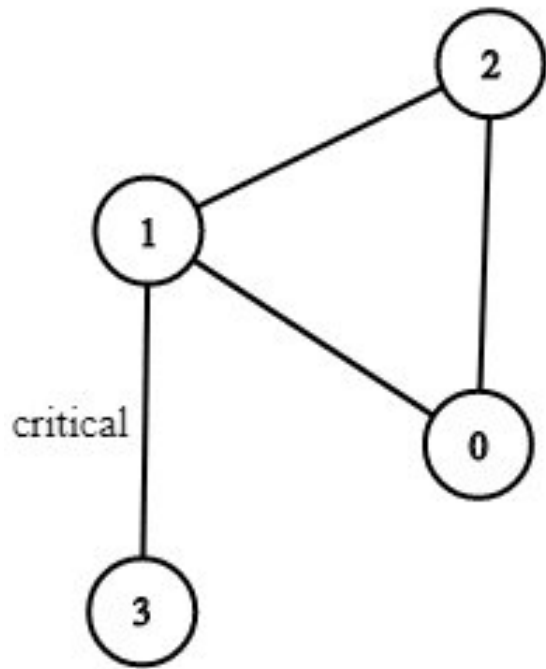
A critical connection is a connection that, if removed, will make some servers unable to reach some other server.

Return all critical connections in the network in any order.

Example 1:

Round 3 · High · Hard

p.1326



Input: $n = 4$, $\text{connections} = [[0,1],[1,2],[2,0],[1,3]]$
 Output: $[[1,3]]$
 Explanation: $[[3,1]]$ is also accepted.

Example 2:

Input: $n = 2$, $\text{connections} = [[0,1]]$
 Output: $[[0,1]]$

Constraints:

- $2 \leq n \leq 105$
- $n - 1 \leq \text{connections.length} \leq 105$
- $0 \leq a_i, b_i \leq n - 1$

Round 3 · High · Hard

p.1327

- $a_i \neq b_i$
- There are no repeated connections.

Community Solutions (Python)

• LeetCode

```

class Solution:
    def criticalConnections(
        self, n: int, connections: List[List[int]]
    ) -> List[List[int]]:
        def tarjan(a: int, fa: int):
            nonlocal now
            now += 1
            dfn[a] = low[a] = now
            for b in g[a]:
                if b == fa:
                    continue
                if not dfn[b]:
                    tarjan(b, a)
                    low[a] = min(low[a], low[b])
                    if low[b] > dfn[a]:
                        ans.append([a, b])
                else:
                    low[a] = min(low[a], dfn[b])

            g = [[] for _ in range(n)]

            for a, b in connections:
                g[a].append(b)
                g[b].append(a)

            dfn = [0] * n
            low = [0] * n
            now = 0
            ans = []
            tarjan(0, -1)
            return ans
  
```

• SimplyLeet

Round 3 · High · Hard

p.1328

```
def criticalConnections(n, connections):
    # Build the adjacency list for the graph
    graph = [[] for _ in range(n)]
    for u, v in connections:
        graph[u].append(v)
        graph[v].append(u)

    # Initialize discovery time and low arrays
    disc = [-1] * n # Discovery times, -1 indicates unvisited
    low = [0] * n   # Low values for each node

    time = 0        # Global timer
    bridges = []     # List to store critical connections (bridges)

    def dfs(u, parent):
        nonlocal time
        disc[u] = low[u] = time
        time += 1

        # Traverse all adjacent nodes
        for v in graph[u]:
            if disc[v] == -1: # If v is not visited
                dfs(v, u)
                low[u] = min(low[u], low[v])
                # If the smallest reachable vertex from v is after u's
                # discovery, edge (u,v) is a bridge
                if low[v] > disc[u]:
                    bridges.append([u, v])
            elif v != parent: # v is visited and is not the parent (back edge)
                low[u] = min(low[u], disc[v])

    # Start DFS from node 0
    dfs(0, -1)

    return bridges

# Example usage:
n = 4
connections = [[0,1],[1,2],[2,0],[1,3]]
print(criticalConnections(n, connections)) # Output: [[1, 3]]
```

 • LC.ca

Round 3 · High · Hard

p.1329

```
class Solution:
    def criticalConnections(
        self, n: int, connections: List[List[int]]
    ) -> List[List[int]]:
        def tarjan(a: int, fa: int):
            nonlocal now
            now += 1
            dfn[a] = low[a] = now
            for b in g[a]:
                if b == fa:
                    continue
                if not dfn[b]:
                    tarjan(b, a)
                    low[a] = min(low[a], low[b])
                    if low[b] > dfn[a]:
                        ans.append([a, b])
                else:
                    low[a] = min(low[a], dfn[b])

        g = [[] for _ in range(n)]

        for a, b in connections:
            g[a].append(b)
            g[b].append(a)

        dfn = [0] * n
        low = [0] * n
        now = 0
        ans = []
        tarjan(0, -1)
        return ans
```

◆ Quick Review

Key Insights

- The problem is about identifying bridges in an undirected graph.

Round 3 · High · Hard

p.1330

- Use Depth-First Search (DFS) to explore the graph.
- During DFS, track discovery times and low-link values for each node.
- For an edge (u, v) , if $\text{low}[v] > \text{disc}[u]$ then (u, v) is a critical connection.
- Tarjan's algorithm is well-suited for solving this problem in $O(n + m)$ time.

Space & Time

Time Complexity: $O(n + m)$ where n is the number of nodes and m is the number of connections.

Space Complexity: $O(n + m)$ for the graph representation and DFS recursion stack.

Solution

We solve this problem using a DFS-based approach (Tarjan's algorithm). First, we transform the list of connections into an adjacency list for efficient graph traversal. We then start a DFS from an arbitrary node (node 0, since the graph is connected) while maintaining two arrays: one for discovery times (disc) and one for low-link values (low). The discovery time is the time when the node

is first visited, and the low-link value is the smallest discovery time that can be reached from that node (including via back edges). For each edge (u, v) , after a DFS call on v , if $\text{low}[v] > \text{disc}[u]$ then the edge (u, v) is a bridge because v (and its descendants) cannot reach u or any of its ancestors without this edge. This edge is then recorded as a critical connection.

Depth First Search (DFS)

#2246 Longest Path With Different Adjacent Characters

HAR

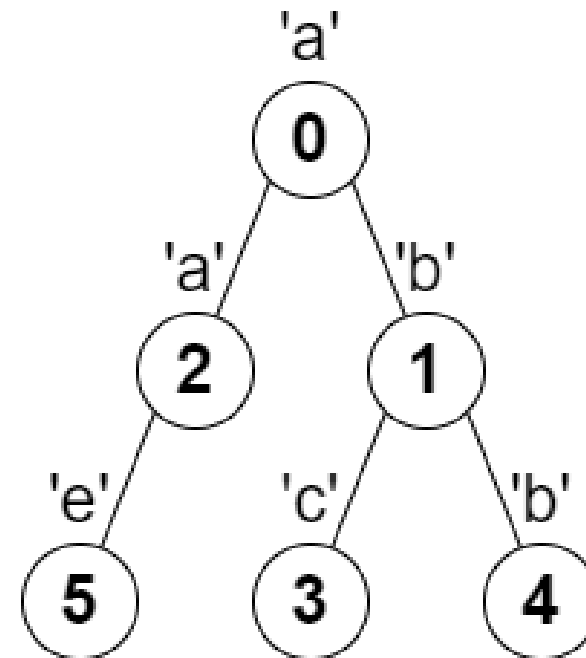
LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · String · Tree · Depth-First Search ·
 Graph Theory · Topological Sort
 Lists: AlgoMaster 600

Problem

You are given a tree (i.e. a connected, undirected graph that has no cycles) rooted at node 0 consisting of n nodes numbered from 0 to $n - 1$. The tree is represented by a 0-indexed array `parent` of size n , where `parent[i]` is the parent of node i . Since node 0 is the root, `parent[0] == -1`. You are also given a string `s` of length n , where `s[i]` is the character assigned to node i .

Return the length of the longest path in the tree such that no pair of adjacent nodes on the path have the same character assigned to them.

Example 1:



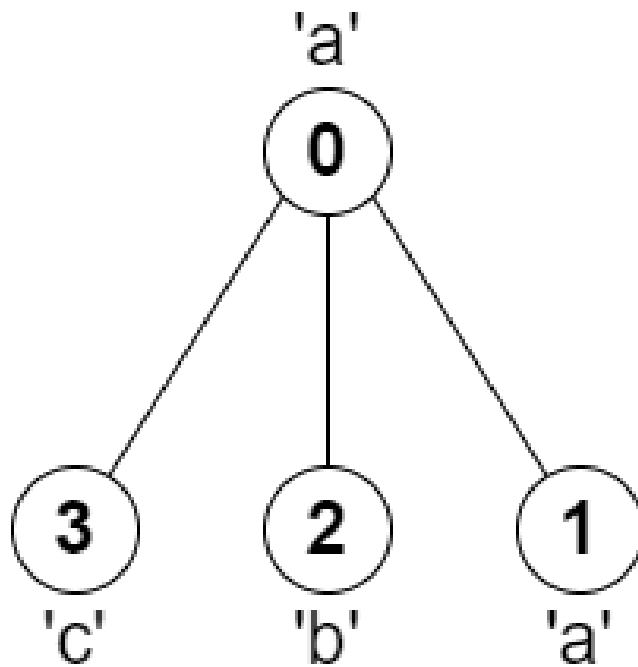
Input: `parent = [-1,0,0,1,1,2]`, `s = "abacbe"`

Output: 3

Explanation: The longest path where each two adjacent nodes have different characters in the tree is the path: 0 → 1 → 3. The length of this path is 3, so 3 is returned.

It can be proven that there is no longer path that satisfies the conditions.

Example 2:



Input: parent = [-1,0,0,0], s = "aabc"

Output: 3

Explanation: The longest path where each two adjacent nodes have different characters is the path: 2 → 0 → 3. The length of this path is 3, so 3 is returned.

Constraints:

- $n == \text{parent.length} == \text{s.length}$
- $1 \leq n \leq 105$
- $0 \leq \text{parent}[i] \leq n - 1$ for all $i \geq 1$
- $\text{parent}[0] == -1$
- parent represents a valid tree.

Round 3 · High · Hard

p.1335

- s consists of only lowercase English letters.

Community Solutions (Python)

• WalkCC

```

class Solution:
    def longestPath(self, parent: list[int], s: str) -> int:
        n = len(parent)
        ans = 0
        graph = [[] for _ in range(n)]

        for i in range(1, n):
            graph[parent[i]].append(i)

        def longestPathDownFrom(u: int) -> int:

            nonlocal ans
            max1 = 0
            max2 = 0

            for v in graph[u]:
                res = longestPathDownFrom(v)
                if s[u] == s[v]:
                    continue
                if res > max1:
                    max2 = max1
                    max1 = res
                elif res > max2:
                    max2 = res

            ans = max(ans, 1 + max1 + max2)
            return 1 + max1

        longestPathDownFrom(0)
        return ans
  
```

• LeetDoocs

Round 3 · High · Hard

p.1336


```
class Solution:
    def longestPath(self, parent: List[int], s: str) -> int:
        def dfs(i: int) -> int:
            mx = 0
            nonlocal ans
            for j in g[i]:
                x = dfs(j) + 1
                if s[i] != s[j]:
                    ans = max(ans, mx + x)
                    mx = max(mx, x)

            return mx

        g = defaultdict(list)
        for i in range(1, len(parent)):
            g[parent[i]].append(i)
        ans = 0
        dfs(0)
        return ans + 1
```

• SimplyLeet

```
# Python solution for "Longest Path With Different Adjacent Characters"

def longestPath(parent, s):
    from collections import defaultdict

    # Build the adjacency list for the tree
    tree = defaultdict(list)
    n = len(parent)
    for node in range(1, n):
        tree[parent[node]].append(node)
```

Round 3 · High · Hard

p.1337

```
# Global variable to store the longest valid path length
max_path = 0

# DFS function to compute the longest valid branch starting from the
current node
def dfs(node):
    nonlocal max_path
    # The two longest valid branch lengths from the children (without
current node)
    longest, second_longest = 0, 0

    # Explore each child
    for child in tree[node]:
        branch_length = dfs(child) # Compute branch length for child
        subtree

        # Only extend the branch if the character at child is different
        if s[child] != s[node]:
            if branch_length > longest:
                second_longest = longest
                longest = branch_length
            elif branch_length > second_longest:
                second_longest = branch_length

    # Update the global maximum with the sum of the two best branches from
the node plus itself
    max_path = max(max_path, longest + second_longest + 1)
    # Return the length of the longest branch including the current node
    return longest + 1

dfs(0)
return max_path

# Example usage:

# parent = [-1,0,0,1,1,2]
# s = "abacbe"
# print(longestPath(parent, s)) # Output: 3
```

Round 3 · High · Hard

p.1338

Breadth First Search (BFS)

Round 3 · High · Hard · 4 questions

Round 3 · High · Hard

p.1339

Breadth First Search (BFS)

□ #127 Word Ladder

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · BFS
Lists: Grind 169

Problem

A transformation sequence from word `beginWord` to word `endWord` using a dictionary `wordList` is a sequence of words `beginWord` -> `s1` -> `s2` -> ... -> `sk` such that:

- Every adjacent pair of words differs by a single letter.
- Every `si` for $1 \leq i \leq k$ is in `wordList`. Note that `beginWord` does not need to be in `wordList`.
- `sk == endWord`

Given two words, `beginWord` and `endWord`, and a dictionary `wordList`, return the number of words in the shortest transformation sequence from `beginWord` to `endWord`, or 0 if no such sequence exists.

Round 3 · High · Hard

p.1340

Example 1:

Input: beginWord = "hit", endWord = "cog", wordList = ["hot","dot","dog","lot","log","cog"]
 Output: 5
 Explanation: One shortest transformation sequence is "hit" -> "hot" -> "dot" -> "dog" -> "cog", which is 5 words long.

Example 2:

Input: beginWord = "hit", endWord = "cog", wordList = ["hot","dot","dog","lot","log"]
 Output: 0
 Explanation: The endWord "cog" is not in wordList, therefore there is no valid transformation sequence.

Constraints:

- $1 \leq \text{beginWord.length} \leq 10$
- $\text{endWord.length} == \text{beginWord.length}$
- $1 \leq \text{wordList.length} \leq 5000$
- $\text{wordList}[i].\text{length} == \text{beginWord.length}$
- beginWord, endWord, and wordList[i] consist of lowercase English letters.
- $\text{beginWord} \neq \text{endWord}$
- All the words in wordList are unique.

Community Solutions (Python)

- WalkCC

Round 3 · High · Hard

p.1341

```
class Solution:
    def ladderLength(
        self,
        beginWord: str,
        endWord: str,
        wordList: list[str],
    ) -> int:
        wordSet = set(wordList)
        if endWord not in wordSet:
            return 0

        q = collections.deque([beginWord])

        step = 1
        while q:
            for _ in range(len(q)):
                wordList = list(q.popleft())
                for i, cache in enumerate(wordList):
                    for c in string.ascii_lowercase:
                        wordList[i] = c

                word = ''.join(wordList)
                if word == endWord:
                    return step + 1
                if word in wordSet:
                    q.append(word)
                    wordSet.remove(word)
                wordList[i] = cache
            step += 1

        return 0
```

- LeetDoocs

Round 3 · High · Hard

p.1342

```

class Solution:
    def ladderLength(self, beginWord: str, endWord: str, wordList: List[str])
-> int:
    words = set(wordList)
    q = deque([beginWord])
    ans = 1
    while q:
        ans += 1
        for _ in range(len(q)):
            s = q.popleft()
            s = list(s)

            for i in range(len(s)):
                ch = s[i]
                for j in range(26):
                    s[i] = chr(ord('a') + j)
                    t = ''.join(s)
                    if t not in words:
                        continue
                    if t == endWord:
                        return ans
                    q.append(t)

            words.remove(t)
            s[i] = ch

    return 0

```

• SimplyLeet

```

# Python solution using BFS

from collections import defaultdict, deque

def ladderLength(beginWord, endWord, wordList):
    # Create a set for fast lookup
    wordSet = set(wordList)
    if endWord not in wordSet:
        return 0

```

Round 3 · High · Hard

p.1343

```

# Preprocessing: build a dictionary of intermediate states to words
L = len(beginWord)
all_combo_dict = defaultdict(list)
for word in wordSet:
    for i in range(L):
        # Key with a wildcard at the i-th position
        intermediate = word[:i] + '*' + word[i+1:]
        all_combo_dict[intermediate].append(word)

# Initialize BFS

queue = deque([(beginWord, 1)])
visited = set([beginWord])

while queue:
    current_word, level = queue.popleft()
    for i in range(L):
        intermediate = current_word[:i] + '*' + current_word[i+1:]
        # Check all words sharing the same intermediate state
        for word in all_combo_dict[intermediate]:
            if word == endWord:
                return level + 1
            if word not in visited:
                visited.add(word)
                queue.append((word, level + 1))

# Once processed, clear the intermediate state list to save
processing time
all_combo_dict[intermediate] = []

return 0

# Example usage:
# print(ladderLength("hit", "cog", ["hot","dot","dog","lot","log","cog"]))

```

• LC.ca

Round 3 · High · Hard

p.1344

```

# native BFS
class Solution:
    def ladderLength(self, beginWord: str, endWord: str, wordList: List[str])
-> int:
    words = set(wordList)
    q = deque([beginWord])
    ans = 1
    while q:
        ans += 1
        for _ in range(len(q)):
            s = q.popleft()

            s = list(s) # must convert to list, cannot directly update
            s[i]='a'

            for i in range(len(s)):
                ch = s[i]
                for j in range(26):
                    # will re-generate s itself
                    # but s not in words-set, s removed after added to
                    words-set

                    s[i] = chr(ord('a') + j)
                    t = ''.join(s)
                    if t not in words:
                        continue

            if t == endWord:
                return ans
            q.append(t)
            words.remove(t) # equivalent to set t as visited
            s[i] = ch # restore

    return 0

#####

```

Round 3 · High · Hard

p.1345

```

'''
BFS BFS
'''

1. q1, q2
2. m1, m2
3.
4.

class Solution:
    def ladderLength(self, beginWord: str, endWord: str, wordList: List[str])
-> int:

    def extend(m1, m2, q):
        for _ in range(len(q)):
            s = q.popleft() # so, every time, starting from start-word...
            and later if t in m1 will skip repeated part...
            step = m1[s]
            s = list(s) # s = "abc", list(s) ==> ['a', 'b', 'c']
            for i in range(len(s)):
                ch = s[i]
                for j in range(26):
                    s[i] = chr(ord('a') + j)
                    t = ''.join(s)

                    if t in m1 or t not in words:
                        continue
                    if t in m2:
                        return step + 1 + m2[t]
                    m1[t] = step + 1
                    q.append(t)
                    s[i] = ch

            return -1

    words = set(wordList)

```

Round 3 · High · Hard

p.1346

```

    if endWord not in words:
        return 0
    q1, q2 = deque([beginWord]), deque([endWord])
    m1, m2 = {beginWord: 0}, {endWord: 0}
    while q1 and q2:
        t = extend(m1, m2, q1) if len(q1) <= len(q2) else extend(m2, m1, q
2)

        if t != -1:
            return t + 1
    return 0

```

```
#####
```

```

import string
from collections import deque

```

```

class Solution(object):
    def ladderLength(self, beginWord, endWord, wordList):
        """
        :type beginWord: str

```

```

        :type endWord: str
        :type wordList: Set[str]
        :rtype: int
        """

```

```

    def getNbrs(src, dest, wordList):
        res = []
        for c in string.ascii_lowercase:
            for i in range(0, len(src)):
                newWord = src[:i] + c + src[i + 1:]

```

Round 3 · High · Hard

p.1347

```

    if newWord == src:
        continue
    if newWord in wordList or newWord == dest:
        # about yield https://stackoverflow.com/questions/231767/what-does
-the-yield-keyword-do-in-python
        # yield is a keyword that is used like return, except the function
will return a generator.
        yield newWord

```

```

queue = deque([beginWord])
length = 0
while queue:

```

```

    length += 1
    for k in range(0, len(queue)):
        top = queue.popleft()
        for nbr in getNbrs(top, endWord, wordList):
            wordList.remove(nbr)
            if nbr == endWord:
                return length + 1
            queue.append(nbr)
    return 0

```

```
#####
```

```

while q1 and q2:
    if len(q1) <= len(q2):
        # Prioritize the queue with fewer elements for expansion
        extend(m1, m2, q1)
    else:
        extend(m2, m1, q2)

```

Round 3 · High · Hard

p.1348

```
def extend(m1, m2, q):
    # New round of expansion
    for _ in range(len(q)):
        p = q.popleft()
        step = m1[p]
        for t in next(p):
            if t in m1:
                # Already visited before
                continue
            if t in m2:
                # The other direction has been searched, indicating that a
                # shortest path has been found
                return step + 1 + m2[t]
        q.append(t)
        m1[t] = step + 1
```

◆ Quick Review

Key Insights

- Use Breadth-First Search (BFS) to explore transformations level by level to guarantee the shortest path.
- Preprocess words by generating intermediate representations (e.g., replacing each letter with a wildcard) to quickly find neighbors.
- Use a visited set to avoid cycles.
- Each transformation counts as a level in the BFS, so the answer is the number of levels traversed.

Round 3 · High · Hard

p.1349

Space & Time

Time Complexity: $O(N * M^2)$ where N is the number of words and M is the length of each word.

Space Complexity: $O(N * M)$ for the storage of intermediate mappings and the BFS queue.

Solution

We solve the problem using a BFS approach. First, we preprocess the wordList by creating a mapping of all possible intermediate states. For example, for the word "dog" and wildcard position, we generate "og", "d g", and "do*". This helps us find all adjacent words (neighbors) that are only one letter different in constant time. We then initialize a queue with the beginWord and initiate the BFS. For each word dequeued, we explore all valid transformations using the precomputed intermediate mapping. If the endWord is reached during this process, we return the current level (transformation sequence length). Otherwise, we continue the BFS until all possibilities are exhausted. If no valid transformation sequence is found, we return 0.

Round 3 · High · Hard

p.1350

Breadth First Search (BFS)

#815 Bus Routes

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table · BFS
 Lists: Grind 169

Problem

You are given an array `routes` representing bus routes where `routes[i]` is a bus route that the `i`th bus repeats forever.

- For example, if `routes[0] = [1, 5, 7]`, this means that the 0th bus travels in the sequence `1 -> 5 -> 7 -> 1 -> 5 -> 7 -> 1 -> ...` forever.

You will start at the bus stop `source` (You are not on any bus initially), and you want to go to the bus stop `target`. You can travel between bus stops by buses only.

Return the least number of buses you must take to travel from `source` to `target`. Return `-1` if it is not possible.

Example 1:

Round 3 · High · Hard

p.1351

Input: `routes = [[1,2,7],[3,6,7]]`, `source = 1`, `target = 6`

Output: 2

Explanation: The best strategy is take the first bus to the bus stop 7, then take the second bus to the bus stop 6.

Example 2:

Input: `routes = [[7,12],[4,5,15],[6],[15,19],[9,12,13]]`, `source = 15`, `target = 12`

Output: -1

Constraints:

- `1 <= routes.length <= 500`.
- `1 <= routes[i].length <= 105`
- All the values of `routes[i]` are unique.
- `sum(routes[i].length) <= 105`
- `0 <= routes[i][j] < 106`
- `0 <= source, target < 106`

Community Solutions (Python)

• WalkCC

```
class Solution:
    def numBusesToDestination(
        self,
        routes: list[list[int]],
        source: int,
        target: int,
    ) -> int:
        if source == target:
            return 0
```

Round 3 · High · Hard

p.1352


```

graph = collections.defaultdict(list)
usedBuses = set()

for i in range(len(routes)):
    for route in routes[i]:
        graph[route].append(i)

q = collections.deque([source])

step = 1

while q:
    for _ in range(len(q)):
        for bus in graph[q.popleft()]:
            if bus in usedBuses:
                continue
            usedBuses.add(bus)
            for nextRoute in routes[bus]:
                if nextRoute == target:
                    return step
                q.append(nextRoute)

    step += 1

return -1

```

• LeetCode

```

class Solution:
    def numBusesToDestination(
        self, routes: List[List[int]], source: int, target: int
    ) -> int:
        if source == target:
            return 0
        g = defaultdict(list)
        for i, route in enumerate(routes):
            for stop in route:
                g[stop].append(i)

```

Round 3 · High · Hard

p.1353

```

if source not in g or target not in g:
    return -1
q = [(source, 0)]
vis_bus = set()
vis_stop = {source}
for stop, bus_count in q:
    if stop == target:
        return bus_count
    for bus in g[stop]:
        if bus not in vis_bus:
            vis_bus.add(bus)
            for next_stop in routes[bus]:
                if next_stop not in vis_stop:
                    vis_stop.add(next_stop)
                    q.append((next_stop, bus_count + 1))

return -1

```

• SimplyLeet

```

from collections import defaultdict, deque

def numBusesToDestination(routes, source, target):
    # If source equals target, no buses are needed.
    if source == target:
        return 0

    # Build mapping: bus stop -> list of bus indices
    stop_to_buses = defaultdict(list)
    for bus_index, stops in enumerate(routes):
        for stop in stops:
            stop_to_buses[stop].append(bus_index)

    visited_buses = set()
    visited_stops = set([source])
    queue = deque()

    # Initialize queue with all buses that include the source stop.
    for bus in stop_to_buses[source]:
        visited_buses.add(bus)

```

Round 3 · High · Hard

p.1354

```

        queue.append((bus, 1))

# Perform BFS over bus routes.
while queue:
    current_bus, buses_taken = queue.popleft()
    for stop in routes[current_bus]:
        if stop == target:
            return buses_taken
        if stop not in visited_stops:
            visited_stops.add(stop)

            for neighbor_bus in stop_to_buses[stop]:
                if neighbor_bus not in visited_buses:
                    visited_buses.add(neighbor_bus)
                    queue.append((neighbor_bus, buses_taken + 1))

return -1

# Example usage:
# routes = [[1,2,7],[3,6,7]]
# print(numBusesToDestination(routes, 1, 6))

```

• LC.ca

```

class Solution:
    def numBusesToDestination(
        self, routes: List[List[int]], source: int, target: int
    ) -> int:
        if source == target:
            return 0

        s = [set(r) for r in routes]
        d = defaultdict(list)
        for i, r in enumerate(routes):

```

Round 3 · High · Hard

p.1355

```

        for v in r:
            d[v].append(i)

        g = defaultdict(list)
        for ids in d.values():
            m = len(ids)
            for i in range(m):
                for j in range(i + 1, m):
                    a, b = ids[i], ids[j]
                    g[a].append(b)

                    g[b].append(a)

        q = deque(d[source])
        ans = 1
        vis = set(d[source])
        while q:
            for _ in range(len(q)):
                i = q.popleft()
                if target in s[i]:
                    return ans
                for j in g[i]:
                    if j not in vis:
                        vis.add(j)
                        q.append(j)

            ans += 1
        return -1

```

◆ Quick Review

Key Insights

- Model the problem as a graph where nodes are bus routes.
- Two bus routes are connected if they share at least one common bus stop.

Round 3 · High · Hard

p.1356

- Use a hash map to quickly look up bus routes available at any bus stop.
- Employ Breadth-First Search (BFS) on the bus routes graph to find the minimum number of bus transfers.
- Mark visited bus routes and bus stops to avoid redundant work.

Space & Time

Time Complexity: $O(N * L)$ in the worst-case scenario, where N is the number of bus routes and L is the average number of stops per route.

Space Complexity: $O(N * L)$ due to storage of the stop-to-buses mapping and BFS auxiliary data structures.

Solution

We begin by constructing a mapping from each bus stop to the list of buses passing through it. This allows quick retrieval of buses available at a given stop. Starting at the source stop, initialize the BFS queue with all buses that can be boarded at that stop. Then, for each bus route in the queue, inspect each stop along that route. If a stop is the target, return

the number of buses taken so far. Otherwise, for each unvisited stop, add all adjacent bus routes (buses that pass through this stop) to the BFS queue with an incremented bus count. Continue until the queue is empty, which means the target cannot be reached.

Breadth First Search (BFS)

□ #1293 Shortest Path in a Grid with Obstacles Elimination

HAR

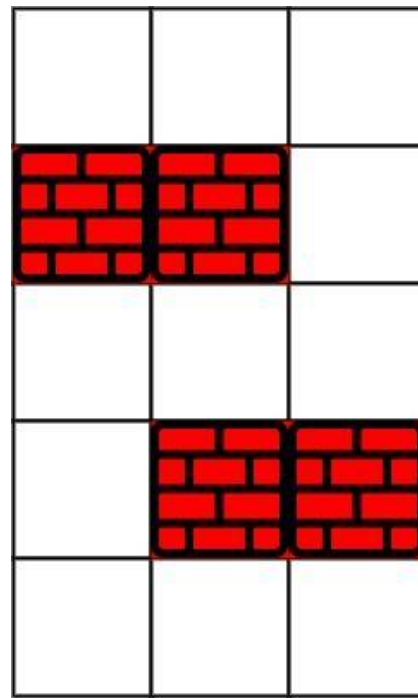
LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Breadth-First Search · Matrix
 Lists: AlgoMaster 600

Problem

You are given an $m \times n$ integer matrix grid where each cell is either 0 (empty) or 1 (obstacle). You can move up, down, left, or right from and to an empty cell in one step.

Return the minimum number of steps to walk from the upper left corner (0, 0) to the lower right corner ($m - 1$, $n - 1$) given that you can eliminate at most k obstacles. If it is not possible to find such walk return -1.

Example 1:



Input: grid = [[0,0,0],[1,1,0],[0,0,0],[0,1,1],[0,0,0]], k = 1

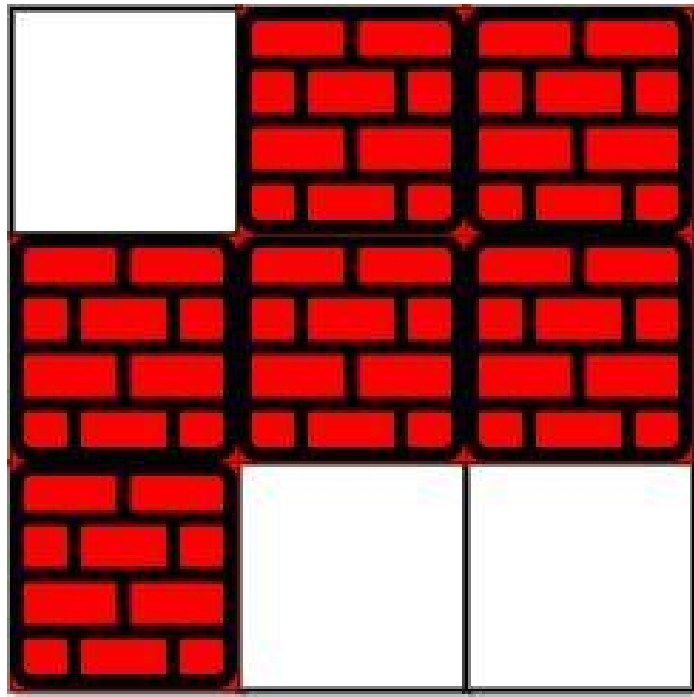
Output: 6

Explanation:

The shortest path without eliminating any obstacle is 10.

The shortest path with one obstacle elimination at position (3,2) is 6. Such path is (0,0) -> (0,1) -> (0,2) -> (1,2) -> (2,2) -> (3,2) -> (4,2).

Example 2:



Input: grid = [[0,1,1],[1,1,1],[1,0,0]], k = 1

Output: -1

Explanation: We need to eliminate at least two obstacles to find such a walk.

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq m, n \leq 40$
- $1 \leq k \leq m * n$
- $\text{grid}[i][j]$ is either 0 or 1.
- $\text{grid}[0][0] == \text{grid}[m - 1][n - 1] == 0$

Round 3 · High · Hard

p.1361

Community Solutions (Python)

• WalkCC

```
class Solution:
    def shortestPath(self, grid: list[list[int]], k: int) -> int:
        m = len(grid)
        n = len(grid[0])
        if m == 1 and n == 1:
            return 0

        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        q = collections.deque([(0, 0, k)])
        seen = {(0, 0, k)}
```

```
        step = 0
        while q:
            step += 1
            for _ in range(len(q)):
                i, j, eliminate = q.popleft()
                for dx, dy in DIRS:
                    x = i + dx
                    y = j + dy
                    if x < 0 or x == m or y < 0 or y == n:
```

```
                        continue
                    if x == m - 1 and y == n - 1:
                        return step
                    if grid[x][y] == 1 and eliminate == 0:
                        continue
                    newEliminate = eliminate - grid[x][y]
                    if (x, y, newEliminate) in seen:
                        continue
                    q.append((x, y, newEliminate))
                    seen.add((x, y, newEliminate))
```

```
        return -1
```

• LeetCode

Round 3 · High · Hard

p.1362

```
class Solution:
    def shortestPath(self, grid: List[List[int]], k: int) -> int:
        m, n = len(grid), len(grid[0])
        if k >= m + n - 3:
            return m + n - 2
        q = deque([(0, 0, k)])
        vis = {(0, 0, k)}
        ans = 0
        while q:
            ans += 1

            for _ in range(len(q)):
                i, j, k = q.popleft()
                for a, b in [(0, -1), (0, 1), (1, 0), (-1, 0)]:
                    x, y = i + a, j + b
                    if 0 <= x < m and 0 <= y < n:
                        if x == m - 1 and y == n - 1:
                            return ans
                        if grid[x][y] == 0 and (x, y, k) not in vis:
                            q.append((x, y, k))
                            vis.add((x, y, k))

            if grid[x][y] == 1 and k > 0 and (x, y, k - 1) not in vis:
                q.append((x, y, k - 1))
                vis.add((x, y, k - 1))

        return -1
```

• SimplyLeet

```
from collections import deque

def shortestPath(grid, k):
    # Dimensions of the grid
    rows, cols = len(grid), len(grid[0])

    # If k is large enough, we can take a direct Manhattan route
    if k >= rows + cols - 2:
        return rows + cols - 2
```

Round 3 · High · Hard

p.1363

```
# Queue for BFS: each element is (row, col, obstaclesEliminated, steps)
queue = deque([(0, 0, 0, 0)])

# 3D visited: visited[row][col] stores the smallest number of obstacles
eliminated so far
visited = [[False] * (k + 1) for _ in range(cols)] for _ in range(rows)]
visited[0][0][0] = True

# Possible directions: right, left, down, up
directions = [(0, 1), (0, -1), (1, 0), (-1, 0)]

while queue:
    row, col, eliminated, steps = queue.popleft()

    # If reached destination, return the number of steps
    if row == rows - 1 and col == cols - 1:
        return steps

    # Explore neighbors in all four directions
    for dr, dc in directions:
        newRow, newCol = row + dr, col + dc

        # Check if new position is inside the grid bounds
        if 0 <= newRow < rows and 0 <= newCol < cols:
            newEliminated = eliminated + grid[newRow][newCol]
            # Only proceed if the new elimination count is within limit and
            state not visited
            if newEliminated <= k and not visited[newRow][newCol][newElimi-
nated]:
                visited[newRow][newCol][newEliminated] = True
                queue.append((newRow, newCol, newEliminated, steps + 1))

    # If destination is unreachable, return -1
    return -1

# Example Usage:
print(shortestPath([[0,0,0],[1,1,0],[0,0,0],[0,1,1],[0,0,0]], 1))
```

Round 3 · High · Hard

p.1364

Breadth First Search (BFS)

#2290 Minimum Obstacle Removal to Reach Corner

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Breadth-First Search · Graph Theory ·
 Heap (Priority Queue) · Matrix · Shortest Path
 Lists: AlgoMaster 600

Problem

You are given a 0-indexed 2D integer array `grid` of size $m \times n$. Each cell has one of two values:

- 0 represents an empty cell,
- 1 represents an obstacle that may be removed.

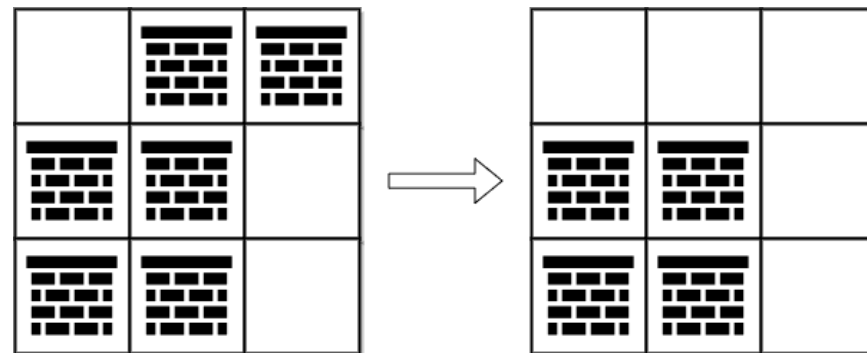
You can move up, down, left, or right from and to an empty cell.

Return the minimum number of obstacles to remove so you can move from the upper left corner $(0, 0)$ to the lower right corner $(m - 1, n - 1)$.

Example 1:

Round 3 · High · Hard

p.1365



Input: `grid = [[0,1,1],[1,1,0],[1,1,0]]`

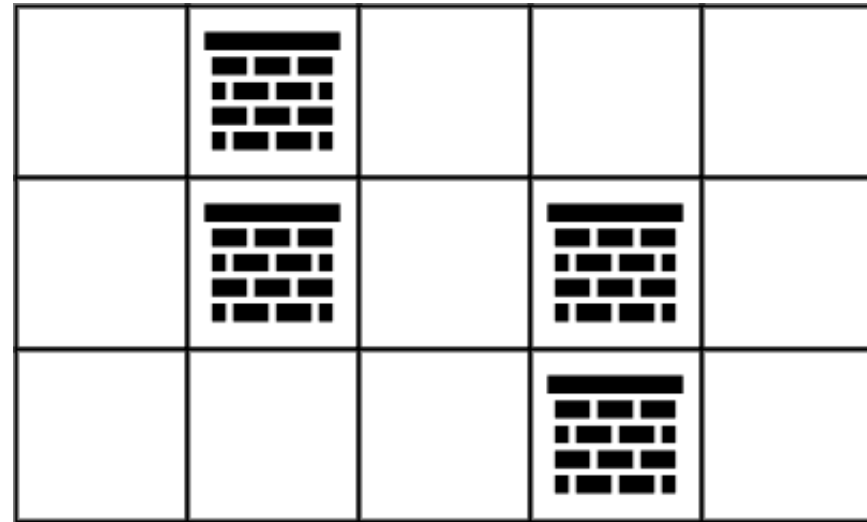
Output: 2

Explanation: We can remove the obstacles at $(0, 1)$ and $(0, 2)$ to create a path from $(0, 0)$ to $(2, 2)$.

It can be shown that we need to remove at least 2 obstacles, so we return 2.

Note that there may be other ways to remove 2 obstacles to create a path.

Example 2:



Round 3 · High · Hard

p.1366

Input: grid = [[0,1,0,0,0],[0,1,0,1,0],[0,0,0,1,0]]

Output: 0

Explanation: We can move from (0, 0) to (2, 4) without removing any obstacles, so we return 0.

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq m, n \leq 105$
- $2 \leq m * n \leq 105$
- $\text{grid}[i][j]$ is either 0 or 1.
- $\text{grid}[0][0] == \text{grid}[m - 1][n - 1] == 0$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minimumObstacles(self, grid: list[list[int]]) -> int:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(grid)
        n = len(grid[0])
        minHeap = [(grid[0][0], 0, 0)] # (d, i, j)
        dist = [[math.inf] * n for _ in range(m)]
        dist[0][0] = grid[0][0]

        while minHeap:
```

```
d, i, j = heapq.heappop(minHeap)
if i == m - 1 and j == n - 1:
    return d
for dx, dy in DIRS:
    x = i + dx
    y = j + dy
    if x < 0 or x == m or y < 0 or y == n:
        continue
    newDist = d + grid[i][j]
    if newDist < dist[x][y]:

        dist[x][y] = newDist
        heapq.heappush(minHeap, (newDist, x, y))

return dist[m - 1][n - 1]
```

• LeetCode

```
class Solution:
    def minimumObstacles(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        q = deque([(0, 0, 0)])
        vis = set()
        dirs = (-1, 0, 1, 0, -1)
        while 1:
            i, j, k = q.popleft()
            if i == m - 1 and j == n - 1:
                return k

            if (i, j) in vis:
                continue
            vis.add((i, j))
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                if 0 <= x < m and 0 <= y < n:
                    if grid[x][y] == 0:
                        q.appendleft((x, y, k))
                    else:
                        q.append((x, y, k + 1))
```

• SimplyLeet


```

from collections import deque

def minimumObstacles(grid):
    # Number of rows and columns
    m, n = len(grid), len(grid[0])
    # Initialize deque with the starting position and initial cost 0
    dq = deque([(0, 0, 0)]) # (row, col, obstacles_removed)

    # Create a 2D list to track the minimum obstacles removed to reach each cell
    min_removed = [[float('inf')] * n for _ in range(m)]

    min_removed[0][0] = 0

    # Directions: up, down, left, right
    directions = [(0, 1), (0, -1), (1, 0), (-1, 0)]

    # Process the deque BFS style using 0-1 BFS approach
    while dq:
        r, c, cost = dq.popleft()
        # If destination is reached, return the cost
        if r == m - 1 and c == n - 1:
            return cost

        for dr, dc in directions:
            nr, nc = r + dr, c + dc
            # Check bounds
            if 0 <= nr < m and 0 <= nc < n:
                # Calculate new_cost: add 1 if there is an obstacle, else add 0
                new_cost = cost + grid[nr][nc]
                # Only proceed if this new_cost is better than any previously found
                if new_cost < min_removed[nr][nc]:

```

Round 3 · High · Hard

p.1369

```

        min_removed[nr][nc] = new_cost
        # For cost 0, append left; for cost 1, append right
        if grid[nr][nc] == 0:
            dq.appendleft((nr, nc, new_cost))
        else:
            dq.append((nr, nc, new_cost))

        # If destination is unreachable, though the problem guarantees it is reachable.
        return -1

# Example usage:

grid_example = [[0, 1, 1], [1, 1, 0], [1, 1, 0]]
print(minimumObstacles(grid_example))

```

Round 3 · High · Hard

p.1370

Round 4 · Mid · Easy

Round 4

Mid Priority · Easy

38 questions · 15 patterns

Bit Manipulation #136 #190 #191 #231 #268
#338 #461 #645

Prefix Sum #303 #724 #1480 #1854

Matrix (2D Array) #867

LinkedList In-place Reversal #206 #234

Fast and Slow Pointers #141 #202 #876

Monotonic Stack #496 #1475

Queues #346 #933 #2073

Recursion #21

Divide and Conquer #1763

BST / Ordered Set #270 #653 #700 #938

Heaps #1046

Top K Elements #703

Round 3 · High · Hard

p.1371

☐ ● Intervals #228 #252 ↗

☐ ● Data Structure Design #225 #232 #359 ↗

☐ ● Greedy #455 #3074 ↗

Round 4 · Mid · Easy

p.1372

Bit Manipulation**Round 4 · Mid · Easy · 8 questions****Round 4 · Mid · Easy****p.1373****Bit Manipulation**

#136 Single Number**EAS****LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Bit Manipulation****Lists: Grind 169****Problem**

Given a non-empty array of integers `nums`, every element appears twice except for one. Find that single one.

You must implement a solution with a linear runtime complexity and use only constant extra space.

Example 1:**Input: `nums = [2,2,1]`****Output: 1****Example 2:****Input: `nums = [4,1,2,1,2]`****Output: 4****Example 3:****Input: `nums = [1]`****Output: 1****Round 4 · Mid · Easy****p.1374**

Constraints:

- $1 \leq \text{nums.length} \leq 3 * 10^4$
- $-3 * 10^4 \leq \text{nums}[i] \leq 3 * 10^4$
- Each element in the array appears twice except for one element which appears only once.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def singleNumber(self, nums: List[int]) -> int:
        return functools.reduce(operator.xor, nums, 0)
```

• LeetDoocs

```
class Solution:
    def singleNumber(self, nums: List[int]) -> int:
        return reduce(xor, nums)
```

• SimplyLeet

```
# Function to find the single number in the list
def singleNumber(nums):
    # Initialize result to 0; any number XOR 0 is the number itself
    result = 0
    # Iterate through every number in the list
    for num in nums:
        # XOR the current result with the current number
        result ^= num # Using XOR to cancel duplicates
    # Return the unique number that did not cancel out
    return result
```

```
# Example usage
nums = [4, 1, 2, 1, 2]
print(singleNumber(nums)) # Output should be 4
```

• LC.ca

```
'''
>>> from functools import reduce
>>> reduce(lambda x, y: x ^ y, [3, 5, 3])
5
'''

from functools import reduce

class Solution:
    def singleNumber(self, nums: List[int]) -> int:

        return reduce(lambda x, y: x ^ y, nums)

#####

class Solution(object):
    def singleNumber(self, nums):
        """
        :type nums: List[int]
        :rtype: int
        """

        for i in range(1, len(nums)):
            nums[0] ^= nums[i]
        return nums[0]
```

◆ Quick Review

Key Insights

- Using bitwise XOR: XOR-ing a number with itself cancels out to 0.

- XOR is both commutative and associative, so the order of operations does not matter.
- A single traversal (linear time) and constant extra space is sufficient for solving the problem.

Space & Time

Time Complexity: $O(n)$ – We only traverse the array once.

Space Complexity: $O(1)$ – Only a single extra variable is used regardless of input size.

Solution

We can solve this problem by initializing a variable to 0 and then traversing the array while XOR-ing each element with the variable. Due to the properties of XOR, pairs of identical numbers will cancel each other out ($x \text{ XOR } x = 0$) and the unique number will be left as the final result. This approach uses constant extra space and performs in linear time.

Bit Manipulation

☐ #190 Reverse Bits

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Divide and Conquer · Bit Manipulation
Lists: Grind 169

Problem

Reverse bits of a given 32 bits signed integer.

Example 1:

Input: $n = 43261596$

Output: 964176192

Explanation:

Example 2:

Input: $n = 2147483644$

Output: 1073741822

Explanation:

Constraints:

- $0 \leq n \leq 2^{31} - 2$
- n is even.

Follow up: If this function is called many times, how would you optimize it?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def reverseBits(self, n: int) -> int:
        ans = 0

        for i in range(32):
            if n >> i & 1:
                ans |= 1 << 31 - i

        return ans
```

• LeetCode

```
class Solution:
    def reverseBits(self, n: int) -> int:
        ans = 0
        for i in range(32):
            ans |= (n & 1) << (31 - i)
            n >>= 1
        return ans
```

• SimplyLeet

```
# Function to reverse bits of a 32-bit binary string.
def reverse_bits(n: str) -> int:
    # Reverse the binary string using slicing.
    reversed_binary = n[::-1]
    # Convert the reversed binary string to an integer.
    result = int(reversed_binary, 2)
    return result

# Example usage:
binary_input = "00000010100101000001111010011100"

print(reverse_bits(binary_input)) # Expected output: 964176192
```

• LC.ca

Round 4 · Mid · Easy

p.1379

```
class Solution:
    def reverseBits(self, n: int) -> int:
        res = 0
        for i in range(32):
            res |= (n & 1) << (31 - i)
            n >>= 1
        return res
```

◆ Quick Review

Key Insights

- The input is a binary string of fixed length (32 bits).
- Reversing the bits is equivalent to reversing the string.
- The reversed binary string can be converted back into its integer representation.
- Since the number of bits is constant (32), the solution can be achieved in constant time and constant space.
- For performance optimization when this function is called many times, caching may be applied for common intermediate results.

Space & Time

Time Complexity: $O(1)$ – We always process 32 bits regardless of the input.

Round 4 · Mid · Easy

p.1380

Space Complexity: $O(1)$ – Only constant extra space is used.

Solution

We solve the problem by reversing the 32-character binary string representing the unsigned integer. Once reversed, the new binary string is converted into its integer value. An alternative solution involves using bitwise operations where you iterate over each bit, shift the result to the left, and add the current bit. This approach does the reversal one bit at a time. The chosen approach leverages string reversal which is straightforward to implement and understand.

Bit Manipulation

#191 Number of 1 Bits

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Divide and Conquer · Bit Manipulation
Lists: Grind 169

Problem

Given a positive integer n , write a function that returns the number of set bits in its binary representation (also known as the Hamming weight).

Example 1:

Input: $n = 11$

Output: 3

Explanation:

The input binary string 1011 has a total of three set bits.

Example 2:

Input: $n = 128$

Output: 1

Explanation:

The input binary string 10000000 has a total of one set bit.

Example 3:

Input: n = 2147483645

Output: 30

Explanation:

The input binary string 1111111111111111111111111111101 has a total of thirty set bits.

Constraints:

- $1 \leq n \leq 2^{31} - 1$

Follow up: If this function is called many times, how would you optimize it?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def hammingWeight(self, n: int) -> int:
        ans = 0

        for i in range(32):
            if (n >> i) & 1:
                ans += 1

        return ans

class Solution:
    def hammingWeight(self, n: int) -> int:
        return n.bit_count()
```

Round 4 · Mid · Easy

p.1383

• LeetCode

```
class Solution:
    def hammingWeight(self, n: int) -> int:
        ans = 0
        while n:
            n &= n - 1
            ans += 1
        return ans
```

• SimplyLeet

```
# Function to count the number of set bits in a given integer n
def hammingWeight(n: int) -> int:
    count = 0 # Initialize count of set bits
    while n:
        # Remove the lowest set bit from n
        n &= n - 1
        count += 1 # Increment count for each set bit removed
    return count

# Example usage:
print(hammingWeight(11)) # Output: 3
```

• LC.ca

```
class Solution:
    def hammingWeight(self, n: int) -> int:
        ans = 0
        while n:
            n &= n - 1
            ans += 1
        return ans
```

◆ Quick Review

Key Insights

Round 4 · Mid · Easy

p.1384

- The problem is essentially counting the number of 1s in the binary representation of a number.
- A common efficient approach uses bit manipulation: repeatedly clear the lowest set bit until n becomes zero.
- The expression $n \& (n - 1)$ clears the lowest set bit of n .
- This method only iterates as many times as there are set bits, making it efficient.

Space & Time

Time Complexity: $O(k)$, where k is the number of set bits, worst-case $O(\log n)$ (or $O(1)$ considering 32-bit integers).

Space Complexity: $O(1)$.

Solution

To solve the problem, we use the bit manipulation trick $n \& (n - 1)$ which removes the lowest set bit from n . The algorithm is as follows:

– Initialize a counter to 0. – While n is not zero, update n by $n = n \& (n - 1)$, and increment the counter. – The counter will hold the number of set bits by the end of the loop. This approach

leverages efficient bit-level operations and works well even when the function is called frequently.

Bit Manipulation

#231 Power of Two

EAS

LeetCode · SimplyLeet · WalkCC · LeetCode
Math · Bit Manipulation · Recursion
Lists: AlgoMaster 600

Problem

Given an integer n , return true if it is a power of two. Otherwise, return false.

An integer n is a power of two, if there exists an integer x such that $n == 2^x$.

Example 1:

Input: $n = 1$
Output: true
Explanation: $2^0 = 1$

Example 2:

Input: $n = 16$
Output: true
Explanation: $2^4 = 16$

Example 3:

Input: $n = 3$
Output: false

Constraints:

- $-2^{31} \leq n \leq 2^{31} - 1$

Round 4 · Mid · Easy

p.1387

Follow up: Could you solve it without loops/recursion?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isPowerOfTwo(self, n: int) -> bool:
        return n >= 0 and n.bit_count() == 1
```

• LeetCode

```
class Solution:
    def isPowerOfTwo(self, n: int) -> bool:
        return n > 0 and (n & (n - 1)) == 0

class Solution:
    def isPowerOfTwo(self, n: int) -> bool:
        return n > 0 and n == n & (-n)
```

• SimplyLeet

```
# Function to determine if a number is a power of two
def isPowerOfTwo(n):
    # n must be positive and only one bit should be set in its binary
    # representation.
    return n > 0 and (n & (n - 1)) == 0

# Example usage:
print(isPowerOfTwo(1)) # True, because 2^0 = 1
print(isPowerOfTwo(16)) # True, because 2^4 = 16
print(isPowerOfTwo(3)) # False, 3 is not a power of two
```

Round 4 · Mid · Easy

p.1388

Bit Manipulation

☐ #268 Missing Number

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table · Math · Binary Search · Bit
 Manipulation

Lists: Grind 169

Problem

Given an array `nums` containing n distinct numbers in the range $[0, n]$, return the only number in the range that is missing from the array.

Example 1:

Input: `nums = [3,0,1]`

Output: 2

Explanation:

$n = 3$ since there are 3 numbers, so all numbers are in the range $[0,3]$. 2 is the missing number in the range since it does not appear in `nums`.

Example 2:

Input: `nums = [0,1]`

Output: 2

Round 4 · Mid · Easy

p.1389

Explanation:

$n = 2$ since there are 2 numbers, so all numbers are in the range $[0,2]$. 2 is the missing number in the range since it does not appear in `nums`.

Example 3:

Input: `nums = [9,6,4,2,3,5,7,0,1]`

Output: 8

Explanation:

$n = 9$ since there are 9 numbers, so all numbers are in the range $[0,9]$. 8 is the missing number in the range since it does not appear in `nums`.

Constraints:

- $n == \text{nums.length}$
- $1 \leq n \leq 10^4$
- $0 \leq \text{nums}[i] \leq n$
- All the numbers of `nums` are unique.

Follow up: Could you implement a solution using only $O(1)$ extra space complexity and $O(n)$ runtime complexity?

Community Solutions (Python)

- WalkCC

Round 4 · Mid · Easy

p.1390

```
class Solution:
    def missingNumber(self, nums: list[int]) -> int:
        ans = len(nums)

        for i, num in enumerate(nums):
            ans ^= i ^ num

        return ans
```

• LeetCode

```
class Solution:
    def missingNumber(self, nums: List[int]) -> int:
        return reduce(xor, (i ^ v for i, v in enumerate(nums, 1)))

class Solution:
    def missingNumber(self, nums: List[int]) -> int:
        n = len(nums)
        return (1 + n) * n // 2 - sum(nums)
```

• SimplyLeet

```
# Function to find the missing number in the array
def missingNumber(nums):
    # Calculate n, where n is the length of the nums array
    n = len(nums)
    # Compute the expected total sum using the arithmetic sum formula for
    # numbers 0 to n
    expected_sum = n * (n + 1) // 2
    # Compute the actual sum of the numbers in the array
    actual_sum = sum(nums)
    # The missing number is the difference between expected and actual sums
    return expected_sum - actual_sum
```

```
# Example Usage
print(missingNumber([3, 0, 1])) # Output: 2
```

• LC.ca

Round 4 · Mid · Easy

p.1391

```
class Solution:
    def missingNumber(self, nums: List[int]) -> int:
        return reduce(xor, (i ^ v for i, v in enumerate(nums, 1)))
```

◆ Quick Review

Key Insights

- The range of numbers is continuous from 0 to n, but one number is missing.
- The sum of numbers from 0 to n can be computed using the arithmetic sum formula.
- Subtracting the sum of the given array from the expected sum gives the missing number.
- There is a follow-up challenge to solve the problem in O(n) time and O(1) extra space.

Space & Time

Time Complexity: O(n)

Space Complexity: O(1)

Solution

We compute the expected sum of all numbers from 0 to n using the formula $\text{sum} = n*(n+1)/2$. Then, we compute the actual sum of the numbers in the array. The missing number is obtained by subtracting the actual sum from the expected sum. This approach

Round 4 · Mid · Easy

p.1392

uses arithmetic operations ($O(1)$ extra space) and loops over the array once ($O(n)$ time).

Bit Manipulation

□ #338 Counting Bits

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Dynamic Programming · Bit Manipulation
Lists: Grind 169

Problem

Given an integer n , return an array `ans` of length $n + 1$ such that for each i ($0 \leq i \leq n$), `ans[i]` is the number of 1's in the binary representation of i .

Example 1:

```
Input: n = 2
Output: [0,1,1]
Explanation:
0 --> 0
1 --> 1
2 --> 10
```

Example 2:

```
Input: n = 5
Output: [0,1,1,2,1,2]
Explanation:
0 --> 0
1 --> 1
2 --> 10
3 --> 11
4 --> 100
```

Constraints:

- $0 \leq n \leq 105$

Follow up:

- It is very easy to come up with a solution with a runtime of $O(n \log n)$. Can you do it in linear time $O(n)$ and possibly in a single pass?
- Can you do it without using any built-in function (i.e., like `__builtin_popcount` in C++)?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def countBits(self, n: int) -> List[int]:
        # f(i) := i's number of 1s in bitmask
        # f(i) = f(i / 2) + i % 2
        ans = [0] * (n + 1)

        for i in range(1, n + 1):
            ans[i] = ans[i // 2] + (i & 1)

        return ans
```

• LeetCode

```
class Solution:
    def countBits(self, n: int) -> List[int]:
        return [i.bit_count() for i in range(n + 1)]

class Solution:
    def countBits(self, n: int) -> List[int]:
        ans = [0] * (n + 1)
        for i in range(1, n + 1):
            ans[i] = ans[i & (i - 1)] + 1
        return ans
```

• SimplyLeet

```
def countBits(n):
    # Create a list to store the number of 1's for each number from 0 to n
    dp = [0] * (n + 1)
    # Loop over each number starting from 1, since dp[0] is already 0
    for i in range(1, n + 1):
        # dp[i] is computed as the count for i//2 plus 1 if i is odd (i & 1)
        dp[i] = dp[i >> 1] + (i & 1)
    return dp

# Example usage:

# print(countBits(5)) -> [0, 1, 1, 2, 1, 2]
```

• LC.ca

```
class Solution:
    def countBits(self, n: int) -> List[int]:
        ans = [0] * (n + 1)
        for i in range(1, n + 1):
            ans[i] = ans[i & (i - 1)] + 1
        return ans
```

◆ Quick Review

Key Insights

- We can use dynamic programming to build the solution by utilizing the count of bits for smaller numbers.
- The relationship $dp[i] = dp[i \gg 1] + (i \& 1)$ holds, where $i \gg 1$ gives the count for i divided by 2 (dropping the least significant bit) and $(i \& 1)$ checks if the least significant bit is 1.

- This approach computes the result for each number in constant time, resulting in a linear time solution.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Solution

We use a dynamic programming approach where an array (or list) `dp` is initialized with size $n+1$. The key idea is to iterate from 1 to n and for each number, use the formula $dp[i] = dp[i \gg 1] + (i \& 1)$.

– The expression $(i \gg 1)$ effectively divides i by 2, giving the count of 1's for that half number. – The $(i \& 1)$ operation checks if i is odd, adding 1 if true. This method ensures each $dp[i]$ is computed in constant time, leading to an overall $O(n)$ time complexity.

Bit Manipulation

□ #461 Hamming Distance

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode

Bit Manipulation

Lists: AlgoMaster 600

Problem

The Hamming distance between two integers is the number of positions at which the corresponding bits are different.

Given two integers x and y , return the Hamming distance between them.

Example 1:

```
Input: x = 1, y = 4
Output: 2
Explanation:
1 (0 0 0 1)
4 (0 1 0 0)
  ↑  ↑
  The above arrows point to positions where the corresponding bits are different.
```

Example 2:

```
Input: x = 3, y = 1
Output: 1
```

Constraints:

- $0 \leq x, y \leq 2^{31} - 1$

Note: This question is the same as 2220: Minimum Bit Flips to Convert Number.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def hammingDistance(self, x: int, y: int) -> int:
        ans = 0

        while x > 0 or y > 0:
            ans += (x & 1) ^ (y & 1)
            x >>= 1
            y >>= 1

        return ans
```

• LeetCode

```
class Solution:
    def hammingDistance(self, x: int, y: int) -> int:
        return (x ^ y).bit_count()
```

• SimplyLeet

```
# Function to calculate Hamming distance between two integers.
def hammingDistance(x, y):
    # Compute XOR of x and y to highlight differing bits.
    xor_result = x ^ y
    count = 0
    # Count the number of set bits in xor_result.
    while xor_result:
        # Remove the lowest set bit.
        xor_result &= xor_result - 1
        count += 1

    return count

# Example usage:
print(hammingDistance(1, 4)) # Output: 2
```

Round 4 · Mid · Easy

p.1399

Bit Manipulation

□ #645 Set Mismatch

EAS

LeetCode · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Bit Manipulation · Sorting
Lists: AlgoMaster 600

Problem

You have a set of integers s , which originally contains all the numbers from 1 to n . Unfortunately, due to some error, one of the numbers in s got duplicated to another number in the set, which results in repetition of one number and loss of another number.

You are given an integer array `nums` representing the data status of this set after the error.

Find the number that occurs twice and the number that is missing and return them in the form of an array.

Example 1:

Input: `nums = [1,2,2,4]`
Output: `[2,3]`

Example 2:

Round 4 · Mid · Easy

p.1400

Input: nums = [1,1]
Output: [1,2]

Constraints:

- $2 \leq \text{nums.length} \leq 104$
- $1 \leq \text{nums}[i] \leq 104$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findErrorNums(self, nums: list[int]) -> list[int]:
        for num in nums:
            if nums[abs(num) - 1] < 0:
                duplicate = abs(num)
            else:
                nums[abs(num) - 1] *= -1

        for i, num in enumerate(nums):
            if num > 0:

                return [duplicate, i + 1]
```

• SimplyLeet

```
# Python solution with clear line-by-line comments.
class Solution:
    def findErrorNums(self, nums: List[int]) -> List[int]:
        # initialize a list to count frequency; size n+1 for 1-indexing
        n = len(nums)
        count = [0] * (n + 1)

        duplicate = -1
        missing = -1
```

```
# count frequency for each number in the array
for num in nums:
    count[num] += 1
    # if count goes to 2, it's the duplicate
    if count[num] == 2:
        duplicate = num

# check for the missing number in range 1 to n
for i in range(1, n + 1):
    if count[i] == 0:

        missing = i
        break

# return the duplicate and missing numbers as required
return [duplicate, missing]
```

Prefix Sum

Round 4 · Mid · Easy · 4 questions

Round 4 · Mid · Easy

p.1403

Prefix Sum

□ #303 Range Sum Query – Immutable

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Design · Prefix Sum

Lists: AlgoMaster 600

Problem

Given an integer array `nums`, handle multiple queries of the following type:

1. Calculate the sum of the elements of `nums` between indices `left` and `right` inclusive where `left <= right`.

Implement the `NumArray` class:

- `NumArray(int[] nums)` Initializes the object with the integer array `nums`.
- `int sumRange(int left, int right)` Returns the sum of the elements of `nums` between indices `left` and `right` inclusive (i.e. `nums[left] + nums[left + 1] + ... + nums[right]`).

Example 1:

Round 4 · Mid · Easy

p.1404

```

Input
["NumArray", "sumRange", "sumRange", "sumRange"]
[[-2, 0, 3, -5, 2, -1]], [0, 2], [2, 5], [0, 5]]
Output
[null, 1, -1, -3]

Explanation
NumArray numArray = new NumArray([-2, 0, 3, -5, 2, -1]);

```

Constraints:

- $1 \leq \text{nums.length} \leq 104$
- $-105 \leq \text{nums}[i] \leq 105$
- $0 \leq \text{left} \leq \text{right} < \text{nums.length}$
- At most 104 calls will be made to `sumRange`.

Community Solutions (Python)

• WalkCC

```

class NumArray:
    def __init__(self, nums: List[int]):
        self.prefix = list(itertools.accumulate(nums, initial=0))

    def sumRange(self, left: int, right: int) -> int:
        return self.prefix[right + 1] - self.prefix[left]

```

• LeetCode

```

class NumArray:
    def __init__(self, nums: List[int]):
        self.s = list(accumulate(nums, initial=0))

    def sumRange(self, left: int, right: int) -> int:
        return self.s[right + 1] - self.s[left]

# Your NumArray object will be instantiated and called as such:
# obj = NumArray(nums)

# param_1 = obj.sumRange(left,right)

typedef struct {
    int* s;
} NumArray;

NumArray* numArrayCreate(int* nums, int n) {
    int* s = malloc(sizeof(int) * (n + 1));
    s[0] = 0;
    for (int i = 0; i < n; i++) {
        s[i + 1] = s[i] + nums[i];
    }

    NumArray* obj = malloc(sizeof(NumArray));
    obj->s = s;
    return obj;
}

int numArraySumRange(NumArray* obj, int left, int right) {
    return obj->s[right + 1] - obj->s[left];
}

void numArrayFree(NumArray* obj) {

```

```

    free(obj->s);
    free(obj);
}

/**
 * Your NumArray struct will be instantiated and called as such:
 * NumArray* obj = numArrayCreate(nums, numsSize);
 * int param_1 = numArraySumRange(obj, left, right);
 *
 * numArrayFree(obj);
 */

```

• SimplyLeet

```

class NumArray:
    def __init__(self, nums: list[int]):
        # Create prefix sum array with an extra 0 at the beginning.
        self.prefix = [0] * (len(nums) + 1)
        for i in range(len(nums)):
            # Each element at prefix[i+1] is the sum of nums[0] to nums[i].
            self.prefix[i + 1] = self.prefix[i] + nums[i]

    def sumRange(self, left: int, right: int) -> int:
        # Return sum of elements from index left to right inclusive.

        return self.prefix[right + 1] - self.prefix[left]

# Example Usage:
# numArray = NumArray([-2, 0, 3, -5, 2, -1])
# print(numArray.sumRange(0, 2)) # Output: 1
# print(numArray.sumRange(2, 5)) # Output: -1
# print(numArray.sumRange(0, 5)) # Output: -3

```

Prefix Sum

□ #724 Find Pivot Index

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Prefix Sum
 Lists: AlgoMaster 600

Problem

Given an array of integers `nums`, calculate the pivot index of this array.

The pivot index is the index where the sum of all the numbers strictly to the left of the index is equal to the sum of all the numbers strictly to the index's right.

If the index is on the left edge of the array, then the left sum is 0 because there are no elements to the left. This also applies to the right edge of the array.

Return the leftmost pivot index. If no such index exists, return `-1`.

Example 1:

Input: nums = [1,7,3,6,5,6]
 Output: 3
 Explanation:
 The pivot index is 3.
 Left sum = nums[0] + nums[1] + nums[2] = 1 + 7 + 3 = 11
 Right sum = nums[4] + nums[5] = 5 + 6 = 11

Example 2:

Input: nums = [1,2,3]
 Output: -1
 Explanation:
 There is no index that satisfies the conditions in the problem statement.

Example 3:

Input: nums = [2,1,-1]
 Output: 0
 Explanation:
 The pivot index is 0.
 Left sum = 0 (no elements to the left of index 0)
 Right sum = nums[1] + nums[2] = 1 + -1 = 0

Constraints:

- $1 \leq \text{nums.length} \leq 104$
- $-1000 \leq \text{nums}[i] \leq 1000$

Note: This question is the same as 1991: <https://leetcode.com/problems/find-the-middle-index-in-array/>

Community Solutions (Python)

- WalkCC

```
class Solution:
    def pivotIndex(self, nums: list[int]) -> int:
        summ = sum(nums)
        prefix = 0

        for i, num in enumerate(nums):
            if prefix == summ - prefix - num:
                return i
            prefix += num

        return -1
```

• LeetCode

```
class Solution:
    def pivotIndex(self, nums: List[int]) -> int:
        left, right = 0, sum(nums)
        for i, x in enumerate(nums):
            right -= x
            if left == right:
                return i
            left += x
        return -1
```

• SimplyLeet

```
# Python solution with line-by-line comments

def pivotIndex(nums):
    # Calculate the total sum of the array
    total_sum = sum(nums)

    # Initialize left sum to 0
    left_sum = 0

    # Iterate over each index and element in the array
```

```

for i, num in enumerate(nums):
    # Right sum is total sum minus left sum minus the current element
    right_sum = total_sum - left_sum - num

    # Check if left sum equals right sum
    if left_sum == right_sum:
        return i # Return the pivot index

    # Update left sum by adding the current element
    left_sum += num

# No pivot index found
return -1

# Example usage
print(pivotIndex([1,7,3,6,5,6])) # Output: 3

```

Prefix Sum

□ #1480 Running Sum of 1d Array

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Prefix Sum
 Lists: AlgoMaster 600

Problem

Given an array `nums`. We define a running sum of an array as `runningSum[i] = sum(nums[0]...nums[i])`.

Return the running sum of `nums`.

Example 1:

Input: `nums = [1,2,3,4]`
 Output: `[1,3,6,10]`
 Explanation: Running sum is obtained as follows: `[1, 1+2, 1+2+3, 1+2+3+4]`.

Example 2:

Input: `nums = [1,1,1,1,1]`
 Output: `[1,2,3,4,5]`
 Explanation: Running sum is obtained as follows: `[1, 1+1, 1+1+1, 1+1+1+1, 1+1+1+1+1]`.

Example 3:

Input: `nums = [3,1,2,10,1]`
 Output: `[3,4,6,16,17]`

Constraints:

- $1 \leq \text{nums.length} \leq 1000$

- $-10^6 \leq \text{nums}[i] \leq 10^6$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def runningSum(self, nums: List[int]) -> List[int]:
        return itertools.accumulate(nums)
```

• LeetCode

```
class Solution:
    def runningSum(self, nums: List[int]) -> List[int]:
        return list(accumulate(nums))
```

• SimplyLeet

```
# Function to compute running sum of a 1d array.
def running_sum(nums):
    # Initialize an empty list to store the running sum.
    result = []
    # Initialize the cumulative sum as 0.
    cumulative = 0
    # Iterate over each value in the input list.
    for num in nums:
        # Update the cumulative sum with the current number.
        cumulative += num

        # Append the updated cumulative sum to the result list.
        result.append(cumulative)
    # Return the list containing the running sums.
    return result

# Example usage:
print(running_sum([1, 2, 3, 4])) # Expected output: [1, 3, 6, 10]
```

Prefix Sum

□ #1854 Maximum Population Year

EAS

LeetCode · SimplyLeet · WalkCC · LeetCode
 Array · Counting · Prefix Sum
 Lists: AlgoMaster 600

Problem

You are given a 2D integer array logs where each logs[i] = [birth_i, death_i] indicates the birth and death years of the ith person.

The population of some year x is the number of people alive during that year. The ith person is counted in year x's population if x is in the inclusive range [birth_i, death_i - 1]. Note that the person is not counted in the year that they die. Return the earliest year with the maximum population.

Example 1:

```
Input: logs = [[1993,1999],[2000,2010]]
Output: 1993
Explanation: The maximum population is 1, and 1993 is the earliest year with this population.
```

Example 2:

Input: logs = [[1950,1961],[1960,1971],[1970,1981]]

Output: 1960

Explanation:

The maximum population is 2, and it had happened in years 1960 and 1970.

The earlier year between them is 1960.

Constraints:

- $1 \leq \text{logs.length} \leq 100$
- $1950 \leq \text{birth}_i < \text{death}_i \leq 2050$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maximumPopulation(self, logs: list[list[int]]) -> int:
        MIN_YEAR = 1950
        MAX_YEAR = 2050
        ans = 0
        maxPopulation = 0
        runningPopulation = 0
        # population[i] := the population of year i
        population = [0] * (MAX_YEAR + 1)

        for birth, death in logs:
            population[birth] += 1
            population[death] -= 1

        for year in range(MIN_YEAR, MAX_YEAR + 1):
            runningPopulation += population[year]
            if runningPopulation > maxPopulation:
                maxPopulation = runningPopulation
                ans = year

        return ans
```

• LeetCode

Round 4 · Mid · Easy

p.1415

```
class Solution:
    def maximumPopulation(self, logs: List[List[int]]) -> int:
        d = [0] * 101
        offset = 1950
        for a, b in logs:
            a, b = a - offset, b - offset
            d[a] += 1
            d[b] -= 1
        s = mx = j = 0
        for i, x in enumerate(d):
            s += x
            if mx < s:
                mx, j = s, i
        return j + offset
```

• SimplyLeet

```
# Python solution using a difference array approach

def maximumPopulation(logs):
    # Define the start and end years based on problem constraints.
    start_year = 1950
    end_year = 2050
    # Create an array to hold the changes in population for each year.
    # We need an extra space to handle the last decrement.
    population_changes = [0] * (end_year + 2)

    # For each log, increment count at birth year and decrement count at death
    year.
    for birth, death in logs:
        population_changes[birth] += 1 # Person becomes alive in the birth
        year
        population_changes[death] -= 1 # Person is not alive in the death year

    max_population = 0 # To store maximum observed population
    max_year = start_year # To store the corresponding earliest year
    current_population = 0 # Running total of the population

    # Iterate from start_year to end_year to compute population count per year.
```

Round 4 · Mid · Easy

p.1416


```
for year in range(start_year, end_year + 1):
    current_population += population_changes[year]
    # Update maximum and earliest year if a new maximum population is
    found.
    if current_population > max_population:
        max_population = current_population
        max_year = year

return max_year

# Example usage:

logs = [[1950,1961],[1960,1971],[1970,1981]]
print(maximumPopulation(logs)) # Expected output: 1960
```

Round 4 · Mid · Easy

p.1417

Matrix (2D Array)

Round 4 · Mid · Easy · 1 question

Round 4 · Mid · Easy

p.1418

Matrix (2D Array)

#867 Transpose Matrix

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Matrix · Simulation
 Lists: CodeSignal

Problem

Given a 2D integer array matrix, return the transpose of matrix.

The transpose of a matrix is the matrix flipped over its main diagonal, switching the matrix's row and column indices.

2	4	-1
-10	5	11
18	-7	6

→

2	-10	18
4	5	-7
-1	11	6

Example 1:

Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]
 Output: [[1,4,7],[2,5,8],[3,6,9]]

Example 2:

Round 4 · Mid · Easy

p.1419

Input: matrix = [[1,2,3],[4,5,6]]
 Output: [[1,4],[2,5],[3,6]]

Constraints:

- $m == \text{matrix.length}$
- $n == \text{matrix}[i].\text{length}$
- $1 \leq m, n \leq 1000$
- $1 \leq m * n \leq 105$
- $-109 \leq \text{matrix}[i][j] \leq 109$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def transpose(self, A: List[List[int]]) -> List[List[int]]:
        ans = [[0] * len(A) for _ in range(len(A[0]))]

        for i in range(len(A)):
            for j in range(len(A[0])):
                ans[j][i] = A[i][j]

        return ans
```

• LeetDoocs

```
class Solution:
    def transpose(self, matrix: List[List[int]]) -> List[List[int]]:
        return list(zip(*matrix))
```

• SimplyLeet

Round 4 · Mid · Easy

p.1420

```
# Python solution with clear comments

def transpose(matrix):
    # Number of rows in the original matrix
    num_rows = len(matrix)
    # Number of columns in the original matrix
    num_cols = len(matrix[0])

    # Create a new matrix with dimensions swapped: num_cols x num_rows
    transposed = [[0] * num_rows for _ in range(num_cols)]

    # Iterate through each element in the original matrix
    for i in range(num_rows):
        for j in range(num_cols):
            # Place the element at the swapped index in the transposed matrix
            transposed[j][i] = matrix[i][j]

    return transposed

# Example usage:

matrix = [[1, 2, 3], [4, 5, 6]]
print(transpose(matrix))
```

• LC.ca

```
class Solution:
    def transpose(self, matrix: List[List[int]]) -> List[List[int]]:
        return list(zip(*matrix))
```

◆ Quick Review

Key Insights

- The value at position (i, j) in the original matrix moves to (j, i) in the transposed matrix.

- The transposed matrix dimensions are the reverse of the original: if the original is $m \times n$, then the transposed is $n \times m$.
- A simple double loop is sufficient to swap the indices, ensuring $O(m*n)$ time complexity.
- This problem handles both square and rectangular matrices.

Space & Time

Time Complexity: $O(mn)$

Space Complexity: $O(mn)$ for the new matrix holding the transposed values

Solution

The solution involves iterating over each element of the original matrix and assigning it to the corresponding position in the new matrix where the indexes are swapped. We create the transposed matrix with dimensions based on the number of columns and rows of the original matrix, respectively. The approach leverages basic array manipulation with nested loops to fill in the new matrix.

LinkedList In-place Reversal

Round 4 · Mid · Easy · 2 questions

Round 4 · Mid · Easy

p.1423

LinkedList In-place Reversal

#206 Reverse Linked List

EAS

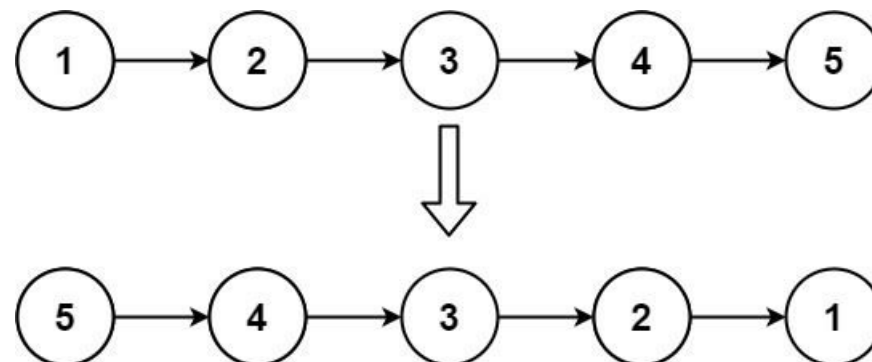
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · Recursion

Lists: Grind 169

Problem

Given the head of a singly linked list, reverse the list, and return the reversed list.

Example 1:

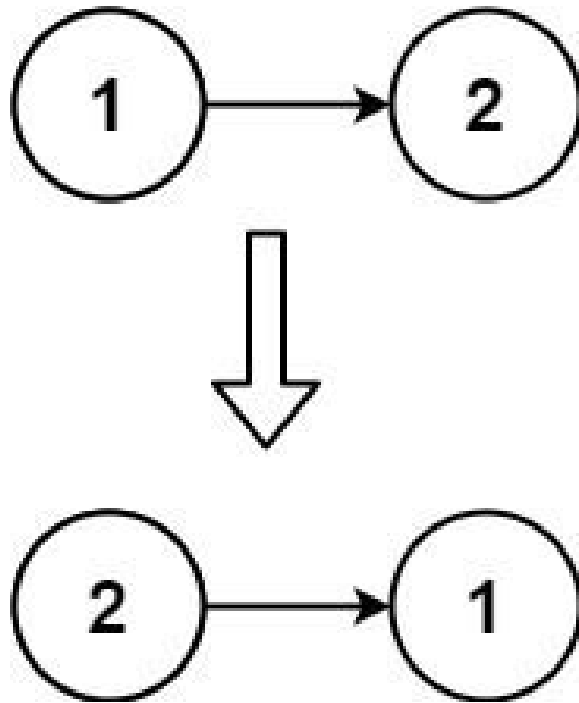


Input: head = [1,2,3,4,5]
Output: [5,4,3,2,1]

Example 2:

Round 4 · Mid · Easy

p.1424



Input: head = [1,2]
Output: [2,1]

Example 3:

Input: head = []
Output: []

Constraints:

- The number of nodes in the list is the range [0, 5000].
- $-5000 \leq \text{Node.val} \leq 5000$

Follow up: A linked list can be reversed either iteratively or recursively. Could you implement both?

Community Solutions (Python)

• WalkCC

```

class Solution:
    def reverseList(self, head: ListNode | None) -> ListNode | None:
        if not head or not head.next:
            return head

        newHead = self.reverseList(head.next)
        head.next.next = head
        head.next = None
        return newHead
  
```

```

class Solution:
    def reverseList(self, head: ListNode | None) -> ListNode | None:
        prev = None
        curr = head

        while curr:
            next = curr.next
            curr.next = prev
            prev = curr
            curr = next

        return prev
  
```

• LeetCode

```
# Definition for singly-linked list.
# class ListNode:
# def __init__(self, val=0, next=None):
# self.val = val
# self.next = next
class Solution:
    def reverseList(self, head: ListNode) -> ListNode:
        dummy = ListNode()
        curr = head
        while curr:
            next = curr.next
            curr.next = dummy.next
            dummy.next = curr
            curr = next
        return dummy.next
```

• SimplyLeet

```
# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val # Store the node value
        self.next = next # Pointer to the next node

# Iterative approach to reverse the linked list.
def reverseList(head):
    prev = None # Initially, there is no previous node
    current = head # Start with the head of the list

    while current:
        next_node = current.next # Temporarily store the next node
        current.next = prev # Reverse the current node's pointer
        prev = current # Move previous pointer one step forward
        current = next_node # Move to the next node in the list
    return prev # The new head of the reversed list

# Recursive approach to reverse the linked list.
def reverseListRecursive(head):
    # Base case: empty list or single node
```

```
if not head or not head.next:
    return head
new_head = reverseListRecursive(head.next) # Reverse the rest of the list
head.next.next = head # Reverse the pointer for the current node
head.next = None # Set the next pointer of the current node to None
return new_head # Return the new head of the reversed list
```

• LC.ca

```
# Definition for singly-linked list.
# class ListNode:
# def __init__(self, val=0, next=None):
# self.val = val
# self.next = next
class Solution:
    def reverseList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        pre, p = None, head
        while p:
            q = p.next

            p.next = pre
            pre = p
            p = q
        return pre

class Solution(object): # recursive
    def reverseList(self, root):
        if not root or not root.next:
            return root

        ret = self.reverseList(root.next)
        root.next.next = root # root.next is end of the newly reversed list ret
        root.next = None
        return ret

class Solution:
    def reverseList(self, head: ListNode) -> ListNode:
        dummy = ListNode()
```

```

curr = head
while curr:
    next = curr.next
    curr.next = dummy.next
    dummy.next = curr
    curr = next
return dummy.next

```

◆ Quick Review

Key Insights

- The problem can be solved using either an iterative or recursive approach.
- Iterative method uses constant space by updating pointers in-place.
- Recursive method leverages the call stack to reverse the list, which could use $O(n)$ space in the worst case.
- Both approaches traverse each node exactly once, yielding linear time complexity.

Space & Time

Time Complexity: $O(n)$ for both iterative and recursive approaches, where n is the number of nodes.

Space Complexity: $O(1)$ for the iterative approach and $O(n)$ for the recursive approach because of the recursion call stack.

Solution

We can solve the problem by iterating through the list while reversing the next pointers of each node. We maintain two pointers (previous and current) and update them as we traverse the list, so that by the end, the previous pointer is at the new head of the reversed list. Alternatively, a recursive approach reverses the rest of the list first and then adjusts the pointers so that the original head becomes the tail.

LinkedList In-place Reversal

#234 Palindrome Linked List

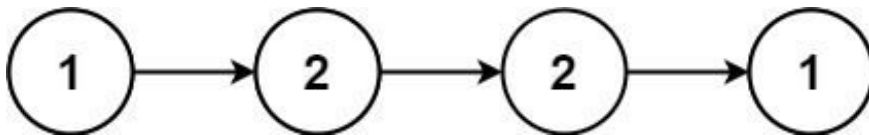
EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Linked List · Two Pointers · Recursion
 Lists: Grind 169

Problem

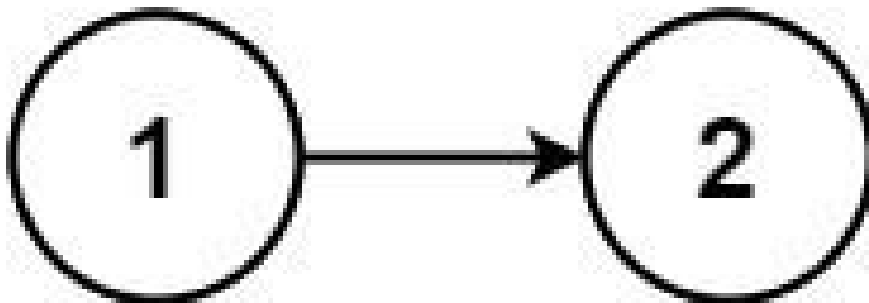
Given the head of a singly linked list, return true if it is a palindrome or false otherwise.

Example 1:



Input: head = [1,2,2,1]
 Output: true

Example 2:



Round 4 · Mid · Easy

p.1431

Input: head = [1,2]
 Output: false

Constraints:

- The number of nodes in the list is in the range [1, 105].
- $0 \leq \text{Node.val} \leq 9$

Follow up: Could you do it in $O(n)$ time and $O(1)$ space?

Community Solutions (Python)

• WalkCC

```

class Solution:
    def isPalindrome(self, head: ListNode) -> bool:
        def reverseList(head: ListNode) -> ListNode:
            prev = None
            curr = head
            while curr:
                next = curr.next
                curr.next = prev
                prev = curr
                curr = next

            return prev

        slow = head
        fast = head

        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next

        if fast:

```

Round 4 · Mid · Easy

p.1432


```

    slow = slow.next
    slow = reverseList(slow)

    while slow:
        if slow.val != head.val:
            return False
        slow = slow.next
        head = head.next

    return True

```

• LeetCode

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def isPalindrome(self, head: Optional[ListNode]) -> bool:
        slow, fast = head, head.next
        while fast and fast.next:
            slow, fast = slow.next, fast.next.next

        pre, cur = None, slow.next
        while cur:
            t = cur.next
            cur.next = pre
            pre, cur = cur, t
        while pre:
            if pre.val != head.val:
                return False
            pre, head = pre.next, head.next
        return True

```

• SimplyLeet

Round 4 · Mid · Easy

p.1433

```

# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def isPalindrome(self, head: ListNode) -> bool:
        # Find the midpoint using slow and fast pointers
        slow, fast = head, head

        while fast and fast.next:
            slow = slow.next          # Move slow pointer by 1
            fast = fast.next.next      # Move fast pointer by 2

        # Reverse the second half of the list
        prev = None
        while slow:
            next_node = slow.next      # Store the next node
            slow.next = prev           # Reverse the current node's pointer
            prev = slow                # Move prev to current node

            slow = next_node            # Move to next node

        # Compare the first half and the reversed second half node by node
        left, right = head, prev
        while right:                    # Only need to compare till the end of
            second half
            if left.val != right.val:
                return False           # Mismatch found, not a palindrome
            left = left.next            # Move left pointer
            right = right.next          # Move right pointer

        return True                    # All nodes matched, it's a palindrome

```

• LC.ca

Round 4 · Mid · Easy

p.1434

```
# Definition for singly-linked list.
# class ListNode:
# def __init__(self, val=0, next=None):
# self.val = val
# self.next = next
class Solution:
    def isPalindrome(self, head: Optional[ListNode]) -> bool:
        slow, fast = head, head.next
        while fast and fast.next:
            slow, fast = slow.next, fast.next.next

        pre, cur = None, slow.next
        while cur:
            t = cur.next
            cur.next = pre
            pre, cur = cur, t
        while pre:
            if pre.val != head.val:
                return False
            pre, head = pre.next, head.next
        return True
```

◆ Quick Review

Key Insights

- Use two pointers (slow and fast) to reach the middle of the list.
- Reverse the second half of the list.
- Compare the nodes from the start of the list to the reversed half.
- This approach meets the $O(n)$ time and $O(1)$ space follow-up requirement.

Space & Time

Round 4 · Mid · Easy

p.1435

Time Complexity: $O(n)$ – each node is visited a constant number of times.

Space Complexity: $O(1)$ – only constant extra space is used (in-place reversal).

Solution

We first use the two-pointer technique to locate the midpoint of the linked list. The slow pointer moves one step at a time while the fast pointer moves two steps. Once the middle is found, we reverse the second half of the list in-place. Finally, we traverse both halves concurrently to compare the node values. If all corresponding values match, the linked list is a palindrome. A potential gotcha is handling the odd number of nodes; in that case, we ignore the middle element when comparing.

Round 4 · Mid · Easy

p.1436

Fast and Slow Pointers

Round 4 · Mid · Easy · 3 questions

Round 4 · Mid · Easy

p.1437

Fast and Slow Pointers

☐ #141 Linked List Cycle

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Linked List · Two Pointers
Lists: Grind 169

Problem

Given head, the head of a linked list, determine if the linked list has a cycle in it.

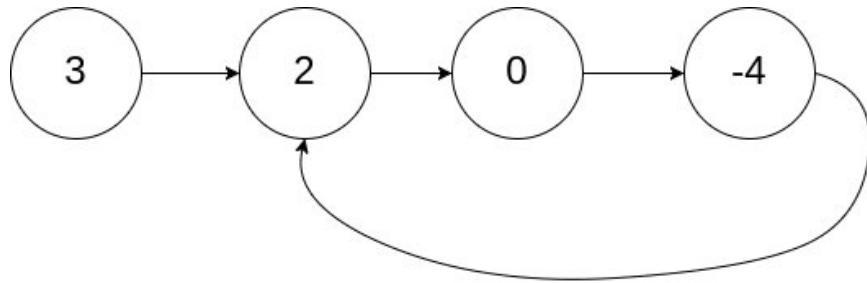
There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the next pointer.

Internally, pos is used to denote the index of the node that tail's next pointer is connected to. Note that pos is not passed as a parameter. Return true if there is a cycle in the linked list. Otherwise, return false.

Example 1:

Round 4 · Mid · Easy

p.1438

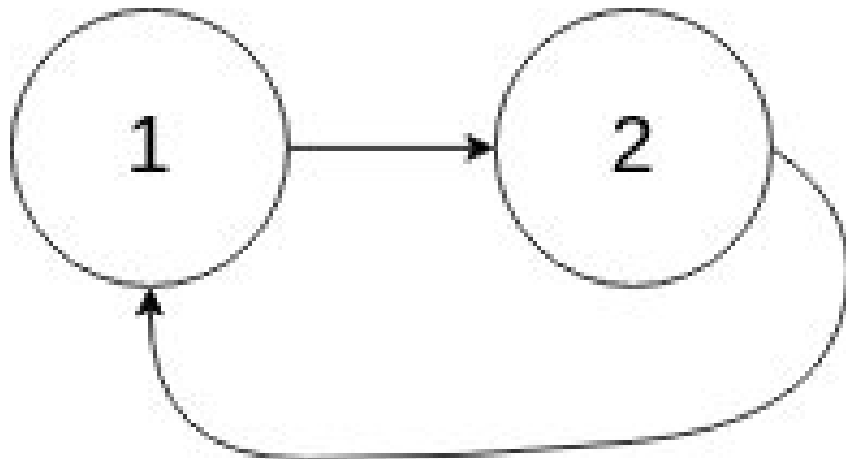


Input: head = [3,2,0,-4], pos = 1

Output: true

Explanation: There is a cycle in the linked list, where the tail connects to the 1st node (0-indexed).

Example 2:



Input: head = [1,2], pos = 0

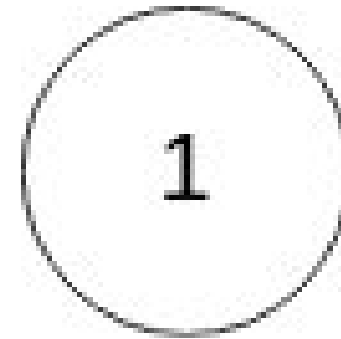
Output: true

Explanation: There is a cycle in the linked list, where the tail connects to the 0th node.

Example 3:

Round 4 · Mid · Easy

p.1439



Input: head = [1], pos = -1

Output: false

Explanation: There is no cycle in the linked list.

Constraints:

- The number of the nodes in the list is in the range [0, 104].
- $-105 \leq \text{Node.val} \leq 105$
- pos is -1 or a valid index in the linked-list.

Follow up: Can you solve it using O(1) (i.e. constant) memory?

Community Solutions (Python)

- WalkCC

Round 4 · Mid · Easy

p.1440

```
class Solution:
    def hasCycle(self, head: ListNode) -> bool:
        slow = head
        fast = head

        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
            if slow == fast:
                return True

        return False
```

• LeetCode

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, x):
#         self.val = x
#         self.next = None

class Solution:
    def hasCycle(self, head: Optional[ListNode]) -> bool:
        s = set()

        while head:
            if head in s:
                return True
            s.add(head)
            head = head.next
        return False
```

• SimplyLeet

Round 4 · Mid · Easy

p.1441

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next

class Solution:
    def hasCycle(self, head: Optional[ListNode]) -> bool:
        # Initialize two pointers, slow and fast, starting from the head.
        slow = head
        fast = head

        # Traverse the list until fast pointer reaches the end.
        while fast and fast.next:
            slow = slow.next           # Move slow pointer one step.
            fast = fast.next.next      # Move fast pointer two steps.

            # If both pointers meet, there is a cycle.
            if slow == fast:
                return True

        # If loop terminates, there is no cycle.
        return False
```

• LC.ca

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, x):
#         self.val = x
#         self.next = None

class Solution:
    def hasCycle(self, head: ListNode) -> bool:
        slow = fast = head
        while fast and fast.next:
            slow, fast = slow.next, fast.next.next
            if slow == fast:
                return True
        return False
```

Round 4 · Mid · Easy

p.1442

◆ Quick Review

Key Insights

- Use two pointers (slow and fast) to traverse the list.
- The fast pointer moves two steps at a time and the slow pointer moves one step.
- If the fast pointer ever meets the slow pointer, a cycle exists.
- If the fast pointer reaches the end of the list (null), the list has no cycle.

Space & Time

Time Complexity: $O(n)$ where n is the number of nodes in the linked list.

Space Complexity: $O(1)$ as only two pointers are used.

Solution

We use the Floyd's Cycle Detection algorithm (Tortoise and Hare approach). Two pointers, slow and fast, are initialized at the head of the list. The slow pointer moves one step at a time while the fast pointer moves two steps. If the linked list has a cycle, the fast pointer will

eventually meet the slow pointer. If the fast pointer reaches the end (i.e., a null pointer), the linked list does not have a cycle.

Fast and Slow Pointers

□ #202 Happy Number

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Math · Two Pointers
Lists: NeetCode 150

Problem

Write an algorithm to determine if a number n is happy.

A happy number is a number defined by the following process:

- Starting with any positive integer, replace the number by the sum of the squares of its digits.
- Repeat the process until the number equals 1 (where it will stay), or it loops endlessly in a cycle which does not include 1.
- Those numbers for which this process ends in 1 are happy.

Return true if n is a happy number, and false if not.

Example 1:

Round 4 · Mid · Easy

p.1445

Input: $n = 19$
Output: true
Explanation:
 $1^2 + 9^2 = 82$
 $8^2 + 2^2 = 68$
 $6^2 + 8^2 = 100$
 $1^2 + 0^2 + 0^2 = 1$

Example 2:

Input: $n = 2$
Output: false

Constraints:

- $1 \leq n \leq 2^{31} - 1$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isHappy(self, n: int) -> bool:
        def squaredSum(n: int) -> bool:
            summ = 0
            while n > 0:
                summ += pow(n % 10, 2)
                n //= 10
            return summ

        slow = squaredSum(n)

        fast = squaredSum(squaredSum(n))

        while slow != fast:
            slow = squaredSum(slow)
            fast = squaredSum(squaredSum(fast))

        return slow == 1
```

• LeetDoocs

Round 4 · Mid · Easy

p.1446

```
class Solution:
    def isHappy(self, n: int) -> bool:
        vis = set()
        while n != 1 and n not in vis:
            vis.add(n)
            x = 0
            while n:
                n, v = divmod(n, 10)
                x += v * v
            n = x
```

```
        return n == 1
```

```
class Solution:
    def isHappy(self, n: int) -> bool:
        def next(x):
            y = 0
            while x:
                x, v = divmod(x, 10)
                y += v * v
            return y
```

```
        slow, fast = n, next(n)
```

```
        while slow != fast:
            slow, fast = next(slow), next(next(fast))
        return slow == 1
```

• SimplyLeet

```
# Function to compute the sum of squares of digits
def sum_of_squares(num):
    total = 0
    while num:
        digit = num % 10      # Extract the last digit
        total += digit * digit # Add its square to the total
        num //= 10           # Remove the last digit
    return total

def isHappy(n):
```

```
seen = set()          # Initialize a set to keep track of seen numbers
# Iterate until n becomes 1 or we detect a cycle
while n != 1 and n not in seen:
    seen.add(n)        # Add current n to the set
    n = sum_of_squares(n) # Compute new n as the sum of squares of digits
return n == 1         # Return True if n is 1, else False
```

```
# Example usage:
```

```
print(isHappy(19)) # Expected output: True
```

```
print(isHappy(2))  # Expected output: False
```


Fast and Slow Pointers

#876 Middle of the Linked List

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Linked List · Two Pointers
 Lists: Grind 169

Problem

Given the head of a singly linked list, return the middle node of the linked list.

If there are two middle nodes, return the second middle node.

Example 1:

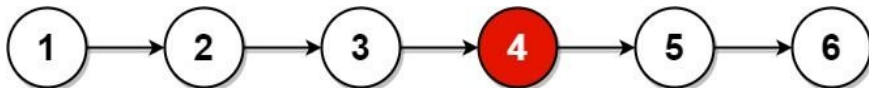


Input: head = [1,2,3,4,5]

Output: [3,4,5]

Explanation: The middle node of the list is node 3.

Example 2:



Round 4 · Mid · Easy

p.1449

Input: head = [1,2,3,4,5,6]

Output: [4,5,6]

Explanation: Since the list has two middle nodes with values 3 and 4, we return the second one.

Constraints:

- The number of nodes in the list is in the range [1, 100].
- $1 \leq \text{Node.val} \leq 100$

Community Solutions (Python)

• WalkCC

```

class Solution:
    def middleNode(self, head: ListNode) -> ListNode:
        slow = head
        fast = head

        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next

        return slow
  
```

• LeetDoocs

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def middleNode(self, head: ListNode) -> ListNode:
        slow = fast = head
        while fast and fast.next:
            slow, fast = slow.next, fast.next.next

        return slow
  
```

Round 4 · Mid · Easy

p.1450

• SimplyLeet

```
# Definition for singly-linked list.
# class ListNode:
# def __init__(self, val=0, next=None):
# self.val = val
# self.next = next

class Solution:
    def middleNode(self, head: ListNode) -> ListNode:
        # Initialize slow and fast pointers at the head of the list
        slow = head

        fast = head
        # Loop until fast reaches the end of the list
        while fast and fast.next:
            # Move slow pointer one step
            slow = slow.next
            # Move fast pointer two steps
            fast = fast.next.next
        # When loop ends, slow pointer points to the middle node
        return slow
```

• LC.ca

```
# Definition for singly-linked list.
# class ListNode:
# def __init__(self, val=0, next=None):
# self.val = val
# self.next = next
class Solution:
    def middleNode(self, head: ListNode) -> ListNode:
        slow = fast = head
        while fast and fast.next:
            slow, fast = slow.next, fast.next.next
        return slow
```

◆ Quick Review

Key Insights

Round 4 · Mid · Easy

p.1451

- Use two pointers (slow and fast) to traverse the list.
- The fast pointer moves two steps at a time while the slow pointer moves one step.
- When the fast pointer reaches the end, the slow pointer will be at the middle node.
- This approach naturally handles both odd and even lengths of the list.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes in the list.

Space Complexity: $O(1)$, constant extra space.

Solution

The solution uses the two-pointer technique:

- Initialize two pointers, slow and fast, at the head.
- Traverse the list, moving slow by one step and fast by two steps during each iteration.
- When fast reaches the end or has no next node, slow will be at the middle.
- Return the slow pointer (the middle node).

This strategy ensures linear time complexity and constant space, efficiently finding the middle node even for lists with an even

Round 4 · Mid · Easy

p.1452

number of elements.

Monotonic Stack

Round 4 · Mid · Easy · 2 questions

Round 4 · Mid · Easy

p.1453

Round 4 · Mid · Easy

p.1454

Monotonic Stack

☐ #496 Next Greater Element I

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Stack · Monotonic Stack
Lists: AlgoMaster 600

Problem

The next greater element of some element x in an array is the first greater element that is to the right of x in the same array.

You are given two distinct 0-indexed integer arrays `nums1` and `nums2`, where `nums1` is a subset of `nums2`.

For each $0 \leq i < \text{nums1.length}$, find the index j such that `nums1[i] == nums2[j]` and determine the next greater element of `nums2[j]` in `nums2`. If there is no next greater element, then the answer for this query is `-1`.

Return an array `ans` of length `nums1.length` such that `ans[i]` is the next greater element as described above.

Example 1:

Round 4 · Mid · Easy

p.1455

Input: `nums1 = [4,1,2]`, `nums2 = [1,3,4,2]`

Output: `[-1,3,-1]`

Explanation: The next greater element for each value of `nums1` is as follows:

- 4 is underlined in `nums2 = [1,3,4,2]`. There is no next greater element, so the answer is `-1`.
- 1 is underlined in `nums2 = [1,3,4,2]`. The next greater element is 3.
- 2 is underlined in `nums2 = [1,3,4,2]`. There is no next greater element, so the answer is `-1`.

Example 2:

Input: `nums1 = [2,4]`, `nums2 = [1,2,3,4]`

Output: `[3,-1]`

Explanation: The next greater element for each value of `nums1` is as follows:

- 2 is underlined in `nums2 = [1,2,3,4]`. The next greater element is 3.
- 4 is underlined in `nums2 = [1,2,3,4]`. There is no next greater element, so the answer is `-1`.

Constraints:

- $1 \leq \text{nums1.length} \leq \text{nums2.length} \leq 1000$
- $0 \leq \text{nums1}[i], \text{nums2}[i] \leq 10^4$
- All integers in `nums1` and `nums2` are unique.
- All the integers of `nums1` also appear in `nums2`.

Follow up: Could you find an $O(\text{nums1.length} + \text{nums2.length})$ solution?

Community Solutions (Python)

- WalkCC

Round 4 · Mid · Easy

p.1456

```
class Solution:
    def nextGreaterElement(self, nums1: list[int], nums2: list[int]) -> list[int]:
        numToNextGreater = {}
        stack = [] # a decreasing stack

        for num in nums2:
            while stack and stack[-1] < num:
                numToNextGreater[stack.pop()] = num
            stack.append(num)

        return [numToNextGreater.get(num, -1) for num in nums1]
```

• LeetCode

```
class Solution:
    def nextGreaterElement(self, nums1: List[int], nums2: List[int]) -> List[int]:
        stk = []
        d = {}
        for x in nums2[::-1]:
            while stk and stk[-1] < x:
                stk.pop()
            if stk:
                d[x] = stk[-1]
            stk.append(x)

        return [d.get(x, -1) for x in nums1]
```

• SimplyLeet

Round 4 · Mid · Easy

p.1457

```
# Python solution for Next Greater Element I

def nextGreaterElement(nums1, nums2):
    # Create a dictionary to hold the next greater element for each number in nums2
    next_greater = {}
    # Initialize an empty stack to keep track of numbers for which we haven't found the next greater element yet
    stack = []

    # Iterate over every number in nums2
    for num in nums2:
        # While the current number is greater than the last number in stack,
        # it is the next greater element for that number.
        while stack and num > stack[-1]:
            previous_num = stack.pop()
            next_greater[previous_num] = num
        # Push the current number onto the stack
        stack.append(num)

    # For numbers left in the stack, no next greater element exists so assign -1
    while stack:
        previous_num = stack.pop()
        next_greater[previous_num] = -1

    # Build the result array for nums1 using the hash map
    return [next_greater[x] for x in nums1]

# Example usage
nums1 = [4,1,2]
nums2 = [1,3,4,2]
print(nextGreaterElement(nums1, nums2))
```

Round 4 · Mid · Easy

p.1458

Monotonic Stack

#1475 Final Prices With a Special Discount in a Shop

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Stack · Monotonic Stack
Lists: AlgoMaster 600

Problem

You are given an integer array `prices` where `prices[i]` is the price of the i th item in a shop. There is a special discount for items in the shop. If you buy the i th item, then you will receive a discount equivalent to `prices[j]` where j is the minimum index such that $j > i$ and `prices[j] ≤ prices[i]`. Otherwise, you will not receive any discount at all.

Return an integer array `answer` where `answer[i]` is the final price you will pay for the i th item of the shop, considering the special discount.

Example 1:

Round 4 · Mid · Easy

p.1459

Input: `prices = [8,4,6,2,3]`

Output: `[4,2,4,2,3]`

Explanation:

For item 0 with `price[0]=8` you will receive a discount equivalent to `prices[1]=4`, therefore, the final price you will pay is $8 - 4 = 4$.
For item 1 with `price[1]=4` you will receive a discount equivalent to `prices[3]=2`, therefore, the final price you will pay is $4 - 2 = 2$.
For item 2 with `price[2]=6` you will receive a discount equivalent to `prices[3]=2`, therefore, the final price you will pay is $6 - 2 = 4$.
For items 3 and 4 you will not receive any discount at all.

Example 2:

Input: `prices = [1,2,3,4,5]`

Output: `[1,2,3,4,5]`

Explanation: In this case, for all items, you will not receive any discount at all.

Example 3:

Input: `prices = [10,1,1,6]`

Output: `[9,0,1,6]`

Constraints:

- $1 \leq \text{prices.length} \leq 500$
- $1 \leq \text{prices}[i] \leq 1000$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def finalPrices(self, prices: list[int]) -> list[int]:
        ans = prices.copy()
        stack = []

        for i, price in enumerate(prices):
            # stack[-1] := i in the problem description.
            while stack and prices[stack[-1]] >= price:
                ans[stack.pop()] -= price
            stack.append(i)
```

Round 4 · Mid · Easy

p.1460

```
return ans
```

• LeetCode

```
class Solution:
    def finalPrices(self, prices: List[int]) -> List[int]:
        stk = []
        for i in reversed(range(len(prices))):
            x = prices[i]
            while stk and x < stk[-1]:
                stk.pop()
            if stk:
                prices[i] -= stk[-1]
            stk.append(x)
```

```
return prices
```

• SimplyLeet

```
# Python solution using a monotonic stack.

def finalPrices(prices):
    # Stack to store the indices of prices which are waiting for a discount.
    stack = []
    # Iterate over each price by its index.
    for i, price in enumerate(prices):
        # While there are indices in stack and the current price qualifies as a
        # discount
        while stack and prices[stack[-1]] >= price:
            # Pop index from the stack; current price is the discount for that
            # price.
```

```
            index = stack.pop()
            prices[index] -= price # Update the price at that index by
            # applying the discount.
            # Push the current index onto the stack.
            stack.append(i)
    return prices
```

```
# Example usage:
# print(finalPrices([8,4,6,2,3])) would output [4,2,4,2,3].
```

Round 4 · Mid · Easy

p.1461

Queues

Round 4 · Mid · Easy · 3 questions

Round 4 · Mid · Easy

p.1462

Queues

★ #346 Moving Average from Data Stream ★

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Design · Queue · Data Stream
 Lists: ★ Premium | Premium 98

Problem

Given a stream of integers and a window size, calculate the moving average of all integers in the sliding window.

Implement the `MovingAverage` class:

- `MovingAverage(int size)` Initializes the object with the size of the window size.
- `double next(int val)` Returns the moving average of the last size values of the stream.

Example 1:

Input
 ["MovingAverage", "next", "next", "next", "next"]
 [[3], [1], [10], [3], [5]]
 Output
 [null, 1.0, 5.5, 4.66667, 6.0]

Explanation
 MovingAverage movingAverage = new MovingAverage(3);

Constraints:

- $1 \leq \text{size} \leq 1000$
- $-105 \leq \text{val} \leq 105$
- At most 104 calls will be made to `next`.

Community Solutions (Python)

• WalkCC

```
class MovingAverage:
    def __init__(self, size: int):
        self.size = size
        self.sum = 0
        self.q = collections.deque()

    def next(self, val: int) -> float:
        if len(self.q) == self.size:
            self.sum -= self.q.popleft()
            self.sum += val

        self.q.append(val)
        return self.sum / len(self.q)
```

• LeetDoocs

```
class MovingAverage:
    def __init__(self, size: int):
        self.s = 0
        self.data = [0] * size
        self.cnt = 0

    def next(self, val: int) -> float:
        i = self.cnt % len(self.data)
        self.s += val - self.data[i]
```



```

        self.data[i] = val
        self.cnt += 1
        return self.s / min(self.cnt, len(self.data))

# Your MovingAverage object will be instantiated and called as such:
# obj = MovingAverage(size)
# param_1 = obj.next(val)

```

```

class MovingAverage:
    def __init__(self, size: int):
        self.n = size
        self.s = 0
        self.q = deque()

    def next(self, val: int) -> float:
        if len(self.q) == self.n:
            self.s -= self.q.popleft()
        self.q.append(val)

```

```

        self.s += val
        return self.s / len(self.q)

```

```

# Your MovingAverage object will be instantiated and called as such:
# obj = MovingAverage(size)
# param_1 = obj.next(val)

```

• SimplyLeet

```

# Python solution for Moving Average from Data Stream

from collections import deque # Import deque for efficient queue operations

class MovingAverage:
    def __init__(self, size):
        # Initialize with the maximum window size
        self.size = size # Maximum number of elements in the
        window
        self.queue = deque() # Queue to store current window values
        self.current_sum = 0 # Running sum of the current window

```

Round 4 · Mid · Easy

p.1465

```

def next(self, val):
    # Add new value to the window
    if len(self.queue) == self.size:
        # If the window is full, remove the oldest element and update the
        sum
        removed_val = self.queue.popleft() # Remove the front element
        self.current_sum -= removed_val # Subtract its value from the
        running sum
    # Append the new value to the queue and update the sum
    self.queue.append(val)
    self.current_sum += val

    # Calculate and return the moving average by dividing the running sum
    by the current window size
    return self.current_sum / len(self.queue)

```

• LC.ca

```

class MovingAverage:
    def __init__(self, size: int):
        self.arr = [0] * size
        self.s = 0
        self.cnt = 0

    def next(self, val: int) -> float:
        idx = self.cnt % len(self.arr) # circular array
        self.s += val - self.arr[idx]
        self.arr[idx] = val

        self.cnt += 1
        return self.s / min(self.cnt, len(self.arr))

```

```

# Your MovingAverage object will be instantiated and called as such:
# obj = MovingAverage(size)
# param_1 = obj.next(val)

```

```

#####

```

Round 4 · Mid · Easy

p.1466

```

from collections import deque

# with no sum variable, calculate on the fly
class MovingAverage:
    def __init__(self, size: int):
        self.queue = deque()
        self.size = size
        self.avg = 0

    def next(self, val: int) -> float:

        if len(self.queue) < self.size:
            self.queue.append(val)
            self.avg = sum(self.queue) / len(self.queue)
            return self.avg
        else:
            head = self.queue.popleft()
            self.queue.append(val)
            minus = head / self.size
            add = val / self.size
            self.avg = self.avg + add - minus

```

```

        return self.avg

```

```

#####

```

```

from collections import deque

```

```

class MovingAverage(object):

```

```

    def __init__(self, size):
        """

```

```

        Initialize your data structure here.

```

```

        :type size: int
        """

```

```

        self.windowSize = size
        self.windowSum = 0.0
        self.data = deque([])

```

```

    def next(self, val):
        """

```

```

        :type val: int

```

Round 4 · Mid · Easy

p.1467

```

        :rtype: float
        """
        self.windowSum += val
        data = self.data

        leftTop = 0
        if len(data) >= self.windowSize:
            leftTop = data.popleft()
            data.append(val)

```

```

        self.windowSum -= leftTop
        if len(data) < self.windowSize:
            return self.windowSum / len(data)
        return self.windowSum / self.windowSize

```

```

# Your MovingAverage object will be instantiated and called as such:
# obj = MovingAverage(size)
# param_1 = obj.next(val)

```

◆ Quick Review

Key Insights

- Use a queue (or circular buffer) to keep track of the current window elements.
- Maintain a running sum to quickly update the window's total when a new element comes in.
- When the window is full, subtract the value of the element that falls out as a new one is added.
- Each call to next() is executed in constant time, O(1).

Round 4 · Mid · Easy

p.1468

Space & Time

Time Complexity: $O(1)$ per call to next()

Space Complexity: $O(\text{window size})$

Solution

The solution leverages a queue data structure to store the elements of the sliding window. A running sum variable is maintained for efficient calculation of the moving average.

When a new integer is added:

– If the size of the queue is equal to the maximum window size, remove the oldest element and subtract its value from the running sum. – Add the new element to both the queue and the running sum. – Compute and return the moving average as the running sum divided by the number of elements in the queue. This approach ensures that each update is done in constant time without having to sum all elements in the window repeatedly.

Queues

□ #933 Number of Recent Calls

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Design · Queue · Data Stream
Lists: AlgoMaster 600

Problem

You have a RecentCounter class which counts the number of recent requests within a certain time frame.

Implement the RecentCounter class:

- RecentCounter() Initializes the counter with zero recent requests.
- int ping(int t) Adds a new request at time t, where t represents some time in milliseconds, and returns the number of requests that has happened in the past 3000 milliseconds (including the new request). Specifically, return the number of requests that have happened in the inclusive range $[t - 3000, t]$.

It is guaranteed that every call to ping uses a strictly larger value of t than the previous call.

Example 1:

Input
 ["RecentCounter", "ping", "ping", "ping", "ping"]
 [[], [1], [100], [3001], [3002]]

Output
 [null, 1, 2, 3, 3]

Explanation
 RecentCounter recentCounter = new RecentCounter();

Constraints:

- $1 \leq t \leq 109$
- Each test case will call ping with strictly increasing values of t.
- At most 104 calls will be made to ping.

Community Solutions (Python)

• WalkCC

```
class RecentCounter:
    def __init__(self):
        self.q = collections.deque()

    def ping(self, t: int) -> int:
        self.q.append(t)
        while self.q[0] < t - 3000:
            self.q.popleft()
        return len(self.q)
```

• LeetCode

```
class RecentCounter:
    def __init__(self):
        self.q = deque()

    def ping(self, t: int) -> int:
        self.q.append(t)
        while self.q[0] < t - 3000:
            self.q.popleft()
        return len(self.q)
```

```
# Your RecentCounter object will be instantiated and called as such:
# obj = RecentCounter()
# param_1 = obj.ping(t)
```

```
class RecentCounter:
    def __init__(self):
        self.s = []

    def ping(self, t: int) -> int:
        self.s.append(t)
        return len(self.s) - bisect_left(self.s, t - 3000)
```

```
# Your RecentCounter object will be instantiated and called as such:
```

```
# obj = RecentCounter()
# param_1 = obj.ping(t)
```

• SimplyLeet

```
# Define the RecentCounter class
class RecentCounter:
    def __init__(self):
        # Initialize a list to act as the queue for storing timestamps
        self.queue = []

    def ping(self, t: int) -> int:
        # Append the new timestamp to the queue
        self.queue.append(t)
        # Remove timestamps from the queue that are older than (t - 3000)
```

```

while self.queue[0] < t - 3000:
    self.queue.pop(0) # Remove the oldest timestamp
# Return the count of timestamps in the current valid window
return len(self.queue)

```

```

# Example usage:
# recentCounter = RecentCounter()
# print(recentCounter.ping(1)) # returns 1
# print(recentCounter.ping(100)) # returns 2
# print(recentCounter.ping(3001)) # returns 3

# print(recentCounter.ping(3002)) # returns 3

```

Queues

☐ #2073 Time Needed to Buy Tickets

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Queue · Simulation
 Lists: AlgoMaster 600

Problem

There are n people in a line queuing to buy tickets, where the 0th person is at the front of the line and the $(n - 1)$ th person is at the back of the line.

You are given a 0-indexed integer array `tickets` of length n where the number of tickets that the i th person would like to buy is `tickets[i]`.

Each person takes exactly 1 second to buy a ticket. A person can only buy 1 ticket at a time and has to go back to the end of the line (which happens instantaneously) in order to buy more tickets. If a person does not have any tickets left to buy, the person will leave the line.

Return the time taken for the person initially at position k (0-indexed) to finish buying tickets.

Example 1:**Input:** tickets = [2,3,2], k = 2**Output:** 6**Explanation:**

- The queue starts as [2,3,2], where the kth person is underlined.
- After the person at the front has bought a ticket, the queue becomes [3,2,1] at 1 second.
- Continuing this process, the queue becomes [2,1,2] at 2 seconds.
- Continuing this process, the queue becomes [1,2,1] at 3 seconds.
- Continuing this process, the queue becomes [2,1] at 4 seconds. Note: the person at the front left the queue.
- Continuing this process, the queue becomes [1,1] at 5 seconds.
- Continuing this process, the queue becomes [1] at 6 seconds. The kth person has bought all their tickets, so return 6.

Example 2:**Input:** tickets = [5,1,1,1], k = 0**Output:** 8**Explanation:**

Round 4 · Mid · Easy

p.1475

- The queue starts as [5,1,1,1], where the kth person is underlined.
- After the person at the front has bought a ticket, the queue becomes [1,1,1,4] at 1 second.
- Continuing this process for 3 seconds, the queue becomes [4] at 4 seconds.
- Continuing this process for 4 seconds, the queue becomes [] at 8 seconds. The kth person has bought all their tickets, so return 8.

Constraints:

- $n == \text{tickets.length}$
- $1 \leq n \leq 100$
- $1 \leq \text{tickets}[i] \leq 100$
- $0 \leq k < n$

Community Solutions (Python)

- WalkCC

Round 4 · Mid · Easy

p.1476

```
class Solution:
    def timeRequiredToBuy(self, tickets: list[int], k: int) -> int:
        ans = 0

        for i, ticket in enumerate(tickets):
            if i <= k:
                ans += min(ticket, tickets[k])
            else:
                ans += min(ticket, tickets[k] - 1)

        return ans
```

• LeetCode

```
class Solution:
    def timeRequiredToBuy(self, tickets: List[int], k: int) -> int:
        ans = 0
        for i, x in enumerate(tickets):
            ans += min(x, tickets[k] if i <= k else tickets[k] - 1)
        return ans
```

• SimplyLeet

```
# Function to calculate the time needed for kth person to finish buying
tickets.
def time_required_to_buy(tickets, k):
    kth_tickets = tickets[k]
    time = 0
    # Loop over each person in the queue.
    for i, ticket in enumerate(tickets):
        # If the person is after kth, they only get up to kth_tickets - 1
        turns.
        if i > k:
            time += min(ticket, kth_tickets - 1)
        else:
            time += min(ticket, kth_tickets)
    return time

# Example usage:
print(time_required_to_buy([2,3,2], 2)) # Expected output: 6
```

Round 4 · Mid · Easy

p.1477

Recursion

Round 4 · Mid · Easy · 1 question

Round 4 · Mid · Easy

p.1478

Recursion

#21 Merge Two Sorted Lists

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Linked List · Recursion
 Lists: Grind 169

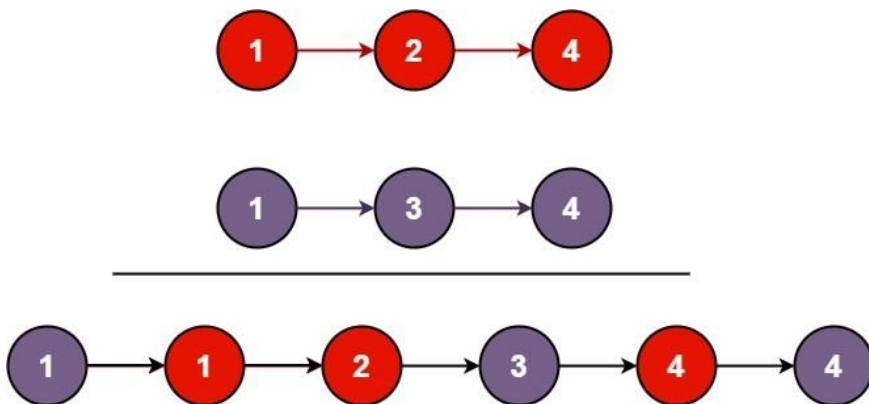
Problem

You are given the heads of two sorted linked lists list1 and list2.

Merge the two lists into one sorted list. The list should be made by splicing together the nodes of the first two lists.

Return the head of the merged linked list.

Example 1:



Round 4 · Mid · Easy

p.1479

Input: list1 = [1,2,4], list2 = [1,3,4]
 Output: [1,1,2,3,4,4]

Example 2:

Input: list1 = [], list2 = []
 Output: []

Example 3:

Input: list1 = [], list2 = [0]
 Output: [0]

Constraints:

- The number of nodes in both lists is in the range [0, 50].
- $-100 \leq \text{Node.val} \leq 100$
- Both list1 and list2 are sorted in non-decreasing order.

Community Solutions (Python)

• WalkCC

```

class Solution:
    def mergeTwoLists(
        self,
        list1: ListNode | None,
        list2: ListNode | None,
    ) -> ListNode | None:
        if not list1 or not list2:
            return list1 if list1 else list2
        if list1.val > list2.val:
            list1, list2 = list2, list1

        list1.next = self.mergeTwoLists(list1.next, list2)
        return list1
  
```

Round 4 · Mid · Easy

p.1480

• LeetCode

```
# Definition for singly-linked list.
# class ListNode:
# def __init__(self, val=0, next=None):
# self.val = val
# self.next = next
class Solution:
    def mergeTwoLists(
        self, list1: Optional[ListNode], list2: Optional[ListNode]
    ) -> Optional[ListNode]:
        if list1 is None or list2 is None:

            return list1 or list2
        if list1.val <= list2.val:
            list1.next = self.mergeTwoLists(list1.next, list2)
            return list1
        else:
            list2.next = self.mergeTwoLists(list1, list2.next)
            return list2
```

• SimplyLeet

```
# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def mergeTwoLists(list1: ListNode, list2: ListNode) -> ListNode:
    # Create a dummy node to serve as the start of the merged list
    dummy = ListNode(-1)
    current = dummy
```

Round 4 · Mid · Easy

p.1481

```
# Iterate while both lists have nodes
while list1 and list2:
    # Compare the values of the current nodes of both lists
    if list1.val <= list2.val:
        current.next = list1 # Link the smaller node to the merged list
        list1 = list1.next # Move the pointer in list1
    else:
        current.next = list2
        list2 = list2.next

    current = current.next # Move the pointer in the merged list

# Once one list is exhausted, attach the remaining nodes from the other
list
if list1:
    current.next = list1
else:
    current.next = list2

# The merged list head is at dummy.next
return dummy.next
```

• LC.ca

```
# Definition for singly-linked list.
# class ListNode:
# def __init__(self, val=0, next=None):
# self.val = val
# self.next = next
# without ops after while loop
class Solution:
    def mergeTwoLists(self, list1: Optional[ListNode], list2: Optional[ListNode]) -> Optional[ListNode]:
        dummy = ListNode()
        current = dummy
```

Round 4 · Mid · Easy

p.1482

```

while list1 or list2:
    v1 = list1.val if list1 else float('inf')
    v2 = list2.val if list2 else float('inf')

    if v1 < v2:
        current.next = list1
        list1 = list1.next
    else:
        current.next = list2
        list2 = list2.next

    current = current.next

return dummy.next

#####

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def mergeTwoLists(
        self, list1: Optional[ListNode], list2: Optional[ListNode]
    ) -> Optional[ListNode]:
        dummy = ListNode()
        curr = dummy
        while list1 and list2:
            if list1.val <= list2.val:
                curr.next = list1
                list1 = list1.next
            else:
                curr.next = list2
                list2 = list2.next
            curr = curr.next
        curr.next = list1 or list2
        return dummy.next

```

Round 4 · Mid · Easy

p.1483

```

better than:

    if list1:
        current.next = list1
    if list2:
        current.next = list2

#####

# recursion
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:

    def mergeTwoLists(self, l1: ListNode, l2: ListNode) -> ListNode:
        if not l1: # If l1 is empty, return l2
            return l2
        if not l2: # If l2 is empty, return l1
            return l1

        if l1.val < l2.val:
            l1.next = self.mergeTwoLists(l1.next, l2)
            return l1
        else:
            l2.next = self.mergeTwoLists(l1, l2.next)
            return l2

```

◆ Quick Review

Key Insights

- Leverage the fact that both lists are sorted.

Round 4 · Mid · Easy

p.1484

- Use two pointers (one for each list) to compare nodes sequentially.
- Create a dummy node to simplify edge cases and build the resulting list.
- Iterate until one list is exhausted, then append the remaining nodes from the other list.
- There is also a recursive approach that can simplify the logic but may use more stack space.

Space & Time

Time Complexity: $O(n + m)$, where n and m are the lengths of the two lists.

Space Complexity: $O(1)$ for the iterative solution ($O(n + m)$ stack space for the recursive solution).

Solution

We use an iterative approach with a dummy node to merge two sorted linked lists. Two pointers traverse list1 and list2; at each step, the node with the smaller value is appended to our merged list. This process repeats until one of the lists is exhausted, after which we append the remaining nodes from the other

list. This approach works in one pass over both lists, ensuring linear time complexity and constant extra space usage.

Divide and Conquer

Round 4 · Mid · Easy · 1 question

Round 4 · Mid · Easy

p.1487

Divide and Conquer

□ #1763 Longest Nice Substring

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Divide and Conquer · Bit
Manipulation · Sliding Window
Lists: AlgoMaster 600

Problem

A string s is nice if, for every letter of the alphabet that s contains, it appears both in uppercase and lowercase. For example, "abABB" is nice because 'A' and 'a' appear, and 'B' and 'b' appear. However, "abA" is not because 'b' appears, but 'B' does not.

Given a string s , return the longest substring of s that is nice. If there are multiple, return the substring of the earliest occurrence. If there are none, return an empty string.

Example 1:

Input: $s = \text{"YazaAay"}$
Output: "aAa"
Explanation: "aAa" is a nice string because 'A/a' is the only letter of the alphabet in s , and both 'A' and 'a' appear.
"aAa" is the longest nice substring.

Round 4 · Mid · Easy

p.1488

Example 2:

Input: s = "Bb"

Output: "Bb"

Explanation: "Bb" is a nice string because both 'B' and 'b' appear. The whole string is a substring.

Example 3:

Input: s = "c"

Output: ""

Explanation: There are no nice substrings.

Constraints:

- $1 \leq s.length \leq 100$
- s consists of uppercase and lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def longestNiceSubstring(self, s: str) -> str:
        if len(s) < 2:
            return ''

        seen = set(s)

        for i, c in enumerate(s):
            # If both upper and lower case letters exists in the string, keep moving,
            # else take the erroneous character as a partition and check for its left

            # and right parts to be nice strings.
            if c.swapcase() not in seen:
                prefix = self.longestNiceSubstring(s[:i])
                suffix = self.longestNiceSubstring(s[i + 1:])
                return max(prefix, suffix, key=len)

        return s
```

Round 4 · Mid · Easy

p.1489

• LeetCode

```
class Solution:
    def longestNiceSubstring(self, s: str) -> str:
        n = len(s)
        ans = ''
        for i in range(n):
            ss = set()
            for j in range(i, n):
                ss.add(s[j])
                if (
                    all(c.lower() in ss and c.upper() in ss for c in ss)
                    and len(ans) < j - i + 1
                ):
                    ans = s[i : j + 1]
        return ans
```

• SimplyLeet

```
# Python solution for Longest Nice Substring
def longestNiceSubstring(s: str) -> str:
    # Helper function to check if a substring is nice
    def is_nice(sub: str) -> bool:
        # Create a set of characters in the substring
        char_set = set(sub)
        # For each character, check if its opposite case exists
        for ch in char_set:
            if ch.swapcase() not in char_set:
                return False
        return True

    # Recursive function using divide and conquer approach
    def helper(sub: str) -> str:
        # Base case: if the substring is too short, it cannot be nice
        if len(sub) < 2:
            return ''
        # For each character, if it's offending (not forming a nice pair),
        # split and solve recursively
        for i, ch in enumerate(sub):
            if ch.swapcase() not in sub:
                return max(helper(sub[:i]), helper(sub[i+1:]), key=len)
```

Round 4 · Mid · Easy

p.1490

```
        # Recurse on the left part and right part
        left = helper(sub[:i])
        right = helper(sub[i+1:])
        # Return the longer substring; if same length, left portion
        appears earlier
        return left if len(left) >= len(right) else right
    # If the loop finishes, the substring is nice
    return sub

    return helper(s)
```

```
# Example usage:
print(longestNiceSubstring("YazaAay")) # Expected output: "aAa"
```

Round 4 · Mid · Easy**p.1491**

BST / Ordered Set

Round 4 · Mid · Easy · 4 questions**Round 4 · Mid · Easy****p.1492**

BST / Ordered Set

□ ★ #270 Closest Binary Search Tree Value **EAS**

★

LeetDoocs · SimplyLeet · WalkCC · LeetCode

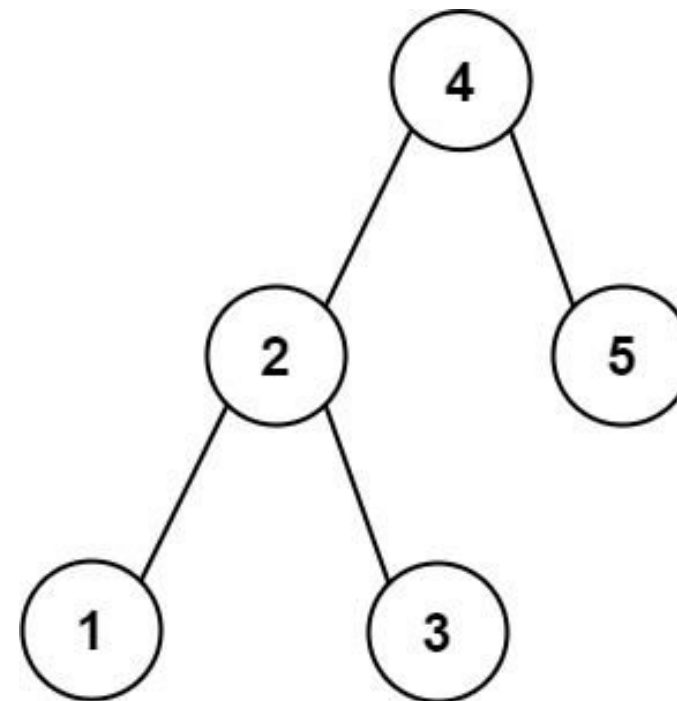
Binary Search · Tree · DFS · BST

Lists: ★ Premium | Premium 98

Problem

Given the root of a binary search tree and a target value, return the value in the BST that is closest to the target. If there are multiple answers, print the smallest.

Example 1:



Input: root = [4,2,5,1,3], target = 3.714286
Output: 4

Example 2:

Input: root = [1], target = 4.428571
Output: 1

Constraints:

- The number of nodes in the tree is in the range [1, 104].
- $0 \leq \text{Node.val} \leq 109$
- $-109 \leq \text{target} \leq 109$

Round 4 · Mid · Easy

p.1493

Round 4 · Mid · Easy

p.1494

Community Solutions (Python)

• WalkCC

```
class Solution:
    def closestValue(self, root: TreeNode | None, target: float) -> int:
        # If target < root.val, search the left subtree.
        if target < root.val and root.left:
            left = self.closestValue(root.left, target)
            if abs(left - target) <= abs(root.val - target):
                return left

        # If target > root.val, search the right subtree.
        if target > root.val and root.right:
            right = self.closestValue(root.right, target)
            if abs(right - target) < abs(root.val - target):
                return right

        return root.val
```

• LeetDoocs

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def closestValue(self, root: Optional[TreeNode], target: float) -> int:
        def dfs(node: Optional[TreeNode]):
            if node is None:
```

```
        return
        nxt = abs(target - node.val)
        nonlocal ans, diff
        if nxt < diff or (nxt == diff and node.val < ans):
            diff = nxt
            ans = node.val
        node = node.left if target < node.val else node.right
        dfs(node)
```

```
ans = 0
```

```
diff = inf
dfs(root)
return ans
```

• SimplyLeet

```
# Python solution for Closest Binary Search Tree Value
```

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
```

```
class Solution:
```

```
    def closestValue(self, root: TreeNode, target: float) -> int:
        # Initialize the best value with the value of the root
        best_value = root.val

        # Traverse the BST iteratively
        current = root
        while current:
            # Calculate the difference between current node's value and target
            current_diff = abs(current.val - target)
            best_diff = abs(best_value - target)
```



```

        # Update best_value if a closer difference is found, or if equal
        and current value is smaller
        if current_diff < best_diff or (current_diff == best_diff and curr
ent.val < best_value):
            best_value = current.val

        # Decide the traversal direction based on the target comparison
        if target < current.val:
            current = current.left
        else:
            current = current.right

    return best_value

```

• LC.ca

```

# Definition for a binary tree node.
# class TreeNode(object):
# def __init__(self, x):
# self.val = x
# self.left = None
# self.right = None

```

```

class Solution(object):
    def closestValue(self, p, target):
        """

```

```

:type root: TreeNode
:type target: float
:rtype: int
"""

```

```

ans = p.val
while p:
    if abs(target - p.val) < abs(ans - target):
        ans = p.val
    if target < p.val:
        p = p.left

```

Round 4 · Mid · Easy

p.1497

```

        else:
            p = p.right
        return ans

#####

# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):

# self.val = val
# self.left = left
# self.right = right
class Solution:
    def closestValue(self, root: Optional[TreeNode], target: float) -> int:
        ans, mi = root.val, inf
        while root:
            t = abs(root.val - target)
            if t < mi:
                mi = t

        ans = root.val
        if root.val > target:
            root = root.left
        else:
            root = root.right
        return ans

```

◆ Quick Review

Key Insights

- BST property allows us to traverse the tree efficiently by comparing the target with node values.
- We can iteratively traverse the tree, moving left if the target is smaller than the current

Round 4 · Mid · Easy

p.1498

node's value and moving right if greater.

- At each node, calculate the absolute difference between node's value and the target, and update the answer if the difference is smaller—or equal with a smaller candidate value.
- This approach minimizes the search area by leveraging the inherent ordering in BSTs.

Space & Time

Time Complexity: $O(h)$ on average, where h is the height of the tree ($O(\log n)$ for a balanced tree, and $O(n)$ in the worst-case scenario for a skewed tree).

Space Complexity: $O(1)$ for an iterative solution (or $O(h)$ if using recursion due to the call stack).

Solution

We solve the problem by performing an iterative traversal of the BST. Starting at the root, we maintain a variable to store the value closest to the target. For every node encountered, we:

- Compute the absolute difference between the node's value and the target.
- Update the

stored closest value if the computed difference is less than the current smallest difference; if the difference is the same, choose the smaller node value. – Depending on whether the target is less than or greater than the current node's value, move to the left or right child accordingly. This approach efficiently defines a search path that is consistent with the properties of a BST.

BST / Ordered Set

#653 Two Sum IV – Input is a BST

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Two Pointers · Tree · Depth-First
Search · Breadth-First Search · Binary Search
Tree · Binary Tree

Lists: AlgoMaster 600

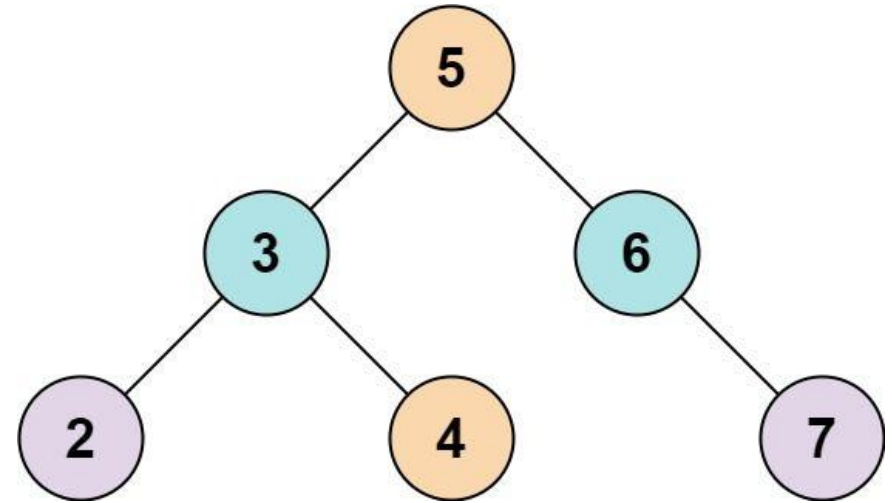
Problem

Given the root of a binary search tree and an integer k , return true if there exist two elements in the BST such that their sum is equal to k , or false otherwise.

Example 1:

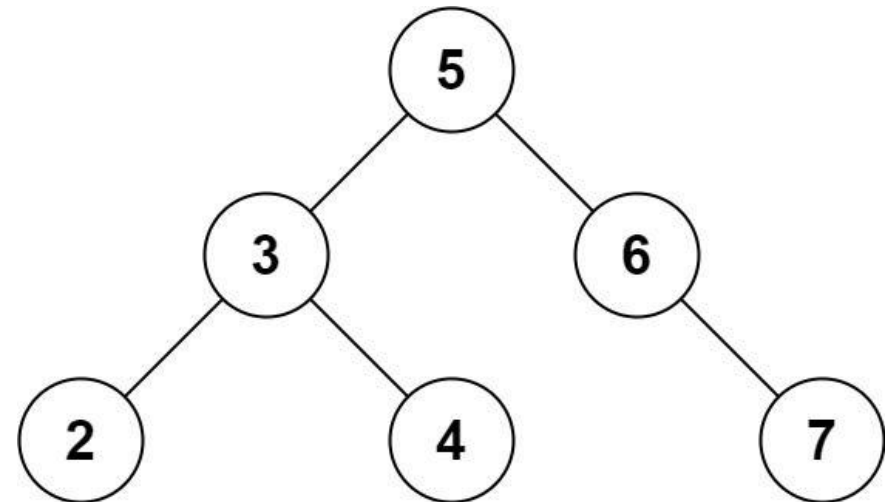
Round 4 · Mid · Easy

p.1501



Input: root = [5,3,6,2,4,null,7], k = 9
Output: true

Example 2:



Round 4 · Mid · Easy

p.1502

Input: root = [5,3,6,2,4,null,7], k = 28
Output: false

Constraints:

- The number of nodes in the tree is in the range [1, 104].
- $-104 \leq \text{Node.val} \leq 104$
- root is guaranteed to be a valid binary search tree.
- $-105 \leq k \leq 105$

Community Solutions (Python)

• WalkCC

```
class BSTIterator:
    def __init__(self, root: TreeNode | None, leftToRight: bool):
        self.stack = []
        self.leftToRight = leftToRight
        self._pushUntilNone(root)

    def next(self) -> int:
        node = self.stack.pop()
        if self.leftToRight:
            self._pushUntilNone(node.right)

        else:
            self._pushUntilNone(node.left)
        return node.val

    def _pushUntilNone(self, root: TreeNode | None):
        while root:
            self.stack.append(root)
            root = root.left if self.leftToRight else root.right
```

Round 4 · Mid · Easy

p.1503

```
class Solution:
    def findTarget(self, root: TreeNode | None, k: int) -> bool:
        if not root:
            return False

        left = BSTIterator(root, True)
        right = BSTIterator(root, False)

        l = left.next()
        r = right.next()

        while l < r:
            summ = l + r
            if summ == k:
                return True
            if summ < k:
                l = left.next()
            else:
                r = right.next()

        return False
```

• LeetCode

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def findTarget(self, root: Optional[TreeNode], k: int) -> bool:
        def dfs(root):
            if root is None:
                return False
            if k - root.val in vis:
                return True
            vis.add(root.val)
            return dfs(root.left) or dfs(root.right)

        vis = set()
        return dfs(root)
```

Round 4 · Mid · Easy

p.1504

• SimplyLeet

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def findTarget(self, root: TreeNode, k: int) -> bool:
        seen = set() # Set to store the values of visited nodes

        def dfs(node):
            if not node: # Base case: if the node is null, return False
                return False
            # Check if the complement of the current node's value exists in the set
            if (k - node.val) in seen:
                return True
            # Add the current node's value to the set
            seen.add(node.val)
            # Continue DFS traversal on left and right children

            return dfs(node.left) or dfs(node.right)

        return dfs(root) # Start DFS from the root node
```

Round 4 · Mid · Easy

p.1505

BST / Ordered Set

□ #700 Search in a Binary Search Tree

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · Binary Search Tree · Binary Tree
Lists: AlgoMaster 600

Problem

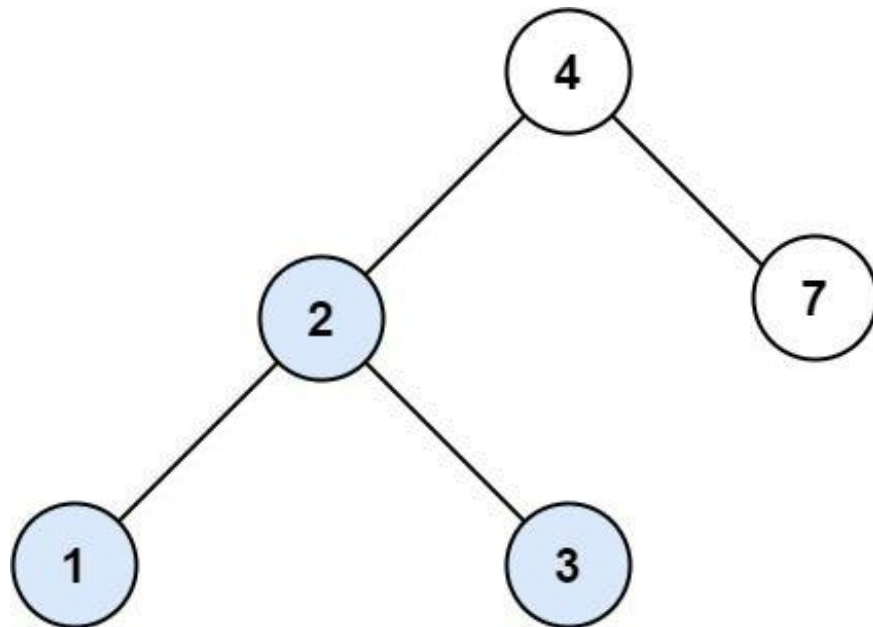
You are given the root of a binary search tree (BST) and an integer val.

Find the node in the BST that the node's value equals val and return the subtree rooted with that node. If such a node does not exist, return null.

Example 1:

Round 4 · Mid · Easy

p.1506

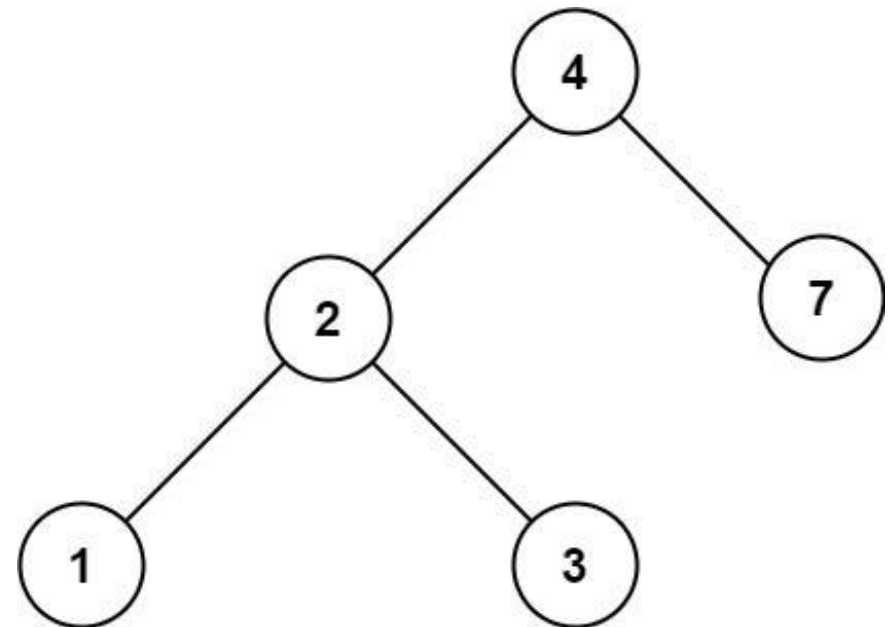


Input: root = [4,2,7,1,3], val = 2
Output: [2,1,3]

Example 2:

Round 4 · Mid · Easy

p.1507



Input: root = [4,2,7,1,3], val = 5
Output: []

Constraints:

- The number of nodes in the tree is in the range [1, 5000].
- $1 \leq \text{Node.val} \leq 107$
- root is a binary search tree.
- $1 \leq \text{val} \leq 107$

Community Solutions (Python)

- WalkCC

Round 4 · Mid · Easy

p.1508

```
class Solution:
    def searchBST(self, root: TreeNode | None, val: int) -> TreeNode | None:
        if not root:
            return None
        if root.val == val:
            return root
        if root.val > val:
            return self.searchBST(root.left, val)
        return self.searchBST(root.right, val)
```

• LeetCode

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def searchBST(self, root: Optional[TreeNode], val: int) -> Optional[TreeNode]:
        if root is None or root.val == val:
            return root

        return (
            self.searchBST(root.left, val)
            if root.val > val
            else self.searchBST(root.right, val)
        )
```

• SimplyLeet

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

    def searchBST(self, root: TreeNode, val: int) -> TreeNode:
        # Base case: if root is null or found the target value, return the root
        if not root or root.val == val:
```

```
        return root
        # If target value is less than current node's value, search in the left
        # subtree
        if val < root.val:
            return searchBST(root.left, val)
        # Otherwise, search in the right subtree
        return searchBST(root.right, val)
```

BST / Ordered Set

#938 Range Sum of BST

EAS

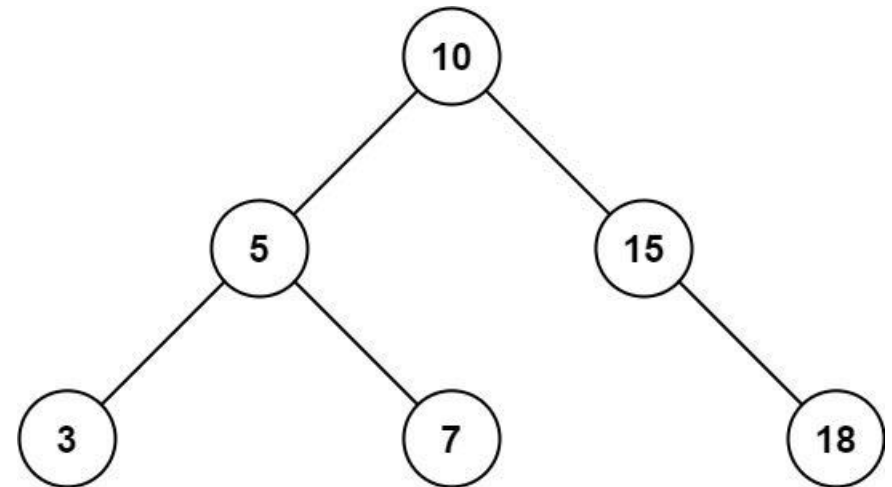
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · Depth-First Search · Binary Search Tree ·
Binary Tree

Lists: AlgoMaster 600

Problem

Given the root node of a binary search tree and two integers *low* and *high*, return the sum of values of all nodes with a value in the inclusive range *[low, high]*.

Example 1:

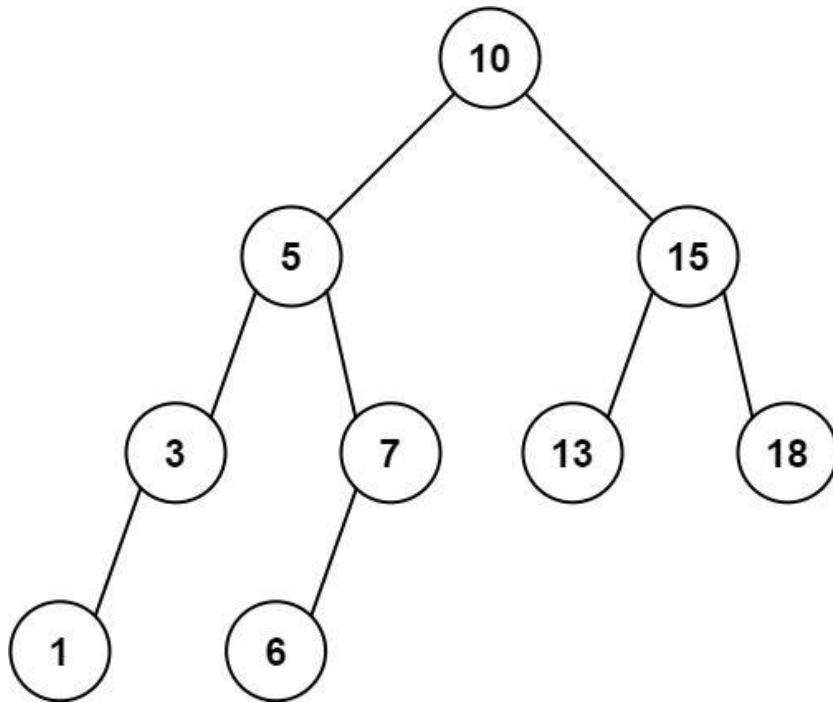


Input: root = [10,5,15,3,7,null,18], low = 7, high = 15

Output: 32

Explanation: Nodes 7, 10, and 15 are in the range [7, 15]. $7 + 10 + 15 = 32$.

Example 2:



Input: root = [10,5,15,3,7,13,18,1,null,6], low = 6, high = 10
 Output: 23
 Explanation: Nodes 6, 7, and 10 are in the range [6, 10]. 6 + 7 + 10 = 23.

Constraints:

- The number of nodes in the tree is in the range [1, 2 * 10⁴].
- 1 ≤ Node.val ≤ 105
- 1 ≤ low ≤ high ≤ 105
- All Node.val are unique.

Community Solutions (Python)

• LeetCode

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def rangeSumBST(self, root: Optional[TreeNode], low: int, high: int) -> int:
        def dfs(root: Optional[TreeNode]) -> int:
            if root is None:
                return 0
            x = root.val
            ans = x if low <= x <= high else 0
            if x > low:
                ans += dfs(root.left)
            if x < high:
                ans += dfs(root.right)
            return ans
        return dfs(root)
  
```

• SimplyLeet

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def rangeSumBST(self, root: TreeNode, low: int, high: int) -> int:
        # Base case: if the current node is None, return sum 0.
  
```

```
if not root:
    return 0

# Initialize sum for the current node.
sum_val = 0

# If the current node's value is within [low, high], add its value.
if low <= root.val <= high:
    sum_val += root.val

# If the current node's value is greater than low, explore left
subtree.
if root.val > low:
    sum_val += self.rangeSumBST(root.left, low, high)

# If the current node's value is less than high, explore right subtree.
if root.val < high:
    sum_val += self.rangeSumBST(root.right, low, high)

return sum_val
```

Round 4 · Mid · Easy

p.1515

Heaps

Round 4 · Mid · Easy · 1 question

Round 4 · Mid · Easy

p.1516

Heaps

#1046 Last Stone Weight

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Heap (Priority Queue)

Lists: NeetCode 150

Problem

You are given an array of integers `stones` where `stones[i]` is the weight of the *i*th stone.

We are playing a game with the stones. On each turn, we choose the heaviest two stones and smash them together. Suppose the heaviest two stones have weights *x* and *y* with $x \leq y$. The result of this smash is:

- If $x == y$, both stones are destroyed, and
- If $x \neq y$, the stone of weight *x* is destroyed, and the stone of weight *y* has new weight $y - x$.

At the end of the game, there is at most one stone left.

Return the weight of the last remaining stone. If there are no stones left, return 0.

Round 4 · Mid · Easy

p.1517

Example 1:

Input: `stones = [2,7,4,1,8,1]`
Output: 1
Explanation:
We combine 7 and 8 to get 1 so the array converts to `[2,4,1,1,1]` then,
we combine 2 and 4 to get 2 so the array converts to `[2,1,1,1]` then,
we combine 2 and 1 to get 1 so the array converts to `[1,1,1]` then,
we combine 1 and 1 to get 0 so the array converts to `[1]` then that's the value of the last stone.

Example 2:

Input: `stones = [1]`
Output: 1

Constraints:

- $1 \leq \text{stones.length} \leq 30$
- $1 \leq \text{stones}[i] \leq 1000$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def lastStoneWeight(self, stones: list[int]) -> int:
        pq = [-stone for stone in stones]
        heapq.heapify(pq)

        while len(pq) >= 2:
            n1 = -heapq.heappop(pq)
            n2 = -heapq.heappop(pq)
            if n1 != n2:
                heapq.heappush(pq, -(n1 - n2))

        return 0 if not pq else -pq[0]
```

• LeetDoocs

Round 4 · Mid · Easy

p.1518

```
class Solution:
    def lastStoneWeight(self, stones: List[int]) -> int:
        h = [-x for x in stones]
        heapify(h)
        while len(h) > 1:
            y, x = -heappop(h), -heappop(h)
            if x != y:
                heappush(h, x - y)
        return 0 if not h else -h[0]
```

• SimplyLeet

```
import heapq

def lastStoneWeight(stones):
    # Create a max heap by inserting negative values of stones
    max_heap = [-stone for stone in stones]
    heapq.heapify(max_heap)

    # While more than one stone remains
    while len(max_heap) > 1:
        # Get the heaviest stone (largest value, but stored as negative)

        heaviest = -heapq.heappop(max_heap)
        # Get the next heaviest stone
        next_heaviest = -heapq.heappop(max_heap)

        # If the stones are not equal, push the difference back into the heap
        if heaviest != next_heaviest:
            diff = heaviest - next_heaviest
            heapq.heappush(max_heap, -diff)

    # If there is a stone left, return its weight. Otherwise, return 0.

    return -max_heap[0] if max_heap else 0

# Example usage:
# print(lastStoneWeight([2,7,4,1,8,1]))
```

Round 4 · Mid · Easy

p.1519

Top K Elements

Round 4 · Mid · Easy · 1 question

Round 4 · Mid · Easy

p.1520

Top K Elements

□ #703 Kth Largest Element in a Stream

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode Tree · Design · Binary Search Tree · Heap (Priority Queue) · Binary Tree · Data Stream Lists: NeetCode 150

Problem

You are part of a university admissions office and need to keep track of the k th highest test score from applicants in real-time. This helps to determine cut-off marks for interviews and admissions dynamically as new applicants submit their scores.

You are tasked to implement a class which, for a given integer k , maintains a stream of test scores and continuously returns the k th highest test score after a new score has been submitted. More specifically, we are looking for the k th highest score in the sorted list of all scores.

Implement the `KthLargest` class:

- `KthLargest(int k, int[] nums)` Initializes the object with the integer k and the stream of

Round 4 · Mid · Easy

p.1521

test scores `nums`.

- `int add(int val)` Adds a new test score `val` to the stream and returns the element representing the k th largest element in the pool of test scores so far.

Example 1:

Input: `["KthLargest", "add", "add", "add", "add", "add"]` `[[3, [4, 5, 8, 2]], [3], [5], [10], [9], [4]]`

Output: `[null, 4, 5, 5, 8, 8]`

Explanation:

```
KthLargest kthLargest = new KthLargest(3, [4, 5, 8, 2]);
kthLargest.add(3); // return 4
kthLargest.add(5); // return 5
kthLargest.add(10); // return 5
kthLargest.add(9); // return 8
kthLargest.add(4); // return 8
```

Example 2:

Input: `["KthLargest", "add", "add", "add", "add"]` `[[4, [7, 7, 7, 7, 8, 3]], [2], [10], [9], [9]]`

Output: `[null, 7, 7, 7, 8]`

Explanation:

Constraints:

- $0 \leq \text{nums.length} \leq 104$
- $1 \leq k \leq \text{nums.length} + 1$

Round 4 · Mid · Easy

p.1522

- $-104 \leq \text{nums}[i] \leq 104$
- $-104 \leq \text{val} \leq 104$
- At most 104 calls will be made to add.

Community Solutions (Python)

• LeetCode

```
class KthLargest:

    def __init__(self, k: int, nums: List[int]):
        self.k = k
        self.min_q = []
        for x in nums:
            self.add(x)

    def add(self, val: int) -> int:
        heappush(self.min_q, val)

        if len(self.min_q) > self.k:
            heappop(self.min_q)
        return self.min_q[0]

# Your KthLargest object will be instantiated and called as such:
# obj = KthLargest(k, nums)
# param_1 = obj.add(val)
```

• SimplyLeet

```
import heapq

class KthLargest:
    def __init__(self, k, nums):
        # Save kth value and initialize the min-heap.
        self.k = k
        self.min_heap = nums
        heapq.heapify(self.min_heap)
        # Reduce the heap size to exactly k elements.
        while len(self.min_heap) > k:
```

```
        heapq.heappop(self.min_heap)

    def add(self, val):
        if len(self.min_heap) < self.k:
            heapq.heappush(self.min_heap, val)
        elif val > self.min_heap[0]:
            heapq.heapreplace(self.min_heap, val)
        # The kth largest element is at the heap root.
        return self.min_heap[0]
```

```
# Example usage:
# kthLargest = KthLargest(3, [4, 5, 8, 2])
# print(kthLargest.add(3)) # returns 4
# print(kthLargest.add(5)) # returns 5
# print(kthLargest.add(10)) # returns 5
# print(kthLargest.add(9)) # returns 8
# print(kthLargest.add(4)) # returns 8
```

Intervals

Round 4 · Mid · Easy · 2 questions

Round 4 · Mid · Easy

p.1525

Intervals

□ #228 Summary Ranges

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array

Lists: AlgoMaster 600

Problem

You are given a sorted unique integer array
nums.

A range [a,b] is the set of all integers from a to b
(inclusive).

Return the smallest sorted list of ranges that
cover all the numbers in the array exactly. That
is, each element of nums is covered by exactly
one of the ranges, and there is no integer x such
that x is in one of the ranges but not in nums.

Each range [a,b] in the list should be output as:

- "a->b" if a != b
- "a" if a == b

Example 1:

Round 4 · Mid · Easy

p.1526

Input: nums = [0,1,2,4,5,7]
 Output: ["0->2","4->5","7"]
 Explanation: The ranges are:
 [0,2] --> "0->2"
 [4,5] --> "4->5"
 [7,7] --> "7"

Example 2:

Input: nums = [0,2,3,4,6,8,9]
 Output: ["0","2->4","6","8->9"]
 Explanation: The ranges are:
 [0,0] --> "0"
 [2,4] --> "2->4"
 [6,6] --> "6"
 [8,9] --> "8->9"

Constraints:

- $0 \leq \text{nums.length} \leq 20$
- $-231 \leq \text{nums}[i] \leq 231 - 1$
- All the values of nums are unique.
- nums is sorted in ascending order.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def summaryRanges(self, nums: list[int]) -> list[str]:
        ans = []

        i = 0
        while i < len(nums):
            begin = nums[i]
            while i < len(nums) - 1 and nums[i] == nums[i + 1] - 1:
                i += 1
            end = nums[i]
```

```
if begin == end:
    ans.append(str(begin))
else:
    ans.append(str(begin) + "->" + str(end))
i += 1

return ans
```

• LeetCode

```
class Solution:
    def summaryRanges(self, nums: List[int]) -> List[str]:
        def f(i: int, j: int) -> str:
            return str(nums[i]) if i == j else f'{nums[i]}->{nums[j]}'

        i = 0
        n = len(nums)
        ans = []
        while i < n:
            j = i

            while j + 1 < n and nums[j + 1] == nums[j] + 1:
                j += 1
            ans.append(f(i, j))
            i = j + 1
        return ans
```

• SimplyLeet

```
# Function to summarize ranges in a sorted unique integer array.
def summary_ranges(nums):
    ranges = [] # List to store the resulting range strings
    n = len(nums)
    i = 0 # Pointer to iterate through nums

    # Iterate over the numbers in the array
    while i < n:
        start = nums[i] # Start of a new range
        # Traverse the consecutive sequence
```



```

while i + 1 < n and nums[i + 1] == nums[i] + 1:
    i += 1
end = nums[i] # End of the current range
# If start and end are the same, it's a single element range
if start == end:
    ranges.append(str(start))
else:
    ranges.append(f"{start}->{end}")
i += 1 # Move to the next number after the current range
return ranges

```

```

# Example test cases
print(summary_ranges([0,1,2,4,5,7])) # Output: ["0->2", "4->5", "7"]
print(summary_ranges([0,2,3,4,6,8,9])) # Output: ["0", "2->4", "6", "8->9"]

```

Intervals

☐ ★ #252 Meeting Rooms ★

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Sorting

Lists: ★ Premium | Grind 169 | Premium 98

Problem

Given an array of meeting time intervals where $\text{intervals}[i] = [\text{start}_i, \text{end}_i]$, determine if a person could attend all meetings.

Example 1:

Input: $\text{intervals} = [[0,30],[5,10],[15,20]]$
Output: false

Example 2:

Input: $\text{intervals} = [[7,10],[2,4]]$
Output: true

Constraints:

- $0 \leq \text{intervals.length} \leq 104$
- $\text{intervals}[i].\text{length} == 2$
- $0 \leq \text{start}_i < \text{end}_i \leq 106$

Community Solutions (Python)

- ☐ WalkCC

```
class Solution:
    def canAttendMeetings(self, intervals: list[list[int]]) -> bool:
        intervals.sort()

        for i in range(1, len(intervals)):
            if intervals[i - 1][1] > intervals[i][0]:
                return False

        return True
```

• LeetCode

```
class Solution:
    def canAttendMeetings(self, intervals: List[List[int]]) -> bool:
        intervals.sort()
        return all(a[1] <= b[0] for a, b in pairwise(intervals))
```

• SimplyLeet

```
# Python solution for checking if one can attend all meetings

def canAttendMeetings(intervals):
    # Sort intervals based on start time
    intervals.sort(key=lambda x: x[0])
    # Iterate through intervals to find any overlap
    for i in range(1, len(intervals)):
        # Check if the previous meeting ends after the current meeting starts
        if intervals[i-1][1] > intervals[i][0]:
            return False

    return True

# Example usage:
print(canAttendMeetings([[0, 30], [5, 10], [15, 20]])) # Expected output:
False
print(canAttendMeetings([[7, 10], [2, 4]])) # Expected output:
True
```

• LC.ca

Round 4 · Mid · Easy

p.1531

```
class Solution:
    def canAttendMeetings(self, intervals: List[List[int]]) -> bool:
        intervals.sort()
        return all(a[1] <= b[0] for a, b in pairwise(intervals))

#####

from itertools import pairwise
class Solution:
    def canAttendMeetings(self, intervals: List[List[int]]) -> bool:

        intervals.sort(key=lambda x: x[0])
        return not any(a[1] > b[0] for a,b in pairwise(intervals))
        # any is True, then conflict found

#####

class Solution:
    def canAttendMeetings(self, intervals: List[List[int]]) -> bool:
        intervals.sort(key=lambda x: x[0])
        for i in range(len(intervals) - 1):

            if intervals[i][1] > intervals[i + 1][0]:
                return False
            return True

#####

# Definition for an interval.
# class Interval(object):
# def __init__(self, s=0, e=0):
# self.start = s

# self.end = e

class Solution(object):
    def canAttendMeetings(self, intervals):
        """
        :type intervals: List[Interval]
        :rtype: bool
        """
        intervals = sorted(intervals, key=lambda x: x.start)
        for i in range(1, len(intervals)):
```

Round 4 · Mid · Easy

p.1532

```
if intervals[i].start < intervals[i - 1].end:  
    return False  
return True
```

◆ Quick Review

Key Insights

- Sort the intervals based on their starting times.
- Compare each interval's end time with the next interval's start time.
- Detect overlap if the end time of one meeting exceeds the start time of the following meeting.
- If no overlaps are detected, all meetings can be attended.

Space & Time

Time Complexity: $O(n \log n)$ due to sorting the intervals.

Space Complexity: $O(1)$ if sorting is done in-place; otherwise $O(n)$.

Solution

The solution starts by sorting the intervals in ascending order by their start times. After sorting, iterate through the intervals and for

each consecutive pair, check if the previous meeting's end time exceeds the next meeting's start time. An overlap indicates that the person cannot attend all meetings. The primary data structure used is an array (or list) for holding the intervals. Sorting makes it easier to compare adjacent intervals for potential overlaps.

Data Structure Design

Round 4 · Mid · Easy · 3 questions

Round 4 · Mid · Easy

p.1535

Data Structure Design

□ #225 Implement Stack using Queues

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Stack · Design · Queue

Lists: AlgoMaster 600

Problem

Implement a last-in-first-out (LIFO) stack using only two queues. The implemented stack should support all the functions of a normal stack (push, top, pop, and empty).

Implement the MyStack class:

- **void push(int x)** Pushes element x to the top of the stack.
- **int pop()** Removes the element on the top of the stack and returns it.
- **int top()** Returns the element on the top of the stack.
- **boolean empty()** Returns true if the stack is empty, false otherwise.

Notes:

Round 4 · Mid · Easy

p.1536

- You must use only standard operations of a queue, which means that only push to back, peek/pop from front, size and is empty operations are valid.
- Depending on your language, the queue may not be supported natively. You may simulate a queue using a list or deque (double-ended queue) as long as you use only a queue's standard operations.

Example 1:

Input
 ["MyStack", "push", "push", "top", "pop", "empty"]
 [[], [1], [2], [], [], []]
 Output
 [null, null, null, 2, 2, false]

Explanation
 MyStack myStack = new MyStack();

Constraints:

- $1 \leq x \leq 9$
- At most 100 calls will be made to push, pop, top, and empty.
- All the calls to pop and top are valid.

Follow-up: Can you implement the stack using only one queue?

Community Solutions (Python)

- WalkCC

Round 4 · Mid · Easy

p.1537

```
class MyStack:
    def __init__(self):
        self.q = collections.deque()

    def push(self, x: int) -> None:
        self.q.append(x)
        for _ in range(len(self.q) - 1):
            self.q.append(self.q.popleft())

    def pop(self) -> int:
        return self.q.popleft()

    def top(self) -> int:
        return self.q[0]

    def empty(self) -> bool:
        return not self.q
```

• LeetDoocs

```
class MyStack:
    def __init__(self):
        self.q1 = deque()
        self.q2 = deque()

    def push(self, x: int) -> None:
        self.q2.append(x)
        while self.q1:
            self.q2.append(self.q1.popleft())
        self.q1, self.q2 = self.q2, self.q1

    def pop(self) -> int:
        return self.q1.popleft()

    def top(self) -> int:
        return self.q1[0]

    def empty(self) -> bool:
        return len(self.q1) == 0
```

Round 4 · Mid · Easy

p.1538

```
# Your MyStack object will be instantiated and called as such:
# obj = MyStack()
# obj.push(x)
# param_2 = obj.pop()
# param_3 = obj.top()
# param_4 = obj.empty()
```

• SimplyLeet

```
from collections import deque

class MyStack:
    def __init__(self):
        # Initialize two queues using deque
        self.q1 = deque()
        self.q2 = deque()

    def push(self, x: int) -> None:
        # Push x into q2 (auxiliary queue)

        self.q2.append(x)
        # Transfer all elements from q1 to q2 to reverse the order
        while self.q1:
            self.q2.append(self.q1.popleft())
        # Swap q1 and q2 so that q1 always holds the current elements
        self.q1, self.q2 = self.q2, self.q1

    def pop(self) -> int:
        # Pop element from the front of q1 which is the top of the stack
        return self.q1.popleft()

    def top(self) -> int:
        # Peek at the front of q1 to get the top element
        return self.q1[0]

    def empty(self) -> bool:
        # Return True if q1 is empty, else False
        return not self.q1
```

Data Structure Design

□ #232 Implement Queue using Stacks

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Stack · Design · Queue
Lists: Grind 169

Problem

Implement a first in first out (FIFO) queue using only two stacks. The implemented queue should support all the functions of a normal queue (push, peek, pop, and empty).

Implement the MyQueue class:

- void push(int x) Pushes element x to the back of the queue.
- int pop() Removes the element from the front of the queue and returns it.
- int peek() Returns the element at the front of the queue.
- boolean empty() Returns true if the queue is empty, false otherwise.

Notes:

- You must use only standard operations of a stack, which means only push to top, peek/pop from top, size, and is empty operations are valid.
- Depending on your language, the stack may not be supported natively. You may simulate a stack using a list or deque (double-ended queue) as long as you use only a stack's standard operations.

Example 1:

Input
 ["MyQueue", "push", "push", "peek", "pop", "empty"]
 [[], [1], [2], [], [], []]
 Output
 [null, null, null, 1, 1, false]

Explanation
 MyQueue myQueue = new MyQueue();

Constraints:

- $1 \leq x \leq 9$
- At most 100 calls will be made to push, pop, peek, and empty.
- All the calls to pop and peek are valid.

Follow-up: Can you implement the queue such that each operation is amortized $O(1)$ time complexity? In other words, performing n operations will take overall $O(n)$ time even if one of those operations may take longer.

Community Solutions (Python)

• WalkCC

```
class MyQueue:
    def __init__(self):
        self.input = []
        self.output = []

    def push(self, x: int) -> None:
        self.input.append(x)

    def pop(self) -> int:
        self.peak()

        return self.output.pop()

    def peek(self) -> int:
        if not self.output:
            while self.input:
                self.output.append(self.input.pop())
        return self.output[-1]

    def empty(self) -> bool:
        return not self.input and not self.output
```

• LeetDoocs

```
class MyQueue:
    def __init__(self):
        self.stk1 = []
        self.stk2 = []

    def push(self, x: int) -> None:
        self.stk1.append(x)

    def pop(self) -> int:
        self.move()
```

```

        return self.stk2.pop()

    def peek(self) -> int:
        self.move()
        return self.stk2[-1]

    def empty(self) -> bool:
        return not self.stk1 and not self.stk2

    def move(self):

        if not self.stk2:
            while self.stk1:
                self.stk2.append(self.stk1.pop())

# Your MyQueue object will be instantiated and called as such:
# obj = MyQueue()
# obj.push(x)
# param_2 = obj.pop()
# param_3 = obj.peek()

# param_4 = obj.empty()

```

• SimplyLeet

```

class MyQueue:
    def __init__(self):
        # Stack for enqueue operations.
        self.input_stack = []
        # Stack for dequeue operations.
        self.output_stack = []

    def push(self, x: int) -> None:
        # Always push new elements onto the input stack.
        self.input_stack.append(x)

```

Round 4 · Mid · Easy

p.1543

```

    def pop(self) -> int:
        # Ensure the output stack has the current queue order.
        if not self.output_stack:
            while self.input_stack:
                # Move all elements from input_stack to output_stack.
                self.output_stack.append(self.input_stack.pop())
        # The top element of output_stack is the front of the queue.
        return self.output_stack.pop()

    def peek(self) -> int:
        # Ensure the output stack has the current queue order.
        if not self.output_stack:
            while self.input_stack:
                # Move all elements from input_stack to output_stack.
                self.output_stack.append(self.input_stack.pop())
        # Peek the top of output_stack without popping.
        return self.output_stack[-1]

    def empty(self) -> bool:

        # The queue is empty if both stacks are empty.
        return not self.input_stack and not self.output_stack

```

• LC.ca

```

class MyQueue:
    def __init__(self):
        self.stk1 = []
        self.stk2 = [] # reversed order

    def push(self, x: int) -> None:
        self.stk1.append(x)

    def pop(self) -> int:
        self.move()

```

Round 4 · Mid · Easy

p.1544


```

        return self.stk2.pop()

    def peek(self) -> int:
        self.move()
        return self.stk2[-1]

    def empty(self) -> bool:
        return not self.stk1 and not self.stk2

    def move(self):

        if not self.stk2: # only when stk2 is empty
            while self.stk1:
                self.stk2.append(self.stk1.pop())

# Your MyQueue object will be instantiated and called as such:
# obj = MyQueue()
# obj.push(x)
# param_2 = obj.pop()
# param_3 = obj.peek()

# param_4 = obj.empty()

#####

class MyQueue:

    def __init__(self):
        self.sk = []
        self.rsk = [] # reversed

    def push(self, x: int) -> None:
        self.sk.append(x);

    def pop(self) -> int:
        self.peek()
        return self.rsk.pop()

    def peek(self) -> int:
        if self.rsk:

```

Round 4 · Mid · Easy

p.1545

```

        return self.rsk[-1]
    else:
        while self.sk:
            self.rsk.append(self.sk.pop())
        return self.rsk[-1]

    def empty(self) -> bool:

        return not (self.sk or self.rsk)

# Your MyQueue object will be instantiated and called as such:
# obj = MyQueue()
# obj.push(x)
# param_2 = obj.pop()
# param_3 = obj.peek()
# param_4 = obj.empty()

```

◆ Quick Review

Key Insights

- Use two stacks:
- One (input stack) for enqueue operations.
- The other (output stack) for dequeue operations.
- When performing pop or peek, if the output stack is empty, transfer all elements from the input stack to the output stack. This reversal makes the oldest element accessible.

Round 4 · Mid · Easy

p.1546

- Each element is moved at most once between the stacks, ensuring an amortized $O(1)$ cost per operation.
- The space complexity is $O(n)$ where n is the number of elements in the queue.

Space & Time

Time Complexity: Amortized $O(1)$ per operation

Space Complexity: $O(n)$

Solution

We maintain two stacks: an input stack for pushing new elements and an output stack for popping or peeking the front element of the queue. When we need to access the front of the queue (either during a pop or peek), we check if the output stack is empty. If it is, we transfer all elements from the input stack to the output stack. This reversal of order allows us to simulate the FIFO nature of the queue. When performing a push, the element is simply added to the input stack.

Data Structure Design

□ #359 Logger Rate Limiter

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Design · Data Stream
Lists: AlgoMaster 600

Problem

Design a logger system that receives a stream of messages along with their timestamps. Each unique message should only be printed at most every 10 seconds (i.e. a message printed at timestamp t will prevent other identical messages from being printed until timestamp $t + 10$).

All messages will come in chronological order. Several messages may arrive at the same timestamp.

Implement the Logger class:

- `Logger()` Initializes the logger object.
- `bool shouldPrintMessage(int timestamp, string message)` Returns true if the message should be printed in the given timestamp,

otherwise returns false.

Example 1:

Input
 ["Logger", "shouldPrintMessage", "shouldPrintMessage", "shouldPrintMessage", "shouldPrintMessage", "shouldPrintMessage", "shouldPrintMessage"]
 [[], [1, "foo"], [2, "bar"], [3, "foo"], [8, "bar"], [10, "foo"], [11, "foo"]]
Output
 [null, true, true, false, false, false, true]

Explanation
 Logger logger = new Logger();

Constraints:

- $0 \leq \text{timestamp} \leq 109$
- Every timestamp will be passed in non-decreasing order (chronological order).
- $1 \leq \text{message.length} \leq 30$
- At most 104 calls will be made to shouldPrintMessage.

Community Solutions (Python)

• WalkCC

```
class Logger:
    def __init__(self):
        # [(timestamp, message)]
        self.messageQueue = collections.deque()
        self.messageSet = set()

    def shouldPrintMessage(self, timestamp: int, message: str) -> bool:
        # Remove the messages that are 10 seconds from the current timestamp.
        while self.messageQueue:
            headTimestamp, headMessage = self.messageQueue[0]
```

```
        if timestamp < headTimestamp + 10:
            break
        self.messageQueue.popleft()
        self.messageSet.remove(headMessage)

        if message in self.messageSet:
            return False

        self.messageQueue.append((timestamp, message))
        self.messageSet.add(message)
```

```
        return True
```

```
class Logger:
    def __init__(self):
        self.okTime = {} # {message: ok time}

    def shouldPrintMessage(self, timestamp: int, message: str) -> bool:
        if timestamp < self.okTime.get(message, 0):
            return False

        self.okTime[message] = timestamp + 10
        return True
```

• LeetDoocs

```
class Logger:

    def __init__(self):
        self.ts = {}

    def shouldPrintMessage(self, timestamp: int, message: str) -> bool:
        t = self.ts.get(message, 0)
        if t > timestamp:
            return False
        self.ts[message] = timestamp + 10

        return True
```

```
# Your Logger object will be instantiated and called as such:
# obj = Logger()
# param_1 = obj.shouldPrintMessage(timestamp,message)
```

• SimplyLeet

```
# Python code with detailed comments

class Logger:
    def __init__(self):
        # Dictionary to record the next allowed timestamp for each message.
        self.message_dict = {}

    def shouldPrintMessage(self, timestamp: int, message: str) -> bool:
        # If message not seen before or current timestamp is >= expected
        timestamp.
        if message not in self.message_dict or timestamp >= self.message_dict[
message]:

            # Update next allowed time to current timestamp + 10.
            self.message_dict[message] = timestamp + 10
            # Return True since message should be printed.
            return True
        # Else, the message cannot be printed yet.
        return False

# Example usage:
# logger = Logger()
# print(logger.shouldPrintMessage(1, "foo")) # True

# print(logger.shouldPrintMessage(3, "foo")) # False
```

Greedy

Round 4 · Mid · Easy · 2 questions

Round 4 · Mid · Easy

p.1551

Round 4 · Mid · Easy

p.1552

Greedy

□ #455 Assign Cookies

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Two Pointers · Greedy · Sorting
 Lists: AlgoMaster 600

Problem

Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.

Each child i has a greed factor $g[i]$, which is the minimum size of a cookie that the child will be content with; and each cookie j has a size $s[j]$. If $s[j] \geq g[i]$, we can assign the cookie j to the child i , and the child i will be content. Your goal is to maximize the number of your content children and output the maximum number.

Example 1:

Input: $g = [1,2,3]$, $s = [1,1]$
 Output: 1
 Explanation: You have 3 children and 2 cookies. The greed factors of 3 children are 1, 2, 3.
 And even though you have 2 cookies, since their size is both 1, you could only make the child whose greed factor is 1 content.
 You need to output 1.

Round 4 · Mid · Easy

p.1553

Example 2:

Input: $g = [1,2]$, $s = [1,2,3]$
 Output: 2
 Explanation: You have 2 children and 3 cookies. The greed factors of 2 children are 1, 2.
 You have 3 cookies and their sizes are big enough to gratify all of the children,
 You need to output 2.

Constraints:

- $1 \leq g.length \leq 3 \times 10^4$
- $0 \leq s.length \leq 3 \times 10^4$
- $1 \leq g[i], s[j] \leq 231 - 1$

Note: This question is the same as 2410: Maximum Matching of Players With Trainers.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findContentChildren(self, g: list[int], s: list[int]) -> int:
        g.sort()
        s.sort()

        i = 0
        for cookie in s:
            if i < len(g) and g[i] <= cookie:
                i += 1

        return i
```

• LeetDoocs

Round 4 · Mid · Easy

p.1554

```
class Solution:
    def findContentChildren(self, g: List[int], s: List[int]) -> int:
        g.sort()
        s.sort()
        j = 0
        for i, x in enumerate(g):
            while j < len(s) and s[j] < g[i]:
                j += 1
            if j >= len(s):
                return i
            j += 1
        return len(g)
```

• SimplyLeet

```
# Python solution for assigning cookies

def findContentChildren(g, s):
    # Sort greed factors and cookie sizes
    g.sort() # Sorting children by greed factor
    s.sort() # Sorting cookies by size

    child = 0 # Pointer for children
    cookie = 0 # Pointer for cookies

    # Iterate until we exhaust either cookies or children
    while child < len(g) and cookie < len(s):
        # If current cookie satisfies current child's greed factor
        if s[cookie] >= g[child]:
            child += 1 # Assign cookie to child, move to next child
            cookie += 1 # Move to next cookie regardless of assignment

    return child # Number of content children

# Example usage:

print(findContentChildren([1,2,3], [1,1])) # Expected output: 1
print(findContentChildren([1,2], [1,2,3])) # Expected output: 2
```

Greedy

□ #3074 Apple Redistribution into Boxes

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Greedy · Sorting
 Lists: AlgoMaster 600

Problem

You are given an array `apple` of size `n` and an array capacity of size `m`.

There are `n` packs where the `i`th pack contains `apple[i]` apples. There are `m` boxes as well, and the `i`th box has a capacity of `capacity[i]` apples. Return the minimum number of boxes you need to select to redistribute these `n` packs of apples into boxes.

Note that, apples from the same pack can be distributed into different boxes.

Example 1:

```
Input: apple = [1,3,2], capacity = [4,3,1,5,2]
Output: 2
Explanation: We will use boxes with capacities 4 and 5.
It is possible to distribute the apples as the total capacity is greater than
or equal to the total number of apples.
```

Example 2:

Input: apple = [5,5,5], capacity = [2,4,2,7]
 Output: 4
 Explanation: We will need to use all the boxes.

Constraints:

- $1 \leq n \leq \text{apple.length} \leq 50$
- $1 \leq m \leq \text{capacity.length} \leq 50$
- $1 \leq \text{apple}[i], \text{capacity}[i] \leq 50$
- The input is generated such that it's possible to redistribute packs of apples into boxes.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minimumBoxes(self, apple: list[int], capacity: list[int]) -> int:
        appleSum = sum(apple)
        capacitySum = 0

        for i, c in enumerate(sorted(capacity, reverse=True)):
            capacitySum += c
            if capacitySum >= appleSum:
                return i + 1

        return len(capacity)
```

• LeetDoocs

```
class Solution:
    def minimumBoxes(self, apple: List[int], capacity: List[int]) -> int:
        capacity.sort(reverse=True)
        s = sum(apple)
        for i, c in enumerate(capacity, 1):
            s -= c
            if s <= 0:
                return i
```

• SimplyLeet

```
def min_boxes(apple, capacity):
    # Calculate total number of apples
    total_apples = sum(apple)
    # Sort capacities in descending order for optimal box selection
    capacity.sort(reverse=True)
    cumulative_capacity = 0
    box_count = 0

    # Iterate over sorted capacity list
    for cap in capacity:
        cumulative_capacity += cap    # Add the current box capacity

        box_count += 1                # Count the box
        if cumulative_capacity >= total_apples:
            break                    # Enough capacity reached

    return box_count

# Example usage:
print(min_boxes([1, 3, 2], [4, 3, 1, 5, 2])) # Output: 2
print(min_boxes([5, 5, 5], [2, 4, 2, 7]))    # Output: 4
```

Round 5 · Mid · Medium

Round 5

Mid Priority · Medium
160 questions · 25 patterns

Bit Manipulation #137 #201 #260 #371
Prefix Sum #304 #370 #523 #525 #560 #974
#1314 #2536
Kadane's Algorithm #53 #152 #918 #978 #1014
#1186 #1749
Matrix (2D Array) #36 #48 #54 #59 #73 #289
#498 #723
LinkedList In-place Reversal #92
Monotonic Stack #402 #456 #503 #581 #739
#769 #901 #907 #962 #1019 #1762 #2104
Queues #950 #1429 #1823 #2327
Monotonic Queue #1438 #1673 #1696 #2762
#3578
Recursion #50 #394

Round 4 · Mid · Easy

p.1559

☐ ● Divide and Conquer #109 #427 #654 ↗
☐ ● Merge Sort #148 #912 ↗
☐ ● QuickSort / QuickSelect #75 #215 #1985 ↗
☐ ● Backtracking #17 #22 #39 #40 #46 #47 #77 #78 ↗
☐ ● #90 #93 #131 #216 ↗
☐ ● BST / Ordered Set #99 #235 #285 #333 #426 ↗
☐ ● #450 #510 #669 #701 #729 #731 #1008 #2034 ↗
☐ ● Tries #208 #211 #421 #648 #720 #1233 #1268 ↗
☐ ● #2452 #3043 ↗
☐ ● Heaps #1167 #1405 #1642 #1834 #1882 ↗
☐ ● Top K Elements #347 #973 ↗
☐ ● Intervals #56 #57 #253 #435 #452 #763 #986 ↗
☐ ● #1094 #1229 #1288 #1353 #2054 ↗
☐ ● K-Way Merge #373 #378 ↗
☐ ● Data Structure Design #146 #271 #355 #362 ↗
☐ ● #380 #622 #641 #981 #1146 #1472 #2353 ↗
☐ ● Greedy #45 #55 #134 #406 #621 #670 #826 ↗
☐ ● #921 #1488 #1717 #1871 #1975 ↗
☐ ● Topological Sort #210 #310 #802 #1136 #1462 ↗
☐ ● #2115 ↗
☐ ● Union Find #261 #323 #399 #547 #684 #721 ↗
☐ ● #947 ↗

Round 5 · Mid · Medium

p.1560

Minimum Spanning Tree #1135 #1584
Shortest Path #505 #743 #787 #1514 #1631
#2976 #3341 #3650

Bit Manipulation

Round 5 · Mid · Medium · 4 questions

Round 5 · Mid · Medium

p.1561

Round 5 · Mid · Medium

p.1562

Bit Manipulation

#137 Single Number II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Bit Manipulation
 Lists: AlgoMaster 600

Problem

Given an integer array `nums` where every element appears three times except for one, which appears exactly once. Find the single element and return it.

You must implement a solution with a linear runtime complexity and use only constant extra space.

Example 1:

Input: `nums = [2,2,3,2]`
 Output: 3

Example 2:

Input: `nums = [0,1,0,1,0,1,99]`
 Output: 99

Constraints:

- $1 \leq \text{nums.length} \leq 3 \times 10^4$
- $-2^{31} \leq \text{nums}[i] \leq 2^{31} - 1$

Round 5 · Mid · Medium

p.1563

- Each element in `nums` appears exactly three times except for one element which appears once.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def singleNumber(self, nums: List[int]) -> int:
        ones = 0
        twos = 0

        for num in nums:
            ones ^= num & ~twos
            twos ^= num & ~ones

        return ones
```

• LeetDoocs

```
class Solution:
    def singleNumber(self, nums: List[int]) -> int:
        ans = 0
        for i in range(32):
            cnt = sum(num >> i & 1 for num in nums)
            if cnt % 3:
                if i == 31:
                    ans -= 1 << i
                else:
                    ans |= 1 << i

        return ans
```

Round 5 · Mid · Medium

p.1564

```
class Solution:
    def singleNumber(self, nums: List[int]) -> int:
        a = b = 0
        for c in nums:
            aa = (~a & b & c) | (a & ~b & ~c)
            bb = ~a & (b ^ c)
            a, b = aa, bb
        return b
```

• SimplyLeet

```
# Python solution for Single Number II
def singleNumber(nums):
    # Variables to store bits seen once and twice
    ones = 0
    twos = 0
    # Process each number in the input list
    for num in nums:
        # Update ones with the current number and clear bits already in twos
        new_ones = (ones ^ num) & ~twos
        # Update twos with the current number and clear bits just added to
        new_twos = (twos ^ num) & ~new_ones
        ones, twos = new_ones, new_twos
    return ones

# Example usage:
print(singleNumber([2,2,3,2])) # Output: 3
```

Bit Manipulation

□ #201 Bitwise AND of Numbers Range

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Bit Manipulation
 Lists: AlgoMaster 600

Problem

Given two integers left and right that represent the range [left, right], return the bitwise AND of all numbers in this range, inclusive.

Example 1:

Input: left = 5, right = 7
 Output: 4

Example 2:

Input: left = 0, right = 0
 Output: 0

Example 3:

Input: left = 1, right = 2147483647
 Output: 0

Constraints:

- $0 \leq \text{left} \leq \text{right} \leq 2^{31} - 1$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def rangeBitwiseAnd(self, m: int, n: int) -> int:
        return self.rangeBitwiseAnd(m >> 1, n >> 1) << 1 if m < n else m
```

• LeetCode

```
class Solution:
    def rangeBitwiseAnd(self, left: int, right: int) -> int:
        while left < right:
            right &= right - 1
        return right
```

• SimplyLeet

```
# Function to calculate bitwise AND of numbers in range [left, right]
def rangeBitwiseAnd(left: int, right: int) -> int:
    shift = 0 # Counter for number of shifts performed
    # Shift both numbers right until they are equal
    while left != right:
        left >>= 1 # Right shift left by 1
        right >>= 1 # Right shift right by 1
        shift += 1 # Increment shift counter
    # Shift left to restore the common prefix in its original position
    return left << shift # Left shift by the number of shifts
```

```
# Example usage
print(rangeBitwiseAnd(5, 7)) # Expected output: 4
```

Bit Manipulation

#260 Single Number III

MED

LeetCode · SimplyLeet · WalkCC · LeetCode
Array · Bit Manipulation
Lists: AlgoMaster 600

Problem

Given an integer array `nums`, in which exactly two elements appear only once and all the other elements appear exactly twice. Find the two elements that appear only once. You can return the answer in any order.

You must write an algorithm that runs in linear runtime complexity and uses only constant extra space.

Example 1:

Input: `nums = [1,2,1,3,2,5]`
Output: `[3,5]`
Explanation: `[5, 3]` is also a valid answer.

Example 2:

Input: `nums = [-1,0]`
Output: `[-1,0]`

Example 3:

Input: nums = [0,1]
Output: [1,0]

Constraints:

- $2 \leq \text{nums.length} \leq 3 * 10^4$
- $-231 \leq \text{nums}[i] \leq 231 - 1$
- Each integer in nums will appear twice, only two integers will appear once.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def singleNumber(self, nums: List[int]) -> List[int]:
        xors = functools.reduce(operator.xor, nums)
        lowbit = xors & -xors
        ans = [0, 0]

        # Separate `nums` into two groups by `lowbit`.
        for num in nums:
            if num & lowbit:
                ans[0] ^= num

            else:
                ans[1] ^= num

        return ans
```

• LeetDoocs

```
class Solution:
    def singleNumber(self, nums: List[int]) -> List[int]:
        xs = reduce(xor, nums)
        a = 0
        lb = xs & -xs
        for x in nums:
            if x & lb:
                a ^= x
        b = xs ^ a
        return [a, b]
```

• SimplyLeet

```
# Python implementation for "Single Number III"
def singleNumber(nums):
    # Step 1: XOR all numbers to get xorTotal = unique1 XOR unique2
    xorTotal = 0
    for num in nums:
        xorTotal ^= num

    # Step 2: Find a set bit (rightmost set bit) in xorTotal
    diff_bit = xorTotal & -xorTotal

    # Step 3: Partition numbers into two groups and XOR separately
    unique1, unique2 = 0, 0
    for num in nums:
        if num & diff_bit:
            unique1 ^= num
        else:
            unique2 ^= num

    return [unique1, unique2]

# Example usage:
# result = singleNumber([1,2,1,3,2,5])
# print(result) # Output might be [3, 5] or [5, 3]
```

Bit Manipulation

#371 Sum of Two Integers

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · Bit Manipulation
Lists: NeetCode 150

Problem

Given two integers a and b , return the sum of the two integers without using the operators $+$ and $-$.

Example 1:

Input: $a = 1, b = 2$
Output: 3

Example 2:

Input: $a = 2, b = 3$
Output: 5

Constraints:

- $-1000 \leq a, b \leq 1000$

Community Solutions (Python)

- WalkCC

Round 5 · Mid · Medium

p.1571

```
class Solution:
    def getSum(self, a: int, b: int) -> int:
        mask = 0xFFFFFFFF
        MAX = 2000

        while b != 0:
            a, b = (a ^ b) & mask, ((a & b) << 1) & mask

        return a if a < MAX else ~(a ^ mask)
```

• LeetDoocs

```
class Solution:
    def getSum(self, a: int, b: int) -> int:
        a, b = a & 0xFFFFFFFF, b & 0xFFFFFFFF
        while b:
            carry = ((a & b) << 1) & 0xFFFFFFFF
            a, b = a ^ b, carry
        return a if a < 0x80000000 else ~(a ^ 0xFFFFFFFF)
```

• SimplyLeet

```
# Python implementation to add two integers without using + or -
def getSum(a, b):
    # Continue till there is no carry
    while b != 0:
        # XOR gets the sum excluding carries
        sum_without_carry = a ^ b
        # AND finds carries, left shift to apply them to the next bit
        carry = (a & b) << 1
        # Update a with sum without carry and b with carry
        a = sum_without_carry

        b = carry
    return a

# Example usage:
result = getSum(1, 2)
print("Sum:", result) # Output: Sum: 3
```

Round 5 · Mid · Medium

p.1572

Prefix Sum

Round 5 · Mid · Medium · 8 questions

Round 5 · Mid · Medium

p.1573

Prefix Sum

□ #304 Range Sum Query 2D – Immutable **MED**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Design · Matrix · Prefix Sum
Lists: AlgoMaster 600

Problem

Given a 2D matrix matrix, handle multiple queries of the following type:

- Calculate the sum of the elements of matrix inside the rectangle defined by its upper left corner (row1, col1) and lower right corner (row2, col2).

Implement the NumMatrix class:

- NumMatrix(int[][] matrix) Initializes the object with the integer matrix matrix.
- int sumRegion(int row1, int col1, int row2, int col2) Returns the sum of the elements of matrix inside the rectangle defined by its upper left corner (row1, col1) and lower right corner (row2, col2).

Round 5 · Mid · Medium

p.1574

You must design an algorithm where `sumRegion` works on $O(1)$ time complexity.

Example 1:

3	0	1	4	2
5	6	3	2	1
1	2	0	1	5
4	1	0	1	7
1	0	3	0	5

Input
 ["NumMatrix", "sumRegion", "sumRegion", "sumRegion"]
 [[[[[3, 0, 1, 4, 2], [5, 6, 3, 2, 1], [1, 2, 0, 1, 5], [4, 1, 0, 1, 7], [1, 0, 3, 0, 5]]], [2, 1, 4, 3], [1, 1, 2, 2], [1, 2, 2, 4]]
Output
 [null, 8, 11, 12]

Explanation
 NumMatrix numMatrix = new NumMatrix([[3, 0, 1, 4, 2], [5, 6, 3, 2, 1], [1, 2, 0, 1, 5], [4, 1, 0, 1, 7], [1, 0, 3, 0, 5]]);

Round 5 · Mid · Medium

p.1575

Constraints:

- $m == \text{matrix.length}$
- $n == \text{matrix}[i].\text{length}$
- $1 \leq m, n \leq 200$
- $-104 \leq \text{matrix}[i][j] \leq 104$
- $0 \leq \text{row1} \leq \text{row2} < m$
- $0 \leq \text{col1} \leq \text{col2} < n$
- At most 104 calls will be made to `sumRegion`.

Community Solutions (Python)

• WalkCC

```
class NumMatrix:
    def __init__(self, matrix: list[list[int]]):
        if not matrix:
            return

        m = len(matrix)
        n = len(matrix[0])
        # prefix[i][j] := the sum of matrix[0..i][0..j]
        self.prefix = [[0] * (n + 1) for _ in range(m + 1)]

        for i in range(m):
            for j in range(n):
                self.prefix[i + 1][j + 1] = (matrix[i][j] + self.prefix[i][j + 1] +
                                              self.prefix[i + 1][j] - self.prefix[i][j])

        def sumRegion(self, row1: int, col1: int, row2: int, col2: int) -> int:
            return (self.prefix[row2 + 1][col2 + 1] - self.prefix[row1][col2 + 1] -
                    self.prefix[row2 + 1][col1] + self.prefix[row1][col1])
```

Round 5 · Mid · Medium

p.1576

• LeetCode

```
class NumMatrix:
    def __init__(self, matrix: List[List[int]]):
        m, n = len(matrix), len(matrix[0])
        self.s = [[0] * (n + 1) for _ in range(m + 1)]
        for i, row in enumerate(matrix):
            for j, v in enumerate(row):
                self.s[i + 1][j + 1] = (
                    self.s[i][j + 1] + self.s[i + 1][j] - self.s[i][j] + v
                )

    def sumRegion(self, row1: int, col1: int, row2: int, col2: int) -> int:
        return (
            self.s[row2 + 1][col2 + 1]
            - self.s[row2 + 1][col1]
            - self.s[row1][col2 + 1]
            + self.s[row1][col1]
        )

# Your NumMatrix object will be instantiated and called as such:
# obj = NumMatrix(matrix)
# param_1 = obj.sumRegion(row1,col1,row2,col2)
```

• SimplyLeet

```
# Class definition for NumMatrix using prefix sum 2D array.
class NumMatrix:
    def __init__(self, matrix):
        # Get dimensions of the matrix.
        m = len(matrix)
        n = len(matrix[0]) if m > 0 else 0
        # Create prefix sum matrix with extra row and column.
        self.dp = [[0] * (n + 1) for _ in range(m + 1)]
        # Precompute the prefix sum values.
        for i in range(m):
```

```
        for j in range(n):
            # Sum of current cell, top, left, and subtract overlapping
            top-left.
            self.dp[i+1][j+1] = self.dp[i][j+1] + self.dp[i+1][j] - self.d
            p[i][j] + matrix[i][j]

    def sumRegion(self, row1, col1, row2, col2):
        # Calculate the sum using the inclusion-exclusion principle.
        return self.dp[row2+1][col2+1] - self.dp[row1][col2+1] - self.dp[row2+
            1][col1] + self.dp[row1][col1]
```

Prefix Sum

#370 Range Addition

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Prefix Sum

Lists: AlgoMaster 600

Problem

You are given an integer `length` and an array `updates` where `updates[i] = [startIdxi, endIdxi, inci]`.

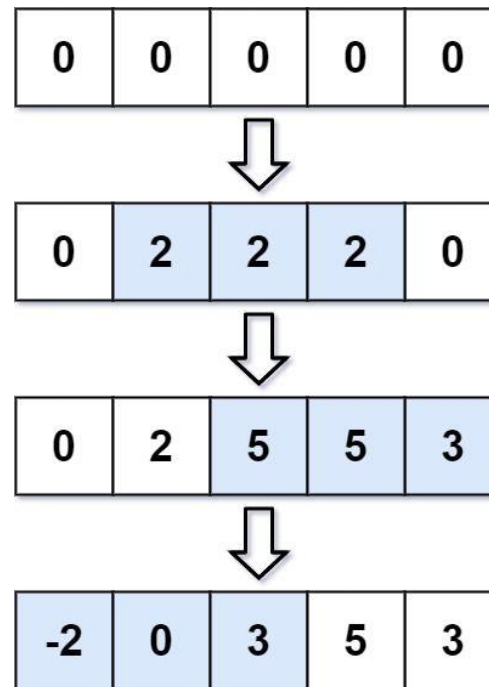
You have an array `arr` of length `length` with all zeros, and you have some operation to apply on `arr`. In the `i`th operation, you should increment all the elements `arr[startIdxi]`, `arr[startIdxi + 1]`, ..., `arr[endIdxi]` by `inci`.

Return `arr` after applying all the updates.

Example 1:

Round 5 · Mid · Medium

p.1579



Input: `length = 5, updates = [[1,3,2],[2,4,3],[0,2,-2]]`
Output: `[-2,0,3,5,3]`

Example 2:

Input: `length = 10, updates = [[2,4,6],[5,6,8],[1,9,-4]]`
Output: `[0,-4,2,2,2,4,4,-4,-4,-4]`

Constraints:

- $1 \leq \text{length} \leq 105$
- $0 \leq \text{updates.length} \leq 104$
- $0 \leq \text{startIdxi} \leq \text{endIdxi} < \text{length}$
- $-1000 \leq \text{inci} \leq 1000$

Round 5 · Mid · Medium

p.1580

Community Solutions (Python)

• WalkCC

```
class Solution:
    def getModifiedArray(
        self,
        length: int,
        updates: List[List[int]],
    ) -> List[int]:
        line = [0] * length

        for start, end, inc in updates:
            line[start] += inc

            if end + 1 < length:
                line[end + 1] -= inc

        return itertools.accumulate(line)
```

• LeetCode

```
class Solution:
    def getModifiedArray(self, length: int, updates: List[List[int]]) -> List[int]:
        d = [0] * length
        for l, r, c in updates:
            d[l] += c
            if r + 1 < length:
                d[r + 1] -= c
        return list(accumulate(d))

class BinaryIndexedTree:
    __slots__ = "n", "c"

    def __init__(self, n: int):
        self.n = n
        self.c = [0] * (n + 1)

    def update(self, x: int, delta: int) -> None:
        while x <= self.n:
            self.c[x] += delta
```

Round 5 · Mid · Medium

p.1581

```
x += x & -x

def query(self, x: int) -> int:
    s = 0
    while x:
        s += self.c[x]
        x -= x & -x
    return s
```

```
class Solution:
    def getModifiedArray(self, length: int, updates: List[List[int]]) -> List[int]:
        tree = BinaryIndexedTree(length)
        for l, r, c in updates:
            tree.update(l + 1, c)
            tree.update(r + 2, -c)
        return [tree.query(i + 1) for i in range(length)]
```

• SimplyLeet

```
# Python solution for Range Addition

def getModifiedArray(length, updates):
    # Initialize the result list with zeros
    result = [0] * length

    # Process each update by marking the start and end+1 positions
    for update in updates:
        start, end, inc = update
        result[start] += inc # Add inc at the start index

        if end + 1 < length:
            result[end + 1] -= inc # Subtract inc just after the end index

    # Convert the difference array to the final result using a running sum
    for i in range(1, length):
        result[i] += result[i - 1]

    return result

# Example usage:
```

Round 5 · Mid · Medium

p.1582

```
length = 5
updates = [[1, 3, 2], [2, 4, 3], [0, 2, -2]]
print(getModifiedArray(length, updates)) # Output: [-2, 0, 3, 5, 3]
```

Prefix Sum

□ #523 Continuous Subarray Sum

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table · Math · Prefix Sum
 Lists: AlgoMaster 600

Problem

Given an integer array `nums` and an integer `k`, return `true` if `nums` has a good subarray or `false` otherwise.

A good subarray is a subarray where:

- its length is at least two, and
- the sum of the elements of the subarray is a multiple of `k`.

Note that:

- A subarray is a contiguous part of the array.
- An integer `x` is a multiple of `k` if there exists an integer `n` such that $x = n * k$. 0 is always a multiple of `k`.

Example 1:

Input: nums = [23,2,4,6,7], k = 6
 Output: true
 Explanation: [2, 4] is a continuous subarray of size 2 whose elements sum up to 6.

Example 2:

Input: nums = [23,2,6,4,7], k = 6
 Output: true
 Explanation: [23, 2, 6, 4, 7] is an continuous subarray of size 5 whose elements sum up to 42.
 42 is a multiple of 6 because $42 = 7 * 6$ and 7 is an integer.

Example 3:

Input: nums = [23,2,6,4,7], k = 13
 Output: false

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $0 \leq \text{nums}[i] \leq 109$
- $0 \leq \text{sum}(\text{nums}[i]) \leq 231 - 1$
- $1 \leq k \leq 231 - 1$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def checkSubarraySum(self, nums: List[int], k: int) -> bool:
        prefix = 0
        prefixToIndex = {0: -1}

        for i, num in enumerate(nums):
            prefix += num
            if k != 0:
                prefix %= k
            if prefix in prefixToIndex:
```

```
        if i - prefixToIndex[prefix] > 1:
            return True
        else:
            # Set a new key if it's absent because the previous index is better.
            prefixToIndex[prefix] = i

        return False
```

• LeetCode

```
class Solution:
    def checkSubarraySum(self, nums: List[int], k: int) -> bool:
        d = {0: -1}
        s = 0
        for i, x in enumerate(nums):
            s = (s + x) % k
            if s not in d:
                d[s] = i
            elif i - d[s] > 1:
                return True

        return False
```

• SimplyLeet

```
# Python solution using prefix sum with modulo and a dictionary to store remainders.

def checkSubarraySum(nums, k):
    # Dictionary to store remainder and its earliest index.
    remainder_index = {0: -1}
    cumulative_sum = 0

    # Iterate over the array elements.
    for index, num in enumerate(nums):
        cumulative_sum += num
```

```

remainder = cumulative_sum % k

# If the remainder is seen before, check the subarray length.
if remainder in remainder_index:
    # If subarray length is at least 2 (current index - previous index)
    if index - remainder_index[remainder] > 1:
        return True
else:
    # Store the index for this remainder the first time it is seen.
    remainder_index[remainder] = index

return False

# Example usage:
# print(checkSubarraySum([23,2,4,6,7], 6)) # Expected output: True

```

Prefix Sum

#525 Contiguous Array

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table · Prefix Sum
 Lists: Grind 169

Problem

Given a binary array `nums`, return the maximum length of a contiguous subarray with an equal number of 0 and 1.

Example 1:

Input: `nums = [0,1]`
 Output: 2
 Explanation: `[0, 1]` is the longest contiguous subarray with an equal number of 0 and 1.

Example 2:

Input: `nums = [0,1,0]`
 Output: 2
 Explanation: `[0, 1]` (or `[1, 0]`) is a longest contiguous subarray with equal number of 0 and 1.

Example 3:

Input: `nums = [0,1,1,1,1,0,0,0]`
 Output: 6
 Explanation: `[1,1,1,0,0,0]` is the longest contiguous subarray with equal number of 0 and 1.

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $\text{nums}[i]$ is either 0 or 1.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findMaxLength(self, nums: list[int]) -> int:
        ans = 0
        prefix = 0
        prefixToIndex = {0: -1}

        for i, num in enumerate(nums):
            prefix += 1 if num else -1
            ans = max(ans, i - prefixToIndex.setdefault(prefix, i))

        return ans
```

• LeetDoocs

```
class Solution:
    def findMaxLength(self, nums: List[int]) -> int:
        d = {0: -1}
        ans = s = 0
        for i, x in enumerate(nums):
            s += 1 if x else -1
            if s in d:
                ans = max(ans, i - d[s])
            else:
                d[s] = i

        return ans
```

• SimplyLeet

Round 5 · Mid · Medium

p.1589

```
# Python solution for Contiguous Array

def findMaxLength(nums):
    # Dictionary to store the earliest occurrence of each prefix sum
    prefix_indices = {0: -1}
    prefix_sum = 0
    max_length = 0

    # Iterate over the array
    for index, num in enumerate(nums):

        # Convert 0 to -1, keep 1 as is
        if num == 0:
            prefix_sum -= 1
        else:
            prefix_sum += 1

        # If the prefix sum has been seen before, a balanced subarray exists
        if prefix_sum in prefix_indices:
            # Calculate the length of the subarray
            current_length = index - prefix_indices[prefix_sum]

            max_length = max(max_length, current_length)
        else:
            # Store the index for this prefix sum if not already seen
            prefix_indices[prefix_sum] = index

    # Return the maximum length of balanced subarray
    return max_length

# Example usage:
# print(findMaxLength([0,1,0])) # Expected output: 2
```

• LC.ca

Round 5 · Mid · Medium

p.1590

```
class Solution:
    def findMaxLength(self, nums: List[int]) -> int:
        s = ans = 0
        mp = {0: -1}
        for i, v in enumerate(nums):
            s += 1 if v == 1 else -1
            if s in mp:
                ans = max(ans, i - mp[s])
            else:
                mp[s] = i
        return ans
```

◆ Quick Review

Key Insights

- Convert 0s to -1 so that a balanced subarray (equal number of 0s and 1s) sums to zero.
- Use a prefix sum to track the cumulative sum as you iterate through the array.
- Use a hash table (or dictionary) to store the first occurrence of each prefix sum.
- When the same prefix sum is encountered again, it indicates that the subarray between these indices is balanced.

Space & Time

Time Complexity: $O(n)$ – We iterate through the array once.

Space Complexity: $O(n)$ – In the worst case, we store an entry for each prefix sum.

Solution

We use a technique where we convert each 0 in the array to -1. Then, we compute a running prefix sum as we iterate through the array. A prefix sum of 0 at any point indicates that the subarray from the beginning to the current index is balanced. For each prefix sum, if we have seen it before, the subarray between the previous index (where the prefix sum was first seen) and the current index sums to 0. We update the maximum length accordingly. We use a hash table to store the first index where each prefix sum occurs.

Prefix Sum

#560 Subarray Sum Equals K

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table · Prefix Sum
 Lists: Grind 169

Problem

Given an array of integers `nums` and an integer `k`, return the total number of subarrays whose sum equals to `k`.

A subarray is a contiguous non-empty sequence of elements within an array.

Example 1:

Input: `nums = [1,1,1]`, `k = 2`
 Output: 2

Example 2:

Input: `nums = [1,2,3]`, `k = 3`
 Output: 2

Constraints:

- $1 \leq \text{nums.length} \leq 2 * 10^4$
- $-1000 \leq \text{nums}[i] \leq 1000$
- $-10^7 \leq k \leq 10^7$

Round 5 · Mid · Medium

p.1593

Community Solutions (Python)

• WalkCC

```
class Solution:
    def subarraySum(self, nums: List[int], k: int) -> int:
        ans = 0
        prefix = 0
        count = collections.Counter({0: 1})

        for num in nums:
            prefix += num
            ans += count[prefix - k]
            count[prefix] += 1

        return ans
```

• LeetDoocs

```
class Solution:
    def subarraySum(self, nums: List[int], k: int) -> int:
        cnt = Counter({0: 1})
        ans = s = 0
        for x in nums:
            s += x
            ans += cnt[s - k]
            cnt[s] += 1
        return ans
```

• SimplyLeet

```
# Function to calculate the number of subarrays that sum to k
def subarraySum(nums, k):
    # Dictionary to store frequency of prefix sums
    prefix_counts = {0: 1} # Initialize with prefix sum 0
    current_sum = 0 # Running sum initialization
    count = 0 # To count the valid subarrays
    # Iterate over the array
    for num in nums:
        current_sum += num # Update the running (prefix) sum
        # Check if there's a prefix sum that differs from current_sum by k
```

Round 5 · Mid · Medium

p.1594

```
count += prefix_counts.get(current_sum - k, 0)
# Update the frequency count for the current sum
prefix_counts[current_sum] = prefix_counts.get(current_sum, 0) + 1
return count
```

Example usage:

```
print(subarraySum([1,1,1], 2)) # Expected output: 2
```

• LC.ca

```
class Solution:
    def subarraySum(self, nums: List[int], k: int) -> int:
        dp = Counter({0: 1})
        ans = s = 0
        for num in nums:
            s += num
            ans += dp[s - k] # not matter if the same num like [1,1]
            dp[s] += 1
        return ans
```

#####

```
class Solution: # over time limit, not fast enough
    def subarraySum(self, nums: List[int], k: int) -> int:
        count = 0

        # sum value from 1 to i-th element
        sum_arr = [0] * (len(nums) + 1)

        # cached sum

        for i in range(1, len(nums) + 1):
            sum_arr[i] = sum_arr[i - 1] + nums[i - 1]

        for start in range(len(nums)):
            for end in range(start + 1, len(nums) + 1):
                if sum_arr[end] - sum_arr[start] == k:
                    count += 1

        return count
```

◆ Quick Review

Key Insights

- Utilize the prefix sum technique to simplify sum calculation for subarrays.
- Use a hash table (or dictionary/map) to store the frequency of prefix sums encountered.
- For each element, check if (current prefix sum - k) exists in the hash table; if it does, it indicates a valid subarray.
- Initialize the hash table with {0: 1} to manage cases where a subarray's sum is exactly k.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Solution

We solve the problem by maintaining a running sum (prefix sum) and using a hash table to record the number of times each sum has occurred. As we iterate through the array, we update the running sum. For each new sum, we check if (current sum - k) exists in our hash table. If it does, the value associated with (current sum - k) gives the number of

subarrays ending at the current index that sum to k. This method efficiently computes the result in a single pass through the array.

Prefix Sum

□ #974 Subarray Sums Divisible by K

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table · Prefix Sum
 Lists: AlgoMaster 600

Problem

Given an integer array `nums` and an integer `k`, return the number of non-empty subarrays that have a sum divisible by `k`.

A subarray is a contiguous part of an array.

Example 1:

```
Input: nums = [4,5,0,-2,-3,1], k = 5
Output: 7
Explanation: There are 7 subarrays with a sum divisible by k = 5:
[4, 5, 0, -2, -3, 1], [5], [5, 0], [5, 0, -2, -3], [0], [0, -2, -3], [-2, -3]
```

Example 2:

```
Input: nums = [5], k = 9
Output: 0
```

Constraints:

- $1 \leq \text{nums.length} \leq 3 \times 10^4$
- $-10^4 \leq \text{nums}[i] \leq 10^4$
- $2 \leq k \leq 10^4$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def subarraysDivByK(self, nums: List[int], k: int) -> int:
        ans = 0
        prefix = 0
        count = [0] * k
        count[0] = 1

        for num in nums:
            prefix = (prefix + num % k + k) % k
            ans += count[prefix]

        count[prefix] += 1

        return ans
```

• LeetDoocs

```
class Solution:
    def subarraysDivByK(self, nums: List[int], k: int) -> int:
        cnt = Counter({0: 1})
        ans = s = 0
        for x in nums:
            s = (s + x) % k
            ans += cnt[s]
            cnt[s] += 1
        return ans
```

• SimplyLeet

Round 5 · Mid · Medium

p.1599

Python Solution

```
def subarraysDivByK(nums, k):
    # Dictionary to store frequency of each remainder
    remainder_count = {0: 1} # initialize with remainder 0 for subarrays
    # starting at index 0
    current_sum = 0 # current prefix sum
    count = 0 # counter for valid subarrays

    # Iterate through the given array
    for num in nums:
        current_sum += num # update the prefix sum
        # Compute the modulo while handling negative values
        remainder = current_sum % k

        # If this remainder has been seen before, add its count to our answer
        count += remainder_count.get(remainder, 0)

        # Update the frequency count for this remainder
        remainder_count[remainder] = remainder_count.get(remainder, 0) + 1

    return count

# Example Usage:
# print(subarraysDivByK([4,5,0,-2,-3,1], 5))
```

Round 5 · Mid · Medium

p.1600

Prefix Sum

□ #1314 Matrix Block Sum

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Matrix · Prefix Sum
 Lists: AlgoMaster 600

Problem

Given a $m \times n$ matrix mat and an integer k , return a matrix $answer$ where each $answer[i][j]$ is the sum of all elements $mat[r][c]$ for:

- $i - k \leq r \leq i + k$,
- $j - k \leq c \leq j + k$, and
- (r, c) is a valid position in the matrix.

Example 1:

Input: $mat = [[1,2,3],[4,5,6],[7,8,9]]$, $k = 1$
 Output: $[[12,21,16],[27,45,33],[24,39,28]]$

Example 2:

Input: $mat = [[1,2,3],[4,5,6],[7,8,9]]$, $k = 2$
 Output: $[[45,45,45],[45,45,45],[45,45,45]]$

Constraints:

- $m == mat.length$
- $n == mat[i].length$
- $1 \leq m, n, k \leq 100$

Round 5 · Mid · Medium

p.1601

- $1 \leq mat[i][j] \leq 100$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def matrixBlockSum(self, mat: list[list[int]], k: int) -> list[list[int]]:
        m = len(mat)
        n = len(mat[0])
        ans = [[0] * n for _ in range(m)]
        prefix = [[0] * (n + 1) for _ in range(m + 1)]

        for i in range(m):
            for j in range(n):
                prefix[i + 1][j + 1] = (mat[i][j] + prefix[i][j + 1] +

                prefix[i + 1][j] - prefix[i][j])

        for i in range(m):
            for j in range(n):
                r1 = max(0, i - k) + 1
                c1 = max(0, j - k) + 1
                r2 = min(m - 1, i + k) + 1
                c2 = min(n - 1, j + k) + 1
                ans[i][j] = (prefix[r2][c2] - prefix[r2][c1 - 1] -
                prefix[r1 - 1][c2] + prefix[r1 - 1][c1 - 1])

        return ans
```

• LeetDoocs

Round 5 · Mid · Medium

p.1602

```

class Solution:
    def matrixBlockSum(self, mat: List[List[int]], k: int) -> List[List[int]]:
        m, n = len(mat), len(mat[0])
        s = [[0] * (n + 1) for _ in range(m + 1)]
        for i, row in enumerate(mat, 1):
            for j, x in enumerate(row, 1):
                s[i][j] = s[i - 1][j] + s[i][j - 1] - s[i - 1][j - 1] + x
        ans = [[0] * n for _ in range(m)]
        for i in range(m):
            for j in range(n):
                x1, y1 = max(i - k, 0), max(j - k, 0)
                x2, y2 = min(m - 1, i + k), min(n - 1, j + k)
                ans[i][j] = (
                    s[x2 + 1][y2 + 1] - s[x1][y2 + 1] - s[x2 + 1][y1] + s[x1][
y1]
                )
        return ans

```

• SimplyLeet

```

# Python solution using prefix sum
def matrixBlockSum(mat, k):
    # dimensions of the matrix
    m = len(mat)
    n = len(mat[0])

    # create a prefix sum matrix with extra row and column for easier
    calculations
    prefix = [[0] * (n + 1) for _ in range(m + 1)]

    # Build prefix sum matrix

```

```

        for i in range(1, m + 1):
            for j in range(1, n + 1):
                prefix[i][j] = mat[i-1][j-1] + prefix[i-1][j] + prefix[i][j-1] - p
refix[i-1][j-1]

        # function to get sum of submatrix from (r1, c1) to (r2, c2) inclusive
        def get_sum(r1, c1, r2, c2):
            return prefix[r2+1][c2+1] - prefix[r1][c2+1] - prefix[r2+1][c1] + pref
ix[r1][c1]

        # Build result matrix
        result = [[0] * n for _ in range(m)]

        for i in range(m):
            for j in range(n):
                # Determine block boundaries considering matrix limits
                r1 = max(0, i - k)
                c1 = max(0, j - k)
                r2 = min(m - 1, i + k)
                c2 = min(n - 1, j + k)
                result[i][j] = get_sum(r1, c1, r2, c2)

        return result

# Example usage:
# print(matrixBlockSum([[1,2,3],[4,5,6],[7,8,9]], 1))

```

Prefix Sum

#2536 Increment Submatrices by One

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode

Array · Matrix · Prefix Sum

Lists: AlgoMaster 600

Problem

You are given a positive integer n , indicating that we initially have an $n \times n$ 0-indexed integer matrix mat filled with zeroes.

You are also given a 2D integer array $query$. For each $query[i] = [row1i, col1i, row2i, col2i]$, you should do the following operation:

- Add 1 to every element in the submatrix with the top left corner $(row1i, col1i)$ and the bottom right corner $(row2i, col2i)$. That is, add 1 to $mat[x][y]$ for all $row1i \leq x \leq row2i$ and $col1i \leq y \leq col2i$.

Return the matrix mat after performing every query.

Example 1:

Round 5 · Mid · Medium

p.1605

0	0	0		0	0	0		1	1	0
0	0	0	→	0	1	1	→	1	2	1
0	0	0		0	1	1		0	1	1

Input: $n = 3$, $queries = [[1,1,2,2],[0,0,1,1]]$

Output: $[[1,1,0],[1,2,1],[0,1,1]]$

Explanation: The diagram above shows the initial matrix, the matrix after the first query, and the matrix after the second query.

- In the first query, we add 1 to every element in the submatrix with the top left corner $(1, 1)$ and bottom right corner $(2, 2)$.
- In the second query, we add 1 to every element in the submatrix with the top left corner $(0, 0)$ and bottom right corner $(1, 1)$.

Example 2:

0	0		1	1
0	0	→	1	1

Input: $n = 2$, $queries = [[0,0,1,1]]$

Output: $[[1,1],[1,1]]$

Explanation: The diagram above shows the initial matrix and the matrix after the first query.

- In the first query we add 1 to every element in the matrix.

Constraints:

- $1 \leq n \leq 500$
- $1 \leq queries.length \leq 104$
- $0 \leq row1i \leq row2i < n$

Round 5 · Mid · Medium

p.1606

- $0 \leq \text{col1} \leq \text{col2} < n$

Community Solutions (Python)

• LeetCode

```
class Solution:
    def rangeAddQueries(self, n: int, queries: List[List[int]]) -> List[List[int]]:
        mat = [[0] * n for _ in range(n)]
        for x1, y1, x2, y2 in queries:
            mat[x1][y1] += 1
            if x2 + 1 < n:
                mat[x2 + 1][y1] -= 1
            if y2 + 1 < n:
                mat[x1][y2 + 1] -= 1
            if x2 + 1 < n and y2 + 1 < n:
                mat[x2 + 1][y2 + 1] += 1

        for i in range(n):
            for j in range(n):
                if i:
                    mat[i][j] += mat[i - 1][j]
                if j:
                    mat[i][j] += mat[i][j - 1]
                if i and j:
                    mat[i][j] -= mat[i - 1][j - 1]

        return mat
```

• SimplyLeet

```
# Python Solution
class Solution:
    def rangeAddQueries(self, n: int, queries: List[List[int]]) -> List[List[int]]:
        # Initialize a difference matrix of size (n+1) x (n+1) with zeros.
        diff = [[0] * (n + 1) for _ in range(n + 1)]

        # Process each query to update the difference matrix.
        for row1, col1, row2, col2 in queries:
            diff[row1][col1] += 1 # Increment top-left corner.
            if col2 + 1 < n:
                diff[row1][col2 + 1] -= 1 # Subtract right to the submatrix.
            if row2 + 1 < n:
                diff[row2 + 1][col1] -= 1 # Subtract below the submatrix.
            if row2 + 1 < n and col2 + 1 < n:
                diff[row2 + 1][col2 + 1] += 1 # Correct double subtraction.

        # Build the final matrix from the difference matrix using prefix sum.
        # First, process prefix sums row-wise.
        for i in range(n):
            for j in range(1, n):
                diff[i][j] += diff[i][j-1]

        # Then, process prefix sums column-wise.
        for j in range(n):
            for i in range(1, n):
                diff[i][j] += diff[i-1][j]

        # Extract the first n x n matrix as result.
        return [row[:n] for row in diff[:n]]
```


Kadane's Algorithm

Round 5 · Mid · Medium · 7 questions

Round 5 · Mid · Medium

p.1609

Kadane's Algorithm

□ #53 Maximum Subarray

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Divide and Conquer

Lists: Grind 169

Problem

Given an integer array `nums`, find the subarray with the largest sum, and return its sum.

Example 1:

Input: `nums = [-2,1,-3,4,-1,2,1,-5,4]`
Output: 6
Explanation: The subarray `[4,-1,2,1]` has the largest sum 6.

Example 2:

Input: `nums = [1]`
Output: 1
Explanation: The subarray `[1]` has the largest sum 1.

Example 3:

Input: `nums = [5,4,-1,7,8]`
Output: 23
Explanation: The subarray `[5,4,-1,7,8]` has the largest sum 23.

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $-10^4 \leq \text{nums}[i] \leq 10^4$

Round 5 · Mid · Medium

p.1610

Follow up: If you have figured out the $O(n)$ solution, try coding another solution using the divide and conquer approach, which is more subtle.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxSubArray(self, nums: list[int]) -> int:
        # dp[i] := the maximum sum subarray ending in i
        dp = [0] * len(nums)

        dp[0] = nums[0]
        for i in range(1, len(nums)):
            dp[i] = max(nums[i], dp[i - 1] + nums[i])

        return max(dp)
```

```
class Solution:
    def maxSubArray(self, nums: list[int]) -> int:
        ans = -math.inf
        summ = 0

        for num in nums:
            summ = max(num, summ + num)
            ans = max(ans, summ)

        return ans
```

Round 5 · Mid · Medium

p.1611

```
from dataclasses import dataclass

@dataclass(frozen=True)
class T:
    summ: int
    # the sum of the subarray starting from the first number
    maxSubarraySumLeft: int
    # the sum of the subarray ending in the last number
    maxSubarraySumRight: int

    maxSubarraySum: int

class Solution:
    def maxSubArray(self, nums: list[int]) -> int:
        def divideAndConquer(l: int, r: int) -> T:
            if l == r:
                return T(nums[l], nums[l], nums[l], nums[l])
            m = (l + r) // 2
            left = divideAndConquer(l, m)

            right = divideAndConquer(m + 1, r)
            maxSubarraySumLeft = max(left.maxSubarraySumLeft,
                                     left.summ + right.maxSubarraySumLeft)
            maxSubarraySumRight = max(
                left.maxSubarraySumRight + right.summ, right.maxSubarraySumRight)
            maxSubarraySum = max(left.maxSubarraySumRight + right.maxSubarraySumLeft
                                ,
                                left.maxSubarraySum, right.maxSubarraySum)
            summ = left.summ + right.summ
            return T(summ, maxSubarraySumLeft, maxSubarraySumRight, maxSubarraySum)

        return divideAndConquer(0, len(nums) - 1).maxSubarraySum
```

• LeetDoocs

Round 5 · Mid · Medium

p.1612

```

class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
        ans = f = nums[0]
        for x in nums[1:]:
            f = max(f, 0) + x
            ans = max(ans, f)
        return ans

class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
        def crossMaxSub(nums, left, mid, right):
            lsum = rsum = 0
            lmx = rmx = -inf
            for i in range(mid, left - 1, -1):
                lsum += nums[i]
                lmx = max(lmx, lsum)
            for i in range(mid + 1, right + 1):
                rsum += nums[i]
                rmx = max(rmx, rsum)
            return lmx + rmx

        def maxSub(nums, left, right):
            if left == right:
                return nums[left]
            mid = (left + right) >> 1
            lsum = maxSub(nums, left, mid)
            rsum = maxSub(nums, mid + 1, right)
            csum = crossMaxSub(nums, left, mid, right)

            return max(lsum, rsum, csum)

        left, right = 0, len(nums) - 1
        return maxSub(nums, left, right)

```

• SimplyLeet

Round 5 · Mid · Medium

p.1613

```

# Python solution using Kadane's Algorithm
def maxSubArray(nums):
    # Initialize currentSum and maxSum to the first element
    current_sum = max_sum = nums[0]
    # Traverse through the array starting from the second element
    for num in nums[1:]:
        # If current_sum + num is less than num, restart at num
        current_sum = max(num, current_sum + num)
        # Update max_sum if current_sum is greater
        max_sum = max(max_sum, current_sum)

    return max_sum

# Example usage
if __name__ == "__main__":
    sample = [-2,1,-3,4,-1,2,1,-5,4]
    print(maxSubArray(sample)) # Output: 6

```

• LC.ca

```

class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
        res = cur_sum = nums[0]
        for num in nums[1:]:
            cur_sum = num + max(cur_sum, 0)
            res = max(res, cur_sum)
        return res

```

◆ Quick Review

Key Insights

- Use Kadane's Algorithm to solve the problem in $O(n)$ time.
- Keep track of a running sum (currentSum) and update it with the maximum between the current element and the sum including the

Round 5 · Mid · Medium

p.1614

current element.

- The maximum sum encountered during the iteration (maxSum) is the answer.
- A divide and conquer approach also exists, splitting the array and merging solutions, but Kadane's method is optimal for this problem.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Solution

The solution uses Kadane's Algorithm. Iterate over the array while maintaining two variables: currentSum, which tracks the maximum subarray sum ending at the current position, and maxSum, which stores the best sum seen so far. For each element, update currentSum to be either the current element or the sum of currentSum and the current element, whichever is higher. Update maxSum accordingly. This approach efficiently computes the maximum contiguous subarray sum in a single pass.

Kadane's Algorithm

□ #152 Maximum Product Subarray

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming

Lists: Grind 169

Problem

Given an integer array nums, find a subarray that has the largest product, and return the product.

The test cases are generated so that the answer will fit in a 32-bit integer.

Note that the product of an array with a single element is the value of that element.

Example 1:

Input: nums = [2,3,-2,4]
Output: 6
Explanation: [2,3] has the largest product 6.

Example 2:

Input: nums = [-2,0,-1]
Output: 0
Explanation: The result cannot be 2, because [-2,-1] is not a subarray.

Constraints:

- $1 \leq \text{nums.length} \leq 2 * 10^4$

- $-10 \leq \text{nums}[i] \leq 10$
- The product of any subarray of nums is guaranteed to fit in a 32-bit integer.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxProduct(self, nums: list[int]) -> int:
        ans = nums[0]
        dpMin = nums[0] # the minimum so far
        dpMax = nums[0] # the maximum so far

        for i in range(1, len(nums)):
            num = nums[i]
            prevMin = dpMin # dpMin[i - 1]
            prevMax = dpMax # dpMax[i - 1]

            if num < 0:
                dpMin = min(prevMax * num, num)
                dpMax = max(prevMin * num, num)
            else:
                dpMin = min(prevMin * num, num)
                dpMax = max(prevMax * num, num)

            ans = max(ans, dpMax)

        return ans
```

• LeetDoocs

```
class Solution:
    def maxProduct(self, nums: List[int]) -> int:
        ans = f = g = nums[0]
        for x in nums[1:]:
            ff, gg = f, g
            f = max(x, ff * x, gg * x)
            g = min(x, ff * x, gg * x)
            ans = max(ans, f)
        return ans
```

• SimplyLeet

```
def maxProduct(nums):
    # If the input array is empty, return 0 (though problem guarantees at least
    # one element)
    if not nums:
        return 0
    # Initialize maximum, minimum product ending here and the result.
    max_ending_here = nums[0]
    min_ending_here = nums[0]
    global_max = nums[0]

    # Iterate over the array starting from the second element.
    for num in nums[1:]:
        # If the current number is negative, swap max and min.
        if num < 0:
            max_ending_here, min_ending_here = min_ending_here, max_ending_her
e
        # Update the max product at the current index.
        max_ending_here = max(num, max_ending_here * num)
        # Update the min product at the current index.
        min_ending_here = min(num, min_ending_here * num)
        # Update the global maximum product.
        global_max = max(global_max, max_ending_here)

    return global_max

# Example usage:
# print(maxProduct([2,3,-2,4])) # Output: 6
```

• LC.ca

```
class Solution:
    def maxProduct(self, nums: List[int]) -> int:
        ans = f = g = nums[0]
        for x in nums[1:]:
            ff, gg = f, g
            f = max(x, ff * x, gg * x)
            g = min(x, ff * x, gg * x)
            ans = max(ans, f)
        return ans
```

◆ Quick Review

Key Insights

- Maintain both the maximum and minimum product at the current index because a negative number can swap the max and min.
- A subarray must be contiguous.
- Use dynamic programming to update both the maximum product and the minimum product ending at each index.
- Resetting the product in case of a zero is crucial since the product becomes 0.

Space & Time

Time Complexity: $O(n)$ – The solution iterates through the array once.

Space Complexity: $O(1)$ – Constant space is used for tracking products.

Round 5 · Mid · Medium

p.1619

Solution

The solution uses an iterative dynamic programming approach. At each index, compute the maximum product (maxEndingHere) and minimum product (minEndingHere) ending at that index. If the current element is negative, swap these two values, because multiplying a negative with a minimum product (possibly negative) might yield a larger product. After processing each element, update the global maximum. This method handles edge cases like zeros and negatives efficiently with constant additional space.

Round 5 · Mid · Medium

p.1620

Kadane's Algorithm

#918 Maximum Sum Circular Subarray

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Divide and Conquer · Dynamic
 Programming · Queue · Monotonic Queue
 Lists: AlgoMaster 600

Problem

Given a circular integer array `nums` of length `n`, return the maximum possible sum of a non-empty subarray of `nums`.

A circular array means the end of the array connects to the beginning of the array. Formally, the next element of `nums[i]` is `nums[(i + 1) % n]` and the previous element of `nums[i]` is `nums[(i - 1 + n) % n]`.

A subarray may only include each element of the fixed buffer `nums` at most once. Formally, for a subarray `nums[i]`, `nums[i + 1]`, ..., `nums[j]`, there does not exist `i <= k1, k2 <= j` with `k1 % n == k2 % n`.

Example 1:

Round 5 · Mid · Medium

p.1621

Input: `nums = [1,-2,3,-2]`
 Output: 3
 Explanation: Subarray [3] has maximum sum 3.

Example 2:

Input: `nums = [5,-3,5]`
 Output: 10
 Explanation: Subarray [5,5] has maximum sum 5 + 5 = 10.

Example 3:

Input: `nums = [-3,-2,-3]`
 Output: -2
 Explanation: Subarray [-2] has maximum sum -2.

Constraints:

- `n == nums.length`
- `1 <= n <= 3 * 104`
- `-3 * 104 <= nums[i] <= 3 * 104`

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxSubarraySumCircular(self, nums: list[int]) -> int:
        totalSum = 0
        currMaxSum = 0
        currMinSum = 0
        maxSum = -math.inf
        minSum = math.inf

        for num in nums:
            totalSum += num
```

Round 5 · Mid · Medium

p.1622

```

currMaxSum = max(currMaxSum + num, num)
currMinSum = min(currMinSum + num, num)
maxSum = max(maxSum, currMaxSum)
minSum = min(minSum, currMinSum)

return maxSum if maxSum < 0 else max(maxSum, totalSum - minSum)

```

• LeetCode

```

class Solution:
    def maxSubarraySumCircular(self, nums: List[int]) -> int:
        pmi, pmx = 0, -inf
        ans, s, smi = -inf, 0, inf
        for x in nums:
            s += x
            ans = max(ans, s - pmi)
            smi = min(smi, s - pmx)
            pmi = min(pmi, s)
            pmx = max(pmx, s)

        return max(ans, s - smi)

```

• SimplyLeet

```

# Python solution

def maxSubarraySumCircular(nums):
    # Initialize variables for Kadane's algorithm for maximum subarray sum
    max_current = max_global = nums[0]
    # Initialize variables for finding the minimum subarray sum
    min_current = min_global = nums[0]
    # Calculate total sum starting with the first element
    total_sum = nums[0]

```

```

# Iterate over the array from the second element
for num in nums[1:]:
    total_sum += num # Update total sum

    # Update maximum subarray sum ending at current position
    max_current = max(num, max_current + num)
    max_global = max(max_global, max_current)

    # Update minimum subarray sum ending at current position
    min_current = min(num, min_current + num)

    min_global = min(min_global, min_current)

# If all numbers are negative, max_global is the maximum
if max_global < 0:
    return max_global
else:
    # Otherwise, the answer is the maximum of non-circular and circular
    case
    return max(max_global, total_sum - min_global)

# Example usage:

print(maxSubarraySumCircular([1, -2, 3, -2]))

```


Kadane's Algorithm

#978 Longest Turbulent Subarray

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Dynamic Programming · Sliding Window
 Lists: AlgoMaster 600

Problem

Given an integer array `arr`, return the length of a maximum size turbulent subarray of `arr`.

A subarray is turbulent if the comparison sign flips between each adjacent pair of elements in the subarray.

More formally, a subarray `[arr[i], arr[i + 1], ..., arr[j]]` of `arr` is said to be turbulent if and only if:

- For $i \leq k < j$: $arr[k] > arr[k + 1]$ when k is odd, and
- $arr[k] < arr[k + 1]$ when k is even.
- $arr[k] > arr[k + 1]$ when k is even, and
- $arr[k] < arr[k + 1]$ when k is odd.

Example 1:

Input: `arr = [9,4,2,10,7,8,8,1,9]`
 Output: 5
 Explanation: `arr[1] > arr[2] < arr[3] > arr[4] < arr[5]`

Round 5 · Mid · Medium

p.1625

Example 2:

Input: `arr = [4,8,12,16]`
 Output: 2

Example 3:

Input: `arr = [100]`
 Output: 1

Constraints:

- $1 \leq arr.length \leq 4 * 10^4$
- $0 \leq arr[i] \leq 10^9$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxTurbulenceSize(self, arr: list[int]) -> int:
        ans = 1
        increasing = 1
        decreasing = 1

        for i in range(1, len(arr)):
            if arr[i] > arr[i - 1]:
                increasing = decreasing + 1
                decreasing = 1

            elif arr[i] < arr[i - 1]:
                decreasing = increasing + 1
                increasing = 1
            else:
                increasing = 1
                decreasing = 1
            ans = max(ans, max(increasing, decreasing))

        return ans
```

• LeetDoocs

Round 5 · Mid · Medium

p.1626

```
class Solution:
    def maxTurbulenceSize(self, arr: List[int]) -> int:
        ans = f = g = 1
        for a, b in pairwise(arr):
            ff = g + 1 if a < b else 1
            gg = f + 1 if a > b else 1
            f, g = ff, gg
            ans = max(ans, f, g)
        return ans
```

• SimplyLeet

```
# Function to find the length of the longest turbulent subarray.
def maxTurbulenceSize(arr):
    n = len(arr)
    if n == 1:
        return 1

    # Initialize maximum turbulent subarray length and current subarray length.
    maxLength = 1
    currentLength = 1

    # Iterate through the array.
    for i in range(1, n):
        # If current and previous elements are equal, reset the current
        # sequence.
        if arr[i] == arr[i - 1]:
            currentLength = 1
        else:
            # For the first pair or if alternating condition holds with
            # previous comparison.
            if i == 1 or (arr[i - 1] - arr[i - 2]) * (arr[i] - arr[i - 1]) < 0:
                currentLength += 1
            else:
                currentLength = 1
```

Round 5 · Mid · Medium

p.1627

```
length 2.
        # Reset: the current pair forms a new turbulent subarray of
        # length 2.
        currentLength = 2
        # Update the maximum observed length.
        maxLength = max(maxLength, currentLength)
    return maxLength

# Example usage:
print(maxTurbulenceSize([9,4,2,10,7,8,8,1,9])) # Expected output: 5
```

Round 5 · Mid · Medium

p.1628

Kadane's Algorithm

#1014 Best Sightseeing Pair

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Dynamic Programming
 Lists: AlgoMaster 600

Problem

You are given an integer array `values` where `values[i]` represents the value of the i th sightseeing spot. Two sightseeing spots i and j have a distance $j - i$ between them.

The score of a pair ($i < j$) of sightseeing spots is `values[i] + values[j] + i - j`: the sum of the values of the sightseeing spots, minus the distance between them.

Return the maximum score of a pair of sightseeing spots.

Example 1:

Input: `values = [8,1,5,2,6]`
 Output: 11
 Explanation: $i = 0, j = 2, \text{values}[i] + \text{values}[j] + i - j = 8 + 5 + 0 - 2 = 11$

Example 2:

Round 5 · Mid · Medium

p.1629

Input: `values = [1,2]`
 Output: 2

Constraints:

- $2 \leq \text{values.length} \leq 5 * 10^4$
- $1 \leq \text{values}[i] \leq 1000$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxScoreSightseeingPair(self, values: List[int]) -> int:
        ans = 0
        bestPrev = 0

        for value in values:
            ans = max(ans, value + bestPrev)
            bestPrev = max(bestPrev, value) - 1

        return ans
```

• LeetDoocs

```
class Solution:
    def maxScoreSightseeingPair(self, values: List[int]) -> int:
        ans = mx = 0
        for j, x in enumerate(values):
            ans = max(ans, mx + x - j)
            mx = max(mx, x + j)
        return ans
```

• SimplyLeet

Round 5 · Mid · Medium

p.1630

```
def maxScoreSightseeing(values):
    # best_left holds the maximum value of (values[i] + i) encountered up to
    # the current index
    best_left = values[0] + 0 # Initialize with the first element and index
    max_score = 0 # Track the maximum score

    # Iterate through the array starting from the second element
    for j in range(1, len(values)):
        # Calculate current score using the best value so far and the current
        # spot's contribution
        current_score = best_left + values[j] - j
        # Update max_score if the current score is greater

        max_score = max(max_score, current_score)
        # Update best_left with the maximum of itself and (values[j] + j)
        best_left = max(best_left, values[j] + j)
    return max_score

# Example usage:
print(maxScoreSightseeing([8, 1, 5, 2, 6])) # Expected output: 11
```

Kadane's Algorithm

#1186 Maximum Subarray Sum with One Deletion MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Dynamic Programming
 Lists: AlgoMaster 600

Problem

Given an array of integers, return the maximum sum for a non-empty subarray (contiguous elements) with at most one element deletion. In other words, you want to choose a subarray and optionally delete one element from it so that there is still at least one element left and the sum of the remaining elements is maximum possible.

Note that the subarray needs to be non-empty after deleting one element.

Example 1:

Input: arr = [1,-2,0,3]
 Output: 4
 Explanation: Because we can choose [1, -2, 0, 3] and drop -2, thus the subarray [1, 0, 3] becomes the maximum value.

Example 2:

Input: arr = [1,-2,-2,3]

Output: 3

Explanation: We just choose [3] and it's the maximum sum.

Example 3:

Input: arr = [-1,-1,-1,-1]

Output: -1

Explanation: The final subarray needs to be non-empty. You can't choose [-1] and delete -1 from it, then get an empty subarray to make the sum equals to 0.

Constraints:

- $1 \leq \text{arr.length} \leq 105$
- $-104 \leq \text{arr}[i] \leq 104$

Community Solutions (Python)

• WalkCC

```
class Solution:
    # Similar to 53. Maximum Subarray
    def maximumSum(self, arr: list[int]) -> int:
        # dp[0][i] := the maximum sum subarray ending in i (no deletion)
        # dp[1][i] := the maximum sum subarray ending in i (at most 1 deletion)
        dp = [[0] * len(arr) for _ in range(2)]

        dp[0][0] = arr[0]
        dp[1][0] = arr[0]
        for i in range(1, len(arr)):

            dp[0][i] = max(arr[i], dp[0][i - 1] + arr[i])
            dp[1][i] = max(arr[i], dp[1][i - 1] + arr[i], dp[0][i - 1])

        return max(dp[1])
```

• LeetDoocs

```
class Solution:
    def maximumSum(self, arr: List[int]) -> int:
        n = len(arr)
        left = [0] * n
        right = [0] * n
        s = 0
        for i, x in enumerate(arr):
            s = max(s, 0) + x
            left[i] = s
            s = 0

        for i in range(n - 1, -1, -1):
            s = max(s, 0) + arr[i]
            right[i] = s
        ans = max(left)
        for i in range(1, n - 1):
            ans = max(ans, left[i - 1] + right[i + 1])
        return ans
```

• SimplyLeet

```
# Python solution for Maximum Subarray Sum with One Deletion

def maximumSum(arr):
    n = len(arr)
    # Initialize dp arrays
    dp_no_del = [0] * n # Maximum subarray sum ending here without deletion
    dp_del = [0] * n # Maximum subarray sum ending here with one deletion

    dp_no_del[0] = arr[0]
    dp_del[0] = 0 # Cannot delete the first element (subarray must be
non-empty)
```

```

max_sum = arr[0]

# Iterate through the array, updating dp values
for i in range(1, n):
    # Maximum no deletion: either extend the previous subarray or start
    fresh
    dp_no_del[i] = max(arr[i], dp_no_del[i-1] + arr[i])
    # Maximum with one deletion: either delete current element using
    previous dp_no_del,
    # or extend a subarray that already had one deletion
    dp_del[i] = max(dp_no_del[i-1], dp_del[i-1] + arr[i])

    # Update overall maximum sum considering both states
    max_sum = max(max_sum, dp_no_del[i], dp_del[i])

return max_sum

# Example usage:
print(maximumSum([1, -2, 0, 3])) # Expected output: 4

```

Kadane's Algorithm

□ #1749 Maximum Absolute Sum of Any Subarray

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Dynamic Programming
 Lists: AlgoMaster 600

Problem

You are given an integer array `nums`. The absolute sum of a subarray `[numsl, numsl+1, ..., numsr-1, numsr]` is `abs(numsl + numsl+1 + ... + numsr-1 + numsr)`.

Return the maximum absolute sum of any (possibly empty) subarray of `nums`.

Note that `abs(x)` is defined as follows:

- If `x` is a negative integer, then `abs(x) = -x`.
- If `x` is a non-negative integer, then `abs(x) = x`.

Example 1:

Input: `nums = [1,-3,2,3,-4]`
 Output: 5
 Explanation: The subarray `[2,3]` has absolute sum = `abs(2+3) = abs(5) = 5`.

Example 2:

Input: nums = [2,-5,1,-4,3,-2]
 Output: 8
 Explanation: The subarray [-5,1,-4] has absolute sum = $\text{abs}(-5+1-4) = \text{abs}(-8) = 8$.

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $-104 \leq \text{nums}[i] \leq 104$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxAbsoluteSum(self, nums):
        ans = -math.inf
        maxSum = 0
        minSum = 0

        for num in nums:
            maxSum = max(num, maxSum + num)
            minSum = min(num, minSum + num)
            ans = max(ans, maxSum, -minSum)
```

```
return ans
```

```
class Solution:
    def maxAbsoluteSum(self, nums):
        summ = 0
        maxPrefix = 0
        minPrefix = 0

        for num in nums:
            summ += num
            maxPrefix = max(maxPrefix, summ)
            minPrefix = min(minPrefix, summ)
```

```
return maxPrefix - minPrefix
```

• LeetCode

```
class Solution:
    def maxAbsoluteSum(self, nums: List[int]) -> int:
        f = g = 0
        ans = 0
        for x in nums:
            f = max(f, 0) + x
            g = min(g, 0) + x
            ans = max(ans, f, abs(g))
        return ans
```

• SimplyLeet

```
# Python solution using Kadane's algorithm for both max and min subarray sums.
def maxAbsoluteSum(nums):
    # Initialize variables for the maximum subarray sum
    max_ending_here = 0
    max_sum = 0

    # Initialize variables for the minimum subarray sum
    min_ending_here = 0
    min_sum = 0

    # Iterate over each number in the list
    for num in nums:
        # Update the maximum sum subarray ending at current position
        max_ending_here = max(0, max_ending_here + num)
        max_sum = max(max_sum, max_ending_here)

        # Update the minimum sum subarray ending at current position
        min_ending_here = min(0, min_ending_here + num)
        min_sum = min(min_sum, min_ending_here)

    # The maximum absolute sum is the maximum of the absolute min sum and max sum
    return max(max_sum, abs(min_sum))

# Example usage:
nums = [1, -3, 2, 3, -4]
print(maxAbsoluteSum(nums)) # Output: 5
```

Matrix (2D Array)

Round 5 · Mid · Medium · 8 questions

Round 5 · Mid · Medium

p.1639

Matrix (2D Array)

□ #36 Valid Sudoku

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Matrix

Lists: Grind 169

Problem

Determine if a 9 x 9 Sudoku board is valid. Only the filled cells need to be validated according to the following rules:

1. Each row must contain the digits 1–9 without repetition.
2. Each column must contain the digits 1–9 without repetition.
3. Each of the nine 3 x 3 sub-boxes of the grid must contain the digits 1–9 without repetition.

Note:

- A Sudoku board (partially filled) could be valid but is not necessarily solvable.
- Only the filled cells need to be validated according to the mentioned rules.

Round 5 · Mid · Medium

p.1640

Example 1:

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Input: board =
 [["5","3",".", ".", "7",".", ".", ".", ".", "."],
 ["6",".", ".", ".", "1","9","5",".", ".", ".", "."],
 [".", "9","8",".", ".", ".", ".", ".", "6","."],
 ["8",".", ".", ".", ".", "6",".", ".", ".", "3"],
 ["4",".", ".", ".", "8",".", "3",".", ".", "1"],
 ["7",".", ".", ".", ".", "2",".", ".", ".", "6"],
 [".", "6",".", ".", ".", ".", ".", "2","8","."],
 [".", ".", ".", "4","1","9",".", ".", ".", "5"],
 [".", ".", ".", ".", "8",".", ".", "7","9"]]

Example 2:

Round 5 · Mid · Medium

p.1641

Input: board =
 [["8","3",".", ".", "7",".", ".", ".", ".", "."],
 ["6",".", ".", ".", "1","9","5",".", ".", ".", "."],
 [".", "9","8",".", ".", ".", ".", ".", "6","."],
 ["8",".", ".", ".", ".", "6",".", ".", ".", "3"],
 ["4",".", ".", ".", "8",".", "3",".", ".", "1"],
 ["7",".", ".", ".", ".", "2",".", ".", ".", "6"],
 [".", "6",".", ".", ".", ".", ".", "2","8","."]]

Constraints:

- board.length == 9
- board[i].length == 9
- board[i][j] is a digit 1-9 or '.'.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isValidSudoku(self, board: list[list[str]]) -> bool:
        seen = set()

        for i in range(9):
            for j in range(9):
                c = board[i][j]
                if c == '.':
                    continue
                if (c + '@row ' + str(i) in seen or
                    c + '@col ' + str(j) in seen or
                    c + '@box ' + str(i // 3) + str(j // 3) in seen):
                    return False
                seen.add(c + '@row ' + str(i))
                seen.add(c + '@col ' + str(j))
                seen.add(c + '@box ' + str(i // 3) + str(j // 3))

        return True
```

• LeetCode

Round 5 · Mid · Medium

p.1642

```
class Solution:
    def isValidSudoku(self, board: List[List[str]]) -> bool:
        row = [[False] * 9 for _ in range(9)]
        col = [[False] * 9 for _ in range(9)]
        sub = [[False] * 9 for _ in range(9)]
        for i in range(9):
            for j in range(9):
                c = board[i][j]
                if c == '.':
                    continue

                num = int(c) - 1
                k = i // 3 * 3 + j // 3
                if row[i][num] or col[j][num] or sub[k][num]:
                    return False
                row[i][num] = True
                col[j][num] = True
                sub[k][num] = True

        return True
```

• SimplyLeet

```
# Python solution with inline comments
def isValidSudoku(board):
    # Initialize sets for rows, columns, and boxes
    rows = [set() for _ in range(9)]
    cols = [set() for _ in range(9)]
    boxes = [set() for _ in range(9)]

    # Iterate over each cell in the board
    for r in range(9):
        for c in range(9):

            num = board[r][c]
            # Skip empty cells
            if num == '.':
                continue
            # Compute box index using integer division
            box_index = (r // 3) * 3 + (c // 3)
            # Check if the number is already present in the row, column, or box
            if num in rows[r] or num in cols[c] or num in boxes[box_index]:
                return False
            # Add number to row, column, and box sets
```

Round 5 · Mid · Medium

p.1643

```
        rows[r].add(num)
        cols[c].add(num)
        boxes[box_index].add(num)

    return True

# Example usage:
if __name__ == "__main__":
    board = [
        ["5","3",".",".","7",".",".",".","."],
        ["6",".",".","1","9","5",".",".","."],
        [".","9","8",".",".",".","6",".","."],
        ["8",".",".","6",".",".",".","3","."],
        ["4",".",".","8",".","3",".",".","1"],
        ["7",".",".","2",".",".","6",".","."],
        [".","6",".",".","2","8",".","."],
        [".",".","4","1","9",".",".","5"],
        [".",".",".","8",".",".","7","9"]
    ]
    print(isValidSudoku(board)) # Expected output: True
```

• LC.ca

```
class Solution:
    def isValidSudoku(self, board: List[List[str]]) -> bool:
        row = [[False] * 9 for _ in range(9)]
        col = [[False] * 9 for _ in range(9)]
        sub = [[False] * 9 for _ in range(9)]
        for i in range(9):
            for j in range(9):
                c = board[i][j]
                if c == '.':
                    continue

                num = int(c) - 1
                k = i // 3 * 3 + j // 3
                if row[i][num] or col[j][num] or sub[k][num]:
                    return False
                row[i][num] = True
                col[j][num] = True
                sub[k][num] = True

        return True
```

Round 5 · Mid · Medium

p.1644

◆ Quick Review

Key Insights

- Validate each row by checking for duplicate numbers while ignoring empty cells.
- Validate each column in the same way.
- Divide the board into nine 3x3 sub-boxes and check each for duplicate numbers.
- Use hash sets (or equivalent data structures) to keep track of seen digits for rows, columns, and boxes.
- The board size is fixed (9x9), so the overall time complexity is constant with respect to input size.

Space & Time

Time Complexity: $O(1)$ — Since the board size is fixed at 9x9.

Space Complexity: $O(1)$ — The extra space used by the sets is bounded by the fixed board size.

Solution

The solution uses three main data structures: an array of sets for rows, an array of sets for

Round 5 · Mid · Medium

p.1645

columns, and an array of sets for 3x3 sub-boxes. For each cell, if it is not empty, check if the number is already present in the corresponding row, column, or box. If found, return false immediately. Otherwise, add the number to the respective sets. Compute the index for the 3x3 sub-box using the formula: $(\text{row_index} // 3) * 3 + (\text{col_index} // 3)$. If no duplicates are found after traversing the entire board, the board is considered valid.

Round 5 · Mid · Medium

p.1646

Matrix (2D Array)

#48 Rotate Image

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode

Array · Math · Matrix

Lists: Grind 169 | CodeSignal

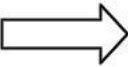
Problem

You are given an $n \times n$ 2D matrix representing an image, rotate the image by 90 degrees (clockwise).

You have to rotate the image in-place, which means you have to modify the input 2D matrix directly. DO NOT allocate another 2D matrix and do the rotation.

Example 1:

1	2	3
4	5	6
7	8	9



7	4	1
8	5	2
9	6	3

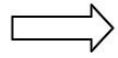
Round 5 · Mid · Medium

p.1647

Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]
Output: [[7,4,1],[8,5,2],[9,6,3]]

Example 2:

5	1	9	11
2	4	8	10
13	3	6	7
15	14	12	16



15	13	2	5
14	3	4	1
12	6	8	9
16	7	10	11

Input: matrix = [[5,1,9,11],[2,4,8,10],[13,3,6,7],[15,14,12,16]]
Output: [[15,13,2,5],[14,3,4,1],[12,6,8,9],[16,7,10,11]]

Constraints:

- $n == \text{matrix.length} == \text{matrix}[i].\text{length}$
- $1 \leq n \leq 20$
- $-1000 \leq \text{matrix}[i][j] \leq 1000$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def rotate(self, matrix: list[list[int]]) -> None:
        matrix.reverse()
        for i, j in itertools.combinations(range(len(matrix)), 2):
            matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
```

Round 5 · Mid · Medium

p.1648

```
class Solution:
    def rotate(self, matrix: List[List[int]]) -> None:
        for mn in range(len(matrix) // 2):
            mx = len(matrix) - mn - 1
            for i in range(mn, mx):
                offset = i - mn
                top = matrix[mn][i]
                matrix[mn][i] = matrix[mx - offset][mn]
                matrix[mx - offset][mn] = matrix[mx][mx - offset]
                matrix[mx][mx - offset] = matrix[i][mx]

            matrix[i][mx] = top
```

• LeetCode

```
class Solution:
    def rotate(self, matrix: List[List[int]]) -> None:
        n = len(matrix)
        for i in range(n >> 1):
            for j in range(n):
                matrix[i][j], matrix[n - i - 1][j] = matrix[n - i - 1][j], matrix[i][j]
            for i in range(n):
                for j in range(i):
                    matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
```

• SimplyLeet

```
def rotate(matrix):
    n = len(matrix)
    # Transpose the matrix by swapping elements across the diagonal
    for i in range(n):
        for j in range(i, n):
            matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
    # Reverse each row to complete the clockwise rotation
    for i in range(n):
        matrix[i].reverse()
```

```
# Example usage:
matrix = [[1,2,3],[4,5,6],[7,8,9]]
rotate(matrix)
print(matrix) # Outputs: [[7,4,1],[8,5,2],[9,6,3]]
```

• LC.ca

```
class Solution(object):
    def rotate(self, matrix):
        """
        :type matrix: List[List[int]]
        :rtype: void Do not return anything, modify matrix in-place instead.
        """

        if len(matrix) == 0:
            return
        h = len(matrix)
        w = len(matrix[0])

        for i in range(0, h): # mirror
            for j in range(0, w / 2):
                matrix[i][j], matrix[i][w - j - 1] = matrix[i][w - j - 1], matrix[i][j]

        for i in range(0, h): # transpos
            for j in range(0, w - 1 - i):
                matrix[i][j], matrix[w - 1 - j][h - 1 - i] = matrix[w - 1 - j][h - 1 - i], matrix[i][j]
```

#####

```
class Solution:
    def rotate(self, matrix: List[List[int]]) -> None:
        n = len(matrix)
        for i in range(n >> 1):
            for j in range(n):
                matrix[i][j], matrix[n - i - 1][j] = matrix[n - i - 1][j], matrix[i][j]
            for i in range(n):
                for j in range(i):
                    matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
```

◆ Quick Review

Key Insights

- The rotation can be achieved in two steps: first transpose the matrix and then reverse each row.
- Transposing the matrix swaps $\text{matrix}[i][j]$ with $\text{matrix}[j][i]$, converting rows into columns.
- Reversing each row post-transposition yields the desired clockwise rotation.
- This approach works in-place with $O(1)$ extra space.

Space & Time

Time Complexity: $O(n^2)$ since every element is processed during the transpose and row reversal.

Space Complexity: $O(1)$ as the operations are performed in-place without extra data structures.

Solution

The solution leverages a two-step in-place transformation:

- Transpose the matrix: Iterate through the matrix and swap each element (i, j) with

element (j, i) for $j \geq i$. – Reverse each row:
After the transpose, each row is reversed to rotate the matrix 90 degrees clockwise. The key trick is to realize that a transpose followed by a reverse of each row achieves the rotation without needing extra space.

Matrix (2D Array)

□ #54 Spiral Matrix

MED

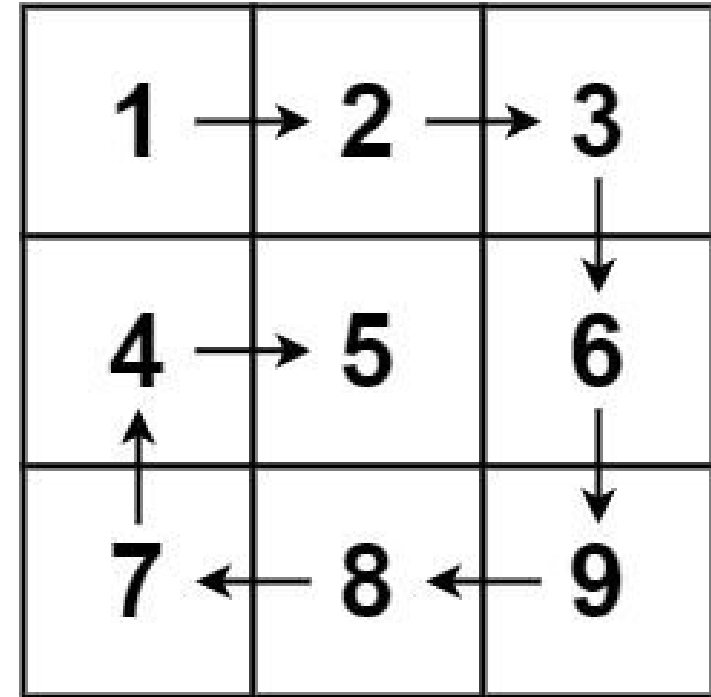
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Matrix · Simulation

Lists: Grind 169 | CodeSignal

Problem

Given an $m \times n$ matrix, return all elements of the matrix in spiral order.

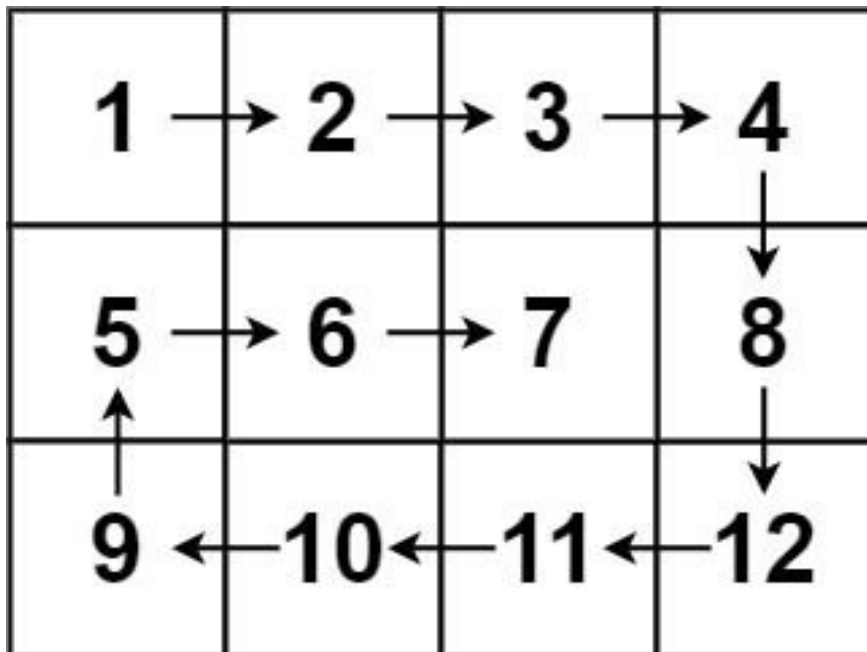
Example 1:



Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]

Output: [1,2,3,6,9,8,7,4,5]

Example 2:



Input: matrix = [[1,2,3,4],[5,6,7,8],[9,10,11,12]]
 Output: [1,2,3,4,8,12,11,10,9,5,6,7]

Constraints:

- $m == \text{matrix.length}$
- $n == \text{matrix}[i].\text{length}$
- $1 \leq m, n \leq 10$
- $-100 \leq \text{matrix}[i][j] \leq 100$

Community Solutions (Python)

- WalkCC

Round 5 · Mid · Medium

p.1655

```
class Solution:
    def spiralOrder(self, matrix: list[list[int]]) -> list[int]:
        if not matrix:
            return []

        m = len(matrix)
        n = len(matrix[0])
        ans = []
        r1 = 0
        c1 = 0

        r2 = m - 1
        c2 = n - 1

        # Repeatedly add matrix[r1..r2][c1..c2] to `ans`.
        while len(ans) < m * n:
            j = c1
            while j <= c2 and len(ans) < m * n:
                ans.append(matrix[r1][j])
                j += 1
            i = r1 + 1

            while i <= r2 - 1 and len(ans) < m * n:
                ans.append(matrix[i][c2])
                i += 1
            j = c2
            while j >= c1 and len(ans) < m * n:
                ans.append(matrix[r2][j])
                j -= 1
            i = r2 - 1
            while i >= r1 + 1 and len(ans) < m * n:
                ans.append(matrix[i][c1])
                i -= 1
            r1 += 1
            c1 += 1
            r2 -= 1
            c2 -= 1

        return ans
```

- LeetCode

Round 5 · Mid · Medium

p.1656


```
class Solution:
    def spiralOrder(self, matrix: List[List[int]]) -> List[int]:
        m, n = len(matrix), len(matrix[0])
        dirs = (0, 1, 0, -1, 0)
        vis = [[False] * n for _ in range(m)]
        i = j = k = 0
        ans = []
        for _ in range(m * n):
            ans.append(matrix[i][j])
            vis[i][j] = True

            x, y = i + dirs[k], j + dirs[k + 1]
            if x < 0 or x >= m or y < 0 or y >= n or vis[x][y]:
                k = (k + 1) % 4
            i += dirs[k]
            j += dirs[k + 1]
        return ans
```

• SimplyLeet

```
def spiralOrder(matrix):
    # Initialize the boundaries of the matrix
    top, bottom = 0, len(matrix) - 1
    left, right = 0, len(matrix[0]) - 1
    result = [] # This will store the spiral order

    # Loop until the pointers cross each other
    while top <= bottom and left <= right:
        # Traverse from left to right on the current top row
        for col in range(left, right + 1):
            result.append(matrix[top][col])
            top += 1 # Move the top boundary down

        # Traverse downwards on the current right column
        for row in range(top, bottom + 1):
            result.append(matrix[row][right])
            right -= 1 # Move the right boundary to the left

        # Traverse from right to left on the current bottom row, if valid
        if top <= bottom:
```

Round 5 · Mid · Medium

p.1657

```
        for col in range(right, left - 1, -1):
            result.append(matrix[bottom][col])
            bottom -= 1 # Move the bottom boundary up

        # Traverse upwards on the current left column, if valid
        if left <= right:
            for row in range(bottom, top - 1, -1):
                result.append(matrix[row][left])
                left += 1 # Move the left boundary right

    return result

# Example usage:
# print(spiralOrder([[1,2,3],[4,5,6],[7,8,9]]))
```

• LC.ca

```
class Solution:
    def spiralOrder(self, matrix: List[List[int]]) -> List[int]:
        m, n = len(matrix), len(matrix[0])
        dirs = (0, 1, 0, -1, 0)
        i = j = k = 0
        ans = []
        vis = set()
        for _ in range(m * n):
            ans.append(matrix[i][j])
            vis.add((i, j))

            x, y = i + dirs[k], j + dirs[k + 1]
            if not 0 <= x < m or not 0 <= y < n or (x, y) in vis:
                k = (k + 1) % 4
            i = i + dirs[k]
            j = j + dirs[k + 1]
        return ans
```

◆ Quick Review

Key Insights

Round 5 · Mid · Medium

p.1658

- Use four boundaries: top, bottom, left, and right to track the current layer.
- Traverse right across the top row, then down the right column, then left on the bottom row, and finally up the left column.
- After each direction, adjust the corresponding boundary to move inward.
- Continue the traversal until the boundaries cross.

Space & Time

Time Complexity: $O(m * n)$ since each element is visited exactly once.

Space Complexity: $O(1)$ extra space (excluding the space required for the output list).

Solution

The solution uses a simulation approach with boundary pointers (top, bottom, left, right). At each step, the boundaries indicate the current submatrix to traverse. You iterate as follows:

- Traverse from left to right along the top boundary.
- Traverse downward along the right boundary.
- If the top boundary has not passed the bottom, traverse from right to left along the bottom boundary.
- If the left

boundary has not passed the right, traverse upward along the left boundary. After each traversal, the boundaries are adjusted inward. This continues until all elements have been visited.

Matrix (2D Array)

□ #59 Spiral Matrix II

MED

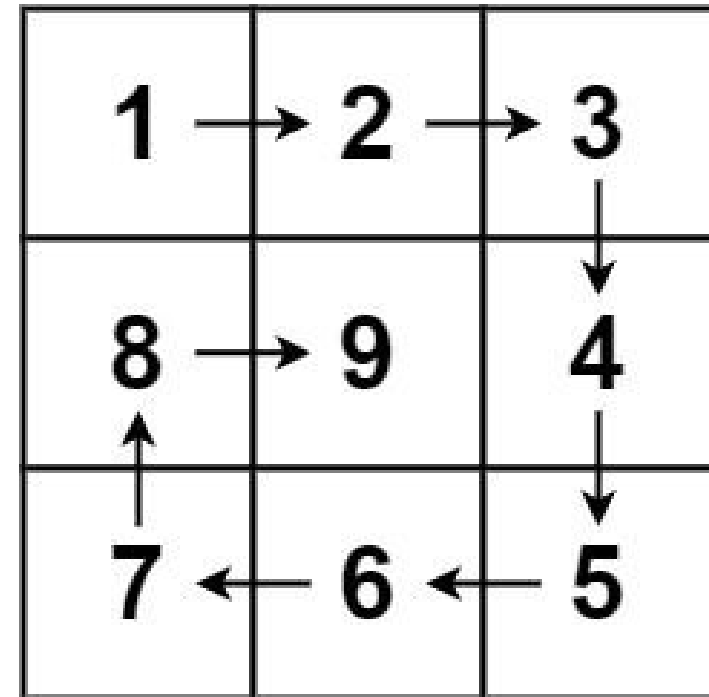
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Matrix · Simulation

Lists: CodeSignal

Problem

Given a positive integer n , generate an $n \times n$ matrix filled with elements from 1 to n^2 in spiral order.

Example 1:



Input: $n = 3$
Output: `[[1,2,3],[8,9,4],[7,6,5]]`

Example 2:

Input: $n = 1$
Output: `[[1]]`

Constraints:

- $1 \leq n \leq 20$

Community Solutions (Python)

- WalkCC

```
class Solution:
    def generateMatrix(self, n: int) -> list[list[int]]:
        ans = [[0] * n for _ in range(n)]
        count = 1

        for mn in range(n // 2):
            mx = n - mn - 1
            for i in range(mn, mx):
                ans[mn][i] = count
                count += 1

            for i in range(mn, mx):
                ans[i][mx] = count
                count += 1
            for i in range(mx, mn, -1):
                ans[mx][i] = count
                count += 1
            for i in range(mx, mn, -1):
                ans[i][mn] = count
                count += 1

        if n % 2 == 1:
            ans[n // 2][n // 2] = count

        return ans
```

• LeetCode

```
class Solution:
    def generateMatrix(self, n: int) -> List[List[int]]:
        ans = [[0] * n for _ in range(n)]
        dirs = (0, 1, 0, -1, 0)
        i = j = k = 0
        for v in range(1, n * n + 1):
            ans[i][j] = v
            x, y = i + dirs[k], j + dirs[k + 1]
            if x < 0 or x >= n or y < 0 or y >= n or ans[x][y]:
                k = (k + 1) % 4

            i, j = i + dirs[k], j + dirs[k + 1]
        return ans
```

Round 5 · Mid · Medium

p.1663

• SimplyLeet

```
# Function to generate spiral matrix of size n x n
def generateMatrix(n):
    # Initialize an n x n matrix with zeros
    matrix = [[0] * n for _ in range(n)]
    # Initialize the starting number to fill
    num = 1
    # Set the initial boundaries for rows and columns
    top, bottom = 0, n - 1
    left, right = 0, n - 1

    # Continue the loop until the boundaries are valid
    while top <= bottom and left <= right:
        # Traverse from left to right along the top row
        for j in range(left, right + 1):
            matrix[top][j] = num
            num += 1
        top += 1 # Move the top boundary down

        # Traverse from top to bottom along the right column
        for i in range(top, bottom + 1):
            matrix[i][right] = num
            num += 1
        right -= 1 # Move the right boundary left

        # Traverse from right to left along the bottom row if still in valid
        # bounds
        if top <= bottom:
            for j in range(right, left - 1, -1):
                matrix[bottom][j] = num
                num += 1
            bottom -= 1 # Move the bottom boundary up
```

Round 5 · Mid · Medium

p.1664

```

# Traverse from bottom to top along the left column if still in valid
bounds
if left <= right:
    for i in range(bottom, top - 1, -1):
        matrix[i][left] = num
        num += 1
    left += 1 # Move the left boundary right

return matrix

# Example usage:
print(generateMatrix(3))

```

• LC.ca

```

class Solution:
    def generateMatrix(self, n: int) -> List[List[int]]:
        ans = [[0] * n for _ in range(n)]
        dirs = ((0, 1), (1, 0), (0, -1), (-1, 0))
        i = j = k = 0
        for v in range(1, n * n + 1):
            ans[i][j] = v
            x, y = i + dirs[k][0], j + dirs[k][1]
            if x < 0 or y < 0 or x >= n or y >= n or ans[x][y]:
                k = (k + 1) % 4
            i, j = x, y
        return ans

```

◆ Quick Review

Key Insights

- Use four pointers (top, bottom, left, right) to keep track of the matrix boundaries.

Round 5 · Mid · Medium

p.1665

- Traverse the matrix in a spiral: move right along the top row, then down the right column, left along the bottom row, and finally up the left column.
- Update the boundary pointers after each complete traversal of one side.
- Continue until all numbers from 1 to n^2 are filled into the matrix.

Space & Time

Time Complexity: $O(n^2)$

Space Complexity: $O(n^2)$ (required for the output matrix)

Solution

We simulate the spiral movement by initializing four pointers: top, bottom, left, and right, which denote the current boundaries of the matrix that have not yet been filled. For each iteration, we:

- Traverse from left to right along the top boundary, then increment the top pointer.
- Traverse from top to bottom along the right boundary, then decrement the right pointer.
- If there are rows remaining, traverse from right to left along the bottom boundary, then

Round 5 · Mid · Medium

p.1666

decrement the bottom pointer. – If there are columns remaining, traverse from bottom to top along the left boundary, then increment the left pointer. This algorithm ensures that the matrix is filled in a spiral order until all numbers are placed.

Matrix (2D Array)

□ #73 Set Matrix Zeroes

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table · Matrix
 Lists: Grind 169

Problem

Given an $m \times n$ integer matrix `matrix`, if an element is 0, set its entire row and column to 0's.

You must do it in place.

Example 1:

1	1	1
1	0	1
1	1	1

→

1	0	1
0	0	0
1	0	1

Input: `matrix = [[1,1,1],[1,0,1],[1,1,1]]`
 Output: `[[1,0,1],[0,0,0],[1,0,1]]`

Example 2:

0	1	2	0
3	4	5	2
1	3	1	5



0	0	0	0
0	4	5	0
0	3	1	0

Input: matrix = [[0,1,2,0],[3,4,5,2],[1,3,1,5]]

Output: [[0,0,0,0],[0,4,5,0],[0,3,1,0]]

Constraints:

- $m == \text{matrix.length}$
- $n == \text{matrix[0].length}$
- $1 \leq m, n \leq 200$
- $-231 \leq \text{matrix[i][j]} \leq 231 - 1$

Follow up:

- A straightforward solution using $O(mn)$ space is probably a bad idea.
- A simple improvement uses $O(m + n)$ space, but still not the best solution.
- Could you devise a constant space solution?

Community Solutions (Python)

- WalkCC

Round 5 · Mid · Medium

p.1669

```
class Solution:
    def setZeroes(self, matrix: List[List[int]]) -> None:
        m = len(matrix)
        n = len(matrix[0])
        shouldFillFirstRow = 0 in matrix[0]
        shouldFillFirstCol = 0 in list(zip(*matrix))[0]

        # Store the information in the first row and the first column.
        for i in range(1, m):
            for j in range(1, n):
                if matrix[i][j] == 0:
                    matrix[i][0] = 0
                    matrix[0][j] = 0

        # Fill 0s for the matrix except the first row and the first column.
        for i in range(1, m):
            for j in range(1, n):
                if matrix[i][0] == 0 or matrix[0][j] == 0:
                    matrix[i][j] = 0

        # Fill 0s for the first row if needed.
        if shouldFillFirstRow:
            matrix[0] = [0] * n

        # Fill 0s for the first column if needed.
        if shouldFillFirstCol:
            for row in matrix:
                row[0] = 0
```

• LeetCode

```
class Solution:
    def setZeroes(self, matrix: List[List[int]]) -> None:
        m, n = len(matrix), len(matrix[0])
        row = [False] * m
        col = [False] * n
        for i in range(m):
            for j in range(n):
                if matrix[i][j] == 0:
                    row[i] = col[j] = True
        for i in range(m):
            for j in range(n):
                if row[i] or col[j]:
                    matrix[i][j] = 0
```

Round 5 · Mid · Medium

p.1670

```
for j in range(n):
    if row[i] or col[j]:
        matrix[i][j] = 0
```

• SimplyLeet

Python solution for the Set Matrix Zeroes problem

```
def setZeroes(matrix):
```

Step 1: Determine the dimensions of matrix

```
m = len(matrix)
```

```
n = len(matrix[0])
```

Determine if first row or first column should be zeroed

```
first_row_zero = any(matrix[0][j] == 0 for j in range(n))
```

```
first_col_zero = any(matrix[i][0] == 0 for i in range(m))
```

Step 2: Use first row and column as markers

```
for i in range(1, m):
```

```
    for j in range(1, n):
```

```
        if matrix[i][j] == 0:
```

```
            matrix[i][0] = 0 # mark the first element of the row
```

```
            matrix[0][j] = 0 # mark the first element of the column
```

Step 3: Zero out cells based on the markers in first row and column

```
for i in range(1, m):
```

```
    for j in range(1, n):
```

```
        if matrix[i][0] == 0 or matrix[0][j] == 0:
```

```
            matrix[i][j] = 0
```

Step 4: Zero out the first row if needed

```
if first_row_zero:
```

```
    for j in range(n):
```

```
        matrix[0][j] = 0
```

Step 5: Zero out the first column if needed

```
if first_col_zero:
    for i in range(m):
        matrix[i][0] = 0
```

Example usage:

```
# matrix = [[1,1,1],[1,0,1],[1,1,1]]
```

```
# setZeroes(matrix)
```

```
# print(matrix)
```

• LC.ca

```
class Solution:
```

```
    def setZeroes(self, matrix: List[List[int]]) -> None:
```

```
        m, n = len(matrix), len(matrix[0])
```

```
        i0 = any(v == 0 for v in matrix[0])
```

```
        j0 = any(matrix[i][0] == 0 for i in range(m))
```

```
        for i in range(1, m):
```

```
            for j in range(1, n):
```

```
                if matrix[i][j] == 0:
```

```
                    matrix[i][0] = matrix[0][j] = 0
```

```
        for i in range(1, m):
```

```
            for j in range(1, n):
```

```
                if matrix[i][0] == 0 or matrix[0][j] == 0:
```

```
                    matrix[i][j] = 0
```

```
        if i0:
```

```
            for j in range(n):
```

```
                matrix[0][j] = 0
```

```
        if j0:
```

```
            for i in range(m):
```

```
                matrix[i][0] = 0
```

◆ Quick Review

Key Insights

- The challenge is to mark rows and columns that need to be zeroed without using extra space.

- A common trick is to use the first row and first column as markers.
- Special care is needed for the first row and first column, as they are also used as markers.
- The algorithm involves two passes: one for marking and one for updating the matrix based on the markers.

Space & Time

Time Complexity: $O(m * n)$

Space Complexity: $O(1)$ (in-place, using only constant extra space)

Solution

We can solve this problem by using the first row and first column of the matrix as markers. First, determine if the first row or first column needs to be zeroed, then traverse the rest of the matrix and mark the corresponding first row and column positions if a zero is encountered. Finally, update the matrix backward using the markers, and finally update the first row and column if needed.

The algorithm works in three main steps:

- Check if the first row or first column should be zeroed (store this in two variables). –

Round 5 · Mid · Medium

p.1673

Iterate through the rest of the matrix; for any element that is zero, mark its row and column by setting the corresponding element in the first row and first column to zero. – Iterate through the matrix again (excluding the first row and column) and set elements to zero if their row or column marker is zero. – Finally, zero out the first row and/or first column if needed.

This approach avoids extra space usage apart from the two boolean flags while ensuring that the matrix is updated correctly.

Round 5 · Mid · Medium

p.1674

Matrix (2D Array)

□ #289 Game of Life

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Matrix · Simulation
 Lists: AlgoMaster 600

Problem

According to Wikipedia's article: "The Game of Life, also known simply as Life, is a cellular automaton devised by the British mathematician John Horton Conway in 1970."

The board is made up of an $m \times n$ grid of cells, where each cell has an initial state: live (represented by a 1) or dead (represented by a 0). Each cell interacts with its eight neighbors (horizontal, vertical, diagonal) using the following four rules (taken from the above Wikipedia article):

1. Any live cell with fewer than two live neighbors dies as if caused by under-population.
2. Any live cell with two or three live neighbors lives on to the next generation.

Round 5 · Mid · Medium

p.1675

3. Any live cell with more than three live neighbors dies, as if by over-population.
4. Any dead cell with exactly three live neighbors becomes a live cell, as if by reproduction.

The next state of the board is determined by applying the above rules simultaneously to every cell in the current state of the $m \times n$ grid board. In this process, births and deaths occur simultaneously.

Given the current state of the board, update the board to reflect its next state.

Note that you do not need to return anything.

Example 1:

Round 5 · Mid · Medium

p.1676

0	1	0
0	0	1
1	1	1
0	0	0

→

0	0	0
1	0	1
0	1	1
0	1	0

Input: board = [[0,1,0],[0,0,1],[1,1,1],[0,0,0]]
 Output: [[0,0,0],[1,0,1],[0,1,1],[0,1,0]]

Example 2:

1	1
1	0

→

1	1
1	1

Input: board = [[1,1],[1,0]]
 Output: [[1,1],[1,1]]

Constraints:

Round 5 · Mid · Medium

p.1677

- $m == \text{board.length}$
- $n == \text{board}[i].\text{length}$
- $1 \leq m, n \leq 25$
- $\text{board}[i][j]$ is 0 or 1.

Follow up:

- Could you solve it in-place? Remember that the board needs to be updated simultaneously: You cannot update some cells first and then use their updated values to update other cells.
- In this question, we represent the board using a 2D array. In principle, the board is infinite, which would cause problems when the active area encroaches upon the border of the array (i.e., live cells reach the border). How would you address these problems?

Community Solutions (Python)

- WalkCC

Round 5 · Mid · Medium

p.1678

```
class Solution:
    def gameOfLife(self, board: List[List[int]]) -> None:
        m = len(board)
        n = len(board[0])
```

```
        for i in range(m):
            for j in range(n):
                ones = 0
                for x in range(max(0, i - 1), min(m, i + 2)):
                    for y in range(max(0, j - 1), min(n, j + 2)):
```

```
                        ones += board[x][y] & 1
                # Any live cell with two or three live neighbors lives on to the next
                # generation.
                if board[i][j] == 1 and (ones == 3 or ones == 4):
                    board[i][j] |= 0b10
                # Any dead cell with exactly three live neighbors becomes a live cell,
                # as if by reproduction.
                if board[i][j] == 0 and ones == 3:
                    board[i][j] |= 0b10
```

```
        for i in range(m):
            for j in range(n):
                board[i][j] >>= 1
```

• LeetCode

```
class Solution:
    def gameOfLife(self, board: List[List[int]]) -> None:
        m, n = len(board), len(board[0])
        for i in range(m):
            for j in range(n):
                live = -board[i][j]
                for x in range(i - 1, i + 2):
                    for y in range(j - 1, j + 2):
                        if 0 <= x < m and 0 <= y < n and board[x][y] > 0:
                            live += 1
```

```
                if board[i][j] and (live < 2 or live > 3):
                    board[i][j] = 2
                if board[i][j] == 0 and live == 3:
                    board[i][j] = -1
            for i in range(m):
                for j in range(n):
                    if board[i][j] == 2:
                        board[i][j] = 0
                    elif board[i][j] == -1:
                        board[i][j] = 1
```

• SimplyLeet

```
# Python solution
class Solution:
    def gameOfLife(self, board: List[List[int]]) -> None:
        # Dimensions of the board
        rows, cols = len(board), len(board[0])

        # Helper function to count live neighbors
        def count_live_neighbors(r, c):
            live = 0
            for i in range(r-1, r+2): # Loop over row neighbors

                for j in range(c-1, c+2): # Loop over column neighbors
                    if (i == r and j == c) or i < 0 or j < 0 or i >= rows or j
                    >= cols:
                        continue # Skip the cell itself and out-of-bound
                    indices
                    # If neighbor was originally live (1 or -1)
                    if abs(board[i][j]) == 1:
                        live += 1
                return live

        # First pass: update board using intermediate states
        for r in range(rows):
```

```

    for c in range(cols):
        live_neighbors = count_live_neighbors(r, c)
        # Rule 1 or Rule 3: live cell dies if fewer than 2 or more than
3 neighbors
        if board[r][c] == 1 and (live_neighbors < 2 or live_neighbors
> 3):
            board[r][c] = -1 # Mark as cell that was live but now
becomes dead
            # Rule 4: dead cell becomes live if exactly 3 live neighbors
            if board[r][c] == 0 and live_neighbors == 3:
                board[r][c] = 2 # Mark as cell that was dead but now
becomes live

        # Second pass: finalize the board by converting intermediate states to
        final states

    for r in range(rows):
        for c in range(cols):
            # If cell has positive value, set to live; otherwise, set to
dead
            board[r][c] = 1 if board[r][c] > 0 else 0

```

Matrix (2D Array)

□ #498 Diagonal Traverse

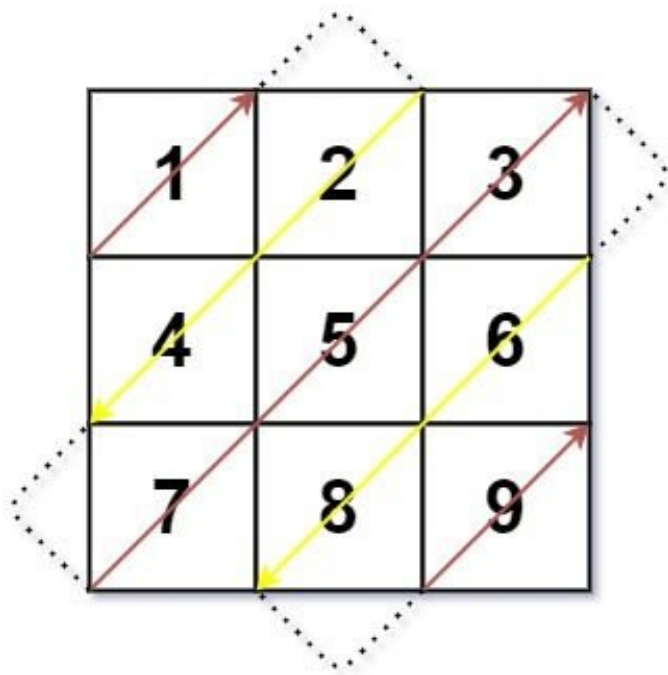
MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Matrix · Simulation
 Lists: CodeSignal

Problem

Given an $m \times n$ matrix `mat`, return an array of all the elements of the array in a diagonal order.

Example 1:



Input: mat = [[1,2,3],[4,5,6],[7,8,9]]
Output: [1,2,4,7,5,3,6,8,9]

Example 2:

Input: mat = [[1,2],[3,4]]
Output: [1,2,3,4]

Constraints:

- $m == \text{mat.length}$
- $n == \text{mat}[i].\text{length}$
- $1 \leq m, n \leq 104$
- $1 \leq m * n \leq 104$

Round 5 · Mid · Medium

p.1683

- $-105 \leq \text{mat}[i][j] \leq 105$

Community Solutions (Python)

• LeetCode

```
class Solution:
    def findDiagonalOrder(self, mat: List[List[int]]) -> List[int]:
        m, n = len(mat), len(mat[0])
        ans = []
        for k in range(m + n - 1):
            t = []
            i = 0 if k < n else k - n + 1
            j = k if k < n else n - 1
            while i < m and j >= 0:
                t.append(mat[i][j])
                i += 1
                j -= 1
            if k % 2 == 0:
                t = t[::-1]
            ans.extend(t)
        return ans
```

• SimplyLeet

```
def findDiagonalOrder(mat):
    if not mat or not mat[0]:
        return []

    m, n = len(mat), len(mat[0])
    result = []
    row, col = 0, 0
    direction = 1 # 1 means upward, -1 means downward

    while len(result) < m * n:
```

Round 5 · Mid · Medium

p.1684

```

result.append(mat[row][col])
if direction == 1: # moving upward
    if col == n - 1: # Hit the right boundary
        row += 1
        direction = -1
    elif row == 0: # Hit the top boundary
        col += 1
        direction = -1
    else:
        row -= 1
        col += 1
else: # moving downward
    if row == m - 1: # Hit the bottom boundary
        col += 1
        direction = 1
    elif col == 0: # Hit the left boundary
        row += 1
        direction = 1
    else:
        row += 1
        col -= 1
return result

# Example usage:
print(findDiagonalOrder([[1,2,3],[4,5,6],[7,8,9]]))

```

• LC.ca

```

class Solution:
    def findDiagonalOrder(self, mat: List[List[int]]) -> List[int]:
        m, n = len(mat), len(mat[0])
        ans = []
        for k in range(m + n - 1):
            t = []
            i = 0 if k < n else k - n + 1
            j = k if k < n else n - 1
            while i < m and j >= 0:
                t.append(mat[i][j])

```

```

i += 1
j -= 1
if k % 2 == 0:
    t = t[::-1]
ans.extend(t)
return ans

```

◆ Quick Review

Key Insights

- Traverse the matrix diagonally with alternating directions: upward (top-right) and downward (bottom-left).
- When the traversal reaches a boundary (top, bottom, left, or right), adjust the position and change the direction.
- Use simulation to process each element exactly once while managing the indices based on the current direction.

Space & Time

Time Complexity: $O(m * n)$ where m is the number of rows and n is the number of columns, as every element is processed once.

Space Complexity: $O(m * n)$ for the output array (excluding the input matrix), while additional space usage remains constant.

Solution

The solution simulates the diagonal traversal using a direction flag (1 for upward and -1 for downward).

- Begin at the matrix's top-left corner.
- Append the current element to the result.
- Depending on the current direction: If moving upward, decrease the row and increase the column. If moving downward, increase the row and decrease the column.
- Adjust for boundary hits: When moving upward: If at the last column, increment the row and switch direction. If at the first row, increment the column and switch direction. When moving downward: If at the last row, increment the column and switch direction. If at the first column, increment the row and switch direction. This process continues until all elements are added to the result array.

Round 5 · Mid · Medium

p.1687

Matrix (2D Array)

★ #723 Candy Crush ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Two Pointers · Matrix · Simulation
 Lists: ★ Premium | Premium 98

Problem

This question is about implementing a basic elimination algorithm for Candy Crush.

Given an $m \times n$ integer array `board` representing the grid of candy where `board[i][j]` represents the type of candy. A value of `board[i][j] == 0` represents that the cell is empty.

The given board represents the state of the game following the player's move. Now, you need to restore the board to a stable state by crushing candies according to the following rules:

- If three or more candies of the same type are adjacent vertically or horizontally, crush them all at the same time – these positions become empty.

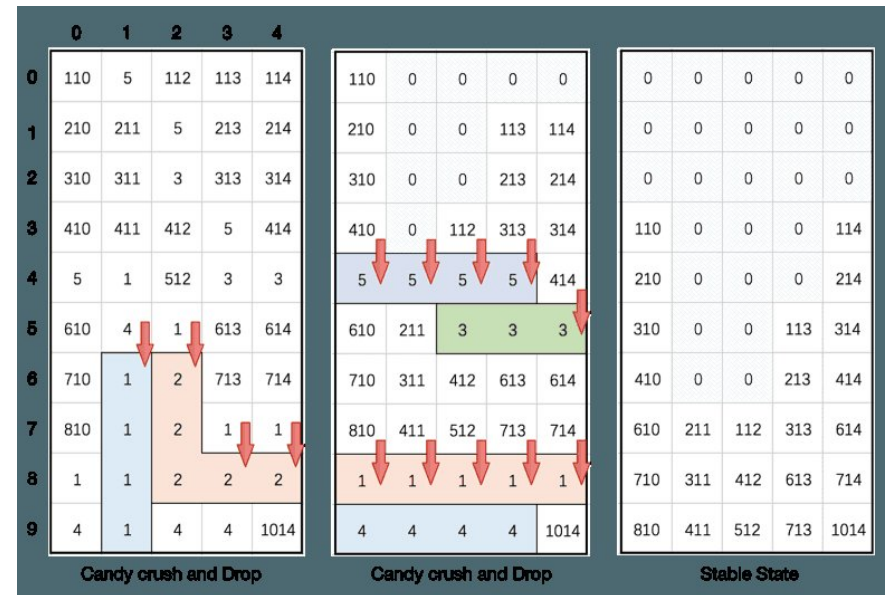
Round 5 · Mid · Medium

p.1688

- After crushing all candies simultaneously, if an empty space on the board has candies on top of itself, then these candies will drop until they hit a candy or bottom at the same time. No new candies will drop outside the top boundary.
- After the above steps, there may exist more candies that can be crushed. If so, you need to repeat the above steps.
- If there does not exist more candies that can be crushed (i.e., the board is stable), then return the current board.

You need to perform the above rules until the board becomes stable, then return the stable board.

Example 1:



Input: board = [[110,5,112,113,114],[210,211,5,213,214],[310,311,3,313,314],[410,411,412,5,414],[5,1,512,3,3],[610,4,1,613,614],[710,1,2,713,714],[810,1,2,1,1],[1,1,2,2,2],[4,1,4,4,1014]]
 Output: [[0,0,0,0,0],[0,0,0,0,0],[0,0,0,0,0],[110,0,0,0,114],[210,0,0,0,214],[310,0,0,113,314],[410,0,0,213,414],[610,211,112,313,614],[710,311,412,613,714],[810,411,512,713,1014]]

Example 2:

Input: board = [[1,3,5,5,2],[3,4,3,3,1],[3,2,4,5,2],[2,4,4,5,5],[1,4,4,1,1]]
 Output: [[1,3,0,0,0],[3,4,0,5,2],[3,2,0,3,1],[2,4,0,5,2],[1,4,3,1,1]]

Constraints:

- $m == \text{board.length}$
- $n == \text{board}[i].\text{length}$
- $3 \leq m, n \leq 50$
- $1 \leq \text{board}[i][j] \leq 2000$

Community Solutions (Python)

• LeetCode

```
class Solution:
    def candyCrush(self, board: List[List[int]]) -> List[List[int]]:
        m, n = len(board), len(board[0])
        run = True
        while run:
            run = False
            for i in range(m):
                for j in range(2, n):
                    if board[i][j] and abs(board[i][j]) == abs(board[i][j - 1]) == abs(
                        board[i][j - 2]
                    ):
                        run = True
                        board[i][j] = board[i][j - 1] = board[i][j - 2] = -abs(
                            board[i][j]
                        )
            for j in range(n):
                for i in range(2, m):
                    if board[i][j] and abs(board[i][j]) == abs(board[i - 1][j]) == abs(
                        board[i - 2][j]
                    ):
                        run = True
                        board[i][j] = board[i - 1][j] = board[i - 2][j] = -abs(
                            board[i][j]
                        )
            if run:
                for j in range(n):
                    k = m - 1
                    for i in range(m - 1, -1, -1):
                        if board[i][j] > 0:
                            board[k][j] = board[i][j]
                            k -= 1
                    while k >= 0:
                        board[k][j] = 0
                        k -= 1
                return board
```

Round 5 · Mid · Medium

p.1691

```
        k -= 1
        while k >= 0:
            board[k][j] = 0
            k -= 1
        return board
```

• SimplyLeet

```
def candyCrush(board):
    # Retrieve board dimensions
    m, n = len(board), len(board[0])
    # Loop until no crushable candies remain
    while True:
        # Set to hold all positions that need to be crushed
        crush_set = set()

        # Check horizontally for crushable candies
        for i in range(m):
            for j in range(n - 2):
                # Check if candy exists and three consecutive candies are the
                same
                if board[i][j] != 0 and abs(board[i][j]) == abs(board[i][j+1])
                == abs(board[i][j+2]):
                    crush_set |= {(i, j), (i, j+1), (i, j+2)}

        # Check vertically for crushable candies
        for i in range(m - 2):
            for j in range(n):
                if board[i][j] != 0 and abs(board[i][j]) == abs(board[i+1][j])
                == abs(board[i+2][j]):
                    crush_set |= {(i, j), (i+1, j), (i+2, j)}
```

Round 5 · Mid · Medium

p.1692

```

        # If no candies need to be crushed, the board is stable; break out of
        the loop.
        if not crush_set:
            break

        # Set all marked positions to 0 (simulate crushing)
        for i, j in crush_set:
            board[i][j] = 0

        # Apply gravity: For each column, move uncrushed candies downwards

    for j in range(n):
        # Pointer for placing the next candy from bottom-up
        write_index = m - 1
        # Traverse each column from bottom to top
        for i in range(m - 1, -1, -1):
            if board[i][j] != 0:
                board[write_index][j] = board[i][j]
                write_index -= 1
        # Fill remaining positions with 0
        for i in range(write_index, -1, -1):

            board[i][j] = 0

    return board

# Example usage:
board = [[110,5,112,113,114],[210,211,5,213,214],[310,311,3,313,314],
         [410,411,412,5,414],[5,1,512,3,3],[610,4,1,613,614],
         [710,1,2,713,714],[810,1,2,1,1],[1,1,2,2,2],[4,1,4,4,1014]]
print(candyCrush(board))

```

• LC.ca

Round 5 · Mid · Medium

p.1693

```

class Solution:
    def candyCrush(self, board: List[List[int]]) -> List[List[int]]:
        m, n = len(board), len(board[0])
        run = True
        while run:
            run = False
            for i in range(m):
                for j in range(n - 2):
                    if (
                        board[i][j] != 0
                        and abs(board[i][j]) == abs(board[i][j + 1])
                        and abs(board[i][j]) == abs(board[i][j + 2])
                    ):
                        run = True
                        board[i][j] = board[i][j + 1] = board[i][j + 2] = -abs(
                            board[i][j]
                        )
            for j in range(n):
                for i in range(m - 2):
                    if (
                        board[i][j] != 0
                        and abs(board[i][j]) == abs(board[i + 1][j])
                        and abs(board[i][j]) == abs(board[i + 2][j])
                    ):
                        run = True
                        board[i][j] = board[i + 1][j] = board[i + 2][j] = -abs(
                            board[i][j]
                        )
            if run:
                for j in range(n):

```

Round 5 · Mid · Medium

p.1694

```

curr = m - 1
for i in range(m - 1, -1, -1):
    if board[i][j] > 0:
        board[curr][j] = board[i][j]
        curr -= 1
while curr > -1:
    board[curr][j] = 0
    curr -= 1
return board

```

◆ Quick Review

Key Insights

- Use a simulation approach: repeatedly detect crushable candies and simulate their removal.
- Traverse horizontally and vertically to mark candies that are in groups of three or more.
- Instead of removing candies immediately, mark them for removal and update the board once for the entire pass.
- After crushing, simulate gravity by shifting non-zero candies downward in each column.
- Continue the process until no further candy crushes can be detected.

Space & Time

Time Complexity: $O((m \cdot n)^2)$ in the worst case since each iteration includes scanning the $m \times n$ grid and then

Round 5 · Mid · Medium

p.1695

applying gravity.

Space Complexity: $O(m \cdot n)$

for maintaining additional markers or flags (or using the board itself with in-place modifications).

Solution

The approach involves iteratively scanning the board to identify candy groups that should be crushed using two passes:

- Horizontal Check – For each row, check segments of 3 consecutive candies.
- Vertical Check – For each column, check segments of 3 consecutive candies. If any candy qualifies, mark its position for crushing. After marking, update the board by turning all marked positions into 0. Then, for each column, drop down non-zero candies such that all empty cells (zeros) are at the top. Repeat this process until no group of 3 or more adjacent candies is found. Key data structures include the board array and a temporary marker (can be implicit by using signs or a separate boolean matrix) for tracking candies to crush. The algorithm ensures a stable board before returning it.

Round 5 · Mid · Medium

p.1696

LinkedList In-place Reversal

Round 5 · Mid · Medium · 1 question

Round 5 · Mid · Medium

p.1697

LinkedList In-place Reversal

□ #92 Reverse Linked List II

MED

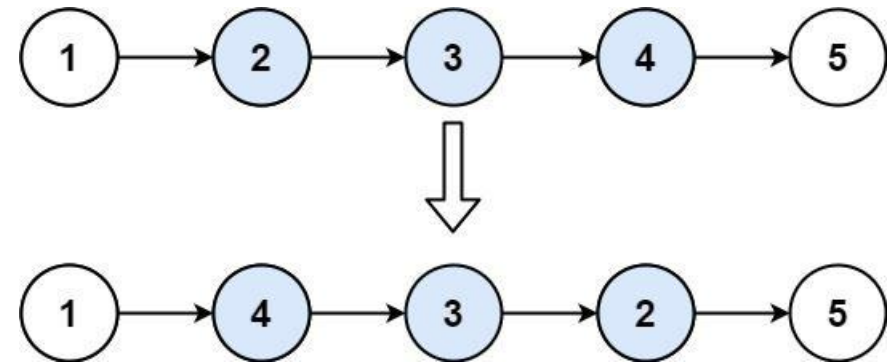
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List

Lists: AlgoMaster 600

Problem

Given the head of a singly linked list and two integers $left$ and $right$ where $left \leq right$, reverse the nodes of the list from position $left$ to position $right$, and return the reversed list.

Example 1:



Input: head = [1,2,3,4,5], left = 2, right = 4
Output: [1,4,3,2,5]

Example 2:

Round 5 · Mid · Medium

p.1698

Input: head = [5], left = 1, right = 1
Output: [5]

Constraints:

- The number of nodes in the list is n.
- $1 \leq n \leq 500$
- $-500 \leq \text{Node.val} \leq 500$
- $1 \leq \text{left} \leq \text{right} \leq n$

Follow up: Could you do it in one pass?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def reverseBetween(
        self,
        head: ListNode | None,
        left: int,
        right: int,
    ) -> ListNode | None:
        if left == 1:
            return self.reverseN(head, right)

        head.next = self.reverseBetween(head.next, left - 1, right - 1)
        return head

    def reverseN(self, head: ListNode | None, n: int) -> ListNode | None:
        if n == 1:
            return head

        newHead = self.reverseN(head.next, n - 1)
        headNext = head.next
        head.next = headNext.next
```

Round 5 · Mid · Medium

p.1699

```
headNext.next = head
return newHead
```

```
class Solution:
    def reverseBetween(self, head: ListNode, m: int, n: int) -> ListNode:
        if not head and m == n:
            return head

        dummy = ListNode(0, head)
        prev = dummy

        for _ in range(m - 1):
            prev = prev.next # Point to the node before the sublist [m, n].

        tail = prev.next # Be the tail of the sublist [m, n].

        # Reverse the sublist [m, n] one by one.
        for _ in range(n - m):
            cache = tail.next
            tail.next = cache.next
            cache.next = prev.next
            prev.next = cache

        return dummy.next
```

• LeetDoocs

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def reverseBetween(
        self, head: Optional[ListNode], left: int, right: int
    ) -> Optional[ListNode]:
        if head.next is None or left == right:
```

Round 5 · Mid · Medium

p.1700

```

        return head
    dummy = ListNode(0, head)
    pre = dummy
    for _ in range(left - 1):
        pre = pre.next
    p, q = pre, pre.next
    cur = q
    for _ in range(right - left + 1):
        t = cur.next
        cur.next = pre

        pre, cur = cur, t
    p.next = pre
    q.next = cur
    return dummy.next

```

• SimplyLeet

```

# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def reverseBetween(self, head: ListNode, left: int, right: int) -> ListNod
e:
        # Create a dummy node that points to the head to simplify edge cases.
        dummy = ListNode(0)

        dummy.next = head
        # Initialize pointer 'prev' to the dummy node.
        prev = dummy
        # Move 'prev' to the node immediately before the left-th node.
        for i in range(left - 1):
            prev = prev.next
        # 'curr' will point to the first node of the sublist to be reversed.
        curr = prev.next
        # Reverse the sublist from left to right.
        for i in range(right - left):

```

```

        temp = curr.next          # 'temp' is the node to be moved.
        curr.next = temp.next     # Bypass 'temp' in the current sublist.
        temp.next = prev.next    # Insert 'temp' right after 'prev'.
        prev.next = temp         # Update 'prev' to point to the moved
node.
        # Return the new head, which is dummy.next.
    return dummy.next

```

Monotonic Stack

Round 5 · Mid · Medium · 12 questions

Round 5 · Mid · Medium

p.1703

Monotonic Stack

□ #402 Remove K Digits

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Stack · Greedy · Monotonic Stack
Lists: AlgoMaster 600

Problem

Given string `num` representing a non-negative integer `num`, and an integer `k`, return the smallest possible integer after removing `k` digits from `num`.

Example 1:

Input: `num = "1432219"`, `k = 3`
Output: `"1219"`
Explanation: Remove the three digits 4, 3, and 2 to form the new number 1219 which is the smallest.

Example 2:

Input: `num = "10200"`, `k = 1`
Output: `"200"`
Explanation: Remove the leading 1 and the number is 200. Note that the output must not contain leading zeroes.

Example 3:

Input: `num = "10"`, `k = 2`
Output: `"0"`
Explanation: Remove all the digits from the number and it is left with nothing which is 0.

Round 5 · Mid · Medium

p.1704

Constraints:

- $1 \leq k \leq \text{num.length} \leq 105$
- num consists of only digits.
- num does not have any leading zeros except for the zero itself.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def removeKdigits(self, num: str, k: int) -> str:
        if len(num) == k:
            return '0'

        ans = []
        stack = []

        for i, digit in enumerate(num):
            while k > 0 and stack and stack[-1] > digit:

                stack.pop()
                k -= 1
            stack.append(digit)

        for _ in range(k):
            stack.pop()

        for c in stack:
            if c == '0' and not ans:
                continue
            ans.append(c)

        return ''.join(ans) if ans else '0'
```

• LeetCode

Round 5 · Mid · Medium

p.1705

```
class Solution:
    def removeKdigits(self, num: str, k: int) -> str:
        stk = []
        remain = len(num) - k
        for c in num:
            while k and stk and stk[-1] > c:
                stk.pop()
                k -= 1
            stk.append(c)
        return ''.join(stk[:remain]).lstrip('0') or '0'
```

• SimplyLeet

```
# Python solution with line-by-line comments
def removeKdigits(num: str, k: int) -> str:
    # Initialize a stack to build the final number.
    stack = []

    # Traverse each digit in the input number.
    for digit in num:
        # Remove digits from the stack if the current digit is smaller,
        # making sure we still have removals available.
        while k > 0 and stack and stack[-1] > digit:

            stack.pop()
            k -= 1
        # Append the current digit to the stack.
        stack.append(digit)

    # If there are still removals left, remove digits from the end.
    while k > 0:
        stack.pop()
        k -= 1

    # Construct the result string and remove any leading zeros.
    result = ''.join(stack).lstrip('0')

    # Return "0" if the resulting string is empty.
    return result if result else "0"

# Example usage:
# print(removeKdigits("1432219", 3)) # expected output: "1219"
```

Round 5 · Mid · Medium

p.1706

Monotonic Stack

#456 132 Pattern

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Binary Search · Stack · Monotonic Stack
 · Ordered Set

Lists: AlgoMaster 600

Problem

Given an array of n integers `nums`, a 132 pattern is a subsequence of three integers `nums[i]`, `nums[j]` and `nums[k]` such that $i < j < k$ and $nums[i] < nums[k] < nums[j]$.

Return true if there is a 132 pattern in `nums`, otherwise, return false.

Example 1:

Input: `nums = [1,2,3,4]`
 Output: false
 Explanation: There is no 132 pattern in the sequence.

Example 2:

Input: `nums = [3,1,4,2]`
 Output: true
 Explanation: There is a 132 pattern in the sequence: [1, 4, 2].

Example 3:

Round 5 · Mid · Medium

p.1707

Input: `nums = [-1,3,2,0]`
 Output: true
 Explanation: There are three 132 patterns in the sequence: [-1, 3, 2], [-1, 3, 0] and [-1, 2, 0].

Constraints:

- $n == \text{nums.length}$
- $1 \leq n \leq 2 * 10^5$
- $-109 \leq \text{nums}[i] \leq 109$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def find132pattern(self, nums: list[int]) -> bool:
        stack = [] # a decreasing stack
        ak = -math.inf # Find a seq, where ai < ak < aj.

        for num in reversed(nums):
            # If ai < ak, done because ai must < aj.
            if num < ak:
                return True
            while stack and stack[-1] < num:
                ak = stack[-1]
                stack.pop()
            stack.append(num) # `nums[i]` is a candidate of aj.

        return False
```

• LeetDoocs

Round 5 · Mid · Medium

p.1708

```
class Solution:
    def find132pattern(self, nums: List[int]) -> bool:
        vk = -inf
        stk = []
        for x in nums[::-1]:
            if x < vk:
                return True
            while stk and stk[-1] < x:
                vk = stk.pop()
            stk.append(x)
```

```
        return False
```

```
class BinaryIndexedTree:
    def __init__(self, n):
        self.n = n
        self.c = [0] * (n + 1)

    def update(self, x, delta):
        while x <= self.n:
            self.c[x] += delta
            x += x & -x
```

```
    def query(self, x):
        s = 0
        while x:
            s += self.c[x]
            x -= x & -x
        return s
```

```
class Solution:
    def find132pattern(self, nums: List[int]) -> bool:
```

```
s = sorted(set(nums))
n = len(nums)
left = [inf] * (n + 1)
for i, x in enumerate(nums):
    left[i + 1] = min(left[i], x)
tree = BinaryIndexedTree(len(s))
for i in range(n - 1, -1, -1):
    x = bisect_left(s, nums[i]) + 1
    y = bisect_left(s, left[i]) + 1
    if x > y and tree.query(x - 1) > tree.query(y):
```

```
        return True
    tree.update(x, 1)
    return False
```

• SimplyLeet

Python solution for the 132 Pattern problem

```
def find132pattern(nums):
    # Initialize candidate for nums[k] with a very small value
    candidate = float('-inf')
    stack = [] # Stack to keep potential nums[j] elements

    # Traverse the array in reverse order
    for num in reversed(nums):
        # If current number is less than candidate, we found a valid 132
        pattern
```

```
        if num < candidate:
            return True
        # Update the candidate while current number is greater than the element
        at stack's top
        while stack and num > stack[-1]:
            candidate = stack.pop()
        # Push the current number onto the stack for potential future use
        stack.append(num)
    return False
```

Example usage:

```
# print(find132pattern([3, 1, 4, 2])) # Output: True
```

Monotonic Stack

#503 Next Greater Element II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Stack · Monotonic Stack
 Lists: AlgoMaster 600

Problem

Given a circular integer array `nums` (i.e., the next element of `nums[nums.length - 1]` is `nums[0]`), return the next greater number for every element in `nums`.

The next greater number of a number `x` is the first greater number to its traversing-order next in the array, which means you could search circularly to find its next greater number. If it doesn't exist, return `-1` for this number.

Example 1:

Input: `nums = [1,2,1]`
 Output: `[2,-1,2]`
 Explanation: The first 1's next greater number is 2;
 The number 2 can't find next greater number.
 The second 1's next greater number needs to search circularly, which is also 2.

Example 2:

Input: `nums = [1,2,3,4,3]`
 Output: `[2,3,4,-1,4]`

Round 5 · Mid · Medium

p.1711

Constraints:

- $1 \leq \text{nums.length} \leq 104$
- $-109 \leq \text{nums}[i] \leq 109$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def nextGreaterElements(self, nums: List[int]) -> List[int]:
        n = len(nums)
        ans = [-1] * n
        stack = [] # a decreasing stack storing indices

        for i in range(n * 2):
            num = nums[i % n]
            while stack and nums[stack[-1]] < num:
                ans[stack.pop()] = num

            if i < n:
                stack.append(i)

        return ans
```

• LeetDoocs

```
class Solution:
    def nextGreaterElements(self, nums: List[int]) -> List[int]:
        n = len(nums)
        ans = [-1] * n
        stk = []
        for i in range(n * 2 - 1, -1, -1):
            i %= n
            while stk and stk[-1] <= nums[i]:
                stk.pop()
            if stk:
                ans[i] = stk[-1]
            stk.append(nums[i])
        return ans
```

Round 5 · Mid · Medium

p.1712

• SimplyLeet

```
# Python solution
def nextGreaterElements(nums):
    n = len(nums)
    result = [-1] * n # Initialize result list with -1
    stack = [] # Stack to store indices

    # Traverse the list twice to account for circularity
    for i in range(2 * n - 1, -1, -1):
        current_index = i % n # Get current index in circular manner
        # Remove elements that are less than or equal to nums[current_index]

        while stack and nums[stack[-1]] <= nums[current_index]:
            stack.pop()

        # If stack is not empty, the top element is the next greater element
        if stack:
            result[current_index] = nums[stack[-1]]

        # Push current index to the stack for future comparisons
        stack.append(current_index)

    return result

# Example usage:
# nums = [1, 2, 1]
# print(nextGreaterElements(nums)) # Output: [2, -1, 2]
```

Monotonic Stack

□ #581 Shortest Unsorted Continuous Subarray

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Stack · Greedy · Sorting
· Monotonic Stack

Lists: AlgoMaster 600

Problem

Given an integer array `nums`, you need to find one continuous subarray such that if you only sort this subarray in non-decreasing order, then the whole array will be sorted in non-decreasing order.

Return the shortest such subarray and output its length.

Example 1:

Input: `nums = [2,6,4,8,10,9,15]`
Output: 5
Explanation: You need to sort `[6, 4, 8, 10, 9]` in ascending order to make the whole array sorted in ascending order.

Example 2:

Input: `nums = [1,2,3,4]`
Output: 0

Example 3:

Input: nums = [1]
Output: 0

Constraints:

- $1 \leq \text{nums.length} \leq 104$
- $-105 \leq \text{nums}[i] \leq 105$

Follow up: Can you solve it in $O(n)$ time complexity?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findUnsortedSubarray(self, nums: List[int]) -> int:
        mn = math.inf
        mx = -math.inf
        flag = False

        for i in range(1, len(nums)):
            if nums[i] < nums[i - 1]:
                flag = True
            if flag:
                mn = min(mn, nums[i])

        flag = False

        for i in reversed(range(len(nums) - 1)):
            if nums[i] > nums[i + 1]:
                flag = True
            if flag:
                mx = max(mx, nums[i])

        return mx - mn + 1
```

Round 5 · Mid · Medium

p.1715

```
for l in range(len(nums)):
    if nums[l] > mn:
        break

for r, num in reversed(list(enumerate(nums))):
    if num < mx:
        break

return 0 if l >= r else r - l + 1
```

• LeetCode

```
class Solution:
    def findUnsortedSubarray(self, nums: List[int]) -> int:
        arr = sorted(nums)
        l, r = 0, len(nums) - 1
        while l <= r and nums[l] == arr[l]:
            l += 1
        while l <= r and nums[r] == arr[r]:
            r -= 1
        return r - l + 1
```

```
class Solution:
    def findUnsortedSubarray(self, nums: List[int]) -> int:
        mi, mx = inf, -inf
        l = r = -1
        n = len(nums)
        for i, x in enumerate(nums):
            if mx > x:
                r = i
            else:
                mx = x

        if mi < nums[n - i - 1]:
            l = n - i - 1
        else:
            mi = nums[n - i - 1]
        return 0 if r == -1 else r - l + 1
```

• SimplyLeet

Round 5 · Mid · Medium

p.1716

```
# Python solution with detailed comments
def findUnsortedSubarray(nums):
    n = len(nums)
    left, right = -1, -1
    max_seen = nums[0]
    min_seen = nums[-1]

    # Scan from left to right to find the right boundary
    for i in range(1, n):
        max_seen = max(max_seen, nums[i])

    # if current element is less than the max seen so far, update right
    # boundary
    if nums[i] < max_seen:
        right = i

    # Scan from right to left to find the left boundary
    for i in range(n - 2, -1, -1):
        min_seen = min(min_seen, nums[i])
    # if current element is greater than the min seen so far, update left
    # boundary
    if nums[i] > min_seen:
        left = i

    # If right boundary was never updated, array is sorted.
    if right == -1:
        return 0
    return right - left + 1

# Example usage:
print(findUnsortedSubarray([2,6,4,8,10,9,15])) # Output: 5
```

Round 5 · Mid · Medium

p.1717

Monotonic Stack

□ #739 Daily Temperatures

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Stack · Monotonic Stack
 Lists: Grind 169

Problem

Given an array of integers temperatures represents the daily temperatures, return an array answer such that answer[i] is the number of days you have to wait after the ith day to get a warmer temperature. If there is no future day for which this is possible, keep answer[i] == 0 instead.

Example 1:

Input: temperatures = [73,74,75,71,69,72,76,73]
 Output: [1,1,4,2,1,1,0,0]

Example 2:

Input: temperatures = [30,40,50,60]
 Output: [1,1,1,0]

Example 3:

Input: temperatures = [30,60,90]
 Output: [1,1,0]

Round 5 · Mid · Medium

p.1718

Constraints:

- $1 \leq \text{temperatures.length} \leq 105$
- $30 \leq \text{temperatures}[i] \leq 100$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def dailyTemperatures(self, temperatures: List[int]) -> List[int]:
        ans = [0] * len(temperatures)
        stack = [] # a decreasing stack

        for i, temperature in enumerate(temperatures):
            while stack and temperature > temperatures[stack[-1]]:
                index = stack.pop()
                ans[index] = i - index
            stack.append(i)

        return ans
```

• LeetCode

```
class Solution:
    def dailyTemperatures(self, temperatures: List[int]) -> List[int]:
        stk = []
        n = len(temperatures)
        ans = [0] * n
        for i in range(n - 1, -1, -1):
            while stk and temperatures[stk[-1]] <= temperatures[i]:
                stk.pop()
            if stk:
                ans[i] = stk[-1] - i
            stk.append(i)
        return ans
```

• SimplyLeet

Round 5 · Mid · Medium

p.1719

```
def dailyTemperatures(temperatures):
    n = len(temperatures)
    # Initialize answer list with zeros
    answer = [0] * n
    # Stack to keep indices of temperatures with unresolved next warmer days
    stack = []
    # Iterate through the temperature list
    for i, temp in enumerate(temperatures):
        # Check if the current temperature resolves any previous indices
        while stack and temperatures[stack[-1]] < temp:
            prev_index = stack.pop() # Get the index of the previous colder day
            answer[prev_index] = i - prev_index # Update the number of days waited
        # Push the current index onto the stack
        stack.append(i)
    return answer

# Example usage:
if __name__ == "__main__":
    temps = [73,74,75,71,69,72,76,73]
    print(dailyTemperatures(temps))
```

• LC.ca

```
class Solution:
    def dailyTemperatures(self, temperatures: List[int]) -> List[int]:
        ans = [0] * len(temperatures)
        stk = []
        for i, t in enumerate(temperatures):
            while stk and temperatures[stk[-1]] < t:
                j = stk.pop()
                ans[j] = i - j
            stk.append(i)
        return ans
```

◆ Quick Review

Key Insights

Round 5 · Mid · Medium

p.1720

- Use a monotonic stack to keep track of indices of days with unresolved warmer temperatures.
- As you iterate through the temperatures, resolve previous days that have found a warmer temperature.
- The index differences yield the number of days waited until a warmer temperature.

Space & Time

Time Complexity: $O(n)$, where n is the number of temperatures, because each index is pushed and popped from the stack at most once.

Space Complexity: $O(n)$ for storing the stack and the answer array.

Solution

The solution uses a monotonic stack to track the indices whose next warmer temperature has not been found yet. As you iterate through the temperature list, compare the current temperature with the temperature at the index stored at the top of the stack. If the current temperature is higher, it means you've found a warmer day for the day represented by the index from the stack. Pop that index, calculate

the difference between the current index and the popped index, and store the result. Push the current index onto the stack for future comparisons. This approach efficiently finds the answer without nested iterations.

Monotonic Stack

☐ #769 Max Chunks To Make Sorted

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Stack · Greedy · Sorting · Monotonic Stack

Lists: AlgoMaster 600

Problem

You are given an integer array `arr` of length `n` that represents a permutation of the integers in the range `[0, n - 1]`.

We split `arr` into some number of chunks (i.e., partitions), and individually sort each chunk. After concatenating them, the result should equal the sorted array.

Return the largest number of chunks we can make to sort the array.

Example 1:

Input: `arr = [4,3,2,1,0]`
Output: 1
Explanation:
Splitting into two or more chunks will not return the required result.
For example, splitting into `[4, 3]`, `[2, 1, 0]` will result in `[3, 4, 0, 1, 2]`, which isn't sorted.

Example 2:

Round 5 · Mid · Medium

p.1723

Input: `arr = [1,0,2,3,4]`
Output: 4
Explanation:
We can split into two chunks, such as `[1, 0]`, `[2, 3, 4]`.
However, splitting into `[1, 0]`, `[2]`, `[3]`, `[4]` is the highest number of chunks possible.

Constraints:

- `n == arr.length`
- `1 <= n <= 10`
- `0 <= arr[i] < n`
- All the elements of `arr` are unique.

Community Solutions (Python)

☐ • WalkCC

```
class Solution:
    def maxChunksToSorted(self, arr: list[int]) -> int:
        ans = 0
        mx = -math.inf

        for i, a in enumerate(arr):
            mx = max(mx, a)
            if mx == i:
                ans += 1

        return ans
```

☐ • LeetDoocs

Round 5 · Mid · Medium

p.1724

```
class Solution:
    def maxChunksToSorted(self, arr: List[int]) -> int:
        mx = ans = 0
        for i, v in enumerate(arr):
            mx = max(mx, v)
            if i == mx:
                ans += 1
        return ans
```

```
class Solution:
    def maxChunksToSorted(self, arr: List[int]) -> int:
        stk = []
        for v in arr:
            if not stk or v >= stk[-1]:
                stk.append(v)
            else:
                mx = stk.pop()
                while stk and stk[-1] > v:
                    stk.pop()
                stk.append(mx)
        return len(stk)
```

• SimplyLeet

```
# Define the function
def max_chunks_to_sorted(arr):
    # Initialize count of chunks and the maximum value seen so far
    chunk_count = 0
    max_so_far = 0

    # Iterate over the array with index
    for i, num in enumerate(arr):
        # Update the maximum value encountered
        max_so_far = max(max_so_far, num)
```

```
# Check if the current max equals the index
if max_so_far == i:
    # Valid chunk identified, increment count
    chunk_count += 1

# Return the total number of chunks
return chunk_count

# Sample test
print(max_chunks_to_sorted([1,0,2,3,4])) # Expected output: 4
```

Monotonic Stack

□ #901 Online Stock Span

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode Stack · Design · Monotonic Stack · Data Stream Lists: AlgoMaster 600

Problem

Design an algorithm that collects daily price quotes for some stock and returns the span of that stock's price for the current day.

The span of the stock's price in one day is the maximum number of consecutive days (starting from that day and going backward) for which the stock price was less than or equal to the price of that day.

- For example, if the prices of the stock in the last four days is [7,2,1,2] and the price of the stock today is 2, then the span of today is 4 because starting from today, the price of the stock was less than or equal 2 for 4 consecutive days.
- Also, if the prices of the stock in the last four days is [7,34,1,2] and the price of the

Round 5 · Mid · Medium

p.1727

stock today is 8, then the span of today is 3 because starting from today, the price of the stock was less than or equal 8 for 3 consecutive days.

Implement the StockSpanner class:

- StockSpanner() Initializes the object of the class.
- int next(int price) Returns the span of the stock's price given that today's price is price.

Example 1:

```
Input
["StockSpanner", "next", "next", "next", "next", "next", "next", "next"]
[[], [100], [80], [60], [70], [60], [75], [85]]
Output
[null, 1, 1, 1, 2, 1, 4, 6]

Explanation
StockSpanner stockSpanner = new StockSpanner();
```

Constraints:

- $1 \leq \text{price} \leq 105$
- At most 104 calls will be made to next.

Community Solutions (Python)

- • WalkCC

Round 5 · Mid · Medium

p.1728

```
class StockSpanner:
    def __init__(self):
        self.stack = [] # (price, span)

    def next(self, price: int) -> int:
        span = 1
        while self.stack and self.stack[-1][0] <= price:
            span += self.stack.pop()[1]
        self.stack.append((price, span))
        return span
```

• LeetCode

```
class StockSpanner:
    def __init__(self):
        self.stk = []

    def next(self, price: int) -> int:
        cnt = 1
        while self.stk and self.stk[-1][0] <= price:
            cnt += self.stk.pop()[1]
        self.stk.append((price, cnt))
        return cnt
```

```
# Your StockSpanner object will be instantiated and called as such:
# obj = StockSpanner()
# param_1 = obj.next(price)
```

• SimplyLeet

Round 5 · Mid · Medium

p.1729

```
class StockSpanner:
    def __init__(self):
        # Stack will store pairs of (price, span)
        self.stack = []

    def next(self, price: int) -> int:
        span = 1 # Start with a span of 1 for the current price
        # While the stack is not empty and the current price is greater than or
        # equal to the price on top of the stack
        while self.stack and self.stack[-1][0] <= price:
            # Add the span from the popped element to the current span

            span += self.stack.pop()[1]
            # Push the current price and its span onto the stack
            self.stack.append((price, span))
        return span

# Example usage:
# stockSpanner = StockSpanner()
# print(stockSpanner.next(100)) # Output: 1
# print(stockSpanner.next(80)) # Output: 1
# print(stockSpanner.next(60)) # Output: 1

# print(stockSpanner.next(70)) # Output: 2
# print(stockSpanner.next(60)) # Output: 1
# print(stockSpanner.next(75)) # Output: 4
# print(stockSpanner.next(85)) # Output: 6
```

Round 5 · Mid · Medium

p.1730

Monotonic Stack

#907 Sum of Subarray Minimums

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Dynamic Programming · Stack ·
 Monotonic Stack
 Lists: AlgoMaster 600

Problem

Given an array of integers `arr`, find the sum of `min(b)`, where `b` ranges over every (contiguous) subarray of `arr`. Since the answer may be large, return the answer modulo $10^9 + 7$.

Example 1:

Input: `arr = [3,1,2,4]`
 Output: 17
 Explanation:
 Subarrays are [3], [1], [2], [4], [3,1], [1,2], [2,4], [3,1,2], [1,2,4], [3,1,2,4].
 Minimums are 3, 1, 2, 4, 1, 1, 2, 1, 1, 1.
 Sum is 17.

Example 2:

Input: `arr = [11,81,94,43,3]`
 Output: 444

Constraints:

- $1 \leq \text{arr.length} \leq 3 \times 10^4$

Round 5 · Mid · Medium

p.1731

- $1 \leq \text{arr}[i] \leq 3 \times 10^4$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def sumSubarrayMins(self, arr: list[int]) -> int:
        MOD = 1_000_000_007
        n = len(arr)
        ans = 0
        # prevMin[i] := index k s.t. arr[k] is the previous minimum in arr[:i]
        prevMin = [-1] * n
        # nextMin[i] := index k s.t. arr[k] is the next minimum in arr[i + 1:]
        nextMin = [n] * n
        stack = []

        for i, a in enumerate(arr):
            while stack and arr[stack[-1]] > a:
                index = stack.pop()
                nextMin[index] = i
            if stack:
                prevMin[i] = stack[-1]
            stack.append(i)

        for i, a in enumerate(arr):
            ans += a * (i - prevMin[i]) * (nextMin[i] - i)
            ans %= MOD

        return ans
```

• LeetDoocs

Round 5 · Mid · Medium

p.1732

```

class Solution:
    def sumSubarrayMins(self, arr: List[int]) -> int:
        n = len(arr)
        left = [-1] * n
        right = [n] * n
        stk = []
        for i, v in enumerate(arr):
            while stk and arr[stk[-1]] >= v:
                stk.pop()
            if stk:
                left[i] = stk[-1]
            stk.append(i)

        stk = []
        for i in range(n - 1, -1, -1):
            while stk and arr[stk[-1]] > arr[i]:
                stk.pop()
            if stk:
                right[i] = stk[-1]
            stk.append(i)

        mod = 10**9 + 7
        return sum((i - left[i]) * (right[i] - i) * v for i, v in enumerate(arr)) % mod

```

• SimplyLeet

```

# Python solution with detailed comments
class Solution:
    def sumSubarrayMins(self, arr):
        MOD = 10**9 + 7 # modulo value
        n = len(arr)
        left = [0] * n # stores the number of contiguous subarrays ending at
        i where arr[i] is the minimum
        right = [0] * n # stores the number of contiguous subarrays starting
        at i where arr[i] is the minimum

        stack = []
        # Calculate left spans

```

Round 5 · Mid · Medium

p.1733

```

for i in range(n):
    count = 1 # each element stands as a subarray itself
    # Pop while top element is greater than arr[i]
    while stack and arr[stack[-1][0]] > arr[i]:
        count += stack.pop()[1]
    stack.append((i, count))
    left[i] = count

stack = []
# Calculate right spans by iterating from right to left

for i in range(n - 1, -1, -1):
    count = 1
    # Use >= here for the right side to correctly handle duplicates
    while stack and arr[stack[-1][0]] >= arr[i]:
        count += stack.pop()[1]
    stack.append((i, count))
    right[i] = count

total = 0
# Combine the contributions of each element

for i in range(n):
    total = (total + arr[i] * left[i] * right[i]) % MOD
return total

```

Round 5 · Mid · Medium

p.1734

Monotonic Stack

#962 Maximum Width Ramp

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Stack · Monotonic Stack
 Lists: Denny Zhang

Problem

A ramp in an integer array `nums` is a pair (i, j) for which $i < j$ and $nums[i] \leq nums[j]$. The width of such a ramp is $j - i$.

Given an integer array `nums`, return the maximum width of a ramp in `nums`. If there is no ramp in `nums`, return 0.

Example 1:

Input: `nums = [6,0,8,2,1,5]`
 Output: 4
 Explanation: The maximum width ramp is achieved at $(i, j) = (1, 5)$: `nums[1] = 0` and `nums[5] = 5`.

Example 2:

Input: `nums = [9,8,1,0,1,9,4,0,4,1]`
 Output: 7
 Explanation: The maximum width ramp is achieved at $(i, j) = (2, 9)$: `nums[2] = 1` and `nums[9] = 1`.

Constraints:

- $2 \leq \text{nums.length} \leq 5 * 10^4$

Round 5 · Mid · Medium

p.1735

- $0 \leq \text{nums}[i] \leq 5 * 10^4$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxWidthRamp(self, nums: List[int]) -> int:
        ans = 0
        stack = []

        for i, num in enumerate(nums):
            if stack == [] or num <= nums[stack[-1]]:
                stack.append(i)

        for i, num in reversed(list(enumerate(nums))):
            while stack and num >= nums[stack[-1]]:
                ans = max(ans, i - stack.pop())

        return ans
```

• LeetDoocs

```
class Solution:
    def maxWidthRamp(self, nums: List[int]) -> int:
        stk = []
        for i, v in enumerate(nums):
            if not stk or nums[stk[-1]] > v:
                stk.append(i)

        ans = 0
        for i in range(len(nums) - 1, -1, -1):
            while stk and nums[stk[-1]] <= nums[i]:
                ans = max(ans, i - stk.pop())

        if not stk:
            break
        return ans
```

• SimplyLeet

Round 5 · Mid · Medium

p.1736


```
# Python solution for Maximum Width Ramp

def maxWidthRamp(nums):
    n = len(nums)
    stack = []
    # Build a stack of indices where array values are in decreasing order
    for i in range(n):
        if not stack or nums[i] < nums[stack[-1]]:
            stack.append(i)
    max_width = 0

    # Iterate backwards to find the maximum width ramp
    for j in range(n - 1, -1, -1):
        while stack and nums[j] >= nums[stack[-1]]:
            max_width = max(max_width, j - stack.pop())
    return max_width

# Example usage:
nums = [6, 0, 8, 2, 1, 5]
print(maxWidthRamp(nums)) # Output: 4
```

• LC.ca

```
class Solution:
    def maxWidthRamp(self, nums: List[int]) -> int:
        stk = []
        for i, v in enumerate(nums):
            if not stk or nums[stk[-1]] > v:
                stk.append(i)
        ans = 0
        for i in range(len(nums) - 1, -1, -1):
            while stk and nums[stk[-1]] <= nums[i]:
                ans = max(ans, i - stk.pop())

        if not stk:
            break
        return ans
```

◆ Quick Review

Key Insights

Round 5 · Mid · Medium

p.1737

- We need to identify pairs (i, j) where $i < j$ and $\text{nums}[i] \leq \text{nums}[j]$.
- A monotonic decreasing stack can be constructed to maintain candidate starting indices.
- By iterating from left to right, we push indices onto the stack only when they have a smaller value than the last.
- Then, a backward pass from the end of the array allows us to efficiently match each element with the smallest possible candidate index from the stack.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Solution

The solution is built in two phases:

- Build a decreasing stack of indices by iterating through the array (only push an index if its value is less than the value at the current top of the stack).
- Iterate through the array backwards. For each index j, check and pop elements from the stack while $\text{nums}[j]$ is greater than or equal to the nums value at the

Round 5 · Mid · Medium

p.1738

popped index. For each valid ramp found, compute the width ($j - \text{popped index}$) and update the maximum width accordingly. This approach guarantees we examine each element a constant number of times ensuring an efficient solution.

Monotonic Stack

□ #1019 Next Greater Node In Linked List **MED**

LeetDoocs · SimplyLeet · WalkCC · LeetCode Array · Linked List · Stack · Monotonic Stack Lists: AlgoMaster 600

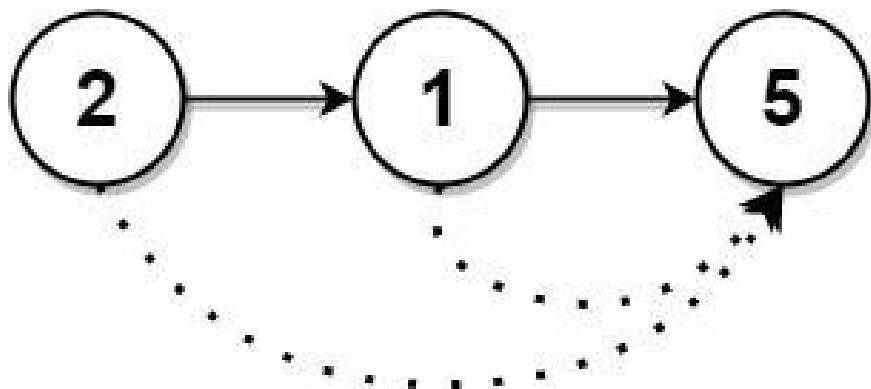
Problem

You are given the head of a linked list with n nodes.

For each node in the list, find the value of the next greater node. That is, for each node, find the value of the first node that is next to it and has a strictly larger value than it.

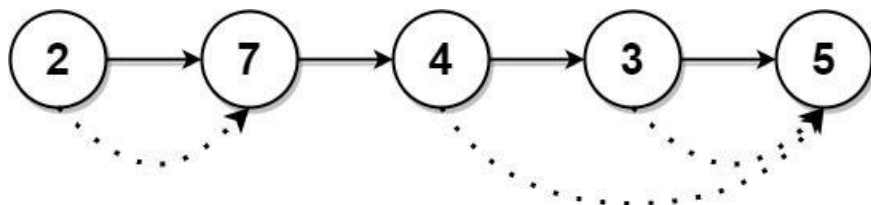
Return an integer array `answer` where `answer[i]` is the value of the next greater node of the i th node (1-indexed). If the i th node does not have a next greater node, set `answer[i] = 0`.

Example 1:



Input: head = [2,1,5]
Output: [5,5,0]

Example 2:



Input: head = [2,7,4,3,5]
Output: [7,0,5,5,0]

Constraints:

- The number of nodes in the list is n.
- $1 \leq n \leq 104$
- $1 \leq \text{Node.val} \leq 109$

Community Solutions (Python)

- WalkCC

Round 5 · Mid · Medium

p.1741

```
class Solution:
    def nextLargerNodes(self, head: ListNode) -> list[int]:
        ans = []
        stack = []

        while head:
            while stack and head.val > ans[stack[-1]]:
                index = stack.pop()
                ans[index] = head.val
            stack.append(len(ans))

            ans.append(head.val)
            head = head.next

        for i in stack:
            ans[i] = 0

        return ans
```

• LeetCode

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def nextLargerNodes(self, head: Optional[ListNode]) -> List[int]:
        nums = []
        while head:
            nums.append(head.val)
            head = head.next

        stk = []
        n = len(nums)
        ans = [0] * n
        for i in range(n - 1, -1, -1):
            while stk and stk[-1] <= nums[i]:
                stk.pop()
            if stk:
                ans[i] = stk[-1]
            stk.append(nums[i])
```

Round 5 · Mid · Medium

p.1742

```
return ans
```

• SimplyLeet

```
# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def nextLargerNodes(self, head: ListNode) -> list:
        # Convert linked list to array for easy access by index.
        values = []

        current = head
        while current:
            values.append(current.val)
            current = current.next

        # Initialize result array with zeros.
        result = [0] * len(values)
        stack = [] # Stack to store indices of nodes.

        # Iterate through the list using the monotonic stack approach.

        for i, value in enumerate(values):
            # Find all nodes in the stack that have a lesser value than the
            # current node.
            while stack and values[stack[-1]] < value:
                index = stack.pop()
                result[index] = value
                stack.append(i)

        return result
```

Round 5 · Mid · Medium

p.1743

Monotonic Stack

★ #1762 Buildings With an Ocean View ★ **MED**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Stack · Monotonic Stack
 Lists: ★ Premium | Premium 98

Problem

There are n buildings in a line. You are given an integer array `heights` of size n that represents the heights of the buildings in the line.

The ocean is to the right of the buildings. A building has an ocean view if the building can see the ocean without obstructions. Formally, a building has an ocean view if all the buildings to its right have a smaller height.

Return a list of indices (0-indexed) of buildings that have an ocean view, sorted in increasing order.

Example 1:

Input: `heights = [4,2,3,1]`
 Output: `[0,2,3]`
 Explanation: Building 1 (0-indexed) does not have an ocean view because building 2 is taller.

Example 2:

Round 5 · Mid · Medium

p.1744

Input: heights = [4,3,2,1]
 Output: [0,1,2,3]
 Explanation: All the buildings have an ocean view.

Example 3:

Input: heights = [1,3,2,4]
 Output: [3]
 Explanation: Only building 3 has an ocean view.

Constraints:

- $1 \leq \text{heights.length} \leq 105$
- $1 \leq \text{heights}[i] \leq 109$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findBuildings(self, heights: list[int]) -> list[int]:
        stack = []

        for i, height in enumerate(heights):
            while stack and heights[stack[-1]] <= height:
                stack.pop()
            stack.append(i)

        return stack
```

• LeetDoocs

```
class Solution:
    def findBuildings(self, heights: List[int]) -> List[int]:
        ans = []
        mx = 0
        for i in range(len(heights) - 1, -1, -1):
            if heights[i] > mx:
                ans.append(i)
                mx = heights[i]
        return ans[::-1]
```

• SimplyLeet

```
def findBuildingsWithOceanView(heights):
    n = len(heights)
    result = []
    max_height = 0
    # Iterate from the rightmost building to the leftmost building.
    for i in range(n - 1, -1, -1):
        if heights[i] > max_height:
            result.append(i)
            max_height = heights[i]
    # Reverse the result to return indices in increasing order.

    result.reverse()
    return result

# Example usage:
# heights = [4,2,3,1]
# print(findBuildingsWithOceanView(heights)) # Output: [0,2,3]
```

• LC.ca

```
class Solution:
    def findBuildings(self, heights: List[int]) -> List[int]:
        ans = []
        mx = 0
        for i in range(len(heights) - 1, -1, -1):
            if heights[i] > mx:
                ans.append(i)
                mx = heights[i]
        return ans[::-1]
```

◆ Quick Review

Key Insights

- Processing the buildings from right to left simplifies the problem because the rightmost building always has an ocean view.

- Keep track of the highest building encountered while iterating from right to left.
- A building has an ocean view if its height is greater than the maximum height seen so far.
- Append indices meeting the criteria, then reverse the order of indices for the solution since they were collected in reverse order.

Space & Time

Time Complexity: $O(n)$ where n is the number of buildings.

Space Complexity: $O(n)$ for storing the result in the worst-case scenario.

Solution

Iterate over the "heights" array from right to left while maintaining a variable to store the maximum height encountered so far. For each building, if its height is greater than this maximum height, add its index to the results list and update the maximum height. Since indices are collected in reverse order, reverse the list before returning it. This approach ensures that each building is visited only once.

Monotonic Stack

□ #2104 Sum of Subarray Ranges

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Stack · Monotonic Stack
 Lists: AlgoMaster 600

Problem

You are given an integer array `nums`. The range of a subarray of `nums` is the difference between the largest and smallest element in the subarray.

Return the sum of all subarray ranges of `nums`.
 A subarray is a contiguous non-empty sequence of elements within an array.

Example 1:

```
Input: nums = [1,2,3]
Output: 4
Explanation: The 6 subarrays of nums are the following:
[1], range = largest - smallest = 1 - 1 = 0
[2], range = 2 - 2 = 0
[3], range = 3 - 3 = 0
[1,2], range = 2 - 1 = 1
[1,2,3], range = 3 - 1 = 2
[2,3], range = 3 - 2 = 1
```

Example 2:

Input: nums = [1,3,3]
 Output: 4
 Explanation: The 6 subarrays of nums are the following:
 [1], range = largest - smallest = 1 - 1 = 0
 [3], range = 3 - 3 = 0
 [3], range = 3 - 3 = 0
 [1,3], range = 3 - 1 = 2
 [3,3], range = 3 - 3 = 0

Example 3:

Input: nums = [4,-2,-3,4,1]
 Output: 59
 Explanation: The sum of all subarray ranges of nums is 59.

Constraints:

- $1 \leq \text{nums.length} \leq 1000$
- $-109 \leq \text{nums}[i] \leq 109$

Follow-up: Could you find a solution with $O(n)$ time complexity?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def subArrayRanges(self, nums: list[int]) -> int:
        prevGt, nextGt = self._getPrevNext(nums, operator.lt)
        prevLt, nextLt = self._getPrevNext(nums, operator.gt)
        return sum(num * (i - prevGt[i]) * (nextGt[i] - i) -
                  num * (i - prevLt[i]) * (nextLt[i] - i)
                  for i, num in enumerate(nums))

    def _getPrevNext(
        self,
```

```
nums: list[int],
op: callable
) -> tuple[list[int], list[int]]:
    """
    Returns `prev` and `next`, that store the indices of the nearest numbers
    that are smaller or larger than the current number depending on `op`.
    """

    n = len(nums)
    prev = [-1] * n
    next = [n] * n

    stack = []
    for i, num in enumerate(nums):
        while stack and op(nums[stack[-1]], num):
            next[stack.pop()] = i
        if stack:
            prev[i] = stack[-1]
        stack.append(i)
    return prev, next
```

• LeetDoocs

```
class Solution:
    def subArrayRanges(self, nums: List[int]) -> int:
        ans, n = 0, len(nums)
        for i in range(n - 1):
            mi = mx = nums[i]
            for j in range(i + 1, n):
                mi = min(mi, nums[j])
                mx = max(mx, nums[j])
                ans += mx - mi
        return ans
```

```
class Solution:
    def subArrayRanges(self, nums: List[int]) -> int:
        def f(nums):
            stk = []
            n = len(nums)
            left = [-1] * n
            right = [n] * n
            for i, v in enumerate(nums):
                while stk and nums[stk[-1]] <= v:
                    stk.pop()
```

```

        if stk:
            left[i] = stk[-1]
            stk.append(i)
        stk = []
        for i in range(n - 1, -1, -1):
            while stk and nums[stk[-1]] < nums[i]:
                stk.pop()
            if stk:
                right[i] = stk[-1]
            stk.append(i)

    return sum((i - left[i]) * (right[i] - i) * v for i, v in enumerate(
e(nums)))

mx = f(nums)
mi = f([-v for v in nums])
return mx + mi

```

• SimplyLeet

```

def subArrayRanges(nums):
    n = len(nums)
    # Initialize arrays to store distance contributions for min and max
    left_min = [0] * n
    right_min = [0] * n
    left_max = [0] * n
    right_max = [0] * n

    # Compute left_min: distance to previous less element (strictly less)
    stack = []

    for i in range(n):
        # Maintain an increasing stack for minimums
        while stack and nums[stack[-1]] > nums[i]:
            stack.pop()
        left_min[i] = i - stack[-1] if stack else i + 1
        stack.append(i)

    # Compute right_min: distance to next less or equal element
    stack = []
    for i in range(n - 1, -1, -1):

```

```

        # Use >= to handle duplicates correctly
        while stack and nums[stack[-1]] >= nums[i]:
            stack.pop()
        right_min[i] = stack[-1] - i if stack else n - i
        stack.append(i)

    # Compute left_max: distance to previous greater element (strictly greater)
    stack = []
    for i in range(n):
        # Maintain a decreasing stack for maximums

        while stack and nums[stack[-1]] < nums[i]:
            stack.pop()
        left_max[i] = i - stack[-1] if stack else i + 1
        stack.append(i)

    # Compute right_max: distance to next greater or equal element
    stack = []
    for i in range(n - 1, -1, -1):
        # Use <= to handle duplicates correctly
        while stack and nums[stack[-1]] <= nums[i]:

            stack.pop()
            right_max[i] = stack[-1] - i if stack else n - i
            stack.append(i)

    # Calculate the final answer using contributions from max and min roles
    total = 0
    for i in range(n):
        total += nums[i] * left_max[i] * right_max[i]
        total -= nums[i] * left_min[i] * right_min[i]
    return total

# Example usage:
print(subArrayRanges([1, 2, 3])) # Output: 4

```


Queues

Round 5 · Mid · Medium · 4 questions

Round 5 · Mid · Medium

p.1753

Queues

□ #950 Reveal Cards In Increasing Order

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Queue · Sorting · Simulation
 Lists: AlgoMaster 600

Problem

You are given an integer array `deck`. There is a deck of cards where every card has a unique integer. The integer on the i th card is `deck[i]`. You can order the deck in any order you want. Initially, all the cards start face down (unrevealed) in one deck.

You will do the following steps repeatedly until all cards are revealed:

1. Take the top card of the deck, reveal it, and take it out of the deck.
2. If there are still cards in the deck then put the next top card of the deck at the bottom of the deck.
3. If there are still unrevealed cards, go back to step 1. Otherwise, stop.

Round 5 · Mid · Medium

p.1754

Return an ordering of the deck that would reveal the cards in increasing order.

Note that the first entry in the answer is considered to be the top of the deck.

Example 1:

Input: deck = [17,13,11,2,3,5,7]
 Output: [2,13,3,11,5,17,7]
 Explanation:
 We get the deck in the order [17,13,11,2,3,5,7] (this order does not matter), and reorder it.
 After reordering, the deck starts as [2,13,3,11,5,17,7], where 2 is the top of the deck.
 We reveal 2, and move 13 to the bottom. The deck is now [3,11,5,17,7,13].
 We reveal 3, and move 11 to the bottom. The deck is now [5,17,7,13,11].
 We reveal 5, and move 17 to the bottom. The deck is now [7,13,11,17].

Example 2:

Input: deck = [1,1000]
 Output: [1,1000]

Constraints:

- $1 \leq \text{deck.length} \leq 1000$
- $1 \leq \text{deck}[i] \leq 106$
- All the values of deck are unique.

Community Solutions (Python)

- WalkCC

```
class Solution:
    def deckRevealedIncreasing(self, deck: list[int]) -> list[int]:
        dq = collections.deque()

        for card in reversed(sorted(deck)):
            dq.rotate()
            dq.appendleft(card)

        return list(dq)
```

• LeetCode

```
class Solution:
    def deckRevealedIncreasing(self, deck: List[int]) -> List[int]:
        q = deque()
        for v in sorted(deck, reverse=True):
            if q:
                q.appendleft(q.pop())
            q.appendleft(v)
        return list(q)
```

• SimplyLeet

```
# Python solution using collections.deque for efficient O(1) pop and append
operations on both ends.

from collections import deque

def deckRevealedIncreasing(deck):
    # Sort the deck in ascending order.
    sorted_deck = sorted(deck)
    # Initialize an empty deque to simulate the reverse process.
    dq = deque()
```

```

# Process each card from the largest to smallest
for card in reversed(sorted_deck):
    # If the deque is not empty, simulate moving the last element to the
    front.
    if dq:
        dq.appendleft(dq.pop())
    # Place the current card at the front of the deque.
    dq.appendleft(card)

# Convert deque back to list which is the required order.
return list(dq)

# Example usage
deck = [17, 13, 11, 2, 3, 5, 7]
print(deckRevealedIncreasing(deck)) # Expected output: [2, 13, 3, 11, 5, 17,
7]

```

Queues

★ #1429 First Unique Number ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table · Design · Queue · Data
 Stream

Lists: ★ Premium | Premium 98

Problem

You have a queue of integers, you need to retrieve the first unique integer in the queue.

Implement the FirstUnique class:

- FirstUnique(int[] nums) Initializes the object with the numbers in the queue.
- int showFirstUnique() returns the value of the first unique integer of the queue, and returns -1 if there is no such integer.
- void add(int value) insert value to the queue.

Example 1:

```

Input:
["FirstUnique","showFirstUnique","add","showFirstUnique","add","showFirstUnique",
"add","showFirstUnique"]
[[[2,3,5]],[],[5],[],[2],[],[3],[]]
Output:
[null,2,null,2,null,3,null,-1]
Explanation:
FirstUnique firstUnique = new FirstUnique([2,3,5]);
firstUnique.showFirstUnique(); // return 2

```

Example 2:

```

Input:
["FirstUnique","showFirstUnique","add","add","add","add","add","showFirstUnique"]
[[[7,7,7,7,7,7]],[],[7],[3],[3],[7],[17],[]]
Output:
[null,-1,null,null,null,null,null,17]
Explanation:
FirstUnique firstUnique = new FirstUnique([7,7,7,7,7,7]);
firstUnique.showFirstUnique(); // return -1

```

Example 3:

```

Input:
["FirstUnique","showFirstUnique","add","showFirstUnique"]
[[[809]],[],[809],[]]
Output:
[null,809,null,-1]
Explanation:
FirstUnique firstUnique = new FirstUnique([809]);
firstUnique.showFirstUnique(); // return 809

```

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $1 \leq \text{nums}[i] \leq 10^8$
- $1 \leq \text{value} \leq 10^8$
- At most 50000 calls will be made to `showFirstUnique` and `add`.

Community Solutions (Python)

Round 5 · Mid · Medium

p.1759

• WalkCC

```

class FirstUnique:
    def __init__(self, nums: list[int]):
        self.seen = set()
        self.unique = {}
        for num in nums:
            self.add(num)

    def showFirstUnique(self) -> int:
        return next(iter(self.unique), -1)

```

```

def add(self, value: int) -> None:
    if value not in self.seen:
        self.seen.add(value)
        self.unique[value] = 1
    elif value in self.unique:
        # We have added this value before, and this is the second time we're
        # adding it. So, erase the value from `unique`.
        self.unique.pop(value)

```

• LeetDoocs

```

class FirstUnique:
    def __init__(self, nums: List[int]):
        self.cnt = Counter(nums)
        self.unique = OrderedDict({v: 1 for v in nums if self.cnt[v] == 1})

    def showFirstUnique(self) -> int:
        return -1 if not self.unique else next(v for v in self.unique.keys())

    def add(self, value: int) -> None:
        self.cnt[value] += 1

```

Round 5 · Mid · Medium

p.1760

```

if self.cnt[value] == 1:
    self.unique[value] = 1
elif value in self.unique:
    self.unique.pop(value)

```

```

# Your FirstUnique object will be instantiated and called as such:
# obj = FirstUnique(nums)
# param_1 = obj.showFirstUnique()
# obj.add(value)

```

• SimplyLeet

```
import collections
```

```
class FirstUnique:
```

```
    # Initialize the object with the numbers in the queue.
```

```
    def __init__(self, nums):
```

```
        # Using deque for efficient popleft operations.
```

```
        self.queue = collections.deque()
```

```
        # Dictionary to store frequency count.
```

```
        self.count = collections.defaultdict(int)
```

```
        # Process the initial list of numbers.
```

```
        for num in nums:
            self.add(num)
```

```
    # Returns the value of the first unique integer of the queue.
```

```
    def showFirstUnique(self):
```

```
        # Remove numbers from the front if they are no longer unique.
```

```
        while self.queue and self.count[self.queue[0]] > 1:
```

```
            self.queue.popleft()
```

```
        # Return the first unique number if available.
```

```
        return self.queue[0] if self.queue else -1
```

```
    # Inserts value to the queue.
```

```
    def add(self, value):
```

```
        self.count[value] += 1
```

```
        # If it is the first occurrence, append it to the queue.
```

```
        if self.count[value] == 1:
```

```
            self.queue.append(value)
```

Round 5 · Mid · Medium

p.1761

• LC.ca

```
class FirstUnique:
```

```
    def __init__(self, nums: List[int]):
```

```
        self.cnt = Counter(nums)
```

```
        self.unique = OrderedDict({v: 1 for v in nums if self.cnt[v] == 1})
```

```
    def showFirstUnique(self) -> int:
```

```
        return -1 if not self.unique else next(v for v in self.unique.keys())
```

```
    def add(self, value: int) -> None:
```

```
        self.cnt[value] += 1
```

```
    if self.cnt[value] == 1:
```

```
        self.unique[value] = 1
```

```
    elif value in self.unique:
```

```
        self.unique.pop(value)
```

```
    # Your FirstUnique object will be instantiated and called as such:
```

```
    # obj = FirstUnique(nums)
```

```
    # param_1 = obj.showFirstUnique()
```

```
    # obj.add(value)
```

◆ Quick Review

Key Insights

- Use a queue to maintain the order of potential unique numbers.
- Use a hash table (or map) to count the occurrences of each number.
- When showing the first unique number, remove numbers from the front of the queue if their frequency is more than one.

Round 5 · Mid · Medium

p.1762

- All operations (add and showFirstUnique) can be done in $O(1)$ amortized time.

Space & Time

Time Complexity: $O(1)$ amortized for each add and showFirstUnique call.

Space Complexity: $O(n)$, where n is the number of distinct integers encountered.

Solution

We design the FirstUnique class using two main data structures: a queue and a hash map (or dictionary). The queue preserves the order of insertion for unique candidates, while the hash map holds the frequency count for each integer.

When a new number is added:

– Increment its count in the hash map. – If the count is one, append it to the queue.

To retrieve the first unique number:

– While the queue is not empty and the count for the element at the front is greater than one, remove it from the queue. – Return the front element of the queue if it exists, otherwise return -1 .

This approach ensures that we are always up-to-date with the current first unique number.

Queues

#1823 Find the Winner of the Circular Game

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Math · Recursion · Queue · Simulation
 Lists: AlgoMaster 600

Problem

There are n friends that are playing a game. The friends are sitting in a circle and are numbered from 1 to n in clockwise order. More formally, moving clockwise from the i th friend brings you to the $(i+1)$ th friend for $1 \leq i < n$, and moving clockwise from the n th friend brings you to the 1st friend.

The rules of the game are as follows:

1. Start at the 1st friend.
2. Count the next k friends in the clockwise direction including the friend you started at. The counting wraps around the circle and may count some friends more than once.
3. The last friend you counted leaves the circle and loses the game.

Round 5 · Mid · Medium

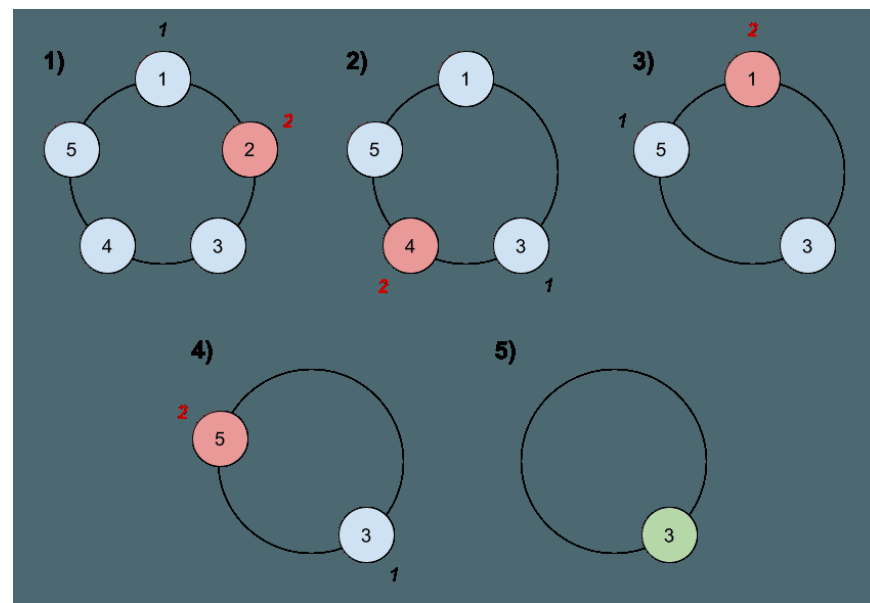
p.1765

4. If there is still more than one friend in the circle, go back to step 2 starting from the friend immediately clockwise of the friend who just lost and repeat.

5. Else, the last friend in the circle wins the game.

Given the number of friends, n , and an integer k , return the winner of the game.

Example 1:



Round 5 · Mid · Medium

p.1766

Input: $n = 5, k = 2$

Output: 3

Explanation: Here are the steps of the game:

- 1) Start at friend 1.
- 2) Count 2 friends clockwise, which are friends 1 and 2.
- 3) Friend 2 leaves the circle. Next start is friend 3.
- 4) Count 2 friends clockwise, which are friends 3 and 4.
- 5) Friend 4 leaves the circle. Next start is friend 5.

Example 2:

Input: $n = 6, k = 5$

Output: 1

Explanation: The friends leave in this order: 5, 4, 6, 2, 3. The winner is friend 1.

Constraints:

- $1 \leq k \leq n \leq 500$

Follow up:

Could you solve this problem in linear time with constant space?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findTheWinner(self, n: int, k: int) -> int:
        # True if i-th friend is left
        friends = [False] * n

        friendCount = n
        fp = 0 # friends' index

        while friendCount > 1:
            for _ in range(k):
```

```
while friends[fp % n]: # The friend is not there.
    fp += 1 # Point to the next one.
    fp += 1
    friends[(fp - 1) % n] = True
    friendCount -= 1
```

```
fp = 0
while friends[fp]:
    fp += 1
```

```
return fp + 1
```

```
class Solution:
    def findTheWinner(self, n: int, k: int) -> int:
        # e.g. n = 4, k = 2.
        # By using 0-indexed notation, we have the following circle:
        #
        # 0 -> 1 -> 2 -> 3 -> 0
        # x
        # 0 -> 1 -> 2 -> 0
        #
        # After the first round, 1 is removed.
```

```
# So, 2 becomes 0, 3 becomes 1, and 0 becomes 2.
# Let's denote that oldIndex = f(n, k) and newIndex = f(n - 1, k).
# By observation, we know f(n, k) = (f(n - 1, k) + k) % n.
def f(n: int, k: int) -> int:
    if n == 1:
        return 0
    return (f(n - 1, k) + k) % n

# Converts back to 1-indexed.
return f(n, k) + 1
```

• LeetCode

```
class Solution:
    def findTheWinner(self, n: int, k: int) -> int:
        if n == 1:
            return 1
        ans = (k + self.findTheWinner(n - 1, k)) % n
        return n if ans == 0 else ans
```


• SimplyLeet

```
# Python implementation for finding the winner in the circular game

def findWinner(n, k):
    # Initialize winner to 0 for the base case of 1 friend (0-indexed)
    winner = 0
    # Process from 2 friends to n friends
    for i in range(2, n + 1):
        # Update winner index based on recurrence relation
        winner = (winner + k) % i
    # Convert the 0-indexed winner to 1-indexed result

    return winner + 1

# Example usage:
n = 5
k = 2
print(findWinner(n, k)) # Expected output: 3
```

Round 5 · Mid · Medium

p.1769

Queues

□ #2327 Number of People Aware of a Secret

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Dynamic Programming · Queue · Simulation
Lists: AlgoMaster 600

Problem

On day 1, one person discovers a secret.

You are given an integer *delay*, which means that each person will share the secret with a new person every day, starting from *delay* days after discovering the secret. You are also given an integer *forget*, which means that each person will forget the secret *forget* days after discovering it. A person cannot share the secret on the same day they forgot it, or on any day afterwards.

Given an integer *n*, return the number of people who know the secret at the end of day *n*. Since the answer may be very large, return it modulo $10^9 + 7$.

Example 1:

Round 5 · Mid · Medium

p.1770

Input: $n = 6$, $\text{delay} = 2$, $\text{forget} = 4$

Output: 5

Explanation:

Day 1: Suppose the first person is named A. (1 person)

Day 2: A is the only person who knows the secret. (1 person)

Day 3: A shares the secret with a new person, B. (2 people)

Day 4: A shares the secret with a new person, C. (3 people)

Day 5: A forgets the secret, and B shares the secret with a new person, D. (3 people)

Example 2:

Input: $n = 4$, $\text{delay} = 1$, $\text{forget} = 3$

Output: 6

Explanation:

Day 1: The first person is named A. (1 person)

Day 2: A shares the secret with B. (2 people)

Day 3: A and B share the secret with 2 new people, C and D. (4 people)

Day 4: A forgets the secret. B, C, and D share the secret with 3 new people. (6 people)

Constraints:

- $2 \leq n \leq 1000$
- $1 \leq \text{delay} < \text{forget} \leq n$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def peopleAwareOfSecret(self, n: int, delay: int, forget: int) -> int:
        MOD = 1_000_000_007
        share = 0
        # dp[i] := the number of people know the secret at day i
        dp = [0] * n # Maps day i to i + 1.
        dp[0] = 1

        for i in range(1, n):
            if i - delay >= 0:
```

```
share += dp[i - delay]
if i - forget >= 0:
    share -= dp[i - forget]
share %= MOD
dp[i] = share

# People before day `n - forget - 1` already forget the secret.
return sum(dp[-forget:]) % MOD
```

• LeetDoocs

```
class Solution:
    def peopleAwareOfSecret(self, n: int, delay: int, forget: int) -> int:
        m = (n << 1) + 10
        d = [0] * m
        cnt = [0] * m
        cnt[1] = 1
        for i in range(1, n + 1):
            if cnt[i]:
                d[i] += cnt[i]
                d[i + forget] -= cnt[i]

        nxt = i + delay
        while nxt < i + forget:
            cnt[nxt] += cnt[i]
            nxt += 1

        mod = 10**9 + 7
        return sum(d[: n + 1]) % mod
```

• SimplyLeet

```
MOD = 10**9 + 7

def peopleAwareOfSecret(n, delay, forget):
    # Initialize dp and prefix arrays.
    dp = [0] * (n + 1)
    prefix = [0] * (n + 1)

    # On day 1, one person discovers the secret.
    dp[1] = 1
    prefix[1] = 1
```

```

    # For each day from 2 to n, determine how many new people learn the secret.
    for day in range(2, n + 1):
        # Calculate the start contribution: people eligible to share from day
        "day-delay".
        if day - delay >= 1:
            share_start = prefix[day - delay]
        else:
            share_start = 0

        # Subtract those who have forgotten the secret.

        if day - forget >= 1:
            forget_end = prefix[day - forget]
        else:
            forget_end = 0

        dp[day] = (share_start - forget_end) % MOD
        prefix[day] = (prefix[day - 1] + dp[day]) % MOD

    # Sum up individuals who have not forgotten the secret by day n.
    total = 0

    # People that discovered the secret on day d will still remember if d > n -
    forget.
    for day in range(n - forget + 1, n + 1):
        if day >= 1:
            total = (total + dp[day]) % MOD
    return total

# Example usage:
print(peopleAwareOfSecret(6, 2, 4))

```

Round 5 · Mid · Medium

p.1773

Monotonic Queue

Round 5 · Mid · Medium · 5 questions

Round 5 · Mid · Medium

p.1774

Monotonic Queue

☐ #1438 Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Queue · Sliding Window · Heap (Priority Queue) · Ordered Set · Monotonic Queue
 Lists: AlgoMaster 600

Problem

Given an array of integers `nums` and an integer `limit`, return the size of the longest non-empty subarray such that the absolute difference between any two elements of this subarray is less than or equal to `limit`.

Example 1:

Input: `nums = [8,2,4,7]`, `limit = 4`
 Output: 2
 Explanation: All subarrays are:
 [8] with maximum absolute diff $|8-8| = 0 \leq 4$.
 [8,2] with maximum absolute diff $|8-2| = 6 > 4$.
 [8,2,4] with maximum absolute diff $|8-2| = 6 > 4$.
 [8,2,4,7] with maximum absolute diff $|8-2| = 6 > 4$.
 [2] with maximum absolute diff $|2-2| = 0 \leq 4$.

Example 2:

Input: `nums = [10,1,2,4,7,2]`, `limit = 5`
 Output: 4
 Explanation: The subarray [2,4,7,2] is the longest since the maximum absolute diff is $|2-7| = 5 \leq 5$.

Example 3:

Input: `nums = [4,2,2,2,4,4,2,2]`, `limit = 0`
 Output: 3

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $1 \leq \text{nums}[i] \leq 109$
- $0 \leq \text{limit} \leq 109$

Community Solutions (Python)

☐ • LeetDoocs

```
class Solution:
    def longestSubarray(self, nums: List[int], limit: int) -> int:
        sl = SortedList()
        ans = j = 0
        for i, x in enumerate(nums):
            sl.add(x)
            while sl[-1] - sl[0] > limit:
                sl.remove(nums[j])
                j += 1
            ans = max(ans, i - j + 1)

        return ans
```

```
class Solution:
    def longestSubarray(self, nums: List[int], limit: int) -> int:
        def check(k: int) -> bool:
            min_q = deque()
            max_q = deque()
            for i, x in enumerate(nums):
                if min_q and i - min_q[0] + 1 > k:
                    min_q.popleft()
                if max_q and i - max_q[0] + 1 > k:
                    max_q.popleft()

                while min_q and nums[min_q[-1]] >= x:
                    min_q.pop()
                while max_q and nums[max_q[-1]] <= x:
                    max_q.pop()
                min_q.append(i)
                max_q.append(i)
                if i >= k - 1 and nums[max_q[0]] - nums[min_q[0]] <= limit:
                    return True
            return False

        l, r = 1, len(nums)
        while l < r:
            mid = (l + r + 1) >> 1
            if check(mid):
                l = mid
            else:
                r = mid - 1
        return l
```

```
class Solution:
    def longestSubarray(self, nums: List[int], limit: int) -> int:
        maxq = deque()
        minq = deque()
        l, n = 0, len(nums)
        for r, x in enumerate(nums):
            while maxq and nums[maxq[-1]] < x:
                maxq.pop()
            while minq and nums[minq[-1]] > x:
                minq.pop()
```

Round 5 · Mid · Medium

p.1777

```
maxq.append(r)
minq.append(r)
if nums[maxq[0]] - nums[minq[0]] > limit:
    l += 1
    if maxq[0] < l:
        maxq.popleft()
    if minq[0] < l:
        minq.popleft()
return n - l
```

• SimplyLeet

```
from collections import deque

def longestSubarray(nums, limit):
    maxDeque = deque() # stores indices for the maximum values
    minDeque = deque() # stores indices for the minimum values
    left = 0
    maxLength = 0

    # Iterate over each index with right pointer
    for right in range(len(nums)):

        # Maintain decreasing order in maxDeque
        while maxDeque and nums[right] > nums[maxDeque[-1]]:
            maxDeque.pop()
        maxDeque.append(right)

        # Maintain increasing order in minDeque
        while minDeque and nums[right] < nums[minDeque[-1]]:
            minDeque.pop()
        minDeque.append(right)
```

Round 5 · Mid · Medium

p.1778

```

# If window is invalid, shrink from the left
while nums[maxDeque[0]] - nums[minDeque[0]] > limit:
    # Move the left pointer forward and remove indices if they are out
    # of the new window
    if maxDeque[0] == left:
        maxDeque.popleft()
    if minDeque[0] == left:
        minDeque.popleft()
    left += 1

# Update maxLength with the valid window size

maxLength = max(maxLength, right - left + 1)

return maxLength

# Example usage:
print(longestSubarray([8,2,4,7], 4))

```

Monotonic Queue

#1673 Find the Most Competitive Subsequence

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Stack · Greedy · Monotonic Stack
 Lists: AlgoMaster 600

Problem

Given an integer array `nums` and a positive integer `k`, return the most competitive subsequence of `nums` of size `k`.

An array's subsequence is a resulting sequence obtained by erasing some (possibly zero) elements from the array.

We define that a subsequence `a` is more competitive than a subsequence `b` (of the same length) if in the first position where `a` and `b` differ, subsequence `a` has a number less than the corresponding number in `b`. For example, `[1,3,4]` is more competitive than `[1,3,5]` because the first position they differ is at the final number, and 4 is less than 5.

Example 1:

Input: nums = [3,5,2,6], k = 2

Output: [2,6]

Explanation: Among the set of every possible subsequence: {[3,5], [3,2], [3,6], [5,2], [5,6], [2,6]}, [2,6] is the most competitive.

Example 2:

Input: nums = [2,4,3,3,5,4,9,6], k = 4

Output: [2,3,3,4]

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $0 \leq \text{nums}[i] \leq 109$
- $1 \leq k \leq \text{nums.length}$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def mostCompetitive(self, nums: list[int], k: int) -> list[int]:
        ans = []

        for i, num in enumerate(nums):
            # If |ans| - 1 + |nums[i..n]| >= k, then it means we still have enough
            # numbers, and we can safely pop an element from ans.
            while ans and ans[-1] > nums[i] and len(ans) - 1 + len(nums) - i >= k:
                ans.pop()
            if len(ans) < k:
                ans.append(nums[i])

        return ans
```

• LeetDoocs

```
class Solution:
    def mostCompetitive(self, nums: List[int], k: int) -> List[int]:
        stk = []
        n = len(nums)
        for i, v in enumerate(nums):
            while stk and stk[-1] > v and len(stk) + n - i > k:
                stk.pop()
            if len(stk) < k:
                stk.append(v)
        return stk
```

• SimplyLeet

```
# Function to find the most competitive subsequence using a greedy monotonic
# stack.
def mostCompetitive(nums, k):
    stack = [] # Stack to hold the resulting subsequence
    n = len(nums) # Total number of elements in the input array

    # Iterate over each number with its index.
    for i, num in enumerate(nums):
        # While the stack is not empty, and the last element in the stack is
        # greater than the current element,
        # and we have enough remaining elements to fill the subsequence of size
        # k:
        while stack and num < stack[-1] and (len(stack) - 1 + (n - i)) >= k:
            stack.pop() # Remove the last element from the stack
        # If the current subsequence is not yet of length k, add the current
        # element.
        if len(stack) < k:
            stack.append(num)
    return stack

# Example usage:
print(mostCompetitive([3, 5, 2, 6], 2)) # Expected Output: [2, 6]
```

Monotonic Queue

#1696 Jump Game VI

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Dynamic Programming · Queue · Heap
 (Priority Queue) · Monotonic Queue
 Lists: AlgoMaster 600

Problem

You are given a 0-indexed integer array `nums` and an integer `k`.

You are initially standing at index 0. In one move, you can jump at most `k` steps forward without going outside the boundaries of the array. That is, you can jump from index `i` to any index in the range `[i + 1, min(n - 1, i + k)]` inclusive.

You want to reach the last index of the array (index `n - 1`). Your score is the sum of all `nums[j]` for each index `j` you visited in the array. Return the maximum score you can get.

Example 1:

Round 5 · Mid · Medium

p.1783

Input: `nums = [1,-1,-2,4,-7,3]`, `k = 2`

Output: 7

Explanation: You can choose your jumps forming the subsequence `[1,-1,4,3]` (underlined above). The sum is 7.

Example 2:

Input: `nums = [10,-5,-2,4,0,3]`, `k = 3`

Output: 17

Explanation: You can choose your jumps forming the subsequence `[10,4,3]` (underlined above). The sum is 17.

Example 3:

Input: `nums = [1,-5,-20,4,-1,3,-6,-3]`, `k = 2`

Output: 0

Constraints:

- $1 \leq \text{nums.length}$, $k \leq 105$
- $-104 \leq \text{nums}[i] \leq 104$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxResult(self, nums: List[int], k: int) -> int:
        # Stores dp[i] within the bounds.
        maxQ = collections.deque([0])
        # dp[i] := the maximum score to consider nums[0..i]
        dp = [0] * len(nums)
        dp[0] = nums[0]

        for i in range(1, len(nums)):
            # Pop the index if it's out-of-bounds.
```

Round 5 · Mid · Medium

p.1784


```

if maxQ[0] + k < i:
    maxQ.popleft()
dp[i] = dp[maxQ[0]] + nums[i]
# Pop indices that won't be chosen in the future.
while maxQ and dp[maxQ[-1]] <= dp[i]:
    maxQ.pop()
maxQ.append(i)

return dp[-1]

```

• LeetCode

```

class Solution:
    def maxResult(self, nums: List[int], k: int) -> int:
        n = len(nums)
        f = [0] * n
        q = deque([0])
        for i in range(n):
            if i - q[0] > k:
                q.popleft()
            f[i] = nums[i] + f[q[0]]
            while q and f[q[-1]] <= f[i]:
                q.pop()
            q.append(i)
        return f[-1]

```

• SimplyLeet

```

# Python solution using deque for the sliding window maximum
from collections import deque

def maxResult(nums, k):
    n = len(nums)
    # Initialize dp array where dp[i] represents the maximum score to reach
    # index i
    dp = [0] * n
    dp[0] = nums[0]
    # Use deque to keep track of indices with maximum dp values in the current
    # window
    dq = deque([0])

```

```

# Process each index from 1 to n-1
for i in range(1, n):
    # Remove indices that are out of the window (i - k)
    while dq and dq[0] < i - k:
        dq.popleft()
    # dp[i] equals nums[i] plus the maximum score within the window
    dp[i] = nums[i] + dp[dq[0]]
    # Maintain deque in decreasing order of dp values
    while dq and dp[i] >= dp[dq[-1]]:
        dq.pop()
    dq.append(i)
return dp[-1]

```

```

# Example usage:
if __name__ == "__main__":
    print(maxResult([1, -1, -2, 4, -7, 3], 2)) # Output: 7

```

Monotonic Queue

#2762 Continuous Subarrays

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Queue · Sliding Window · Heap (Priority Queue) · Ordered Set · Monotonic Queue
 Lists: AlgoMaster 600

Problem

You are given a 0-indexed integer array `nums`. A subarray of `nums` is called continuous if:

- Let $i, i + 1, \dots, j$ be the indices in the subarray. Then, for each pair of indices $i \leq i1, i2 \leq j$, $0 \leq |nums[i1] - nums[i2]| \leq 2$.

Return the total number of continuous subarrays.

A subarray is a contiguous non-empty sequence of elements within an array.

Example 1:

Input: `nums = [5,4,2,4]`
 Output: 8
 Explanation:
 Continuous subarray of size 1: [5], [4], [2], [4].
 Continuous subarray of size 2: [5,4], [4,2], [2,4].
 Continuous subarray of size 3: [4,2,4].
 There are no subarrays of size 4.
 Total continuous subarrays = 4 + 3 + 1 = 8.

Round 5 · Mid · Medium

p.1787

Example 2:

Input: `nums = [1,2,3]`
 Output: 6
 Explanation:
 Continuous subarray of size 1: [1], [2], [3].
 Continuous subarray of size 2: [1,2], [2,3].
 Continuous subarray of size 3: [1,2,3].
 Total continuous subarrays = 3 + 2 + 1 = 6.

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $1 \leq \text{nums}[i] \leq 109$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def continuousSubarrays(self, nums: list[int]) -> int:
        ans = 1 # [nums[0]]
        left = nums[0] - 2
        right = nums[0] + 2
        l = 0

        # nums[l..r] is a valid window.
        for r in range(1, len(nums)):
            if left <= nums[r] <= right:

                left = max(left, nums[r] - 2)
                right = min(right, nums[r] + 2)
            else:
                # nums[r] is out-of-bounds, so reconstruct the window.
                left = nums[r] - 2
                right = nums[r] + 2
                l = r

        # If we consistently move leftward in each iteration, it implies that
        # the entire left subarray satisfies the given condition. For every
        # subarray with l in the range [0, r], the condition is met, preventing
```

Round 5 · Mid · Medium

p.1788

```
# the code from reaching the final "else" condition. Instead, it stops
# at the "if" condition.
while nums[r] - 2 <= nums[l] <= nums[r] + 2:
    left = max(left, nums[l] - 2)
    right = min(right, nums[l] + 2)
    l -= 1
    l += 1
# nums[l..r], num[l + 1..r], ..., nums[r]
ans += r - l + 1

return ans
```

• LeetDoocs

```
class Solution:
    def continuousSubarrays(self, nums: List[int]) -> int:
        ans = i = 0
        sl = SortedList()
        for x in nums:
            sl.add(x)
            while sl[-1] - sl[0] > 2:
                sl.remove(nums[i])
                i += 1
            ans += len(sl)

        return ans
```

• SimplyLeet

```
from collections import deque

def continuousSubarrays(nums):
    # Deques for storing indices for minimum and maximum values
    min_deque = deque()
    max_deque = deque()
    left = 0
    count = 0

    # Iterate over each element as the right end of the window
```

```
for right, num in enumerate(nums):
    # Maintain min_deque: remove elements greater than current num
    while min_deque and nums[min_deque[-1]] > num:
        min_deque.pop()
    min_deque.append(right)

    # Maintain max_deque: remove elements smaller than current num
    while max_deque and nums[max_deque[-1]] < num:
        max_deque.pop()
    max_deque.append(right)

    # If current window doesn't satisfy condition, slide left pointer
    while nums[max_deque[0]] - nums[min_deque[0]] > 2:
        # Move left pointer and remove if it is out of window
        if min_deque[0] == left:
            min_deque.popleft()
        if max_deque[0] == left:
            max_deque.popleft()
        left += 1

    # Count all subarrays ending at right index
    count += (right - left + 1)

return count

# Example usage:
nums = [5,4,2,4]
print(continuousSubarrays(nums)) # Expected output: 8
```

Monotonic Queue

□ #3578 Count Partitions With Max-Min Difference at Most K

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Dynamic Programming · Queue ·
 Sliding Window · Prefix Sum · Monotonic Queue
 Lists: AlgoMaster 600

Problem

You are given an integer array `nums` and an integer `k`. Your task is to partition `nums` into one or more non-empty contiguous segments such that in each segment, the difference between its maximum and minimum elements is at most `k`.

Return the total number of ways to partition `nums` under this condition.

Since the answer may be too large, return it modulo $10^9 + 7$.

Example 1:

Input: `nums = [9,4,1,3,7]`, `k = 4`

Output: 6

Explanation:

Round 5 · Mid · Medium

p.1791

There are 6 valid partitions where the difference between the maximum and minimum elements in each segment is at most `k = 4`:

- `[[9], [4], [1], [3], [7]]`
- `[[9], [4], [1, 3], [7]]`
- `[[9], [4], [1, 3], [7]]`
- `[[9], [4, 1], [3], [7]]`
- `[[9], [4, 1], [3, 7]]`
- `[[9], [4, 1, 3], [7]]`

Example 2:

Input: `nums = [3,3,4]`, `k = 0`

Output: 2

Explanation:

There are 2 valid partitions that satisfy the given conditions:

- `[[3], [3], [4]]`
- `[[3, 3], [4]]`

Constraints:

- $2 \leq \text{nums.length} \leq 5 \times 10^4$
- $1 \leq \text{nums}[i] \leq 10^9$
- $0 \leq k \leq 10^9$

Community Solutions (Python)

Round 5 · Mid · Medium

p.1792

• LeetCode

```
class Solution:
    def countPartitions(self, nums: List[int], k: int) -> int:
        mod = 10**9 + 7
        sl = SortedList()
        n = len(nums)
        f = [1] + [0] * n
        g = [1] + [0] * n
        l = 1
        for r, x in enumerate(nums, 1):
            sl.add(x)

            while sl[-1] - sl[0] > k:
                sl.remove(nums[l - 1])
                l += 1
            f[r] = (g[r - 1] - (g[l - 2] if l >= 2 else 0) + mod) % mod
            g[r] = (g[r - 1] + f[r]) % mod
        return f[n]
```

Recursion

Round 5 · Mid · Medium · 2 questions

Round 5 · Mid · Medium

p.1793

Round 5 · Mid · Medium

p.1794

Recursion

#50 Pow(x, n)

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode

Math · Recursion

Lists: Grind 169

Problem

Implement $\text{pow}(x, n)$, which calculates x raised to the power n (i.e., x^n).

Example 1:

Input: $x = 2.00000$, $n = 10$
Output: 1024.00000

Example 2:

Input: $x = 2.10000$, $n = 3$
Output: 9.26100

Example 3:

Input: $x = 2.00000$, $n = -2$
Output: 0.25000
Explanation: $2^{-2} = 1/2^2 = 1/4 = 0.25$

Constraints:

- $-100.0 < x < 100.0$
- $-231 \leq n \leq 231-1$
- n is an integer.

Round 5 · Mid · Medium

p.1795

- Either x is not zero or $n > 0$.
- $-104 \leq x^n \leq 104$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def myPow(self, x: float, n: int) -> float:
        if n == 0:
            return 1
        if n < 0:
            return 1 / self.myPow(x, -n)
        if n % 2 == 1:
            return x * self.myPow(x, n - 1)
        return self.myPow(x * x, n // 2)
```

• LeetCode

```
class Solution:
    def myPow(self, x: float, n: int) -> float:
        def qpown(a: float, n: int) -> float:
            ans = 1
            while n:
                if n & 1:
                    ans *= a
                a *= a
                n >>= 1
            return ans

        return qpown(x, n) if n >= 0 else 1 / qpown(x, -n)
```

• SimplyLeet

Round 5 · Mid · Medium

p.1796

```
# Python implementation of pow(x, n) using exponentiation by squaring
def myPow(x, n):
    # if exponent is negative, invert x and make n positive
    if n < 0:
        x = 1 / x
        n = -n

    result = 1.0 # initialize result
    while n:
        # if the current exponent is odd, multiply result by the base

        if n % 2:
            result *= x
        x *= x # square the base for the next iteration
        n //= 2 # divide the exponent by 2
    return result

# Example usage:
print(myPow(2.0, 10)) # Output: 1024.0
print(myPow(2.1, 3))  # Output: 9.261000000000001
print(myPow(2.0, -2)) # Output: 0.25
```

• LC.ca

```
class Solution:
    def myPow(self, x: float, n: int) -> float:
    def qpow(a: float, n: int) -> float:
        ans = 1
        while n:
            if n & 1:
                ans *= a
            a *= a
            n >>= 1
        return ans

    return qpow(x, n) if n >= 0 else 1 / qpow(x, -n)
```

◆ Quick Review

Key Insights

Round 5 · Mid · Medium

p.1797

- Use exponentiation by squaring to reduce the number of multiplications.
- For a negative exponent, compute the power for $|n|$ and then take the reciprocal.
- Handle the edge case when n is 0 (return 1).
- The algorithm works in $O(\log n)$ time complexity.

Space & Time

Time Complexity: $O(\log |n|)$

Space Complexity: $O(1)$ for the iterative solution, or $O(\log |n|)$ if implemented recursively

Solution

We use exponentiation by squaring, an algorithm that reduces the number of multiplications by repeatedly squaring the base and halving the exponent. Here's how it works:

- If n is negative, switch x to $1/x$ and n to $-n$.
- Initialize the result as 1.
- Loop while n is greater than 0: If the current n is odd, multiply the result by x . Square the base x . Divide n by 2 (using integer division). This iterative approach efficiently computes the power in

Round 5 · Mid · Medium

p.1798

logarithmic steps. The same approach can be implemented recursively.

Round 5 · Mid · Medium

p.1799

Recursion

□ #394 Decode String

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Stack · Recursion
Lists: Grind 169

Problem

Given an encoded string, return its decoded string.

The encoding rule is: $k[\text{encoded_string}]$, where the `encoded_string` inside the square brackets is being repeated exactly k times. Note that k is guaranteed to be a positive integer.

You may assume that the input string is always valid; there are no extra white spaces, square brackets are well-formed, etc. Furthermore, you may assume that the original data does not contain any digits and that digits are only for those repeat numbers, k . For example, there will not be input like `3a` or `2[4]`.

The test cases are generated so that the length of the output will never exceed 105.

Round 5 · Mid · Medium

p.1800

Example 1:

Input: s = "3[a]2[bc]"
Output: "aaabcbc"

Example 2:

Input: s = "3[a2[c]]"
Output: "accaccacc"

Example 3:

Input: s = "2[abc]3[cd]ef"
Output: "abcbccdcdcdef"

Constraints:

- $1 \leq s.length \leq 30$
- s consists of lowercase English letters, digits, and square brackets '['].
- s is guaranteed to be a valid input.
- All the integers in s are in the range [1, 300].

Community Solutions (Python)

• WalkCC

```
class Solution:
    def decodeString(self, s: str) -> str:
        stack = [] # (prevStr, repeatCount)
        currStr = ''
        currNum = 0

        for c in s:
            if c.isdigit():
                currNum = currNum * 10 + int(c)
            else:
```

```
if c == '[':
    stack.append((currStr, currNum))
    currStr = ''
    currNum = 0
elif c == ']':
    prevStr, num = stack.pop()
    currStr = prevStr + num * currStr
else:
    currStr += c
```

```
return currStr
```

```
class Solution:
    def decodeString(self, s: str) -> str:
        ans = ''

        while self.i < len(s) and s[self.i] != ']':
            if s[self.i].isdigit():
                k = 0
                while self.i < len(s) and s[self.i].isdigit():
                    k = k * 10 + int(s[self.i])
                    self.i += 1

                self.i += 1 # '['
                decodedString = self.decodeString(s)
                self.i += 1 # ']'
                ans += k * decodedString
            else:
                ans += s[self.i]
                self.i += 1

        return ans
```

```
i = 0
```

• LeetDoocs

```
class Solution:
    def decodeString(self, s: str) -> str:
        s1, s2 = [], []
        num, res = 0, ''
        for c in s:
            if c.isdigit():
                num = num * 10 + int(c)
            elif c == '[':
                s1.append(num)
                s2.append(res)

            num, res = 0, ''
            elif c == ']':
                res = s2.pop() + res * s1.pop()
            else:
                res += c
        return res
```

• SimplyLeet

```
def decodeString(s: str) -> str:
    # index pointer as a list so it can be mutable within recursion
    def helper(i: int) -> (str, int):
        decoded = ''
        while i < len(s):
            if s[i].isdigit():
                # build the number (it might be more than one digit)
                k = 0
                while i < len(s) and s[i].isdigit():
                    k = k * 10 + int(s[i])

                i += 1
                # skip the '[' character
                i += 1 # skip '['
                # recursively solve the substring inside the brackets
                sub, i = helper(i)
                # append the substring repeated k times
                decoded += sub * k
            elif s[i] == ']':
                # returning decoded string and new index after ']'
                return decoded, i + 1
```

```
        else:
            # accumulate alphabetic characters directly
            decoded += s[i]
            i += 1
        return decoded, i

    result, _ = helper(0)
    return result
```

Example usage

```
#print(decodeString("3[a2[c]]")) # Output: "accaccacc"
```

• LC.ca

```
class Solution:
    def decodeString(self, s: str) -> str:
        num_stk, str_stk = [], []
        num, res = 0, ''
        for c in s:
            if c.isdigit():
                num = num * 10 + int(c)
            elif c == '[':
                num_stk.append(num)
                str_stk.append(res) # initial, res being pushed is empty str ''

            num, res = 0, ''
            elif c == ']':
                res = str_stk.pop() + res * num_stk.pop()
            else:
                res += c
        return res
```

◆ Quick Review

Key Insights

- Use a stack or recursion to manage nested encoded patterns.

- Process the string character by character, accumulating digits for the repeat count.
- When encountering a '[', start a new context (recursively or via a new stack frame) for the encoded substring.
- When encountering a ']', finish the current context and repeat the accumulated substring as needed.
- Careful handling of multi-digit numbers is essential.

Space & Time

Time Complexity: $O(n)$, where n is the length of the string.

Space Complexity: $O(n)$, for the recursion stack or the auxiliary stack used.

Solution

We can solve this problem using a recursive approach. When we see a digit, we determine the full repeat count. Upon encountering a '[', we recursively read the encoded substring until the corresponding ']'. Each recursive call returns the decoded substring until it hits a ']', which is then repeated based on the previously computed count.

Round 5 · Mid · Medium

p.1805

Alternatively, an iterative solution using a stack is also feasible. As we parse the string, push the current string and number onto the stack when encountering '[', then pop and combine when ']' is reached. This approach correctly handles nested patterns.

Round 5 · Mid · Medium

p.1806

Divide and Conquer

Round 5 · Mid · Medium · 3 questions

Round 5 · Mid · Medium

p.1807

Divide and Conquer

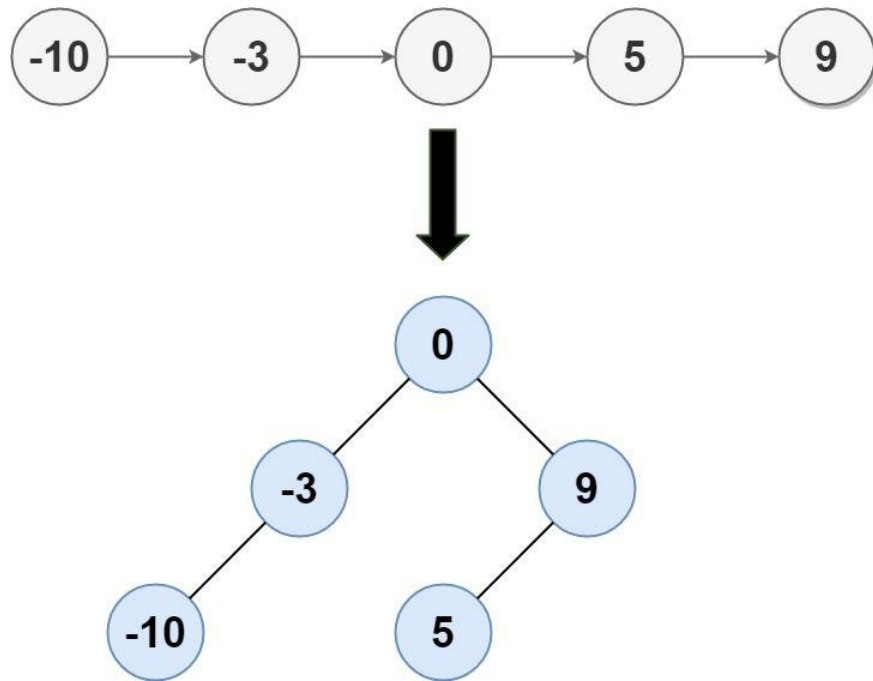
☐ #109 Convert Sorted List to Binary Search Tree

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · Divide and Conquer · Tree ·
Binary Search Tree · Binary Tree
Lists: AlgoMaster 600
Problem
Given the head of a singly linked list where
elements are sorted in ascending order, convert
it to a height-balanced binary search tree.
Example 1:

Round 5 · Mid · Medium

p.1808



Input: head = [-10,-3,0,5,9]
 Output: [0,-3,9,-10,null,5]
 Explanation: One possible answer is [0,-3,9,-10,null,5], which represents the shown height balanced BST.

Example 2:

Input: head = []
 Output: []

Constraints:

- The number of nodes in head is in the range [0, 2 * 10⁴].
- -105 ≤ Node.val ≤ 105

Community Solutions (Python)

• WalkCC

```

class Solution:
    def sortedListToBST(self, head: ListNode) -> TreeNode:
        def findMid(head: ListNode) -> ListNode:
            prev = None
            slow = head
            fast = head

            while fast and fast.next:
                prev = slow
                slow = slow.next
                fast = fast.next.next
            prev.next = None

            return slow

        if not head:
            return None
        if not head.next:
            return TreeNode(head.val)

        mid = findMid(head)
        root = TreeNode(mid.val)
        root.left = self.sortedListToBST(head)
        root.right = self.sortedListToBST(mid.next)
        return root

class Solution:
    def sortedListToBST(self, head: ListNode | None) -> TreeNode | None:
        arr = []

        # Construct the array.
        curr = head
        while curr:
            arr.append(curr.val)
            curr = curr.next
  
```

```
def helper(l: int, r: int) -> TreeNode | None:
    if l > r:
        return None
    m = (l + r) // 2
    root = TreeNode(arr[m])
    root.left = helper(l, m - 1)
    root.right = helper(m + 1, r)
    return root

return helper(0, len(arr) - 1)

class Solution:
    def sortedListToBST(self, head: ListNode | None) -> TreeNode | None:
        def helper(l: int, r: int) -> TreeNode | None:
            nonlocal head
            if l > r:
                return None

            m = (l + r) // 2

            # Simulate inorder traversal: recursively form the left half.

            left = helper(l, m - 1)

            # Once the left half is traversed, process the current node.
            root = TreeNode(head.val)
            root.left = left

            # Maintain the invariance.
            head = head.next

            # Simulate inorder traversal: recursively form the right half.

            root.right = helper(m + 1, r)
            return root

        return helper(0, self._getLength(head) - 1)

    def _getLength(self, head: ListNode | None) -> int:
        length = 0
        curr = head
        while curr:
            length += 1
```

Round 5 · Mid · Medium

p.1811

```
curr = curr.next
return length
```

• LeetCode

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left

# self.right = right
class Solution:
    def sortedListToBST(self, head: Optional[ListNode]) -> Optional[TreeNode]:
        def dfs(i: int, j: int) -> Optional[TreeNode]:
            if i > j:
                return None
            mid = (i + j) // 2
            l, r = dfs(i, mid - 1), dfs(mid + 1, j)
            return TreeNode(nums[mid], l, r)

        nums = []
        while head:
            nums.append(head.val)
            head = head.next
        return dfs(0, len(nums) - 1)
```

• SimplyLeet

Round 5 · Mid · Medium

p.1812

```
# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def sortedListToBST(self, head: ListNode) -> TreeNode:
        # Convert linked list to array for easier middle element access
        values = []
        while head:
            values.append(head.val)
            head = head.next

        # Helper function to build BST from sorted array
        def buildBST(left, right):
            if left > right:
                return None
            mid = (left + right) // 2 # Choose middle element as root for
            balance

            node = TreeNode(values[mid])
            # Recursively build left subtree
            node.left = buildBST(left, mid - 1)
            # Recursively build right subtree

            node.right = buildBST(mid + 1, right)
            return node

        # Build BST using the helper function
        return buildBST(0, len(values) - 1)
```

Divide and Conquer

□ #427 Construct Quad Tree

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Divide and Conquer · Tree · Matrix
 Lists: AlgoMaster 600

Problem

Given a $n * n$ matrix grid of 0's and 1's only. We want to represent grid with a Quad-Tree.

Return the root of the Quad-Tree representing grid.

A Quad-Tree is a tree data structure in which each internal node has exactly four children.

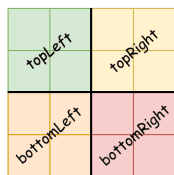
Besides, each node has two attributes:

- **val**: True if the node represents a grid of 1's or False if the node represents a grid of 0's. Notice that you can assign the val to True or False when isLeaf is False, and both are accepted in the answer.
- **isLeaf**: True if the node is a leaf node on the tree or False if the node has four children.

```
class Node {
public boolean val;
public boolean isLeaf;
public Node topLeft;
public Node topRight;
public Node bottomLeft;
public Node bottomRight;
}
```

We can construct a Quad-Tree from a two-dimensional area using the following steps:

1. If the current grid has the same value (i.e all 1's or all 0's) set isLeaf True and set val to the value of the grid and set the four children to Null and stop.
2. If the current grid has different values, set isLeaf to False and set val to any value and divide the current grid into four sub-grids as shown in the photo.
3. Recurse for each of the children with the proper sub-grid.



If you want to know more about the Quad-Tree, you can refer to the wiki.

Quad-Tree format:

Round 5 · Mid · Medium

p.1815

You don't need to read this section for solving the problem. This is only if you want to understand the output format here. The output represents the serialized format of a Quad-Tree using level order traversal, where null signifies a path terminator where no node exists below. It is very similar to the serialization of the binary tree. The only difference is that the node is represented as a list [isLeaf, val].

If the value of isLeaf or val is True we represent it as 1 in the list [isLeaf, val] and if the value of isLeaf or val is False we represent it as 0.

Example 1:

0	1
1	0

Input: grid = [[0,1],[1,0]]

Output: [[0,1],[1,0],[1,1],[1,1],[1,0]]

Explanation: The explanation of this example is shown below:

Notice that 0 represents False and 1 represents True in the photo representing the Quad-Tree.

Example 2:

Round 5 · Mid · Medium

p.1816

1	1	1	1	0	0	0	0
1	1	1	1	0	0	0	0
1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1
1	1	1	1	0	0	0	0
1	1	1	1	0	0	0	0
1	1	1	1	0	0	0	0
1	1	1	1	0	0	0	0

Input: grid = [[1,1,1,1,0,0,0,0],[1,1,1,1,0,0,0,0],[1,1,1,1,1,1,1,1],[1,1,1,1,1,1,1,1],[1,1,1,1,0,0,0,0],[1,1,1,1,0,0,0,0],[1,1,1,1,0,0,0,0],[1,1,1,1,0,0,0,0]]

Output:

[[0,1],[1,1],[0,1],[1,1],[1,0],null,null,null,null,[1,0],[1,0],[1,1],[1,1]]

Explanation: All values in the grid are not the same. We divide the grid into four sub-grids.

The topLeft, bottomLeft and bottomRight each has the same value.

The topRight have different values so we divide it into 4 sub-grids where each has the same value.

Explanation is shown in the photo below:

Constraints:

- $n == \text{grid.length} == \text{grid}[i].\text{length}$
- $n == 2^x$ where $0 \leq x \leq 6$

Community Solutions (Python)

- WalkCC

```
class Solution:
    def construct(self, grid: list[list[int]]) -> 'Node':
        return self._helper(grid, 0, 0, len(grid))

    def _helper(self, grid: list[list[int]], i: int, j: int, w: int) -> 'Node':
        if self._allSame(grid, i, j, w):
            return Node(grid[i][j] == 1, True)
        half = w // 2
        return Node(True, False,
                    self._helper(grid, i, j, half),
                    self._helper(grid, i + half, j, half),
                    self._helper(grid, i + half, j + half, half))

    def _allSame(self, grid: list[list[int]], i: int, j: int, w: int) -> bool:
        return all(grid[x][y] == grid[i][j]
                   for x in range(i, i + w)
                   for y in range(j, j + w))
```

• LeetDoocs

```
"""
# Definition for a QuadTree node.
class Node:
    def __init__(self, val, isLeaf, topLeft, topRight, bottomLeft,
                 bottomRight):
        self.val = val
        self.isLeaf = isLeaf
        self.topLeft = topLeft
        self.topRight = topRight
        self.bottomLeft = bottomLeft
        self.bottomRight = bottomRight
```

```

class Solution:
    def construct(self, grid: List[List[int]]) -> 'Node':
        def dfs(a, b, c, d):
            zero = one = 0
            for i in range(a, c + 1):
                for j in range(b, d + 1):
                    if grid[i][j] == 0:
                        zero = 1
                    else:
                        one = 1
            isLeaf = zero + one == 1
            val = isLeaf and one
            if isLeaf:
                return Node(grid[a][b], True)
            topLeft = dfs(a, b, (a + c) // 2, (b + d) // 2)
            topRight = dfs(a, (b + d) // 2 + 1, (a + c) // 2, d)
            bottomLeft = dfs((a + c) // 2 + 1, b, c, (b + d) // 2)
            bottomRight = dfs((a + c) // 2 + 1, (b + d) // 2 + 1, c, d)
            return Node(val, isLeaf, topLeft, topRight, bottomLeft, bottomRight)

        return dfs(0, 0, len(grid) - 1, len(grid[0]) - 1)

```

• SimplyLeet

Round 5 · Mid · Medium

p.1819

```

# Definition for a Quad-Tree Node.
class Node:
    def __init__(self, val, isLeaf, topLeft=None, topRight=None, bottomLeft=None, bottomRight=None):
        self.val = val # True for 1, False for 0.
        self.isLeaf = isLeaf # Boolean indicating if this node is a leaf.
        self.topLeft = topLeft # Child node for the top-left quadrant.
        self.topRight = topRight # Child node for the top-right quadrant.
        self.bottomLeft = bottomLeft # Child node for the bottom-left quadrant.
        self.bottomRight = bottomRight # Child node for the bottom-right quadrant.

    def construct(grid):
        # Helper function to determine if the current grid section is uniform.
        def is_uniform(r0, c0, length):
            value = grid[r0][c0]
            for i in range(r0, r0 + length):
                for j in range(c0, c0 + length):
                    if grid[i][j] != value:
                        return False, value
            return True, value

        # Recursive function to build the Quad Tree.
        def helper(r0, c0, length):
            uniform, value = is_uniform(r0, c0, length)
            if uniform:
                # All cells are the same: create a leaf node.
                return Node(val=(value == 1), isLeaf=True)
            else:
                half = length // 2
                # Recursively construct each quadrant.
                topLeft = helper(r0, c0, half)

```

Round 5 · Mid · Medium

p.1820

```

    topRight = helper(r0, c0 + half, half)
    bottomLeft = helper(r0 + half, c0, half)
    bottomRight = helper(r0 + half, c0 + half, half)
    # Create an internal node with four children.
    return Node(val=True, isLeaf=False, topLeft=topLeft, topRight=topR
ight,
                bottomLeft=bottomLeft, bottomRight=bottomRight)

n = len(grid)
return helper(0, 0, n)

```

Divide and Conquer

□ #654 Maximum Binary Tree

MED

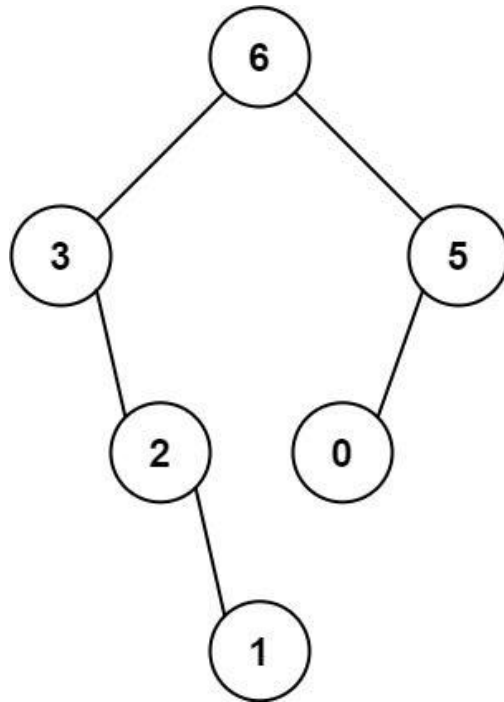
LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Divide and Conquer · Stack · Tree ·
 Monotonic Stack · Binary Tree
 Lists: AlgoMaster 600

Problem

You are given an integer array `nums` with no duplicates. A maximum binary tree can be built recursively from `nums` using the following algorithm:

1. Create a root node whose value is the maximum value in `nums`.
2. Recursively build the left subtree on the subarray prefix to the left of the maximum value.
3. Recursively build the right subtree on the subarray suffix to the right of the maximum value.

Return the maximum binary tree built from `nums`.

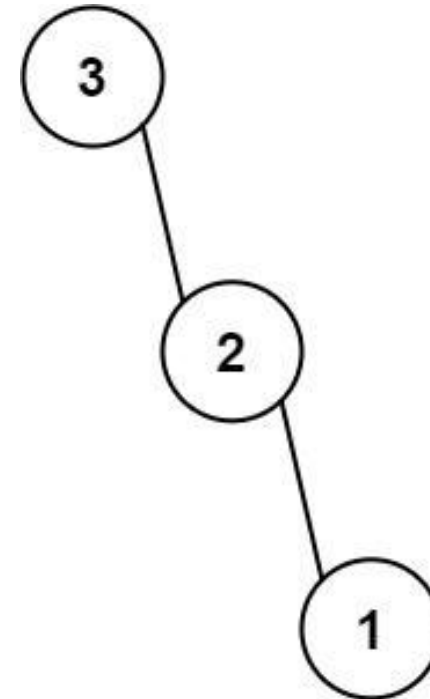
Example 1:

Input: nums = [3,2,1,6,0,5]

Output: [6,3,5,null,2,0,null,null,1]

Explanation: The recursive calls are as follow:

- The largest value in [3,2,1,6,0,5] is 6. Left prefix is [3,2,1] and right suffix is [0,5].
- The largest value in [3,2,1] is 3. Left prefix is [] and right suffix is [2,1].
- Empty array, so no child.
- The largest value in [2,1] is 2. Left prefix is [] and right suffix is [1].
- Empty array, so no child.

Example 2:

Input: nums = [3,2,1]

Output: [3,null,2,null,1]

Constraints:

- $1 \leq \text{nums.length} \leq 1000$
- $0 \leq \text{nums}[i] \leq 1000$
- All integers in nums are unique.

Community Solutions (Python)

- WalkCC

```
class Solution:
    def constructMaximumBinaryTree(self, nums: list[int]) -> TreeNode | None:
        def build(i: int, j: int) -> TreeNode | None:
            if i > j:
                return None

            maxNum = max(nums[i:j + 1])
            maxIndex = nums.index(maxNum)

            root = TreeNode(maxNum)

            root.left = build(i, maxIndex - 1)
            root.right = build(maxIndex + 1, j)
            return root

        return build(0, len(nums) - 1)
```

• LeetCode

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def constructMaximumBinaryTree(self, nums: List[int]) -> Optional[TreeNode]:
        def dfs(nums):
            if not nums:
                return None
            val = max(nums)
            i = nums.index(val)
            root = TreeNode(val)
            root.left = dfs(nums[:i])
            root.right = dfs(nums[i + 1 :])
            return root

        return dfs(nums)
```

• SimplyLeet

Round 5 · Mid · Medium

p.1825

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val          # Node value
        self.left = left        # Left child
        self.right = right       # Right child

class Solution:
    def constructMaximumBinaryTree(self, nums: list[int]) -> TreeNode:
        # Base case: if nums is empty return None

        if not nums:
            return None

        # Find index of the maximum element in the list
        max_index = nums.index(max(nums))
        # Create a new TreeNode with the maximum element
        node = TreeNode(nums[max_index])
        # Recursively build the left subtree using elements before max_index
        node.left = self.constructMaximumBinaryTree(nums[:max_index])
        # Recursively build the right subtree using elements after max_index
        node.right = self.constructMaximumBinaryTree(nums[max_index + 1:])

        # Return the created tree node
        return node
```

Round 5 · Mid · Medium

p.1826

Merge Sort

Round 5 · Mid · Medium · 2 questions

Round 5 · Mid · Medium

p.1827

Merge Sort

#148 Sort List

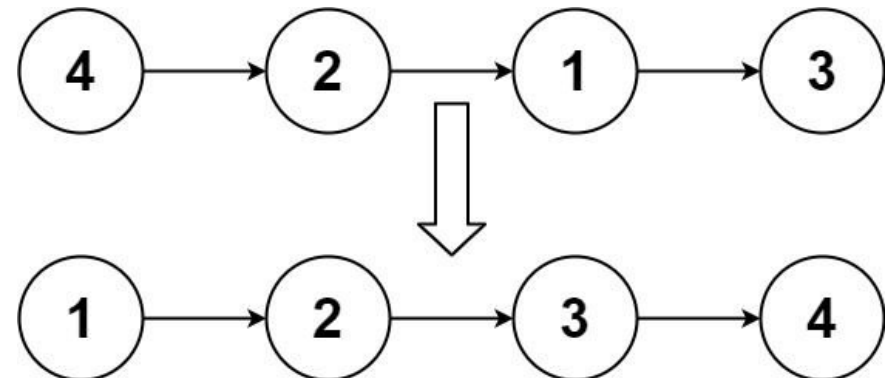
MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Linked List · Two Pointers · Divide and Conquer
 · Sorting · Merge Sort
 Lists: Grind 169

Problem

Given the head of a linked list, return the list after sorting it in ascending order.

Example 1:

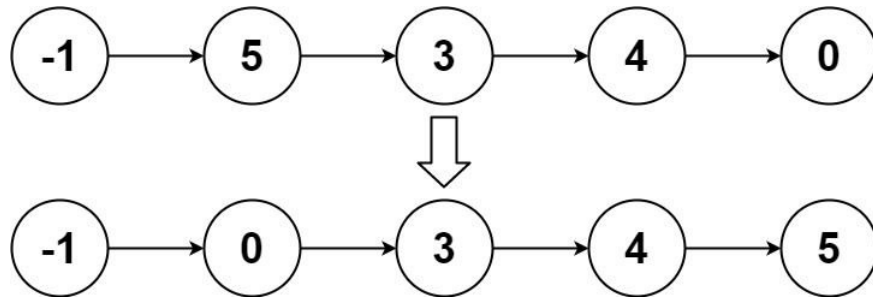


Input: head = [4,2,1,3]
 Output: [1,2,3,4]

Example 2:

Round 5 · Mid · Medium

p.1828



Input: head = [-1,5,3,4,0]
Output: [-1,0,3,4,5]

Example 3:

Input: head = []
Output: []

Constraints:

- The number of nodes in the list is in the range $[0, 5 * 10^4]$.
- $-105 \leq \text{Node.val} \leq 105$

Follow up: Can you sort the linked list in $O(n \log n)$ time and $O(1)$ memory (i.e. constant space)?

Community Solutions (Python)

- WalkCC

```

class Solution:
    def sortList(self, head: ListNode) -> ListNode:
        def split(head: ListNode, k: int) -> ListNode:
            while k > 1 and head:
                head = head.next
                k -= 1
            rest = head.next if head else None
            if head:
                head.next = None
            return rest

        def merge(l1: ListNode, l2: ListNode) -> tuple:
            dummy = ListNode(0)
            tail = dummy

            while l1 and l2:
                if l1.val > l2.val:
                    l1, l2 = l2, l1
                tail.next = l1
                l1 = l1.next

            tail = tail.next
            tail.next = l1 if l1 else l2
            while tail.next:
                tail = tail.next

            return dummy.next, tail

        length = 0
        curr = head
        while curr:
            length += 1
            curr = curr.next

        dummy = ListNode(0, head)

        k = 1
        while k < length:
            curr = dummy.next
            tail = dummy
            while curr:
  
```

```

l = curr
r = split(l, k)
curr = split(r, k)
mergedHead, mergedTail = merge(l, r)
tail.next = mergedHead
tail = mergedTail
k *= 2

return dummy.next

```

• LeetCode

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def sortList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        if head is None or head.next is None:
            return head
        slow, fast = head, head.next

```

```

        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
        l1, l2 = head, slow.next
        slow.next = None
        l1, l2 = self.sortList(l1), self.sortList(l2)
        dummy = ListNode()
        tail = dummy
        while l1 and l2:
            if l1.val <= l2.val:

```

```

                tail.next = l1
                l1 = l1.next
            else:
                tail.next = l2
                l2 = l2.next
            tail = tail.next
        tail.next = l1 or l2
        return dummy.next

```

Round 5 · Mid · Medium

p.1831

• SimplyLeet

```

# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def sortList(self, head: ListNode) -> ListNode:
        # Base case: if list is empty or contains a single element.
        if not head or not head.next:

```

```

            return head

```

```

        # Function to split the list into two halves
        def getMid(head):
            slow, fast = head, head.next
            # Move fast twice as fast as slow to locate middle
            while fast and fast.next:
                slow = slow.next
                fast = fast.next.next
            return slow

```

```

        # Split the linked list into two halves
        mid = getMid(head)
        right_head = mid.next
        mid.next = None # cut the list into two parts

        # Recursive calls to sort each half
        left = self.sortList(head)
        right = self.sortList(right_head)

```

Round 5 · Mid · Medium

p.1832


```

        # Merge two sorted lists
        return merge(left, right)

def merge(l1, l2):
    dummy = ListNode(0)
    tail = dummy
    # Merge the lists by comparing nodes
    while l1 and l2:
        if l1.val < l2.val:
            tail.next = l1
            l1 = l1.next
        else:
            tail.next = l2
            l2 = l2.next
        tail = tail.next
    # Append the remaining nodes
    tail.next = l1 if l1 else l2
    return dummy.next

```

• LC.ca

```

# Definition for singly-linked list.
# class ListNode:
# def __init__(self, val=0, next=None):
# self.val = val
# self.next = next
class Solution:
    def sortList(self, head: ListNode) -> ListNode:
        if head is None or head.next is None:
            return head
        slow, fast = head, head.next

        while fast and fast.next:
            slow, fast = slow.next, fast.next.next
            t = slow.next
            slow.next = None
            l1, l2 = self.sortList(head), self.sortList(t)
            dummy = ListNode()
            cur = dummy
            while l1 and l2:
                if l1.val <= l2.val:
                    cur.next = l1
                    l1 = l1.next
                else:
                    cur.next = l2
                    l2 = l2.next
                cur = cur.next
            cur.next = l1 if l1 else l2
            return dummy.next

```

Round 5 · Mid · Medium

p.1833

```

        l1 = l1.next
    else:
        cur.next = l2
        l2 = l2.next
        cur = cur.next
    cur.next = l1 or l2 # add the rest
    return dummy.next

```

◆ Quick Review

Key Insights

- Merge sort is well-suited for linked lists; it naturally relies on splitting the list and merging sorted sublists.
- The slow and fast pointer technique is used to find the middle point of the list.
- A recursive merge sort implementation may use $O(\log n)$ space for recursion stack. To achieve $O(1)$ extra space, an iterative bottom-up merge sort approach would be preferred.
- Merging two sorted lists is efficient with linked lists because of their sequential access.

Space & Time

Time Complexity: $O(n \log n)$

Space Complexity: $O(1)$ extra space for an iterative bottom-up approach ($O(\log n)$ for

Round 5 · Mid · Medium

p.1834

recursive calls if using recursion)

Solution

The solution employs the merge sort algorithm tailored for linked lists. The overall steps are:

- Use the slow and fast pointers to find the middle of the list and split it into two halves.
- Recursively sort the two halves.
- Merge the two sorted halves by comparing node values.
- Ensure that during merging, pointers are adjusted properly to build the sorted list.

For an $O(1)$ space solution, an iterative bottom-up merge sort can be implemented. However, the recursive merge sort is easier to understand and implement, though it uses $O(\log n)$ space on the call stack.

Merge Sort

□ #912 Sort an Array

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode Array · Divide and Conquer · Sorting · Heap (Priority Queue) · Merge Sort · Bucket Sort · Radix Sort · Counting Sort

Lists: AlgoMaster 600

Problem

Given an array of integers `nums`, sort the array in ascending order and return it.

You must solve the problem without using any built-in functions in $O(n \log(n))$ time complexity and with the smallest space complexity possible.

Example 1:

Input: `nums = [5,2,3,1]`
 Output: `[1,2,3,5]`
 Explanation: After sorting the array, the positions of some numbers are not changed (for example, 2 and 3), while the positions of other numbers are changed (for example, 1 and 5).

Example 2:

Input: `nums = [5,1,1,2,0,0]`
 Output: `[0,0,1,1,2,5]`
 Explanation: Note that the values of `nums` are not necessarily unique.

Constraints:

- $1 \leq \text{nums.length} \leq 5 * 10^4$
- $-5 * 10^4 \leq \text{nums}[i] \leq 5 * 10^4$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def sortArray(self, nums: list[int]) -> list[int]:
        self._mergeSort(nums, 0, len(nums) - 1)
        return nums

    def _mergeSort(self, nums: list[int], l: int, r: int) -> None:
        if l >= r:
            return

        def merge(nums: list[int], l: int, m: int, r: int) -> None:

            sorted = [0] * (r - l + 1)
            k = 0 # sorted's index
            i = l # left's index
            j = m + 1 # right's index

            while i <= m and j <= r:
                if nums[i] < nums[j]:
                    sorted[k] = nums[i]
                    k += 1
                    i += 1
                else:
                    sorted[k] = nums[j]
                    k += 1
                    j += 1

            # Put the possible remaining left part into the sorted array.
            while i <= m:
                sorted[k] = nums[i]
                k += 1
                i += 1
```

Round 5 · Mid · Medium

p.1837

```
# Put the possible remaining right part into the sorted array.
while j <= r:
    sorted[k] = nums[j]
    k += 1
    j += 1

nums[l:l + len(sorted)] = sorted

m = (l + r) // 2
```

```
self._mergeSort(nums, l, m)
self._mergeSort(nums, m + 1, r)
merge(nums, l, m, r)
```

```
class Solution:
    def sortArray(self, nums: list[int]) -> list[int]:
        self._heapSort(nums)
        return nums

    def _heapSort(self, nums: list[int]) -> None:
        def maxHeapify(nums: list[int], i: int, heapSize: int) -> None:
            l = 2 * i + 1
            r = 2 * i + 2
            largest = i

            if l < heapSize and nums[largest] < nums[l]:
                largest = l
            if r < heapSize and nums[largest] < nums[r]:
                largest = r
            if largest != i:
                nums[largest], nums[i] = nums[i], nums[largest]
                maxHeapify(nums, largest, heapSize)

        def buildMaxHeap(nums: list[int]) -> None:
            for i in range(len(nums) // 2, -1, -1):
```

Round 5 · Mid · Medium

p.1838

```

        maxHeapify(nums, i, len(nums))

    buildMaxHeap(nums)
    heapSize = len(nums)
    for i in reversed(range(1, len(nums))):
        nums[i], nums[0] = nums[0], nums[i]
        heapSize -= 1
        maxHeapify(nums, 0, heapSize)

class Solution:
    def sortArray(self, nums: List[int]) -> List[int]:
        self._quickSort(nums, 0, len(nums) - 1)
        return nums

    def _quickSort(self, nums: List[int], l: int, r: int) -> None:
        if l >= r:
            return

        def partition(nums: List[int], l: int, r: int) -> int:

            randIndex = random.randint(0, r - l) + l
            nums[randIndex], nums[r] = nums[r], nums[randIndex]
            pivot = nums[r]
            nextSwapped = l
            for i in range(l, r):
                if nums[i] <= pivot:
                    nums[nextSwapped], nums[i] = nums[i], nums[nextSwapped]
                    nextSwapped += 1
            nums[nextSwapped], nums[r] = nums[r], nums[nextSwapped]
            return nextSwapped

        m = partition(nums, l, r)
        self._quickSort(nums, l, m - 1)
        self._quickSort(nums, m + 1, r)

```

• LeetCode

Round 5 · Mid · Medium

p.1839

```

class Solution:
    def sortArray(self, nums: List[int]) -> List[int]:
        def quick_sort(l, r):
            if l >= r:
                return
            x = nums[randint(l, r)]
            i, j, k = l - 1, r + 1, l
            while k < j:
                if nums[k] < x:
                    nums[i + 1], nums[k] = nums[k], nums[i + 1]

                i, k = i + 1, k + 1
                elif nums[k] > x:
                    j -= 1
                    nums[j], nums[k] = nums[k], nums[j]

                else:
                    k = k + 1
            quick_sort(l, i)
            quick_sort(j, r)

        quick_sort(0, len(nums) - 1)

        return nums

class Solution:
    def sortArray(self, nums: List[int]) -> List[int]:
        def merge_sort(l, r):
            if l >= r:
                return
            mid = (l + r) >> 1
            merge_sort(l, mid)
            merge_sort(mid + 1, r)
            i, j = l, mid + 1
            tmp = []

```

Round 5 · Mid · Medium

p.1840

```

while i <= mid and j <= r:
    if nums[i] <= nums[j]:
        tmp.append(nums[i])
        i += 1
    else:
        tmp.append(nums[j])
        j += 1
if i <= mid:
    tmp.extend(nums[i : mid + 1])
if j <= r:

```

```

    tmp.extend(nums[j : r + 1])
for i in range(l, r + 1):
    nums[i] = tmp[i - l]

```

```

merge_sort(0, len(nums) - 1)
return nums

```

• SimplyLeet

```

def heapify(nums, heap_size, root_index):
    # Initialize largest as the root index
    largest = root_index
    left_child = 2 * root_index + 1 # left child index
    right_child = 2 * root_index + 2 # right child index

    # If left child exists and is larger than the current largest, update
    largest
    if left_child < heap_size and nums[left_child] > nums[largest]:
        largest = left_child

```

```

    # If right child exists and is larger than the current largest, update
    largest
    if right_child < heap_size and nums[right_child] > nums[largest]:
        largest = right_child

```

```

    # If largest is not the root, swap the values and heapify the affected
    subtree
    if largest != root_index:
        nums[root_index], nums[largest] = nums[largest], nums[root_index]
        heapify(nums, heap_size, largest)

```

```

def sortArray(nums):

```

```

    n = len(nums)
    # Build a max heap by heapifying each non-leaf node (from n//2 - 1 down to
    0)
    for i in range(n // 2 - 1, -1, -1):
        heapify(nums, n, i)

```

```

    # Extract elements one by one from the heap
    for i in range(n - 1, 0, -1):
        # Move current root to the end (swap)
        nums[0], nums[i] = nums[i], nums[0]
        # Heapify the reduced heap to maintain max heap invariant

```

```

        heapify(nums, i, 0)

```

```

    return nums

```

```

# Example usage:
# print(sortArray([5,2,3,1]))

```

QuickSort / QuickSelect

Round 5 · Mid · Medium · 3 questions

Round 5 · Mid · Medium

p.1843

QuickSort / QuickSelect

□ #75 Sort Colors

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Sorting
Lists: Grind 169

Problem

Given an array `nums` with `n` objects colored red, white, or blue, sort them in-place so that objects of the same color are adjacent, with the colors in the order red, white, and blue.

We will use the integers 0, 1, and 2 to represent the color red, white, and blue, respectively.

You must solve this problem without using the library's sort function.

Example 1:

```
Input: nums = [2,0,2,1,1,0]
Output: [0,0,1,1,2,2]
```

Example 2:

```
Input: nums = [2,0,1]
Output: [0,1,2]
```

Constraints:

- `n == nums.length`

Round 5 · Mid · Medium

p.1844

- $1 \leq n \leq 300$
- `nums[i]` is either 0, 1, or 2.

Follow up: Could you come up with a one-pass algorithm using only constant extra space?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def sortColors(self, nums: list[int]) -> None:
        zero = -1
        one = -1
        two = -1

        for num in nums:
            if num == 0:
                two += 1
                one += 1

            zero += 1
            nums[two] = 2
            nums[one] = 1
            nums[zero] = 0
            elif num == 1:
                two += 1
                one += 1
                nums[two] = 2
                nums[one] = 1
            else:
                two += 1
                nums[two] = 2
```

```
class Solution:
    def sortColors(self, nums: list[int]) -> None:
        l = 0 # The next 0 should be placed in l.
        r = len(nums) - 1 # The next 2 should be placed in r.

        i = 0
        while i <= r:
            if nums[i] == 0:
                nums[i], nums[l] = nums[l], nums[i]
                i += 1

            l += 1
            elif nums[i] == 1:
                i += 1
            else:
                # We may swap a 0 to index i, but we're still not sure whether this 0
                # is placed in the correct index, so we can't move pointer i.
                nums[i], nums[r] = nums[r], nums[i]
                r -= 1
```

• LeetCode

```
class Solution:
    def sortColors(self, nums: List[int]) -> None:
        i, j, k = -1, len(nums), 0
        while k < j:
            if nums[k] == 0:
                i += 1
                nums[i], nums[k] = nums[k], nums[i]
                k += 1
            elif nums[k] == 2:
                j -= 1
                nums[j], nums[k] = nums[k], nums[j]
            else:
                k += 1
```

• SimplyLeet

```
# Function to sort the colors using Dutch National Flag algorithm
def sortColors(nums):
    # Initialize pointers for the start, current, and end positions
    low, mid, high = 0, 0, len(nums) - 1

    # Process elements until mid pointer exceeds high pointer
    while mid <= high:
        # If the current element is 0 (red)
        if nums[mid] == 0:
            # Swap current element with the element at low pointer

            nums[low], nums[mid] = nums[mid], nums[low]
            # Increment both low and mid pointers
            low += 1
            mid += 1

        # If the current element is 1 (white)
        elif nums[mid] == 1:
            # Move mid pointer forward since it's already in the correct place
            mid += 1

        # If the current element is 2 (blue)
        else:
            # Swap current element with the element at high pointer
            nums[mid], nums[high] = nums[high], nums[mid]
            # Decrement high pointer
            high -= 1

    # Example usage:
    nums = [2,0,2,1,1,0]
    sortColors(nums)
    print(nums) # Output: [0,0,1,1,2,2]
```

• LC.ca

Round 5 · Mid · Medium

p.1847

```
class Solution:
    def sortColors(self, nums: List[int]) -> None:
        i, j, k = -1, len(nums), 0
        while k < j:
            if nums[k] == 0:
                i += 1
                nums[i], nums[k] = nums[k], nums[i]
                k += 1
            elif nums[k] == 2:
                j -= 1
                nums[j], nums[k] = nums[k], nums[j]
            else:
                k += 1
```

◆ Quick Review

Key Insights

- Use the Dutch National Flag algorithm to partition the array into three sections.
- Maintain three pointers: one for the next position for 0 (low), one for the next position for 2 (high), and one to traverse the array (mid).
- Swap elements appropriately to ensure that all 0's are at the beginning, all 2's are at the end, and 1's remain in the middle.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Round 5 · Mid · Medium

p.1848

Solution

The solution uses a three-pointer approach (Dutch National Flag algorithm). Initialize three pointers: low, mid, and high. As you iterate through the array:

- If the element is 0, swap it with the element at the low pointer and increment both low and mid.
- If the element is 1, simply move mid forward.
- If the element is 2, swap it with the element at the high pointer and decrement high.

This single pass method ensures that the array is sorted in-place with constant extra space. Key details include correctly handling swaps without losing any data and ensuring that the pointers are updated in the proper order.

QuickSort / QuickSelect

□ #215 Kth Largest Element in an Array

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Divide and Conquer · Sorting · Heap · Quickselect

Lists: Grind 169

Problem

Given an integer array `nums` and an integer `k`, return the `k`th largest element in the array.

Note that it is the `k`th largest element in the sorted order, not the `k`th distinct element.

Can you solve it without sorting?

Example 1:

Input: `nums = [3,2,1,5,6,4]`, `k = 2`
 Output: 5

Example 2:

Input: `nums = [3,2,3,1,2,4,5,5,6]`, `k = 4`
 Output: 4

Constraints:

- $1 \leq k \leq \text{nums.length} \leq 105$
- $-104 \leq \text{nums}[i] \leq 104$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findKthLargest(self, nums: List[int], k: int) -> int:
        minHeap = []

        for num in nums:
            heapq.heappush(minHeap, num)
            if len(minHeap) > k:
                heapq.heappop(minHeap)

        return minHeap[0]

class Solution:
    def findKthLargest(self, nums: List[int], k: int) -> int:
        def quickSelect(l: int, r: int, k: int) -> int:
            pivot = nums[r]

            nextSwapped = l
            for i in range(l, r):
                if nums[i] >= pivot:
                    nums[nextSwapped], nums[i] = nums[i], nums[nextSwapped]
                    nextSwapped += 1

            nums[nextSwapped], nums[r] = nums[r], nums[nextSwapped]

            count = nextSwapped - l + 1 # Number of nums >= pivot
            if count == k:
                return nums[nextSwapped]
            if count > k:
                return quickSelect(l, nextSwapped - 1, k)
            return quickSelect(nextSwapped + 1, r, k - count)

        return quickSelect(0, len(nums) - 1, k)
```

• LeetDoocs

```
class Solution:
    def findKthLargest(self, nums: List[int], k: int) -> int:
        def quick_sort(l: int, r: int) -> int:
            if l == r:
                return nums[l]
            i, j = l - 1, r + 1
            x = nums[(l + r) >> 1]
            while i < j:
                while 1:
                    i += 1
                    if nums[i] >= x:
                        break
                while 1:
                    j -= 1
                    if nums[j] <= x:
                        break
                if i < j:
                    nums[i], nums[j] = nums[j], nums[i]
            if j < k:
                return quick_sort(j + 1, r)

            return quick_sort(l, j)

        n = len(nums)
        k = n - k
        return quick_sort(0, n - 1)

class Solution:
    def findKthLargest(self, nums: List[int], k: int) -> int:
        return nlargest(k, nums)[-1]

class Solution:
    def findKthLargest(self, nums: List[int], k: int) -> int:
        cnt = Counter(nums)
        for i in count(max(cnt), -1):
            k -= cnt[i]
            if k <= 0:
                return i
```

• SimplyLeet

```
import heapq

def findKthLargest(nums, k):
    # Create a min-heap with the first k elements
    min_heap = nums[:k]
    heapq.heapify(min_heap) # O(k) time to build the initial heap

    # Process the remaining elements
    for num in nums[k:]:
        # If the current element is larger than the smallest in the heap,
        # replace it

        if num > min_heap[0]:
            heapq.heapreplace(min_heap, num) # O(log k)
    # The smallest element in the heap is the kth largest overall
    return min_heap[0]

# Example usage:
nums = [3, 2, 1, 5, 6, 4]
k = 2
print(findKthLargest(nums, k)) # Expected output: 5
```

• LC.ca

```
class Solution:
    def findKthLargest(self, nums: List[int], k: int) -> int:
        lo, hi = 0, len(nums) - 1

        while lo <= hi:
            pos = self.partition(nums, lo, hi)
            if pos == k - 1: # pos starting from 0, so -1
                return nums[pos] # partially sorted, desc
            elif pos > k - 1:
                hi = pos - 1
```

Round 5 · Mid · Medium

p.1853

```
        else: # pos < k - 1
            lo = pos + 1

        return -1 # or raise exception

def partition(self, nums: List[int], lo: int, hi: int) -> int:
    pivot, l, r = nums[lo], lo + 1, hi
    while l <= r:
        if nums[l] < pivot and nums[r] > pivot:
            # larger num at left of pivot, easier to count for k-th largest

            nums[l], nums[r] = nums[r], nums[l]
            l += 1
            r -= 1
        if nums[l] >= pivot:
            l += 1
        if nums[r] <= pivot:
            r -= 1

    # use nums[l] will lead to infinite looping
    nums[lo], nums[r] = nums[r], nums[lo]
    # possible there is duplicated num, but will be covered here

    return r

#####

class Solution:
    def findKthLargest(self, nums: List[int], k: int) -> int:
        def quick_sort(left, right, k):
            if left == right:
                return nums[left]
            i, j = left - 1, right + 1

            x = nums[(left + right) >> 1]
            while i < j:
                while 1:
                    i += 1
                    if nums[i] >= x:
                        break
                while 1:
                    j -= 1
                    if nums[j] <= x:
                        break
```

Round 5 · Mid · Medium

p.1854

```

        if i < j:
            nums[i], nums[j] = nums[j], nums[i]
    if j < k:
        return quick_sort(j + 1, right, k)
    return quick_sort(left, j, k)

n = len(nums)
return quick_sort(0, n - 1, n - k)

#####

class Solution:
    def findKthLargest(self, nums: List[int], k: int) -> int:
        def quick_sort(left, right, k):
            if left == right:
                return nums[left]
            i, j = left - 1, right + 1
            x = nums[(left + right) >> 1]
            while i < j:

                while 1:
                    i += 1
                    if nums[i] >= x:
                        break
                while 1:
                    j -= 1
                    if nums[j] <= x:
                        break
                if i < j:
                    nums[i], nums[j] = nums[j], nums[i]

            if j < k:
                return quick_sort(j + 1, right, k)
            return quick_sort(left, j, k)

n = len(nums)
return quick_sort(0, n - 1, n - k)

```

◆ Quick Review

Round 5 · Mid · Medium

p.1855

Key Insights

- kth largest element can be converted to finding the element at index $n-k$ if the array were sorted.
- A min-heap of size k is a practical method to maintain the k largest elements seen so far.
- The Quickselect algorithm can find the kth largest element in average $O(n)$ time with in-place partitioning.
- Choose an appropriate method based on performance needs and worst-case scenarios.

Space & Time

Time Complexity:

- Min-Heap: $O(n \log k)$ by maintaining a heap of size k .
- Quickselect: Average $O(n)$ but worst-case $O(n^2)$ with poor pivot selection.

Space Complexity:

- Min-Heap: $O(k)$
- Quickselect: $O(1)$ additional space with in-place partitioning.

Solution

Round 5 · Mid · Medium

p.1856

We can solve the problem using either a min-heap or the Quickselect algorithm.

– **Min-Heap Approach:** Initialize a min-heap with the first k elements. Iterate through the remaining elements; if an element is greater than the heap's root (the smallest in the heap), replace the root. After processing all elements, the root of the min-heap is the kth largest element. – **Quickselect Approach:** Choose a pivot and partition the array so that elements larger than the pivot come before it. Determine the position of the pivot; if it matches the index corresponding to the kth largest element, return it. Recurse into the appropriate part of the array.

Key points to consider include maintaining heap size for the first approach and proper pivot selection for the Quickselect approach.

Round 5 · Mid · Medium

p.1857

QuickSort / QuickSelect

☐ #1985 Find the Kth Largest Integer in the Array

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · String · Divide and Conquer · Sorting ·
Heap (Priority Queue) · Quickselect
Lists: AlgoMaster 600

Problem

You are given an array of strings `nums` and an integer `k`. Each string in `nums` represents an integer without leading zeros.

Return the string that represents the kth largest integer in `nums`.

Note: Duplicate numbers should be counted distinctly. For example, if `nums` is `["1","2","2"]`, "2" is the first largest integer, "2" is the second-largest integer, and "1" is the third-largest integer.

Example 1:

Input: `nums = ["3","6","7","10"]`, `k = 4`
Output: "3"
Explanation:
The numbers in `nums` sorted in non-decreasing order are `["3","6","7","10"]`.
The 4th largest integer in `nums` is "3".

Round 5 · Mid · Medium

p.1858

Example 2:

Input: nums = ["2","21","12","1"], k = 3
 Output: "2"
 Explanation:
 The numbers in nums sorted in non-decreasing order are ["1","2","12","21"].
 The 3rd largest integer in nums is "2".

Example 3:

Input: nums = ["0","0"], k = 2
 Output: "0"
 Explanation:
 The numbers in nums sorted in non-decreasing order are ["0","0"].
 The 2nd largest integer in nums is "0".

Constraints:

- $1 \leq k \leq \text{nums.length} \leq 104$
- $1 \leq \text{nums}[i].\text{length} \leq 100$
- `nums[i]` consists of only digits.
- `nums[i]` will not have any leading zeros.

Community Solutions (Python)

• WalkCC

```
class Solution:
    # Similar to 215. Kth Largest Element in an Array
    def kthLargestNumber(self, nums: list[str], k: int) -> str:
        minHeap = []

        for num in nums:
            heapq.heappush(minHeap, int(num))
            if len(minHeap) > k:
                heapq.heappop(minHeap)

        return str(minHeap[0])
```

• LeetCode

```
class Solution:
    def kthLargestNumber(self, nums: List[str], k: int) -> str:
        return nlargest(k, nums, key=lambda x: int(x))[k - 1]
```

• SimplyLeet

```
# Define a function to find the kth largest integer in the array
def kthLargestNumber(nums, k):
    # Sort the list 'nums' in descending order by using a custom key:
    # 1. Compare by the length of the string (more digits means larger number)
    # 2. If lengths are equal, compare lexicographically.
    nums.sort(key=lambda x: (len(x), x), reverse=True)
    # Return the kth largest element (0-indexed array so k-1)
    return nums[k - 1]

# Example usage:

if __name__ == "__main__":
    nums = ["3","6","7","10"]
    k = 4
    result = kthLargestNumber(nums, k)
    print(result) # Expected output: "3"
```

Backtracking**Round 5 · Mid · Medium · 12 questions****Round 5 · Mid · Medium****p.1861****Backtracking**☐ **#17 Letter Combinations of a Phone Number****MED****LeetDoocs · SimplyLeet · WalkCC · LeetCode Hash Table · String · Backtracking Lists: Grind 169****Problem**

Given a string containing digits from 2–9 inclusive, return all possible letter combinations that the number could represent. Return the answer in any order.

A mapping of digits to letters (just like on the telephone buttons) is given below. Note that 1 does not map to any letters.

Round 5 · Mid · Medium**p.1862**



Example 1:

Input: digits = "23"
Output: ["ad","ae","af","bd","be","bf","cd","ce","cf"]

Example 2:

Input: digits = "2"
Output: ["a","b","c"]

Constraints:

- $1 \leq \text{digits.length} \leq 4$
- $\text{digits}[i]$ is a digit in the range ['2', '9'].

Community Solutions (Python)

• WalkCC

```
class Solution:
    def letterCombinations(self, digits: str) -> List[str]:
        if not digits:
            return []

        digitToLetters = ['', '', 'abc', 'def', 'ghi',
                          'jkl', 'mno', 'pqrs', 'tuv', 'wxyz']

        ans = []

        def dfs(i: int, path: List[str]) -> None:

            if i == len(digits):
                ans.append(''.join(path))
                return

            for letter in digitToLetters[int(digits[i])]:
                path.append(letter)
                dfs(i + 1, path)
                path.pop()

        dfs(0, [])

        return ans
```

• LeetCode

```
class Solution:
    def letterCombinations(self, digits: str) -> List[str]:
        if not digits:
            return []

        d = ["abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"]
        ans = [""]

        for i in digits:
            s = d[int(i) - 2]
            ans = [a + b for a in ans for b in s]

        return ans
```



```
class Solution:
    def letterCombinations(self, digits: str) -> List[str]:
        def dfs(i: int):
            if i >= len(digits):
                ans.append(''.join(t))
                return
            for c in d[int(digits[i]) - 2]:
                t.append(c)
                dfs(i + 1)
                t.pop()

        if not digits:
            return []
        d = ["abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"]
        ans = []
        t = []
        dfs(0)
        return ans
```

• SimplyLeet

Python solution for Letter Combinations of a Phone Number

```
def letterCombinations(digits):
    # Mapping of digits to corresponding letters
    digit_to_letters = {
        "2": "abc",
        "3": "def",
        "4": "ghi",
        "5": "jkl",
        "6": "mno",

        "7": "pqrs",
        "8": "tuv",
        "9": "wxyz"
    }

    # If input is empty, return an empty list
    if not digits:
        return []

    result = []
```

Round 5 · Mid · Medium

p.1865

```
# Backtracking function to generate combinations
def backtrack(index, current_combination):
    # If the current combination length equals the digits length,
    # add the combination to the result list.
    if index == len(digits):
        result.append(current_combination)
        return

    # Get the corresponding letters for the current digit

    letters = digit_to_letters[digits[index]]
    # Explore all possible letter routes for the current digit.
    for letter in letters:
        backtrack(index + 1, current_combination + letter)

    # Initiate backtracking starting from index 0 and empty combination.
    backtrack(0, "")
    return result

# Example usage:

#print(letterCombinations("23"))
```

• LC.ca

```
class Solution:
    def letterCombinations(self, digits: str) -> List[str]:
        if not digits:
            return []
        d = ["abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"]
        ans = [""]
        for i in digits:
            s = d[int(i) - 2]
            ans = [a + b for a in ans for b in s]
        return ans
```

◆ Quick Review

Key Insights

Round 5 · Mid · Medium

p.1866

- Use a mapping from digits to the respective letters as defined by a telephone keypad.
- Backtracking is an ideal approach to generate all possible combinations.
- Each digit adds a level in the recursive tree where you append one letter at a time.
- Handling edge cases such as an empty input string is crucial.

Space & Time

Time Complexity: $O(4^n * n)$ where n is the length of the input string (each digit may map to up to 4 letters and adding a combination takes $O(n)$ time).

Space Complexity: $O(n)$ for the recursion call stack, not counting the space required for storing the output.

Solution

We can solve the problem using a backtracking algorithm. First, we define a dictionary mapping each digit to its corresponding letters. Then we initiate a recursive function called `backtrack` that takes the current combination and the index of the next digit to process. At each call, if the combination is

complete (length equal to the input string), we add it to the result list. Otherwise, we iterate through the letters corresponding to the current digit, append a letter to the combination, and recursively call `backtrack` with the next index. This approach ensures that all possible letter combinations are generated.

Backtracking

#22 Generate Parentheses

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming · Backtracking
Lists: Grind 169

Problem

Given n pairs of parentheses, write a function to generate all combinations of well-formed parentheses.

Example 1:

Input: $n = 3$
Output: ["((()))", "(()())", "(())()", "()(())", "()()()"]

Example 2:

Input: $n = 1$
Output: ["()"]

Constraints:

- $1 \leq n \leq 8$

Community Solutions (Python)

- WalkCC

Round 5 · Mid · Medium

p.1869

```
class Solution:
    def generateParenthesis(self, n):
        ans = []

        def dfs(l: int, r: int, s: list[str]) -> None:
            if l == 0 and r == 0:
                ans.append(''.join(s))
            if l > 0:
                s.append('(')
                dfs(l - 1, r, s)

            s.pop()
            if l < r:
                s.append(')')
                dfs(l, r - 1, s)
                s.pop()

        dfs(n, n, [])
        return ans
```

• LeetDoocs

```
class Solution:
    def generateParenthesis(self, n: int) -> List[str]:
        def dfs(l: int, r: int, t: str):
            if l > n or r > n or l < r:
                return
            if l == n and r == n:
                ans.append(t)
                return
            dfs(l + 1, r, t + "(")
            dfs(l, r + 1, t + ")")

        ans = []
        dfs(0, 0, "")
        return ans
```

• SimplyLeet

Round 5 · Mid · Medium

p.1870

```
def generateParenthesis(n):
    # List to store valid combinations
    result = []

    # Helper function for backtracking
    def backtrack(current, open_count, close_count):
        # If current string reaches the maximum length, it's a valid
        # combination
        if len(current) == 2 * n:
            result.append(current)
            return

        # If we can add an opening parenthesis, do it
        if open_count < n:
            backtrack(current + '(', open_count + 1, close_count)

        # If adding a closing parenthesis maintains the validity, do it
        if close_count < open_count:
            backtrack(current + ')', open_count, close_count + 1)

    # Start the recursion with an empty string and zero counts for both
    backtrack("", 0, 0)
    return result
```

```
# Example usage:
print(generateParenthesis(3))
```

• LC.ca

```
class Solution:
    def generateParenthesis(self, n: int) -> List[str]:
        def dfs(l, r, t):
            if l > n or r > n or l < r:
                return
            if l == n and r == n:
                ans.append(t)
                return
            dfs(l + 1, r, t + '(')
            dfs(l, r + 1, t + ')')
```

Round 5 · Mid · Medium

p.1871

```
ans = []
dfs(0, 0, '')
return ans

#####

class Solution(object):
    def generateParenthesis(self, n):
        """
        :type n: int
        :rtype: List[str]
        """

        def dfs(left, path, res, n):
            if len(path) == 2 * n:
                if left == 0:
                    res.append("".join(path))
                return

            if left < n:
                path.append("(")
                dfs(left + 1, path, res, n)
                path.pop()

            if left > 0:
                path.append(")")
                dfs(left - 1, path, res, n)
                path.pop()

        res = []

        dfs(0, [], res, n)
        return res
```

◆ Quick Review

Key Insights

Round 5 · Mid · Medium

p.1872

- Use a backtracking approach to build the valid combinations step by step.
- At any recursion step, you can add an opening parenthesis if you still have one available.
- You can add a closing parenthesis only if it would not result in an invalid sequence (i.e., the number of closing parentheses is less than the number of opening ones).
- The solution leverages recursion to explore all potential valid sequences while pruning invalid paths early.

Space & Time

Time Complexity: $O(4^n / (n^{3/2}))$

Space Complexity: $O(n)$ for the recursion stack, plus space for storing all valid combinations.

Solution

The solution uses a recursive backtracking technique to generate the combinations. The algorithm maintains counters for the number of '(' and ')' added to the current string. It recursively adds a '(' if the count is less than n and adds a ')' if doing so does not make the string invalid (i.e., if the count of ')' is less than

the count of '('). When a valid combination reaches a length of $2*n$, it is added to the result list.

Backtracking

#39 Combination Sum

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Backtracking
 Lists: Grind 169

Problem

Given an array of distinct integers `candidates` and a target integer `target`, return a list of all unique combinations of candidates where the chosen numbers sum to `target`. You may return the combinations in any order.

The same number may be chosen from `candidates` an unlimited number of times. Two combinations are unique if the frequency of at least one of the chosen numbers is different.

The test cases are generated such that the number of unique combinations that sum up to `target` is less than 150 combinations for the given input.

Example 1:

Round 5 · Mid · Medium

p.1875

Input: `candidates = [2,3,6,7]`, `target = 7`

Output: `[[2,2,3],[7]]`

Explanation:

2 and 3 are candidates, and $2 + 2 + 3 = 7$. Note that 2 can be used multiple times.

7 is a candidate, and $7 = 7$.

These are the only two combinations.

Example 2:

Input: `candidates = [2,3,5]`, `target = 8`

Output: `[[2,2,2,2],[2,3,3],[3,5]]`

Example 3:

Input: `candidates = [2]`, `target = 1`

Output: `[]`

Constraints:

- $1 \leq \text{candidates.length} \leq 30$
- $2 \leq \text{candidates}[i] \leq 40$
- All elements of `candidates` are distinct.
- $1 \leq \text{target} \leq 40$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def combinationSum(self, candidates: list[int],
                      target: int) -> list[list[int]]:
        ans = []

        def dfs(s: int, target: int, path: list[int]) -> None:
            if target < 0:
                return
            if target == 0:
                ans.append(path.clone())
```

Round 5 · Mid · Medium

p.1876

```

        return

    for i in range(s, len(candidates)):
        path.append(candidates[i])
        dfs(i, target - candidates[i], path)
        path.pop()

    candidates.sort()
    dfs(0, target, [])
    return ans

```

• LeetCode

```

class Solution:
    def combinationSum(self, candidates: List[int], target: int) -> List[List[int]]:
        def dfs(i: int, s: int):
            if s == 0:
                ans.append(t[:])
                return
            if s < candidates[i]:
                return
            for j in range(i, len(candidates)):
                t.append(candidates[j])

                dfs(j, s - candidates[j])
                t.pop()

        candidates.sort()
        t = []
        ans = []
        dfs(0, target)
        return ans

```

• SimplyLeet

Round 5 · Mid · Medium

p.1877

```

# Function to find all unique combinations that sum to the target
def combinationSum(candidates, target):
    # List to store all valid combinations
    result = []
    # Sort candidates to facilitate pruning of the search
    candidates.sort()

    # Recursive helper function for backtracking
    def backtrack(remaining, start, path):
        # If remaining is 0, path is a valid combination so add a copy of it to result
        if remaining == 0:
            result.append(list(path))
            return
        # Loop through candidates starting from the current index
        for i in range(start, len(candidates)):
            # If candidate is greater than remaining, break out for efficiency
            if candidates[i] > remaining:
                break
            # Include candidate in the current combination path
            path.append(candidates[i])

            # Recursively call backtrack with updated remaining and same index (allowing reuse)
            backtrack(remaining - candidates[i], i, path)
            # Backtrack by removing the candidate added in this iteration
            path.pop()

        # Start backtracking from index 0 with an empty path
        backtrack(target, 0, [])
        return result

    # Example usage
    print(combinationSum([2,3,6,7], 7))

```

• LC.ca

Round 5 · Mid · Medium

p.1878

```

class Solution:
    def combinationSum(self, candidates: List[int], target: int) -> List[List[
int]]:
    # i: start index for this recursion
    # s: sum
    def dfs(i, s):
        if s == target:
            ans.append(t.copy())
            return
        if s > target:
            return

        for j in range(i, len(candidates)):
            c = candidates[j]
            t.append(c)
            dfs(j, s + c)
            t.pop()

    ans = []
    t = []
    # candidates.sort() # diff from combinationSum-II, no need sorting it
    dfs(0, 0)

    return ans

#####

# dp version
class Solution_dp:
    def combinationSum(self, candidates: List[int], target: int) -> List[List[
int]]:
    # for each-target (from 1 to target), its dp[i][j]
    # => so 3-D array dp[][][]
    dp = []

```

Round 5 · Mid · Medium

p.1879

```

        candidates.sort()

        for i in range(1, target+1):
            cur = []
            for j in range(len(candidates)):
                if candidates[j] > i:
                    break
                if candidates[j] == i:
                    one = [candidates[j]]
                    cur.append(one)

                    break
                for a in dp[i - candidates[j] - 1]:
                    if candidates[j] > a[0]:
                        continue
                    deepCopied = a.copy()
                    deepCopied.insert(0, candidates[j])
                    cur.append(deepCopied)
            dp.append(cur)

        return dp[-1]

#####

class Solution(object):
    def combinationSum(self, candidates, target):
        """"
        :type candidates: List[int]
        :type target: int
        :rtype: List[List[int]]

        """"

        def dfs(candidates, start, target, path, res):
            if target == 0:
                return res.append(path + [])

            for i in range(start, len(candidates)):
                if target - candidates[i] >= 0:
                    path.append(candidates[i])
                    dfs(candidates, i, target - candidates[i], path, res)

```

Round 5 · Mid · Medium

p.1880


```

        path.pop()

    res = []
    dfs(candidates, 0, target, [], res)
    return res

#####

class Solution:
    def combinationSum(self, candidates: List[int], target: int) -> List[List[int]]:

        def dfs(i: int, s: int):
            if s == 0:
                ans.append(t[:])
                return
            if i >= len(candidates) or s < candidates[i]:
                return
            dfs(i + 1, s)
            t.append(candidates[i])
            dfs(i, s - candidates[i])
            t.pop()

        candidates.sort()
        t = []
        ans = []
        dfs(0, target)
        return ans

```

◆ Quick Review

Key Insights

- Backtracking is a natural fit to explore all potential combinations.
- Sorting candidates simplifies the process by allowing early termination if the current

Round 5 · Mid · Medium

p.1881

candidate exceeds the remaining target.

- Recursion is used to build combinations and backtrack once the current path exceeds or meets the target.
- The same candidate can be reused in the combination, so the recursive call does not advance the index.

Space & Time

Time Complexity:

$O(2^{(target/\min(candidates))})$ in the worst case where many combinations are possible.

Space Complexity: $O(target/\min(candidates))$ for the recursion stack and temporary space used to store combinations.

Solution

The solution uses a backtracking approach. First, sort the candidates to enable pruning of recursive calls once the candidate exceeds the remaining target. A recursive helper function builds up a combination, subtracting the candidate value from the target until it reaches zero (a valid combination) or becomes negative (an invalid path). The recursion allows the same candidate to be chosen again.

Round 5 · Mid · Medium

p.1882

Each valid combination is then added to the result list.

Backtracking

□ #40 Combination Sum II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Backtracking
 Lists: NeetCode 150

Problem

Given a collection of candidate numbers (candidates) and a target number (target), find all unique combinations in candidates where the candidate numbers sum to target.

Each number in candidates may only be used once in the combination.

Note: The solution set must not contain duplicate combinations.

Example 1:

```
Input: candidates = [10,1,2,7,6,1,5], target = 8
Output:
[
  [1,1,6],
  [1,2,5],
  [1,7],
  [2,6]
]
```

Example 2:

Input: candidates = [2,5,2,1,2], target = 5

Output:
[
[1,2,2],
[5]
]

Constraints:

- $1 \leq \text{candidates.length} \leq 100$
- $1 \leq \text{candidates}[i] \leq 50$
- $1 \leq \text{target} \leq 30$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def combinationSum2(self, candidates: list[int],
                       target: int) -> list[list[int]]:

        ans = []

        def dfs(s: int, target: int, path: list[int]) -> None:
            if target < 0:
                return
            if target == 0:
                ans.append(path.copy())

            return

        for i in range(s, len(candidates)):
            if i > s and candidates[i] == candidates[i - 1]:
                continue
            path.append(candidates[i])
            dfs(i + 1, target - candidates[i], path)
            path.pop()

        candidates.sort()

        dfs(0, target, [])
        return ans
```

Round 5 · Mid · Medium

p.1885

• LeetCode

```
class Solution:
    def combinationSum2(self, candidates: List[int], target: int) -> List[List[int]]:
        def dfs(i: int, s: int):
            if s == 0:
                ans.append(t[:])
                return
            if i >= len(candidates) or s < candidates[i]:
                return
            for j in range(i, len(candidates)):
                if j > i and candidates[j] == candidates[j - 1]:
                    continue
                t.append(candidates[j])
                dfs(j + 1, s - candidates[j])
                t.pop()

        candidates.sort()
        ans = []
        t = []
        dfs(0, target)
        return ans
```

• SimplyLeet

```
# Python Solution for Combination Sum II

def combinationSum2(candidates, target):
    # Sort candidates to handle duplicates easily.
    candidates.sort()
    results = []

    def backtrack(start, current_combination, current_sum):
        # If the current sum equals target, add a copy of the current
        # combination to results.
        if current_sum == target:
```

Round 5 · Mid · Medium

p.1886

```

        results.append(list(current_combination))
        return
    # Iterate over possible candidates starting from index 'start'.
    for i in range(start, len(candidates)):
        # Skip duplicate elements.
        if i > start and candidates[i] == candidates[i - 1]:
            continue
        new_sum = current_sum + candidates[i]
        # If the new sum exceeds target, break the loop.
        if new_sum > target:
            break
        # Include the candidate and recurse.
        current_combination.append(candidates[i])
        backtrack(i + 1, current_combination, new_sum)
        # Backtrack: remove the last element added.
        current_combination.pop()

    backtrack(0, [], 0)
    return results

```

```

# Example usage:
print(combinationSum2([10,1,2,7,6,1,5], 8))

```

Backtracking

□ #46 Permutations

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Backtracking
 Lists: Grind 169

Problem

Given an array `nums` of distinct integers, return all the possible permutations. You can return the answer in any order.

Example 1:

Input: `nums = [1,2,3]`
 Output: `[[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]`

Example 2:

Input: `nums = [0,1]`
 Output: `[[0,1],[1,0]]`

Example 3:

Input: `nums = [1]`
 Output: `[[1]]`

Constraints:

- `1 <= nums.length <= 6`
- `-10 <= nums[i] <= 10`
- All the integers of `nums` are unique.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def permute(self, nums: List[int]) -> List[List[int]]:
        ans = []
        used = [False] * len(nums)

        def dfs(path: List[int]) -> None:
            if len(path) == len(nums):
                ans.append(path.copy())
                return

            for i, num in enumerate(nums):
                if used[i]:
                    continue
                used[i] = True
                path.append(num)
                dfs(path)
                path.pop()
                used[i] = False

        dfs([])

        return ans
```

• LeetCode

```
class Solution:
    def permute(self, nums: List[int]) -> List[List[int]]:
        def dfs(i: int):
            if i >= n:
                ans.append(t[:])
                return
            for j, x in enumerate(nums):
                if not vis[j]:
                    vis[j] = True
                    t[i] = x
```

Round 5 · Mid · Medium

p.1889

```
        dfs(i + 1)
        vis[j] = False
```

```
n = len(nums)
vis = [False] * n
t = [0] * n
ans = []
dfs(0)
return ans
```

• SimplyLeet

```
def permute(nums):
    result = []

    def backtrack(current, remaining):
        # If no remaining elements, current permutation is complete.
        if not remaining:
            result.append(current.copy())
            return
        # Explore each candidate number.
        for i in range(len(remaining)):

            # Add the chosen number to the current permutation.
            current.append(remaining[i])
            # Recursively generate permutations with the remaining numbers.
            backtrack(current, remaining[:i] + remaining[i+1:])
            # Backtrack by removing the chosen number.
            current.pop()

    backtrack([], nums)
    return result

# Example usage:
print(permute([1,2,3]))
```

• LC.ca

Round 5 · Mid · Medium

p.1890

```
'''
remove an element by its value from a set
```

```
>>> my_set = {1, 2, 3, 4, 5}
>>> my_set.remove(3)
>>> print(my_set)
{1, 2, 4, 5}
```

```
-----
```

cannot remove an element by index from a set in Python3
need to convert the set to a list

```
>>> my_set = {1, 2, 3, 4, 5}
>>> my_list = list(my_set)
>>> del my_list[2] # remove the element at index 2
>>> my_set = set(my_list)
>>> print(my_set)
{1, 2, 4, 5}
'''
```

```
class Solution: # iterative
    def permute(self, nums: List[int]) -> List[List[int]]:
        res = [[]]
```

```
        if nums is None or len(nums) == 0:
            return ans
```

```
        for num in nums:
            new_res = []
```

```
            for perm in res:
                for i in range(len(perm) + 1):
                    new_perm = perm[:i] + [num] + perm[i:]
                    new_res.append(new_perm)
```

```
            res = new_res
```

```
        return res
```

```
#####
```

```
class Solution: # iterative, single_perm.insert(index, num)
    def permute(self, nums: List[int]) -> List[List[int]]:
        ans = [[]]
```

```
        if nums is None or len(nums) == 0:
            return ans
```

```
        for num in nums:
            tmp_list = []
```

```
            for single_perm in ans:
                for index in range(len(single_perm) + 1):
                    single_perm.insert(index, num)
                    tmp_list.append(single_perm.copy())
                    single_perm.pop(index)
```

```
            ans = tmp_list
```

```
        return ans
```

```
#####
```

```
'''
```

In Python 3, both the discard() and remove() methods of a set object are used to remove an element from the set, but there is one key difference:

* discard() removes the specified element from the set if it is present, but does nothing if the element is not present.

* remove() removes the specified element from the set if it is present, but raises a KeyError exception if the element is not present.

```
>>> a
{33, 66, 11, 44, 22, 55}
>>> a.discard(22)
>>> a
{33, 66, 11, 44, 55}

>>> a.discard(200)
>>>
>>> a.remove(200)
Traceback (most recent call last):
```

```
File "<stdin>", line 1, in <module>
KeyError: 200
'''
```

```
'''
>>> v1 = set([])
>>> v2 = set()
>>>
>>> v1
set()
```

```
>>> v2
set()
'''
class Solution:
    def permute(self, nums: List[int]) -> List[List[int]]:
        res = []
        visited = set([])

        def dfs(nums, path, res, visited):
            if len(path) == len(nums):
```

```
        res.append(path + [])
        return

    for i in range(0, len(nums)):
        if i not in visited:
            visited.add(i)
            path.append(nums[i])
            dfs(nums, path, res, visited)
            path.pop()
            visited.discard(i) # remove(i) will throw exception if i
not existing
```

```
    dfs(nums, [], res, visited)
    return res
```

```
#####
```

```
class Solution:
    def permute(self, nums: List[int]) -> List[List[int]]:
        def dfs(i):
            if i == n:
```

```
            ans.append(t[:])
            return
        for j in range(n):
            if not vis[j]:
                vis[j] = True
                t[i] = nums[j]
                dfs(i + 1)
                vis[j] = False
```

```
        n = len(nums)
```

```
        vis = [False] * n
        t = [0] * n
        ans = []
        dfs(0)
        return ans
```

◆ Quick Review

Key Insights

- Use backtracking to generate all possible permutations.
- Build each permutation by selecting numbers one at a time.
- When the current permutation reaches the length of nums, add it to the result.
- Backtracking allows you to undo choices and explore different orderings.

Space & Time

Time Complexity: $O(n * n!)$ – There are $n!$ permutations, and constructing each permutation takes $O(n)$ time.

Space Complexity: $O(n)$ – Recursion depth is limited to n , plus additional space for the current permutation.

Solution

The solution employs a backtracking algorithm. A helper function is used to build permutations recursively. At each step, the algorithm iterates through the available numbers, adds one to the current permutation if it has not been used yet, and recurses to handle the remaining numbers. Once the

Round 5 · Mid · Medium

p.1895

current permutation matches the length of the input array, it is added to the result list. Then, by "backtracking" (removing the last number added), the algorithm explores alternative possibilities. This systematic exploration ensures that all possible permutations are generated.

Round 5 · Mid · Medium

p.1896

Backtracking

#47 Permutations II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Backtracking · Sorting
 Lists: AlgoMaster 600

Problem

Given a collection of numbers, `nums`, that might contain duplicates, return all possible unique permutations in any order.

Example 1:

Input: `nums = [1,1,2]`
 Output:
`[[1,1,2],`
`[1,2,1],`
`[2,1,1]]`

Example 2:

Input: `nums = [1,2,3]`
 Output: `[[1,2,3], [1,3,2], [2,1,3], [2,3,1], [3,1,2], [3,2,1]]`

Constraints:

- $1 \leq \text{nums.length} \leq 8$
- $-10 \leq \text{nums}[i] \leq 10$

Community Solutions (Python)

Round 5 · Mid · Medium

p.1897

• WalkCC

```
class Solution:
    def permuteUnique(self, nums: List[int]) -> List[List[int]]:
        ans = []
        used = [False] * len(nums)

        def dfs(path: List[int]) -> None:
            if len(path) == len(nums):
                ans.append(path.copy())
                return

            for i, num in enumerate(nums):
                if used[i]:
                    continue
                if i > 0 and nums[i] == nums[i - 1] and not used[i - 1]:
                    continue
                used[i] = True
                path.append(num)
                dfs(path)
                path.pop()
                used[i] = False

        nums.sort()
        dfs([])
        return ans
```

• LeetDoocs

```
class Solution:
    def permuteUnique(self, nums: List[int]) -> List[List[int]]:
        def dfs(i: int):
            if i == n:
                ans.append(t[:])
                return
            for j in range(n):
                if vis[j] or (j and nums[j] == nums[j - 1] and not vis[j - 1]):
                    continue
                t[i] = nums[j]
```

Round 5 · Mid · Medium

p.1898

```

        vis[j] = True
        dfs(i + 1)
        vis[j] = False

```

```

n = len(nums)
nums.sort()
ans = []
t = [0] * n
vis = [False] * n
dfs(0)

```

```

return ans

```

• SimplyLeet

```

# Function to return all unique permutations of the input list nums.

```

```

def permuteUnique(nums):
    # Sort the numbers to facilitate duplicate checking.
    nums.sort()
    results = []
    permutation = []
    used = [False] * len(nums)

```

```

# The helper function for backtracking.

```

```

def backtrack():

```

```

    # If the current permutation is complete, add a copy to the results.
    if len(permutation) == len(nums):
        results.append(permutation.copy())
        return

```

```

    # Iterate over each number in nums.

```

```

    for i in range(len(nums)):
        # Skip used elements.
        if used[i]:
            continue

```

```

        # Skip duplicates: if the current element is the same as the
        previous and
        # the previous element was not used, then skip the current element.
        if i > 0 and nums[i] == nums[i - 1] and not used[i - 1]:
            continue
        # Mark the element as used.
        used[i] = True
        permutation.append(nums[i])
        # Recurse.
        backtrack()
        # Backtrack: unmark the element and remove it from the current
        permutation.

```

```

        used[i] = False
        permutation.pop()

```

```

    backtrack()
    return results

```

```

# Example usage:

```

```

# print(permuteUnique([1, 1, 2]))

```

Backtracking

#77 Combinations

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Backtracking

Lists: AlgoMaster 600

Problem

Given two integers n and k , return all possible combinations of k numbers chosen from the range $[1, n]$.

You may return the answer in any order.

Example 1:

Input: $n = 4, k = 2$
Output: $[[1,2],[1,3],[1,4],[2,3],[2,4],[3,4]]$
Explanation: There are 4 choose 2 = 6 total combinations.
Note that combinations are unordered, i.e., $[1,2]$ and $[2,1]$ are considered to be the same combination.

Example 2:

Input: $n = 1, k = 1$
Output: $[[1]]$
Explanation: There is 1 choose 1 = 1 total combination.

Constraints:

- $1 \leq n \leq 20$
- $1 \leq k \leq n$

Round 5 · Mid · Medium

p.1901

Community Solutions (Python)

• WalkCC

```
class Solution:
    def combine(self, n: int, k: int) -> list[list[int]]:
        ans = []

        def dfs(s: int, path: list[int]) -> None:
            if len(path) == k:
                ans.append(path.copy())
                return

            for i in range(s, n + 1):

                path.append(i)
                dfs(i + 1, path)
                path.pop()

        dfs(1, [])
        return ans
```

• LeetDoocs

```
class Solution:
    def combine(self, n: int, k: int) -> List[List[int]]:
        def dfs(i: int):
            if len(t) == k:
                ans.append(t[:])
                return
            if i > n:
                return
            t.append(i)
            dfs(i + 1)

            t.pop()
            dfs(i + 1)

        ans = []
        t = []
        dfs(1)
        return ans
```

Round 5 · Mid · Medium

p.1902

• SimplyLeet

```
# Define the function to get combinations
def combine(n, k):
    # This list will store all the valid combinations
    result = []

    # Helper function for backtracking
    def backtrack(start, current):
        # If current combination is of the required length, add a copy to
        result
        if len(current) == k:
            result.append(current.copy())

        return

    # Iterate from the current start to n
    # Prune the loop: Stop if remaining numbers are less than needed
    for num in range(start, n + 1):
        # Add the current number to the combination
        current.append(num)
        # Move on to the next number; increment start to ensure increasing
        backtrack(num + 1, current)
        # Remove the last added number to backtrack

    current.pop()

    # Initialize backtracking with starting number 1 and an empty combination
    backtrack(1, [])
    return result

# Example usage
print(combine(4, 2))
```

Round 5 · Mid · Medium

p.1903

Backtracking

#78 Subsets

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Backtracking · Bit Manipulation
Lists: Grind 169

Problem

Given an integer array `nums` of unique elements, return all possible subsets (the power set).

The solution set must not contain duplicate subsets. Return the solution in any order.

Example 1:

Input: `nums = [1,2,3]`
Output: `[[], [1], [2], [1,2], [3], [1,3], [2,3], [1,2,3]]`

Example 2:

Input: `nums = [0]`
Output: `[[], [0]]`

Constraints:

- $1 \leq \text{nums.length} \leq 10$
- $-10 \leq \text{nums}[i] \leq 10$
- All the numbers of `nums` are unique.

Community Solutions (Python)

Round 5 · Mid · Medium

p.1904

• WalkCC

```
class Solution:
    def subsets(self, nums: list[int]) -> list[list[int]]:
        ans = []

        def dfs(s: int, path: list[int]) -> None:
            ans.append(path)

            for i in range(s, len(nums)):
                dfs(i + 1, path + [nums[i]])

        dfs(0, [])
        return ans
```

• LeetCode

```
class Solution:
    def subsets(self, nums: List[int]) -> List[List[int]]:
        def dfs(i: int):
            if i == len(nums):
                ans.append(t[:])
                return
            dfs(i + 1)
            t.append(nums[i])
            dfs(i + 1)
            t.pop()

        ans = []
        t = []
        dfs(0)
        return ans
```

• SimplyLeet

Round 5 · Mid · Medium

p.1905

```
# Define the function to generate all subsets using backtracking.
def subsets(nums):
    # This list will store all the subsets
    result = []

    # Recursive helper function to generate subsets.
    def backtrack(start, current):
        # Append a copy of the current subset to the final result.
        result.append(current.copy())
        # Iterate over the possible next elements starting from index 'start'

        for i in range(start, len(nums)):
            # Include nums[i] in the current subset.
            current.append(nums[i])
            # Recurse with the next starting index.
            backtrack(i + 1, current)
            # Backtrack: remove the last element to explore other
            # possibilities.
            current.pop()

        # Start the recursion with an empty path.
        backtrack(0, [])

    return result

# Example usage:
print(subsets([1, 2, 3]))
```

• LC.ca

```
from typing import List

class Solution: # bfs
    def subsets(self, nums: List[int]) -> List[List[int]]:
        if not nums:
            return [[]]

        nums.sort() # sort() not necessary if no duplicates
        result = [[]]
```

Round 5 · Mid · Medium

p.1906

```

    for num in nums:
        result += [subset + [num] for subset in result]

    return result

class Solution: # dfs
    def subsets(self, nums: List[int]) -> List[List[int]]:
        def dfs(u, t):
            ans.append(t[:]) # or, t.copy()

            for i in range(u, len(nums)):
                t.append(nums[i])
                dfs(i + 1, t)
                t.pop()

        ans = []
        nums.sort() # sort() not necessary if no duplicates
        dfs(0, [])
        return ans

```

```

class Solution: # dfs, just pass down the final path
    def subsets(self, nums: List[int]) -> List[List[int]]:
        def dfs(nums, index, path, ans):
            ans.append(path)
            [dfs(nums, i + 1, path + [nums[i]], ans) for i in range(index, len
(nums))]

        ans = []
        dfs(nums, 0, [], ans)
        return ans

```

```

class Solution: # dfs, but running slower, since need to reference from parent
method for 'nums' and 'ans'
    def subsets(self, nums: List[int]) -> List[List[int]]:
        def dfs(index, path):
            ans.append(path)
            [dfs(i + 1, path + [nums[i]]) for i in range(index, len(nums))]

        ans = []
        dfs(0, [])

        return ans

```

◆ Quick Review

Key Insights

- The problem is equivalent to generating the power set of the given set.
- A recursive backtracking approach can be used to explore both including and excluding each element.
- Iterative (bit manipulation) solutions are also possible since there are 2^n subsets in total.
- The maximum array length is small (up to 10), making exponential time solutions feasible.

Space & Time

Time Complexity: $O(2^n * n)$ – There are 2^n subsets and creating each subset may take up

to $O(n)$ time.

Space Complexity: $O(2^n * n)$ – Space for storing all subsets. The recursion stack has a maximum depth of $O(n)$.

Solution

We solve the problem using backtracking, where we decide for each element whether to include it in the current subset or not. We recursively build all subsets starting at an empty list and at each step appending the current subset to our result list. We can also use iterative bit manipulation but here we focus on the backtracking approach. This algorithm leverages recursion and a list to maintain the current state, leading to a clear and concise implementation.

Backtracking

#90 Subsets II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Backtracking · Bit Manipulation
 Lists: Denny Zhang

Problem

Given an integer array `nums` that may contain duplicates, return all possible subsets (the power set).

The solution set must not contain duplicate subsets. Return the solution in any order.

Example 1:

```
Input: nums = [1,2,2]
Output: [[], [1], [1,2], [1,2,2], [2], [2,2]]
```

Example 2:

```
Input: nums = [0]
Output: [[], [0]]
```

Constraints:

- $1 \leq \text{nums.length} \leq 10$
- $-10 \leq \text{nums}[i] \leq 10$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def subsetsWithDup(self, nums: List[int]) -> List[List[int]]:
        ans = []

        def dfs(s: int, path: List[int]) -> None:
            ans.append(path)
            if s == len(nums):
                return

            for i in range(s, len(nums)):

                if i > s and nums[i] == nums[i - 1]:
                    continue
                dfs(i + 1, path + [nums[i]])

        nums.sort()
        dfs(0, [])
        return ans
```

• LeetCode

```
class Solution:
    def subsetsWithDup(self, nums: List[int]) -> List[List[int]]:
        def dfs(i: int):
            if i == len(nums):
                ans.append(t[:])
                return
            t.append(nums[i])
            dfs(i + 1)
            x = t.pop()
            while i + 1 < len(nums) and nums[i + 1] == x:
                i += 1
            dfs(i + 1)

        nums.sort()
        ans = []
        t = []
        dfs(0)
        return ans
```

Round 5 · Mid · Medium

p.1911

• SimplyLeet

```
def subsetsWithDup(nums):
    # Sort the numbers to handle duplicates
    nums.sort()
    result = [] # To store all subsets
    subset = [] # Current subset being built

    def backtrack(start):
        # Append a copy of the current subset to the result
        result.append(list(subset))
        # Iterate over possible elements to include

        for i in range(start, len(nums)):
            # If the current element is a duplicate and not at the start of
            this loop, skip it
            if i > start and nums[i] == nums[i - 1]:
                continue
            # Include current element into the subset
            subset.append(nums[i])
            # Recurse with next start index
            backtrack(i + 1)
            # Backtrack and remove the last element added
            subset.pop()

    backtrack(0)
    return result

# Example usage:
print(subsetsWithDup([1, 2, 2]))
```

• LC.ca

Round 5 · Mid · Medium

p.1912


```
class Solution:
    def subsetsWithDup(self, nums: List[int]) -> List[List[int]]:
        def dfs(u, t):
            ans.append(t[:]) # or, t.copy()
            for i in range(u, len(nums)):
                # also good for [1,5,5], second '5' will be skipped here if
                # but second '5' is always covered, because i==u when
                ans=[1,5] and i=2
                if i != u and nums[i] == nums[i - 1]:
                    continue
                t.append(nums[i])
```

```
        dfs(i + 1, t)
        t.pop()
```

```
    ans = []
    nums.sort()
    dfs(0, [])
    return ans
```

```
# iteration
```

```
class Solution:
    def subsetsWithDup(self, nums: List[int]) -> List[List[int]]:
        if not nums:
            return [[]]

        nums.sort() # Sorting is necessary to handle duplicates.
        result = [[]]
        start_idx = 0 # Start index for the new subsets
        new_subsets_size = 0
```

```
        for i in range(len(nums)):
            size = len(result)
            # If the current number is the same as the previous one,
            # we only want to extend the subsets added in the previous
            iteration.
            if i > 0 and nums[i] == nums[i - 1]:
                start_idx = size - new_subsets_size
            else:
                start_idx = 0

            new_subsets_size = 0 # Reset the size of newly created subsets
```

```
        for j in range(start_idx, size):
            current_subset = result[j] + [nums[i]]
            result.append(current_subset)
            new_subsets_size += 1
```

```
    return result
```

```
#####
```

```
class Solution:
    def subsetsWithDup(self, nums: List[int]) -> List[List[int]]:
        nums.sort()
        n = len(nums)
        ans = []
        for mask in range(1 << n):
            ok = True
            t = []
            for i in range(n):
                if mask >> i & 1:

                    if i and (mask >> (i - 1) & 1) == 0 and nums[i] == nums[i
- 1]:
                        ok = False
                        break
                    t.append(nums[i])
            if ok:
                ans.append(t)
        return ans
```

◆ Quick Review

Key Insights

- Sort the array first so that duplicate elements are adjacent.
- Use backtracking to generate all possible subsets.
- Skip duplicate elements in the recursion to avoid forming duplicate subsets.
- The use of sorted order and conditionally skipping elements is the trick to handle duplicates.

Space & Time

Time Complexity: $O(2^n * n)$ since each subset creation may take $O(n)$ in the worst case.

Space Complexity: $O(2^n * n)$ for storing the subsets, plus $O(n)$ for recursion stack.

Solution

The solution uses a backtracking technique that builds subsets incrementally. The input array is first sorted to bring duplicates together. During recursion, if the current element is the same as the previous one and it has not been used in the current recursion

path, it is skipped to prevent duplicate subsets. This ensures that each subset is unique. The algorithm stores each valid subset by copying the current subset state into the result list.

Backtracking

#93 Restore IP Addresses

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Backtracking
Lists: AlgoMaster 600

Problem

A valid IP address consists of exactly four integers separated by single dots. Each integer is between 0 and 255 (inclusive) and cannot have leading zeros.

- For example, "0.1.2.201" and "192.168.1.1" are valid IP addresses, but "0.011.255.245", "192.168.1.312" and "192.168@1.1" are invalid IP addresses.

Given a string *s* containing only digits, return all possible valid IP addresses that can be formed by inserting dots into *s*. You are not allowed to reorder or remove any digits in *s*. You may return the valid IP addresses in any order.

Example 1:

Input: *s* = "25525511135"
Output: ["255.255.11.135", "255.255.111.35"]

Round 5 · Mid · Medium

p.1917

Example 2:

Input: *s* = "0000"
Output: ["0.0.0.0"]

Example 3:

Input: *s* = "101023"
Output: ["1.0.10.23", "1.0.102.3", "10.1.0.23", "10.10.2.3", "101.0.2.3"]

Constraints:

- $1 \leq s.length \leq 20$
- *s* consists of digits only.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def restoreIpAddresses(self, s: str) -> list[str]:
        ans = []

        def dfs(start: int, path: list[int]) -> None:
            if len(path) == 4 and start == len(s):
                ans.append(path[0] + '.' + path[1] + '.' + path[2] + '.' + path[3])
                return
            if len(path) == 4 or start == len(s):
                return

            for length in range(1, 4):
                if start + length > len(s):
                    return # out-of-bounds
                if length > 1 and s[start] == '0':
                    return # leading '0'
                num = s[start: start + length]
                if int(num) > 255:
                    return
                dfs(start + length, path + [num])
```

Round 5 · Mid · Medium

p.1918

```
dfs(0, [])
return ans
```

• LeetCode

```
class Solution:
    def restoreIpAddresses(self, s: str) -> List[str]:
        def check(i: int, j: int) -> bool:
            if s[i] == "0" and i != j:
                return False
            return 0 <= int(s[i : j + 1]) <= 255

        def dfs(i: int):
            if i >= n and len(t) == 4:
                ans.append(".".join(t))

            return
            if i >= n or len(t) >= 4:
                return
            for j in range(i, min(i + 3, n)):
                if check(i, j):
                    t.append(s[i : j + 1])
                    dfs(j + 1)
                    t.pop()

        n = len(s)

        ans = []
        t = []
        dfs(0)
        return ans
```

• SimplyLeet

Round 5 · Mid · Medium

p.1919

```
# Python solution using backtracking

class Solution:
    def restoreIpAddresses(self, s: str) -> list:
        result = [] # List to store valid IP addresses

        def backtrack(start: int, path: list):
            # If 4 parts are collected and we've used all characters, record
            # the result
            if len(path) == 4 and start == len(s):
                result.append(".".join(path))

            return
            # If either 4 parts are collected or not enough parts remain, prune
            # the search
            if len(path) == 4 or start == len(s):
                return

            # Try each segment length from 1 to 3
            for length in range(1, 4):
                # Avoid index overflow
                if start + length > len(s):
                    break

                segment = s[start:start+length]
                # Check for leading zeros (segment "0" is valid)
                if len(segment) > 1 and segment[0] == '0':
                    continue
                # Convert segment to integer and check valid range [0, 255]
                if int(segment) > 255:
                    continue
                # Append segment to path and backtrack for next segment
                backtrack(start + length, path + [segment])

        backtrack(0, [])
        return result

# Example usage:
# sol = Solution()
# print(sol.restoreIpAddresses("25525511135"))
```

Round 5 · Mid · Medium

p.1920

Backtracking

#131 Palindrome Partitioning

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming · Backtracking
Lists: NeetCode 150

Problem

Given a string s , partition s such that every substring of the partition is a palindrome. Return all possible palindrome partitioning of s .

Example 1:

Input: $s = \text{"aab"}$
Output: $[[\text{"a"}, \text{"a"}, \text{"b"}], [\text{"aa"}, \text{"b"}]]$

Example 2:

Input: $s = \text{"a"}$
Output: $[[\text{"a"}]]$

Constraints:

- $1 \leq s.length \leq 16$
- s contains only lowercase English letters.

Community Solutions (Python)

- WalkCC

Round 5 · Mid · Medium

p.1921

```
class Solution:
    def partition(self, s: str) -> list[list[str]]:
        ans = []

        def isPalindrome(s: str) -> bool:
            return s == s[::-1]

        def dfs(s: str, j: int, path: list[str], ans: list[list[str]]) -> None:
            if j == len(s):
                ans.append(path)

            return

            for i in range(j, len(s)):
                if isPalindrome(s[j: i + 1]):
                    dfs(s, i + 1, path + [s[j: i + 1]], ans)

        dfs(s, 0, [], ans)
        return ans
```

• LeetDoocs

```
class Solution:
    def partition(self, s: str) -> List[List[str]]:
        def dfs(i: int):
            if i == n:
                ans.append(t[:])
                return
            for j in range(i, n):
                if f[i][j]:
                    t.append(s[i: j + 1])
                    dfs(j + 1)

            t.pop()

        n = len(s)
        f = [[True] * n for _ in range(n)]
        for i in range(n - 1, -1, -1):
            for j in range(i + 1, n):
                f[i][j] = s[i] == s[j] and f[i + 1][j - 1]

        ans = []
        t = []
        dfs(0)
```

Round 5 · Mid · Medium

p.1922

```
return ans
```

• SimplyLeet

```
def partition(s):
    # Helper function to check if a given substring is a palindrome
    def is_palindrome(sub):
        return sub == sub[::-1]

    result = [] # final list to store all possible partitions

    # backtracking helper function: current_index points to the starting index
    # of the current search,
    # current_partition stores the partition done so far.
    def backtrack(current_index, current_partition):

        # if we reached the end of the string, add the current_partition to the
        # result list
        if current_index == len(s):
            result.append(list(current_partition))
            return

        # try all possible substrings starting from current_index
        for i in range(current_index, len(s)):
            substring = s[current_index:i+1] # current substring
            # if the substring is a palindrome, add it to current_partition
            if is_palindrome(substring):
                current_partition.append(substring)

                backtrack(i+1, current_partition)
                # backtrack: remove the last added substring
                current_partition.pop()

        backtrack(0, [])
    return result

# Example usage:
print(partition("aab"))
```

Backtracking

□ #216 Combination Sum III

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Backtracking
 Lists: AlgoMaster 600

Problem

Find all valid combinations of k numbers that sum up to n such that the following conditions are true:

- Only numbers 1 through 9 are used.
- Each number is used at most once.

Return a list of all possible valid combinations. The list must not contain the same combination twice, and the combinations may be returned in any order.

Example 1:

```
Input: k = 3, n = 7
Output: [[1,2,4]]
Explanation:
1 + 2 + 4 = 7
There are no other valid combinations.
```

Example 2:

Input: k = 3, n = 9
 Output: [[1,2,6],[1,3,5],[2,3,4]]
 Explanation:
 1 + 2 + 6 = 9
 1 + 3 + 5 = 9
 2 + 3 + 4 = 9
 There are no other valid combinations.

Example 3:

Input: k = 4, n = 1
 Output: []
 Explanation: There are no valid combinations.
 Using 4 different numbers in the range [1,9], the smallest sum we can get is 1+2+3+4 = 10 and since 10 > 1, there are no valid combination.

Constraints:

- $2 \leq k \leq 9$
- $1 \leq n \leq 60$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def combinationSum3(self, k: int, n: int) -> list[list[int]]:
        ans = []

        def dfs(k: int, n: int, s: int, path: list[int]) -> None:
            if k == 0 and n == 0:
                ans.append(path)
                return
            if k == 0 or n < 0:
                return

            for i in range(s, 10):
                dfs(k - 1, n - i, i + 1, path + [i])

        dfs(k, n, 1, [])
        return ans
```

Round 5 · Mid · Medium

p.1925

• LeetCode

```
class Solution:
    def combinationSum3(self, k: int, n: int) -> List[List[int]]:
        def dfs(i: int, s: int):
            if s == 0:
                if len(t) == k:
                    ans.append(t[:])
                return
            if i > 9 or i > s or len(t) >= k:
                return
            t.append(i)

            dfs(i + 1, s - i)
            t.pop()
            dfs(i + 1, s)

        ans = []
        t = []
        dfs(1, n)
        return ans
```

```
class Solution:
    def combinationSum3(self, k: int, n: int) -> List[List[int]]:
        ans = []
        for mask in range(1 << 9):
            if mask.bit_count() == k:
                t = [i + 1 for i in range(9) if mask >> i & 1]
                if sum(t) == n:
                    ans.append(t)
        return ans
```

• SimplyLeet

Round 5 · Mid · Medium

p.1926

```
def combinationSum3(k, n):
    result = []

    def backtrack(start, path, current_sum):
        # If we've chosen k numbers, check the sum.
        if len(path) == k:
            if current_sum == n:
                result.append(path.copy())
            return

        # Iterate through numbers from 'start' to 9.
        for num in range(start, 10):
            if current_sum + num > n: # Early pruning.
                break
            path.append(num)
            backtrack(num + 1, path, current_sum + num)
            path.pop() # Backtrack.

    backtrack(1, [], 0)
    return result
```

```
# Example usage:
print(combinationSum3(3, 7)) # Output: [[1,2,4]]
```

Round 5 · Mid · Medium

p.1927

BST / Ordered Set

Round 5 · Mid · Medium · 13 questions

Round 5 · Mid · Medium

p.1928

BST / Ordered Set

□ #99 Recover Binary Search Tree

MED

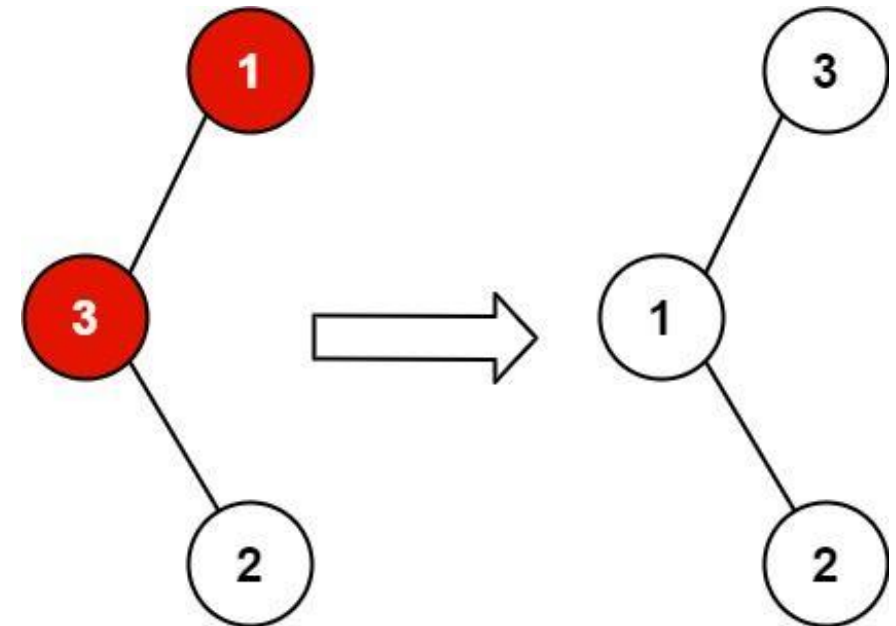
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · Depth-First Search · Binary Search Tree · Binary Tree

Lists: AlgoMaster 600

Problem

You are given the root of a binary search tree (BST), where the values of exactly two nodes of the tree were swapped by mistake. Recover the tree without changing its structure.

Example 1:

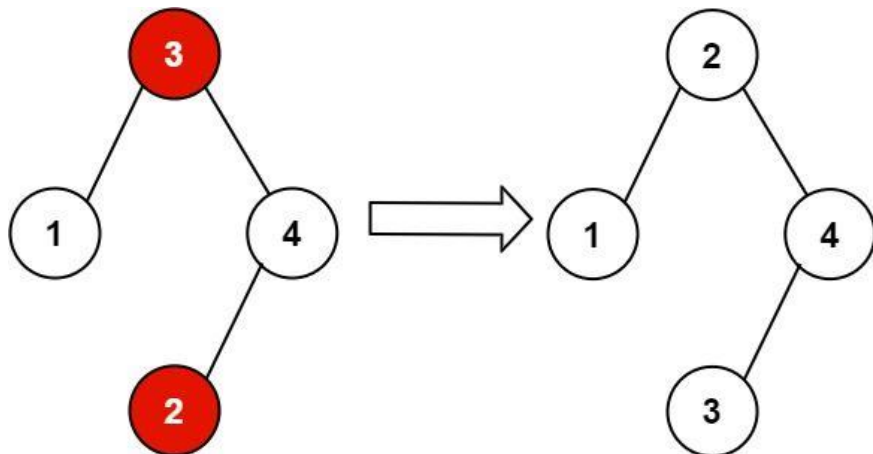


Input: root = [1,3,null,null,2]

Output: [3,1,null,null,2]

Explanation: 3 cannot be a left child of 1 because $3 > 1$. Swapping 1 and 3 makes the BST valid.

Example 2:



Input: root = [3,1,4,null,null,2]

Output: [2,1,4,null,null,3]

Explanation: 2 cannot be in the right subtree of 3 because $2 < 3$. Swapping 2 and 3 makes the BST valid.

Constraints:

- The number of nodes in the tree is in the range $[2, 1000]$.
- $-231 \leq \text{Node.val} \leq 231 - 1$

Follow up: A solution using $O(n)$ space is pretty straight-forward. Could you devise a constant $O(1)$ space solution?

Community Solutions (Python)

- WalkCC

```
class Solution:
    def recoverTree(self, root: TreeNode | None) -> None:
        def swap(x: TreeNode | None, y: TreeNode | None) -> None:
            temp = x.val
            x.val = y.val
            y.val = temp
```

```
    def inorder(root: TreeNode | None) -> None:
        if not root:
            return
```

```
        inorder(root.left)
```

```
        if self.pred and root.val < self.pred.val:
            self.y = root
            if not self.x:
                self.x = self.pred
            else:
                return
            self.pred = root
```

```
        inorder(root.right)
```

```
        inorder(root)
        swap(self.x, self.y)
```

```
        pred = None
        x = None # the first wrong node
        y = None # the second wrong node
```

```
class Solution:
    def recoverTree(self, root: TreeNode | None) -> None:
        pred = None
        x = None # the first wrong node
        y = None # the second wrong node
        stack = []
```

```
        while root or stack:
            while root:
                stack.append(root)
```

```

        root = root.left
    root = stack.pop()
    if pred and root.val < pred.val:
        y = root
        if not x:
            x = pred
        pred = root
    root = root.right

```

```
def swap(x: TreeNode | None, y: TreeNode | None) -> None:
```

```

    temp = x.val
    x.val = y.val
    y.val = temp

```

```
swap(x, y)
```

• LeetCode

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def recoverTree(self, root: Optional[TreeNode]) -> None:
        """
        Do not return anything, modify root in-place instead.
        """

```

```

    def dfs(root):
        if root is None:
            return
        nonlocal prev, first, second
        dfs(root.left)
        if prev and prev.val > root.val:
            if first is None:
                first = prev

```

Round 5 · Mid · Medium

p.1933

```

        second = root
        prev = root
        dfs(root.right)

```

```

prev = first = second = None
dfs(root)
first.val, second.val = second.val, first.val

```

• SimplyLeet

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

```

```

def recoverTree(root):
    # Initialize variables to track the first and second swapped nodes and the
    # previous node in in-order traversal
    first = second = prev = None

```

```

# Helper function to perform in-order traversal
def inorder(node):
    nonlocal first, second, prev
    if not node:
        return
    # Traverse the left subtree
    inorder(node.left)
    # Check for swapped nodes: if previous node's value is greater than the
    # current node's value, violation occurs
    if prev and prev.val > node.val:

```

Round 5 · Mid · Medium

p.1934

```

    if not first:
        # Record the first element when the first violation is
        encountered
        first = prev
        # Update the second element in every violation
        second = node
        # Update the previous node to the current node
        prev = node
        # Traverse the right subtree
        inorder(node.right)

inorder(root)
# Swap the values of the two nodes to recover the BST
first.val, second.val = second.val, first.val

```

BST / Ordered Set

☐ #235 Lowest Common Ancestor of a Binary Search Tree

MED

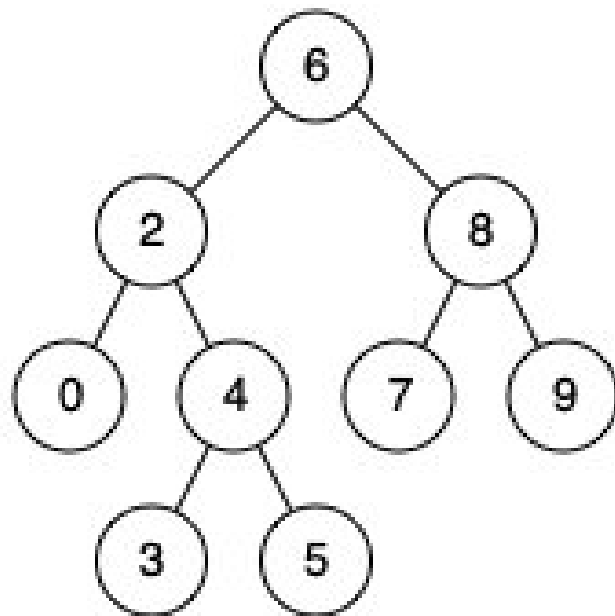
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · BST
Lists: Grind 169

Problem

Given a binary search tree (BST), find the lowest common ancestor (LCA) node of two given nodes in the BST.

According to the definition of LCA on Wikipedia: “The lowest common ancestor is defined between two nodes p and q as the lowest node in T that has both p and q as descendants (where we allow a node to be a descendant of itself).”

Example 1:

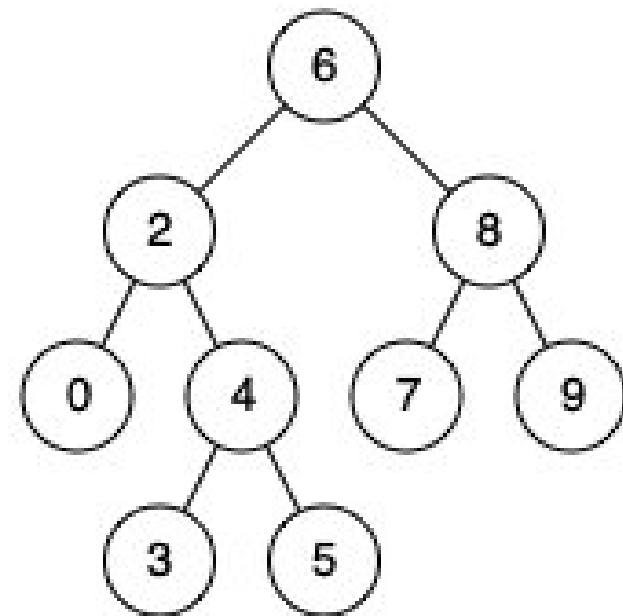


Input: root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 8
 Output: 6
 Explanation: The LCA of nodes 2 and 8 is 6.

Example 2:

Round 5 · Mid · Medium

p.1937



Input: root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 4
 Output: 2
 Explanation: The LCA of nodes 2 and 4 is 2, since a node can be a descendant of itself according to the LCA definition.

Example 3:

Input: root = [2,1], p = 2, q = 1
 Output: 2

Constraints:

- The number of nodes in the tree is in the range [2, 105].
- $-109 \leq \text{Node.val} \leq 109$

Round 5 · Mid · Medium

p.1938

- All Node.val are unique.
- $p \neq q$
- p and q will exist in the BST.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def lowestCommonAncestor(
        self,
        root: 'TreeNode',
        p: 'TreeNode',
        q: 'TreeNode',
    ) -> 'TreeNode':
        if root.val > max(p.val, q.val):
            return self.lowestCommonAncestor(root.left, p, q)
        if root.val < min(p.val, q.val):
            return self.lowestCommonAncestor(root.right, p, q)
        return root
```

• LeetDoocs

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None
```

```
class Solution:
    def lowestCommonAncestor(
```

```
self, root: 'TreeNode', p: 'TreeNode', q: 'TreeNode'
) -> 'TreeNode':
    while 1:
        if root.val < min(p.val, q.val):
            root = root.right
        elif root.val > max(p.val, q.val):
            root = root.left
        else:
            return root
```

• SimplyLeet

```
def lowestCommonAncestor(root, p, q):
    # Iterate until we find the lowest common ancestor
    while root:
        # If both p and q are less than the current node, move to left subtree
        if p.val < root.val and q.val < root.val:
            root = root.left
        # If both are greater, move to right subtree
        elif p.val > root.val and q.val > root.val:
            root = root.right
        else:
            # Found the split point; return current node as LCA
            return root
```

• LC.ca

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None

class Solution:
    def lowestCommonAncestor(
        self, root: 'TreeNode', p: 'TreeNode', q: 'TreeNode'
```

```

    ) -> 'TreeNode':
        while 1:
            if root.val < min(p.val, q.val): # no =, so root is p or q is in
else block
            root = root.right
            elif root.val > max(p.val, q.val):
            root = root.left
            else:
            return root

```

```
#####
```

```

class Solution:
    def lowestCommonAncestor(self, root, p, q):
        if root is None:
            return root

        if root.val > max(p.val, q.val):
            return self.lowestCommonAncestor(root.left, p, q)
        elif root.val < min(p.val, q.val):
            return self.lowestCommonAncestor(root.right, p, q)

```

```

        else:
            return root

```

```
#####
```

```

class Solution(object):
    def lowestCommonAncestor(self, root, p, q):
        """
        :type root: TreeNode
        :type p: TreeNode

```

```

:type q: TreeNode
:rtype: TreeNode
"""
a, b = sorted([p.val, q.val])
while not a <= root.val <= b:
    if a > root.val:
        root = root.right
    else:
        root = root.left
return root

```

◆ Quick Review

Key Insights

- Utilize the BST property: for any node, values in the left subtree are less and values in the right subtree are greater.
- If both p and q have values less than the current node, the LCA must lie in the left subtree.
- If both p and q have values greater than the current node, the LCA must lie in the right subtree.
- When one node value is less than and the other is greater than (or equal to) the current node, the current node is the lowest common ancestor.
- An iterative approach is efficient and uses constant extra space.

Space & Time

Time Complexity: $O(h)$, where h is the height of the tree ($O(\log n)$ for balanced trees and $O(n)$ in the worst-case skewed tree).

Space Complexity: $O(1)$ with an iterative approach ($O(h)$ if using recursion due to call stack).

Solution

The solution leverages the BST properties.

Algorithm:

- Start at the root node.
- While the current node exists: If both p and q have values less than the current node's value, move to the left child. If both p and q have values greater than the current node's value, move to the right child. Otherwise, the current node is the split point where p and q diverge, so it is the lowest common ancestor.
- Return the current node.

This approach efficiently finds the LCA using a single traversal from the root.

BST / Ordered Set

□ #285 Inorder Successor in BST

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode Tree · DFS · BST

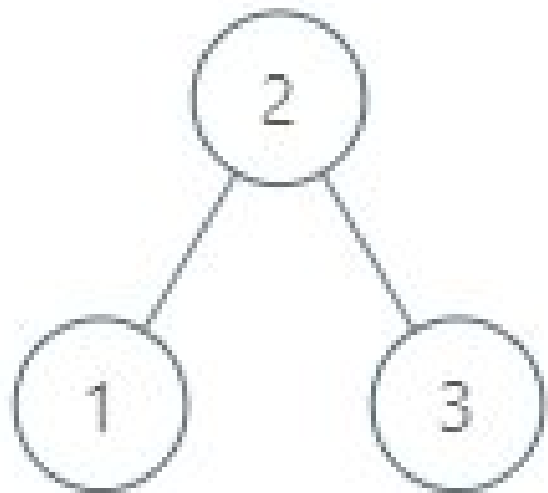
Lists: Grind 169

Problem

Given the root of a binary search tree and a node p in it, return the in-order successor of that node in the BST. If the given node has no in-order successor in the tree, return null.

The successor of a node p is the node with the smallest key greater than $p.val$.

Example 1:



Input: root = [2,1,3], p = 1

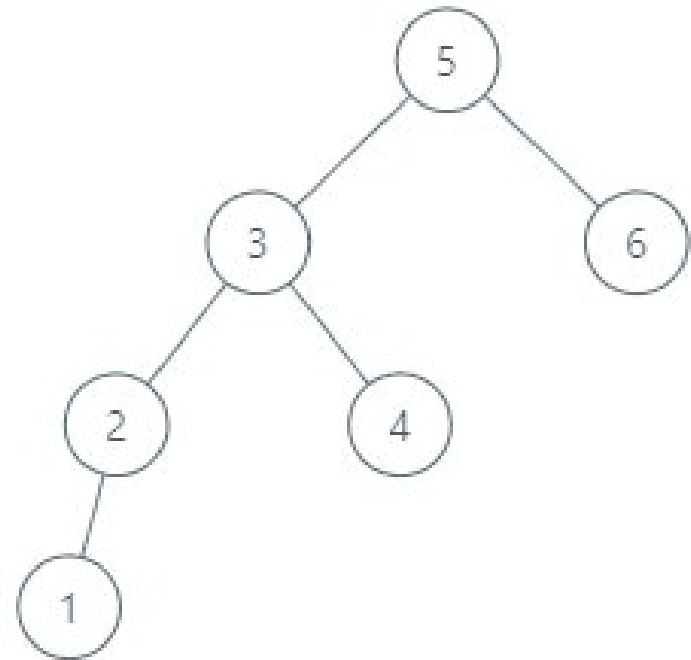
Output: 2

Explanation: 1's in-order successor node is 2. Note that both p and the return value is of TreeNode type.

Example 2:

Round 5 · Mid · Medium

p.1945



Input: root = [5,3,6,2,4,null,null,1], p = 6

Output: null

Explanation: There is no in-order successor of the current node, so the answer is null.

Constraints:

- The number of nodes in the tree is in the range [1, 104].
- $-105 \leq \text{Node.val} \leq 105$
- All Nodes will have unique values.

Community Solutions (Python)

Round 5 · Mid · Medium

p.1946

• WalkCC

```
class Solution:
    def inorderSuccessor(
        self,
        root: TreeNode | None,
        p: TreeNode | None,
    ) -> TreeNode | None:
        if not root:
            return None
        if root.val <= p.val:
            return self.inorderSuccessor(root.right, p)

        return self.inorderSuccessor(root.left, p) or root
```

• LeetCode

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None

class Solution:
    def inorderSuccessor(self, root: TreeNode, p: TreeNode) -> Optional[TreeNode]:
        ans = None
        while root:
            if root.val > p.val:
                ans = root
                root = root.left
            else:
                root = root.right
        return ans
```

• SimplyLeet

Round 5 · Mid · Medium

p.1947

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def inorderSuccessor(self, root: TreeNode, p: TreeNode) -> TreeNode:
        # If p has a right subtree, the successor is the left-most node in that
        # subtree.

        if p.right:
            successor = p.right
            while successor.left:
                successor = successor.left
            return successor

        # Otherwise, start from the root and keep track of a potential
        # successor.
        successor = None
        current = root
        while current:
            if p.val < current.val:
                # current is a candidate, so record it and move left to find a
                # smaller candidate
                successor = current
                current = current.left
            else:
                # If p.val is greater or equal, go right.
                current = current.right
        return successor
```

• LC.ca

Round 5 · Mid · Medium

p.1948

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None

class Solution:
    def inorderSuccessor(self, root: TreeNode, p: TreeNode) -> Optional[TreeNode]:

        ans = None
        while root:
            if root.val > p.val:
                ans = root
                root = root.left
            else:
                root = root.right
        return ans
```

◆ Quick Review

Key Insights

- In a BST, an in-order traversal yields nodes in ascending order.
- If the node p has a right subtree, the in-order successor is the left-most node in that right subtree.
- If p has no right subtree, the successor is one of its ancestors; traverse from the root towards p while keeping track of the last node that is greater than p .

Round 5 · Mid · Medium

p.1949

- An iterative approach is efficient and avoids recursion stack overhead.

Space & Time

Time Complexity: $O(h)$, where h is the height of the tree. In the worst-case (skewed tree), it can be $O(n)$.

Space Complexity: $O(1)$ as only a constant amount of extra space is used.

Solution

We solve the problem using two cases:

- If node p has a right subtree, we simply find the left-most node in that right subtree.
- If node p does not have a right subtree, initiate a search from the BST root. As we traverse: If the current node's value is greater than $p.val$, the current node is a candidate for the successor; move to the left subtree to look for a smaller candidate. Otherwise, move to the right subtree. By the end of the traversal, the candidate (if exists) is the in-order successor. This approach benefits from the BST property, limiting traversal to $O(h)$ time.

Round 5 · Mid · Medium

p.1950

BST / Ordered Set

★ #333 Largest BST Subtree ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Dynamic Programming · Tree · DFS · BST
 Lists: ★ Premium | Premium 98

Problem

Given the root of a binary tree, find the largest subtree, which is also a Binary Search Tree (BST), where the largest means subtree has the largest number of nodes.

A Binary Search Tree (BST) is a tree in which all the nodes follow the below-mentioned properties:

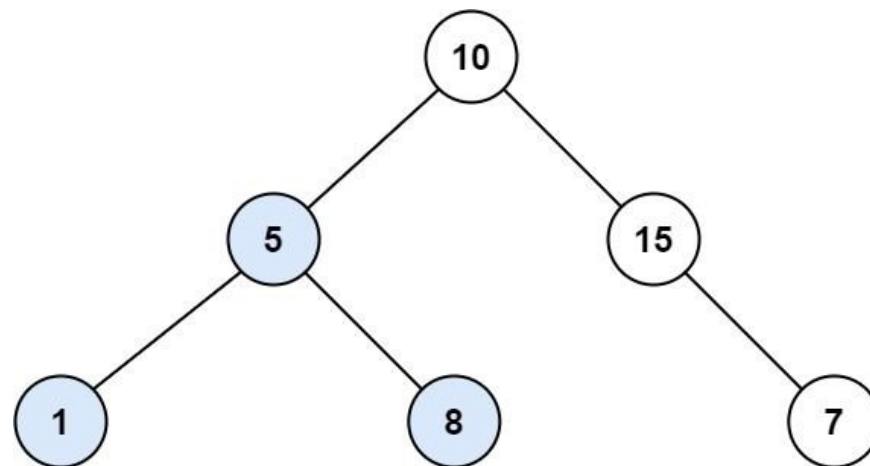
- The left subtree values are less than the value of their parent (root) node's value.
- The right subtree values are greater than the value of their parent (root) node's value.

Note: A subtree must include all of its descendants.

Example 1:

Round 5 · Mid · Medium

p.1951



Input: root = [10,5,15,1,8,null,7]

Output: 3

Explanation: The Largest BST Subtree in this case is the highlighted one. The return value is the subtree's size, which is 3.

Example 2:

Input: root = [4,2,7,2,3,5,null,2,null,null,null,null,null,1]

Output: 2

Constraints:

- The number of nodes in the tree is in the range [0, 104].
- $-104 \leq \text{Node.val} \leq 104$

Follow up: Can you figure out ways to solve it with $O(n)$ time complexity?

Community Solutions (Python)

- WalkCC

Round 5 · Mid · Medium

p.1952

```
from dataclasses import dataclass
```

```
@dataclass(frozen=True)
```

```
class T:
    mn: int # the minimum value in the subtree
    mx: int # the maximum value in the subtree
    size: int # the size of the subtree
```

```
class Solution:
```

```
    def largestBSTSubtree(self, root: TreeNode | None) -> int:
```

```
        def dfs(root: TreeNode | None) -> T:
```

```
            if not root:
                return T(math.inf, -math.inf, 0)
```

```
            l = dfs(root.left)
            r = dfs(root.right)
```

```
            if l.mx < root.val < r.mn:
```

```
                return T(min(l.mn, root.val), max(r.mx, root.val), 1 + l.size + r.size
            )
```

```
            # Mark one as invalid, but still record the size of children.
            # Return (-inf, inf) because no node will be > inf or < -inf.
            return T(-math.inf, math.inf, max(l.size, r.size))
```

```
        return dfs(root).size
```

• LeetDoocs

```
# Definition for a binary tree node.
```

```
# class TreeNode:
```

```
# def __init__(self, val=0, left=None, right=None):
```

```
# self.val = val
```

```
# self.left = left
```

```
# self.right = right
```

```
class Solution:
```

```
    def largestBSTSubtree(self, root: Optional[TreeNode]) -> int:
```

```
        def dfs(root):
```

```
            if root is None:
```

```
        return inf, -inf, 0
```

```
        lmi, lmx, ln = dfs(root.left)
```

```
        rmi, rmx, rn = dfs(root.right)
```

```
        nonlocal ans
```

```
        if lmx < root.val < rmi:
```

```
            ans = max(ans, ln + rn + 1)
```

```
            return min(lmi, root.val), max(rmx, root.val), ln + rn + 1
```

```
        return -inf, inf, 0
```

```
        ans = 0
```

```
        dfs(root)
```

```
        return ans
```

• SimplyLeet

```
# Definition for a binary tree node.
```

```
# class TreeNode:
```

```
# def __init__(self, val=0, left=None, right=None):
```

```
# self.val = val
```

```
# self.left = left
```

```
# self.right = right
```

```
class Solution:
```

```
    def largestBSTSubtree(self, root: TreeNode) -> int:
```

```
        # Global variable to store the size of the largest BST subtree found.
```

```
        self.max_bst_size = 0
```

```
        def helper(node):
```

```
            # Returns a tuple of: (isBST, size, min_val, max_val)
```

```
            if not node:
```

```
                # An empty node is considered a BST of size 0 with extreme min
                and max values.
```

```
                return (True, 0, float('inf'), float('-inf'))
```

```
                left_is_bst, left_size, left_min, left_max = helper(node.left)
```

```
                right_is_bst, right_size, right_min, right_max = helper(node.right)
```

```
            )
```

```

        # Check if the current node forms a BST with its left and right
        subtrees.
        if left_is_bst and right_is_bst and node.val > left_max and node.val < right_min:
            current_size = left_size + right_size + 1
            self.max_bst_size = max(self.max_bst_size, current_size)
            current_min = min(left_min, node.val)
            current_max = max(right_max, node.val)
            return (True, current_size, current_min, current_max)
        else:
            # Not a BST: Return false and the size of the largest BST found
            # so far in its subtrees.

            return (False, max(left_size, right_size), 0, 0)

    helper(root)
    return self.max_bst_size

```

• LC.ca

```

'''
>>> float("inf")
inf
>>> float("inf") + 1
inf
>>> import math
>>> float("inf") == math.inf
True
'''
# Definition for a binary tree node.

# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def largestBSTSubtree(self, root: Optional[TreeNode]) -> int:
        def dfs(root):
            if root is None:
                return inf, -inf, 0

```

Round 5 · Mid · Medium

p.1955

```

        lmi, lmx, ln = dfs(root.left)
        rmi, rmx, rn = dfs(root.right)
        nonlocal ans
        # this if can check if left/right is a bst
        # eg. if left is not bst, the returned lmi=-inf and lmx=inf, so
        # 'lmx < root.val < rmi' is false
        if lmx < root.val < rmi:
            ans = max(ans, ln + rn + 1)
            # lmi is guranteed < root.val, except when left(right) is None
            return min(lmi, root.val), max(rmx, root.val), ln + rn + 1
        return -inf, inf, 0 # if this one not bst, then all parents will
        not be bst

    ans = 0
    dfs(root)
    return ans

#####

# Definition for a binary tree node.
# class TreeNode(object):
#     def __init__(self, x):

# self.val = x
# self.left = None
# self.right = None

class Solution(object):
    def largestBSTSubtree(self, root):
        """
        :type root: TreeNode
        :rtype: int
        """

```

Round 5 · Mid · Medium

p.1956

```
def helper(root):
    if not root:
        return (0, 0, float("inf"), float("-inf")) # numBST, maxNumBST, min,
max
    lnumBST, lmaxNumBST, lmin, lmax = helper(root.left)
    rnumBST, rmaxNumBST, rmin, rmax = helper(root.right)
    numBST = -1
    if lmax < root.val < rmin and lnumBST != -1 and rnumBST != -1:
        numBST = 1 + lnumBST + rnumBST
    maxNumBST = max(1, lmaxNumBST, rmaxNumBST, numBST)

    return numBST, maxNumBST, min(lmin, rmin, root.val), max(lmax, rmax, root.val)

return helper(root)[1]
```

◆ Quick Review

Key Insights

- Use a bottom-up DFS (post-order traversal) to process each node.
- For each node, determine whether its left and right subtrees are BSTs.
- Maintain additional data per subtree: size (number of nodes), minimum value, and maximum value.
- A subtree rooted at the current node is a BST if:
 - Its left subtree is a BST.
 - Its right subtree is a BST.

Round 5 · Mid · Medium

p.1957

- The maximum value in the left subtree is less than the current node's value.
- The current node's value is less than the minimum value in the right subtree.
- Track the size of the largest BST identified during the traversal.

Space & Time

Time Complexity: $O(n)$ (each node is visited once)

Space Complexity: $O(h)$, where h is the height of the tree (due to recursion stack)

Solution

Perform a post-order traversal of the binary tree. For each node, return a tuple (or object in some languages) that provides:

- Whether the subtree rooted at this node is a BST.
- The size of the subtree if it is a BST (or the size of the largest BST found within it).
- The minimum value in the subtree.
- The maximum value in the subtree.

At each node, if both left and right children form valid BSTs and the current node's value fits the BST condition (greater than left's max and less than right's min), then the subtree

Round 5 · Mid · Medium

p.1958

rooted at the node is also a BST. Calculate its size and update the current global maximum if necessary. If the condition fails, return the maximum BST size found in its subtrees.

BST / Ordered Set

☐ #426 Convert Binary Search Tree to Sorted Doubly Linked List

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · Stack · Tree · Depth-First Search
· Binary Search Tree · Binary Tree ·
Doubly-Linked List

Lists: AlgoMaster 600

Problem

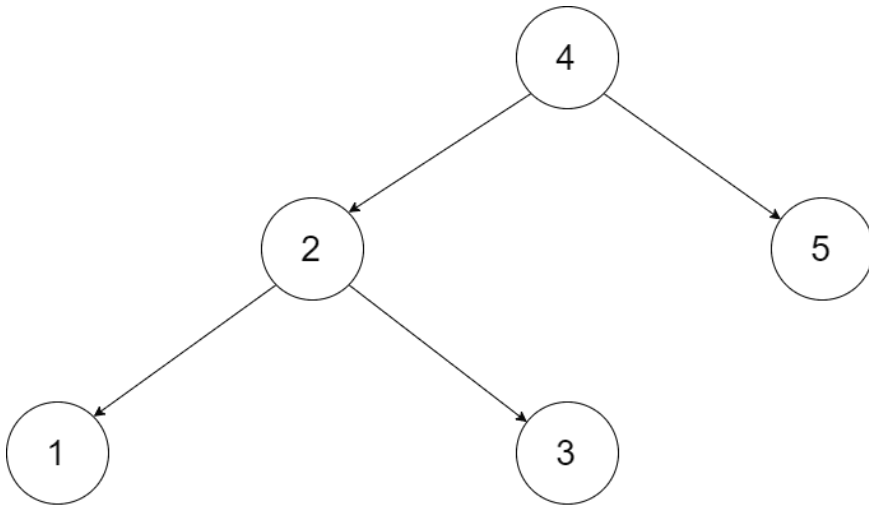
Convert a Binary Search Tree to a sorted Circular Doubly-Linked List in place.

You can think of the left and right pointers as synonymous to the predecessor and successor pointers in a doubly-linked list. For a circular doubly linked list, the predecessor of the first element is the last element, and the successor of the last element is the first element.

We want to do the transformation in place. After the transformation, the left pointer of the tree node should point to its predecessor, and the right pointer should point to its successor. You should return the pointer to the smallest

element of the linked list.

Example 1:



Input: root = [4,2,5,1,3]

Output: [1,2,3,4,5]

Explanation: The figure below shows the transformed BST. The solid line indicates the successor relationship, while the dashed line means the predecessor relationship.

Example 2:

Input: root = [2,1,3]

Output: [1,2,3]

Constraints:

- The number of nodes in the tree is in the range [0, 2000].
- $-1000 \leq \text{Node.val} \leq 1000$

Round 5 · Mid · Medium

p.1961

- All the values of the tree are unique.

Community Solutions (Python)

• WalkCC

```

class Solution:
    def treeToDoublyList(self, root: 'Node | None') -> 'Node | None':
        if not root:
            return None
        leftHead = self.treeToDoublyList(root.left)
        rightHead = self.treeToDoublyList(root.right)
        root.left = leftHead
        root.right = rightHead
        return self._connect(self._connect(leftHead, root), rightHead)

    def _connect(self, node1: 'Node | None', node2: 'Node | None') -> 'Node | None':
        if not node1:
            return node2
        if not node2:
            return node1

        tail1 = node1.left
        tail2 = node2.left

        # Connect node1's tail with node2.

        tail1.right = node2
        node2.left = tail1

        # Connect node2's tail with node1.
        tail2.right = node1
        node1.left = tail2
        return node1
  
```

• LeetCode

Round 5 · Mid · Medium

p.1962

```

"""
# Definition for a Node.
class Node:
    def __init__(self, val, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
"""

class Solution:
    def treeToDoublyList(self, root: 'Optional[Node]') -> 'Optional[Node]':
        def dfs(root):
            if root is None:
                return
            nonlocal prev, head
            dfs(root.left)
            if prev:
                prev.right = root
                root.left = prev
            else:
                head = root
            prev = root
            dfs(root.right)

        if root is None:
            return None
        head = prev = None
        dfs(root)
        prev.right = head

        head.left = prev
        return head

```

• SimplyLeet

Round 5 · Mid · Medium

p.1963

```

# Definition for a Node.
# class Node:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left # Pointer to predecessor in DLL
#         self.right = right # Pointer to successor in DLL

def treeToDoublyList(root):
    # If the BST is empty, return None
    if not root:

        return None

    # Initialize the pointers for the previous node and head node of the doubly
    linked list
    prev = None
    head = None

    # Helper function to perform in-order traversal
    def inorder(node):
        nonlocal prev, head
        if not node:

            return

        # Traverse left subtree
        inorder(node.left)

        # Process current node: if prev is None, it means this is the smallest
        element
        if prev is None:
            head = node
        else:
            # Connect the previous node (predecessor) with the current node
            (successor)

```

Round 5 · Mid · Medium

p.1964

```

    prev.right = node
    node.left = prev

    # Update prev to current node for the next iteration
    prev = node

    # Traverse right subtree
    inorder(node.right)

# Start in-order traversal starting from the root

inorder(root)

# After traversal, prev points to the last node. Connect head and tail to
# make DLL circular.
head.left = prev
prev.right = head

# Return the head node of the circular doubly linked list.
return head

```

Round 5 · Mid · Medium

p.1965

BST / Ordered Set

□ #450 Delete Node in a BST

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Tree · Binary Search Tree · Binary Tree
 Lists: AlgoMaster 600

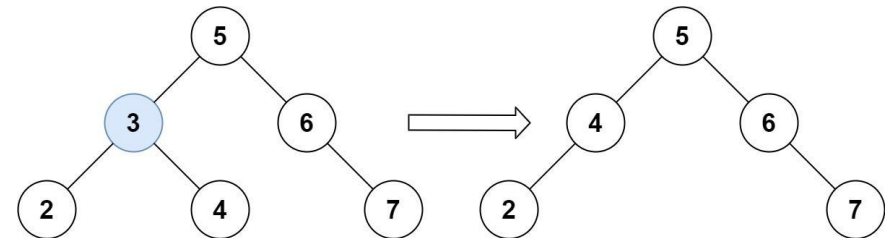
Problem

Given a root node reference of a BST and a key, delete the node with the given key in the BST. Return the root node reference (possibly updated) of the BST.

Basically, the deletion can be divided into two stages:

1. Search for a node to remove.
2. If the node is found, delete the node.

Example 1:



Round 5 · Mid · Medium

p.1966

Input: root = [5,3,6,2,4,null,7], key = 3
 Output: [5,4,6,2,null,null,7]
 Explanation: Given key to delete is 3. So we find the node with value 3 and delete it.
 One valid answer is [5,4,6,2,null,null,7], shown in the above BST.
 Please notice that another valid answer is [5,2,6,null,4,null,7] and it's also accepted.

Example 2:

Input: root = [5,3,6,2,4,null,7], key = 0
 Output: [5,3,6,2,4,null,7]
 Explanation: The tree does not contain a node with value = 0.

Example 3:

Input: root = [], key = 0
 Output: []

Constraints:

- The number of nodes in the tree is in the range [0, 104].
- $-105 \leq \text{Node.val} \leq 105$
- Each node has a unique value.
- root is a valid binary search tree.
- $-105 \leq \text{key} \leq 105$

Follow up: Could you solve it with time complexity $O(\text{height of tree})$?

Community Solutions (Python)

- WalkCC

```
class Solution:
    def deleteNode(self, root: TreeNode | None, key: int) -> TreeNode | None:
        if not root:
            return None
        if root.val == key:
            if not root.left:
                return root.right
            if not root.right:
                return root.left
            minNode = self._getMin(root.right)

            root.right = self.deleteNode(root.right, minNode.val)
            minNode.left = root.left
            minNode.right = root.right
            root = minNode
        elif root.val < key:
            root.right = self.deleteNode(root.right, key)
        else: # root.val > key
            root.left = self.deleteNode(root.left, key)
        return root

    def _getMin(self, node: TreeNode | None) -> TreeNode | None:
        while node.left:
            node = node.left
        return node
```

• LeetCode

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def deleteNode(self, root: Optional[TreeNode], key: int) -> Optional[TreeNode]:
        if root is None:
            return None
```

```

if root.val > key:
    root.left = self.deleteNode(root.left, key)
    return root
if root.val < key:
    root.right = self.deleteNode(root.right, key)
    return root
if root.left is None:
    return root.right
if root.right is None:
    return root.left

```

```

node = root.right
while node.left:
    node = node.left
node.left = root.left
root = root.right
return root

```

• SimplyLeet

```

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val          # Node's value
        self.left = left        # Pointer to left child
        self.right = right       # Pointer to right child

class Solution:
    def deleteNode(self, root: TreeNode, key: int) -> TreeNode:
        if not root:
            # If the tree is empty, return None.
            return None
        if key < root.val:
            # If the key is less, go to the left subtree.
            root.left = self.deleteNode(root.left, key)
        elif key > root.val:
            # If the key is greater, go to the right subtree.
            root.right = self.deleteNode(root.right, key)
        else:
            # Key found; now perform deletion.

```

```

if not root.left:
    # Node with only right child or no child.
    return root.right
elif not root.right:
    # Node with only left child.
    return root.left
else:
    # Node has two children.
    # Find the inorder successor (smallest in the right subtree).
    temp = root.right

```

```

while temp.left:
    temp = temp.left
# Replace the value of the node to be deleted.
root.val = temp.val
# Delete the inorder successor.
root.right = self.deleteNode(root.right, temp.val)
return root

```

BST / Ordered Set

□ #510 Inorder Successor in BST II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · Binary Search Tree · Binary Tree
Lists: AlgoMaster 600

Problem

Given a node in a binary search tree, return the in-order successor of that node in the BST. If that node has no in-order successor, return null.

The successor of a node is the node with the smallest key greater than node.val.

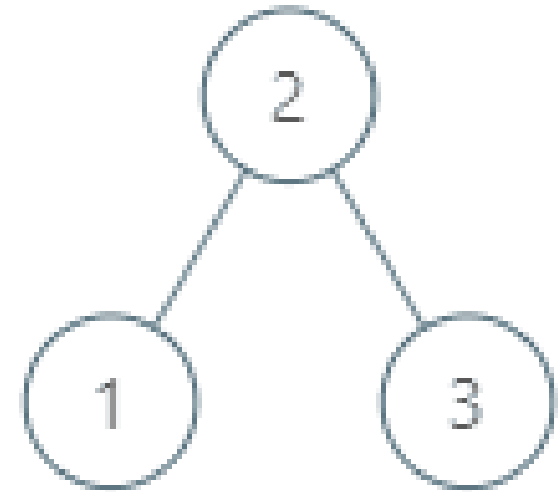
You will have direct access to the node but not to the root of the tree. Each node will have a reference to its parent node. Below is the definition for Node:

```
class Node {
public int val;
public Node left;
public Node right;
public Node parent;
}
```

Example 1:

Round 5 · Mid · Medium

p.1971



Input: tree = [2,1,3], node = 1

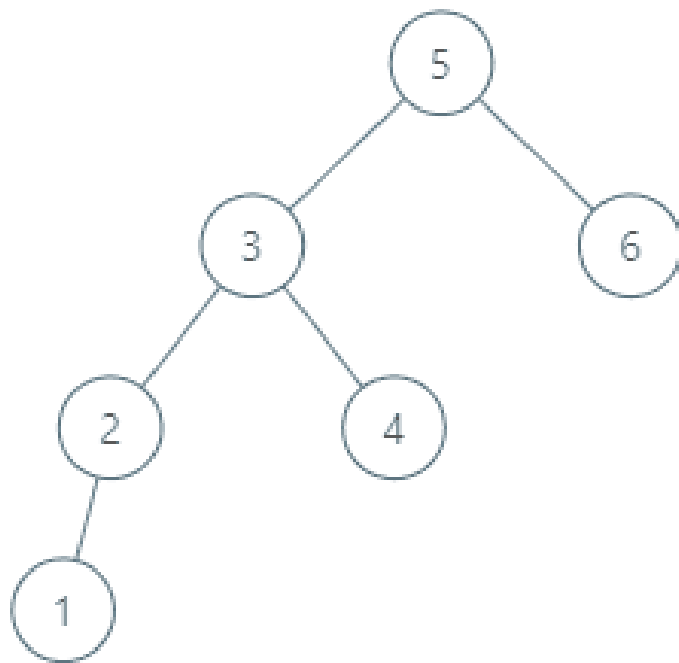
Output: 2

Explanation: 1's in-order successor node is 2. Note that both the node and the return value is of Node type.

Example 2:

Round 5 · Mid · Medium

p.1972



Input: tree = [5,3,6,2,4,null,null,1], node = 6

Output: null

Explanation: There is no in-order successor of the current node, so the answer is null.

Constraints:

- The number of nodes in the tree is in the range [1, 104].
- $-105 \leq \text{Node.val} \leq 105$
- All Nodes will have unique values.

Follow up: Could you solve it without looking up any of the node's values?

Round 5 · Mid · Medium

p.1973

Community Solutions (Python)

• LeetCode

```

#####
# Definition for a Node.
class Node:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None
        self.parent = None
#####

class Solution:
    def inorderSuccessor(self, node: 'Node') -> 'Optional[Node]':
        if node.right:
            node = node.right
            while node.left:
                node = node.left
            return node
        while node.parent and node.parent.right is node:
            node = node.parent
        return node.parent

```

• SimplyLeet

```

# Python solution with line-by-line comments

class Node:
    def __init__(self, val):
        self.val = val          # Node value
        self.left = None       # Left child node
        self.right = None      # Right child node
        self.parent = None     # Parent node

    def inorderSuccessor(self, node):

```

Round 5 · Mid · Medium

p.1974

```

# If there is a right subtree, find its leftmost node.
if node.right:
    successor = node.right
    # Loop to find the leftmost child
    while successor.left:
        successor = successor.left
    return successor
# If no right subtree, traverse up using parent pointers.
parent = node.parent
while parent and node == parent.right:

    node = parent
    parent = parent.parent
return parent # This will be the in-order successor or None if not found

```

BST / Ordered Set

#669 Trim a Binary Search Tree

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · Depth-First Search · Binary Search Tree · Binary Tree

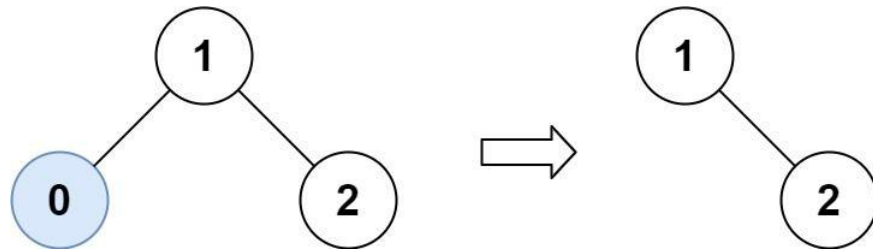
Lists: AlgoMaster 600

Problem

Given the root of a binary search tree and the lowest and highest boundaries as low and high, trim the tree so that all its elements lies in [low, high]. Trimming the tree should not change the relative structure of the elements that will remain in the tree (i.e., any node's descendant should remain a descendant). It can be proven that there is a unique answer.

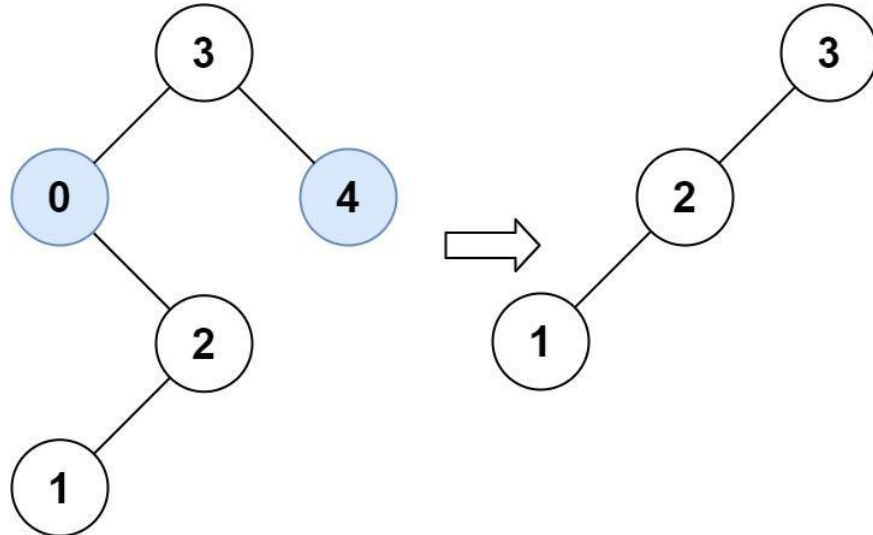
Return the root of the trimmed binary search tree. Note that the root may change depending on the given bounds.

Example 1:



Input: root = [1,0,2], low = 1, high = 2
 Output: [1,null,2]

Example 2:



Input: root = [3,0,4,null,2,null,null,1], low = 1, high = 3
 Output: [3,2,null,1]

Constraints:

- The number of nodes in the tree is in the range [1, 104].

Round 5 · Mid · Medium

p.1977

- $0 \leq \text{Node.val} \leq 104$
- The value of each node in the tree is unique.
- root is guaranteed to be a valid binary search tree.
- $0 \leq \text{low} \leq \text{high} \leq 104$

Community Solutions (Python)

• LeetCode

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def trimBST(
        self, root: Optional[TreeNode], low: int, high: int
    ) -> Optional[TreeNode]:

        def dfs(root):
            if root is None:
                return root
            if root.val > high:
                return dfs(root.left)
            if root.val < low:
                return dfs(root.right)
            root.left = dfs(root.left)
            root.right = dfs(root.right)
            return root

        return dfs(root)
  
```

• SimplyLeet

Round 5 · Mid · Medium

p.1978

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val      # Node's value
        self.left = left    # Left child pointer
        self.right = right  # Right child pointer

class Solution:
    def trimBST(self, root: TreeNode, low: int, high: int) -> TreeNode:
        # Base case: if node is None, return None.

        if not root:
            return None

        # If node's value is less than low, trim right subtree.
        if root.val < low:
            return self.trimBST(root.right, low, high)

        # If node's value is greater than high, trim left subtree.
        if root.val > high:
            return self.trimBST(root.left, low, high)

        # Otherwise, recursively trim left and right subtrees.
        root.left = self.trimBST(root.left, low, high)
        root.right = self.trimBST(root.right, low, high)
        return root
```

Round 5 · Mid · Medium

p.1979

BST / Ordered Set

□ #701 Insert into a Binary Search Tree

MED

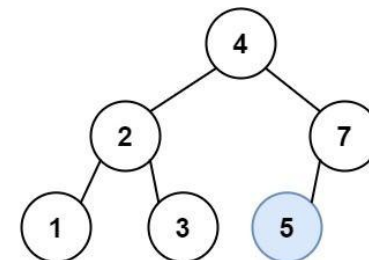
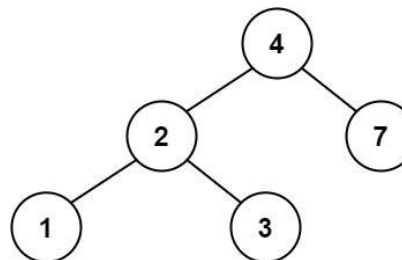
LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Tree · Binary Search Tree · Binary Tree
 Lists: AlgoMaster 600

Problem

You are given the root node of a binary search tree (BST) and a value to insert into the tree. Return the root node of the BST after the insertion. It is guaranteed that the new value does not exist in the original BST.

Notice that there may exist multiple valid ways for the insertion, as long as the tree remains a BST after insertion. You can return any of them.

Example 1:



Round 5 · Mid · Medium

p.1980

Input: root = [4,2,7,1,3], val = 5
 Output: [4,2,7,1,3,5]
 Explanation: Another accepted tree is:

Example 2:

Input: root = [40,20,60,10,30,50,70], val = 25
 Output: [40,20,60,10,30,50,70,null,null,25]

Example 3:

Input: root = [4,2,7,1,3,null,null,null,null,null,null], val = 5
 Output: [4,2,7,1,3,5]

Constraints:

- The number of nodes in the tree will be in the range [0, 104].
- $-108 \leq \text{Node.val} \leq 108$
- All the values Node.val are unique.
- $-108 \leq \text{val} \leq 108$
- It's guaranteed that val does not exist in the original BST.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def insertIntoBST(self, root: TreeNode | None, val: int) -> TreeNode | None:
        if not root:
            return TreeNode(val)
        if root.val > val:
            root.left = self.insertIntoBST(root.left, val)
        else:
            root.right = self.insertIntoBST(root.right, val)
        return root
```

• LeetDoocs

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def insertIntoBST(self, root: Optional[TreeNode], val: int) -> Optional[TreeNode]:
        if root is None:
            return TreeNode(val)

        if root.val > val:
            root.left = self.insertIntoBST(root.left, val)
        else:
            root.right = self.insertIntoBST(root.right, val)
        return root
```

• SimplyLeet

```
# Define the structure for a tree node
class TreeNode:
    def __init__(self, x):
        self.val = x
        self.left = None
        self.right = None

def insertIntoBST(root, val):
    # If reaching a null node, insert the new node here.
    if root is None:
        return TreeNode(val)
    # If the value to insert is less than the current node's value,
    # move to the left subtree.
    if val < root.val:
        root.left = insertIntoBST(root.left, val)
    # Otherwise, move to the right subtree.
    else:
        root.right = insertIntoBST(root.right, val)
    # Return the unchanged root pointer.
    return root
```

```
# Example usage:
# root = TreeNode(4)
# root.left = TreeNode(2)
# root.right = TreeNode(7)
# root.left.left = TreeNode(1)
# root.left.right = TreeNode(3)
# new_root = insertIntoBST(root, 5)
```

BST / Ordered Set

□ #729 My Calendar I

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Binary Search · Design · Segment Tree
 · Ordered Set

Lists: AlgoMaster 600

Problem

You are implementing a program to use as your calendar. We can add a new event if adding the event will not cause a double booking.

A double booking happens when two events have some non-empty intersection (i.e., some moment is common to both events.).

The event can be represented as a pair of integers `startTime` and `endTime` that represents a booking on the half-open interval `[startTime, endTime)`, the range of real numbers `x` such that `startTime ≤ x < endTime`.

Implement the `MyCalendar` class:

- `MyCalendar()` Initializes the calendar object.

- **boolean book(int startTime, int endTime)**
Returns true if the event can be added to the calendar successfully without causing a double booking. Otherwise, return false and do not add the event to the calendar.

Example 1:

Input
["MyCalendar", "book", "book", "book"]
[[], [10, 20], [15, 25], [20, 30]]
Output
[null, true, false, true]

Explanation
MyCalendar myCalendar = new MyCalendar();

Constraints:

- $0 \leq \text{start} < \text{end} \leq 109$
- At most 1000 calls will be made to book.

Community Solutions (Python)

• WalkCC

```
class MyCalendar:
    def __init__(self):
        self.line = []

    def book(self, startTime: int, endTime: int) -> bool:
        for s, e in self.line:
            if max(startTime, s) < min(endTime, e):
                return False
        self.line.append((startTime, endTime))
        return True
```

```
from dataclasses import dataclass
```

```
@dataclass
class Node:
    start: int
    end: int
    left = None
    right = None
```

```
class Tree:
    def __init__(self):
        self.root = None

    def insert(self, node: Node, root: Node = None) -> bool:
        if not root:
            if not self.root:
                self.root = node
                return True
```

```
        else:
            root = self.root
```

```
        if node.start >= root.end:
            if not root.right:
                root.right = node
                return True
            return self.insert(node, root.right)
        elif node.end <= root.start:
            if not root.left:
```

```
                root.left = node
                return True
            return self.insert(node, root.left)
        else:
            return False
```

```
class MyCalendar:
    def __init__(self):
        self.tree = Tree()
```

```
def book(self, start: int, end: int) -> bool:
    return self.tree.insert(Node(start, end))
```

• LeetCode

```
class MyCalendar:

    def __init__(self):
        self.sd = SortedDict()

    def book(self, start: int, end: int) -> bool:
        idx = self.sd.bisect_right(start)
        if idx < len(self.sd) and self.sd.values()[idx] < end:
            return False
        self.sd[end] = start

        return True
```

```
# Your MyCalendar object will be instantiated and called as such:
# obj = MyCalendar()
# param_1 = obj.book(start,end)
```

• SimplyLeet

```
# Python implementation for My Calendar I

class MyCalendar:
    def __init__(self):
        # List to store booked intervals as (start, end) tuples
        self.bookings = []

    def book(self, start: int, end: int) -> bool:
        # Check each existing booking for an overlap
        for s, e in self.bookings:
```

```
# If new event overlaps with existing event, return False
if not (end <= s or start >= e):
    return False

# Could not find overlap, so add the new event
self.bookings.append((start, end))
return True
```

```
# Example usage:
# myCalendar = MyCalendar()
# print(myCalendar.book(10, 20)) # True

# print(myCalendar.book(15, 25)) # False
# print(myCalendar.book(20, 30)) # True
```

BST / Ordered Set

□ #731 My Calendar II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Binary Search · Design · Segment Tree
 · Prefix Sum · Ordered Set

Lists: AlgoMaster 600

Problem

You are implementing a program to use as your calendar. We can add a new event if adding the event will not cause a triple booking.

A triple booking happens when three events have some non-empty intersection (i.e., some moment is common to all the three events.).

The event can be represented as a pair of integers `startTime` and `endTime` that represents a booking on the half-open interval `[startTime, endTime)`, the range of real numbers `x` such that `startTime ≤ x < endTime`.

Implement the `MyCalendarTwo` class:

- `MyCalendarTwo()` Initializes the calendar object.

Round 5 · Mid · Medium

p.1989

- `boolean book(int startTime, int endTime)`
 Returns true if the event can be added to the calendar successfully without causing a triple booking. Otherwise, return false and do not add the event to the calendar.

Example 1:

```
Input
["MyCalendarTwo", "book", "book", "book", "book", "book", "book"]
[[], [10, 20], [50, 60], [10, 40], [5, 15], [5, 10], [25, 55]]
Output
[null, true, true, true, false, true, true]
```

```
Explanation
MyCalendarTwo myCalendarTwo = new MyCalendarTwo();
```

Constraints:

- `0 ≤ start < end ≤ 109`
- At most 1000 calls will be made to `book`.

Community Solutions (Python)

• LeetDoocs

```
class MyCalendarTwo:

    def __init__(self):
        self.sd = SortedDict()

    def book(self, startTime: int, endTime: int) -> bool:
        self.sd[startTime] = self.sd.get(startTime, 0) + 1
        self.sd[endTime] = self.sd.get(endTime, 0) - 1
        s = 0
        for v in self.sd.values():
```

Round 5 · Mid · Medium

p.1990

```

        s += v
        if s > 2:
            self.sd[startTime] -= 1
            self.sd[endTime] += 1
            return False
        return True

```

```

# Your MyCalendarTwo object will be instantiated and called as such:
# obj = MyCalendarTwo()

```

```

# param_1 = obj.book(startTime, endTime)

```

```

class Node:
    def __init__(self, l: int, r: int):
        self.left = None
        self.right = None
        self.l = l
        self.r = r
        self.mid = (l + r) >> 1
        self.v = 0
        self.add = 0

```

```

class SegmentTree:
    def __init__(self):
        self.root = Node(1, 10**9 + 1)

    def modify(self, l: int, r: int, v: int, node: Node = None):
        if l > r:
            return
        if node is None:
            node = self.root

```

```

        if node.l >= l and node.r <= r:
            node.v += v
            node.add += v
            return
        self.pushdown(node)
        if l <= node.mid:
            self.modify(l, r, v, node.left)
        if r > node.mid:
            self.modify(l, r, v, node.right)
        self.pushup(node)

```

```

def query(self, l: int, r: int, node: Node = None) -> int:
    if l > r:
        return 0
    if node is None:
        node = self.root
    if node.l >= l and node.r <= r:
        return node.v
    self.pushdown(node)
    v = 0

```

```

        if l <= node.mid:
            v = max(v, self.query(l, r, node.left))
        if r > node.mid:
            v = max(v, self.query(l, r, node.right))
        return v

```

```

def pushup(self, node: Node):
    node.v = max(node.left.v, node.right.v)

```

```

def pushdown(self, node: Node):

```

```

        if node.left is None:
            node.left = Node(node.l, node.mid)
        if node.right is None:
            node.right = Node(node.mid + 1, node.r)
        if node.add:
            node.left.v += node.add
            node.right.v += node.add
            node.left.add += node.add
            node.right.add += node.add
            node.add = 0

```



```
class MyCalendarTwo:
    def __init__(self):
        self.tree = SegmentTree()

    def book(self, startTime: int, endTime: int) -> bool:
        if self.tree.query(startTime + 1, endTime) >= 2:
            return False
        self.tree.modify(startTime + 1, endTime, 1)

        return True
```

```
# Your MyCalendarTwo object will be instantiated and called as such:
# obj = MyCalendarTwo()
# param_1 = obj.book(startTime,endTime)
```

• SimplyLeet

```
# Python implementation of MyCalendarTwo
class MyCalendarTwo:
    def __init__(self):
        # List to store all the booked events
        self.booked = []
        # List to store all the double-booked intervals (overlaps)
        self.overlaps = []

    def book(self, start: int, end: int) -> bool:
        # Check if new event overlaps with any double-booked interval.
```

Round 5 · Mid · Medium

p.1993

```
        for os, oe in self.overlaps:
            # If there's any overlap with a double-booked interval, triple
            booking occurs.
            if start < oe and end > os:
                return False

        # For all previously booked events, check if they overlap with the new
        event.
        for bs, be in self.booked:
            if start < be and end > bs:
                # Calculate the overlapping segment and record it as a double
                booking.
                self.overlaps.append((max(start, bs), min(end, be)))

        # Add the new event to the booked list as it doesn't cause a triple
        booking.
        self.booked.append((start, end))
        return True

# Example usage:
# myCalendar = MyCalendarTwo()
# print(myCalendar.book(10, 20)) # True
# print(myCalendar.book(50, 60)) # True
# print(myCalendar.book(10, 40)) # True

# print(myCalendar.book(5, 15)) # False
```

Round 5 · Mid · Medium

p.1994

BST / Ordered Set

□ #1008 Construct Binary Search Tree from Preorder Traversal

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Stack · Tree · Binary Search Tree ·
 Monotonic Stack · Binary Tree

Lists: AlgoMaster 600

Problem

Given an array of integers preorder, which represents the preorder traversal of a BST (i.e., binary search tree), construct the tree and return its root.

It is guaranteed that there is always possible to find a binary search tree with the given requirements for the given test cases.

A binary search tree is a binary tree where for every node, any descendant of Node.left has a value strictly less than Node.val, and any descendant of Node.right has a value strictly greater than Node.val.

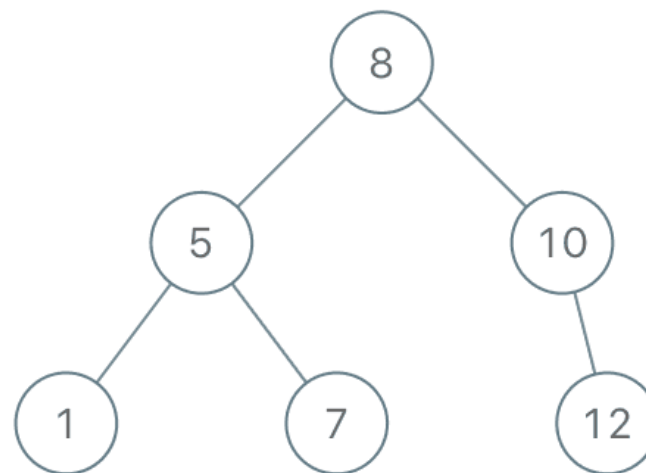
A preorder traversal of a binary tree displays the value of the node first, then traverses

Round 5 · Mid · Medium

p.1995

Node.left, then traverses Node.right.

Example 1:



Input: preorder = [8,5,1,7,10,12]
 Output: [8,5,10,1,7,null,12]

Example 2:

Input: preorder = [1,3]
 Output: [1,null,3]

Constraints:

- $1 \leq \text{preorder.length} \leq 100$
- $1 \leq \text{preorder}[i] \leq 1000$
- All the values of preorder are unique.

Round 5 · Mid · Medium

p.1996

Community Solutions (Python)

• WalkCC

```
class Solution:
    def bstFromPreorder(self, preorder: List[int]) -> TreeNode | None:
        root = TreeNode(preorder[0])
        stack = [root]

        for i in range(1, len(preorder)):
            parent = stack[-1]
            child = TreeNode(preorder[i])
            # Adjust the parent.
            while stack and stack[-1].val < child.val:
                parent = stack.pop()
            # Create parent-child link according to BST property.
            if parent.val > child.val:
                parent.left = child
            else:
                parent.right = child
            stack.append(child)

        return root
```

• LeetDoocs

```
class Solution:
    def bstFromPreorder(self, preorder: List[int]) -> Optional[TreeNode]:
        def dfs(i: int, j: int) -> Optional[TreeNode]:
            if i > j:
                return None
            root = TreeNode(preorder[i])
            l, r = i + 1, j + 1
            while l < r:
                mid = (l + r) >> 1
                if preorder[mid] > preorder[i]:
```

```
        r = mid
    else:
        l = mid + 1
    root.left = dfs(i + 1, l - 1)
    root.right = dfs(l, j)
    return root

    return dfs(0, len(preorder) - 1)
```

• SimplyLeet

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

# Recursive helper function that utilizes bounds to construct the BST.
def bstFromPreorder(preorder):
    # use a list to simulate a pointer that holds the current index reference

    index = [0]

    # Recursive function to build BST within the lower and upper bounds
    def buildBST(lower, upper):
        # If all elements are used, return None.
        if index[0] == len(preorder):
            return None
        # next value in the preorder sequence
        val = preorder[index[0]]
        # If the value does not satisfy BST constraints, return None.
```

```

if val < lower or val > upper:
    return None

# Accept the value and move to the next element
index[0] += 1
# Create a new tree node with the current value.
node = TreeNode(val)
# All values for the left subtree must be less than the current node's
value.
node.left = buildBST(lower, val)
# All values for the right subtree must be greater than the current
node's value.

node.right = buildBST(val, upper)
return node

return buildBST(float('-inf'), float('inf'))

# Example usage:
# preorder = [8,5,1,7,10,12]
# root = bstFromPreorder(preorder)

```

BST / Ordered Set

#2034 Stock Price Fluctuation

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Hash Table · Design · Heap (Priority Queue) ·
 Data Stream · Ordered Set
 Lists: AlgoMaster 600

Problem

You are given a stream of records about a particular stock. Each record contains a timestamp and the corresponding price of the stock at that timestamp.

Unfortunately due to the volatile nature of the stock market, the records do not come in order. Even worse, some records may be incorrect. Another record with the same timestamp may appear later in the stream correcting the price of the previous wrong record.

Design an algorithm that:

- Updates the price of the stock at a particular timestamp, correcting the price from any previous records at the timestamp.

- Finds the latest price of the stock based on the current records. The latest price is the price at the latest timestamp recorded.
- Finds the maximum price the stock has been based on the current records.
- Finds the minimum price the stock has been based on the current records.

Implement the StockPrice class:

- `StockPrice()` Initializes the object with no price records.
- `void update(int timestamp, int price)`
Updates the price of the stock at the given timestamp.
- `int current()` Returns the latest price of the stock.
- `int maximum()` Returns the maximum price of the stock.
- `int minimum()` Returns the minimum price of the stock.

Example 1:

```
Input
["StockPrice", "update", "update", "current", "maximum", "update", "maximum",
"update", "minimum"]
[[], [1, 10], [2, 5], [], [], [1, 3], [], [4, 2], []]
Output
[null, null, null, 5, 10, null, 5, null, 2]
```

```
Explanation
StockPrice stockPrice = new StockPrice();
```

Constraints:

- $1 \leq \text{timestamp}, \text{price} \leq 109$
- At most 105 calls will be made in total to `update`, `current`, `maximum`, and `minimum`.
- `current`, `maximum`, and `minimum` will be called only after `update` has been called at least once.

Community Solutions (Python)

• WalkCC

```
from sortedcontainers import SortedDict

class StockPrice:
    def __init__(self):
        self.timestampToPrice = SortedDict()
        self.pricesCount = SortedDict()

    def update(self, timestamp: int, price: int) -> None:
        if timestamp in self.timestampToPrice:
```

```

prevPrice = self.timestampToPrice[timestamp]
self.pricesCount[prevPrice] -= 1
if self.pricesCount[prevPrice] == 0:
    del self.pricesCount[prevPrice]
self.timestampToPrice[timestamp] = price
self.pricesCount[price] = self.pricesCount.get(price, 0) + 1

```

```

def current(self) -> int:
    return self.timestampToPrice.peekitem(-1)[1]

```

```

def maximum(self) -> int:
    return self.pricesCount.peekitem(-1)[0]

```

```

def minimum(self) -> int:
    return self.pricesCount.peekitem(0)[0]

```

• LeetCode

```

class StockPrice:
    def __init__(self):
        self.d = {}
        self.ls = SortedList()
        self.last = 0

    def update(self, timestamp: int, price: int) -> None:
        if timestamp in self.d:
            self.ls.remove(self.d[timestamp])
        self.d[timestamp] = price

```

```

        self.ls.add(price)
        self.last = max(self.last, timestamp)

```

```

    def current(self) -> int:
        return self.d[self.last]

```

```

    def maximum(self) -> int:
        return self.ls[-1]

```

```

    def minimum(self) -> int:

```

```

    return self.ls[0]

```

```

# Your StockPrice object will be instantiated and called as such:
# obj = StockPrice()
# obj.update(timestamp,price)
# param_2 = obj.current()
# param_3 = obj.maximum()
# param_4 = obj.minimum()

```

• SimplyLeet

```

# Python solution
import heapq

class StockPrice:
    def __init__(self):
        # Dictionary mapping timestamp to price
        self.price_by_timestamp = {}
        # Track the latest timestamp seen so far
        self.latest_timestamp = 0
        # Min heap to obtain the minimum price quickly; stores (price,
        timestamp)

```

```

        self.min_heap = []
        # Max heap to obtain the maximum price quickly; stores (-price,
        timestamp)
        self.max_heap = []

```

```

    def update(self, timestamp: int, price: int) -> None:
        # Update the dictionary with the new price for the given timestamp
        self.price_by_timestamp[timestamp] = price
        # Update the latest timestamp if necessary
        self.latest_timestamp = max(self.latest_timestamp, timestamp)
        # Push the new price with its timestamp into the min heap

```

```

        heapq.heappush(self.min_heap, (price, timestamp))
        # Push the negative of price to simulate a max heap into the max heap
        heapq.heappush(self.max_heap, (-price, timestamp))

    def current(self) -> int:
        # Return the price corresponding to the latest timestamp
        return self.price_by_timestamp[self.latest_timestamp]

    def maximum(self) -> int:
        # Remove outdated entries from the max heap

        while self.max_heap:
            # Get the price (as negative) and associated timestamp from the top
            # of the heap
            neg_price, timestamp = self.max_heap[0]
            # If the price in the dictionary doesn't match the current heap
            # entry, it's outdated
            if self.price_by_timestamp[timestamp] != -neg_price:
                heapq.heappop(self.max_heap)
            else:
                return -neg_price
        return -1 # Edge case (should not occur as current is called after an
        update)

    def minimum(self) -> int:
        # Remove outdated entries from the min heap
        while self.min_heap:
            price, timestamp = self.min_heap[0]
            # If the price in the dictionary doesn't match, pop it as it's
            # outdated
            if self.price_by_timestamp[timestamp] != price:
                heapq.heappop(self.min_heap)
            else:
                return price
        return -1 # Edge case (should not occur as current is called after an
        update)

```

Round 5 · Mid · Medium

p.2005

Tries

Round 5 · Mid · Medium · 9 questions

Round 5 · Mid · Medium

p.2006

Tries

#208 Implement Trie (Prefix Tree)

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Design · Trie
Lists: Grind 169

Problem

A trie (pronounced as "try") or prefix tree is a tree data structure used to efficiently store and retrieve keys in a dataset of strings. There are various applications of this data structure, such as autocomplete and spellchecker.

Implement the Trie class:

- **Trie()** Initializes the trie object.
- **void insert(String word)** Inserts the string word into the trie.
- **boolean search(String word)** Returns true if the string word is in the trie (i.e., was inserted before), and false otherwise.
- **boolean startsWith(String prefix)** Returns true if there is a previously inserted string word that has the prefix prefix, and false

Round 5 · Mid · Medium

p.2007

otherwise.

Example 1:

```
Input
["Trie", "insert", "search", "search", "startsWith", "insert", "search"]
[[], ["apple"], ["apple"], ["app"], ["app"], ["app"], ["app"]]
Output
[null, null, true, false, true, null, true]
```

Explanation
Trie trie = new Trie();

Constraints:

- $1 \leq \text{word.length}, \text{prefix.length} \leq 2000$
- word and prefix consist only of lowercase English letters.
- At most $3 * 10^4$ calls in total will be made to insert, search, and startsWith.

Community Solutions (Python)

• WalkCC

```
class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.isWord = False

class Trie:
    def __init__(self):
        self.root = TrieNode()
```

Round 5 · Mid · Medium

p.2008


```
def insert(self, word: str) -> None:
    node: TrieNode = self.root
    for c in word:
        node = node.children.setdefault(c, TrieNode())
        node.isWord = True

def search(self, word: str) -> bool:
    node = self._find(word)
    return node is not None and node.isWord
```

```
def startsWith(self, prefix: str) -> bool:
    return self._find(prefix) is not None

def _find(self, prefix: str) -> TrieNode | None:
    node: TrieNode = self.root
    for c in prefix:
        if c not in node.children:
            return None
        node = node.children[c]
    return node
```

• LeetCode

```
class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.is_end = False

    def insert(self, word: str) -> None:
        node = self
        for c in word:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
```

Round 5 · Mid · Medium

p.2009

```
        node.children[idx] = Trie()
        node = node.children[idx]
        node.is_end = True

    def search(self, word: str) -> bool:
        node = self._search_prefix(word)
        return node is not None and node.is_end

    def startsWith(self, prefix: str) -> bool:
        node = self._search_prefix(prefix)

        return node is not None

    def _search_prefix(self, prefix: str):
        node = self
        for c in prefix:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                return None
            node = node.children[idx]
        return node
```

```
# Your Trie object will be instantiated and called as such:
# obj = Trie()
# obj.insert(word)
# param_2 = obj.search(word)
# param_3 = obj.startsWith(prefix)
```

• SimplyLeet

```
class TrieNode:
    def __init__(self):
        # Dictionary to store child nodes and a flag for end of word.
        self.children = {}
        self.is_end_of_word = False

class Trie:
    def __init__(self):
        # Initialize the Trie with the root node.
        self.root = TrieNode()
```

Round 5 · Mid · Medium

p.2010

```
def insert(self, word: str) -> None:
    # Start from the root.
    node = self.root
    for char in word:
        # If character doesn't exist, add a new TrieNode.
        if char not in node.children:
            node.children[char] = TrieNode()
        node = node.children[char]
    # Mark the end of word.
```

```
node.is_end_of_word = True
```

```
def search(self, word: str) -> bool:
    # Traverse the Trie for each character.
    node = self.root
    for char in word:
        if char not in node.children:
            return False
        node = node.children[char]
    # Returns True if word ends exactly here.
```

```
return node.is_end_of_word
```

```
def startsWith(self, prefix: str) -> bool:
    # Traverse checking for the prefix.
    node = self.root
    for char in prefix:
        if char not in node.children:
            return False
        node = node.children[char]
    return True
```

• LC.ca

Round 5 · Mid · Medium

p.2011

```
import collections
```

```
class TrieNode:
```

```
def __init__(self):
    ...
```

Usually, a Python dictionary throws a KeyError if you try to get an item with a key that is not currently in the dictionary.

The defaultdict in contrast will simply create any items that you try to access

<https://stackoverflow.com/questions/5900578/how-does-collections-defaultdict-work>

```
...
self.child = collections.defaultdict(TrieNode)
self.is_word = False
```

```
class Trie:
```

```
def __init__(self):
    self.root = TrieNode()
```

```
def insert(self, word: str) -> None:
```

```
cur = self.root
for letter in word:
    cur = cur.child[letter]
cur.is_word = True
```

```
def search(self, word: str) -> bool:
    cur = self.root
    for letter in word:
        # cur = cur.child[letter] ==> will not work, it will create a default node for
        # this letter
        cur = cur.child.get(letter)
```

Round 5 · Mid · Medium

p.2012

```

if not cur:
    return False
return cur.is_word

def startsWith(self, prefix: str) -> bool:
    cur = self.root
    for letter in prefix:
        # cur = cur.child[letter] ==> will not work, it will creat a default node for
        # this letter
        cur = cur.child.get(letter)
    if not cur:
        return False
    return True

# Your Trie object will be instantiated and called as such:
# obj = Trie()
# obj.insert(word)
# param_2 = obj.search(word)
# param_3 = obj.startsWith(prefix)

#####

class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.is_end = False

    def insert(self, word: str) -> None:
        node = self
        for c in word:

```

Round 5 · Mid · Medium

p.2013

```

        idx = ord(c) - ord('a')
        if node.children[idx] is None:
            node.children[idx] = Trie()
        node = node.children[idx]
        node.is_end = True

def search(self, word: str) -> bool:
    node = self._search_prefix(word)
    return node is not None and node.is_end

def startsWith(self, prefix: str) -> bool:
    node = self._search_prefix(prefix)
    return node is not None

def _search_prefix(self, prefix: str):
    node = self
    for c in prefix:
        idx = ord(c) - ord('a')
        if node.children[idx] is None:
            return None

        node = node.children[idx]
    return node

# Your Trie object will be instantiated and called as such:
# obj = Trie()
# obj.insert(word)
# param_2 = obj.search(word)
# param_3 = obj.startsWith(prefix)

# below is using reduce()
class Trie(object):

    def __init__(self):
        T = lambda: collections.defaultdict(T)
        self.root = T()

    def insert(self, word):

```

Round 5 · Mid · Medium

p.2014

```

reduce(dict.__getitem__, word, self.root)['#'] = True

def search(self, word):
    return '#' in reduce(lambda cur, c: cur.get(c, {}), word, self.root)

def startsWith(self, prefix):
    return bool(reduce(lambda cur, c: cur.get(c, {}), prefix, self.root))

```

◆ Quick Review

Key Insights

- Use a tree-like structure where each node represents a character.
- Each node stores its children in a dictionary (or hash map) for $O(1)$ average time access.
- Mark the end of a word with a boolean flag.
- Insertion is done character by character, creating new nodes as needed.
- Searching and checking prefixes follow traversals down the tree.

Space & Time

Time Complexity:

- Insert: $O(m)$ where m is the length of the word.
- Search: $O(m)$
- startsWith: $O(m)$

Space Complexity:

Round 5 · Mid · Medium

p.2015

- $O(N * M)$ in the worst-case scenario, where N is the number of words and M is the average length of the words, due to storing each character in the Trie.

Solution

We implement the Trie using a node-based approach. Each TrieNode contains a hash map (or dictionary) of its children and a boolean indicating if the node marks the end of a word. The Trie class uses these nodes to build out the words character by character. The insert method traverses and builds the necessary nodes; the search method verifies that all characters exist in order and that the final node marks the end of a word; the startsWith method checks if the prefix exists in the Trie. No tricky edge cases exist beyond handling null or empty inputs, as constraints guarantee valid lowercase strings.

Round 5 · Mid · Medium

p.2016

Tries

□ #211 Design Add and Search Words Data Structure MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · DFS · Design · Trie
Lists: Grind 169

Problem

Design a data structure that supports adding new words and finding if a string matches any previously added string.

Implement the WordDictionary class:

- WordDictionary() Initializes the object.
- void addWord(word) Adds word to the data structure, it can be matched later.
- bool search(word) Returns true if there is any string in the data structure that matches word or false otherwise. word may contain dots '.' where dots can be matched with any letter.

Example:

```
Input
["WordDictionary","addWord","addWord","addWord","search","search","search","search"]
[[],["bad"],["dad"],["mad"],["pad"],["bad"],[".ad"],["b.."]]
Output
[null,null,null,null,false,true,true,true]
```

```
Explanation
WordDictionary wordDictionary = new WordDictionary();
```

Constraints:

- $1 \leq \text{word.length} \leq 25$
- word in addWord consists of lowercase English letters.
- word in search consist of '.' or lowercase English letters.
- There will be at most 2 dots in word for search queries.
- At most 104 calls will be made to addWord and search.

Community Solutions (Python)

□ • WalkCC

```
class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.isWord = False

class WordDictionary:
    def __init__(self):
        self.root = TrieNode()
```

```
def addWord(self, word: str) -> None:
    node: TrieNode = self.root
    for c in word:
        node = node.children.setdefault(c, TrieNode())
    node.isWord = True

def search(self, word: str) -> bool:
    return self._dfs(word, 0, self.root)

def _dfs(self, word: str, s: int, node: TrieNode) -> bool:

    if s == len(word):
        return node.isWord
    if word[s] != '.':
        child: TrieNode = node.children.get(word[s], None)
        return self._dfs(word, s + 1, child) if child else False
    return any(self._dfs(word, s + 1, child) for child in node.children.values
())
```

• LeetCode

```
class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.is_end = False

class WordDictionary:
    def __init__(self):
        self.trie = Trie()

    def addWord(self, word: str) -> None:
        node = self.trie
        for c in word:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
        node.is_end = True

    def search(self, word: str) -> bool:
```

Round 5 · Mid · Medium

p.2019

```
def search(word, node):
    for i in range(len(word)):
        c = word[i]
        idx = ord(c) - ord('a')
        if c != '.' and node.children[idx] is None:
            return False
        if c == '.':
            for child in node.children:
                if child is not None and search(word[i + 1:], child):
                    return True

        return False
        node = node.children[idx]
    return node.is_end

    return search(word, self.trie)

# Your WordDictionary object will be instantiated and called as such:
# obj = WordDictionary()
# obj.addWord(word)

# param_2 = obj.search(word)
```

• SimplyLeet

```
class TrieNode:
    def __init__(self):
        # Dictionary to store child nodes
        self.children = {}
        # Flag to indicate if a word ends here
        self.is_end = False

class WordDictionary:
    def __init__(self):
        # Initialize the Trie with an empty root node
```

Round 5 · Mid · Medium

p.2020

```

self.root = TrieNode()

def addWord(self, word: str) -> None:
    # Start at the root of the Trie
    node = self.root
    for char in word:
        # Create a new node if the character is not present
        if char not in node.children:
            node.children[char] = TrieNode()
        node = node.children[char]

    # Mark the end of the word
    node.is_end = True

def search(self, word: str) -> bool:
    # DFS helper function
    def dfs(index, node):
        if index == len(word):
            return node.is_end
        char = word[index]
        if char == '.':
            # If wildcard, search all children nodes
            for child in node.children.values():
                if dfs(index + 1, child):
                    return True
            return False
        else:
            # If character not found, word does not exist
            if char not in node.children:
                return False
            return dfs(index + 1, node.children[char])

    return dfs(0, self.root)

# Example usage:
wd = WordDictionary()
wd.addWord("bad")
wd.addWord("dad")
wd.addWord("mad")
print(wd.search("pad")) # Output: False
print(wd.search("bad")) # Output: True
print(wd.search(".ad")) # Output: True

```

Round 5 · Mid · Medium

p.2021

```
print(wd.search("b..")) # Output: True
```

• LC.ca

```

class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.is_end = False

class WordDictionary:
    def __init__(self):
        self.trie = Trie()

    def addWord(self, word: str) -> None:
        node = self.trie
        for c in word:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
        node.is_end = True

    def search(self, word: str) -> bool:
        def search(word, node):
            for i in range(len(word)):
                c = word[i]
                idx = ord(c) - ord('a')
                if c != '.' and node.children[idx] is None:
                    return False
                if c == '.':
                    for child in node.children:
                        if child is not None and search(word[i + 1:], child):
                            return True

```

Round 5 · Mid · Medium

p.2022

```

        return False
    node = node.children[idx]
    return node.is_end

    return search(word, self.trie)

```

```

# Your WordDictionary object will be instantiated and called as such:
# obj = WordDictionary()
# obj.addWord(word)

```

```

# param_2 = obj.search(word)

```

◆ Quick Review

Key Insights

- Utilize a Trie (prefix tree) to efficiently store words.
- Each node in the Trie contains a dictionary for its children and a flag indicating the end of a word.
- The search operation is implemented with a depth-first search (DFS) approach.
- When the search query contains a dot, explore all possible child nodes.

Space & Time

Time Complexity:

- addWord: $O(w)$, where w is the length of the word.

- search: Worst-case $O(26^w)$ for words with multiple wildcards, but practical performance is better given constraints.

Space Complexity: $O(n * w)$, where n is the number of words and w is the average word length.

Solution

The WordDictionary is implemented using a Trie. For adding, we insert each character of the word into the Trie. For searching, we use DFS. When encountering a wildcard '.', we recursively search all children nodes. This structure combined with DFS efficiently addresses the wildcard matching and ensures good performance for multiple operations.

Tries

☐ #421 Maximum XOR of Two Numbers in an Array

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table · Bit Manipulation · Trie
 Lists: AlgoMaster 600

Problem

Given an integer array `nums`, return the maximum result of `nums[i] XOR nums[j]`, where $0 \leq i \leq j < n$.

Example 1:

Input: `nums = [3,10,5,25,2,8]`
 Output: 28
 Explanation: The maximum result is 5 XOR 25 = 28.

Example 2:

Input: `nums = [14,70,53,83,49,91,36,80,92,51,66,70]`
 Output: 127

Constraints:

- $1 \leq \text{nums.length} \leq 2 * 10^5$
- $0 \leq \text{nums}[i] \leq 2^{31} - 1$

Community Solutions (Python)

- WalkCC

Round 5 · Mid · Medium

p.2025

```
class Solution:
    def findMaximumXOR(self, nums: list[int]) -> int:
        maxNum = max(nums)
        if maxNum == 0:
            return 0
        maxBit = int(math.log2(maxNum))
        ans = 0
        prefixMask = 0 # `prefixMask` grows like: 10000 -> 11000 -> ... -> 11111.

        # If ans is 11100 when i = 2, it means that before we reach the last two

        # bits, 11100 is the maximum XOR we have, and we're going to explore if we
        # can get another two 1s and put them into `ans`.
        for i in range(maxBit, -1, -1):
            prefixMask |= 1 << i
            # We only care about the left parts,
            # If i = 2, nums = [1110, 1011, 0111]
            # -> prefixes = [1100, 1000, 0100]
            prefixes = set([num & prefixMask for num in nums])
            # If i = 1 and before this iteration, the ans is 10100, it means that we
            # want to grow the ans to 10100 | 1 << 1 = 10110 and we're looking for

            # XOR of two prefixes = candidate.
            candidate = ans | 1 << i
            for prefix in prefixes:
                if prefix ^ candidate in prefixes:
                    ans = candidate
                    break

        return ans

class TrieNode:
    def __init__(self):
        self.children: list[TrieNode | None] = [None] * 2

class BitTrie:
    def __init__(self, maxBit: int):
        self.maxBit = maxBit
        self.root = TrieNode()
```

Round 5 · Mid · Medium

p.2026

```
def insert(self, num: int) -> None:
    node = self.root
    for i in range(self.maxBit, -1, -1):
        bit = num >> i & 1
        if not node.children[bit]:
            node.children[bit] = TrieNode()
        node = node.children[bit]

def getMaxXor(self, num: int) -> int:
    maxXor = 0

    node = self.root
    for i in range(self.maxBit, -1, -1):
        bit = num >> i & 1
        toggleBit = bit ^ 1
        if node.children[toggleBit]:
            maxXor = maxXor | 1 << i
            node = node.children[toggleBit]
        elif node.children[bit]:
            node = node.children[bit]
        else: # There's nothing in the Bit Trie.
```

```
        return 0
    return maxXor
```

```
class Solution:
    def findMaximumXOR(self, nums: List[int]) -> int:
        maxNum = max(nums)
        if maxNum == 0:
            return 0
        maxBit = int(math.log2(maxNum))
```

```
        ans = 0
        bitTrie = BitTrie(maxBit)

        for num in nums:
            ans = max(ans, bitTrie.getMaxXor(num))
            bitTrie.insert(num)
```

```
    return ans
```

• LeetCode

Round 5 · Mid · Medium

p.2027

```
class Trie:
    __slots__ = ("children",)

    def __init__(self):
        self.children: List[Trie | None] = [None, None]

    def insert(self, x: int):
        node = self
        for i in range(30, -1, -1):
            v = x >> i & 1
```

```
            if node.children[v] is None:
                node.children[v] = Trie()
            node = node.children[v]
```

```
    def search(self, x: int) -> int:
        node = self
        ans = 0
        for i in range(30, -1, -1):
            v = x >> i & 1
            if node.children[v ^ 1]:
```

```
                ans |= 1 << i
                node = node.children[v ^ 1]
            else:
                node = node.children[v]
        return ans
```

```
class Solution:
    def findMaximumXOR(self, nums: List[int]) -> int:
        trie = Trie()
```

```
        for x in nums:
            trie.insert(x)
        return max(trie.search(x) for x in nums)
```

• SimplyLeet

Round 5 · Mid · Medium

p.2028

```

# Define the node for Trie
class TrieNode:
    def __init__(self):
        self.children = {} # Dictionary to hold child nodes for bits 0 and 1

# Define the Trie data structure
class Trie:
    def __init__(self):
        self.root = TrieNode()

# Insert a number's binary representation into the Trie
def insert(self, num):
    node = self.root
    # Consider 32-bit integers (from bit 31 to 0)
    for i in range(31, -1, -1):
        bit = (num >> i) & 1 # Extract the i-th bit
        if bit not in node.children:
            node.children[bit] = TrieNode()
        node = node.children[bit]

# Query for the maximum XOR partner for the given number
def query(self, num):
    node = self.root
    xor_sum = 0
    for i in range(31, -1, -1):
        bit = (num >> i) & 1
        toggled = 1 - bit # Complement of the current bit
        if toggled in node.children:
            xor_sum |= (1 << i) # Set the i-th bit in the result
            node = node.children[toggled]
        else:
            node = node.children.get(bit, node)
    return xor_sum

class Solution:
    def findMaximumXOR(self, nums):
        trie = Trie()
        max_xor = 0
        # Insert the first number to initialize the Trie
        trie.insert(nums[0])

```

Round 5 · Mid · Medium

p.2029

```

for num in nums[1:]:
    # For each number, query the best possible XOR companion
    current_xor = trie.query(num)
    max_xor = max(max_xor, current_xor)
    trie.insert(num)
return max_xor

# Example usage:
# sol = Solution()
# print(sol.findMaximumXOR([3,10,5,25,2,8])) # Expected output: 28

```

Round 5 · Mid · Medium

p.2030

Tries

□ #648 Replace Words

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table · String · Trie
 Lists: AlgoMaster 600

Problem

In English, we have a concept called root, which can be followed by some other word to form another longer word – let's call this word derivative. For example, when the root "help" is followed by the word "ful", we can form a derivative "helpful".

Given a dictionary consisting of many roots and a sentence consisting of words separated by spaces, replace all the derivatives in the sentence with the root forming it. If a derivative can be replaced by more than one root, replace it with the root that has the shortest length.

Return the sentence after the replacement.

Example 1:

Round 5 · Mid · Medium

p.2031

Input: dictionary = ["cat","bat","rat"], sentence = "the cattle was rattled by the battery"
 Output: "the cat was rat by the bat"

Example 2:

Input: dictionary = ["a","b","c"], sentence = "aadsfasf absbs bbab cadsfafs"
 Output: "a a b c"

Constraints:

- $1 \leq \text{dictionary.length} \leq 1000$
- $1 \leq \text{dictionary}[i].\text{length} \leq 100$
- $\text{dictionary}[i]$ consists of only lower-case letters.
- $1 \leq \text{sentence.length} \leq 10^6$
- sentence consists of only lower-case letters and spaces.
- The number of words in sentence is in the range $[1, 1000]$
- The length of each word in sentence is in the range $[1, 1000]$
- Every two consecutive words in sentence will be separated by exactly one space.
- sentence does not have leading or trailing spaces.

Community Solutions (Python)

- WalkCC

Round 5 · Mid · Medium

p.2032

```

class Solution:
    def __init__(self):
        self.root = {}

    def insert(self, word: str) -> None:
        node = self.root
        for c in word:
            if c not in node:
                node[c] = {}
            node = node[c]

        node['word'] = word

    def search(self, word: str) -> str:
        node = self.root
        for c in word:
            if 'word' in node:
                return node['word']
            if c not in node:
                return word
            node = node[c]

        return word

    def replaceWords(self, dictionary: list[str], sentence: str) -> str:
        for word in dictionary:
            self.insert(word)

        words = sentence.split(' ')
        return ' '.join([self.search(word) for word in words])

```

• LeetCode

```

class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.is_end = False

    def insert(self, w: str) -> None:
        node = self
        for c in w:
            idx = ord(c) - ord("a")
            if node.children[idx] is None:

```

Round 5 · Mid · Medium

p.2033

```

        node.children[idx] = Trie()
        node = node.children[idx]
        node.is_end = True

    def search(self, w: str) -> str:
        node = self
        for i, c in enumerate(w, 1):
            idx = ord(c) - ord("a")
            if node.children[idx] is None:
                return w

            node = node.children[idx]
            if node.is_end:
                return w[:i]

        return w

class Solution:
    def replaceWords(self, dictionary: List[str], sentence: str) -> str:
        trie = Trie()
        for w in dictionary:
            trie.insert(w)

        return " ".join(trie.search(w) for w in sentence.split())

```

• SimplyLeet

```

# Define a Trie Node class
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end = False

# Define a Trie class to insert roots and search for shortest prefix
class Trie:
    def __init__(self):
        self.root = TrieNode()

```

Round 5 · Mid · Medium

p.2034

```

# Insert a word into the Trie
def insert(self, word: str) -> None:
    node = self.root
    for char in word:
        if char not in node.children:
            node.children[char] = TrieNode()
        node = node.children[char]
    node.is_end = True

# Search for the shortest prefix of the word that is in the Trie
def search_shortest_prefix(self, word: str) -> str:
    node = self.root
    prefix = ""
    for char in word:
        if char not in node.children:
            break
        node = node.children[char]
        prefix += char
        if node.is_end:
            return prefix
    return word

# Main function to replace words in the sentence
def replaceWords(dictionary: list, sentence: str) -> str:
    trie = Trie()
    # Insert all roots from dictionary into the Trie
    for root in dictionary:
        trie.insert(root)

    words = sentence.split()
    result = []
    # Process each word in the sentence
    for word in words:
        result.append(trie.search_shortest_prefix(word))
    return " ".join(result)

# Example usage:
# dictionary = ["cat", "bat", "rat"]
# sentence = "the cattle was rattled by the battery"

```

```

# print(replaceWords(dictionary, sentence))

```

Tries

#720 Longest Word in Dictionary

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table · String · Trie · Sorting
 Lists: Denny Zhang

Problem

Given an array of strings `words` representing an English Dictionary, return the longest word in `words` that can be built one character at a time by other words in `words`.

If there is more than one possible answer, return the longest word with the smallest lexicographical order. If there is no answer, return the empty string.

Note that the word should be built from left to right with each additional character being added to the end of a previous word.

Example 1:

Input: `words = ["w","wo","wor","worl","world"]`
 Output: `"world"`
 Explanation: The word "world" can be built one character at a time by "w", "wo", "wor", and "worl".

Example 2:

Round 5 · Mid · Medium

p.2037

Input: `words = ["a","banana","app","appl","ap","apply","apple"]`
 Output: `"apple"`
 Explanation: Both "apply" and "apple" can be built from other words in the dictionary. However, "apple" is lexicographically smaller than "apply".

Constraints:

- $1 \leq \text{words.length} \leq 1000$
- $1 \leq \text{words}[i].\text{length} \leq 30$
- `words[i]` consists of lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def longestWord(self, words: list[str]) -> str:
        root = {}

        for word in words:
            node = root
            for c in word:
                if c not in node:
                    node[c] = {}
                node = node[c]

            node['word'] = word

        def dfs(node: dict) -> str:
            ans = node['word'] if 'word' in node else ''

            for child in node:
                if 'word' in node[child] and len(node[child]['word']) > 0:
                    childWord = dfs(node[child])
                    if len(childWord) > len(ans) or (
                        len(childWord) == len(ans) and childWord < ans):
                        ans = childWord

            return ans

        return dfs(root)
```

Round 5 · Mid · Medium

p.2038

```

        ans = childWord

    return ans

return dfs(root)

```

• LeetDoocs

```

class Trie:
    def __init__(self):
        self.children: List[Optional[Trie]] = [None] * 26
        self.is_end = False

    def insert(self, w: str):
        node = self
        for c in w:
            idx = ord(c) - ord("a")
            if node.children[idx] is None:
                node.children[idx] = Trie()
                node = node.children[idx]
            node.is_end = True

    def search(self, w: str) -> bool:
        node = self
        for c in w:
            idx = ord(c) - ord("a")
            if node.children[idx] is None:
                return False
            node = node.children[idx]
            if not node.is_end:
                return False
        return True

class Solution:
    def longestWord(self, words: List[str]) -> str:
        trie = Trie()
        for w in words:

```

```

        trie.insert(w)
    ans = ""
    for w in words:
        if trie.search(w) and (
            len(ans) < len(w) or (len(ans) == len(w) and ans > w)
        ):
            ans = w
    return ans

```

• SimplyLeet

```

# Python solution for Longest Word in Dictionary

def longestWord(words):
    # Sort words by length and lexicographical order
    words.sort(key=lambda x: (len(x), x))

    valid_words = set()
    result = ""

    # Process each word
    for word in words:
        # For single-letter words, they are valid if they exist in the list
        if len(word) == 1 or word[:-1] in valid_words:
            valid_words.add(word)
            # Update result if the current word is longer or if equal length
            # but lexicographically smaller
            if len(word) > len(result) or (len(word) == len(result) and word <
                result):
                result = word

    return result

# Example usage:
# print(longestWord(["a", "banana", "app", "appl", "ap", "apply", "apple"]))

```

• LC.ca


```
class Solution:
    def longestWord(self, words: List[str]) -> str:
        cnt, ans = 0, ''
        s = set(words)
        for w in s:
            n = len(w)
            if all(w[:i] in s for i in range(1, n)):
                if cnt < n:
                    cnt, ans = n, w
                elif cnt == n and w < ans:
                    ans = w
        return ans
```

◆ Quick Review

Key Insights

- The word must be built character by character; hence every prefix (removing one character from the end at a time) must exist in the dictionary.
- Sorting the input list can help in ensuring that smaller words (prefixes) are processed first.
- Using a hash set to store valid words enables rapid prefix checks.
- Lexicographical order plays a role when words have the same length.

Space & Time

Round 5 · Mid · Medium

p.2041

Time Complexity: $O(n * L + n \log n)$ where n is the number of words and L is the maximum length of a word ($n * L$ for prefix checking, and $n \log n$ for sorting).

Space Complexity: $O(n * L)$ for storing words in the set.

Solution

The approach begins by sorting the list of words. Sorting is done primarily by word length and secondarily by lexicographical order so that when we iterate through the words, all possible prefixes for a word have already been considered. Initialize a set to keep track of valid words that can be built one character at a time. For each word in the sorted list, check if all its prefixes (word without the last character progressively) exist in the set. If a word qualifies, add it to the set and check if it should be the new result based on length and lexicographical order. The set lookup ensures that checking each prefix is efficient.

Round 5 · Mid · Medium

p.2042

Tries

☐ #1233 Remove Sub-Folders from the Filesystem

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · String · Depth-First Search · Trie
 Lists: AlgoMaster 600

Problem

Given a list of folders `folder`, return the folders after removing all sub-folders in those folders. You may return the answer in any order.

If a `folder[i]` is located within another `folder[j]`, it is called a sub-folder of it. A sub-folder of `folder[j]` must start with `folder[j]`, followed by a `"/`. For example, `"/a/b"` is a sub-folder of `"/a"`, but `"/b"` is not a sub-folder of `"/a/b/c"`.

The format of a path is one or more concatenated strings of the form: `'/'` followed by one or more lowercase English letters.

- For example, `"/leetcode"` and `"/leetcode/problems"` are valid paths while an empty string and `"/"` are not.

Example 1:

Round 5 · Mid · Medium

p.2043

Input: `folder = ["/a", "/a/b", "/c/d", "/c/d/e", "/c/f"]`
 Output: `["/a", "/c/d", "/c/f"]`
 Explanation: Folders `"/a/b"` is a subfolder of `"/a"` and `"/c/d/e"` is inside of folder `"/c/d"` in our filesystem.

Example 2:

Input: `folder = ["/a", "/a/b/c", "/a/b/d"]`
 Output: `["/a"]`
 Explanation: Folders `"/a/b/c"` and `"/a/b/d"` will be removed because they are subfolders of `"/a"`.

Example 3:

Input: `folder = ["/a/b/c", "/a/b/ca", "/a/b/d"]`
 Output: `["/a/b/c", "/a/b/ca", "/a/b/d"]`

Constraints:

- $1 \leq \text{folder.length} \leq 4 * 10^4$
- $2 \leq \text{folder}[i].\text{length} \leq 100$
- `folder[i]` contains only lowercase letters and `'/'`.
- `folder[i]` always starts with the character `'/'`.
- Each folder name is unique.

Community Solutions (Python)

- WalkCC

Round 5 · Mid · Medium

p.2044

```
class Solution:
    def removeSubfolders(self, folder: List[str]) -> List[str]:
        ans = []
        prev = ""

        folder.sort()

        for f in folder:
            if len(prev) > 0 and f.startswith(prev) and f[len(prev)] == '/':
                continue

            ans.append(f)
            prev = f

        return ans
```

• LeetCode

```
class Solution:
    def removeSubfolders(self, folder: List[str]) -> List[str]:
        folder.sort()
        ans = [folder[0]]
        for f in folder[1:]:
            m, n = len(ans[-1]), len(f)
            if m >= n or not (ans[-1] == f[:m] and f[m] == '/'):
                ans.append(f)
        return ans
```

```
class Trie:
    def __init__(self):
        self.children = {}
        self.fid = -1

    def insert(self, i, f):
        node = self
        ps = f.split('/')
        for p in ps[1:]:
            if p not in node.children:
```

```
        node.children[p] = Trie()
        node = node.children[p]
        node.fid = i

    def search(self):
    def dfs(root):
        if root.fid != -1:
            ans.append(root.fid)
            return
        for child in root.children.values():
            dfs(child)

    ans = []
    dfs(self)
    return ans
```

```
class Solution:
    def removeSubfolders(self, folder: List[str]) -> List[str]:
        trie = Trie()

        for i, f in enumerate(folder):
            trie.insert(i, f)
        return [folder[i] for i in trie.search()]
```

• SimplyLeet

```
# Python solution with line-by-line comments
class Solution:
    def removeSubfolders(self, folder: List[str]) -> List[str]:
        # Sort folders lexicographically so parent folders come before
        # sub-folders
        folder.sort()
        # This will store the final result with no sub-folders
        result = []
        for f in folder:
            # If result is empty or f is not a sub-folder of the last folder in
            # result,
            # then add f to the result list.
```

```

if not result or not f.startswith(result[-1] + '/'):
    result.append(f)
return result

```

Tries

□ #1268 Search Suggestions System

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · String · Binary Search · Trie · Sorting ·
 Heap (Priority Queue)
 Lists: AlgoMaster 600

Problem

You are given an array of strings `products` and a string `searchWord`.

Design a system that suggests at most three product names from `products` after each character of `searchWord` is typed. Suggested products should have common prefix with `searchWord`. If there are more than three products with a common prefix return the three lexicographically minimums products.

Return a list of lists of the suggested products after each character of `searchWord` is typed.

Example 1:

Input: products = ["mobile","mouse","moneypot","monitor","mousepad"],
searchWord = "mouse"
Output: [["mobile","moneypot","monitor"],["mobile","moneypot","monitor"],["mouse","mousepad"],["mouse","mousepad"],["mouse","mousepad"]]
Explanation: products sorted lexicographically =
["mobile","moneypot","monitor","mouse","mousepad"].
After typing m and mo all products match and we show user
["mobile","moneypot","monitor"].
After typing mou, mous and mouse the system suggests ["mouse","mousepad"].

Example 2:

Input: products = ["havana"], searchWord = "havana"
Output: [["havana"],["havana"],["havana"],["havana"],["havana"],["havana"]]
Explanation: The only word "havana" will be always suggested while typing the search word.

Constraints:

- $1 \leq \text{products.length} \leq 1000$
- $1 \leq \text{products}[i].\text{length} \leq 3000$
- $1 \leq \sum(\text{products}[i].\text{length}) \leq 2 * 10^4$
- All the strings of products are unique.
- products[i] consists of lowercase English letters.
- $1 \leq \text{searchWord.length} \leq 1000$
- searchWord consists of lowercase English letters.

Community Solutions (Python)

- WalkCC

```
class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.word: str | None = None

class Solution:
    def suggestedProducts(
        self,
        products: list[str],

        searchWord: str
    ) -> list[list[str]]:
        ans = []
        root = TrieNode()

        def insert(word: str) -> None:
            node = root
            for c in word:
                node = node.children.setdefault(c, TrieNode())
            node.word = word

        def search(node: TrieNode | None) -> list[str]:
            res: list[str] = []
            dfs(node, res)
            return res

        def dfs(node: TrieNode | None, res: list[str]) -> None:
            if len(res) == 3:
                return
            if not node:
                return
            if node.word:
                res.append(node.word)
            for c in string.ascii_lowercase:
                if c in node.children:
                    dfs(node.children[c], res)

        for product in products:
            insert(product)
```

```

node = root

for c in searchWord:
    if not node or c not in node.children:
        node = None
        ans.append([])
        continue
    node = node.children[c]
    ans.append(search(node))

return ans

```

• LeetDoocs

```

class Trie:
    def __init__(self):
        self.children: List[Union[Trie, None]] = [None] * 26
        self.v: List[int] = []

    def insert(self, w, i):
        node = self
        for c in w:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
            if len(node.v) < 3:
                node.v.append(i)

    def search(self, w):
        node = self
        ans = [[] for _ in range(len(w))]
        for i, c in enumerate(w):
            idx = ord(c) - ord('a')

```

```

        if node.children[idx] is None:
            break
        node = node.children[idx]
        ans[i] = node.v
        return ans

class Solution:
    def suggestedProducts(
        self, products: List[str], searchWord: str
    ) -> List[List[str]]:
        products.sort()
        trie = Trie()
        for i, w in enumerate(products):
            trie.insert(w, i)
        return [[products[i] for i in v] for v in trie.search(searchWord)]

```

• SimplyLeet

```

# Python solution for Search Suggestions System

import bisect

def suggestedProducts(products, searchWord):
    # Sort products lexicographically
    products.sort()
    result = []
    prefix = ""

    # For each character in searchWord, update the prefix
    for char in searchWord:
        prefix += char
        # Find the start index of matching products using binary search
        start = bisect.bisect_left(products, prefix)
        suggestions = []
        # Check next three products starting from 'start'
        for i in range(start, min(start + 3, len(products))):
            if products[i].startswith(prefix):
                suggestions.append(products[i])

```

```
        else:
            break
        result.append(suggestions)
    return result

# Example usage:
# products = ["mobile", "mouse", "moneypot", "monitor", "mousepad"]
# searchWord = "mouse"
# print(suggestedProducts(products, searchWord))
```

Tries

☐ #2452 Words Within Two Edits of Dictionary

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · String · Trie
Lists: AlgoMaster 600

Problem

You are given two string arrays, queries and dictionary. All words in each array comprise of lowercase English letters and have the same length.

In one edit you can take a word from queries, and change any letter in it to any other letter. Find all words from queries that, after a maximum of two edits, equal some word from dictionary.

Return a list of all words from queries, that match with some word from dictionary after a maximum of two edits. Return the words in the same order they appear in queries.

Example 1:

Input: queries = ["word","note","ants","wood"], dictionary = ["wood","joke","moat"]
 Output: ["word","note","wood"]
 Explanation:
 - Changing the 'r' in "word" to 'o' allows it to equal the dictionary word "wood".
 - Changing the 'n' to 'j' and the 't' to 'k' in "note" changes it to "joke".
 - It would take more than 2 edits for "ants" to equal a dictionary word.
 - "wood" can remain unchanged (0 edits) and match the corresponding dictionary word.
 Thus, we return ["word","note","wood"].

Example 2:

Input: queries = ["yes"], dictionary = ["not"]
 Output: []
 Explanation:
 Applying any two edits to "yes" cannot make it equal to "not". Thus, we return an empty array.

Constraints:

- $1 \leq \text{queries.length}, \text{dictionary.length} \leq 100$
- $n == \text{queries}[i].\text{length} == \text{dictionary}[j].\text{length}$
- $1 \leq n \leq 100$
- All queries[i] and dictionary[j] are composed of lowercase English letters.

Community Solutions (Python)

- WalkCC

```
class Solution:
    def twoEditWords(
        self,
        queries: List[str],
        dictionary: List[str],
    ) -> List[str]:
        return [query for query in queries
                if any(sum(a != b for a, b in zip(query, word)) < 3
                       for word in dictionary)]
```

• LeetCode

```
class Solution:
    def twoEditWords(self, queries: List[str], dictionary: List[str]) -> List[str]:
        ans = []
        for s in queries:
            for t in dictionary:
                if sum(a != b for a, b in zip(s, t)) < 3:
                    ans.append(s)
                    break
        return ans
```

• SimplyLeet

```
def two_edits_away(queries, dictionary):
    valid_words = [] # List to hold valid query words
    # Iterate through every query word
    for word in queries:
        # Compare query with each dictionary word
        for dict_word in dictionary:
            # Count number of positions with different characters
            differences = sum(1 for q_char, d_char in zip(word, dict_word) if
                             q_char != d_char)
            # If edit count is within two, mark word as valid
            if differences <= 2:
```



```

        valid_words.append(word)
        break # Stop checking other dictionary words for this query
    return valid_words

# Example usage:
queries = ["word", "note", "ants", "wood"]
dictionary = ["wood", "joke", "moat"]
print(two_edits_away(queries, dictionary)) # Output: ["word", "note", "wood"]

```

Tries

#3043 Find the Length of the Longest Common Prefix

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table · String · Trie
 Lists: AlgoMaster 600

Problem

You are given two arrays with positive integers `arr1` and `arr2`.

A prefix of a positive integer is an integer formed by one or more of its digits, starting from its leftmost digit. For example, 123 is a prefix of the integer 12345, while 234 is not.

A common prefix of two integers `a` and `b` is an integer `c`, such that `c` is a prefix of both `a` and `b`. For example, 5655359 and 56554 have common prefixes 565 and 5655 while 1223 and 43456 do not have a common prefix.

You need to find the length of the longest common prefix between all pairs of integers (`x`, `y`) such that `x` belongs to `arr1` and `y` belongs to `arr2`.

Return the length of the longest common prefix among all pairs. If no common prefix exists among them, return 0.

Example 1:

Input: arr1 = [1,10,100], arr2 = [1000]
 Output: 3
 Explanation: There are 3 pairs (arr1[i], arr2[j]):
 - The longest common prefix of (1, 1000) is 1.
 - The longest common prefix of (10, 1000) is 10.
 - The longest common prefix of (100, 1000) is 100.
 The longest common prefix is 100 with a length of 3.

Example 2:

Input: arr1 = [1,2,3], arr2 = [4,4,4]
 Output: 0
 Explanation: There exists no common prefix for any pair (arr1[i], arr2[j]), hence we return 0.
 Note that common prefixes between elements of the same array do not count.

Constraints:

- $1 \leq \text{arr1.length}, \text{arr2.length} \leq 5 * 10^4$
- $1 \leq \text{arr1}[i], \text{arr2}[i] \leq 10^8$

Community Solutions (Python)

• WalkCC

```
class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word: str) -> None:
```

Round 5 · Mid · Medium

p.2059

```
node: TrieNode = self.root
for c in word:
    node = node.children.setdefault(c, TrieNode())
node.isWord = True

def search(self, word: str) -> int:
    prefixLength = 0
    node = self.root
    for c in word:
        if c not in node.children:
            break
        node = node.children[c]
        prefixLength += 1
    return prefixLength

class Solution:
    def longestCommonPrefix(self, arr1: list[int], arr2: list[int]) -> int:
        trie = Trie()

        for num in arr1:
            trie.insert(str(num))

        return max(trie.search(str(num)) for num in arr2)
```

• LeetDoocs

```
class Solution:
    def longestCommonPrefix(self, arr1: List[int], arr2: List[int]) -> int:
        s = set()
        for x in arr1:
            while x:
                s.add(x)
                x //= 10
        mx = 0
        for x in arr2:
            while x:
```

Round 5 · Mid · Medium

p.2060

```

        if x in s:
            mx = max(mx, x)
            break
        x //= 10
    return len(str(mx)) if mx else 0

```

• SimplyLeet

Python solution with line-by-line comments

```

def longest_common_prefix_length(arr1, arr2):
    # Helper function to generate all prefixes of a given number string
    def get_prefixes(num_str):
        prefixes = set()
        for i in range(1, len(num_str) + 1):
            prefixes.add(num_str[:i])
        return prefixes

    # Build a set of prefixes from arr1
    prefixes1 = set()
    for num in arr1:
        s = str(num) # Convert the number to a string
        prefixes1.update(get_prefixes(s)) # Add all prefixes of the number

    # Build a set of prefixes from arr2
    prefixes2 = set()
    for num in arr2:
        s = str(num) # Convert the number to a string

        prefixes2.update(get_prefixes(s)) # Add all prefixes of the number

    # Find common prefixes between arr1 and arr2
    common_prefixes = prefixes1.intersection(prefixes2)

    # Determine the longest common prefix length
    max_length = 0
    for prefix in common_prefixes:
        max_length = max(max_length, len(prefix))

```

Round 5 · Mid · Medium

p.2061

```

    return max_length

# Example usage:
print(longest_common_prefix_length([1,10,100], [1000])) # Expected output: 3
print(longest_common_prefix_length([1,2,3], [4,4,4]))    # Expected output:
0

```

Round 5 · Mid · Medium

p.2062

Heaps

Round 5 · Mid · Medium · 5 questions

Round 5 · Mid · Medium

p.2063

Heaps

★ #1167 Minimum Cost to Connect Sticks ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Greedy · Heap

Lists: ★ Premium | Premium 98

Problem

You have some number of sticks with positive integer lengths. These lengths are given as an array sticks, where sticks[i] is the length of the ith stick.

You can connect any two sticks of lengths x and y into one stick by paying a cost of x + y. You must connect all the sticks until there is only one stick remaining.

Return the minimum cost of connecting all the given sticks into one stick in this way.

Example 1:

Input: sticks = [2,4,3]

Output: 14

Explanation: You start with sticks = [2,4,3].

1. Combine sticks 2 and 3 for a cost of 2 + 3 = 5. Now you have sticks = [5,4].

2. Combine sticks 5 and 4 for a cost of 5 + 4 = 9. Now you have sticks = [9].

There is only one stick left, so you are done. The total cost is 5 + 9 = 14.

Round 5 · Mid · Medium

p.2064

Example 2:

Input: sticks = [1,8,3,5]

Output: 30

Explanation: You start with sticks = [1,8,3,5].

1. Combine sticks 1 and 3 for a cost of $1 + 3 = 4$. Now you have sticks = [4,8,5].

2. Combine sticks 4 and 5 for a cost of $4 + 5 = 9$. Now you have sticks = [9,8].

3. Combine sticks 9 and 8 for a cost of $9 + 8 = 17$. Now you have sticks = [17]. There is only one stick left, so you are done. The total cost is $4 + 9 + 17 = 30$.

Example 3:

Input: sticks = [5]

Output: 0

Explanation: There is only one stick, so you don't need to do anything. The total cost is 0.

Constraints:

- $1 \leq \text{sticks.length} \leq 104$
- $1 \leq \text{sticks}[i] \leq 104$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def connectSticks(self, sticks: list[int]) -> int:
        ans = 0
        heapq.heapify(sticks)

        while len(sticks) > 1:
            x = heapq.heappop(sticks)
            y = heapq.heappop(sticks)
            ans += x + y
            heapq.heappush(sticks, x + y)

        return ans
```

• LeetCode

```
class Solution:
    def connectSticks(self, sticks: List[int]) -> int:
        heapq.heapify(sticks)
        ans = 0
        while len(sticks) > 1:
            z = heapq.heappop(sticks) + heapq.heappop(sticks)
            ans += z
            heapq.heappush(sticks, z)
        return ans
```

• SimplyLeet

```
import heapq

def connectSticks(sticks):
    # Initialize the heap with the provided stick lengths
    heapq.heapify(sticks)

    # Initialize total cost to 0
    total_cost = 0

    # Continue connecting sticks until there is only one left
    while len(sticks) > 1:
        # Extract the two smallest sticks
        stick1 = heapq.heappop(sticks)
        stick2 = heapq.heappop(sticks)
        # Calculate the cost to connect these two sticks
        cost = stick1 + stick2
        # Add the cost to the total cost
        total_cost += cost
        # Push the combined stick back into the heap
        heapq.heappush(sticks, cost)

    return total_cost

# Example usage:
# print(connectSticks([2, 4, 3])) # Output: 14
```

• LC.ca

```
class Solution:
    def connectSticks(self, sticks: List[int]) -> int:
        heapify(sticks)
        ans = 0
        while len(sticks) > 1:
            z = heappop(sticks) + heappop(sticks)
            ans += z
            heappush(sticks, z)
        return ans
```

◆ Quick Review

Key Insights

- Always connect the two smallest sticks available to minimize the incremental cost.
- A min-heap (or priority queue) is an ideal data structure to efficiently retrieve the smallest sticks.
- The process continues until only one stick remains.

Space & Time

Time Complexity: $O(n \log n)$, where n is the number of sticks (due to heap insertion and removal operations).

Space Complexity: $O(n)$, for storing the sticks in the heap.

Solution

Round 5 · Mid · Medium

p.2067

The solution uses a greedy algorithm with a min-heap:

– Insert all stick lengths into a min-heap. – While there are at least two sticks in the heap: Extract the two smallest sticks. Compute the cost of connecting them (sum the two lengths). Add this cost to the total result. Push the combined stick (with length equal to their sum) back into the heap. – Once the heap is reduced to one element, return the total cost. This approach ensures that at every connection, the smallest possible sticks are merged, which leads to the minimum overall cost.

Round 5 · Mid · Medium

p.2068

Heaps

#1405 Longest Happy String

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Greedy · Heap (Priority Queue)
Lists: AlgoMaster 600

Problem

A string s is called happy if it satisfies the following conditions:

- s only contains the letters 'a', 'b', and 'c'.
- s does not contain any of "aaa", "bbb", or "ccc" as a substring.
- s contains at most a occurrences of the letter 'a'.
- s contains at most b occurrences of the letter 'b'.
- s contains at most c occurrences of the letter 'c'.

Given three integers a , b , and c , return the longest possible happy string. If there are multiple longest happy strings, return any of them. If there is no such string, return the empty

Round 5 · Mid · Medium

p.2069

string "".

A substring is a contiguous sequence of characters within a string.

Example 1:

Input: $a = 1, b = 1, c = 7$
Output: "ccaccbcc"
Explanation: "ccbccacc" would also be a correct answer.

Example 2:

Input: $a = 7, b = 1, c = 0$
Output: "aabaa"
Explanation: It is the only correct answer in this case.

Constraints:

- $0 \leq a, b, c \leq 100$
- $a + b + c > 0$

Community Solutions (Python)

• LeetDoocs

```
class Solution:
    def longestDiverseString(self, a: int, b: int, c: int) -> str:
        h = []
        if a > 0:
            heappush(h, [-a, 'a'])
        if b > 0:
            heappush(h, [-b, 'b'])
        if c > 0:
            heappush(h, [-c, 'c'])
```

Round 5 · Mid · Medium

p.2070

```

ans = []
while len(h) > 0:
    cur = heappop(h)
    if len(ans) >= 2 and ans[-1] == cur[1] and ans[-2] == cur[1]:
        if len(h) == 0:
            break
        nxt = heappop(h)
        ans.append(nxt[1])
        if -nxt[0] > 1:
            nxt[0] += 1

        heappush(h, nxt)
        heappush(h, cur)
    else:
        ans.append(cur[1])
        if -cur[0] > 1:
            cur[0] += 1
            heappush(h, cur)

return ''.join(ans)

```

• SimplyLeet

```

import heapq

def longestDiverseString(a: int, b: int, c: int) -> str:
    # Create a max heap using negative counts (heapq is a min heap by default)
    max_heap = []
    for count, char in [(-a, 'a'), (-b, 'b'), (-c, 'c')]:
        if count < 0:
            heapq.heappush(max_heap, (count, char))

    result = []

```

```

while max_heap:
    # Get the character with the highest remaining count
    count, char = heapq.heappop(max_heap)
    # Check if appending this character causes three consecutive same
    characters
    if len(result) >= 2 and result[-1] == result[-2] == char:
        # If no alternative, break out of the loop
        if not max_heap:
            break
        # Use the next most frequent character instead

    count2, char2 = heapq.heappop(max_heap)
    result.append(char2)
    count2 += 1 # Decrease the count (since counts are negative)
    if count2 < 0:
        heapq.heappush(max_heap, (count2, char2))
    # Push the original character back for future consideration
    heapq.heappush(max_heap, (count, char))
else:
    result.append(char)
    count += 1 # Decrement count (count is negative so adding 1
    increases it)

    if count < 0:
        heapq.heappush(max_heap, (count, char))

return ''.join(result)

```

```

# Example usage:
print(longestDiverseString(1, 1, 7)) # could output "ccaccbcc" or another
valid answer

```


Heaps

□ #1642 Furthest Building You Can Reach **MED**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Greedy · Heap (Priority Queue)
Lists: AlgoMaster 600

Problem

You are given an integer array `heights` representing the heights of buildings, some bricks, and some ladders.

You start your journey from building 0 and move to the next building by possibly using bricks or ladders.

While moving from building i to building $i+1$ (0-indexed),

- If the current building's height is greater than or equal to the next building's height, you do not need a ladder or bricks.
- If the current building's height is less than the next building's height, you can either use one ladder or $(h[i+1] - h[i])$ bricks.

Round 5 · Mid · Medium

p.2073

Return the furthest building index (0-indexed) you can reach if you use the given ladders and bricks optimally.

Example 1:



Round 5 · Mid · Medium

p.2074

Input: heights = [4,2,7,6,9,14,12], bricks = 5, ladders = 1

Output: 4

Explanation: Starting at building 0, you can follow these steps:

- Go to building 1 without using ladders nor bricks since $4 \geq 2$.
- Go to building 2 using 5 bricks. You must use either bricks or ladders because $2 < 7$.
- Go to building 3 without using ladders nor bricks since $7 \geq 6$.
- Go to building 4 using your only ladder. You must use either bricks or ladders because $6 < 9$.

It is impossible to go beyond building 4 because you do not have any more bricks or ladders.

Example 2:

Input: heights = [4,12,2,7,3,18,20,3,19], bricks = 10, ladders = 2

Output: 7

Example 3:

Input: heights = [14,3,19,3], bricks = 17, ladders = 0

Output: 3

Constraints:

- $1 \leq \text{heights.length} \leq 105$
- $1 \leq \text{heights}[i] \leq 106$
- $0 \leq \text{bricks} \leq 109$
- $0 \leq \text{ladders} \leq \text{heights.length}$

Community Solutions (Python)

- WalkCC

```
class Solution:
    def furthestBuilding(
        self,
        heights: List[int],
        bricks: int,
        ladders: int,
    ) -> int:
        minHeap = []

        for i, (a, b) in enumerate(itertools.pairwise(heights)):

            diff = b - a
            if diff <= 0:
                continue
            heapq.heappush(minHeap, diff)
            # If we run out of ladders, greedily use as less bricks as possible.
            if len(minHeap) > ladders:
                bricks -= heapq.heappop(minHeap)
            if bricks < 0:
                return i

        return len(heights) - 1
```

• LeetDoocs

```
class Solution:
    def furthestBuilding(self, heights: List[int], bricks: int, ladders: int)
-> int:
        h = []
        for i, a in enumerate(heights[:-1]):
            b = heights[i + 1]
            d = b - a
            if d > 0:
                heappush(h, d)
                if len(h) > ladders:
                    bricks -= heappop(h)

            if bricks < 0:
                return i
        return len(heights) - 1
```

• SimplyLeet

```
import heapq

def furthestBuilding(heights, bricks, ladders):
    # Min-heap to store the positive height differences where resources are
    # used
    min_heap = []

    # Loop through each building
    for i in range(len(heights) - 1):
        climb = heights[i+1] - heights[i]
        # Only consider climbs that require additional resources

        if climb > 0:
            heapq.heappush(min_heap, climb)

        # If we have more climbs than ladders, use bricks for the smallest
        climb
        if len(min_heap) > ladders:
            bricks -= heapq.heappop(min_heap)

        # If bricks run out, return the current building index
        if bricks < 0:
            return i

    # If we overcame all challenges, return the last building index
    return len(heights) - 1

# Example usage:
#print(furthestBuilding([4,2,7,6,9,14,12], 5, 1))
```

Round 5 · Mid · Medium

p.2077

Heaps

□ #1834 Single-Threaded CPU

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Sorting · Heap (Priority Queue)
 Lists: AlgoMaster 600

Problem

You are given n tasks labeled from 0 to $n - 1$ represented by a 2D integer array `tasks`, where `tasks[i] = [enqueueTimei, processingTimei]` means that the i th task will be available to process at `enqueueTimei` and will take `processingTimei` to finish processing.

You have a single-threaded CPU that can process at most one task at a time and will act in the following way:

- If the CPU is idle and there are no available tasks to process, the CPU remains idle.
- If the CPU is idle and there are available tasks, the CPU will choose the one with the shortest processing time. If multiple tasks have the same shortest processing time, it will choose the task with the smallest index.

Round 5 · Mid · Medium

p.2078

- Once a task is started, the CPU will process the entire task without stopping.
- The CPU can finish a task then start a new one instantly.

Return the order in which the CPU will process the tasks.

Example 1:

Input: tasks = [[1,2],[2,4],[3,2],[4,1]]
 Output: [0,2,3,1]
 Explanation: The events go as follows:
 - At time = 1, task 0 is available to process. Available tasks = {0}.
 - Also at time = 1, the idle CPU starts processing task 0. Available tasks = {}.
 - At time = 2, task 1 is available to process. Available tasks = {1}.
 - At time = 3, task 2 is available to process. Available tasks = {1, 2}.
 - Also at time = 3, the CPU finishes task 0 and starts processing task 2 as it is the shortest. Available tasks = {1}.

Example 2:

Input: tasks = [[7,10],[7,12],[7,5],[7,4],[7,2]]
 Output: [4,3,2,0,1]
 Explanation: The events go as follows:
 - At time = 7, all the tasks become available. Available tasks = {0,1,2,3,4}.
 - Also at time = 7, the idle CPU starts processing task 4. Available tasks = {0,1,2,3}.
 - At time = 9, the CPU finishes task 4 and starts processing task 3. Available tasks = {0,1,2}.
 - At time = 13, the CPU finishes task 3 and starts processing task 2. Available tasks = {0,1}.
 - At time = 18, the CPU finishes task 2 and starts processing task 0. Available tasks = {1}.

Constraints:

- tasks.length == n
- 1 <= n <= 105

- 1 <= enqueueTime_i, processingTime_i <= 109

Community Solutions (Python)

• WalkCC

```
class Solution:
    def getOrder(self, tasks: list[list[int]]) -> list[int]:
        n = len(tasks)
        A = [[*task, i] for i, task in enumerate(tasks)]
        ans = []
        minHeap = []
        i = 0 # tasks' index
        time = 0 # the current time

        A.sort()

        while i < n or minHeap:
            if not minHeap:
                time = max(time, A[i][0])
            while i < n and time >= A[i][0]:
                heapq.heappush(minHeap, (A[i][1], A[i][2]))
                i += 1
            procTime, index = heapq.heappop(minHeap)
            time += procTime
            ans.append(index)

        return ans
```

• LeetDoocs

```
class Solution:
    def getOrder(self, tasks: List[List[int]]) -> List[int]:
        for i, task in enumerate(tasks):
            task.append(i)
        tasks.sort()
        ans = []
        q = []
        n = len(tasks)
        i = t = 0
        while q or i < n:
            if not q:
                t = max(t, tasks[i][0])
            while i < n and tasks[i][0] <= t:
                heappush(q, (tasks[i][1], tasks[i][2]))
                i += 1
            pt, j = heappop(q)
            ans.append(j)
            t += pt
        return ans
```

• SimplyLeet

```
import heapq

class Solution:
    def getOrder(self, tasks):
        # Augment each task with its original index: (enqueueTime,
        # processingTime, index)
        indexed_tasks = [(task[0], task[1], i) for i, task in enumerate(tasks)]

        # Sort tasks by enqueueTime
        indexed_tasks.sort(key=lambda x: x[0])

        result = [] # This list will store the order of task indices
```

```
min_heap = [] # Heap to select the next task by processing time and
index
time = 0 # Current time
i = 0 # Pointer to tasks in indexed_tasks list
n = len(tasks)

while i < n or min_heap:
    # If no available tasks, jump time to the next task's enqueueTime
    if not min_heap and time < indexed_tasks[i][0]:
        time = indexed_tasks[i][0]
    # Add all tasks that have become available by current time to the
    heap

    while i < n and indexed_tasks[i][0] <= time:
        enqueue_time, processing_time, index = indexed_tasks[i]
        heapq.heappush(min_heap, (processing_time, index))
        i += 1
    # Process the task at the top of the heap (shortest processing time
    # then smallest index)
    proc_time, index = heapq.heappop(min_heap)
    time += proc_time # Update the current time by the processing time
    # of the task
    result.append(index)
return result

# Example usage:
# sol = Solution()
# print(sol.getOrder([[1,2],[2,4],[3,2],[4,1]]))
```

Heaps

□ #1882 Process Tasks Using Servers

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Heap (Priority Queue)

Lists: AlgoMaster 600

Problem

You are given two 0-indexed integer arrays `servers` and `tasks` of lengths `n` and `m` respectively. `servers[i]` is the weight of the `i`th server, and `tasks[j]` is the time needed to process the `j`th task in seconds.

Tasks are assigned to the servers using a task queue. Initially, all servers are free, and the queue is empty.

At second `j`, the `j`th task is inserted into the queue (starting with the 0th task being inserted at second 0). As long as there are free servers and the queue is not empty, the task in the front of the queue will be assigned to a free server with the smallest weight, and in case of a tie, it is assigned to a free server with the smallest index.

Round 5 · Mid · Medium

p.2083

If there are no free servers and the queue is not empty, we wait until a server becomes free and immediately assign the next task. If multiple servers become free at the same time, then multiple tasks from the queue will be assigned in order of insertion following the weight and index priorities above.

A server that is assigned task `j` at second `t` will be free again at second `t + tasks[j]`.

Build an array `ans` of length `m`, where `ans[j]` is the index of the server the `j`th task will be assigned to.

Return the array `ans`.

Example 1:

```
Input: servers = [3,3,2], tasks = [1,2,3,2,1,2]
Output: [2,2,0,2,1,2]
Explanation: Events in chronological order go as follows:
- At second 0, task 0 is added and processed using server 2 until second 1.
- At second 1, server 2 becomes free. Task 1 is added and processed using
  server 2 until second 3.
- At second 2, task 2 is added and processed using server 0 until second 5.
- At second 3, server 2 becomes free. Task 3 is added and processed using
  server 2 until second 5.
- At second 4, task 4 is added and processed using server 1 until second 5.
```

Example 2:

Round 5 · Mid · Medium

p.2084

Input: servers = [5,1,4,3,2], tasks = [2,1,2,4,5,2,1]

Output: [1,4,1,4,1,3,2]

Explanation: Events in chronological order go as follows:

- At second 0, task 0 is added and processed using server 1 until second 2.
- At second 1, task 1 is added and processed using server 4 until second 2.
- At second 2, servers 1 and 4 become free. Task 2 is added and processed using server 1 until second 4.
- At second 3, task 3 is added and processed using server 4 until second 7.
- At second 4, server 1 becomes free. Task 4 is added and processed using server 1 until second 9.

Constraints:

- `servers.length == n`
- `tasks.length == m`
- $1 \leq n, m \leq 2 * 10^5$
- $1 \leq \text{servers}[i], \text{tasks}[j] \leq 2 * 10^5$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def assignTasks(self, servers: list[int], tasks: list[int]) -> list[int]:
        ans = []
        free = [] # (weight, index, freeTime)
        used = [] # (freeTime, weight, index)

        for i, weight in enumerate(servers):
            heapq.heappush(free, (weight, i, 0))

        for i, executionTime in enumerate(tasks): # i := the current time
```

```
# Poll all servers that'll be free at time i.
while used and used[0][0] <= i:
    curr = heapq.heappop(used)
    heapq.heappush(free, (curr[1], curr[2], curr[0]))
if free:
    curr = heapq.heappop(free)
    ans.append(curr[1])
    heapq.heappush(used, (i + executionTime, curr[0], curr[1]))
else:
    curr = heapq.heappop(used)

ans.append(curr[2])
heapq.heappush(used, (curr[0] + executionTime, curr[1], curr[2]))

return ans
```

• LeetCode

```
class Solution:
    def assignTasks(self, servers: List[int], tasks: List[int]) -> List[int]:
        idle = [(x, i) for i, x in enumerate(servers)]
        heapq.heapify(idle)
        busy = []
        ans = []
        for j, t in enumerate(tasks):
            while busy and busy[0][0] <= j:
                _, s, i = heapq.heappop(busy)
                heapq.heappush(idle, (s, i))

            if idle:
                s, i = heapq.heappop(idle)
                heapq.heappush(busy, (j + t, s, i))
            else:
                w, s, i = heapq.heappop(busy)
                heapq.heappush(busy, (w + t, s, i))
            ans.append(i)
        return ans
```

• SimplyLeet

```
# Python solution using heaps from the heapq module
import heapq

def processTasks(servers, tasks):
    # free_servers heap: (server_weight, server_index)
    free_servers = [(weight, idx) for idx, weight in enumerate(servers)]
    heapq.heapify(free_servers)

    # busy_servers heap: (finish_time, server_weight, server_index)
    busy_servers = []

    n = len(tasks)
    result = [0] * n # To store the server assigned to each task
    time = 0 # Current time

    # Iterate through each task (arrival time is j)
    for j, task_time in enumerate(tasks):
        time = max(time, j) # Ensure current time is at least j

        # Move servers that have completed tasks from busy_servers to
        free_servers

        while busy_servers and busy_servers[0][0] <= time:
            finish_time, weight, server_idx = heapq.heappop(busy_servers)
            heapq.heappush(free_servers, (weight, server_idx))

        # If no free servers, fast-forward time to when the next server becomes
        free

        if not free_servers:
            time = busy_servers[0][0]
            while busy_servers and busy_servers[0][0] <= time:
                finish_time, weight, server_idx = heapq.heappop(busy_servers)
                heapq.heappush(free_servers, (weight, server_idx))
```

```
# Assign the task to the free server with smallest weight (and index)
weight, server_idx = heapq.heappop(free_servers)
result[j] = server_idx

# Add the server into busy_servers with its new finish time
heapq.heappush(busy_servers, (time + task_time, weight, server_idx))

return result

# Example usage:
# servers = [3,3,2]
# tasks = [1,2,3,2,1,2]
# print(processTasks(servers, tasks)) # Output: [2,2,0,2,1,2]
```


Top K Elements

Round 5 · Mid · Medium · 2 questions

Round 5 · Mid · Medium

p.2089

Top K Elements

□ #347 Top K Frequent Elements

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Divide and Conquer ·
Sorting · Heap · Bucket Sort · Counting ·
Quickselect

Lists: Denny Zhang

Problem

Given an integer array `nums` and an integer `k`, return the `k` most frequent elements. You may return the answer in any order.

Example 1:

Input: `nums = [1,1,1,2,2,3]`, `k = 2`

Output: `[1,2]`

Example 2:

Input: `nums = [1]`, `k = 1`

Output: `[1]`

Example 3:

Input: `nums = [1,2,1,2,1,2,3,1,3,2]`, `k = 2`

Output: `[1,2]`

Constraints:

Round 5 · Mid · Medium

p.2090

- $1 \leq \text{nums.length} \leq 105$
- $-104 \leq \text{nums}[i] \leq 104$
- k is in the range $[1, \text{the number of unique elements in the array}]$.
- It is guaranteed that the answer is unique.

Follow up: Your algorithm's time complexity must be better than $O(n \log n)$, where n is the array's size.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:
        ans = []
        bucket = [[] for _ in range(len(nums) + 1)]

        for num, freq in collections.Counter(nums).items():
            bucket[freq].append(num)

        for b in reversed(bucket):
            ans += b

        if len(ans) == k:
            return ans
```

• LeetDoocs

```
class Solution:
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:
        cnt = Counter(nums)
        return [x for x, _ in cnt.most_common(k)]
```

• SimplyLeet

Round 5 · Mid · Medium

p.2091

```
# Python solution using bucket sort

def topKFrequent(nums, k):
    # Count the frequency of each element in nums
    frequency = {}
    for num in nums:
        frequency[num] = frequency.get(num, 0) + 1

    # Create buckets where index represents frequency
    n = len(nums)

    buckets = [[] for _ in range(n + 1)]
    for num, count in frequency.items():
        buckets[count].append(num)

    # Gather the k most frequent elements
    result = []
    # Iterate over buckets in reverse order (highest frequency first)
    for freq in range(n, 0, -1):
        for num in buckets[freq]:
            result.append(num)

    if len(result) == k:
        return result

# Example usage:
# print(topKFrequent([1,1,1,2,2,3], 2)) -> Output: [1, 2]
```

• LC.ca

```
'''
>>> nums = ["a", "a", "b", "c", "c"]

>>> cnt = Counter(nums)
>>> cnt
Counter({'a': 2, 'c': 2, 'b': 1})

# default to keys
>>> sorted_freqs = sorted(cnt, key=lambda x: (-cnt[x], x))
>>> sorted_freqs
```

Round 5 · Mid · Medium

p.2092

```

['a', 'c', 'b']
'''
from collections import Counter

class Solution:
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:
        # Count the frequency of each element in the list
        cnt = Counter(nums)

        # Sort the elements by frequency in decreasing order

        # diff from below solution: sorted(cnt), not sorted(cnt.items())
        # more in https://leetcode.ca/2017-10-22-692-Top-K-Frequent-Words/
        sorted_freqs = sorted(cnt, key=lambda x: (-cnt[x], x))

        return sorted_freqs[:k]

#####

class Solution:
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:

        # Count the frequency of each element in the list
        freqs = Counter(nums)

        # Sort the elements by frequency in decreasing order
        sorted_freqs = sorted(freqs.items(), key=lambda x: x[1], reverse=True)
        # also passing 0J: sorted_freqs = sorted(freqs.items(), key=lambda x:
-x[1])

        # Take the top k elements
        top_k = [num for num, freq in sorted_freqs[:k]]

```

Round 5 · Mid · Medium

p.2093

```

        return top_k

#####

'''
>>> from heapq import heappush
>>> h = []
>>> heappush(h, (3,1))
>>> heappush(h, (1,1))
>>> heappush(h, (2,1))

>>> h
[(1, 1), (3, 1), (2, 1)]

>>> from heapq import heappop
>>> heappop(h)
(1, 1)
>>> heappop(h)
(2, 1)
>>> heappop(h)
(3, 1)

'''

from collections import Counter

class Solution: # heap
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:
        cnt = Counter(nums)
        hp = []
        for num, freq in cnt.items():
            heappush(hp, (freq, num)) # freq first, default sort by 1st element

        if len(hp) > k:
            heappop(hp)
        return [v[1] for v in hp]

#####

from typing import List
import random
# O(n log(n)) in the average case
# O(n^2) worst case

```

Round 5 · Mid · Medium

p.2094

```

class Solution:
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:
        freq_map = {}
        for num in nums:
            freq_map[num] = freq_map.get(num, 0) + 1

        freq_list = list(freq_map.items())
        self.quick_select(freq_list, 0, len(freq_list) - 1, k)

        return [num for num, freq in freq_list[:k]]

    def quick_select(self, freq_list, start, end, k):
        if start == end:
            return

        pivot_idx = random.randint(start, end)
        pivot_freq = freq_list[pivot_idx][1]

        left = start
        right = end

        while left <= right:
            while freq_list[left][1] > pivot_freq:
                left += 1
            while freq_list[right][1] < pivot_freq:
                right -= 1

            if left <= right:
                freq_list[left], freq_list[right] = freq_list[right], freq_list[left]
                left += 1
                right -= 1

        return freq_list[start:k]

```

Round 5 · Mid · Medium

p.2095

```

        right -= 1

        if k <= right:
            self.quick_select(freq_list, start, right, k)
        elif k >= left:
            self.quick_select(freq_list, left, end, k)

        #####

'''
>>> x = [1, 2, 3]
>>> x.append([4, 5])
>>> print(x)
[1, 2, 3, [4, 5]]

>>> x = [1, 2, 3]
>>> x.extend([4, 5])

>>> print(x)
[1, 2, 3, 4, 5]
'''

class Solution:
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:
        freq = Counter(nums)

        # not always filled, max possible frequency is len(nums) [1,1,1,1,...]
        buckets = [[] for _ in range(len(nums) + 1)]
        for num, freq in freq.items():

            buckets[freq].append(num)

        res = []
        for bucket in reversed(buckets):
            if bucket:
                res.extend(bucket)
                if len(res) >= k:
                    break

        return res[:k]

```

Round 5 · Mid · Medium

p.2096

```
#####

class Solution(object):
    def topKFrequent(self, nums, k):
        """
        :type nums: List[int]
        :type k: int
        :rtype: List[int]
        """

        d = {}
        res = []
        ans = []
        buckets = [[] for _ in range(len(nums) + 1)]

        for num in nums:
            d[num] = d.get(num, 0) + 1

        for key in d:
            res.append((d[key], key))

        for t in res:
            freq, key = t
            buckets[freq].append(key)

        buckets.reverse()

        for item in buckets:
            if item and k > 0:
                while item and k > 0:
                    ans.append(item.pop())
                    k -= 1
            if k == 0:
                return ans

        return ans
```

◆ Quick Review

Round 5 · Mid · Medium

p.2097

Key Insights

- Count the frequency of each element using a hash table.
- Use bucket sort to achieve linear time complexity by grouping numbers by their frequency.
- Traverse the frequency buckets from highest to lowest to collect the k most frequent elements.
- Alternative approaches include using heaps or Quickselect, but bucket sort meets the follow-up requirement of better than $O(n \log n)$ time complexity.

Space & Time

Time Complexity: $O(n)$ where n is the length of the array (bucket sort and frequency counting are linear).

Space Complexity: $O(n)$ for storing the counts and buckets.

Solution

The solution starts by counting the frequency of each element with a hash table (or dictionary). Then, we create an array of buckets where the index represents the

Round 5 · Mid · Medium

p.2098

frequency. Each bucket holds the numbers that appear with that frequency. Finally, we iterate over the buckets in reverse order (from high frequency to low) and collect elements until we have k of them. This bucket sort approach avoids the higher complexity of sorting a list of frequencies and guarantees linear time performance.

Top K Elements

□ #973 K Closest Points to Origin

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode Array · Math · Divide and Conquer · Sorting · Heap

Lists: Grind 169

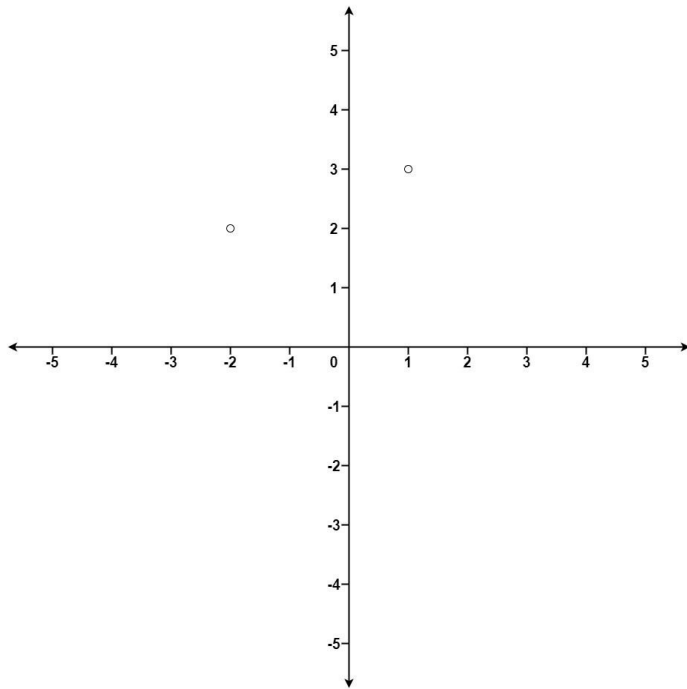
Problem

Given an array of points where $\text{points}[i] = [x_i, y_i]$ represents a point on the X-Y plane and an integer k, return the k closest points to the origin (0, 0).

The distance between two points on the X-Y plane is the Euclidean distance (i.e., $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$).

You may return the answer in any order. The answer is guaranteed to be unique (except for the order that it is in).

Example 1:



Input: points = [[1,3],[-2,2]], k = 1

Output: [[-2,2]]

Explanation:

The distance between (1, 3) and the origin is $\sqrt{10}$.

The distance between (-2, 2) and the origin is $\sqrt{8}$.

Since $\sqrt{8} < \sqrt{10}$, (-2, 2) is closer to the origin.

We only want the closest k = 1 points from the origin, so the answer is just [[-2,2]].

Example 2:

Input: points = [[3,3],[5,-1],[-2,4]], k = 2

Output: [[3,3],[-2,4]]

Explanation: The answer [[-2,4],[3,3]] would also be accepted.

Constraints:

- $1 \leq k \leq \text{points.length} \leq 104$

Round 5 · Mid · Medium

p.2101

- $-104 \leq x_i, y_i \leq 104$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def kClosest(self, points: list[list[int]], k: int) -> list[list[int]]:
        maxHeap = []

        for x, y in points:
            heapq.heappush(maxHeap, (- x * x - y * y, [x, y]))
            if len(maxHeap) > k:
                heapq.heappop(maxHeap)

        return [pair[1] for pair in maxHeap]

class Solution:
    def kClosest(self, points: list[list[int]], k: int) -> list[list[int]]:
        def squareDist(p: list[int]) -> int:
            return p[0] * p[0] + p[1] * p[1]

        def quickSelect(l: int, r: int, k: int) -> None:
            pivot = points[r]

            nextSwapped = l
            for i in range(l, r):
                if squareDist(points[i]) <= squareDist(pivot):
                    points[nextSwapped], points[i] = points[i], points[nextSwapped]
                    nextSwapped += 1
            points[nextSwapped], points[r] = points[r], points[nextSwapped]

            count = nextSwapped - l + 1  # the number of points <= pivot
            if count == k:
                return
            if count > k:
                quickSelect(l, nextSwapped - 1, k)
```

Round 5 · Mid · Medium

p.2102

```

else:
    quickSelect(nextSwapped + 1, r, k - count)

quickSelect(0, len(points) - 1, k)
return points[0:k]

```

• LeetCode

```

class Solution:
    def kClosest(self, points: List[List[int]], k: int) -> List[List[int]]:
        points.sort(key=lambda p: hypot(p[0], p[1]))
        return points[:k]

```

• SimplyLeet

```

# Python Solution
def kClosest(points, k):
    # Sort the points based on their squared Euclidean distance from the origin.
    points.sort(key=lambda point: point[0]**2 + point[1]**2)
    # Return the first k points from the sorted list.
    return points[:k]

# Example usage:
# points = [[1,3], [-2,2]]
# k = 1

# print(kClosest(points, k))

```

• LC.ca

```

class Solution:
    def kClosest(self, points: List[List[int]], k: int) -> List[List[int]]:
        points.sort(key=lambda p: p[0] * p[0] + p[1] * p[1])
        return points[:k]

```

◆ Quick Review

Key Insights

Round 5 · Mid · Medium

p.2103

- Calculate the squared Euclidean distance ($x^2 + y^2$) to avoid the overhead of square root computation.
- Sorting the points based on their distance gives an $O(n \log n)$ solution.
- Alternatively, using a max-heap to keep track of the k closest points results in $O(n \log k)$ time, which is effective if k is much smaller than n .
- Using Quickselect can achieve an average time complexity of $O(n)$, though the worst-case is $O(n^2)$.
- The problem does not require the output to be sorted in any specific order.

Space & Time

Time Complexity: $O(n \log n)$ with a sorting approach, $O(n \log k)$ with a heap approach, or average $O(n)$ with a Quickselect approach.

Space Complexity: $O(n)$ if extra space is used for sorting, or $O(k)$ for the heap approach.

Solution

We can solve this problem by first computing the squared Euclidean distance for each point to avoid unnecessary square root operations.

Round 5 · Mid · Medium

p.2104

Then, we can sort the points based on their computed distance and select the first k points. Alternatively, a max-heap or Quickselect algorithm can be used:

- Max-Heap: Maintain a heap of size k while iterating through the points, ensuring that only the k points with the smallest distances are kept.
- Quickselect: Partition the array like in QuickSort to position the kth closest point in its correct position. Once partitioned, all points to the left of that position will be the k closest. The sorting solution is straightforward and efficient enough given the constraints.

Round 5 · Mid · Medium

p.2105

Intervals

Round 5 · Mid · Medium · 12 questions

Round 5 · Mid · Medium

p.2106

Intervals

#56 Merge Intervals

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Sorting
 Lists: Grind 169

Problem

Given an array of intervals where $\text{intervals}[i] = [\text{start}_i, \text{end}_i]$, merge all overlapping intervals, and return an array of the non-overlapping intervals that cover all the intervals in the input.

Example 1:

Input: $\text{intervals} = [[1,3],[2,6],[8,10],[15,18]]$
 Output: $[[1,6],[8,10],[15,18]]$
 Explanation: Since intervals $[1,3]$ and $[2,6]$ overlap, merge them into $[1,6]$.

Example 2:

Input: $\text{intervals} = [[1,4],[4,5]]$
 Output: $[[1,5]]$
 Explanation: Intervals $[1,4]$ and $[4,5]$ are considered overlapping.

Example 3:

Input: $\text{intervals} = [[4,7],[1,4]]$
 Output: $[[1,7]]$
 Explanation: Intervals $[1,4]$ and $[4,7]$ are considered overlapping.

Constraints:

- $1 \leq \text{intervals.length} \leq 104$

Round 5 · Mid · Medium

p.2107

- $\text{intervals}[i].\text{length} == 2$
- $0 \leq \text{start}_i \leq \text{end}_i \leq 104$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def merge(self, intervals: List[List[int]]) -> List[List[int]]:
        ans = []

        for interval in sorted(intervals):
            if not ans or ans[-1][1] < interval[0]:
                ans.append(interval)
            else:
                ans[-1][1] = max(ans[-1][1], interval[1])

        return ans
```

• LeetDoocs

```
class Solution:
    def merge(self, intervals: List[List[int]]) -> List[List[int]]:
        intervals.sort()
        ans = []
        st, ed = intervals[0]
        for s, e in intervals[1:]:
            if ed < s:
                ans.append([st, ed])
                st, ed = s, e
            else:
                ed = max(ed, e)
        ans.append([st, ed])
        return ans
```

• SimplyLeet

Round 5 · Mid · Medium

p.2108

```
def merge(intervals):
    # Edge case: if the list is empty, return an empty list
    if not intervals:
        return []
    # Sort intervals based on the starting time
    intervals.sort(key=lambda x: x[0])
    merged = [] # List to store merged intervals

    # Iterate over each interval in the sorted list
    for interval in intervals:

        # If merged list is not empty and the current interval
        # overlaps with the last one in merged, merge them
        if merged and interval[0] <= merged[-1][1]:
            # Update the end of the last interval in merged if needed
            merged[-1][1] = max(merged[-1][1], interval[1])
        else:
            # No overlap, so add the current interval to merged
            merged.append(interval)
    return merged

# Example usage:
print(merge([[1,3],[2,6],[8,10],[15,18]]))
```

• LC.ca

```
# Definition for an interval.
# class Interval(object):
# def __init__(self, s=0, e=0):
# self.start = s
# self.end = e

class Solution(object):
    def merge(self, intervals):
        """
        :type intervals: List[Interval]
```

```
:rtype: List[Interval]
"""
ans = []
for intv in sorted(intervals, key=lambda x: x.start):
    if ans and ans[-1].end >= intv.start:
        ans[-1].end = max(ans[-1].end, intv.end)
    else:
        ans.append(intv)
return ans

#####

'''
>>> intervals = [[111,222],[1,3],[2,6],[8,10],[15,18]]
>>> intervals.sort()
>>> intervals
[[1, 3], [2, 6], [8, 10], [15, 18], [111, 222]]
'''

class Solution:
    def merge(self, intervals: List[List[int]]) -> List[List[int]]:

        intervals.sort() # default sort also ok
        ans = [intervals[0]]
        for s, e in intervals[1:]:
            if ans[-1][1] < s:
                ans.append([s, e])
            else:
                ans[-1][1] = max(ans[-1][1], e)
        return ans
```

◆ Quick Review

Key Insights

- Sort the intervals based on their start values.
- Use a pointer or iteration to compare the current interval with the last merged interval.

- If the current interval overlaps with the previous one, merge them by updating the end time.
- Otherwise, add the current interval to the results as a new merged interval.

Space & Time

Time Complexity: $O(n \log n)$ due to sorting the intervals.

Space Complexity: $O(n)$ to store the merged intervals.

Solution

First, sort the array of intervals based on the starting time. Then, iterate through the sorted list and check if the current interval overlaps with the last interval in the merged list. If an overlap is detected (i.e., the start of the current interval is less than or equal to the end of the last merged interval), update the last merged interval's end to be the maximum of both ends. If there is no overlap, simply add the current interval to the merged list. This approach efficiently merges overlapping intervals and ensures that all intervals are processed in a single pass after sorting.

Round 5 · Mid · Medium

p.2111

Intervals

□ #57 Insert Interval

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode Array

Lists: Grind 169

Problem

You are given an array of non-overlapping intervals intervals where $\text{intervals}[i] = [\text{start}_i, \text{end}_i]$ represent the start and the end of the i th interval and intervals is sorted in ascending order by start_i . You are also given an interval $\text{newInterval} = [\text{start}, \text{end}]$ that represents the start and end of another interval.

Insert newInterval into intervals such that intervals is still sorted in ascending order by start_i and intervals still does not have any overlapping intervals (merge overlapping intervals if necessary).

Return intervals after the insertion.

Note that you don't need to modify intervals in-place. You can make a new array and return it.

Round 5 · Mid · Medium

p.2112

Example 1:

Input: intervals = [[1,3],[6,9]], newInterval = [2,5]
 Output: [[1,5],[6,9]]

Example 2:

Input: intervals = [[1,2],[3,5],[6,7],[8,10],[12,16]], newInterval = [4,8]
 Output: [[1,2],[3,10],[12,16]]
 Explanation: Because the new interval [4,8] overlaps with [3,5],[6,7],[8,10].

Constraints:

- $0 \leq \text{intervals.length} \leq 104$
- $\text{intervals}[i].\text{length} == 2$
- $0 \leq \text{start}_i \leq \text{end}_i \leq 105$
- intervals is sorted by start_i in ascending order.
- $\text{newInterval.length} == 2$
- $0 \leq \text{start} \leq \text{end} \leq 105$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def insert(self, intervals: list[list[int]],
               newInterval: list[int]) -> list[list[int]]:
        n = len(intervals)
        ans = []
        i = 0

        while i < n and intervals[i][1] < newInterval[0]:
            ans.append(intervals[i])
            i += 1
```

```
# Merge overlapping intervals.
while i < n and intervals[i][0] <= newInterval[1]:
    newInterval[0] = min(newInterval[0], intervals[i][0])
    newInterval[1] = max(newInterval[1], intervals[i][1])
    i += 1

ans.append(newInterval)

while i < n:
    ans.append(intervals[i])
    i += 1

return ans
```

• LeetCode

```
class Solution:
    def insert(
        self, intervals: List[List[int]], newInterval: List[int]
    ) -> List[List[int]]:
        def merge(intervals: List[List[int]]) -> List[List[int]]:
            intervals.sort()
            ans = [intervals[0]]
            for s, e in intervals[1:]:
                if ans[-1][1] < s:
                    ans.append([s, e])
                else:
                    ans[-1][1] = max(ans[-1][1], e)
            return ans

        intervals.append(newInterval)
        return merge(intervals)
```

• SimplyLeet

Python solution for Insert Interval

```
class Solution:
    def insert(self, intervals, newInterval):
        # List to store the resulting intervals
        merged_intervals = []
        i = 0
        n = len(intervals)

        # Add all intervals that end before newInterval starts

        while i < n and intervals[i][1] < newInterval[0]:
            merged_intervals.append(intervals[i])
            i += 1

        # Merge overlapping intervals with newInterval
        while i < n and intervals[i][0] <= newInterval[1]:
            # Update the start and end of newInterval by merging with
            overlapping intervals
            newInterval[0] = min(newInterval[0], intervals[i][0])
            newInterval[1] = max(newInterval[1], intervals[i][1])
            i += 1

        # Add the merged newInterval
        merged_intervals.append(newInterval)

        # Add any remaining intervals that come after newInterval
        while i < n:
            merged_intervals.append(intervals[i])
            i += 1

        return merged_intervals
```

• LC.ca

Round 5 · Mid · Medium

p.2115

```
'''
>>> a = [1,2,3,4,5]
>>> a[1]
2
>>> a[~1]
4
'''

class Solution:
    def insert(self, intervals: List[List[int]], newInterval: List[int]) -> List[List[int]]:
        start = newInterval[0]

        end = newInterval[1]

        left = list(filter(lambda x: x[1] < start, intervals)) # cast via list()
        right = list(filter(lambda x: x[0] > end, intervals))

        if left + right != intervals:
            start = min(start, intervals[len(left)][0]) # note, left not -1, because index starts at 0
            end = max(end, intervals[~len(right)][1]) # same, starting at right with index=0

        return left + [[start, end]] + right

#####

class Solution: # re-use Leetcode-56's merge solution
    def insert(self, intervals: List[List[int]], newInterval: List[int]) -> List[List[int]]:
        intervals.append(newInterval)
        return self.merge(intervals)

    def merge(self, intervals: List[List[int]]) -> List[List[int]]:
        ans = []
```

Round 5 · Mid · Medium

p.2116

```

for intv in sorted(intervals, key=lambda x: x[0]):
    if ans and ans[-1][1] >= intv[0]:
        ans[-1][1] = max(ans[-1][1], intv[1])
    else:
        ans.append(intv)
return ans

```

```
#####
```

```
class Solution:
```

```

def insert(
    self, intervals: List[List[int]], newInterval: List[int]
) -> List[List[int]]:
    def merge(intervals: List[List[int]]) -> List[List[int]]:
        intervals.sort()
        ans = [intervals[0]]
        for s, e in intervals[1:]:
            if ans[-1][1] < s:
                ans.append([s, e])
            else:

```

```

                ans[-1][1] = max(ans[-1][1], e)
        return ans

```

```

intervals.append(newInterval)
return merge(intervals)

```

◆ Quick Review

Key Insights

- The intervals array is sorted by start times.
- Three main steps are used:
- Add intervals that come completely before the new interval.

- Merge overlapping intervals with the new interval.
- Append intervals that start after the new (merged) interval.
- Iterating through the list once allows an $O(n)$ solution.

Space & Time

Time Complexity: $O(n)$ since we scan the list once.

Space Complexity: $O(n)$ to store the resulting list of intervals.

Solution

Traverse the intervals list and perform the following:

- Append all intervals that end before the new interval starts.
- For intervals that overlap with newInterval, update newInterval to merge them (using min for start and max for end).
- Append the new merged interval.
- Append all remaining intervals. This ensures that all intervals remain non-overlapping and sorted with a single pass.

Intervals

★ #253 Meeting Rooms II ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Two Pointers · Greedy · Sorting · Heap
 Lists: ★ Premium | Grind 169 | Premium 98

Problem

Given an array of meeting time intervals intervals where intervals[i] = [starti, endi], return the minimum number of conference rooms required.

Example 1:

Input: intervals = [[0,30],[5,10],[15,20]]
 Output: 2

Example 2:

Input: intervals = [[7,10],[2,4]]
 Output: 1

Constraints:

- $1 \leq \text{intervals.length} \leq 104$
- $0 \leq \text{starti} < \text{endi} \leq 106$

Community Solutions (Python)

- WalkCC

Round 5 · Mid · Medium

p.2119

```
class Solution:
    def minMeetingRooms(self, intervals: list[list[int]]) -> int:
        minHeap = [] # Store the end times of each room.

        for start, end in sorted(intervals):
            # There's no overlap, so we can reuse the same room.
            if minHeap and start >= minHeap[0]:
                heapq.heappop(minHeap)
            heapq.heappush(minHeap, end)

        return len(minHeap)

class Solution:
    def minMeetingRooms(self, intervals: list[list[int]]) -> int:
        n = len(intervals)
        ans = 0
        starts = []
        ends = []

        for start, end in intervals:
            starts.append(start)
            ends.append(end)

        starts.sort()
        ends.sort()

        j = 0
        for i in range(n):
            if starts[i] < ends[j]:
                ans += 1
            else:
                j += 1

        return ans
```

- LeetDoocs

Round 5 · Mid · Medium

p.2120


```
class Solution:
    def minMeetingRooms(self, intervals: List[List[int]]) -> int:
        m = max(e[1] for e in intervals)
        d = [0] * (m + 1)
        for l, r in intervals:
            d[l] += 1
            d[r] -= 1
        ans = s = 0
        for v in d:
            s += v

            ans = max(ans, s)
        return ans
```

```
class Solution:
    def minMeetingRooms(self, intervals: List[List[int]]) -> int:
        d = defaultdict(int)
        for l, r in intervals:
            d[l] += 1
            d[r] -= 1
        ans = s = 0
        for _, v in sorted(d.items()):
            s += v
            ans = max(ans, s)

        return ans
```

• SimplyLeet

```
import heapq

def minMeetingRooms(intervals):
    # Sort intervals based on start time
    intervals.sort(key=lambda x: x[0])

    # Initialize a min-heap to store end times of meetings that are currently
    # using rooms
    min_heap = []

    # Process each meeting interval
```

Round 5 · Mid · Medium

p.2121

```
for interval in intervals:
    start, end = interval

    # If the heap is not empty and the current meeting starts after the
    # earliest ended meeting
    if min_heap and start >= min_heap[0]:
        heapq.heappop(min_heap) # Reuse the room

    # Allocate the current meeting end time into the heap (new or reused
    # room)
    heapq.heappush(min_heap, end)

# The number of rooms is the number of simultaneous meetings
return len(min_heap)

# Example usage:
intervals = [[0,30],[5,10],[15,20]]
print(minMeetingRooms(intervals)) # Output: 2
```

• LC.ca

```
'''
data = [3, 4, 6, 2, 1, 9, 0, 7, 5, 8]

list(accumulate(data, operator.mul)) # running product
[3, 12, 72, 144, 144, 1296, 0, 0, 0, 0]

list(accumulate(data, max)) # running maximum
[3, 4, 6, 6, 6, 6, 9, 9, 9, 9]
```

```
>>> from itertools import accumulate
>>> data = [3, 4, 6, 2, 1, 9, 0, 7, 5, 8]
>>> accumulate(data)
<itertools.accumulate object at 0x10fd78440>
>>> list(accumulate(data))
[3, 7, 13, 15, 16, 25, 25, 32, 37, 45]
>>> max(accumulate(data))
45

https://docs.python.org/3/library/itertools.html#itertools.accumulate
```

Round 5 · Mid · Medium

p.2122

```
# Amortize a 5% loan of 1000 with 4 annual payments of 90
cashflows = [1000, -90, -90, -90, -90]
list(accumulate(cashflows, lambda bal, pmt: bal*1.05 + pmt))
[1000, 960.0, 918.0, 873.9000000000001, 827.5950000000001]
'''

class Solution:
    def minMeetingRooms(self, intervals: List[List[int]]) -> int:
        delta = [0] * 1000010

        for start, end in intervals:
            delta[start] += 1
            delta[end] -= 1
        return max(accumulate(delta))
# why not delta.sort()?
# because accumulate() will go by order from index 0 to index final
# just like, from sortedcontainers import SortedDict

#####

class Solution(object):
    def minMeetingRooms(self, intervals):
        """
        :type intervals: List[Interval]
        :rtype: int
        """
        meetings = []
        for i in intervals:
            meetings.append((i.start, 1))
            meetings.append((i.end, 0))

        meetings.sort()
        ans = 0
        count = 0
        for meeting in meetings:
            if meeting[1] == 1:
                count += 1
            else:
                count -= 1
            ans = max(ans, count)
        return ans
```

◆ Quick Review

Key Insights

- Sort the intervals based on start times to process meetings in order.
- Use a min-heap (priority queue) to keep track of meeting end times.
- When processing a new meeting, check if its start time is greater than or equal to the smallest end time from the heap.
- If so, it means a meeting room has been freed (the meeting with the smallest end time has ended) and can be reused.
- If not, allocate a new room by pushing the current meeting's end time into the heap.
- The size of the heap at any moment represents the number of rooms currently in use.

Space & Time

Time Complexity: $O(n \log n)$ due to sorting and each heap operation (push/pop) taking $O(\log n)$.

Space Complexity: $O(n)$ in the worst-case scenario when all meetings overlap, requiring all end times in the heap.

Solution

The solution involves first sorting the meeting intervals by their start times. Then, we utilize a min-heap (priority queue) to keep track of the end times of ongoing meetings. For each meeting, we compare its start time with the smallest end time in the heap:

- If the meeting can start after the meeting at the top of the heap has ended (i.e., its start time is greater than or equal to the smallest end), we remove that end time from the heap (freeing up a room).
- Then, we add the current meeting's end time to the heap. By the end of processing all meetings, the size of the heap represents the minimum number of conference rooms required.

Intervals

□ #435 Non-overlapping Intervals

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode Array · Dynamic Programming · Greedy · Sorting

Lists: Grind 169

Problem

Given an array of intervals `intervals` where `intervals[i] = [starti, endi]`, return the minimum number of intervals you need to remove to make the rest of the intervals non-overlapping.

Note that intervals which only touch at a point are non-overlapping. For example, `[1, 2]` and `[2, 3]` are non-overlapping.

Example 1:

Input: `intervals = [[1,2],[2,3],[3,4],[1,3]]`
 Output: 1
 Explanation: `[1,3]` can be removed and the rest of the intervals are non-overlapping.

Example 2:

Input: `intervals = [[1,2],[1,2],[1,2]]`
 Output: 2
 Explanation: You need to remove two `[1,2]` to make the rest of the intervals non-overlapping.

Example 3:

Input: intervals = [[1,2],[2,3]]

Output: 0

Explanation: You don't need to remove any of the intervals since they're already non-overlapping.

Constraints:

- $1 \leq \text{intervals.length} \leq 105$
- $\text{intervals}[i].\text{length} == 2$
- $-5 * 10^4 \leq \text{start}_i < \text{end}_i \leq 5 * 10^4$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def eraseOverlapIntervals(self, intervals: list[list[int]]) -> int:
        ans = 0
        currentEnd = -math.inf

        for interval in sorted(intervals, key=lambda x: x[1]):
            if interval[0] >= currentEnd:
                currentEnd = interval[1]
            else:
                ans += 1

        return ans
```

• LeetCode

```
class Solution:
    def eraseOverlapIntervals(self, intervals: List[List[int]]) -> int:
        intervals.sort(key=lambda x: x[1])
        ans = len(intervals)
        pre = -inf
        for l, r in intervals:
            if pre <= l:
                ans -= 1
                pre = r
        return ans
```

• SimplyLeet

```
# Python implementation for Non-overlapping Intervals

def eraseOverlapIntervals(intervals):
    # Sort intervals based on the end time
    intervals.sort(key=lambda x: x[1])

    # Initialize the count of intervals in the non-overlapping set
    count = 0
    # Initialize the end of the last added interval to the minimum possible
    # value
    last_included_end = float('-inf')

    # Iterate over every interval in the sorted list
    for interval in intervals:
        # If the current interval does not overlap with the last added interval
        if interval[0] >= last_included_end:
            count += 1          # We include this interval
            last_included_end = interval[1] # Update end for subsequent
            comparisons
        # The minimum number of removals is total intervals minus non-overlapping
        # intervals count
    return len(intervals) - count

# Example usage:
print(eraseOverlapIntervals([[1,2],[2,3],[3,4],[1,3]])) # Expected output: 1
```

• LC.ca

```
class Solution:
    def eraseOverlapIntervals(self, intervals: List[List[int]]) -> int:
        intervals.sort(key=lambda x: x[1])
        ans, t = 0, intervals[0][1]
        for s, e in intervals[1:]:
            if s >= t:
                t = e
            else:
                ans += 1
        return ans
```

◆ Quick Review

Key Insights

- **Sorting is the key:** By sorting intervals by their end times, we can greedily choose the optimal set of non-overlapping intervals.
- **Greedy approach:** Always select the interval that ends the earliest, as it leaves maximum room for subsequent intervals.
- **Counting removals:** The answer is the total number of intervals minus the count of intervals selected by this greedy approach.
- **Edge observation:** Intervals that touch at the boundary are not overlapping, so the check uses “>=” instead of “>”.

Space & Time

Round 5 · Mid · Medium

p.2129

Time Complexity: $O(n \log n)$ due to sorting the intervals.

Space Complexity: $O(n)$ if considering the storage for the sorted list (or $O(1)$ if sorting in-place).

Solution

The solution involves first sorting the intervals by their end times. Initialize a variable to track the end of the last chosen interval. Iterate over the sorted intervals: if the current interval's start is greater than or equal to the last chosen interval's end, update the last end and include this interval in the non-overlapping set. Otherwise, count the interval as one that must be removed. The final answer is the removal count, which is computed as the total number of intervals minus the count of non-overlapping intervals selected.

Round 5 · Mid · Medium

p.2130

Intervals

#452 Minimum Number of Arrows to Burst Balloons

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Greedy · Sorting
 Lists: AlgoMaster 600

Problem

There are some spherical balloons taped onto a flat wall that represents the XY-plane. The balloons are represented as a 2D integer array `points` where `points[i] = [xstart, xend]` denotes a balloon whose horizontal diameter stretches between `xstart` and `xend`. You do not know the exact y-coordinates of the balloons.

Arrows can be shot up directly vertically (in the positive y-direction) from different points along the x-axis. A balloon with `xstart` and `xend` is burst by an arrow shot at `x` if `xstart ≤ x ≤ xend`. There is no limit to the number of arrows that can be shot. A shot arrow keeps traveling up infinitely, bursting any balloons in its path.

Round 5 · Mid · Medium

p.2131

Given the array `points`, return the minimum number of arrows that must be shot to burst all balloons.

Example 1:

```
Input: points = [[10,16],[2,8],[1,6],[7,12]]
Output: 2
Explanation: The balloons can be burst by 2 arrows:
- Shoot an arrow at x = 6, bursting the balloons [2,8] and [1,6].
- Shoot an arrow at x = 11, bursting the balloons [10,16] and [7,12].
```

Example 2:

```
Input: points = [[1,2],[3,4],[5,6],[7,8]]
Output: 4
Explanation: One arrow needs to be shot for each balloon for a total of 4 arrows.
```

Example 3:

```
Input: points = [[1,2],[2,3],[3,4],[4,5]]
Output: 2
Explanation: The balloons can be burst by 2 arrows:
- Shoot an arrow at x = 2, bursting the balloons [1,2] and [2,3].
- Shoot an arrow at x = 4, bursting the balloons [3,4] and [4,5].
```

Constraints:

- $1 \leq \text{points.length} \leq 105$
- $\text{points}[i].\text{length} == 2$
- $-231 \leq \text{xstart} < \text{xend} \leq 231 - 1$

Community Solutions (Python)

- WalkCC

Round 5 · Mid · Medium

p.2132

```
class Solution:
    def findMinArrowShots(self, points: list[list[int]]) -> int:
        ans = 0
        arrowX = -math.inf

        for point in sorted(points, key=lambda x: x[1]):
            if point[0] > arrowX:
                ans += 1
                arrowX = point[1]

        return ans
```

• LeetCode

```
class Solution:
    def findMinArrowShots(self, points: List[List[int]]) -> int:
        ans, last = 0, -inf
        for a, b in sorted(points, key=lambda x: x[1]):
            if a > last:
                ans += 1
                last = b
        return ans
```

• SimplyLeet

```
def findMinArrowShots(points):
    # If there are no balloons, return 0
    if not points:
        return 0

    # Sort the balloons by their end coordinate
    points.sort(key=lambda x: x[1])

    # Start with one arrow at the end of the first balloon
    arrows = 1
```

```
current_arrow_position = points[0][1]

# Traverse through each balloon interval
for x_start, x_end in points:
    # Only shoot a new arrow if the current balloon is not covered by the
    last arrow
    if x_start > current_arrow_position:
        arrows += 1 # Increment the arrow count
        current_arrow_position = x_end # Update the arrow position

return arrows

# Example usage:
print(findMinArrowShots([[10,16],[2,8],[1,6],[7,12]])) # Output: 2
```

Intervals

#763 Partition Labels

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Two Pointers · String · Greedy
Lists: NeetCode 150

Problem

You are given a string *s*. We want to partition the string into as many parts as possible so that each letter appears in at most one part. For example, the string "ababcc" can be partitioned into ["abab", "cc"], but partitions such as ["aba", "bcc"] or ["ab", "ab", "cc"] are invalid.

Note that the partition is done so that after concatenating all the parts in order, the resultant string should be *s*.

Return a list of integers representing the size of these parts.

Example 1:

Input: *s* = "ababcbacadefegdehijhklij"

Output: [9,7,8]

Explanation:

The partition is "ababcbaca", "defegde", "hijhklij".

This is a partition so that each letter appears in at most one part.

A partition like "ababcbacadefegde", "hijhklij" is incorrect, because it splits *s* into less parts.

Example 2:

Input: *s* = "eccbbbbbdec"

Output: [10]

Constraints:

- $1 \leq s.length \leq 500$
- *s* consists of lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def partitionLabels(self, s: str) -> list[int]:
        ans = []
        letterToRightmostIndex = {c: i for i, c in enumerate(s)}

        l = 0 # the leftmost index of the current running string
        r = 0 # the rightmost index of the current running string

        for i, c in enumerate(s):
            r = max(r, letterToRightmostIndex[c])

            if i == r:
                ans.append(r - l + 1)
                l = r + 1

        return ans
```

• LeetDoocs


```
class Solution:
    def partitionLabels(self, s: str) -> List[int]:
        last = {c: i for i, c in enumerate(s)}
        mx = j = 0
        ans = []
        for i, c in enumerate(s):
            mx = max(mx, last[c])
            if mx == i:
                ans.append(i - j + 1)
                j = i + 1
        return ans
```

• SimplyLeet

```
# Python Solution
class Solution:
    def partitionLabels(self, s: str) -> list:
        # Step 1: Record the last occurrence of each character.
        last_occurrence = {char: i for i, char in enumerate(s)}
        # List to store the sizes of partitions.
        partitions = []
        start, end = 0, 0

        # Step 2: Iterate over the string to form partitions.

        for i, char in enumerate(s):
            # Update the end index for the current partition.
            end = max(end, last_occurrence[char])
            # When we reach the end of the partition.
            if i == end:
                partitions.append(end - start + 1)
                start = i + 1 # Move start to the next index.

        return partitions
```

Intervals

□ #986 Interval List Intersections

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Two Pointers
 Lists: Denny Zhang

Problem

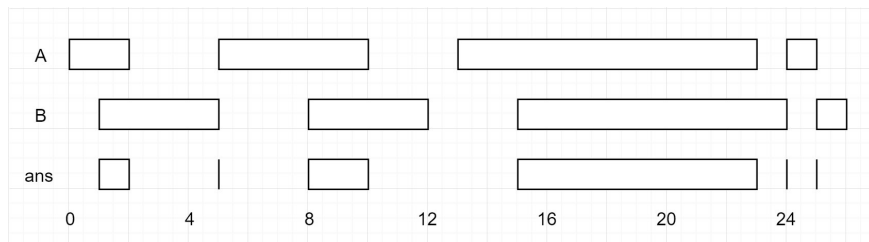
You are given two lists of closed intervals, firstList and secondList, where firstList[i] = [starti, endi] and secondList[j] = [startj, endj]. Each list of intervals is pairwise disjoint and in sorted order.

Return the intersection of these two interval lists.

A closed interval [a, b] (with a ≤ b) denotes the set of real numbers x with a ≤ x ≤ b.

The intersection of two closed intervals is a set of real numbers that are either empty or represented as a closed interval. For example, the intersection of [1, 3] and [2, 4] is [2, 3].

Example 1:



Input: firstList = [[0,2],[5,10],[13,23],[24,25]], secondList = [[1,5],[8,12],[15,24],[25,26]]
 Output: [[1,2],[5,5],[8,10],[15,23],[24,24],[25,25]]

Example 2:

Input: firstList = [[1,3],[5,9]], secondList = []
 Output: []

Constraints:

- $0 \leq \text{firstList.length}, \text{secondList.length} \leq 1000$
- $\text{firstList.length} + \text{secondList.length} \geq 1$
- $0 \leq \text{start}_i < \text{end}_i \leq 109$
- $\text{end}_i < \text{start}_{i+1}$
- $0 \leq \text{start}_j < \text{end}_j \leq 109$
- $\text{end}_j < \text{start}_{j+1}$

Community Solutions (Python)

- WalkCC

```
class Solution:
    def intervalIntersection(self, firstList: list[list[int]],
                           secondList: list[list[int]]) -> list[list[int]]:

        ans = []
        i = 0
        j = 0

        while i < len(firstList) and j < len(secondList):
            # lo := the start of the intersection
            # hi := the end of the intersection

            lo = max(firstList[i][0], secondList[j][0])
            hi = min(firstList[i][1], secondList[j][1])
            if lo <= hi:
                ans.append([lo, hi])
            if firstList[i][1] < secondList[j][1]:
                i += 1
            else:
                j += 1

        return ans
```

• LeetDoocs

```
class Solution:
    def intervalIntersection(
        self, firstList: List[List[int]], secondList: List[List[int]]
    ) -> List[List[int]]:
        i = j = 0
        ans = []
        while i < len(firstList) and j < len(secondList):
            s1, e1, s2, e2 = *firstList[i], *secondList[j]
            l, r = max(s1, s2), min(e1, e2)
            if l <= r:
                ans.append([l, r])
            if e1 < e2:
                i += 1
            else:
                j += 1
        return ans
```

• SimplyLeet

```
# Function to compute the interval intersections using two pointers
def intervalIntersection(firstList, secondList):
    intersections = [] # List to store the resulting intersections
    i, j = 0, 0 # Pointers for firstList and secondList

    # Loop until either list is fully traversed
    while i < len(firstList) and j < len(secondList):
        start1, end1 = firstList[i]
        start2, end2 = secondList[j]

        # Find the intersection boundaries
        start_overlap = max(start1, start2)
        end_overlap = min(end1, end2)

        # Check if there is an intersection
        if start_overlap <= end_overlap:
            intersections.append([start_overlap, end_overlap])

        # Move the pointer that has the smaller interval ending
        if end1 < end2:
            i += 1
        else:
            j += 1

    return intersections

# Example usage:
firstList = [[0,2],[5,10],[13,23],[24,25]]
secondList = [[1,5],[8,12],[15,24],[25,26]]
print(intervalIntersection(firstList, secondList)) # Output:
[[1,2],[5,5],[8,10],[15,23],[24,24],[25,25]]
```

• LC.ca

Round 5 · Mid · Medium

p.2141

```
class Solution:
    def intervalIntersection(
        self, firstList: List[List[int]], secondList: List[List[int]]
    ) -> List[List[int]]:
        i = j = 0
        ans = []
        while i < len(firstList) and j < len(secondList):
            s1, e1, s2, e2 = *firstList[i], *secondList[j]
            l, r = max(s1, s2), min(e1, e2)
            if l <= r:
                ans.append([l, r])
                if e1 < e2:
                    i += 1
                else:
                    j += 1
        return ans
```

◆ Quick Review

Key Insights

- Use two pointers to traverse both lists simultaneously.
- Determine the overlapping interval by taking the maximum of the start points and the minimum of the end points.
- If the overlapping condition ($\text{start} \leq \text{end}$) is met, add the intersection to the result.
- Move the pointer that has the smaller end value to explore further potential intersections efficiently.

Round 5 · Mid · Medium

p.2142

- Achieve an overall time complexity of $O(m+n)$, where m and n are the lengths of the two interval lists.

Space & Time

Time Complexity: $O(m + n)$ where m and n are the lengths of `firstList` and `secondList` respectively.

Space Complexity: $O(1)$ extra space (excluding the space used for the output).

Solution

The solution uses a two pointers approach. Initialize two indices, one for each list, and iterate through both lists. For each pair of intervals, the potential intersection is computed by:

- Calculating the maximum of the two start values.
- Calculating the minimum of the two end values.

If the maximum start is less than or equal to the minimum end, there is an intersection, and it is added to the result list. The pointer corresponding to the interval with the smaller end value is then incremented since that interval cannot intersect with any further intervals from the other list. This

process repeats until one of the lists is fully traversed.

Intervals

#1094 Car Pooling

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Sorting · Heap (Priority Queue) ·
 Simulation · Prefix Sum
 Lists: AlgoMaster 600

Problem

There is a car with capacity empty seats. The vehicle only drives east (i.e., it cannot turn around and drive west).

You are given the integer capacity and an array trips where $\text{trips}[i] = [\text{numPassengers}_i, \text{from}_i, \text{to}_i]$ indicates that the i th trip has numPassengers_i passengers and the locations to pick them up and drop them off are from_i and to_i respectively. The locations are given as the number of kilometers due east from the car's initial location.

Return true if it is possible to pick up and drop off all passengers for all the given trips, or false otherwise.

Example 1:

Round 5 · Mid · Medium

p.2145

Input: trips = [[2,1,5],[3,3,7]], capacity = 4
 Output: false

Example 2:

Input: trips = [[2,1,5],[3,3,7]], capacity = 5
 Output: true

Constraints:

- $1 \leq \text{trips.length} \leq 1000$
- $\text{trips}[i].\text{length} == 3$
- $1 \leq \text{numPassengers}_i \leq 100$
- $0 \leq \text{from}_i < \text{to}_i \leq 1000$
- $1 \leq \text{capacity} \leq 105$

Community Solutions (Python)

• LeetDoocs

```
class Solution:
    def carPooling(self, trips: List[List[int]], capacity: int) -> bool:
        mx = max(e[2] for e in trips)
        d = [0] * (mx + 1)
        for x, f, t in trips:
            d[f] += x
            d[t] -= x
        return all(s <= capacity for s in accumulate(d))
```

• SimplyLeet

Round 5 · Mid · Medium

p.2146

```
# Python solution with detailed comments

def carPooling(trips, capacity):
    # events will store tuples (location, change_in_passengers)
    events = []

    # Create events for each trip: pickup event (+num_passengers) and drop-off
    # event (-num_passengers)
    for num_passengers, start, end in trips:
        events.append((start, num_passengers)) # When passengers get on
        events.append((end, -num_passengers))   # When passengers get off

    # Sort events by location,
    # and in case of ties, process drop-offs (-num_passengers) before pick-ups
    # (+num_passengers)
    events.sort(key=lambda x: (x[0], x[1]))

    current_passengers = 0

    # Process each event in order of location
    for location, change in events:
        current_passengers += change

    # If at any point, the number of passengers exceeds capacity, return
    # False
    if current_passengers > capacity:
        return False

    # If we've processed all events without exceeding capacity, return True
    return True

# Example usage:
print(carPooling([[2,1,5],[3,3,7]], 4)) # Expected output: False
print(carPooling([[2,1,5],[3,3,7]], 5)) # Expected output: True
```

Intervals

□ #1229 Meeting Scheduler

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Two Pointers · Sorting
 Lists: AlgoMaster 600

Problem

Given the availability time slots arrays `slots1` and `slots2` of two people and a meeting duration `duration`, return the earliest time slot that works for both of them and is of duration `duration`.

If there is no common time slot that satisfies the requirements, return an empty array.

The format of a time slot is an array of two elements `[start, end]` representing an inclusive time range from `start` to `end`.

It is guaranteed that no two availability slots of the same person intersect with each other. That is, for any two time slots `[start1, end1]` and `[start2, end2]` of the same person, either `start1 > end2` or `start2 > end1`.

Example 1:

Input: slots1 = [[10,50],[60,120],[140,210]], slots2 = [[0,15],[60,70]],
 duration = 8
 Output: [60,68]

Example 2:

Input: slots1 = [[10,50],[60,120],[140,210]], slots2 = [[0,15],[60,70]],
 duration = 12
 Output: []

Constraints:

- $1 \leq \text{slots1.length}, \text{slots2.length} \leq 104$
- $\text{slots1}[i].\text{length}, \text{slots2}[i].\text{length} == 2$
- $\text{slots1}[i][0] < \text{slots1}[i][1]$
- $\text{slots2}[i][0] < \text{slots2}[i][1]$
- $0 \leq \text{slots1}[i][j], \text{slots2}[i][j] \leq 109$
- $1 \leq \text{duration} \leq 106$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minAvailableDuration(
        self,
        slots1: List[List[int]],
        slots2: List[List[int]],
        duration: int,
    ) -> List[int]:
        slots1.sort()
        slots2.sort()
```

```
i = 0 # slots1's index
j = 0 # slots2's index

while i < len(slots1) and j < len(slots2):
    start = max(slots1[i][0], slots2[j][0])
    end = min(slots1[i][1], slots2[j][1])
    if start + duration <= end:
        return [start, start + duration]
    if slots1[i][1] < slots2[j][1]:
        i += 1
    else:
        j += 1

return []
```

• LeetCode

```
class Solution:
    def minAvailableDuration(
        self, slots1: List[List[int]], slots2: List[List[int]], duration: int
    ) -> List[int]:
        slots1.sort()
        slots2.sort()
        m, n = len(slots1), len(slots2)
        i = j = 0
        while i < m and j < n:
            start = max(slots1[i][0], slots2[j][0])

            end = min(slots1[i][1], slots2[j][1])
            if end - start >= duration:
                return [start, start + duration]
            if slots1[i][1] < slots2[j][1]:
                i += 1
            else:
                j += 1
        return []
```

• SimplyLeet

```
# Python solution for Meeting Scheduler
```

```
def meeting_scheduler(slots1, slots2, duration):
```

```
    # Sort both lists based on start time
```

```
    slots1.sort(key=lambda x: x[0])
```

```
    slots2.sort(key=lambda x: x[0])
```

```
    # Initialize two pointers for both lists
```

```
    i, j = 0, 0
```

```
    # Iterate through both lists until one pointer reaches the end
```

```
    while i < len(slots1) and j < len(slots2):
```

```
        # Calculate the latest start and earliest end of the two slots
```

```
        start = max(slots1[i][0], slots2[j][0])
```

```
        end = min(slots1[i][1], slots2[j][1])
```

```
        # Check if overlapping duration is enough for the meeting
```

```
        if end - start >= duration:
```

```
            return [start, start + duration]
```

```
    # Move the pointer that has the earlier end time
```

```
    if slots1[i][1] < slots2[j][1]:
```

```
        i += 1
```

```
    else:
```

```
        j += 1
```

```
    # If no valid meeting slot is found, return an empty list
```

```
    return []
```

```
# Example usage:
```

```
# print(meeting_scheduler([[10,50],[60,120],[140,210]], [[0,15],[60,70]], 8))
```

Intervals

□ #1288 Remove Covered Intervals

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Sorting

Lists: AlgoMaster 600

Problem

Given an array intervals where intervals[i] = [li, ri] represent the interval [li, ri), remove all intervals that are covered by another interval in the list.

The interval [a, b) is covered by the interval [c, d) if and only if c ≤ a and b ≤ d.

Return the number of remaining intervals.

Example 1:

Input: intervals = [[1,4],[3,6],[2,8]]

Output: 2

Explanation: Interval [3,6] is covered by [2,8], therefore it is removed.

Example 2:

Input: intervals = [[1,4],[2,3]]

Output: 1

Constraints:

- 1 ≤ intervals.length ≤ 1000

- `intervals[i].length == 2`
- `0 <= li < ri <= 105`
- All the given intervals are unique.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def removeCoveredIntervals(self, intervals: list[list[int]]) -> int:
        ans = 0
        prevEnd = 0

        for _, end in sorted(intervals, key=lambda x: (x[0], -x[1])):
            # The current interval is not covered by the previous one.
            if prevEnd < end:
                prevEnd = end
                ans += 1

        return ans
```

• LeetDoocs

```
class Solution:
    def removeCoveredIntervals(self, intervals: List[List[int]]) -> int:
        intervals.sort(key=lambda x: (x[0], -x[1]))
        ans = 0
        pre = -inf
        for _, cur in intervals:
            if cur > pre:
                ans += 1
                pre = cur
        return ans
```

• SimplyLeet

```
# Python solution to remove covered intervals with line-by-line comments
class Solution:
    def removeCoveredIntervals(self, intervals):
        # Sort intervals by start ascending and then by end descending.
        intervals.sort(key=lambda x: (x[0], -x[1]))
        count = 0 # Initialize counter for non-covered intervals.
        max_end = 0 # Track the maximum end encountered.
```

```
        # Iterate over the sorted intervals.
        for start, end in intervals:

            # If the current interval's end is greater than the maximum end
            # seen,
            # it is not covered by any previous interval.
            if end > max_end:
                count += 1 # Increment count of valid intervals.
                max_end = end # Update the maximum end.
            # Return the count of remaining intervals.
        return count
```

```
# Example usage:
sol = Solution()
```

```
print(sol.removeCoveredIntervals([[1,4],[3,6],[2,8]]))
```

Intervals

#1353 Maximum Number of Events That Can Be Attended MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Greedy · Sorting · Heap (Priority Queue)
 Lists: AlgoMaster 600

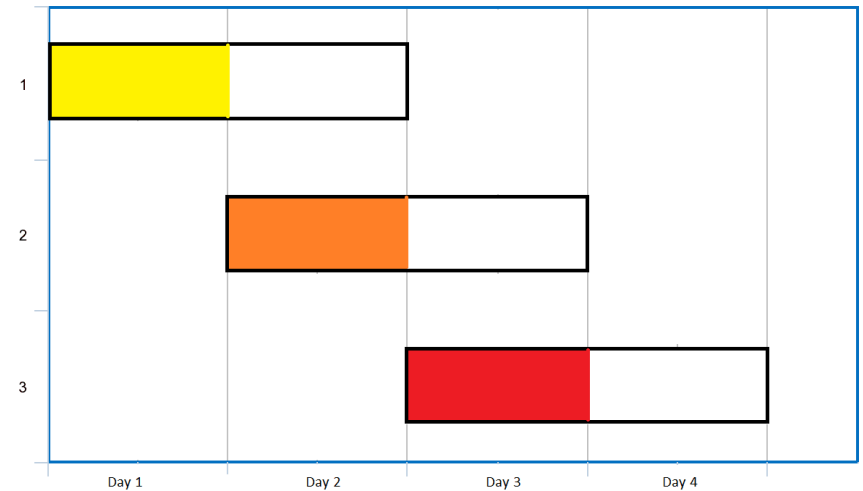
Problem

You are given an array of events where $\text{events}[i] = [\text{startDay}_i, \text{endDay}_i]$. Every event i starts at startDay_i and ends at endDay_i .

You can attend an event i at any day d where $\text{startDay}_i \leq d \leq \text{endDay}_i$. You can only attend one event at any time d .

Return the maximum number of events you can attend.

Example 1:



Input: $\text{events} = [[1,2],[2,3],[3,4]]$

Output: 3

Explanation: You can attend all the three events.

One way to attend them all is as shown.

Attend the first event on day 1.

Attend the second event on day 2.

Attend the third event on day 3.

Example 2:

Input: $\text{events} = [[1,2],[2,3],[3,4],[1,2]]$

Output: 4

Constraints:

- $1 \leq \text{events.length} \leq 105$
- $\text{events}[i].\text{length} == 2$
- $1 \leq \text{startDay}_i \leq \text{endDay}_i \leq 105$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxEvents(self, events: list[list[int]]) -> int:
        ans = 0
        minHeap = []
        i = 0 # events' index

        events.sort(key=lambda x: x[0])

        while minHeap or i < len(events):
            # If no events are available to attend today, let time flies to the next

            # available event.
            if not minHeap:
                d = events[i][0]
            # All the events starting from today are newly available.
            while i < len(events) and events[i][0] == d:
                heapq.heappush(minHeap, events[i][1])
                i += 1
            # Greedily attend the event that'll end the earliest since it has higher
            # chance can't be attended in the future.
            heapq.heappop(minHeap)

            ans += 1
            d += 1
            # Pop the events that can't be attended.
            while minHeap and minHeap[0] < d:
                heapq.heappop(minHeap)

        return ans
```

• LeetCode

```
class Solution:
    def maxEvents(self, events: List[List[int]]) -> int:
        g = defaultdict(list)
        l, r = inf, 0
        for s, e in events:
            g[s].append(e)
            l = min(l, s)
            r = max(r, e)
        pq = []
        ans = 0

        for s in range(l, r + 1):
            while pq and pq[0] < s:
                heappop(pq)
            for e in g[s]:
                heappush(pq, e)
            if pq:
                heappop(pq)
                ans += 1
        return ans
```

• SimplyLeet

```
import heapq

def maxEvents(events):
    # Sort events by start day
    events.sort(key=lambda x: x[0])
    # Initialize a min-heap and index for events
    min_heap = []
    i = 0
    total_events = 0
    n = len(events)
```

```

# Determine the last possible day to check (max end day among events)
last_day = max(event[1] for event in events)

# Iterate over each day
for day in range(1, last_day + 1):
    # Add events starting on the current day to the heap with their end
    days
    while i < n and events[i][0] == day:
        heapq.heappush(min_heap, events[i][1])
        i += 1

    # Remove events that have already ended (end day before current day)
    while min_heap and min_heap[0] < day:
        heapq.heappop(min_heap)

    # If there is an available event, attend the one ending earliest
    if min_heap:
        heapq.heappop(min_heap)
        total_events += 1

return total_events

```

```

# Example usage:
# events = [[1,2],[2,3],[3,4]]
# print(maxEvents(events)) # Expected Output: 3

```

Intervals

#2054 Two Best Non-Overlapping Events

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Binary Search · Dynamic Programming ·
 Sorting · Heap (Priority Queue)

Lists: AlgoMaster 600

Problem

You are given a 0-indexed 2D integer array of events where $\text{events}[i] = [\text{startTime}_i, \text{endTime}_i, \text{value}_i]$. The i th event starts at startTime_i and ends at endTime_i , and if you attend this event, you will receive a value of value_i . You can choose at most two non-overlapping events to attend such that the sum of their values is maximized.

Return this maximum sum.

Note that the start time and end time is inclusive: that is, you cannot attend two events where one of them starts and the other ends at the same time. More specifically, if you attend an event with end time t , the next event must

start at or after $t + 1$.

Example 1:

Time	1	2	3	4	5
Event 0	2				
Event 1				2	
Event 2		3			

Input: events = [[1,3,2],[4,5,2],[2,4,3]]

Output: 4

Explanation: Choose the green events, 0 and 1 for a sum of $2 + 2 = 4$.

Example 2:

Time	1	2	3	4	5
Event 0	2				
Event 1				2	
Event 2	5				

Input: events = [[1,3,2],[4,5,2],[1,5,5]]

Output: 5

Explanation: Choose event 2 for a sum of 5.

Example 3:

Time	1	2	3	4	5	6
Event 0	3					
Event 1	1					
Event 2						5

Input: events = [[1,5,3],[1,5,1],[6,6,5]]

Output: 8

Explanation: Choose events 0 and 2 for a sum of $3 + 5 = 8$.

Constraints:

- $2 \leq \text{events.length} \leq 105$
- $\text{events}[i].\text{length} == 3$
- $1 \leq \text{startTime}_i \leq \text{endTime}_i \leq 109$
- $1 \leq \text{value}_i \leq 106$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxTwoEvents(self, events: list[list[int]]) -> int:
        ans = 0
        maxValue = 0
        evts = [] # (time, isStart, value)

        for s, e, v in events:
            evts.append((s, 1, v))
            evts.append((e + 1, 0, v))

        # When two events have the same time, the one is not start will be in the front
        evts.sort()

        for _, isStart, value in evts:
            if isStart:
                ans = max(ans, value + maxValue)
            else:
                maxValue = max(maxValue, value)

        return ans
```

• LeetDoocs

```
class Solution:
    def maxTwoEvents(self, events: List[List[int]]) -> int:
        events.sort()
        n = len(events)
        f = [events[-1][2]] * n
        for i in range(n - 2, -1, -1):
            f[i] = max(f[i + 1], events[i][2])
        ans = 0
        for _, e, v in events:
            idx = bisect_right(events, e, key=lambda x: x[0])

            if idx < n:
                v += f[idx]
            ans = max(ans, v)
        return ans
```

• SimplyLeet

```
import bisect
from typing import List

class Solution:
    def maxTwoEvents(self, events: List[List[int]]) -> int:
        # Sort events by their start time
        events.sort(key=lambda x: x[0])
        n = len(events)
        # Extract start times to use for binary search later
        start_times = [event[0] for event in events]

        # Build a suffix max array: suffix_max[i] holds the max event value
        # from events[i] to end
        suffix_max = [0] * n
        suffix_max[-1] = events[-1][2]
        for i in range(n - 2, -1, -1):
            suffix_max[i] = max(events[i][2], suffix_max[i + 1])
        max_value = 0
        # Iterate over each event and use binary search to find the next
        # non-overlapping event
        for i in range(n):
            current_value = events[i][2]
            # Find the first event starting at or after events[i][1] + 1
```

```
next_index = bisect.bisect_left(start_times, events[i][1] + 1)
if next_index < n:
    current_value += suffix_max[next_index]
max_value = max(max_value, current_value)
return max_value
```

K-Way Merge

Round 5 · Mid · Medium · 2 questions

Round 5 · Mid · Medium

p.2165

K-Way Merge

□ #373 Find K Pairs with Smallest Sums

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Heap (Priority Queue)

Lists: AlgoMaster 600

Problem

You are given two integer arrays `nums1` and `nums2` sorted in non-decreasing order and an integer `k`.

Define a pair (u, v) which consists of one element from the first array and one element from the second array.

Return the k pairs $(u_1, v_1), (u_2, v_2), \dots, (u_k, v_k)$ with the smallest sums.

Example 1:

Input: `nums1 = [1,7,11], nums2 = [2,4,6], k = 3`
 Output: `[[1,2],[1,4],[1,6]]`
 Explanation: The first 3 pairs are returned from the sequence:
`[1,2], [1,4], [1,6], [7,2], [7,4], [11,2], [7,6], [11,4], [11,6]`

Example 2:

Input: `nums1 = [1,1,2], nums2 = [1,2,3], k = 2`
 Output: `[[1,1],[1,1]]`
 Explanation: The first 2 pairs are returned from the sequence:
`[1,1], [1,1], [1,2], [2,1], [1,2], [2,2], [1,3], [1,3], [2,3]`

Round 5 · Mid · Medium

p.2166

Constraints:

- $1 \leq \text{nums1.length}, \text{nums2.length} \leq 105$
- $-109 \leq \text{nums1}[i], \text{nums2}[i] \leq 109$
- nums1 and nums2 both are sorted in non-decreasing order.
- $1 \leq k \leq 104$
- $k \leq \text{nums1.length} * \text{nums2.length}$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def kSmallestPairs(self, nums1: list[int],
                      nums2: list[int],
                      k: int) -> list[list[int]]:
        minHeap = []

        for i in range(min(k, len(nums1))):
            heapq.heappush(minHeap, (nums1[i] + nums2[0], i, 0))

        ans = []

        while minHeap and len(ans) < k:
            _, i, j = heapq.heappop(minHeap)
            ans.append([nums1[i], nums2[j]])
            if j + 1 < len(nums2):
                heapq.heappush(minHeap, (nums1[i] + nums2[j + 1], i, j + 1))

        return ans
```

• LeetDoocs

```
class Solution:
    def kSmallestPairs(
        self, nums1: List[int], nums2: List[int], k: int
    ) -> List[List[int]]:
        q = [(u + nums2[0], i, 0) for i, u in enumerate(nums1[:k])]
        heapify(q)
        ans = []
        while q and k > 0:
            _, i, j = heappop(q)
            ans.append([nums1[i], nums2[j]])

            k -= 1
            if j + 1 < len(nums2):
                heappush(q, [nums1[i] + nums2[j + 1], i, j + 1])
        return ans
```

• SimplyLeet

```
import heapq

def kSmallestPairs(nums1, nums2, k):
    # Edge case: if either array is empty, return an empty list
    if not nums1 or not nums2 or k <= 0:
        return []

    result = []
    minHeap = []

    # Initialize the heap with the first element from nums2 for each element in
    # nums1 (up to k)
    for i in range(min(k, len(nums1))):
        # Each entry: (sum, index in nums1, index in nums2)
        heapq.heappush(minHeap, (nums1[i] + nums2[0], i, 0))

    # Extract the smallest pairs until we found k pairs or the heap is empty
    while k > 0 and minHeap:
        currentSum, i, j = heapq.heappop(minHeap)
        result.append([nums1[i], nums2[j]])
        k -= 1
```



```
# If there's any next element in nums2, push the new pair into the heap
if j + 1 < len(nums2):
    heapq.heappush(minHeap, (nums1[i] + nums2[j + 1], i, j + 1))

return result

# Example usage:
print(kSmallestPairs([1,7,11], [2,4,6], 3))
```

K-Way Merge

☐ #378 Kth Smallest Element in a Sorted Matrix

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Binary Search · Sorting · Heap (Priority Queue) · Matrix

Lists: AlgoMaster 600

Problem

Given an $n \times n$ matrix where each of the rows and columns is sorted in ascending order, return the k th smallest element in the matrix.

Note that it is the k th smallest element in the sorted order, not the k th distinct element.

You must find a solution with a memory complexity better than $O(n^2)$.

Example 1:

Input: matrix = [[1,5,9],[10,11,13],[12,13,15]], k = 8
 Output: 13
 Explanation: The elements in the matrix are [1,5,9,10,11,12,13,13,15], and the 8th smallest number is 13

Example 2:

Input: matrix = [[-5]], k = 1
 Output: -5

Constraints:

- $n == \text{matrix.length} == \text{matrix}[i].\text{length}$
- $1 \leq n \leq 300$
- $-109 \leq \text{matrix}[i][j] \leq 109$
- All the rows and columns of matrix are guaranteed to be sorted in non-decreasing order.
- $1 \leq k \leq n^2$

Follow up:

- Could you solve the problem with a constant memory (i.e., $O(1)$ memory complexity)?
- Could you solve the problem in $O(n)$ time complexity? The solution may be too advanced for an interview but you may find reading this paper fun.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def kthSmallest(self, matrix: list[list[int]], k: int) -> int:
        minHeap = [] # (matrix[i][j], i, j)

        i = 0
        while i < k and i < len(matrix):
            heapq.heappush(minHeap, (matrix[i][0], i, 0))
            i += 1

        while k > 1:
```

Round 5 · Mid · Medium

p.2171

```
k -= 1
_, i, j = heapq.heappop(minHeap)
if j + 1 < len(matrix[0]):
    heapq.heappush(minHeap, (matrix[i][j + 1], i, j + 1))

return minHeap[0][0]

class Solution:
    def kthSmallest(self, matrix: list[list[int]], k: int) -> int:
    def numsNoGreaterThan(m: int) -> int:
        count = 0
        j = len(matrix[0]) - 1
        # For each row, find the first index j s.t. row[j] <= m s.t. the number
        of
        # numbers <= m for this row will be j + 1.
        for row in matrix:
            while j >= 0 and row[j] > m:
                j -= 1

            count += j + 1
        return count

    l = matrix[0][0]
    r = matrix[-1][-1]
    return bisect.bisect_left(range(l, r), k, key=numsNoGreaterThan) + l
```

• LeetCode

```
class Solution:
    def kthSmallest(self, matrix: List[List[int]], k: int) -> int:
    def check(matrix, mid, k, n):
        count = 0
        i, j = n - 1, 0
        while i >= 0 and j < n:
            if matrix[i][j] <= mid:
                count += i + 1
                j += 1
            else:
```

Round 5 · Mid · Medium

p.2172

```

        i -= 1
        return count >= k

n = len(matrix)
left, right = matrix[0][0], matrix[n - 1][n - 1]
while left < right:
    mid = (left + right) >> 1
    if check(matrix, mid, k, n):
        right = mid
    else:
        left = mid + 1
return left

```

• SimplyLeet

```

# Python solution using binary search

def kthSmallest(matrix, k):
    n = len(matrix)
    # Set initial low and high based on the smallest and largest values in the matrix
    low, high = matrix[0][0], matrix[-1][-1]

    def count_less_equal(x):
        # Count the number of elements less than or equal to x
        count = 0

        # Start from the bottom-left corner of the matrix
        row, col = n - 1, 0
        while row >= 0 and col < n:
            if matrix[row][col] <= x:
                # All elements in this column up to this row are <= x
                count += row + 1
                col += 1
            else:
                row -= 1
        return count

```

```

# Binary search to find the kth smallest element
while low < high:
    mid = low + (high - low) // 2
    if count_less_equal(mid) < k:
        low = mid + 1 # kth element is greater than mid
    else:
        high = mid # kth element is less than or equal to mid
return low

# Example usage:
# print(kthSmallest([[1,5,9],[10,11,13],[12,13,15]], 8))

```

Data Structure Design

Round 5 · Mid · Medium · 11 questions

Round 5 · Mid · Medium

p.2175

Data Structure Design

□ #146 LRU Cache

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Linked List · Design
Lists: Grind 169

Problem

Design a data structure that follows the constraints of a Least Recently Used (LRU) cache.

Implement the LRUCache class:

- `LRUCache(int capacity)` Initialize the LRU cache with positive size capacity.
- `int get(int key)` Return the value of the key if the key exists, otherwise return -1.
- `void put(int key, int value)` Update the value of the key if the key exists. Otherwise, add the key-value pair to the cache. If the number of keys exceeds the capacity from this operation, evict the least recently used key.

The functions `get` and `put` must each run in $O(1)$ average time complexity.

Round 5 · Mid · Medium

p.2176

Example 1:

Input
 ["LRUCache", "put", "put", "get", "put", "get", "put", "get", "get", "get"]
 [[2], [1, 1], [2, 2], [1], [3, 3], [2], [4, 4], [1], [3], [4]]

Output
 [null, null, null, 1, null, -1, null, -1, 3, 4]

Explanation
 LRUCache lRUCache = new LRUCache(2);

Constraints:

- $1 \leq \text{capacity} \leq 3000$
- $0 \leq \text{key} \leq 104$
- $0 \leq \text{value} \leq 105$
- At most $2 * 105$ calls will be made to get and put.

Community Solutions (Python)

• WalkCC

```
class Node:
    def __init__(self, key: int, value: int):
        self.key = key
        self.value = value
        self.prev = None
        self.next = None

class LRUCache:
    def __init__(self, capacity: int):
```

```
self.capacity = capacity
self.keyToNode = {}
self.head = Node(-1, -1)
self.tail = Node(-1, -1)
self.join(self.head, self.tail)
```

```
def get(self, key: int) -> int:
    if key not in self.keyToNode:
        return -1
```

```
node = self.keyToNode[key]
self.remove(node)
self.moveToHead(node)
return node.value
```

```
def put(self, key: int, value: int) -> None:
    if key in self.keyToNode:
        node = self.keyToNode[key]
        node.value = value
        self.remove(node)
```

```
self.moveToHead(node)
return
```

```
if len(self.keyToNode) == self.capacity:
    lastNode = self.tail.prev
    del self.keyToNode[lastNode.key]
    self.remove(lastNode)
```

```
self.moveToHead(Node(key, value))
self.keyToNode[key] = self.head.next
```

```
def join(self, node1: Node, node2: Node):
    node1.next = node2
    node2.prev = node1
```

```
def moveToHead(self, node: Node):
    self.join(node, self.head.next)
    self.join(self.head, node)
```

```
def remove(self, node: Node):
```

```
self.join(node.prev, node.next)
```

• SimplyLeet

```
# Define a node for the doubly linked list.
class Node:
    def __init__(self, key, value):
        self.key = key          # key of the node
        self.value = value      # value of the node
        self.prev = None       # pointer to previous node
        self.next = None       # pointer to next node

class LRUCache:
    def __init__(self, capacity):

        self.capacity = capacity          # maximum capacity of the cache
        self.cache = {}                   # dictionary mapping key ->
        node

        # Create dummy head and tail nodes.
        self.head = Node(0, 0)
        self.tail = Node(0, 0)
        self.head.next = self.tail
        self.tail.prev = self.head

        # Helper function to remove a node from the linked list.
        def _remove(self, node):

            prev_node = node.prev
            next_node = node.next
            prev_node.next = next_node
            next_node.prev = prev_node

        # Helper function to add a node right after the head.
        def _add(self, node):
            node.prev = self.head
            node.next = self.head.next
            self.head.next.prev = node
```

Round 5 · Mid · Medium

p.2179

```
self.head.next = node
```

```
# Returns the value of the key if it exists; otherwise, returns -1.
def get(self, key):
    if key in self.cache:
        node = self.cache[key]
        # Move the accessed node to the head (marking it as recently used)
        self._remove(node)
        self._add(node)
        return node.value

    return -1

# Updates the value of the key if it exists; otherwise, adds the key-value
pair.
# Removes the least recently used key if the cache exceeds its capacity.
def put(self, key, value):
    if key in self.cache:
        # Remove the old node.
        self._remove(self.cache[key])
    node = Node(key, value)
    self._add(node)

    self.cache[key] = node
    if len(self.cache) > self.capacity:
        # Remove the least recently used element.
        lru = self.tail.prev
        self._remove(lru)
        del self.cache[lru.key]
```

• LC.ca

```
class Node:
    def __init__(self, key=0, val=0):
        self.key = key
        self.val = val
        self.prev = None
        self.next = None

class LRUCache:
    def __init__(self, capacity: int):
```

Round 5 · Mid · Medium

p.2180

```

self.cache = {} # key ==> Node(val)
self.head = Node() # dummy node
self.tail = Node() # dummy node
self.capacity = capacity
self.size = 0
self.head.next = self.tail # note: key setup
self.tail.prev = self.head

```

```

def get(self, key: int) -> int:
    if key not in self.cache:

```

```

        return -1
    node = self.cache[key]
    self.move_to_head(node)
    return node.val

```

```

def put(self, key: int, value: int) -> None:
    if key in self.cache:
        node = self.cache[key]
        node.val = value
        self.move_to_head(node)

```

```

    else:
        node = Node(key, value)
        self.cache[key] = node
        self.add_to_head(node)
        self.size += 1
        if self.size > self.capacity:
            tail = self.remove_tail()
            self.cache.pop(tail.key)
            self.size -= 1

```

```

def move_to_head(self, node):
    self.remove_node(node)
    self.add_to_head(node)

```

```

def remove_node(self, node):
    node.prev.next = node.next
    node.next.prev = node.prev

```

```

def add_to_head(self, node):
    node.next = self.head.next

```

```

node.prev = self.head
self.head.next = node
node.next.prev = node

```

```

def remove_tail(self):
    node = self.tail.prev
    self.remove_node(node)
    return node

```

```

# Your LRUcache object will be instantiated and called as such:
# obj = LRUcache(capacity)
# param_1 = obj.get(key)
# obj.put(key,value)

```

```

#####

```

```

'''
example:

```

```

>>> od = collections.OrderedDict()
>>>
>>> od[1]=1
>>> od[2]=2
>>> od[3]=3
>>>
>>> od
OrderedDict([(1, 1), (2, 2), (3, 3)])
>>> od.move_to_end(1)
>>> od

```

```

OrderedDict([(2, 2), (3, 3), (1, 1)])
>>>
>>> od.get(1)
1
>>> od.popitem()
(1, 1)
>>> od
OrderedDict([(2, 2), (3, 3)])
>>>
>>> od[1]=1

```

```
>>> od
OrderedDict([(2, 2), (3, 3), (1, 1)])
>>>
>>> od.popitem(last=False)
(2, 2)
>>> od
OrderedDict([(3, 3), (1, 1)])
'''
```

```
import collections
```

```
class LRUCache:
    def __init__(self, capacity: 'int'):
        self.cache = collections.OrderedDict()
        self.remain = capacity
```

```
    def get(self, key: 'int') -> 'int':
        if key not in self.cache:
            return -1
```

```
        self.cache.move_to_end(key) # meaning end is the most recently used
        return self.cache.get(key)
```

```
    def put(self, key: 'int', value: 'int') -> 'None':
        if key not in self.cache:
            if self.remain > 0:
                self.remain -= 1
            else:
                self.cache.popitem(last=False) # pop start position
        else:
            self.cache[key] = value
```

```
        self.cache.pop(key)
        self.cache[key] = value # add to end of dict, meaning most recently
used
```

```
#####
```

```
### below solution with no ordered-dict
```

```
class DLinkedList:
```

```
    def __init__(self, key=0, value=0):
```

```
        self.key = key
        self.val = value
        self.next = None
        self.prev = None
```

```
class DLinkedList:
```

```
    def __init__(self):
        self.head = DLinkedList() # dummy head, its next is real head
```

```
        self.tail = DLinkedList() # dummy tail, its prev is real tail
        self.head.next, self.tail.prev = self.tail, self.head
```

```
    def add_node(self, node): # add to head
        node.prev, node.next = self.head, self.head.next
        node.next.prev, node.prev.next = node, node
```

```
    def remove_node(self, node):
        node.prev.next, node.next.prev = node.next, node.prev
        return node.key
```



```

def remove_tail(self):
    return self.remove_node(self.tail.prev) # dummy tail's prev is real
tail

def move_to_head(self, node):
    self.remove_node(node)
    self.add_node(node)

# def __repr__(self):
#     ans = []

#     h = self.head
#     while h:
#         ans.append(str(h.val))
#         h = h.next
#     return '<DLinkedList: {}>'.format('>'.join(ans))

class LRUCache:

    def __init__(self, capacity: int):
        self.capacity = capacity

        self.nodes_map = {}
        self.cache_list = DLinkedList()

    def get(self, key: int) -> int:
        node = self.nodes_map.get(key, None)
        if node:
            self.cache_list.move_to_head(node)
            return node.val
        else:
            return -1

```

```

def put(self, key: int, value: int) -> None:
    node = self.nodes_map.get(key, None)
    if not node:
        if self.capacity > 0:
            self.capacity -= 1
        else:
            rm_key = self.cache_list.remove_tail()
            self.nodes_map.pop(rm_key) # note api
            new_node = DLinkedList(key, value)

            self.nodes_map[key] = new_node
            self.cache_list.add_node(new_node)
    else:
        self.cache_list.move_to_head(node)
        node.val = value

```

◆ Quick Review

Key Insights

- Use a Hash Table (dictionary) to allow $O(1)$ access to nodes by key.
- Use a Doubly Linked List to record the usage order. The head represents the most recently used element while the tail represents the least recently used.
- When a key is accessed or updated, move the corresponding node to the front (head) of the list.
- When the cache exceeds capacity, remove the node at the tail end which is the least recently used entry.

Space & Time

Time Complexity: $O(1)$ per get and put operation.

Space Complexity: $O(\text{capacity})$, storing key-node pairs and linked list nodes.

Solution

The solution combines a hash table (or dictionary) and a doubly linked list:

- The hash table stores keys and corresponding node addresses.
- The doubly linked list maintains the order of usage with the most recently used items near the head.
- For get(key), check the dictionary. If found, move the node to the head and return its value; if not, return -1.
- For put(key, value), if the key exists, update its value and move it to the head. If not, create a new node and add it right after the head. If the capacity is reached, remove the tail node (least recently used) and update the dictionary accordingly.
- Dummy head and tail nodes are used to simplify node addition and removal procedures without worrying about edge cases.

Data Structure Design

★ #271 Encode and Decode Strings ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · String · Design

Lists: ★ Premium | Grind 169 | Premium 98

Problem

Design an algorithm to encode a list of strings to a string. The encoded string is then sent over the network and is decoded back to the original list of strings.

Machine 1 (sender) has the function:

```
string encode(vector<string> strs) {
    // ... your code
    return encoded_string;
}
```

Machine 2 (receiver) has the function:

```
vector<string> decode(string s) {
    //... your code
    return strs;
}
```

So Machine 1 does:

```
string encoded_string = encode(strs);
```

and Machine 2 does:

```
vector<string> strs2 = decode(encoded_string);
```

strs2 in Machine 2 should be the same as strs in Machine 1.

Implement the encode and decode methods.

You are not allowed to solve the problem using any serialize methods (such as eval).

Example 1:

```
Input: dummy_input = ["Hello","World"]
Output: ["Hello","World"]
Explanation:
Machine 1:
Codec encoder = new Codec();
String msg = encoder.encode(strs);
Machine 1 ---msg--> Machine 2
```

Example 2:

```
Input: dummy_input = [""]
Output: [""]
```

Constraints:

- $1 \leq \text{strs.length} \leq 200$
- $0 \leq \text{strs}[i].\text{length} \leq 200$
- $\text{strs}[i]$ contains any possible characters out of 256 valid ASCII characters.

Follow up: Could you write a generalized algorithm to work on any possible set of characters?

Community Solutions (Python)

- WalkCC

```
class Codec:
    def encode(self, strs: list[str]) -> str:
        """Encodes a list of strings to a single string."""
        return ''.join(str(len(s)) + '/' + s for s in strs)

    def decode(self, s: str) -> list[str]:
        """Decodes a single string to a list of strings."""
        decoded = []

        i = 0

        while i < len(s):
            slash = s.find('/', i)
            length = int(s[i:slash])
            i = slash + length + 1
            decoded.append(s[slash + 1:i])

        return decoded
```

• LeetDoocs

```
class Codec:
    def encode(self, strs: List[str]) -> str:
        """Encodes a list of strings to a single string."""
        ans = []
        for s in strs:
            ans.append('{:4}'.format(len(s)) + s)
        return ''.join(ans)

    def decode(self, s: str) -> List[str]:
        """Decodes a single string to a list of strings."""

        ans = []
        i, n = 0, len(s)
        while i < n:
            size = int(s[i : i + 4])
            i += 4
            ans.append(s[i : i + size])
            i += size
        return ans
```

```
# Your Codec object will be instantiated and called as such:
# codec = Codec()
# codec.decode(codec.encode(strs))
```

• SimplyLeet

```
class Codec:
    # Encodes a list of strings to a single string.
    def encode(self, strs):
        encoded = []
        # Iterate through each string in the list.
        for s in strs:
            # Append the length of the string, a '#' as delimiter, and the
            # string.
            encoded.append(str(len(s)) + '#' + s)
        # Join all parts into one encoded string.
        return ''.join(encoded)
```

```
# Decodes a single string back to a list of strings.
def decode(self, s):
    decoded = []
    i = 0 # Pointer to traverse the encoded string.
    while i < len(s):
        j = i
        # Find the '#' delimiter to determine the length.
        while s[j] != '#':
            j += 1
```

```
        # Convert the substring from i to j into an integer length.
        length = int(s[i:j])
        # Extract the actual string using the obtained length.
        string = s[j+1:j+1+length]
        decoded.append(string)
        # Move the pointer to the start of the next encoded segment.
        i = j + 1 + length
    return decoded
```

```
# Example usage:
```

```
# codec = Codec()
# encoded_str = codec.encode(["Hello", "World"])
# decoded_list = codec.decode(encoded_str)
```

Round 5 · Mid · Medium

p.2191

• LC.ca

```
'''
>>> s = 'abcde'
>>> '{:4}'.format(len(s)) + s
' 5abcde'
>>>
>>> '{:4}'.format(333)
' 333'
>>>
>>> '{:4}'.format(1234)
'1234'
```

```
>>> '{:4}'.format(12345)
'12345'
```

```
>>> '{0:4}'.format(12345)
'12345'
>>> '{1:4}'.format(12345)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: Replacement index 1 out of range for positional args tuple
>>> '{1:4}'.format(12345, 678910)
```

```
'678910'
>>> '{:4}'.format(12345, 678910)
'12345'
>>> '{0:4}'.format(12345, 678910)
'12345'
'''
```

```
class Codec: # no splitter
    def encode(self, strs: List[str]) -> str:
        """Encodes a list of strings to a single string."""
        ans = []
```

Round 5 · Mid · Medium

p.2192

```

for s in strs:
    ans.append('{:4}'.format(len(s)) + s)
return ''.join(ans)

def decode(self, s: str) -> List[str]:
    """Decodes a single string to a list of strings."""
    ans = []
    i, n = 0, len(s)
    while i < n:
        size = int(s[i : i + 4]) # so here potential overflow, if larger
                                # than int-max, need to clarify assumption with interviewer
        i += 4
        ans.append(s[i : i + size]) # note: exclusive for right index
        i += size
    return ans

# Your Codec object will be instantiated and called as such:
# codec = Codec()
# codec.decode(codec.encode(strs))

#####

class Codec:
    def encode(self, strs: List[str]) -> str:
        encoded = ""
        for s in strs:
            # cover for input: ["abcd", "####"]
            encoded += str(len(s)) + "#" + s.replace("#", "$$") + "#"

```

```

return encoded

def decode(self, s: str) -> List[str]:
    decoded = []
    i = 0
    while i < len(s):
        j = s.find("#", i)
        length = int(s[i:j])
        decoded.append(s[j+1:j+1+length].replace("$", "#"))
        i = j + 1 + length + 1

return decoded

#####

class Codec: # wrong compared with above, for input: ["abcd", "####"]
    def encode(self, strs):
        encoded = ""
        for s in strs:
            encoded += str(len(s)) + "#" + s

return encoded

def decode(self, s):
    decoded = []
    i = 0
    while i < len(s):
        delimiter = s.index("#", i)
        length = int(s[i:delimiter])
        decoded.append(s[delimiter+1:delimiter+1+length])
        i = delimiter + 1 + length

return decoded

```

◆ Quick Review

Key Insights

- Use length–prefix encoding for each string to avoid ambiguity.
- Append a delimiter (e.g., "#") after the length to separate it from the string content.
- This method ensures that even if the original strings contain special characters, including the delimiter, the decoding remains unambiguous.
- The approach handles edge cases, such as empty strings.

Space & Time

Time Complexity: $O(n)$, where n is the total number of characters across all strings.

Space Complexity: $O(n)$, required for the encoded string and the resulting decoded list.

Solution

We encode by iterating over each string and building a new string that contains the length of the string, a delimiter (e.g., "#"), and then the string itself. This way, during decoding, we can safely determine how many characters to extract by first reading until the delimiter to get the length, and then reading that number of characters. This length–prefix approach

avoids conflicts with potential characters in the strings.

Data Structure Design

#355 Design Twitter

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Linked List · Design · Heap
(Priority Queue)

Lists: NeetCode 150

Problem

Design a simplified version of Twitter where users can post tweets, follow/unfollow another user, and is able to see the 10 most recent tweets in the user's news feed.

Implement the Twitter class:

- `Twitter()` Initializes your twitter object.

- `void postTweet(int userId, int tweetId)`

Composes a new tweet with ID `tweetId` by the user `userId`. Each call to this function will be made with a unique `tweetId`.

- `List<Integer> getNewsFeed(int userId)`

Retrieves the 10 most recent tweet IDs in the user's news feed. Each item in the news feed must be posted by users who the user

followed or by the user themselves. Tweets must be ordered from most recent to least recent.

- `void follow(int followerId, int followeeId)`

The user with ID `followerId` started following the user with ID `followeeId`.

- `void unfollow(int followerId, int followeeId)`

The user with ID `followerId` started unfollowing the user with ID `followeeId`.

Example 1:

```
Input
["Twitter", "postTweet", "getNewsFeed", "follow", "postTweet", "getNewsFeed",
"unfollow", "getNewsFeed"]
[[], [1, 5], [1], [1, 2], [2, 6], [1], [1, 2], [1]]
Output
[null, null, [5], null, null, [6, 5], null, [5]]
```

Explanation

```
Twitter twitter = new Twitter();
```

Constraints:

- $1 \leq \text{userId}, \text{followerId}, \text{followeeId} \leq 500$
- $0 \leq \text{tweetId} \leq 104$
- All the tweets have unique IDs.
- At most $3 * 10^4$ calls will be made to `postTweet`, `getNewsFeed`, `follow`, and `unfollow`.
- A user cannot follow himself.

Community Solutions (Python)

• WalkCC

```
class Twitter:
    def __init__(self):
        self.timer = itertools.count(step=-1)
        self.tweets = collections.defaultdict(deque)
        self.followees = collections.defaultdict(set)

    def postTweet(self, userId: int, tweetId: int) -> None:
        self.tweets[userId].appendleft((next(self.timer), tweetId))
        if len(self.tweets[userId]) > 10:
            self.tweets[userId].pop()

    def getNewsFeed(self, userId: int) -> list[int]:
        tweets = list(
            heapq.merge(
                *
                (self.tweets[followee]
                 for followee in self.followees[userId] | {userId}))
        )
        return [tweetId for _, tweetId in tweets[:10]]

    def follow(self, followerId: int, followeeId: int) -> None:

        self.followees[followerId].add(followeeId)

    def unfollow(self, followerId: int, followeeId: int) -> None:
        self.followees[followerId].discard(followeeId)
```

• LeetDoocs

```
class Twitter:
    def __init__(self):
        """
        Initialize your data structure here.
        """
        self.user_tweets = defaultdict(list)
        self.user_following = defaultdict(set)
        self.tweets = defaultdict()
        self.time = 0
```

Round 5 · Mid · Medium

p.2199

```
def postTweet(self, userId: int, tweetId: int) -> None:
    """
    Compose a new tweet.
    """
    self.time += 1
    self.user_tweets[userId].append(tweetId)
    self.tweets[tweetId] = self.time
```

```
def getNewsFeed(self, userId: int) -> List[int]:
    """
```

Retrieve the 10 most recent tweet ids in the user's news feed. Each item in the news feed must be posted by users who the user followed or by the user herself. Tweets must be ordered from most recent to least recent.

```
    """
    following = self.user_following[userId]
    users = set(following)
    users.add(userId)
    tweets = [self.user_tweets[u][::-1][:10] for u in users]
    tweets = sum(tweets, [])
    return nlargest(10, tweets, key=lambda tweet: self.tweets[tweet])
```

```
def follow(self, followerId: int, followeeId: int) -> None:
```

```
    """
    Follower follows a followee. If the operation is invalid, it should be
    a no-op.
    """
```

```
    self.user_following[followerId].add(followeeId)
```

```
def unfollow(self, followerId: int, followeeId: int) -> None:
    """
```

Follower unfollows a followee. If the operation is invalid, it should be a no-op.

```
    """
    following = self.user_following[followerId]
```

Round 5 · Mid · Medium

p.2200


```
if followeeId in following:
    following.remove(followeeId)
```

```
# Your Twitter object will be instantiated and called as such:
# obj = Twitter()
# obj.postTweet(userId,tweetId)
# param_2 = obj.getNewsFeed(userId)
# obj.follow(followerId,followeeId)
# obj.unfollow(followerId,followeeId)
```

• SimplyLeet

```
# Python implementation of the Twitter design problem
import heapq
```

```
class Twitter:
    def __init__(self):
        # Mapping from userId to a set of followeeIds
        self.follows = {}
        # Mapping from userId to list of (timestamp, tweetId)
        self.tweets = {}
        # Global timestamp to order tweets

        self.timestamp = 0

    def postTweet(self, userId: int, tweetId: int) -> None:
        # Ensure the user follows itself for easier news feed retrieval
        if userId not in self.follows:
            self.follows[userId] = set()
        self.follows[userId].add(userId)

        # Append the tweet with the current timestamp, then increment timestamp
        if userId not in self.tweets:
```

```
self.tweets[userId] = []
self.tweets[userId].append((self.timestamp, tweetId))
self.timestamp += 1
```

```
def getNewsFeed(self, userId: int) -> list:
    # If the user does not exist in follows mapping, return empty feed
    if userId not in self.follows:
        return []
```

```
# Use a heap (as a max-heap using negative timestamp)
```

```
heap = []
# Iterate over all followees of the user
for followeeId in self.follows[userId]:
    if followeeId in self.tweets and self.tweets[followeeId]:
        # Start from the most recent tweet of the followee
        index = len(self.tweets[followeeId]) - 1
        ts, tid = self.tweets[followeeId][index]
        # Push the tweet along with followeeId and the index of the
        # tweet in the list
        heapq.heappush(heap, (-ts, tid, followeeId, index))
```

```
result = []
# Retrieve up to 10 tweets
while heap and len(result) < 10:
    neg_ts, tweetId, followeeId, idx = heapq.heappop(heap)
    result.append(tweetId)
    # If there is an older tweet from this followee, push it onto the
    # heap
    if idx - 1 >= 0:
        ts, tid = self.tweets[followeeId][idx - 1]
        heapq.heappush(heap, (-ts, tid, followeeId, idx - 1))
    return result
```

```
def follow(self, followerId: int, followeeId: int) -> None:
    # A user should always follow itself
    if followerId not in self.follows:
        self.follows[followerId] = set()
    self.follows[followerId].add(followerId)
    # Add followeeId to follower's follow set if not self-follow
    if followerId != followeeId:
        self.follows[followerId].add(followeeId)

def unfollow(self, followerId: int, followeeId: int) -> None:
    # A user cannot unfollow itself
    if followerId in self.follows and followeeId != followerId:
        self.follows[followerId].discard(followeeId)
```

Data Structure Design

□ #362 Design Hit Counter

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table · Binary Search · Design · Queue

Lists: Grind 169

Problem

Design a hit counter which counts the number of hits received in the past 5 minutes (i.e., the past 300 seconds).

Your system should accept a timestamp parameter (in seconds granularity), and you may assume that calls are being made to the system in chronological order (i.e., timestamp is monotonically increasing). Several hits may arrive roughly at the same time.

Implement the HitCounter class:

- HitCounter() Initializes the object of the hit counter system.
- void hit(int timestamp) Records a hit that happened at timestamp (in seconds). Several

hits may happen at the same timestamp.

- `int getHits(int timestamp)` Returns the number of hits in the past 5 minutes from timestamp (i.e., the past 300 seconds).

Example 1:

```
Input
["HitCounter", "hit", "hit", "hit", "getHits", "hit", "getHits", "getHits"]
[[], [1], [2], [3], [4], [300], [300], [301]]
Output
[null, null, null, null, 3, null, 4, 3]
```

```
Explanation
HitCounter hitCounter = new HitCounter();
```

Constraints:

- $1 \leq \text{timestamp} \leq 2 * 10^9$
- All the calls are being made to the system in chronological order (i.e., timestamp is monotonically increasing).
- At most 300 calls will be made to `hit` and `getHits`.

Follow up: What if the number of hits per second could be huge? Does your design scale?

Community Solutions (Python)

- WalkCC

```
class HitCounter:
    def __init__(self):
        self.timestamps = [0] * 300
        self.hits = [0] * 300

    def hit(self, timestamp: int) -> None:
        i = timestamp % 300
        if self.timestamps[i] == timestamp:
            self.hits[i] += 1
        else:
            self.timestamps[i] = timestamp
            self.hits[i] = 1 # Reset the hit count to 1.

    def getHits(self, timestamp: int) -> int:
        return sum(h for t, h in zip(self.timestamps, self.hits)
                    if timestamp - t < 300)
```

• LeetCode

```
class HitCounter:
    def __init__(self):
        self.ts = []

    def hit(self, timestamp: int) -> None:
        self.ts.append(timestamp)

    def getHits(self, timestamp: int) -> int:
        return len(self.ts) - bisect_left(self.ts, timestamp - 300 + 1)
```

```
# Your HitCounter object will be instantiated and called as such:
# obj = HitCounter()
# obj.hit(timestamp)
# param_2 = obj.getHits(timestamp)
```

• SimplyLeet

```
# Python implementation of HitCounter

from collections import deque

class HitCounter:
    def __init__(self):
        # Initialize the deque to store (timestamp, count) pairs.
        self.hits_queue = deque()

    def hit(self, timestamp: int) -> None:

        # If the last recorded hit is at the same timestamp, increment its
        count.
        if self.hits_queue and self.hits_queue[-1][0] == timestamp:
            self.hits_queue[-1][1] += 1
        else:
            # Append new (timestamp, count) pair.
            self.hits_queue.append([timestamp, 1])

    def getHits(self, timestamp: int) -> int:
        # Remove outdated hits from the front of the queue.
        while self.hits_queue and self.hits_queue[0][0] <= timestamp - 300:

            self.hits_queue.popleft()

        # Sum counts of the remaining hit entries.
        return sum(count for _, count in self.hits_queue)
```

• LC.ca

```
from collections import deque

class HitCounter: # queue

    def __init__(self):
        """
        Initialize your data structure here.
        """
        self.hits = deque()
```

Round 5 · Mid · Medium

p.2207

```
def hit(self, timestamp: int) -> None:
    """
    Record a hit.
    @param timestamp - The current timestamp (in seconds granularity).
    """
    self.hits.append(timestamp)

def getHits(self, timestamp: int) -> int:
    """
    Return the number of hits in the past 5 minutes.

    @param timestamp - The current timestamp (in seconds granularity).
    """
    while self.hits and self.hits[0] <= timestamp - 300:
        self.hits.popleft()
    return len(self.hits)

class HitCounter: # follow-up
    def __init__(self):
        self.times = [0] * 300

        self.hits = [0] * 300

    def hit(self, timestamp: int) -> None:
        idx = timestamp % 300
        if self.times[idx] != timestamp:
            self.times[idx] = timestamp
            self.hits[idx] = 1
        else:
            self.hits[idx] += 1

    def getHits(self, timestamp: int) -> int:
        res = 0
        for i in range(300):
            if timestamp - self.times[i] < 300:
                res += self.hits[i]
        return res

# Your HitCounter object will be instantiated and called as such:
# obj = HitCounter()
# obj.hit(timestamp)
```

Round 5 · Mid · Medium

p.2208

```
# param_2 = obj.getHits(timestamp)

#####

class HitCounter: # extra o(N) space
    def __init__(self):
        """
        Initialize your data structure here.
        """
        self.counter = Counter()

    def hit(self, timestamp: int) -> None:
        """
        Record a hit.
        @param timestamp - The current timestamp (in seconds granularity).
        """
        self.counter[timestamp] += 1

    def getHits(self, timestamp: int) -> int:
        """
        Return the number of hits in the past 5 minutes.
        @param timestamp - The current timestamp (in seconds granularity).
        """
        return sum([v for t, v in self.counter.items() if t + 300 > timestamp])

# Your HitCounter object will be instantiated and called as such:
# obj = HitCounter()
# obj.hit(timestamp)
# param_2 = obj.getHits(timestamp)
```

◆ Quick Review

Key Insights

- Use a sliding window of 300 seconds to count hits.

- Since timestamps are monotonically increasing, outdated hits can be removed from the front.
- A queue (or circular array) is an efficient data structure to maintain and remove expired hit records.
- Grouping consecutive hits at the same timestamp can optimize space.

Space & Time

Time Complexity: $O(1)$ per hit and `getHits` operation on average, since each hit is enqueued and later dequeued only once.

Space Complexity: $O(W)$ where W is the window size (300 seconds) in the worst case.

Solution

The solution uses a queue (or deque) data structure where each element stores a (timestamp, count) pair. When a hit is recorded, if the last element of the queue shares the same timestamp, then its count is incremented; otherwise, a new element is added. When `getHits(timestamp)` is called, the algorithm removes elements from the front of the queue whose timestamp is out of the

300-second window relative to the current timestamp, and then sums the counts of the remaining elements to return the result. This ensures that only relevant hits within the past 5 minutes are counted. The design scales well when hit frequency is high because hits at the same timestamp are grouped together instead of inserting individual records.

Data Structure Design

□ #380 Insert Delete GetRandom O(1)

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table · Math · Design
 Lists: Grind 169

Problem

Implement the RandomizedSet class:

- RandomizedSet() Initializes the RandomizedSet object.
- bool insert(int val) Inserts an item val into the set if not present. Returns true if the item was not present, false otherwise.
- bool remove(int val) Removes an item val from the set if present. Returns true if the item was present, false otherwise.
- int getRandom() Returns a random element from the current set of elements (it's guaranteed that at least one element exists when this method is called). Each element must have the same probability of being returned.

You must implement the functions of the class such that each function works in average $O(1)$ time complexity.

Example 1:

Input
["RandomizedSet", "insert", "remove", "insert", "getRandom", "remove", "insert", "getRandom"]
[[], [1], [2], [2], [1], [1], [2], []]
Output
[null, true, false, true, 2, true, false, 2]

Explanation
RandomizedSet randomizedSet = new RandomizedSet();

Constraints:

- $-231 \leq \text{val} \leq 231 - 1$
- At most $2 * 10^5$ calls will be made to insert, remove, and getRandom.
- There will be at least one element in the data structure when getRandom is called.

Community Solutions (Python)

• WalkCC

```
class RandomizedSet:
    def __init__(self):
        self.vals = []
        self.valToIndex = collections.defaultdict(int) # {val: index in vals}

    def insert(self, val: int) -> bool:
        if val in self.valToIndex:
            return False
        self.valToIndex[val] = len(self.vals)
        self.vals.append(val)
```

Round 5 · Mid · Medium

p.2213

```
return True

def remove(self, val: int) -> bool:
    if val not in self.valToIndex:
        return False
    index = self.valToIndex[val]
    # The order of the following two lines is important when vals.size() == 1.
    self.valToIndex[self.vals[-1]] = index
    del self.valToIndex[val]
    self.vals[index] = self.vals[-1]

    self.vals.pop()
    return True

def getRandom(self) -> int:
    index = random.randint(0, len(self.vals) - 1)
    return self.vals[index]
```

• LeetDoocs

```
class RandomizedSet:
    def __init__(self):
        self.d = {}
        self.q = []

    def insert(self, val: int) -> bool:
        if val in self.d:
            return False
        self.d[val] = len(self.q)
        self.q.append(val)

    def remove(self, val: int) -> bool:
        if val not in self.d:
            return False
        i = self.d[val]
        self.d[self.q[-1]] = i
        self.q[i] = self.q[-1]
        self.q.pop()
        self.d.pop(val)
```

Round 5 · Mid · Medium

p.2214

```
return True
```

```
def getRandom(self) -> int:
    return choice(self.q)
```

```
# Your RandomizedSet object will be instantiated and called as such:
# obj = RandomizedSet()
# param_1 = obj.insert(val)
# param_2 = obj.remove(val)
```

```
# param_3 = obj.getRandom()
```

• SimplyLeet

```
import random
```

```
class RandomizedSet:
```

```
    def __init__(self):
```

```
        # List to store the values
```

```
        self.values = []
```

```
        # Dictionary to map a value to its index in the list
```

```
        self.val_to_index = {}
```

```
    def insert(self, val: int) -> bool:
```

```
        # If the value is already present, do not insert.
```

```
        if val in self.val_to_index:
```

```
            return False
```

```
        # Append the value and store its index.
```

```
        self.val_to_index[val] = len(self.values)
```

```
        self.values.append(val)
```

```
        return True
```

```
    def remove(self, val: int) -> bool:
```

```
        # If the value is not present, return False.
```

```
if val not in self.val_to_index:
```

```
    return False
```

```
# Index of element to remove.
```

```
index = self.val_to_index[val]
```

```
# Get the last element.
```

```
last_element = self.values[-1]
```

```
# Place the last element at the index of the element to remove.
```

```
self.values[index] = last_element
```

```
self.val_to_index[last_element] = index
```

```
# Remove the last element from the list.
```

```
self.values.pop()
```

```
# Delete the mapping for the removed element.
```

```
del self.val_to_index[val]
```

```
return True
```

```
def getRandom(self) -> int:
```

```
    # Return a random element from the list.
```

```
    return random.choice(self.values)
```

• LC.ca

```
'''
my_dict.pop('b') # vs del d[k]
```

```
my_dict = {'a': 1, 'b': 2, 'c': 3}
```

```
val = my_dict.pop('b')
```

```
print(my_dict) # {'a': 1, 'c': 3}
```

```
print(val) # 2
```

```
>>> item = my_dict.pop()
```

```
Traceback (most recent call last):
```

```
File "<stdin>", line 1, in <module>
```

```
TypeError: pop expected at least 1 argument, got 0
```

```
>>> my_dict = {'a': 1, 'b': 2, 'c': 3}
```

```
>>> del my_dict['b']
```

```
>>> my_dict
```

```
{'a': 1, 'c': 3}
```

```
'''
```

```
from collections import defaultdict
```



```

from random import choice

class RandomizedSet(object):
    def __init__(self):
        """
        Initialize your data structure here.
        """

        # value (map) -> its index -> value (list)
        self.d = {}

        self.a = [] # introduce it purely for random

    def insert(self, val):
        """
        Inserts a value to the set. Returns true if the set did not already contain
        the specified element.
        :type val: int
        :rtype: bool
        """
        if val in self.d:
            return False

        self.a.append(val)
        self.d[val] = len(self.a) - 1
        return True

    def remove(self, val):
        """
        Removes a value from the set. Returns true if the set contained the
        specified element.
        :type val: int
        :rtype: bool
        """

```

Round 5 · Mid · Medium

p.2217

```

        if val not in self.d:
            return False
        index = self.d[val]

        # process last index/val
        self.a[index] = self.a[-1]
        self.d[self.a[-1]] = index

        # process to be deleted index/val
        self.a.pop() # delete in list

        del self.d[val] # delete in dict
        # or, self.d.pop(val)
        return True

    def getRandom(self):
        """
        Get a random element from the set.
        :rtype: int
        """
        return self.a[random.randrange(0, len(self.a))]

        # return random.choice(self.a)

#####

# Your RandomizedSet object will be instantiated and called as such:
# obj = RandomizedSet()
# param_1 = obj.insert(val)
# param_2 = obj.remove(val)
# param_3 = obj.getRandom()

'''
>>> import random
>>> random.choice([1,2,3,4,5])
3
>>> random.choice([1,2,3,4,5])
4
>>> random.choice([1,2,3,4,5])
4
'''

class RandomizedSet:

```

Round 5 · Mid · Medium

p.2218

```
def __init__(self):
    self.m = {}
    self.l = []
```

```
def insert(self, val: int) -> bool:
    if val in self.m:
        return False
    self.m[val] = len(self.l)
    self.l.append(val)
    return True
```

```
def remove(self, val: int) -> bool:
    if val not in self.m:
        return False
    idx = self.m[val]
    self.l[idx] = self.l[-1]
    self.m[self.l[-1]] = idx
    self.l.pop()
    self.m.pop(val)
    return True
```

```
def getRandom(self) -> int:
    return random.choice(self.l)
```

```
# Your RandomizedSet object will be instantiated and called as such:
# obj = RandomizedSet()
# param_1 = obj.insert(val)
# param_2 = obj.remove(val)
# param_3 = obj.getRandom()
```

◆ Quick Review

Key Insights

- Use a dynamic array (or list) to store the elements to allow $O(1)$ access by index.

Round 5 · Mid · Medium

p.2219

- Use a hash table (or dictionary/map) to store the mapping from value to its index in the array.
- For removal, swap the element to be removed with the last element in the array, then delete the last element. This avoids expensive shifting of elements.
- getRandom can be implemented by generating a random index into the array.

Space & Time

Time Complexity: $O(1)$ on average for insert, remove, and getRandom operations.

Space Complexity: $O(n)$ where n is the number of elements stored in the data structure.

Solution

The solution involves maintaining a list to store the values and a hash map to quickly check for the presence of a value and to know its index in the list. When an element is removed, swap it with the last element of the list and update the map accordingly; then, remove the last element in $O(1)$ time. This approach ensures that insert, remove, and getRandom can all be achieved in constant

Round 5 · Mid · Medium

p.2220

time on average.

Round 5 · Mid · Medium

p.2221

Data Structure Design

□ #622 Design Circular Queue

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Linked List · Design · Queue
Lists: Denny Zhang

Problem

Design your implementation of the circular queue. The circular queue is a linear data structure in which the operations are performed based on FIFO (First In First Out) principle, and the last position is connected back to the first position to make a circle. It is also called "Ring Buffer".

One of the benefits of the circular queue is that we can make use of the spaces in front of the queue. In a normal queue, once the queue becomes full, we cannot insert the next element even if there is a space in front of the queue. But using the circular queue, we can use the space to store new values.

Implement the MyCircularQueue class:

Round 5 · Mid · Medium

p.2222

- **MyCircularQueue(k)** Initializes the object with the size of the queue to be k.
- **int Front()** Gets the front item from the queue. If the queue is empty, return -1.
- **int Rear()** Gets the last item from the queue. If the queue is empty, return -1.
- **boolean enqueue(int value)** Inserts an element into the circular queue. Return true if the operation is successful.
- **boolean dequeue()** Deletes an element from the circular queue. Return true if the operation is successful.
- **boolean isEmpty()** Checks whether the circular queue is empty or not.
- **boolean isFull()** Checks whether the circular queue is full or not.

You must solve the problem without using the built-in queue data structure in your programming language.

Example 1:

```
Input
["MyCircularQueue", "enqueue", "enqueue", "enqueue", "enqueue", "Rear",
"isFull", "dequeue", "enqueue", "Rear"]
[[3], [1], [2], [3], [4], [1], [1], [1], [4], [1]]
Output
[null, true, true, true, false, 3, true, true, true, 4]
```

```
Explanation
MyCircularQueue myCircularQueue = new MyCircularQueue(3);
```

Constraints:

- $1 \leq k \leq 1000$
- $0 \leq \text{value} \leq 1000$
- At most 3000 calls will be made to enqueue, dequeue, Front, Rear, isEmpty, and isFull.

Community Solutions (Python)

• LeetCode

```
class MyCircularQueue:

    def __init__(self, k: int):
        self.q = [0] * k
        self.size = 0
        self.capacity = k
        self.front = 0

    def enqueue(self, value: int) -> bool:
        if self.isFull():
```

```

        return False
    self.q[(self.front + self.size) % self.capacity] = value
    self.size += 1
    return True

def deQueue(self) -> bool:
    if self.isEmpty():
        return False
    self.front = (self.front + 1) % self.capacity
    self.size -= 1

    return True

def Front(self) -> int:
    return -1 if self.isEmpty() else self.q[self.front]

def Rear(self) -> int:
    if self.isEmpty():
        return -1
    return self.q[(self.front + self.size - 1) % self.capacity]

def isEmpty(self) -> bool:
    return self.size == 0

def isFull(self) -> bool:
    return self.size == self.capacity

# Your MyCircularQueue object will be instantiated and called as such:
# obj = MyCircularQueue(k)
# param_1 = obj.enQueue(value)

# param_2 = obj.deQueue()
# param_3 = obj.Front()
# param_4 = obj.Rear()
# param_5 = obj.isEmpty()
# param_6 = obj.isFull()

```

• SimplyLeet

Round 5 · Mid · Medium

p.2225

```

# Python implementation of MyCircularQueue

class MyCircularQueue:
    def __init__(self, k):
        # Initialize the queue with capacity k
        self.capacity = k
        self.queue = [0] * k # fixed-size array
        self.front = 0 # pointer to the front element
        self.rear = 0 # pointer to the next insertion position
        self.count = 0 # current number of elements in the queue

    def enQueue(self, value):
        # Check if the queue is full
        if self.isFull():
            return False
        # Insert element at rear pointer and update pointer using modulo for
        # circularity
        self.queue[self.rear] = value
        self.rear = (self.rear + 1) % self.capacity
        self.count += 1
        return True

    def deQueue(self):
        # Check if the queue is empty
        if self.isEmpty():
            return False
        # Move front pointer to the next element using modulo for circularity
        self.front = (self.front + 1) % self.capacity
        self.count -= 1
        return True

```

Round 5 · Mid · Medium

p.2226

```

def Front(self):
    # Return -1 if the queue is empty, else return the front element
    if self.isEmpty():
        return -1
    return self.queue[self.front]

def Rear(self):
    # Return -1 if the queue is empty
    if self.isEmpty():
        return -1

    # Since rear points to the next insertion position, get the last
    # element index
    return self.queue[(self.rear - 1 + self.capacity) % self.capacity]

def isEmpty(self):
    # Check if count is 0
    return self.count == 0

def isFull(self):
    # Check if the queue has reached its capacity
    return self.count == self.capacity

```

• LC.ca

```

class MyCircularQueue:
    def __init__(self, k: int):
        self.q = [0] * k
        self.front = 0
        self.size = 0
        self.capacity = k

    def enqueue(self, value: int) -> bool:
        if self.isFull():
            return False

```

Round 5 · Mid · Medium

p.2227

```

        idx = (self.front + self.size) % self.capacity
        self.q[idx] = value
        self.size += 1
        return True

    def dequeue(self) -> bool:
        if self.isEmpty():
            return False
        self.front = (self.front + 1) % self.capacity
        self.size -= 1

        return True

    def Front(self) -> int:
        return -1 if self.isEmpty() else self.q[self.front]

    def Rear(self) -> int:
        if self.isEmpty():
            return -1
        idx = (self.front + self.size - 1) % self.capacity
        return self.q[idx]

    def isEmpty(self) -> bool:
        return self.size == 0

    def isFull(self) -> bool:
        return self.size == self.capacity

```

Your MyCircularQueue object will be instantiated and called as such:
obj = MyCircularQueue(k)

```

# param_1 = obj.enqueue(value)
# param_2 = obj.dequeue()
# param_3 = obj.front()
# param_4 = obj.rear()
# param_5 = obj.isEmpty()
# param_6 = obj.isFull()

```

#####

```

class MyCircularQueue(object):

```

Round 5 · Mid · Medium

p.2228

```
def __init__(self, k):
    """
    Initialize your data structure here. Set the size of the queue to be k.
    :type k: int
    """
    self.queue = []
    self.size = k
    self.front = 0
    self.rear = 0

def enqueue(self, value):
    """
    Insert an element into the circular queue. Return true if the operation
    is successful.
    :type value: int
    :rtype: bool
    """
    if self.rear - self.front < self.size:
        self.queue.append(value)
        self.rear += 1

    return True
else:
    return False

def dequeue(self):
    """
    Delete an element from the circular queue. Return true if the operation
    is successful.
    :rtype: bool
    """
    if self.rear - self.front > 0:
```

Round 5 · Mid · Medium

p.2229

```
        self.front += 1
        return True
    else:
        return False

def Front(self):
    """
    Get the front item from the queue.
    :rtype: int
    """

    if self.isEmpty():
        return -1
    else:
        return self.queue[self.front]

def Rear(self):
    """
    Get the last item from the queue.
    :rtype: int
    """

    if self.isEmpty():
        return -1
    else:
        return self.queue[self.rear - 1]

def isEmpty(self):
    """
    Checks whether the circular queue is empty or not.
    :rtype: bool
    """

    return self.front == self.rear

def isFull(self):
    """
    Checks whether the circular queue is full or not.
    :rtype: bool
    """
    return self.rear - self.front == self.size
```

Round 5 · Mid · Medium

p.2230

```
# Your MyCircularQueue object will be instantiated and called as such:
# obj = MyCircularQueue(k)
# param_1 = obj.enqueue(value)
# param_2 = obj.dequeue()
# param_3 = obj.front()
# param_4 = obj.rear()
# param_5 = obj.isEmpty()
# param_6 = obj.isFull()
```

◆ Quick Review

Key Insights

- Leverage a fixed-size array to store the queue elements.
- Use two pointers: one for the front and one for the rear.
- Maintain a count variable (or use pointer arithmetic) to differentiate between an empty and a full queue.
- Use modulo arithmetic to wrap pointers when they reach the end of the array.
- Ensure all operations (enqueue, dequeue, front, rear, isEmpty, isFull) have $O(1)$ time complexity.

Space & Time

Time Complexity: $O(1)$ per operation for enqueue, dequeue, front, rear, isEmpty, and

isFull.

Space Complexity: $O(k)$, where k is the capacity of the circular queue.

Solution

We implement the circular queue using a fixed-size array alongside two pointers, front and rear, and a counter to keep track of the number of elements in the queue. The front pointer indicates the index of the first element, and the rear pointer indicates the index where the next element will be enqueued. When enqueueing an element, we insert the element at the rear index and then increment the rear pointer modulo the size of the array. Similarly, when dequeuing, we remove the element at the front index and increment the front pointer modulo the size. The counter helps in quickly checking if the queue is empty or full (count equals 0 or count equals capacity, respectively). This design ensures all operations remain $O(1)$ and correctly wraps around, forming a circular structure.

Data Structure Design

#641 Design Circular Deque

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Linked List · Design · Queue
 Lists: AlgoMaster 600

Problem

Design your implementation of the circular double-ended queue (deque).

Implement the MyCircularDeque class:

- **MyCircularDeque(int k)** Initializes the deque with a maximum size of k.
- **boolean insertFront()** Adds an item at the front of Deque. Returns true if the operation is successful, or false otherwise.
- **boolean insertLast()** Adds an item at the rear of Deque. Returns true if the operation is successful, or false otherwise.
- **boolean deleteFront()** Deletes an item from the front of Deque. Returns true if the operation is successful, or false otherwise.

Round 5 · Mid · Medium

p.2233

- **boolean deleteLast()** Deletes an item from the rear of Deque. Returns true if the operation is successful, or false otherwise.
- **int getFront()** Returns the front item from the Deque. Returns -1 if the deque is empty.
- **int getRear()** Returns the last item from Deque. Returns -1 if the deque is empty.
- **boolean isEmpty()** Returns true if the deque is empty, or false otherwise.
- **boolean isFull()** Returns true if the deque is full, or false otherwise.

Example 1:

```
Input
["MyCircularDeque", "insertLast", "insertLast", "insertFront", "insertFront",
"getRear", "isFull", "deleteLast", "insertFront", "getFront"]
[[3], [1], [2], [3], [4], [], [], [], [4], []]
Output
[null, true, true, true, false, 2, true, true, true, 4]
```

```
Explanation
MyCircularDeque myCircularDeque = new MyCircularDeque(3);
```

Constraints:

- $1 \leq k \leq 1000$
- $0 \leq \text{value} \leq 1000$
- At most 2000 calls will be made to insertFront, insertLast, deleteFront, deleteLast, getFront, getRear, isEmpty, isFull.

Round 5 · Mid · Medium

p.2234

Community Solutions (Python)

• LeetCode

```
class MyCircularDeque:
    def __init__(self, k: int):
        """
        Initialize your data structure here. Set the size of the deque to be k.
        """
        self.q = [0] * k
        self.front = 0
        self.size = 0
        self.capacity = k

    def insertFront(self, value: int) -> bool:
        """
        Adds an item at the front of Deque. Return true if the operation is
        successful.
        """
        if self.isFull():
            return False
        if not self.isEmpty():
            self.front = (self.front - 1 + self.capacity) % self.capacity
        self.q[self.front] = value
        self.size += 1

        return True

    def insertLast(self, value: int) -> bool:
        """
        Adds an item at the rear of Deque. Return true if the operation is
        successful.
        """
        if self.isFull():
            return False
        idx = (self.front + self.size) % self.capacity
        self.q[idx] = value
```

Round 5 · Mid · Medium

p.2235

```
        self.size += 1
        return True

    def deleteFront(self) -> bool:
        """
        Deletes an item from the front of Deque. Return true if the operation
        is successful.
        """
        if self.isEmpty():
            return False
        self.front = (self.front + 1) % self.capacity

        self.size -= 1
        return True

    def deleteLast(self) -> bool:
        """
        Deletes an item from the rear of Deque. Return true if the operation is
        successful.
        """
        if self.isEmpty():
            return False
        self.size -= 1

        return True

    def getFront(self) -> int:
        """
        Get the front item from the deque.
        """
        if self.isEmpty():
            return -1
        return self.q[self.front]
```

Round 5 · Mid · Medium

p.2236

```
def getRear(self) -> int:
    """
    Get the last item from the deque.
    """
    if self.isEmpty():
        return -1
    idx = (self.front + self.size - 1) % self.capacity
    return self.q[idx]
```

```
def isEmpty(self) -> bool:
```

```
    """
    Checks whether the circular deque is empty or not.
    """
    return self.size == 0
```

```
def isFull(self) -> bool:
```

```
    """
    Checks whether the circular deque is full or not.
    """
    return self.size == self.capacity
```

```
# Your MyCircularDeque object will be instantiated and called as such:
# obj = MyCircularDeque(k)
# param_1 = obj.insertFront(value)
# param_2 = obj.insertLast(value)
# param_3 = obj.deleteFront()
# param_4 = obj.deleteLast()
# param_5 = obj.getFront()
# param_6 = obj.getRear()
```

```
# param_7 = obj.isEmpty()
# param_8 = obj.isFull()
```

• SimplyLeet

Round 5 · Mid · Medium

p.2237

```
class MyCircularDeque:
    def __init__(self, k: int):
        # Initialize the deque with a fixed capacity of k
        self.capacity = k
        self.deque = [0] * k
        self.size = 0
        self.front = 0          # Points to the current front element
        self.rear = 0           # Points to the next insertion position at rear
```

```
    def insertFront(self, value: int) -> bool:
```

```
        if self.isFull():
            return False
        # Move front pointer backward using modulo arithmetic
        self.front = (self.front - 1 + self.capacity) % self.capacity
        self.deque[self.front] = value
        self.size += 1
        return True
```

```
    def insertLast(self, value: int) -> bool:
        if self.isFull():
```

```
            return False
        # Insert element at the rear pointer and update it circularly
        self.deque[self.rear] = value
        self.rear = (self.rear + 1) % self.capacity
        self.size += 1
        return True
```

```
    def deleteFront(self) -> bool:
        if self.isEmpty():
            return False
```

Round 5 · Mid · Medium

p.2238

```

# Move front pointer forward to delete the current front element
self.front = (self.front + 1) % self.capacity
self.size -= 1
return True

def deleteLast(self) -> bool:
    if self.isEmpty():
        return False
    # Move rear pointer backward to delete the last element
    self.rear = (self.rear - 1 + self.capacity) % self.capacity

    self.size -= 1
    return True

def getFront(self) -> int:
    if self.isEmpty():
        return -1
    return self.deque[self.front]

def getRear(self) -> int:
    if self.isEmpty():
        return -1
    # Since rear pointer indicates next inserting slot, adjust to get last
    # element
    return self.deque[(self.rear - 1 + self.capacity) % self.capacity]

def isEmpty(self) -> bool:
    return self.size == 0

def isFull(self) -> bool:
    return self.size == self.capacity

```

Data Structure Design

□ #981 Time Based Key-Value Store

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Binary Search · Design
Lists: Grind 169

Problem

Design a time-based key-value data structure that can store multiple values for the same key at different time stamps and retrieve the key's value at a certain timestamp.

Implement the TimeMap class:

- TimeMap() Initializes the object of the data structure.
- void set(String key, String value, int timestamp) Stores the key key with the value value at the given time timestamp.
- String get(String key, int timestamp) Returns a value such that set was called previously, with timestamp_prev ≤ timestamp. If there are multiple such values, it returns the value associated with the largest timestamp_prev. If

there are no values, it returns "".

Example 1:

```
Input
["TimeMap", "set", "get", "get", "set", "get", "get"]
[[], ["foo", "bar", 1], ["foo", 1], ["foo", 3], ["foo", "bar2", 4], ["foo", 4],
["foo", 5]]
Output
[null, null, "bar", "bar", null, "bar2", "bar2"]
```

```
Explanation
TimeMap timeMap = new TimeMap();
```

Constraints:

- $1 \leq \text{key.length}$, $\text{value.length} \leq 100$
- key and value consist of lowercase English letters and digits.
- $1 \leq \text{timestamp} \leq 107$
- All the timestamps timestamp of set are strictly increasing.
- At most $2 * 10^5$ calls will be made to set and get.

Community Solutions (Python)

- WalkCC

```
class TimeMap:
    def __init__(self):
        self.values = collections.defaultdict(list)
        self.timestamps = collections.defaultdict(list)

    def set(self, key: str, value: str, timestamp: int) -> None:
        self.values[key].append(value)
        self.timestamps[key].append(timestamp)

    def get(self, key: str, timestamp: int) -> str:
        if key not in self.timestamps:
            return ''
        i = bisect.bisect(self.timestamps[key], timestamp)
        return self.values[key][i - 1] if i > 0 else ''
```

• LeetCode

```
class TimeMap:
    def __init__(self):
        self.ktv = defaultdict(list)

    def set(self, key: str, value: str, timestamp: int) -> None:
        self.ktv[key].append((timestamp, value))

    def get(self, key: str, timestamp: int) -> str:
        if key not in self.ktv:
            return ''

        tv = self.ktv[key]
        i = bisect_right(tv, (timestamp, chr(127)))
        return tv[i - 1][1] if i else ''

# Your TimeMap object will be instantiated and called as such:
# obj = TimeMap()
# obj.set(key,value,timestamp)
# param_2 = obj.get(key,timestamp)
```

• SimplyLeet

```
# TimeMap class in Python with line-by-line comments.
class TimeMap:
    def __init__(self):
        # Initialize the dictionary to hold key and list of (timestamp, value)
        # pairs.
        self.store = {}

    def set(self, key: str, value: str, timestamp: int) -> None:
        # Append the new (timestamp, value) pair for the given key.
        if key not in self.store:
            self.store[key] = []

        self.store[key].append((timestamp, value))

    def get(self, key: str, timestamp: int) -> str:
        # Return the value for the largest timestamp <= the given timestamp.
        if key not in self.store:
            return ""

        values = self.store[key]
        left, right = 0, len(values) - 1

        # Binary search for the rightmost index with timestamp <= given
        # timestamp.
        while left <= right:
            mid = (left + right) // 2
            if values[mid][0] <= timestamp:
                left = mid + 1
            else:
                right = mid - 1

        # If no valid timestamp exists, right will be -1.
        if right == -1:
```

```
        return ""
        return values[right][1]

# Example usage:
# timeMap = TimeMap()
# timeMap.set("foo", "bar", 1)
# print(timeMap.get("foo", 1)) # Output: "bar"
# print(timeMap.get("foo", 3)) # Output: "bar"
# timeMap.set("foo", "bar2", 4)
# print(timeMap.get("foo", 4)) # Output: "bar2"

# print(timeMap.get("foo", 5)) # Output: "bar2"
```

• LC.ca

```
class TimeMap:
    def __init__(self):
        self.ktv = defaultdict(list)

    def set(self, key: str, value: str, timestamp: int) -> None:
        self.ktv[key].append((timestamp, value))

    def get(self, key: str, timestamp: int) -> str:
        if key not in self.ktv:
            return ""

        tv = self.ktv[key]
        i = bisect_right(tv, (timestamp, chr(127)))
        return tv[i - 1][1] if i else ""

# Your TimeMap object will be instantiated and called as such:
# obj = TimeMap()
# obj.set(key,value,timestamp)
# param_2 = obj.get(key,timestamp)
```

◆ Quick Review

Key Insights

- Use a dictionary (hash table) to map each key to a list of (timestamp, value) pairs.
- Timestamps are strictly increasing when setting the values, which allows efficient appending.
- For getting the value, perform a binary search on the list of timestamps to find the largest timestamp that is $\leq$ the queried timestamp.
- If no valid timestamp exists, return an empty string.

Space & Time

Time Complexity:

- set operation: $O(1)$ per call (amortized).
- get operation: $O(\log N)$ per call, where N is the number of timestamps stored for the key.

Space Complexity: $O(\text{total number of set calls})$, to store all (timestamp, value) pairs.

Solution

We use a hash table to store keys mapping to a list of their timestamps and corresponding values. Since the timestamps are inserted in strictly increasing order, the list remains

sorted. For the get operation, a binary search is applied to quickly locate the appropriate value for a given timestamp. This approach minimizes the lookup time and efficiently handles the problem constraints.

Data Structure Design

#1146 Snapshot Array

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table · Binary Search · Design
 Lists: AlgoMaster 600

Problem

Implement a SnapshotArray that supports the following interface:

- SnapshotArray(int length) initializes an array-like data structure with the given length. Initially, each element equals 0.
- void set(index, val) sets the element at the given index to be equal to val.
- int snap() takes a snapshot of the array and returns the snap_id: the total number of times we called snap() minus 1.
- int get(index, snap_id) returns the value at the given index, at the time we took the snapshot with the given snap_id

Example 1:

```
Input: ["SnapshotArray","set","snap","set","get"]
[[3],[0,5],[],[0,6],[0,0]]
Output: [null,null,0,null,5]
Explanation:
SnapshotArray snapshotArr = new SnapshotArray(3); // set the length to be 3
snapshotArr.set(0,5); // Set array[0] = 5
snapshotArr.snap(); // Take a snapshot, return snap_id = 0
snapshotArr.set(0,6);
```

Constraints:

- $1 \leq \text{length} \leq 5 * 10^4$
- $0 \leq \text{index} < \text{length}$
- $0 \leq \text{val} \leq 10^9$
- $0 \leq \text{snap_id} < (\text{the total number of times we call snap()})$
- At most $5 * 10^4$ calls will be made to set, snap, and get.

Community Solutions (Python)

• WalkCC

```
class SnapshotArray:
    def __init__(self, length: int):
        self.snaps = [[0, 0] for _ in range(length)]
        self.snap_id = 0

    def set(self, index: int, val: int) -> None:
        snap = self.snaps[index][-1]
        if snap[0] == self.snap_id:
            snap[1] = val
        else:
```



```

        self.snaps[index].append([self.snap_id, val])

    def snap(self) -> int:
        self.snap_id += 1
        return self.snap_id - 1

    def get(self, index: int, snap_id: int) -> int:
        i = bisect.bisect_left(self.snaps[index], [snap_id + 1]) - 1
        return self.snaps[index][i][1]

```

• LeetCode

```

class SnapshotArray:

    def __init__(self, length: int):
        self.arr = [[] for _ in range(length)]
        self.i = 0

    def set(self, index: int, val: int) -> None:
        self.arr[index].append((self.i, val))

    def snap(self) -> int:

        self.i += 1
        return self.i - 1

    def get(self, index: int, snap_id: int) -> int:
        i = bisect_left(self.arr[index], (snap_id, inf)) - 1
        return 0 if i < 0 else self.arr[index][i][1]

# Your SnapshotArray object will be instantiated and called as such:
# obj = SnapshotArray(length)

# obj.set(index,val)
# param_2 = obj.snap()
# param_3 = obj.get(index,snap_id)

```

• SimplyLeet

```

class SnapshotArray:
    def __init__(self, length: int):
        # Initialize a history for each index with an initial value 0 at
        # snap_id 0.
        self.data = [[(0, 0)] for _ in range(length)]
        # Current snapshot id starts at 0.
        self.snap_id = 0

    def set(self, index: int, val: int) -> None:
        # Get the history list for the index.
        history = self.data[index]

        # If the most recent set was made in the current snapshot, update it.
        if history[-1][0] == self.snap_id:
            history[-1] = (self.snap_id, val)
        else:
            # Otherwise, append a new record for this snapshot.
            history.append((self.snap_id, val))

    def snap(self) -> int:
        # Return the current snap_id then increment for the next snapshot.
        self.snap_id += 1

        return self.snap_id - 1

    def get(self, index: int, snap_id: int) -> int:
        history = self.data[index]
        # Binary search for the right-most record not exceeding snap_id.
        l, r = 0, len(history) - 1
        while l <= r:
            mid = (l + r) // 2
            if history[mid][0] <= snap_id:
                l = mid + 1

```

```

        else:
            r = mid - 1
            # r holds the latest index where the snapshot id is <= snap_id.
            return history[r][1]

# Example usage:
# snapshotArr = SnapshotArray(3)
# snapshotArr.set(0, 5)
# print(snapshotArr.snap()) # returns 0
# snapshotArr.set(0, 6)

# print(snapshotArr.get(0, 0)) # returns 5

```

Data Structure Design

□ #1472 Design Browser History

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Linked List · Stack · Design ·
 Doubly-Linked List · Data Stream
 Lists: AlgoMaster 600

Problem

You have a browser of one tab where you start on the homepage and you can visit another url, get back in the history number of steps or move forward in the history number of steps.

Implement the BrowserHistory class:

- **BrowserHistory(string homepage)** Initializes the object with the homepage of the browser.
- **void visit(string url)** Visits url from the current page. It clears up all the forward history.
- **string back(int steps)** Move steps back in history. If you can only return x steps in the history and steps > x, you will return only x steps. Return the current url after moving

back in history at most steps.

• string forward(int steps) Move steps forward in history. If you can only forward x steps in the history and steps > x, you will forward only x steps. Return the current url after forwarding in history at most steps.

Example:

Input:
 ["BrowserHistory","visit","visit","visit","back","back","forward","visit","forward","back","back"]
 [{"leetcode.com"}, {"google.com"}, {"facebook.com"}, {"youtube.com"}, [1], [1], [1], [{"linkedin.com"}, [2], [2], [7]]
 Output:
 [null,null,null,null,"facebook.com","google.com","facebook.com",null,"linkedin.com","google.com","leetcode.com"]

Explanation:

BrowserHistory browserHistory = new BrowserHistory("leetcode.com");

Constraints:

- $1 \leq \text{homepage.length} \leq 20$
- $1 \leq \text{url.length} \leq 20$
- $1 \leq \text{steps} \leq 100$
- homepage and url consist of '.' or lower case English letters.
- At most 5000 calls will be made to visit, back, and forward.

Community Solutions (Python)

- WalkCC

Round 5 · Mid · Medium

p.2253

```
class BrowserHistory:
    def __init__(self, homepage: str):
        self.urls = []
        self.index = -1
        self.lastIndex = -1
        self.visit(homepage)

    def visit(self, url: str) -> None:
        self.index += 1
        if self.index < len(self.urls):

            self.urls[self.index] = url
        else:
            self.urls.append(url)
        self.lastIndex = self.index

    def back(self, steps: int) -> str:
        self.index = max(0, self.index - steps)
        return self.urls[self.index]

    def forward(self, steps: int) -> str:

        self.index = min(self.lastIndex, self.index + steps)
        return self.urls[self.index]

class BrowserHistory:
    def __init__(self, homepage: str):
        self.history = []
        self.visit(homepage)

    def visit(self, url: str) -> None:
        self.history.append(url)
        self.future = []

    def back(self, steps: int) -> str:
```

Round 5 · Mid · Medium

p.2254

```

while len(self.history) > 1 and steps > 0:
    self.future.append(self.history.pop())
    steps -= 1
return self.history[-1]

```

```

def forward(self, steps: int) -> str:
    while self.future and steps > 0:
        self.history.append(self.future.pop())
        steps -= 1
    return self.history[-1]

```

```

class Node:
    def __init__(self, url: str):
        self.prev = None
        self.next = None
        self.url = url

```

```

class BrowserHistory:
    def __init__(self, homepage: str):
        self.curr = Node(homepage)

```

```

def visit(self, url: str) -> None:
    self.curr.next = Node(url)
    self.curr.next.prev = self.curr
    self.curr = self.curr.next

```

```

def back(self, steps: int) -> str:
    while self.curr.prev and steps > 0:
        self.curr = self.curr.prev
        steps -= 1

```

```

return self.curr.url

```

```

def forward(self, steps: int) -> str:
    while self.curr.next and steps > 0:
        self.curr = self.curr.next
        steps -= 1
    return self.curr.url

```

• LeetDoocs

Round 5 · Mid · Medium

p.2255

```

class BrowserHistory:
    def __init__(self, homepage: str):
        self.stk1 = []
        self.stk2 = []
        self.visit(homepage)

```

```

def visit(self, url: str) -> None:
    self.stk1.append(url)
    self.stk2.clear()

```

```

def back(self, steps: int) -> str:
    while steps and len(self.stk1) > 1:
        self.stk2.append(self.stk1.pop())
        steps -= 1
    return self.stk1[-1]

```

```

def forward(self, steps: int) -> str:
    while steps and self.stk2:
        self.stk1.append(self.stk2.pop())
        steps -= 1

```

```

return self.stk1[-1]

```

```

# Your BrowserHistory object will be instantiated and called as such:
# obj = BrowserHistory(homepage)
# obj.visit(url)
# param_2 = obj.back(steps)
# param_3 = obj.forward(steps)

```

• SimplyLeet

Round 5 · Mid · Medium

p.2256

Python Implementation of BrowserHistory

```
class BrowserHistory:
    def __init__(self, homepage: str):
        # Initialize history with the homepage and set current index to 0
        self.history = [homepage]
        self.curr = 0

    def visit(self, url: str) -> None:
        # Clear forward history by slicing the current history up to and
        # including current page

        self.history = self.history[:self.curr + 1]
        # Append the new URL to history
        self.history.append(url)
        # Move current pointer to the end of the history
        self.curr += 1

    def back(self, steps: int) -> str:
        # Move the pointer 'steps' back but don't go below index 0
        self.curr = max(0, self.curr - steps)
        # Return the URL at the new current position

        return self.history[self.curr]

    def forward(self, steps: int) -> str:
        # Move the pointer 'steps' forward but don't exceed the last index in
        # history
        self.curr = min(len(self.history) - 1, self.curr + steps)
        # Return the URL at the new current position
        return self.history[self.curr]
```

Data Structure Design

□ #2353 Design a Food Rating System

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table · String · Design · Heap
 (Priority Queue) · Ordered Set
 Lists: AlgoMaster 600

Problem

Design a food rating system that can do the following:

- Modify the rating of a food item listed in the system.
- Return the highest-rated food item for a type of cuisine in the system.

Implement the FoodRatings class:

- FoodRatings(String[] foods, String[] cuisines, int[] ratings) Initializes the system. The food items are described by foods, cuisines and ratings, all of which have a length of n. foods[i] is the name of the ith food,
- cuisines[i] is the type of cuisine of the ith food, and

• ratings[i] is the initial rating of the ith food. Note that a string x is lexicographically smaller than string y if x comes before y in dictionary order, that is, either x is a prefix of y, or if i is the first position such that x[i] != y[i], then x[i] comes before y[i] in alphabetic order.

Example 1:

```
Input
["FoodRatings", "highestRated", "highestRated", "changeRating",
 "highestRated", "changeRating", "highestRated"]
[[["kimchi", "miso", "sushi", "moussaka", "ramen", "bulgogi"], ["korean",
 "japanese", "japanese", "greek", "japanese", "korean"], [9, 12, 8, 15, 14, 7]],
 [{"korean"}, {"japanese"}, {"sushi", 16}, {"japanese"}, {"ramen", 16},
 {"japanese"}]]
Output
[null, "kimchi", "ramen", null, "sushi", null, "ramen"]
```

Explanation
FoodRatings foodRatings = new FoodRatings(["kimchi", "miso", "sushi", "moussaka", "ramen", "bulgogi"], ["korean", "japanese", "japanese", "greek", "japanese", "korean"], [9, 12, 8, 15, 14, 7]);

Constraints:

- $1 \leq n \leq 2 * 10^4$
- $n == \text{foods.length} == \text{cuisines.length} == \text{ratings.length}$
- $1 \leq \text{foods}[i].\text{length}, \text{cuisines}[i].\text{length} \leq 10$
- foods[i], cuisines[i] consist of lowercase English letters.
- $1 \leq \text{ratings}[i] \leq 10^8$

- All the strings in foods are distinct.
- food will be the name of a food item in the system across all calls to changeRating.
- cuisine will be a type of cuisine of at least one food item in the system across all calls to highestRated.
- At most $2 * 10^4$ calls in total will be made to changeRating and highestRated.

Community Solutions (Python)

• WalkCC

```
from sortedcontainers import SortedSet

class FoodRatings:
    def __init__(self, foods: list[str], cuisines: list[str], ratings: list[int]):
        self.cuisineToRatingAndFoods = collections.defaultdict(
            lambda: SortedSet(key=lambda x: (-x[0], x[1])))
        self.foodToCuisine = {}
        self.foodToRating = {}

        for food, cuisine, rating in zip(foods, cuisines, ratings):
            self.cuisineToRatingAndFoods[cuisine].add((rating, food))
            self.foodToCuisine[food] = cuisine
            self.foodToRating[food] = rating

        def changeRating(self, food: str, newRating: int) -> None:
            cuisine = self.foodToCuisine[food]
            oldRating = self.foodToRating[food]
            ratingAndFoods = self.cuisineToRatingAndFoods[cuisine]
            ratingAndFoods.remove((oldRating, food))
```

```
ratingAndFoods.add((newRating, food))
self.foodToRating[food] = newRating

def highestRated(self, cuisine: str) -> str:
    return self.cuisineToRatingAndFoods[cuisine][0][1]
```

• LeetCode

```
class FoodRatings:

    def __init__(self, foods: List[str], cuisines: List[str], ratings: List[int]):
        self.d = defaultdict(SortedList)
        self.g = {}
        for food, cuisine, rating in zip(foods, cuisines, ratings):
            self.d[cuisine].add((-rating, food))
            self.g[food] = (rating, cuisine)

    def changeRating(self, food: str, newRating: int) -> None:

        oldRating, cuisine = self.g[food]
        self.g[food] = (newRating, cuisine)
        self.d[cuisine].remove((-oldRating, food))
        self.d[cuisine].add((-newRating, food))

    def highestRated(self, cuisine: str) -> str:
        return self.d[cuisine][0][1]

# Your FoodRatings object will be instantiated and called as such:

# obj = FoodRatings(foods, cuisines, ratings)
# obj.changeRating(food,newRating)
# param_2 = obj.highestRated(cuisine)
```

• SimplyLeet

Round 5 · Mid · Medium

p.2261

```
import heapq

class FoodRatings:
    def __init__(self, foods, cuisines, ratings):
        # Map food to its current rating.
        self.food_to_rating = {}
        # Map food to its cuisine.
        self.food_to_cuisine = {}
        # Map each cuisine to a heap of (-rating, food) pairs.
        self.cuisine_to_heap = {}

    for food, cuisine, rating in zip(foods, cuisines, ratings):
        self.food_to_rating[food] = rating
        self.food_to_cuisine[food] = cuisine
        if cuisine not in self.cuisine_to_heap:
            self.cuisine_to_heap[cuisine] = []
        # Store negative rating to simulate a max-heap; food name ensures
        # lexicographic order.
        heapq.heappush(self.cuisine_to_heap[cuisine], (-rating, food))

    def changeRating(self, food, newRating):

        # Retrieve corresponding cuisine.
        cuisine = self.food_to_cuisine[food]
        # Update the food's rating.
        self.food_to_rating[food] = newRating
        # Push the new rating into the cuisine heap.
        heapq.heappush(self.cuisine_to_heap[cuisine], (-newRating, food))

    def highestRated(self, cuisine):
        heap = self.cuisine_to_heap[cuisine]
        # Remove stale entries until the top of the heap is valid.

        while heap:
            rating_neg, food = heap[0]
            if -rating_neg == self.food_to_rating[food]:
                return food
            heapq.heappop(heap)
        return ""
```

Round 5 · Mid · Medium

p.2262

Greedy

Round 5 · Mid · Medium · 12 questions

Round 5 · Mid · Medium

p.2263

Greedy

□ #45 Jump Game II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Dynamic Programming · Greedy
 Lists: NeetCode 150

Problem

You are given a 0-indexed array of integers `nums` of length `n`. You are initially positioned at index 0.

Each element `nums[i]` represents the maximum length of a forward jump from index `i`. In other words, if you are at index `i`, you can jump to any index `(i + j)` where:

- $0 \leq j \leq \text{nums}[i]$ and
- $i + j < n$

Return the minimum number of jumps to reach index `n - 1`. The test cases are generated such that you can reach index `n - 1`.

Example 1:

Input: `nums = [2,3,1,1,4]`

Output: 2

Explanation: The minimum number of jumps to reach the last index is 2. Jump 1 step from index 0 to 1, then 3 steps to the last index.

Round 5 · Mid · Medium

p.2264

Example 2:

Input: nums = [2,3,0,1,4]
Output: 2

Constraints:

- $1 \leq \text{nums.length} \leq 104$
- $0 \leq \text{nums}[i] \leq 1000$
- It's guaranteed that you can reach $\text{nums}[n - 1]$.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def jump(self, nums: list[int]) -> int:
        ans = 0
        end = 0
        farthest = 0

        # Start an implicit BFS.
        for i in range(len(nums) - 1):
            farthest = max(farthest, i + nums[i])
            if farthest >= len(nums) - 1:
                ans += 1
                break
        if i == end:
            # Visited all the items on the current level.
            ans += 1
            end = farthest
            # Make the queue size for the next level.

        return ans
```

• LeetDoocs

```
class Solution:
    def jump(self, nums: List[int]) -> int:
        ans = mx = last = 0
        for i, x in enumerate(nums[:-1]):
            mx = max(mx, i + x)
            if last == i:
                ans += 1
                last = mx
        return ans
```

• SimplyLeet

```
# Function to calculate minimum number of jumps to reach the last index
def jump(nums):
    # If the list has only one element, we're already at the destination.
    if len(nums) == 1:
        return 0

    n = len(nums) # Length of the input array
    jumps = 0     # To count the number of jumps required
    currentEnd = 0 # The end of the current range that can be reached
    furthest = 0  # The furthest index that can be reached from the current range

    # Iterate till the second last element because we don't need to jump from the last element
    for i in range(n - 1):
        # Update the furthest index reachable from index i
        furthest = max(furthest, i + nums[i])
        # If we have reached the end of the current jump range
        if i == currentEnd:
            jumps += 1 # Increase the jump count as we make a jump
            currentEnd = furthest # Update the currentEnd to the furthest reachable index
    return jumps

# Example usage:
print(jump([2,3,1,1,4])) # Output: 2
```

Greedy

#55 Jump Game

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Dynamic Programming · Greedy
 Lists: Grind 169

Problem

You are given an integer array `nums`. You are initially positioned at the array's first index, and each element in the array represents your maximum jump length at that position.

Return `true` if you can reach the last index, or `false` otherwise.

Example 1:

Input: `nums = [2,3,1,1,4]`
 Output: `true`
 Explanation: Jump 1 step from index 0 to 1, then 3 steps to the last index.

Example 2:

Input: `nums = [3,2,1,0,4]`
 Output: `false`
 Explanation: You will always arrive at index 3 no matter what. Its maximum jump length is 0, which makes it impossible to reach the last index.

Constraints:

- $1 \leq \text{nums.length} \leq 104$

Round 5 · Mid · Medium

p.2267

- $0 \leq \text{nums}[i] \leq 105$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def canJump(self, nums: List[int]) -> bool:
        i = 0
        reach = 0

        while i < len(nums) and i <= reach:
            reach = max(reach, i + nums[i])
            i += 1

        return i == len(nums)
```

• LeetDoocs

```
class Solution:
    def canJump(self, nums: List[int]) -> bool:
        mx = 0
        for i, x in enumerate(nums):
            if mx < i:
                return False
            mx = max(mx, i + x)
        return True
```

• SimplyLeet

```
# Function to determine if you can reach the last index of the array
def canJump(nums):
    # Initialize maxReach which is the max index reachable at start (index 0)
    maxReach = 0
    # Iterate through each element and its index
    for i, jump in enumerate(nums):
        # If current index is greater than maximum reachable index, return
        False
        if i > maxReach:
            return False
        # Update maxReach with the maximum of current maxReach and i + jump
        length at index i
```

Round 5 · Mid · Medium

p.2268

```

maxReach = max(maxReach, i + jump)
# If current maxReach is already at or beyond the last index, return
True
    if maxReach >= len(nums) - 1:
        return True
# Return True if the end is reachable after processing all indices
return True

# Example usage:
# print(canJump([2,3,1,1,4])) # Expected output: True
# print(canJump([3,2,1,0,4])) # Expected output: False

```

• LC.ca

```

class Solution:
    def canJump(self, nums: List[int]) -> bool:
        mx = 0
        for i, x in enumerate(nums):
            if mx < i:
                return False
            mx = max(mx, i + x)
        return True

```

◆ Quick Review

Key Insights

- The greedy approach is optimal for this problem.
- Track the furthest index reachable as you iterate through the array.
- If the current index exceeds the maximum reachable index, it is impossible to proceed further.

- The solution only requires constant extra space.

Space & Time

Time Complexity: $O(n)$ where n is the length of the array.

Space Complexity: $O(1)$

Solution

The solution uses a greedy algorithm. As you iterate through each array element, you maintain a variable (`maxReach`) that represents the furthest index you can reach so far. For each index i , if i is greater than `maxReach` then it is not possible to reach index i and hence return false immediately. Otherwise, update `maxReach` to the maximum of its current value and $i + \text{nums}[i]$. If you can iterate through the array without i exceeding `maxReach`, then you can reach the end and return true.

Greedy

☐ #134 Gas Station

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode

Array · Greedy

Lists: Grind 169

Problem

There are n gas stations along a circular route, where the amount of gas at the i th station is $gas[i]$.

You have a car with an unlimited gas tank and it costs $cost[i]$ of gas to travel from the i th station to its next $(i + 1)$ th station. You begin the journey with an empty tank at one of the gas stations.

Given two integer arrays gas and $cost$, return the starting gas station's index if you can travel around the circuit once in the clockwise direction, otherwise return -1 . If there exists a solution, it is guaranteed to be unique.

Example 1:

```
Input: gas = [1,2,3,4,5], cost = [3,4,5,1,2]
Output: 3
Explanation:
Start at station 3 (index 3) and fill up with 4 unit of gas. Your tank = 0 + 4 = 4
Travel to station 4. Your tank = 4 - 1 + 5 = 8
Travel to station 0. Your tank = 8 - 2 + 1 = 7
Travel to station 1. Your tank = 7 - 3 + 2 = 6
Travel to station 2. Your tank = 6 - 4 + 3 = 5
```

Example 2:

```
Input: gas = [2,3,4], cost = [3,4,3]
Output: -1
Explanation:
You can't start at station 0 or 1, as there is not enough gas to travel to the next station.
Let's start at station 2 and fill up with 4 unit of gas. Your tank = 0 + 4 = 4
Travel to station 0. Your tank = 4 - 3 + 2 = 3
Travel to station 1. Your tank = 3 - 3 + 3 = 3
You cannot travel back to station 2, as it requires 4 unit of gas but you only have 3.
```

Constraints:

- $n == gas.length == cost.length$
- $1 \leq n \leq 10^5$
- $0 \leq gas[i], cost[i] \leq 10^4$
- The input is generated such that the answer is unique.

Community Solutions (Python)

- WalkCC

```
class Solution:
    def canCompleteCircuit(self, gas: list[int], cost: list[int]) -> int:
        ans = 0
        net = 0
        summ = 0

        # Try to start from each index.
        for i in range(len(gas)):
            net += gas[i] - cost[i]
            summ += gas[i] - cost[i]

        if summ < 0:
            summ = 0
            ans = i + 1 # Start from the next index.

        return -1 if net < 0 else ans
```

• LeetCode

```
class Solution:
    def canCompleteCircuit(self, gas: List[int], cost: List[int]) -> int:
        n = len(gas)
        i = j = n - 1
        cnt = s = 0
        while cnt < n:
            s += gas[j] - cost[j]
            cnt += 1
            j = (j + 1) % n
            while s < 0 and cnt < n:
                i -= 1
                s += gas[i] - cost[i]
                cnt += 1
        return -1 if s < 0 else i
```

• SimplyLeet

Round 5 · Mid · Medium

p.2273

```
# Define a function to find starting gas station index
def canCompleteCircuit(gas, cost):
    total_surplus = 0 # Total gas surplus after completing the route
    current_surplus = 0 # Gas surplus while iterating from a candidate start station
    start_index = 0 # Candidate starting station index

    # Loop through each station
    for i in range(len(gas)):
        # Calculate net gas after leaving station i
        surplus = gas[i] - cost[i]

        total_surplus += surplus
        current_surplus += surplus

        # If current surplus becomes negative, station i is not a valid start.
        if current_surplus < 0:
            # Reset starting station to the next station
            start_index = i + 1
            current_surplus = 0 # Reset current surplus
    # Check if overall surplus is non-negative
    return start_index if total_surplus >= 0 else -1

# Example usage:
gas = [1,2,3,4,5]
cost = [3,4,5,1,2]
print(canCompleteCircuit(gas, cost)) # Output: 3
```

• LC.ca

```
class Solution:
    def canCompleteCircuit(self, gas: List[int], cost: List[int]) -> int:
        n = len(gas)
        i = j = n - 1
        cnt = s = 0
        while cnt < n:
            s += gas[j] - cost[j]
            cnt += 1
            j = (j + 1) % n
            while s < 0 and cnt < n:
```

Round 5 · Mid · Medium

p.2274

```

i -= 1
s += gas[i] - cost[i]
cnt += 1
return -1 if s < 0 else i

```

◆ Quick Review

Key Insights

- If the total amount of gas available is less than the total travel cost, then completing the circuit is impossible.
- A greedy approach can be used by keeping track of the current surplus of gas. If the surplus becomes negative at any station, the journey cannot start from any station before that point.
- Restart the journey from the next station whenever the surplus goes negative.
- The problem guarantees that if a solution exists, it is unique.

Space & Time

Time Complexity: $O(n)$, where n is the number of gas stations, as we traverse the list only once.

Space Complexity: $O(1)$, as no extra space is used other than a few variables.

Round 5 · Mid · Medium

p.2275

Solution

The solution involves iterating over the list of stations while maintaining two variables: one for the current gas surplus and one for the total gas surplus. If at any station the current surplus drops below zero, the current starting station is not viable and we set the starting station to the next index and reset the current surplus. After one pass, if the total gas surplus is non-negative, then the identified station is the valid starting point.

Round 5 · Mid · Medium

p.2276

Greedy

#406 Queue Reconstruction by Height

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Indexed Tree · Segment Tree ·
Sorting

Lists: AlgoMaster 600

Problem

You are given an array of people, `people`, which are the attributes of some people in a queue (not necessarily in order). Each `people[i] = [hi, ki]` represents the *i*th person of height *hi* with exactly *ki* other people in front who have a height greater than or equal to *hi*.

Reconstruct and return the queue that is represented by the input array `people`. The returned queue should be formatted as an array `queue`, where `queue[j] = [hj, kj]` is the attributes of the *j*th person in the queue (`queue[0]` is the person at the front of the queue).

Example 1:

Round 5 · Mid · Medium

p.2277

Input: `people = [[7,0],[4,4],[7,1],[5,0],[6,1],[5,2]]`

Output: `[[5,0],[7,0],[5,2],[6,1],[4,4],[7,1]]`

Explanation:

Person 0 has height 5 with no other people taller or the same height in front.
Person 1 has height 7 with no other people taller or the same height in front.
Person 2 has height 5 with two persons taller or the same height in front, which is person 0 and 1.
Person 3 has height 6 with one person taller or the same height in front, which is person 1.
Person 4 has height 4 with four people taller or the same height in front, which are people 0, 1, 2, and 3.

Example 2:

Input: `people = [[6,0],[5,0],[4,0],[3,2],[2,2],[1,4]]`

Output: `[[4,0],[5,0],[2,2],[3,2],[1,4],[6,0]]`

Constraints:

- $1 \leq \text{people.length} \leq 2000$
- $0 \leq \text{hi} \leq 106$
- $0 \leq \text{ki} < \text{people.length}$
- It is guaranteed that the queue can be reconstructed.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def reconstructQueue(self, people: list[list[int]]) -> list[list[int]]:
        ans = []

        people.sort(key=lambda x: (-x[0], x[1]))

        for person in people:
            ans.insert(person[1], person)

        return ans
```

Round 5 · Mid · Medium

p.2278

• LeetCode

```
class Solution:
    def reconstructQueue(self, people: List[List[int]]) -> List[List[int]]:
        people.sort(key=lambda x: (-x[0], x[1]))
        ans = []
        for p in people:
            ans.insert(p[1], p)
        return ans
```

• SimplyLeet

```
# Python solution with line-by-line comments

class Solution:
    def reconstructQueue(self, people):
        # Sort people: first by descending height; if heights are equal, sort
        # by ascending k value.
        people.sort(key=lambda person: (-person[0], person[1]))

        # Initialize result list
        queue = []

        # Insert each person in the list at the index equal to their k-value.
        for person in people:
            queue.insert(person[1], person) # person[1] is the k value
            indicating the index
        return queue

# Example usage:
# solution = Solution()
# print(solution.reconstructQueue([[7,0],[4,4],[7,1],[5,0],[6,1],[5,2]]))
```

Greedy

#621 Task Scheduler

MED

LeetCode · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Greedy · Heap · Counting
Lists: Grind 169

Problem

You are given an array of CPU tasks, each labeled with a letter from A to Z, and a number n. Each CPU interval can be idle or allow the completion of one task. Tasks can be completed in any order, but there's a constraint: there has to be a gap of at least n intervals between two tasks with the same label.

Return the minimum number of CPU intervals required to complete all tasks.

Example 1:

Input: tasks = ["A","A","A","B","B","B"], n = 2

Output: 8

Explanation: A possible sequence is: A -> B -> idle -> A -> B -> idle -> A -> B.

After completing task A, you must wait two intervals before doing A again. The same applies to task B. In the 3rd interval, neither A nor B can be done, so you idle. By the 4th interval, you can do A again as 2 intervals have passed.

Example 2:

Input: tasks = ["A","C","A","B","D","B"], n = 1

Output: 6

Explanation: A possible sequence is: A -> B -> C -> D -> A -> B.

With a cooling interval of 1, you can repeat a task after just one other task.

Example 3:

Input: tasks = ["A","A","A", "B","B","B"], n = 3

Output: 10

Explanation: A possible sequence is: A -> B -> idle -> idle -> A -> B -> idle -> idle -> A -> B.

There are only two types of tasks, A and B, which need to be separated by 3 intervals. This leads to idling twice between repetitions of these tasks.

Constraints:

- $1 \leq \text{tasks.length} \leq 104$
- tasks[i] is an uppercase English letter.

- $0 \leq n \leq 100$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def leastInterval(self, tasks: list[str], n: int) -> int:
        count = collections.Counter(tasks)
        maxFreq = max(count.values())
        # Put the most frequent task in the slot first.
        maxFreqTaskOccupy = (maxFreq - 1) * (n + 1)
        # Get the number of tasks with same frequency as maxFreq, we'll append them
        # after the
        # `maxFreqTaskOccupy`.
        nMaxFreq = sum(value == maxFreq for value in count.values())
        # max(
        # the most frequent task is frequent enough to force some idle slots,
        # the most frequent task is not frequent enough to force idle slots
        # )
        return max(maxFreqTaskOccupy + nMaxFreq, len(tasks))
```

• LeetDoocs

```
class Solution:
    def leastInterval(self, tasks: List[str], n: int) -> int:
        cnt = Counter(tasks)
        x = max(cnt.values())
        s = sum(v == x for v in cnt.values())
        return max(len(tasks), (x - 1) * (n + 1) + s)
```

• SimplyLeet

```
# Python solution for Task Scheduler
from collections import Counter

def leastInterval(tasks, n):
    # Count the frequency of each task
    task_counts = Counter(tasks)
    # Find the maximum frequency among tasks
    max_freq = max(task_counts.values())
    # Count how many tasks have the maximum frequency
    max_count = sum(1 for count in task_counts.values() if count == max_freq)

    # Calculate the minimum intervals required based on the most frequent tasks
    part_count = max_freq - 1          # Number of parts between the
    most frequent task
    part_length = n - (max_count - 1)    # Effective empty slots per
    part after placing max frequency tasks
    empty_slots = part_count * part_length # Total empty slots to fill
    with idle or other tasks
    available_tasks = len(tasks) - max_freq * max_count # Remaining tasks to
    fill the empty slots
    idles = max(0, empty_slots - available_tasks) # Remaining idle slots if
    available tasks are not enough

    # The total intervals is the sum of all tasks and idle slots inserted
    return len(tasks) + idles

# Example usage:
if __name__ == "__main__":
    tasks1 = ["A","A","A","B","B","B"]
    n1 = 2
    print(leastInterval(tasks1, n1)) # Output: 8
```

• LC.ca

```
class Solution:
    def leastInterval(self, tasks: List[str], n: int) -> int:
        cnt = Counter(tasks)
        x = max(cnt.values())
        s = sum(v == x for v in cnt.values())
        return max(len(tasks), (x - 1) * (n + 1) + s)
```

Round 5 · Mid · Medium

p.2283

◆ Quick Review

Key Insights

- Count the frequency of each task.
- The task with the highest frequency determines the base structure.
- Identify how many tasks share the maximum frequency.
- Calculate the total time based on the formula: $\max(\text{total tasks}, (\text{max frequency} - 1) * (n + 1) + \text{count of tasks with maximum frequency})$.
- This approach avoids simulating the entire scheduling by using a greedy strategy.

Space & Time

Time Complexity: $O(N)$ where N is the number of tasks (counting frequency, then iterating over constant 26 letters).

Space Complexity: $O(1)$ since the frequency count uses an array or hash map with fixed size (26 letters).

Solution

We first count the frequency of each task. The task(s) with the maximum frequency establish

Round 5 · Mid · Medium

p.2284

the minimum structure required. Compute the ideal idle slots based on the count of these maximum tasks and the cooling period n . Finally, compare this computed length with the total number of tasks to get the final answer. This greedy strategy avoids explicit simulation and leverages mathematical formulation.

Greedy

□ #670 Maximum Swap

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · Greedy
Lists: AlgoMaster 600

Problem

You are given an integer num . You can swap two digits at most once to get the maximum valued number.

Return the maximum valued number you can get.

Example 1:

```
Input: num = 2736
Output: 7236
Explanation: Swap the number 2 and the number 7.
```

Example 2:

```
Input: num = 9973
Output: 9973
Explanation: No swap.
```

Constraints:

- $0 \leq num \leq 10^8$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maximumSwap(self, num: int) -> int:
        s = list(str(num))
        dict = {c: i for i, c in enumerate(s)}

        for i, c in enumerate(s):
            for digit in reversed(string.digits):
                if digit <= c:
                    break
                if digit in dict and dict[digit] > i:

                    s[i], s[dict[digit]] = digit, s[i]
                    return int(''.join(s))

        return num
```

• LeetDoocs

```
class Solution:
    def maximumSwap(self, num: int) -> int:
        s = list(str(num))
        n = len(s)
        d = list(range(n))
        for i in range(n - 2, -1, -1):
            if s[i] <= s[d[i + 1]]:
                d[i] = d[i + 1]
        for i, j in enumerate(d):
            if s[i] < s[j]:

                s[i], s[j] = s[j], s[i]
                break
        return int(''.join(s))
```

• SimplyLeet

Round 5 · Mid · Medium

p.2287

```
def maximumSwap(num):
    # Convert number to a list of characters (digits)
    digits = list(str(num))
    # Create a mapping of the last occurrence of each digit
    last = {int(digit): index for index, digit in enumerate(digits)}

    # Iterate over the digits from left to right
    for i, digit in enumerate(digits):
        # Check for a digit larger than the current one from 9 down to
        current+1
        for d in range(9, int(digit), -1):

            # If a larger digit appears later in the number, swap and return
            result
            if last.get(d, -1) > i:
                digits[i], digits[last[d]] = digits[last[d]], digits[i]
                return int(''.join(digits))

    return num

# Test example
print(maximumSwap(2736)) # Expected output: 7236
```

Round 5 · Mid · Medium

p.2288

Greedy

#826 Most Profit Assigning Work

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Binary Search · Greedy ·
Sorting

Lists: AlgoMaster 600

Problem

You have n jobs and m workers. You are given three arrays: `difficulty`, `profit`, and `worker` where:

- `difficulty[i]` and `profit[i]` are the difficulty and the profit of the i th job, and
- `worker[j]` is the ability of j th worker (i.e., the j th worker can only complete a job with difficulty at most `worker[j]`).

Every worker can be assigned at most one job, but one job can be completed multiple times.

- For example, if three workers attempt the same job that pays \$1, then the total profit will be \$3. If a worker cannot complete any job, their profit is \$0.

Round 5 · Mid · Medium

p.2289

Return the maximum profit we can achieve after assigning the workers to the jobs.

Example 1:

Input: `difficulty = [2,4,6,8,10]`, `profit = [10,20,30,40,50]`, `worker = [4,5,6,7]`
Output: 100
Explanation: Workers are assigned jobs of difficulty `[4,4,6,6]` and they get a profit of `[20,20,30,30]` separately.

Example 2:

Input: `difficulty = [85,47,57]`, `profit = [24,66,99]`, `worker = [40,25,25]`
Output: 0

Constraints:

- $n == \text{difficulty.length}$
- $n == \text{profit.length}$
- $m == \text{worker.length}$
- $1 \leq n, m \leq 104$
- $1 \leq \text{difficulty}[i], \text{profit}[i], \text{worker}[i] \leq 105$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxProfitAssignment(
        self,
        difficulty: list[int],
        profit: list[int],
        worker: list[int],
    ) -> int:
        ans = 0
        jobs = sorted(zip(difficulty, profit))
        worker.sort(reverse=1)
```

Round 5 · Mid · Medium

p.2290

```

i = 0
maxProfit = 0

for w in sorted(worker):
    while i < len(jobs) and w >= jobs[i][0]:
        maxProfit = max(maxProfit, jobs[i][1])
        i += 1
    ans += maxProfit

return ans

```

• LeetDoocs

```

class Solution:
    def maxProfitAssignment(
        self, difficulty: List[int], profit: List[int], worker: List[int]
    ) -> int:
        worker.sort()
        jobs = sorted(zip(difficulty, profit))
        ans = mx = i = 0
        for w in worker:
            while i < len(jobs) and jobs[i][0] <= w:
                mx = max(mx, jobs[i][1])
                i += 1
            ans += mx
        return ans

```

• SimplyLeet

```

# Python solution with detailed comments

import bisect

def maxProfitAssignment(difficulty, profit, worker):
    # Pair up difficulty and profit for jobs and sort by difficulty
    jobs = sorted(zip(difficulty, profit))

    max_profit_up_to = []
    current_max = 0

```

```

# Create a list of difficulties and update max profit seen so far.
difficulties = []
for d, p in jobs:
    current_max = max(current_max, p)
    difficulties.append(d)
    max_profit_up_to.append(current_max)

total_profit = 0
# For each worker, binary search for the most difficult job they can work
on
for ability in worker:

    # bisect_right returns insertion point, subtract 1 to get valid index
    idx = bisect.bisect_right(difficulties, ability) - 1
    if idx >= 0:
        total_profit += max_profit_up_to[idx]
return total_profit

# Example usage:
# print(maxProfitAssignment([2,4,6,8,10], [10,20,30,40,50], [4,5,6,7]))

```

Greedy

☐ #921 Minimum Add to Make Parentheses Valid MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Stack · Greedy
Lists: AlgoMaster 600

Problem

A parentheses string is valid if and only if:

- It is the empty string,
- It can be written as AB (A concatenated with B), where A and B are valid strings, or
- It can be written as (A), where A is a valid string.

You are given a parentheses string *s*. In one move, you can insert a parenthesis at any position of the string.

- For example, if *s* = "())", you can insert an opening parenthesis to be "(())" or a closing parenthesis to be "()())".

Return the minimum number of moves required to make *s* valid.

Example 1:

Round 5 · Mid · Medium

p.2293

Input: *s* = "())"
Output: 1

Example 2:

Input: *s* = "((("
Output: 3

Constraints:

- $1 \leq s.length \leq 1000$
- *s*[*i*] is either '(' or ')'.

Community Solutions (Python)

☐ • WalkCC

```
class Solution:
    def minAddToMakeValid(self, s: str) -> int:
        l = 0
        r = 0

        for c in s:
            if c == '(':
                l += 1
            else:
                if l == 0:
                    r += 1
                else:
                    l -= 1

        return l + r
```

☐ • LeetDoocs

Round 5 · Mid · Medium

p.2294

```
class Solution:
    def minAddToMakeValid(self, s: str) -> int:
        stk = []
        for c in s:
            if c == ')' and stk and stk[-1] == '(':
                stk.pop()
            else:
                stk.append(c)
        return len(stk)
```

```
class Solution:
    def minAddToMakeValid(self, s: str) -> int:
        ans = cnt = 0
        for c in s:
            if c == '(':
                cnt += 1
            elif cnt:
                cnt -= 1
            else:
                ans += 1
```

```
ans += cnt
return ans
```

• SimplyLeet

```
# Function to determine the minimum moves required
def minAddToMakeValid(s):
    moves = 0 # Counts the required insertions for unmatched ')'
    count = 0 # Counts unmatched '(' characters
    for char in s:
        if char == '(':
            count += 1 # Increase count for an open parenthesis
        elif char == ')':
            # If no matching open, it's an error, need one move to balance
            if count == 0:
```

```
        moves += 1
    else:
        count -= 1 # Found a matching pair
    # Add moves needed for any leftover unmatched '('
    return moves + count

# Example usage:
print(minAddToMakeValid("()")) # Expected output: 1
print(minAddToMakeValid("(((")) # Expected output: 3
```


Greedy

#1488 Avoid Flood in The City

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table · Binary Search · Greedy ·
 Heap (Priority Queue)

Lists: AlgoMaster 600

Problem

Your country has 109 lakes. Initially, all the lakes are empty, but when it rains over the n th lake, the n th lake becomes full of water. If it rains over a lake that is full of water, there will be a flood. Your goal is to avoid floods in any lake.

Given an integer array rains where:

- $\text{rains}[i] > 0$ means there will be rains over the $\text{rains}[i]$ lake.
- $\text{rains}[i] == 0$ means there are no rains this day and you must choose one lake this day and dry it.

Return an array ans where:

- $\text{ans.length} == \text{rains.length}$

Round 5 · Mid · Medium

p.2297

- $\text{ans}[i] == -1$ if $\text{rains}[i] > 0$.
- $\text{ans}[i]$ is the lake you choose to dry in the i th day if $\text{rains}[i] == 0$.

If there are multiple valid answers return any of them. If it is impossible to avoid flood return an empty array.

Notice that if you chose to dry a full lake, it becomes empty, but if you chose to dry an empty lake, nothing changes.

Example 1:

```
Input: rains = [1,2,3,4]
Output: [-1,-1,-1,-1]
Explanation: After the first day full lakes are [1]
After the second day full lakes are [1,2]
After the third day full lakes are [1,2,3]
After the fourth day full lakes are [1,2,3,4]
There's no day to dry any lake and there is no flood in any lake.
```

Example 2:

```
Input: rains = [1,2,0,0,2,1]
Output: [-1,-1,2,1,-1,-1]
Explanation: After the first day full lakes are [1]
After the second day full lakes are [1,2]
After the third day, we dry lake 2. Full lakes are [1]
After the fourth day, we dry lake 1. There is no full lakes.
After the fifth day, full lakes are [2].
After the sixth day, full lakes are [1,2].
```

Example 3:

```
Input: rains = [1,2,0,1,2]
Output: []
Explanation: After the second day, full lakes are [1,2]. We have to dry one lake in the third day.
After that, it will rain over lakes [1,2]. It's easy to prove that no matter which lake you choose to dry in the 3rd day, the other one will flood.
```

Round 5 · Mid · Medium

p.2298

Constraints:

- $1 \leq \text{rains.length} \leq 105$
- $0 \leq \text{rains}[i] \leq 109$

Community Solutions (Python)

• WalkCC

```
from sortedcontainers import SortedSet

class Solution:
    def avoidFlood(self, rains: list[int]) -> list[int]:
        ans = [-1] * len(rains)
        lakeIdToFullDay = {}
        emptyDays = SortedSet() # indices of rains[i] == 0

        for i, lakeId in enumerate(rains):

            if lakeId == 0:
                emptyDays.add(i)
                continue
            # The lake was full in a previous day. Greedily find the closest day
            # to make the lake empty.
            if lakeId in lakeIdToFullDay:
                fullDay = lakeIdToFullDay[lakeId]
                emptyDayIndex = emptyDays.bisect_right(fullDay)
                if emptyDayIndex == len(emptyDays): # Not found.
                    return []

            # Empty the lake at this day.
            emptyDay = emptyDays[emptyDayIndex]
            ans[emptyDay] = lakeId
            emptyDays.discard(emptyDay)
            # The lake with `lakeId` becomes full at the day `i`.
            lakeIdToFullDay[lakeId] = i

        # Empty an arbitrary lake if there are remaining empty days.
        for emptyDay in emptyDays:
            ans[emptyDay] = 1
```

Round 5 · Mid · Medium

p.2299

```
return ans
```

• LeetCode

```
class Solution:
    def avoidFlood(self, rains: List[int]) -> List[int]:
        n = len(rains)
        ans = [-1] * n
        sunny = SortedList()
        rainy = {}
        for i, v in enumerate(rains):
            if v:
                if v in rainy:
                    idx = sunny.bisect_right(rainy[v])

                    if idx == len(sunny):
                        return []
                    ans[sunny[idx]] = v
                    sunny.discard(sunny[idx])
                rainy[v] = i
            else:
                sunny.add(i)
                ans[i] = 1
        return ans
```

• SimplyLeet

```
# Import bisect for binary search operations on the sorted list of dry day
indices
import bisect

def avoidFlood(rains):
    n = len(rains)
    result = [-1] * n # By default, set rainy day result to -1; dry day will
    get updated
    lake_last_rain = {} # Dictionary to map lake to the last day it rained
    dry_days = [] # Sorted list to keep track of indices of available dry days

    for i, lake in enumerate(rains):
```

Round 5 · Mid · Medium

p.2300

```

if lake > 0: # It is raining over the lake
    if lake in lake_last_rain:
        # Find a dry day index that is greater than the last rain day
        prev_rain_day = lake_last_rain[lake]
        idx = bisect.bisect_right(dry_days, prev_rain_day)
        if idx == len(dry_days):
            # No available dry day to dry the lake before it rains
            again -> flood
            return []
        # Use the found dry day to dry this lake
        dry_day_index = dry_days[idx]

        result[dry_day_index] = lake # Dry the lake on that dry day
        dry_days.pop(idx) # Remove the used dry day from the available
list
    # Update the last rain day for this lake
    lake_last_rain[lake] = i
    result[i] = -1 # On rainy days, the result is always -1
else:
    # It's a dry day; add index to the list of dry days (initially set
to arbitrary 1)
    bisect.insort(dry_days, i)
    result[i] = 1 # Default value, may be overridden later if the day
is used to dry a lake

return result

# Example usage:
#print(avoidFlood([1,2,0,0,2,1]))

```

Greedy

#1717 Maximum Score From Removing Substrings

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Stack · Greedy
Lists: AlgoMaster 600

Problem

You are given a string s and two integers x and y . You can perform two types of operations any number of times.

- Remove substring "ab" and gain x points.
For example, when removing "ab" from "cabxbae" it becomes "cxbae".
- For example, when removing "ba" from "cabxbae" it becomes "cabxe".

Return the maximum points you can gain after applying the above operations on s .

Example 1:

Input: s = "cdbcbbaaabab", x = 4, y = 5

Output: 19

Explanation:

- Remove the "ba" underlined in "cdbcbbaaabab". Now, s = "cdbcbbaaab" and 5 points are added to the score.
 - Remove the "ab" underlined in "cdbcbbaaab". Now, s = "cdbcbbaa" and 4 points are added to the score.
 - Remove the "ba" underlined in "cdbcbbaa". Now, s = "cdbcbba" and 5 points are added to the score.
 - Remove the "ba" underlined in "cdbcbba". Now, s = "cdbcb" and 5 points are added to the score.
- Total score = 5 + 4 + 5 + 5 = 19.

Example 2:

Input: s = "aabbaaxybbaabb", x = 5, y = 4

Output: 20

Constraints:

- $1 \leq s.length \leq 105$
- $1 \leq x, y \leq 104$
- s consists of lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maximumGain(self, s: str, x: int, y: int) -> int:
        # The assumption that gain('ab') > gain('ba') while removing 'ba' first is
        # optimal is contradicted. Only 'b(ab)a' satisfies the condition of
        # preventing two 'ba' removals, but after removing 'ab', we can still
        # remove one 'ba', resulting in a higher gain. Thus, removing 'ba' first is
        # not optimal.
        return (self._gain(s, 'ab', x, 'ba', y) if x > y else
                self._gain(s, 'ba', y, 'ab', x))
```

```
# Returns the points gained by first removing sub1 ('ab' | 'ba') from s with
# point1, then removing sub2 ('ab' | 'ba') from s with point2.
def _gain(self, s: str, sub1: str, point1: int, sub2: str, point2: int) -> int:
    points = 0
    stack1 = []
    stack2 = []

    # Remove 'sub1' from s with point1 gain.
    for c in s:
        if stack1 and stack1[-1] == sub1[0] and c == sub1[1]:
            stack1.pop()
            points += point1
        else:
            stack1.append(c)

    # Remove 'sub2' from s with point2 gain.
    for c in stack1:
        if stack2 and stack2[-1] == sub2[0] and c == sub2[1]:
            stack2.pop()
            points += point2
        else:
            stack2.append(c)

    return points
```

• LeetDoocs

```
class Solution:
    def maximumGain(self, s: str, x: int, y: int) -> int:
        a, b = "a", "b"
        if x < y:
            x, y = y, x
            a, b = b, a
        ans = cnt1 = cnt2 = 0
        for c in s:
            if c == a:
                cnt1 += 1
```

```

        elif c == b:
            if cnt1:
                ans += x
                cnt1 -= 1
            else:
                cnt2 += 1
        else:
            ans += min(cnt1, cnt2) * y
            cnt1 = cnt2 = 0
    ans += min(cnt1, cnt2) * y

    return ans

```

• SimplyLeet

```

# Python solution with detailed comments
def maximumGain(s: str, x: int, y: int) -> int:
    total_points = 0

    # Decide which pair to remove first based on their points
    if x >= y:
        first_pair, first_score = "ab", x
        second_pair, second_score = "ba", y
    else:
        first_pair, first_score = "ba", y
        second_pair, second_score = "ab", x

    # First pass: remove first_pair using a greedy stack approach
    stack = []
    remaining_chars = []
    for char in s:
        if stack and (stack[-1] + char == first_pair):
            stack.pop() # Remove the matching pair from the stack
            total_points += first_score # Add score for this removal
        else:
            stack.append(char)

    # The characters remaining after first pass
    remaining_chars = stack

    # Second pass: remove second_pair from the remaining characters
    stack = []
    for char in remaining_chars:
        if stack and (stack[-1] + char == second_pair):
            stack.pop() # removal of second pair
            total_points += second_score
        else:
            stack.append(char)

    return total_points

```

```

        stack.append(char)
    # The characters remaining after first pass
    remaining_chars = stack

    # Second pass: remove second_pair from the remaining characters
    stack = []
    for char in remaining_chars:
        if stack and (stack[-1] + char == second_pair):
            stack.pop() # removal of second pair
            total_points += second_score
        else:
            stack.append(char)

    return total_points

```

```

# Example usage:
s = "cdbcbbaaabab"
x = 4
y = 5
print(maximumGain(s, x, y)) # Expected output: 19

```

Greedy

#1871 Jump Game VII

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming · Sliding
Window · Prefix Sum
Lists: AlgoMaster 600

Problem

You are given a 0-indexed binary string s and two integers minJump and maxJump . In the beginning, you are standing at index 0, which is equal to '0'. You can move from index i to index j if the following conditions are fulfilled:

- $i + \text{minJump} \leq j \leq \min(i + \text{maxJump}, s.length - 1)$, and
- $s[j] == '0'$.

Return true if you can reach index $s.length - 1$ in s , or false otherwise.

Example 1:

Input: $s = "011010"$, $\text{minJump} = 2$, $\text{maxJump} = 3$
Output: true
Explanation:
In the first step, move from index 0 to index 3.
In the second step, move from index 3 to index 5.

Round 5 · Mid · Medium

p.2307

Example 2:

Input: $s = "01101110"$, $\text{minJump} = 2$, $\text{maxJump} = 3$
Output: false

Constraints:

- $2 \leq s.length \leq 105$
- $s[i]$ is either '0' or '1'.
- $s[0] == '0'$
- $1 \leq \text{minJump} \leq \text{maxJump} < s.length$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def canReach(self, s: str, minJump: int, maxJump: int) -> bool:
        count = 0
        dp = [True] + [False] * (len(s) - 1)

        for i in range(minJump, len(s)):
            count += dp[i - minJump]
            if i - maxJump > 0:
                count -= dp[i - maxJump - 1]
            dp[i] = count and s[i] == '0'

        return dp[-1]
```

• LeetDoocs

Round 5 · Mid · Medium

p.2308

```
class Solution:
    def canReach(self, s: str, minJump: int, maxJump: int) -> bool:
        n = len(s)
        pre = [0] * (n + 1)
        pre[1] = 1
        f = [True] + [False] * (n - 1)
        for i in range(1, n):
            if s[i] == "0":
                l, r = max(0, i - maxJump), i - minJump
                f[i] = l <= r and pre[r + 1] - pre[l] > 0

            pre[i + 1] = pre[i] + f[i]
        return f[-1]
```

• SimplyLeet

```
def canReach(s: str, minJump: int, maxJump: int) -> bool:
    n = len(s)
    # dp[i] is True if index i is reachable, False otherwise
    dp = [False] * n
    dp[0] = True

    # variable to keep track of the cumulative count of reachable indices in
    # the window
    reachable = 0
    for i in range(1, n):
        # If i is at least minJump steps ahead, add the dp value that enters
        # the window.
        if i >= minJump:
            reachable += dp[i - minJump]
        # If i is beyond maxJump, subtract the dp value that leaves the window.
        if i > maxJump:
            reachable -= dp[i - maxJump - 1]
        # Mark index i as reachable if s[i] is '0' and there is at least one
        # reachable index in the window.
        if s[i] == '0' and reachable > 0:
            dp[i] = True
    return dp[-1]

# Example usage:
print(canReach("011010", 2, 3)) # Expected output: True
```

Round 5 · Mid · Medium

p.2309

Greedy

#1975 Maximum Matrix Sum

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Greedy · Matrix
 Lists: AlgoMaster 600

Problem

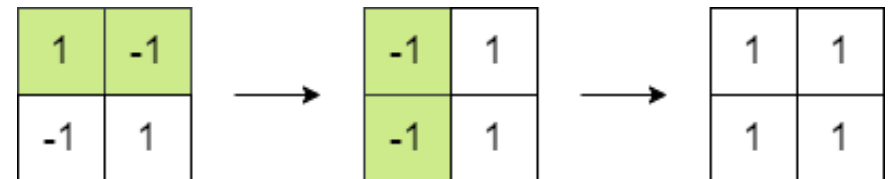
You are given an $n \times n$ integer matrix. You can do the following operation any number of times:

- Choose any two adjacent elements of matrix and multiply each of them by -1 .

Two elements are considered adjacent if and only if they share a border.

Your goal is to maximize the summation of the matrix's elements. Return the maximum sum of the matrix's elements using the operation mentioned above.

Example 1:



Round 5 · Mid · Medium

p.2310

Input: matrix = [[1,-1],[-1,1]]

Output: 4

Explanation: We can follow the following steps to reach sum equals 4:

- Multiply the 2 elements in the first row by -1.
- Multiply the 2 elements in the first column by -1.

Example 2:

1	2	3
-1	-2	-3
1	2	3

→

1	2	3
-1	2	3
1	2	3

Input: matrix = [[1,2,3],[-1,-2,-3],[1,2,3]]

Output: 16

Explanation: We can follow the following step to reach sum equals 16:

- Multiply the 2 last elements in the second row by -1.

Constraints:

- $n == \text{matrix.length} == \text{matrix}[i].\text{length}$
- $2 \leq n \leq 250$
- $-105 \leq \text{matrix}[i][j] \leq 105$

Community Solutions (Python)

- WalkCC

Round 5 · Mid · Medium

p.2311

```
class Solution:
    def maxMatrixSum(self, matrix: list[list[int]]) -> int:
        absSum = 0
        minAbs = math.inf
        # 0 := even number of negatives
        # 1 := odd number of negatives
        oddNeg = 0

        for row in matrix:
            for num in row:
                absSum += abs(num)
                minAbs = min(minAbs, abs(num))
                if num < 0:
                    oddNeg ^= 1

        return absSum - oddNeg * minAbs * 2
```

• LeetCode

```
class Solution:
    def maxMatrixSum(self, matrix: List[List[int]]) -> int:
        mi = inf
        s = cnt = 0
        for row in matrix:
            for x in row:
                cnt += x < 0
                y = abs(x)
                mi = min(mi, y)
                s += y

        return s if cnt % 2 == 0 else s - mi * 2
```

• SimplyLeet

Round 5 · Mid · Medium

p.2312


```
# Python solution for Maximum Matrix Sum

class Solution:
    def maxMatrixSum(self, matrix: List[List[int]]) -> int:
        total_sum = 0          # Sum of absolute values of all elements
        min_abs_value = float('inf') # Minimum absolute value in the matrix
        negative_count = 0     # Counter for negative numbers

        # Traverse through the matrix
        for row in matrix:

            for value in row:
                abs_value = abs(value)
                total_sum += abs_value # Add the absolute value to total_sum
                min_abs_value = min(min_abs_value, abs_value) # Update
minimum absolute value
            if value < 0:
                negative_count += 1 # Count negatives

        # If the number of negatives is odd, subtract twice the smallest
absolute value
        if negative_count % 2 != 0:
            total_sum -= 2 * min_abs_value

        return total_sum
```

Round 5 · Mid · Medium

p.2313

Topological Sort

Round 5 · Mid · Medium · 6 questions

Round 5 · Mid · Medium

p.2314

Topological Sort

#210 Course Schedule II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode

DFS · BFS · Graph · Topological Sort

Lists: Grind 169

Problem

There are a total of `numCourses` courses you have to take, labeled from 0 to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [ai, bi]` indicates that you must take course `bi` first if you want to take course `ai`.

- For example, the pair `[0, 1]`, indicates that to take course 0 you have to first take course 1.

Return the ordering of courses you should take to finish all courses. If there are many valid answers, return any of them. If it is impossible to finish all courses, return an empty array.

Example 1:

Input: `numCourses = 2, prerequisites = [[1,0]]`
 Output: `[0,1]`
 Explanation: There are a total of 2 courses to take. To take course 1 you should have finished course 0. So the correct course order is `[0,1]`.

Example 2:

Round 5 · Mid · Medium

p.2315

Input: `numCourses = 4, prerequisites = [[1,0],[2,0],[3,1],[3,2]]`
 Output: `[0,2,1,3]`
 Explanation: There are a total of 4 courses to take. To take course 3 you should have finished both courses 1 and 2. Both courses 1 and 2 should be taken after you finished course 0.
 So one correct course order is `[0,1,2,3]`. Another correct ordering is `[0,2,1,3]`.

Example 3:

Input: `numCourses = 1, prerequisites = []`
 Output: `[0]`

Constraints:

- `1 <= numCourses <= 2000`
- `0 <= prerequisites.length <= numCourses * (numCourses - 1)`
- `prerequisites[i].length == 2`
- `0 <= ai, bi < numCourses`
- `ai != bi`
- All the pairs `[ai, bi]` are distinct.

Community Solutions (Python)

• WalkCC

```
from enum import Enum

class State(Enum):
    INIT = 0
    VISITING = 1
    VISITED = 2

class Solution:
```

Round 5 · Mid · Medium

p.2316

```
def findOrder(
    self,
    numCourses: int,
    prerequisites: List[List[int]],
) -> List[int]:
    ans = []
    graph = [[] for _ in range(numCourses)]
    states = [State.INIT] * numCourses

    for v, u in prerequisites:
        graph[u].append(v)

    def hasCycle(u: int) -> bool:
        if states[u] == State.VISITING:
            return True
        if states[u] == State.VISITED:
            return False
        states[u] = State.VISITING
        if any(hasCycle(v) for v in graph[u]):
            return True
        states[u] = State.VISITED
        ans.append(u)
        return False

    if any(hasCycle(i) for i in range(numCourses)):
        return []

    return ans[::-1]
```

```
class Solution:
    def findOrder(
        self,
        numCourses: int,
        prerequisites: List[List[int]],
    ) -> List[int]:
        ans = []
        graph = [[] for _ in range(numCourses)]
        inDegrees = [0] * numCourses
        q = collections.deque()
```

```
# Build the graph.
for v, u in prerequisites:
    graph[u].append(v)
    inDegrees[v] += 1

# Perform topological sorting.
q = collections.deque([i for i, d in enumerate(inDegrees) if d == 0])

while q:
    u = q.popleft()
    ans.append(u)
    for v in graph[u]:
        inDegrees[v] -= 1
        if inDegrees[v] == 0:
            q.append(v)

return ans if len(ans) == numCourses else []
```

• LeetCode

```
class Solution:
    def findOrder(self, numCourses: int, prerequisites: List[List[int]]) -> List[int]:
        g = defaultdict(list)
        indeg = [0] * numCourses
        for a, b in prerequisites:
            g[b].append(a)
            indeg[a] += 1
        ans = []
        q = deque(i for i, x in enumerate(indeg) if x == 0)
        while q:
            i = q.popleft()
            ans.append(i)
            for j in g[i]:
                indeg[j] -= 1
                if indeg[j] == 0:
                    q.append(j)
            return ans if len(ans) == numCourses else []
```

• SimplyLeet

```
# Python solution using BFS (Kahn's Algorithm)
from collections import deque

def findOrder(numCourses, prerequisites):
    # Build the graph and the in-degree array
    in_degree = [0] * numCourses
    graph = {i: [] for i in range(numCourses)}
    for course, pre in prerequisites:
        graph[pre].append(course)
        in_degree[course] += 1

    # Initialize the queue with all courses having zero in-degree
    queue = deque([i for i in range(numCourses) if in_degree[i] == 0])
    order = []

    # Process courses
    while queue:
        course = queue.popleft()
        order.append(course)
        for neighbor in graph[course]:
            in_degree[neighbor] -= 1
            if in_degree[neighbor] == 0:
                queue.append(neighbor)

    # If the ordering includes all courses, return it; otherwise, return an
    # empty list
    return order if len(order) == numCourses else []

# Example usage:
# print(findOrder(4, [[1,0],[2,0],[3,1],[3,2]]))
```

• LC.ca

Round 5 · Mid · Medium

p.2319

```
class Solution:
    def findOrder(self, numCourses: int, prerequisites: List[List[int]]) -> List[int]:
        g = defaultdict(list)
        indeg = [0] * numCourses
        for a, b in prerequisites:
            g[b].append(a)
            indeg[a] += 1
        q = deque([i for i, v in enumerate(indeg) if v == 0])
        ans = [] # added from previous question
        while q:
            i = q.popleft()
            ans.append(i)
            # assumption is only one path
            # as in question 'You may assume that there are no duplicate edges
            # in the input prerequisites.'
            for j in g[i]:
                indeg[j] -= 1
                if indeg[j] == 0:
                    q.append(j)
        return ans if len(ans) == numCourses else []
```

◆ Quick Review

Key Insights

- This is a topological sort problem on a directed graph.
- The courses are nodes, and prerequisites form directed edges.
- We can solve the problem by either:
- Using DFS with cycle detection and a stack for topological ordering.

Round 5 · Mid · Medium

p.2320

- Using BFS (Kahn's Algorithm) to process nodes with zero in-degrees.
- If at any point no node with a zero in-degree is available, a cycle exists and no valid order can be produced.

Space & Time

Time Complexity: $O(V + E)$ where V is the number of courses and E is the number of prerequisite pairs.

Space Complexity: $O(V + E)$ for storing the graph and in-degree array.

Solution

We treat the problem as a topological sort on a directed acyclic graph (DAG). For a BFS approach using Kahn's algorithm:

- Build an adjacency list for the graph and an in-degree array to record the number of prerequisites (incoming edges) for each course.
- Initialize a queue with all courses that have an in-degree of zero (i.e., courses that can be taken without prerequisites).
- Dequeue nodes from the queue, add them to the ordering, and reduce the in-degree of their neighbors.
- If a neighbor's in-degree

becomes zero, add it to the queue. – If the final ordering includes all courses, return the ordering; otherwise, return an empty array indicating a cycle. The chosen data structures are:

- A list (or array) for the in-degree counts.
- A dictionary (or array of lists) for the adjacency list.
- A queue to process nodes with zero in-degree.

Topological Sort

#310 Minimum Height Trees

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 DFS · BFS · Graph · Topological Sort
 Lists: Grind 169

Problem

A tree is an undirected graph in which any two vertices are connected by exactly one path. In other words, any connected graph without simple cycles is a tree.

Given a tree of n nodes labelled from 0 to $n - 1$, and an array of $n - 1$ edges where $\text{edges}[i] = [a_i, b_i]$ indicates that there is an undirected edge between the two nodes a_i and b_i in the tree, you can choose any node of the tree as the root. When you select a node x as the root, the result tree has height h . Among all possible rooted trees, those with minimum height (i.e. $\min(h)$) are called minimum height trees (MHTs).

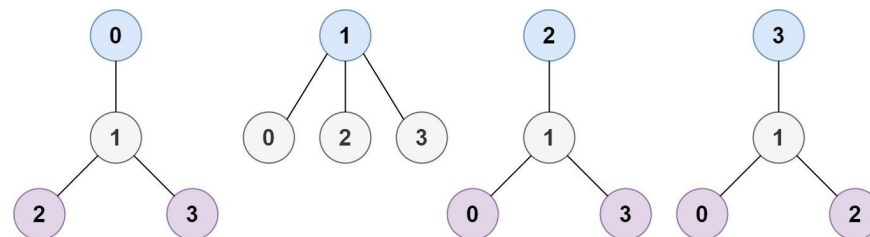
Return a list of all MHTs' root labels. You can return the answer in any order.

Round 5 · Mid · Medium

p.2323

The height of a rooted tree is the number of edges on the longest downward path between the root and a leaf.

Example 1:

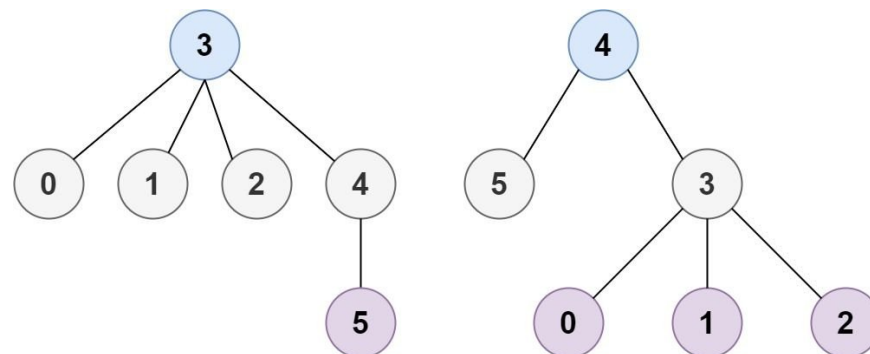


Input: $n = 4$, $\text{edges} = [[1,0],[1,2],[1,3]]$

Output: [1]

Explanation: As shown, the height of the tree is 1 when the root is the node with label 1 which is the only MHT.

Example 2:



Input: $n = 6$, $\text{edges} = [[3,0],[3,1],[3,2],[3,4],[5,4]]$

Output: [3,4]

Constraints:

Round 5 · Mid · Medium

p.2324

- $1 \leq n \leq 2 * 10^4$
- `edges.length == n - 1`
- $0 \leq a_i, b_i < n$
- $a_i \neq b_i$
- All the pairs (a_i, b_i) are distinct.
- The given input is guaranteed to be a tree and there will be no repeated edges.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findMinHeightTrees(self, n: int, edges: list[list[int]]) -> list[int]:
        if n == 1 or not edges:
            return [0]

        ans = []
        graph = collections.defaultdict(set)

        for u, v in edges:
            graph[u].add(v)

            graph[v].add(u)

        for label, children in graph.items():
            if len(children) == 1:
                ans.append(label)

        while n > 2:
            n -= len(ans)
            nextLeaves = []
            for leaf in ans:
```

```
u = next(iter(graph[leaf]))
graph[u].remove(leaf)
if len(graph[u]) == 1:
    nextLeaves.append(u)
ans = nextLeaves

return ans
```

• LeetCode

```
class Solution:
    def findMinHeightTrees(self, n: int, edges: List[List[int]]) -> List[int]:
        if n == 1:
            return [0]
        g = [[] for _ in range(n)]
        degree = [0] * n
        for a, b in edges:
            g[a].append(b)
            g[b].append(a)
            degree[a] += 1

            degree[b] += 1
        q = deque(i for i in range(n) if degree[i] == 1)
        ans = []
        while q:
            ans.clear()
            for _ in range(len(q)):
                a = q.popleft()
                ans.append(a)
                for b in g[a]:
                    degree[b] -= 1

            if degree[b] == 1:
                q.append(b)

        return ans
```

• SimplyLeet

```
# Python solution with line-by-line comments
class Solution:
    def findMinHeightTrees(self, n: int, edges: List[List[int]]) -> List[int]:
        # Special case: if there's only one node, return it as the only MHT
        # root.
        if n == 1:
            return [0]

        # Build the graph using an adjacency list with sets for neighbors.
        graph = [set() for _ in range(n)]
        for u, v in edges:
            graph[u].add(v)
            graph[v].add(u)

        # Initialize the first layer of leaves (nodes with one neighbor).
        leaves = [i for i in range(n) if len(graph[i]) == 1]

        # Remove leaves layer by layer until at most 2 nodes remain.
        remaining_nodes = n
        while remaining_nodes > 2:
            remaining_nodes -= len(leaves)

            new_leaves = []
            # Remove current leaves.
            for leaf in leaves:
                # Since it's a leaf, there is only one neighbor.
                neighbor = graph[leaf].pop()
                # Remove the connection from the neighbor to the leaf.
                graph[neighbor].remove(leaf)
                # If the neighbor becomes a leaf, add it to the new leaves
                list.
                if len(graph[neighbor]) == 1:
                    new_leaves.append(neighbor)

            leaves = new_leaves

        # The remaining nodes are the roots of the MHTs.
        return leaves
```

☐ • LC.ca

Round 5 · Mid · Medium

p.2327

```
from typing import List, Set
from collections import defaultdict

class Solution:
    def findMinHeightTrees(self, n: int, edges: List[List[int]]) -> List[int]:
        if n == 1:
            return [0]

        graph = defaultdict(set)

        for a, b in edges:
            graph[a].add(b)
            graph[b].add(a)

        # just check neighbours[] size, not indgree[] for each node
        leaves = [i for i in range(n) if len(graph[i]) == 1]

        while n > 2:
            n -= len(leaves)
            new_leaves = []

            for leaf in leaves:
                # should be only one in hashset pop(), because it's a leaf node
                neighbor = graph[leaf].pop()
                graph[neighbor].remove(leaf)
                if len(graph[neighbor]) == 1:
                    new_leaves.append(neighbor)
            leaves = new_leaves

        return leaves

#####

class Solution:
    def findMinHeightTrees(self, n: int, edges: List[List[int]]) -> List[int]:
        if n == 1:
            return [0]
        g = defaultdict(list)
        degree = [0] * n
        for a, b in edges:
            g[a].append(b)
```

Round 5 · Mid · Medium

p.2328


```

        g[b].append(a)
        degree[a] += 1 # not needed, just len(g[a]) is good enough, will
update it in next round updating
        degree[b] += 1
        q = deque()
        for i in range(n):
            if degree[i] == 1:
                q.append(i)
        ans = []
        while q:
            n = len(q)

            ans.clear()
            for _ in range(n):
                a = q.popleft()
                ans.append(a)
                for b in g[a]:
                    degree[b] -= 1
                    if degree[b] == 1: # final round only 2 left (a,b), then
degree[b] here will be 0, and no more node enqueue
                        q.append(b)
            return ans

```

◆ Quick Review

Key Insights

- A tree can have at most two centers that will serve as the roots for its minimum height trees.
- Instead of trying all possible roots, we can remove leaves layer by layer until we are left with the central node(s).
- The process is similar to a topological sort on the tree.

- When there are 2 or fewer nodes left, these nodes are the centers (roots) of MHTs.

Space & Time

Time Complexity: $O(n)$ where n is the number of nodes, since we process each node and edge once.

Space Complexity: $O(n)$ for storing the graph and additional data structures like the leaves list.

Solution

The solution builds an undirected graph from the input, then identifies and removes node-leaves iteratively. A leaf is defined as a node connected to only one other node. By repeatedly removing the leaves and updating the graph (decreasing neighbor degrees), the algorithm peels away the outer layers of the tree. When only one or two nodes remain, these nodes are the centers of the tree and form the roots of the minimum height trees.

Topological Sort

□ #802 Find Eventual Safe States

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Depth-First Search · Breadth-First Search ·
 Graph Theory · Topological Sort
 Lists: AlgoMaster 600

Problem

There is a directed graph of n nodes with each node labeled from 0 to $n - 1$. The graph is represented by a 0-indexed 2D integer array `graph` where `graph[i]` is an integer array of nodes adjacent to node i , meaning there is an edge from node i to each node in `graph[i]`.

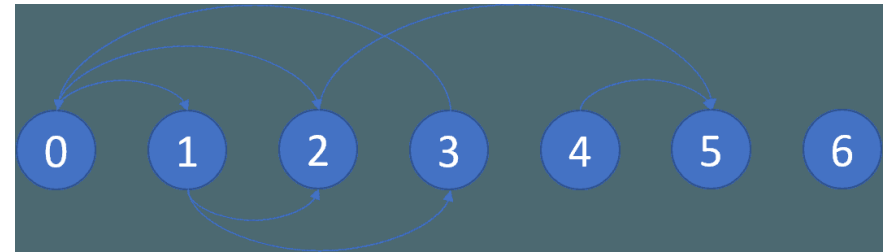
A node is a terminal node if there are no outgoing edges. A node is a safe node if every possible path starting from that node leads to a terminal node (or another safe node).

Return an array containing all the safe nodes of the graph. The answer should be sorted in ascending order.

Example 1:

Round 5 · Mid · Medium

p.2331



Input: `graph = [[1,2],[2,3],[5],[0],[5],[],[[]]]`
 Output: `[2,4,5,6]`
 Explanation: The given graph is shown above.
 Nodes 5 and 6 are terminal nodes as there are no outgoing edges from either of them.
 Every path starting at nodes 2, 4, 5, and 6 all lead to either node 5 or 6.

Example 2:

Input: `graph = [[1,2,3,4],[1,2],[3,4],[0,4],[[]]]`
 Output: `[4]`
 Explanation:
 Only node 4 is a terminal node, and every path starting at node 4 leads to node 4.

Constraints:

- $n == \text{graph.length}$
- $1 \leq n \leq 104$
- $0 \leq \text{graph}[i].\text{length} \leq n$
- $0 \leq \text{graph}[i][j] \leq n - 1$
- `graph[i]` is sorted in a strictly increasing order.
- The graph may contain self-loops.
- The number of edges in the graph will be in the range $[1, 4 * 104]$.

Round 5 · Mid · Medium

p.2332

Community Solutions (Python)

• WalkCC

```
from enum import Enum

class State(Enum):
    INIT = 0
    VISITING = 1
    VISITED = 2

class Solution:

    def eventualSafeNodes(self, graph: List[List[int]]) -> List[int]:
        states = [State.INIT] * len(graph)

        def hasCycle(u: int) -> bool:
            if states[u] == State.VISITING:
                return True
            if states[u] == State.VISITED:
                return False
            states[u] = State.VISITING
            if any(hasCycle(v) for v in graph[u]):
                return True
            states[u] = State.VISITED
            return False

        return [i for i in range(len(graph)) if not hasCycle(i)]
```

• LeetCode

Round 5 · Mid · Medium

p.2333

```
class Solution:
    def eventualSafeNodes(self, graph: List[List[int]]) -> List[int]:
        rg = defaultdict(list)
        indeg = [0] * len(graph)
        for i, vs in enumerate(graph):
            for j in vs:
                rg[j].append(i)
                indeg[i] = len(vs)
        q = deque([i for i, v in enumerate(indeg) if v == 0])
        while q:
            i = q.popleft()
            for j in rg[i]:
                indeg[j] -= 1
                if indeg[j] == 0:
                    q.append(j)
        return [i for i, v in enumerate(indeg) if v == 0]

class Solution:
    def eventualSafeNodes(self, graph: List[List[int]]) -> List[int]:
        def dfs(i):
            if color[i]:
                return color[i] == 2
            color[i] = 1
            for j in graph[i]:
                if not dfs(j):
                    return False
            color[i] = 2

        return True

        n = len(graph)
        color = [0] * n
        return [i for i in range(n) if dfs(i)]
```

• SimplyLeet

Round 5 · Mid · Medium

p.2334

```
# Python solution using DFS and state tracking.
# States: 0 = unvisited, 1 = visiting, 2 = safe.
class Solution:
    def eventualSafeNodes(self, graph: List[List[int]]) -> List[int]:
        n = len(graph)
        state = [0] * n # Initialize all nodes as unvisited.

        def dfs(node):
            # If node is already processed, return True if safe.
            if state[node] != 0:
                return state[node] == 2
            state[node] = 1 # Mark node as visiting.
            for neighbor in graph[node]:
                # If any neighbor leads to a cycle, node is unsafe.
                if not dfs(neighbor):
                    return False
            state[node] = 2 # All neighbors are safe, mark node as safe.
            return True

        safe_nodes = []

        for i in range(n):
            if dfs(i):
                safe_nodes.append(i)
        return safe_nodes
```

Round 5 · Mid · Medium

p.2335

Topological Sort

 ★ #1136 Parallel Courses ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Graph · Topological Sort
Lists: ★ Premium | Premium 98

Problem

You are given an integer n , which indicates that there are n courses labeled from 1 to n . You are also given an array `relations` where `relations[i] = [prevCoursei, nextCoursei]`, representing a prerequisite relationship between course `prevCoursei` and course `nextCoursei`: course `prevCoursei` has to be taken before course `nextCoursei`.

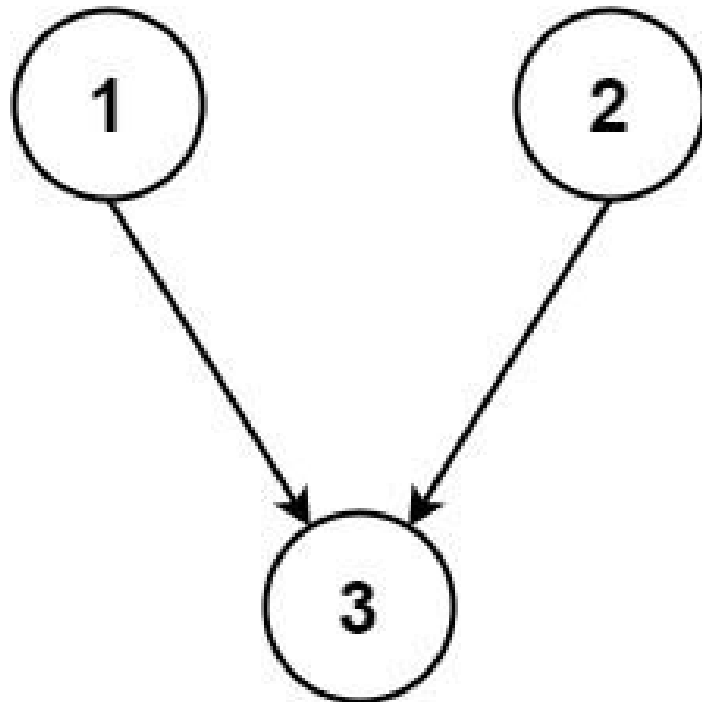
In one semester, you can take any number of courses as long as you have taken all the prerequisites in the previous semester for the courses you are taking.

Return the minimum number of semesters needed to take all courses. If there is no way to take all the courses, return -1 .

Example 1:

Round 5 · Mid · Medium

p.2336



Input: $n = 3$, $\text{relations} = [[1,3],[2,3]]$

Output: 2

Explanation: The figure above represents the given graph.

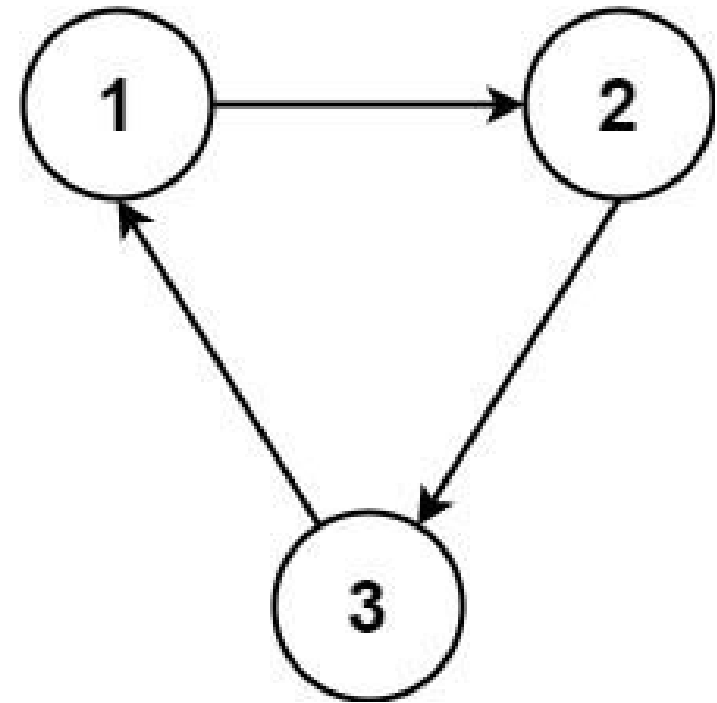
In the first semester, you can take courses 1 and 2.

In the second semester, you can take course 3.

Example 2:

Round 5 · Mid · Medium

p.2337



Input: $n = 3$, $\text{relations} = [[1,2],[2,3],[3,1]]$

Output: -1

Explanation: No course can be studied because they are prerequisites of each other.

Constraints:

- $1 \leq n \leq 5000$
- $1 \leq \text{relations.length} \leq 5000$
- $\text{relations}[i].\text{length} == 2$
- $1 \leq \text{prevCourse}_i, \text{nextCourse}_i \leq n$
- $\text{prevCourse}_i \neq \text{nextCourse}_i$

Round 5 · Mid · Medium

p.2338

- All the pairs [prevCoursei, nextCoursei] are unique.

Community Solutions (Python)

• WalkCC

```
from enum import Enum

class State(Enum):
    INIT = 0
    VISITING = 1
    VISITED = 2

class Solution:

    def minimumSemesters(self, n: int, relations: list[list[int]]) -> int:
        graph = [[] for _ in range(n)]
        states = [State.INIT] * n
        depth = [1] * n

        for u, v in relations:
            graph[u - 1].append(v - 1)

        def hasCycle(u: int) -> bool:
            if states[u] == State.VISITING:
                return True
            if states[u] == State.VISITED:
                return False
            states[u] = State.VISITING
            for v in graph[u]:
                if hasCycle(v):
                    return True
            depth[u] = max(depth[u], 1 + depth[v])
            states[u] = State.VISITED
            return False
```

```
if any(hasCycle(i) for i in range(n)):
    return -1
return max(depth)

class Solution:
    def minimumSemesters(self, n: int, relations: list[list[int]]) -> int:
        graph = [[] for _ in range(n)]
        inDegrees = [0] * n

        # Build the graph.
        for u, v in relations:
            graph[u - 1].append(v - 1)
            inDegrees[v - 1] += 1

        # Perform topological sorting.
        q = collections.deque([i for i, d in enumerate(inDegrees) if d == 0])

        step = 0
        while q:
            for _ in range(len(q)):
                u = q.popleft()
                n -= 1
                for v in graph[u]:
                    inDegrees[v] -= 1

            if inDegrees[v] == 0:
                q.append(v)
            step += 1

        return step if n == 0 else -1
```

• LeetCode

```
class Solution:
    def minimumSemesters(self, n: int, relations: List[List[int]]) -> int:
        g = defaultdict(list)
        indeg = [0] * n
        for prev, nxt in relations:
            prev, nxt = prev - 1, nxt - 1
            g[prev].append(nxt)
            indeg[nxt] += 1
        q = deque(i for i, v in enumerate(indeg) if v == 0)
        ans = 0
```

```
        while q:
            ans += 1
            for _ in range(len(q)):
                i = q.popleft()
                n -= 1
                for j in g[i]:
                    indeg[j] -= 1
                    if indeg[j] == 0:
                        q.append(j)
            return -1 if n else ans
```

• SimplyLeet

Python Implementation

```
from collections import deque
```

```
def minimumSemesters(numCourses, relations):
    # Build graph and in-degree dictionary.
    graph = {i: [] for i in range(1, numCourses + 1)}
    in_degree = {i: 0 for i in range(1, numCourses + 1)}
```

```
    # Construct graph from relations.
```

```
    for pre, course in relations:
        graph[pre].append(course)
        in_degree[course] += 1
```

```
    # Initialize queue with all courses having no prerequisites.
    queue = deque()
    for course in range(1, numCourses + 1):
        if in_degree[course] == 0:
            queue.append(course)
```

```
    semesters = 0
    takenCourses = 0
```

```
    # Process each level (semester) using BFS.
    while queue:
        semesters += 1
        current_level_size = len(queue)
        for _ in range(current_level_size):
            course = queue.popleft()
            takenCourses += 1
```

```
    # Decrement in-degree for neighboring courses.
    for neighbor in graph[course]:
        in_degree[neighbor] -= 1
        if in_degree[neighbor] == 0:
            queue.append(neighbor)
```

```
    # If not all courses have been taken, a cycle exists.
    return semesters if takenCourses == numCourses else -1
```

```
# Example usage:
```

```
print(minimumSemesters(3, [[1, 3], [2, 3]])) # Expected output: 2
```

• LC.ca

```

class Solution:
    def minimumSemesters(self, n: int, relations: List[List[int]]) -> int:
        g = defaultdict(list)
        indeg = [0] * n
        for prev, nxt in relations:
            prev, nxt = prev - 1, nxt - 1
            g[prev].append(nxt)
            indeg[nxt] += 1
        q = deque(i for i, v in enumerate(indeg) if v == 0)
        ans = 0

        while q:
            ans += 1
            for _ in range(len(q)):
                i = q.popleft()
                n -= 1
                for j in g[i]:
                    indeg[j] -= 1
                    if indeg[j] == 0:
                        q.append(j)
            return -1 if n else ans

```

◆ Quick Review

Key Insights

- Model the problem as a directed graph where nodes are courses and edges represent prerequisites.
- Use topological sorting (Kahn's algorithm) to process courses with zero prerequisites.
- Each level (or layer) of processing represents one semester.
- Detect cycles by ensuring all courses are eventually processed; if not, a cycle exists.

- The number of BFS levels gives the minimum number of semesters necessary.

Space & Time

Time Complexity: $O(n + m)$ where n is the number of courses and m is the length of relations (prerequisites).

Space Complexity: $O(n + m)$ for storing the graph, in-degree counts, and the processing queue.

Solution

We solve the problem using a topological sort with Kahn's algorithm. First, construct a graph representation using an adjacency list and compute the in-degree for each course. Initialize a queue with courses that have no prerequisites (in-degree == 0). Then, process courses level by level. Each level processed represents one semester. For every course taken in that semester, decrease the in-degree of all its neighbors. Add any neighbor with an in-degree that becomes zero to the queue for the next semester. If, after processing, the total courses taken is less than n , then a cycle exists and return -1 . Otherwise, the number of

semesters processed is the answer.

Round 5 · Mid · Medium

p.2345

Topological Sort

□ #1462 Course Schedule IV

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Depth-First Search · Breadth-First Search ·
 Graph Theory · Topological Sort
 Lists: AlgoMaster 600

Problem

There are a total of `numCourses` courses you have to take, labeled from 0 to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [ai, bi]` indicates that you must take course `ai` first if you want to take course `bi`.

- For example, the pair `[0, 1]` indicates that you have to take course 0 before you can take course 1.

Prerequisites can also be indirect. If course `a` is a prerequisite of course `b`, and course `b` is a prerequisite of course `c`, then course `a` is a prerequisite of course `c`.

You are also given an array `queries` where `queries[j] = [uj, vj]`. For the `j`th query, you should answer whether course `uj` is a prerequisite of

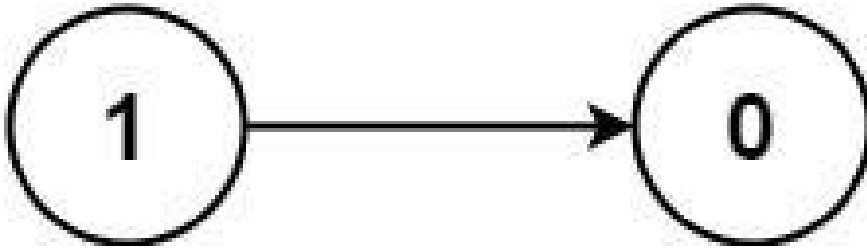
Round 5 · Mid · Medium

p.2346

course v_j or not.

Return a boolean array `answer`, where `answer[j]` is the answer to the j th query.

Example 1:

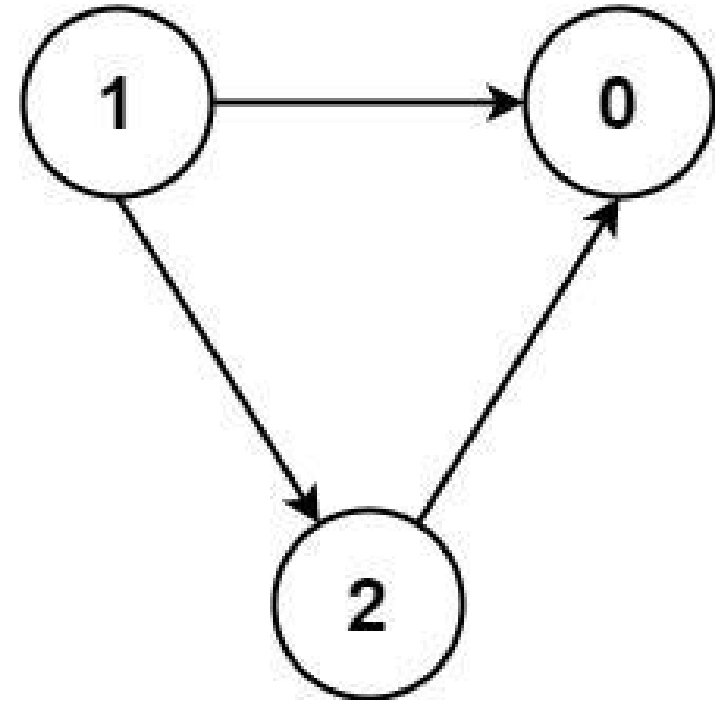


Input: `numCourses = 2, prerequisites = [[1,0]], queries = [[0,1],[1,0]]`
 Output: `[false,true]`
 Explanation: The pair `[1, 0]` indicates that you have to take course 1 before you can take course 0.
 Course 0 is not a prerequisite of course 1, but the opposite is true.

Example 2:

Input: `numCourses = 2, prerequisites = [], queries = [[1,0],[0,1]]`
 Output: `[false,false]`
 Explanation: There are no prerequisites, and each course is independent.

Example 3:



Input: `numCourses = 3, prerequisites = [[1,2],[1,0],[2,0]], queries = [[1,0],[1,2]]`
 Output: `[true,true]`

Constraints:

- $2 \leq \text{numCourses} \leq 100$
- $0 \leq \text{prerequisites.length} \leq (\text{numCourses} * (\text{numCourses} - 1) / 2)$
- `prerequisites[i].length == 2`
- $0 \leq a_i, b_i \leq \text{numCourses} - 1$
- $a_i \neq b_i$

- All the pairs [ai, bi] are unique.
- The prerequisites graph has no cycles.
- $1 \leq \text{queries.length} \leq 104$
- $0 \leq \text{ui}, \text{vi} \leq \text{numCourses} - 1$
- $\text{ui} \neq \text{vi}$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def checkIfPrerequisite(
        self,
        numCourses: int,
        prerequisites: list[list[int]],
        queries: list[list[int]],
    ) -> list[bool]:
        graph = [[] for _ in range(numCourses)]
        # isPrerequisite[i][j] := True if course i is a prerequisite of course j.
        isPrerequisite = [[False] * numCourses for _ in range(numCourses)]

        for u, v in prerequisites:
            graph[u].append(v)

        # DFS from every course.
        for i in range(numCourses):
            self._dfs(graph, i, isPrerequisite[i])

        return [isPrerequisite[u][v] for u, v in queries]

    def _dfs(self, graph: list[list[int]], u: int, used: list[bool]) -> None:
        for v in graph[u]:
            if used[v]:
                continue
            used[v] = True
            self._dfs(graph, v, used)
```

Round 5 · Mid · Medium

p.2349

• LeetCode

```
class Solution:
    def checkIfPrerequisite(
        self, n: int, prerequisites: List[List[int]], queries: List[List[int]]
    ) -> List[bool]:
        f = [[False] * n for _ in range(n)]
        for a, b in prerequisites:
            f[a][b] = True
        for k in range(n):
            for i in range(n):
                for j in range(n):
                    if f[i][k] and f[k][j]:
                        f[i][j] = True
        return [f[a][b] for a, b in queries]
```

• SimplyLeet

```
# Python solution using Floyd Warshall algorithm
def check_prerequisites(numCourses, prerequisites, queries):
    # Create a 2D reachability matrix initialized to False
    reachable = [[False] * numCourses for _ in range(numCourses)]

    # Set direct prerequisites in the matrix
    for pre, course in prerequisites:
        reachable[pre][course] = True

    # Compute transitive closure with Floyd Warshall algorithm

    for k in range(numCourses):
        for i in range(numCourses):
            if reachable[i][k]:
                for j in range(numCourses):
                    if reachable[k][j]:
                        reachable[i][j] = True

    # Process each query and return the results
    return [reachable[u][v] for u, v in queries]
```

Round 5 · Mid · Medium

p.2350

```
# Example usage:
numCourses = 3
prerequisites = [[1,2],[1,0],[2,0]]
queries = [[1,0],[1,2]]
print(check_prerequisites(numCourses, prerequisites, queries)) # Expected
output: [True, True]
```

Topological Sort

#2115 Find All Possible Recipes from Given Supplies

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · String · Graph Theory ·
Topological Sort

Lists: AlgoMaster 600

Problem

You have information about n different recipes. You are given a string array `recipes` and a 2D string array `ingredients`. The i th recipe has the name `recipes[i]`, and you can create it if you have all the needed ingredients from `ingredients[i]`. A recipe can also be an ingredient for other recipes, i.e., `ingredients[i]` may contain a string that is in `recipes`.

You are also given a string array `supplies` containing all the ingredients that you initially have, and you have an infinite supply of all of them.

Return a list of all the recipes that you can create. You may return the answer in any order.

Note that two recipes may contain each other in their ingredients.

Example 1:

Input: recipes = ["bread"], ingredients = [["yeast", "flour"]], supplies = ["yeast", "flour", "corn"]
 Output: ["bread"]
 Explanation:
 We can create "bread" since we have the ingredients "yeast" and "flour".

Example 2:

Input: recipes = ["bread", "sandwich"], ingredients = [["yeast", "flour"], ["bread", "meat"]], supplies = ["yeast", "flour", "meat"]
 Output: ["bread", "sandwich"]
 Explanation:
 We can create "bread" since we have the ingredients "yeast" and "flour".
 We can create "sandwich" since we have the ingredient "meat" and can create the ingredient "bread".

Example 3:

Input: recipes = ["bread", "sandwich", "burger"], ingredients = [["yeast", "flour"], ["bread", "meat"], ["sandwich", "meat", "bread"]], supplies = ["yeast", "flour", "meat"]
 Output: ["bread", "sandwich", "burger"]
 Explanation:
 We can create "bread" since we have the ingredients "yeast" and "flour".
 We can create "sandwich" since we have the ingredient "meat" and can create the ingredient "bread".
 We can create "burger" since we have the ingredient "meat" and can create the ingredients "bread" and "sandwich".

Constraints:

- $n == \text{recipes.length} == \text{ingredients.length}$
- $1 \leq n \leq 100$
- $1 \leq \text{ingredients}[i].\text{length}, \text{supplies.length} \leq 100$

- $1 \leq \text{recipes}[i].\text{length}, \text{ingredients}[i][j].\text{length}, \text{supplies}[k].\text{length} \leq 10$
- $\text{recipes}[i]$, $\text{ingredients}[i][j]$, and $\text{supplies}[k]$ consist only of lowercase English letters.
- All the values of recipes and supplies combined are unique.
- Each $\text{ingredients}[i]$ does not contain any duplicate values.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findAllRecipes(
        self,
        recipes: list[str],
        ingredients: list[list[str]],
        supplies: list[str],
    ) -> list[str]:
        ans = []
        supplies = set(supplies)
        graph = collections.defaultdict(list)

        inDegrees = collections.Counter()
        q = collections.deque()

        # Build the graph.
        for i, recipe in enumerate(recipes):
            for ingredient in ingredients[i]:
                if ingredient not in supplies:
                    graph[ingredient].append(recipe)
                    inDegrees[recipe] += 1
```

```
# Perform topological sorting.
for recipe in recipes:
    if inDegrees[recipe] == 0:
        q.append(recipe)

while q:
    u = q.popleft()
    ans.append(u)
    for v in graph[u]:
        inDegrees[v] -= 1

    if inDegrees[v] == 0:
        q.append(v)

return ans
```

• LeetCode

```
class Solution:
    def findAllRecipes(
        self, recipes: List[str], ingredients: List[List[str]], supplies: List[
            str]
        ) -> List[str]:
        g = defaultdict(list)
        indeg = defaultdict(int)
        for a, b in zip(recipes, ingredients):
            for v in b:
                g[v].append(a)
                indeg[a] += len(b)

        q = supplies
        ans = []
        for i in q:
            for j in g[i]:
                indeg[j] -= 1
                if indeg[j] == 0:
                    ans.append(j)
                    q.append(j)

        return ans
```

• SimplyLeet

Round 5 · Mid · Medium

p.2355

```
# Python solution using BFS and topological sort.
from collections import deque, defaultdict

def findAllRecipes(recipes, ingredients, supplies):
    # Map each ingredient to the recipes that need it.
    ingredient_to_recipes = defaultdict(list)
    # Dictionary to count required number of ingredients for each recipe.
    recipe_indegree = {}

    # Build graph: for each recipe, iterate over its ingredients.

    for recipe, reqs in zip(recipes, ingredients):
        recipe_indegree[recipe] = len(reqs)
        for ingredient in reqs:
            ingredient_to_recipes[ingredient].append(recipe)

    result = []
    q = deque(supplies) # initialize queue with initial supplies

    # Process available ingredients and recipes.
    while q:

        current = q.popleft()
        # If the current available item is used in any recipe, process it.
        for rec in ingredient_to_recipes[current]:
            # Decrease the recipe's requirement count.
            recipe_indegree[rec] -= 1
            # If all ingredients for the recipe are available, add recipe as an
            ingredient.

            if recipe_indegree[rec] == 0:
                result.append(rec)
                q.append(rec)

    return result

# Example usage:
recipes = ["bread", "sandwich"]
ingredients = [["yeast", "flour"], ["bread", "meat"]]
supplies = ["yeast", "flour", "meat"]
print(findAllRecipes(recipes, ingredients, supplies)) # Output: ["bread",
"sandwich"]
```

Round 5 · Mid · Medium

p.2356

Union Find

Round 5 · Mid · Medium · 7 questions

Round 5 · Mid · Medium

p.2357

Union Find

□ ★ #261 Graph Valid Tree ★

MED

[LeetDoocs](#) · [SimplyLeet](#) · [WalkCC](#) · [LeetCode](#)
[DFS](#) · [BFS](#) · [Union Find](#) · [Graph](#)Lists: ★ [Premium](#) | [Grind 169](#) | [Premium 98](#)

Problem

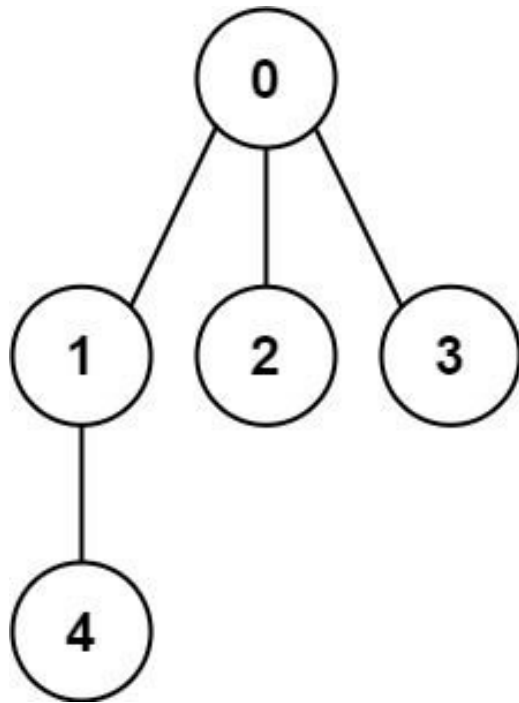
You have a graph of n nodes labeled from 0 to $n - 1$. You are given an integer n and a list of edges where $\text{edges}[i] = [a_i, b_i]$ indicates that there is an undirected edge between nodes a_i and b_i in the graph.

Return true if the edges of the given graph make up a valid tree, and false otherwise.

Example 1:

Round 5 · Mid · Medium

p.2358

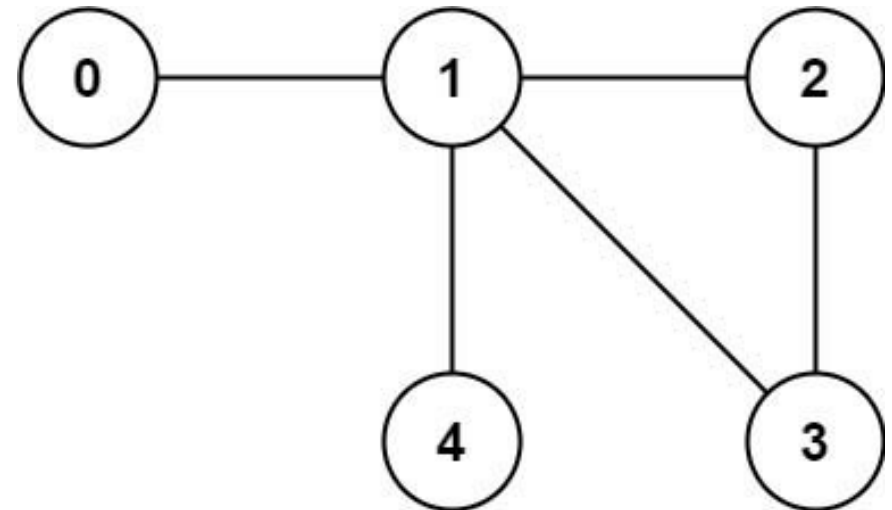


Input: $n = 5$, $edges = [[0,1],[0,2],[0,3],[1,4]]$
 Output: true

Example 2:

Round 5 · Mid · Medium

p.2359



Input: $n = 5$, $edges = [[0,1],[1,2],[2,3],[1,3],[1,4]]$
 Output: false

Constraints:

- $1 \leq n \leq 2000$
- $0 \leq edges.length \leq 5000$
- $edges[i].length == 2$
- $0 \leq a_i, b_i < n$
- $a_i \neq b_i$
- There are no self-loops or repeated edges.

Community Solutions (Python)

- WalkCC

Round 5 · Mid · Medium

p.2360


```
class Solution:
    def validTree(self, n: int, edges: list[list[int]]) -> bool:
        if n == 0 or len(edges) != n - 1:
            return False
```

```
graph = [[] for _ in range(n)]
q = collections.deque([0])
seen = {0}
```

```
for u, v in edges:
```

```
    graph[u].append(v)
    graph[v].append(u)
```

```
while q:
    u = q.popleft()
    for v in graph[u]:
        if v not in seen:
            q.append(v)
            seen.add(v)
```

```
return len(seen) == n
```

```
class UnionFind:
    def __init__(self, n: int):
        self.count = n
        self.id = list(range(n))
        self.rank = [0] * n
```

```
    def unionByRank(self, u: int, v: int) -> None:
        i = self._find(u)
        j = self._find(v)
        if i == j:
```

```
        return
        if self.rank[i] < self.rank[j]:
            self.id[i] = j
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
        else:
            self.id[i] = j
            self.rank[j] += 1
        self.count -= 1
```

```
    def _find(self, u: int) -> int:
        if self.id[u] != u:
            self.id[u] = self._find(self.id[u])
        return self.id[u]
```

```
class Solution:
    def validTree(self, n: int, edges: list[list[int]]) -> bool:
        if n == 0 or len(edges) != n - 1:
            return False
```

```
uf = UnionFind(n)
```

```
for u, v in edges:
    uf.unionByRank(u, v)
```

```
return uf.count == 1
```

• LeetCode

```
class Solution:
    def validTree(self, n: int, edges: List[List[int]]) -> bool:
        def find(x: int) -> int:
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        p = list(range(n))
        for a, b in edges:
            pa, pb = find(a), find(b)
```

```

        if pa == pb:
            return False
        p[pa] = pb
        n -= 1
        return n == 1

class Solution:
    def validTree(self, n: int, edges: List[List[int]]) -> bool:
        def dfs(i: int):
            vis.add(i)
            for j in g[i]:
                if j not in vis:
                    dfs(j)

        if len(edges) != n - 1:
            return False

        g = [[] for _ in range(n)]
        for a, b in edges:
            g[a].append(b)
            g[b].append(a)
        vis = set()
        dfs(0)
        return len(vis) == n

```

• SimplyLeet

```

# Python solution with DFS

def validTree(n, edges):
    # If the number of edges is not exactly n - 1, it cannot be a tree
    if len(edges) != n - 1:
        return False

    # Build the graph as an adjacency list
    graph = {i: [] for i in range(n)}
    for a, b in edges:

```

Round 5 · Mid · Medium

p.2363

```

        graph[a].append(b)
        graph[b].append(a)

        visited = set()

        # DFS helper function that accepts current node and its parent node
        def dfs(node, parent):
            # Add the current node to visited
            visited.add(node)
            # Traverse all the neighboring nodes

            for neighbor in graph[node]:
                # Skip the neighbor if it is the parent to avoid trivial cycle
                # detection
                if neighbor == parent:
                    continue
                # If neighbor is already visited, a cycle is detected
                if neighbor in visited:
                    return False
                # Recurse deeper; if cycle is found in recursion, propagate False
                # upward
                if not dfs(neighbor, node):
                    return False

            return True

        # Start DFS from node 0, with parent set to -1 (non-existent)
        if not dfs(0, -1):
            return False

        # Check if all nodes were visited (i.e., the graph is connected)
        return len(visited) == n

# Example usage:

print(validTree(5, [[0, 1], [0, 2], [0, 3], [1, 4]])) # Expected output: True
print(validTree(5, [[0, 1], [1, 2], [2, 3], [1, 3], [1, 4]])) # Expected
output: False

```

• LC.ca

Round 5 · Mid · Medium

p.2364

```

class Solution:
    def validTree(self, n: int, edges: List[List[int]]) -> bool:
        def find(x):
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        p = list(range(n))
        for a, b in edges:
            if find(a) == find(b):
                return False
            p[find(a)] = find(b)
            n -= 1
        return n == 1

```

◆ Quick Review

Key Insights

- A tree must be acyclic and connected.
- For n nodes, a valid tree must have exactly $n - 1$ edges.
- A DFS/BFS traversal can check connectivity and simultaneously detect cycles.
- The Union-Find (disjoint set) approach provides an alternative for cycle detection and connectivity verification.

Space & Time

Time Complexity: $O(n + e)$, where e is the number of edges

Space Complexity: $O(n + e)$ for storing the graph and traversal/union-find structures

Solution

We can solve the problem using either a DFS/BFS approach or a Union-Find algorithm. For the DFS/BFS method, we first check if the number of edges is exactly $n - 1$. If not, it cannot be a tree. Then, we build an adjacency list for the graph and perform a DFS starting from node 0 while keeping track of visited nodes and parent information to avoid false-positive cycle detections. If during DFS we encounter a node that has been visited (and is not the parent), then a cycle exists. Finally, if all nodes are reached from node 0 and no cycle is found, the graph is a valid tree. Using Union-Find, we iterate over each edge and try to union the two nodes. If they already share the same parent (i.e., are in the same set), a cycle exists. After processing all edges, we confirm that exactly one connected component exists.

Union Find

★ #323 Number of Connected Components in an Undirected Graph ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
DFS · BFS · Union Find · Graph

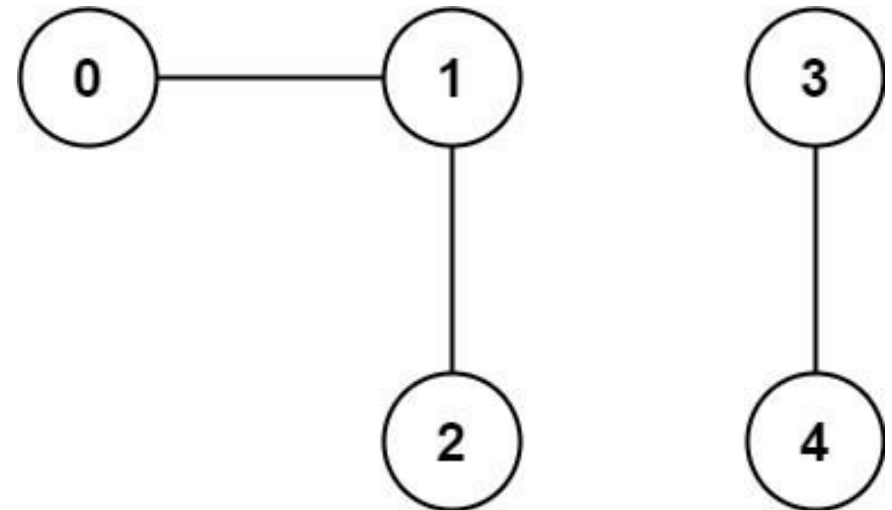
Lists: ★ Premium | Grind 169 | Premium 98

Problem

You have a graph of n nodes. You are given an integer n and an array `edges` where `edges[i] = [ai, bi]` indicates that there is an edge between a_i and b_i in the graph.

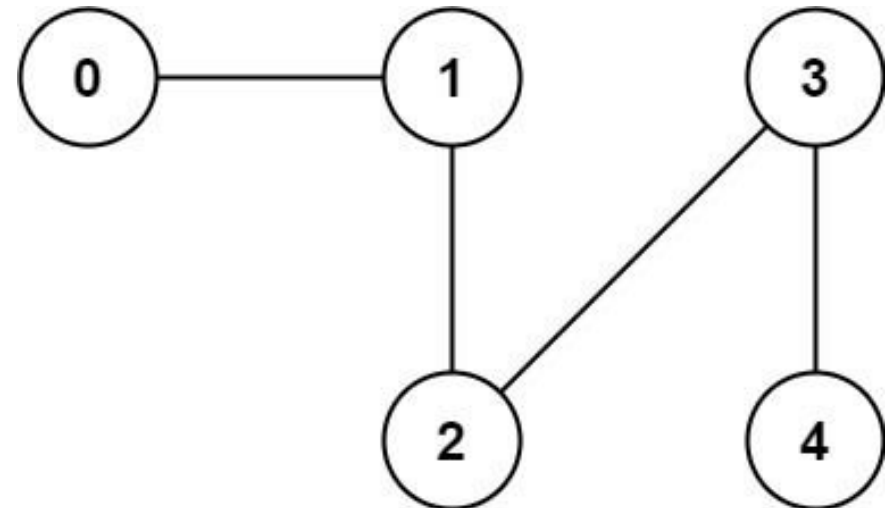
Return the number of connected components in the graph.

Example 1:



Input: $n = 5$, `edges = [[0,1],[1,2],[3,4]]`
Output: 2

Example 2:



Round 5 · Mid · Medium

p.2367

Round 5 · Mid · Medium

p.2368

Input: n = 5, edges = [[0,1],[1,2],[2,3],[3,4]]
Output: 1

Constraints:

- $1 \leq n \leq 2000$
- $1 \leq \text{edges.length} \leq 5000$
- $\text{edges}[i].\text{length} == 2$
- $0 \leq a_i \leq b_i < n$
- $a_i \neq b_i$
- There are no repeated edges.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def countComponents(self, n: int, edges: list[list[int]]) -> int:
        ans = 0
        graph = [[] for _ in range(n)]
        seen = set()

        for u, v in edges:
            graph[u].append(v)
            graph[v].append(u)

        def bfs(node: int, seen: set[int]) -> None:
            q = collections.deque([node])
            seen.add(node)

            while q:
                u = q.pop()
                for v in graph[u]:
                    if v not in seen:
                        q.append(v)
                        seen.add(v)
```

Round 5 · Mid · Medium

p.2369

```
for i in range(n):
    if i not in seen:
        bfs(i, seen)
        ans += 1

return ans

class UnionFind:
    def __init__(self, n: int):
        self.count = n
        self.id = list(range(n))
        self.rank = [0] * n

    def unionByRank(self, u: int, v: int) -> None:
        i = self._find(u)
        j = self._find(v)
        if i == j:

            return
        if self.rank[i] < self.rank[j]:
            self.id[i] = j
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
        else:
            self.id[i] = j
            self.rank[j] += 1
        self.count -= 1

    def _find(self, u: int) -> int:
        if self.id[u] != u:
            self.id[u] = self._find(self.id[u])
        return self.id[u]
```

```
class Solution:
    def countComponents(self, n: int, edges: list[list[int]]) -> int:
        uf = UnionFind(n)
```

Round 5 · Mid · Medium

p.2370

```

for u, v in edges:
    uf.unionByRank(u, v)

return uf.count

```

• LeetCode

```

class Solution:
    def countComponents(self, n: int, edges: List[List[int]]) -> int:
        def dfs(i: int) -> int:
            if i in vis:
                return 0
            vis.add(i)
            for j in g[i]:
                dfs(j)
            return 1

```

```

g = [[] for _ in range(n)]
for a, b in edges:
    g[a].append(b)
    g[b].append(a)
vis = set()
return sum(dfs(i) for i in range(n))

```

```

class UnionFind:
    def __init__(self, n):
        self.p = list(range(n))
        self.size = [1] * n

    def find(self, x):
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]

```

```

def union(self, a, b):
    pa, pb = self.find(a), self.find(b)
    if pa == pb:
        return False
    if self.size[pa] > self.size[pb]:
        self.p[pb] = pa
        self.size[pa] += self.size[pb]
    else:
        self.p[pa] = pb
        self.size[pb] += self.size[pa]

```

```

return True

```

```

class Solution:
    def countComponents(self, n: int, edges: List[List[int]]) -> int:
        uf = UnionFind(n)
        for a, b in edges:
            n -= uf.union(a, b)
        return n

```

```

class Solution:
    def countComponents(self, n: int, edges: List[List[int]]) -> int:
        g = [[] for _ in range(n)]
        for a, b in edges:
            g[a].append(b)
            g[b].append(a)
        vis = set()
        ans = 0
        for i in range(n):
            if i in vis:

```

```

                continue
            vis.add(i)
            q = deque([i])
            while q:
                a = q.popleft()
                for b in g[a]:
                    if b not in vis:
                        vis.add(b)
                        q.append(b)
            ans += 1

```

Round 5 · Mid · Medium

p.2371

Round 5 · Mid · Medium

p.2372

```
return ans
```

• SimplyLeet

```
# Function to count the number of connected components in an undirected graph.
def count_components(n, edges):
    # Build the adjacency list for the graph.
    graph = {i: [] for i in range(n)}
    for a, b in edges:
        graph[a].append(b)
        graph[b].append(a)

    # Set to keep track of visited nodes.
    visited = set()

    # Helper function to perform DFS traversal starting from a node.
    def dfs(node):
        stack = [node]
        while stack:
            current = stack.pop()
            if current not in visited:
                visited.add(current)
                for neighbor in graph[current]:
                    if neighbor not in visited:
                        stack.append(neighbor)

    # Count connected components by initiating DFS on unvisited nodes.
    count = 0
    for i in range(n):
        if i not in visited:
            dfs(i)
            count += 1
    return count

# Example usage:
print(count_components(5, [[0,1],[1,2],[3,4]])) # Expected output: 2
```

• LC.ca

Round 5 · Mid · Medium

p.2373

```
'''
example-2: [[0, 1], [1, 2], [2, 3], [3, 4]]

for loop

[0, 1]
    0 parent set to 1
    1 parent set to 1

[1, 2]

    1 parent set to 2
    2 parent set to 2

[2, 3]
    2 parent set to 3
    3 parent set to 3

final:
    0 parent set to 1
    1 parent set to 2

    2 parent set to 3
    3 parent set to 3

==> so filter for 'i == find(i)'
'''

class Solution:
    def countComponents(self, n: int, edges: List[List[int]]) -> int:
        def find(x):
            if p[x] != x:
                p[x] = find(p[x])

        return p[x]

        p = list(range(n))
        for a, b in edges:
            p[find(a)] = find(b)
        return sum(i == find(i) for i in range(n)) # or len(set(parents))

#####

class Solution: # more literal
```

Round 5 · Mid · Medium

p.2374

```
def countComponents(self, n: int, edges: List[List[int]]) -> int:
    # Initialize parent array for union-find
    parent = [i for i in range(n)]

    # Perform union-find on each edge
    for u, v in edges:
        self.union(u, v, parent)

    # Count the number of distinct parents (connected components)
    count = len(set(self.find(x, parent) for x in range(n)))

    return count

def union(self, x: int, y: int, parent: List[int]) -> None:
    rootX = self.find(x, parent)
    rootY = self.find(y, parent)
    if rootX != rootY:
        parent[rootX] = rootY

def find(self, x: int, parent: List[int]) -> int:
    if parent[x] != x:
        parent[x] = self.find(parent[x], parent)
    return parent[x]
```

◆ Quick Review

Key Insights

- Use an adjacency list to represent the graph.
- A DFS (or BFS) traversal from an unvisited node will traverse an entire connected component.
- Each new DFS/BFS call from an unvisited node signals a separate connected component.

Round 5 · Mid · Medium

p.2375

- Alternatively, the Union-Find data structure can be used to union connected nodes and count distinct sets.

Space & Time

Time Complexity: $O(n + e)$, where n is the number of nodes and e is the number of edges.

Space Complexity: $O(n + e)$ due to the storage of the graph and tracking of visited nodes.

Solution

We solve the problem by iterating over each node and performing a DFS for any node that hasn't been visited. The DFS uses an iterative approach with a stack to traverse all nodes in a connected component. Each DFS initiation corresponds to one connected component. This method efficiently explores the graph using an adjacency list and marks nodes as visited to prevent redundant traversals.

Round 5 · Mid · Medium

p.2376

Union Find

#399 Evaluate Division

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode

Array · String · Depth-First Search ·
Breadth-First Search · Union-Find · Graph
Theory · Shortest Path

Lists: AlgoMaster 600

Problem

You are given an array of variable pairs equations and an array of real numbers values, where $\text{equations}[i] = [A_i, B_i]$ and $\text{values}[i]$ represent the equation $A_i / B_i = \text{values}[i]$. Each A_i or B_i is a string that represents a single variable.

You are also given some queries, where $\text{queries}[j] = [C_j, D_j]$ represents the j th query where you must find the answer for $C_j / D_j = ?$. Return the answers to all queries. If a single answer cannot be determined, return -1.0 .

Note: The input is always valid. You may assume that evaluating the queries will not result in division by zero and that there is no

Round 5 · Mid · Medium

p.2377

contradiction.

Note: The variables that do not occur in the list of equations are undefined, so the answer cannot be determined for them.

Example 1:

```
Input: equations = [["a","b"],["b","c"]], values = [2.0,3.0], queries =
[["a","c"],["b","a"],["a","e"],["a","a"],["x","x"]]
Output: [6.00000,0.50000,-1.00000,1.00000,-1.00000]
Explanation:
Given: a / b = 2.0, b / c = 3.0
queries are: a / c = ?, b / a = ?, a / e = ?, a / a = ?, x / x = ?
return: [6.0, 0.5, -1.0, 1.0, -1.0 ]
note: x is undefined => -1.0
```

Example 2:

```
Input: equations = [["a","b"],["b","c"],["bc","cd"]], values = [1.5,2.5,5.0],
queries = [["a","c"],["c","b"],["bc","cd"],["cd","bc"]]
Output: [3.75000,0.40000,5.00000,0.20000]
```

Example 3:

```
Input: equations = [["a","b"]], values = [0.5], queries =
[["a","b"],["b","a"],["a","c"],["x","y"]]
Output: [0.50000,2.00000,-1.00000,-1.00000]
```

Constraints:

- $1 \leq \text{equations.length} \leq 20$
- $\text{equations}[i].\text{length} == 2$
- $1 \leq A_i.\text{length}, B_i.\text{length} \leq 5$
- $\text{values.length} == \text{equations.length}$
- $0.0 < \text{values}[i] \leq 20.0$
- $1 \leq \text{queries.length} \leq 20$
- $\text{queries}[i].\text{length} == 2$

Round 5 · Mid · Medium

p.2378

- $1 \leq C_j.length, D_j.length \leq 5$
- A_i, B_i, C_j, D_j consist of lower case English letters and digits.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def calcEquation(
        self,
        equations: list[list[str]],
        values: list[float],
        queries: list[list[str]],
    ) -> list[float]:
        ans = []
        # graph[A][B] := A / B
        graph = collections.defaultdict(dict)

        for (A, B), value in zip(equations, values):
            graph[A][B] = value
            graph[B][A] = 1 / value

        def devide(A: str, C: str, seen: set[str]) -> float:
            """Returns A / C."""
            if A == C:
                return 1.0

            seen.add(A)

            # value := A / B
            for B, value in graph[A].items():
                if B in seen:
                    continue
                res = devide(B, C, seen) # B / C
                if res > 0: # valid result
                    return value * res # (A / B) * (B / C) = A / C
```

Round 5 · Mid · Medium

p.2379

```
        return -1.0 # invalid result

    for A, C in queries:
        if A not in graph or C not in graph:
            ans.append(-1.0)
        else:
            ans.append(devide(A, C, set()))

    return ans
```

• LeetDoocs

```
class Solution:
    def calcEquation(
        self, equations: List[List[str]], values: List[float], queries: List[L
ist[str]]
    ) -> List[float]:
        def find(x):
            if p[x] != x:
                origin = p[x]
                p[x] = find(p[x])
                w[x] *= w[origin]
            return p[x]

        w = defaultdict(lambda: 1)
        p = defaultdict()
        for a, b in equations:
            p[a], p[b] = a, b
        for i, v in enumerate(values):
            a, b = equations[i]
            pa, pb = find(a), find(b)
            if pa == pb:
                continue

            p[pa] = pb
            w[pa] = w[b] * v / w[a]
        return [
            -1 if c not in p or d not in p or find(c) != find(d) else w[c] / w
[d]
            for c, d in queries
        ]
```

Round 5 · Mid · Medium

p.2380

• SimplyLeet

Python solution using DFS

```
def calcEquation(equations, values, queries):
    # Build the graph as a dictionary of dictionaries
    graph = {}
    for (dividend, divisor), value in zip(equations, values):
        if dividend not in graph:
            graph[dividend] = {}
        if divisor not in graph:
            graph[divisor] = {}

    graph[dividend][divisor] = value
    graph[divisor][dividend] = 1 / value

    def dfs(current, target, accumulated, visited):
        # If the current node equals the target, return the accumulated product
        if current == target:
            return accumulated
        visited.add(current)
        # Traverse all neighbors
        for neighbor, value in graph[current].items():
            if neighbor in visited:
                continue
            result = dfs(neighbor, target, accumulated * value, visited)
            if result != -1:
                return result
        return -1

    results = []
    for start, end in queries:
        if start not in graph or end not in graph:
```

```
        results.append(-1.0)
    elif start == end:
        results.append(1.0)
    else:
        visited = set()
        result = dfs(start, end, 1, visited)
        results.append(result)

    return results
```

Example usage:

```
if __name__ == "__main__":
    equations = [["a", "b"], ["b", "c"]]
    values = [2.0, 3.0]
    queries = [["a", "c"], ["b", "a"], ["a", "e"], ["a", "a"], ["x", "x"]]
    print(calcEquation(equations, values, queries))
```

Union Find

#547 Number of Provinces

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Depth-First Search · Breadth-First Search ·
 Union-Find · Graph Theory
 Lists: AlgoMaster 600

Problem

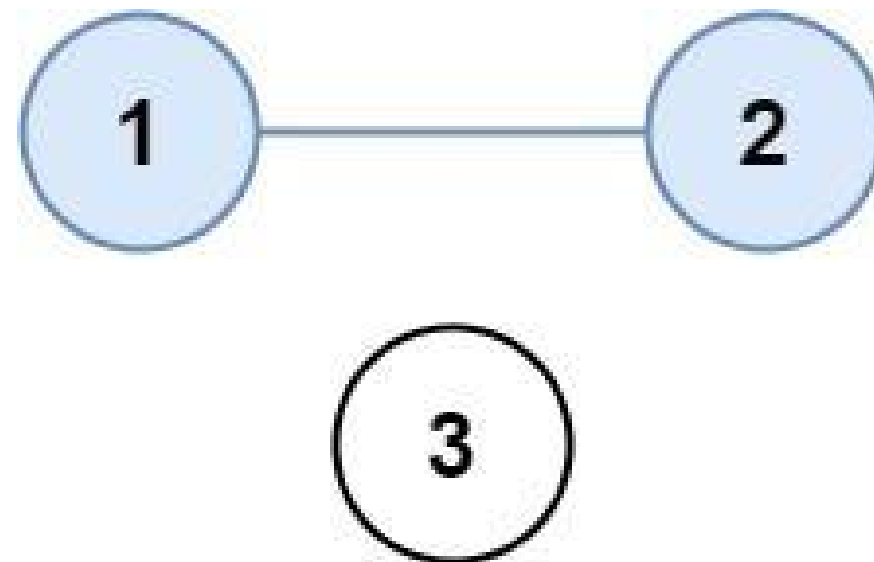
There are n cities. Some of them are connected, while some are not. If city a is connected directly with city b , and city b is connected directly with city c , then city a is connected indirectly with city c .

A province is a group of directly or indirectly connected cities and no other cities outside of the group.

You are given an $n \times n$ matrix `isConnected` where `isConnected[i][j] = 1` if the i th city and the j th city are directly connected, and `isConnected[i][j] = 0` otherwise.

Return the total number of provinces.

Example 1:



Input: `isConnected = [[1,1,0],[1,1,0],[0,0,1]]`
 Output: 2

Example 2:



Input: isConnected = [[1,0,0],[0,1,0],[0,0,1]]
Output: 3

Constraints:

- $1 \leq n \leq 200$
- $n == \text{isConnected.length}$
- $n == \text{isConnected}[i].\text{length}$
- $\text{isConnected}[i][j]$ is 1 or 0.
- $\text{isConnected}[i][i] == 1$
- $\text{isConnected}[i][j] == \text{isConnected}[j][i]$

Community Solutions (Python)

- WalkCC

Round 5 · Mid · Medium

p.2385

```
class UnionFind:
    def __init__(self, n: int):
        self.count = n
        self.id = list(range(n))
        self.rank = [0] * n

    def unionByRank(self, u: int, v: int) -> None:
        i = self._find(u)
        j = self._find(v)
        if i == j:
            return

        if self.rank[i] < self.rank[j]:
            self.id[i] = j
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
        else:
            self.id[i] = j
            self.rank[j] += 1
        self.count -= 1

    def _find(self, u: int) -> int:
        if self.id[u] != u:
            self.id[u] = self._find(self.id[u])
        return self.id[u]

class Solution:
    def findCircleNum(self, isConnected: list[list[int]]) -> int:
        n = len(isConnected)
        uf = UnionFind(n)

        for i in range(n):
            for j in range(i, n):
                if isConnected[i][j] == 1:
                    uf.unionByRank(i, j)

        return uf.count
```

- LeetDoocs

Round 5 · Mid · Medium

p.2386

```
class Solution:
    def findCircleNum(self, isConnected: List[List[int]]) -> int:
        def dfs(i: int):
            vis[i] = True
            for j, x in enumerate(isConnected[i]):
                if not vis[j] and x:
                    dfs(j)

        n = len(isConnected)
        vis = [False] * n
```

```
        ans = 0
        for i in range(n):
            if not vis[i]:
                dfs(i)
                ans += 1
        return ans
```

```
class Solution:
    def findCircleNum(self, isConnected: List[List[int]]) -> int:
        def find(x: int) -> int:
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]
```

```
        n = len(isConnected)
        p = list(range(n))
        ans = n
```

```
        for i in range(n):
            for j in range(i + 1, n):
                if isConnected[i][j]:
                    pa, pb = find(i), find(j)
                    if pa != pb:
                        p[pa] = pb
                        ans -= 1
        return ans
```

☐ • SimplyLeet

Round 5 · Mid · Medium

p.2387

```
# Function to count the number of provinces using DFS
def findCircleNum(isConnected):
    n = len(isConnected) # Total number of cities
    visited = [False] * n # Track visited cities

    # Helper function for Depth-First Search
    def dfs(city):
        visited[city] = True # Mark the current city as visited
        # Explore all neighbors of the current city
        for neighbor in range(n):

            # If the neighbor is connected and not visited yet, explore deeper
            if isConnected[city][neighbor] == 1 and not visited[neighbor]:
                dfs(neighbor)

    provinces = 0 # Initialize province counter
    # Iterate over each city
    for city in range(n):
        if not visited[city]:
            dfs(city) # Start DFS for a new province
            provinces += 1 # Increment province count

    return provinces

# Example usage:
if __name__ == "__main__":
    isConnected = [[1, 1, 0], [1, 1, 0], [0, 0, 1]]
    print(findCircleNum(isConnected)) # Expected Output: 2
```

Round 5 · Mid · Medium

p.2388

Union Find

#684 Redundant Connection

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode

Depth-First Search · Breadth-First Search ·

Union-Find · Graph Theory

Lists: NeetCode 150

Problem

In this problem, a tree is an undirected graph that is connected and has no cycles.

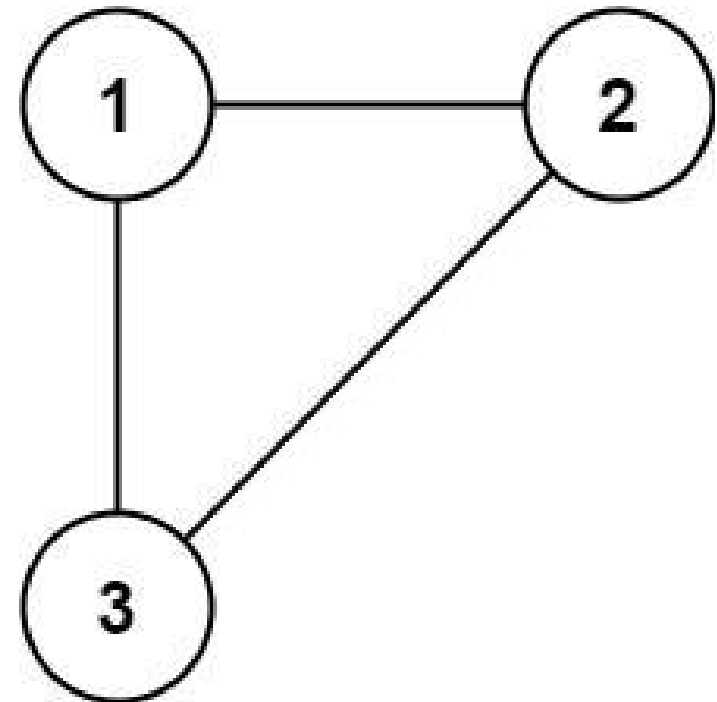
You are given a graph that started as a tree with n nodes labeled from 1 to n , with one additional edge added. The added edge has two different vertices chosen from 1 to n , and was not an edge that already existed. The graph is represented as an array `edges` of length n where `edges[i] = [ai, bi]` indicates that there is an edge between nodes ai and bi in the graph.

Return an edge that can be removed so that the resulting graph is a tree of n nodes. If there are multiple answers, return the answer that occurs last in the input.

Example 1:

Round 5 · Mid · Medium

p.2389

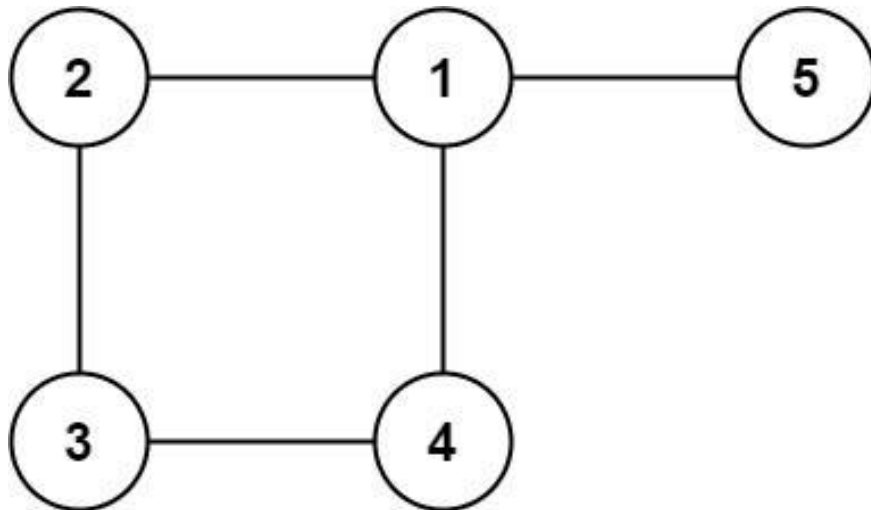


Input: edges = [[1,2],[1,3],[2,3]]
Output: [2,3]

Example 2:

Round 5 · Mid · Medium

p.2390



Input: edges = [[1,2],[2,3],[3,4],[1,4],[1,5]]
 Output: [1,4]

Constraints:

- $n == \text{edges.length}$
- $3 \leq n \leq 1000$
- $\text{edges}[i].\text{length} == 2$
- $1 \leq a_i < b_i \leq \text{edges.length}$
- $a_i \neq b_i$
- There are no repeated edges.
- The given graph is connected.

Community Solutions (Python)

- WalkCC

Round 5 · Mid · Medium

p.2391

```

class UnionFind:
    def __init__(self, n: int):
        self.id = list(range(n))
        self.rank = [0] * n

    def unionByRank(self, u: int, v: int) -> bool:
        i = self._find(u)
        j = self._find(v)
        if i == j:
            return False

        if self.rank[i] < self.rank[j]:
            self.id[i] = j
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
        else:
            self.id[i] = j
            self.rank[j] += 1
        return True

    def _find(self, u: int) -> int:
        if self.id[u] != u:
            self.id[u] = self._find(self.id[u])
        return self.id[u]

class Solution:
    def findRedundantConnection(self, edges: list[list[int]]) -> list[int]:
        uf = UnionFind(len(edges) + 1)

        for edge in edges:
            u, v = edge
            if not uf.unionByRank(u, v):
                return edge
  
```

- LeetCode

Round 5 · Mid · Medium

p.2392


```
class Solution:
    def findRedundantConnection(self, edges: List[List[int]]) -> List[int]:
        def find(x: int) -> int:
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        p = list(range(len(edges)))
        for a, b in edges:
            pa, pb = find(a - 1), find(b - 1)

            if pa == pb:
                return [a, b]
            p[pa] = pb
```

```
class UnionFind:
    __slots__ = "p", "size"

    def __init__(self, n: int):
        self.p: List[int] = list(range(n))
        self.size: List[int] = [1] * n

    def find(self, x: int) -> int:
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])

        return self.p[x]

    def union(self, a: int, b: int) -> bool:
        pa, pb = self.find(a), self.find(b)
        if pa == pb:
            return False
        if self.size[pa] > self.size[pb]:
            self.p[pb] = pa
            self.size[pa] += self.size[pb]
        else:
```

```
        self.p[pa] = pb
        self.size[pb] += self.size[pa]
        return True
```

```
class Solution:
    def findRedundantConnection(self, edges: List[List[int]]) -> List[int]:
        uf = UnionFind(len(edges))
        for a, b in edges:
            if not uf.union(a - 1, b - 1):

                return [a, b]
```

• SimplyLeet

Python solution using Union Find with path compression.

```
class UnionFind:
    def __init__(self, size):
        # Initialize parent pointers for each node.
        self.parent = list(range(size))

    def find(self, x):
        # Find the root of x with path compression.
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, x, y):
        # Union the sets of x and y.
        rootX = self.find(x)
        rootY = self.find(y)
        if rootX == rootY:
            return False # Cycle detected.
        self.parent[rootY] = rootX
```

```

        return True

def findRedundantConnection(edges):
    # Number of nodes is equal to len(edges) because edges were from a tree.
    n = len(edges)
    uf = UnionFind(n + 1) # Account for node labels starting at 1.
    redundant_edge = None
    for edge in edges:
        a, b = edge
        # If union returns False, a cycle is detected.

        if not uf.union(a, b):
            redundant_edge = edge
    return redundant_edge

# Example usage:
edges = [[1,2],[1,3],[2,3]]
print(findRedundantConnection(edges)) # Output: [2, 3]

```

Union Find

#721 Accounts Merge

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · String · DFS · BFS · Union Find
 Lists: Grind 169

Problem

Given a list of accounts where each element `accounts[i]` is a list of strings, where the first element `accounts[i][0]` is a name, and the rest of the elements are emails representing emails of the account.

Now, we would like to merge these accounts. Two accounts definitely belong to the same person if there is some common email to both accounts. Note that even if two accounts have the same name, they may belong to different people as people could have the same name. A person can have any number of accounts initially, but all of their accounts definitely have the same name.

After merging the accounts, return the accounts in the following format: the first element of each

account is the name, and the rest of the elements are emails in sorted order. The accounts themselves can be returned in any order.

Example 1:

Input: accounts = [["John","johnsmith@mail.com","john_newyork@mail.com"],["John","johnsmith@mail.com","john00@mail.com"],["Mary","mary@mail.com"],["John","johnnybravo@mail.com"]]
 Output: [["John","john00@mail.com","john_newyork@mail.com","johnsmith@mail.com"],["Mary","mary@mail.com"],["John","johnnybravo@mail.com"]]
 Explanation:
 The first and second John's are the same person as they have the common email "johnsmith@mail.com".
 The third John and Mary are different people as none of their email addresses are used by other accounts.
 We could return these lists in any order, for example the answer [['Mary', 'mary@mail.com'], ['John', 'johnnybravo@mail.com'], ['John', 'john00@mail.com', 'john_newyork@mail.com', 'johnsmith@mail.com']] would still be accepted.

Example 2:

Input: accounts = [["Gabe","Gabe0@m.co","Gabe3@m.co","Gabe1@m.co"],["Kevin","Kevin3@m.co","Kevin5@m.co","Kevin0@m.co"],["Ethan","Ethan5@m.co","Ethan4@m.co","Ethan0@m.co"],["Hanzo","Hanzo3@m.co","Hanzo1@m.co","Hanzo0@m.co"],["Fern","Fern5@m.co","Fern1@m.co","Fern0@m.co"]]
 Output: [["Ethan","Ethan0@m.co","Ethan4@m.co","Ethan5@m.co"],["Gabe","Gabe0@m.co","Gabe1@m.co","Gabe3@m.co"],["Hanzo","Hanzo0@m.co","Hanzo1@m.co","Hanzo3@m.co"],["Kevin","Kevin0@m.co","Kevin3@m.co","Kevin5@m.co"],["Fern","Fern0@m.co","Fern1@m.co","Fern5@m.co"]]

Constraints:

- $1 \leq \text{accounts.length} \leq 1000$
- $2 \leq \text{accounts}[i].\text{length} \leq 10$
- $1 \leq \text{accounts}[i][j].\text{length} \leq 30$
- $\text{accounts}[i][0]$ consists of English letters.
- $\text{accounts}[i][j]$ (for $j > 0$) is a valid email.

Round 5 · Mid · Medium

p.2397

Community Solutions (Python)

• LeetCode

```
class UnionFind:
    def __init__(self, n):
        self.p = list(range(n))
        self.size = [1] * n

    def find(self, x):
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]

    def union(self, a, b):
        pa, pb = self.find(a), self.find(b)
        if pa == pb:
            return False
        if self.size[pa] > self.size[pb]:
            self.p[pb] = pa
            self.size[pa] += self.size[pb]
        else:
            self.p[pa] = pb
            self.size[pb] += self.size[pa]

        return True

class Solution:
    def accountsMerge(self, accounts: List[List[str]]) -> List[List[str]]:
        uf = UnionFind(len(accounts))
        d = {}
        for i, (_, *emails) in enumerate(accounts):
            for email in emails:
                if email in d:
                    uf.union(i, d[email])
                else:
                    d[email] = i
```

Round 5 · Mid · Medium

p.2398

```

        uf.union(i, d[email])
    else:
        d[email] = i
g = defaultdict(set)
for i, (_, *emails) in enumerate(accounts):
    root = uf.find(i)
    g[root].update(emails)
return [[accounts[root][0]] + sorted(emails) for root, emails in g.items()]

```

• SimplyLeet

```

# Python solution for Accounts Merge

# Define the UnionFind class for disjoint set operations
class UnionFind:
    def __init__(self):
        self.parent = {}

    # Find with path compression
    def find(self, x):
        if self.parent.setdefault(x, x) != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    # Union two nodes
    def union(self, x, y):
        self.parent[self.find(x)] = self.find(y)

class Solution:
    def accountsMerge(self, accounts):
        uf = UnionFind()

```

```

email_to_name = {}

# Iterate over each account and union emails
for account in accounts:
    name = account[0]
    first_email = account[1]
    for email in account[1:]:
        uf.union(first_email, email)
        email_to_name[email] = name

```

```

# Group emails by their root parent
groups = {}
for email in email_to_name:
    parent = uf.find(email)
    groups.setdefault(parent, []).append(email)

# Return the merged account list with sorted emails
result = []
for parent, emails in groups.items():
    result.append([email_to_name[parent]] + sorted(emails))

return result

```

```

# Example usage:
# accounts = [["John", "johnsmith@mail.com", "john_newyork@mail.com"],
#              ["John", "johnsmith@mail.com", "john00@mail.com"],
#              ["Mary", "mary@mail.com"],
#              ["John", "johnnybravo@mail.com"]]
# solution = Solution()
# print(solution.accountsMerge(accounts))

```

• LC.ca

```

class Solution:
    def accountsMerge(self, accounts: List[List[str]]) -> List[List[str]]:
        def find(x):
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        n = len(accounts)
        p = list(range(n))
        email_id = {}

        for i, account in enumerate(accounts):
            name = account[0]
            for email in account[1:]:
                if email in email_id:
                    p[find(i)] = find(email_id[email])
                else:
                    email_id[email] = i
        mp = defaultdict(set)
        for i, account in enumerate(accounts):
            for email in account[1:]:
                mp[find(i)].add(email)

        ans = []
        for i, emails in mp.items():
            t = [accounts[i][0]]
            t.extend(sorted(emails))
            ans.append(t)
        return ans

```

◆ Quick Review

Key Insights

- Two accounts are merged if they share at least one common email.
- Treat each email as a node in a graph; accounts represent connected components.

Round 5 · Mid · Medium

p.2401

- Both DFS/BFS and Union-Find can be used to find connected components.
- Keep a mapping from email to name because all emails in a connected component belong to the same person.
- After grouping, sort the emails and prepend the name to form the final account list.

Space & Time

Time Complexity: $O(N * M * \log(M))$ where N is the number of accounts and M is the number of emails per account (owing to sorting and union operations).

Space Complexity: $O(N * M)$, used for storing email mappings and union-find structures.

Solution

The solution involves using a Union-Find (Disjoint Set Union) data structure to merge emails that are connected by being on the same account. For each account, we union the first email with every other email in that account. We also maintain a mapping from email to owner name. After processing all accounts, we group the emails by their root parent derived from the union-find structure.

Round 5 · Mid · Medium

p.2402

For each group, we sort the emails and prepend the owner's name. Finally, we return the merged list of accounts.

Union Find

☐ #947 Most Stones Removed with Same Row or Column

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Depth-First Search · Union-Find ·
Graph Theory

Lists: AlgoMaster 600

Problem

On a 2D plane, we place n stones at some integer coordinate points. Each coordinate point may have at most one stone.

A stone can be removed if it shares either the same row or the same column as another stone that has not been removed.

Given an array `stones` of length n where `stones[i] = [xi, yi]` represents the location of the i th stone, return the largest possible number of stones that can be removed.

Example 1:

Input: stones = [[0,0],[0,1],[1,0],[1,2],[2,1],[2,2]]

Output: 5

Explanation: One way to remove 5 stones is as follows:

1. Remove stone [2,2] because it shares the same row as [2,1].
2. Remove stone [2,1] because it shares the same column as [0,1].
3. Remove stone [1,2] because it shares the same row as [1,0].
4. Remove stone [1,0] because it shares the same column as [0,0].
5. Remove stone [0,1] because it shares the same row as [0,0].

Example 2:

Input: stones = [[0,0],[0,2],[1,1],[2,0],[2,2]]

Output: 3

Explanation: One way to make 3 moves is as follows:

1. Remove stone [2,2] because it shares the same row as [2,0].
 2. Remove stone [2,0] because it shares the same column as [0,0].
 3. Remove stone [0,2] because it shares the same row as [0,0].
- Stones [0,0] and [1,1] cannot be removed since they do not share a row/column with another stone still on the plane.

Example 3:

Input: stones = [[0,0]]

Output: 0

Explanation: [0,0] is the only stone on the plane, so you cannot remove it.

Constraints:

- $1 \leq \text{stones.length} \leq 1000$
- $0 \leq x_i, y_i \leq 104$
- No two stones are at the same coordinate point.

Community Solutions (Python)

- [LeetDoocs](#)

```
class UnionFind:
    def __init__(self, n):
        self.p = list(range(n))
        self.size = [1] * n

    def find(self, x):
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]

    def union(self, a, b):
        pa, pb = self.find(a), self.find(b)
        if pa == pb:
            return False
        if self.size[pa] > self.size[pb]:
            self.p[pb] = pa
            self.size[pa] += self.size[pb]
        else:
            self.p[pa] = pb
            self.size[pb] += self.size[pa]

        return True

class Solution:
    def removeStones(self, stones: List[List[int]]) -> int:
        uf = UnionFind(len(stones))
        ans = 0
        for i, (x1, y1) in enumerate(stones):
            for j, (x2, y2) in enumerate(stones[:i]):
                if x1 == x2 or y1 == y2:
                    ans += uf.union(i, j)

        return ans
```

- [SimplyLeet](#)

```
# Python solution using DFS

def removeStones(stones):
    # Build mappings from row and column to stone indices
    row_map = {}
    col_map = {}
    n = len(stones)
    for i, (x, y) in enumerate(stones):
        row_map.setdefault(x, []).append(i)
        col_map.setdefault(y, []).append(i)

    visited = [False] * n

    # DFS to traverse all stones connected to the current stone
    def dfs(index):
        if visited[index]:
            return
        visited[index] = True
        x, y = stones[index]
        # Traverse stones in the same row
        for neighbor in row_map.get(x, []):
            if not visited[neighbor]:
                dfs(neighbor)
        # Traverse stones in the same column
        for neighbor in col_map.get(y, []):
            if not visited[neighbor]:
                dfs(neighbor)

    components = 0
    # Perform DFS for each stone that hasn't been visited
    for i in range(n):
        if not visited[i]:
            dfs(i)
            components += 1

    # The result is total stones removed = n - number of connected components
    return n - components

# Example usage:
stones = [[0,0],[0,1],[1,0],[1,2],[2,1],[2,2]]
```

Round 5 · Mid · Medium

p.2407

```
print(removeStones(stones)) # Expected Output: 5
```

Round 5 · Mid · Medium

p.2408

Minimum Spanning Tree

Round 5 · Mid · Medium · 2 questions

Round 5 · Mid · Medium

p.2409

Minimum Spanning Tree

□ #1135 Connecting Cities With Minimum Cost

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Union-Find · Graph Theory · Heap (Priority Queue) · Minimum Spanning Tree

Lists: AlgoMaster 600

Problem

There are n cities labeled from 1 to n . You are given the integer n and an array `connections` where `connections[i] = [xi, yi, costi]` indicates that the cost of connecting city x_i and city y_i (bidirectional connection) is $cost_i$.

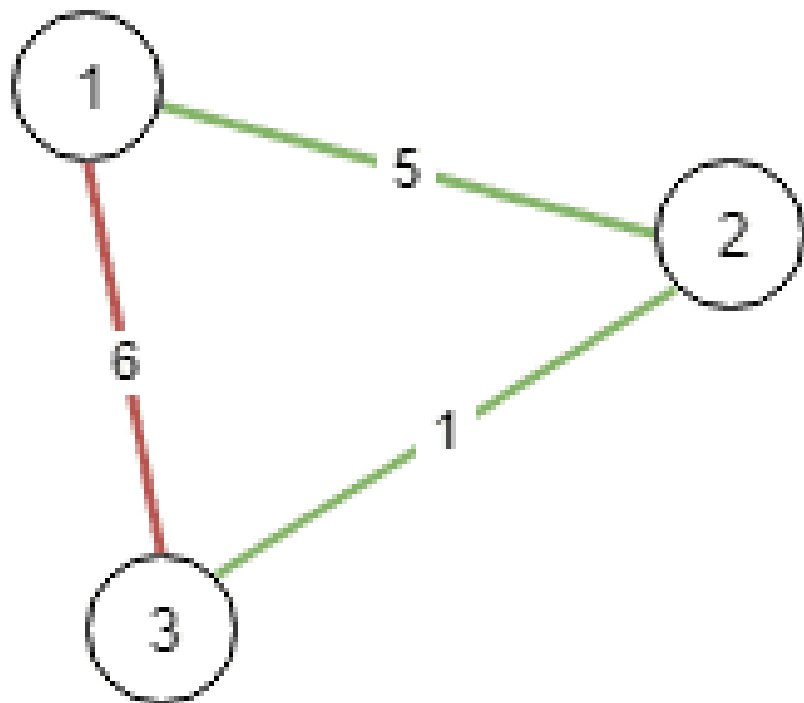
Return the minimum cost to connect all the n cities such that there is at least one path between each pair of cities. If it is impossible to connect all the n cities, return -1 ,

The cost is the sum of the connections' costs used.

Example 1:

Round 5 · Mid · Medium

p.2410

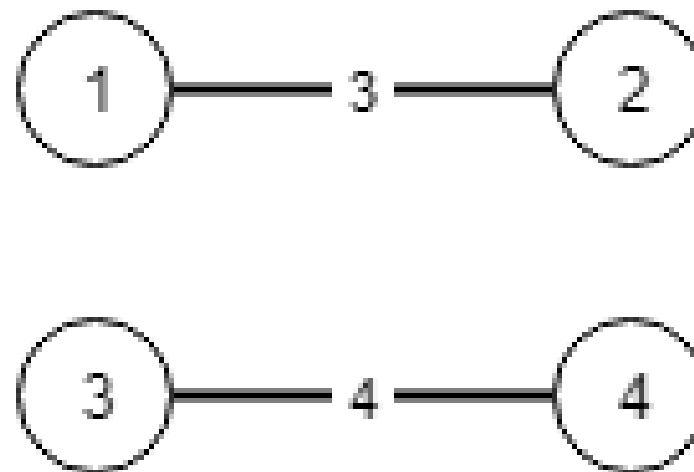


Input: $n = 3$, $\text{connections} = [[1,2,5],[1,3,6],[2,3,1]]$
 Output: 6
 Explanation: Choosing any 2 edges will connect all cities so we choose the minimum 2.

Example 2:

Round 5 · Mid · Medium

p.2411



Input: $n = 4$, $\text{connections} = [[1,2,3],[3,4,4]]$
 Output: -1
 Explanation: There is no way to connect all cities even if all edges are used.

Constraints:

- $1 \leq n \leq 104$
- $1 \leq \text{connections.length} \leq 104$
- $\text{connections}[i].\text{length} == 3$
- $1 \leq x_i, y_i \leq n$
- $x_i \neq y_i$
- $0 \leq \text{cost}_i \leq 105$

Community Solutions (Python)

- WalkCC

Round 5 · Mid · Medium

p.2412

```

class UnionFind:
    def __init__(self, n: int):
        self.id = list(range(n))
        self.rank = [0] * n

    def unionByRank(self, u: int, v: int) -> None:
        i = self.find(u)
        j = self.find(v)
        if i == j:
            return

        if self.rank[i] < self.rank[j]:
            self.id[i] = j
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
        else:
            self.id[i] = j
            self.rank[j] += 1

    def find(self, u: int) -> int:
        if self.id[u] != u:
            self.id[u] = self.find(self.id[u])
        return self.id[u]

class Solution:
    def minimumCost(self, n: int, connections: list[list[int]]) -> int:
        ans = 0
        uf = UnionFind(n + 1)

        # Sort by cost.
        connections.sort(key=lambda x: x[2])

        for u, v, cost in connections:
            if uf.find(u) == uf.find(v):
                continue
            uf.unionByRank(u, v)
            ans += cost

        root = uf.find(1)
        if any(uf.find(i) != root for i in range(1, n + 1)):

```

Round 5 · Mid · Medium

p.2413

```

        return -1

    return ans

```

• LeetCode

```

class Solution:
    def minimumCost(self, n: int, connections: List[List[int]]) -> int:
        def find(x):
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        connections.sort(key=lambda x: x[2])
        p = list(range(n))
        ans = 0

        for x, y, cost in connections:
            x, y = x - 1, y - 1
            if find(x) == find(y):
                continue
            p[find(x)] = find(y)
            ans += cost
            n -= 1
            if n == 1:
                return ans
        return -1

```

• SimplyLeet

```

# Python solution using Union Find (Kruskal's Algorithm)

# Define the Union find class to manage connected components
class UnionFind:
    def __init__(self, size):
        # Parent list: each node is initially its own parent.
        self.parent = list(range(size))

    def find(self, x):
        # Path Compression: make every node on the path point directly to the
        root

```

Round 5 · Mid · Medium

p.2414

```

    if self.parent[x] != x:
        self.parent[x] = self.find(self.parent[x])
    return self.parent[x]

def union(self, x, y):
    # Union the roots of x and y, if they are different
    rootX = self.find(x)
    rootY = self.find(y)
    if rootX == rootY:
        return False # x and y are already in the same set

    self.parent[rootY] = rootX
    return True

def minimumCost(n, connections):
    # Sort the connections based on cost
    connections.sort(key=lambda x: x[2])

    uf = UnionFind(n)
    total_cost = 0
    edges_used = 0

    # Iterate over each connection in sorted order
    for city1, city2, cost in connections:
        # Adjust city labels to 0-indexed
        if uf.union(city1 - 1, city2 - 1):
            total_cost += cost
            edges_used += 1
            # Once we use n-1 edges, all cities are connected
            if edges_used == n - 1:
                return total_cost

    # If we haven't connected all cities, return -1
    return -1

# Example usage:
# print(minimumCost(3, [[1,2,5],[1,3,6],[2,3,1]]))

```

Minimum Spanning Tree

#1584 Min Cost to Connect All Points

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Union-Find · Graph Theory · Minimum
 Spanning Tree
 Lists: NeetCode 150

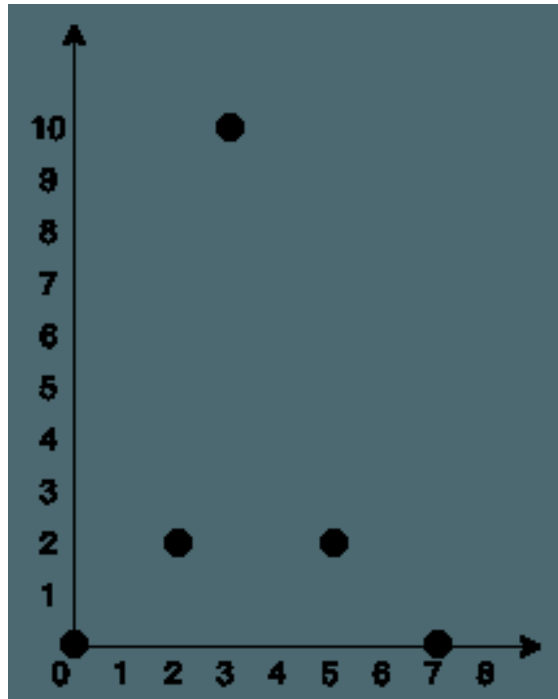
Problem

You are given an array points representing integer coordinates of some points on a 2D-plane, where $\text{points}[i] = [x_i, y_i]$.

The cost of connecting two points $[x_i, y_i]$ and $[x_j, y_j]$ is the manhattan distance between them: $|x_i - x_j| + |y_i - y_j|$, where $|val|$ denotes the absolute value of val .

Return the minimum cost to make all points connected. All points are connected if there is exactly one simple path between any two points.

Example 1:



Input: points = [[0,0],[2,2],[3,10],[5,2],[7,0]]
 Output: 20
 Explanation:

We can connect the points as shown above to get the minimum cost of 20.
 Notice that there is a unique path between every pair of points.

Example 2:

Input: points = [[3,12],[-2,5],[-4,1]]
 Output: 18

Constraints:

- $1 \leq \text{points.length} \leq 1000$
- $-106 \leq x_i, y_i \leq 106$

Round 5 · Mid · Medium

p.2417

- All pairs (x_i, y_i) are distinct.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minCostConnectPoints(self, points: List[int]) -> int:
        # dist[i] := the minimum distance to connect the points[i]
        dist = [math.inf] * len(points)
        ans = 0

        for i in range(len(points) - 1):
            for j in range(i + 1, len(points)):
                # Try to connect the points[i] with the points[j].
                dist[j] = min(dist[j], abs(points[i][0] - points[j][0]) +
                               abs(points[i][1] - points[j][1]))
                # Swap the points[j] (the point with the minimum distance) with the
                # points[i + 1].
                if dist[j] < dist[i + 1]:
                    points[j], points[i + 1] = points[i + 1], points[j]
                    dist[j], dist[i + 1] = dist[i + 1], dist[j]
            ans += dist[i + 1]

        return ans
```

• LeetCode

```
class Solution:
    def minCostConnectPoints(self, points: List[List[int]]) -> int:
        n = len(points)
        g = [[0] * n for _ in range(n)]
        dist = [inf] * n
        vis = [False] * n
        for i, (x1, y1) in enumerate(points):
            for j in range(i + 1, n):
                x2, y2 = points[j]
                t = abs(x1 - x2) + abs(y1 - y2)
```

Round 5 · Mid · Medium

p.2418

```

        g[i][j] = g[j][i] = t
    dist[0] = 0
    ans = 0
    for _ in range(n):
        i = -1
        for j in range(n):
            if not vis[j] and (i == -1 or dist[j] < dist[i]):
                i = j
        vis[i] = True
        ans += dist[i]

```

```

        for j in range(n):
            if not vis[j]:
                dist[j] = min(dist[j], g[i][j])
    return ans

```

```

class Solution:
    def minCostConnectPoints(self, points: List[List[int]]) -> int:
        def find(x: int) -> int:
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

```

```

    n = len(points)
    g = []
    for i, (x1, y1) in enumerate(points):

```

```

        for j in range(i + 1, n):
            x2, y2 = points[j]
            t = abs(x1 - x2) + abs(y1 - y2)
            g.append((t, i, j))
    p = list(range(n))
    ans = 0
    for cost, i, j in sorted(g):
        pa, pb = find(i), find(j)
        if pa == pb:
            continue

```

Round 5 · Mid · Medium

p.2419

```

        p[pa] = pb
        ans += cost
        n -= 1
        if n == 1:
            break
    return ans

```

• SimplyLeet

Python solution using Prim's algorithm

```

class Solution:
    def minCostConnectPoints(self, points):
        # number of points
        n = len(points)
        # initialize cost for each point as infinity, except the first point
        minCost = [float('inf')] * n
        minCost[0] = 0 # start from the first point

```

```

        # keep track of which points are included in the MST
        inMST = [False] * n

```

```

        # total cost of connecting all points
        total_cost = 0

```

```

        for _ in range(n):
            # select the next point with the smallest connection cost
            curr = -1
            curr_cost = float('inf')

```

```

            for i in range(n):
                if not inMST[i] and minCost[i] < curr_cost:
                    curr = i
                    curr_cost = minCost[i]

```

```

            # add the cost of the chosen point
            total_cost += curr_cost
            inMST[curr] = True

```

```

            # update the connection costs for the remaining points

```

Round 5 · Mid · Medium

p.2420

```
    for next_point in range(n):
        if not inMST[next_point]:
            # compute the Manhattan distance between the current and
            next point
            (x1, y1), (x2, y2) = points[curr], points[next_point]
            dist = abs(x1 - x2) + abs(y1 - y2)
            if dist < minCost[next_point]:
                minCost[next_point] = dist

    return total_cost

# Example usage:
# sol = Solution()
# print(sol.minCostConnectPoints([[0,0],[2,2],[3,10],[5,2],[7,0]]))
```

Round 5 · Mid · Medium

p.2421

Shortest Path

Round 5 · Mid · Medium · 8 questions

Round 5 · Mid · Medium

p.2422

Shortest Path

★ #505 The Maze II ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · DFS · BFS · Graph · Matrix · Heap ·
 Shortest Path

Lists: ★ Premium | Premium 98

Problem

There is a ball in a maze with empty spaces (represented as 0) and walls (represented as 1). The ball can go through the empty spaces by rolling up, down, left or right, but it won't stop rolling until hitting a wall. When the ball stops, it could choose the next direction.

Given the $m \times n$ maze, the ball's start position and the destination, where $\text{start} = [\text{startrow}, \text{startcol}]$ and $\text{destination} = [\text{destinationrow}, \text{destinationcol}]$, return the shortest distance for the ball to stop at the destination. If the ball cannot stop at destination, return -1.

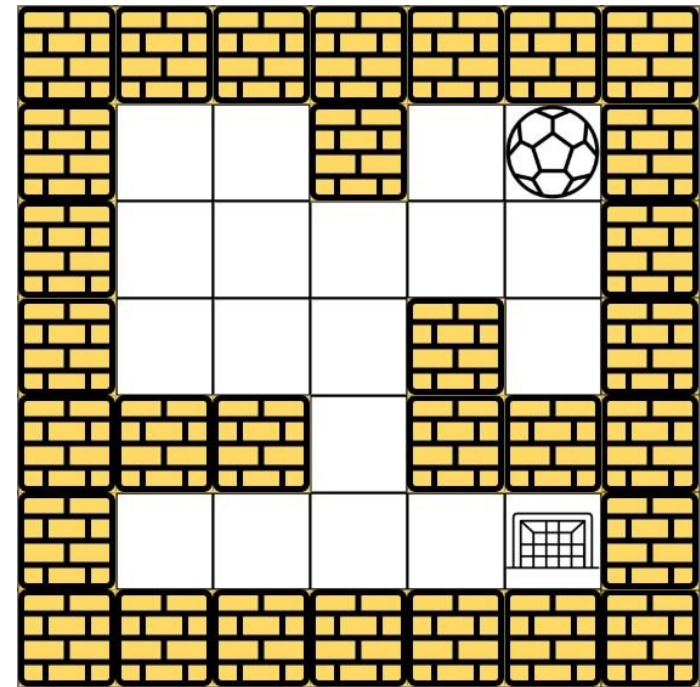
The distance is the number of empty spaces traveled by the ball from the start position (excluded) to the destination (included).

Round 5 · Mid · Medium

p.2423

You may assume that the borders of the maze are all walls (see examples).

Example 1:

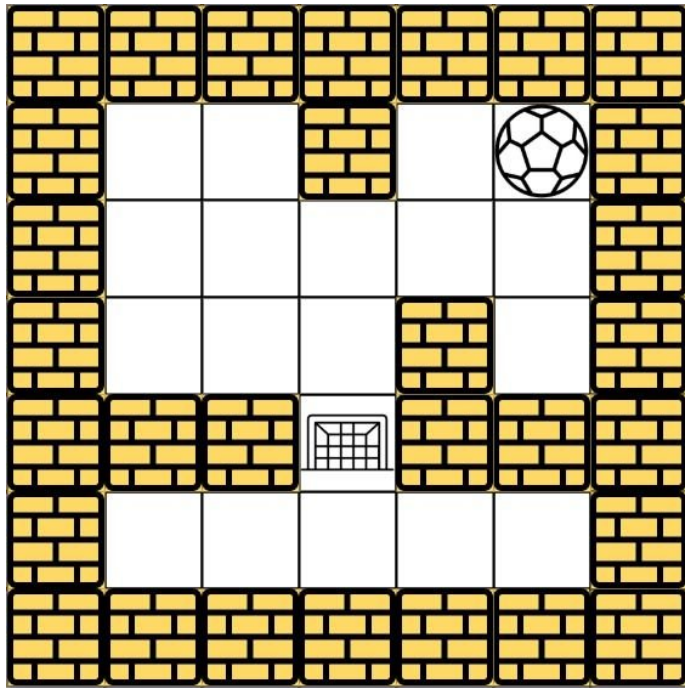


Input: $\text{maze} = [[0,0,1,0,0],[0,0,0,0,0],[0,0,0,1,0],[1,1,0,1,1],[0,0,0,0,0]]$,
 $\text{start} = [0,4]$, $\text{destination} = [4,4]$
 Output: 12
 Explanation: One possible way is : left -> down -> left -> down -> right ->
 down -> right.
 The length of the path is $1 + 1 + 3 + 1 + 2 + 2 + 2 = 12$.

Example 2:

Round 5 · Mid · Medium

p.2424



Input: maze = [[0,0,1,0,0],[0,0,0,0,0],[0,0,0,1,0],[1,1,0,1,1],[0,0,0,0,0]],
start = [0,4], destination = [3,2]

Output: -1

Explanation: There is no way for the ball to stop at the destination. Notice that you can pass through the destination but you cannot stop there.

Example 3:

Input: maze = [[0,0,0,0,0],[1,1,0,0,1],[0,0,0,0,0],[0,1,0,0,1],[0,1,0,0,0]],
start = [4,3], destination = [0,1]

Output: -1

Constraints:

- $m == \text{maze.length}$
- $n == \text{maze}[i].\text{length}$

Round 5 · Mid · Medium

p.2425

- $1 \leq m, n \leq 100$
- $\text{maze}[i][j]$ is 0 or 1.
- $\text{start.length} == 2$
- $\text{destination.length} == 2$
- $0 \leq \text{startrow}, \text{destinationrow} < m$
- $0 \leq \text{startcol}, \text{destinationcol} < n$
- Both the ball and the destination exist in an empty space, and they will not be in the same position initially.
- The maze contains at least 2 empty spaces.

Community Solutions (Python)

• LeetCode

```
class Solution:
    def shortestDistance(
        self, maze: List[List[int]], start: List[int], destination: List[int]
    ) -> int:
        m, n = len(maze), len(maze[0])
        dirs = (-1, 0, 1, 0, -1)
        si, sj = start
        di, dj = destination
        q = deque([(si, sj)])
        dist = [[inf] * n for _ in range(m)]
```

Round 5 · Mid · Medium

p.2426

```

dist[si][sj] = 0
while q:
    i, j = q.popleft()
    for a, b in pairwise(dirs):
        x, y, k = i, j, dist[i][j]
        while 0 <= x + a < m and 0 <= y + b < n and maze[x + a][y + b]
== 0:
            x, y, k = x + a, y + b, k + 1
            if k < dist[x][y]:
                dist[x][y] = k
                q.append((x, y))

return -1 if dist[di][dj] == inf else dist[di][dj]

```

• SimplyLeet

```

import heapq

def shortestDistance(maze, start, destination):
    m, n = len(maze), len(maze[0])
    # Initialize distance matrix with infinity
    dist = [[float('inf')] * n for _ in range(m)]
    dist[start[0]][start[1]] = 0
    # Priority queue (min-heap) storing tuples (distance, row, col)
    heap = [(0, start[0], start[1])]
    # Directions: up, down, left, right

    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    while heap:
        d, row, col = heapq.heappop(heap)
        # If we have reached the destination, return the corresponding distance
        if [row, col] == destination:
            return d
        # Skip if we already found a better path to (row, col)
        if d > dist[row][col]:
            continue

```

```

for dr, dc in directions:
    next_row, next_col, count = row, col, 0
    # Roll the ball until it hits a wall
    while 0 <= next_row + dr < m and 0 <= next_col + dc < n and maze[
ext_row + dr][next_col + dc] == 0:
        next_row += dr
        next_col += dc
        count += 1
    # Check if the new distance is shorter
    if d + count < dist[next_row][next_col]:
        dist[next_row][next_col] = d + count

    heapq.heappush(heap, (dist[next_row][next_col], next_row, next
_col))
return -1

```

• LC.ca

```

class Solution:
    def shortestDistance(
        self, maze: List[List[int]], start: List[int], destination: List[int]
    ) -> int:
        m, n = len(maze), len(maze[0])
        dirs = (-1, 0, 1, 0, -1)
        si, sj = start
        di, dj = destination
        q = deque([(si, sj)])
        dist = [[inf] * n for _ in range(m)]

        dist[si][sj] = 0
        while q:
            i, j = q.popleft()
            for a, b in pairwise(dirs):
                x, y, k = i, j, dist[i][j]
                while 0 <= x + a < m and 0 <= y + b < n and maze[x + a][y + b]
== 0:
                    x, y, k = x + a, y + b, k + 1
                    if k < dist[x][y]:
                        dist[x][y] = k
                        q.append((x, y))

        return -1 if dist[di][dj] == inf else dist[di][dj]

```

◆ Quick Review

Key Insights

- The ball rolls until it hits a wall, so each move is not one step but a continuous travel in a direction.
- The problem can be modeled as finding the shortest path in a weighted graph where nodes are positions in the maze and edge weights are the distance traveled in that roll.
- A priority queue (min-heap) based algorithm, such as Dijkstra's algorithm, is ideal since each roll covers variable distances.
- Always check if the new computed distance for a cell is less than a previously recorded one before pushing it to the heap.

Space & Time

Time Complexity: $O(m * n * \log(m * n))$, where m and n are the maze dimensions (each cell may get processed and queued with a logarithmic cost).

Space Complexity: $O(m * n)$ for maintaining the distance matrix and the priority queue.

Solution

Round 5 · Mid · Medium

p.2429

We solve the problem using a modified version of Dijkstra's algorithm. The maze grid is treated as a graph where each cell is a node and an edge exists between the current cell and the cell where the ball stops after rolling in one direction. For every cell popped from the priority queue, we roll the ball in all four directions until it hits a wall, counting the number of steps taken. If the distance calculated from this roll is shorter than a previously stored distance for the stopping cell, we update the distance and add it to the priority queue. This process continues until we either reach the destination (and return the distance) or exhaust all possibilities (returning -1).

Round 5 · Mid · Medium

p.2430

Shortest Path

#743 Network Delay Time

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Depth-First Search · Breadth-First Search ·
 Graph Theory · Heap (Priority Queue) · Shortest
 Path

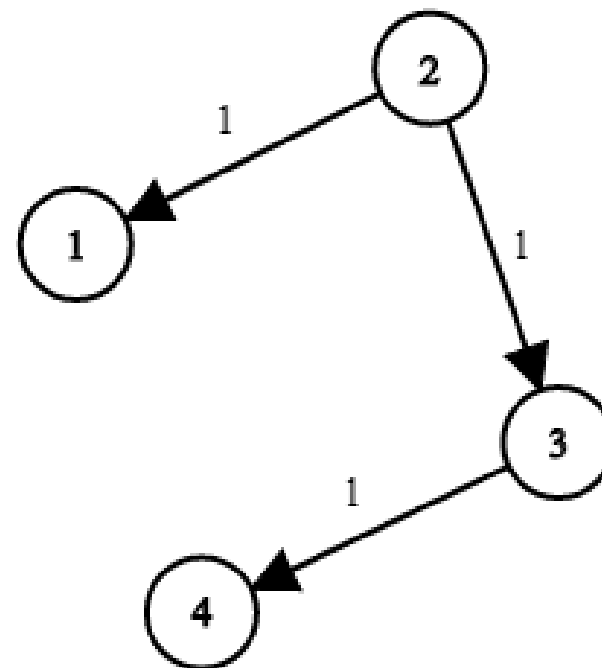
Lists: NeetCode 150

Problem

You are given a network of n nodes, labeled from 1 to n . You are also given times, a list of travel times as directed edges $\text{times}[i] = (u_i, v_i, w_i)$, where u_i is the source node, v_i is the target node, and w_i is the time it takes for a signal to travel from source to target.

We will send a signal from a given node k . Return the minimum time it takes for all the n nodes to receive the signal. If it is impossible for all the n nodes to receive the signal, return -1 .

Example 1:



Input: $\text{times} = [[2,1,1],[2,3,1],[3,4,1]]$, $n = 4$, $k = 2$
 Output: 2

Example 2:

Input: $\text{times} = [[1,2,1]]$, $n = 2$, $k = 1$
 Output: 1

Example 3:

Input: $\text{times} = [[1,2,1]]$, $n = 2$, $k = 2$
 Output: -1

Constraints:

- $1 \leq k \leq n \leq 100$

- $1 \leq \text{times.length} \leq 6000$
- $\text{times}[i].\text{length} == 3$
- $1 \leq u_i, v_i \leq n$
- $u_i \neq v_i$
- $0 \leq w_i \leq 100$
- All the pairs (u_i, v_i) are unique. (i.e., no multiple edges.)

Community Solutions (Python)

• WalkCC

```
class Solution:
    def networkDelayTime(self, times: list[list[int]], n: int, k: int) -> int:
        graph = [[] for _ in range(n)]

        for u, v, w in times:
            graph[u - 1].append((v - 1, w))

        return self._dijkstra(graph, k - 1)

    def _dijkstra(self, graph: list[list[tuple[int, int]]], src: int) -> int:

        dist = [math.inf] * len(graph)

        dist[src] = 0
        minHeap = [(dist[src], src)] # (d, u)

        while minHeap:
            d, u = heapq.heappop(minHeap)
            if d > dist[u]:
                continue
            for v, w in graph[u]:
```

Round 5 · Mid · Medium

p.2433

```
if d + w < dist[v]:
    dist[v] = d + w
    heapq.heappush(minHeap, (dist[v], v))

maxDist = max(dist)
return maxDist if maxDist != math.inf else -1
```

• LeetCode

```
class Solution:
    def networkDelayTime(self, times: List[List[int]], n: int, k: int) -> int:
        g = [[inf] * n for _ in range(n)]
        for u, v, w in times:
            g[u - 1][v - 1] = w
        dist = [inf] * n
        dist[k - 1] = 0
        vis = [False] * n
        for _ in range(n):
            t = -1

            for j in range(n):
                if not vis[j] and (t == -1 or dist[t] > dist[j]):
                    t = j
            vis[t] = True
            for j in range(n):
                dist[j] = min(dist[j], dist[t] + g[t][j])
        ans = max(dist)
        return -1 if ans == inf else ans
```

• SimplyLeet

```
import heapq

def networkDelayTime(times, n, k):
    # Build the graph as an adjacency list
    graph = {i: [] for i in range(1, n + 1)}
    for u, v, w in times:
        graph[u].append((v, w))

    # Initialize distances: infinity for all nodes except the starting node k
    # with distance 0
    dist = {i: float('inf') for i in range(1, n + 1)}
```

Round 5 · Mid · Medium

p.2434

```

dist[k] = 0
# Use a heap as a priority queue storing (distance, node) pairs
heap = [(0, k)]

while heap:
    current_dist, node = heapq.heappop(heap)
    # If we have already found a better path, skip
    if current_dist > dist[node]:
        continue
    # Update the path for neighbors

    for neighbor, weight in graph[node]:
        new_dist = current_dist + weight
        if new_dist < dist[neighbor]:
            dist[neighbor] = new_dist
            heapq.heappush(heap, (new_dist, neighbor))

# Maximum distance among all nodes represents the time taken for the signal
to reach every node
max_time = max(dist.values())
return max_time if max_time != float('inf') else -1

# Example usage:
print(networkDelayTime([[2,1,1],[2,3,1],[3,4,1]], 4, 2))

```

Shortest Path

□ #787 Cheapest Flights Within K Stops

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Dynamic Programming · DFS · BFS · Graph · Heap

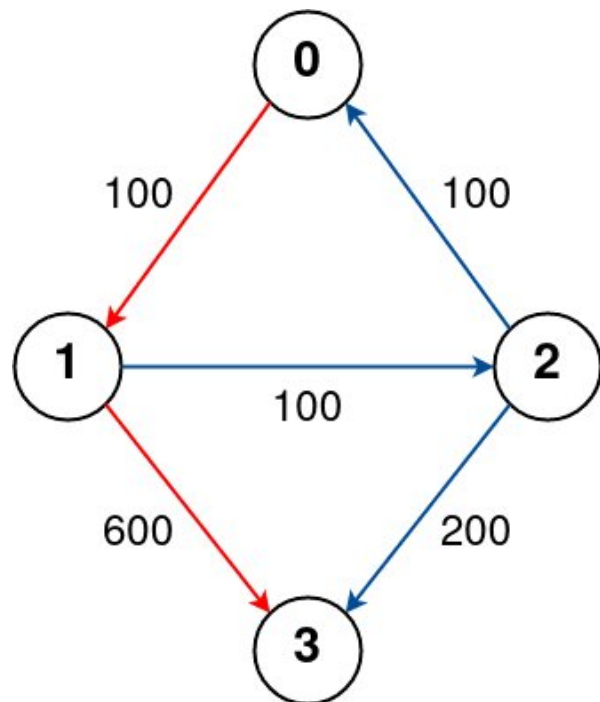
Lists: Grind 169

Problem

There are n cities connected by some number of flights. You are given an array `flights` where `flights[i] = [fromi, toi, pricei]` indicates that there is a flight from city `fromi` to city `toi` with cost `pricei`.

You are also given three integers `src`, `dst`, and `k`, return the cheapest price from `src` to `dst` with at most `k` stops. If there is no such route, return `-1`.

Example 1:

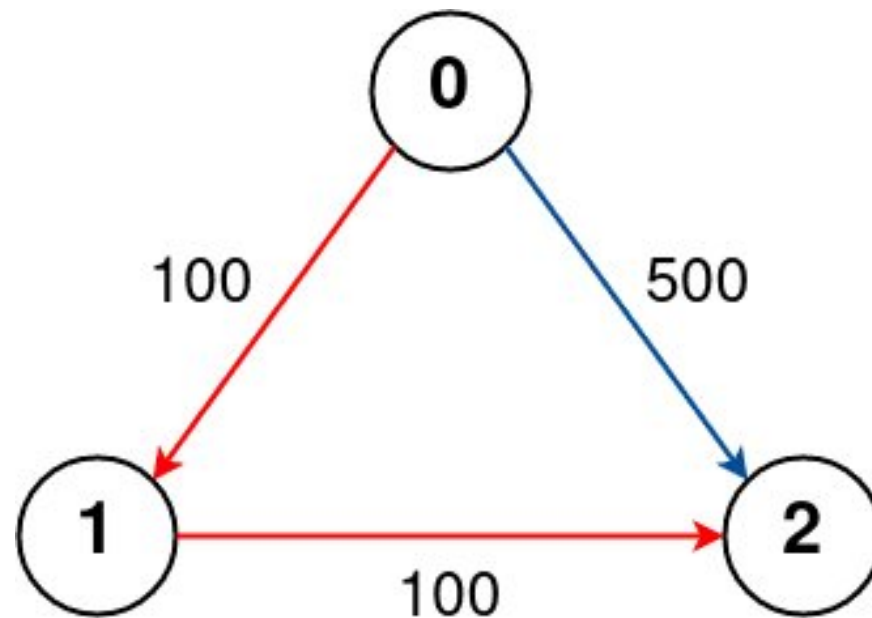


Input: $n = 4$, flights = $[[0,1,100],[1,2,100],[2,0,100],[1,3,600],[2,3,200]]$,
 src = 0, dst = 3, k = 1
 Output: 700
 Explanation:
 The graph is shown above.
 The optimal path with at most 1 stop from city 0 to 3 is marked in red and has
 cost $100 + 600 = 700$.
 Note that the path through cities $[0,1,2,3]$ is cheaper but is invalid because
 it uses 2 stops.

Example 2:

Round 5 · Mid · Medium

p.2437

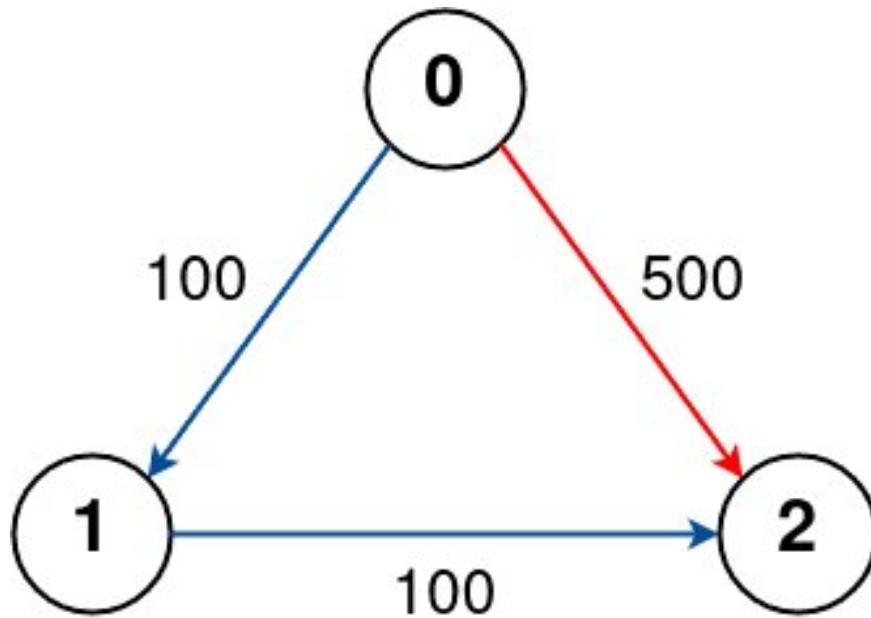


Input: $n = 3$, flights = $[[0,1,100],[1,2,100],[0,2,500]]$, src = 0, dst = 2, k = 1
 Output: 200
 Explanation:
 The graph is shown above.
 The optimal path with at most 1 stop from city 0 to 2 is marked in red and has
 cost $100 + 100 = 200$.

Example 3:

Round 5 · Mid · Medium

p.2438



Input: $n = 3$, $flights = [[0,1,100],[1,2,100],[0,2,500]]$, $src = 0$, $dst = 2$, $k = 0$
 Output: 500
 Explanation:
 The graph is shown above.
 The optimal path with no stops from city 0 to 2 is marked in red and has cost 500.

Constraints:

- $2 \leq n \leq 100$
- $0 \leq flights.length \leq (n * (n - 1) / 2)$
- $flights[i].length == 3$
- $0 \leq from_i, to_i < n$
- $from_i \neq to_i$

Round 5 · Mid · Medium

p.2439

- $1 \leq price_i \leq 104$
- There will not be any multiple flights between two cities.
- $0 \leq src, dst, k < n$
- $src \neq dst$

Community Solutions (Python)

• WalkCC

```

class Solution:
    def findCheapestPrice(
        self,
        n: int,
        flights: list[list[int]],
        src: int,
        dst: int,
        k: int,
    ) -> int:
        graph = [[] for _ in range(n)]

        for u, v, w in flights:
            graph[u].append((v, w))

        return self._dijkstra(graph, src, dst, k)

    def _dijkstra(
        self,
        graph: list[list[tuple[int, int]]],
        src: int,
    
```

Round 5 · Mid · Medium

p.2440


```

    dst: int,
    k: int,
) -> int:
    dist = [[math.inf] * (k + 2) for _ in range(len(graph))]

    dist[src][k + 1] = 0
    minHeap = [(dist[src][k + 1], src, k + 1)] # (d, u, stops)

    while minHeap:
        d, u, stops = heapq.heappop(minHeap)

        if u == dst:
            return d
        if stops == 0 or d > dist[u][stops]:
            continue
        for v, w in graph[u]:
            if d + w < dist[v][stops - 1]:
                dist[v][stops - 1] = d + w
                heapq.heappush(minHeap, (dist[v][stops - 1], v, stops - 1))

    return -1

```

• LeetCode

```

class Solution:
    def findCheapestPrice(
        self, n: int, flights: List[List[int]], src: int, dst: int, k: int
    ) -> int:
        INF = 0x3F3F3F3F
        dist = [INF] * n
        dist[src] = 0
        for _ in range(k + 1):
            backup = dist.copy()
            for f, t, p in flights:
                dist[t] = min(dist[t], backup[f] + p)
        return -1 if dist[dst] == INF else dist[dst]

```

• SimplyLeet

Round 5 · Mid · Medium

p.2441

```

import heapq

def findCheapestPrice(n, flights, src, dst, k):
    # Build graph: each city maps to a list of (neighboring city, flight price)
    graph = {i: [] for i in range(n)}
    for from_city, to_city, price in flights:
        graph[from_city].append((to_city, price))

    # Priority queue: (total_cost, current_city, stops so far)
    heap = [(0, src, 0)]

    # Dictionary to store the best cost to reach a city with a given stop count
    best = {}

    while heap:
        cost, city, stops = heapq.heappop(heap)
        # Return cost if destination is reached
        if city == dst:
            return cost

        # If the current number of stops exceeds k, skip further processing
        if stops > k:
            continue

        # Expand the neighboring flights
        for neighbor, price in graph[city]:
            new_cost = cost + price
            # Only proceed if this route is cheaper or not yet explored for
            # this many stops
            if (neighbor, stops + 1) not in best or new_cost < best[(neighbor,
            stops + 1)]:
                best[(neighbor, stops + 1)] = new_cost

                heapq.heappush(heap, (new_cost, neighbor, stops + 1))

    return -1

# Example usage:
print(findCheapestPrice(4, [[0,1,100],[1,2,100],[2,0,100],[1,3,600],[2,3,200]],
, 0, 3, 1))

```

• LC.ca

Round 5 · Mid · Medium

p.2442

```

class Solution:
    def findCheapestPrice(
        self, n: int, flights: List[List[int]], src: int, dst: int, k: int
    ) -> int:
        INF = 0x3F3F3F3F
        dist = [INF] * n
        dist[src] = 0
        for _ in range(k + 1):
            backup = dist.copy()
            for f, t, p in flights:
                dist[t] = min(dist[t], backup[f] + p)
        return -1 if dist[dst] == INF else dist[dst]

```

◆ Quick Review

Key Insights

- Build the graph as an adjacency list of flights and prices.
- Use a min-heap so the cheapest partial route is always processed first.
- Track states as `(city, flightsUsed)` instead of only `city`, because the stop count matters.
- For each popped state, try all outgoing flights and push the updated cost and flight count.
- Do not expand a state once it has already used more than `k + 1` flights.
- Keep the best known cost for each `(city, flightsUsed)` state so worse duplicate states

Round 5 · Mid · Medium

p.2443

can be skipped.

Space & Time

Time Complexity: $O((V + E) * \log(V * (k + 1)))$ for the heap-based search over `(city, stopsUsed)` states.

Space Complexity: $O(E + V * (k + 1))$ for the graph plus the best-cost tracking table.

Solution

We solve the problem using a Dijkstra-style min-heap search, but the state must include both the city and how many flights have been used so far. We start from `(0, src, 0)` and always pop the cheapest state from the heap. For each state, if we reach `dst`, that cost is the answer. Otherwise, if we still have flights available within the `k + 1` limit, we explore all outgoing edges and push the next states into the heap. To avoid useless work, we store the best cost seen for each `(city, flightsUsed)` pair and only continue when the new state improves that record. This guarantees the cheapest valid route under the stop constraint.

Round 5 · Mid · Medium

p.2444

Shortest Path

#1514 Path with Maximum Probability

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Graph Theory · Heap (Priority Queue) ·
 Shortest Path

Lists: AlgoMaster 600

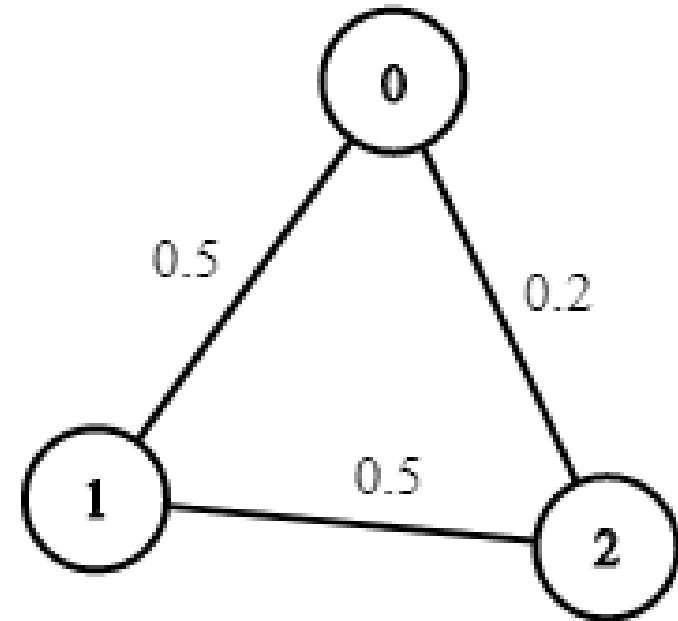
Problem

You are given an undirected weighted graph of n nodes (0-indexed), represented by an edge list where $\text{edges}[i] = [a, b]$ is an undirected edge connecting the nodes a and b with a probability of success of traversing that edge $\text{succProb}[i]$.

Given two nodes start and end , find the path with the maximum probability of success to go from start to end and return its success probability.

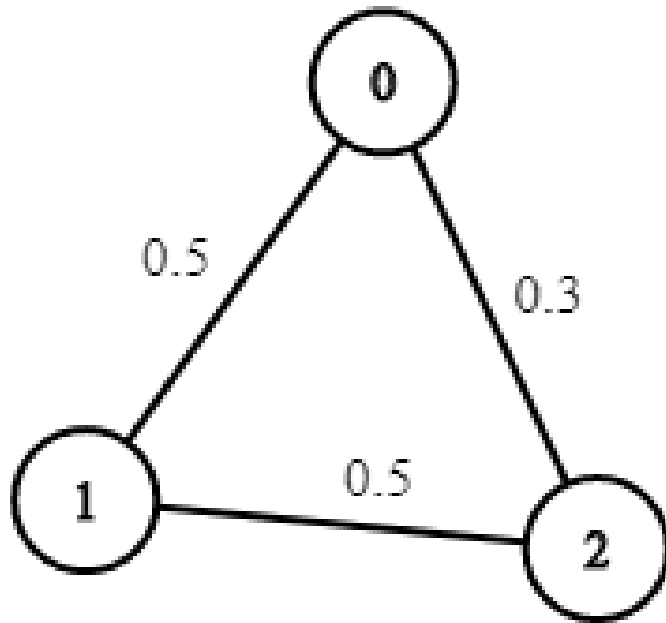
If there is no path from start to end , return 0.
 Your answer will be accepted if it differs from the correct answer by at most $1e-5$.

Example 1:



Input: $n = 3$, $\text{edges} = [[0,1],[1,2],[0,2]]$, $\text{succProb} = [0.5,0.5,0.2]$, $\text{start} = 0$, $\text{end} = 2$
 Output: 0.25000
 Explanation: There are two paths from start to end , one having a probability of success = 0.2 and the other has $0.5 * 0.5 = 0.25$.

Example 2:

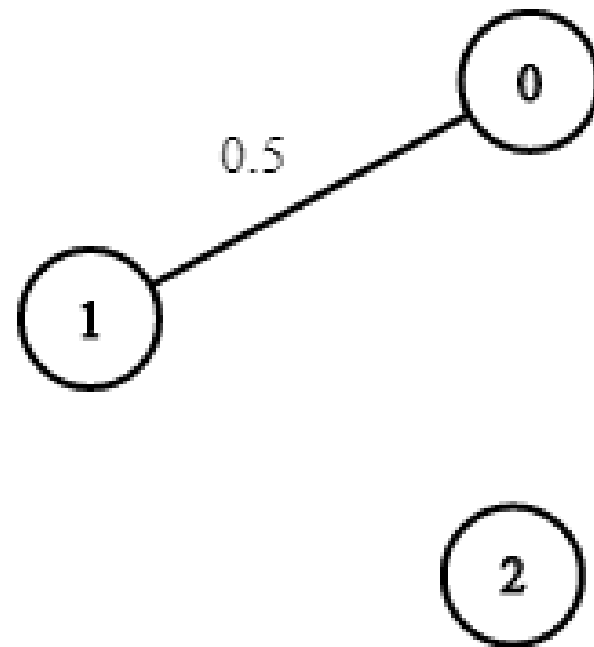


Input: $n = 3$, $edges = [[0,1],[1,2],[0,2]]$, $succProb = [0.5,0.5,0.3]$, $start = 0$, $end = 2$
 Output: 0.30000

Example 3:

Round 5 · Mid · Medium

p.2447



Input: $n = 3$, $edges = [[0,1]]$, $succProb = [0.5]$, $start = 0$, $end = 2$
 Output: 0.00000
 Explanation: There is no path between 0 and 2.

Constraints:

- $2 \leq n \leq 10^4$
- $0 \leq start, end < n$
- $start \neq end$
- $0 \leq a, b < n$
- $a \neq b$

Round 5 · Mid · Medium

p.2448

- $0 \leq \text{succProb.length} == \text{edges.length} \leq 2 \cdot 10^4$
- $0 \leq \text{succProb}[i] \leq 1$
- There is at most one edge between every two nodes.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxProbability(
        self,
        n: int,
        edges: list[list[int]],
        succProb: list[float],
        start: int,
        end: int,
    ) -> float:
        graph = [[] for _ in range(n)] # {a: [(b, probability_ab)]}

        maxHeap = [(-1.0, start)] # (the probability to reach u, u)
        seen = [False] * n

        for i, ((u, v), prob) in enumerate(zip(edges, succProb)):
            graph[u].append((v, prob))
            graph[v].append((u, prob))

        while maxHeap:
            prob, u = heapq.heappop(maxHeap)
            prob *= -1
```

```
if u == end:
    return prob
if seen[u]:
    continue
seen[u] = True
for nextNode, edgeProb in graph[u]:
    if seen[nextNode]:
        continue
    heapq.heappush(maxHeap, (-prob * edgeProb, nextNode))
```

```
return 0
```

• LeetCode

```
class Solution:
    def maxProbability(
        self,
        n: int,
        edges: List[List[int]],
        succProb: List[float],
        start_node: int,
        end_node: int,
    ) -> float:
        g: List[List[Tuple[int, float]]] = [[] for _ in range(n)]

        for (a, b), p in zip(edges, succProb):
            g[a].append((b, p))
            g[b].append((a, p))
        pq = [(-1, start_node)]
        dist = [0] * n
        dist[start_node] = 1
        while pq:
            w, a = heapq.heappop(pq)
            w = -w
            if dist[a] > w:
                continue
            for b, p in g[a]:
                if (t := w * p) > dist[b]:
                    dist[b] = t
                    heapq.heappush(pq, (-t, b))
        return dist[end_node]
```

• SimplyLeet

```
import heapq
from collections import defaultdict

def maxProbability(n, edges, succProb, start, end):
    # Build the graph as an adjacency list
    graph = defaultdict(list)
    for (u, v), prob in zip(edges, succProb):
        graph[u].append((v, prob))
        graph[v].append((u, prob))

    # Create a list to store the maximum probability for reaching each node
    max_probs = [0.0] * n
    max_probs[start] = 1.0

    # Use a max-heap with negative probabilities (since heapq is a min-heap)
    heap = [(-1.0, start)]

    # Process nodes in the order of maximum probability
    while heap:
        # Pop the node with the current highest probability path

        prob, node = heapq.heappop(heap)
        current_prob = -prob

        # If we reached the end node, return the probability
        if node == end:
            return current_prob

        # If we encounter a probability smaller than the recorded one, skip it
        if current_prob < max_probs[node]:
            continue

        # Update the probabilities for neighboring nodes
        for neighbor, edge_prob in graph[node]:
            new_prob = current_prob * edge_prob
            if new_prob > max_probs[neighbor]:
                max_probs[neighbor] = new_prob
                heapq.heappush(heap, (-new_prob, neighbor))

    # Return 0 if end is unreachable
    return 0.0
```

```
# Update the probabilities for neighboring nodes
for neighbor, edge_prob in graph[node]:
    new_prob = current_prob * edge_prob
    if new_prob > max_probs[neighbor]:
        max_probs[neighbor] = new_prob
        heapq.heappush(heap, (-new_prob, neighbor))

# Return 0 if end is unreachable
return 0.0

# Example usage:
if __name__ == "__main__":
    n = 3
    edges = [[0,1],[1,2],[0,2]]
    succProb = [0.5,0.5,0.2]
    start, end = 0, 2
    print(maxProbability(n, edges, succProb, start, end)) # Expected output:
0.25
```

Shortest Path

#1631 Path With Minimum Effort

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Binary Search · Depth-First Search ·
 Breadth-First Search · Union-Find · Heap
 (Priority Queue) · Matrix

Lists: AlgoMaster 600

Problem

You are a hiker preparing for an upcoming hike. You are given heights, a 2D array of size rows x columns, where heights[row][col] represents the height of cell (row, col). You are situated in the top-left cell, (0, 0), and you hope to travel to the bottom-right cell, (rows-1, columns-1) (i.e., 0-indexed). You can move up, down, left, or right, and you wish to find a route that requires the minimum effort.

A route's effort is the maximum absolute difference in heights between two consecutive cells of the route.

Return the minimum effort required to travel from the top-left cell to the bottom-right cell.

Round 5 · Mid · Medium

p.2453

Example 1:

1	2	2
3	8	2
5	3	5

Input: heights = [[1,2,2],[3,8,2],[5,3,5]]

Output: 2

Explanation: The route of [1,3,5,3,5] has a maximum absolute difference of 2 in consecutive cells.

This is better than the route of [1,2,2,2,5], where the maximum absolute difference is 3.

Example 2:

Round 5 · Mid · Medium

p.2454

1	2	3
3	8	4
5	3	5

Input: heights = [[1,2,3],[3,8,4],[5,3,5]]

Output: 1

Explanation: The route of [1,2,3,4,5] has a maximum absolute difference of 1 in consecutive cells, which is better than route [1,3,5,3,5].

Example 3:

Round 5 · Mid · Medium

p.2455

1	2	1	1	1
1	2	1	2	1
1	2	1	2	1
1	2	1	2	1
1	1	1	2	1

Input: heights = [[1,2,1,1,1],[1,2,1,2,1],[1,2,1,2,1],[1,2,1,2,1],[1,1,1,2,1]]

Output: 0

Explanation: This route does not require any effort.

Constraints:

- rows == heights.length
- columns == heights[i].length
- 1 <= rows, columns <= 100
- 1 <= heights[i][j] <= 106

Community Solutions (Python)

Round 5 · Mid · Medium

p.2456

• WalkCC

```
class Solution:
    def minimumEffortPath(self, heights: List[List[int]]) -> int:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(heights)
        n = len(heights[0])
        # diff[i][j] := the maximum absolute difference to reach (i, j)
        diff = [[math.inf] * n for _ in range(m)]
        seen = set()

        minHeap = [(0, 0, 0)] # (d, i, j)

        diff[0][0] = 0

        while minHeap:
            d, i, j = heapq.heappop(minHeap)
            if i == m - 1 and j == n - 1:
                return d
            seen.add((i, j))
            for dx, dy in DIRS:
                x = i + dx
                y = j + dy

                if x < 0 or x == m or y < 0 or y == n:
                    continue
                if (x, y) in seen:
                    continue
                newDiff = abs(heights[i][j] - heights[x][y])
                maxDiff = max(diff[i][j], newDiff)
                if diff[x][y] > maxDiff:
                    diff[x][y] = maxDiff
                    heapq.heappush(minHeap, (diff[x][y], x, y))
```

• LeetDoocs

```
class UnionFind:
    def __init__(self, n):
        self.p = list(range(n))
        self.size = [1] * n

    def find(self, x):
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]

    def union(self, a, b):
        pa, pb = self.find(a), self.find(b)
        if pa == pb:
            return False
        if self.size[pa] > self.size[pb]:
            self.p[pb] = pa
            self.size[pa] += self.size[pb]
        else:
            self.p[pa] = pb
            self.size[pb] += self.size[pa]

        return True

    def connected(self, a, b):
        return self.find(a) == self.find(b)

class Solution:
    def minimumEffortPath(self, heights: List[List[int]]) -> int:
        m, n = len(heights), len(heights[0])
        uf = UnionFind(m * n)
```

```

e = []
dirs = (0, 1, 0)
for i in range(m):
    for j in range(n):
        for a, b in pairwise(dirs):
            x, y = i + a, j + b
            if 0 <= x < m and 0 <= y < n:
                e.append(
                    (abs(heights[i][j] - heights[x][y]), i * n + j, x
* n + y)
                )

e.sort()
for h, a, b in e:
    uf.union(a, b)
    if uf.connected(0, m * n - 1):
        return h
return 0

class Solution:
    def minimumEffortPath(self, heights: List[List[int]]) -> int:
        def check(h: int) -> bool:
            q = deque([(0, 0)])
            vis = {(0, 0)}
            dirs = (-1, 0, 1, 0, -1)
            while q:
                for _ in range(len(q)):
                    i, j = q.popleft()
                    if i == m - 1 and j == n - 1:
                        return True
                for a, b in pairwise(dirs):
                    x, y = i + a, j + b
                    if (
                        0 <= x < m
                        and 0 <= y < n
                        and (x, y) not in vis
                        and abs(heights[i][j] - heights[x][y]) <= h
                    ):
                        q.append((x, y))

```

Round 5 · Mid · Medium

p.2459

```

        vis.add((x, y))
        return False

m, n = len(heights), len(heights[0])
return bisect_left(range(10**6), True, key=check)

class Solution:
    def minimumEffortPath(self, heights: List[List[int]]) -> int:
        m, n = len(heights), len(heights[0])
        dist = [[inf] * n for _ in range(m)]
        dist[0][0] = 0
        dirs = (-1, 0, 1, 0, -1)
        q = [(0, 0, 0)]
        while q:
            t, i, j = heappop(q)
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                if (
                    0 <= x < m
                    and 0 <= y < n
                    and (d := max(t, abs(heights[i][j] - heights[x][y]))) < di
st[x][y]
                ):
                    dist[x][y] = d
                    heappush(q, (d, x, y))
        return int(dist[-1][-1])

```

• SimplyLeet

```

import heapq

def minimumEffortPath(heights):
    rows, cols = len(heights), len(heights[0])
    # Directions: up, down, left, right
    directions = [(1,0), (-1,0), (0,1), (0,-1)]

    # Distance matrix to track minimum effort found for each cell.
    effort = [[float('inf')] * cols for _ in range(rows)]
    effort[0][0] = 0

```

Round 5 · Mid · Medium

p.2460

```
# Priority queue containing (current effort, row, col)
minHeap = [(0, 0, 0)]

while minHeap:
    currentEffort, row, col = heapq.heappop(minHeap)
    if row == rows - 1 and col == cols - 1:
        return currentEffort

    for dr, dc in directions:
        newRow, newCol = row + dr, col + dc
        if 0 <= newRow < rows and 0 <= newCol < cols:
            # The cost to move to the neighbor is the absolute difference
            in heights
            stepEffort = abs(heights[newRow][newCol] - heights[row][col])
            # The effort for the new cell is the maximum effort so far
            along the path.
            newEffort = max(currentEffort, stepEffort)
            if newEffort < effort[newRow][newCol]:
                effort[newRow][newCol] = newEffort
                heapq.heappush(minHeap, (newEffort, newRow, newCol))

return 0

# Example usage:
heights = [[1,2,2],[3,8,2],[5,3,5]]
print(minimumEffortPath(heights))
```

Shortest Path

□ #2976 Minimum Cost to Convert String I **MED**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · String · Graph Theory · Shortest Path
 Lists: AlgoMaster 600

Problem

You are given two 0-indexed strings *source* and *target*, both of length *n* and consisting of lowercase English letters. You are also given two 0-indexed character arrays *original* and *changed*, and an integer array *cost*, where *cost*[*i*] represents the cost of changing the character *original*[*i*] to the character *changed*[*i*].

You start with the string *source*. In one operation, you can pick a character *x* from the string and change it to the character *y* at a cost of *z* if there exists any index *j* such that *cost*[*j*] == *z*, *original*[*j*] == *x*, and *changed*[*j*] == *y*.

Return the minimum cost to convert the string *source* to the string *target* using any number of operations. If it is impossible to convert *source* to *target*, return -1.

Note that there may exist indices i, j such that $\text{original}[j] == \text{original}[i]$ and $\text{changed}[j] == \text{changed}[i]$.

Example 1:

Input: source = "abcd", target = "acbe", original = ["a","b","c","c","e","d"], changed = ["b","c","b","e","b","e"], cost = [2,5,5,1,2,20]
Output: 28
Explanation: To convert the string "abcd" to string "acbe":
- Change value at index 1 from 'b' to 'c' at a cost of 5.
- Change value at index 2 from 'c' to 'e' at a cost of 1.
- Change value at index 2 from 'e' to 'b' at a cost of 2.
- Change value at index 3 from 'd' to 'e' at a cost of 20.
The total cost incurred is $5 + 1 + 2 + 20 = 28$.

Example 2:

Input: source = "aaaa", target = "bbbb", original = ["a","c"], changed = ["c","b"], cost = [1,2]
Output: 12
Explanation: To change the character 'a' to 'b' change the character 'a' to 'c' at a cost of 1, followed by changing the character 'c' to 'b' at a cost of 2, for a total cost of $1 + 2 = 3$. To change all occurrences of 'a' to 'b', a total cost of $3 * 4 = 12$ is incurred.

Example 3:

Input: source = "abcd", target = "abce", original = ["a"], changed = ["e"], cost = [10000]
Output: -1
Explanation: It is impossible to convert source to target because the value at index 3 cannot be changed from 'd' to 'e'.

Constraints:

- $1 \leq \text{source.length} == \text{target.length} \leq 105$
- source, target consist of lowercase English letters.
- $1 \leq \text{cost.length} == \text{original.length} == \text{changed.length} \leq 2000$

- $\text{original}[i], \text{changed}[i]$ are lowercase English letters.
- $1 \leq \text{cost}[i] \leq 106$
- $\text{original}[i] \neq \text{changed}[i]$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minimumCost(
        self,
        source: str,
        target: str,
        original: list[str],
        changed: list[str],
        cost: list[int],
    ) -> int:
        ans = 0

        # dist[u][v] := the minimum distance to change ('a' + u) to ('a' + v)
        dist = [[math.inf] * 26 for _ in range(26)]

        for a, b, c in zip(original, changed, cost):
            u = ord(a) - ord('a')
            v = ord(b) - ord('a')
            dist[u][v] = min(dist[u][v], c)

        for k in range(26):
            for i in range(26):
```

```

    if dist[i][k] < math.inf:
        for j in range(26):
            if dist[k][j] < math.inf:
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])

for s, t in zip(source, target):
    if s == t:
        continue
    u = ord(s) - ord('a')
    v = ord(t) - ord('a')

    if dist[u][v] == math.inf:
        return -1
    ans += dist[u][v]

return ans

```

• LeetCode

```

class Solution:
    def minimumCost(
        self,
        source: str,
        target: str,
        original: List[str],
        changed: List[str],
        cost: List[int],
    ) -> int:
        g = [[inf] * 26 for _ in range(26)]

        for i in range(26):
            g[i][i] = 0

        for x, y, z in zip(original, changed, cost):
            x = ord(x) - ord('a')
            y = ord(y) - ord('a')
            g[x][y] = min(g[x][y], z)

        for k in range(26):
            for i in range(26):
                for j in range(26):
                    g[i][j] = min(g[i][j], g[i][k] + g[k][j])

```

Round 5 · Mid · Medium

p.2465

```

ans = 0
for a, b in zip(source, target):
    if a != b:
        x, y = ord(a) - ord('a'), ord(b) - ord('a')
        if g[x][y] >= inf:
            return -1
        ans += g[x][y]
return ans

```

• SimplyLeet

```

# Python solution with line-by-line comments
def minimumCost(source, target, original, changed, cost):
    # There are 26 letters, so we use an indexed list to represent the
    # transformation costs
    INF = float('inf')
    n_letters = 26
    # Initialize the cost matrix with INF and 0 cost for same letter
    # transformations
    dist = [[INF] * n_letters for _ in range(n_letters)]
    for i in range(n_letters):
        dist[i][i] = 0

```

```

# Update the cost matrix with the provided operations
for op_orig, op_changed, op_cost in zip(original, changed, cost):
    i = ord(op_orig) - ord('a')
    j = ord(op_changed) - ord('a')
    # Ensure we store the minimum cost if multiple operations exist
    dist[i][j] = min(dist[i][j], op_cost)

# Apply Floyd Warshall to compute minimum cost for all pairs of letters
for k in range(n_letters):
    for i in range(n_letters):

```

Round 5 · Mid · Medium

p.2466

```

        for j in range(n_letters):
            if dist[i][k] + dist[k][j] < dist[i][j]:
                dist[i][j] = dist[i][k] + dist[k][j]

# Calculate the total conversion cost from source to target
total_cost = 0
for s_char, t_char in zip(source, target):
    # If the characters are the same, no cost is needed
    if s_char == t_char:
        continue

    i = ord(s_char) - ord('a')
    j = ord(t_char) - ord('a')
    # If the conversion is impossible, return -1
    if dist[i][j] == INF:
        return -1
    total_cost += dist[i][j]
return total_cost

# Example usage:
print(minimumCost("abcd", "acbe", ["a","b","c","c","e","d"], ["b","c","b","e",
"b","e"], [2,5,5,1,2,20]))

print(minimumCost("aaaa", "bbbb", ["a","c"], ["c","b"], [1,2]))
print(minimumCost("abcd", "abce", ["a"], ["e"], [10000]))

```

Shortest Path

#3341 Find Minimum Time to Reach Last Room I MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Graph Theory · Heap (Priority Queue) ·
 Matrix · Shortest Path

Lists: AlgoMaster 600

Problem

There is a dungeon with $n \times m$ rooms arranged as a grid.

You are given a 2D array `moveTime` of size $n \times m$, where `moveTime[i][j]` represents the minimum time in seconds after which the room opens and can be moved to. You start from the room (0, 0) at time $t = 0$ and can move to an adjacent room. Moving between adjacent rooms takes exactly one second.

Return the minimum time to reach the room $(n - 1, m - 1)$.

Two rooms are adjacent if they share a common wall, either horizontally or vertically.

Example 1:

Input: moveTime = [[0,4],[4,4]]

Output: 6

Explanation:

The minimum time required is 6 seconds.

- At time $t == 4$, move from room (0, 0) to room (1, 0) in one second.
- At time $t == 5$, move from room (1, 0) to room (1, 1) in one second.

Example 2:

Input: moveTime = [[0,0,0],[0,0,0]]

Output: 3

Explanation:

The minimum time required is 3 seconds.

- At time $t == 0$, move from room (0, 0) to room (1, 0) in one second.
- At time $t == 1$, move from room (1, 0) to room (1, 1) in one second.
- At time $t == 2$, move from room (1, 1) to room (1, 2) in one second.

Example 3:

Input: moveTime = [[0,1],[1,2]]

Output: 3

Constraints:

- $2 \leq n \leq \text{moveTime.length} \leq 50$
- $2 \leq m \leq \text{moveTime}[i].\text{length} \leq 50$
- $0 \leq \text{moveTime}[i][j] \leq 109$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minTimeToReach(self, moveTime: list[list[int]]) -> int:
        return self._dijkstra(moveTime,
                               (0, 0), (len(moveTime) - 1, len(moveTime[0]) - 1))

    def _dijkstra(
        self,
        moveTime: list[list[int]],
        src: tuple[int, int],
        dst: tuple[int, int]
    ) -> int:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(moveTime)
        n = len(moveTime[0])
        dist = [[math.inf] * n for _ in range(m)]

        dist[0][0] = 0
        minHeap = [(0, src)] # (d, u)

        while minHeap:
            d, u = heapq.heappop(minHeap)
            if u == dst:
                return d
            i, j = u
            if d > dist[i][j]:
                continue
            for dx, dy in DIRS:
                x = i + dx
                y = j + dy
                if x < 0 or x == m or y < 0 or y == n:
```

```

        continue
    newDist = max(moveTime[x][y], d) + 1
    if newDist < dist[x][y]:
        dist[x][y] = newDist
        heapq.heappush(minHeap, (newDist, (x, y)))

return -1

```

• LeetCode

```

class Solution:
    def minTimeToReach(self, moveTime: List[List[int]]) -> int:
        n, m = len(moveTime), len(moveTime[0])
        dist = [[inf] * m for _ in range(n)]
        dist[0][0] = 0
        pq = [(0, 0, 0)]
        dirs = (-1, 0, 1, 0, -1)
        while 1:
            d, i, j = heapq.heappop(pq)
            if i == n - 1 and j == m - 1:
                return d
            if d > dist[i][j]:
                continue
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                if 0 <= x < n and 0 <= y < m:
                    t = max(moveTime[x][y], dist[i][j]) + 1
                    if dist[x][y] > t:
                        dist[x][y] = t
                        heapq.heappush(pq, (t, x, y))

```

• SimplyLeet

Round 5 · Mid · Medium

p.2471

```

import heapq

def minTimeToReachLastRoom(moveTime):
    n = len(moveTime)
    m = len(moveTime[0])

    # Directions for adjacent moves: up, right, down, left
    directions = [(-1, 0), (0, 1), (1, 0), (0, -1)]

    # Initialize distance matrix with infinity and set start (0,0)
    dist = [[float('inf')] * m for _ in range(n)]
    dist[0][0] = 0

    # Priority queue holding (current time, row, column)
    heap = [(0, 0, 0)]

    while heap:
        current_time, row, col = heapq.heappop(heap)

        # If destination is reached, return the current time
        if row == n - 1 and col == m - 1:
            return current_time

        # Skip if a better route to this cell is already known
        if current_time > dist[row][col]:
            continue

        # Explore adjacent cells
        for dr, dc in directions:
            new_row, new_col = row + dr, col + dc

```

Round 5 · Mid · Medium

p.2472


```

    if 0 <= new_row < n and 0 <= new_col < m:
        # Calculate arrival time: moving takes 1 second and may require
        waiting
        next_time = max(current_time + 1, moveTime[new_row][new_col])
        if next_time < dist[new_row][new_col]:
            dist[new_row][new_col] = next_time
            heapq.heappush(heap, (next_time, new_row, new_col))

    return -1

# Example test case:
print(minTimeToReachLastRoom([[0,4],[4,4]])) # Expected output: 6

```

Shortest Path

#3650 Minimum Cost Path with Edge Reversals

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Graph Theory · Heap (Priority Queue) · Shortest Path

Lists: AlgoMaster 600

Problem

You are given a directed, weighted graph with n nodes labeled from 0 to $n - 1$, and an array edges where edges[i] = [ui, vi, wi] represents a directed edge from node ui to node vi with cost wi.

Each node ui has a switch that can be used at most once: when you arrive at ui and have not yet used its switch, you may activate it on one of its incoming edges vi → ui reverse that edge to ui → vi and immediately traverse it.

The reversal is only valid for that single move, and using a reversed edge costs $2 * wi$.

Return the minimum total cost to travel from node 0 to node $n - 1$. If it is not possible, return

-1.

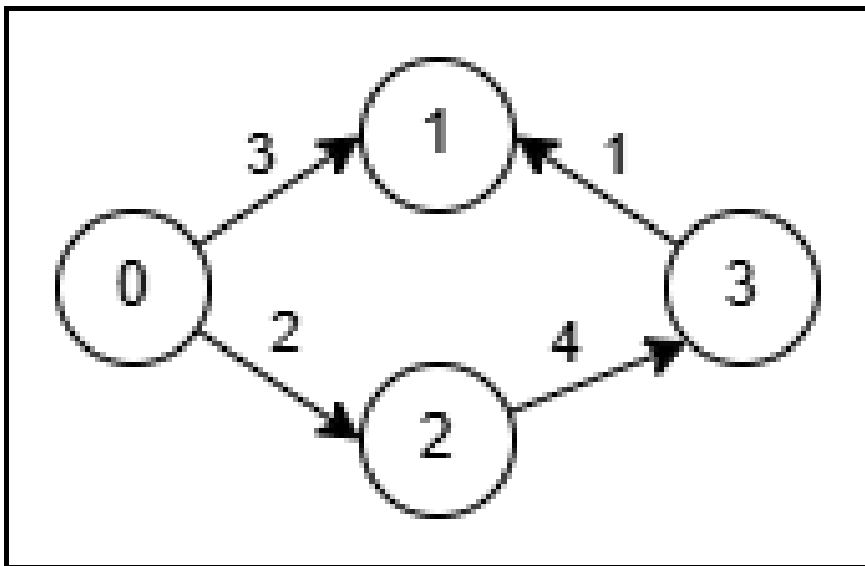
Example 1:

Input: $n = 4$, $edges =$

$[[0,1,3],[3,1,1],[2,3,4],[0,2,2]]$

Output: 5

Explanation:



- Use the path $0 \rightarrow 1$ (cost 3).
- At node 1 reverse the original edge $3 \rightarrow 1$ into $1 \rightarrow 3$ and traverse it at cost $2 * 1 = 2$.
- Total cost is $3 + 2 = 5$.

Example 2:

Round 5 · Mid · Medium

p.2475

Input: $n = 4$, $edges =$

$[[0,2,1],[2,1,1],[1,3,1],[2,3,3]]$

Output: 3

Explanation:

- No reversal is needed. Take the path $0 \rightarrow 2$ (cost 1), then $2 \rightarrow 1$ (cost 1), then $1 \rightarrow 3$ (cost 1).
- Total cost is $1 + 1 + 1 = 3$.

Constraints:

- $2 \leq n \leq 5 * 10^4$
- $1 \leq edges.length \leq 10^5$
- $edges[i] = [ui, vi, wi]$
- $0 \leq ui, vi \leq n - 1$
- $1 \leq wi \leq 1000$

Community Solutions (Python)

• LeetCode

```

class Solution:
    def minCost(self, n: int, edges: List[List[int]]) -> int:
        g = [[] for _ in range(n)]
        for u, v, w in edges:
            g[u].append((v, w))
            g[v].append((u, w * 2))
        pq = [(0, 0)]
        dist = [inf] * n
        dist[0] = 0
        while pq:

```

Round 5 · Mid · Medium

p.2476

```

d, u = heappop(pq)
if d > dist[u]:
    continue
if u == n - 1:
    return d
for v, w in g[u]:
    nd = d + w
    if nd < dist[v]:
        dist[v] = nd
        heappush(pq, (nd, v))

return -1

```

Round 6 · Mid · Hard

Round 6

Mid Priority · Hard

50 questions · 17 patterns

- ☐ ● **LinkedList In-place Reversal #25** ↗
- ☐ ● **Monotonic Stack #84 #321 #1944** ↗
- ☐ ● **Monotonic Queue #239 #862 #1425 #1499** ↗
- ☐ ● **Recursion #273 #761** ↗
- ☐ ● **Merge Sort #327 #493** ↗
- ☐ ● **Backtracking #37 #51 #301 #980** ↗
- ☐ ● **Tries #212 #336 #425 #472 #1032** ↗
- ☐ ● **Heaps #1383** ↗
- ☐ ● **Two Heaps #295 #480 #502** ↗
- ☐ ● **Intervals #2402** ↗
- ☐ ● **K-Way Merge #23 #632** ↗
- ☐ ● **Data Structure Design #432 #460 #588 #642** ↗
- ☐ ● **#715 #716 #895 #2296** ↗

Round 5 · Mid · Medium

p.2477

Round 5 · Mid · Medium

p.2478

Greedy #135 #757 #857 #871
Topological Sort #269 #1203
Union Find #305 #924
Minimum Spanning Tree #1168 #1489 #3600
Shortest Path #778 #1368 #2203

LinkedList In-place Reversal

Round 6 · Mid · Hard · 1 question

Round 6 · Mid · Hard

p.2479

Round 6 · Mid · Hard

p.2480

LinkedList In-place Reversal

#25 Reverse Nodes in k-Group

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Linked List · Recursion
 Lists: Grind 169

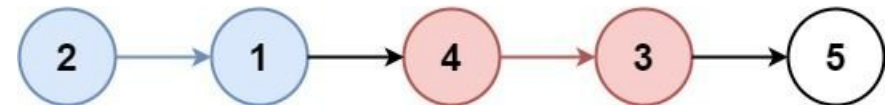
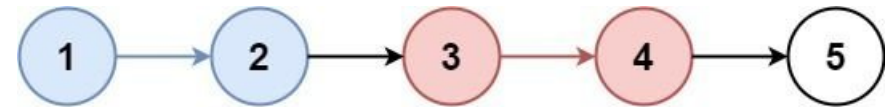
Problem

Given the head of a linked list, reverse the nodes of the list k at a time, and return the modified list.

k is a positive integer and is less than or equal to the length of the linked list. If the number of nodes is not a multiple of k then left-out nodes, in the end, should remain as it is.

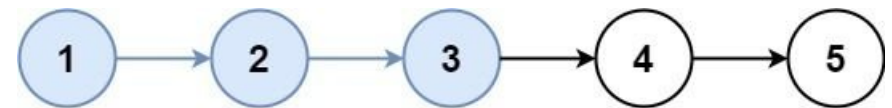
You may not alter the values in the list's nodes, only nodes themselves may be changed.

Example 1:



Input: head = [1,2,3,4,5], k = 2
 Output: [2,1,4,3,5]

Example 2:



Input: head = [1,2,3,4,5], k = 3
 Output: [3,2,1,4,5]

Constraints:

- The number of nodes in the list is n .
- $1 \leq k \leq n \leq 5000$
- $0 \leq \text{Node.val} \leq 1000$

Follow-up: Can you solve the problem in $O(1)$ extra memory space?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def reverseKGroup(self, head: ListNode | None, k: int) -> ListNode | None:
        if not head:
            return None

        tail = head

        for _ in range(k):
            # There are less than k nodes in the list, do nothing.
            if not tail:
                return head
            tail = tail.next

        newHead = self._reverse(head, tail)
        head.next = self.reverseKGroup(tail, k)
        return newHead

    def _reverse(
        self,
        head: ListNode | None,
        tail: ListNode | None,
    ) -> ListNode | None:
        """Reverses [head, tail)."""
        prev = None
        curr = head
        while curr != tail:
            next = curr.next
            curr.next = prev
            prev = curr
            curr = next

        return prev
```

Round 6 · Mid · Hard

p.2483

```
class Solution:
    def reverseKGroup(self, head: ListNode, k: int) -> ListNode:
        if not head or k == 1:
            return head

        def getLength(head: ListNode) -> int:
            length = 0
            while head:
                length += 1
                head = head.next

            return length

        length = getLength(head)
        dummy = ListNode(0, head)
        prev = dummy
        curr = head

        for _ in range(length // k):
            for _ in range(k - 1):
                next = curr.next

                curr.next = next.next
                next.next = prev.next
                prev.next = next
                prev = curr
                curr = curr.next

        return dummy.next
```

• LeetDoocs

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def reverseKGroup(self, head: Optional[ListNode], k: int) -> Optional[ListNode]:
        def reverse(head: Optional[ListNode]) -> Optional[ListNode]:
            dummy = ListNode()
            cur = head
```

Round 6 · Mid · Hard

p.2484

```

while cur:
    nxt = cur.next
    cur.next = dummy.next
    dummy.next = cur
    cur = nxt
return dummy.next

```

```

dummy = pre = ListNode(next=head)
while pre:
    cur = pre

```

```

for _ in range(k):
    cur = cur.next
    if cur is None:
        return dummy.next
node = pre.next
nxt = cur.next
cur.next = None
pre.next = reverse(node)
node.next = nxt
pre = node

```

```

return dummy.next

```

• SimplyLeet

```

# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def reverseKGroup(self, head: ListNode, k: int) -> ListNode:
        # Create a dummy node to simplify edge cases
        dummy = ListNode(0)

```

Round 6 · Mid · Hard

p.2485

```

dummy.next = head
group_prev = dummy

while True:
    # Find the kth node from group_prev
    kth = self.getKthNode(group_prev, k)
    if not kth:
        break
    group_next = kth.next

```

```

# Reverse the group starting at group_prev.next up to kth
prev = kth.next
curr = group_prev.next
while curr != group_next:
    temp = curr.next # Store next node
    curr.next = prev # Reverse pointer
    prev = curr # Move prev forward
    curr = temp # Move curr forward

# Reconnect the reversed group with the previous part

```

```

temp = group_prev.next # temp is the start of the original group,
                        # becomes the new tail
group_prev.next = kth # Connect previous part to the kth node
                        # (new head of reversed group)
group_prev = temp # Move group_prev to the end of the
                  # reversed group

```

```

return dummy.next

```

```

# Helper function to get the kth node from the current node
def getKthNode(self, curr, k):
    while curr and k > 0:
        curr = curr.next

```

```

k -= 1
return curr

```

• LC.ca

Round 6 · Mid · Hard

p.2486

```
# Definition for singly-linked list.
# class ListNode:
# def __init__(self, val=0, next=None):
# self.val = val
# self.next = next
class Solution:
    def reverseKGroup(self, head: ListNode, k: int) -> ListNode:
        '''
        for reverse 1->2->3->4->5, process is like
        1->None, 2->3->4->5

        2->1->None, 3->4->5
        3->2->1->None, 4->5
        4->3->2->1->None, 5
        5->4->3->2->1->None, None
        '''
        def reverseList(head):
            pre, p = None, head
            while p:
                pnext = p.next
                p.next = pre
                pre = p
                p = pnext
            return pre

        dummy = ListNode(next=head)
        pre = cur = dummy
        while cur.next:
            for _ in range(k):
                cur = cur.next
            if cur is None:
                return dummy.next
```

Round 6 · Mid · Hard

p.2487

```
                return dummy.next
            t = cur.next
            cur.next = None # cut from next k-group, so to reverseList() for
current k-group
            start = pre.next
            pre.next = reverseList(start)
            start.next = t # so now 'start' is the last node of k-group
            pre = cur = start # same as the reset dummy before while loop
            return dummy.next

#####

# Definition for singly-linked list.
# class ListNode(object):
# def __init__(self, x):
# self.val = x
# self.next = None

class Solution(object):
    def reverseKGroup(self, head, k):
        '''
        :type head: ListNode
        :type k: int
        :rtype: ListNode
        '''

        def reverseList(head, k):
            pre = None
            cur = head
            while cur and k > 0:
                tmp = cur.next
```

Round 6 · Mid · Hard

p.2488


```

    cur.next = pre
    pre = cur
    cur = tmp
    k -= 1
    head.next = cur
    return cur, pre

length = 0
p = head
while p:

    length += 1
    p = p.next
    if length < k:
        return head
    step = length / k
    ret = None
    pre = None
    p = head
    while p and step:
        next, newHead = reverseList(p, k)

        if ret is None:
            ret = newHead
        if pre:
            pre.next = newHead
        pre = p
        p = next
        step -= 1
    return ret

```

◆ Quick Review

Key Insights

- Use a dummy node to simplify edge cases.
- Identify groups of k nodes using a helper function.

Round 6 · Mid · Hard

p.2489

- Reverse the nodes in each group using pointer manipulation.
- If a remaining group has fewer than k nodes, leave it unchanged.
- The reversal is done in-place with O(1) extra space.

Space & Time

Time Complexity: O(n) where n is the number of nodes (each node is processed a constant number of times)

Space Complexity: O(1) (only constant extra space is used)

Solution

The solution involves iterating through the linked list by identifying groups of k consecutive nodes. For each group:

- Find the kth node from the current starting point. If it does not exist, the remainder of the list is not reversed.
- Reverse the nodes in the identified group in-place by adjusting the next pointers.
- Reconnect the reversed group with the previous part of the list and update pointers to move to the next group. The approach leverages a dummy node for easier

Round 6 · Mid · Hard

p.2490

handling of head modifications and uses a helper function to fetch the kth node. The in-place reversal ensures that the extra memory usage is constant.

Round 6 · Mid · Hard

p.2491

Monotonic Stack

Round 6 · Mid · Hard · 3 questions

Round 6 · Mid · Hard

p.2492

Monotonic Stack

#84 Largest Rectangle in Histogram

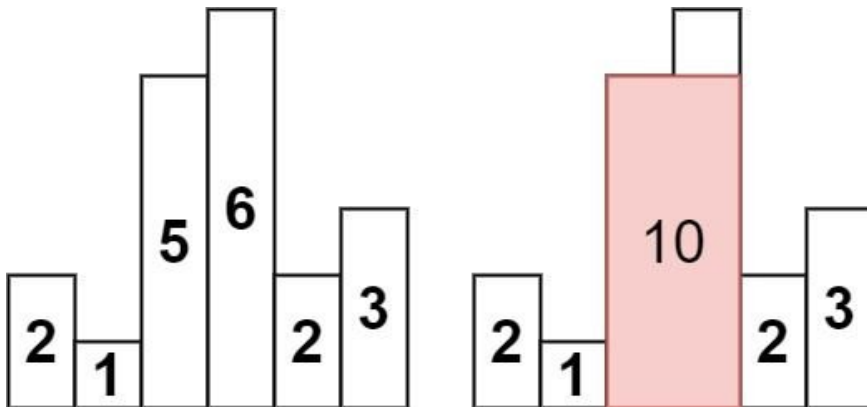
HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Stack · Monotonic Stack
 Lists: Grind 169

Problem

Given an array of integers heights representing the histogram's bar height where the width of each bar is 1, return the area of the largest rectangle in the histogram.

Example 1:



Round 6 · Mid · Hard

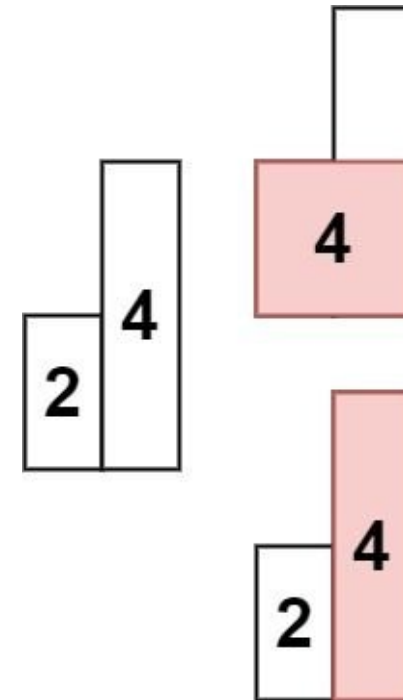
p.2493

Input: heights = [2,1,5,6,2,3]

Output: 10

Explanation: The above is a histogram where width of each bar is 1.
 The largest rectangle is shown in the red area, which has an area = 10 units.

Example 2:



Input: heights = [2,4]

Output: 4

Constraints:

- $1 \leq \text{heights.length} \leq 105$
- $0 \leq \text{heights}[i] \leq 104$

Round 6 · Mid · Hard

p.2494

Community Solutions (Python)

• WalkCC

```
class Solution:
    def largestRectangleArea(self, heights: List[int]) -> int:
        ans = 0
        stack = []

        for i in range(len(heights) + 1):
            while stack and (i == len(heights) or heights[stack[-1]] > heights[i]):
                h = heights[stack.pop()]
                w = i - stack[-1] - 1 if stack else i
                ans = max(ans, h * w)

            stack.append(i)

        return ans
```

• LeetDoocs

```
class Solution:
    def largestRectangleArea(self, heights: List[int]) -> int:
        n = len(heights)
        stk = []
        left = [-1] * n
        right = [n] * n
        for i, h in enumerate(heights):
            while stk and heights[stk[-1]] >= h:
                right[stk[-1]] = i
                stk.pop()

            if stk:
                left[i] = stk[-1]
            stk.append(i)
        return max(h * (right[i] - left[i] - 1) for i, h in enumerate(heights))
```

• SimplyLeet

Round 6 · Mid · Hard

p.2495

Python solution

```
def largestRectangleArea(heights):
    # Append a zero to flush out the remaining bars in the stack
    heights.append(0)
    stack = [] # Stack to store indices of bars
    max_area = 0

    # Iterate over each index and bar height
    for i, h in enumerate(heights):

        # Process bars that are taller than the current bar
        while stack and heights[stack[-1]] > h:
            height = heights[stack.pop()] # Height of the bar at the index
            # Calculate width: if stack is empty, width is the current index;
            # otherwise, it's the difference
            width = i if not stack else i - stack[-1] - 1
            max_area = max(max_area, height * width)
            stack.append(i)

    return max_area

# Example usage:
if __name__ == "__main__":
    histogram = [2, 1, 5, 6, 2, 3]
    print(largestRectangleArea(histogram))
```

• LC.ca

```
'''
    0000000000 h 00000000000000000000 00(<) h 000 left_i, right_i

    0000000000 h * (right_i - left_i - 1)0000000000
    '''
class Solution:
    def largestRectangleArea(self, heights: List[int]) -> int:
        n = len(heights)
        stk = []
        left = [-1] * n
```

Round 6 · Mid · Hard

p.2496

```

right = [n] * n
for i, h in enumerate(heights):
    while stk and heights[stk[-1]] >= h:
        right[stk[-1]] = i
        stk.pop()
    if stk:
        left[i] = stk[-1] # same as below: stk[-1] in 'i - stack[-1] -
1'

    stk.append(i)

# valid for one element input [3]

return max(h * (right[i] - left[i] - 1) for i, h in enumerate(heights)
)

#####

class Solution:
    def largestRectangleArea(self, heights: List[int]) -> int:
        if not heights:
            return 0

heights.append(-1) # make for loop running
stack = []
ans = 0
for i in range(0, len(heights)):
    while stack and heights[i] < heights[stack[-1]]:
        currentLowestBarIndex = stack.pop()
        h = heights[currentLowestBarIndex]
        # stack[-1] is after pop(). why not i-currentLowestBarIndex?
        # because not working for tie case in the stack,
        # eg. for input=[4,2,0,3,2,5], expected=6 but output=5

```

```

# 000000000000 00(<) h 000 left_i, right_i
# 0000000000 00 h 0000000000 right-left+1
# 000000 00 h00000right-left+1 00 0000 000000index000000+1 -2 0 -1
w = i - stack[-1] - 1 if stack else i
ans = max(ans, h * w)
stack.append(i)

heights.pop() # restore, pop -1
return ans

```

◆ Quick Review

Key Insights

- Each bar can potentially extend left and right as long as the neighboring bars are not shorter.
- A monotonic (increasing) stack can be used to efficiently track indices of bars.
- When encountering a bar lower than the one on the top of the stack, pop the stack and calculate the area of the rectangle formed with the popped bar height.
- A dummy bar (of height 0) is appended at the end to ensure all bars are processed.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Solution

The solution employs a monotonic stack to traverse the histogram once. For each bar, while the current bar's height is less than the height at the top index of the stack, it pops from the stack and calculates the area using the popped height as the smallest bar. The width for the rectangle is determined by the difference between the current index and the index from the stack (or the current index if the stack is empty). This effectively finds the maximum rectangular area possible. A sentinel value (0) is added at the end of the histogram to flush any remaining bars from the stack.

Round 6 · Mid · Hard

p.2499

Monotonic Stack

□ #321 Create Maximum Number

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Stack · Greedy ·
Monotonic Stack

Lists: AlgoMaster 600

Problem

You are given two integer arrays `nums1` and `nums2` of lengths `m` and `n` respectively. `nums1` and `nums2` represent the digits of two numbers. You are also given an integer `k`.

Create the maximum number of length $k \leq m + n$ from digits of the two numbers. The relative order of the digits from the same array must be preserved.

Return an array of the `k` digits representing the answer.

Example 1:

Input: `nums1 = [3,4,6,5]`, `nums2 = [9,1,2,5,8,3]`, `k = 5`
Output: `[9,8,6,5,3]`

Example 2:

Round 6 · Mid · Hard

p.2500

Input: nums1 = [6,7], nums2 = [6,0,4], k = 5
Output: [6,7,6,0,4]

Example 3:

Input: nums1 = [3,9], nums2 = [8,9], k = 3
Output: [9,8,9]

Constraints:

- $m == \text{nums1.length}$
- $n == \text{nums2.length}$
- $1 \leq m, n \leq 500$
- $0 \leq \text{nums1}[i], \text{nums2}[i] \leq 9$
- $1 \leq k \leq m + n$
- nums1 and nums2 do not have leading zeros.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxNumber(self, nums1: list[int], nums2: list[int], k: int) -> list[int]:
        :
        def maxArray(nums: list[int], k: int) -> list[int]:
            res = []
            toTop = len(nums) - k
            for num in nums:
                while res and res[-1] < num and toTop > 0:
                    res.pop()
                    toTop -= 1
                res.append(num)
```

```
return res[:k]

def merge(nums1: list[int], nums2: list[int]) -> list[int]:
    return [max(nums1, nums2).pop(0) for _ in nums1 + nums2]

return max(merge(maxArray(nums1, i), maxArray(nums2, k - i))
            for i in range(k + 1)
            if i <= len(nums1) and k - i <= len(nums2))
```

• LeetDoocs

```
class Solution:
    def maxNumber(self, nums1: List[int], nums2: List[int], k: int) -> List[int]:
        def f(nums: List[int], k: int) -> List[int]:
            n = len(nums)
            stk = [0] * k
            top = -1
            remain = n - k
            for x in nums:
                while top >= 0 and stk[top] < x and remain > 0:
                    top -= 1
                    remain -= 1
                if top + 1 < k:
                    top += 1
                    stk[top] = x
                else:
                    remain -= 1
            return stk

        def compare(nums1: List[int], nums2: List[int], i: int, j: int) -> bool:
            if i >= len(nums1):
```

```

        return False
    if j >= len(nums2):
        return True
    if nums1[i] > nums2[j]:
        return True
    if nums1[i] < nums2[j]:
        return False
    return compare(nums1, nums2, i + 1, j + 1)

def merge(nums1: List[int], nums2: List[int]) -> List[int]:

    m, n = len(nums1), len(nums2)
    i = j = 0
    ans = [0] * (m + n)
    for k in range(m + n):
        if compare(nums1, nums2, i, j):
            ans[k] = nums1[i]
            i += 1
        else:
            ans[k] = nums2[j]
            j += 1

    return ans

    m, n = len(nums1), len(nums2)
    l, r = max(0, k - n), min(k, m)
    ans = [0] * k
    for x in range(l, r + 1):
        arr1 = f(nums1, x)
        arr2 = f(nums2, k - x)
        arr = merge(arr1, arr2)
        if ans < arr:

            ans = arr
    return ans

```

• SimplyLeet

Round 6 · Mid · Hard

p.2503

```

def maxNumber(nums1, nums2, k):
    # Helper function to get the maximum subsequence of given length t from
    array nums
    def getMaxSubsequence(nums, t):
        drop = len(nums) - t # Number of elements that can be dropped
        stack = []
        for num in nums:
            # Remove smaller digits when possible to achieve a larger leading
            number

            while drop and stack and stack[-1] < num:
                stack.pop()
                drop -= 1

            stack.append(num)
        # Limit the result to length t
        return stack[:t]

    # Helper function to merge two subsequences into the maximum possible
    number
    def merge(seq1, seq2):
        merged = []
        while seq1 or seq2:
            # Lexicographical comparison of remaining parts
            if seq1 > seq2:

                merged.append(seq1[0])
                seq1 = seq1[1:]
            else:
                merged.append(seq2[0])
                seq2 = seq2[1:]
        return merged

    max_combo = []
    m, n = len(nums1), len(nums2)
    # Iterate over all possible splits: select i digits from nums1 and k-i from
    nums2

```

Round 6 · Mid · Hard

p.2504


```

for i in range(max(0, k - n), min(k, m) + 1):
    subseq1 = getMaxSubsequence(nums1, i)
    subseq2 = getMaxSubsequence(nums2, k - i)
    candidate = merge(subseq1, subseq2)
    if candidate > max_combo:
        max_combo = candidate
return max_combo

```

```

# Example usage:
print(maxNumber([3,4,6,5], [9,1,2,5,8,3], 5)) # Expected Output: [9,8,6,5,3]

```

Monotonic Stack

#1944 Number of Visible People in a Queue

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Stack · Monotonic Stack
 Lists: AlgoMaster 600

Problem

There are n people standing in a queue, and they numbered from 0 to $n - 1$ in left to right order. You are given an array heights of distinct integers where heights[i] represents the height of the i th person.

A person can see another person to their right in the queue if everybody in between is shorter than both of them. More formally, the i th person can see the j th person if $i < j$ and $\min(\text{heights}[i], \text{heights}[j]) > \max(\text{heights}[i+1], \text{heights}[i+2], \dots, \text{heights}[j-1])$.

Return an array answer of length n where answer[i] is the number of people the i th person can see to their right in the queue.

Example 1:



Input: heights = [10,6,8,5,11,9]

Output: [3,1,2,1,1,0]

Explanation:

Person 0 can see person 1, 2, and 4.

Person 1 can see person 2.

Person 2 can see person 3 and 4.

Person 3 can see person 4.

Person 4 can see person 5.

Example 2:

Input: heights = [5,1,2,3,10]

Output: [4,1,1,1,0]

Constraints:

- $n == \text{heights.length}$
- $1 \leq n \leq 105$
- $1 \leq \text{heights}[i] \leq 105$
- All the values of heights are unique.

Community Solutions (Python)

- WalkCC

Round 6 · Mid · Hard

p.2507

```
class Solution:
    def canSeePersonsCount(self, heights: list[int]) -> list[int]:
        ans = [0] * len(heights)
        stack = []

        for i, height in enumerate(heights):
            while stack and heights[stack[-1]] <= height:
                ans[stack.pop()] += 1
            if stack:
                ans[stack[-1]] += 1

            stack.append(i)

        return ans
```

• LeetCode

```
class Solution:
    def canSeePersonsCount(self, heights: List[int]) -> List[int]:
        n = len(heights)
        ans = [0] * n
        stk = []
        for i in range(n - 1, -1, -1):
            while stk and stk[-1] < heights[i]:
                ans[i] += 1
                stk.pop()
            if stk:
                ans[i] += 1
                stk.append(heights[i])
        return ans
```

• SimplyLeet

Round 6 · Mid · Hard

p.2508

```
# Python solution using a monotonic stack

def canSeePersonsCount(heights):
    n = len(heights) # Total number of people in the queue
    answer = [0] * n # Initialize the answer array with zeros
    stack = [] # Stack to keep track of heights

    # Iterate from rightmost person to leftmost person
    for i in range(n - 1, -1, -1):
        count = 0 # Count for the number of people the current person can see

        # Pop all people shorter than the current person as they are visible
        while stack and heights[i] > stack[-1]:
            stack.pop()
            count += 1 # Each popped person is visible from the current
            person's view

        # If the stack is not empty, the current person can see one more person
        # (taller than them)
        if stack:
            count += 1

        answer[i] = count # Save the computed count for the current person
        stack.append(heights[i]) # Push the current person's height onto the
        stack

    return answer

# Example usage:
# heights = [10,6,8,5,11,9] should output [3,1,2,1,1,0]
print(canSeePersonsCount([10,6,8,5,11,9]))
```

Round 6 · Mid · Hard

p.2509

Monotonic Queue

Round 6 · Mid · Hard · 4 questions

Round 6 · Mid · Hard

p.2510

Monotonic Queue

#239 Sliding Window Maximum

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Queue · Sliding Window · Monotonic Queue

Lists: Grind 169

Problem

You are given an array of integers `nums`, there is a sliding window of size `k` which is moving from the very left of the array to the very right. You can only see the `k` numbers in the window. Each time the sliding window moves right by one position.

Return the max sliding window.

Example 1:

```
Input: nums = [1,3,-1,-3,5,3,6,7], k = 3
Output: [3,3,5,5,6,7]
Explanation:
Window position Max
-----
[1 3 -1] -3 5 3 6 7 3
1 [3 -1 -3] 5 3 6 7 3
1 3 [-1 -3 5] 3 6 7 5
```

Example 2:

Round 6 · Mid · Hard

p.2511

```
Input: nums = [1], k = 1
Output: [1]
```

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $-104 \leq \text{nums}[i] \leq 104$
- $1 \leq k \leq \text{nums.length}$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxSlidingWindow(self, nums: list[int], k: int) -> list[int]:
        ans = []
        maxQ = collections.deque()

        for i, num in enumerate(nums):
            while maxQ and maxQ[-1] < num:
                maxQ.pop()
            maxQ.append(num)
            if i >= k and nums[i - k] == maxQ[0]: # out-of-bounds

                maxQ.popleft()
            if i >= k - 1:
                ans.append(maxQ[0])

        return ans
```

• LeetDoocs

Round 6 · Mid · Hard

p.2512

```
class Solution:
    def maxSlidingWindow(self, nums: List[int], k: int) -> List[int]:
        q = [(-v, i) for i, v in enumerate(nums[: k - 1])]
        heapify(q)
        ans = []
        for i in range(k - 1, len(nums)):
            heappush(q, (-nums[i], i))
            while q[0][1] <= i - k:
                heappop(q)
            ans.append(-q[0][0])
```

```
        return ans
```

```
class Solution:
    def maxSlidingWindow(self, nums: List[int], k: int) -> List[int]:
        q = deque()
        ans = []
        for i, x in enumerate(nums):
            if q and i - q[0] >= k:
                q.popleft()
            while q and nums[q[-1]] <= x:
                q.pop()
            q.append(i)
```

```
        if i >= k - 1:
            ans.append(nums[q[0]])
        return ans
```

• SimplyLeet

```
from collections import deque

def maxSlidingWindow(nums, k):
    # Create a deque to store indices of elements in the current window
    deq = deque()
    result = []

    for i, num in enumerate(nums):
        # Remove indices from the front if they are out of the current window
        if deq and deq[0] == i - k:
```

```
        deq.popleft()
```

```
        # Remove elements from the back while the current number is larger
        # than the number at the indices stored in the deque
        while deq and nums[deq[-1]] < num:
            deq.pop()
```

```
        # Append the current index to the deque
        deq.append(i)
```

```
        # If we've hit the window size, append the current max to result
        if i >= k - 1:
            result.append(nums[deq[0]])
```

```
    return result
```

```
# Example usage:
# print(maxSlidingWindow([1,3,-1,-3,5,3,6,7], 3))
```

• LC.ca

```
from collections import deque
```

```
class Solution:
    def maxSlidingWindow(self, nums: List[int], k: int) -> List[int]:
        q = deque()
        ans = []
        for i, v in enumerate(nums):
            if q and i - k + 1 > q[0]:
                q.popleft() # remove index if out of window left
            while q and nums[q[-1]] <= v: # `<==`, not `<`, to ensure the bigger
                index stored in deque
```

```

        q.pop()
        q.append(i)
        if i >= k - 1:
            ans.append(nums[q[0]])
        return ans

#####

class Solution(object):
    def maxSlidingWindow(self, nums, k):
        """
        :type nums: List[int]
        :type k: int
        :rtype: List[int]
        """
        if k == 0:
            return []
        ans = [0 for _ in range(len(nums) - k + 1)]
        stack = collections.deque([])
        for i in range(0, k):
            while stack and nums[stack[-1]] < nums[i]:
                stack.pop()
            stack.append(i)
            ans[0] = nums[stack[0]]
            idx = 0
        for i in range(k, len(nums)):
            idx += 1
            if stack and stack[0] == i - k:
                stack.popleft()
            while stack and nums[stack[-1]] < nums[i]:
                stack.pop()
            stack.append(i)
            ans[idx] = nums[stack[0]]

        return ans

```

◆ Quick Review

Round 6 · Mid · Hard

p.2515

Key Insights

- Use a deque (double-ended queue) to maintain the indices of potential maximum elements in a decreasing order.
- Ensure that the deque always has the current window's indices by removing elements that fall outside the current window.
- Only add elements that are greater than the ones at the back of the deque, thus preserving a monotonic decreasing order.
- The maximum for the current window is at the front of the deque.

Space & Time

Time Complexity: $O(n)$ where n is the number of elements in `nums`, since each element is processed at most twice.

Space Complexity: $O(k)$, as the deque stores indices for at most k elements at any time.

Solution

The solution uses a monotonic deque to maintain indices of potential maximum elements in the current window. As the window slides, we:

Round 6 · Mid · Hard

p.2516

– Remove indices from the front if they are out of the window. – Remove indices from the back if the current element is larger, because they cannot be the maximum if the current element is in the window. – Append the current index. – After the first k elements, the element at the front of the deque is the maximum for that window. This approach ensures that each element is pushed and popped from the deque only once, achieving an overall $O(n)$ time complexity.

Monotonic Queue

☐ #862 Shortest Subarray with Sum at Least K

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode Array · Binary Search · Queue · Sliding Window · Heap (Priority Queue) · Prefix Sum · Monotonic Queue

Lists: AlgoMaster 600

Problem

Given an integer array `nums` and an integer `k`, return the length of the shortest non-empty subarray of `nums` with a sum of at least `k`. If there is no such subarray, return `-1`.

A subarray is a contiguous part of an array.

Example 1:

Input: `nums = [1]`, `k = 1`
Output: `1`

Example 2:

Input: `nums = [1,2]`, `k = 4`
Output: `-1`

Example 3:

Input: `nums = [2,-1,2]`, `k = 3`
Output: `3`

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $-105 \leq \text{nums}[i] \leq 105$
- $1 \leq k \leq 109$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def shortestSubarray(self, nums: List[int], k: int) -> int:
        n = len(nums)
        ans = n + 1
        dq = collections.deque()
        prefix = list(itertools.accumulate(nums, initial=0))

        for i in range(n + 1):
            while dq and prefix[i] - prefix[dq[0]] >= k:
                ans = min(ans, i - dq.popleft())

            while dq and prefix[i] <= prefix[dq[-1]]:
                dq.pop()
            dq.append(i)

        return ans if ans <= n else -1
```

• LeetCode

```
class Solution:
    def shortestSubarray(self, nums: List[int], k: int) -> int:
        s = list(accumulate(nums, initial=0))
        q = deque()
        ans = inf
        for i, v in enumerate(s):
            while q and v - s[q[0]] >= k:
                ans = min(ans, i - q.popleft())
            while q and s[q[-1]] >= v:
                q.pop()
            q.append(i)
```

Round 6 · Mid · Hard

p.2519

```
q.append(i)
return -1 if ans == inf else ans
```

• SimplyLeet

```
# Python solution for Shortest Subarray with Sum at Least K
from collections import deque
```

```
def shortestSubarray(nums, k):
    # Build the prefix sums, starting with an initial 0
    prefix_sums = [0]
    for num in nums:
        prefix_sums.append(prefix_sums[-1] + num)

    # Deque to store indices of prefix_sums in increasing order
```

```
d = deque()
res = float('inf')
```

```
# Iterate over prefix sums
for i, current_sum in enumerate(prefix_sums):
    # If the difference between current and oldest prefix sum is at least
    k,
    # update the result and remove the element from deque front.
    while d and current_sum - prefix_sums[d[0]] >= k:
        res = min(res, i - d[0])
        d.popleft()
```

```
# Maintain increasing order of prefix sums in deque
while d and prefix_sums[d[-1]] >= current_sum:
    d.pop()
```

```
# Add current index
d.append(i)
```

```
return res if res != float('inf') else -1
```

```
# Example usage
if __name__ == "__main__":
    print(shortestSubarray([2, -1, 2], 3)) # Expected output: 3
```

Round 6 · Mid · Hard

p.2520

Monotonic Queue

□ #1425 Constrained Subsequence Sum

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Dynamic Programming · Queue ·
 Sliding Window · Heap · Monotonic Queue
 Lists: Denny Zhang

Problem

Given an integer array `nums` and an integer `k`, return the maximum sum of a non-empty subsequence of that array such that for every two consecutive integers in the subsequence, `nums[i]` and `nums[j]`, where $i < j$, the condition $j - i \leq k$ is satisfied.

A subsequence of an array is obtained by deleting some number of elements (can be zero) from the array, leaving the remaining elements in their original order.

Example 1:

Input: `nums = [10,2,-10,5,20]`, `k = 2`
 Output: 37
 Explanation: The subsequence is `[10, 2, 5, 20]`.

Example 2:

Round 6 · Mid · Hard

p.2521

Input: `nums = [-1,-2,-3]`, `k = 1`
 Output: -1
 Explanation: The subsequence must be non-empty, so we choose the largest number.

Example 3:

Input: `nums = [10,-2,-10,-5,20]`, `k = 2`
 Output: 23
 Explanation: The subsequence is `[10, -2, -5, 20]`.

Constraints:

- $1 \leq k \leq \text{nums.length} \leq 105$
- $-104 \leq \text{nums}[i] \leq 104$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def constrainedSubsetSum(self, nums: list[int], k: int) -> int:
        # dp[i] := the maximum the sum of non-empty subsequences in nums[0..i]
        dp = [0] * len(nums)
        # dq stores dp[i - k], dp[i - k + 1], ..., dp[i - 1] whose values are > 0
        # in decreasing order.
        dq = collections.deque()

        for i, num in enumerate(nums):
            if dq:
                dp[i] = max(dq[0], 0) + num
            else:
                dp[i] = num
            while dq and dq[-1] < dp[i]:
                dq.pop()
            dq.append(dp[i])
            if i >= k and dp[i - k] == dq[0]:
                dq.popleft()

        return max(dp)
```

Round 6 · Mid · Hard

p.2522

• LeetCode

```
class Solution:
    def constrainedSubsetSum(self, nums: List[int], k: int) -> int:
        q = deque([0])
        n = len(nums)
        f = [0] * n
        ans = -inf
        for i, x in enumerate(nums):
            while i - q[0] > k:
                q.popleft()
            f[i] = max(0, f[q[0]]) + x

            ans = max(ans, f[i])
            while q and f[q[-1]] <= f[i]:
                q.pop()
            q.append(i)
        return ans
```

• SimplyLeet

```
# Python solution using a monotonic deque for efficient window maximum
computation.
from collections import deque

def constrained_subsequence_sum(nums, k):
    n = len(nums)
    dp = [0] * n          # dp[i] stores the maximum subsequence sum ending at
    index i
    dp[0] = nums[0]
    result = dp[0]

    # Deque to store indices in a decreasing order of dp values
```

Round 6 · Mid · Hard

p.2523

```
deq = deque([0])

for i in range(1, n):
    # Remove indices from the front which are out of the window
    while deq and i - deq[0] > k:
        deq.popleft()

    # The maximum dp in the window is at the front of the deque
    max_in_window = dp[deq[0]] if dp[deq[0]] > 0 else 0

    # Calculate dp[i] by either starting a new subsequence or extending the
    best one from the window
    dp[i] = nums[i] + max_in_window

    # Update overall answer
    result = max(result, dp[i])

    # Maintain the deque monotonically so that dp indices are in decreasing
    order
    while deq and dp[i] >= dp[deq[-1]]:
        deq.pop()
    deq.append(i)

return result

# Example usage:
# print(constrained_subsequence_sum([10,2,-10,5,20], 2))
```

• LC.ca

```
class Solution:
    def constrainedSubsetSum(self, nums: List[int], k: int) -> int:
        n = len(nums)
        dp = [0] * n
        ans = -inf
        q = deque()
        for i, v in enumerate(nums):
            if q and i - q[0] > k:
                q.popleft()
            dp[i] = max(0, 0 if not q else dp[q[0]]) + v
```

Round 6 · Mid · Hard

p.2524

```

while q and dp[q[-1]] <= dp[i]:
    q.pop()
q.append(i)
ans = max(ans, dp[i])
return ans

```

◆ Quick Review

Key Insights

- Use dynamic programming where $dp[i]$ represents the maximum sum of a valid subsequence ending at index i .
- For each index i , consider taking $nums[i]$ alone or appending it to a valid subsequence ending at an index j (where $i - j \leq k$) that maximizes the sum.
- To efficiently obtain the maximum $dp[j]$ in a sliding window of the last k indices, use a monotonic (decreasing) deque.
- The idea is to maintain the window maximum and update it as we proceed through the array.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$ ($O(k)$ extra space for the deque, with dp array using $O(n)$)

Solution

We solve the problem by defining a dp array where $dp[i] = nums[i] + \max(0, \text{maximum dp value in the window } [i-k, i-1])$. The trick is to include $\max(0, \dots)$ to allow starting a new subsequence when previous sums are negative. To access the maximum dp in the window in constant time, a monotonic deque is maintained that stores indices with their dp values in decreasing order. At each index, we:

- Remove elements from the front of the deque if they are out of the current window ($i - \text{index} > k$).
- Use the front of the deque (if non-empty) as the maximum dp value from the previous k indices.
- Compute $dp[i]$ and update the global maximum answer.
- Remove from the back of the deque all indices with dp values less than the current $dp[i]$ since they are not useful.
- Insert the current index into the deque.

Monotonic Queue

#1499 Max Value of Equation

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Queue · Sliding Window · Heap (Priority Queue) · Monotonic Queue

Lists: AlgoMaster 600

Problem

You are given an array `points` containing the coordinates of points on a 2D plane, sorted by the `x`-values, where `points[i] = [xi, yi]` such that $x_i < x_j$ for all $1 \leq i < j \leq \text{points.length}$. You are also given an integer `k`.

Return the maximum value of the equation $y_i + y_j + |x_i - x_j|$ where $|x_i - x_j| \leq k$ and $1 \leq i < j \leq \text{points.length}$.

It is guaranteed that there exists at least one pair of points that satisfy the constraint $|x_i - x_j| \leq k$.

Example 1:

Round 6 · Mid · Hard

p.2527

Input: `points = [[1,3],[2,0],[5,10],[6,-10]]`, `k = 1`

Output: 4

Explanation: The first two points satisfy the condition $|x_i - x_j| \leq 1$ and if we calculate the equation we get $3 + 0 + |1 - 2| = 4$. Third and fourth points also satisfy the condition and give a value of $10 + -10 + |5 - 6| = 1$. No other pairs satisfy the condition, so we return the max of 4 and 1.

Example 2:

Input: `points = [[0,0],[3,0],[9,2]]`, `k = 3`

Output: 3

Explanation: Only the first two points have an absolute difference of 3 or less in the `x`-values, and give the value of $0 + 0 + |0 - 3| = 3$.

Constraints:

- $2 \leq \text{points.length} \leq 105$
- `points[i].length == 2`
- $-108 \leq x_i, y_i \leq 108$
- $0 \leq k \leq 2 * 108$
- $x_i < x_j$ for all $1 \leq i < j \leq \text{points.length}$
- `xi` form a strictly increasing sequence.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findMaxValueOfEquation(self, points: list[list[int]], k: int) -> int:
        ans = -math.inf
        maxHeap = [] # (y - x, x)

        for x, y in points:
            while maxHeap and x + maxHeap[0][1] > k:
                heapq.heappop(maxHeap)
            if maxHeap:
                ans = max(ans, x + y - maxHeap[0][0])
```

Round 6 · Mid · Hard

p.2528

```
heapq.heappush(maxHeap, (x - y, -x))
```

```
return ans
```

• LeetCode

```
class Solution:
    def findMaxValueOfEquation(self, points: List[List[int]], k: int) -> int:
        ans = -inf
        pq = []
        for x, y in points:
            while pq and x - pq[0][1] > k:
                heappop(pq)
            if pq:
                ans = max(ans, x + y - pq[0][0])
            heappush(pq, (x - y, x))
```

```
return ans
```

• SimplyLeet

```
# Python solution implementation with detailed comments.
from collections import deque

def findMaxValueOfEquation(points, k):
    # Initialize deque to store tuples of (x, y+x) in a monotonic decreasing
    # order for y+x values.
    mono_deque = deque()
    max_value = float('-inf')

    # Iterate over each point in the sorted list
    for x, y in points:
```

```
# Remove points from the front that are out of the window (x difference
> k)
while mono_deque and x - mono_deque[0][0] > k:
    mono_deque.popleft()

# If deque is not empty, a candidate exists to maximize the equation.
if mono_deque:
    # The candidate is the current point's (y-x) plus the maximum (y+x)
    # from the deque front.
    max_value = max(max_value, y - x + mono_deque[0][1])

# Prepare the current value for future comparisons

current_value = y + x

# Before appending the current point, pop from the back if
# current_value is greater
while mono_deque and mono_deque[-1][1] <= current_value:
    mono_deque.pop()

# Append the current point (store x and current value)
mono_deque.append((x, current_value))

return max_value
```

```
# Example usage:
# points = [[1,3],[2,0],[5,10],[6,-10]]
# k = 1
# print(findMaxValueOfEquation(points, k)) # Expected output: 4
```

Recursion

Round 6 · Mid · Hard · 2 questions

Round 6 · Mid · Hard

p.2531

Recursion

□ #273 Integer to English Words

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · String · Recursion
Lists: AlgoMaster 600

Problem

Convert a non-negative integer num to its English words representation.

Example 1:

Input: num = 123
Output: "One Hundred Twenty Three"

Example 2:

Input: num = 12345
Output: "Twelve Thousand Three Hundred Forty Five"

Example 3:

Input: num = 1234567
Output: "One Million Two Hundred Thirty Four Thousand Five Hundred Sixty Seven"

Constraints:

- $0 \leq \text{num} \leq 2^{31} - 1$

Community Solutions (Python)

- WalkCC

Round 6 · Mid · Hard

p.2532

```

class Solution:
    def numberToWords(self, num: int) -> str:
        if num == 0:
            return 'Zero'

        belowTwenty = ['', 'One', 'Two', 'Three',
                        'Four', 'Five', 'Six', 'Seven',
                        'Eight', 'Nine', 'Ten', 'Eleven',
                        'Twelve', 'Thirteen', 'Fourteen', 'Fifteen',
                        'Sixteen', 'Seventeen', 'Eighteen', 'Nineteen']

        tens = ['', 'Ten', 'Twenty', 'Thirty', 'Forty',
                'Fifty', 'Sixty', 'Seventy', 'Eighty', 'Ninety']

        def helper(num: int) -> str:
            if num < 20:
                s = belowTwenty[num]
            elif num < 100:
                s = tens[num // 10] + ' ' + belowTwenty[num % 10]
            elif num < 1000:
                s = helper(num // 100) + ' Hundred ' + helper(num % 100)

            elif num < 1000000:
                s = helper(num // 1000) + ' Thousand ' + helper(num % 1000)
            elif num < 1000000000:
                s = helper(num // 1000000) + ' Million ' + helper(num % 1000000)
            else:
                s = helper(num // 1000000000) + ' Billion ' + helper(num % 1000000000)
            return s.strip()

        return helper(num)

```

• LeetCode

Round 6 · Mid · Hard

p.2533

```

class Solution:
    def numberToWords(self, num: int) -> str:
        if num == 0:
            return 'Zero'

        lt20 = [
            '',
            'One',
            'Two',
            'Three',

            'Four',
            'Five',
            'Six',
            'Seven',
            'Eight',
            'Nine',
            'Ten',
            'Eleven',
            'Twelve',
            'Thirteen',

            'Fourteen',
            'Fifteen',
            'Sixteen',
            'Seventeen',
            'Eighteen',
            'Nineteen',
        ]
        tens = [
            '',
            'Ten',

            'Twenty',
            'Thirty',
            'Forty',
            'Fifty',
            'Sixty',
            'Seventy',
            'Eighty',
            'Ninety',
        ]
        thousands = ['Billion', 'Million', 'Thousand', '']

```

Round 6 · Mid · Hard

p.2534

```
def transfer(num):
    if num == 0:
        return ''
    if num < 20:
        return lt20[num] + ' '
    if num < 100:
        return tens[num // 10] + ' ' + transfer(num % 10)
    return lt20[num // 100] + ' Hundred ' + transfer(num % 100)
```

```
res = []
i, j = 1000000000, 0
while i > 0:
    if num // i != 0:
        res.append(transfer(num // i))
        res.append(thousands[j])
        res.append(' ')
        num %= i
    j += 1
    i //= 1000
```

```
return ''.join(res).strip()
```

• SimplyLeet

```
# Define mappings for number words
ones = ["", "One", "Two", "Three", "Four", "Five", "Six", "Seven", "Eight", "Nine"]
teens = ["Ten", "Eleven", "Twelve", "Thirteen", "Fourteen", "Fifteen", "Sixteen", "Seventeen", "Eighteen", "Nineteen"]
tens = ["", "", "Twenty", "Thirty", "Forty", "Fifty", "Sixty", "Seventy", "Eighty", "Ninety"]
scales = ["", "Thousand", "Million", "Billion"]
```

```
def three_digit_to_words(num):
    # Convert a number less than 1000 to words
    if num == 0:
        return ""
```

```
result = ""
# Handle hundreds place
if num >= 100:
    result += ones[num // 100] + " Hundred "
    num %= 100
# Handle tens and ones
if num >= 20:
    result += tens[num // 10] + " "
    num %= 10
elif num >= 10: # Handle teen numbers specifically
```

```
    result += teens[num - 10] + " "
    num = 0
# Handle remaining ones (if any)
if num > 0:
    result += ones[num] + " "
return result
```

```
def number_to_words(num):
    # Special case: if number is 0, return "Zero"
    if num == 0:
```

```
        return "Zero"
```

```
    words = ""
    scale_index = 0
    # Process each group of 3 digits
    while num > 0:
        current_chunk = num % 1000 # take last three digits
        if current_chunk != 0:
            # Convert the current chunk and add the corresponding scale
            words = three_digit_to_words(current_chunk) + scales[scale_index] + " " + words
```



```

num //= 1000
scale_index += 1

# Return the cleaned up result
return words.strip()

# Example Usage:
print(number_to_words(123))      # Output: "One Hundred Twenty Three"
print(number_to_words(12345))   # Output: "Twelve Thousand Three Hundred
Forty Five"
print(number_to_words(1234567)) # Output: "One Million Two Hundred Thirty
Four Thousand Five Hundred Sixty Seven"

```

Recursion

□ #761 Special Binary String

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 String · Divide and Conquer · Sorting
 Lists: AlgoMaster 600

Problem

Special binary strings are binary strings with the following two properties:

- The number of 0's is equal to the number of 1's.
- Every prefix of the binary string has at least as many 1's as 0's.

You are given a special binary string s .

A move consists of choosing two consecutive, non-empty, special substrings of s , and swapping them. Two strings are consecutive if the last character of the first string is exactly one index before the first character of the second string.

Return the lexicographically largest resulting string possible after applying the mentioned

operations on the string.

Example 1:

Input: s = "11011000"
 Output: "11100100"
 Explanation: The strings "10" [occurring at s[1]] and "1100" [at s[3]] are swapped.
 This is the lexicographically largest string possible after some number of swaps.

Example 2:

Input: s = "10"
 Output: "10"

Constraints:

- $1 \leq s.length \leq 50$
- s[i] is either '0' or '1'.
- s is a special binary string.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def makeLargestSpecial(self, s: str) -> str:
        specials = []
        count = 0

        i = 0
        for j, c in enumerate(s):
            count += 1 if c == '1' else -1
            if count == 0:
                specials.append(

                    '1' + self.makeLargestSpecial(s[i + 1:j]) + '0')
                i = j + 1

        return ''.join(sorted(specials)[::-1])
```

Round 6 · Mid · Hard

p.2539

• LeetCode

```
class Solution:
    def makeLargestSpecial(self, s: str) -> str:
        if s == '':
            return ''
        ans = []
        cnt = 0
        i = j = 0
        while i < len(s):
            cnt += 1 if s[i] == '1' else -1
            if cnt == 0:
                ans.append('1' + self.makeLargestSpecial(s[j + 1 : i]) + '0')
                j = i + 1
                i += 1
        ans.sort(reverse=True)
        return ''.join(ans)
```

• SimplyLeet

```
# Python solution with detailed comments

def makeLargestSpecial(s: str) -> str:
    # Base case: if s is empty, return empty string
    if not s:
        return s

    segments = [] # List to hold valid special substrings
    count = 0 # Counter to track the balance of '1's and '0's
    start = 0 # Starting index for a potential special substring

    # Iterate over each character in string s
    for i, char in enumerate(s):
        # Increase counter for '1' and decrease for '0'
        if char == '1':
            count += 1
        else:
            count -= 1

        # When the count becomes 0, we found a valid special substring
```

Round 6 · Mid · Hard

p.2540

```
    if count == 0:
        # Recursively process the inner part (excluding the outer '1' and
'0')
        inner = makeLargestSpecial(s[start+1:i])
        # Wrap the processed inner substring with outer characters '1' and
'0'
        segments.append("1" + inner + "0")
        start = i + 1 # Update start for the next segment

# Sort segments in reverse lexicographical order to maximize value
segments.sort(reverse=True)
# Join sorted segments and return as final answer

return "".join(segments)

# Example usage:
print(makeLargestSpecial("11011000"))
```

Round 6 · Mid · Hard**p.2541**

Merge Sort

Round 6 · Mid · Hard · 2 questions**Round 6 · Mid · Hard****p.2542**

Merge Sort

#327 Count of Range Sum

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Binary Search · Divide and Conquer ·
 Binary Indexed Tree · Segment Tree · Merge
 Sort · Ordered Set

Lists: AlgoMaster 600

Problem

Given an integer array `nums` and two integers `lower` and `upper`, return the number of range sums that lie in `[lower, upper]` inclusive.

Range sum $S(i, j)$ is defined as the sum of the elements in `nums` between indices `i` and `j` inclusive, where $i \leq j$.

Example 1:

Input: `nums = [-2,5,-1]`, `lower = -2`, `upper = 2`
 Output: 3
 Explanation: The three ranges are: `[0,0]`, `[2,2]`, and `[0,2]` and their respective sums are: -2, -1, 2.

Example 2:

Input: `nums = [0]`, `lower = 0`, `upper = 0`
 Output: 1

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $-231 \leq \text{nums}[i] \leq 231 - 1$
- $-105 \leq \text{lower} \leq \text{upper} \leq 105$
- The answer is guaranteed to fit in a 32-bit integer.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def countRangeSum(self, nums: list[int], lower: int, upper: int) -> int:
        n = len(nums)
        self.ans = 0
        prefix = list(itertools.accumulate(nums, initial=0))

        self._mergeSort(prefix, 0, n, lower, upper)
        return self.ans

    def _mergeSort(
        self,
        prefix: list[int],
        l: int,
        r: int,
        lower: int,
        upper: int,
    ) -> None:
        if l >= r:
            return
```

```

m = (l + r) // 2
self._mergeSort(prefix, l, m, lower, upper)
self._mergeSort(prefix, m + 1, r, lower, upper)
self._merge(prefix, l, m, r, lower, upper)

def _merge(
    self,
    prefix: list[int],
    l: int,
    m: int,

    r: int,
    lower: int,
    upper: int,
) -> None:
    lo = m + 1 # the first index s.t. prefix[lo] - prefix[l] >= lower
    hi = m + 1 # the first index s.t. prefix[hi] - prefix[l] > upper

    # For each index i in range [l, m], add hi - lo to `ans`.
    for i in range(l, m + 1):
        while lo <= r and prefix[lo] - prefix[i] < lower:
            lo += 1
        while hi <= r and prefix[hi] - prefix[i] <= upper:
            hi += 1
        self.ans += hi - lo

    sorted = [0] * (r - l + 1)
    k = 0 # sorted's index
    i = l # left's index
    j = m + 1 # right's index

    while i <= m and j <= r:
        if prefix[i] < prefix[j]:
            sorted[k] = prefix[i]
            k += 1
            i += 1
        else:
            sorted[k] = prefix[j]
            k += 1
            j += 1

```

Round 6 · Mid · Hard

p.2545

```

# Put the possible remaining left part into the sorted array.
while i <= m:
    sorted[k] = prefix[i]
    k += 1
    i += 1

# Put the possible remaining right part into the sorted array.
while j <= r:
    sorted[k] = prefix[j]
    k += 1
    j += 1

prefix[l:l + len(sorted)] = sorted

```

• LeetDoocs

```

class BinaryIndexedTree:
    def __init__(self, n):
        self.n = n
        self.c = [0] * (n + 1)

    def update(self, x, v):
        while x <= self.n:
            self.c[x] += v
            x = x & -x

    def query(self, x):
        s = 0
        while x > 0:
            s += self.c[x]
            x = x & -x
        return s

class Solution:
    def countRangeSum(self, nums: List[int], lower: int, upper: int) -> int:

```

Round 6 · Mid · Hard

p.2546

```
s = list(accumulate(nums, initial=0))
arr = sorted(set(v for x in s for v in (x, x - lower, x - upper)))
tree = BinaryIndexedTree(len(arr))
ans = 0
for x in s:
    l = bisect_left(arr, x - upper) + 1
    r = bisect_left(arr, x - lower) + 1
    ans += tree.query(r) - tree.query(l - 1)
    tree.update(bisect_left(arr, x) + 1, 1)
return ans
```

• SimplyLeet

Python implementation using merge sort style divide and conquer

```
def countRangeSum(nums, lower, upper):
    # Build prefix sum array - prefix[i] is the sum of nums[0] up to nums[i-1].
    prefix = [0]
    for num in nums:
        prefix.append(prefix[-1] + num)
```

Helper function to count valid range sums during merge sort.

```
def merge_sort(start, end):
```

```
    if end - start <= 1: # Base case: one element is already sorted.
        return 0
```

```
    mid = (start + end) // 2
```

```
    count = merge_sort(start, mid) + merge_sort(mid, end)
```

Two pointers for the merging process

```
    j = k = mid
```

```
    temp = []
```

```
    r = mid
```

Iterate over the left part

```
    for left in prefix[start:mid]:
```

Increase k while the condition is not satisfied.

```
while k < end and prefix[k] - left < lower:
```

```
    k += 1
```

Increase j while the condition is met.

```
while j < end and prefix[j] - left <= upper:
```

```
    j += 1
```

```
count += j - k
```

Standard merge sort merge operation.

```
l, r = start, mid
```

```
sorted_list = []
```

```
while l < mid and r < end:
```

```
    if prefix[l] <= prefix[r]:
```

```
        sorted_list.append(prefix[l])
```

```
        l += 1
```

```
    else:
```

```
        sorted_list.append(prefix[r])
```

```
        r += 1
```

Append remaining elements.

```
sorted_list.extend(prefix[l:mid])
```

```
sorted_list.extend(prefix[r:end])
```

Write back sorted list into prefix.

```
prefix[start:end] = sorted_list
```

```
return count
```

```
return merge_sort(0, len(prefix))
```

Example usage:

```
print(countRangeSum([-2,5,-1], -2, 2)) # Output: 3
```

Merge Sort

#493 Reverse Pairs

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Binary Search · Divide and Conquer ·
 Binary Indexed Tree · Segment Tree · Merge
 Sort · Ordered Set

Lists: AlgoMaster 600

Problem

Given an integer array `nums`, return the number of reverse pairs in the array.

A reverse pair is a pair (i, j) where:

- $0 \leq i < j < \text{nums.length}$ and
- $\text{nums}[i] > 2 * \text{nums}[j]$.

Example 1:

Input: `nums = [1,3,2,3,1]`
 Output: 2
 Explanation: The reverse pairs are:
 $(1, 4) \rightarrow \text{nums}[1] = 3, \text{nums}[4] = 1, 3 > 2 * 1$
 $(3, 4) \rightarrow \text{nums}[3] = 3, \text{nums}[4] = 1, 3 > 2 * 1$

Example 2:

Input: `nums = [2,4,3,5,1]`
 Output: 3
 Explanation: The reverse pairs are:
 $(1, 4) \rightarrow \text{nums}[1] = 4, \text{nums}[4] = 1, 4 > 2 * 1$
 $(2, 4) \rightarrow \text{nums}[2] = 3, \text{nums}[4] = 1, 3 > 2 * 1$
 $(3, 4) \rightarrow \text{nums}[3] = 5, \text{nums}[4] = 1, 5 > 2 * 1$

Constraints:

- $1 \leq \text{nums.length} \leq 5 * 10^4$
- $-2^{31} \leq \text{nums}[i] \leq 2^{31} - 1$

Community Solutions (Python)

• LeetDoocs

```
class Solution:
    def reversePairs(self, nums: List[int]) -> int:
        def merge_sort(l, r):
            if l >= r:
                return 0
            mid = (l + r) >> 1
            ans = merge_sort(l, mid) + merge_sort(mid + 1, r)
            t = []
            i, j = l, mid + 1
            while i <= mid and j <= r:
                if nums[i] <= 2 * nums[j]:
                    i += 1
                else:
                    ans += mid - i + 1
                    j += 1
            i, j = l, mid + 1
            while i <= mid and j <= r:
                if nums[i] <= nums[j]:
                    t.append(nums[i])
                    i += 1
```

```

        else:
            t.append(nums[j])
            j += 1
        t.extend(nums[i : mid + 1])
        t.extend(nums[j : r + 1])
        nums[l : r + 1] = t
        return ans

    return merge_sort(0, len(nums) - 1)

class BinaryIndexedTree:
    def __init__(self, n):
        self.n = n
        self.c = [0] * (n + 1)

    @staticmethod
    def lowbit(x):
        return x & -x

    def update(self, x, delta):

        while x <= self.n:
            self.c[x] += delta
            x += BinaryIndexedTree.lowbit(x)

    def query(self, x):
        s = 0
        while x > 0:
            s += self.c[x]
            x -= BinaryIndexedTree.lowbit(x)
        return s

class Solution:
    def reversePairs(self, nums: List[int]) -> int:
        s = set()
        for num in nums:
            s.add(num)
            s.add(num * 2)
        alls = sorted(s)
        m = {v: i for i, v in enumerate(alls, 1)}

```

Round 6 · Mid · Hard

p.2551

```

        ans = 0
        tree = BinaryIndexedTree(len(m))
        for num in nums[::-1]:
            ans += tree.query(m[num] - 1)
            tree.update(m[num * 2], 1)
        return ans

class Node:
    def __init__(self):
        self.l = 0
        self.r = 0
        self.v = 0

class SegmentTree:
    def __init__(self, n):
        self.tr = [Node() for _ in range(4 * n)]

        self.build(1, 1, n)

    def build(self, u, l, r):
        self.tr[u].l = l
        self.tr[u].r = r
        if l == r:
            return
        mid = (l + r) >> 1
        self.build(u << 1, l, mid)
        self.build(u << 1 | 1, mid + 1, r)

    def modify(self, u, x, v):
        if self.tr[u].l == x and self.tr[u].r == x:
            self.tr[u].v += 1
            return
        mid = (self.tr[u].l + self.tr[u].r) >> 1
        if x <= mid:
            self.modify(u << 1, x, v)
        else:
            self.modify(u << 1 | 1, x, v)

```

Round 6 · Mid · Hard

p.2552


```

        self.pushup(u)

    def pushup(self, u):
        self.tr[u].v = self.tr[u << 1].v + self.tr[u << 1 | 1].v

    def query(self, u, l, r):
        if self.tr[u].l >= l and self.tr[u].r <= r:
            return self.tr[u].v
        mid = (self.tr[u].l + self.tr[u].r) >> 1
        v = 0

        if l <= mid:
            v += self.query(u << 1, l, r)
        if r > mid:
            v += self.query(u << 1 | 1, l, r)
        return v

class Solution:
    def reversePairs(self, nums: List[int]) -> int:
        s = set()

        for num in nums:
            s.add(num)
            s.add(num * 2)
        alls = sorted(s)
        m = {v: i for i, v in enumerate(alls, 1)}
        tree = SegmentTree(len(m))
        ans = 0
        for v in nums[::-1]:
            x = m[v]
            ans += tree.query(1, 1, x - 1)

            tree.modify(1, m[v * 2], 1)
        return ans

```

• SimplyLeet

Round 6 · Mid · Hard

p.2553

```

# Define the Solution class
class Solution:
    def reversePairs(self, nums):
        # Helper function that uses merge sort to count reverse pairs
        def merge_sort(left, right):
            # Base case: if the segment has one or no elements, no reverse
            # pairs exist
            if left >= right:
                return 0
            mid = (left + right) // 2
            # Count reverse pairs in the left and right halves

            count = merge_sort(left, mid) + merge_sort(mid+1, right)
            j = mid + 1
            # Count the reverse pairs: for every element in left half,
            # move pointer j in right half until condition fails
            for i in range(left, mid+1):
                while j <= right and nums[i] > 2 * nums[j]:
                    j += 1
                count += j - (mid + 1)
            # Merge the two sorted halves
            temp = []

            p1, p2 = left, mid+1
            while p1 <= mid and p2 <= right:
                if nums[p1] <= nums[p2]:
                    temp.append(nums[p1])
                    p1 += 1
                else:
                    temp.append(nums[p2])
                    p2 += 1
            while p1 <= mid:
                temp.append(nums[p1])
                p1 += 1
            while p2 <= right:
                temp.append(nums[p2])
                p2 += 1
            # Copy the sorted elements back into the original array
            nums[left:right+1] = temp
            return count

        return merge_sort(0, len(nums)-1)

```

Round 6 · Mid · Hard

p.2554

Backtracking**Round 6 · Mid · Hard · 4 questions****Round 6 · Mid · Hard****p.2555****Backtracking****#37 Sudoku Solver****HAR****LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Backtracking · Matrix
Lists: Grind 169****Problem****Write a program to solve a Sudoku puzzle by filling the empty cells.****A sudoku solution must satisfy all of the following rules:**

- 1. Each of the digits 1–9 must occur exactly once in each row.**
- 2. Each of the digits 1–9 must occur exactly once in each column.**
- 3. Each of the digits 1–9 must occur exactly once in each of the 9 3x3 sub-boxes of the grid.**

The '.' character indicates empty cells.**Example 1:****Round 6 · Mid · Hard****p.2556**

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Input: board = `[["5","3",".",".","7",".",".",".","."],["6",".",".","1","9","5",".",".","."],[".","9","8",".",".",".","6",".","."],["8",".",".","6",".","3",".",".","."],["4",".","8",".","3",".","1","2","."],["7",".","2",".","4","1","9","."],["6",".","8",".","7","9"]]`

Output: `[["5","3","4","6","7","8","9","1","2"],["6","7","2","1","9","5","3","4","8"],["1","9","8","3","4","2","5","6","7"],["8","5","9","7","6","1","4","2","3"],["4","2","6","8","5","3","7","9","1"],["7","1","3","9","2","4","8","5","6"],["9","6","1","5","3","7","2","8","4"],["2","8","7","4","1","9","6","3","5"],["3","4","5","2","8","6","1","7","9"]]`

Explanation: The input board is shown above and the only valid solution is shown below:

Constraints:

- `board.length == 9`
- `board[i].length == 9`

- `board[i][j]` is a digit or '.'.
- It is guaranteed that the input board has only one solution.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def solveSudoku(self, board: list[list[str]]) -> None:
        def isValid(row: int, col: int, c: str) -> bool:
            for i in range(9):
                if (board[i][col] == c or
                    board[row][i] == c or
                    board[3 * (row // 3) + i // 3][3 * (col // 3) + i % 3] == c):
                    return False
            return True

        def solve(s: int) -> bool:
            if s == 81:
                return True

            i = s // 9
            j = s % 9

            if board[i][j] != '.':
                return solve(s + 1)

            for c in string.digits[1:]:
                if isValid(i, j, c):
                    board[i][j] = c
                    if solve(s + 1):
                        return True
                    board[i][j] = '.'

            return False

        solve(0)
```

• SimplyLeet

```
def solveSudoku(board):
    # Check if placing the digit 'char' at board[row][col] is valid
    def is_valid(row, col, char):
        for i in range(9):
            # Check the row
            if board[row][i] == char:
                return False
            # Check the column
            if board[i][col] == char:
                return False

        # Check the 3x3 sub-box
        sub_row = 3 * (row // 3) + i // 3
        sub_col = 3 * (col // 3) + i % 3
        if board[sub_row][sub_col] == char:
            return False
        return True

    # Backtracking function to fill the board
    def backtrack():
        for row in range(9):
            for col in range(9):
                if board[row][col] == '.':
                    for char in map(str, range(1, 10)):
                        if is_valid(row, col, char):
                            board[row][col] = char # Place the digit
                            if backtrack():
                                return True # Recurse; if solution found,
                                bubble up True
                            board[row][col] = '.' # Backtrack if not valid
                                further down
                        return False # No valid digit found; trigger backtracking
                    return True # Board completely filled
```

Round 6 · Mid · Hard

p.2559

• LC.ca

```
class Solution:
    def solveSudoku(self, board: List[List[str]]) -> None:
        def dfs(k):
            nonlocal ok
            if k == len(t):
                ok = True
                return
            i, j = t[k]
            for v in range(9):
                if row[i][v] == col[j][v] == block[i // 3][j // 3][v] == False:
                    row[i][v] = col[j][v] = block[i // 3][j // 3][v] = True
                    board[i][j] = str(v + 1)
                    dfs(k + 1)
                    row[i][v] = col[j][v] = block[i // 3][j // 3][v] = False
            if ok:
                return

        row = [[False] * 9 for _ in range(9)]
        col = [[False] * 9 for _ in range(9)]
        block = [[[False] * 9 for _ in range(3)] for _ in range(3)]
```

Round 6 · Mid · Hard

p.2560

```

t = []
ok = False
for i in range(9):
    for j in range(9):
        if board[i][j] == '.':
            t.append((i, j))
        else:
            v = int(board[i][j]) - 1
            row[i][v] = col[j][v] = block[i // 3][j // 3][v] = True
dfs(0)

```

◆ Quick Review

Key Insights

- Use backtracking to explore digit placements for each empty cell.
- Validate placements by ensuring that the chosen digit is not already present in the corresponding row, column, or 3x3 sub-box.
- Employ recursion to fill the board incrementally and backtrack upon encountering invalid configurations.
- Although the worst-case time complexity is exponential, the fixed board size (9x9) keeps the problem computationally manageable.

Space & Time

Time Complexity: In the worst-case scenario, the algorithm can explore up to $O(9^n)$

possibilities where n is the number of empty cells. However, since n is at most 81 and many branches are pruned early, the practical runtime is far less.

Space Complexity: $O(n)$ due to the recursion stack, where n is the number of empty cells; given the board's fixed size, this space is constant in practice.

Solution

The solution uses a backtracking approach. For each empty cell on the board, we attempt to place a digit from '1' to '9'. Before placing a digit, we check if the placement is valid by ensuring that:

- The digit is not already present in the same row.
- The digit is not already present in the same column.
- The digit is not already present in the corresponding 3x3 sub-box.

If a valid digit is found, we place it on the board and recursively attempt to solve the rest of the board. If no valid digit can be placed, the algorithm backtracks by reverting the last move and trying a different digit. This recursive search continues until the board is

completely filled with a valid Sudoku configuration.

Round 6 · Mid · Hard

p.2563

Backtracking

□ #51 N-Queens

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Backtracking
Lists: Grind 169

Problem

The n-queens puzzle is the problem of placing n queens on an n x n chessboard such that no two queens attack each other.

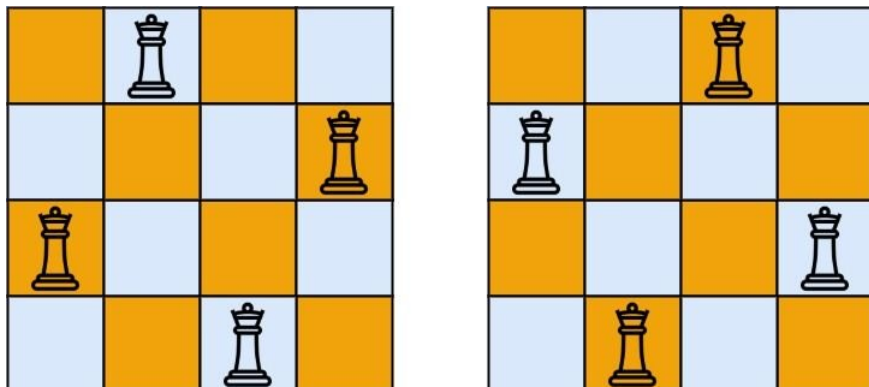
Given an integer n, return all distinct solutions to the n-queens puzzle. You may return the answer in any order.

Each solution contains a distinct board configuration of the n-queens' placement, where 'Q' and '.' both indicate a queen and an empty space, respectively.

Example 1:

Round 6 · Mid · Hard

p.2564



Input: $n = 4$
 Output: `[[".Q..","...Q","Q...", "..Q."],["..Q.", "Q...", "...Q", ".Q.."]]`
 Explanation: There exist two distinct solutions to the 4-queens puzzle as shown above

Example 2:

Input: $n = 1$
 Output: `[["Q"]]`

Constraints:

- $1 \leq n \leq 9$

Community Solutions (Python)

- WalkCC

```
class Solution:
    def solveNQueens(self, n: int) -> list[list[str]]:
        ans = []
        cols = [False] * n
        diag1 = [False] * (2 * n - 1)
        diag2 = [False] * (2 * n - 1)

        def dfs(i: int, board: list[int]) -> None:
            if i == n:
                ans.append(board)
                return

            for j in range(n):
                if cols[j] or diag1[i + j] or diag2[j - i + n - 1]:
                    continue
                cols[j] = diag1[i + j] = diag2[j - i + n - 1] = True
                dfs(i + 1, board + ['. ' * j + 'Q' + '.' * (n - j - 1)])
                cols[j] = diag1[i + j] = diag2[j - i + n - 1] = False

        dfs(0, [])

        return ans
```

• LeetCode

```
class Solution:
    def solveNQueens(self, n: int) -> List[List[str]]:
        def dfs(i: int):
            if i == n:
                ans.append(["".join(row) for row in g])
                return
            for j in range(n):
                if col[j] + dg[i + j] + udg[n - i + j] == 0:
                    g[i][j] = "Q"
                    col[j] = dg[i + j] = udg[n - i + j] = 1
```

```

        dfs(i + 1)
        col[j] = dg[i + j] = udg[n - i + j] = 0
        g[i][j] = "."

```

```

ans = []
g = [["."] * n for _ in range(n)]
col = [0] * n
dg = [0] * (n << 1)
udg = [0] * (n << 1)
dfs(0)

```

```

return ans

```

• SimplyLeet

```

def solveNQueens(n):
    result = []

    # Helper function to backtrack
    def backtrack(row, columns, diag1, diag2, board):
        # Base case: if all rows are filled, add a copy of board to result
        if row == n:
            result.append(list(board))
            return

        # Try to place a queen in each column in the current row
        for col in range(n):
            # Check if the column or diagonals are not under attack
            if col in columns or (row - col) in diag1 or (row + col) in diag2:
                continue

            # Place the queen
            board[row] = '.' * col + 'Q' + '.' * (n - col - 1)
            # Mark the column and diagonals as occupied
            columns.add(col)

```

Round 6 · Mid · Hard

p.2567

```

        diag1.add(row - col)
        diag2.add(row + col)

        # Move to the next row
        backtrack(row + 1, columns, diag1, diag2, board)

        # Backtrack: remove the queen and unmark the sets
        columns.remove(col)
        diag1.remove(row - col)
        diag2.remove(row + col)

```

```

# Initialize board with empty rows
board = [ "." * n for _ in range(n)]
backtrack(0, set(), set(), set(), board)
return result

```

```

# Example usage:
print(solveNQueens(4))

```

• LC.ca

```

class Solution:
    def solveNQueens(self, n: int) -> List[List[str]]:
        def dfs(i: int):
            if i == n:
                ans.append(["".join(row) for row in g])
                return
            for j in range(n):
                if col[j] + dg[i + j] + udg[n - i + j] == 0:
                    g[i][j] = "Q"
                    col[j] = dg[i + j] = udg[n - i + j] = 1

                    dfs(i + 1)
                    col[j] = dg[i + j] = udg[n - i + j] = 0
                    g[i][j] = "."

        ans = []
        g = [ "." * n for _ in range(n)]
        col = [0] * n
        dg = [0] * (n << 1)
        udg = [0] * (n << 1)
        dfs(0)

```

Round 6 · Mid · Hard

p.2568


```
return ans
```

◆ Quick Review

Key Insights

- Use backtracking to explore potential positions row by row.
- Use sets to track which columns and diagonals are under attack.
- Diagonals can be tracked by $(row - col)$ and $(row + col)$, ensuring that queens do not conflict along any diagonal.
- Recursively build each valid board configuration and backtrack when a conflict arises.

Space & Time

Time Complexity: $O(n!)$ in the worst-case scenario, as the solution explores permutations for queen placements.

Space Complexity: $O(n^2)$ for storing board configurations and $O(n)$ for recursion stack and conflict-tracking sets.

Solution

Round 6 · Mid · Hard

p.2569

We use a backtracking algorithm that places queens row by row. For each row, we try placing a queen in each column. Before placing, we check if the column, main diagonal ($row - col$), and anti-diagonal ($row + col$) are free using dedicated sets. If a queen can be safely placed, we mark the column and diagonals as occupied and recurse to the next row. Once all rows are processed, the board configuration is a valid solution and is added to the result. After that, we backtrack by unmarking the sets and removing the queen from the board. This systematic approach ensures all possible configurations are considered.

Round 6 · Mid · Hard

p.2570

Backtracking

#301 Remove Invalid Parentheses

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode String · Backtracking · Breadth-First Search Lists: AlgoMaster 600

Problem

Given a string *s* that contains parentheses and letters, remove the minimum number of invalid parentheses to make the input string valid. Return a list of unique strings that are valid with the minimum number of removals. You may return the answer in any order.

Example 1:

Input: *s* = "()())()"

Output: ["()()()", "()()())"]

Example 2:

Input: *s* = "(a)())()"

Output: ["(a)()()", "(a)()())"]

Example 3:

Input: *s* = ")("

Output: [""]

Constraints:

- $1 \leq s.length \leq 25$
- *s* consists of lowercase English letters and parentheses '(' and ')'. There will be at most 20 parentheses in *s*.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def removeInvalidParentheses(self, s: str) -> list[str]:
        # Similar to 921. Minimum Add to Make Parentheses Valid
        def getLeftAndRightCounts(s: str) -> tuple[int, int]:
            """Returns how many '(' and ')' need to be deleted."""
            l = 0
            r = 0

            for c in s:
                if c == '(':
                    l += 1
                elif c == ')':
                    if l == 0:
                        r += 1
                    else:
                        l -= 1

            return l, r

        def isValid(s: str):
```

```

opened = 0 # the number of '(' - # of ')'
for c in s:
    if c == '(':
        opened += 1
    elif c == ')':
        opened -= 1
    if opened < 0:
        return False
return True # opened == 0

```

```
ans = []
```

```

def dfs(s: str, start: int, l: int, r: int) -> None:
    if l == 0 and r == 0 and isValid(s):
        ans.append(s)
        return

    for i in range(start, len(s)):
        if i > start and s[i] == s[i - 1]:
            continue

```

```

    if r > 0 and s[i] == ')': # Delete s[i]
        dfs(s[:i] + s[i + 1:], i, l, r - 1)
    elif l > 0 and s[i] == '(': # Delete s[i]
        dfs(s[:i] + s[i + 1:], i, l - 1, r)

```

```

l, r = getLeftAndRightCounts(s)
dfs(s, 0, l, r)
return ans

```

• LeetCode

```

class Solution:
    def removeInvalidParentheses(self, s: str) -> List[str]:
        def dfs(i, l, r, lcnt, rcnt, t):
            if i == n:
                if l == 0 and r == 0:
                    ans.add(t)
                return
            if n - i < l + r or lcnt < rcnt:
                return
            if s[i] == '(' and l:

```

Round 6 · Mid · Hard

p.2573

```

        dfs(i + 1, l - 1, r, lcnt, rcnt, t)
    elif s[i] == ')' and r:
        dfs(i + 1, l, r - 1, lcnt, rcnt, t)
    dfs(i + 1, l, r, lcnt + (s[i] == '('), rcnt + (s[i] == ')'), t + s[i])

    l = r = 0
    for c in s:
        if c == '(':
            l += 1
        elif c == ')':

```

```

            if l:
                l -= 1
            else:
                r += 1
    ans = set()
    n = len(s)
    dfs(0, l, r, 0, 0, '')
    return list(ans)

```

• SimplyLeet

```

# Python solution using BFS
from collections import deque

def removeInvalidParentheses(s):
    # Helper function to check if a string is valid.
    def isValid(string):
        count = 0
        for char in string:
            if char == '(':
                count += 1

```

Round 6 · Mid · Hard

p.2574

```

        elif char == ')':
            count -= 1
            if count < 0:
                return False
        return count == 0

result = []
visited = set([s]) # Set to avoid re-processing the same string
queue = deque([s])
found = False # Flag indicating if we've found a valid expression at
current level

while queue:
    current = queue.popleft()
    if isValid(current):
        result.append(current)
        found = True
    # Continue exploring only if a valid expression hasn't been found at
    this BFS level.
    if found:
        continue
    for i in range(len(current)):

        # Skip non-parenthesis characters.
        if current[i] != '(' and current[i] != ')':
            continue
        next_str = current[:i] + current[i+1:]
        if next_str not in visited:
            visited.add(next_str)
            queue.append(next_str)

return result

# Example usage:

# print(removeInvalidParentheses("()())()"))

```

Round 6 · Mid · Hard

p.2575

Backtracking

□ #980 Unique Paths III

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Backtracking · Bit Manipulation · Matrix
 Lists: AlgoMaster 600

Problem

You are given an $m \times n$ integer array grid where $grid[i][j]$ could be:

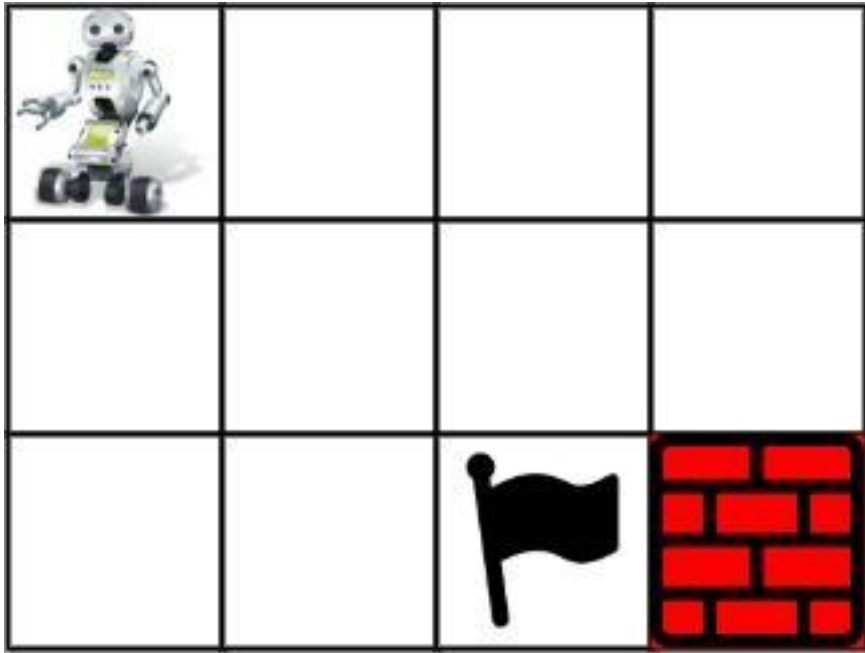
- 1 representing the starting square. There is exactly one starting square.
- 2 representing the ending square. There is exactly one ending square.
- 0 representing empty squares we can walk over.
- -1 representing obstacles that we cannot walk over.

Return the number of 4-directional walks from the starting square to the ending square, that walk over every non-obstacle square exactly once.

Example 1:

Round 6 · Mid · Hard

p.2576



Input: grid = [[1,0,0,0],[0,0,0,0],[0,0,2,-1]]

Output: 2

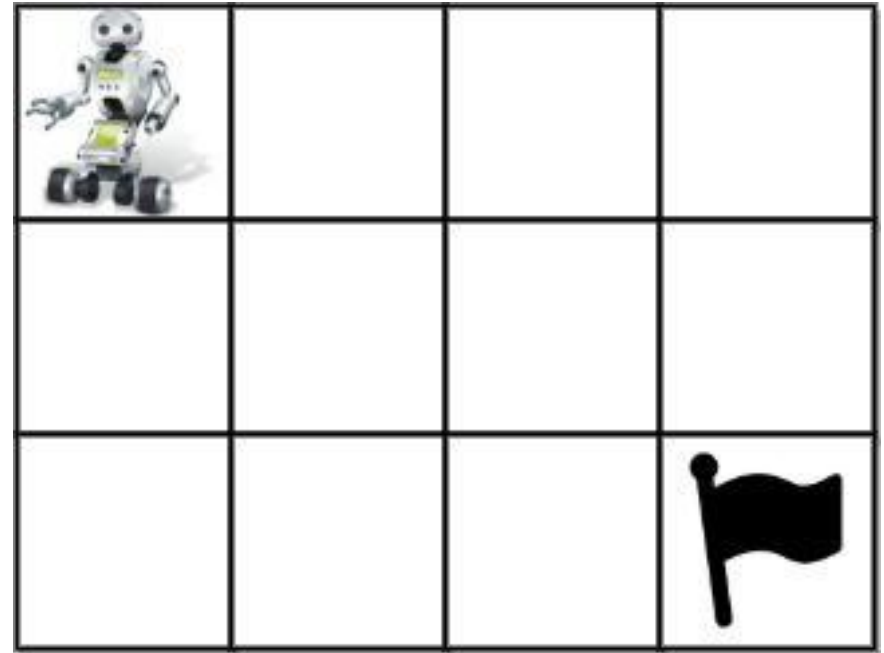
Explanation: We have the following two paths:

1. (0,0),(0,1),(0,2),(0,3),(1,3),(1,2),(1,1),(1,0),(2,0),(2,1),(2,2)
2. (0,0),(1,0),(2,0),(2,1),(1,1),(0,1),(0,2),(0,3),(1,3),(1,2),(2,2)

Example 2:

Round 6 · Mid · Hard

p.2577



Input: grid = [[1,0,0,0],[0,0,0,0],[0,0,0,2]]

Output: 4

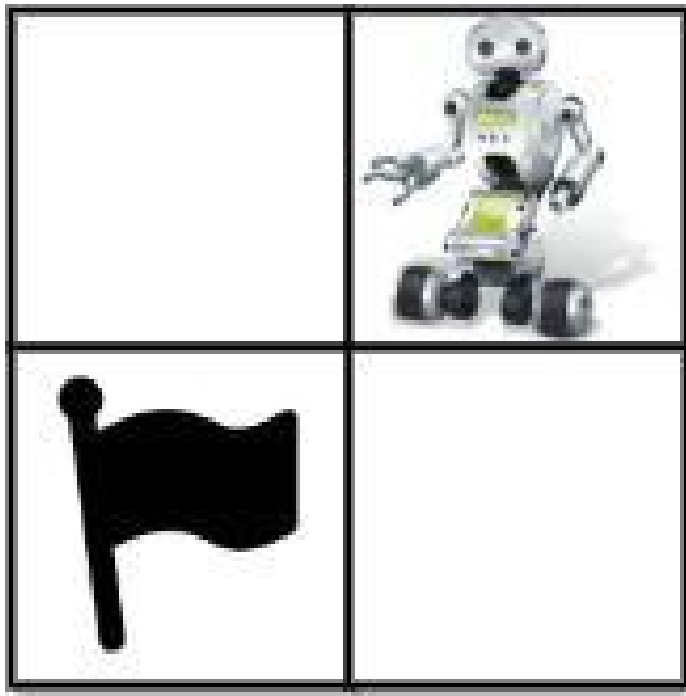
Explanation: We have the following four paths:

1. (0,0),(0,1),(0,2),(0,3),(1,3),(1,2),(1,1),(1,0),(2,0),(2,1),(2,2),(2,3)
2. (0,0),(0,1),(1,1),(1,0),(2,0),(2,1),(2,2),(1,2),(0,2),(0,3),(1,3),(2,3)
3. (0,0),(1,0),(2,0),(2,1),(2,2),(1,2),(1,1),(0,1),(0,2),(0,3),(1,3),(2,3)
4. (0,0),(1,0),(2,0),(2,1),(1,1),(0,1),(0,2),(0,3),(1,3),(1,2),(2,2),(2,3)

Example 3:

Round 6 · Mid · Hard

p.2578



Input: grid = [[0,1],[2,0]]

Output: 0

Explanation: There is no path that walks over every empty square exactly once. Note that the starting and ending square can be anywhere in the grid.

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq m, n \leq 20$
- $1 \leq m * n \leq 20$
- $-1 \leq \text{grid}[i][j] \leq 2$

Round 6 · Mid · Hard

p.2579

- There is exactly one starting cell and one ending cell.

Community Solutions (Python)

• LeetCode

```
class Solution:
    def uniquePathsIII(self, grid: List[List[int]]) -> int:
        def dfs(i: int, j: int, k: int) -> int:
            if grid[i][j] == 2:
                return int(k == cnt + 1)
            ans = 0
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                if 0 <= x < m and 0 <= y < n and (x, y) not in vis and grid[x]
                [y] != -1:
                    vis.add((x, y))
                    ans += dfs(x, y, k + 1)
                    vis.remove((x, y))
            return ans

        m, n = len(grid), len(grid[0])
        start = next((i, j) for i in range(m) for j in range(n) if grid[i][j]
        == 1)
        dirs = (-1, 0, 1, 0, -1)
        cnt = sum(row.count(0) for row in grid)
        vis = {start}
        return dfs(*start, 0)
```

• SimplyLeet

Round 6 · Mid · Hard

p.2580

```
def uniquePathsIII(grid):
    # Get the dimensions of the grid
    rows, cols = len(grid), len(grid[0])
    empty = 0
    start = None

    # Count the number of non-obstacle cells and locate the starting cell
    for i in range(rows):
        for j in range(cols):
            if grid[i][j] != -1:

                empty += 1
                if grid[i][j] == 1:
                    start = (i, j)

    paths = 0
    # DFS helper to explore the grid
    def dfs(i, j, count):
        nonlocal paths
        # When the ending cell is reached, check if all non-obstacle cells are
        visited
        if grid[i][j] == 2:

            if count == empty:
                paths += 1
            return

        # Mark the current cell as visited by setting it as an obstacle
        temp = grid[i][j]
        grid[i][j] = -1

        # Explore all four directions
        for di, dj in [(1,0), (-1,0), (0,1), (0,-1)]:
```

Round 6 · Mid · Hard

p.2581

```
        new_i, new_j = i + di, j + dj
        if 0 <= new_i < rows and 0 <= new_j < cols and grid[new_i][new_j]
        != -1:

            dfs(new_i, new_j, count + 1)

        # Backtrack: restore the original cell value
        grid[i][j] = temp

    # Start the DFS from the starting point with a count of 1
    dfs(start[0], start[1], 1)
    return paths

# Example usage:
if __name__ == '__main__':
    grid = [[1,0,0,0],[0,0,0,0],[0,0,2,-1]]
    print(uniquePathsIII(grid)) # Expected output: 2
```

Round 6 · Mid · Hard

p.2582

Tries

Round 6 · Mid · Hard · 5 questions

Round 6 · Mid · Hard

p.2583

Tries

□ #212 Word Search II

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · String · Backtracking · Trie · Matrix
Lists: Grind 169

Problem

Given an $m \times n$ board of characters and a list of strings words, return all words on the board. Each word must be constructed from letters of sequentially adjacent cells, where adjacent cells are horizontally or vertically neighboring. The same letter cell may not be used more than once in a word.

Example 1:

Round 6 · Mid · Hard

p.2584

o	a	a	n
e	t	a	e
i	h	k	r
i	f	l	v

Input: board =
 [["o","a","a","n"],["e","t","a","e"],["i","h","k","r"],["i","f","l","v"]],
 words = ["oath","pea","eat","rain"]
 Output: ["eat","oath"]

Example 2:

Round 6 · Mid · Hard

p.2585

a	b
c	d

Input: board = [["a","b"],["c","d"]], words = ["abcb"]
 Output: []

Constraints:

- $m == \text{board.length}$
- $n == \text{board}[i].\text{length}$
- $1 \leq m, n \leq 12$
- $\text{board}[i][j]$ is a lowercase English letter.
- $1 \leq \text{words.length} \leq 3 * 10^4$
- $1 \leq \text{words}[i].\text{length} \leq 10$

Round 6 · Mid · Hard

p.2586

- words[i] consists of lowercase English letters.
- All the strings of words are unique.

Community Solutions (Python)

• WalkCC

```
class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.word: str | None = None

class Solution:
    def findWords(self, board: list[list[str]], words: list[str]) -> list[str]:
        m = len(board)
        n = len(board[0])

        ans = []
        root = TrieNode()

        def insert(word: str) -> None:
            node = root
            for c in word:
                node = node.children.setdefault(c, TrieNode())
            node.word = word

        for word in words:
            insert(word)

        def dfs(i: int, j: int, node: TrieNode) -> None:
            if i < 0 or i == m or j < 0 or j == n:
                return
            if board[i][j] == '*':
                return

            c = board[i][j]
            if c not in node.children:
```

```
        return

        child = node.children[c]
        if child.word:
            ans.append(child.word)
            child.word = None

        board[i][j] = '*'
        dfs(i + 1, j, child)
        dfs(i - 1, j, child)

        dfs(i, j + 1, child)
        dfs(i, j - 1, child)
        board[i][j] = c

        for i in range(m):
            for j in range(n):
                dfs(i, j, root)

        return ans
```

• SimplyLeet

```
# Define a TrieNode for building the Trie.
class TrieNode:
    def __init__(self):
        self.children = {}
        self.word = None # Stores complete word at the end node

class Solution:
    def findWords(self, board, words):
        # Build the Trie from the list of words.
        root = TrieNode()
```

```

for word in words:
    node = root
    for char in word:
        if char not in node.children:
            node.children[char] = TrieNode()
        node = node.children[char]
    node.word = word # Mark the end of a word

```

```

result = []
m, n = len(board), len(board[0])

```

```

# Define the DFS helper function.
def dfs(i, j, node):
    char = board[i][j]
    if char not in node.children:
        return
    next_node = node.children[char]
    if next_node.word:
        result.append(next_node.word)
        next_node.word = None # Avoid duplicate entries

```

```

board[i][j] = '#' # Mark the current cell as visited
# Explore all four adjacent directions.
for dx, dy in ((0, 1), (1, 0), (0, -1), (-1, 0)):
    ni, nj = i + dx, j + dy
    if 0 <= ni < m and 0 <= nj < n and board[ni][nj] != '#':
        dfs(ni, nj, next_node)
    board[i][j] = char # Restore the original character

```

```

# Start DFS from each cell.

```

```

for i in range(m):
    for j in range(n):
        dfs(i, j, root)
return result

```

• LC.ca

Round 6 · Mid · Hard

p.2589

```

class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.w = ''
        # minor ajust, not a boolean is_end, but the whole word
        # so it's easier for dfs to save the word

```

```

def insert(self, w):
    node = self
    for c in w:

```

```

        idx = ord(c) - ord('a')
        if node.children[idx] is None:
            node.children[idx] = Trie()
        node = node.children[idx]
    node.w = w

```

```

class Solution:
    def findWords(self, board: List[List[str]], words: List[str]) -> List[str]
    :
        def dfs(node, i, j):

```

```

            idx = ord(board[i][j]) - ord('a')
            if node.children[idx] is None:
                return
            node = node.children[idx]
            if node.w:
                ans.add(node.w)
            c = board[i][j]
            board[i][j] = '0' # 0 for visited already
            for a, b in [[0, -1], [0, 1], [1, 0], [-1, 0]]:
                x, y = i + a, j + b

```

Round 6 · Mid · Hard

p.2590

```

        if 0 <= x < m and 0 <= y < n and board[x][y] != '0':
            dfs(node, x, y)
        board[i][y] = c

```

```

trie = Trie()
for w in words:
    trie.insert(w)
ans = set()
m, n = len(board), len(board[0])
for i in range(m):

```

```

    for j in range(n):
        dfs(trie, i, j)
return list(ans)

```

```
#####
```

```

class Trie:
    def __init__(self):

```

```

        self.children: List[Trie | None] = [None] * 26
        self.ref: int = -1

```

```

    def insert(self, w: str, ref: int):
        node = self
        for c in w:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]

```

```

node.ref = ref

```

```

class Solution:
    def findWords(self, board: List[List[str]], words: List[str]) -> List[str]
:

```

```

        def dfs(node: Trie, i: int, j: int):
            idx = ord(board[i][j]) - ord('a')
            if node.children[idx] is None:
                return
            node = node.children[idx]

```

```

        if node.ref >= 0:
            ans.append(words[node.ref])
            node.ref = -1
        c = board[i][j]
        board[i][j] = '#'
        for a, b in pairwise((-1, 0, 1, 0, -1)):
            x, y = i + a, j + b
            if 0 <= x < m and 0 <= y < n and board[x][y] != '#':
                dfs(node, x, y)
        board[i][j] = c

```

```

tree = Trie()
for i, w in enumerate(words):
    tree.insert(w, i)
m, n = len(board), len(board[0])
ans = []
for i in range(m):
    for j in range(n):
        dfs(tree, i, j)
return ans

```

◆ Quick Review

Key Insights

- Use Depth-First Search (DFS) to explore possible words starting from each board cell.

- Build a Trie (prefix tree) to store the words for $O(1)$ prefix checks and efficient pruning.
- Mark visited cells and restore them after DFS to avoid revisiting in the current search path.
- Prune search paths early if the current prefix is not part of any word in the Trie.

Space & Time

Time Complexity: $O(m * n * 4^L)$ in the worst case, where L is the maximum word length; however, using a Trie prunes unnecessary paths.

Space Complexity: $O(K)$ for the Trie storage, where K is the total number of letters in all words, plus $O(m * n)$ auxiliary space for recursion and board marking.

Solution

We first insert all words into a Trie data structure to allow quick lookups of prefixes. Then, for each cell in the board, we perform DFS. During DFS, if the current path matches a Trie node that represents a complete word, we add that word to the result list and mark it as found to avoid duplicates. We continue exploring adjacent cells (up, down, left, right)

while marking the cell as visited. After exploring, we restore the original letter for further DFS calls. This combination of Trie and DFS with backtracking efficiently finds valid words on the board.

Tries

#336 Palindrome Pairs

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table · String · Trie
 Lists: Grind 169

Problem

You are given a 0-indexed array of unique strings words.

A palindrome pair is a pair of integers (i, j) such that:

- $0 \leq i, j < \text{words.length}$,
- $i \neq j$, and
- $\text{words}[i] + \text{words}[j]$ (the concatenation of the two strings) is a palindrome.

Return an array of all the palindrome pairs of words.

You must write an algorithm with $O(\text{sum of words}[i].\text{length})$ runtime complexity.

Example 1:

Input: words = ["abcd","dcba","lls","s","sssll"]
 Output: [[0,1],[1,0],[3,2],[2,4]]
 Explanation: The palindromes are ["abccddcba","dcbaabcd","slls","llsllsll"]

Round 6 · Mid · Hard

p.2595

Example 2:

Input: words = ["bat","tab","cat"]
 Output: [[0,1],[1,0]]
 Explanation: The palindromes are ["battab","tabbat"]

Example 3:

Input: words = ["a",""]
 Output: [[0,1],[1,0]]
 Explanation: The palindromes are ["a","a"]

Constraints:

- $1 \leq \text{words.length} \leq 5000$
- $0 \leq \text{words}[i].\text{length} \leq 300$
- $\text{words}[i]$ consists of lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def palindromePairs(self, words: list[str]) -> list[list[int]]:
        ans = []
        dict = {word[::-1]: i for i, word in enumerate(words)}

        for i, word in enumerate(words):
            if "" in dict and dict[""] != i and word == word[::-1]:
                ans.append([i, dict[""]])

            for j in range(1, len(word) + 1):
```

Round 6 · Mid · Hard

p.2596

```

l = word[:j]
r = word[j:]
if l in dict and dict[l] != i and r == r[::-1]:
    ans.append([i, dict[l]])
if r in dict and dict[r] != i and l == l[::-1]:
    ans.append([dict[r], i])

return ans

```

• LeetCode

```

class Solution:
    def palindromePairs(self, words: List[str]) -> List[List[int]]:
        d = {w: i for i, w in enumerate(words)}
        ans = []
        for i, w in enumerate(words):
            for j in range(len(w) + 1):
                a, b = w[:j], w[j:]
                ra, rb = a[::-1], b[::-1]
                if ra in d and d[ra] != i and b == rb:
                    ans.append([i, d[ra]])

            if j and rb in d and d[rb] != i and a == ra:
                ans.append([d[rb], i])

        return ans

```

• SimplyLeet

```

# Python implementation for Palindrome Pairs

def is_palindrome(s):
    # Helper function to check if a string is a palindrome
    return s == s[::-1]

def palindromePairs(words):
    result = []
    # Create a dictionary mapping each word to its index
    word_dict = {word: i for i, word in enumerate(words)}

```

Round 6 · Mid · Hard

p.2597

```

# Iterate over each word with its index
for i, word in enumerate(words):
    # For each possible split position (including edge empty string splits)
    for j in range(len(word) + 1):
        prefix = word[:j]
        suffix = word[j:]

        # If prefix is palindrome, check for reversed suffix in dictionary
        if is_palindrome(prefix):

            reversed_suffix = suffix[::-1]
            # Ensure that the reversed_suffix exists and is not the same
            word

            if reversed_suffix in word_dict and word_dict[reversed_suffix]
            != i:

                result.append([word_dict[reversed_suffix], i])

        # Check the suffix; avoid duplicate pairs when j equals len(word)
        because

        # the case when suffix is empty was handled in the first condition
        if j != len(word) and is_palindrome(suffix):
            reversed_prefix = prefix[::-1]
            if reversed_prefix in word_dict and word_dict[reversed_prefix]
            != i:

                result.append([i, word_dict[reversed_prefix]])

        return result

```

• LC.ca

Round 6 · Mid · Hard

p.2598

```
class Solution:
    def palindromePairs(self, words: List[str]) -> List[List[int]]:
        d = {w: i for i, w in enumerate(words)}
        ans = []
        for i, w in enumerate(words):
            for j in range(len(w) + 1):
                a, b = w[:j], w[j:]
                ra, rb = a[::-1], b[::-1]
                if ra in d and d[ra] != i and b == rb:
                    ans.append([i, d[ra]])

            if j and rb in d and d[rb] != i and a == ra:
                ans.append([d[rb], i])
        return ans
```

◆ Quick Review

Key Insights

- A palindrome is a string that reads the same forwards and backwards.
- For any given word, if you split it into two parts, one part (either prefix or suffix) might be a palindrome.
- If the other part's reverse exists elsewhere in the array, then that reverse can be concatenated to yield a palindrome.
- Carefully handle edge cases like empty strings, since an empty string is a palindrome by default.

Space & Time

Round 6 · Mid · Hard

p.2599

Time Complexity: $O(n * k^2)$ in the worst case where n is the number of words and k is the maximum word length, but with the palindrome checking optimized this can be considered $O(\text{sum of word lengths})$: Each word is processed once and each split is checked in $O(k)$ time.

Space Complexity: $O(n * k)$ due to the hashmap storing all words and their indices.

Solution

We use a hashmap (dictionary) to store each word and its corresponding index. For every word, we iterate through every possible split position to divide the word into a prefix and a suffix. For each split:

- If the prefix is a palindrome, we look for the reverse of the suffix in the hashmap. If found (and not the same word), then combining the reverse with the current word forms a palindrome.
- Similarly, if the suffix is a palindrome, we look for the reverse of the prefix.

We must be cautious with edge cases, especially when either part is empty, as the

Round 6 · Mid · Hard

p.2600

empty string is considered a palindrome. This method avoids checking every possible pair explicitly and leverages string reversal and palindrome checks to achieve efficiency.

Tries

☐ #425 Word Squares

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · String · Backtracking · Trie
 Lists: AlgoMaster 600

Problem

Given an array of unique strings words, return all the word squares you can build from words. The same word from words can be used multiple times. You can return the answer in any order.

A sequence of strings forms a valid word square if the kth row and column read the same string, where $0 \leq k < \max(\text{numRows}, \text{numColumns})$.

- For example, the word sequence ["ball", "area", "lead", "lady"] forms a word square because each word reads the same both horizontally and vertically.

Example 1:

Input: words = ["area","lead","wall","lady","ball"]
 Output: [["ball","area","lead","lady"],["wall","area","lead","lady"]]
 Explanation:
 The output consists of two word squares. The order of output does not matter (just the order of words in each word square matters).

Example 2:

Input: words = ["abat","baba","atan","atal"]
 Output: [["baba","abat","baba","atal"],["baba","abat","baba","atan"]]
 Explanation:
 The output consists of two word squares. The order of output does not matter (just the order of words in each word square matters).

Constraints:

- $1 \leq \text{words.length} \leq 1000$
- $1 \leq \text{words}[i].\text{length} \leq 4$
- All words[i] have the same length.
- words[i] consists of only lowercase English letters.
- All words[i] are unique.

Community Solutions (Python)

• WalkCC

```
class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.startsWith: list[str] = []

class Trie:
    def __init__(self, words: list[str]):
        self.root = TrieNode()
        for word in words:
```

```
        self._insert(word)

    def findBy(self, prefix: str) -> list[str]:
        node = self.root
        for c in prefix:
            if c not in node.children:
                return []
            node = node.children[c]
        return node.startsWith

    def _insert(self, word: str) -> None:
        node = self.root
        for c in word:
            node = node.children.setdefault(c, TrieNode())
            node.startsWith.append(word)

class Solution:
    def wordSquares(self, words: list[str]) -> list[list[str]]:
        if not words:
            return []

        n = len(words[0])
        ans = []
        path = []
        trie = Trie(words)

        for word in words:
            path.append(word)
            self._dfs(trie, n, path, ans)

            path.pop()

        return ans

    def _dfs(self, trie: Trie, n: int, path: list[str], ans: list[list[str]]):
        if len(path) == n:
            ans.append(path.copy())
            return

        prefix = self._getPrefix(path)
```

```

for s in trie.findBy(prefix):
    path.append(s)
    self._dfs(trie, n, path, ans)
    path.pop()

```

```

def _getPrefix(self, path: list[str]) -> str:
    """
    e.g. path = ["wall",
                 "area"]
    """

```

```

    prefix = "le.."
    """

```

```

    prefix = []
    index = len(path)
    for s in path:
        prefix.append(s[index])
    return ''.join(prefix)

```

• LeetDoocs

```

class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.v = []

    def insert(self, w, i):
        node = self
        for c in w:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:

```

```

                node.children[idx] = Trie()
            node = node.children[idx]
            node.v.append(i)

```

```

    def search(self, w):
        node = self
        for c in w:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                return []

```

Round 6 · Mid · Hard

p.2605

```

        node = node.children[idx]
        return node.v

```

```

class Solution:
    def wordSquares(self, words: List[str]) -> List[List[str]]:
        def dfs(t):
            if len(t) == len(words[0]):
                ans.append(t[:])
                return

```

```

            idx = len(t)
            pref = [v[idx] for v in t]
            indexes = trie.search(''.join(pref))
            for i in indexes:
                t.append(words[i])
                dfs(t)
                t.pop()

```

```

        trie = Trie()
        ans = []

```

```

        for i, w in enumerate(words):
            trie.insert(w, i)
        for w in words:
            dfs([w])
        return ans

```

• SimplyLeet

Python solution for Word Squares

```

class WordSquares:
    def __init__(self, words):
        self.N = len(words[0])
        self.words = words
        self.prefix_map = {}
        # Build prefix mapping: for every prefix, map to list of words with
        # that prefix.
        for word in words:
            for i in range(len(word) + 1):

```

Round 6 · Mid · Hard

p.2606

```

        prefix = word[:i]
        if prefix in self.prefix_map:
            self.prefix_map[prefix].append(word)
        else:
            self.prefix_map[prefix] = [word]

```

```

def find_word_squares(self):
    results = []
    square = []
    # Begin backtracking starting with each word.

```

```

    for word in self.words:
        square = [word]
        self.backtrack(1, square, results)
    return results

```

```

def backtrack(self, step, square, results):
    # If the current square is complete, add a copy to results
    if step == self.N:
        results.append(list(square))
        return

```

```

    # Build the prefix for the next word in the square using the kth letter
    # of each current word.
    prefix = ''.join([word[step] for word in square])
    for candidate in self.prefix_map.get(prefix, []):
        square.append(candidate)
        self.backtrack(step + 1, square, results)
    square.pop() # backtrack if no valid square found

```

```

# Example usage:
# words = ["area", "lead", "wall", "lady", "ball"]

```

```

# ws = WordSquares(words)
# print(ws.find_word_squares())

```

Tries

□ #472 Concatenated Words

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · String · Dynamic Programming ·
 Depth-First Search · Trie · Sorting
 Lists: AlgoMaster 600

Problem

Given an array of strings words (without duplicates), return all the concatenated words in the given list of words.

A concatenated word is defined as a string that is comprised entirely of at least two shorter words (not necessarily distinct) in the given array.

Example 1:

```

Input: words = ["cat","cats","catsdogcats","dog","dogcatsdog","hippopotamuses",
               ,"rat","ratcatdogcat"]
Output: ["catsdogcats","dogcatsdog","ratcatdogcat"]
Explanation: "catsdogcats" can be concatenated by "cats", "dog" and "cats";
"dogcatsdog" can be concatenated by "dog", "cats" and "dog";
"ratcatdogcat" can be concatenated by "rat", "cat", "dog" and "cat".

```

Example 2:

```

Input: words = ["cat","dog","catdog"]
Output: ["catdog"]

```

Constraints:

- $1 \leq \text{words.length} \leq 104$
- $1 \leq \text{words}[i].\text{length} \leq 30$
- $\text{words}[i]$ consists of only lowercase English letters.
- All the strings of words are unique.
- $1 \leq \text{sum}(\text{words}[i].\text{length}) \leq 105$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findAllConcatenatedWordsInADict(self, words: list[str]) -> list[str]:
        wordSet = set(words)

        @functools.lru_cache(None)
        def isConcat(word: str) -> bool:
            for i in range(1, len(word)):
                prefix = word[:i]
                suffix = word[i:]
                if prefix in wordSet and (suffix in wordSet or isConcat(suffix)):

                    return True

            return False

        return [word for word in words if isConcat(word)]
```

• LeetDoocs

```
class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.is_end = False

    def insert(self, w):
        node = self
        for c in w:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
        node.is_end = True

class Solution:
    def findAllConcatenatedWordsInADict(self, words: List[str]) -> List[str]:
        def dfs(w):
            if not w:
                return True

            node = trie
            for i, c in enumerate(w):
                idx = ord(c) - ord('a')
                if node.children[idx] is None:
                    return False
                node = node.children[idx]
                if node.is_end and dfs(w[i + 1 :]):
                    return True
            return False

        trie = Trie()
        ans = []
        words.sort(key=lambda x: len(x))
        for w in words:
            if dfs(w):
                ans.append(w)
            else:
                trie.insert(w)
        return ans
```

• SimplyLeet

```
def findAllConcatenatedWordsInADict(words):
    # Sort the words by length to ensure that smaller words are processed
    # first.
    words.sort(key=len)
    concatenated = []
    word_set = set()

    # Function to check if the word can be formed by words in the set.
    def can_form(word):
        if not word:
            return False

        dp = [False] * (len(word) + 1)
        dp[0] = True
        # Try all possible splits.
        for i in range(1, len(word) + 1):
            # Limit inner loop based on maximum word length of 30.
            for j in range(max(0, i - 30), i):
                if dp[j] and word[j:i] in word_set:
                    dp[i] = True
                    break
        return dp[len(word)]

    # Process each word.
    for word in words:
        if not word:
            continue
        if can_form(word):
            concatenated.append(word)
            word_set.add(word)

    return concatenated

# Example usage:
words = ["cat", "cats", "catsdogcats", "dog", "dogcatsdog", "hippopotamuses",
        "rat", "ratcatdogcat"]
print(findAllConcatenatedWordsInADict(words))
```

Round 6 · Mid · Hard

p.2611

Tries

□ #1032 Stream of Characters

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · String · Design · Trie · Data Stream
 Lists: AlgoMaster 600

Problem

Design an algorithm that accepts a stream of characters and checks if a suffix of these characters is a string of a given array of strings words.

For example, if words = ["abc", "xyz"] and the stream added the four characters (one by one) 'a', 'x', 'y', and 'z', your algorithm should detect that the suffix "xyz" of the characters "axyz" matches "xyz" from words.

Implement the StreamChecker class:

- StreamChecker(String[] words) Initializes the object with the strings array words.
- boolean query(char letter) Accepts a new character from the stream and returns true if any non-empty suffix from the stream forms

Round 6 · Mid · Hard

p.2612

a word that is in words.

Example 1:

Input
 ["StreamChecker", "query", "query", "query", "query", "query", "query",
 "query", "query", "query", "query", "query", "query"]
 [["cd", "f", "kl"], ["a"], ["b"], ["c"], ["d"], ["e"], ["f"], ["g"], ["h"],
 ["i"], ["j"], ["k"], ["l"]]
Output
 [null, false, false, false, true, false, true, false, false, false, false,
 false, true]

Explanation
 StreamChecker streamChecker = new StreamChecker(["cd", "f", "kl"]);

Constraints:

- $1 \leq \text{words.length} \leq 2000$
- $1 \leq \text{words}[i].\text{length} \leq 200$
- $\text{words}[i]$ consists of lowercase English letters.
- letter is a lowercase English letter.
- At most $4 * 10^4$ calls will be made to query.

Community Solutions (Python)

• WalkCC

```
class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.isWord = False

class StreamChecker:
    def __init__(self, words: list[str]):
        self.root = TrieNode()
        self.letters = []
```

Round 6 · Mid · Hard

p.2613

```
for word in words:
    self._insert(word)

def query(self, letter: str) -> bool:
    self.letters.append(letter)
    node = self.root
    for c in reversed(self.letters):
        if c not in node.children:
            return False
        node = node.children[c]

    if node.isWord:
        return True
    return False

def _insert(self, word: str) -> None:
    node = self.root
    for c in reversed(word):
        node = node.children.setdefault(c, TrieNode())
    node.isWord = True
```

• LeetDoocs

```
class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.is_end = False

    def insert(self, w: str):
        node = self
        for c in w[::-1]:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
```

Round 6 · Mid · Hard

p.2614

```

        node.children[idx] = Trie()
        node = node.children[idx]
        node.is_end = True

```

```

def search(self, w: List[str]) -> bool:
    node = self
    for c in w[::-1]:
        idx = ord(c) - ord('a')
        if node.children[idx] is None:
            return False

```

```

        node = node.children[idx]
        if node.is_end:
            return True
    return False

```

```

class StreamChecker:
    def __init__(self, words: List[str]):
        self.trie = Trie()
        self.cs = []

```

```

        self.limit = 201
        for w in words:
            self.trie.insert(w)

    def query(self, letter: str) -> bool:
        self.cs.append(letter)
        return self.trie.search(self.cs[-self.limit :])

```

Your StreamChecker object will be instantiated and called as such:

```

# obj = StreamChecker(words)
# param_1 = obj.query(letter)

```

• SimplyLeet

Round 6 · Mid · Hard

p.2615

```

# Python implementation of StreamChecker
class StreamChecker:
    def __init__(self, words: List[str]):
        # Build a reverse Trie, where each node is a dictionary
        self.trie = {}
        self.max_length = 0 # maximum length of a word in words
        for word in words:
            self.max_length = max(self.max_length, len(word))
            node = self.trie
            # Insert word in reverse order

```

```

            for char in reversed(word):
                if char not in node:
                    node[char] = {}
                node = node[char]
            # Mark the end of a word
            node['#'] = True
        self.stream = [] # buffer to store characters from query

```

```

    def query(self, letter: str) -> bool:
        # Append the new letter to our stream buffer

```

```

        self.stream.append(letter)
        # Keep the stream buffer size within the maximum word length
        if len(self.stream) > self.max_length:
            self.stream.pop(0)

        node = self.trie
        # Traverse the stream backwards (i.e., the most recent characters)
        for ch in reversed(self.stream):
            if ch not in node:
                return False # no further matching path in Trie

```

```

            node = node[ch]
            # Check if we have reached the end of a word
            if '#' in node:
                return True
        return False

```

```

# Example usage:
# streamChecker = StreamChecker(["cd", "f", "kl"])
# print(streamChecker.query("a"))
# print(streamChecker.query("b"))

```

Round 6 · Mid · Hard

p.2616


```
# print(streamChecker.query("c"))
# print(streamChecker.query("d"))
```

Heaps

Round 6 · Mid · Hard · 1 question

Round 6 · Mid · Hard

p.2617

Round 6 · Mid · Hard

p.2618

Heaps

☐ #1383 Maximum Performance of a Team **HAR**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Greedy · Sorting · Heap (Priority Queue)
Lists: AlgoMaster 600

Problem

You are given two integers n and k and two integer arrays `speed` and `efficiency` both of length n . There are n engineers numbered from 1 to n . `speed[i]` and `efficiency[i]` represent the speed and efficiency of the i th engineer respectively.

Choose at most k different engineers out of the n engineers to form a team with the maximum performance.

The performance of a team is the sum of its engineers' speeds multiplied by the minimum efficiency among its engineers.

Return the maximum performance of this team. Since the answer can be a huge number, return it modulo $10^9 + 7$.

Round 6 · Mid · Hard

p.2619

Example 1:

Input: $n = 6$, `speed` = [2,10,3,1,5,8], `efficiency` = [5,4,3,9,7,2], $k = 2$
Output: 60
Explanation:
We have the maximum performance of the team by selecting engineer 2 (with speed=10 and efficiency=4) and engineer 5 (with speed=5 and efficiency=7). That is, $\text{performance} = (10 + 5) * \min(4, 7) = 60$.

Example 2:

Input: $n = 6$, `speed` = [2,10,3,1,5,8], `efficiency` = [5,4,3,9,7,2], $k = 3$
Output: 68
Explanation:
This is the same example as the first but $k = 3$. We can select engineer 1, engineer 2 and engineer 5 to get the maximum performance of the team. That is, $\text{performance} = (2 + 10 + 5) * \min(5, 4, 7) = 68$.

Example 3:

Input: $n = 6$, `speed` = [2,10,3,1,5,8], `efficiency` = [5,4,3,9,7,2], $k = 4$
Output: 72

Constraints:

- $1 \leq k \leq n \leq 105$
- `speed.length` == n
- `efficiency.length` == n
- $1 \leq \text{speed}[i] \leq 105$
- $1 \leq \text{efficiency}[i] \leq 108$

Community Solutions (Python)

- WalkCC

Round 6 · Mid · Hard

p.2620

```
class Solution:
    # Similar to 857. Minimum Cost to Hire K Workers
    def maxPerformance(
        self,
        n: int,
        speed: List[int],
        efficiency: List[int],
        k: int,
    ) -> int:
        MOD = 1_000_000_007

        ans = 0
        speedSum = 0
        # (efficiency[i], speed[i]) sorted by efficiency[i] in descending order
        A = sorted([(e, s) for s, e in zip(speed, efficiency)], reverse=True)
        minHeap = []

        for e, s in A:
            heapq.heappush(minHeap, s)
            speedSum += s
            if len(minHeap) > k:

                speedSum -= heapq.heappop(minHeap)
                ans = max(ans, speedSum * e)

        return ans % MOD
```

• LeetCode

```
class Solution:
    def maxPerformance(
        self, n: int, speed: List[int], efficiency: List[int], k: int
    ) -> int:
        t = sorted(zip(speed, efficiency), key=lambda x: -x[1])
        ans = tot = 0
        mod = 10**9 + 7
        h = []
        for s, e in t:
            tot += s
```

Round 6 · Mid · Hard

p.2621

```
ans = max(ans, tot * e)
heappush(h, s)
if len(h) == k:
    tot -= heappop(h)
return ans % mod
```

• SimplyLeet

```
import heapq

def maxPerformance(n, speed, efficiency, k):
    MOD = 10**9 + 7
    # Create pairs of (efficiency, speed) and sort by efficiency descending
    engineers = sorted(zip(efficiency, speed), key=lambda x: -x[0])

    max_perf = 0      # Store maximum performance encountered
    speed_sum = 0     # Running sum of speeds in the current team
    speed_heap = []   # Min-heap for the speeds in the team

    # Iterate through each engineer sorted by descending efficiency
    for curr_eff, curr_speed in engineers:
        # Add current engineer's speed to the heap
        heapq.heappush(speed_heap, curr_speed)
        speed_sum += curr_speed

        # If there are more than k engineers, remove the one with the smallest
        speed

        if len(speed_heap) > k:
            speed_sum -= heapq.heappop(speed_heap)

        # Calculate team performance with current engineer as minimum
        efficiency
        curr_perf = speed_sum * curr_eff
        max_perf = max(max_perf, curr_perf)

    return max_perf % MOD

# Example usage:
print(maxPerformance(6, [2,10,3,1,5,8], [5,4,3,9,7,2], 2))
```

Round 6 · Mid · Hard

p.2622

Two Heaps

Round 6 · Mid · Hard · 3 questions

Round 6 · Mid · Hard

p.2623

Two Heaps

□ #295 Find Median from Data Stream

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Two Pointers · Design · Sorting · Heap
 Lists: Grind 169

Problem

The median is the middle value in an ordered integer list. If the size of the list is even, there is no middle value, and the median is the mean of the two middle values.

- For example, for $arr = [2,3,4]$, the median is 3.
- For example, for $arr = [2,3]$, the median is $(2 + 3) / 2 = 2.5$.

Implement the MedianFinder class:

- `MedianFinder()` initializes the `MedianFinder` object.
- `void addNum(int num)` adds the integer `num` from the data stream to the data structure.
- `double findMedian()` returns the median of all elements so far. Answers within 10^{-5} of

Round 6 · Mid · Hard

p.2624

the actual answer will be accepted.

Example 1:

Input
 ["MedianFinder", "addNum", "addNum", "findMedian", "addNum", "findMedian"]
 [[], [1], [2], [], [3], []]
Output
 [null, null, null, 1.5, null, 2.0]

Explanation
 MedianFinder medianFinder = new MedianFinder();

Constraints:

- $-105 \leq \text{num} \leq 105$
- There will be at least one element in the data structure before calling findMedian.
- At most $5 * 10^4$ calls will be made to addNum and findMedian.

Follow up:

- If all integer numbers from the stream are in the range $[0, 100]$, how would you optimize your solution?
- If 99% of all integer numbers from the stream are in the range $[0, 100]$, how would you optimize your solution?

Community Solutions (Python)

- WalkCC

```
class MedianFinder:
    def __init__(self):
        self.maxHeap = []
        self.minHeap = []

    def addNum(self, num: int) -> None:
        if not self.maxHeap or num <= -self.maxHeap[0]:
            heapq.heappush(self.maxHeap, -num)
        else:
            heapq.heappush(self.minHeap, num)

        # Balance the two heaps s.t.
        # |maxHeap| >= |minHeap| and |maxHeap| - |minHeap| <= 1.
        if len(self.maxHeap) < len(self.minHeap):
            heapq.heappush(self.maxHeap, -heapq.heappop(self.minHeap))
        elif len(self.maxHeap) - len(self.minHeap) > 1:
            heapq.heappush(self.minHeap, -heapq.heappop(self.maxHeap))

    def findMedian(self) -> float:
        if len(self.maxHeap) == len(self.minHeap):
            return (-self.maxHeap[0] + self.minHeap[0]) / 2.0
        return -self.maxHeap[0]
```

• LeetCode

```
class MedianFinder:
    def __init__(self):
        self.minq = []
        self.maxq = []

    def addNum(self, num: int) -> None:
        heappush(self.minq, -heappushpop(self.maxq, -num))
        if len(self.minq) - len(self.maxq) > 1:
            heappush(self.maxq, -heappop(self.minq))
```

```
def findMedian(self) -> float:
    if len(self.minq) == len(self.maxq):
        return (self.minq[0] + self.maxq[0]) / 2
    return self.minq[0]
```

```
# Your MedianFinder object will be instantiated and called as such:
# obj = MedianFinder()
# obj.addNum(num)
```

```
# param_2 = obj.findMedian()
```

• SimplyLeet

```
import heapq

class MedianFinder:
    def __init__(self):
        # max-heap for the lower half (store negatives to simulate max-heap)
        self.lowerHalf = []
        # min-heap for the upper half
        self.upperHalf = []

    def addNum(self, num: int) -> None:

        # Push to max-heap (simulate by pushing negative)
        heapq.heappush(self.lowerHalf, -num)
        # Ensure every element in lowerHalf is <= every element in upperHalf
        # Move maximum of lowerHalf to upperHalf
        heapq.heappush(self.upperHalf, -heapq.heappop(self.lowerHalf))

        # Balance the heaps: if upperHalf exceeds lowerHalf in size
        if len(self.upperHalf) > len(self.lowerHalf):
            heapq.heappush(self.lowerHalf, -heapq.heappop(self.upperHalf))
```

Round 6 · Mid · Hard

p.2627

```
def findMedian(self) -> float:
    # If both heaps are balanced, median is average of both heaps' top
    elements
    if len(self.lowerHalf) == len(self.upperHalf):
        return (-self.lowerHalf[0] + self.upperHalf[0]) / 2.0
    # Otherwise, median is the top element of the larger heap
    return -self.lowerHalf[0]

# Example usage:
# medianFinder = MedianFinder()
# medianFinder.addNum(1)

# medianFinder.addNum(2)
# print(medianFinder.findMedian()) # Outputs 1.5
# medianFinder.addNum(3)
# print(medianFinder.findMedian()) # Outputs 2.0
```

• LC.ca

```
from heapq import heappush, heappop

class MedianFinder:

    def __init__(self):
        # the smaller half of the list, max heap (invert min-heap)
        self.smallerHalf = []
        # the larger half of the list, min heap
        self.largerHalf = []

    def addNum(self, num: int) -> None:

        # trick for smaller half, use -1*val
        # heapq in python does NOT have comparator like in Java
        heappush(self.smallerHalf, -num)
        heappush(self.largerHalf, -heappop(self.smallerHalf)) # note: not
        self.smallerHalf.pop()

        if len(self.smallerHalf) < len(self.largerHalf):
            heappush(self.smallerHalf, -heappop(self.largerHalf))
```

Round 6 · Mid · Hard

p.2628

```
def findMedian(self) -> float:
    if len(self.smallerHalf) == len(self.largerHalf):
        return (-self.smallerHalf[0] + self.largerHalf[0]) / 2.0
    else:
        return float(-self.smallerHalf[0])
```

```
# Your MedianFinder object will be instantiated and called as such:
# obj = MedianFinder()
# obj.addNum(num)
```

```
# param_2 = obj.findMedian()
```

```
# follow up
```

```
class MedianFinder:
    def __init__(self):
        self.counts = [0] * 101
        self.total = 0
```

```
    def addNum(self, num: int) -> None:
```

```
        self.counts[num] += 1
        self.total += 1
```

```
    def findMedian(self) -> float:
        if self.total % 2 == 0:
            # even number of elements
            middle1 = self.total // 2
            middle2 = middle1 + 1
            count = 0
            i1 = i2 = 0
```

```
            for i in range(101):
                count += self.counts[i]
                if count >= middle1 and i1 == 0:
                    i1 = i
                if count >= middle2:
                    i2 = i
                    break
            return (i1 + i2) / 2
        else:
            # odd number of elements
```

```
        middle = self.total // 2 + 1
        count = 0
        for i in range(101):
            count += self.counts[i]
            if count >= middle:
                return i
```

```
#####
```

```
class MedianFinder:
    def __init__(self):
        """
        initialize your data structure here.
        """
        self.h1 = []
        self.h2 = []
```

```
    def addNum(self, num: int) -> None:
        heappush(self.h1, num)
```

```
        heappush(self.h2, -heappop(self.h1))
        if len(self.h2) - len(self.h1) > 1:
            heappush(self.h1, -heappop(self.h2))
```

```
    def findMedian(self) -> float:
        if len(self.h2) > len(self.h1):
            return -self.h2[0]
        return (self.h1[0] - self.h2[0]) / 2
```

```
# Your MedianFinder object will be instantiated and called as such:
```

```
# obj = MedianFinder()
# obj.addNum(num)
# param_2 = obj.findMedian()
```

◆ Quick Review

Key Insights

- Use two heaps: a max-heap for the lower half and a min-heap for the upper half of numbers.
- Balancing the heaps ensures the median can be computed in $O(1)$ time once numbers are added.
- Rebalance the heaps after each insertion to ensure size properties hold (difference in sizes is at most 1).

Space & Time

Time Complexity:

- addNum: $O(\log n)$ per insertion (heap operations).
- findMedian: $O(1)$ (accessing the top elements).

Space Complexity:

- $O(n)$, where n is the number of elements added in the data stream, as they are stored in the two heaps.

Solution

We maintain two heaps:

- A max-heap (lowerHalf) for the smaller half of the numbers.
- A min-heap (upperHalf) for

the larger half of the numbers. On inserting a new number, decide the heap membership based on the current median. After each insertion, ensure that the heaps have balanced sizes (the size difference is at most one). When computing the median: – If both heaps have the same number of elements, the median is the average of the max of lowerHalf and the min of upperHalf. – If one heap has more elements, the median is the top element of that heap.

This method makes it efficient to dynamically update and retrieve the median as new numbers are inserted.

Two Heaps

□ #480 Sliding Window Median

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Sliding Window · Heap
(Priority Queue)

Lists: AlgoMaster 600

Problem

The median is the middle value in an ordered integer list. If the size of the list is even, there is no middle value. So the median is the mean of the two middle values.

- For examples, if arr = [2,3,4], the median is 3.
- For examples, if arr = [1,2,3,4], the median is $(2 + 3) / 2 = 2.5$.

You are given an integer array nums and an integer k. There is a sliding window of size k which is moving from the very left of the array to the very right. You can only see the k numbers in the window. Each time the sliding window moves right by one position.

Round 6 · Mid · Hard

p.2633

Return the median array for each window in the original array. Answers within 10⁻⁵ of the actual value will be accepted.

Example 1:

```
Input: nums = [1,3,-1,-3,5,3,6,7], k = 3
Output: [1.00000,-1.00000,-1.00000,3.00000,5.00000,6.00000]
Explanation:
Window position Median
-----
[1 3 -1] -3 5 3 6 7 1
1 [3 -1 -3] 5 3 6 7 -1
1 3 [-1 -3 5] 3 6 7 -1
```

Example 2:

```
Input: nums = [1,2,3,4,2,3,1,4,2], k = 3
Output: [2.00000,3.00000,3.00000,3.00000,2.00000,3.00000,2.00000]
```

Constraints:

- $1 \leq k \leq \text{nums.length} \leq 105$
- $-231 \leq \text{nums}[i] \leq 231 - 1$

Community Solutions (Python)

• LeetDoocs

```
class MedianFinder:
    def __init__(self, k: int):
        self.k = k
        self.small = []
        self.large = []
        self.delayed = defaultdict(int)
        self.small_size = 0
        self.large_size = 0

    def add_num(self, num: int):
```

Round 6 · Mid · Hard

p.2634

```

    if not self.small or num <= -self.small[0]:
        heappush(self.small, -num)
        self.small_size += 1
    else:
        heappush(self.large, num)
        self.large_size += 1
    self.rebalance()

def find_median(self) -> float:
    return -self.small[0] if self.k & 1 else (-self.small[0] + self.large[0]) / 2

def remove_num(self, num: int):
    self.delayed[num] += 1
    if num <= -self.small[0]:
        self.small_size -= 1
        if num == -self.small[0]:
            self.prune(self.small)
    else:
        self.large_size -= 1
        if num == self.large[0]:
            self.prune(self.large)
    self.rebalance()

def prune(self, pq: List[int]):
    sign = -1 if pq is self.small else 1
    while pq and sign * pq[0] in self.delayed:
        self.delayed[sign * pq[0]] -= 1
        if self.delayed[sign * pq[0]] == 0:
            self.delayed.pop(sign * pq[0])
        heappop(pq)

```

```

def rebalance(self):
    if self.small_size > self.large_size + 1:
        heappush(self.large, -heappop(self.small))
        self.small_size -= 1
        self.large_size += 1
        self.prune(self.small)
    elif self.small_size < self.large_size:
        heappush(self.small, -heappop(self.large))
        self.large_size -= 1
        self.small_size += 1
        self.prune(self.large)

class Solution:
    def medianSlidingWindow(self, nums: List[int], k: int) -> List[float]:
        finder = MedianFinder(k)
        for x in nums[:k]:
            finder.add_num(x)
        ans = [finder.find_median()]

        for i in range(k, len(nums)):
            finder.add_num(nums[i])
            finder.remove_num(nums[i - k])
            ans.append(finder.find_median())
        return ans

class Solution:
    def medianSlidingWindow(self, nums: List[int], k: int) -> List[float]:
        l = SortedList()
        r = SortedList()
        ans = []
        for i, x in enumerate(nums):
            r.add(x)
            l.add(r.pop(0))
            while len(l) - len(r) > 1:
                r.add(l.pop())

```

```

        j = i - k + 1
        if j >= 0:
            ans.append(l[-1] if k & 1 else (l[-1] + r[0]) / 2)
            if nums[j] in l:
                l.remove(nums[j])
            else:
                r.remove(nums[j])
        return ans

```

• SimplyLeet

```

import heapq

class DualHeap:
    def __init__(self, k):
        self.small = [] # max heap (store negatives)
        self.large = [] # min heap
        self.k = k
        self.delayed = {} # counts for lazy removal
        self.smallSize = 0 # effective size of small heap
        self.largeSize = 0 # effective size of large heap

```

```

    def prune(self, heap):
        # Remove elements from the top that are marked for lazy deletion.
        while heap:
            num = heap[0] if heap is self.large else -heap[0]
            if num in self.delayed and self.delayed[num] > 0:
                heapq.heappop(heap)
                self.delayed[num] -= 1
                if self.delayed[num] == 0:
                    del self.delayed[num]

```

```

        else:
            break

```

```

    def balance(self):
        # Ensure the two heaps' sizes differ by at most one.
        if self.smallSize > self.largeSize + 1:
            val = -heapq.heappop(self.small)
            self.smallSize -= 1
            heapq.heappush(self.large, val)
            self.largeSize += 1

```

```

        self.prune(self.small)
        elif self.smallSize < self.largeSize:
            val = heapq.heappop(self.large)
            self.largeSize -= 1
            heapq.heappush(self.small, -val)
            self.smallSize += 1
            self.prune(self.large)

    def add(self, num):
        if not self.small or num <= -self.small[0]:

```

```

            heapq.heappush(self.small, -num)
            self.smallSize += 1
        else:
            heapq.heappush(self.large, num)
            self.largeSize += 1
            self.balance()

```

```

    def remove(self, num):
        # Mark the element for lazy removal.
        self.delayed[num] = self.delayed.get(num, 0) + 1

```

```

        if self.small and num <= -self.small[0]:
            self.smallSize -= 1
            if num == -self.small[0]:
                self.prune(self.small)
        else:
            self.largeSize -= 1
            if self.large and num == self.large[0]:
                self.prune(self.large)
            self.balance()

```

```

    def median(self):
        if self.k % 2:
            return float(-self.small[0])
        else:
            return (-self.small[0] + self.large[0]) / 2.0

```

```

    def slidingWindowMedian(nums, k):
        result = []
        dh = DualHeap(k)
        for i, num in enumerate(nums):

```

```

        dh.add(num)
        if i >= k - 1:
            result.append(dh.median())
            dh.remove(nums[i - k + 1])
        return result

# Example usage:
nums = [1, 3, -1, -3, 5, 3, 6, 7]
k = 3
print(slidingWindowMedian(nums, k))

```

Two Heaps

□ #502 IPO

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Greedy · Sorting · Heap (Priority Queue)
 Lists: AlgoMaster 600

Problem

Suppose LeetCode will start its IPO soon. In order to sell a good price of its shares to Venture Capital, LeetCode would like to work on some projects to increase its capital before the IPO. Since it has limited resources, it can only finish at most k distinct projects before the IPO. Help LeetCode design the best way to maximize its total capital after finishing at most k distinct projects.

You are given n projects where the i th project has a pure profit $profits[i]$ and a minimum capital of $capital[i]$ is needed to start it.

Initially, you have w capital. When you finish a project, you will obtain its pure profit and the profit will be added to your total capital.

Pick a list of at most k distinct projects from given projects to maximize your final capital, and return the final maximized capital.

The answer is guaranteed to fit in a 32-bit signed integer.

Example 1:

Input: $k = 2, w = 0, \text{profits} = [1, 2, 3], \text{capital} = [0, 1, 1]$
 Output: 4
 Explanation: Since your initial capital is 0, you can only start the project indexed 0.
 After finishing it you will obtain profit 1 and your capital becomes 1.
 With capital 1, you can either start the project indexed 1 or the project indexed 2.
 Since you can choose at most 2 projects, you need to finish the project indexed 2 to get the maximum capital.
 Therefore, output the final maximized capital, which is $0 + 1 + 3 = 4$.

Example 2:

Input: $k = 3, w = 0, \text{profits} = [1, 2, 3], \text{capital} = [0, 1, 2]$
 Output: 6

Constraints:

- $1 \leq k \leq 105$
- $0 \leq w \leq 109$
- $n == \text{profits.length}$
- $n == \text{capital.length}$
- $1 \leq n \leq 105$
- $0 \leq \text{profits}[i] \leq 104$
- $0 \leq \text{capital}[i] \leq 109$

Community Solutions (Python)

Round 6 · Mid · Hard

p.2641

• LeetCode

```
class Solution:
    def findMaximizedCapital(
        self, k: int, w: int, profits: List[int], capital: List[int]
    ) -> int:
        h1 = [(c, p) for c, p in zip(capital, profits)]
        heapify(h1)
        h2 = []
        while k:
            while h1 and h1[0][0] <= w:
                heappush(h2, -heappop(h1)[1])

            if not h2:
                break
            w -= heappop(h2)
            k -= 1
        return w
```

• SimplyLeet

```
import heapq

def findMaximizedCapital(k, w, profits, capital):
    # Create project pairs (required capital, profit)
    projects = sorted(zip(capital, profits), key=lambda x: x[0])
    max_heap = [] # max heap (simulate by pushing negative profits)
    idx = 0
    n = len(projects)

    # Iterate up to k projects

    for _ in range(k):
        # Add all projects that are affordable with current capital w
        while idx < n and projects[idx][0] <= w:
            # Push negative profit to simulate max-heap since heapq is a
            min-heap
            heapq.heappush(max_heap, -projects[idx][1])
            idx += 1

        # If no project is affordable, break out early
        if not max_heap:
            break
```

Round 6 · Mid · Hard

p.2642

```
# Select project with maximum profit and update capital
w += -heapq.heappop(max_heap)

return w

# Example usage:
# k = 2, w = 0, profits = [1,2,3], capital = [0,1,1]
print(findMaximizedCapital(2, 0, [1, 2, 3], [0, 1, 1])) # Output: 4
```

Intervals

Round 6 · Mid · Hard · 1 question

Intervals

□ #2402 Meeting Rooms III

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Sorting · Heap (Priority Queue) · Simulation

Lists: AlgoMaster 600

Problem

You are given an integer n . There are n rooms numbered from 0 to $n - 1$.

You are given a 2D integer array `meetings` where `meetings[i] = [starti, endi]` means that a meeting will be held during the half-closed time interval `[starti, endi)`. All the values of `starti` are unique.

Meetings are allocated to rooms in the following manner:

1. Each meeting will take place in the unused room with the lowest number.
2. If there are no available rooms, the meeting will be delayed until a room becomes free. The delayed meeting should have the same

duration as the original meeting.

3. When a room becomes unused, meetings that have an earlier original start time should be given the room.

Return the number of the room that held the most meetings. If there are multiple rooms, return the room with the lowest number.

A half-closed interval `[a, b)` is the interval between `a` and `b` including `a` and not including `b`.

Example 1:

```
Input: n = 2, meetings = [[0,10],[1,5],[2,7],[3,4]]
Output: 0
Explanation:
- At time 0, both rooms are not being used. The first meeting starts in room 0.
- At time 1, only room 1 is not being used. The second meeting starts in room 1.
- At time 2, both rooms are being used. The third meeting is delayed.
- At time 3, both rooms are being used. The fourth meeting is delayed.
- At time 5, the meeting in room 1 finishes. The third meeting starts in room 1 for the time period [5,10).
```

Example 2:

```
Input: n = 3, meetings = [[1,20],[2,10],[3,5],[4,9],[6,8]]
Output: 1
Explanation:
- At time 1, all three rooms are not being used. The first meeting starts in room 0.
- At time 2, rooms 1 and 2 are not being used. The second meeting starts in room 1.
- At time 3, only room 2 is not being used. The third meeting starts in room 2.
- At time 4, all three rooms are being used. The fourth meeting is delayed.
- At time 5, the meeting in room 2 finishes. The fourth meeting starts in room 2 for the time period [5,10).
```

Constraints:

- $1 \leq n \leq 100$
- $1 \leq \text{meetings.length} \leq 105$
- $\text{meetings}[i].\text{length} == 2$
- $0 \leq \text{start}_i < \text{end}_i \leq 5 * 105$
- All the values of start_i are unique.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def mostBooked(self, n: int, meetings: list[list[int]]) -> int:
        count = [0] * n

        meetings.sort()

        occupied = [] # (endTime, roomId)
        availableRoomIds = [i for i in range(n)]
        heapq.heapify(availableRoomIds)

        for start, end in meetings:
            # Push meetings ending before this `meeting` in occupied to the
            # `availableRoomsIds`.
            while occupied and occupied[0][0] <= start:
                heapq.heappush(availableRoomIds, heapq.heappop(occupied)[1])
            if availableRoomIds:
                roomId = heapq.heappop(availableRoomIds)
                count[roomId] += 1
                heapq.heappush(occupied, (end, roomId))
            else:
                newStart, roomId = heapq.heappop(occupied)
                count[roomId] += 1
                heapq.heappush(occupied, (newStart + (end - start), roomId))

        return count.index(max(count))
```

Round 6 · Mid · Hard

p.2647

• LeetCode

```
class Solution:
    def mostBooked(self, n: int, meetings: List[List[int]]) -> int:
        meetings.sort(key=lambda x: x[0])
        busy = []
        idle = list(range(n))
        heapq.heapify(idle)
        cnt = [0] * n
        for s, e in meetings:
            while busy and busy[0][0] <= s:
                heapq.push(idle, heapq.pop(busy)[1])

            i = 0
            if idle:
                i = heapq.pop(idle)
                heapq.push(busy, (e, i))
            else:
                time_end, i = heapq.pop(busy)
                heapq.push(busy, (time_end + e - s, i))
            cnt[i] += 1

        ans = 0
        for i in range(n):
            if cnt[ans] < cnt[i]:
                ans = i

        return ans
```

• SimplyLeet

```
import heapq

def mostBooked(n, meetings):
    # Sort meetings by their original start times.
    meetings.sort(key=lambda x: x[0])

    # Initialize available rooms heap and ongoing meetings heap.
    available_rooms = list(range(n))
    heapq.heapify(available_rooms)
    busy_rooms = [] # Each element is (end_time, room)
```

Round 6 · Mid · Hard

p.2648


```
# Count of meetings held in each room.
meeting_count = [0] * n

for start, end in meetings:
    # Free up all rooms that have finished before the current meeting
    start.
    while busy_rooms and busy_rooms[0][0] <= start:
        end_time, room = heapq.heappop(busy_rooms)
        heapq.heappush(available_rooms, room)

    duration = end - start

    if available_rooms:
        # If a room is available, assign the meeting immediately.
        room = heapq.heappop(available_rooms)
        meeting_start = start
        meeting_end = start + duration
    else:
        # Delay the meeting: get the room that will be free the earliest.
        earliest_end, room = heapq.heappop(busy_rooms)

        meeting_start = earliest_end
        meeting_end = earliest_end + duration

    # Update the room meeting count and busy heap.
    meeting_count[room] += 1
    heapq.heappush(busy_rooms, (meeting_end, room))

# Return the room with the maximum meeting count (lowest numbered in case
of tie).
return meeting_count.index(max(meeting_count))

# Example usage:
print(mostBooked(2, [[0,10],[1,5],[2,7],[3,4]])) # Expected output: 0
```

Round 6 · Mid · Hard

p.2649

K-Way Merge

Round 6 · Mid · Hard · 2 questions

Round 6 · Mid · Hard

p.2650

K-Way Merge

#23 Merge k Sorted Lists

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Linked List · Divide and Conquer · Heap ·
 Merge Sort
 Lists: Grind 169

Problem

You are given an array of k linked-lists lists, each linked-list is sorted in ascending order. Merge all the linked-lists into one sorted linked-list and return it.

Example 1:

```
Input: lists = [[1,4,5],[1,3,4],[2,6]]
Output: [1,1,2,3,4,4,5,6]
Explanation: The linked-lists are:
[
  1->4->5,
  1->3->4,
  2->6
]
```

Example 2:

```
Input: lists = []
Output: []
```

Example 3:

Round 6 · Mid · Hard

p.2651

```
Input: lists = [[]]
Output: []
```

Constraints:

- $k == \text{lists.length}$
- $0 \leq k \leq 104$
- $0 \leq \text{lists}[i].\text{length} \leq 500$
- $-104 \leq \text{lists}[i][j] \leq 104$
- $\text{lists}[i]$ is sorted in ascending order.
- The sum of $\text{lists}[i].\text{length}$ will not exceed 104.

Community Solutions (Python)

• WalkCC

```
from queue import PriorityQueue

class Solution:
    def mergeKLists(self, lists: list[ListNode]) -> ListNode:
        dummy = ListNode(0)
        curr = dummy
        pq = PriorityQueue()

        for i, lst in enumerate(lists):
```

Round 6 · Mid · Hard

p.2652

```

if lst:
    pq.put((lst.val, i, lst))

while not pq.empty():
    _, i, minNode = pq.get()
    if minNode.next:
        pq.put((minNode.next.val, i, minNode.next))
    curr.next = minNode
    curr = curr.next

```

```

return dummy.next

```

• LeetCode

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def mergeKLists(self, lists: List[Optional[ListNode]]) -> Optional[ListNode]:
        setattr(ListNode, "__lt__", lambda a, b: a.val < b.val)
        pq = [head for head in lists if head]
        heapify(pq)

        dummy = cur = ListNode()
        while pq:
            node = heappop(pq)
            if node.next:
                heappush(pq, node.next)
            cur.next = node
            cur = cur.next
        return dummy.next

```

• SimplyLeet

Round 6 · Mid · Hard

p.2653

```

# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

import heapq

class Solution:
    def mergeKLists(self, lists):
        # Create a min-heap
        min_heap = []
        # Heap entry: (node value, unique id, node) to handle duplicate values
        for i, node in enumerate(lists):
            if node:
                heapq.heappush(min_heap, (node.val, i, node))

        # Dummy node to form the start of merged list
        dummy = ListNode(0)
        current = dummy

        # While there are elements in the heap
        while min_heap:
            # Pop the smallest node from the heap
            val, idx, node = heapq.heappop(min_heap)
            # Add the smallest node to the merged list
            current.next = node
            current = current.next
            # If there is a next node in the list, push it into the heap
            if node.next:
                heapq.heappush(min_heap, (node.next.val, idx, node.next))

        return dummy.next

```

• LC.ca

Round 6 · Mid · Hard

p.2654

```
# Definition for singly-linked list.
# class ListNode:
# def __init__(self, val=0, next=None):
# self.val = val
# self.next = next
class Solution:
    def mergeKLists(self, lists: List[Optional[ListNode]]) -> Optional[ListNode]:
        dummy = cur = ListNode()
        while pq:
            node = heappop(pq)
            if node.next:
                heappush(pq, node.next)
            cur.next = node
            cur = cur.next
        return dummy.next
```

```
# Definition for singly-linked list.
# class ListNode:
# def __init__(self, val=0, next=None):
# self.val = val
# self.next = next

#####

import heapq
```

Round 6 · Mid · Hard

p.2655

```
'''
__lt__(self, other) for <
__le__(self, other) for <=
__gt__(self, other) for >
__ge__(self, other) for >=
'''
```

```
# or create a wrapper class, without modifying existing node class
class NodeWrapper:
    def __init__(self, node):
        self.node = node
```

```
    def __lt__(self, other):
        return self.node.val < other.node.val
```

```
# overwrite the comparison function, so the node can be comparable
# or else, error, TypeError: '<' not supported between instances of 'ListNode'
and 'ListNode'
```

```
# define 'gt' will also work, so either lt or gt will do the compare job for
ListNode
# ListNode.__gt__ = lambda x, y: (x.val > y.val)
```

```
ListNode.__lt__ = lambda x, y: (x.val < y.val)
```

```
class Solution:
```

Round 6 · Mid · Hard

p.2656

```

def mergeKLists(self, lists: List[Optional[ListNode]]) -> Optional[ListNode]:
    dummy = current = ListNode()
    heap = []
    for i, node in enumerate(lists):
        if node:
            # need to override
            # or else error: TypeError: '<' not supported between instances
            # of 'ListNode' and 'ListNode'
            heapq.heappush(heap, node)

    while heap:

        anode = heapq.heappop(heap)

        current.next = anode
        current = current.next

        if anode.next:
            heapq.heappush(heap, anode.next)

    return dummy.next

'''
tried to use node.val to order heap, but still got error TypeError: '<' not
supported between instances of 'ListNode' and 'ListNode':

heapq.heappush(heap, (node.val, node))

'''

#####

# based on merge 2 lists

```

Round 6 · Mid · Hard

p.2657

```

# note for empty input
class Solution:
    def mergeKLists(self, lists: List[Optional[ListNode]]) -> Optional[ListNode]:
        if lists is None or len(lists) == 0:
            return None
        return self.mergeKListsHelper(lists, 0, len(lists) - 1)

    def mergeKListsHelper(self, lists: List[Optional[ListNode]], start: int, end: int) -> Optional[ListNode]:
        if start == end:
            return lists[start]

        mid = int((start + end) / 2)

        left = self.mergeKListsHelper(lists, start, mid)
        # print(left)

        right = self.mergeKListsHelper(lists, mid + 1, end)
        # print(right)

        return self.mergeTwoLists(left, right)

    def mergeTwoLists(self, list1: Optional[ListNode], list2: Optional[ListNode]) -> Optional[ListNode]:
        dummy = ListNode()
        current = dummy
        # print("merging: ", list1)
        # print("merging: ", list2)
        while list1 or list2:
            v1 = list1.val if list1 else float('inf')
            v2 = list2.val if list2 else float('inf')

```

Round 6 · Mid · Hard

p.2658

```

    if v1 < v2:
        current.next = list1
        list1 = list1.next
    else:
        current.next = list2
        list2 = list2.next
    current = current.next

```

```

# print("merging result: ", dummy.next)

```

```

return dummy.next

```

```

#####

```

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next

```

```

class Solution:
    def mergeKLists(self, lists: List[ListNode]) -> ListNode:
        n = len(lists)
        if n == 0:
            return None
        for i in range(n - 1): # this is o(N) times of merge, while if use
            divide-mid-merge then it's o(logN) times of merge
            lists[i + 1] = self.mergeTwoLists(lists[i], lists[i + 1])
        return lists[-1]

    def mergeTwoLists(self, l1: ListNode, l2: ListNode) -> ListNode:

```

```

dummy = ListNode()
cur = dummy
while l1 and l2:
    if l1.val <= l2.val:
        cur.next = l1
        l1 = l1.next
    else:
        cur.next = l2
        l2 = l2.next
    cur = cur.next

```

```

cur.next = l1 or l2
return dummy.next

```

```

if __name__ == '__main__':

```

```

    l1 = ListNode(1)
    l1.next = ListNode(4)
    l1.next.next = ListNode(5)

```

```

    l2 = ListNode(1)
    l2.next = ListNode(3)
    l2.next.next = ListNode(4)

```

```

    l3 = ListNode(2)
    l3.next = ListNode(6)

```

```

    print(Solution().mergeKLists([l1, l2, l3]))

```

```

print(Solution().mergeKLists([]))
print(Solution().mergeKLists([[]]))

```

◆ Quick Review

Key Insights

- Use a min-heap (priority queue) to efficiently extract the smallest value among the current heads of the lists.
- By pushing the head of each non-empty list into the heap, we can pull the smallest node, then push that node's next element into the heap.
- Alternatively, a divide and conquer approach can be used by recursively merging pairs of lists.
- Careful management of pointers (or references) is key to constructing the new linked list.

Space & Time

Time Complexity: $O(N \log k)$ where N is the total number of nodes across all lists and k is the number of lists.

Space Complexity: $O(k)$ due to the heap containing at most one node from each list.

Solution

We use a min-heap to always retrieve the smallest node among the current nodes in the linked lists. First, initialize the heap with the head of each list. Then, continuously extract

the smallest node from the heap, attach it to the merged list, and if the extracted node has a next node, insert that into the heap. The process repeats until the heap is empty.

K-Way Merge

#632 Smallest Range Covering Elements from K Lists HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode Array · Hash Table · Greedy · Heap · Sliding Window · Sorting

Lists: Grind 169

Problem

You have k lists of sorted integers in non-decreasing order. Find the smallest range that includes at least one number from each of the k lists.

We define the range $[a, b]$ is smaller than range $[c, d]$ if $b - a < d - c$ or $a < c$ if $b - a == d - c$.

Example 1:

Input: `nums = [[4,10,15,24,26],[0,9,12,20],[5,18,22,30]]`
 Output: `[20,24]`
 Explanation:
 List 1: `[4, 10, 15, 24,26]`, 24 is in range `[20,24]`.
 List 2: `[0, 9, 12, 20]`, 20 is in range `[20,24]`.
 List 3: `[5, 18, 22, 30]`, 22 is in range `[20,24]`.

Example 2:

Input: `nums = [[1,2,3],[1,2,3],[1,2,3]]`
 Output: `[1,1]`

Constraints:

- `nums.length == k`
- `1 <= k <= 3500`
- `1 <= nums[i].length <= 50`
- `-105 <= nums[i][j] <= 105`
- `nums[i]` is sorted in non-decreasing order.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def smallestRange(self, nums: list[list[int]]) -> list[int]:
        minHeap = [(row[0], i, 0) for i, row in enumerate(nums)]
        heapq.heapify(minHeap)

        maxRange = max(row[0] for row in nums)
        minRange = heapq.nsmallest(1, minHeap)[0][0]
        ans = [minRange, maxRange]

        while len(minHeap) == len(nums):
            num, r, c = heapq.heappop(minHeap)
            if c + 1 < len(nums[r]):
                heapq.heappush(minHeap, (nums[r][c + 1], r, c + 1))
                maxRange = max(maxRange, nums[r][c + 1])
                minRange = heapq.nsmallest(1, minHeap)[0][0]
                if maxRange - minRange < ans[1] - ans[0]:
                    ans[0], ans[1] = minRange, maxRange

        return ans
```

• LeetDoocs


```
class Solution:
    def smallestRange(self, nums: List[List[int]]) -> List[int]:
        t = [(x, i) for i, v in enumerate(nums) for x in v]
        t.sort()
        cnt = Counter()
        ans = [-inf, inf]
        j = 0
        for i, (b, v) in enumerate(t):
            cnt[v] += 1
            while len(cnt) == len(nums):
                a = t[j][0]
                x = b - a - (ans[1] - ans[0])
                if x < 0 or (x == 0 and a < ans[0]):
                    ans = [a, b]
                w = t[j][1]
                cnt[w] -= 1
                if cnt[w] == 0:
                    cnt.pop(w)
                j += 1
        return ans
```

• SimplyLeet

```
import heapq

def smallestRange(nums):
    # Initialize the min-heap and current maximum value
    heap = []
    current_max = float('-inf')

    # Add the first element of each list to the heap
    for i, lst in enumerate(nums):
        value = lst[0]
```

Round 6 · Mid · Hard

p.2665

```
        heapq.heappush(heap, (value, i, 0))
        current_max = max(current_max, value)

        best_range = [float('-inf'), float('inf')]

        # Process until one list is exhausted
        while heap:
            current_min, list_idx, elem_idx = heapq.heappop(heap)

            # Update the best range if the current range is smaller

            if current_max - current_min < best_range[1] - best_range[0] or \
               (current_max - current_min == best_range[1] - best_range[0] and cur
                rent_min < best_range[0]):
                best_range = [current_min, current_max]

            # If there is no next element in the same list, break out
            if elem_idx + 1 == len(nums[list_idx]):
                break
            next_value = nums[list_idx][elem_idx + 1]
            heapq.heappush(heap, (next_value, list_idx, elem_idx + 1))
            current_max = max(current_max, next_value)

        return best_range

# Example Usage:
nums = [[4,10,15,24,26],[0,9,12,20],[5,18,22,30]]
print(smallestRange(nums))
```

• LC.ca

```
class Solution:
    def smallestRange(self, nums: List[List[int]]) -> List[int]:
        t = [(x, i) for i, v in enumerate(nums) for x in v]
        t.sort()
        cnt = Counter()
        ans = [-inf, inf]
        j = 0
        for b, v in t:
            cnt[v] += 1
            while len(cnt) == len(nums):
```

Round 6 · Mid · Hard

p.2666

```

a = t[j][0]
x = b - a - (ans[1] - ans[0])
if x < 0 or (x == 0 and a < ans[0]):
    ans = [a, b]
w = t[j][1]
cnt[w] -= 1
if cnt[w] == 0:
    cnt.pop(w)
j += 1
return ans

```

◆ Quick Review

Key Insights

- Use a min heap to maintain the smallest current element among the selected elements from each list.
- Track the current maximum among the chosen elements to easily compute the range.
- Iteratively replace the minimum element with the next element from the same list and update the current maximum.
- Stop when one of the lists is exhausted, as the range must include at least one element from each list.

Space & Time

Time Complexity: $O(N \log k)$, where N is the total number of elements across all lists.

Space Complexity: $O(k)$ for the heap, plus additional space for storing the range.

Solution

The solution uses a min heap (priority queue) that stores a tuple containing the current element, the index of the list it comes from, and its index in that list. Initially, the first element from each list is inserted into the heap and the current maximum among them is tracked. In each iteration, the smallest element is removed (heap root) which provides the current minimum, and the range $[current_min, current_max]$ is assessed. If the list from which the minimum element was extracted has a next element, that element is inserted into the heap and the current maximum is updated if necessary. This process continues until one of the lists is exhausted.

Data Structure Design

Round 6 · Mid · Hard · 8 questions

Round 6 · Mid · Hard

p.2669

Data Structure Design

□ #432 All O'one Data Structure

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Design · Doubly-Linked List ·
Linked List

Lists: Denny Zhang

Problem

Design a data structure to store the strings' count with the ability to return the strings with minimum and maximum counts.

Implement the AllOne class:

- AllOne() Initializes the object of the data structure.
- inc(String key) Increments the count of the string key by 1. If key does not exist in the data structure, insert it with count 1.
- dec(String key) Decrements the count of the string key by 1. If the count of key is 0 after the decrement, remove it from the data structure. It is guaranteed that key exists in the data structure before the decrement.

Round 6 · Mid · Hard

p.2670

- **getMaxKey()** Returns one of the keys with the maximal count. If no element exists, return an empty string "".
- **getMinKey()** Returns one of the keys with the minimum count. If no element exists, return an empty string "".

Note that each function must run in $O(1)$ average time complexity.

Example 1:

Input
 ["AllOne", "inc", "inc", "getMaxKey", "getMinKey", "inc", "getMaxKey", "getMinKey"]
 [[], ["hello"], ["hello"], [], [], ["leet"], [], []]
 Output
 [null, null, null, "hello", "hello", null, "hello", "leet"]

Explanation
 AllOne allOne = new AllOne();

Constraints:

- $1 \leq \text{key.length} \leq 10$
- key consists of lowercase English letters.
- It is guaranteed that for each call to dec, key is existing in the data structure.
- At most $5 * 10^4$ calls will be made to inc, dec, getMaxKey, and getMinKey.

Community Solutions (Python)

- WalkCC

```
from dataclasses import dataclass

@dataclass
class Node:
    def __init__(self, count: int, key: str | None = None):
        self.count = count
        self.keys: set[str] = {key} if key else set()
        self.prev: Node | None = None
        self.next: Node | None = None

    def __eq__(self, other) -> bool:
        if not isinstance(other, Node):
            return NotImplemented
        return self.count == other.count and self.keys == other.keys

class AllOne:
    def __init__(self):
        self.keyToNode: dict[str, Node] = {}

        self.head = Node(0)
        self.tail = Node(0)
        self.head.next = self.tail
        self.tail.prev = self.head

    def inc(self, key: str) -> None:
        if key in self.keyToNode:
            self._incrementExistingKey(key)
        else:
            self._addNewKey(key)

    def dec(self, key: str) -> None:
        # It is guaranteed that key exists in the data structure before the
        # decrement.
        self._decrementExistingKey(key)

    def getMaxKey(self) -> str:
        return '' if self.tail.prev == self.head \
            else next(iter(self.tail.prev.keys))
```

```

def getMinKey(self) -> str:
    return ' ' if self.head.next == self.tail \
        else next(iter(self.head.next.keys))

def _addNewKey(self, key: str) -> None:
    """Adds a new node with frequency 1."""
    if self.head.next.count == 1:
        self.head.next.keys.add(key)
    else:
        self._insertAfter(self.head, Node(1, key))

    self.keyToNode[key] = self.head.next

def _incrementExistingKey(self, key: str) -> None:
    """Increments the frequency of the key by 1."""
    node = self.keyToNode[key]
    node.keys.remove(key)
    if node.next == self.tail or node.next.count > node.count + 1:
        self._insertAfter(node, Node(node.count + 1))
    node.next.keys.add(key)
    self.keyToNode[key] = node.next

    if not node.keys:
        self._remove(node)

def _decrementExistingKey(self, key: str) -> None:
    """Decrements the count of the key by 1."""
    node = self.keyToNode[key]
    node.keys.remove(key)
    if node.count > 1:
        if node.prev == self.head or node.prev.count != node.count - 1:
            self._insertAfter(node.prev, Node(node.count - 1))

    node.prev.keys.add(key)
    self.keyToNode[key] = node.prev
    else:
        del self.keyToNode[key]
    if not node.keys:
        self._remove(node)

def _insertAfter(self, node: Node, newNode: Node) -> None:
    newNode.prev = node
    newNode.next = node.next

```

Round 6 · Mid · Hard

p.2673

```

node.next.prev = newNode
node.next = newNode

def _remove(self, node: Node) -> None:
    node.prev.next = node.next
    node.next.prev = node.prev

```

• LeetCode

```

class Node:
    def __init__(self, key='', cnt=0):
        self.prev = None
        self.next = None
        self.cnt = cnt
        self.keys = {key}

    def insert(self, node):
        node.prev = self
        node.next = self.next

        node.prev.next = node
        node.next.prev = node
        return node

    def remove(self):
        self.prev.next = self.next
        self.next.prev = self.prev

class AllOne:

    def __init__(self):
        self.root = Node()
        self.root.next = self.root
        self.root.prev = self.root
        self.nodes = {}

    def inc(self, key: str) -> None:
        root, nodes = self.root, self.nodes
        if key not in nodes:
            if root.next == root or root.next.cnt > 1:

```

Round 6 · Mid · Hard

p.2674

```

        nodes[key] = root.insert(Node(key, 1))
    else:
        root.next.keys.add(key)
        nodes[key] = root.next
    else:
        curr = nodes[key]
        next = curr.next
        if next == root or next.cnt > curr.cnt + 1:
            nodes[key] = curr.insert(Node(key, curr.cnt + 1))
        else:
            next.keys.add(key)
            nodes[key] = next
            curr.keys.discard(key)
            if not curr.keys:
                curr.remove()

def dec(self, key: str) -> None:
    root, nodes = self.root, self.nodes
    curr = nodes[key]
    if curr.cnt == 1:
        nodes.pop(key)
    else:
        prev = curr.prev
        if prev == root or prev.cnt < curr.cnt - 1:
            nodes[key] = prev.insert(Node(key, curr.cnt - 1))
        else:
            prev.keys.add(key)
            nodes[key] = prev
            curr.keys.discard(key)
            if not curr.keys:
                curr.remove()

def getMaxKey(self) -> str:
    return next(iter(self.root.prev.keys))

def getMinKey(self) -> str:
    return next(iter(self.root.next.keys))

```

Your AllOne object will be instantiated and called as such:

Round 6 · Mid · Hard

p.2675

```

# obj = AllOne()
# obj.inc(key)
# obj.dec(key)
# param_3 = obj.getMaxKey()
# param_4 = obj.getMinKey()

class AllOne {
    Node root = new Node();
    Map<String, Node> nodes = new HashMap<>();

    public AllOne() {
        root.next = root;
        root.prev = root;
    }

    public void inc(String key) {
        if (!nodes.containsKey(key)) {
            if (root.next == root || root.next.cnt > 1) {
                nodes.put(key, root.insert(new Node(key, 1)));
            } else {
                root.next.keys.add(key);
                nodes.put(key, root.next);
            }
        } else {
            Node curr = nodes.get(key);
            Node next = curr.next;

            if (next == root || next.cnt > curr.cnt + 1) {
                nodes.put(key, curr.insert(new Node(key, curr.cnt + 1)));
            } else {
                next.keys.add(key);
                nodes.put(key, next);
            }
            curr.keys.remove(key);
            if (curr.keys.isEmpty()) {
                curr.remove();
            }
        }
    }
}

```

Round 6 · Mid · Hard

p.2676

```

    }
}

public void dec(String key) {
    Node curr = nodes.get(key);
    if (curr.cnt == 1) {
        nodes.remove(key);
    } else {
        Node prev = curr.prev;
        if (prev == root || prev.cnt < curr.cnt - 1) {

            nodes.put(key, prev.insert(new Node(key, curr.cnt - 1)));
        } else {
            prev.keys.add(key);
            nodes.put(key, prev);
        }
    }

    curr.keys.remove(key);
    if (curr.keys.isEmpty()) {
        curr.remove();
    }
}

public String getMaxKey() {
    return root.prev.keys.iterator().next();
}

public String getMinKey() {
    return root.next.keys.iterator().next();
}
}

class Node {
    Node prev;
    Node next;
    int cnt;
    Set<String> keys = new HashSet<>();

    public Node() {
        this("", 0);
    }
}

```

Round 6 · Mid · Hard

p.2677

```

    }

    public Node(String key, int cnt) {
        this.cnt = cnt;
        keys.add(key);
    }

    public Node insert(Node node) {
        node.prev = this;
        node.next = this.next;

        node.prev.next = node;
        node.next.prev = node;
        return node;
    }

    public void remove() {
        this.prev.next = this.next;
        this.next.prev = this.prev;
    }
}

/**
 * Your AllOne object will be instantiated and called as such:
 * AllOne obj = new AllOne();
 * obj.inc(key);
 * obj.dec(key);
 * String param_3 = obj.getMaxKey();
 * String param_4 = obj.getMinKey();
 */

```

• SimplyLeet

Round 6 · Mid · Hard

p.2678

```
# Node class to represent a frequency bucket in the doubly linked list
class Node:
    def __init__(self, count):
        self.count = count          # frequency count for keys in this
node                                # node

        self.keys = set()           # set of keys with this frequency
        self.prev = None            # previous node in the linked list
        self.next = None            # next node in the linked list

class AllOne:
    def __init__(self):

        # Use sentinel nodes for easier handling of edge cases.
        self.head = Node(float('-inf'))
        self.tail = Node(float('inf'))
        self.head.next = self.tail
        self.tail.prev = self.head
        # Map each key to its corresponding node.
        self.keyToNode = {}

        # Helper: insert newNode after prevNode in the linked list.
        def _insertNodeAfter(self, prevNode, newNode):

            newNode.prev = prevNode
            newNode.next = prevNode.next
            prevNode.next.prev = newNode
            prevNode.next = newNode

        # Helper: remove a node if it has no keys.
        def _removeNode(self, node):
            node.prev.next = node.next
            node.next.prev = node.prev
```

Round 6 · Mid · Hard

p.2679

```
def inc(self, key: str) -> None:
    # If key is new, treat current node as head.
    if key not in self.keyToNode:
        curr = self.head
    else:
        curr = self.keyToNode[key]
        curr.keys.remove(key)

    # Check if the next node has count = current count + 1.
    targetCount = (0 if curr == self.head else curr.count) + 1
    if curr.next == self.tail or curr.next.count != targetCount:

        # Create new node with targetCount.
        newNode = Node(targetCount)
        self._insertNodeAfter(curr, newNode)
    else:
        newNode = curr.next
        newNode.keys.add(key)
        self.keyToNode[key] = newNode
    # Remove the current node if it is not a sentinel and has no keys.
    if curr != self.head and len(curr.keys) == 0:
        self._removeNode(curr)

def dec(self, key: str) -> None:
    curr = self.keyToNode[key]
    curr.keys.remove(key)
    # If decrementing causes the count to be 0, remove key completely.
    if curr.count == 1:
        del self.keyToNode[key]
    else:
        targetCount = curr.count - 1
        # Check if the previous node has the target count.

        if curr.prev == self.head or curr.prev.count != targetCount:
            newNode = Node(targetCount)
            # Insert newNode before current.
            self._insertNodeAfter(curr.prev, newNode)
        else:
            newNode = curr.prev
            newNode.keys.add(key)
            self.keyToNode[key] = newNode
    # Clean up current node if empty.
    if len(curr.keys) == 0:
```

Round 6 · Mid · Hard

p.2680


```

        self._removeNode(curr)

    def getMaxKey(self) -> str:
        # The node before tail has the maximal count.
        return next(iter(self.tail.prev.keys)) if self.tail.prev != self.head
    else ""

    def getMinKey(self) -> str:
        # The node after head has the minimal count.
        return next(iter(self.head.next.keys)) if self.head.next != self.tail
    else ""

# Example usage:
# obj = AllOne()
# obj.inc("hello")
# obj.inc("hello")
# print(obj.getMaxKey()) # "hello"
# print(obj.getMinKey()) # "hello"
# obj.inc("leet")
# print(obj.getMaxKey()) # "hello"
# print(obj.getMinKey()) # "leet"

```

• LC.ca

```

class Node:
    def __init__(self, key='', cnt=0):
        self.prev = None
        self.next = None
        self.cnt = cnt
        self.keys = {key}

    def insert(self, node):
        node.prev = self
        node.next = self.next

```

Round 6 · Mid · Hard

p.2681

```

        node.prev.next = node
        node.next.prev = node
        return node

    def remove(self):
        self.prev.next = self.next
        self.next.prev = self.prev

class AllOne:

    def __init__(self):
        self.root = Node()
        self.root.next = self.root
        self.root.prev = self.root
        self.nodes = {}

    def inc(self, key: str) -> None:
        root, nodes = self.root, self.nodes
        if key not in nodes:
            if root.next == root or root.next.cnt > 1:
                nodes[key] = root.insert(Node(key, 1))
            else:
                root.next.keys.add(key)
                nodes[key] = root.next
        else:
            curr = nodes[key]
            next = curr.next
            if next == root or next.cnt > curr.cnt + 1:
                nodes[key] = curr.insert(Node(key, curr.cnt + 1))
            else:
                next.keys.add(key)
                nodes[key] = next
                curr.keys.discard(key)
                if not curr.keys:
                    curr.remove()

    def dec(self, key: str) -> None:
        root, nodes = self.root, self.nodes
        curr = nodes[key]
        if curr.cnt == 1:

```

Round 6 · Mid · Hard

p.2682

```

        nodes.pop(key)
    else:
        prev = curr.prev
        if prev == root or prev.cnt < curr.cnt - 1:
            nodes[key] = prev.insert(Node(key, curr.cnt - 1))
        else:
            prev.keys.add(key)
            nodes[key] = prev
        curr.keys.discard(key)
        if not curr.keys:

```

```

        curr.remove()

```

```

def getMaxKey(self) -> str:
    return next(iter(self.root.prev.keys))

```

```

def getMinKey(self) -> str:
    return next(iter(self.root.next.keys))

```

```

# Your AllOne object will be instantiated and called as such:

```

```

# obj = AllOne()
# obj.inc(key)
# obj.dec(key)
# param_3 = obj.getMaxKey()
# param_4 = obj.getMinKey()

```

◆ Quick Review

Key Insights

- Use a Doubly-Linked List (DLL) where each node holds a unique count value and a set of keys with that count.
- Maintain a Hash Map (keyToNode) to quickly locate the node corresponding to each key.

Round 6 · Mid · Hard

p.2683

- Insert new nodes in the DLL when a key's count is incremented or decremented and the adjacent node does not have the target count.
- Remove nodes from the DLL when no keys remain in that count.
- The head of the DLL holds the minimum count and the tail holds the maximum count.

Space & Time

Time Complexity: $O(1)$ average for inc, dec, getMaxKey, and getMinKey.

Space Complexity: $O(n)$ where n is the number of unique keys in the data structure.

Solution

The solution uses a combination of a doubly-linked list and hash maps. Each node in the DLL represents a frequency and maintains a set of keys that have that frequency. A hash map maps keys to their corresponding node. When inc(key) or dec(key) is called, we update the key's frequency and move the key to the appropriate node in the DLL. If the adjacent node with the required frequency doesn't exist, we create a new node and insert it in the correct position. Removal of

Round 6 · Mid · Hard

p.2684

a key from a node and deletion of empty nodes ensures that `getMinKey()` and `getMaxKey()` can be returned directly from the head and tail of the DLL.

Round 6 · Mid · Hard

p.2685

Data Structure Design

□ #460 LFU Cache

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Linked List · Design ·
Doubly-Linked List
Lists: Denny Zhang

Problem

Design and implement a data structure for a Least Frequently Used (LFU) cache.

Implement the `LFUCache` class:

- `LFUCache(int capacity)` Initializes the object with the capacity of the data structure.
- `int get(int key)` Gets the value of the key if the key exists in the cache. Otherwise, returns `-1`.
- `void put(int key, int value)` Update the value of the key if present, or inserts the key if not already present. When the cache reaches its capacity, it should invalidate and remove the least frequently used key before inserting a new item. For this problem, when there is a tie

Round 6 · Mid · Hard

p.2686

(i.e., two or more keys with the same frequency), the least recently used key would be invalidated.

To determine the least frequently used key, a use counter is maintained for each key in the cache. The key with the smallest use counter is the least frequently used key.

When a key is first inserted into the cache, its use counter is set to 1 (due to the put operation). The use counter for a key in the cache is incremented either a get or put operation is called on it.

The functions get and put must each run in $O(1)$ average time complexity.

Example 1:

```
Input
["LFUCache", "put", "put", "get", "put", "get", "get", "put", "get", "get",
"get"]
[[2], [1, 1], [2, 2], [1], [3, 3], [2], [3], [4, 4], [1], [3], [4]]
Output
[null, null, null, 1, null, -1, 3, null, -1, 3, 4]
```

Explanation
// cnt(x) = the use counter for key x

Constraints:

- $1 \leq \text{capacity} \leq 104$
- $0 \leq \text{key} \leq 105$
- $0 \leq \text{value} \leq 109$

- At most $2 * 105$ calls will be made to get and put.

Community Solutions (Python)

• WalkCC

```
class Node:
    def __init__(self, key: int, value: int, freq: int, it):
        self.key = key
        self.value = value
        self.freq = freq
        self.it = it

class LFUCache:
    def __init__(self, capacity: int):

        self.capacity = capacity
        self.minFreq = 0
        self.keyToNode = {}
        self.freqToList = {}

    def get(self, key: int) -> int:
        if key not in self.keyToNode:
            return -1

        node = self.keyToNode[key]

        self._touch(node)
        return node.value

    def put(self, key: int, value: int) -> None:
        if self.capacity == 0:
            return

        if key in self.keyToNode:
            node = self.keyToNode[key]
            node.value = value
```

```

        self._touch(node)
        return

    if len(self.keyToNode) == self.capacity:
        # Evict an LRU key from `minFreq` list
        keyToEvict = self.freqToList[self.minFreq][-1]
        self.freqToList[self.minFreq].pop()
        del self.keyToNode[keyToEvict]

    self.minFreq = 1

    if 1 not in self.freqToList:
        self.freqToList[1] = []
    self.freqToList[1].insert(0, key)
    self.keyToNode[key] = Node(key, value, 1, 0) # Using index 0 as iterator

def _touch(self, node: Node) -> None:
    # Update the node's frequency
    prevFreq = node.freq
    node.freq += 1
    newFreq = node.freq

    # Remove the key from `prevFreq`'s list
    self.freqToList[prevFreq].remove(node.key)
    if not self.freqToList[prevFreq]:
        del self.freqToList[prevFreq]
    # Update `minFreq` if needed
    if prevFreq == self.minFreq:
        self.minFreq += 1

    # Insert the key to the front of `newFreq`'s list

    if newFreq not in self.freqToList:
        self.freqToList[newFreq] = []
    self.freqToList[newFreq].insert(0, node.key)
    node.it = 0 # Using index 0 as iterator

```

• SimplyLeet

Round 6 · Mid · Hard

p.2689

```

from collections import OrderedDict

class LFUCache:
    def __init__(self, capacity: int):
        # Initialize cache capacity and helper variables.
        self.capacity = capacity
        self.min_freq = 0 # Track the minimum frequency in cache.
        # Map key -> (value, frequency)
        self.key_to_val_freq = {}
        # Map frequency -> OrderedDict of keys for LRU ordering.

        self.freq_to_keys = {}

    def update_freq(self, key):
        # Increase frequency of the key since it was accessed.
        value, freq = self.key_to_val_freq[key]
        # Remove key from its current frequency list.
        self.freq_to_keys[freq].pop(key)
        # If current frequency list is empty, update min_freq if necessary.
        if not self.freq_to_keys[freq]:
            del self.freq_to_keys[freq]

        if self.min_freq == freq:
            self.min_freq += 1
        new_freq = freq + 1
        # Insert key into the list corresponding to the new frequency.
        if new_freq not in self.freq_to_keys:
            self.freq_to_keys[new_freq] = OrderedDict()
        self.freq_to_keys[new_freq][key] = None
        # Update the frequency entry for the key.
        self.key_to_val_freq[key] = (value, new_freq)

    def get(self, key: int) -> int:
        # Return -1 if the key is not present.
        if key not in self.key_to_val_freq:
            return -1
        # Update frequency due to access.
        self.update_freq(key)
        return self.key_to_val_freq[key][0]

    def put(self, key: int, value: int) -> None:
        # If capacity is zero, nothing can be added.

```

Round 6 · Mid · Hard

p.2690

```

if self.capacity <= 0:
    return
if key in self.key_to_val_freq:
    # Update value and frequency if key exists.
    self.key_to_val_freq[key] = (value, self.key_to_val_freq[key][1])
    self.update_freq(key)
else:
    # If cache is full, evict the least frequently used (and least
    recently used) key.
    if len(self.key_to_val_freq) >= self.capacity:
        # Remove the first key from the lowest frequency list.

        key_to_remove, _ = self.freq_to_keys[self.min_freq].popitem(la
st=False)

        del self.key_to_val_freq[key_to_remove]
        if not self.freq_to_keys[self.min_freq]:
            del self.freq_to_keys[self.min_freq]
        # Insert the new key with frequency 1.
        self.key_to_val_freq[key] = (value, 1)
        if 1 not in self.freq_to_keys:
            self.freq_to_keys[1] = OrderedDict()
        self.freq_to_keys[1][key] = None
        self.min_freq = 1 # Reset min_freq.

```

• LC.ca

```

from collections import defaultdict

class LFUCache:
    def __init__(self, capacity: int):
        self.capacity = capacity
        self.key_to_value = {}
        self.key_to_freq = defaultdict(int)
        self.freq_to_keys = defaultdict(OrderedDict)
        self.min_freq = 0

```

Round 6 · Mid · Hard

p.2691

```

def get(self, key: int) -> int:
    if key not in self.key_to_value:
        return -1

    # Update the frequency
    self._update_frequency(key)
    return self.key_to_value[key]

def put(self, key: int, value: int) -> None:
    if self.capacity == 0:

```

```

        return

    if key in self.key_to_value:
        # Update the value and frequency
        self.key_to_value[key] = value
        self._update_frequency(key)
    else:
        if len(self.key_to_value) >= self.capacity:
            # Evict the least frequently used key
            self._evict()

```

```

        # Add the new key-value pair
        self.key_to_value[key] = value
        self.key_to_freq[key] = 1
        self.freq_to_keys[1][key] = None
        self.min_freq = 1

    def _update_frequency(self, key: int) -> None:
        freq = self.key_to_freq[key]
        del self.freq_to_keys[freq][key]

```

```

        if not self.freq_to_keys[freq]:
            # If there are no keys with the previous frequency, update min_freq
            if self.min_freq == freq:
                self.min_freq += 1
            del self.freq_to_keys[freq]

        freq += 1
        self.key_to_freq[key] = freq
        self.freq_to_keys[freq][key] = None

```

Round 6 · Mid · Hard

p.2692

```
def _evict(self) -> None:
    keys = self.freq_to_keys[self.min_freq]
    evict_key, _ = keys.popitem(last=False)
    del self.key_to_value[evict_key]
    del self.key_to_freq[evict_key]

# Your LFUCache object will be instantiated and called as such:
# obj = LFUCache(capacity)
# param_1 = obj.get(key)
```

```
# obj.put(key,value)
```

```
#####
```

```
class Node:
    def __init__(self, key: int, value: int) -> None:
        self.key = key
        self.value = value
        self.freq = 1
```

```
self.prev = None
self.next = None
```

```
class DoublyLinkedList:
    def __init__(self) -> None:
        self.head = Node(-1, -1)
        self.tail = Node(-1, -1)
        self.head.next = self.tail
        self.tail.prev = self.head
```

```
def add_first(self, node: Node) -> None:
    node.prev = self.head
    node.next = self.head.next
    self.head.next.prev = node
    self.head.next = node
```

```
def remove(self, node: Node) -> Node:
    node.next.prev = node.prev
    node.prev.next = node.next
```

Round 6 · Mid · Hard

p.2693

```
node.next, node.prev = None, None
return node
```

```
def remove_last(self) -> Node:
    return self.remove(self.tail.prev)
```

```
def is_empty(self) -> bool:
    return self.head.next == self.tail
```

```
class LFUCache:
    def __init__(self, capacity: int):
        self.capacity = capacity
        self.min_freq = 0
        self.map = defaultdict(Node)
        self.freq_map = defaultdict(DoublyLinkedList)
```

```
def get(self, key: int) -> int:
    if self.capacity == 0 or key not in self.map:
        return -1
```

```
node = self.map[key]
self.incr_freq(node)
return node.value
```

```
def put(self, key: int, value: int) -> None:
    if self.capacity == 0:
        return
    if key in self.map:
        node = self.map[key]
        node.value = value
```

```
self.incr_freq(node)
return
if len(self.map) == self.capacity:
    ls = self.freq_map[self.min_freq]
    node = ls.remove_last()
    self.map.pop(node.key)
node = Node(key, value)
self.add_node(node)
self.map[key] = node
self.min_freq = 1
```

Round 6 · Mid · Hard

p.2694

```
def incr_freq(self, node: Node) -> None:
    freq = node.freq
    ls = self.freq_map[freq]
    ls.remove(node)
    if ls.is_empty():
        self.freq_map.pop(freq)
        if freq == self.min_freq:
            self.min_freq += 1
    node.freq += 1
```

```
self.add_node(node)
```

```
def add_node(self, node: Node) -> None:
    freq = node.freq
    ls = self.freq_map[freq]
    ls.add_first(node)
    self.freq_map[freq] = ls
```

```
# Your LFUCache object will be instantiated and called as such:
```

```
# obj = LFUCache(capacity)
# param_1 = obj.get(key)
# obj.put(key, value)
```

◆ Quick Review

Key Insights

- Maintain a mapping from key to its value and frequency.
- Use an additional mapping from frequency to keys (maintaining the order of insertion or recent use) for quick eviction.

- Keep a global minFreq variable to track the lowest frequency present in the cache.
- When a key is accessed or updated, increase its frequency and relocate it in the frequency mapping.
- Evictions occur by removing the oldest key from the lowest frequency bucket.

Space & Time

Time Complexity: $O(1)$ average for both get and put.

Space Complexity: $O(\text{capacity})$, where capacity is the maximum number of keys stored.

Solution

The solution utilizes two main data structures:

- A hash map (keyToValFreq) that maps keys to a pair (value, frequency).
- A second hash map (freqToKeys) mapping frequencies to an ordered list/set of keys. This data structure preserves the order in which keys were added to allow removal of the least recently used key when multiple keys share the same frequency.

A global minFreq variable is maintained to quickly identify the bucket with the least frequency. When a key is accessed or updated,

the key is removed from its current frequency list and placed in the next higher frequency list. If the frequency list for the minimal frequency becomes empty, minFreq is updated. This design guarantees that both get and put operations execute in constant time on average.

Round 6 · Mid · Hard

p.2697

Data Structure Design

□ ★ #588 Design In-Memory File System ★ **HAR**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Design · Trie
Lists: ★ Premium | Grind 169 | Premium 98

Problem

Design a data structure that simulates an in-memory file system.

Implement the FileSystem class:

- **FileSystem()** Initializes the object of the system.
- **List<String> ls(String path)** If path is a file path, returns a list that only contains this file's name.
- If path is a directory path, returns the list of file and directory names in this directory.
- If filePath does not exist, creates that file containing given content.
- If filePath already exists, appends the given content to original content.

Example 1:

Round 6 · Mid · Hard

p.2698

Operation	Output	Explanation
FileSystem fs = new FileSystem()	null	The constructor returns nothing.
fs.ls("/")	[]	Initially, directory <code>/</code> has nothing. So return empty list.
fs.mkdir("/a/b/c")	null	Create directory <code>a</code> in directory <code>/</code> . Then create directory <code>b</code> in directory <code>a</code> . Finally, create directory <code>c</code> in directory <code>b</code> .
fs.addContentToFile("/a/b/c/d", "hello")	null	Create a file named <code>d</code> with content <code>"hello"</code> in directory <code>/a/b/c</code> .
fs.ls("/")	["a"]	Only directory <code>a</code> is in directory <code>/</code> .
fs.readContentFromFile("/a/b/c/d")	"hello"	Output the file content.

Input
 ["FileSystem", "ls", "mkdir", "addContentToFile", "ls", "readContentFromFile"]
 [[], ["/"], ["/a/b/c"], ["/a/b/c/d", "hello"], ["/"], ["/a/b/c/d"]]

Output
 [null, [], null, null, ["a"], "hello"]

Explanation
 FileSystem fileSystem = new FileSystem();

Constraints:

- $1 \leq \text{path.length}, \text{filePath.length} \leq 100$
- `path` and `filePath` are absolute paths which begin with `'/'` and do not end with `'/'` except that the path is just `"/"`.
- You can assume that all directory names and file names only contain lowercase letters, and the same names will not exist in the same directory.

- You can assume that all operations will be passed valid parameters, and users will not attempt to retrieve file content or list a directory or file that does not exist.
- You can assume that the parent directory for the file in `addContentToFile` will exist.
- $1 \leq \text{content.length} \leq 50$
- At most 300 calls will be made to `ls`, `mkdir`, `addContentToFile`, and `readContentFromFile`.

Community Solutions (Python)

• LeetCode

```
class Trie:
    def __init__(self):
        self.name = None
        self.isFile = False
        self.content = []
        self.children = {}

    def insert(self, path, isFile):
        node = self
        ps = path.split('/')
        for p in ps:
```

```

for p in ps[1:]:
    if p not in node.children:
        node.children[p] = Trie()
    node = node.children[p]
node.isFile = isFile
if isFile:
    node.name = ps[-1]
return node

```

```
def search(self, path):
```

```

    node = self
    if path == '/':
        return node
    ps = path.split('/')
    for p in ps[1:]:
        if p not in node.children:
            return None
        node = node.children[p]
    return node

```

```
class FileSystem:
```

```

    def __init__(self):
        self.root = Trie()

```

```

    def ls(self, path: str) -> List[str]:
        node = self.root.search(path)
        if node is None:
            return []
        if node.isFile:

```

```

            return [node.name]
        return sorted(node.children.keys())

```

```

    def mkdir(self, path: str) -> None:
        self.root.insert(path, False)

```

```

    def addContentToFile(self, filePath: str, content: str) -> None:
        node = self.root.insert(filePath, True)
        node.content.append(content)

```

Round 6 · Mid · Hard

p.2701

```

def readContentFromFile(self, filePath: str) -> str:
    node = self.root.search(filePath)
    return ''.join(node.content)

```

```

# Your FileSystem object will be instantiated and called as such:
# obj = FileSystem()
# param_1 = obj.ls(path)
# obj.mkdir(path)
# obj.addContentToFile(filePath,content)

```

```
# param_4 = obj.readContentFromFile(filePath)
```

• SimplyLeet

```
class Node:
```

```

    def __init__(self):
        # Indicates whether this node represents a file or a directory
        self.is_file = False
        # For directories, mapping from name to Node
        self.children = {}
        # For files, store content
        self.content = ""

```

```
class FileSystem:
```

```

    def __init__(self):
        # Root directory node initialization
        self.root = Node()

```

```

    def ls(self, path: str) -> list:
        # Traverse the path to obtain the target node
        node, name = self.traverse(path)
        # If the node is a file, return the single element list containing its
        name
        if node.is_file:

```

Round 6 · Mid · Hard

p.2702

```

        return [name]
    # Otherwise, list all keys (names) in the directory, sorted
    lexicographically
    return sorted(node.children.keys())

def mkdir(self, path: str) -> None:
    # Create directory by traversing the path and creating nodes if
    necessary
    self.traverse(path)

def addContentToFile(self, filePath: str, content: str) -> None:
    # Traverse to the file's node; if it does not exist, create necessary
    nodes along the path

    node, name = self.traverse(filePath)
    # If not already a file, mark the node as a file
    if not node.is_file:
        node.is_file = True
        node.content = ""
    # Append the new content to the file's content
    node.content += content

def readContentFromFile(self, filePath: str) -> str:
    # Traverse to the file's node

    node, _ = self.traverse(filePath)
    # Return the content of the file
    return node.content

def traverse(self, path: str):
    # Start from the root node
    node = self.root
    if path == "/":
        # Special case for root path
        return node, ""

```

Round 6 · Mid · Hard

p.2703

```

    # Split the path by "/" and traverse the nodes
    parts = path.split("/")[1:]
    for part in parts:
        # Create the node if it does not exist
        if part not in node.children:
            node.children[part] = Node()
        node = node.children[part]
    # Return the target node and its name (last part of the path)
    return node, parts[-1]

```

```

# Example usage:
# fs = FileSystem()
# print(fs.ls("/")) # prints []
# fs.mkdir("/a/b/c")
# fs.addContentToFile("/a/b/c/d", "hello")
# print(fs.ls("/")) # prints ['a']
# print(fs.readContentFromFile("/a/b/c/d")) # prints "hello"

```

• LC.ca

```

'''
>>> "/a/b/c".split('/')
['', 'a', 'b', 'c']
'''

class Trie:
    def __init__(self):
        self.name = None
        self.isFile = False
        self.content = []

        self.children = {}

    def insert(self, path, isFile):
        node = self
        ps = path.split('/')
        for p in ps[1:]:
            if p not in node.children:
                node.children[p] = Trie()
            node = node.children[p]
        node.isFile = isFile

```

Round 6 · Mid · Hard

p.2704

```

    if isFile:
        node.name = ps[-1]
    return node

def search(self, path):
    node = self
    if path == '/':
        return node
    ps = path.split('/')
    for p in ps[1:]:
        if p not in node.children:
            return None
        node = node.children[p]
    return node

class FileSystem:
    def __init__(self):
        self.root = Trie()

    def ls(self, path: str) -> List[str]:
        node = self.root.search(path)
        if node is None:
            return []
        if node.isFile:
            return [node.name]
        return sorted(node.children.keys())

    def mkdir(self, path: str) -> None:
        self.root.insert(path, False)

    def addContentToFile(self, filePath: str, content: str) -> None:
        node = self.root.insert(filePath, True)
        node.content.append(content)

    def readContentFromFile(self, filePath: str) -> str:
        node = self.root.search(filePath)
        return ''.join(node.content)

```

Round 6 · Mid · Hard

p.2705

```

# Your FileSystem object will be instantiated and called as such:
# obj = FileSystem()
# param_1 = obj.ls(path)
# obj.mkdir(path)
# obj.addContentToFile(filePath,content)
# param_4 = obj.readContentFromFile(filePath)

```

◆ Quick Review

Key Insights

- Use a tree (trie) to represent directories and files.
- Each node can be either a directory or a file; directories maintain a mapping of names to child nodes.
- For ls, if the given path is a file, return only its name. If it's a directory, return the sorted list of names of its children.
- mkdir must create all intermediate directories if they do not already exist.
- File nodes store content and support content appending.

Space & Time

Time Complexity:

- ls: $O(k \log k)$ where k is the number of entries in the directory (due to sorting), or

Round 6 · Mid · Hard

p.2706

$O(n)$ if traversing a long path.

– `mkdir`, `addContentToFile`,
`readContentFromFile`: $O(d)$ where d is the
depth of the file path.

Space Complexity:

– $O(T)$ where T is the total number of
characters stored in directory names and file
contents.

Solution

The solution uses a tree-based approach where each node represents either a directory or a file. Each directory node stores its children in a hash table (or dictionary) mapping names to node objects. File nodes store their content as a string and carry a flag indicating they are files. For the `ls` operation, check if the node is a file—if so, return its name; otherwise, list and sort the names of all children in that directory. The `mkdir` operation traverses (or creates) nodes for each segment of the provided path. The `addContentToFile` method accesses or creates the file node and appends the content. Finally, `readContentFromFile` retrieves the stored

content from the file node.

Data Structure Design

★ #642 Design Search Autocomplete System ★

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode String · Design · Trie · Data Stream · Heap Lists: ★ Premium | Premium 98

Problem

Design a search autocomplete system for a search engine. Users may input a sentence (at least one word and end with a special character '#').

You are given a string array `sentences` and an integer array `times` both of length `n` where `sentences[i]` is a previously typed sentence and `times[i]` is the corresponding number of times the sentence was typed. For each input character except '#', return the top 3 historical hot sentences that have the same prefix as the part of the sentence already typed.

Here are the specific rules:

- The hot degree for a sentence is defined as the number of times a user typed the exactly

Round 6 · Mid · Hard

p.2709

same sentence before.

- The returned top 3 hot sentences should be sorted by hot degree (The first is the hottest one). If several sentences have the same hot degree, use ASCII-code order (smaller one appears first).
- If less than 3 hot sentences exist, return as many as you can.
- When the input is a special character, it means the sentence ends, and in this case, you need to return an empty list.

Implement the `AutocompleteSystem` class:

- `AutocompleteSystem(String[] sentences, int[] times)` Initializes the object with the `sentences` and `times` arrays.
- `List<String> input(char c)` This indicates that the user typed the character `c`. Returns an empty array `[]` if `c == '#'` and stores the inputted sentence in the system.
- Returns the top 3 historical hot sentences that have the same prefix as the part of the sentence already typed. If there are fewer than 3 matches, return them all.

Example 1:

Round 6 · Mid · Hard

p.2710

```

Input
["AutocompleteSystem", "input", "input", "input", "input"]
[[["i love you", "island", "iroman", "i love leetcode"], [5, 3, 2, 2]], ["i"],
[" "], ["a"], ["#"]]
Output
[null, ["i love you", "island", "i love leetcode"], ["i love you", "i love
leetcode"], [], []]

```

Explanation

```

AutocompleteSystem obj = new AutocompleteSystem(["i love you", "island",
"iroman", "i love leetcode"], [5, 3, 2, 2]);

```

Constraints:

- $n == \text{sentences.length}$
- $n == \text{times.length}$
- $1 \leq n \leq 100$
- $1 \leq \text{sentences}[i].\text{length} \leq 100$
- $1 \leq \text{times}[i] \leq 50$
- c is a lowercase English letter, a hash '#', or space ' '.
- Each tested sentence will be a sequence of characters c that end with the character '#'.
 • Each tested sentence will have a length in the range $[1, 200]$.
- The words in each input sentence are separated by single spaces.
- At most 5000 calls will be made to input.

Community Solutions (Python)

- WalkCC

Round 6 · Mid · Hard

p.2711

```

class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.s: str | None = None
        self.time = 0
        self.top3: list[TrieNode] = []

    def __lt__(self, other):
        if self.time == other.time:
            return self.s < other.s

        return self.time > other.time

    def update(self, node) -> None:
        if node not in self.top3:
            self.top3.append(node)
            self.top3.sort()
            if len(self.top3) > 3:
                self.top3.pop()

class AutocompleteSystem:
    def __init__(self, sentences: list[str], times: list[int]):
        self.root = TrieNode()
        self.curr = self.root
        self.s: list[str] = []

        for sentence, time in zip(sentences, times):
            self._insert(sentence, time)

    def input(self, c: str) -> list[str]:

        if c == '#':
            self._insert(''.join(self.s), 1)
            self.curr = self.root
            self.s = []
            return []

        self.s.append(c)

        if self.curr:
            self.curr = self.curr.children.get(c, None)

```

Round 6 · Mid · Hard

p.2712


```

if not self.curr:
    return []
return [node.s for node in self.curr.top3]

def _insert(self, sentence: str, time: int) -> None:
    node = self.root
    for c in sentence:
        node = node.children.setdefault(c, TrieNode())
    node.s = sentence
    node.time += time

```

```

leaf = node
node: TrieNode = self.root
for c in sentence:
    node = node.children[c]
    node.update(leaf)

```

• LeetDoocs

```

class Trie:
    def __init__(self):
        self.children = [None] * 27
        self.v = 0
        self.w = ''

    def insert(self, w, t):
        node = self
        for c in w:
            idx = 26 if c == ' ' else ord(c) - ord('a')

            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
        node.v += t
        node.w = w

    def search(self, pref):
        node = self
        for c in pref:
            idx = 26 if c == ' ' else ord(c) - ord('a')

```

```

if node.children[idx] is None:
    return None
node = node.children[idx]
return node

```

```

class AutocompleteSystem:
    def __init__(self, sentences: List[str], times: List[int]):
        self.trie = Trie()
        for a, b in zip(sentences, times):

```

```

        self.trie.insert(a, b)
        self.t = []

```

```

    def input(self, c: str) -> List[str]:
        def dfs(node):
            if node is None:
                return
            if node.v:
                res.append((node.v, node.w))
            for nxt in node.children:

```

```

                dfs(nxt)

```

```

            if c == '#':
                s = ''.join(self.t)
                self.trie.insert(s, 1)
                self.t = []
                return []

```

```

        res = []
        self.t.append(c)

```

```

        node = self.trie.search(''.join(self.t))
        if node is None:
            return res
        dfs(node)
        res.sort(key=lambda x: (-x[0], x[1]))
        return [v[1] for v in res[:3]]

```

```

# Your AutocompleteSystem object will be instantiated and called as such:
# obj = AutocompleteSystem(sentences, times)

```

Round 6 · Mid · Hard

p.2713

Round 6 · Mid · Hard

p.2714

```
# param_1 = obj.input(c)
```

• SimplyLeet

```
class TrieNode:
    def __init__(self):
        # Mapping from character to TrieNode
        self.children = {}
        # Dictionary mapping sentence to its frequency count
        self.counts = {}

class AutocompleteSystem:
    def __init__(self, sentences, times):
        self.root = TrieNode()

        self.current_node = self.root
        self.current_input = ""
        # Initialize Trie with the given sentences and their times
        for sentence, time in zip(sentences, times):
            self.addSentence(sentence, time)

    def addSentence(self, sentence, freq):
        node = self.root
        for char in sentence:
            if char not in node.children:
                node.children[char] = TrieNode()
            node = node.children[char]
            # Update sentence frequency at each node
            node.counts[sentence] = node.counts.get(sentence, 0) + freq

    def input(self, c):
        results = []
        if c == '#':
            # End of input: add current sentence to Trie
            self.addSentence(self.current_input, 1)
```

Round 6 · Mid · Hard

p.2715

```
# Reset the current input and traversal pointer
self.current_input = ""
self.current_node = self.root
return results

self.current_input += c
# Move to the next Trie node if it exists
if self.current_node:
    self.current_node = self.current_node.children.get(c, None)
if not self.current_node:

    return results

# Retrieve sentences and sort them by frequency (descending) and
lexicographical order (ascending)
data = list(self.current_node.counts.items())
data.sort(key=lambda x: (-x[1], x[0]))
for i in range(min(3, len(data))):
    results.append(data[i][0])
return results
```

```
# Example usage:
```

```
# auto = AutocompleteSystem(["i love you", "island", "iroman", "i love
leetcode"], [5, 3, 2, 2])
# print(auto.input("i"))
# print(auto.input(" "))
# print(auto.input("a"))
# print(auto.input("#"))
```

• LC.ca

```
class Trie:
    def __init__(self):
        # 26+1, extra 1 for space ' ' node
        self.children = [None] * 27
        self.v = 0
        self.w = ''

    def insert(self, w, t):
        node = self
        for c in w:
```

Round 6 · Mid · Hard

p.2716

```

        idx = 26 if c == ' ' else ord(c) - ord('a')
        if node.children[idx] is None:
            node.children[idx] = Trie()
        node = node.children[idx]
    node.v += t
    node.w = w

def search(self, pref):
    node = self
    for c in pref:
        idx = 26 if c == ' ' else ord(c) - ord('a')
        if node.children[idx] is None:
            return None
        node = node.children[idx]
    return node

class AutocompleteSystem:
    def __init__(self, sentences: List[str], times: List[int]):
        self.trie = Trie()

        for a, b in zip(sentences, times):
            self.trie.insert(a, b)
        self.t = [] # target

    def input(self, c: str) -> List[str]:
        def dfs(node):
            if node is None:
                return
            if node.v:
                res.append((node.v, node.w))

            for nxt in node.children:
                dfs(nxt)

        if c == '#':
            s = ''.join(self.t)
            self.trie.insert(s, 1)
            self.t = []
            return []

        res = []

```

Round 6 · Mid · Hard

p.2717

```

        self.t.append(c)
        node = self.trie.search(''.join(self.t))
        if node is None:
            return res
        dfs(node)
        res.sort(key=lambda x: (-x[0], x[1]))
        return [v[1] for v in res[:3]]

# Your AutocompleteSystem object will be instantiated and called as such:
# obj = AutocompleteSystem(sentences, times)
# param_1 = obj.input(c)

```

◆ Quick Review

Key Insights

- Use a Trie to efficiently store and retrieve sentences based on their prefixes.
- Maintain a mapping at each Trie node to record sentences (or their frequencies) that pass through that node.
- Use sorting or a min-heap to select the top three sentences based on their frequency (hot degree) and lexicographical order if frequencies tie.
- On receiving '#', update the Trie with the current sentence and reset the current search state.

Space & Time

Round 6 · Mid · Hard

p.2718

Time Complexity: $O(L * \log k)$ per character input query, where L is the length of the current prefix and k is 3 (constant factor), thus ensuring efficient autocompletion.

Space Complexity: $O(N * L)$, where N is the number of unique sentences and L is the average sentence length, for maintaining the Trie.

Solution

The solution revolves around building a Trie where each node contains:

- A mapping to its child nodes.
- A hash map that records all sentences passing through the node along with their occurrence counts.

For each character entered (except '#'), the system:

- Traverses the Trie to find the current prefix node.
- Retrieves stored sentences at that node and sorts them based on the following criteria: descending by frequency and ascending lexicographically if frequencies are equal.
- Returns the top three sentences from the sorted result.

When the user inputs '#', the current sentence (tracked as user input so far) is added or its frequency updated in the Trie, and the input state resets.

Data Structure Design

☐ #715 Range Module

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Design · Segment Tree · Ordered Set
Lists: AlgoMaster 600

Problem

A Range Module is a module that tracks ranges of numbers. Design a data structure to track the ranges represented as half-open intervals and query about them.

A half-open interval $[left, right)$ denotes all the real numbers x where $left \leq x < right$.

Implement the RangeModule class:

- `RangeModule()` Initializes the object of the data structure.
- `void addRange(int left, int right)` Adds the half-open interval $[left, right)$, tracking every real number in that interval. Adding an interval that partially overlaps with currently tracked numbers should add any numbers in the interval $[left, right)$ that are not already

Round 6 · Mid · Hard

p.2721

tracked.

- `boolean queryRange(int left, int right)` Returns true if every real number in the interval $[left, right)$ is currently being tracked, and false otherwise.
- `void removeRange(int left, int right)` Stops tracking every real number currently being tracked in the half-open interval $[left, right)$.

Example 1:

```
Input
["RangeModule", "addRange", "removeRange", "queryRange", "queryRange",
 "queryRange"]
[[], [10, 20], [14, 16], [10, 14], [13, 15], [16, 17]]
Output
[null, null, null, true, false, true]
```

```
Explanation
RangeModule rangeModule = new RangeModule();
```

Constraints:

- $1 \leq left < right \leq 10^9$
- At most 104 calls will be made to `addRange`, `queryRange`, and `removeRange`.

Community Solutions (Python)

- WalkCC

Round 6 · Mid · Hard

p.2722

```
class RangeModule:
    def __init__(self):
        self.A = []

    def addRange(self, left: int, right: int) -> None:
        i = bisect_left(self.A, left)
        j = bisect_right(self.A, right)
        self.A[i:j] = [left] * (i % 2 == 0) + [right] * (j % 2 == 0)

    def queryRange(self, left: int, right: int) -> bool:

        i = bisect_right(self.A, left)
        j = bisect_left(self.A, right)
        return i == j and i % 2 == 1

    def removeRange(self, left: int, right: int) -> None:
        i = bisect_left(self.A, left)
        j = bisect_right(self.A, right)
        self.A[i:j] = [left] * (i % 2 == 1) + [right] * (j % 2 == 1)
```

• LeetCode

```
class Node:
    __slots__ = ['left', 'right', 'add', 'v']

    def __init__(self):
        self.left = None
        self.right = None
        self.add = 0
        self.v = False

class SegmentTree:
    __slots__ = ['root']

    def __init__(self):
        self.root = Node()

    def modify(self, left, right, v, l=1, r=int(1e9), node=None):
        if node is None:
            node = self.root
        if l >= left and r <= right:
```

```
        if v == 1:
            node.add = 1
            node.v = True
        else:
            node.add = -1
            node.v = False
        return
    self.pushdown(node)
    mid = (l + r) >> 1
    if left <= mid:

        self.modify(left, right, v, l, mid, node.left)
    if right > mid:
        self.modify(left, right, v, mid + 1, r, node.right)
    self.pushup(node)

    def query(self, left, right, l=1, r=int(1e9), node=None):
        if node is None:
            node = self.root
        if l >= left and r <= right:
            return node.v

        self.pushdown(node)
        mid = (l + r) >> 1
        v = True
        if left <= mid:
            v = v and self.query(left, right, l, mid, node.left)
        if right > mid:
            v = v and self.query(left, right, mid + 1, r, node.right)
        return v

    def pushup(self, node):
```

```

node.v = bool(node.left and node.left.v and node.right and node.right.
v)

def pushdown(self, node):
    if node.left is None:
        node.left = Node()
    if node.right is None:
        node.right = Node()
    if node.add:
        node.left.add = node.right.add = node.add
        node.left.v = node.add == 1

    node.right.v = node.add == 1
    node.add = 0

class RangeModule:
    def __init__(self):
        self.tree = SegmentTree()

    def addRange(self, left: int, right: int) -> None:
        self.tree.modify(left, right - 1, 1)

    def queryRange(self, left: int, right: int) -> bool:
        return self.tree.query(left, right - 1)

    def removeRange(self, left: int, right: int) -> None:
        self.tree.modify(left, right - 1, -1)

# Your RangeModule object will be instantiated and called as such:
# obj = RangeModule()

# obj.addRange(left, right)
# param_2 = obj.queryRange(left, right)
# obj.removeRange(left, right)

```

• SimplyLeet

Round 6 · Mid · Hard

p.2725

```

# Python implementation with detailed line-by-line comments
class RangeModule:
    def __init__(self):
        # List to hold the non-overlapping intervals sorted by start
        self.intervals = []

    def addRange(self, left: int, right: int) -> None:
        new_intervals = []
        i = 0
        # Add all intervals that end before 'left'
        while i < len(self.intervals) and self.intervals[i][1] < left:
            new_intervals.append(self.intervals[i])
            i += 1

        # Merge all overlapping intervals with [left, right)
        while i < len(self.intervals) and self.intervals[i][0] <= right:
            left = min(left, self.intervals[i][0])
            right = max(right, self.intervals[i][1])
            i += 1

        # Add the merged interval
        new_intervals.append([left, right])

        # Append the remaining intervals that start after 'right'
        while i < len(self.intervals):
            new_intervals.append(self.intervals[i])
            i += 1

        self.intervals = new_intervals

    def queryRange(self, left: int, right: int) -> bool:
        # Binary search to find the position where 'left' would be inserted
        lo, hi = 0, len(self.intervals)
        while lo < hi:
            mid = (lo + hi) // 2
            if self.intervals[mid][1] < left:
                lo = mid + 1
            else:
                hi = mid
        # If no interval covers [left, right), return false

```

Round 6 · Mid · Hard

p.2726

```

        if lo < len(self.intervals) and self.intervals[lo][0] <= left and self
.intervals[lo][1] >= right:
            return True
        return False

def removeRange(self, left: int, right: int) -> None:
    new_intervals = []
    i = 0
    # Add all intervals that end before 'left'
    while i < len(self.intervals) and self.intervals[i][1] <= left:
        new_intervals.append(self.intervals[i])

        i += 1

    # Process intervals that overlap with [left, right)
    while i < len(self.intervals) and self.intervals[i][0] < right:
        # If part of the interval is left of 'left', keep it
        if self.intervals[i][0] < left:
            new_intervals.append([self.intervals[i][0], left])
        # If part of the interval is right of 'right', keep it
        if self.intervals[i][1] > right:
            new_intervals.append([right, self.intervals[i][1]])

        i += 1

    # Append the remaining intervals that start after 'right'
    while i < len(self.intervals):
        new_intervals.append(self.intervals[i])
        i += 1

    self.intervals = new_intervals

```

Data Structure Design

#716 Max Stack

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Stack · Design · Doubly-Linked List · Ordered Set

Lists: Denny Zhang

Problem

Design a max stack data structure that supports the stack operations and supports finding the stack's maximum element.

Implement the MaxStack class:

- **MaxStack()** Initializes the stack object.
- **void push(int x)** Pushes element x onto the stack.
- **int pop()** Removes the element on top of the stack and returns it.
- **int top()** Gets the element on the top of the stack without removing it.
- **int peekMax()** Retrieves the maximum element in the stack without removing it.

- **int popMax()** Retrieves the maximum element in the stack and removes it. If there is more than one maximum element, only remove the top-most one.

You must come up with a solution that supports $O(1)$ for each top call and $O(\log n)$ for each other call.

Example 1:

```
Input
["MaxStack", "push", "push", "push", "top", "popMax", "top", "peekMax", "pop",
"top"]
[[], [5], [1], [5], [], [], [], [], [], []]
Output
[null, null, null, null, 5, 5, 1, 5, 1, 5]
```

```
Explanation
MaxStack stk = new MaxStack();
```

Constraints:

- $-107 \leq x \leq 107$
- At most 105 calls will be made to push, pop, top, peekMax, and popMax.
- There will be at least one element in the stack when pop, top, peekMax, or popMax is called.

Community Solutions (Python)

- SimplyLeet

Round 6 · Mid · Hard

p.2729

```
# Python solution implementing the MaxStack data structure
# We use collections.OrderedDict for demonstration.
# However, in Python we do not have a built in balanced BST;
# one can use "sortedcontainers" or manually maintain a heap and dictionary.
# For clarity, we simulate an ordered map with a TreeMap-like structure using
sorted list of keys.
# In production, one may use "SortedDict" from sortedcontainers.

from collections import defaultdict

class Node:

    def __init__(self, val):
        self.val = val
        self.prev = None
        self.next = None

class DoubleLinkedList:
    def __init__(self):
        # Create dummy head and tail nodes.
        self.head = Node(0)
        self.tail = Node(0)

        self.head.next = self.tail
        self.tail.prev = self.head

    def add_last(self, node):
        # Add node right before the tail.
        node.prev = self.tail.prev
        node.next = self.tail
        self.tail.prev.next = node
        self.tail.prev = node
```

Round 6 · Mid · Hard

p.2730

```

def pop(self, node=None):
    # If no node is provided, pop from the end.
    if not node:
        node = self.tail.prev
        node.prev.next = node.next
        node.next.prev = node.prev
        return node

def top(self):
    # Return the last node's value.

    return self.tail.prev.val

class MaxStack:
    def __init__(self):
        self.dll = DoubleLinkedList()
        # Dictionary mapping value to list of nodes
        self.valToNodes = defaultdict(list)
        # Sorted list to simulate ordered set of keys.
        self.sortedKeys = []

def push(self, x: int) -> None:
    node = Node(x)
    self.dll.add_last(node)
    self.valToNodes[x].append(node)
    # Insert x into sortedKeys if new, otherwise maintain count.
    if len(self.valToNodes[x]) == 1:
        # binary insertion into sortedKeys
        lo, hi = 0, len(self.sortedKeys)
        while lo < hi:
            mid = (lo + hi) // 2

            if self.sortedKeys[mid] < x:
                lo = mid + 1
            else:
                hi = mid
        self.sortedKeys.insert(lo, x)

def pop(self) -> int:
    node = self.dll.pop()
    x = node.val
    self.valToNodes[x].pop()

```

Round 6 · Mid · Hard

p.2731

```

if not self.valToNodes[x]:
    # Remove x from sortedKeys
    self.sortedKeys.remove(x)
    return x

def top(self) -> int:
    return self.dll.top()

def peekMax(self) -> int:
    # The maximum is the last element in sortedKeys.

    return self.sortedKeys[-1]

def popMax(self) -> int:
    max_val = self.peekMax()
    # Get the most recent node with max_val.
    node = self.valToNodes[max_val].pop()
    if not self.valToNodes[max_val]:
        self.sortedKeys.pop() # removes the last element
    # Remove the node from the doubly-linked list.
    self.dll.pop(node)

    return max_val

# Example usage:
# stk = MaxStack()
# stk.push(5)
# stk.push(1)
# stk.push(5)
# print(stk.top()) # returns 5
# print(stk.popMax()) # returns 5
# print(stk.top()) # returns 1

# print(stk.peekMax()) # returns 5
# print(stk.pop()) # returns 1
# print(stk.top()) # returns 5

```

☐ • LC.ca

Round 6 · Mid · Hard

p.2732

```

class MaxStack(object):

    def __init__(self):
        """
        initialize your data structure here.
        """
        self.stack = []
        self.max_stack = []
        # or the same trick in min-stack
        # self.max_stack = [-math.inf]

    def push(self, x):
        """
        :type x: int
        :rtype: void
        """
        self.stack.append(x)
        self.max_stack.append(max(x, self.max_stack[-1]) if self.max_stack else x)

    def pop(self):
        """
        :rtype: int
        """
        if len(self.stack) != 0:
            self.max_stack.pop(-1)
            return self.stack.pop(-1)

    def top(self):
        """
        :rtype: int

```

Round 6 · Mid · Hard

p.2733

```

        """
        return self.stack[-1] if self.stack else None

    def peekMax(self):
        """
        :rtype: int
        """
        if len(self.max_stack) != 0:
            return self.max_stack[-1]

    def popMax(self):
        """
        :rtype: int
        """
        val = self.peekMax()
        buff = []
        while self.top() != val:
            # re-use pop(), ops on both max_stack and stack
            buff.append(self.pop())
        self.pop()

        while len(buff) != 0:
            # re-use push(), no need to save buffer for max-stack
            self.push(buff.pop(-1))
        return val

#####

from sortedcontainers import SortedList

class Node:

    def __init__(self, val=0):
        self.val = val
        self.prev: Union[Node, None] = None
        self.next: Union[Node, None] = None

class DoubleLinkedList:
    def __init__(self):
        self.head = Node()
        self.tail = Node()

```

Round 6 · Mid · Hard

p.2734

```

        self.head.next = self.tail
        self.tail.prev = self.head

    def append(self, val) -> Node:
        node = Node(val)
        node.next = self.tail
        node.prev = self.tail.prev
        self.tail.prev = node
        node.prev.next = node
        return node

    @staticmethod
    def remove(node) -> Node:
        node.prev.next = node.next
        node.next.prev = node.prev
        return node

    def pop(self) -> Node:
        return self.remove(self.tail.prev)

    def peek(self):
        return self.tail.prev.val

class MaxStack:
    def __init__(self):
        self.stk = DoubleLinkedList()
        self.sl = SortedList(key=lambda x: x.val)

    def push(self, x: int) -> None:

        node = self.stk.append(x)
        self.sl.add(node)

    def pop(self) -> int:
        node = self.stk.pop()
        self.sl.remove(node)
        return node.val

    def top(self) -> int:
        return self.stk.peek()

```

Round 6 · Mid · Hard

p.2735

```

    def peekMax(self) -> int:
        return self.sl[-1].val

    def popMax(self) -> int:
        node = self.sl.pop()
        DoubleLinkedList.remove(node)
        return node.val

```

```

# Your MaxStack object will be instantiated and called as such:
# obj = MaxStack()
# obj.push(x)
# param_2 = obj.pop()
# param_3 = obj.top()
# param_4 = obj.peekMax()
# param_5 = obj.popMax()

```

◆ Quick Review

Key Insights

- Use a doubly-linked list to support fast insertion and removal from the stack.
- Maintain a balanced tree (or ordered dictionary/multiset) to quickly locate the maximum element in $O(\log n)$ time.
- Map each value to its corresponding node(s) in the doubly-linked list, so that when popMax is called, you can immediately remove the corresponding node from the list.

Round 6 · Mid · Hard

p.2736

- The double-linking allows removal of arbitrary nodes in $O(1)$, once you have the reference.

Space & Time

Time Complexity:

- push: $O(\log n)$ due to insertion in the ordered structure.
- pop: $O(1)$ for removal from the doubly-linked list, plus $O(\log n)$ for updating the tree.
- top: $O(1)$
- peekMax: $O(1)$ or $O(\log n)$ depending on the tree implementation.
- popMax: $O(\log n)$ for looking up and removal from the tree, $O(1)$ for doubly-linked list deletion.

Space Complexity:

- $O(n)$ for storing all nodes in the doubly-linked list and the tree structure mapping values to nodes.

Solution

We use two data structures:

Round 6 · Mid · Hard

p.2737

- A doubly-linked list to store the elements in stack order. This supports fast push and pop operations and allows $O(1)$ removal when given a reference.
- An ordered structure (like a balanced BST, TreeMap in Java, or SortedList in Python) that maps each value to a list of nodes that hold that value in the linked list. This allows us to quickly determine the maximum element. When there are duplicates, we remove the one closest to the top by storing nodes in order. The trick is to keep these two structures synchronized. When an element is pushed, add its node to both the doubly-linked list and the ordered structure. When an element is removed (either via pop or popMax), remove its node from both structures.

Round 6 · Mid · Hard

p.2738

Data Structure Design

#895 Maximum Frequency Stack

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode

Hash Table · Stack · Design

Lists: Grind 169

Problem

Design a stack-like data structure to push elements to the stack and pop the most frequent element from the stack.

Implement the FreqStack class:

- FreqStack() constructs an empty frequency stack.
- void push(int val) pushes an integer val onto the top of the stack.
- int pop() removes and returns the most frequent element in the stack. If there is a tie for the most frequent element, the element closest to the stack's top is removed and returned.

Example 1:

Round 6 · Mid · Hard

p.2739

Input
 ["FreqStack", "push", "push", "push", "push", "push", "push", "pop", "pop", "pop", "pop"]
 [[], [5], [7], [5], [7], [4], [5], [], [], [], []]
Output
 [null, null, null, null, null, null, null, 5, 7, 5, 4]

Explanation
 FreqStack freqStack = new FreqStack();

Constraints:

- $0 \leq \text{val} \leq 109$
- At most $2 * 10^4$ calls will be made to push and pop.
- It is guaranteed that there will be at least one element in the stack before calling pop.

Community Solutions (Python)

• WalkCC

```
class FreqStack:
    def __init__(self):
        self.maxFreq = 0
        self.count = collections.Counter()
        self.countToStack = collections.defaultdict(list)

    def push(self, val: int) -> None:
        self.count[val] += 1
        self.countToStack[self.count[val]].append(val)
        self.maxFreq = max(self.maxFreq, self.count[val])

    def pop(self) -> int:
        val = self.countToStack[self.maxFreq].pop()
        self.count[val] -= 1
        if not self.countToStack[self.maxFreq]:
            self.maxFreq -= 1
        return val
```

Round 6 · Mid · Hard

p.2740

• LeetCode

```
class FreqStack:
    def __init__(self):
        self.cnt = defaultdict(int)
        self.q = []
        self.ts = 0

    def push(self, val: int) -> None:
        self.ts += 1
        self.cnt[val] += 1
        heappush(self.q, (-self.cnt[val], -self.ts, val))

    def pop(self) -> int:
        val = heappop(self.q)[2]
        self.cnt[val] -= 1
        return val
```

Your FreqStack object will be instantiated and called as such:

```
# obj = FreqStack()
# obj.push(val)
```

```
# param_2 = obj.pop()
```

```
class FreqStack:
    def __init__(self):
        self.cnt = defaultdict(int)
        self.d = defaultdict(list)
        self.mx = 0

    def push(self, val: int) -> None:
        self.cnt[val] += 1
        self.d[self.cnt[val]].append(val)
        self.mx = max(self.mx, self.cnt[val])
```

```
def pop(self) -> int:
    val = self.d[self.mx].pop()
    self.cnt[val] -= 1
    if not self.d[self.mx]:
        self.mx -= 1
    return val
```

Your FreqStack object will be instantiated and called as such:

```
# obj = FreqStack()
# obj.push(val)
# param_2 = obj.pop()
```

• SimplyLeet

Python implementation of FreqStack

```
class FreqStack:
    def __init__(self):
        # Dictionary mapping value to its frequency
        self.freq = {}
        # Dictionary mapping frequency to a list (stack) of values
        self.group = {}
        # Variable tracking the current maximum frequency in the stack
        self.maxfreq = 0
```

```
def push(self, val: int) -> None:
    # Update frequency count for the value
    f = self.freq.get(val, 0) + 1
    self.freq[val] = f
    # Update the maximum frequency if necessary
    if f > self.maxfreq:
        self.maxfreq = f
    # Append the value to the appropriate frequency group
    if f not in self.group:
```

```

        self.group[f] = []
        self.group[f].append(val)

    def pop(self) -> int:
        # Remove the value from the stack of the current maximum frequency
        val = self.group[self.maxfreq].pop()
        # Decrement the frequency count for the value
        self.freq[val] -= 1
        # If no more elements have the current max frequency, decrease maxfreq
        if not self.group[self.maxfreq]:

            self.maxfreq -= 1
        return val

```

• LC.ca

```

class FreqStack:
    def __init__(self):
        self.cnt = defaultdict(int)
        self.d = defaultdict(list)
        self.mx = 0

    def push(self, val: int) -> None:
        self.cnt[val] += 1
        self.d[self.cnt[val]].append(val)
        self.mx = max(self.mx, self.cnt[val])

    def pop(self) -> int:
        val = self.d[self.mx].pop()
        self.cnt[val] -= 1
        if not self.d[self.mx]:
            self.mx -= 1
        return val

```

Your FreqStack object will be instantiated and called as such:

```

# obj = FreqStack()
# obj.push(val)
# param_2 = obj.pop()

```

◆ Quick Review

Key Insights

- Use a hash map to count the frequency of each element.
- Maintain another hash map that maps frequencies to stacks (lists) of elements.
- Track the current maximum frequency to enable O(1) retrieval of the most frequent element.
- When pushing an element, update its frequency and add it to the corresponding frequency stack.
- When popping an element, remove it from the stack corresponding to the maximum frequency and update the frequency count; if that stack becomes empty, decrease the current maximum frequency.

Space & Time

Time Complexity: O(1) per push and pop.

Space Complexity: O(n), where n is the number of elements pushed onto the stack.

Solution

The solution involves using two main data structures:

- A hash map (freq) to store the frequency count for each value.
- A hash map (group) that maps a frequency to a stack (list) of values that have that frequency.

During a push:

- Increment the frequency count of the value.
- Push the value onto the stack corresponding to its updated frequency.
- Update the maximum frequency if necessary.

During a pop:

- Pop the top element from the stack corresponding to the current maximum frequency.
- Decrement its frequency count in the freq map.
- If the frequency stack becomes empty after popping, decrement the current maximum frequency. This design enables the retrieval of the most frequent element in constant time while dealing with frequency ties by returning the element closest to the top.

Data Structure Design

□ #2296 Design a Text Editor

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · String · Stack · Design ·
Simulation · Doubly-Linked List

Lists: AlgoMaster 600

Problem

Design a text editor with a cursor that can do the following:

- Add text to where the cursor is.
- Delete text from where the cursor is (simulating the backspace key).
- Move the cursor either left or right.

When deleting text, only characters to the left of the cursor will be deleted. The cursor will also remain within the actual text and cannot be moved beyond it. More formally, we have that $0 \leq \text{cursor.position} \leq \text{currentText.length}$ always holds.

Implement the TextEditor class:

- **TextEditor()** Initializes the object with empty text.
- **void addText(string text)** Appends text to where the cursor is. The cursor ends to the right of text.
- **int deleteText(int k)** Deletes k characters to the left of the cursor. Returns the number of characters actually deleted.
- **string cursorLeft(int k)** Moves the cursor to the left k times. Returns the last min(10, len) characters to the left of the cursor, where len is the number of characters to the left of the cursor.
- **string cursorRight(int k)** Moves the cursor to the right k times. Returns the last min(10, len) characters to the left of the cursor, where len is the number of characters to the left of the cursor.

Example 1:

Input
 ["TextEditor", "addText", "deleteText", "addText", "cursorRight", "cursorLeft", "deleteText", "cursorLeft", "cursorRight"]
 [[], ["leetcode"], [4], ["practice"], [3], [8], [10], [2], [6]]

Output
 [null, null, 4, null, "etpractice", "leet", 4, "", "practi"]

Explanation
 TextEditor textEditor = new TextEditor(); // The current text is "|". (The '|' character represents the cursor)

Constraints:

- $1 \leq \text{text.length}$, $k \leq 40$
- text consists of lowercase English letters.
- At most $2 * 10^4$ calls in total will be made to addText, deleteText, cursorLeft and cursorRight.

Follow-up: Could you find a solution with time complexity of $O(k)$ per call?

Community Solutions (Python)

• WalkCC

```
class TextEditor:
    def __init__(self):
        self.s = []
        self.stack = []

    def addText(self, text: str) -> None:
        for c in text:
            self.s.append(c)

    def deleteText(self, k: int) -> int:

        numDeleted = min(k, len(self.s))
        for _ in range(numDeleted):
            self.s.pop()
        return numDeleted

    def cursorLeft(self, k: int) -> str:
        while self.s and k > 0:
            self.stack.append(self.s.pop())
            k -= 1
        return self._getString()
```

```
def cursorRight(self, k: int) -> str:
    while self.stack and k > 0:
        self.s.append(self.stack.pop())
        k -= 1
    return self._getString()

def _getString(self) -> str:
    if len(self.s) < 10:
        return ''.join(self.s)

    return ''.join(self.s[-10:])
```

• LeetCode

```
class TextEditor:
    def __init__(self):
        self.left = []
        self.right = []

    def addText(self, text: str) -> None:
        self.left.extend(list(text))

    def deleteText(self, k: int) -> int:
        k = min(k, len(self.left))

        for _ in range(k):
            self.left.pop()
        return k

    def cursorLeft(self, k: int) -> str:
        k = min(k, len(self.left))
        for _ in range(k):
            self.right.append(self.left.pop())
        return ''.join(self.left[-10:])
```

Round 6 · Mid · Hard

p.2749

```
def cursorRight(self, k: int) -> str:
    k = min(k, len(self.right))
    for _ in range(k):
        self.left.append(self.right.pop())
    return ''.join(self.left[-10:])

# Your TextEditor object will be instantiated and called as such:
# obj = TextEditor()
# obj.addText(text)

# param_2 = obj.deleteText(k)
# param_3 = obj.cursorLeft(k)
# param_4 = obj.cursorRight(k)
```

• SimplyLeet

```
class TextEditor:
    def __init__(self):
        # left_stack contains text left to the cursor; right_stack contains
        # text right to the cursor.
        self.left_stack = []
        self.right_stack = []

    def addText(self, text: str) -> None:
        # Append each character to left_stack.
        for ch in text:
            self.left_stack.append(ch)

    def deleteText(self, k: int) -> int:
        # Remove up to k characters from the left_stack.
        count = 0
        while self.left_stack and count < k:
            self.left_stack.pop()
            count += 1
        return count

    def cursorLeft(self, k: int) -> str:
```

Round 6 · Mid · Hard

p.2750

```
# Move at most k characters from left_stack to right_stack.
count = 0
while self.left_stack and count < k:
    self.right_stack.append(self.left_stack.pop())
    count += 1
# Get last min(10, len(left_stack)) characters and return them as a
string.
return "".join(self.left_stack[-10:])
```

```
def cursorRight(self, k: int) -> str:
    # Move at most k characters from right_stack back into left_stack.
```

```
count = 0
while self.right_stack and count < k:
    self.left_stack.append(self.right_stack.pop())
    count += 1
# Get last min(10, len(left_stack)) characters and return them as a
string.
return "".join(self.left_stack[-10:])
```

```
# Example usage:
# textEditor = TextEditor()
# textEditor.addText("leetcode")
```

```
# print(textEditor.deleteText(4))
# textEditor.addText("practice")
# print(textEditor.cursorRight(3))
# print(textEditor.cursorLeft(8))
# print(textEditor.deleteText(10))
# print(textEditor.cursorLeft(2))
# print(textEditor.cursorRight(6))
```

Round 6 · Mid · Hard

p.2751

Greedy

Round 6 · Mid · Hard · 4 questions

Round 6 · Mid · Hard

p.2752

Greedy

#135 Candy

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Greedy
 Lists: AlgoMaster 600

Problem

There are n children standing in a line. Each child is assigned a rating value given in the integer array `ratings`.

You are giving candies to these children subjected to the following requirements:

- Each child must have at least one candy.
- Children with a higher rating get more candies than their neighbors.

Return the minimum number of candies you need to have to distribute the candies to the children.

Example 1:

Input: `ratings = [1,0,2]`
 Output: 5
 Explanation: You can allocate to the first, second and third child with 2, 1, 2 candies respectively.

Example 2:

Round 6 · Mid · Hard

p.2753

Input: `ratings = [1,2,2]`
 Output: 4
 Explanation: You can allocate to the first, second and third child with 1, 2, 1 candies respectively.
 The third child gets 1 candy because it satisfies the above two conditions.

Constraints:

- $n == \text{ratings.length}$
- $1 \leq n \leq 2 * 10^4$
- $0 \leq \text{ratings}[i] \leq 2 * 10^4$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def candy(self, ratings: list[int]) -> int:
        n = len(ratings)

        ans = 0
        l = [1] * n
        r = [1] * n

        for i in range(1, n):
            if ratings[i] > ratings[i - 1]:
                l[i] = l[i - 1] + 1

        for i in range(n - 2, -1, -1):
            if ratings[i] > ratings[i + 1]:
                r[i] = r[i + 1] + 1

        for a, b in zip(l, r):
            ans += max(a, b)

        return ans
```

• LeetDoocs

Round 6 · Mid · Hard

p.2754

```
class Solution:
    def candy(self, ratings: List[int]) -> int:
        n = len(ratings)
        left = [1] * n
        right = [1] * n
        for i in range(1, n):
            if ratings[i] > ratings[i - 1]:
                left[i] = left[i - 1] + 1
        for i in range(n - 2, -1, -1):
            if ratings[i] > ratings[i + 1]:
                right[i] = right[i + 1] + 1
        return sum(max(a, b) for a, b in zip(left, right))
```

• SimplyLeet

```
# Function to distribute candies according to ratings.
def candy(ratings):
    n = len(ratings)
    candies = [1] * n # Initialize with 1 candy for each child.

    # Left-to-right pass: ensure right neighbor condition.
    for i in range(1, n):
        if ratings[i] > ratings[i - 1]:
            candies[i] = candies[i - 1] + 1

    # Right-to-left pass: ensure left neighbor condition.
    for i in range(n - 2, -1, -1):
        if ratings[i] > ratings[i + 1]:
            candies[i] = max(candies[i], candies[i + 1] + 1)

    # Sum up all distributed candies.
    return sum(candies)

# Example usage:
ratings = [1, 0, 2]

print(candy(ratings)) # Expected Output: 5
```

Greedy

□ #757 Set Intersection Size At Least Two **HAR**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Greedy · Sorting
Lists: AlgoMaster 600

Problem

You are given a 2D integer array intervals where intervals[i] = [starti, endi] represents all the integers from starti to endi inclusively.

A containing set is an array nums where each interval from intervals has at least two integers in nums.

- For example, if intervals = [[1,3], [3,7], [8,9]], then [1,2,4,7,8,9] and [2,3,4,8,9] are containing sets.

Return the minimum possible size of a containing set.

Example 1:

Input: intervals = [[1,3],[3,7],[8,9]]
Output: 5
Explanation: let nums = [2, 3, 4, 8, 9].
It can be shown that there cannot be any containing array of size 4.

Example 2:

Input: intervals = [[1,3],[1,4],[2,5],[3,5]]
 Output: 3
 Explanation: let nums = [2, 3, 4].
 It can be shown that there cannot be any containing array of size 2.

Example 3:

Input: intervals = [[1,2],[2,3],[2,4],[4,5]]
 Output: 5
 Explanation: let nums = [1, 2, 3, 4, 5].
 It can be shown that there cannot be any containing array of size 4.

Constraints:

- $1 \leq \text{intervals.length} \leq 3000$
- $\text{intervals}[i].\text{length} == 2$
- $0 \leq \text{start}_i < \text{end}_i \leq 108$

Community Solutions (Python)

• LeetCode

```
class Solution:
    def intersectionSizeTwo(self, intervals: List[List[int]]) -> int:
        intervals.sort(key=lambda x: (x[1], -x[0]))
        s = e = -1
        ans = 0
        for a, b in intervals:
            if a <= s:
                continue
            if a > e:
                ans += 2
                s, e = b - 1, b
            else:
                ans += 1
                s, e = e, b
        return ans
```

• SimplyLeet

Round 6 · Mid · Hard

p.2757

```
# Python code solution
from typing import List

class Solution:
    def intersectionSizeTwo(self, intervals: List[List[int]]) -> int:
        # sort intervals by end ascending, start descending when ends are equal
        intervals.sort(key=lambda x: (x[1], -x[0]))

        result = 0 # count of numbers in the chosen set
        # last two selected numbers; initialize them to values that would not
        # interfere with any interval.

        # They represent the two largest numbers added so far that are used to
        # cover intervals.
        # We initialize them to values that are "unused".
        p, q = -1, -1

        for start, end in intervals:
            # Three cases depending on how many of {p, q} fall into the current
            # interval [start, end]
            # Case 1: Both numbers are already in the interval, no addition
            # required

            if start <= p:
                continue
            # Case 2: Only one of the previous numbers is in the interval

            if start <= q:
                # We need to add one number to cover interval, choose end as it
                # is the largest possible in this interval.
                result += 1
                p = q
                q = end
            else:
                # Case 3: None of the previous selected numbers are in the
                # interval.

                # We add two numbers: end-1 and end.
                result += 2
                p = end - 1
```

Round 6 · Mid · Hard

p.2758

```

        q = end

    return result

# Example usage:
# sol = Solution()
# print(sol.intersectionSizeTwo([[1,3],[3,7],[8,9]])) # Expected output: 5

```

Greedy

☐ #857 Minimum Cost to Hire K Workers

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Greedy · Sorting · Heap (Priority Queue)
 Lists: AlgoMaster 600

Problem

There are n workers. You are given two integer arrays `quality` and `wage` where `quality[i]` is the quality of the i th worker and `wage[i]` is the minimum wage expectation for the i th worker. We want to hire exactly k workers to form a paid group. To hire a group of k workers, we must pay them according to the following rules:

1. Every worker in the paid group must be paid at least their minimum wage expectation.
2. In the group, each worker's pay must be directly proportional to their quality. This means if a worker's quality is double that of another worker in the group, then they must be paid twice as much as the other worker.

Given the integer k , return the least amount of money needed to form a paid group satisfying the above conditions. Answers within 10–5 of the actual answer will be accepted.

Example 1:

Input: `quality = [10,20,5]`, `wage = [70,50,30]`, `k = 2`
 Output: 105.00000
 Explanation: We pay 70 to 0th worker and 35 to 2nd worker.

Example 2:

Input: `quality = [3,1,10,10,1]`, `wage = [4,8,2,2,7]`, `k = 3`
 Output: 30.66667
 Explanation: We pay 4 to 0th worker, 13.33333 to 2nd and 3rd workers separately.

Constraints:

- $n == \text{quality.length} == \text{wage.length}$
- $1 \leq k \leq n \leq 104$
- $1 \leq \text{quality}[i], \text{wage}[i] \leq 104$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def mincostToHireWorkers(
        self,
        quality: List[int],
        wage: List[int],
        k: int,
    ) -> float:
        ans = math.inf
        qualitySum = 0
        # (wagePerQuality, quality) sorted by wagePerQuality
```

```
workers = sorted((w / q, q) for q, w in zip(quality, wage))
maxHeap = []
```

```
for wagePerQuality, q in workers:
    heapq.heappush(maxHeap, -q)
    qualitySum += q
    if len(maxHeap) > k:
        qualitySum += heapq.heappop(maxHeap)
    if len(maxHeap) == k:
        ans = min(ans, qualitySum * wagePerQuality)
```

```
return ans
```

• LeetDoocs

```
class Solution:
    def mincostToHireWorkers(
        self, quality: List[int], wage: List[int], k: int
    ) -> float:
        t = sorted(zip(quality, wage), key=lambda x: x[1] / x[0])
        ans, tot = inf, 0
        h = []
        for q, w in t:
            tot += q
            heappush(h, -q)

            if len(h) == k:
                ans = min(ans, w / q * tot)
                tot -= heappop(h)
        return ans
```

• SimplyLeet

```
# Python solution using heapq as a max-heap by pushing negative values.
import heapq

def mincostToHireWorkers(quality, wage, k):
    # Create a list of tuples (ratio, quality) for each worker.
    workers = []
    for q, w in zip(quality, wage):
        ratio = w / q # Minimum acceptable wage per unit quality.
        workers.append((ratio, q))

    # Sort workers by their ratio in ascending order.
    workers.sort(key=lambda x: x[0])

    quality_sum = 0 # The running sum of qualities in the current group.
    max_quality_heap = [] # Max-heap to maintain the largest quality values
    (using negatives).
    min_cost = float('inf') # Initialize the minimum cost as infinity.

    # Process each worker in order of increasing ratio.
    for ratio, q in workers:
        # Add the current worker's quality to the heap (invert sign for
        max-heap simulation).

        heapq.heappush(max_quality_heap, -q)
        quality_sum += q

        # If we have more than k workers, remove the one with the highest
        quality.
        if len(max_quality_heap) > k:
            removed_quality = -heapq.heappop(max_quality_heap)
            quality_sum -= removed_quality

        # When the group size is exactly k, calculate the cost.
        if len(max_quality_heap) == k:
```

```
        current_cost = quality_sum * ratio
        min_cost = min(min_cost, current_cost)

    return min_cost

# Example usage:
# quality = [10,20,5]
# wage = [70,50,30]
# k = 2
# Expected output: 105.00000

#print(mincostToHireWorkers([10,20,5], [70,50,30], 2))
```

Greedy

#871 Minimum Number of Refueling Stops

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Greedy · Heap
(Priority Queue)

Lists: AlgoMaster 600

Problem

A car travels from a starting position to a destination which is target miles east of the starting position.

There are gas stations along the way. The gas stations are represented as an array stations where $\text{stations}[i] = [\text{position}_i, \text{fuel}_i]$ indicates that the i th gas station is position_i miles east of the starting position and has fuel_i liters of gas. The car starts with an infinite tank of gas, which initially has startFuel liters of fuel in it. It uses one liter of gas per one mile that it drives. When the car reaches a gas station, it may stop and refuel, transferring all the gas from the station into the car.

Round 6 · Mid · Hard

p.2765

Return the minimum number of refueling stops the car must make in order to reach its destination. If it cannot reach the destination, return -1.

Note that if the car reaches a gas station with 0 fuel left, the car can still refuel there. If the car reaches the destination with 0 fuel left, it is still considered to have arrived.

Example 1:

Input: target = 1, startFuel = 1, stations = []
Output: 0
Explanation: We can reach the target without refueling.

Example 2:

Input: target = 100, startFuel = 1, stations = [[10,100]]
Output: -1
Explanation: We can not reach the target (or even the first gas station).

Example 3:

Input: target = 100, startFuel = 10, stations = [[10,60],[20,30],[30,30],[60,40]]
Output: 2
Explanation: We start with 10 liters of fuel. We drive to position 10, expending 10 liters of fuel. We refuel from 0 liters to 60 liters of gas. Then, we drive from position 10 to position 60 (expending 50 liters of fuel), and refuel from 10 liters to 50 liters of gas. We then drive to and reach the target. We made 2 refueling stops along the way, so we return 2.

Constraints:

- $1 \leq \text{target}$, $\text{startFuel} \leq 100$
- $0 \leq \text{stations.length} \leq 500$

Round 6 · Mid · Hard

p.2766

- $1 \leq \text{position}_i < \text{position}_{i+1} < \text{target}$
- $1 \leq \text{fuel}_i < 109$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minRefuelStops(
        self,
        target: int,
        startFuel: int,
        stations: List[List[int]],
    ) -> int:
        # dp[i] := the farthest position we can reach w / i refuels
        dp = [startFuel] + [0] * len(stations)

        for i, station in enumerate(stations):
            for j in range(i + 1, 0, -1):
                if dp[j - 1] >= station[0]:
                    dp[j] = max(dp[j], dp[j - 1] + station[1])

        for i, d in enumerate(dp):
            if d >= target:
                return i

        return -1
```

• LeetCode

```
class Solution:
    def minRefuelStops(
        self, target: int, startFuel: int, stations: List[List[int]]
    ) -> int:
        pq = []
        ans = pre = 0
        stations.append([target, 0])
        for pos, fuel in stations:
            dist = pos - pre
            startFuel -= dist
```

Round 6 · Mid · Hard

p.2767

```
while startFuel < 0 and pq:
    startFuel -= heappop(pq)
    ans += 1
if startFuel < 0:
    return -1
heappush(pq, -fuel)
pre = pos
return ans
```

• SimplyLeet

```
import heapq

def minRefuelStops(target, startFuel, stations):
    # Max-heap to store the fuels available at the stations we've passed.
    max_heap = []
    numRefuels = 0
    prev_position = 0
    fuel = startFuel

    # Append target as a final station to unify logic
    stations.append([target, 0])

    for position, station_fuel in stations:
        # Calculate distance from previous station (or start) to current
        # station.
        distance = position - prev_position

        # While we don't have enough fuel to reach the current station,
        # refuel from the stations previously passed (using max fuel from
        # heap).
        while fuel < distance and max_heap:
            added_fuel = -heapq.heappop(max_heap) # Retrieve max fuel
            available
```

Round 6 · Mid · Hard

p.2768

```
        fuel += added_fuel
        numRefuels += 1

    # If we still can't reach the station, return -1.
    if fuel < distance:
        return -1

    # Subtract the fuel used to travel to this station.
    fuel -= distance
    # Store the fuel available at this station into the max-heap.

    heapq.heappush(max_heap, -station_fuel)
    # Update prev_position for next iteration.
    prev_position = position

    return numRefuels

# Example usage:
print(minRefuelStops(100, 10, [[10,60],[20,30],[30,30],[60,40]])) # Expected
output: 2
```

Round 6 · Mid · Hard

p.2769

Topological Sort

Round 6 · Mid · Hard · 2 questions

Round 6 · Mid · Hard

p.2770

Topological Sort

★ #269 Alien Dictionary ★

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · String · DFS · BFS · Graph · Topological Sort

Lists: ★ Premium | Grind 169 | Premium 98

Problem

There is a new alien language that uses the English alphabet. However, the order of the letters is unknown to you.

You are given a list of strings `words` from the alien language's dictionary. Now it is claimed that the strings in `words` are sorted lexicographically by the rules of this new language.

If this claim is incorrect, and the given arrangement of string in `words` cannot correspond to any order of letters, return `""`.

Otherwise, return a string of the unique letters in the new alien language sorted in lexicographically increasing order by the new language's rules. If there are multiple solutions,

return any of them.

Example 1:

```
Input: words = ["wrt","wrf","er","ett","rftt"]
Output: "wertf"
```

Example 2:

```
Input: words = ["z","x"]
Output: "zx"
```

Example 3:

```
Input: words = ["z","x","z"]
Output: ""
Explanation: The order is invalid, so return "".
```

Constraints:

- $1 \leq \text{words.length} \leq 100$
- $1 \leq \text{words}[i].\text{length} \leq 100$
- `words[i]` consists of only lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def alienOrder(self, words: list[str]) -> str:
        graph = {}
        inDegrees = [0] * 26

        self._buildGraph(graph, words, inDegrees)
        return self._topology(graph, inDegrees)

    def _buildGraph(
        self,
```

```

graph: dict[str, set[str]],
words: list[str],
inDegrees: list[int],
) -> None:
    # Create a node for each character in each word.
    for word in words:
        for c in word:
            if c not in graph:
                graph[c] = set()

    for first, second in zip(words, words[1:]):
        length = min(len(first), len(second))
        for j in range(length):
            u = first[j]
            v = second[j]
            if u != v:
                if v not in graph[u]:
                    graph[u].add(v)
                    inDegrees[ord(v) - ord('a')] += 1
                break # The order of characters after this are meaningless.

    # First = 'ab', second = 'a' . invalid
    if j == length - 1 and len(first) > len(second):
        graph.clear()
        return

    def _topology(self, graph: dict[str, set[str]], inDegrees: list[int]) -> str
    :
        s = ''
        q = collections.deque()

        for c in graph:

```

```

        if inDegrees[ord(c) - ord('a')] == 0:
            q.append(c)

        while q:
            u = q.pop()
            s += u
            for v in graph[u]:
                inDegrees[ord(v) - ord('a')] -= 1
                if inDegrees[ord(v) - ord('a')] == 0:
                    q.append(v)

    # Words = ['z', 'x', 'y', 'x']
    return s if len(s) == len(graph) else ''

```

• LeetCode

```

class Solution:
    def alienOrder(self, words: List[str]) -> str:
        g = [[False] * 26 for _ in range(26)]
        s = [False] * 26
        cnt = 0
        n = len(words)
        for i in range(n - 1):
            for c in words[i]:
                if cnt == 26:
                    break

            o = ord(c) - ord('a')
            if not s[o]:
                cnt += 1
                s[o] = True
            m = len(words[i + 1])
            for j in range(m):
                if j >= len(words[i + 1]):
                    return ''
                c1, c2 = words[i][j], words[i + 1][j]
                if c1 == c2:

```

```

        continue
    o1, o2 = ord(c1) - ord('a'), ord(c2) - ord('a')
    if g[o2][o1]:
        return ''
    g[o1][o2] = True
    break
for c in words[n - 1]:
    if cnt == 26:
        break
    o = ord(c) - ord('a')

```

```

    if not s[o]:
        cnt += 1
        s[o] = True

indegree = [0] * 26
for i in range(26):
    for j in range(26):
        if i != j and s[i] and s[j] and g[i][j]:
            indegree[j] += 1
q = deque()

```

```

ans = []
for i in range(26):
    if s[i] and indegree[i] == 0:
        q.append(i)
while q:
    t = q.popleft()
    ans.append(chr(t + ord('a')))
    for i in range(26):
        if s[i] and i != t and g[t][i]:
            indegree[i] -= 1

```

```

    if indegree[i] == 0:
        q.append(i)
return '' if len(ans) < cnt else ''.join(ans)

```

• SimplyLeet

Round 6 · Mid · Hard

p.2775

```

# Python solution for Alien Dictionary
from collections import defaultdict, deque

def alienOrder(words):
    # Create data structures: graph and in_degree for each unique character.
    graph = defaultdict(set)
    in_degree = {char: 0 for word in words for char in word}

    # Build the graph.
    for i in range(len(words) - 1):

        word1, word2 = words[i], words[i+1]
        # Check for invalid case where prefix of next word is the same but
        word1 is longer.
        if len(word1) > len(word2) and word1.startswith(word2):
            return ""
        # Compare the words character by character.
        for c1, c2 in zip(word1, word2):
            if c1 != c2:
                # If we haven't already recorded this edge, add it.
                if c2 not in graph[c1]:
                    graph[c1].add(c2)

                in_degree[c2] += 1
                break

    # Prepare for topological sort by adding all nodes with in_degree == 0.
    queue = deque([char for char in in_degree if in_degree[char] == 0])
    order = []

    # Kahn's algorithm for topological sort.
    while queue:
        current = queue.popleft()

```

Round 6 · Mid · Hard

p.2776


```

        order.append(current)
        for neighbor in graph[current]:
            in_degree[neighbor] -= 1
            if in_degree[neighbor] == 0:
                queue.append(neighbor)

# Check if topological sort is possible.
if len(order) == len(in_degree):
    return "".join(order)
else:
    return ""

# Example usage:
print(alienOrder(["wrt","wrf","er","ett","rftt"]))

```

• LC.ca

```

'''
>>> a = ['a','b','c']
>>> zip(a, a[1:])
[('a', 'b'), ('b', 'c')]

# same as pairwise()
>>> from itertools import pairwise
>>> pairwise(a)
<itertools.pairwise object at 0x108458730>

>>> list(pairwise(a))
[('a', 'b'), ('b', 'c')]
'''

'''
>>> inDegree = {'a':11, 'b':22, 'c':33}

>>> [print(c) for c in inDegree.keys()]
a
b

```

Round 6 · Mid · Hard

p.2777

```

c
[None, None, None]

# same as for c in inDegree.keys()
>>> [print(c) for c in inDegree]
a
b
c
[None, None, None]

>>> [print(c) for c in inDegree.values()]
11
22
33
[None, None, None]

>>> [print(c) for c in inDegree.items()]
('a', 11)
('b', 22)
('c', 33)

[None, None, None]
'''

from collections import defaultdict, deque
from typing import Dict, List, Set

class Solution:
    def alienOrder(self, words: List[str]) -> str:
        graph = defaultdict(set)
        inDegree = defaultdict(int)

```

Round 6 · Mid · Hard

p.2778

```

        self._buildGraph(graph, words, inDegree)
        return self._topology(graph, inDegree)

    def _buildGraph(self, graph: Dict[str, Set[str]], words: List[str], inDegree: Dict[str, int]) -> None:
        # Create a node for each character in each word
        for word in words:
            for c in word:
                inDegree[c] = 0 # necessary for final char counting

        for first, second in zip(words, words[1:]): # or pairwise(words)

            length = min(len(first), len(second))
            for j in range(length):
                u = first[j]
                v = second[j]
                if u != v:
                    if v not in graph[u]:
                        graph[u].add(v)
                        inDegree[v] += 1
                    break # Later characters' order is meaningless
            if j == length - 1 and len(first) > len(second):

                # If 'ab' comes before 'a', it's an invalid order
                graph.clear()
                return

    def _topology(self, graph: Dict[str, Set[str]], inDegree: Dict[str, int]) -> str:
        result = ''
        q = deque([c for c in inDegree if inDegree[c] == 0])

        while q:
            u = q.popleft()

```

Round 6 · Mid · Hard

p.2779

```

        result += u
        for v in graph[u]:
            inDegree[v] -= 1
            if inDegree[v] == 0:
                q.append(v)

        # If there are remaining characters in inDegree, it means there's a
        cycle
        if any(inDegree.values()):
            return ''

        # Words = ['z', 'x', 'y', 'x']
        return result if len(result) == len(indegree) else ''

#####

import collections

class Node(object):
    def __init__(self, val):

        self.val = val
        self.neighbors = []

    def connect(self, node):
        self.neighbors.append(node)

    def getNbrs(self):
        return self.neighbors

```

Round 6 · Mid · Hard

p.2780

```

class Solution(object):
    def alienOrder(self, words):
        """
        :type words: List[str]
        :rtype: str
        """

        def dfs(root, graph, visited):
            visited[root] = 1
            for nbr in graph[root].getNbrs():

                if visited[nbr.val] == 0:
                    if not dfs(nbr.val, graph, visited):
                        return False
                elif visited[nbr.val] == 1:
                    return False

            visited[root] = 2
            self.ans += root
            return True

        self.ans = ""
        graph = {}
        visited = collections.defaultdict(int)
        self.topNum = 0
        for i in range(0, len(words) - 1):
            a = words[i]
            b = words[i + 1]
            i = 0
            while i < len(a) and i < len(b):
                if a[i] != b[i]:
                    nodeA = nodeB = None
                    if a[i] not in graph:
                        nodeA = Node(a[i])
                        graph[a[i]] = nodeA
                    else:
                        nodeA = graph[a[i]]
                    if b[i] not in graph:
                        nodeB = Node(b[i])
                        graph[b[i]] = nodeB
                    else:

```

```

            nodeB = graph[b[i]]
            nodeA.connect(nodeB)
            break
            i += 1
            if i < len(a) and i >= len(b):
                return ""

        for c in graph:
            if visited[c] == 0:
                if not dfs(c, graph, visited):
                    return ""

        unusedSet = set()
        for word in words:
            for c in word:
                unusedSet.add(c)

        for c in unusedSet:
            if c not in graph:
                self.ans += c

        return self.ans[::-1]

```

◆ Quick Review

Key Insights

- Build a graph where each unique character is a node.
- Compare adjacent words to determine the first different character, which gives the ordering (directed edge).
- Ensure all characters, even those without edges, are represented.

- Use a topological sort (using DFS or BFS) to determine a valid character ordering.
- Handle edge cases, such as words where a longer word appears before its prefix which is invalid.

Space & Time

Time Complexity: $O(N * L + V + E)$ where N is the number of words, L is the average length of each word, V is the number of unique characters, and E is the number of edges derived from adjacent word comparisons.

Space Complexity: $O(V + E)$ for storing the graph and additional data structures for topological sorting.

Solution

We use a graph-based approach to solve the problem. First, we initialize a graph (adjacency list) and in-degree dictionary for all unique characters. We then iterate through each pair of consecutive words, comparing them character by character to find the first pair that differs. This difference implies an order: the first character should come before the second character. We add a directed edge

accordingly and update the in-degree for the target character.

To get the final ordering, we perform a topological sort using Kahn's algorithm:

- Enqueue characters with in-degree zero.
 - Remove characters from the queue and append them to our result.
 - Decrement the in-degree of neighboring nodes.
 - If a neighbor's in-degree becomes zero, enqueue it.
- If the result's length equals the number of unique nodes, we have a valid ordering; otherwise, a cycle exists, and we return an empty string.

Topological Sort

□ #1203 Sort Items by Groups Respecting Dependencies HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Depth-First Search · Breadth-First Search ·
 Graph Theory · Topological Sort
 Lists: AlgoMaster 600

Problem

There are n items each belonging to zero or one of m groups where $group[i]$ is the group that the i -th item belongs to and it's equal to -1 if the i -th item belongs to no group. The items and the groups are zero indexed. A group can have no item belonging to it.

Return a sorted list of the items such that:

- The items that belong to the same group are next to each other in the sorted list.
- There are some relations between these items where $beforeItems[i]$ is a list containing all the items that should come before the i -th item in the sorted array (to the left of the i -th item).

Round 6 · Mid · Hard

p.2785

Return any solution if there is more than one solution and return an empty list if there is no solution.

Example 1:

Item	Group	Before
0	-1	
1	-1	6
2	1	5
3	0	6
4	0	3, 6
5	1	
6	0	
7	-1	

Input: $n = 8$, $m = 2$, $group = [-1, -1, 1, 0, 0, 1, 0, -1]$, $beforeItems =$
 $[[], [6], [5], [6], [3, 6], [], [], []]$
 Output: $[6, 3, 4, 1, 5, 2, 0, 7]$

Example 2:

Round 6 · Mid · Hard

p.2786

Input: n = 8, m = 2, group = [-1,-1,1,0,0,1,0,-1], beforeItems =
 [[],[6],[5],[6],[3],[1],[4],[1]]

Output: []

Explanation: This is the same as example 1 except that 4 needs to be before 6 in the sorted list.

Constraints:

- $1 \leq m \leq n \leq 3 * 104$
- `group.length == beforeItems.length == n`
- $-1 \leq group[i] \leq m - 1$
- $0 \leq beforeItems[i].length \leq n - 1$
- $0 \leq beforeItems[i][j] \leq n - 1$
- $i \neq beforeItems[i][j]$
- `beforeItems[i]` does not contain duplicates elements.

Community Solutions (Python)

• LeetCode

```
class Solution:
    def sortItems(
        self, n: int, m: int, group: List[int], beforeItems: List[List[int]]
    ) -> List[int]:
        def topo_sort(degree, graph, items):
            q = deque(i for _, i in enumerate(items) if degree[i] == 0)
            res = []
            while q:
                i = q.popleft()
                res.append(i)
```

```
        for j in graph[i]:
            degree[j] -= 1
            if degree[j] == 0:
                q.append(j)
        return res if len(res) == len(items) else []
```

```
idx = m
group_items = [[] for _ in range(n + m)]
for i, g in enumerate(group):
    if g == -1:
```

```
        group[i] = idx
        idx += 1
        group_items[group[i]].append(i)
```

```
item_degree = [0] * n
group_degree = [0] * (n + m)
item_graph = [[] for _ in range(n)]
group_graph = [[] for _ in range(n + m)]
for i, gi in enumerate(group):
    for j in beforeItems[i]:
```

```
        gj = group[j]
        if gi == gj:
            item_degree[i] += 1
            item_graph[j].append(i)
        else:
            group_degree[gi] += 1
            group_graph[gj].append(gi)
```

```
group_order = topo_sort(group_degree, group_graph, range(n + m))
if not group_order:
```

```
    return []
ans = []
for gi in group_order:
    items = group_items[gi]
    item_order = topo_sort(item_degree, item_graph, items)
    if len(items) != len(item_order):
        return []
    ans.extend(item_order)
return ans
```

• SimplyLeet

```
# Python solution for "Sort Items by Groups Respecting Dependencies"

from collections import defaultdict, deque

def sortItems(n, m, group, beforeItems):
    # Step 1: Reassign group ids for ungrouped items (-1) starting from m
    new_group = m
    for i in range(n):
        if group[i] == -1:
            group[i] = new_group

    new_group += 1

    total_groups = new_group

    # Graphs and indegree arrays for groups and items
    group_graph = defaultdict(list)
    group_indegree = [0] * total_groups
    item_graph = defaultdict(list)
    item_indegree = [0] * n

    # Mapping from group id to items in that group
    group_to_items = defaultdict(list)
    for i in range(n):
        group_to_items[group[i]].append(i)

    # Build graphs for items and groups based on dependencies
    for i in range(n):
        for prev in beforeItems[i]:
            # if dependency is within the same group, link in item graph
            if group[prev] == group[i]:
```

```
        item_graph[prev].append(i)
        item_indegree[i] += 1
    else:
        # dependency across groups: add edge from group[prev] to
        group[i]

        group_graph[group[prev]].append(group[i])
        group_indegree[group[i]] += 1
    # Also build the item graph for dependency within items across
    groups

    item_graph[prev].append(i)
    item_indegree[i] += 1

    # Helper: topological sort function
    def topologicalSort(nodes, graph, indegree):
        order = []
        dq = deque([node for node in nodes if indegree[node] == 0])
        while dq:
            node = dq.popleft()
            order.append(node)
            for neighbor in graph[node]:
                indegree[neighbor] -= 1
                if indegree[neighbor] == 0:
                    dq.append(neighbor)
        if len(order) == len(nodes):
            return order
        else:
            return [] # cycle detected

    # Topologically sort groups
    groups = list(range(total_groups))
    sorted_groups = topologicalSort(groups, group_graph, group_indegree[:])
    if not sorted_groups:
```

```

    return [] # return empty list if cycle in groups detected

# Topologically sort items
items = list(range(n))
sorted_items = topologicalSort(items, item_graph, item_indegree[:])
if not sorted_items:
    return [] # return empty list if cycle in items detected

# Map each item to its order in the sorted_items list
item_order = {item: i for i, item in enumerate(sorted_items)}

# For each group, sort items according to the global item order
for grp in group_to_items:
    group_to_items[grp].sort(key=lambda x: item_order[x])

# Build final result following the order of groups from sorted_groups
result = []
for grp in sorted_groups:
    result.extend(group_to_items[grp])
return result

```

Example usage:

```

if __name__ == "__main__":
    n = 8
    m = 2
    group = [-1,-1,1,0,0,1,0,-1]
    beforeItems = [[],[6],[5],[6],[3,6],[],[],[[]]]
    print(sortItems(n, m, group, beforeItems))

```

Round 6 · Mid · Hard

p.2791

Union Find

Round 6 · Mid · Hard · 2 questions

Round 6 · Mid · Hard

p.2792

Union Find

★ #305 Number of Islands II ★

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table · Union Find · Matrix
 Lists: ★ Premium | Premium 98

Problem

You are given an empty 2D binary grid `grid` of size $m \times n$. The grid represents a map where 0's represent water and 1's represent land. Initially, all the cells of grid are water cells (i.e., all the cells are 0's).

We may perform an add land operation which turns the water at position into a land. You are given an array `positions` where `positions[i] = [ri, ci]` is the position (ri, ci) at which we should operate the i th operation.

Return an array of integers `answer` where `answer[i]` is the number of islands after turning the cell (ri, ci) into a land.

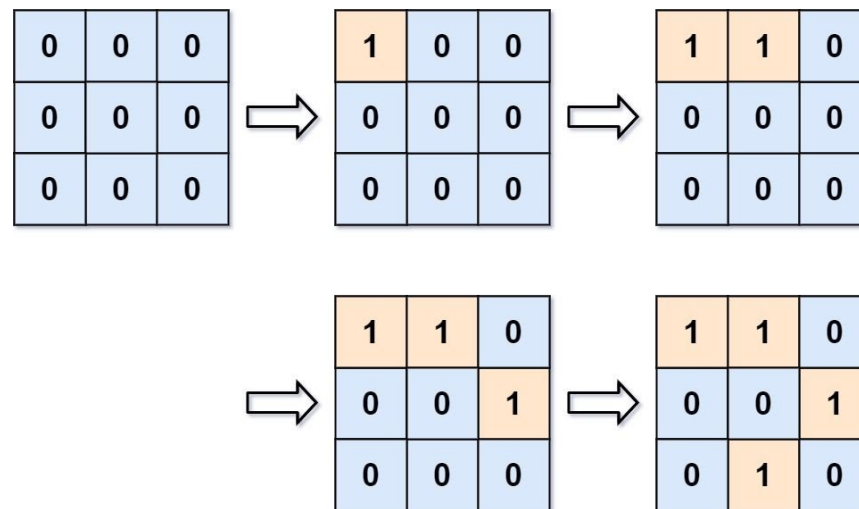
An island is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the

Round 6 · Mid · Hard

p.2793

grid are all surrounded by water.

Example 1:



Input: $m = 3, n = 3, \text{positions} = [[0,0],[0,1],[1,2],[2,1]]$

Output: `[1,1,2,3]`

Explanation:

Initially, the 2d grid is filled with water.

- Operation #1: `addLand(0, 0)` turns the water at `grid[0][0]` into a land. We have 1 island.
- Operation #2: `addLand(0, 1)` turns the water at `grid[0][1]` into a land. We still have 1 island.
- Operation #3: `addLand(1, 2)` turns the water at `grid[1][2]` into a land. We have 2 islands.
- Operation #4: `addLand(2, 1)` turns the water at `grid[2][1]` into a land. We have 3 islands.

Example 2:

Input: $m = 1, n = 1, \text{positions} = [[0,0]]$

Output: `[1]`

Constraints:

Round 6 · Mid · Hard

p.2794

- $1 \leq m, n, \text{positions.length} \leq 104$
- $1 \leq m * n \leq 104$
- $\text{positions}[i].\text{length} == 2$
- $0 \leq ri < m$
- $0 \leq ci < n$

Follow up: Could you solve it in time complexity $O(k \log(mn))$, where $k == \text{positions.length}$?

Community Solutions (Python)

• WalkCC

```
class UnionFind:
    def __init__(self, n: int):
        self.id = [-1] * n
        self.rank = [0] * n

    def unionByRank(self, u: int, v: int) -> None:
        i = self.find(u)
        j = self.find(v)
        if i == j:
            return

        if self.rank[i] < self.rank[j]:
            self.id[i] = j
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
        else:
            self.id[i] = j
            self.rank[j] += 1

    def find(self, u: int) -> int:
        if self.id[u] != u:
```

```
        self.id[u] = self.find(self.id[u])
        return self.id[u]
```

```
class Solution:
    def numIslands2(
        self,
        m: int,
        n: int,
        positions: list[list[int]],
    ) -> list[int]:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        ans = []
        seen = [[False] * n for _ in range(m)]
        uf = UnionFind(m * n)
        count = 0

        def getId(i: int, j: int, n: int) -> int:
            return i * n + j
```

```
        for i, j in positions:
            if seen[i][j]:
                ans.append(count)
                continue
            seen[i][j] = True
            id = getId(i, j, n)
            uf.id[id] = id
            count += 1
            for dx, dy in DIRS:
                x = i + dx
                y = j + dy
                if x < 0 or x == m or y < 0 or y == n:
                    continue
                neighborId = getId(x, y, n)
                if uf.id[neighborId] == -1: # water
                    continue
                currentParent = uf.find(id)
                neighborParent = uf.find(neighborId)
                if currentParent != neighborParent:
                    uf.unionByRank(currentParent, neighborParent)
```

```
count -= 1
ans.append(count)
```

```
return ans
```

• LeetCode

```
class UnionFind:
    def __init__(self, n: int):
        self.p = list(range(n))
        self.size = [1] * n

    def find(self, x: int):
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]

    def union(self, a: int, b: int) -> bool:
        pa, pb = self.find(a - 1), self.find(b - 1)
        if pa == pb:
            return False
        if self.size[pa] > self.size[pb]:
            self.p[pb] = pa
            self.size[pa] += self.size[pb]
        else:
            self.p[pa] = pb
            self.size[pb] += self.size[pa]

        return True

class Solution:
    def numIslands2(self, m: int, n: int, positions: List[List[int]]) -> List[int]:
        uf = UnionFind(m * n)
        grid = [[0] * n for _ in range(m)]
        ans = []
        dirs = (-1, 0, 1, 0, -1)
        cnt = 0
```

Round 6 · Mid · Hard

p.2797

```
for i, j in positions:
    if grid[i][j]:
        ans.append(cnt)
        continue
    grid[i][j] = 1
    cnt += 1
    for a, b in pairwise(dirs):
        x, y = i + a, j + b
        if (
            0 <= x < m
            and 0 <= y < n
            and grid[x][y]
            and uf.union(i * n + j, x * n + y)
        ):
            cnt -= 1
    ans.append(cnt)
return ans
```

• SimplyLeet

```
# Python implementation using Union Find with path compression and union by rank.
class UnionFind:
    def __init__(self, size):
        # parent: stores parent of each node. -1 indicates water.
        self.parent = [-1] * size
        self.rank = [0] * size
        self.count = 0 # number of islands

    def find(self, x):
        # Find with path compression.

        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, x, y):
        # Union the sets; if union is successful, reduce island count.
        rootX = self.find(x)
        rootY = self.find(y)
        if rootX == rootY:
            return False
```

Round 6 · Mid · Hard

p.2798

```
# union by rank.
if self.rank[rootX] < self.rank[rootY]:
    self.parent[rootX] = rootY
elif self.rank[rootX] > self.rank[rootY]:
    self.parent[rootY] = rootX
else:
    self.parent[rootY] = rootX
    self.rank[rootX] += 1
self.count -= 1
return True
```

```
class Solution:
    def numIslands2(self, m, n, positions):
        uf = UnionFind(m * n)
        result = []
        directions = [(0, 1), (1, 0), (0, -1), (-1, 0)]
```

```
for r, c in positions:
    idx = r * n + c
    # If the position is already land, skip.
```

```
if uf.parent[idx] != -1:
    result.append(uf.count)
    continue
```

```
# Mark this cell as land forming a new island.
uf.parent[idx] = idx
uf.count += 1
```

```
# Union with all 4 neighbors if they are already land.
for dr, dc in directions:
```

```
    nr, nc = r + dr, c + dc
    nidx = nr * n + nc
    if 0 <= nr < m and 0 <= nc < n and uf.parent[nidx] != -1:
        uf.union(idx, nidx)
    result.append(uf.count)
return result
```

```
# Example run:
# s = Solution()
# print(s.numIslands2(3, 3, [[0,0],[0,1],[1,2],[2,1]]))
```

Round 6 · Mid · Hard

p.2799

• LC.ca

```
class Solution:
    def numIslands2(self, m: int, n: int, positions: List[List[int]]) -> List[
int]:
        def check(i, j):
            return 0 <= i < m and 0 <= j < n and grid[i][j] == 1

        def find(x):
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]
```

```
p = list(range(m * n))
grid = [[0] * n for _ in range(m)]
res = []
cur = 0 # current island count
for i, j in positions:
    if grid[i][j] == 1: # already counted, same as previous island
        count
```

```
        res.append(cur)
        continue
    grid[i][j] = 1
    cur += 1
```

```
for x, y in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
    if check(i + x, j + y) and find(i * n + j) != find((i + x) * n
+ j + y):
        p[find(i * n + j)] = find((i + x) * n + j + y)
        cur -= 1
        # 1 0 1
        # 0 1 0
        # for above, if setting 1st-row 2nd-col 0-to-1
        # cur will -1 for 3 times
        # but cur += 1 after grid[i][j] = 1 so cur final result is
1
        res.append(cur)
```

Round 6 · Mid · Hard

p.2800

```

        return res

#####

class UnionFind(object):
    def __init__(self, m, n):
        self.dad = [i for i in range(m * n)]
        self.rank = [0 for _ in range(m * n)]

    def find(self, x):

        if self.dad[x] != x:
            self.dad[x] = self.find(self.dad[x])
        return self.dad[x]

    def union(self, xy): # xy is a tuple (x,y), with x and y value inside
        x, y = map(self.find, xy)
        if x == y:
            return False
        if self.rank[x] > self.rank[y]:
            self.dad[y] = x

        elif self.rank[x] < self.rank[y]:
            self.dad[x] = y
        else:
            self.dad[y] = x # search to the left, to find parent
            self.rank[x] += 1 # now x a parent, so +1 its rank
        return True

class Solution(object):
    def numIslands2(self, m, n, positions):

        """
        :type m: int
        :type n: int
        :type positions: List[List[int]]
        :rtype: List[int]
        """
        uf = UnionFind(m, n)
        ans = 0
        ret = []
        dirs = [(0, -1), (0, 1), (1, 0), (-1, 0)]

```

Round 6 · Mid · Hard

p.2801

```

        grid = set()
        for i, j in positions:
            ans += 1
            grid |= {(i, j)}
            for di, dj in dirs:
                ni, nj = i + di, j + dj
                if 0 <= ni < m and 0 <= nj < n and (ni, nj) in grid:
                    if uf.union((ni * n + nj, i * n + j)):
                        ans -= 1
            ret.append(ans)

        return ret

#####

class UnionFind:
    def __init__(self, n: int):
        self.p = list(range(n))
        self.size = [1] * n

    def find(self, x: int):
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]

    def union(self, a: int, b: int) -> bool:
        pa, pb = self.find(a - 1), self.find(b - 1)
        if pa == pb:
            return False

        if self.size[pa] > self.size[pb]:
            self.p[pb] = pa
            self.size[pa] += self.size[pb]
        else:
            self.p[pa] = pb
            self.size[pb] += self.size[pa]
        return True

class Solution:

```

Round 6 · Mid · Hard

p.2802

```
def numIslands2(self, m: int, n: int, positions: List[List[int]]) -> List[int]:
    uf = UnionFind(m * n)
    grid = [[0] * n for _ in range(m)]
    ans = []
    dirs = (-1, 0, 1, 0, -1)
    cnt = 0
    for i, j in positions:
        if grid[i][j]:
            ans.append(cnt)
            continue

        grid[i][j] = 1
        cnt += 1
        for a, b in pairwise(dirs):
            x, y = i + a, j + b
            if (
                0 <= x < m
                and 0 <= y < n
                and grid[x][y]
                and uf.union(i * n + j, x * n + y)
            ):
                cnt -= 1
        ans.append(cnt)
    return ans
```

◆ Quick Review

Key Insights

- Use Union Find (Disjoint Set Union) to efficiently manage connected components (islands).
- Map 2D grid coordinates to a 1D index.
- When adding a new land, check its 4 neighbors (up, down, left, right) and perform

unions if they are also lands.

- Avoid duplicate processing if the same cell is added multiple times.
- Maintain a counter for islands that is updated during each union operation.

Space & Time

Time Complexity: $O(K * \alpha(mn))$ where K is the number of operations and α is the inverse Ackermann function (almost constant time per union/find).

Space Complexity: $O(mn)$ for storing the union-find structure.

Solution

The solution relies on the Union Find data structure to dynamically merge islands. When a new land cell is added:

- If the cell is already land, record the current island count.
- Otherwise, mark the cell as land and increment the island count.
- Check the four neighboring cells; if any neighbor is land, attempt to union the current cell with that neighbor.
- If a union actually happens (i.e., the two cells were in separate islands),

decrement the island counter. – Output the island count after each add-land operation. The key trick is converting 2D indices to a 1D representation ($r * n + c$) and using path compression along with union by rank in the Union Find implementation for optimal performance.

Round 6 · Mid · Hard

p.2805

Union Find

□ #924 Minimize Malware Spread

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table · Depth-First Search ·
 Breadth-First Search · Union-Find · Graph
 Theory

Lists: AlgoMaster 600

Problem

You are given a network of n nodes represented as an $n \times n$ adjacency matrix graph, where the i th node is directly connected to the j th node if $\text{graph}[i][j] == 1$.

Some nodes initial are initially infected by malware. Whenever two nodes are directly connected, and at least one of those two nodes is infected by malware, both nodes will be infected by malware. This spread of malware will continue until no more nodes can be infected in this manner.

Suppose $M(\text{initial})$ is the final number of nodes infected with malware in the entire network after the spread of malware stops. We will

Round 6 · Mid · Hard

p.2806

remove exactly one node from initial.

Return the node that, if removed, would minimize $M(\text{initial})$. If multiple nodes could be removed to minimize $M(\text{initial})$, return such a node with the smallest index.

Note that if a node was removed from the initial list of infected nodes, it might still be infected later due to the malware spread.

Example 1:

Input: graph = [[1,1,0],[1,1,0],[0,0,1]], initial = [0,1]
Output: 0

Example 2:

Input: graph = [[1,0,0],[0,1,0],[0,0,1]], initial = [0,2]
Output: 0

Example 3:

Input: graph = [[1,1,1],[1,1,1],[1,1,1]], initial = [1,2]
Output: 1

Constraints:

- $n == \text{graph.length}$
- $n == \text{graph}[i].\text{length}$
- $2 \leq n \leq 300$
- $\text{graph}[i][j]$ is 0 or 1.
- $\text{graph}[i][j] == \text{graph}[j][i]$
- $\text{graph}[i][i] == 1$
- $1 \leq \text{initial.length} \leq n$

Round 6 · Mid · Hard

p.2807

- $0 \leq \text{initial}[i] \leq n - 1$
- All the integers in initial are unique.

Community Solutions (Python)

• **LeetDoocs**

```
class UnionFind:
    __slots__ = "p", "size"

    def __init__(self, n: int):
        self.p = list(range(n))
        self.size = [1] * n

    def find(self, x: int) -> int:
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])

        return self.p[x]

    def union(self, a: int, b: int) -> bool:
        pa, pb = self.find(a), self.find(b)
        if pa == pb:
            return False
        if self.size[pa] > self.size[pb]:
            self.p[pb] = pa
            self.size[pa] += self.size[pb]
        else:
            self.p[pa] = pb
            self.size[pb] += self.size[pa]
        return True

    def get_size(self, root: int) -> int:
        return self.size[root]

class Solution:
    def minMalwareSpread(self, graph: List[List[int]], initial: List[int]) -> int:
```

Round 6 · Mid · Hard

p.2808


```

n = len(graph)
uf = UnionFind(n)
for i in range(n):
    for j in range(i + 1, n):
        graph[i][j] and uf.union(i, j)
cnt = Counter(uf.find(x) for x in initial)
ans, mx = n, 0
for x in initial:
    root = uf.find(x)
    if cnt[root] > 1:
        continue
    sz = uf.get_size(root)
    if sz > mx or (sz == mx and x < ans):
        ans = x
        mx = sz
return min(initial) if ans == n else ans

```

• SimplyLeet

Python Implementation

```

def minMalwareSpread(graph, initial):
    n = len(graph)
    # visited array to track if a node has been processed.
    visited = [False] * n
    # component mapping: node -> component id, and component size mapping.
    comp_id = [-1] * n
    comp_size = {}
    current_component = 0

```

DFS function to traverse nodes in the same component.

```

def dfs(node):
    stack = [node]
    nodes_in_component = []
    while stack:
        current = stack.pop()
        if not visited[current]:
            visited[current] = True
            comp_id[current] = current_component

```

```

        nodes_in_component.append(current)
        # Check all neighbors.
        for neighbor in range(n):
            if graph[current][neighbor] == 1 and not visited[neighbor]:
                stack.append(neighbor)
        return nodes_in_component

# Identify components.
for i in range(n):
    if not visited[i]:

```

```

        nodes = dfs(i)
        comp_size[current_component] = len(nodes)
        current_component += 1

```

```

# Count initial infected nodes in each component.
init_count = [0] * current_component
for node in initial:
    init_count[comp_id[node]] += 1

```

Determine the best candidate for removal.

```

result = None
max_saved = -1
for node in sorted(initial):
    cid = comp_id[node]
    # Only one infected in the component, meaning removal saves component
    size nodes.
    if init_count[cid] == 1:
        if comp_size[cid] > max_saved:
            max_saved = comp_size[cid]
            result = node

```

If no such node, return the smallest index.

```

if result is None:
    result = min(initial)
return result

```

```

# Example usage:
# print(minMalwareSpread([[1,1,0],[1,1,0],[0,0,1]], [0,1]))

```

Minimum Spanning Tree

Round 6 · Mid · Hard · 3 questions

Round 6 · Mid · Hard

p.2811

Minimum Spanning Tree

□ #1168 Optimize Water Distribution in a Village

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Union-Find · Graph Theory · Heap (Priority Queue) · Minimum Spanning Tree

Lists: AlgoMaster 600

Problem

There are n houses in a village. We want to supply water for all the houses by building wells and laying pipes.

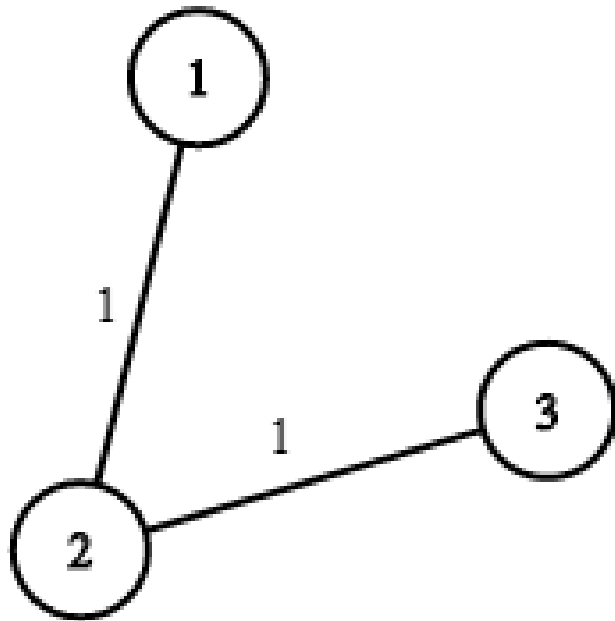
For each house i , we can either build a well inside it directly with cost $wells[i - 1]$ (note the -1 due to 0-indexing), or pipe in water from another well to it. The costs to lay pipes between houses are given by the array $pipes$ where each $pipes[j] = [house1j, house2j, costj]$ represents the cost to connect $house1j$ and $house2j$ together using a pipe. Connections are bidirectional, and there could be multiple valid connections between the same two houses with different costs.

Round 6 · Mid · Hard

p.2812

Return the minimum total cost to supply water to all houses.

Example 1:



Input: $n = 3$, wells = [1,2,2], pipes = [[1,2,1],[2,3,1]]
 Output: 3
 Explanation: The image shows the costs of connecting houses using pipes. The best strategy is to build a well in the first house with cost 1 and connect the other houses to it with cost 2 so the total cost is 3.

Example 2:

Input: $n = 2$, wells = [1,1], pipes = [[1,2,1],[1,2,2]]
 Output: 2
 Explanation: We can supply water with cost two using one of the three options:
 Option 1:
 - Build a well inside house 1 with cost 1.
 - Build a well inside house 2 with cost 1.
 The total cost will be 2.
 Option 2:

Constraints:

- $2 \leq n \leq 104$
- wells.length == n
- $0 \leq \text{wells}[i] \leq 105$
- $1 \leq \text{pipes.length} \leq 104$
- pipes[j].length == 3
- $1 \leq \text{house1j}, \text{house2j} \leq n$
- $0 \leq \text{costj} \leq 105$
- house1j != house2j

Community Solutions (Python)

• WalkCC

```

class Solution:
    def minCostToSupplyWater(
        self,
        n: int,
        wells: list[int],
        pipes: list[list[int]],
    ) -> int:
        ans = 0
        graph = [[] for _ in range(n + 1)]
        minHeap = [] # (d, u)
  
```

```

for u, v, w in pipes:
    graph[u].append((v, w))
    graph[v].append((u, w))

# Connect virtual 0 with nodes 1 to n.
for i, well in enumerate(wells):
    graph[0].append((i + 1, well))
    heapq.heappush(minHeap, (well, i + 1))

```

```

mst = {0}

while len(mst) < n + 1:
    d, u = heapq.heappop(minHeap)
    if u in mst:
        continue
    # Add the new vertex.
    mst.add(u)
    ans += d
    # Expand if possible.

```

```

for v, w in graph[u]:
    if v not in mst:
        heapq.heappush(minHeap, (w, v))

return ans

```

• LeetDoocs

```

class Solution:
    def minCostToSupplyWater(
        self, n: int, wells: List[int], pipes: List[List[int]]
    ) -> int:
        def find(x: int) -> int:
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        for i, w in enumerate(wells, 1):

```

Round 6 · Mid · Hard

p.2815

```

        pipes.append([0, i, w])
    pipes.sort(key=lambda x: x[2])
    p = list(range(n + 1))
    ans = 0
    for a, b, c in pipes:
        pa, pb = find(a), find(b)
        if pa != pb:
            p[pa] = pb
            n -= 1
            ans += c

```

```

        if n == 0:
            return ans

```

```

class UnionFind:
    __slots__ = ("p", "size")

    def __init__(self, n):
        self.p = list(range(n))
        self.size = [1] * n

    def find(self, x: int) -> int:
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])

        return self.p[x]

```

```

    def union(self, a: int, b: int) -> bool:
        pa, pb = self.find(a), self.find(b)
        if pa == pb:
            return False
        if self.size[pa] > self.size[pb]:
            self.p[pb] = pa
            self.size[pa] += self.size[pb]
        else:

```

Round 6 · Mid · Hard

p.2816

```

        self.p[pa] = pb
        self.size[pb] += self.size[pa]
        return True

class Solution:
    def minCostToSupplyWater(
        self, n: int, wells: List[int], pipes: List[List[int]]
    ) -> int:
        for i, w in enumerate(wells, 1):

```

```

            pipes.append([0, i, w])
        pipes.sort(key=lambda x: x[2])
        uf = UnionFind(n + 1)
        ans = 0
        for a, b, c in pipes:
            if uf.union(a, b):
                ans += c
                n -= 1
                if n == 0:
                    return ans

```

• SimplyLeet

```

# Python solution using Kruskal's algorithm and Union-Find

# Define a class to perform Union-Find (Disjoint Set Union)
class UnionFind:
    def __init__(self, size):
        # Initialize parent pointers and rank array for union by rank
        self.parent = list(range(size))
        self.rank = [0] * size

    def find(self, x):

```

```

        # Path compression to find root of x
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, x, y):
        # Union two sets if roots are different; returns True if union
        # performed
        rootX = self.find(x)
        rootY = self.find(y)
        if rootX == rootY:

            return False # They are already connected
        # Attach the tree with lower rank to the root of the tree with higher
        # rank
        if self.rank[rootX] < self.rank[rootY]:
            self.parent[rootX] = rootY
        elif self.rank[rootX] > self.rank[rootY]:
            self.parent[rootY] = rootX
        else:
            self.parent[rootY] = rootX
            self.rank[rootX] += 1
        return True

```

```

def minCostToSupplyWater(n, wells, pipes):
    # Build the edge list, starting with edges from a virtual node 0 to every
    # house
    edges = []
    for i in range(1, n + 1):
        # (cost, house, virtual node 0)
        edges.append((wells[i - 1], i, 0))
    # Add all pipe edges to the list - note these are bidirectional
    for house1, house2, cost in pipes:
        edges.append((cost, house1, house2))

```

```
# Sort edges based on cost
edges.sort()

uf = UnionFind(n + 1)
total_cost = 0

# Process each edge: if it connects two disjoint sets, add it to the MST
for cost, house1, house2 in edges:
    if uf.union(house1, house2):
        total_cost += cost

return total_cost

# Example usage:
n = 3
wells = [1, 2, 2]
pipes = [[1, 2, 1], [2, 3, 1]]
print(minCostToSupplyWater(n, wells, pipes))
```

Minimum Spanning Tree

□ #1489 Find Critical and Pseudo-Critical Edges in Minimum Spanning Tree

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Union-Find · Graph Theory · Sorting ·
 Minimum Spanning Tree · Strongly Connected
 Component

Lists: AlgoMaster 600

Problem

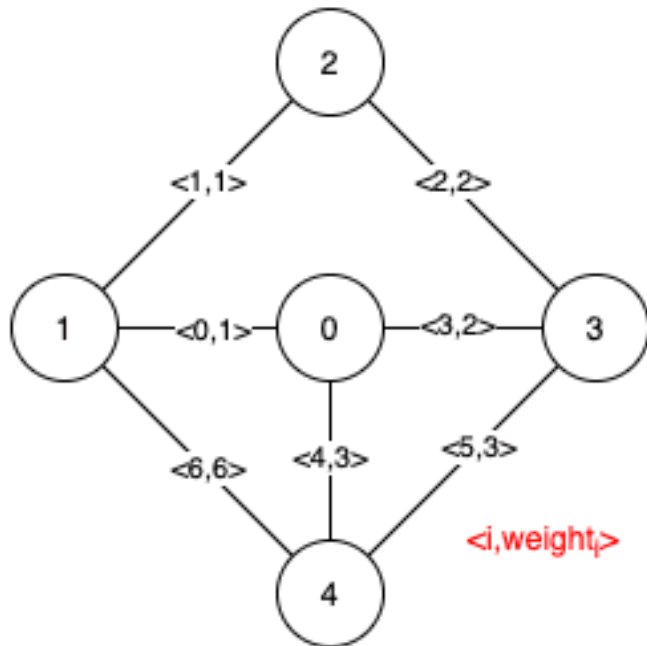
Given a weighted undirected connected graph with n vertices numbered from 0 to $n - 1$, and an array `edges` where `edges[i] = [ai, bi, weighti]` represents a bidirectional and weighted edge between nodes `ai` and `bi`. A minimum spanning tree (MST) is a subset of the graph's edges that connects all vertices without cycles and with the minimum possible total edge weight.

Find all the critical and pseudo-critical edges in the given graph's minimum spanning tree (MST). An MST edge whose deletion from the graph would cause the MST weight to increase is called a critical edge. On the other hand, a

pseudo-critical edge is that which can appear in some MSTs but not all.

Note that you can return the indices of the edges in any order.

Example 1:



Round 6 · Mid · Hard

p.2821

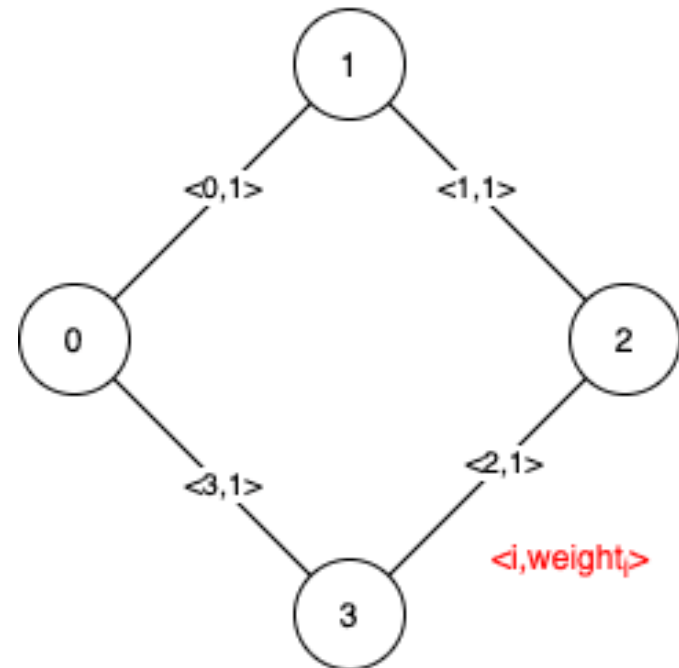
Input: $n = 5$, edges = $[[0,1,1],[1,2,1],[2,3,2],[0,3,2],[0,4,3],[3,4,3],[1,4,6]]$
 Output: $[[0,1],[2,3,4,5]]$

Explanation: The figure above describes the graph.

The following figure shows all the possible MSTs:

Notice that the two edges 0 and 1 appear in all MSTs, therefore they are critical edges, so we return them in the first list of the output. The edges 2, 3, 4, and 5 are only part of some MSTs, therefore they are considered pseudo-critical edges. We add them to the second list of the output.

Example 2:



Input: $n = 4$, edges = $[[0,1,1],[1,2,1],[2,3,1],[0,3,1]]$

Output: $[[],[0,1,2,3]]$

Explanation: We can observe that since all 4 edges have equal weight, choosing any 3 edges from the given 4 will yield an MST. Therefore all 4 edges are pseudo-critical.

Round 6 · Mid · Hard

p.2822

Constraints:

- $2 \leq n \leq 100$
- $1 \leq \text{edges.length} \leq \min(200, n * (n - 1) / 2)$
- $\text{edges}[i].\text{length} == 3$
- $0 \leq a_i < b_i < n$
- $1 \leq \text{weight}_i \leq 1000$
- All pairs (a_i, b_i) are distinct.

Community Solutions (Python)

• WalkCC

```
class UnionFind:
    def __init__(self, n: int):
        self.id = list(range(n))
        self.rank = [0] * n

    def unionByRank(self, u: int, v: int) -> None:
        i = self.find(u)
        j = self.find(v)
        if i == j:
            return

        if self.rank[i] < self.rank[j]:
            self.id[i] = j
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
        else:
            self.id[i] = j
            self.rank[j] += 1

    def find(self, u: int) -> int:
        if self.id[u] != u:
```

```
        self.id[u] = self.find(self.id[u])
        return self.id[u]

class Solution:
    def findCriticalAndPseudoCriticalEdges(self, n: int, edges: list[list[int]])
    -> list[list[int]]:
        criticalEdges = []
        pseudoCriticalEdges = []

        # Record the index information, so edges[i] := (u, v, weight, index).

        for i in range(len(edges)):
            edges[i].append(i)

        # Sort by the weight.
        edges.sort(key=lambda x: x[2])

        def getMSTWeight(
            firstEdge: list[int],
            deletedEdgeIndex: int) -> int | float:
            mstWeight = 0

            uf = UnionFind(n)

            if firstEdge:
                uf.unionByRank(firstEdge[0], firstEdge[1])
                mstWeight += firstEdge[2]

            for u, v, weight, index in edges:
                if index == deletedEdgeIndex:
                    continue
                if uf.find(u) == uf.find(v):
```



```

        continue
    uf.unionByRank(u, v)
    mstWeight += weight

    root = uf.find(0)
    if any(uf.find(i) != root for i in range(n)):
        return math.inf

    return mstWeight

```

```

mstWeight = getMSTWeight([], -1)

for edge in edges:
    index = edge[3]
    # Deleting the `edge` increases the weight of the MST or makes the MST
    # invalid.
    if getMSTWeight([], index) > mstWeight:
        criticalEdges.append(index)
    # If an edge can be in any MST, we can always add `edge` to the edge set.
    elif getMSTWeight(edge, -1) == mstWeight:

```

```

        pseudoCriticalEdges.append(index)

return [criticalEdges, pseudoCriticalEdges]

```

• LeetCode

```

class UnionFind:
    def __init__(self, n):
        self.p = list(range(n))
        self.n = n

    def union(self, a, b):
        if self.find(a) == self.find(b):
            return False
        self.p[self.find(a)] = self.find(b)
        self.n -= 1

```

Round 6 · Mid · Hard

p.2825

```

        return True

    def find(self, x):
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]

class Solution:
    def findCriticalAndPseudoCriticalEdges(
        self, n: int, edges: List[List[int]]
    ) -> List[List[int]]:
        for i, e in enumerate(edges):
            e.append(i)
        edges.sort(key=lambda x: x[2])
        uf = UnionFind(n)
        v = sum(w for f, t, w, _ in edges if uf.union(f, t))
        ans = [[], []]
        for f, t, w, i in edges:
            uf = UnionFind(n)

            k = sum(z for x, y, z, j in edges if j != i and uf.union(x, y))
            if uf.n > 1 or (uf.n == 1 and k > v):
                ans[0].append(i)
                continue

            uf = UnionFind(n)
            uf.union(f, t)
            k = w + sum(z for x, y, z, j in edges if j != i and uf.union(x, y))

            if k == v:
                ans[1].append(i)

        return ans

```

• SimplyLeet

Round 6 · Mid · Hard

p.2826

```
# Python solution with detailed comments

class UnionFind:
    def __init__(self, n):
        # Each node is initially its own parent
        self.parent = list(range(n))
        # Rank for union-by-rank optimization
        self.rank = [0] * n

    def find(self, x):

        # Path compression optimization
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, x, y):
        # Union two sets; return True if they were separate
        xr = self.find(x)
        yr = self.find(y)
        if xr == yr:

            return False
        if self.rank[xr] < self.rank[yr]:
            self.parent[xr] = yr
        elif self.rank[xr] > self.rank[yr]:
            self.parent[yr] = xr
        else:
            self.parent[yr] = xr
            self.rank[xr] += 1
        return True

    def buildMST(n, edges, skip_edge=None, force_edge=None):
        uf = UnionFind(n)
        weight = 0
        count = 0
        # If force_edge is provided, include it prior to processing other edges
        if force_edge is not None:
            u, v, w, idx = force_edge
            if uf.union(u, v):
                weight += w
                count += 1
```

Round 6 · Mid · Hard

p.2827

```
# Process each edge in order of increasing weight
for u, v, w, idx in edges:
    if skip_edge is not None and idx == skip_edge:
        continue
    if uf.union(u, v):
        weight += w
        count += 1
    if count == n - 1:
        break
return weight if count == n - 1 else float('inf')

def findCriticalAndPseudoCriticalEdges(n, edges_input):
    edges = []
    # Append original indices to edges
    for idx, (u, v, w) in enumerate(edges_input):
        edges.append((u, v, w, idx))
    edges.sort(key=lambda x: x[2])
    original_weight = buildMST(n, edges)
    critical = []
    pseudo_critical = []

    # Check each edge
    for u, v, w, idx in edges:
        # Exclude the edge and compute MST weight
        if buildMST(n, edges, skip_edge=idx) > original_weight:
            critical.append(idx)
        else:
            # Force include this edge and compute MST weight
            if buildMST(n, edges, force_edge=(u, v, w, idx)) == original_weight:
                pseudo_critical.append(idx)
    return [critical, pseudo_critical]

# Example usage:
if __name__ == "__main__":
    n = 5
    edges = [[0,1,1],[1,2,1],[2,3,2],[0,3,2],[0,4,3],[3,4,3],[1,4,6]]
    result = findCriticalAndPseudoCriticalEdges(n, edges)
    print(result)
```

Round 6 · Mid · Hard

p.2828

Minimum Spanning Tree

#3600 Maximize Spanning Tree Stability with Upgrades HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Binary Search · Greedy · Union-Find · Graph Theory · Minimum Spanning Tree

Lists: AlgoMaster 600

Problem

You are given an integer n , representing n nodes numbered from 0 to $n - 1$ and a list of edges, where $\text{edges}[i] = [\text{ui}, \text{vi}, \text{si}, \text{musti}]$:

- ui and vi indicates an undirected edge between nodes ui and vi .
- si is the strength of the edge.
- musti is an integer (0 or 1). If $\text{musti} == 1$, the edge must be included in the spanning tree. These edges cannot be upgraded.

You are also given an integer k , the maximum number of upgrades you can perform. Each upgrade doubles the strength of an edge, and each eligible edge (with $\text{musti} == 0$) can be upgraded at most once.

Round 6 · Mid · Hard

p.2829

The stability of a spanning tree is defined as the minimum strength score among all edges included in it.

Return the maximum possible stability of any valid spanning tree. If it is impossible to connect all nodes, return -1 .

Note: A spanning tree of a graph with n nodes is a subset of the edges that connects all nodes together (i.e. the graph is connected) without forming any cycles, and uses exactly $n - 1$ edges.

Example 1:

Input: $n = 3$, $\text{edges} = [[0,1,2,1],[1,2,3,0]]$, $k = 1$

Output: 2

Explanation:

- Edge $[0,1]$ with strength $= 2$ must be included in the spanning tree.
- Edge $[1,2]$ is optional and can be upgraded from 3 to 6 using one upgrade.
- The resulting spanning tree includes these two edges with strengths 2 and 6 .
- The minimum strength in the spanning tree is 2 , which is the maximum possible stability.

Example 2:

Round 6 · Mid · Hard

p.2830

Input: $n = 3$, $edges = [[0,1,4,0],[1,2,3,0],[0,2,1,0]]$,
 $k = 2$

Output: 6

Explanation:

- Since all edges are optional and up to $k = 2$ upgrades are allowed.
- Upgrade edges $[0,1]$ from 4 to 8 and $[1,2]$ from 3 to 6.
- The resulting spanning tree includes these two edges with strengths 8 and 6.
- The minimum strength in the tree is 6, which is the maximum possible stability.

Example 3:

Input: $n = 3$, $edges = [[0,1,1,1],[1,2,1,1],[2,0,1,1]]$,
 $k = 0$

Output: -1

Explanation:

- All edges are mandatory and form a cycle, which violates the spanning tree property of acyclicity. Thus, the answer is -1.

Constraints:

- $2 \leq n \leq 105$
- $1 \leq edges.length \leq 105$
- $edges[i] = [ui, vi, si, musti]$

Round 6 · Mid · Hard

p.2831

- $0 \leq ui, vi < n$
- $ui \neq vi$
- $1 \leq si \leq 105$
- $musti$ is either 0 or 1.
- $0 \leq k \leq n$
- There are no duplicate edges.

Community Solutions (Python)

• LeetCode

```
class UnionFind:
    def __init__(self, n):
        self.p = list(range(n))
        self.size = [1] * n
        self.cnt = n

    def find(self, x):
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]

    def union(self, a, b):
        pa, pb = self.find(a), self.find(b)
        if pa == pb:
            return False
        if self.size[pa] > self.size[pb]:
            self.p[pb] = pa
            self.size[pa] += self.size[pb]
        else:
            self.p[pa] = pb
```

Round 6 · Mid · Hard

p.2832

```

        self.size[pb] += self.size[pa]
        self.cnt -= 1
        return True

class Solution:
    def maxStability(self, n: int, edges: List[List[int]], k: int) -> int:
        def check(lim: int) -> bool:
            uf = UnionFind(n)
            for u, v, s, _ in edges:
                if s >= lim:
                    uf.union(u, v)
            rem = k
            for u, v, s, _ in edges:
                if s * 2 >= lim and rem > 0:
                    if uf.union(u, v):
                        rem -= 1
            return uf.cnt == 1

        uf = UnionFind(n)

        mn = 10**6
        for u, v, s, must in edges:
            if must:
                mn = min(mn, s)
                if not uf.union(u, v):
                    return -1
        for u, v, _, _ in edges:
            uf.union(u, v)
        if uf.cnt > 1:
            return -1

        l, r = 1, mn
        while l < r:
            mid = (l + r + 1) >> 1
            if check(mid):
                l = mid
            else:
                r = mid - 1
        return l

```

Round 6 · Mid · Hard

p.2833

Shortest Path

Round 6 · Mid · Hard · 3 questions

Round 6 · Mid · Hard

p.2834

Shortest Path

#778 Swim in Rising Water

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Binary Search · Depth-First Search ·
 Breadth-First Search · Union-Find · Heap
 (Priority Queue) · Matrix

Lists: NeetCode 150

Problem

You are given an $n \times n$ integer matrix grid where each value $\text{grid}[i][j]$ represents the elevation at that point (i, j) .

It starts raining, and water gradually rises over time. At time t , the water level is t , meaning any cell with elevation less than equal to t is submerged or reachable.

You can swim from a square to another 4-directionally adjacent square if and only if the elevation of both squares individually are at most t . You can swim infinite distances in zero time. Of course, you must stay within the boundaries of the grid during your swim.

Round 6 · Mid · Hard

p.2835

Return the minimum time until you can reach the bottom right square $(n - 1, n - 1)$ if you start at the top left square $(0, 0)$.

Example 1:

0	2
1	3

Input: grid = [[0,2],[1,3]]

Output: 3

Explanation:

At time 0, you are in grid location $(0, 0)$.

You cannot go anywhere else because 4-directionally adjacent neighbors have a higher elevation than $t = 0$.

You cannot reach point $(1, 1)$ until time 3.

When the depth of water is 3, we can swim anywhere inside the grid.

Round 6 · Mid · Hard

p.2836

Example 2:

0	1	2	3	4
24	23	22	21	5
12	13	14	15	16
11	17	18	19	20
10	9	8	7	6

Input: grid =
[[0,1,2,3,4],[24,23,22,21,5],[12,13,14,15,16],[11,17,18,19,20],[10,9,8,7,6]]
Output: 16
Explanation: The final route is shown.
We need to wait until time 16 so that (0, 0) and (4, 4) are connected.

Constraints:

- $n == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq n \leq 50$
- $0 \leq \text{grid}[i][j] < n^2$

Round 6 · Mid · Hard

p.2837

- Each value $\text{grid}[i][j]$ is unique.

Community Solutions (Python)

• LeetCode

```
class Solution:
    def swimInWater(self, grid: List[List[int]]) -> int:
        def find(x: int) -> int:
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        n = len(grid)
        m = n * n
        p = list(range(m))

        hi = [0] * m
        for i, row in enumerate(grid):
            for j, h in enumerate(row):
                hi[h] = i * n + j
        dirs = (-1, 0, 1, 0, -1)
        for t in range(m):
            x, y = divmod(hi[t], n)
            for dx, dy in pairwise(dirs):
                nx, ny = x + dx, y + dy
                if 0 <= nx < n and 0 <= ny < n and grid[nx][ny] <= t:
                    p[find(x * n + y)] = find(nx * n + ny)

            if find(0) == find(m - 1):
                return t
        return 0
```

• SimplyLeet

Round 6 · Mid · Hard

p.2838

```
import heapq

def swimInWater(grid):
    n = len(grid)
    # Initialize a visited set to track cells we have processed.
    visited = [[False] * n for _ in range(n)]
    # Min-heap: (current max elevation along the path, row, col)
    min_heap = [(grid[0][0], 0, 0)]
    visited[0][0] = True
    # Directions for moving up, down, left, and right.

    directions = [(1,0), (-1,0), (0,1), (0,-1)]

    while min_heap:
        time, r, c = heapq.heappop(min_heap)
        # If we reach the bottom-right, return the maximum elevation
        # encountered along the path.
        if r == n - 1 and c == n - 1:
            return time
        for dr, dc in directions:
            nr, nc = r + dr, c + dc
            # Check boundaries and if the cell is not already visited.

            if 0 <= nr < n and 0 <= nc < n and not visited[nr][nc]:
                visited[nr][nc] = True
                # The required time is the maximum of the current time and the
                # elevation at the new cell.
                new_time = max(time, grid[nr][nc])
                heapq.heappush(min_heap, (new_time, nr, nc))

    return -1 # Should not reach here if input is as per constraints.

# Example Usage:
grid = [[0,2],[1,3]]

print(swimInWater(grid)) # Output: 3
```

Shortest Path

□ #1368 Minimum Cost to Make at Least One Valid Path in a Grid

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Breadth-First Search · Graph Theory ·
 Heap (Priority Queue) · Matrix · Shortest Path
 Lists: AlgoMaster 600

Problem

Given an $m \times n$ grid. Each cell of the grid has a sign pointing to the next cell you should visit if you are currently in this cell. The sign of $grid[i][j]$ can be:

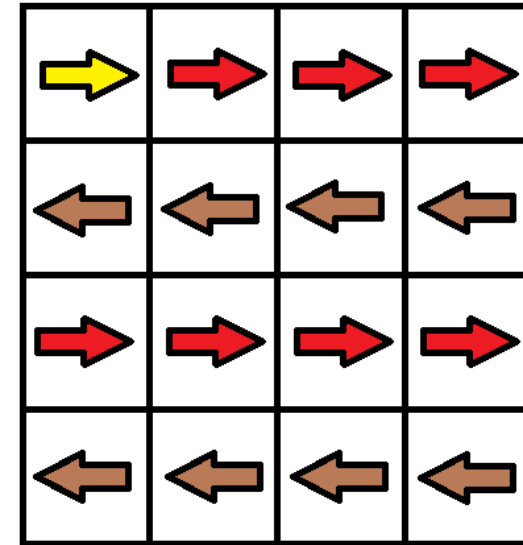
- 1 which means go to the cell to the right. (i.e go from $grid[i][j]$ to $grid[i][j + 1]$)
- 2 which means go to the cell to the left. (i.e go from $grid[i][j]$ to $grid[i][j - 1]$)
- 3 which means go to the lower cell. (i.e go from $grid[i][j]$ to $grid[i + 1][j]$)
- 4 which means go to the upper cell. (i.e go from $grid[i][j]$ to $grid[i - 1][j]$)

Notice that there could be some signs on the cells of the grid that point outside the grid.

You will initially start at the upper left cell (0, 0). A valid path in the grid is a path that starts from the upper left cell (0, 0) and ends at the bottom-right cell (m - 1, n - 1) following the signs on the grid. The valid path does not have to be the shortest.

You can modify the sign on a cell with cost = 1. You can modify the sign on a cell one time only. Return the minimum cost to make the grid have at least one valid path.

Example 1:



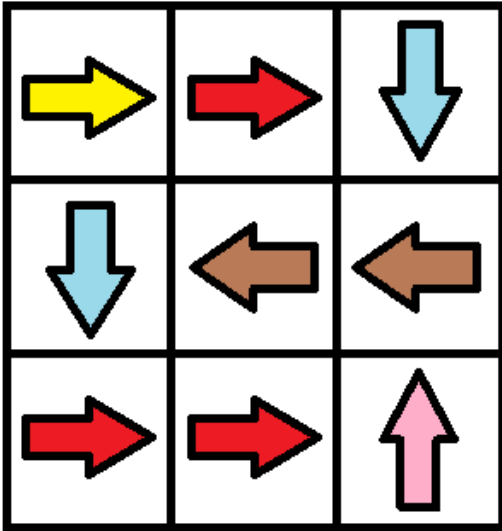
Input: grid = [[1,1,1,1],[2,2,2,2],[1,1,1,1],[2,2,2,2]]

Output: 3

Explanation: You will start at point (0, 0).

The path to (3, 3) is as follows. (0, 0) --> (0, 1) --> (0, 2) --> (0, 3) change the arrow to down with cost = 1 --> (1, 3) --> (1, 2) --> (1, 1) --> (1, 0) change the arrow to down with cost = 1 --> (2, 0) --> (2, 1) --> (2, 2) --> (2, 3) change the arrow to down with cost = 1 --> (3, 3)
The total cost = 3.

Example 2:

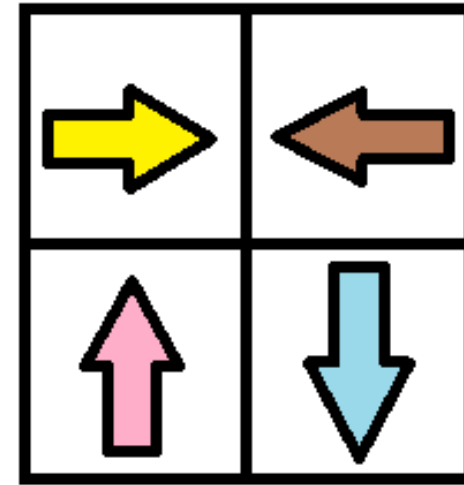


Input: grid = [[1,1,3],[3,2,2],[1,1,4]]
 Output: 0
 Explanation: You can follow the path from (0, 0) to (2, 2).

Example 3:

Round 6 · Mid · Hard

p.2843



Input: grid = [[1,2],[4,3]]
 Output: 1

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq m, n \leq 100$
- $1 \leq \text{grid}[i][j] \leq 4$

Community Solutions (Python)

- WalkCC

Round 6 · Mid · Hard

p.2844

```
class Solution:
    def minCost(self, grid: list[list[int]]) -> int:
        m = len(grid)
        n = len(grid[0])
        DIRS = ((0, 1), (0, -1), (1, 0), (-1, 0))
        dp = [[-1] * n for _ in range(m)]
        q = collections.deque()
```

```
    def dfs(i: int, j: int, cost: int) -> None:
        if i < 0 or i == m or j < 0 or j == n:
```

```
            return
        if dp[i][j] != -1:
            return
        dp[i][j] = cost
        q.append((i, j))
        dx, dy = DIRS[grid[i][j] - 1]
        dfs(i + dx, j + dy, cost)
```

```
    dfs(0, 0, 0)
```

```
    cost = 0
    while q:
        cost += 1
        for _ in range(len(q)):
            i, j = q.popleft()
            for dx, dy in DIRS:
                dfs(i + dx, j + dy, cost)

    return dp[-1][-1]
```

• LeetCode

Round 6 · Mid · Hard

p.2845

```
class Solution:
    def minCost(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        dirs = [[0, 0], [0, 1], [0, -1], [1, 0], [-1, 0]]
        q = deque([(0, 0, 0)])
        vis = set()
        while q:
            i, j, d = q.popleft()
            if (i, j) in vis:
                continue
```

```
            vis.add((i, j))
            if i == m - 1 and j == n - 1:
                return d
            for k in range(1, 5):
                x, y = i + dirs[k][0], j + dirs[k][1]
                if 0 <= x < m and 0 <= y < n:
                    if grid[i][j] == k:
                        q.appendleft((x, y, d))
                    else:
                        q.append((x, y, d + 1))
```

```
        return -1
```

• SimplyLeet

```
from collections import deque

def minCost(grid):
    m, n = len(grid), len(grid[0])
    # Directions: right, left, down, up correspond to 0,1,2,3 in our 0-indexed
    list.
    directions = [(0, 1), (0, -1), (1, 0), (-1, 0)]
    # cost matrix initialized with infinity
    cost = [[float('inf')] * n for _ in range(m)]
    cost[0][0] = 0
    dq = deque()
```

Round 6 · Mid · Hard

p.2846

```

dq.append((0, 0))

while dq:
    i, j = dq.popleft()
    # If reached the destination, return the cost immediately.
    if i == m - 1 and j == n - 1:
        return cost[i][j]
    for k, (di, dj) in enumerate(directions):
        ni, nj = i + di, j + dj
        if 0 <= ni < m and 0 <= nj < n:
            # If the current direction (grid[i][j]) matches the direction
            # index k+1 then costChange is 0; otherwise 1.
            cost_change = 0 if grid[i][j] == k + 1 else 1
            if cost[i][j] + cost_change < cost[ni][nj]:
                cost[ni][nj] = cost[i][j] + cost_change
                if cost_change == 0:
                    dq.appendleft((ni, nj))
                else:
                    dq.append((ni, nj))
    return cost[m-1][n-1]

# Example usage:
grid = [[1,1,1,1],[2,2,2,2],[1,1,1,1],[2,2,2,2]]
print(minCost(grid))

```

Shortest Path

□ #2203 Minimum Weighted Subgraph With the Required Paths

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Graph Theory · Heap (Priority Queue) · Shortest Path

Lists: AlgoMaster 600

Problem

You are given an integer n denoting the number of nodes of a weighted directed graph. The nodes are numbered from 0 to $n - 1$.

You are also given a 2D integer array `edges` where `edges[i] = [fromi, toi, weighti]` denotes that there exists a directed edge from `fromi` to `toi` with weight `weighti`.

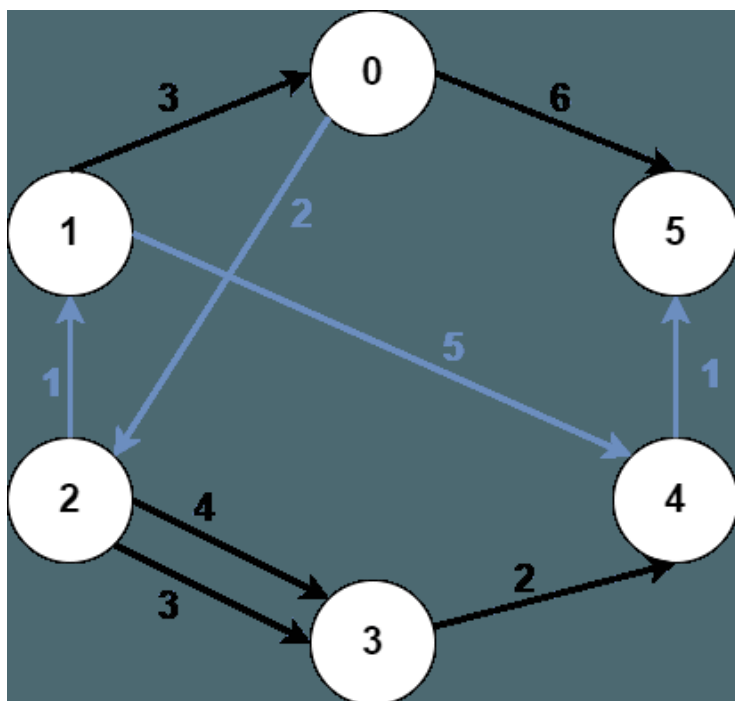
Lastly, you are given three distinct integers `src1`, `src2`, and `dest` denoting three distinct nodes of the graph.

Return the minimum weight of a subgraph of the graph such that it is possible to reach `dest` from both `src1` and `src2` via a set of edges of this subgraph. In case such a subgraph does not

exist, return -1.

A subgraph is a graph whose vertices and edges are subsets of the original graph. The weight of a subgraph is the sum of weights of its constituent edges.

Example 1:



Input: $n = 6$, edges =
 $[[0,2,2],[0,5,6],[1,0,3],[1,4,5],[2,1,1],[2,3,3],[2,3,4],[3,4,2],[4,5,1]]$,
 $src1 = 0$, $src2 = 1$, $dest = 5$
 Output: 9
 Explanation:
 The above figure represents the input graph.
 The blue edges represent one of the subgraphs that yield the optimal answer.
 Note that the subgraph $[[1,0,3],[0,5,6]]$ also yields the optimal answer. It is not possible to get a subgraph with less weight satisfying all the constraints.

Example 2:



Input: $n = 3$, edges = $[[0,1,1],[2,1,1]]$, $src1 = 0$, $src2 = 1$, $dest = 2$
 Output: -1
 Explanation:
 The above figure represents the input graph.
 It can be seen that there does not exist any path from node 1 to node 2, hence there are no subgraphs satisfying all the constraints.

Constraints:

- $3 \leq n \leq 105$
- $0 \leq \text{edges.length} \leq 105$
- $\text{edges}[i].\text{length} == 3$
- $0 \leq \text{from}_i, \text{to}_i, \text{src1}, \text{src2}, \text{dest} \leq n - 1$
- $\text{from}_i \neq \text{to}_i$
- $\text{src1}, \text{src2}$, and dest are pairwise distinct.
- $1 \leq \text{weight}[i] \leq 105$

Community Solutions (Python)

- WalkCC

```

class Solution:
    def minimumWeight(
        self,
        n: int,
        edges: list[list[int]],
        src1: int,
        src2: int,
        dest: int,
    ) -> int:
        graph = [[] for _ in range(n)]

        reversedGraph = [[] for _ in range(n)]

        for u, v, w in edges:
            graph[u].append((v, w))
            reversedGraph[v].append((u, w))

        fromSrc1 = self._dijkstra(graph, src1)
        fromSrc2 = self._dijkstra(graph, src2)
        fromDest = self._dijkstra(reversedGraph, dest)
        minWeight = min(a + b + c for a, b, c in zip(fromSrc1, fromSrc2, fromDest))

        return -1 if minWeight == math.inf else minWeight

    def _dijkstra(
        self,
        graph: list[list[tuple[int, int]]],
        src: int,
    ) -> list[int]:
        dist = [math.inf] * len(graph)

        dist[src] = 0

```

Round 6 · Mid · Hard

p.2851

```

minHeap = [(dist[src], src)] # (d, u)

while minHeap:
    d, u = heapq.heappop(minHeap)
    if d > dist[u]:
        continue
    for v, w in graph[u]:
        if d + w < dist[v]:
            dist[v] = d + w
            heapq.heappush(minHeap, (dist[v], v))

return dist

```

• LeetCode

```

class Solution:
    def minimumWeight(
        self, n: int, edges: List[List[int]], src1: int, src2: int, dest: int
    ) -> int:
        def dijkstra(g, u):
            dist = [inf] * n
            dist[u] = 0
            q = [(0, u)]
            while q:
                d, u = heapq.heappop(q)

                if d > dist[u]:
                    continue
                for v, w in g[u]:
                    if dist[v] > dist[u] + w:
                        dist[v] = dist[u] + w
                        heapq.heappush(q, (dist[v], v))
            return dist

        g = defaultdict(list)
        rg = defaultdict(list)

```

Round 6 · Mid · Hard

p.2852

```

for f, t, w in edges:
    g[f].append((t, w))
    rg[t].append((f, w))
d1 = dijkstra(g, src1)
d2 = dijkstra(g, src2)
d3 = dijkstra(rg, dest)
ans = min(sum(v) for v in zip(d1, d2, d3))
return -1 if ans >= inf else ans

```

• SimplyLeet

```

import heapq

def minimumWeight(n, edges, src1, src2, dest):
    # Build graph in normal direction and reversed direction
    graph = [[] for _ in range(n)]
    reverse_graph = [[] for _ in range(n)]

    # Build both graphs according to the given directed weighted edges
    for u, v, w in edges:
        graph[u].append((v, w))

        reverse_graph[v].append((u, w))

    # Function to perform Dijkstra's algorithm from a source node
    def dijkstra(start, graph):
        dist = [float('inf')] * n
        dist[start] = 0
        minheap = [(0, start)] # (distance, node)
        while minheap:
            d, node = heapq.heappop(minheap)
            # If we have already found a better path, skip this entry

            if d > dist[node]:
                continue
            # Relax edges out of current node
            for neighbor, weight in graph[node]:
                new_d = d + weight
                if new_d < dist[neighbor]:
                    dist[neighbor] = new_d
                    heapq.heappush(minheap, (new_d, neighbor))
        return dist

```

Round 6 · Mid · Hard

p.2853

```

# Compute distances from src1 and src2 on the original graph.
dist_from_src1 = dijkstra(src1, graph)
dist_from_src2 = dijkstra(src2, graph)

# Compute distances from dest on the reversed graph (equivalent to
distances from every node to dest)
dist_to_dest = dijkstra(dest, reverse_graph)

# Compute the minimum combined weight
result = float('inf')
for node in range(n):

    total = dist_from_src1[node] + dist_from_src2[node] + dist_to_dest[nod
e]

    if total < result:
        result = total

# If result remains infinity, then there's no valid subgraph meeting
requirements.
return -1 if result == float('inf') else result

# Example usage:
# n = 6, edges =
[[0,2,2],[0,5,6],[1,0,3],[1,4,5],[2,1,1],[2,3,3],[2,3,4],[3,4,2],[4,5,1]],
src1 = 0, src2 = 1, dest = 5
# Expected output: 9

print(minimumWeight(6, [[0,2,2],[0,5,6],[1,0,3],[1,4,5],[2,1,1],[2,3,3],[2,3,4
],[3,4,2],[4,5,1]], 0, 1, 5))

```

Round 6 · Mid · Hard

p.2854

Round 7 · Low · Easy

Round 7

Low Priority · Easy

11 questions · 4 patterns

- ☐ ● 1-D DP #70 #509 #746 ↗
- ☐ ● 2D Grid DP #118 ↗
- ☐ ● Maths / Geometry #9 #13 #66 #168 #263 #1071 ↗
- ☐ ● String Matching #459 ↗

Round 6 · Mid · Hard

p.2855

1-D DP

Round 7 · Low · Easy · 3 questions

Round 7 · Low · Easy

p.2856

1-D DP

#70 Climbing Stairs

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · Dynamic Programming · Memoization
Lists: Grind 169

Problem

You are climbing a staircase. It takes n steps to reach the top.

Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Example 1:

Input: $n = 2$
Output: 2
Explanation: There are two ways to climb to the top.
1. 1 step + 1 step
2. 2 steps

Example 2:

Input: $n = 3$
Output: 3
Explanation: There are three ways to climb to the top.
1. 1 step + 1 step + 1 step
2. 1 step + 2 steps
3. 2 steps + 1 step

Constraints:

- $1 \leq n \leq 45$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def climbStairs(self, n: int) -> int:
        # dp[i] := the number of ways to climb to the i-th stair
        dp = [1, 1] + [0] * (n - 1)

        for i in range(2, n + 1):
            dp[i] = dp[i - 1] + dp[i - 2]

        return dp[n]
```

```
class Solution:
    def climbStairs(self, n: int) -> int:
        prev1 = 1 # dp[i - 1]
        prev2 = 1 # dp[i - 2]

        for _ in range(2, n + 1):
            dp = prev1 + prev2
            prev2 = prev1
            prev1 = dp
```

```
return prev1
```

• LeetDoocs

```
class Solution:
    def climbStairs(self, n: int) -> int:
        a, b = 0, 1
        for _ in range(n):
            a, b = b, a + b
        return b
```

```
import numpy as np

class Solution:
    def climbStairs(self, n: int) -> int:
        res = np.asmatrix([[1, 1]], np.dtype("0"))
        factor = np.asmatrix([[1, 1], (1, 0)], np.dtype("0"))
        n -= 1
        while n:
            if n & 1:
                res *= factor
            factor *= factor
            n >>= 1
        return res[0, 0]
```

• SimplyLeet

```
# Define a function to compute the number of ways to climb stairs.
def climbStairs(n):
    # Base case: if there is only one step, return 1.
    if n == 1:
        return 1
    # Initialize the dp array with n+1 elements.
    dp = [0] * (n + 1)
    # There is 1 way to climb 1 step.
    dp[1] = 1
    # There are 2 ways to climb 2 steps.

    dp[2] = 2
    # Compute the number of ways for each step from 3 to n.
    for i in range(3, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2] # Sum of ways from the previous two
    steps.
    return dp[n]

# Example usage:
print(climbStairs(3)) # Output: 3
```

• LC.ca

Round 7 · Low · Easy

p.2859

```
class Solution:
    def climbStairs(self, n: int) -> int:
        a, b = 0, 1
        for _ in range(n):
            a, b = b, a + b
        return b
```

◆ Quick Review

Key Insights

- The problem can be solved using dynamic programming because the solution for a given step depends on the solutions of the previous two steps.
- The recurrence relation is: $\text{ways}[i] = \text{ways}[i-1] + \text{ways}[i-2]$.
- Base cases: when there is 1 step, there is 1 way; when there are 2 steps, there are 2 ways.
- This is analogous to the Fibonacci sequence, shifting indices accordingly.

Space & Time

Time Complexity: $O(n)$ Space Complexity: $O(n)$ (can be optimized to $O(1)$ using constant space)

Solution

Round 7 · Low · Easy

p.2860

We use a dynamic programming approach to build a solution gradually. The idea is that to reach step i , you could have come from step $i-1$ (taking one step) or from step $i-2$ (taking two steps). Therefore, the total number of ways to reach step i is the sum of ways to reach step $i-1$ and $i-2$. We initialize $dp[1] = 1$ and $dp[2] = 2$, and iterate from 3 to n to fill $dp[]$. Each code solution below uses this approach with comments to explain the logic.

Round 7 · Low · Easy

p.2861

1-D DP

□ #509 Fibonacci Number

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · Dynamic Programming · Recursion · Memoization

Lists: AlgoMaster 600

Problem

The Fibonacci numbers, commonly denoted $F(n)$ form a sequence, called the Fibonacci sequence, such that each number is the sum of the two preceding ones, starting from 0 and 1. That is,

$$F(0) = 0, F(1) = 1$$

$$F(n) = F(n-1) + F(n-2), \text{ for } n > 1.$$

Given n , calculate $F(n)$.

Example 1:

Input: $n = 2$
Output: 1
Explanation: $F(2) = F(1) + F(0) = 1 + 0 = 1$.

Example 2:

Input: $n = 3$
Output: 2
Explanation: $F(3) = F(2) + F(1) = 1 + 1 = 2$.

Example 3:

Round 7 · Low · Easy

p.2862

Input: n = 4
 Output: 3
 Explanation: F(4) = F(3) + F(2) = 2 + 1 = 3.

Constraints:

- $0 \leq n \leq 30$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def fib(self, n: int) -> int:
        if n < 2:
            return n

        dp = [0, 0, 1]

        for i in range(2, n + 1):
            dp[0] = dp[1]
            dp[1] = dp[2]

            dp[2] = dp[0] + dp[1]

        return dp[2]
```

• LeetDoocs

```
class Solution:
    def fib(self, n: int) -> int:
        a, b = 0, 1
        for _ in range(n):
            a, b = b, a + b
        return a
```

```
import numpy as np

class Solution:
    def fib(self, n: int) -> int:
        factor = np.asmatrix([[1, 1], [1, 0]], np.dtype("0"))
        res = np.asmatrix([[1, 0]], np.dtype("0"))
        while n:
            if n & 1:
                res = res * factor
            factor = factor * factor
            n >>= 1
        return res[0, 1]
```

• SimplyLeet

```
# Function to compute the nth Fibonacci number
def fibonacci(n):
    # If n is 0, return 0 directly
    if n == 0:
        return 0

    # Initialize first two Fibonacci numbers
    a, b = 0, 1
    # Iterate from 2 up to n (inclusive)
    for i in range(2, n + 1):
        # Compute the next Fibonacci number by summing the last two
        a, b = b, a + b # Update a and b for the next iteration
    # For n=1, b already equals 1; for other n, result stored in b
    return b

# Example usage:
print(fibonacci(4)) # Expected output: 3
```

1-D DP

#746 Min Cost Climbing Stairs

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Dynamic Programming
 Lists: NeetCode 150

Problem

You are given an integer array `cost` where `cost[i]` is the cost of *i*th step on a staircase. Once you pay the cost, you can either climb one or two steps.

You can either start from the step with index 0, or the step with index 1.

Return the minimum cost to reach the top of the floor.

Example 1:

Input: `cost = [10,15,20]`
 Output: 15
 Explanation: You will start at index 1.
 - Pay 15 and climb two steps to reach the top.
 The total cost is 15.

Example 2:

Input: `cost = [1,100,1,1,1,100,1,1,100,1]`
 Output: 6
 Explanation: You will start at index 0.
 - Pay 1 and climb two steps to reach index 2.
 - Pay 1 and climb two steps to reach index 4.
 - Pay 1 and climb two steps to reach index 6.
 - Pay 1 and climb one step to reach index 7.
 - Pay 1 and climb two steps to reach index 9.

Constraints:

- $2 \leq \text{cost.length} \leq 1000$
- $0 \leq \text{cost}[i] \leq 999$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minCostClimbingStairs(self, cost: List[int]) -> int:
        cost.append(0)

        for i in range(2, len(cost)):
            cost[i] += min(cost[i - 1], cost[i - 2])

        return cost[-1]
```

• LeetDoocs

```
class Solution:
    def minCostClimbingStairs(self, cost: List[int]) -> int:
        @cache
        def dfs(i: int) -> int:
            if i >= len(cost):
                return 0
            return cost[i] + min(dfs(i + 1), dfs(i + 2))

        return min(dfs(0), dfs(1))
```

```
class Solution:
    def minCostClimbingStairs(self, cost: List[int]) -> int:
        n = len(cost)
        f = [0] * (n + 1)
        for i in range(2, n + 1):
            f[i] = min(f[i - 2] + cost[i - 2], f[i - 1] + cost[i - 1])
        return f[n]
```

```
class Solution:
    def minCostClimbingStairs(self, cost: List[int]) -> int:
        f = g = 0
        for i in range(2, len(cost) + 1):
            f, g = g, min(f + cost[i - 2], g + cost[i - 1])
        return g
```

• SimplyLeet

Python implementation with detailed comments

```
def minCostClimbingStairs(cost):
    n = len(cost) # number of steps
    # dp array to store minimum cost to reach each step
    dp = [0] * n
    dp[0] = cost[0] # cost to start at step 0
    dp[1] = cost[1] # cost to start at step 1

    # Compute minimum cost for each subsequent step

    for i in range(2, n):
        # cost to reach current step is current cost + min cost of the two
        # previous steps
        dp[i] = cost[i] + min(dp[i-1], dp[i-2])

    # The top can be reached from the last or second last step
    return min(dp[n-1], dp[n-2])

# Example usage:
cost = [10, 15, 20]
print(minCostClimbingStairs(cost))
```

Round 7 · Low · Easy

p.2867

2D Grid DP

Round 7 · Low · Easy · 1 question

Round 7 · Low · Easy

p.2868

2D Grid DP

#118 Pascal's Triangle

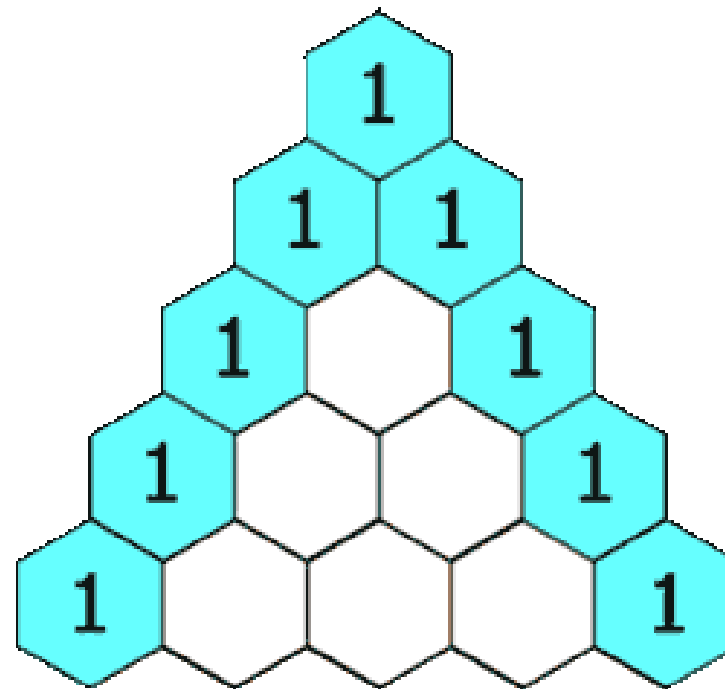
EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Dynamic Programming
 Lists: AlgoMaster 600

Problem

Given an integer numRows, return the first numRows of Pascal's triangle.

In Pascal's triangle, each number is the sum of the two numbers directly above it as shown:



Example 1:

Input: numRows = 5
 Output: [[1],[1,1],[1,2,1],[1,3,3,1],[1,4,6,4,1]]

Example 2:

Input: numRows = 1
 Output: [[1]]

Constraints:

- $1 \leq \text{numRows} \leq 30$

Community Solutions (Python)

Round 7 · Low · Easy

p.2869

Round 7 · Low · Easy

p.2870

• WalkCC

```
class Solution:
    def generate(self, numRows: int) -> list[list[int]]:
        ans = []

        for i in range(numRows):
            ans.append([1] * (i + 1))

        for i in range(2, numRows):
            for j in range(1, len(ans[i]) - 1):
                ans[i][j] = ans[i - 1][j - 1] + ans[i - 1][j]

        return ans
```

• LeetCode

```
class Solution:
    def generate(self, numRows: int) -> List[List[int]]:
        f = [[1]]
        for i in range(numRows - 1):
            g = [1] + [a + b for a, b in pairwise(f[-1])] + [1]
            f.append(g)
        return f
```

• SimplyLeet

```
def generate_pascal_triangle(numRows):
    # Initialize the triangle with the first row
    triangle = [[1]]

    # Generate each row from the second to numRows
    for row_number in range(1, numRows):
        # Start the new row with the number 1
        row = [1]
        # Get the previous row to compute the current row values
        previous_row = triangle[row_number - 1]
```

Round 7 · Low · Easy

p.2871

```
# Compute the inner elements of the row
for j in range(1, row_number):
    # Sum of two adjacent numbers from the previous row
    row.append(previous_row[j - 1] + previous_row[j])
# End the row with the number 1
row.append(1)
# Add the constructed row to the triangle
triangle.append(row)
```

```
return triangle
```

```
# Example usage:
print(generate_pascal_triangle(5))
```

Round 7 · Low · Easy

p.2872

Maths / Geometry

Round 7 · Low · Easy · 6 questions

Round 7 · Low · Easy

p.2873

Maths / Geometry

#9 Palindrome Number

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math

Lists: Grind 169

Problem

Given an integer x , return true if x is a palindrome, and false otherwise.

Example 1:

Input: $x = 121$
Output: true
Explanation: 121 reads as 121 from left to right and from right to left.

Example 2:

Input: $x = -121$
Output: false
Explanation: From left to right, it reads -121. From right to left, it becomes 121-. Therefore it is not a palindrome.

Example 3:

Input: $x = 10$
Output: false
Explanation: Reads 01 from right to left. Therefore it is not a palindrome.

Constraints:

- $-231 \leq x \leq 231 - 1$

Round 7 · Low · Easy

p.2874

Follow up: Could you solve it without converting the integer to a string?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isPalindrome(self, x: int) -> bool:
        if x < 0:
            return False

        rev = 0
        y = x

        while y:
            rev = rev * 10 + y % 10

            y //= 10

        return rev == x
```

• LeetDoocs

```
class Solution:
    def isPalindrome(self, x: int) -> bool:
        if x < 0 or (x and x % 10 == 0):
            return False
        y = 0
        while y < x:
            y = y * 10 + x % 10
            x //= 10
        return x in (y, y // 10)
```

• SimplyLeet

Round 7 · Low · Easy

p.2875

```
# Function to check if a number is a palindrome without converting to a string.
def isPalindrome(x):
    # Negative numbers and numbers ending with 0 (but not 0 itself) are not
    # palindromic.
    if x < 0 or (x % 10 == 0 and x != 0):
        return False
    reversed_half = 0
    # Process: Reverse half of the number.
    while x > reversed_half:
        # Add the last digit of x to reversed_half.
        reversed_half = reversed_half * 10 + x % 10

        # Remove the last digit from x.
        x //= 10
    # For numbers with odd digits, reversed_half // 10 removes the middle
    # digit.
    return x == reversed_half or x == reversed_half // 10

# Example usage:
print(isPalindrome(121)) # Output: True
print(isPalindrome(-121)) # Output: False
print(isPalindrome(10)) # Output: False
```

• LC.ca

```
class Solution:
    def isPalindrome(self, x: int) -> bool:
        if x < 0 or (x and x % 10 == 0):
            return False
        y = 0
        while y < x:
            y = y * 10 + x % 10
            x //= 10
        return x in (y, y // 10)
```

◆ Quick Review

Key Insights

- Negative numbers are not palindromic.

Round 7 · Low · Easy

p.2876

- A number ending in 0 (except 0 itself) cannot be a palindrome.
- Instead of reversing the entire number (risking overflow), reverse only half of the number and compare with the other half.
- For numbers with an odd number of digits, the middle digit can be disregarded.

Space & Time

Time Complexity: $O(\log_{10}(n))$ – Each iteration reduces the number by a factor of 10.

Space Complexity: $O(1)$ – Only constant extra space is used.

Solution

The solution checks for obvious non-palindrome cases first (negative numbers and numbers ending with 0 that are not 0). Then, it reverses half of the digits of the number while the original number is larger than the reversed half. Finally, it compares the remaining part of the original number with the reversed half. In the case of an odd number of digits, the middle digit is removed by dividing the reversed half by 10 before the final comparison.

Maths / Geometry

□ #13 Roman to Integer

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Math · String
Lists: Grind 169

Problem

Roman numerals are represented by seven different symbols: I, V, X, L, C, D and M.

Symbol	Value
I	1
V	5
X	10
L	50
C	100
D	500
M	1000

For example, 2 is written as II in Roman numeral, just two ones added together. 12 is written as XII, which is simply X + II. The number 27 is written as XXVII, which is XX + V + II.

Roman numerals are usually written largest to smallest from left to right. However, the numeral for four is not IIII. Instead, the number four is written as IV. Because the one is before

the five we subtract it making four. The same principle applies to the number nine, which is written as IX. There are six instances where subtraction is used:

- I can be placed before V (5) and X (10) to make 4 and 9.
- X can be placed before L (50) and C (100) to make 40 and 90.
- C can be placed before D (500) and M (1000) to make 400 and 900.

Given a roman numeral, convert it to an integer.

Example 1:

Input: s = "III"
Output: 3
Explanation: III = 3.

Example 2:

Input: s = "LVIII"
Output: 58
Explanation: L = 50, V = 5, III = 3.

Example 3:

Input: s = "MCMXCIV"
Output: 1994
Explanation: M = 1000, CM = 900, XC = 90 and IV = 4.

Constraints:

- $1 \leq s.length \leq 15$

- s contains only the characters ('I', 'V', 'X', 'L', 'C', 'D', 'M').
- It is guaranteed that s is a valid roman numeral in the range [1, 3999].

Community Solutions (Python)

• WalkCC

```
class Solution:
    def romanToInt(self, s: str) -> int:
        ans = 0
        roman = {'I': 1, 'V': 5, 'X': 10, 'L': 50,
                  'C': 100, 'D': 500, 'M': 1000}

        for a, b in zip(s, s[1:]):
            if roman[a] < roman[b]:
                ans -= roman[a]
            else:
                ans += roman[a]

        return ans + roman[s[-1]]
```

• LeetCode

```
class Solution:
    def romanToInt(self, s: str) -> int:
        d = {'I': 1, 'V': 5, 'X': 10, 'L': 50, 'C': 100, 'D': 500, 'M': 1000}
        return sum((-1 if d[a] < d[b] else 1) * d[a] for a, b in pairwise(s))
        + d[s[-1]]
```

• SimplyLeet

```
# Define a function to convert Roman numeral to integer
def romanToInt(s):
    # Map each Roman numeral to its integer value
    roman_values = {'I': 1, 'V': 5, 'X': 10, 'L': 50, 'C': 100, 'D': 500, 'M': 1000}
    total = 0 # Initialize the total integer value

    # Loop over the string using index
    for i in range(len(s)):
        # If this is not the last character and the current value is less than
        # the next value,
        # subtract the current value; otherwise, add the current value.

        if i + 1 < len(s) and roman_values[s[i]] < roman_values[s[i + 1]]:
            total -= roman_values[s[i]]
        else:
            total += roman_values[s[i]]
    return total

# Example usage:
print(romanToInt("MCMXCIV")) # Output: 1994
```

• LC.ca

```
class Solution:
    def romanToInt(self, s: str) -> int:
        d = {'I': 1, 'V': 5, 'X': 10, 'L': 50, 'C': 100, 'D': 500, 'M': 1000}
        return sum((-1 if d[a] < d[b] else 1) * d[a] for a, b in pairwise(s))
    + d[s[-1]]
```

◆ Quick Review

Key Insights

- Map each Roman numeral character to its integer value using a hash table or dictionary.
- Loop through the string and check if the current numeral is less than the next numeral;

Round 7 · Low · Easy

p.2881

if so, subtract its value.

- Otherwise, add its value to the total.
- This approach takes advantage of the subtractive notation used in Roman numerals.

Space & Time

Time Complexity: $O(n)$, where n is the length of the input string.

Space Complexity: $O(1)$, since the mapping and additional variables use constant space.

Solution

We use a dictionary (or hash map) to store the values of each Roman numeral. As we iterate over the string, we compare the current numeral's value with the next numeral's value. When a numeral of smaller value appears before a numeral of larger value, we subtract the current numeral's value; otherwise, we add it. This technique efficiently handles both the standard and subtractive cases in one pass through the string.

Round 7 · Low · Easy

p.2882

Maths / Geometry

#66 Plus One

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Math
 Lists: NeetCode 150

Problem

You are given a large integer represented as an integer array `digits`, where each `digits[i]` is the *i*th digit of the integer. The digits are ordered from most significant to least significant in left-to-right order. The large integer does not contain any leading 0's.

Increment the large integer by one and return the resulting array of digits.

Example 1:

Input: `digits = [1,2,3]`
 Output: `[1,2,4]`
 Explanation: The array represents the integer 123.
 Incrementing by one gives $123 + 1 = 124$.
 Thus, the result should be `[1,2,4]`.

Example 2:

Round 7 · Low · Easy

p.2883

Input: `digits = [4,3,2,1]`
 Output: `[4,3,2,2]`
 Explanation: The array represents the integer 4321.
 Incrementing by one gives $4321 + 1 = 4322$.
 Thus, the result should be `[4,3,2,2]`.

Example 3:

Input: `digits = [9]`
 Output: `[1,0]`
 Explanation: The array represents the integer 9.
 Incrementing by one gives $9 + 1 = 10$.
 Thus, the result should be `[1,0]`.

Constraints:

- $1 \leq \text{digits.length} \leq 100$
- $0 \leq \text{digits}[i] \leq 9$
- `digits` does not contain any leading 0's.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def plusOne(self, digits: list[int]) -> list[int]:
        for i, d in reversed(list(enumerate(digits))):
            if d < 9:
                digits[i] += 1
                return digits
            digits[i] = 0
        return [1] + digits
```

• LeetDoocs

Round 7 · Low · Easy

p.2884

```
class Solution:
    def plusOne(self, digits: List[int]) -> List[int]:
        n = len(digits)
        for i in range(n - 1, -1, -1):
            digits[i] += 1
            digits[i] %= 10
            if digits[i] != 0:
                return digits
        return [1] + digits
```

• SimplyLeet

```
# Function to add one to the array of digits
def plusOne(digits):
    # Process digits starting from the least significant digit
    for i in range(len(digits) - 1, -1, -1):
        # Add one to the current digit
        digits[i] += 1
        # Check if digit is less than 10, no carry needed
        if digits[i] < 10:
            return digits # No further carry, return the result
        # Set current digit to 0 because of carry over

        digits[i] = 0
    # If carry remains after processing all digits, add a new digit at start
    return [1] + digits

# Example usage
# print(plusOne([1, 2, 3])) # Output: [1, 2, 4]
```

Maths / Geometry

□ #168 Excel Sheet Column Title

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · String
Lists: AlgoMaster 600

Problem

Given an integer `columnNumber`, return its corresponding column title as it appears in an Excel sheet.

For example:

```
A -> 1
B -> 2
C -> 3
...
Z -> 26
AA -> 27
AB -> 28
...
```

Example 1:

```
Input: columnNumber = 1
Output: "A"
```

Example 2:

```
Input: columnNumber = 28
Output: "AB"
```

Example 3:

Input: columnNumber = 701
Output: "ZY"

Constraints:

- $1 \leq \text{columnNumber} \leq 231 - 1$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def convertToTitle(self, n: int) -> str:
        return (self.convertToTitle((n - 1) // 26) + chr(ord('A') + (n - 1) % 26)
                if n
                else '')
```

• LeetDoocs

```
class Solution:
    def convertToTitle(self, columnNumber: int) -> str:
        res = []
        while columnNumber:
            columnNumber -= 1
            res.append(chr(ord('A') + columnNumber % 26))
            columnNumber //= 26
        return ''.join(res[::-1])
```

• SimplyLeet

```
# Python solution for converting a column number to Excel sheet column title
def convertToTitle(columnNumber: int) -> str:
    result = [] # list to collect column letters
    while columnNumber > 0:
        columnNumber -= 1 # adjust to 0-indexed
        remainder = columnNumber % 26 # determine current letter index
        # Convert remainder to corresponding uppercase letter
        letter = chr(ord('A') + remainder)
        result.append(letter) # add the letter to the result list
        columnNumber //= 26 # update columnNumber for next iteration
```

```
# Since letters are computed in reverse, reverse them before joining
return ''.join(reversed(result))
```

```
# Example usage:
print(convertToTitle(1)) # Output: "A"
print(convertToTitle(28)) # Output: "AB"
print(convertToTitle(701)) # Output: "ZY"
```


Maths / Geometry

#263 Ugly Number

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math

Lists: AlgoMaster 600

Problem

An ugly number is a positive integer which does not have a prime factor other than 2, 3, and 5.

Given an integer n , return true if n is an ugly number.

Example 1:

Input: $n = 6$
Output: true
Explanation: $6 = 2 \times 3$

Example 2:

Input: $n = 1$
Output: true
Explanation: 1 has no prime factors.

Example 3:

Input: $n = 14$
Output: false
Explanation: 14 is not ugly since it includes the prime factor 7.

Constraints:

- $-231 \leq n \leq 231 - 1$

Round 7 · Low · Easy

p.2889

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isUgly(self, n: int) -> bool:
        if n == 0:
            return False

        for prime in 2, 3, 5:
            while n % prime == 0:
                n //= prime

        return n == 1
```

• LeetDoocs

```
class Solution:
    def isUgly(self, n: int) -> bool:
        if n < 1:
            return False
        for x in [2, 3, 5]:
            while n % x == 0:
                n //= x
        return n == 1
```

• SimplyLeet

```
# Function to check if a number is ugly.
def is_ugly(n):
    # Only positive numbers can be ugly.
    if n <= 0:
        return False
    # Divide out factor 2.
    while n % 2 == 0:
        n //= 2
    # Divide out factor 3.
    while n % 3 == 0:
```

Round 7 · Low · Easy

p.2890

```

    n //= 3
    # Divide out factor 5.
    while n % 5 == 0:
        n //= 5
    # If n is reduced to 1, then it's an ugly number.
    return n == 1

# Example usage.
print(is_ugly(6)) # Output: True
print(is_ugly(14)) # Output: False

```

Maths / Geometry

☐ #1071 Greatest Common Divisor of Strings

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · String
Lists: AlgoMaster 600

Problem

For two strings s and t , we say " t divides s " if and only if $s = t + t + t + \dots + t + t$ (i.e., t is concatenated with itself one or more times). Given two strings $str1$ and $str2$, return the largest string x such that x divides both $str1$ and $str2$.

Example 1:

Input: $str1 = \text{"ABCABC"}$, $str2 = \text{"ABC"}$

Output: "ABC"

Example 2:

Input: $str1 = \text{"ABABAB"}$, $str2 = \text{"ABAB"}$

Output: "AB"

Example 3:

Input: $str1 = \text{"LEET"}$, $str2 = \text{"CODE"}$

Output: ""

Example 4:

Input: str1 = "AAAAAB", str2 = "AAA"

Output: ""

Constraints:

- $1 \leq \text{str1.length}, \text{str2.length} \leq 1000$
- str1 and str2 consist of English uppercase letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def gcdOfStrings(self, str1: str, str2: str) -> str:
        for sz in range(min(len(str1), len(str2)), 0, -1):
            if self._isDivisible(str1, str2, sz):
                return str1[:sz]
        return ''

    def _isDivisible(self, str1: str, str2: str, sz: int) -> bool:
        """Returns True if str1 and str2 are divisible by str1[0..sz)."""
        if len(str1) % sz > 0 or len(str2) % sz > 0:
            return False
        gcd = str1[:sz]
        return str1.replace(gcd, '') == '' and str2.replace(gcd, '') == ''

class Solution:
    def gcdOfStrings(self, str1: str, str2: str) -> str:
        if str1 + str2 != str2 + str1:
            return ''
        g = math.gcd(len(str1), len(str2))
        return str1[:g]
```

Round 7 · Low · Easy

p.2893

• LeetCode

```
class Solution:
    def gcdOfStrings(self, str1: str, str2: str) -> str:
        def check(a, b):
            c = ''
            while len(c) < len(b):
                c += a
            return c == b

        for i in range(min(len(str1), len(str2)), 0, -1):
            t = str1[:i]

            if check(t, str1) and check(t, str2):
                return t
        return ''
```

```
class Solution:
    def gcdOfStrings(self, str1: str, str2: str) -> str:
        if str1 + str2 != str2 + str1:
            return ''
        n = gcd(len(str1), len(str2))
        return str1[:n]
```

• SimplyLeet

```
def gcd(a, b):
    # Compute the greatest common divisor using the Euclidean algorithm
    while b:
        a, b = b, a % b
    return a

def gcdOfStrings(str1, str2):
    # Check if concatenating the strings in different orders gives the same result
    if str1 + str2 != str2 + str1:
        return ''
```

Round 7 · Low · Easy

p.2894

```
# Compute the gcd of the lengths of str1 and str2
gcd_length = gcd(len(str1), len(str2))
# The substring of str1 with length equal to gcd_length is the greatest
common divisor string
return str1[:gcd_length]

# Example usage:
print(gcdOfStrings("ABCABC", "ABC")) # Output: "ABC"
```

Round 7 · Low · Easy

p.2895

String Matching

Round 7 · Low · Easy · 1 question

Round 7 · Low · Easy

p.2896

String Matching

#459 Repeated Substring Pattern

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · String Matching
Lists: AlgoMaster 600

Problem

Given a string s , check if it can be constructed by taking a substring of it and appending multiple copies of the substring together.

Example 1:

Input: $s = \text{"abab"}$
Output: true
Explanation: It is the substring "ab" twice.

Example 2:

Input: $s = \text{"aba"}$
Output: false

Example 3:

Input: $s = \text{"abcabcabcabc"}$
Output: true
Explanation: It is the substring "abc" four times or the substring "abcabc" twice.

Constraints:

- $1 \leq s.length \leq 104$
- s consists of lowercase English letters.

Round 7 · Low · Easy

p.2897

Community Solutions (Python)

• WalkCC

```
class Solution:
    def repeatedSubstringPattern(self, s: str) -> bool:
        return s in (s + s)[1:-1]
```

• LeetDoocs

```
class Solution:
    def repeatedSubstringPattern(self, s: str) -> bool:
        return (s + s).index(s, 1) < len(s)
```

• SimplyLeet

```
# Function to check if the string can be constructed by repeated substring
pattern
def repeatedSubstringPattern(s: str) -> bool:
    # Create a new string by concatenating s with itself and removing first and
    last characters
    doubled_s = (s + s)[1:-1]
    # Check if the original string s is present in the modified string
    return s in doubled_s

# Example usage:
if __name__ == "__main__":
    test_string = "abcabc"

    print(repeatedSubstringPattern(test_string)) # Expected output: True
```

Round 7 · Low · Easy

p.2898

Round 8 · Low · Medium**Round 8****Low Priority · Medium****65 questions · 15 patterns****Bucket Sort #164 #451 #692****1-D DP #198 #213 #343 #646 #740 #1262****0/1 Knapsack #416 #474 #494 #1049****Unbounded Knapsack #279 #322 #518 #983****Longest Increasing Subsequence (LIS) #300****#673 #1027 #1048****2D Grid DP #62 #63 #64 #120 #221 #256 #931****#1277 #1937****String DP #72 #91 #97 #139 #516 #1143 #1653****Tree / Graph DP #95 #337 #1976****Bitmask DP #698 #1066 #1986 #2305****Digit DP #357****Probability DP #688 #808 #837****State Machine DP #309 #714****Maths / Geometry #7 #43 #172 #204 #264 #287****#593 #611 #939 #963 #1922****String Matching #686 #1698****Binary Indexed Tree / Segment Tree #307 #308****Round 7 · Low · Easy****p.2899****Round 8 · Low · Medium****p.2900**

Bucket Sort

Round 8 · Low · Medium · 3 questions

Round 8 · Low · Medium

p.2901

Bucket Sort

□ #164 Maximum Gap

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Sorting · Bucket Sort · Radix Sort
Lists: AlgoMaster 600

Problem

Given an integer array `nums`, return the maximum difference between two successive elements in its sorted form. If the array contains less than two elements, return 0.

You must write an algorithm that runs in linear time and uses linear extra space.

Example 1:

Input: `nums = [3,6,9,1]`
Output: 3
Explanation: The sorted form of the array is `[1,3,6,9]`, either `(3,6)` or `(6,9)` has the maximum difference 3.

Example 2:

Input: `nums = [10]`
Output: 0
Explanation: The array contains less than 2 elements, therefore return 0.

Constraints:

- $1 \leq \text{nums.length} \leq 105$

Round 8 · Low · Medium

p.2902

- $0 \leq \text{nums}[i] \leq 109$

Community Solutions (Python)

• WalkCC

```
class Bucket:
    def __init__(self, mn: int, mx: int):
        self.mn = mn
        self.mx = mx

class Solution:
    def maximumGap(self, nums: list[int]) -> int:
        if len(nums) < 2:
            return 0

        mn = min(nums)
        mx = max(nums)
        if mn == mx:
            return 0

        gap = math.ceil((mx - mn) / (len(nums) - 1))
        bucketSize = (mx - mn) // gap + 1
        buckets = [Bucket(math.inf, -math.inf) for _ in range(bucketSize)]

        for num in nums:
            i = (num - mn) // gap
            buckets[i].mn = min(buckets[i].mn, num)
            buckets[i].mx = max(buckets[i].mx, num)

        ans = 0
        prevMax = mn

        for bucket in buckets:
            if bucket.mn == math.inf:
```

```
            continue # empty bucket
            ans = max(ans, bucket.mn - prevMax)
            prevMax = bucket.mx

        return ans
```

• LeetCode

```
class Solution:
    def maximumGap(self, nums: List[int]) -> int:
        n = len(nums)
        if n < 2:
            return 0
        mi, mx = min(nums), max(nums)
        bucket_size = max(1, (mx - mi) // (n - 1))
        bucket_count = (mx - mi) // bucket_size + 1
        buckets = [[inf, -inf] for _ in range(bucket_count)]
        for v in nums:
            i = (v - mi) // bucket_size
            buckets[i][0] = min(buckets[i][0], v)
            buckets[i][1] = max(buckets[i][1], v)
        ans = 0
        prev = inf
        for curmin, curmax in buckets:
            if curmin > curmax:
                continue
            ans = max(ans, curmin - prev)
            prev = curmax

        return ans
```

• SimplyLeet


```
# Python solution for Maximum Gap
def maximum_gap(nums):
    # If there are fewer than 2 elements, return 0
    if len(nums) < 2:
        return 0

    # Find the minimum and maximum values in the array
    min_val, max_val = min(nums), max(nums)
    if min_val == max_val:
        return 0 # All numbers are the same

    # Calculate the bucket size using a ceiling division method, ensuring at
    # least 1
    n = len(nums)
    bucket_size = max(1, (max_val - min_val) // (n - 1))
    bucket_count = (max_val - min_val) // bucket_size + 1

    # Initialize buckets where each bucket holds [min_value, max_value]
    buckets = [[None, None] for _ in range(bucket_count)]

    # Distribute the numbers into the buckets

    for num in nums:
        # Identify the index for the bucket this number belongs to
        idx = (num - min_val) // bucket_size
        # Update bucket's min and max values
        if buckets[idx][0] is None:
            buckets[idx][0] = num
            buckets[idx][1] = num
        else:
            buckets[idx][0] = min(buckets[idx][0], num)
            buckets[idx][1] = max(buckets[idx][1], num)
```

Round 8 · Low · Medium

p.2905

```
# Compute the maximum gap from the buckets.
max_gap = 0
previous_max = None

for b in buckets:
    # Ignore empty buckets
    if b[0] is None:
        continue
    if previous_max is not None:
        max_gap = max(max_gap, b[0] - previous_max)
        previous_max = b[1]

    return max_gap

# Example usage:
print(maximum_gap([3, 6, 9, 11])) # Output: 3
print(maximum_gap([10]))         # Output: 0
```

Round 8 · Low · Medium

p.2906

Bucket Sort

☐ #451 Sort Characters By Frequency

MED

LeetCode · SimplyLeet · WalkCC · LeetCode Hash Table · String · Sorting · Heap (Priority Queue) · Bucket Sort · Counting Lists: AlgoMaster 600

Problem

Given a string *s*, sort it in decreasing order based on the frequency of the characters. The frequency of a character is the number of times it appears in the string.

Return the sorted string. If there are multiple answers, return any of them.

Example 1:

Input: *s* = "tree"
Output: "eert"
Explanation: 'e' appears twice while 'r' and 't' both appear once. So 'e' must appear before both 'r' and 't'. Therefore "eetr" is also a valid answer.

Example 2:

Input: *s* = "cccaaa"
Output: "aaaccc"
Explanation: Both 'c' and 'a' appear three times, so both "cccaaa" and "aaaccc" are valid answers. Note that "cacaca" is incorrect, as the same characters must be together.

Round 8 · Low · Medium

p.2907

Example 3:

Input: *s* = "Aabb"
Output: "bbAa"
Explanation: "bbaA" is also a valid answer, but "Aabb" is incorrect. Note that 'A' and 'a' are treated as two different characters.

Constraints:

- $1 \leq s.length \leq 5 * 10^5$
- *s* consists of uppercase and lowercase English letters and digits.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def frequencySort(self, s: str) -> str:
        ans = []
        buckets = [[] for _ in range(len(s) + 1)]

        for c, freq in collections.Counter(s).items():
            buckets[freq].append(c)

        for freq in reversed(range(len(buckets))):
            for c in buckets[freq]:
                ans.append(c * freq)

        return ''.join(ans)
```

• LeetCode

```
class Solution:
    def frequencySort(self, s: str) -> str:
        cnt = Counter(s)
        return ''.join(c * v for c, v in sorted(cnt.items(), key=lambda x: -x[1]))
```

Round 8 · Low · Medium

p.2908

• SimplyLeet

```
# Count the frequency of each character using a dictionary
def frequencySort(s: str) -> str:
    char_frequency = {}
    # Build frequency dictionary
    for char in s:
        char_frequency[char] = char_frequency.get(char, 0) + 1

    # Sort characters based on frequency in descending order
    sorted_chars = sorted(char_frequency.items(), key=lambda item: item[1], reverse=True)

    # Form the result string by repeating characters
    result = []
    for char, freq in sorted_chars:
        result.append(char * freq)

    return "".join(result)

# Example usage:
print(frequencySort("tree")) # Output could be "eert" or "eetr"
```

Round 8 · Low · Medium

p.2909

Bucket Sort

□ #692 Top K Frequent Words

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Trie · Sorting · Heap
Lists: Grind 169

Problem

Given an array of strings `words` and an integer `k`, return the `k` most frequent strings.

Return the answer sorted by the frequency from highest to lowest. Sort the words with the same frequency by their lexicographical order.

Example 1:

Input: `words = ["i","love","leetcode","i","love","coding"], k = 2`
Output: `["i","love"]`
Explanation: "i" and "love" are the two most frequent words.
Note that "i" comes before "love" due to a lower alphabetical order.

Example 2:

Input: `words = ["the","day","is","sunny","the","the","the","sunny","is","is"], k = 4`
Output: `["the","is","sunny","day"]`
Explanation: "the", "is", "sunny" and "day" are the four most frequent words, with the number of occurrence being 4, 3, 2 and 1 respectively.

Constraints:

- $1 \leq \text{words.length} \leq 500$

Round 8 · Low · Medium

p.2910

- $1 \leq \text{words}[i].\text{length} \leq 10$
- $\text{words}[i]$ consists of lowercase English letters.
- k is in the range $[1, \text{The number of unique words}[i]]$

Follow-up: Could you solve it in $O(n \log(k))$ time and $O(n)$ extra space?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def topKFrequent(self, words: list[str], k: int) -> list[str]:
        ans = []
        bucket = [[] for _ in range(len(words) + 1)]

        for word, freq in collections.Counter(words).items():
            bucket[freq].append(word)

        for b in reversed(bucket):
            for word in sorted(b):

                ans.append(word)
                if len(ans) == k:
                    return ans

from dataclasses import dataclass

@dataclass(frozen=True)
class T:
    word: str
    freq: int

    def __lt__(self, other):
        if self.freq == other.freq:
```

Round 8 · Low · Medium

p.2911

```
        return self.word > other.word
        return self.freq < other.freq

class Solution:
    def topKFrequent(self, words: list[str], k: int) -> list[str]:
        ans = []
        heap = []

        for word, freq in collections.Counter(words).items():

            heapq.heappush(heap, T(word, freq))
            if len(heap) > k:
                heapq.heappop(heap)

        while heap:
            ans.append(heapq.heappop(heap).word)

        return ans[::-1]
```

• LeetCode

```
class Solution:
    def topKFrequent(self, words: List[str], k: int) -> List[str]:
        cnt = Counter(words)
        return sorted(cnt, key=lambda x: (-cnt[x], x))[:k]
```

• SimplyLeet

```
import heapq
from collections import Counter

def topKFrequent(words, k):
    # Count the frequency of each word
    count = Counter(words)

    # Initialize a min-heap; Python's heapq is a min-heap.
    # The comparator is (frequency, negative word) but use word in reverse
    order:
    # Since we want words with lower alphabetical order to be prioritized in
    the result,
```

Round 8 · Low · Medium

p.2912

```
# and in the min heap if frequencies equal, we want the one with
lexicographically larger word to be popped.
heap = []

for word, freq in count.items():
    # Push each pair into the heap. The tuple ordering:
    # First element is frequency and second element is the word (with
    reverse lex order trick)
    heapq.heappush(heap, (freq, -ord(word[0]), word))
    # However, using -ord(word[0]) is not robust for multi-letter words.
    # Instead, we can use tuple (freq, word) with a custom comparator by
    ensuring we invert the word order by using word in reverse.
    # For simplicity, we push tuple (freq, word) and then pop if heap size
    > k with a custom condition.
```

```
# Reset heap and use a custom comparator implemented by the tuple (-freq,
word) to simulate max heap behavior.
heap = []
for word, freq in count.items():
    # We push with frequency as negative so that larger frequencies come
    first;
    # use word as is to satisfy lexicographical order when frequencies are
    equal.
```

```
    heapq.heappush(heap, (-freq, word))
```

```
# Extract k elements from the heap and sort them accordingly.
result = []
```

```
for _ in range(k):
    freq, word = heapq.heappop(heap)
    result.append(word)
return result
```

```
# Example usage:
print(topKFrequent(["i","love","leetcode","i","love","coding"], 2)) # Output:
['i', 'love']
```



• LC.ca

Round 8 · Low · Medium

p.2913

```
'''
>>> sorted(student_objects, key=attrgetter('grade', 'age'))
[('john', 'A', 15), ('dave', 'B', 10), ('jane', 'B', 12)]

ref: https://docs.python.org/3/howto/sorting.html#operator-module-functions
'''

class Solution:
    def topKFrequent(self, words: List[str], k: int) -> List[str]:
        cnt = Counter(words)
        # multiple comparators "lambda x: (-cnt[x], x)"

        return sorted(cnt, key=lambda x: (-cnt[x], x))[:k]
        # why the returned sorted() not a tuple (word, count), but only word?
        # it's the property of Counter() class
        # so, also pass 0J: return sorted(cnt.keys(), key=lambda x: (-cnt[x],
        x))[:k]

'''
>>> words = ["i","love","leetcode","i","love","coding"]
>>> k = 2
>>> cnt = Counter(words)
>>> cnt

Counter({'i': 2, 'love': 2, 'leetcode': 1, 'coding': 1})

>>> sorted(cnt, key=lambda x: (-cnt[x], x))
['i', 'love', 'coding', 'leetcode']

# default, only for keys()
>>> sorted(cnt)
['coding', 'i', 'leetcode', 'love']
>>> sorted(cnt.keys())
['coding', 'i', 'leetcode', 'love']
```

Round 8 · Low · Medium

p.2914

```
>>> sorted(cnt.values())
[1, 1, 2, 2]
>>> sorted(cnt.items())
[('coding', 1), ('i', 2), ('leetcode', 1), ('love', 2)]
'''
```

```
#####
```

```
'''
```

```
>>> words = ["the","day","is","sunny","the","the","the","sunny","is","is"]
>>> count = collections.Counter(words)
>>> count
Counter({'the': 4, 'is': 3, 'sunny': 2, 'day': 1})
>>>
>>> count.items()
dict_items([('the', 4), ('day', 1), ('is', 3), ('sunny', 2)])
>>>
'''
```

```
'''
cmp doesn't exist in Python 3. If you really want it, you could define it
yourself:
```

```
def cmp(a, b):
    return (a > b) - (a < b)
```

```
'''
```

```
class Solution(object): #python-2
    def topKFrequent(self, words, k):
        '''
```

```
:type words: List[str]
:type k: int
:rtype: List[str]
'''

count = collections.Counter(words)

# https://docs.python.org/3/howto/sorting.html#comparison-functions
def compare(x, y):
    if x[1] == y[1]:
        return cmp(x[0], y[0])

    else:
        return -cmp(x[1], y[1]) # -cmp, so reversed order, bigger ones
at front
return [x[0] for x in sorted(count.items(), cmp = compare)[:k]]
```

◆ Quick Review

Key Insights

- Count the frequency of each unique word using a hash table or dictionary.
- Use a min-heap (priority queue) to keep track of the top k frequent words. In the heap, the ordering is based on frequency; if frequencies are equal, order by lexicographical order (inverted condition for min-heap).
- Alternatively, you can sort the unique words based on frequency and lexicographical order and then pick the top k.
- For an efficient solution, use a min-heap that maintains size k achieving $O(n \log k)$ time

complexity.

Space & Time

Time Complexity: $O(n \log k)$ when using a heap, where n is the number of unique words.

Space Complexity: $O(n)$ for storing the frequency counts and the heap.

Solution

We first calculate the frequency of each word using a hash table (or dictionary). Then, we use a min-heap that stores pairs of (frequency, word). The comparator for the heap is defined such that the root of the heap is the word with the smallest frequency; if two words have the same frequency, the word with the lexicographically larger value is considered “smaller” in our ordering so that it gets popped when the heap size exceeds k . After processing all words, the heap contains the top k frequent words. Finally, the heap elements are extracted and reversed (if needed) to produce the final list sorted from highest to lowest frequency with lexicographic tie-breaking.

Round 8 · Low · Medium

p.2917

1-D DP

Round 8 · Low · Medium · 6 questions

Round 8 · Low · Medium

p.2918

1-D DP

#198 House Robber

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Dynamic Programming
 Lists: Grind 169

Problem

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed, the only constraint stopping you from robbing each of them is that adjacent houses have security systems connected and it will automatically contact the police if two adjacent houses were broken into on the same night.

Given an integer array `nums` representing the amount of money of each house, return the maximum amount of money you can rob tonight without alerting the police.

Example 1:

Input: `nums = [1,2,3,1]`
 Output: 4
 Explanation: Rob house 1 (money = 1) and then rob house 3 (money = 3).
 Total amount you can rob = 1 + 3 = 4.

Round 8 · Low · Medium

p.2919

Example 2:

Input: `nums = [2,7,9,3,1]`
 Output: 12
 Explanation: Rob house 1 (money = 2), rob house 3 (money = 9) and rob house 5 (money = 1).
 Total amount you can rob = 2 + 9 + 1 = 12.

Constraints:

- $1 \leq \text{nums.length} \leq 100$
- $0 \leq \text{nums}[i] \leq 400$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def rob(self, nums: list[int]) -> int:
        if not nums:
            return 0
        if len(nums) == 1:
            return nums[0]

        # dp[i] := max money of robbing nums[0..i]
        dp = [0] * len(nums)
        dp[0] = nums[0]

        dp[1] = max(nums[0], nums[1])

        for i in range(2, len(nums)):
            dp[i] = max(dp[i - 1], dp[i - 2] + nums[i])

        return dp[-1]
```

Round 8 · Low · Medium

p.2920


```
class Solution:
    def rob(self, nums: List[int]) -> int:
        prev1 = 0 # dp[i - 1]
        prev2 = 0 # dp[i - 2]

        for num in nums:
            dp = max(prev1, prev2 + num)
            prev2 = prev1
            prev1 = dp
```

```
return prev1
```

• LeetDoocs

```
class Solution:
    def rob(self, nums: List[int]) -> int:
        @cache
        def dfs(i: int) -> int:
            if i >= len(nums):
                return 0
            return max(nums[i] + dfs(i + 2), dfs(i + 1))

        return dfs(0)
```

```
class Solution:
    def rob(self, nums: List[int]) -> int:
        n = len(nums)
        f = [0] * (n + 1)
        f[1] = nums[0]
        for i in range(2, n + 1):
            f[i] = max(f[i - 1], f[i - 2] + nums[i - 1])
        return f[n]
```

```
class Solution:
    def rob(self, nums: List[int]) -> int:
        f = g = 0
        for x in nums:
            f, g = max(f, g), f + x
        return max(f, g)
```

• SimplyLeet

Round 8 · Low · Medium

p.2921

```
# Definition of the Solution class
class Solution:
    def rob(self, nums: List[int]) -> int:
        # Edge case: if there are no houses, return 0
        if not nums:
            return 0

        # Initialize two variables to keep track of the max amounts
        prev1, prev2 = 0, 0
```

```
# Iterate over each house's money
for money in nums:
    # Calculate the maximum money that can be robbed if we consider the
    current house
    current = max(prev1, prev2 + money) # Either skip current house or
    rob it
    # Update variables: prev2 becomes previous best, and prev1 becomes
    current best
    prev2 = prev1
    prev1 = current

# Return the maximum money that can be robbed
return prev1
```

• LC.ca

```
# greedy
class Solution:
    def rob(self, nums: List[int]) -> int:
        not_rob, rob = 0, nums[0]
        for num in nums[1:]:
            # must max check
            # eg. first robbed 99999, then following ones are just 3,3,3,3,3
            not_rob, rob = rob, max(num + not_rob, rob)
        return rob
```

Round 8 · Low · Medium

p.2922

#####

```

class Solution(object):
    def rob(self, nums):
        """
        :type nums: List[int]
        :rtype: int
        """
        if len(nums) == 0:
            return 0

        if len(nums) <= 2:
            return max(nums)
        dp = [0 for i in range(0, 2)]
        dp[0] = nums[0]
        dp[1] = max(nums[1], nums[0])
        for i in range(2, len(nums)):
            dp[i % 2] = max(dp[(i - 1) % 2], dp[(i - 2) % 2] + nums[i])
        return dp[(len(nums) - 1) % 2]

```

◆ Quick Review

Key Insights

- The problem can be solved using Dynamic Programming.
- For each house, decide whether to rob it or not.
- Recurrence relation: $dp[i] = \max(dp[i-1], dp[i-2] + \text{currentHouseMoney})$
- Use iterative computation to keep track of the optimal solutions without needing extra space for the entire dp array.

Space & Time

Time Complexity: $O(n)$, where n is the number of houses.

Space Complexity: $O(1)$, using only two variables to track the previous decisions.

Solution

We use a dynamic programming approach by iterating through each house in the array. At each step, we consider two scenarios: either we rob the current house (and add its money to the total from two houses ago) or we skip it (keeping the optimal solution up to the previous house). By using two variables to store these intermediate results, we achieve an optimized space complexity of $O(1)$.

1-D DP

□ #213 House Robber II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming

Lists: NeetCode 150

Problem

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed. All houses at this place are arranged in a circle. That means the first house is the neighbor of the last one.

Meanwhile, adjacent houses have a security system connected, and it will automatically contact the police if two adjacent houses were broken into on the same night.

Given an integer array `nums` representing the amount of money of each house, return the maximum amount of money you can rob tonight without alerting the police.

Example 1:

Round 8 · Low · Medium

p.2925

Input: `nums = [2,3,2]`

Output: 3

Explanation: You cannot rob house 1 (money = 2) and then rob house 3 (money = 2), because they are adjacent houses.

Example 2:

Input: `nums = [1,2,3,1]`

Output: 4

Explanation: Rob house 1 (money = 1) and then rob house 3 (money = 3).
Total amount you can rob = 1 + 3 = 4.

Example 3:

Input: `nums = [1,2,3]`

Output: 3

Constraints:

- $1 \leq \text{nums.length} \leq 100$
- $0 \leq \text{nums}[i] \leq 1000$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def rob(self, nums: list[int]) -> int:
        if not nums:
            return 0
        if len(nums) < 2:
            return nums[0]

        def rob(l: int, r: int) -> int:
            dp1 = 0
            dp2 = 0
```

Round 8 · Low · Medium

p.2926

```

for i in range(l, r + 1):
    temp = dp1
    dp1 = max(dp1, dp2 + nums[i])
    dp2 = temp

return dp1

return max(rob(0, len(nums) - 2),
           rob(1, len(nums) - 1))

```

• LeetCode

```

class Solution:
    def rob(self, nums: List[int]) -> int:
        def _rob(nums):
            f = g = 0
            for x in nums:
                f, g = max(f, g), f + x
            return max(f, g)

        if len(nums) == 1:
            return nums[0]

        return max(_rob(nums[1:]), _rob(nums[:-1]))

```

• SimplyLeet

```

# Python solution for House Robber II

def rob(nums):
    # Helper function to handle the linear House Robber problem.
    def rob_linear(houses):
        prev_max, curr_max = 0, 0
        for money in houses:
            temp = curr_max
            curr_max = max(curr_max, prev_max + money)
            prev_max = temp

```

```

return curr_max

n = len(nums)
# When there's only one house, just return its value.
if n == 1:
    return nums[0]

# Rob excluding the first house and excluding the last house.
return max(rob_linear(nums[1:]), rob_linear(nums[:-1]))

# Test example
print(rob([2, 3, 2])) # Expected output: 3

```

1-D DP

#343 Integer Break

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · Dynamic Programming
Lists: AlgoMaster 600

Problem

Given an integer n , break it into the sum of k positive integers, where $k \geq 2$, and maximize the product of those integers.

Return the maximum product you can get.

Example 1:

Input: $n = 2$
Output: 1
Explanation: $2 = 1 + 1$, $1 \times 1 = 1$.

Example 2:

Input: $n = 10$
Output: 36
Explanation: $10 = 3 + 3 + 4$, $3 \times 3 \times 4 = 36$.

Constraints:

- $2 \leq n \leq 58$

Community Solutions (Python)

- WalkCC

Round 8 · Low · Medium

p.2929

```
class Solution:
    def integerBreak(self, n: int) -> int:
        # If an optimal product contains a factor f >= 4, then we can replace it
        # with 2 and f - 2 without losing optimality. As 2(f - 2) = 2f - 4 >= f,
        # we never need a factor >= 4, meaning we only need factors 1, 2, and 3
        # (and 1 is wasteful).
        # Also, 3 * 3 is better than 2 * 2 * 2, so we never use 2 more than twice.
        if n == 2: # 1 * 1
            return 1
        if n == 3: # 1 * 2

            return 2

        ans = 1

        while n > 4:
            n -= 3
            ans *= 3
            ans *= n

        return ans
```

• LeetDoocs

```
class Solution:
    def integerBreak(self, n: int) -> int:
        f = [1] * (n + 1)
        for i in range(2, n + 1):
            for j in range(1, i):
                f[i] = max(f[i], f[i - j] * j, (i - j) * j)
        return f[n]

class Solution:
    def integerBreak(self, n: int) -> int:
        if n < 4:
            return n - 1
        if n % 3 == 0:
            return pow(3, n // 3)
        if n % 3 == 1:
            return pow(3, n // 3 - 1) * 4
        return pow(3, n // 3) * 2
```

Round 8 · Low · Medium

p.2930

• SimplyLeet

```
def integerBreak(n: int) -> int:
    # dp[i] holds the maximum product obtainable for integer i
    dp = [0] * (n + 1)
    # Base cases: for n=2 and n=3, the maximum product is defined by their
    # splits (1*1 and 1*2).
    dp[1] = 1
    dp[2] = 1 # 2 = 1+1, product = 1 (special handling)

    for i in range(3, n + 1):
        # Compute dp[i] by trying all splits j and i-j
        for j in range(1, i):

            # Option 1: Do not split further the part (i - j)
            product1 = j * (i - j)
            # Option 2: Use optimum splitting for the part (i - j)
            product2 = j * dp[i - j]
            dp[i] = max(dp[i], product1, product2)

    return dp[n]

# Example usage:
print(integerBreak(10)) # Expected output: 36
```

1-D DP

□ #646 Maximum Length of Pair Chain

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Greedy ·
Sorting

Lists: AlgoMaster 600

Problem

You are given an array of n pairs pairs where $\text{pairs}[i] = [\text{left}_i, \text{right}_i]$ and $\text{left}_i < \text{right}_i$.

A pair $p_2 = [c, d]$ follows a pair $p_1 = [a, b]$ if $b < c$. A chain of pairs can be formed in this fashion. Return the length longest chain which can be formed.

You do not need to use up all the given intervals. You can select pairs in any order.

Example 1:

Input: $\text{pairs} = [[1,2],[2,3],[3,4]]$
Output: 2
Explanation: The longest chain is $[1,2] \rightarrow [3,4]$.

Example 2:

Input: $\text{pairs} = [[1,2],[7,8],[4,5]]$
Output: 3
Explanation: The longest chain is $[1,2] \rightarrow [4,5] \rightarrow [7,8]$.

Constraints:

- $n == \text{pairs.length}$
- $1 \leq n \leq 1000$
- $-1000 \leq \text{lefti} < \text{righti} \leq 1000$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findLongestChain(self, pairs: list[list[int]]) -> int:
        ans = 0
        prevEnd = -math.inf

        for s, e in sorted(pairs, key=lambda x: x[1]):
            if s > prevEnd:
                ans += 1
                prevEnd = e

        return ans
```

• LeetCode

```
class Solution:
    def findLongestChain(self, pairs: List[List[int]]) -> int:
        pairs.sort(key=lambda x: x[1])
        ans, pre = 0, -inf
        for a, b in pairs:
            if pre < a:
                ans += 1
                pre = b
        return ans
```

• SimplyLeet

Round 8 · Low · Medium

p.2933

```
# Function to compute the maximum length of a pair chain
def findLongestChain(pairs):
    # Sort pairs based on their second element (end value)
    pairs.sort(key=lambda x: x[1])

    # Initialize the number of pairs in the longest chain and the current end
    # marker
    chain_length = 0
    current_end = float('-inf')

    # Iterate over each pair in the sorted array

    for pair in pairs:
        # If the current pair's start is greater than the current end, it can
        # be added to the chain
        if pair[0] > current_end:
            chain_length += 1
            current_end = pair[1] # Update the current end to the end of the
            # current pair

    return chain_length

# Example usage:
pairs = [[1,2], [7,8], [4,5]]

print(findLongestChain(pairs)) # Expected Output: 3
```

Round 8 · Low · Medium

p.2934

1-D DP

#740 Delete and Earn

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table · Dynamic Programming
 Lists: AlgoMaster 600

Problem

You are given an integer array `nums`. You want to maximize the number of points you get by performing the following operation any number of times:

- Pick any `nums[i]` and delete it to earn `nums[i]` points. Afterwards, you must delete every element equal to `nums[i] - 1` and every element equal to `nums[i] + 1`.

Return the maximum number of points you can earn by applying the above operation some number of times.

Example 1:

Input: `nums = [3,4,2]`
 Output: 6
 Explanation: You can perform the following operations:
 - Delete 4 to earn 4 points. Consequently, 3 is also deleted. `nums = [2]`.
 - Delete 2 to earn 2 points. `nums = []`.
 You earn a total of 6 points.

Round 8 · Low · Medium

p.2935

Example 2:

Input: `nums = [2,2,3,3,3,4]`
 Output: 9
 Explanation: You can perform the following operations:
 - Delete a 3 to earn 3 points. All 2's and 4's are also deleted. `nums = [3,3]`.
 - Delete a 3 again to earn 3 points. `nums = [3]`.
 - Delete a 3 once more to earn 3 points. `nums = []`.
 You earn a total of 9 points.

Constraints:

- $1 \leq \text{nums.length} \leq 2 * 10^4$
- $1 \leq \text{nums}[i] \leq 10^4$

Community Solutions (Python)

• LeetDoocs

```
class Solution:
    def deleteAndEarn(self, nums: List[int]) -> int:
        mx = -inf
        for num in nums:
            mx = max(mx, num)
        total = [0] * (mx + 1)
        for num in nums:
            total[num] += num
        first = total[0]
        second = max(total[0], total[1])

        for i in range(2, mx + 1):
            cur = max(first + total[i], second)
            first = second
            second = cur
        return second
```

• SimplyLeet

Round 8 · Low · Medium

p.2936

Python solution for Delete and Earn

```
def deleteAndEarn(nums):
    # Step 1: Build frequency dictionary
    points = {}
    for num in nums:
        if num in points:
            points[num] += num
        else:
            points[num] = num

    # Step 2: Find the maximum number in nums to define the dp array size
    max_num = max(nums)

    # Step 3: Initialize dp array where dp[i] represents maximum points up to i
    dp = [0] * (max_num + 1)
    dp[0] = 0
    dp[1] = points.get(1, 0)

    # Step 4: Fill dp using recurrence relation

    for i in range(2, max_num + 1):
        dp[i] = max(dp[i-1], dp[i-2] + points.get(i, 0))

    # Final answer is maximum points at the maximum number
    return dp[max_num]

# Example usage:
print(deleteAndEarn([3,4,2])) # Output: 6
```

Round 8 · Low · Medium

p.2937

1-D DP

□ #1262 Greatest Sum Divisible by Three

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Greedy ·
Sorting

Lists: AlgoMaster 600

Problem

Given an integer array `nums`, return the maximum possible sum of elements of the array such that it is divisible by three.

Example 1:

Input: `nums = [3,6,5,1,8]`
Output: 18
Explanation: Pick numbers 3, 6, 1 and 8 their sum is 18 (maximum sum divisible by 3).

Example 2:

Input: `nums = [4]`
Output: 0
Explanation: Since 4 is not divisible by 3, do not pick any number.

Example 3:

Input: `nums = [1,2,3,4,4]`
Output: 12
Explanation: Pick numbers 1, 3, 4 and 4 their sum is 12 (maximum sum divisible by 3).

Constraints:

Round 8 · Low · Medium

p.2938

- $1 \leq \text{nums.length} \leq 4 * 10^4$
- $1 \leq \text{nums}[i] \leq 10^4$

Community Solutions (Python)

• LeetCode

```
class Solution:
    def maxSumDivThree(self, nums: List[int]) -> int:
        n = len(nums)
        f = [[-inf] * 3 for _ in range(n + 1)]
        f[0][0] = 0
        for i, x in enumerate(nums, 1):
            for j in range(3):
                f[i][j] = max(f[i - 1][j], f[i - 1][(j - x) % 3] + x)
        return f[n][0]
```

```
class Solution:
    def maxSumDivThree(self, nums: List[int]) -> int:
        f = [0, -inf, -inf]
        for x in nums:
            g = f[:]
            for j in range(3):
                g[j] = max(f[j], f[(j - x) % 3] + x)
            f = g
        return f[0]
```

• SimplyLeet

```
# Python Solution
def maxSumDivThree(nums):
    # Initialize dp array where:
    # dp[0]: maximum sum with remainder 0 (divisible by three)
    # dp[1]: maximum sum with remainder 1
    # dp[2]: maximum sum with remainder 2
    dp = [0, float('-inf'), float('-inf')]

    # Process each number in the list
    for num in nums:
```

```
# Create a copy of current dp to update new states
current = dp.copy()
# Iterate through each remainder state in the current dp
for i in range(3):
    # Calculate new remainder when adding the current number
    new_remainder = (i + num) % 3
    # Update dp for the new remainder with the maximum possible sum
    dp[new_remainder] = max(dp[new_remainder], current[i] + num)
# dp[0] holds the maximum sum divisible by three
return dp[0]
```

```
# Example usage:
# print(maxSumDivThree([3,6,5,1,8])) # Expected output: 18
```

0/1 Knapsack

Round 8 · Low · Medium · 4 questions

Round 8 · Low · Medium

p.2941

0/1 Knapsack

□ #416 Partition Equal Subset Sum

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming

Lists: Grind 169

Problem

Given an integer array `nums`, return `true` if you can partition the array into two subsets such that the sum of the elements in both subsets is equal or `false` otherwise.

Example 1:

Input: `nums = [1,5,11,5]`
Output: `true`
Explanation: The array can be partitioned as `[1, 5, 5]` and `[11]`.

Example 2:

Input: `nums = [1,2,3,5]`
Output: `false`
Explanation: The array cannot be partitioned into equal sum subsets.

Constraints:

- `1 <= nums.length <= 200`
- `1 <= nums[i] <= 100`

Community Solutions (Python)

Round 8 · Low · Medium

p.2942

• WalkCC

```
class Solution:
    def canPartition(self, nums: list[int]) -> bool:
        summ = sum(nums)
        if summ % 2 == 1:
            return False
        return self.knapsack_(nums, summ // 2)

    def knapsack_(self, nums: list[int], subsetSum: int) -> bool:
        n = len(nums)
        # dp[i][j] := True if j can be formed by nums[0..i]

        dp = [[False] * (subsetSum + 1) for _ in range(n + 1)]
        dp[0][0] = True

        for i in range(1, n + 1):
            num = nums[i - 1]
            for j in range(subsetSum + 1):
                if j < num:
                    dp[i][j] = dp[i - 1][j]
                else:
                    dp[i][j] = dp[i - 1][j] or dp[i - 1][j - num]

        return dp[n][subsetSum]
```

• LeetCode

```
class Solution:
    def canPartition(self, nums: List[int]) -> bool:
        m, mod = divmod(sum(nums), 2)
        if mod:
            return False
        n = len(nums)
        f = [[False] * (m + 1) for _ in range(n + 1)]
        f[0][0] = True
        for i, x in enumerate(nums, 1):
            for j in range(m + 1):
                f[i][j] = f[i - 1][j] or (j >= x and f[i - 1][j - x])
        return f[n][m]
```

Round 8 · Low · Medium

p.2943

• SimplyLeet

```
# Python solution for Partition Equal Subset Sum

def canPartition(nums):
    # Calculate the total sum of the array
    total_sum = sum(nums)

    # If the total sum is odd, equal partition is not possible
    if total_sum % 2 != 0:
        return False

    # Set the target sum to half of the total sum
    target = total_sum // 2

    # Initialize dp array where dp[i] is True if a subset sum i is achievable
    dp = [False] * (target + 1)
    dp[0] = True # Base case: subset sum of 0 is always possible

    # Iterate through each number in the array
    for num in nums:
        # Iterate backwards to avoid overwriting dp values needed for current
        # iteration
        for i in range(target, num - 1, -1):
            # Update dp[i] if dp[i-num] is True
            if dp[i - num]:
                dp[i] = True

    # dp[target] is True if a subset adds up to target sum
    return dp[target]

# Example usage:
# print(canPartition([1, 5, 11, 5]))
```

• LC.ca

Round 8 · Low · Medium

p.2944

```
class Solution:
    def canPartition(self, nums: List[int]) -> bool:
        s = sum(nums)
        if s % 2 != 0:
            return False
        n = s >> 1
        dp = [False] * (n + 1)
        dp[0] = True
        for v in nums:
            for target in range(n, v - 1, -1): # including v itself
                dp[target] = dp[target] or dp[target - v]
        return dp[-1]
```

```
class Solution: # recursive, but over time limit in OJ
    def canPartition(self, nums: List[int]) -> bool:
        total_sum = sum(nums)
        if total_sum % 2 != 0:
            return False
        target_sum = total_sum // 2
```

```
memo = {} # ==> or just use @cache for dfs()

def dfs(idx, curr_sum):
    if curr_sum == target_sum:
        return True
    if curr_sum > target_sum or idx >= len(nums):
        return False
    if (idx, curr_sum) in memo:
        return memo[(idx, curr_sum)]
```

```
# Explore two options: include the current number or exclude it
include = dfs(idx + 1, curr_sum + nums[idx])
exclude = dfs(idx + 1, curr_sum)

memo[(idx, curr_sum)] = include or exclude
return memo[(idx, curr_sum)]
```

```
return dfs(0, 0)
```

```
#####
```

Round 8 · Low · Medium

p.2945

```
class Solution:
    def canPartition(self, nums: List[int]) -> bool:
        m, mod = divmod(sum(nums), 2)
        if mod:
            return False
        f = [True] + [False] * m
        for x in nums:
            for j in range(m, x - 1, -1):
                f[j] = f[j] or f[j - x]

        return f[m]
```

◆ Quick Review

Key Insights

- The total sum of the array must be even, otherwise, partitioning into two equal subsets is impossible.
- The problem can be transformed into a "subset sum" problem where we need to check if there exists a subset of numbers that sums to half of the total sum.
- A dynamic programming approach can efficiently decide if such a subset exists.

Space & Time

Time Complexity: $O(n * \text{target})$, where target is half of the total sum.

Round 8 · Low · Medium

p.2946

Space Complexity: $O(\text{target})$, using a one-dimensional DP array.

Solution

First, compute the total sum of the array. If the total is odd, return false since an odd number cannot be equally partitioned. Next, set the target sum to half of the total. Use a dynamic programming array (dp) where $dp[i]$ represents whether a subset with sum i is achievable. Initialize $dp[0]$ as true because a subset sum of 0 is always possible. Then, for each number in the array, iterate through the dp array backwards from target to the number's value and update $dp[i]$ to true if $dp[i - \text{num}]$ is true. Ultimately, if $dp[\text{target}]$ is true, the array can be partitioned into two subsets with equal sum.

Round 8 · Low · Medium

p.2947

0/1 Knapsack

□ #474 Ones and Zeroes

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · String · Dynamic Programming
Lists: AlgoMaster 600

Problem

You are given an array of binary strings `strs` and two integers `m` and `n`.

Return the size of the largest subset of `strs` such that there are at most `m` 0's and `n` 1's in the subset.

A set x is a subset of a set y if all elements of x are also elements of y .

Example 1:

```
Input: strs = ["10","0001","111001","1","0"], m = 5, n = 3
Output: 4
Explanation: The largest subset with at most 5 0's and 3 1's is {"10", "0001", "1", "0"}, so the answer is 4.
Other valid but smaller subsets include {"0001", "1"} and {"10", "1", "0"}. {"111001"} is an invalid subset because it contains 4 1's, greater than the maximum of 3.
```

Example 2:

```
Input: strs = ["10","0","1"], m = 1, n = 1
Output: 2
Explanation: The largest subset is {"0", "1"}, so the answer is 2.
```

Round 8 · Low · Medium

p.2948

Constraints:

- $1 \leq \text{strs.length} \leq 600$
- $1 \leq \text{strs}[i].\text{length} \leq 100$
- $\text{strs}[i]$ consists only of digits '0' and '1'.
- $1 \leq m, n \leq 100$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findMaxForm(self, strs: List[str], m: int, n: int) -> int:
        # dp[i][j] := the maximum size of the subset given i 0s and j 1s are
        # available
        dp = [[0] * (n + 1) for _ in range(m + 1)]

        for s in strs:
            zeros = s.count('0')
            ones = len(s) - zeros
            for i in range(m, zeros - 1, -1):
                for j in range(n, ones - 1, -1):
                    dp[i][j] = max(dp[i][j], dp[i - zeros][j - ones] + 1)

        return dp[m][n]
```

• LeetCode

```
class Solution:
    def findMaxForm(self, strs: List[str], m: int, n: int) -> int:
        sz = len(strs)
        f = [[[0] * (n + 1) for _ in range(m + 1)] for _ in range(sz + 1)]
        for i, s in enumerate(strs, 1):
            a, b = s.count("0"), s.count("1")
            for j in range(m + 1):
                for k in range(n + 1):
                    f[i][j][k] = f[i - 1][j][k]
                    if j >= a and k >= b:
```

```
f[i][j][k] = max(f[i][j][k], f[i - 1][j - a][k - b] + 1)

    return f[sz][m][n]

class Solution:
    def findMaxForm(self, strs: List[str], m: int, n: int) -> int:
        f = [[0] * (n + 1) for _ in range(m + 1)]
        for s in strs:
            a, b = s.count("0"), s.count("1")
            for i in range(m, a - 1, -1):
                for j in range(n, b - 1, -1):
                    f[i][j] = max(f[i][j], f[i - a][j - b] + 1)
        return f[m][n]
```

• SimplyLeet

```
# Python solution for Ones and Zeroes problem

def findMaxForm(strs, m, n):
    # Create a 2D DP array with dimensions (m+1) x (n+1)
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    # Process each binary string in the list
    for binary_str in strs:
        count0 = binary_str.count('0') # count zeros in the string
        count1 = binary_str.count('1') # count ones in the string

        # Traverse the dp array in reverse order to avoid using the same string
        twice
        for zeros in range(m, count0 - 1, -1):
            for ones in range(n, count1 - 1, -1):
                # Decision: take max between not choosing and choosing the
                current string
                dp[zeros][ones] = max(dp[zeros][ones], dp[zeros - count0][ones - count1] + 1)

        return dp[m][n]

# Example usage:
```

```
if __name__ == '__main__':  
    strs = ["10", "0001", "111001", "1", "0"]  
    m = 5  
    n = 3  
    print(findMaxForm(strs, m, n)) # Expected output: 4
```

0/1 Knapsack

□ #494 Target Sum

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Backtracking
Lists: NeetCode 150

Problem

You are given an integer array `nums` and an integer `target`.

You want to build an expression out of `nums` by adding one of the symbols '+' and '-' before each integer in `nums` and then concatenate all the integers.

- For example, if `nums = [2, 1]`, you can add a '+' before 2 and a '-' before 1 and concatenate them to build the expression "+2-1".

Return the number of different expressions that you can build, which evaluates to `target`.

Example 1:

Input: nums = [1,1,1,1,1], target = 3
 Output: 5
 Explanation: There are 5 ways to assign symbols to make the sum of nums be target 3.

```
-1 + 1 + 1 + 1 + 1 = 3
+1 - 1 + 1 + 1 + 1 = 3
+1 + 1 - 1 + 1 + 1 = 3
+1 + 1 + 1 - 1 + 1 = 3
+1 + 1 + 1 + 1 - 1 = 3
```

Example 2:

Input: nums = [1], target = 1
 Output: 1

Constraints:

- $1 \leq \text{nums.length} \leq 20$
- $0 \leq \text{nums}[i] \leq 1000$
- $0 \leq \text{sum}(\text{nums}[i]) \leq 1000$
- $-1000 \leq \text{target} \leq 1000$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findTargetSumWays(self, nums: list[int], target: int) -> int:
        summ = sum(nums)
        if summ < abs(target) or (summ + target) % 2 == 1:
            return 0

        def knapsack(target: int) -> int:
            # dp[i] := the number of ways to sum to i by nums so far
            dp = [1] + [0] * summ
```

```
for num in nums:
    for j in range(summ, num - 1, -1):
        dp[j] += dp[j - num]

return dp[target]

return knapsack((summ + target) // 2)
```

• LeetDoocs

```
class Solution:
    def findTargetSumWays(self, nums: List[int], target: int) -> int:
        s = sum(nums)
        if s < target or (s - target) % 2:
            return 0
        m, n = len(nums), (s - target) // 2
        f = [[0] * (n + 1) for _ in range(m + 1)]
        f[0][0] = 1
        for i, x in enumerate(nums, 1):
            for j in range(n + 1):
                f[i][j] = f[i - 1][j]
                if j >= x:
                    f[i][j] += f[i - 1][j - x]
        return f[m][n]
```

• SimplyLeet

```
# Define a function to compute the number of ways to reach the target sum.
def findTargetSumWays(nums, target):
    # Calculate the total sum of all numbers in the list.
    total_sum = sum(nums)
    # If the target is greater than total_sum, or (target + total_sum) is odd,
    no solution exists.
    if target > total_sum or (target + total_sum) % 2 == 1:
        return 0

    # Compute the required subset sum.
    subset_sum = (target + total_sum) // 2
```

```
# Initialize the dp array with zeros, with dp[0] set to 1.
dp = [0] * (subset_sum + 1)
dp[0] = 1

# Update the dp array for each number in nums.
for num in nums:
    # Iterate backwards to ensure each number is only counted once.
    for s in range(subset_sum, num - 1, -1):
        dp[s] += dp[s - num]
return dp[subset_sum]

# Example usage:
print(findTargetSumWays([1,1,1,1,1], 3)) # Expected output: 5
```

0/1 Knapsack

□ #1049 Last Stone Weight II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Dynamic Programming
 Lists: AlgoMaster 600

Problem

You are given an array of integers stones where stones[i] is the weight of the ith stone.

We are playing a game with the stones. On each turn, we choose any two stones and smash them together. Suppose the stones have weights x and y with $x \leq y$. The result of this smash is:

- If $x == y$, both stones are destroyed, and
- If $x \neq y$, the stone of weight x is destroyed, and the stone of weight y has new weight $y - x$.

At the end of the game, there is at most one stone left.

Return the smallest possible weight of the left stone. If there are no stones left, return 0.

Example 1:

Input: stones = [2,7,4,1,8,1]

Output: 1

Explanation:

We can combine 2 and 4 to get 2, so the array converts to [2,7,1,8,1] then, we can combine 7 and 8 to get 1, so the array converts to [2,1,1,1] then, we can combine 2 and 1 to get 1, so the array converts to [1,1,1] then, we can combine 1 and 1 to get 0, so the array converts to [1], then that's the optimal value.

Example 2:

Input: stones = [31,26,33,21,40]

Output: 5

Constraints:

- $1 \leq \text{stones.length} \leq 30$
- $1 \leq \text{stones}[i] \leq 100$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def lastStoneWeightII(self, stones: list[int]) -> int:
        summ = sum(stones)
        s = 0
        dp = [True] + [False] * summ

        for stone in stones:
            for w in range(summ // 2 + 1)[::-1]:
                if w >= stone:
                    dp[w] = dp[w] or dp[w - stone]

            if dp[w]:
                s = max(s, w)

        return summ - 2 * s
```

• LeetDoocs

```
class Solution:
    def lastStoneWeightII(self, stones: List[int]) -> int:
        s = sum(stones)
        m, n = len(stones), s >> 1
        dp = [[0] * (n + 1) for _ in range(m + 1)]
        for i in range(1, m + 1):
            for j in range(n + 1):
                dp[i][j] = dp[i - 1][j]
                if stones[i - 1] <= j:
                    dp[i][j] = max(
                        dp[i][j], dp[i - 1][j - stones[i - 1]] + stones[i - 1]
                    )
        return s - 2 * dp[-1][-1]
```

• SimplyLeet

```
# Python solution for Last Stone Weight II

def lastStoneWeightII(stones):
    total_sum = sum(stones) # Total sum of all stones
    half_sum = total_sum // 2 # Maximum sum we target for one of the groups
    dp = [False] * (half_sum + 1) # DP array where dp[j] indicates if sum j is
    achievable
    dp[0] = True # Base case: sum 0 is always achievable

    # Process each stone
    for stone in stones:

        # Traverse backwards to avoid using the same stone twice in one
        iteration
        for j in range(half_sum, stone - 1, -1):
            if dp[j - stone]:
                dp[j] = True

    # Find the best achievable subset sum (s1) closest to half_sum
    for s1 in range(half_sum, -1, -1):
        if dp[s1]:
            # The minimal possible remaining stone weight
            return total_sum - 2 * s1
```

```
    return 0 # Fallback, though this line should never be reached

# Example usage
print(lastStoneWeightII([2,7,4,1,8,1])) # Expected output: 1
```

Round 8 · Low · Medium**p.2959**

Unbounded Knapsack

Round 8 · Low · Medium · 4 questions**Round 8 · Low · Medium****p.2960**

Unbounded Knapsack

#279 Perfect Squares

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode Math · Dynamic Programming · Breadth-First Search

Lists: AlgoMaster 600

Problem

Given an integer n , return the least number of perfect square numbers that sum to n .

A perfect square is an integer that is the square of an integer; in other words, it is the product of some integer with itself. For example, 1, 4, 9, and 16 are perfect squares while 3 and 11 are not.

Example 1:

Input: $n = 12$
Output: 3
Explanation: $12 = 4 + 4 + 4$.

Example 2:

Input: $n = 13$
Output: 2
Explanation: $13 = 4 + 9$.

Constraints:

Round 8 · Low · Medium

p.2961

- $1 \leq n \leq 104$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def numSquares(self, n: int) -> int:
        dp = [n] * (n + 1) # 1^2 x n
        dp[0] = 0 # no way
        dp[1] = 1 # 1^2

        for i in range(2, n + 1):
            j = 1
            while j * j <= i:
                dp[i] = min(dp[i], dp[i - j * j] + 1)
                j += 1

        return dp[n]
```

• LeetDoocs

```
class Solution:
    def numSquares(self, n: int) -> int:
        m = int(sqrt(n))
        f = [[inf] * (n + 1) for _ in range(m + 1)]
        f[0][0] = 0
        for i in range(1, m + 1):
            for j in range(n + 1):
                f[i][j] = f[i - 1][j]
                if j >= i * i:
                    f[i][j] = min(f[i][j], f[i][j - i * i] + 1)

        return f[m][n]
```

Round 8 · Low · Medium

p.2962

```
class Solution:
    def numSquares(self, n: int) -> int:
        m = int(sqrt(n))
        f = [0] + [inf] * n
        for i in range(1, m + 1):
            for j in range(i * i, n + 1):
                f[j] = min(f[j], f[j - i * i] + 1)
        return f[n]
```

• SimplyLeet

```
# Python solution using dynamic programming

def numSquares(n: int) -> int:
    # Initialize dp array where dp[i] indicates the least number of perfect
    # squares summing to i.
    dp = [float('inf')] * (n + 1)
    dp[0] = 0 # Base case: 0 perfect squares sum to 0

    # Iterate through each number from 1 to n
    for i in range(1, n + 1):
        # Try every perfect square number j*j less than or equal to i

        j = 1
        while j * j <= i:
            # Update dp[i] with the minimum count found so far
            dp[i] = min(dp[i], dp[i - j*j] + 1)
            j += 1

    return dp[n]

# Example usage:
# print(numSquares(12)) # Output: 3

# print(numSquares(13)) # Output: 2
```

Unbounded Knapsack

□ #322 Coin Change

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Dynamic Programming · BFS
 Lists: Grind 169

Problem

You are given an integer array `coins` representing coins of different denominations and an integer amount representing a total amount of money.

Return the fewest number of coins that you need to make up that amount. If that amount of money cannot be made up by any combination of the coins, return `-1`.

You may assume that you have an infinite number of each kind of coin.

Example 1:

```
Input: coins = [1,2,5], amount = 11
Output: 3
Explanation: 11 = 5 + 5 + 1
```

Example 2:

Input: coins = [2], amount = 3
Output: -1

Example 3:

Input: coins = [1], amount = 0
Output: 0

Constraints:

- $1 \leq \text{coins.length} \leq 12$
- $1 \leq \text{coins}[i] \leq 231 - 1$
- $0 \leq \text{amount} \leq 104$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def coinChange(self, coins: list[int], amount: int) -> int:
        # dp[i] := the minimum number of coins to make up i
        dp = [0] + [amount + 1] * amount

        for coin in coins:
            for i in range(coin, amount + 1):
                dp[i] = min(dp[i], dp[i - coin] + 1)

        return -1 if dp[amount] == amount + 1 else dp[amount]
```

• LeetCode

```
class Solution:
    def coinChange(self, coins: List[int], amount: int) -> int:
        m, n = len(coins), amount
        f = [[inf] * (n + 1) for _ in range(m + 1)]
        f[0][0] = 0
        for i, x in enumerate(coins, 1):
            for j in range(n + 1):
                f[i][j] = f[i - 1][j]
                if j >= x:
                    f[i][j] = min(f[i][j], f[i][j - x] + 1)

        return -1 if f[m][n] >= inf else f[m][n]
```

• SimplyLeet

```
def coinChange(coins, amount):
    # Create a dp array with amount+1 entries set to a value greater than any
    # possible minimum number of coins.
    dp = [amount + 1] * (amount + 1)
    # Base case: 0 coins are needed to form amount 0.
    dp[0] = 0

    # Build up the dp table.
    for current_amount in range(1, amount + 1):
        for coin in coins:
            if coin <= current_amount:
                dp[current_amount] = min(dp[current_amount], dp[current_amount
                - coin] + 1)

    # If dp[amount] remains greater than amount, no solution was found.
    return dp[amount] if dp[amount] != amount + 1 else -1
```

• LC.ca

```

'''
>>> float("inf")
inf
>>> float("inf") + 1
inf
'''

class Solution(object):
    def coinChange(self, coins, amount):
        """
        :type coins: List[int]
        :type amount: int
        :rtype: int
        """

        dp = [float("inf")] * (amount + 1)
        dp[0] = 0
        for i in range(1, amount + 1):
            for coin in coins:
                if i - coin >= 0:

                    dp[i] = min(dp[i], dp[i - coin] + 1)
        return dp[-1] if dp[-1] != float("inf") else -1

#####

class Solution:
    def coinChange(self, coins: List[int], amount: int) -> int:
        dp = [amount + 1] * (amount + 1)
        dp[0] = 0
        for coin in coins:

            for j in range(coin, amount + 1):
                dp[j] = min(dp[j], dp[j - coin] + 1)
        return -1 if dp[-1] > amount else dp[-1]

```

◆ Quick Review

Key Insights

Round 8 · Low · Medium

p.2967

- Use a dynamic programming approach to build up a solution from smaller subproblems.
- Define $dp[x]$ as the minimum number of coins required for amount x .
- The recurrence relation is: $dp[x] = \min(dp[x], dp[x - \text{coin}] + 1)$ for each coin.
- Initialize $dp[0] = 0$ and set other values to a number larger than the maximum possible coins ($\text{amount} + 1$ works as an infinity substitute).
- An alternative approach is through BFS, treating each amount as a node and each coin as an edge, but the DP method is more intuitive here.

Space & Time

Time Complexity: $O(\text{amount} * n)$ where n is the number of coins.

Space Complexity: $O(\text{amount})$ for the dp array.

Solution

We use a bottom-up dynamic programming approach. The idea is to use a one-dimensional dp array where each element at index x represents the minimum number of coins required to form the amount x . Starting

Round 8 · Low · Medium

p.2968

from $dp[0]$ which is 0, we iterate through all amounts up to the target amount and update $dp[x]$ based on each coin denomination. If after filling the dp array the value $dp[amount]$ remains unchanged (i.e., greater than amount), it means the amount cannot be formed and we return -1 .

Unbounded Knapsack

□ #518 Coin Change II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Dynamic Programming
 Lists: NeetCode 150

Problem

You are given an integer array $coins$ representing coins of different denominations and an integer $amount$ representing a total amount of money.

Return the number of combinations that make up that amount. If that amount of money cannot be made up by any combination of the coins, return 0.

You may assume that you have an infinite number of each kind of coin.

The answer is guaranteed to fit into a signed 32-bit integer.

Example 1:

Input: amount = 5, coins = [1,2,5]
 Output: 4
 Explanation: there are four ways to make up the amount:
 5=5
 5=2+2+1
 5=2+1+1+1
 5=1+1+1+1+1

Example 2:

Input: amount = 3, coins = [2]
 Output: 0
 Explanation: the amount of 3 cannot be made up just with coins of 2.

Example 3:

Input: amount = 10, coins = [10]
 Output: 1

Constraints:

- $1 \leq \text{coins.length} \leq 300$
- $1 \leq \text{coins}[i] \leq 5000$
- All the values of coins are unique.
- $0 \leq \text{amount} \leq 5000$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def change(self, amount: int, coins: list[int]) -> int:
        dp = [1] + [0] * amount

        for coin in coins:
            for i in range(coin, amount + 1):
                dp[i] += dp[i - coin]

        return dp[amount]
```

• LeetDoocs

```
class Solution:
    def change(self, amount: int, coins: List[int]) -> int:
        m, n = len(coins), amount
        f = [[0] * (n + 1) for _ in range(m + 1)]
        f[0][0] = 1
        for i, x in enumerate(coins, 1):
            for j in range(n + 1):
                f[i][j] = f[i - 1][j]
                if j >= x:
                    f[i][j] += f[i][j - x]

        return f[m][n]
```

```
class Solution:
    def change(self, amount: int, coins: List[int]) -> int:
        n = amount
        f = [1] + [0] * n
        for x in coins:
            for j in range(x, n + 1):
                f[j] += f[j - x]
        return f[n]
```

• SimplyLeet

```
# Python solution for Coin Change II
def change(amount, coins):
    # Initialize dp array where dp[j] represents the number of ways to form
    # amount j
    dp = [0] * (amount + 1)
    dp[0] = 1 # One way to achieve amount 0 (use no coins)

    # Process each coin
    for coin in coins:
        # Update dp for all amounts from coin to target amount
        for j in range(coin, amount + 1):
            dp[j] += dp[j - coin] # Accumulate ways by using coin
    return dp[amount]

# Example usage:
print(change(5, [1, 2, 5])) # Output: 4
```

Unbounded Knapsack

☐ #983 Minimum Cost For Tickets

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Dynamic Programming
 Lists: AlgoMaster 600

Problem

You have planned some train traveling one year in advance. The days of the year in which you will travel are given as an integer array `days`. Each day is an integer from 1 to 365.

Train tickets are sold in three different ways:

- a 1-day pass is sold for `costs[0]` dollars,
- a 7-day pass is sold for `costs[1]` dollars, and
- a 30-day pass is sold for `costs[2]` dollars.

The passes allow that many days of consecutive travel.

- For example, if we get a 7-day pass on day 2, then we can travel for 7 days: 2, 3, 4, 5, 6, 7, and 8.

Return the minimum number of dollars you need to travel every day in the given list of days.

Round 8 · Low · Medium

p.2973

Example 1:

```
Input: days = [1,4,6,7,8,20], costs = [2,7,15]
Output: 11
Explanation: For example, here is one way to buy passes that lets you travel your travel plan:
On day 1, you bought a 1-day pass for costs[0] = $2, which covered day 1.
On day 3, you bought a 7-day pass for costs[1] = $7, which covered days 3, 4, ..., 9.
On day 20, you bought a 1-day pass for costs[0] = $2, which covered day 20.
In total, you spent $11 and covered all the days of your travel.
```

Example 2:

```
Input: days = [1,2,3,4,5,6,7,8,9,10,30,31], costs = [2,7,15]
Output: 17
Explanation: For example, here is one way to buy passes that lets you travel your travel plan:
On day 1, you bought a 30-day pass for costs[2] = $15 which covered days 1, 2, ..., 30.
On day 31, you bought a 1-day pass for costs[0] = $2 which covered day 31.
In total, you spent $17 and covered all the days of your travel.
```

Constraints:

- $1 \leq \text{days.length} \leq 365$
- $1 \leq \text{days}[i] \leq 365$
- `days` is in strictly increasing order.
- `costs.length == 3`
- $1 \leq \text{costs}[i] \leq 1000$

Community Solutions (Python)

- WalkCC

Round 8 · Low · Medium

p.2974

```
class Solution:
    def mincostTickets(self, days: list[int], costs: list[int]) -> int:
        ans = 0
        last7 = collections.deque()
        last30 = collections.deque()

        for day in days:
            while last7 and last7[0][0] + 7 <= day:
                last7.popleft()
            while last30 and last30[0][0] + 30 <= day:
                last30.popleft()
            last7.append([day, ans + costs[1]])
            last30.append([day, ans + costs[2]])
            ans = min(ans + costs[0], last7[0][1], last30[0][1])

        return ans
```

• LeetDoocs

```
class Solution:
    def mincostTickets(self, days: List[int], costs: List[int]) -> int:
        @cache
        def dfs(i: int) -> int:
            if i >= n:
                return 0
            ans = inf
            for c, v in zip(costs, valid):
                j = bisect_left(days, days[i] + v)
                ans = min(ans, c + dfs(j))

        return ans

n = len(days)
valid = [1, 7, 30]
return dfs(0)
```

Round 8 · Low · Medium

p.2975

```
class Solution:
    def mincostTickets(self, days: List[int], costs: List[int]) -> int:
        m = days[-1]
        f = [0] * (m + 1)
        valid = [1, 7, 30]
        j = 0
        for i in range(1, m + 1):
            if i == days[j]:
                f[i] = inf
                for c, v in zip(costs, valid):
                    f[i] = min(f[i], f[max(0, i - v)] + c)
                j += 1
            else:
                f[i] = f[i - 1]
        return f[m]
```

• SimplyLeet

```
# Python solution with detailed comments
def mincostTickets(days, costs):
    # Create an array dp with size up to the last travel day + 1
    last_day = days[-1]
    dp = [0] * (last_day + 1)
    travel_days = set(days) # Using a set for O(1) look-up

    for day in range(1, last_day + 1):
        if day not in travel_days:
            # If it's not a travel day, cost remains as previous day's cost
            dp[day] = dp[day - 1]
        else:
            # Calculate cost if we buy a 1-day, 7-day, or 30-day pass
            cost1 = dp[day - 1] + costs[0]
            cost7 = dp[max(0, day - 7)] + costs[1]
            cost30 = dp[max(0, day - 30)] + costs[2]
            # Choose the minimum cost option
            dp[day] = min(cost1, cost7, cost30)
    return dp[last_day]

# Example usage:
print(mincostTickets([1,4,6,7,8,20], [2,7,15]))
```

Round 8 · Low · Medium

p.2976

Longest Increasing Subsequence (LIS)

Round 8 · Low · Medium · 4 questions

Round 8 · Low · Medium

p.2977

Longest Increasing Subsequence (LIS)

□ #300 Longest Increasing Subsequence

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Dynamic Programming
Lists: Grind 169

Problem

Given an integer array `nums`, return the length of the longest strictly increasing subsequence.

Example 1:

Input: `nums = [10,9,2,5,3,7,101,18]`
Output: 4
Explanation: The longest increasing subsequence is `[2,3,7,101]`, therefore the length is 4.

Example 2:

Input: `nums = [0,1,0,3,2,3]`
Output: 4

Example 3:

Input: `nums = [7,7,7,7,7,7,7]`
Output: 1

Constraints:

- $1 \leq \text{nums.length} \leq 2500$
- $-104 \leq \text{nums}[i] \leq 104$

Round 8 · Low · Medium

p.2978

Follow up: Can you come up with an algorithm that runs in $O(n \log(n))$ time complexity?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def lengthOfLIS(self, nums: List[int]) -> int:
        if not nums:
            return 0

        # dp[i] := the length of LIS ending in nums[i]
        dp = [1] * len(nums)

        for i in range(1, len(nums)):
            for j in range(i):
                if nums[j] < nums[i]:
                    dp[i] = max(dp[i], dp[j] + 1)

        return max(dp)

class Solution:
    def lengthOfLIS(self, nums: List[int]) -> int:
        # tails[i] := the minimum tails of all the increasing subsequences having
        # length i + 1
        tails = []

        for num in nums:
            if not tails or num > tails[-1]:
                tails.append(num)
            else:
                tails[bisect.bisect_left(tails, num)] = num

        return len(tails)
```

• LeetCode

Round 8 · Low · Medium

p.2979

```
class Solution:
    def lengthOfLIS(self, nums: List[int]) -> int:
        n = len(nums)
        f = [1] * n
        for i in range(1, n):
            for j in range(i):
                if nums[j] < nums[i]:
                    f[i] = max(f[i], f[j] + 1)
        return max(f)
```

```
class BinaryIndexedTree:
    def __init__(self, n: int):
        self.n = n
        self.c = [0] * (n + 1)

    def update(self, x: int, v: int):
        while x <= self.n:
            self.c[x] = max(self.c[x], v)
            x += x & -x
```

```
    def query(self, x: int) -> int:
        mx = 0
        while x:
            mx = max(mx, self.c[x])
            x -= x & -x
        return mx
```

```
class Solution:
    def lengthOfLIS(self, nums: List[int]) -> int:
        s = sorted(set(nums))
        m = len(s)
        tree = BinaryIndexedTree(m)
        for x in nums:
            x = bisect_left(s, x) + 1
            t = tree.query(x - 1) + 1
            tree.update(x, t)
        return tree.query(m)
```

• SimplyLeet

Round 8 · Low · Medium

p.2980

```
import bisect

def lengthOfLIS(nums):
    dp = [] # dp array stores smallest tail for each subsequence length
    for num in nums:
        # Find the leftmost insertion point using binary search
        idx = bisect.bisect_left(dp, num)
        if idx == len(dp):
            dp.append(num) # Extend dp if no larger tail exists
        else:
            dp[idx] = num # Replace to maintain the smallest tail
    return len(dp)
```

```
# Example usage:
# print(lengthOfLIS([10,9,2,5,3,7,101,18])) # Output: 4
```

• LC.ca

```
# find the largest end element in tails that is smaller than nums[i]
# and then replace it with nums[i] and discard the list in the same length
# which is implemented by `tail[idx] = num`
```

```
class Solution(object):
    def lengthOfLIS(self, nums):
        """
        :type nums: List[int]
        :rtype: int
        """
```

```
tail = []
for num in nums:
    # if using bisect_right(tail, num),
    # then input=[7,7,7,7,7,7] will output 7 but expected result is 1
    idx = bisect.bisect_left(tail, num)
    if idx == len(tail): # same as in java: if (i == len) len++;
        tail.append(num)
    else:
        tail[idx] = num
return len(tail)
```

Round 8 · Low · Medium

p.2981

```
# implementation of bisect.bisect_left()
# similar to Leetcode-302, find left/right/top/bottom callable
def bisect_left(a, x, lo=0, hi=None):
    if hi is None:
        hi = len(a)
```

```
while lo < hi:
    mid = (lo + hi) // 2
```

```
if a[mid] < x:
    lo = mid + 1
else:
    hi = mid
```

```
return lo
```

```
#####
```

```
class BinaryIndexedTree:
```

```
def __init__(self, n: int):
    self.n = n
    self.c = [0] * (n + 1)
```

```
def update(self, x: int, v: int):
    while x <= self.n:
        self.c[x] = max(self.c[x], v)
        x += x & -x
```

```
def query(self, x: int) -> int:
```

```
mx = 0
while x:
    mx = max(mx, self.c[x])
    x -= x & -x
return mx
```

```
class Solution:
    def lengthOfLIS(self, nums: List[int]) -> int:
        s = sorted(set(nums))
```

Round 8 · Low · Medium

p.2982

```

m = len(s)
tree = BinaryIndexedTree(m)
for x in nums:
    x = bisect_left(s, x) + 1
    t = tree.query(x - 1) + 1
    tree.update(x, t)
return tree.query(m)

```

◆ Quick Review

Key Insights

- A dynamic programming solution with $O(n^2)$ time complexity exists, but we focus on an optimized approach.
- Using binary search and a dp array, we can achieve $O(n \log n)$ time.
- The dp array maintains the smallest possible tail for increasing subsequences of various lengths.
- For each number, perform binary search to decide if it can extend or replace a value in dp.

Space & Time

Time Complexity: $O(n \log n)$ where n is the length of nums

Space Complexity: $O(n)$ for storing the dp array

Solution

Round 8 · Low · Medium

p.2983

We use a dynamic programming approach combined with binary search. The idea is to maintain a dp array where $dp[i]$ holds the smallest tail value for any increasing subsequence with length $i+1$. For each number in the array, we perform a binary search on dp to find its correct insertion position:

- If the number is larger than all elements, we append it, effectively extending the current longest subsequence.
- Otherwise, we replace the first element in dp that is greater than or equal to the current number to keep the tail as small as possible. This process ensures that dp remains sorted and its length gives the length of the longest increasing subsequence.

Round 8 · Low · Medium

p.2984

Longest Increasing Subsequence (LIS)

#673 Number of Longest Increasing Subsequence

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Binary Indexed Tree · Segment Tree

Lists: AlgoMaster 600

Problem

Given an integer array `nums`, return the number of longest increasing subsequences.

Notice that the sequence has to be strictly increasing.

Example 1:

Input: `nums = [1,3,5,4,7]`
Output: 2
Explanation: The two longest increasing subsequences are `[1, 3, 4, 7]` and `[1, 3, 5, 7]`.

Example 2:

Input: `nums = [2,2,2,2,2]`
Output: 5
Explanation: The length of the longest increasing subsequence is 1, and there are 5 increasing subsequences of length 1, so output 5.

Constraints:

- $1 \leq \text{nums.length} \leq 2000$

Round 8 · Low · Medium

p.2985

- $-106 \leq \text{nums}[i] \leq 106$
- The answer is guaranteed to fit inside a 32-bit integer.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findNumberOfLIS(self, nums: list[int]) -> int:
        ans = 0
        maxLength = 0
        # length[i] := the length of the LIS ending in nums[i]
        length = [1] * len(nums)
        # count[i] := the number of LIS's ending in nums[i]
        count = [1] * len(nums)

        # Calculate the `length` and `count` arrays.

        for i, num in enumerate(nums):
            for j in range(i):
                if nums[j] < num:
                    if length[i] < length[j] + 1:
                        length[i] = length[j] + 1
                        count[i] = count[j]
                    elif length[i] == length[j] + 1:
                        count[i] += count[j]

        # Get the number of LIS.

        for i, l in enumerate(length):
            if l > maxLength:
                maxLength = l
                ans = count[i]
            elif l == maxLength:
                ans += count[i]

        return ans
```

• LeetDoocs

Round 8 · Low · Medium

p.2986

```
class Solution:
    def findNumberOfLIS(self, nums: List[int]) -> int:
        n = len(nums)
        f = [1] * n
        cnt = [1] * n
        mx = 0
        for i in range(n):
            for j in range(i):
                if nums[j] < nums[i]:
                    if f[i] < f[j] + 1:
                        f[i] = f[j] + 1
                        cnt[i] = cnt[j]
                elif f[i] == f[j] + 1:
                    cnt[i] += cnt[j]
            if mx < f[i]:
                mx = f[i]
                ans = cnt[i]
            elif mx == f[i]:
                ans += cnt[i]
        return ans
```

```
class BinaryIndexedTree:
    __slots__ = ["n", "c", "d"]

    def __init__(self, n):
        self.n = n
        self.c = [0] * (n + 1)
        self.d = [0] * (n + 1)

    def update(self, x, v, cnt):
        while x <= self.n:
```

```
            if self.c[x] < v:
                self.c[x] = v
                self.d[x] = cnt
            elif self.c[x] == v:
                self.d[x] += cnt
            x += x & -x
```

```
    def query(self, x):
        v = cnt = 0
        while x:
```

```
        if self.c[x] > v:
            v = self.c[x]
            cnt = self.d[x]
        elif self.c[x] == v:
            cnt += self.d[x]
        x -= x & -x
        return v, cnt
```

```
class Solution:
```

```
    def findNumberOfLIS(self, nums: List[int]) -> int:
        arr = sorted(set(nums))
        m = len(arr)
        tree = BinaryIndexedTree(m)
        for x in nums:
            i = bisect_left(arr, x) + 1
            v, cnt = tree.query(i - 1)
            tree.update(i, v + 1, max(cnt, 1))
        return tree.query(m)[1]
```

• SimplyLeet

```
# Python solution for Number of Longest Increasing Subsequence
```

```
def findNumberOfLIS(nums):
    # Get the length of the input array
    n = len(nums)

    # If the list is empty, return 0 (edge-case)
    if n == 0:
        return 0
```

```

# dp[i] stores the length of the longest increasing subsequence ending at i
dp = [1] * n
# count[i] stores the number of longest increasing subsequences ending at i
count = [1] * n

# Iterate over each element in nums
for i in range(n):
    # Check all previous elements to update dp[i] and count[i]
    for j in range(i):
        if nums[j] < nums[i]:

            # Found a potential sequence ending at i
            if dp[j] + 1 > dp[i]:
                # A longer subsequence is found, update length and count
                dp[i] = dp[j] + 1
                count[i] = count[j]
            elif dp[j] + 1 == dp[i]:
                # Found another subsequence of the same maximum length
                # ending at i, add count
                count[i] += count[j]

    # Find the length of the overall longest increasing subsequence in the
    # array

longest = max(dp)
# Sum the number of ways for all indices that achieve this longest length
ans = sum(c for i, c in enumerate(count) if dp[i] == longest)

return ans

# Example usage
# nums = [1,3,5,4,7]
# print(findNumberOfLIS(nums)) # Output should be 2

```

Longest Increasing Subsequence (LIS)

□ #1027 Longest Arithmetic Subsequence MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table · Binary Search · Dynamic Programming

Lists: AlgoMaster 600

Problem

Given an array `nums` of integers, return the length of the longest arithmetic subsequence in `nums`.

Note that:

- A subsequence is an array that can be derived from another array by deleting some or no elements without changing the order of the remaining elements.
- A sequence `seq` is arithmetic if `seq[i + 1] - seq[i]` are all the same value (for $0 \leq i < \text{seq.length} - 1$).

Example 1:

Input: `nums = [3,6,9,12]`
 Output: 4
 Explanation: The whole array is an arithmetic sequence with steps of length = 3.

Example 2:

Input: nums = [9,4,7,2,10]
 Output: 3
 Explanation: The longest arithmetic subsequence is [4,7,10].

Example 3:

Input: nums = [20,1,15,3,10,5,8]
 Output: 4
 Explanation: The longest arithmetic subsequence is [20,15,10,5].

Constraints:

- $2 \leq \text{nums.length} \leq 1000$
- $0 \leq \text{nums}[i] \leq 500$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def longestArithSeqLength(self, nums: list[int]) -> int:
        n = len(nums)
        ans = 0
        # dp[i][k] := the length of the longest arithmetic subsequence of
        # nums[0..i]
        # with k = diff + 500
        dp = [[0] * 1001 for _ in range(n)]

        for i in range(n):
            for j in range(i):

                k = nums[i] - nums[j] + 500
                dp[i][k] = max(2, dp[j][k] + 1)
                ans = max(ans, dp[i][k])

        return ans
```

• LeetCode

Round 8 · Low · Medium

p.2991

```
class Solution:
    def longestArithSeqLength(self, nums: List[int]) -> int:
        n = len(nums)
        f = [[1] * 1001 for _ in range(n)]
        ans = 0
        for i in range(1, n):
            for k in range(i):
                j = nums[i] - nums[k] + 500
                f[i][j] = max(f[i][j], f[k][j] + 1)
                ans = max(ans, f[i][j])

        return ans
```

• SimplyLeet

```
# Python solution for Longest Arithmetic Subsequence
def longestArithSeqLength(nums):
    # n is the length of the input array
    n = len(nums)
    # dp will be a list of dictionaries.
    # dp[i] will store {diff: length} for subsequences ending at index i
    dp = [{ } for _ in range(n)]
    max_length = 0 # initialize max length

    # iterate over each pair (j, i) where j < i

    for i in range(n):
        for j in range(i):
            # current difference between nums[i] and nums[j]
            diff = nums[i] - nums[j]
            # get the length for this difference from dp[j], default is 1 (only
            # nums[j])

            dp[i][diff] = dp[j].get(diff, 1) + 1
            # update the maximum length found so far
            max_length = max(max_length, dp[i][diff])

        return max_length

# Example usage:
# print(longestArithSeqLength([3,6,9,12])) -> Output: 4
```

Round 8 · Low · Medium

p.2992

Longest Increasing Subsequence (LIS)

#1048 Longest String Chain

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table · Two Pointers · String ·
 Dynamic Programming · Sorting
 Lists: AlgoMaster 600

Problem

You are given an array of words where each word consists of lowercase English letters. wordA is a predecessor of wordB if and only if we can insert exactly one letter anywhere in wordA without changing the order of the other characters to make it equal to wordB.

- For example, "abc" is a predecessor of "abac", while "cba" is not a predecessor of "bcad".

A word chain is a sequence of words [word1, word2, ..., wordk] with $k \geq 1$, where word1 is a predecessor of word2, word2 is a predecessor of word3, and so on. A single word is trivially a word chain with $k == 1$.

Round 8 · Low · Medium

p.2993

Return the length of the longest possible word chain with words chosen from the given list of words.

Example 1:

Input: words = ["a","b","ba","bca","bda","bdca"]
 Output: 4
 Explanation: One of the longest word chains is ["a","ba","bda","bdca"].

Example 2:

Input: words = ["xbc","pcxbcf","xb","cxbc","pcxbc"]
 Output: 5
 Explanation: All the words can be put in a word chain ["xb", "xbc", "cxbc", "pcxbc", "pcxbcf"].

Example 3:

Input: words = ["abcd","dbqca"]
 Output: 1
 Explanation: The trivial word chain ["abcd"] is one of the longest word chains. ["abcd","dbqca"] is not a valid word chain because the ordering of the letters is changed.

Constraints:

- $1 \leq \text{words.length} \leq 1000$
- $1 \leq \text{words}[i].\text{length} \leq 16$
- $\text{words}[i]$ only consists of lowercase English letters.

Community Solutions (Python)

- WalkCC

Round 8 · Low · Medium

p.2994

```
class Solution:
    def longestStrChain(self, words: list[str]) -> int:
        wordsSet = set(words)

        @functools.lru_cache(None)
        def dp(s: str) -> int:
            """Returns the longest chain where s is the last word."""
            ans = 1
            for i in range(len(s)):
                pred = s[:i] + s[i + 1:]

                if pred in wordsSet:
                    ans = max(ans, dp(pred) + 1)
            return ans

        return max(dp(word) for word in words)
```

```
class Solution:
    def longestStrChain(self, words: list[str]) -> int:
        dp = {}

        for word in sorted(words, key=len):
            dp[word] = max(dp.get(word[:i] + word[i + 1:], 0) +
                          1 for i in range(len(word)))

        return max(dp.values())
```

• LeetCode

```
class Solution:
    def longestStrChain(self, words: List[str]) -> int:
        def check(w1, w2):
            if len(w2) - len(w1) != 1:
                return False
            i = j = cnt = 0
            while i < len(w1) and j < len(w2):
                if w1[i] != w2[j]:
                    cnt += 1
                else:
                    i += 1
                    j += 1
            return cnt == 1
```

Round 8 · Low · Medium

p.2995

```
i += 1
j += 1
return cnt < 2 and i == len(w1)

n = len(words)
dp = [1] * (n + 1)
words.sort(key=lambda x: len(x))
res = 1
for i in range(1, n):
    for j in range(i):
        if check(words[j], words[i]):
            dp[i] = max(dp[i], dp[j] + 1)
    res = max(res, dp[i])
return res
```

```
class Solution:
    def longestStrChain(self, words: List[str]) -> int:
        words.sort(key=lambda x: len(x))
        res = 0
        mp = {}
        for word in words:
            x = 1
            for i in range(len(word)):
                pre = word[:i] + word[i + 1:]
                x = max(x, mp.get(pre, 0) + 1)
```

```
mp[word] = x
res = max(res, x)
return res
```

• SimplyLeet

Round 8 · Low · Medium

p.2996

```
# Function to compute the longest string chain using dynamic programming
def longestStrChain(words):
    # Sort words based on their length to ensure potential predecessors come
    # first
    words.sort(key=len)
    # Dictionary to store the longest chain length ending with the word
    dp = {}
    max_chain = 1 # Minimum chain length is 1

    # Process each word in sorted order
    for word in words:

        current_length = 1 # Each word by itself is a chain of length 1
        # Try removing one letter at a time to form a potential predecessor
        for i in range(len(word)):
            # Form the predecessor by skipping the i-th letter
            predecessor = word[:i] + word[i+1:]
            # If predecessor exists, update current chain length
            if predecessor in dp:
                current_length = max(current_length, dp[predecessor] + 1)
        # Save the computed chain length for the current word
        dp[word] = current_length

    # Update the overall maximum chain length
    max_chain = max(max_chain, current_length)

    return max_chain

# Example usage:
words = ["a", "b", "ba", "bca", "bda", "bdca"]
print(longestStrChain(words)) # Expected output: 4
```

Round 8 · Low · Medium

p.2997

2D Grid DP

Round 8 · Low · Medium · 9 questions

Round 8 · Low · Medium

p.2998

2D Grid DP

#62 Unique Paths

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · Dynamic Programming · Combinatorics
Lists: Grind 169

Problem

There is a robot on an $m \times n$ grid. The robot is initially located at the top-left corner (i.e., $\text{grid}[0][0]$). The robot tries to move to the bottom-right corner (i.e., $\text{grid}[m - 1][n - 1]$). The robot can only move either down or right at any point in time.

Given the two integers m and n , return the number of possible unique paths that the robot can take to reach the bottom-right corner.

The test cases are generated so that the answer will be less than or equal to $2 * 10^9$.

Example 1:

Round 8 · Low · Medium

p.2999



Input: $m = 3, n = 7$
Output: 28

Example 2:

Input: $m = 3, n = 2$
Output: 3
Explanation: From the top-left corner, there are a total of 3 ways to reach the bottom-right corner:
1. Right -> Down -> Down
2. Down -> Down -> Right
3. Down -> Right -> Down

Constraints:

- $1 \leq m, n \leq 100$

Community Solutions (Python)

- WalkCC

Round 8 · Low · Medium

p.3000


```
class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        # dp[i][j] := the number of unique paths from (0, 0) to (i, j)
        dp = [[1] * n for _ in range(m)]

        for i in range(1, m):
            for j in range(1, n):
                dp[i][j] = dp[i - 1][j] + dp[i][j - 1]

        return dp[-1][-1]
```

```
class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        dp = [1] * n

        for _ in range(1, m):
            for j in range(1, n):
                dp[j] += dp[j - 1]

        return dp[n - 1]
```

• LeetCode

```
class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        f = [[0] * n for _ in range(m)]
        f[0][0] = 1
        for i in range(m):
            for j in range(n):
                if i:
                    f[i][j] += f[i - 1][j]
                if j:
                    f[i][j] += f[i][j - 1]

        return f[-1][-1]
```

```
class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        f = [[1] * n for _ in range(m)]
        for i in range(1, m):
            for j in range(1, n):
                f[i][j] = f[i - 1][j] + f[i][j - 1]
        return f[-1][-1]
```

Round 8 · Low · Medium

p.3001

```
class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        f = [1] * n
        for _ in range(1, m):
            for j in range(1, n):
                f[j] += f[j - 1]
        return f[-1]
```

• SimplyLeet

```
# Python solution using Dynamic Programming

def uniquePaths(m, n):
    # Initialize a 2D list with 1's for the first row and first column
    dp = [[1] * n for _ in range(m)]

    # Fill in the DP table
    for i in range(1, m):
        for j in range(1, n):
            # The number of ways to reach this cell is the sum of the ways from
            # the cell above and the cell on the left
```

```
        dp[i][j] = dp[i-1][j] + dp[i][j-1]
    # Return the value in the bottom-right cell
    return dp[m-1][n-1]
```

```
# Example usage:
print(uniquePaths(3, 7))
```

• LC.ca

```
class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        # avoid setting dp[][] to 1 for i==0 or j==0 as initialization
        dp = [[1] * n for _ in range(m)]
        for i in range(1, m):
            for j in range(1, n):
                dp[i][j] = dp[i - 1][j] + dp[i][j - 1]
        return dp[-1][-1]
```

◆ Quick Review

Round 8 · Low · Medium

p.3002

Key Insights

- The robot is constrained to only move right or down.
- Any valid path consists of exactly $(m-1)$ down moves and $(n-1)$ right moves.
- The problem can be solved using Dynamic Programming where each cell is the sum of the ways to reach the cell from the top and left.
- Alternatively, a combinatorial solution is possible by calculating the number of unique sequences of moves, equivalent to choosing $(m-1)$ moves from $(m+n-2)$ moves.

Space & Time

Time Complexity: $O(mn)$

Space Complexity: $O(mn)$ – can be optimized to $O(n)$ using a 1D DP array.

Solution

We can solve the problem using a dynamic programming approach. The idea is to create a 2D DP table where each cell (i, j) represents the number of ways to reach that cell. Since

the robot can only come from above or from the left, we update the DP table as follows:

- Initialize $dp[0][j] = 1$ for all j (only one way to move right in the first row).
- Initialize $dp[i][0] = 1$ for all i (only one way to move down in the first column).
- For each other cell, compute $dp[i][j] = dp[i-1][j] + dp[i][j-1]$. The final answer will be at $dp[m-1][n-1]$.

The combinatorial alternative computes $C(m+n-2, m-1)$ or $C(m+n-2, n-1)$ using basic factorial arithmetic, which is efficient for the given constraints.

2D Grid DP

#63 Unique Paths II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Dynamic Programming · Matrix
 Lists: AlgoMaster 600

Problem

You are given an $m \times n$ integer array grid. There is a robot initially located at the top-left corner (i.e., $\text{grid}[0][0]$). The robot tries to move to the bottom-right corner (i.e., $\text{grid}[m - 1][n - 1]$). The robot can only move either down or right at any point in time.

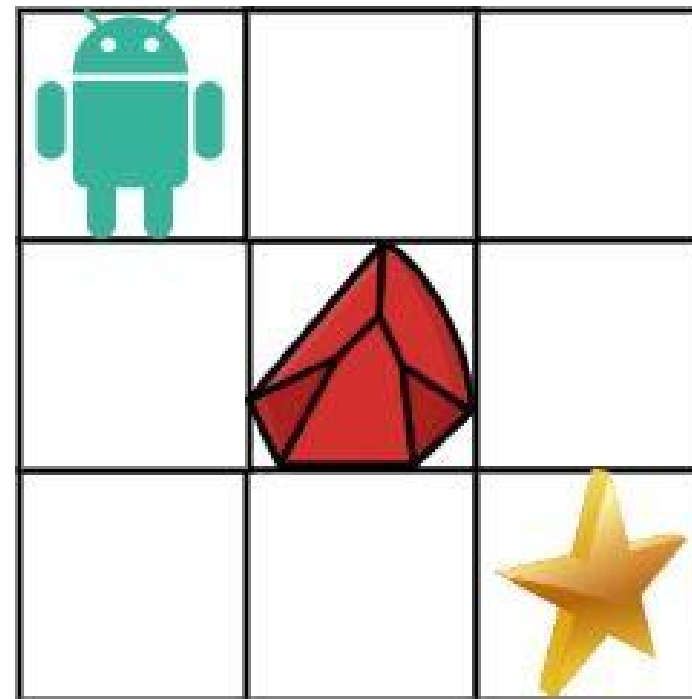
An obstacle and space are marked as 1 or 0 respectively in grid. A path that the robot takes cannot include any square that is an obstacle. Return the number of possible unique paths that the robot can take to reach the bottom-right corner.

The testcases are generated so that the answer will be less than or equal to $2 * 10^9$.

Example 1:

Round 8 · Low · Medium

p.3005

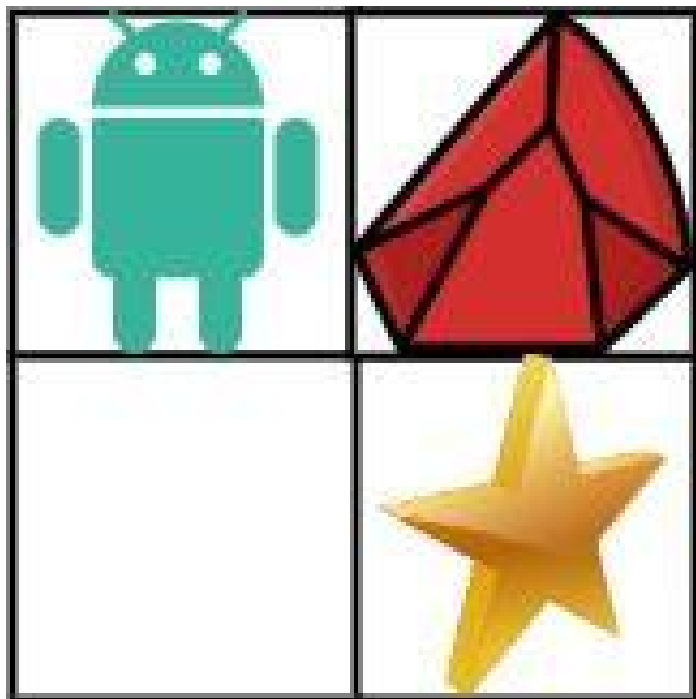


```
Input: obstacleGrid = [[0,0,0],[0,1,0],[0,0,0]]
Output: 2
Explanation: There is one obstacle in the middle of the 3x3 grid above.
There are two ways to reach the bottom-right corner:
1. Right -> Right -> Down -> Down
2. Down -> Down -> Right -> Right
```

Example 2:

Round 8 · Low · Medium

p.3006



Input: obstacleGrid = [[0,1],[0,0]]

Output: 1

Constraints:

- $m == \text{obstacleGrid.length}$
- $n == \text{obstacleGrid}[i].\text{length}$
- $1 \leq m, n \leq 100$
- $\text{obstacleGrid}[i][j]$ is 0 or 1.

Community Solutions (Python)

- WalkCC

Round 8 · Low · Medium

p.3007

```
class Solution:
    def uniquePathsWithObstacles(self, obstacleGrid: List[List[int]]) -> int:
        m = len(obstacleGrid)
        n = len(obstacleGrid[0])
        # dp[i][j] := the number of unique paths from (0, 0) to (i, j)
        dp = [[0] * (n + 1) for _ in range(m + 1)]
        dp[0][1] = 1 # Can also set dp[1][0] = 1.

        for i in range(1, m + 1):
            for j in range(1, n + 1):

                if obstacleGrid[i - 1][j - 1] == 0:
                    dp[i][j] = dp[i - 1][j] + dp[i][j - 1]

        return dp[m][n]
```

• LeetCode

```
class Solution:
    def uniquePathsWithObstacles(self, obstacleGrid: List[List[int]]) -> int:
        @cache
        def dfs(i: int, j: int) -> int:
            if i >= m or j >= n or obstacleGrid[i][j]:
                return 0
            if i == m - 1 and j == n - 1:
                return 1
            return dfs(i + 1, j) + dfs(i, j + 1)

        m, n = len(obstacleGrid), len(obstacleGrid[0])
        return dfs(0, 0)
```

• SimplyLeet

Round 8 · Low · Medium

p.3008

```
# Function to calculate unique paths with obstacles in a grid
def uniquePathsWithObstacles(obstacleGrid):
    # If the starting cell has an obstacle return 0 immediately
    if not obstacleGrid or obstacleGrid[0][0] == 1:
        return 0
    m, n = len(obstacleGrid), len(obstacleGrid[0])

    # Create a dp grid with same dimensions initialized to 0
    dp = [[0] * n for _ in range(m)]
    dp[0][0] = 1 # start position

    # Fill the first column considering obstacles
    for i in range(1, m):
        dp[i][0] = 0 if obstacleGrid[i][0] == 1 else dp[i-1][0]

    # Fill the first row considering obstacles
    for j in range(1, n):
        dp[0][j] = 0 if obstacleGrid[0][j] == 1 else dp[0][j-1]

    # Fill the rest dp table

    for i in range(1, m):
        for j in range(1, n):
            if obstacleGrid[i][j] == 1:
                dp[i][j] = 0 # obstacle cell
            else:
                dp[i][j] = dp[i-1][j] + dp[i][j-1] # paths from top and left
    return dp[m-1][n-1]

# Example usage:
grid = [[0,0,0],[0,1,0],[0,0,0]]

print(uniquePathsWithObstacles(grid)) # Output: 2
```

2D Grid DP

□ #64 Minimum Path Sum

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Dynamic Programming · Matrix
 Lists: Denny Zhang

Problem

Given a $m \times n$ grid filled with non-negative numbers, find a path from top left to bottom right, which minimizes the sum of all numbers along its path.

Note: You can only move either down or right at any point in time.

Example 1:

1	3	1
1	5	1
4	2	1

Input: grid = [[1,3,1],[1,5,1],[4,2,1]]

Output: 7

Explanation: Because the path 1 → 3 → 1 → 1 → 1 minimizes the sum.

Example 2:

Input: grid = [[1,2,3],[4,5,6]]

Output: 12

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq m, n \leq 200$

Round 8 · Low · Medium

p.3011

- $0 \leq \text{grid}[i][j] \leq 200$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minPathSum(self, grid: List[List[int]]) -> int:
        m = len(grid)
        n = len(grid[0])

        for i in range(m):
            for j in range(n):
                if i > 0 and j > 0:
                    grid[i][j] += min(grid[i - 1][j], grid[i][j - 1])
                elif i > 0:
                    grid[i][0] += grid[i - 1][0]
                elif j > 0:
                    grid[0][j] += grid[0][j - 1]

        return grid[m - 1][n - 1]
```

• LeetCode

```
class Solution:
    def minPathSum(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        f = [[0] * n for _ in range(m)]
        f[0][0] = grid[0][0]
        for i in range(1, m):
            f[i][0] = f[i - 1][0] + grid[i][0]
        for j in range(1, n):
            f[0][j] = f[0][j - 1] + grid[0][j]
        for i in range(1, m):
            for j in range(1, n):
                f[i][j] = min(f[i - 1][j], f[i][j - 1]) + grid[i][j]
        return f[-1][-1]
```

• SimplyLeet

Round 8 · Low · Medium

p.3012

```
# Python solution using dynamic programming
def minPathSum(grid):
    # Number of rows and columns in the grid
    rows = len(grid)
    cols = len(grid[0])

    # Initialize the DP table directly in grid to save space
    # Fill the first row by accumulating the sum from the left
    for j in range(1, cols):
        grid[0][j] += grid[0][j-1]

    # Fill the first column by accumulating the sum above
    for i in range(1, rows):
        grid[i][0] += grid[i-1][0]

    # Compute the minimum path sum for the rest of the cells
    for i in range(1, rows):
        for j in range(1, cols):
            # The cell value is added to the minimum of the top or left cell
            grid[i][j] += min(grid[i-1][j], grid[i][j-1])

    # The bottom-right cell contains the answer
    return grid[rows-1][cols-1]

# Example usage
example_grid = [[1,3,1],[1,5,1],[4,2,1]]
print(minPathSum(example_grid)) # Output should be 7
```

• LC.ca

```
class Solution:
    def minPathSum(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        f = [[0] * n for _ in range(m)]
        f[0][0] = grid[0][0]
        for i in range(1, m):
            f[i][0] = f[i-1][0] + grid[i][0]
        for j in range(1, n):
            f[0][j] = f[0][j-1] + grid[0][j]
        for i in range(1, m):
```

Round 8 · Low · Medium

p.3013

```
for j in range(1, n):
    f[i][j] = min(f[i-1][j], f[i][j-1]) + grid[i][j]
return f[-1][-1]
```

◆ Quick Review

Key Insights

- The problem can be solved using Dynamic Programming.
- The optimal solution for each cell depends on the minimum path sum of the cell above it and the cell to its left.
- Since you can only move right or down, only two adjacent cells affect the current cell's result.
- Edge cases include the first row and first column, which have only one direction of movement.

Space & Time

Time Complexity: $O(m * n)$, where m and n are the number of rows and columns.

Space Complexity: $O(m * n)$ for the DP table (this can be optimized to $O(n)$ with in-place computations).

Solution

Round 8 · Low · Medium

p.3014

We use a Dynamic Programming (DP) approach where we create a DP table (or use the grid itself) that stores the minimum path sum to each cell. For each cell (i, j):

- If $i = 0$ (first row), the only possible path is from the left.
- If $j = 0$ (first column), the only possible path is from above.
- For other cells, the minimum path sum is the cell value plus the minimum of the two possible previous cells (above and left). This ensures that by the time we reach the bottom-right cell, we have accumulated the minimum sum required to reach that cell.

2D Grid DP

□ #120 Triangle

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Dynamic Programming
 Lists: AlgoMaster 600

Problem

Given a triangle array, return the minimum path sum from top to bottom.

For each step, you may move to an adjacent number of the row below. More formally, if you are on index i on the current row, you may move to either index i or index $i + 1$ on the next row.

Example 1:

```
Input: triangle = [[2],[3,4],[6,5,7],[4,1,8,3]]
Output: 11
Explanation: The triangle looks like:
  2
 3 4
6 5 7
4 1 8 3
The minimum path sum from top to bottom is 2 + 3 + 5 + 1 = 11 (underlined above).
```

Example 2:

```
Input: triangle = [[-10]]
Output: -10
```

Constraints:

- $1 \leq \text{triangle.length} \leq 200$
- $\text{triangle}[0].\text{length} == 1$
- $\text{triangle}[i].\text{length} == \text{triangle}[i - 1].\text{length} + 1$
- $-104 \leq \text{triangle}[i][j] \leq 104$

Follow up: Could you do this using only $O(n)$ extra space, where n is the total number of rows in the triangle?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minimumTotal(self, triangle: list[list[int]]) -> int:
        for i in range(len(triangle) - 2, -1, -1):
            for j in range(i + 1):
                triangle[i][j] += min(triangle[i + 1][j],
                                      triangle[i + 1][j + 1])

        return triangle[0][0]
```

• LeetDoocs

```
class Solution:
    def minimumTotal(self, triangle: List[List[int]]) -> int:
        n = len(triangle)
        f = [[0] * (n + 1) for _ in range(n + 1)]
        for i in range(n - 1, -1, -1):
            for j in range(i + 1):
                f[i][j] = min(f[i + 1][j], f[i + 1][j + 1]) + triangle[i][j]
        return f[0][0]
```

• SimplyLeet

```
def minimumTotal(triangle):
    # If the triangle is empty, return 0
    if not triangle:
        return 0
    # Start with a copy of the last row which acts as our dp array
    dp = triangle[-1][:]
    # Process the triangle from the second last row up to the top row
    for row in range(len(triangle) - 2, -1, -1):
        for col in range(len(triangle[row])):
            # Update dp array by adding the current triangle element
            # to the minimum of its two children in the dp array
            dp[col] = triangle[row][col] + min(dp[col], dp[col + 1])
    return dp[0]

# Example usage:
triangle1 = [[2], [3, 4], [6, 5, 7], [4, 1, 8, 3]]
print(minimumTotal(triangle1)) # Output: 11
```

2D Grid DP

□ #221 Maximal Square

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Dynamic Programming · Matrix
 Lists: Grind 169

Problem

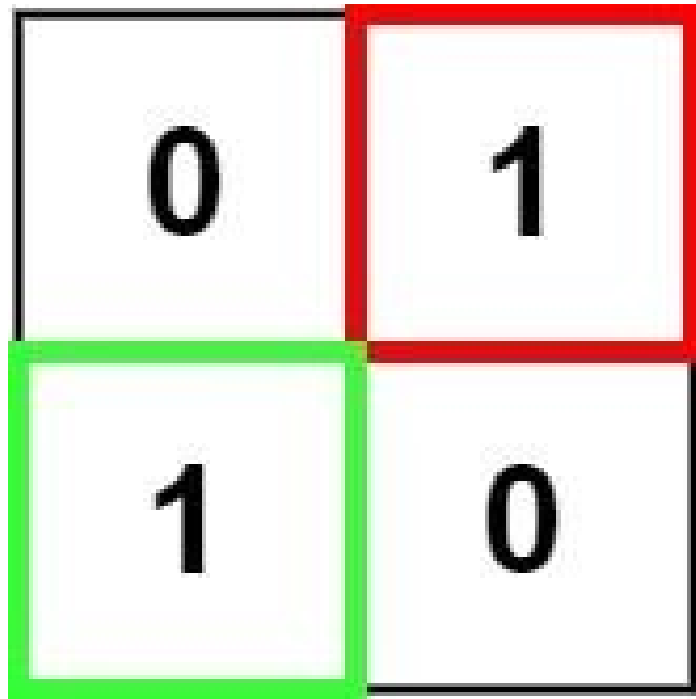
Given an $m \times n$ binary matrix filled with 0's and 1's, find the largest square containing only 1's and return its area.

Example 1:

1	0	1	0	0
1	0	1	1	1
1	1	1	1	1
1	0	0	1	0

Input: matrix = `[["1","0","1","0","0"],["1","0","1","1","1"],["1","1","1","1","1"],["1","0","0","1","0"]]`
 Output: 4

Example 2:



Input: matrix = [["0","1"],["1","0"]]
Output: 1

Example 3:

Input: matrix = [["0"]]
Output: 0

Constraints:

- $m == \text{matrix.length}$
- $n == \text{matrix}[i].\text{length}$
- $1 \leq m, n \leq 300$
- $\text{matrix}[i][j]$ is '0' or '1'.

Round 8 · Low · Medium

p.3021

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maximalSquare(self, matrix: List[List[str]]) -> int:
        m = len(matrix)
        n = len(matrix[0])
        dp = [[0] * n for _ in range(m)]
        maxLength = 0

        for i in range(m):
            for j in range(n):
                if i == 0 or j == 0 or matrix[i][j] == '0':
                    dp[i][j] = 1 if matrix[i][j] == '1' else 0
                else:
                    dp[i][j] = min(dp[i - 1][j - 1], dp[i - 1][j], dp[i][j - 1]) + 1
                maxLength = max(maxLength, dp[i][j])

        return maxLength * maxLength
```

• LeetDoocs

```
class Solution:
    def maximalSquare(self, matrix: List[List[str]]) -> int:
        m, n = len(matrix), len(matrix[0])
        dp = [[0] * (n + 1) for _ in range(m + 1)]
        mx = 0
        for i in range(m):
            for j in range(n):
                if matrix[i][j] == '1':
                    dp[i + 1][j + 1] = min(dp[i][j + 1], dp[i + 1][j], dp[i][j + 1]) + 1
                    mx = max(mx, dp[i + 1][j + 1])

        return mx * mx
```

• SimplyLeet

Round 8 · Low · Medium

p.3022

```
# Python implementation of the Maximal Square problem

def maximalSquare(matrix):
    # Check if matrix is empty
    if not matrix or not matrix[0]:
        return 0
    rows = len(matrix)
    cols = len(matrix[0])
    # DP table with extra row and col for ease of boundary conditions
    dp = [[0] * (cols + 1) for _ in range(rows + 1)]

    max_side = 0
    # Traverse through each cell in the matrix
    for i in range(1, rows + 1):
        for j in range(1, cols + 1):
            # If the current cell is '1'
            if matrix[i - 1][j - 1] == '1':
                # Compute the minimum value among the top, left, and top-left
                # neighbors and add 1
                dp[i][j] = min(dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1]) +
                1

                max_side = max(max_side, dp[i][j])
            # The area of the square is the square of the side length

    return max_side * max_side

# Example usage:
matrix = [
    ["1", "0", "1", "0", "0"],
    ["1", "0", "1", "1", "1"],
    ["1", "1", "1", "1", "1"],
    ["1", "0", "0", "1", "0"]
]
print(maximalSquare(matrix))
```

• LC.ca

Round 8 · Low · Medium

p.3023

```
'''
eg. a 10*10 square with full of 1s
this square move 1 line down
this square move 1 line right

so there needs an extra single 1 at bottom right, to make it a larger full
square
'''

class Solution:
    def maximalSquare(self, matrix: List[List[str]]) -> int:
        m, n = len(matrix), len(matrix[0])

        dp = [[0] * (n + 1) for _ in range(m + 1)]
        mx = 0
        for i in range(m):
            for j in range(n):
                if matrix[i][j] == '1':
                    dp[i + 1][j + 1] = 1 + min(dp[i][j + 1], dp[i + 1][j], dp[
i][j])

                    mx = max(mx, dp[i + 1][j + 1])
        return mx * mx
```

◆ Quick Review

Key Insights

- Use dynamic programming to build a solution where each cell in the DP table represents the side length of the largest square ending at that cell.
- If the current cell is '1', its DP value is the minimum of the top, left, and top-left neighbors plus one.
- Update a global maximum side length during the process to determine the area.

Round 8 · Low · Medium

p.3024

- The square's area is the square of its maximum side length.

Space & Time

Time Complexity: $O(m * n)$, where m is the number of rows and n is the number of columns.

Space Complexity: $O(m * n)$ for the DP table (this can be optimized to $O(n)$ with space reduction techniques).

Solution

We use a two-dimensional dynamic programming approach. We define a DP matrix with one extra row and column to simplify boundary conditions. For each cell (i, j) in the input matrix, if the value is '1', then we calculate the DP value as the minimum of its top, left, and top-left neighboring DP values plus one. This value represents the side length of the square ending at (i, j) . We also keep track of the maximum side length found and finally compute the area of the maximal square by squaring the maximum side length.

2D Grid DP

★ #256 Paint House ★

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode Array · Dynamic Programming

Lists: ★ Premium | Premium 98

Problem

There is a row of n houses, where each house can be painted one of three colors: red, blue, or green. The cost of painting each house with a certain color is different. You have to paint all the houses such that no two adjacent houses have the same color.

The cost of painting each house with a certain color is represented by an $n \times 3$ cost matrix costs.

- For example, $\text{costs}[0][0]$ is the cost of painting house 0 with the color red; $\text{costs}[1][2]$ is the cost of painting house 1 with color green, and so on...

Return the minimum cost to paint all houses.

Example 1:

Input: costs = [[17,2,17],[16,16,5],[14,3,19]]
 Output: 10
 Explanation: Paint house 0 into blue, paint house 1 into green, paint house 2 into blue.
 Minimum cost: 2 + 5 + 3 = 10.

Example 2:

Input: costs = [[7,6,2]]
 Output: 2

Constraints:

- costs.length == n
- costs[i].length == 3
- 1 <= n <= 100
- 1 <= costs[i][j] <= 20

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minCost(self, costs: List[List[int]]) -> List[List[int]]:
        for i in range(1, len(costs)):
            costs[i][0] += min(costs[i-1][1], costs[i-1][2])
            costs[i][1] += min(costs[i-1][0], costs[i-1][2])
            costs[i][2] += min(costs[i-1][0], costs[i-1][1])

        return min(costs[-1])
```

• LeetDoocs

```
class Solution:
    def minCost(self, costs: List[List[int]]) -> int:
        a = b = c = 0
        for ca, cb, cc in costs:
            a, b, c = min(b, c) + ca, min(a, c) + cb, min(a, b) + cc
        return min(a, b, c)
```

• SimplyLeet

```
# Python solution for the Paint House problem

def minCost(costs):
    # Get the number of houses
    n = len(costs)
    # Base case: If there are no houses, return 0 cost
    if n == 0:
        return 0

    # Iterate from the second house to the last house

    for i in range(1, n):
        # Update the cost for painting the current house red
        costs[i][0] += min(costs[i-1][1], costs[i-1][2])
        # Update the cost for painting the current house blue
        costs[i][1] += min(costs[i-1][0], costs[i-1][2])
        # Update the cost for painting the current house green
        costs[i][2] += min(costs[i-1][0], costs[i-1][1])

    # The answer is the minimum cost of painting the last house with any of the colors
    return min(costs[-1])

# Example usage:
example_costs = [[17,2,17],[16,16,5],[14,3,19]]
print(minCost(example_costs)) # Output: 10
```

• LC.ca

```
class Solution:
    def minCost(self, costs: List[List[int]]) -> int:
        a = b = c = 0
        for ca, cb, cc in costs:
            a, b, c = min(b, c) + ca, min(a, c) + cb, min(a, b) + cc
        return min(a, b, c)

#####
...

```

```

Two-dimensional dynamic array dp,
where dp[i][j] represents the minimum cost of brushing to the i+1th house with
color j,

and the state transition equation is:
dp[i][j] = cost[i][j] + min(dp[i-1][(j + 1)% 3], dp[i-1][(j + 2)% 3])

...
class Solution(object):
    def minCost(self, costs):
        """
        :type costs: List[List[int]]
        :rtype: int
        """
        if not costs:
            return 0
        dp = [0] * (len(costs[0]))
        dp[:] = costs[0]
        for i in range(1, len(costs)):
            d0 = d1 = d2 = 0
            for j in range(0, 3):
                if j == 0:
                    d0 = min(dp[1], dp[2]) + costs[i][0]
                elif j == 1:
                    d1 = min(dp[0], dp[2]) + costs[i][1]
                else:
                    d2 = min(dp[0], dp[1]) + costs[i][2]
            dp[0], dp[1], dp[2] = d0, d1, d2
        return min(dp)

```

◆ Quick Review

Key Insights

- The choice for painting each house depends on the previous house's color choices.

- Use dynamic programming to accumulate the cost while ensuring adjacent houses do not have the same color.
- The state of each house can be represented by the minimum cost when the house is painted a specific color, computed using the minimal costs of the previous house from the other two colors.
- The problem only requires updating the cost matrix in-place for optimal space usage.

Space & Time

Time Complexity: $O(n)$, where n is the number of houses.

Space Complexity: $O(1)$ extra space if the input matrix is modified in place.

Solution

We use a dynamic programming approach with iterative in-place updating of the cost matrix. For each house starting from the second one, we update the cost for each color by adding the minimal cost of painting the previous house with either of the other two colors. This guarantees we never use the same color for adjacent houses while accumulating the

minimal total cost. The final answer is the minimum value for the last house.

2D Grid DP

□ #931 Minimum Falling Path Sum

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Matrix
Lists: AlgoMaster 600

Problem

Given an $n \times n$ array of integers matrix, return the minimum sum of any falling path through matrix.

A falling path starts at any element in the first row and chooses the element in the next row that is either directly below or diagonally left/right. Specifically, the next element from position (row, col) will be $(row + 1, col - 1)$, $(row + 1, col)$, or $(row + 1, col + 1)$.

Example 1:

2	1	3
6	5	4
7	8	9

2	1	3
6	5	4
7	8	9

2	1	3
6	5	4
7	8	9

Input: matrix = [[2,1,3],[6,5,4],[7,8,9]]

Output: 13

Explanation: There are two falling paths with a minimum sum as shown.

Example 2:

Round 8 · Low · Medium

p.3033

-19	57
-40	-5

-19	57
-40	-5

Input: matrix = [[-19,57],[-40,-5]]

Output: -59

Explanation: The falling path with a minimum sum is shown.

Constraints:

- $n == \text{matrix.length} == \text{matrix}[i].\text{length}$
- $1 \leq n \leq 100$
- $-100 \leq \text{matrix}[i][j] \leq 100$

Community Solutions (Python)

- WalkCC

Round 8 · Low · Medium

p.3034

```
class Solution:
    def minFallingPathSum(self, A: list[list[int]]) -> int:
        n = len(A)

        for i in range(1, n):
            for j in range(n):
                mn = math.inf
                for k in range(max(0, j - 1), min(n, j + 2)):
                    mn = min(mn, A[i - 1][k])
                A[i][j] += mn

        return min(A[-1])
```

• LeetCode

```
class Solution:
    def minFallingPathSum(self, matrix: List[List[int]]) -> int:
        n = len(matrix)
        f = [0] * n
        for row in matrix:
            g = [0] * n
            for j, x in enumerate(row):
                l, r = max(0, j - 1), min(n, j + 2)
                g[j] = min(f[l:r]) + x
            f = g

        return min(f)
```

• SimplyLeet

```
# Python solution with line-by-line comments

def minFallingPathSum(matrix):
    # Get the dimension of the matrix
    n = len(matrix)

    # Process the matrix from second last row to the first row
    for row in range(n - 2, -1, -1):
        for col in range(n):
            # Initialize the minimum sum from the next row as a large number.
```

```
best_below = matrix[row + 1][col]
# Check the diagonal left if it is within bounds
if col - 1 >= 0:
    best_below = min(best_below, matrix[row + 1][col - 1])
# Check the diagonal right if it is within bounds
if col + 1 < n:
    best_below = min(best_below, matrix[row + 1][col + 1])
# Update the current cell with the sum of its value and the best
move below
matrix[row][col] += best_below

# The result is the minimum value in the first row after processing
return min(matrix[0])

# Example usage:
matrix = [[2,1,3],[6,5,4],[7,8,9]]
print(minFallingPathSum(matrix)) # Expected output: 13
```

2D Grid DP

☐ #1277 Count Square Submatrices with All Ones

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Dynamic Programming · Matrix
 Lists: AlgoMaster 600

Problem

Given a $m * n$ matrix of ones and zeros, return how many square submatrices have all ones.

Example 1:

Input: matrix =
 [
 [0,1,1,1],
 [1,1,1,1],
 [0,1,1,1]
]
 Output: 15
 Explanation:

Example 2:

Input: matrix =
 [
 [1,0,1],
 [1,1,0],
 [1,1,0]
]
 Output: 7
 Explanation:

Constraints:

- $1 \leq \text{arr.length} \leq 300$

Round 8 · Low · Medium

p.3037

- $1 \leq \text{arr}[0].\text{length} \leq 300$
- $0 \leq \text{arr}[i][j] \leq 1$

Community Solutions (Python)

☐ • WalkCC

```
class Solution:
    def countSquares(self, matrix: List[List[int]]) -> int:
        for i in range(len(matrix)):
            for j in range(len(matrix[0])):
                if matrix[i][j] == 1 and i > 0 and j > 0:
                    matrix[i][j] += min(matrix[i - 1][j - 1],
                                         matrix[i - 1][j], matrix[i][j - 1])
                return sum(map(sum, matrix))
```

☐ • LeetDoocs

```
class Solution:
    def countSquares(self, matrix: List[List[int]]) -> int:
        m, n = len(matrix), len(matrix[0])
        f = [[0] * n for _ in range(m)]
        ans = 0
        for i, row in enumerate(matrix):
            for j, v in enumerate(row):
                if v == 0:
                    continue
                if i == 0 or j == 0:
                    f[i][j] = 1
                else:
                    f[i][j] = min(f[i - 1][j - 1], f[i - 1][j], f[i][j - 1]) + 1
                ans += f[i][j]
        return ans
```

☐ • SimplyLeet

Round 8 · Low · Medium

p.3038

```
# Function to count square submatrices with all ones
def countSquares(matrix):
    # Get dimensions of matrix
    rows = len(matrix)
    cols = len(matrix[0])
    total_squares = 0

    # Iterate over each cell in the matrix
    for i in range(rows):
        for j in range(cols):

            # If the cell is not in the first row or first column and is a 1
            if matrix[i][j] == 1 and i > 0 and j > 0:
                # Determine the size of the largest square ending at (i, j)
                matrix[i][j] = min(matrix[i-1][j], matrix[i][j-1], matrix[i-1]
[j-1]) + 1

            # Add current value to the total count of squares
            total_squares += matrix[i][j]

    return total_squares

# Example usage
example_matrix = [

    [0, 1, 1, 1],
    [1, 1, 1, 1],
    [0, 1, 1, 1]
]
print("Total squares:", countSquares(example_matrix))
```

2D Grid DP

#1937 Maximum Number of Points with Cost

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Matrix
Lists: AlgoMaster 600

Problem

You are given an $m \times n$ integer matrix `points` (0-indexed). Starting with 0 points, you want to maximize the number of points you can get from the matrix.

To gain points, you must pick one cell in each row. Picking the cell at coordinates (r, c) will add `points[r][c]` to your score.

However, you will lose points if you pick a cell too far from the cell that you picked in the previous row. For every two adjacent rows r and $r + 1$ (where $0 \leq r < m - 1$), picking cells at coordinates $(r, c1)$ and $(r + 1, c2)$ will subtract $\text{abs}(c1 - c2)$ from your score.

Return the maximum number of points you can achieve.

abs(x) is defined as:

- x for $x \geq 0$.
- $-x$ for $x < 0$.

Example 1:

1	2	3
1	5	1
3	1	1

Input: points = `[[1,2,3],[1,5,1],[3,1,1]]`

Output: 9

Explanation:

The blue cells denote the optimal cells to pick, which have coordinates (0, 2), (1, 1), and (2, 0).

You add $3 + 5 + 3 = 11$ to your score.

However, you must subtract $\text{abs}(2 - 1) + \text{abs}(1 - 0) = 2$ from your score.

Your final score is $11 - 2 = 9$.

Round 8 · Low · Medium

p.3041

Example 2:

1	5
2	3
4	2

Input: points = `[[1,5],[2,3],[4,2]]`

Output: 11

Explanation:

The blue cells denote the optimal cells to pick, which have coordinates (0, 1), (1, 1), and (2, 0).

You add $5 + 3 + 4 = 12$ to your score.

However, you must subtract $\text{abs}(1 - 1) + \text{abs}(1 - 0) = 1$ from your score.

Your final score is $12 - 1 = 11$.

Constraints:

- $m == \text{points.length}$
- $n == \text{points}[r].\text{length}$

Round 8 · Low · Medium

p.3042

- $1 \leq m, n \leq 105$
- $1 \leq m * n \leq 105$
- $0 \leq \text{points}[r][c] \leq 105$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxPoints(self, points: list[list[int]]) -> int:
        n = len(points[0])
        # dp[j] := the maximum number of points you can have if points[i][j] is the
        # most recent cell you picked
        dp = [0] * n

        for row in points:
            leftToRight = [0] * n
            runningMax = 0

            for j in range(n):
                runningMax = max(runningMax - 1, dp[j])
                leftToRight[j] = runningMax

            rightToLeft = [0] * n
            runningMax = 0
            for j in range(n - 1, -1, -1):
                runningMax = max(runningMax - 1, dp[j])
                rightToLeft[j] = runningMax

            for j in range(n):
                dp[j] = max(leftToRight[j], rightToLeft[j]) + row[j]

        return max(dp)
```

• LeetCode

```
class Solution:
    def maxPoints(self, points: List[List[int]]) -> int:
        n = len(points[0])
        f = points[0][:]
        for p in points[1:]:
            g = [0] * n
            lmx = -inf
            for j in range(n):
                lmx = max(lmx, f[j] + j)
                g[j] = max(g[j], p[j] + lmx - j)

            rmx = -inf
            for j in range(n - 1, -1, -1):
                rmx = max(rmx, f[j] - j)
                g[j] = max(g[j], p[j] + rmx + j)
            f = g
        return max(f)
```

• SimplyLeet

```
# Function to calculate the maximum number of points
def maxPoints(points):
    m = len(points)
    n = len(points[0])
    # dp will track the maximum score ending in each column for the previous
    row
    dp = points[0][:] # Copy the first row as initial state

    # Process each row from the second row onward
    for i in range(1, m):
        # Create helper arrays to capture best previous scores adjusting for
        movement costs

        left = [0] * n
        right = [0] * n

        # Build the left helper array from left to right
        left[0] = dp[0]
        for j in range(1, n):
            left[j] = max(left[j - 1] - 1, dp[j])

        # Build the right helper array from right to left
        right[n - 1] = dp[n - 1]
```

```
for j in range(n - 2, -1, -1):
    right[j] = max(right[j + 1] - 1, dp[j])

# Update dp for the current row using the best scores from left and
right arrays
new_dp = [0] * n
for j in range(n):
    new_dp[j] = points[i][j] + max(left[j], right[j])
dp = new_dp # Move to the next row

# The answer is the maximum value in the final dp row

return max(dp)
```

```
# Example usage:
if __name__ == '__main__':
    points_example = [[1,2,3],[1,5,1],[3,1,1]]
    print(maxPoints(points_example)) # Expected output: 9
```

Round 8 · Low · Medium**p.3045**

String DP

Round 8 · Low · Medium · 7 questions**Round 8 · Low · Medium****p.3046**

String DP

#72 Edit Distance

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming
Lists: Denny Zhang

Problem

Given two strings word1 and word2, return the minimum number of operations required to convert word1 to word2.

You have the following three operations permitted on a word:

- Insert a character
- Delete a character
- Replace a character

Example 1:

```
Input: word1 = "horse", word2 = "ros"
Output: 3
Explanation:
horse -> rorse (replace 'h' with 'r')
rorse -> rose (remove 'r')
rose -> ros (remove 'e')
```

Example 2:

```
Input: word1 = "intention", word2 = "execution"
Output: 5
Explanation:
intention -> inention (remove 't')
inention -> enention (replace 'i' with 'e')
enention -> exention (replace 'n' with 'x')
exention -> exection (replace 'n' with 'c')
exection -> execution (insert 'u')
```

Constraints:

- $0 \leq \text{word1.length}, \text{word2.length} \leq 500$
- word1 and word2 consist of lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minDistance(self, word1: str, word2: str) -> int:
        m = len(word1)
        n = len(word2)
        # dp[i][j] := the minimum number of operations to convert word1[0..i) to
        # word2[0..j)
        dp = [[0] * (n + 1) for _ in range(m + 1)]

        for i in range(1, m + 1):
            dp[i][0] = i

        for j in range(1, n + 1):
            dp[0][j] = j

        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if word1[i - 1] == word2[j - 1]:
                    dp[i][j] = dp[i - 1][j - 1]
                else:
                    dp[i][j] = min(dp[i - 1][j - 1], dp[i - 1][j], dp[i][j - 1]) + 1
```



```
return dp[m][n]
```

• LeetDoocs

```
class Solution:
    def minDistance(self, word1: str, word2: str) -> int:
        m, n = len(word1), len(word2)
        f = [[0] * (n + 1) for _ in range(m + 1)]
        for j in range(1, n + 1):
            f[0][j] = j
        for i, a in enumerate(word1, 1):
            f[i][0] = i
            for j, b in enumerate(word2, 1):
                if a == b:
                    f[i][j] = f[i - 1][j - 1]
                else:
                    f[i][j] = min(f[i - 1][j], f[i][j - 1], f[i - 1][j - 1]) + 1
        return f[m][n]
```

• SimplyLeet

```
# Python solution to compute the Edit Distance
def minDistance(word1, word2):
    m = len(word1)
    n = len(word2)

    # Initialize the DP table with size (m+1) x (n+1)
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    # Base cases: transform word1[0:i] to empty word2 requires i deletions
    for i in range(m + 1):
```

```
    dp[i][0] = i
    # Base cases: transform empty word1 to word2[0:j] requires j insertions
    for j in range(n + 1):
        dp[0][j] = j

    # Fill up the dp table
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            # If the characters are the same, no new operation is needed.
            if word1[i - 1] == word2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                # consider insertion, deletion, and replacement
                dp[i][j] = 1 + min(dp[i][j - 1], # Insert
                                   dp[i - 1][j], # Delete
                                   dp[i - 1][j - 1]) # Replace

    return dp[m][n]

# Example usage:
print(minDistance("horse", "ros"))
```

• LC.ca

```
class Solution:
    def minDistance(self, word1: str, word2: str) -> int:
        m, n = len(word1), len(word2)
        dp = [[0] * (n + 1) for _ in range(m + 1)]
        for i in range(m + 1):
            dp[i][0] = i
        for j in range(n + 1):
            dp[0][j] = j
        for i in range(1, m + 1):
            for j in range(1, n + 1):
```

```

        if word1[i - 1] == word2[j - 1]:
            dp[i][j] = dp[i - 1][j - 1]
        else:
            # dp[i - 1][j - 1] meaning replace.
            # e.g. "abc" and "bf", checking "ab" and "b" distance,
            # then replace either way for "c" or "f"
            dp[i][j] = 1 + min(dp[i][j - 1], dp[i - 1][j], dp[i - 1][j
- 1])
        return dp[-1][-1]

#####

class Solution:
    def minDistance(self, word1: str, word2: str) -> int:
        m, n = len(word1), len(word2)
        f = [[0] * (n + 1) for _ in range(m + 1)]
        for j in range(1, n + 1):
            f[0][j] = j
        for i, a in enumerate(word1, 1):
            f[i][0] = i # merged for loops from above solution, but not good
            for readability
            for j, b in enumerate(word2, 1):

                if a == b:
                    f[i][j] = f[i - 1][j - 1]
                else:
                    f[i][j] = min(f[i - 1][j], f[i][j - 1], f[i - 1][j - 1]) +
1
        return f[m][n]

```

◆ Quick Review

Key Insights

- This is a classic dynamic programming problem.
- Use a 2D DP table where $dp[i][j]$ represents the minimum edit distance between

Round 8 · Low · Medium

p.3051

$word1[0...i-1]$ and $word2[0...j-1]$.

- **Base cases:** converting an empty string to a string of length j requires j insertions, and vice versa.
- **Transition:** If characters match then $dp[i][j] = dp[i-1][j-1]$, otherwise take minimum among insertion, deletion, and replacement operations and add 1.

Space & Time

Time Complexity: $O(mn)$, where m and n are the lengths of $word1$ and $word2$ respectively.

Space Complexity: $O(mn)$ due to the DP table used for storing computed distances.

Solution

We use a dynamic programming approach by constructing a 2D table dp of size $(m+1) \times (n+1)$, where m and n are the lengths of $word1$ and $word2$. Each $dp[i][j]$ holds the minimum number of operations needed to convert the first i characters of $word1$ to the first j characters of $word2$. The algorithm includes the following steps:

Round 8 · Low · Medium

p.3052

– Initialize the base cases: $dp[0][j] = j$ (converting an empty word1 to j characters of word2 requires j insertions) $dp[i][0] = i$ (converting i characters of word1 to an empty word2 requires i deletions) – Iterate over each character in word1 and word2: If the current characters are equal, no additional operation is needed and $dp[i][j] = dp[i-1][j-1]$. Otherwise, choose the best operation among insertion ($dp[i][j-1]$), deletion ($dp[i-1][j]$), or substitution ($dp[i-1][j-1]$) and add 1. – The answer is found in $dp[m][n]$.

This method reliably computes the edit distance using a systematic DP approach, ensuring that all subproblems are solved and reused efficiently.

String DP

□ #91 Decode Ways

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming
Lists: Grind 169

Problem

You have intercepted a secret message encoded as a string of numbers. The message is decoded via the following mapping:

"1" -> 'A' "2" -> 'B' ... "25" -> 'Y' "26" -> 'Z'

However, while decoding the message, you realize that there are many different ways you can decode the message because some codes are contained in other codes ("2" and "5" vs "25").

For example, "11106" can be decoded into:

- "AAJF" with the grouping (1, 1, 10, 6)
- "KJF" with the grouping (11, 10, 6)
- The grouping (1, 11, 06) is invalid because "06" is not a valid code (only "6" is valid).

Note: there may be strings that are impossible to decode. Given a string *s* containing only digits, return the number of ways to decode it. If the entire string cannot be decoded in any valid way, return 0.

The test cases are generated so that the answer fits in a 32-bit integer.

Example 1:

Input: *s* = "12"

Output: 2

Explanation:

"12" could be decoded as "AB" (1 2) or "L" (12).

Example 2:

Input: *s* = "226"

Output: 3

Explanation:

"226" could be decoded as "BZ" (2 26), "VF" (22 6), or "BBF" (2 2 6).

Example 3:

Input: *s* = "06"

Output: 0

Explanation:

"06" cannot be mapped to "F" because of the leading zero ("6" is different from "06"). In this case, the string is not a valid encoding, so return 0.

Constraints:

- $1 \leq \text{s.length} \leq 100$
- *s* contains only digits and may contain leading zero(s).

Community Solutions (Python)

• WalkCC

```
class Solution:
    def numDecodings(self, s: str) -> int:
        n = len(s)
        # dp[i] := the number of ways to decode s[i..n]
        dp = [0] * n + [1]

        def isValid(a: str, b=None) -> bool:
            if b:
                return a == '1' or a == '2' and b < '7'
            return a != '0'

        if isValid(s[-1]):
            dp[n - 1] = 1

        for i in reversed(range(n - 1)):
            if isValid(s[i]):
                dp[i] += dp[i + 1]
            if isValid(s[i], s[i + 1]):
                dp[i] += dp[i + 2]

        return dp[0]
```

• LeetCode

```
class Solution:
    def numDecodings(self, s: str) -> int:
        n = len(s)
        f = [1] + [0] * n
        for i, c in enumerate(s, 1):
            if c != "0":
                f[i] = f[i - 1]
            if i > 1 and s[i - 2] != "0" and int(s[i - 2 : i]) <= 26:
                f[i] += f[i - 2]
        return f[n]
```

```
class Solution:
    def numDecodings(self, s: str) -> int:
        f, g = 0, 1
        for i, c in enumerate(s, 1):
            h = g if c != "0" else 0
            if i > 1 and s[i - 2] != "0" and int(s[i - 2 : i]) <= 26:
                h += f
            f, g = g, h
        return g
```

• SimplyLeet

```
# Python solution using dynamic programming
def num_decodings(s: str) -> int:
    # If the string starts with '0', it's not decodable
    if not s or s[0] == '0':
        return 0

    # dp array where dp[i] represents the number of ways to decode s[:i]
    n = len(s)
    dp = [0] * (n + 1)
```

```
# Base case: an empty string has one decoding
dp[0] = 1

for i in range(1, n + 1):
    # Check for single digit decoding (s[i-1] is valid if not '0')
    if s[i - 1] != '0':
        dp[i] += dp[i - 1]
    # Check for two-digit decoding (if i>=2, s[i-2:i] forms a valid number
    # between 10 and 26)
    if i >= 2 and s[i - 2] != '0':
        two_digit = int(s[i - 2:i])
```

```
        if 10 <= two_digit <= 26:
            dp[i] += dp[i - 2]
    return dp[n]
```

```
# Example usage:
print(num_decodings("12")) # Expected output: 2
```

• LC.ca

```
'''
>>> s = "abcdef"
>>> [(i,c) for i, c in enumerate(s, 1)]
[(1, 'a'), (2, 'b'), (3, 'c'), (4, 'd'), (5, 'e'), (6, 'f')]
'''
```

```
class Solution:
    def numDecodings(self, s: str) -> int:
        # f: i-2
        # g: i-1
        f, g = 0, 1

        for i, c in enumerate(s, 1):
            h = g if c != "0" else 0
            if i > 1 and s[i - 2] != "0" and int(s[i - 2 : i]) <= 26:
                h += f
            f, g = g, h
        return g
```

```
#####
```

```
class Solution:
```

```
def numDecodings(self, s: str) -> int:
    if len(s) == 0:
        return 0

    length = len(s)
    dp = [0] * (length + 1) # dp[i] => at index i, its decode ways

    dp[length] = 1 # initiator, just make forloop flow working
    dp[length - 1] = 0 if s[length - 1] == '0' else 1 # assumption is
    starting at i, so starting as 0 is not decodable

    # start at 'len-2'
    # or else below 'dp[i+2]' will out of index boundary
    for i in range(length - 2, -1, -1):
        if s[i] == '0':
            continue
        tem = int(s[i:i + 2])

        if tem > 26:
            dp[i] = dp[i + 1]
        else:
            dp[i] = dp[i + 1] + dp[i + 2]

    return dp[0]

# optimized from above solution, no dp array
class Solution:
    def numDecodings(self, s: str) -> int:
        n = len(s)
        # a: dp[i-2], b: dp[i-1], c: count for current index i
        a, b, c = 0, 1, 0
        for i in range(1, n + 1):
```

```
        c = 0
        if s[i - 1] != '0':
            c += b
        if i > 1 and s[i - 2] != '0' and (int(s[i - 2]) * 10 + int(s[i - 1]
    ]) <= 26):
            c += a
            a, b = b, c
        return c

#####

class Solution:
    def numDecodings(self, s: str) -> int:
        n = len(s)
        dp = [0] * (n + 1)
        dp[0] = 1
        for i in range(1, n + 1):
            if s[i - 1] != '0':
                dp[i] += dp[i - 1]
            if i > 1 and s[i - 2] != '0' and (int(s[i - 2]) * 10 + int(s[i - 1]
    ]) <= 26):
                dp[i] += dp[i - 2]

        return dp[n]
```

◆ Quick Review

Key Insights

- A digit '0' cannot be mapped on its own; it must be part of "10" or "20".
- Use dynamic programming where $dp[i]$ represents the number of ways to decode the substring $s[0...i-1]$.

- For each index i , if the digit at position $i-1$ is non-zero, add $dp[i-1]$ to $dp[i]$.
- If the two-digit number formed by $s[i-2]$ and $s[i-1]$ is between 10 and 26, add $dp[i-2]$ to $dp[i]$.
- Initialize $dp[0]$ as 1 (an empty string has one way to be decoded).

Space & Time

Time Complexity: $O(n)$, where n is the length of the string.

Space Complexity: $O(n)$, due to the dp array (which can be optimized to $O(1)$).

Solution

We solve the problem using a dynamic programming approach. We initialize a dp array where $dp[i]$ indicates the number of ways to decode the substring $s[0...i-1]$. We set $dp[0] = 1$ since an empty string is trivially decodable. We loop through the string, and at each step:

- If $s[i-1]$ is not '0', we add $dp[i-1]$ to $dp[i]$ because $s[i-1]$ can be mapped to a valid letter.
- If the substring $s[i-2:i]$ (for $i \geq 2$) forms a valid two-digit number between 10 and 26, we

add $dp[i-2]$ to $dp[i]$. This cumulative approach yields the total ways to decode the string, and if the string starts with '0' or contains invalid sequences, $dp[n]$ will eventually be 0.

String DP

#97 Interleaving String

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming
Lists: NeetCode 150

Problem

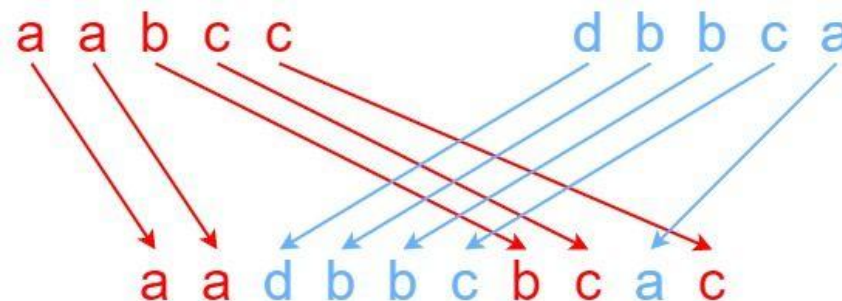
Given strings s_1 , s_2 , and s_3 , find whether s_3 is formed by an interleaving of s_1 and s_2 .

An interleaving of two strings s and t is a configuration where s and t are divided into n and m substrings respectively, such that:

- $s = s_1 + s_2 + \dots + s_n$
- $t = t_1 + t_2 + \dots + t_m$
- $|n - m| \leq 1$
- The interleaving is $s_1 + t_1 + s_2 + t_2 + s_3 + t_3 + \dots$ or $t_1 + s_1 + t_2 + s_2 + t_3 + s_3 + \dots$

Note: $a + b$ is the concatenation of strings a and b .

Example 1:



Input: $s_1 = \text{"aabcc"}$, $s_2 = \text{"dbbca"}$, $s_3 = \text{"aadbcbcbac"}$
Output: true
Explanation: One way to obtain s_3 is:
Split s_1 into $s_1 = \text{"aa"} + \text{"bc"} + \text{"c"}$, and s_2 into $s_2 = \text{"dbbc"} + \text{"a"}$.
Interleaving the two splits, we get $\text{"aa"} + \text{"dbbc"} + \text{"bc"} + \text{"a"} + \text{"c"} = \text{"aadbcbcbac"}$.
Since s_3 can be obtained by interleaving s_1 and s_2 , we return true.

Example 2:

Input: $s_1 = \text{"aabcc"}$, $s_2 = \text{"dbbca"}$, $s_3 = \text{"aadbcbacc"}$
Output: false
Explanation: Notice how it is impossible to interleave s_2 with any other string to obtain s_3 .

Example 3:

Input: $s_1 = \text{""}$, $s_2 = \text{""}$, $s_3 = \text{""}$
Output: true

Constraints:

- $0 \leq s_1.\text{length}, s_2.\text{length} \leq 100$
- $0 \leq s_3.\text{length} \leq 200$
- s_1 , s_2 , and s_3 consist of lowercase English letters.

Follow up: Could you solve it using only $O(s_2.\text{length})$ additional memory space?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isInterleave(self, s1: str, s2: str, s3: str) -> bool:
        m = len(s1)
        n = len(s2)
        if m + n != len(s3):
            return False

        # dp[i][j] := true if s3[0..i+j] is formed by the interleaving of
        # s1[0..i] and s2[0..j]
        dp = [[False] * (n + 1) for _ in range(m + 1)]

        dp[0][0] = True

        for i in range(1, m + 1):
            dp[i][0] = dp[i - 1][0] and s1[i - 1] == s3[i - 1]

        for j in range(1, n + 1):
            dp[0][j] = dp[0][j - 1] and s2[j - 1] == s3[j - 1]

        for i in range(1, m + 1):
            for j in range(1, n + 1):
                dp[i][j] = (dp[i - 1][j] and s1[i - 1] == s3[i + j - 1] or
                           dp[i][j - 1] and s2[j - 1] == s3[i + j - 1])

        return dp[m][n]
```

• LeetCode

```
class Solution:
    def isInterleave(self, s1: str, s2: str, s3: str) -> bool:
        @cache
        def dfs(i: int, j: int) -> bool:
            if i >= m and j >= n:
                return True
            k = i + j
            if i < m and s1[i] == s3[k] and dfs(i + 1, j):
                return True
            if j < n and s2[j] == s3[k] and dfs(i, j + 1):
                return True
            return False

        m, n = len(s1), len(s2)
        if m + n != len(s3):
            return False
        return dfs(0, 0)

class Solution:
    def isInterleave(self, s1: str, s2: str, s3: str) -> bool:
        m, n = len(s1), len(s2)
        if m + n != len(s3):
            return False
        f = [[False] * (n + 1) for _ in range(m + 1)]
        f[0][0] = True
        for i in range(m + 1):
            for j in range(n + 1):
                k = i + j - 1
                if i and s1[i - 1] == s3[k]:
                    f[i][j] = f[i - 1][j]
                if j and s2[j - 1] == s3[k]:
                    f[i][j] |= f[i][j - 1]
        return f[m][n]
```

• SimplyLeet

Round 8 · Low · Medium

p.3065

Round 8 · Low · Medium

p.3066

```
def isInterleave(s1, s2, s3):
    # Check if the combined lengths of s1 and s2 match s3
    if len(s1) + len(s2) != len(s3):
        return False
    # Initialize dp table with dimensions (len(s1)+1) x (len(s2)+1)
    dp = [[False] * (len(s2) + 1) for _ in range(len(s1) + 1)]
    dp[0][0] = True # Base case: empty strings form an empty string
    # Fill first row using characters from s2 only
    for j in range(1, len(s2) + 1):
        dp[0][j] = dp[0][j - 1] and s2[j - 1] == s3[j - 1]

    # Fill first column using characters from s1 only
    for i in range(1, len(s1) + 1):
        dp[i][0] = dp[i - 1][0] and s1[i - 1] == s3[i - 1]
    # Fill the rest of the dp table
    for i in range(1, len(s1) + 1):
        for j in range(1, len(s2) + 1):
            dp[i][j] = (dp[i - 1][j] and s1[i - 1] == s3[i + j - 1]) or \
                (dp[i][j - 1] and s2[j - 1] == s3[i + j - 1])
    return dp[len(s1)][len(s2)]

# Example usage:
print(isInterleave("aabcc", "dbbca", "aadbcbcbac")) # Expected output: True
print(isInterleave("aabcc", "dbbca", "aadbbaacc")) # Expected output: False
```

String DP

□ #139 Word Break

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table · String · Dynamic
 Programming · Trie · Memoization
 Lists: Grind 169

Problem

Given a string *s* and a dictionary of strings *wordDict*, return true if *s* can be segmented into a space-separated sequence of one or more dictionary words.

Note that the same word in the dictionary may be reused multiple times in the segmentation.

Example 1:

Input: *s* = "leetcode", *wordDict* = ["leet","code"]
 Output: true
 Explanation: Return true because "leetcode" can be segmented as "leet code".

Example 2:

Input: *s* = "applepenapple", *wordDict* = ["apple","pen"]
 Output: true
 Explanation: Return true because "applepenapple" can be segmented as "apple pen apple".
 Note that you are allowed to reuse a dictionary word.

Example 3:

Input: s = "catsandog", wordDict = ["cats","dog","sand","and","cat"]
Output: false

Constraints:

- $1 \leq s.length \leq 300$
- $1 \leq wordDict.length \leq 1000$
- $1 \leq wordDict[i].length \leq 20$
- s and wordDict[i] consist of only lowercase English letters.
- All the strings of wordDict are unique.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def wordBreak(self, s: str, wordDict: list[str]) -> bool:
        wordSet = set(wordDict)

        @functools.lru_cache(None)
        def wordBreak(s: str) -> bool:
            """Returns True if s can be segmented."""
            if s in wordSet:
                return True
            return any(s[:i] in wordSet and wordBreak(s[i:]) for i in range(len(s)))

        return wordBreak(s)
```

```
class Solution:
    def wordBreak(self, s: str, wordDict: list[str]) -> bool:
        n = len(s)
        wordSet = set(wordDict)
        # dp[i] := True if s[0..i] can be segmented
        dp = [True] + [False] * n

        for i in range(1, n + 1):
            for j in range(i):
                # s[0..j] can be segmented and s[j..i] is in `wordSet`, so s[0..i] can
                # be segmented.
                if dp[j] and s[j:i] in wordSet:
                    dp[i] = True
                    break

        return dp[n]

class Solution:
    def wordBreak(self, s: str, wordDict: list[str]) -> bool:
        n = len(s)
        maxLength = len(max(wordDict, key=len))
        wordSet = set(wordDict)
        # dp[i] := True if s[0..i] can be segmented
        dp = [True] + [False] * n

        for i in range(1, n + 1):
            for j in range(i - 1, -1, -1):
                if i - j > maxLength:
                    break
                # s[0..j] can be segmented and s[j..i] is in the wordSet, so s[0..i]
                # can be segmented.
                if dp[j] and s[j:i] in wordSet:
                    dp[i] = True
                    break

        return dp[n]
```

• LeetCode

```
class Solution:
    def wordBreak(self, s: str, wordDict: List[str]) -> bool:
        words = set(wordDict)
        n = len(s)
        f = [True] + [False] * n
        for i in range(1, n + 1):
            f[i] = any(f[j] and s[j:i] in words for j in range(i))
        return f[n]
```

```
class Trie:
    def __init__(self):
        self.children: List[Trie | None] = [None] * 26
        self.isEnd = False
```

```
    def insert(self, w: str):
        node = self
        for c in w:
            idx = ord(c) - ord('a')
            if not node.children[idx]:
```

```
                node.children[idx] = Trie()
            node = node.children[idx]
        node.isEnd = True
```

```
class Solution:
    def wordBreak(self, s: str, wordDict: List[str]) -> bool:
        trie = Trie()
        for w in wordDict:
            trie.insert(w)
```

```
        n = len(s)
        f = [False] * (n + 1)
        f[n] = True
        for i in range(n - 1, -1, -1):
            node = trie
            for j in range(i, n):
                idx = ord(s[j]) - ord('a')
                if not node.children[idx]:
                    break
                node = node.children[idx]
```

Round 8 · Low · Medium

p.3071

```
        if node.isEnd and f[j + 1]:
            f[i] = True
            break
        return f[0]
```

• SimplyLeet

Python solution for the Word Break problem

```
def wordBreak(s, wordDict):
    # Convert the list to a set for O(1) look-ups
    wordSet = set(wordDict)

    # dp[i] is True if s[0:i] can be segmented into words from the wordDict
    n = len(s)
    dp = [False] * (n + 1)
```

```
    # Base case: empty string is always segmentable
    dp[0] = True
```

```
    # Iterate through the string positions
    for i in range(1, n + 1):
        # Check every possible partition s[0:j] and s[j:i]
        for j in range(i):
            # If s[0:j] can be segmented and s[j:i] is in the dictionary
            if dp[j] and s[j:i] in wordSet:
                dp[i] = True # s[0:i] can be segmented
```

```
        break # No need to check further partitions
    return dp[n]
```

```
# Example usage:
s = "leetcode"
wordDict = ["leet", "code"]
print(wordBreak(s, wordDict)) # Output: True
```

• LC.ca

Round 8 · Low · Medium

p.3072

```
class Solution:
    def wordBreak(self, s: str, wordDict: List[str]) -> bool:
        words = set(wordDict)
        n = len(s)

        # dp[j] meaning s from 0 to index=i-1 is breakable
        dp = [True] + [False] * n # so actually size is n+1
        for i in range(1, n + 1):
            dp[i] = any( (dp[j] and s[j:i] in words) for j in range(i) )
        return dp[n]
```

#####

```
class Solution:
    def wordBreak(self, s: str, wordDict: Set[str]) -> bool:
        if not s or not wordDict:
            return False

        length = len(s)
```

```
# construct dp, dp[i] meaning position i-1 can from dict or not
dp[-1] meaning at last-index plus 1, checking up to last-index
dp = [False] * (length + 1)
dp[0] = True # initiate
```

```
for i in range(length + 1):
    if not dp[i]:
        continue
    for word in wordDict:
        if i + len(word) > length:
```

```
        continue
        if s[i:i + len(word)] == word:
            dp[i + len(word)] = True
```

```
    return dp[length]
```

#####

```
class Solution:
    def wordBreak(self, s: str, wordDict: List[str]) -> bool:
```

```
words = set(wordDict)
n = len(s)
dp = [False] * (n + 1)
dp[0] = True # i'th char in string, is good
for j in range(1, n + 1):
    for i in range(j): # starting at 0, including dp[0]
        if dp[i] and s[i:j] in words:
            dp[j] = True # j is exclusive, meaning True until index
            break
return dp[-1]
```

#####

```
class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.is_end = False
```

```
    def insert(self, w):
```

```
        node = self
        for c in w:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
        node.is_end = True
```

```
    def search(self, w):
        node = self
```

```

    for c in w:
        idx = ord(c) - ord('a')
        if node.children[idx] is None:
            return False
        node = node.children[idx]
    return node.is_end

class Solution:
    def wordBreak(self, s: str, wordDict: List[str]) -> bool:
        # https://docs.python.org/3/library/functools.html#functools.cache

        # creating a thin wrapper around a dictionary lookup for the function
        arguments
        @cache
        def dfs(s):
            return not s or any(trie.search(s[:i]) and dfs(s[i:]) for i in range(1, len(s) + 1))

        trie = Trie()
        for w in wordDict:
            trie.insert(w)
        return dfs(s)

#####

class Solution(object):
    def wordBreak(self, s, wordDict):
        """
        :type s: str
        :type wordDict: Set[str]
        :rtype: bool
        """
        queue = [0]

```

Round 8 · Low · Medium

p.3075

```

    ls = len(s)
    lenList = [l for l in set(map(len, wordDict))]
    visited = [0 for _ in range(0, ls + 1)]
    while queue:
        start = queue.pop(0)
        for l in lenList:
            if s[start:start + l] in wordDict:
                if start + l == ls:
                    return True
                if visited[start + l] == 0:
                    queue.append(start + l)
                    visited[start + l] = 1
    return False

```

◆ Quick Review

Key Insights

- Use Dynamic Programming (DP) to break the problem into subproblems.
- Create a DP array where $dp[i]$ is true if the substring $s[0:i]$ can be segmented using words from the dictionary.
- Leverage a hash set for fast look-up of dictionary words.
- For every index i , check all possible previous break points j such that $dp[j]$ is true and $s[j:i]$ is in the wordDict.
- The problem can also be approached with recursion and memoization, but DP is more straightforward for iterative implementation.

Round 8 · Low · Medium

p.3076

Space & Time

Time Complexity: $O(n^2)$ in the worst-case scenario, where n is the length of s (each substring check is $O(1)$ if using a hash set, and there could be $O(n^2)$ checks overall).

Space Complexity: $O(n)$ for the dp array and $O(k)$ for the hash set where k is the number of words in the dictionary.

Solution

The solution uses a Dynamic Programming (DP) approach:

- Convert wordDict to a hash set for constant time lookup.
- Create a boolean dp array of size $n+1$ (n is the length of s), where $dp[i]$ indicates if $s[0:i]$ can be segmented.
- Set $dp[0]$ to true because an empty string is always segmentable.
- For each index i from 1 to n , iterate through all indices j from 0 to i : If $dp[j]$ is true and the substring $s[j:i]$ is in the dictionary, then set $dp[i]$ to true. Break early from the inner loop when a valid break is found.
- Return $dp[n]$ as the result.

This method efficiently builds up the solution by utilizing previous results to determine if

the entire string can be segmented.

String DP

#516 Longest Palindromic Subsequence **MED**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming
Lists: Denny Zhang

Problem

Given a string s , find the longest palindromic subsequence's length in s .

A subsequence is a sequence that can be derived from another sequence by deleting some or no elements without changing the order of the remaining elements.

Example 1:

Input: $s = \text{"bbbab"}$
Output: 4
Explanation: One possible longest palindromic subsequence is "bbbb".

Example 2:

Input: $s = \text{"cbdd"}$
Output: 2
Explanation: One possible longest palindromic subsequence is "bb".

Constraints:

- $1 \leq s.length \leq 1000$
- s consists only of lowercase English letters.

Round 8 · Low · Medium

p.3079

Community Solutions (Python)

• WalkCC

```
class Solution:
    def longestPalindromeSubseq(self, s: str) -> int:
        @functools.lru_cache(None)
        def dp(i: int, j: int) -> int:
            """Returns the length of LPS(s[i..j])."""
            if i > j:
                return 0
            if i == j:
                return 1
            if s[i] == s[j]:
                return 2 + dp(i + 1, j - 1)
            return max(dp(i + 1, j), dp(i, j - 1))

        return dp(0, len(s) - 1)
```

```
class Solution:
    def longestPalindromeSubseq(self, s: str) -> int:
        n = len(s)
        # dp[i][j] := the length of LPS(s[i..j])
        dp = [[0] * n for _ in range(n)]

        for i in range(n):
            dp[i][i] = 1

        for d in range(1, n):
            for i in range(n - d):
                j = i + d
                if s[i] == s[j]:
                    dp[i][j] = 2 + dp[i + 1][j - 1]
                else:
                    dp[i][j] = max(dp[i + 1][j], dp[i][j - 1])

        return dp[0][n - 1]
```

• LeetDoocs

Round 8 · Low · Medium

p.3080


```
class Solution:
    def longestPalindromeSubseq(self, s: str) -> int:
        n = len(s)
        f = [[0] * n for _ in range(n)]
        for i in range(n):
            f[i][i] = 1
        for i in range(n - 1, -1, -1):
            for j in range(i + 1, n):
                if s[i] == s[j]:
                    f[i][j] = f[i + 1][j - 1] + 2
                else:
                    f[i][j] = max(f[i + 1][j], f[i][j - 1])
        return f[0][-1]
```

• SimplyLeet

```
# Function to compute the longest palindromic subsequence length
def longest_palindromic_subseq(s):
    n = len(s)
    # Create a 2D DP table initialized to 0
    dp = [[0] * n for _ in range(n)]

    # Base case: every single character is a palindrome of length 1
    for i in range(n):
        dp[i][i] = 1

    # Build the table. 'sub_len' is the current length of substring we are
    # considering.
    for sub_len in range(2, n+1):
        for i in range(n - sub_len + 1):
            j = i + sub_len - 1 # Compute the end index of the substring
            if s[i] == s[j]:
                # Characters match, add 2 to the result from the inner
                # substring if available
                dp[i][j] = 2 + (dp[i+1][j-1] if sub_len > 2 else 0)
            else:
                # Characters don't match, choose the maximum from either
                # excluding left or right char
                dp[i][j] = max(dp[i+1][j], dp[i][j-1])
```

Round 8 · Low · Medium

p.3081

```
# The answer for the full string is stored at dp[0][n-1]
return dp[0][n-1]

# Example usage:
s = "bbbab"
print(longest_palindromic_subseq(s)) # Expected output: 4
```

• LC.ca

```
class Solution:
    def longestPalindromeSubseq(self, s: str) -> int:
        n = len(s)
        dp = [[0] * n for _ in range(n)]
        for i in range(n - 1, -1, -1):
            dp[i][i] = 1 # pre-set before inner loop
            for j in range(i + 1, n):
                if s[i] == s[j]:
                    dp[i][j] = dp[i + 1][j - 1] + 2
                else:
                    dp[i][j] = max(dp[i + 1][j], dp[i][j - 1])
        return dp[0][-1]

#####

class Solution:
    def longestPalindromeSubseq(self, s: str) -> int:
        n = len(s)
        dp = [[0] * n for _ in range(n)]
        for i in range(n):
            dp[i][i] = 1 # cannot move it inside 'for j in range(1, n)'
            # because below 'for j in range' starting at 1
            # so cannot put dp[j][j]=1 inside below for loop
            for j in range(1, n):
                for i in range(j - 1, -1, -1):
                    if s[i] == s[j]:
                        dp[i][j] = dp[i + 1][j - 1] + 2
                        # "aa" => i=0,j=1 => dp[1][0] is 0, still works
                    else:
                        dp[i][j] = max(dp[i + 1][j], dp[i][j - 1])
        return dp[0][-1]
```

Round 8 · Low · Medium

p.3082

◆ Quick Review

Key Insights

- The problem can be solved efficiently using dynamic programming.
- A 2D DP table is used where $dp[i][j]$ represents the length of the longest palindromic subsequence in the substring $s[i...j]$.
- If the characters at the two ends of the substring match ($s[i] == s[j]$), they contribute to the palindrome length.
- Otherwise, the solution is the maximum sequence length by either excluding the left or the right character.
- The approach builds the solution from smaller substrings (length 1) to the entire string.

Space & Time

Time Complexity: $O(n^2)$ where n is the length of the string.

Space Complexity: $O(n^2)$ due to the usage of a 2D DP table.

Round 8 · Low · Medium

p.3083

Solution

We solve this problem using a bottom-up dynamic programming approach. We create a 2D table $dp[][]$ such that $dp[i][j]$ stores the longest palindromic subsequence length for the substring $s[i...j]$. For any single character, $dp[i][i]$ is 1. For substrings of length greater than 1, if the characters at positions i and j ($s[i]$ and $s[j]$) match, then $dp[i][j] = dp[i+1][j-1] + 2$. If they do not match, we take the maximum of $dp[i+1][j]$ and $dp[i][j-1]$. Finally, the $dp[0][n-1]$ holds the result for the entire string.

Round 8 · Low · Medium

p.3084

String DP

□ #1143 Longest Common Subsequence

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming
Lists: Denny Zhang

Problem

Given two strings `text1` and `text2`, return the length of their longest common subsequence. If there is no common subsequence, return 0.

A subsequence of a string is a new string generated from the original string with some characters (can be none) deleted without changing the relative order of the remaining characters.

- For example, "ace" is a subsequence of "abcde".

A common subsequence of two strings is a subsequence that is common to both strings.

Example 1:

Input: `text1 = "abcde", text2 = "ace"`
Output: 3
Explanation: The longest common subsequence is "ace" and its length is 3.

Round 8 · Low · Medium

p.3085

Example 2:

Input: `text1 = "abc", text2 = "abc"`
Output: 3
Explanation: The longest common subsequence is "abc" and its length is 3.

Example 3:

Input: `text1 = "abc", text2 = "def"`
Output: 0
Explanation: There is no such common subsequence, so the result is 0.

Constraints:

- $1 \leq \text{text1.length}, \text{text2.length} \leq 1000$
- `text1` and `text2` consist of only lowercase English characters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def longestCommonSubsequence(self, text1: str, text2: str) -> int:
        m = len(text1)
        n = len(text2)
        # dp[i][j] := the length of LCS(text1[0..i], text2[0..j])
        dp = [[0] * (n + 1) for _ in range(m + 1)]

        for i in range(m):
            for j in range(n):
                dp[i + 1][j + 1] = (1 + dp[i][j] if text1[i] == text2[j]
                                   else max(dp[i][j + 1], dp[i + 1][j]))

        return dp[m][n]
```

• LeetDoocs

Round 8 · Low · Medium

p.3086

```
class Solution:
    def longestCommonSubsequence(self, text1: str, text2: str) -> int:
        m, n = len(text1), len(text2)
        f = [[0] * (n + 1) for _ in range(m + 1)]
        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if text1[i - 1] == text2[j - 1]:
                    f[i][j] = f[i - 1][j - 1] + 1
                else:
                    f[i][j] = max(f[i - 1][j], f[i][j - 1])

        return f[m][n]
```

• SimplyLeet

```
# Function to find the longest common subsequence length
def longestCommonSubsequence(text1: str, text2: str) -> int:
    m, n = len(text1), len(text2)
    # Initialize a (m+1) x (n+1) DP table filled with zeros
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    # Build the DP table bottom-up
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            # If characters match, extend the subsequence length by 1
            if text1[i - 1] == text2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                # Otherwise, choose the maximum from the left or top cell
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])

    return dp[m][n]

# Example usage
if __name__ == "__main__":
    text1 = "abcde"
    text2 = "ace"
    print(longestCommonSubsequence(text1, text2)) # Output: 3
```

• LC.ca

Round 8 · Low · Medium

p.3087

```
class Solution:
    def longestCommonSubsequence(self, text1: str, text2: str) -> int:
        m, n = len(text1), len(text2)
        f = [[0] * (n + 1) for _ in range(m + 1)]
        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if text1[i - 1] == text2[j - 1]:
                    f[i][j] = f[i - 1][j - 1] + 1
                else:
                    f[i][j] = max(f[i - 1][j], f[i][j - 1])

        return f[m][n]
```

◆ Quick Review

Key Insights

- The problem is well-suited for a dynamic programming approach.
- Use a 2D table where $dp[i][j]$ stores the length of the longest common subsequence for $text1[0...i-1]$ and $text2[0...j-1]$.
- If the characters match at the current position, extend the subsequence by 1 from $dp[i-1][j-1]$; otherwise, take the maximum from $dp[i-1][j]$ or $dp[i][j-1]$.
- Filling the table in a bottom-up manner ensures that all subproblems are solved optimally.

Space & Time

Round 8 · Low · Medium

p.3088

Time Complexity: $O(m * n)$, where m and n are the lengths of `text1` and `text2`.

Space Complexity: $O(m * n)$ for maintaining the 2D DP table. Space can be optimized to $O(\min(m, n))$ if needed.

Solution

This solution uses a dynamic programming approach with a 2D array (`dp`) to record the lengths of common subsequences at each substring pair. We initialize a table of dimensions $(m+1) \times (n+1)$ with zeros, where m and n are the lengths of `text1` and `text2`. We iterate over both strings, comparing characters; if they match, we set $dp[i][j] = dp[i-1][j-1] + 1$, otherwise we choose the maximum from $dp[i-1][j]$ and $dp[i][j-1]$. The final answer is found at $dp[m][n]$.

String DP

□ #1653 Minimum Deletions to Make String Balanced

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode String · Dynamic Programming · Stack Lists: AlgoMaster 600

Problem

You are given a string `s` consisting only of characters 'a' and 'b'.

You can delete any number of characters in `s` to make `s` balanced. `s` is balanced if there is no pair of indices (i, j) such that $i < j$ and $s[i] = 'b'$ and $s[j] = 'a'$.

Return the minimum number of deletions needed to make `s` balanced.

Example 1:

```
Input: s = "aababbab"
Output: 2
Explanation: You can either:
Delete the characters at 0-indexed positions 2 and 6 ("aababbab" -> "aaabbbb"),
or
Delete the characters at 0-indexed positions 3 and 6 ("aababbab" -> "aabbbb").
```

Example 2:

Input: s = "bbaaaaabb"

Output: 2

Explanation: The only solution is to delete the first two characters.

Constraints:

- $1 \leq \text{s.length} \leq 105$
- s[i] is 'a' or 'b'.

Community Solutions (Python)

• WalkCC

```
class Solution:
    # Same as 926. Flip String to Monotone Increasing
    def minimumDeletions(self, s: str) -> int:
        dp = 0 # the number of characters to be deleted to make subso far balanced
        countB = 0

        for c in s:
            if c == 'a':
                # 1. Delete 'a'.
                # 2. Keep 'a' and delete the previous 'b's.

                dp = min(dp + 1, countB)
            else:
                countB += 1

        return dp
```

• LeetCode

```
class Solution:
    def minimumDeletions(self, s: str) -> int:
        n = len(s)
        f = [0] * (n + 1)
        b = 0
        for i, c in enumerate(s, 1):
            if c == 'b':
                f[i] = f[i - 1]
                b += 1
            else:
                f[i] = min(f[i - 1] + 1, b)
        return f[n]
```

```
class Solution:
    def minimumDeletions(self, s: str) -> int:
        lb, ra = 0, s.count('a')
        ans = len(s)
        for c in s:
            ra -= c == 'a'
            ans = min(ans, lb + ra)
            lb += c == 'b'
        return ans
```

• SimplyLeet

```
# Function to compute minimum deletions needed to make a string balanced
def minimum_deletions(s):
    deletion_count = 0 # Tracks the minimum number of deletions required so far
    count_b = 0 # Counts the number of 'b' characters encountered
    # Iterate over each character in the string
    for char in s:
        if char == 'b':
            count_b += 1 # Increment count for each 'b'
        else: # char is 'a'
            # Either delete the current 'a' (deletion_count + 1) or remove all previous 'b's (count_b)
```

```
        deletion_count = min(deletion_count + 1, count_b)
    return deletion_count
```

```
# Example usage
```

```
s = "aababbab"
```

```
print(minimum_deletions(s)) # Expected output: 2
```

Round 8 · Low · Medium

p.3093

Tree / Graph DP

Round 8 · Low · Medium · 3 questions

Round 8 · Low · Medium

p.3094

Tree / Graph DP

#95 Unique Binary Search Trees II

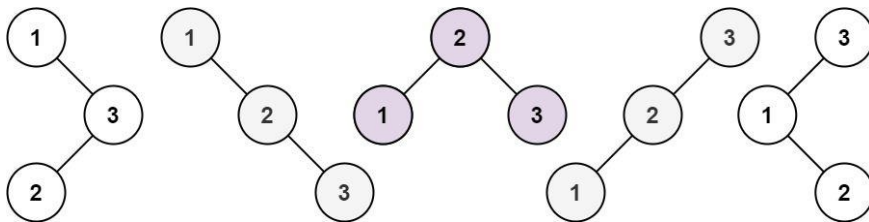
MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Dynamic Programming · Backtracking · Tree ·
 Binary Search Tree · Binary Tree
 Lists: AlgoMaster 600

Problem

Given an integer n , return all the structurally unique BST's (binary search trees), which has exactly n nodes of unique values from 1 to n . Return the answer in any order.

Example 1:


 Input: $n = 3$

Output:

`[[1,null,2,null,3],[1,null,3,2],[2,1,3],[3,1,null,null,2],[3,2,null,1]]`

Example 2:

Round 8 · Low · Medium

p.3095

 Input: $n = 1$

 Output: `[[1]]`

Constraints:

- $1 \leq n \leq 8$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def generateTrees(self, n: int) -> list[TreeNode]:
        if n == 0:
            return []

        def generateTrees(mn: int, mx: int) -> list[int | None]:
            if mn > mx:
                return [None]

            ans = []

            for i in range(mn, mx + 1):
                for left in generateTrees(mn, i - 1):
                    for right in generateTrees(i + 1, mx):
                        ans.append(TreeNode(i))
                        ans[-1].left = left
                        ans[-1].right = right

            return ans

        return generateTrees(1, n)
```

• LeetDoocs

Round 8 · Low · Medium

p.3096


```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def generateTrees(self, n: int) -> List[Optional[TreeNode]]:
        def dfs(i: int, j: int) -> List[Optional[TreeNode]]:
            if i > j:
                return [None]
            ans = []
            for v in range(i, j + 1):
                left = dfs(i, v - 1)
                right = dfs(v + 1, j)
                for l in left:
                    for r in right:
                        ans.append(TreeNode(v, l, r))
            return ans

        return dfs(1, n)
```

• SimplyLeet

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val # Initialize the node's value
        self.left = left # Reference to the left subtree
        self.right = right # Reference to the right subtree

class Solution:
    def generateTrees(self, n: int):
        # Edge-case: if no numbers, return empty list
```

```
if n == 0:
    return []

# Recursive helper function to generate BSTs for a given range
def generate_trees(start, end):
    trees = []
    # Base case: represent an empty tree
    if start > end:
        trees.append(None)
        return trees

    # Loop through each number as a potential root
    for root_val in range(start, end + 1):
        # All possible left subtrees if root_val is chosen
        left_trees = generate_trees(start, root_val - 1)
        # All possible right subtrees if root_val is chosen
        right_trees = generate_trees(root_val + 1, end)
        # Combine each left and right subtree with the root
        for left in left_trees:
            for right in right_trees:
                root = TreeNode(root_val)
                root.left = left
                root.right = right
                trees.append(root)

    return trees

return generate_trees(1, n)

# Example usage:
# sol = Solution()
# trees = sol.generateTrees(3)

# This will generate all unique BSTs for n = 3.
```

Tree / Graph DP

#337 House Robber III

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Dynamic Programming · Tree · Depth-First
 Search · Binary Tree
 Lists: AlgoMaster 600

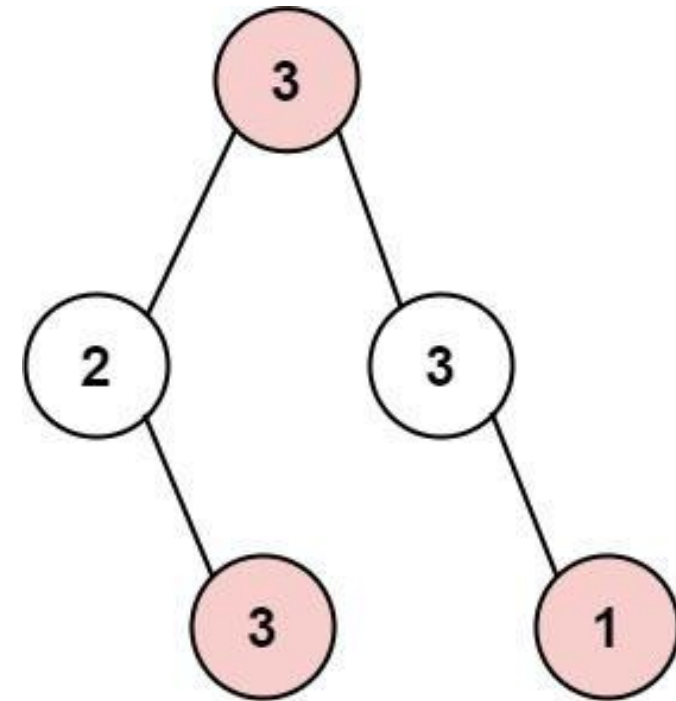
Problem

The thief has found himself a new place for his thievery again. There is only one entrance to this area, called root.

Besides the root, each house has one and only one parent house. After a tour, the smart thief realized that all houses in this place form a binary tree. It will automatically contact the police if two directly-linked houses were broken into on the same night.

Given the root of the binary tree, return the maximum amount of money the thief can rob without alerting the police.

Example 1:

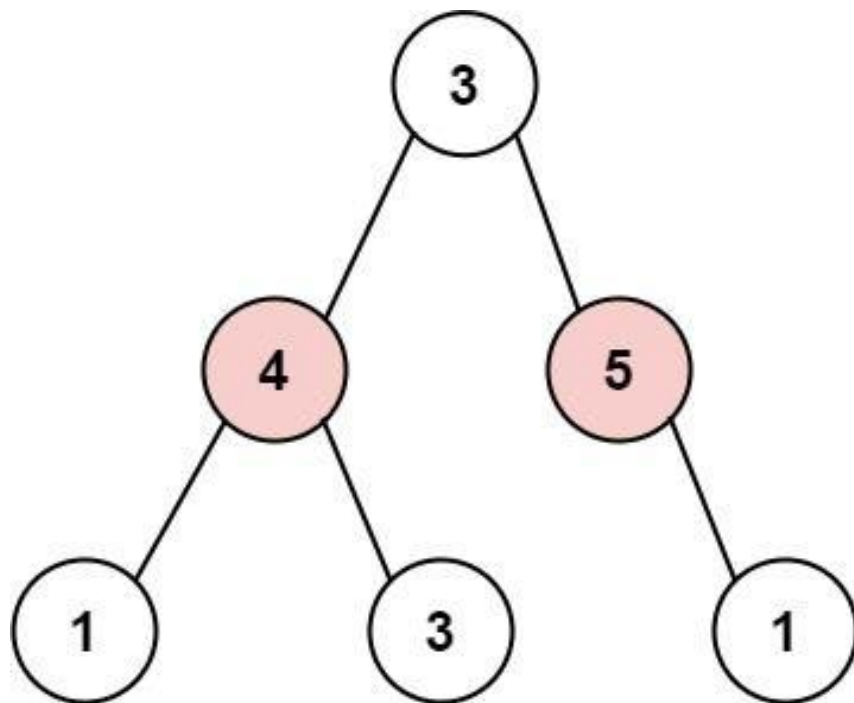


Input: root = [3,2,3,null,3,null,1]

Output: 7

Explanation: Maximum amount of money the thief can rob = 3 + 3 + 1 = 7.

Example 2:



Input: root = [3,4,5,1,3,null,1]

Output: 9

Explanation: Maximum amount of money the thief can rob = 4 + 5 = 9.

Constraints:

- The number of nodes in the tree is in the range [1, 104].
- $0 \leq \text{Node.val} \leq 104$

Community Solutions (Python)

- WalkCC

Round 8 · Low · Medium

p.3101

```

class Solution:
    def rob(self, root: TreeNode | None) -> int:
        def robOrNot(root: TreeNode | None) -> tuple:
            if not root:
                return (0, 0)

            robLeft, notRobLeft = robOrNot(root.left)
            robRight, notRobRight = robOrNot(root.right)

            return (root.val + notRobLeft + notRobRight,
                    max(robLeft, notRobLeft) + max(robRight, notRobRight))

        return max(robOrNot(root))
  
```

• LeetCode

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def rob(self, root: Optional[TreeNode]) -> int:
        def dfs(root: Optional[TreeNode]) -> (int, int):
            if root is None:
                return 0, 0

            la, lb = dfs(root.left)
            ra, rb = dfs(root.right)
            return root.val + lb + rb, max(la, lb) + max(ra, rb)

        return max(dfs(root))
  
```

• SimplyLeet

Round 8 · Low · Medium

p.3102

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val      # value of the node (money in the house)
        self.left = left    # left child
        self.right = right  # right child

def rob(root: TreeNode) -> int:
    # Returns a tuple (rob, notRob) representing:
    # rob -> maximum money when this node is robbed

    # notRob -> maximum money when this node is not robbed
    def dfs(node):
        if not node:
            return (0, 0)
        left = dfs(node.left)
        right = dfs(node.right)
        # If we rob this node, we cannot rob its children
        rob_current = node.val + left[1] + right[1]
        # If we do not rob this node, choose the max for each child
        independently
        not_rob_current = max(left) + max(right)

        return (rob_current, not_rob_current)

    return max(dfs(root))

# Example usage:
# root = TreeNode(3, TreeNode(2, None, TreeNode(3)), TreeNode(3, None,
# TreeNode(1)))
# print(rob(root)) # Output should be 7
```

Tree / Graph DP

□ #1976 Number of Ways to Arrive at Destination

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Dynamic Programming · Graph Theory ·
Topological Sort · Shortest Path

Lists: AlgoMaster 600

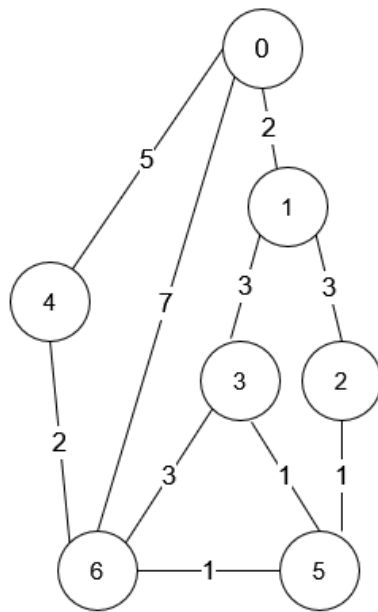
Problem

You are in a city that consists of n intersections numbered from 0 to $n - 1$ with bi-directional roads between some intersections. The inputs are generated such that you can reach any intersection from any other intersection and that there is at most one road between any two intersections.

You are given an integer n and a 2D integer array `roads` where `roads[i] = [ui, vi, timei]` means that there is a road between intersections ui and vi that takes $timei$ minutes to travel. You want to know in how many ways you can travel from intersection 0 to intersection $n - 1$ in the shortest amount of time.

Return the number of ways you can arrive at your destination in the shortest amount of time. Since the answer may be large, return it modulo $10^9 + 7$.

Example 1:



Input: $n = 7$, $roads = [[0,6,7],[0,1,2],[1,2,3],[1,3,3],[6,3,3],[3,5,1],[6,5,1],[2,5,1],[0,4,5],[4,6,2]]$

Output: 4

Explanation: The shortest amount of time it takes to go from intersection 0 to intersection 6 is 7 minutes.

The four ways to get there in 7 minutes are:

- $0 \rightarrow 6$
- $0 \rightarrow 4 \rightarrow 6$
- $0 \rightarrow 1 \rightarrow 2 \rightarrow 5 \rightarrow 6$
- $0 \rightarrow 1 \rightarrow 3 \rightarrow 5 \rightarrow 6$

Example 2:

Input: $n = 2$, $roads = [[1,0,10]]$

Output: 1

Explanation: There is only one way to go from intersection 0 to intersection 1, and it takes 10 minutes.

Constraints:

- $1 \leq n \leq 200$
- $n - 1 \leq roads.length \leq n * (n - 1) / 2$
- $roads[i].length == 3$
- $0 \leq u_i, v_i \leq n - 1$
- $1 \leq time_i \leq 10^9$
- $u_i \neq v_i$
- There is at most one road connecting any two intersections.
- You can reach any intersection from any other intersection.

Community Solutions (Python)

- WalkCC

```
class Solution:
    def countPaths(self, n: int, roads: list[list[int]]) -> int:
        graph = [[] for _ in range(n)]

        for u, v, w in roads:
            graph[u].append((v, w))
            graph[v].append((u, w))

        return self._dijkstra(graph, 0, n - 1)
```

```
def _dijkstra(
    self,
    graph: list[list[tuple[int, int]]],
    src: int,
    dst: int,
) -> int:
    MOD = 10**9 + 7
    ways = [0] * len(graph)
    dist = [math.inf] * len(graph)
```

```
    ways[src] = 1
    dist[src] = 0
    minHeap = [(dist[src], src)]
```

```
    while minHeap:
        d, u = heapq.heappop(minHeap)
        if d > dist[u]:
            continue
        for v, w in graph[u]:
            if d + w < dist[v]:
```

```
                dist[v] = d + w
                ways[v] = ways[u]
                heapq.heappush(minHeap, (dist[v], v))
            elif d + w == dist[v]:
                ways[v] += ways[u]
                ways[v] %= MOD
```

```
    return ways[dst]
```

• LeetCode

Round 8 · Low · Medium

p.3107

```
class Solution:
    def countPaths(self, n: int, roads: List[List[int]]) -> int:
        g = [[inf] * n for _ in range(n)]
        for u, v, t in roads:
            g[u][v] = g[v][u] = t
        g[0][0] = 0
        dist = [inf] * n
        dist[0] = 0
        f = [0] * n
        f[0] = 1
```

```
        vis = [False] * n
        for _ in range(n):
            t = -1
            for j in range(n):
                if not vis[j] and (t == -1 or dist[j] < dist[t]):
                    t = j
            vis[t] = True
            for j in range(n):
                if j == t:
                    continue
```

```
                ne = dist[t] + g[t][j]
                if dist[j] > ne:
                    dist[j] = ne
                    f[j] = f[t]
                elif dist[j] == ne:
                    f[j] += f[t]
        mod = 10**9 + 7
        return f[-1] % mod
```

• SimplyLeet

```
import heapq

def countPaths(n, roads):
    MOD = 10**9 + 7
    # Build the graph as an adjacency list
    graph = [[] for _ in range(n)]
    for u, v, time in roads:
        graph[u].append((v, time))
        graph[v].append((u, time))
```

Round 8 · Low · Medium

p.3108

```
# Initialize distance and ways arrays
dist = [float('inf')] * n
ways = [0] * n
dist[0] = 0
ways[0] = 1

# Min-heap: (distance, node)
heap = [(0, 0)]

# Modified Dijkstra's algorithm

while heap:
    current_dist, u = heapq.heappop(heap)
    if current_dist > dist[u]:
        continue
    for v, time in graph[u]:
        next_dist = current_dist + time
        if next_dist < dist[v]:
            dist[v] = next_dist
            ways[v] = ways[u]
            heapq.heappush(heap, (next_dist, v))

        elif next_dist == dist[v]:
            ways[v] = (ways[v] + ways[u]) % MOD
return ways[n - 1]
```

Round 8 · Low · Medium

p.3109

Bitmask DP

Round 8 · Low · Medium · 4 questions

Round 8 · Low · Medium

p.3110

Bitmask DP

#698 Partition to K Equal Sum Subsets

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Dynamic Programming · Backtracking ·
 Bit Manipulation · Memoization · Bitmask
 Lists: AlgoMaster 600

Problem

Given an integer array `nums` and an integer `k`, return `true` if it is possible to divide this array into `k` non-empty subsets whose sums are all equal.

Example 1:

Input: `nums = [4,3,2,3,5,2,1]`, `k = 4`
 Output: `true`
 Explanation: It is possible to divide it into 4 subsets (5), (1, 4), (2,3), (2,3) with equal sums.

Example 2:

Input: `nums = [1,2,3,4]`, `k = 3`
 Output: `false`

Constraints:

- $1 \leq k \leq \text{nums.length} \leq 16$
- $1 \leq \text{nums}[i] \leq 104$

Round 8 · Low · Medium

p.3111

- The frequency of each element is in the range `[1, 4]`.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def canPartitionKSubsets(self, nums: list[int], k: int) -> bool:
        summ = sum(nums)
        if summ % k != 0:
            return False

        target = summ // k # the target sum of each subset
        if any(num > target for num in nums):
            return False

        def dfs(s: int, remainingGroups: int, currSum: int, used: int) -> bool:
            if remainingGroups == 0:
                return True
            if currSum > target:
                return False
            if currSum == target: # Find a valid group, so fresh start.
                return dfs(0, remainingGroups - 1, 0, used)

            for i in range(s, len(nums)):
                if used >> i & 1:

                    continue
                if dfs(i + 1, remainingGroups, currSum + nums[i], used | 1 << i):
                    return True

            return False

        nums.sort(reverse=True)
        return dfs(0, k, 0, 0)
```

• LeetDoocs

Round 8 · Low · Medium

p.3112


```
class Solution:
    def canPartitionKSubsets(self, nums: List[int], k: int) -> bool:
        def dfs(i: int) -> bool:
            if i == len(nums):
                return True
            for j in range(k):
                if j and cur[j] == cur[j - 1]:
                    continue
                cur[j] += nums[i]
                if cur[j] <= s and dfs(i + 1):
                    return True
                cur[j] -= nums[i]
            return False

        s, mod = divmod(sum(nums), k)
        if mod:
            return False
        cur = [0] * k
        nums.sort(reverse=True)
        return dfs(0)
```

```
class Solution:
    def canPartitionKSubsets(self, nums: List[int], k: int) -> bool:
        @cache
        def dfs(state, t):
            if state == mask:
                return True
            for i, v in enumerate(nums):
                if (state >> i) & 1:
                    continue
                if t + v > s:
                    break
                if dfs(state | 1 << i, (t + v) % s):
                    return True
            return False

        s, mod = divmod(sum(nums), k)
        if mod:
            return False
        nums.sort()
        mask = (1 << len(nums)) - 1
```

```
return dfs(0, 0)
```

```
class Solution:
    def canPartitionKSubsets(self, nums: List[int], k: int) -> bool:
        s = sum(nums)
        if s % k:
            return False
        s //= k
        nums.sort()
        n = len(nums)
        f = [False] * (1 << n)
        cur = [0] * (1 << n)

        f[0] = True
        for i in range(1 << n):
            if not f[i]:
                continue
            for j in range(n):
                if cur[i] + nums[j] > s:
                    break
                if (i >> j & 1) == 0:
                    if not f[i | 1 << j]:
                        cur[i | 1 << j] = (cur[i] + nums[j]) % s

                f[i | 1 << j] = True

        return f[-1]
```

• SimplyLeet

```
# Define the class with the solution method
class Solution:
    def canPartitionKSubsets(self, nums, k):
        # Calculate the total sum and check divisibility by k
        total_sum = sum(nums)
        if total_sum % k != 0:
            return False

        target = total_sum // k
```

```

# Sort the numbers in descending order for optimization
nums.sort(reverse=True)
# If the largest number is greater than the target, partition is
impossible
if nums[0] > target:
    return False

n = len(nums)
used = [False] * n

# Backtracking function: start_index is where to continue searching,
# curr_sum is the current subset sum, count is how many subsets have
been formed so far.
def backtrack(start_index, curr_sum, count):
    # If we've correctly formed k-1 subsets, the rest of the numbers
    form the last subset.
    if count == k - 1:
        return True
    # When current subset sum equals target, start forming a new
    subset.
    if curr_sum == target:
        return backtrack(0, 0, count + 1)
    # Try to assign next numbers to current subset.
    for i in range(start_index, n):
        if not used[i] and curr_sum + nums[i] <= target:
            used[i] = True
            if backtrack(i + 1, curr_sum + nums[i], count):
                return True
            used[i] = False
            # Skip duplicate elements to prevent redundant work.
            while i + 1 < n and nums[i] == nums[i + 1]:
                i += 1
    return False

return backtrack(0, 0, 0)

# Example usage:
# sol = Solution()
# print(sol.canPartitionKSubsets([4,3,2,3,5,2,1], 4)) # Output: True

```

Round 8 · Low · Medium

p.3115

Bitmask DP

#1066 Campus Bikes II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Dynamic Programming · Backtracking ·
 Bit Manipulation · Bitmask
 Lists: AlgoMaster 600

Problem

On a campus represented as a 2D grid, there are n workers and m bikes, with $n \leq m$. Each worker and bike is a 2D coordinate on this grid. We assign one unique bike to each worker so that the sum of the Manhattan distances between each worker and their assigned bike is minimized.

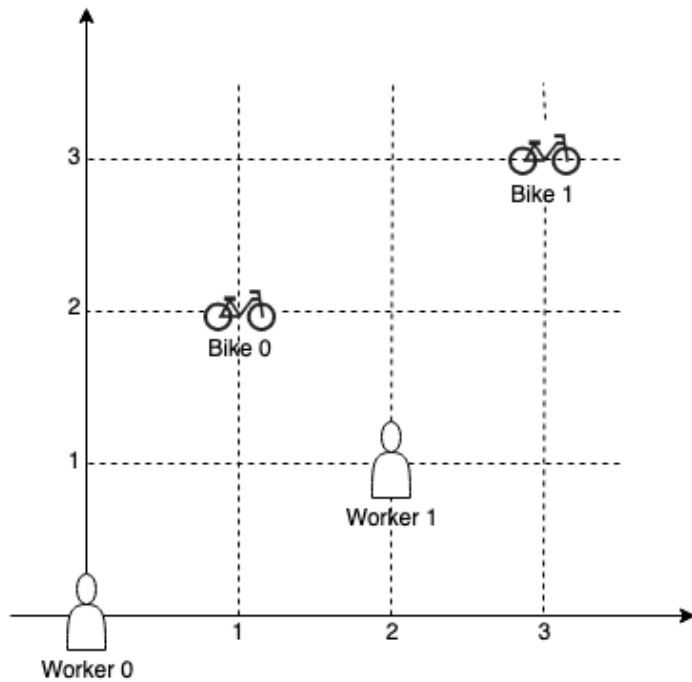
Return the minimum possible sum of Manhattan distances between each worker and their assigned bike.

The Manhattan distance between two points $p1$ and $p2$ is $\text{Manhattan}(p1, p2) = |p1.x - p2.x| + |p1.y - p2.y|$.

Example 1:

Round 8 · Low · Medium

p.3116



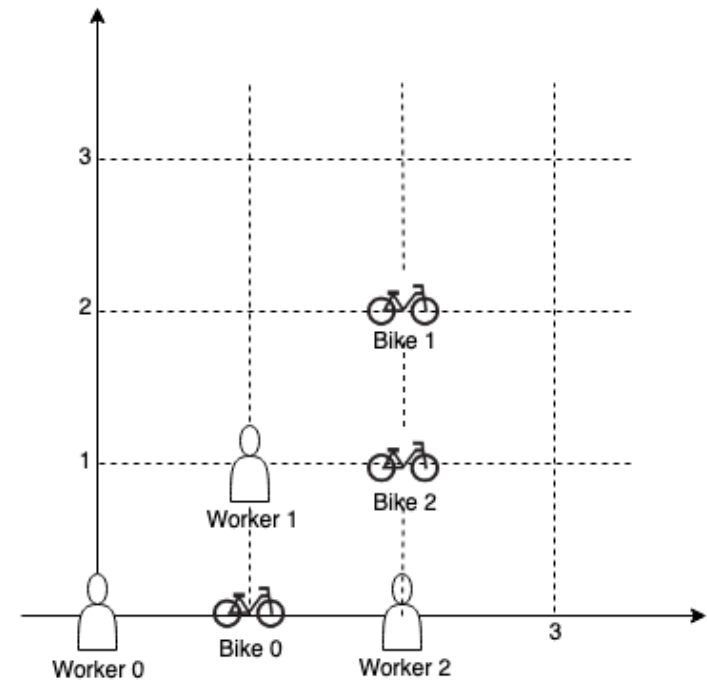
Input: workers = `[[0,0],[2,1]]`, bikes = `[[1,2],[3,3]]`

Output: 6

Explanation:

We assign bike 0 to worker 0, bike 1 to worker 1. The Manhattan distance of both assignments is 3, so the output is 6.

Example 2:



Input: workers = `[[0,0],[1,1],[2,0]]`, bikes = `[[1,0],[2,2],[2,1]]`

Output: 4

Explanation:

We first assign bike 0 to worker 0, then assign bike 1 to worker 1 or worker 2, bike 2 to worker 2 or worker 1. Both assignments lead to sum of the Manhattan distances as 4.

Example 3:

Input: workers = `[[0,0],[1,0],[2,0],[3,0],[4,0]]`, bikes = `[[0,999],[1,999],[2,999],[3,999],[4,999]]`

Output: 4995

Constraints:

- `n == workers.length`

- $m == \text{bikes.length}$
- $1 \leq n \leq m \leq 10$
- $\text{workers}[i].\text{length} == 2$
- $\text{bikes}[i].\text{length} == 2$
- $0 \leq \text{workers}[i][0], \text{workers}[i][1], \text{bikes}[i][0], \text{bikes}[i][1] < 1000$
- All the workers and the bikes locations are unique.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def assignBikes(
        self,
        workers: list[list[int]],
        bikes: list[list[int]],
    ) -> int:
        def dist(p1: list[int], p2: list[int]) -> int:
            return abs(p1[0] - p2[0]) + abs(p1[1] - p2[1])

        @functools.lru_cache(None)

        def dp(workerIndex: int, used: int) -> int:
            """
            Returns the minimum Manhattan distances to assign bikes to
            workers[workerIndex..n], where `used` is the bitmask of the used bikes.
            """
            if workerIndex == len(workers):
                return 0
            return min(
                (dist(workers[workerIndex],
                    bike) + dp(workerIndex + 1, used | 1 << i) for i,
```

```
bike in enumerate(bikes) if not used >> i & 1),
default=math.inf)
```

```
return dp(0, 0)
```

• LeetCode

```
class Solution:
    def assignBikes(self, workers: List[List[int]], bikes: List[List[int]]) ->
    int:
        n, m = len(workers), len(bikes)
        f = [[inf] * (1 << m) for _ in range(n + 1)]
        f[0][0] = 0
        for i, (x1, y1) in enumerate(workers, 1):
            for j in range(1 << m):
                for k, (x2, y2) in enumerate(bikes):
                    if j >> k & 1:
                        f[i][j] = min(
                            f[i][j],
                            f[i - 1][j ^ (1 << k)] + abs(x1 - x2) + abs(y1 - y
2),
                        )
        return min(f[n])
```

• SimplyLeet

```
from functools import lru_cache

def assignBikes(workers, bikes):
    # Helper function to compute Manhattan distance between two points
    def manhattan(p1, p2):
        return abs(p1[0] - p2[0]) + abs(p1[1] - p2[1])

    n = len(workers)
    m = len(bikes)
```

```

# dp(worker, mask): minimum total Manhattan distance starting from worker
index 'worker'
# with bikes represented by 'mask' (1 bit per bike, 1 means assigned)
@lru_cache(maxsize=None)
def dp(worker, mask):
    # Base case: if all workers are assigned, return 0
    if worker == n:
        return 0

    res = float('inf')
    # Iterate over all bikes

    for j in range(m):
        # Check if bike j is free
        if not (mask & (1 << j)):
            # Calculate new mask with bike j assigned and compute total
            cost

            new_mask = mask | (1 << j)
            cost = manhattan(workers[worker], bikes[j]) + dp(worker + 1, n
            ew_mask)

            res = min(res, cost)
    return res

# Start with the first worker and no bikes assigned (mask = 0)

return dp(0, 0)

# Example usage:
# workers = [[0,0], [2,1]]
# bikes = [[1,2], [3,3]]
# print(assignBikes(workers, bikes))

```

Bitmask DP

☐ #1986 Minimum Number of Work Sessions to Finish the Tasks

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Dynamic Programming · Backtracking ·
 Bit Manipulation · Bitmask

Lists: AlgoMaster 600

Problem

There are n tasks assigned to you. The task times are represented as an integer array `tasks` of length n , where the i th task takes `tasks[i]` hours to finish. A work session is when you work for at most `sessionTime` consecutive hours and then take a break.

You should finish the given tasks in a way that satisfies the following conditions:

- If you start a task in a work session, you must complete it in the same work session.
- You can start a new task immediately after finishing the previous one.
- You may complete the tasks in any order.

Given tasks and sessionTime, return the minimum number of work sessions needed to finish all the tasks following the conditions above.

The tests are generated such that sessionTime is greater than or equal to the maximum element in tasks[i].

Example 1:

Input: tasks = [1,2,3], sessionTime = 3
Output: 2
Explanation: You can finish the tasks in two work sessions.
- First work session: finish the first and the second tasks in 1 + 2 = 3 hours.
- Second work session: finish the third task in 3 hours.

Example 2:

Input: tasks = [3,1,3,1,1], sessionTime = 8
Output: 2
Explanation: You can finish the tasks in two work sessions.
- First work session: finish all the tasks except the last one in 3 + 1 + 3 + 1 = 8 hours.
- Second work session: finish the last task in 1 hour.

Example 3:

Input: tasks = [1,2,3,4,5], sessionTime = 15
Output: 1
Explanation: You can finish all the tasks in one work session.

Constraints:

- $n == \text{tasks.length}$
- $1 \leq n \leq 14$
- $1 \leq \text{tasks}[i] \leq 10$
- $\max(\text{tasks}[i]) \leq \text{sessionTime} \leq 15$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minSessions(self, tasks: list[int], sessionTime: int) -> int:
        # Returns True if we can assign tasks[s..n) to `sessions`. Note that
        # `sessions`
        # may be occupied by some tasks.
        def dfs(s: int, sessions: list[int]) -> bool:
            if s == len(tasks):
                return True

            for i, session in enumerate(sessions):
                # Can't assign the tasks[s] to this session.

                if session + tasks[s] > sessionTime:
                    continue
                # Assign the tasks[s] to this session.
                sessions[i] += tasks[s]
                if dfs(s + 1, sessions):
                    return True
                # Backtracking.
                sessions[i] -= tasks[s]
                # If it's the first time we assign the tasks[s] to this session, then
                # future
                # `session`s can't satisfy either.

                if sessions[i] == 0:
                    return False

            return False

        for numSessions in range(1, len(tasks) + 1):
            if dfs(0, [0] * numSessions):
                return numSessions
```

• LeetDoocs

```
class Solution:
    def minSessions(self, tasks: List[int], sessionTime: int) -> int:
        n = len(tasks)
        ok = [False] * (1 << n)
        for i in range(1, 1 << n):
            t = sum(tasks[j] for j in range(n) if i >> j & 1)
            ok[i] = t <= sessionTime
        f = [inf] * (1 << n)
        f[0] = 0
        for i in range(1, 1 << n):
            j = i
            while j:
                if ok[j]:
                    f[i] = min(f[i], f[i ^ j] + 1)
                j = (j - 1) & i
            return f[-1]
```

• SimplyLeet

```
def minSessions(tasks, sessionTime):
    n = len(tasks)
    from functools import lru_cache

    @lru_cache(maxsize=None)
    def dp(mask, time_used):
        # Base condition: if all tasks are completed.
        if mask == (1 << n) - 1:
            return 0
        best = float('inf')

        for i in range(n):
            # If task i is not yet completed.
            if not (mask & (1 << i)):
                if time_used + tasks[i] <= sessionTime:
                    best = min(best, dp(mask | (1 << i), time_used + tasks[i]))
            else:
                best = min(best, 1 + dp(mask | (1 << i), tasks[i]))
        return best

    # We start with one session and 0 time used.
```

Round 8 · Low · Medium

p.3125

```
return 1 + dp(0, 0)

# Example Usage
print(minSessions([1,2,3], 3)) # Output: 2
print(minSessions([3,1,3,1,1], 8)) # Output: 2
print(minSessions([1,2,3,4,5], 15)) # Output: 1
```

Round 8 · Low · Medium

p.3126

Bitmask DP

☐ #2305 Fair Distribution of Cookies

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Dynamic Programming · Backtracking ·
 Bit Manipulation · Bitmask
 Lists: AlgoMaster 600

Problem

You are given an integer array `cookies`, where `cookies[i]` denotes the number of cookies in the *i*th bag. You are also given an integer `k` that denotes the number of children to distribute all the bags of cookies to. All the cookies in the same bag must go to the same child and cannot be split up.

The unfairness of a distribution is defined as the maximum total cookies obtained by a single child in the distribution.

Return the minimum unfairness of all distributions.

Example 1:

Input: `cookies = [8,15,10,20,8]`, `k = 2`

Output: 31

Explanation: One optimal distribution is `[8,15,8]` and `[10,20]`

- The 1st child receives `[8,15,8]` which has a total of $8 + 15 + 8 = 31$ cookies.
 - The 2nd child receives `[10,20]` which has a total of $10 + 20 = 30$ cookies.
- The unfairness of the distribution is $\max(31,30) = 31$.
 It can be shown that there is no distribution with an unfairness less than 31.

Example 2:

Input: `cookies = [6,1,3,2,2,4,1,2]`, `k = 3`

Output: 7

Explanation: One optimal distribution is `[6,1]`, `[3,2,2]`, and `[4,1,2]`

- The 1st child receives `[6,1]` which has a total of $6 + 1 = 7$ cookies.
 - The 2nd child receives `[3,2,2]` which has a total of $3 + 2 + 2 = 7$ cookies.
 - The 3rd child receives `[4,1,2]` which has a total of $4 + 1 + 2 = 7$ cookies.
- The unfairness of the distribution is $\max(7,7,7) = 7$.

It can be shown that there is no distribution with an unfairness less than 7.

Constraints:

- $2 \leq \text{cookies.length} \leq 8$
- $1 \leq \text{cookies}[i] \leq 105$
- $2 \leq k \leq \text{cookies.length}$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def distributeCookies(self, cookies: list[int], k: int) -> int:
        ans = math.inf

        def dfs(s: int, children: list[int]) -> None:
            nonlocal ans
            if s == len(cookies):
                ans = min(ans, max(children))
                return
```



```

for i in range(k):
    children[i] += cookies[s]
    dfs(s + 1, children)
    children[i] -= cookies[s]

```

```

dfs(0, [0] * k)
return ans

```

• LeetCode

```

class Solution:
    def distributeCookies(self, cookies: List[int], k: int) -> int:
        def dfs(i):
            if i >= len(cookies):
                nonlocal ans
                ans = max(cnt)
                return
            for j in range(k):
                if cnt[j] + cookies[i] >= ans or (j and cnt[j] == cnt[j - 1]):
                    continue

                cnt[j] += cookies[i]
                dfs(i + 1)
                cnt[j] -= cookies[i]

        ans = inf
        cnt = [0] * k
        cookies.sort(reverse=True)
        dfs(0)
        return ans

```

• SimplyLeet

```

# Python solution using recursion and backtracking.
def distributeCookies(cookies, k):
    # Initialize the minimum unfairness as a large number.
    min_unfairness = float('inf')
    # Initialize a list to store the running sum of cookies for each child.
    children = [0] * k

    def backtrack(index):
        nonlocal min_unfairness
        # Base case: if we've assigned all bags.

        if index == len(cookies):
            # Update the minimum unfairness with the current maximum value
            # among children.
            min_unfairness = min(min_unfairness, max(children))
            return

        # Try giving the current cookie bag to each child.
        for i in range(k):
            children[i] += cookies[index]
            # Pruning: only proceed if the current child's sum does not exceed
            # known min_unfairness.
            if max(children) < min_unfairness:

                backtrack(index + 1)
                children[i] -= cookies[index]
                # Optimization: if this child hasn't been given a cookie bag yet,
                # break to avoid redundant permutations.
                if children[i] == 0:
                    break

        backtrack(0)
        return min_unfairness

# Example usage:
print(distributeCookies([8,15,10,20,8], 2)) # Output: 31
print(distributeCookies([6,1,3,2,2,4,1,2], 3)) # Output: 7

```

Digit DP

Round 8 · Low · Medium · 1 question

Round 8 · Low · Medium

p.3131

Digit DP

☐ #357 Count Numbers with Unique Digits **MED**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · Dynamic Programming · Backtracking
Lists: AlgoMaster 600

Problem

Given an integer n , return the count of all numbers with unique digits, x , where $0 \leq x < 10^n$.

Example 1:

Input: $n = 2$
Output: 91
Explanation: The answer should be the total numbers in the range of $0 \leq x < 100$, excluding 11,22,33,44,55,66,77,88,99

Example 2:

Input: $n = 0$
Output: 1

Constraints:

- $0 \leq n \leq 8$

Community Solutions (Python)

- ☐
- WalkCC

Round 8 · Low · Medium

p.3132

```
class Solution:
    def countNumbersWithUniqueDigits(self, n: int) -> int:
        if n == 0:
            return 1

        ans = 10
        uniqueDigits = 9
        availableNum = 9

        while n > 1 and availableNum > 0:

            uniqueDigits *= availableNum
            ans += uniqueDigits
            n -= 1
            availableNum -= 1

        return ans
```

• LeetDoocs

```
class Solution:
    def countNumbersWithUniqueDigits(self, n: int) -> int:
        @cache
        def dfs(i: int, mask: int, lead: bool) -> int:
            if i < 0:
                return 1
            ans = 0
            for j in range(10):
                if mask >> j & 1:
                    continue

                if lead and j == 0:
                    ans += dfs(i - 1, mask, True)
                else:
                    ans += dfs(i - 1, mask | 1 << j, False)
            return ans

        return dfs(n - 1, 0, True)
```

• SimplyLeet

Round 8 · Low · Medium

p.3133

```
# Define the function to count numbers with unique digits
def countNumbersWithUniqueDigits(n):
    # For n = 0, only number valid is 0.
    if n == 0:
        return 1

    # Initialize result with one count for '0'
    result = 1

    # For numbers with more than 0 digits, iterate through digit lengths 1 to n
    # (max 10 digits)

    for k in range(1, min(n, 10) + 1):
        # Start with 9 options for the first digit (1 through 9)
        unique_count = 9
        # For remaining digits, each digit's options reduce as digits are used
        for j in range(1, k):
            unique_count *= (10 - j)
        # Add count for k-digit numbers to result
        result += unique_count

    return result

# Example usage:
n = 2
print(countNumbersWithUniqueDigits(n)) # Expected output: 91

# You can test other cases by calling countNumbersWithUniqueDigits with
different n values.
```

Round 8 · Low · Medium

p.3134

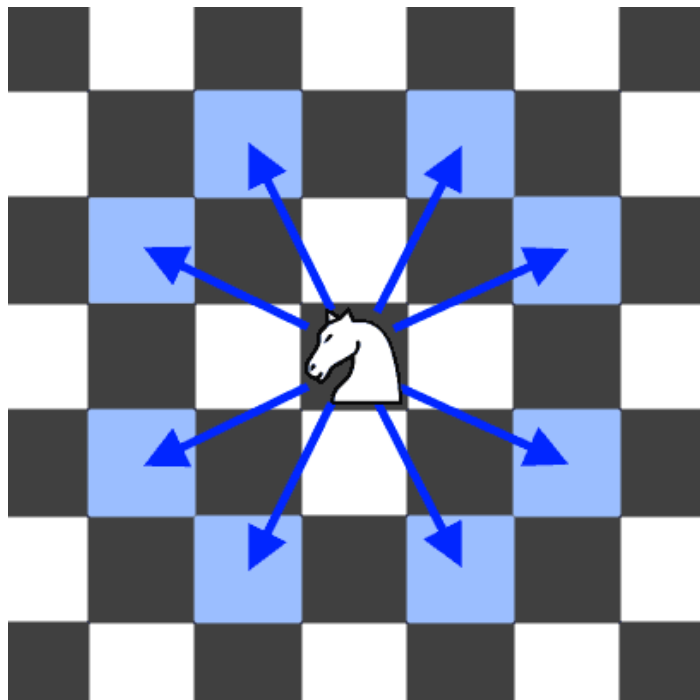
Probability DP**Round 8 · Low · Medium · 3 questions****Round 8 · Low · Medium****p.3135****Probability DP**

□ #688 Knight Probability in Chessboard**MED****LeetDoocs · SimplyLeet · WalkCC · LeetCode
Dynamic Programming****Lists: AlgoMaster 600****Problem**

On an $n \times n$ chessboard, a knight starts at the cell (row, column) and attempts to make exactly k moves. The rows and columns are 0-indexed, so the top-left cell is (0, 0), and the bottom-right cell is ($n - 1$, $n - 1$).

A chess knight has eight possible moves it can make, as illustrated below. Each move is two cells in a cardinal direction, then one cell in an orthogonal direction.

Round 8 · Low · Medium**p.3136**



Each time the knight is to move, it chooses one of eight possible moves uniformly at random (even if the piece would go off the chessboard) and moves there.

The knight continues moving until it has made exactly k moves or has moved off the chessboard.

Return the probability that the knight remains on the board after it has stopped moving.

Round 8 · Low · Medium

p.3137

Example 1:

Input: $n = 3$, $k = 2$, $row = 0$, $column = 0$

Output: 0.06250

Explanation: There are two moves (to (1,2), (2,1)) that will keep the knight on the board.

From each of those positions, there are also two moves that will keep the knight on the board.

The total probability the knight stays on the board is 0.0625.

Example 2:

Input: $n = 1$, $k = 0$, $row = 0$, $column = 0$

Output: 1.00000

Constraints:

- $1 \leq n \leq 25$
- $0 \leq k \leq 100$
- $0 \leq row, column \leq n - 1$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def knightProbability(self, n: int, k: int, row: int, column: int) -> float:
        DIRS = ((1, 2), (2, 1), (2, -1), (1, -2),
                (-1, -2), (-2, -1), (-2, 1), (-1, 2))
        # dp[i][j] := the probability to stand on (i, j)
        dp = [[0] * n for _ in range(n)]
        dp[row][column] = 1.0

        for _ in range(k):
            newDp = [[0] * n for _ in range(n)]
```

Round 8 · Low · Medium

p.3138

```

for i in range(n):
    for j in range(n):
        for dx, dy in DIRS:
            x = i + dx
            y = j + dy
            if 0 <= x < n and 0 <= y < n:
                newDp[i][j] += dp[x][y]
        dp = newDp

return sum(map(sum, dp)) / 8**k

```

• LeetCode

```

class Solution:
    def knightProbability(self, n: int, k: int, row: int, column: int) -> float:
        f = [[[0] * n for _ in range(n)] for _ in range(k + 1)]
        for i in range(n):
            for j in range(n):
                f[0][i][j] = 1
        for h in range(1, k + 1):
            for i in range(n):
                for j in range(n):
                    for a, b in pairwise((-2, -1, 2, 1, -2, 1, 2, -1, -2)):
                        x, y = i + a, j + b
                        if 0 <= x < n and 0 <= y < n:
                            f[h][i][j] += f[h - 1][x][y] / 8
        return f[k][row][column]

```

• SimplyLeet

```

# Python solution using iterative DP

def knightProbability(n, k, row, column):
    # Define the 8 possible moves for a knight
    moves = [(2, 1), (1, 2), (-1, 2), (-2, 1),
              (-2, -1), (-1, -2), (1, -2), (2, -1)]

    # Initialize dp with probability 0 for each cell, and set starting cell to 1
    dp = [[0 for _ in range(n)] for _ in range(n)]
    dp[row][column] = 1.0

    # Iterate for each move
    for move_num in range(k):
        # Create temporary dp for the next move
        dp_next = [[0 for _ in range(n)] for _ in range(n)]
        # Process each cell
        for i in range(n):
            for j in range(n):
                if dp[i][j] > 0:
                    # Try each knight move
                    for dx, dy in moves:
                        ni, nj = i + dx, j + dy
                        # Check board boundaries
                        if 0 <= ni < n and 0 <= nj < n:
                            dp_next[ni][nj] += dp[i][j] / 8.0

        # Update dp for the current move
        dp = dp_next

    # Sum probabilities of all cells for the final answer
    total_probability = sum(map(sum, dp))

    return total_probability

# Example usage:
#print(knightProbability(3, 2, 0, 0))

```

Probability DP

□ #808 Soup Servings

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · Dynamic Programming · Probability and Statistics

Lists: AlgoMaster 600

Problem

You have two soups, A and B, each starting with n mL. On every turn, one of the following four serving operations is chosen at random, each with probability 0.25 independent of all previous turns:

- pour 100 mL from type A and 0 mL from type B
- pour 75 mL from type A and 25 mL from type B
- pour 50 mL from type A and 50 mL from type B
- pour 25 mL from type A and 75 mL from type B

Note:

- There is no operation that pours 0 mL from A and 100 mL from B.
- The amounts from A and B are poured simultaneously during the turn.
- If an operation asks you to pour more than you have left of a soup, pour all that remains of that soup.

The process stops immediately after any turn in which one of the soups is used up.

Return the probability that A is used up before B, plus half the probability that both soups are used up in the same turn. Answers within 10⁻⁵ of the actual answer will be accepted.

Example 1:

```
Input: n = 50
Output: 0.62500
Explanation:
If we perform either of the first two serving operations, soup A will become empty first.
If we perform the third operation, A and B will become empty at the same time.
If we perform the fourth operation, B will become empty first.
So the total probability of A becoming empty first plus half the probability that A and B become empty at the same time, is  $0.25 * (1 + 1 + 0.5 + 0) = 0.625$ .
```

Example 2:

Input: n = 100
Output: 0.71875

Explanation:

If we perform the first serving operation, soup A will become empty first.
If we perform the second serving operations, A will become empty on performing operation [1, 2, 3], and both A and B become empty on performing operation 4.
If we perform the third operation, A will become empty on performing operation [1, 2], and both A and B become empty on performing operation 3.
If we perform the fourth operation, A will become empty on performing operation 1, and both A and B become empty on performing operation 2.
So the total probability of A becoming empty first plus half the probability that A and B become empty at the same time, is 0.71875.

Constraints:

- $0 \leq n \leq 109$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def soupServings(self, n: int) -> float:
        @functools.lru_cache(None)
        def dfs(a: int, b: int) -> float:
            if a <= 0 and b <= 0:
                return 0.5
            if a <= 0:
                return 1.0
            if b <= 0:
                return 0.0

            return 0.25 * (dfs(a - 4, b) +
                          dfs(a - 3, b - 1) +
                          dfs(a - 2, b - 2) +
                          dfs(a - 1, b - 3))

        return 1 if n >= 4800 else dfs((n + 24) // 25, (n + 24) // 25)
```

• LeetCode

```
class Solution:
    def soupServings(self, n: int) -> float:
        @cache
        def dfs(i: int, j: int) -> float:
            if i <= 0 and j <= 0:
                return 0.5
            if i <= 0:
                return 1
            if j <= 0:
                return 0

            return 0.25 * (
                dfs(i - 4, j)
                + dfs(i - 3, j - 1)
                + dfs(i - 2, j - 2)
                + dfs(i - 1, j - 3)
            )

        return 1 if n > 4800 else dfs((n + 24) // 25, (n + 24) // 25)
```

• SimplyLeet

```
import math
from functools import lru_cache

def soupServings(n):
    # For large n, the probability tends to 1 directly.
    if n > 4800:
        return 1.0
    # Scale down n by 25 using ceiling division.
    N = (n + 24) // 25

    @lru_cache(maxsize=None)
    def dp(a, b):
        # Both soups are depleted simultaneously.
        if a <= 0 and b <= 0:
            return 0.5
        # If only soup A is empty, it counts as a valid case.
        if a <= 0:
            return 1.0
        # If only soup B is empty, it's not a favorable outcome.
        if b <= 0:
```



```

    return 0.0
    # Calculate probability by averaging outcomes of the four operations.
    return 0.25 * (dp(a - 4, b) + dp(a - 3, b - 1) +
                  dp(a - 2, b - 2) + dp(a - 1, b - 3))

return dp(N, N)

# Example usage:
#print(soupServings(50)) # Expected output: 0.62500

```

Probability DP

□ #837 New 21 Game

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Math · Dynamic Programming · Sliding Window
 · Probability and Statistics
 Lists: AlgoMaster 600

Problem

Alice plays the following game, loosely based on the card game "21".

Alice starts with 0 points and draws numbers while she has less than k points. During each draw, she gains an integer number of points randomly from the range [1, maxPts], where maxPts is an integer. Each draw is independent and the outcomes have equal probabilities.

Alice stops drawing numbers when she gets k or more points.

Return the probability that Alice has n or fewer points.

Answers within 10⁻⁵ of the actual answer are considered accepted.

Example 1:

Input: $n = 10$, $k = 1$, $\text{maxPts} = 10$
 Output: 1.00000
 Explanation: Alice gets a single card, then stops.

Example 2:

Input: $n = 6$, $k = 1$, $\text{maxPts} = 10$
 Output: 0.60000
 Explanation: Alice gets a single card, then stops.
 In 6 out of 10 possibilities, she is at or below 6 points.

Example 3:

Input: $n = 21$, $k = 17$, $\text{maxPts} = 10$
 Output: 0.73278

Constraints:

- $0 \leq k \leq n \leq 104$
- $1 \leq \text{maxPts} \leq 104$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def new21Game(self, n: int, k: int, maxPts: int) -> float:
        # When the game ends, the point is in [k..k - 1 maxPts].
        # P = 1, if n >= k - 1 + maxPts
        # P = 0, if n < k (note that the constraints already have k <= n)
        if k == 0 or n >= k - 1 + maxPts:
            return 1.0

        ans = 0.0
        dp = [1.0] + [0] * n # dp[i] := the probability to have i points
```

```
windowSum = dp[0] # P(i - 1) + P(i - 2) + ... + P(i - maxPts)

for i in range(1, n + 1):
    # The probability to get i points is
    # P(i) = [P(i - 1) + P(i - 2) + ... + P(i - maxPts)] / maxPts
    dp[i] = windowSum / maxPts
    if i < k:
        windowSum += dp[i]
    else: # The game ends.
        ans += dp[i]

        if i - maxPts >= 0:
            windowSum -= dp[i - maxPts]

return ans
```

• LeetCode

```
class Solution:
    def new21Game(self, n: int, k: int, maxPts: int) -> float:
        @cache
        def dfs(i: int) -> float:
            if i >= k:
                return int(i <= n)
            if i == k - 1:
                return min(n - k + 1, maxPts) / maxPts
            return dfs(i + 1) + (dfs(i + 1) - dfs(i + maxPts + 1)) / maxPts

        return dfs(0)
```

```
class Solution:
    def new21Game(self, n: int, k: int, maxPts: int) -> float:
        f = [0] * (k + maxPts)
        for i in range(k, min(n + 1, k + maxPts)):
            f[i] = 1
        f[k - 1] = min(n - k + 1, maxPts) / maxPts
        for i in range(k - 2, -1, -1):
            f[i] = f[i + 1] + (f[i + 1] - f[i + maxPts + 1]) / maxPts
        return f[0]
```

• SimplyLeet

```
# Python solution for New 21 Game
class Solution:
    def new21Game(self, n: int, k: int, maxPts: int) -> float:
        # Edge case: If k is 0 then Alice doesn't draw any card, always count
        # as 1.
        if k == 0:
            return 1.0

        # dp[i] will represent the probability of reaching exactly i points.
        dp = [0.0] * (n + 1)
        dp[0] = 1.0 # start with 0 points, probability 1

        # W is the cumulative probability for states that can transition into
        # the current state.
        w = 1.0
        result = 0.0

        # Calculate probabilities for each score from 1 to n.
        for i in range(1, n + 1):
            dp[i] = w / maxPts
            # Only add probabilities for states less than k to the window
            # because game ends when score >= k.
            if i < k:
                w += dp[i]
            else:
                # For i >= k, these states represent a terminal state where
                # Alice stops.
                result += dp[i]
                # If the window has exceeded the size maxPts, subtract the oldest
                # probability.
                if i - maxPts >= 0:
                    w -= dp[i - maxPts]

        return result

# Example usage:
# solution = Solution()
# print(solution.new21Game(21, 17, 10))
```

Round 8 · Low · Medium

p.3149

State Machine DP

Round 8 · Low · Medium · 2 questions

Round 8 · Low · Medium

p.3150

State Machine DP

☐ #309 Best Time to Buy and Sell Stock with Cooldown

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Dynamic Programming
 Lists: Denny Zhang

Problem

You are given an array `prices` where `prices[i]` is the price of a given stock on the i th day.

Find the maximum profit you can achieve. You may complete as many transactions as you like (i.e., buy one and sell one share of the stock multiple times) with the following restrictions:

- After you sell your stock, you cannot buy stock on the next day (i.e., cooldown one day).

Note: You may not engage in multiple transactions simultaneously (i.e., you must sell the stock before you buy again).

Example 1:

Input: `prices = [1,2,3,0,2]`
 Output: 3
 Explanation: `transactions = [buy, sell, cooldown, buy, sell]`

Example 2:

Round 8 · Low · Medium

p.3151

Input: `prices = [1]`
 Output: 0

Constraints:

- $1 \leq \text{prices.length} \leq 5000$
- $0 \leq \text{prices}[i] \leq 1000$

Community Solutions (Python)

☐ • WalkCC

```
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        sell = 0
        hold = -math.inf
        prev = 0

        for price in prices:
            cache = sell
            sell = max(sell, hold + price)
            hold = max(hold, prev - price)

            prev = cache

        return sell
```

☐ • LeetDoocs

```
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        @cache
        def dfs(i: int, j: int) -> int:
            if i >= len(prices):
                return 0
            ans = dfs(i + 1, j)
            if j:
                ans = max(ans, prices[i] + dfs(i + 2, 0))
            else:
```

Round 8 · Low · Medium

p.3152

```

        ans = max(ans, -prices[i] + dfs(i + 1, 1))
    return ans

    return dfs(0, 0)

class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        n = len(prices)
        f = [[0] * 2 for _ in range(n)]
        f[0][1] = -prices[0]
        for i in range(1, n):
            f[i][0] = max(f[i - 1][0], f[i - 1][1] + prices[i])
            f[i][1] = max(f[i - 1][1], f[i - 2][0] - prices[i])
        return f[n - 1][0]

class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        f, f0, f1 = 0, 0, -prices[0]
        for x in prices[1:]:
            f, f0, f1 = f0, max(f0, f1 + x), max(f1, f - x)
        return f0

```

• SimplyLeet

```

# Python Solution with line-by-line comments

def maxProfit(prices):
    # Handle edge case: if there are no prices, profit is 0
    if not prices:
        return 0

    # Initialize states:
    # s0: Maximum profit when ready to buy
    # s1: Maximum profit when holding a stock (bought)

```

Round 8 · Low · Medium

p.3153

```

# s2: Maximum profit during cooldown after a sale
s0, s1, s2 = 0, -prices[0], 0

# Iterate over prices starting from day 1
for price in prices[1:]:
    # Store previous state values
    prev_s0, prev_s1, prev_s2 = s0, s1, s2
    # s0 can come either from staying in s0 or coming from a cooldown (s2)
    s0 = max(prev_s0, prev_s2)
    # s1 is updated by either keeping the stock or buying today with profit
    # from s0

    s1 = max(prev_s1, prev_s0 - price)
    # s2 is achieved by selling the stock that we were holding (prev_s1 +
    price)
    s2 = prev_s1 + price
    # The final maximum profit is the maximum of not holding a stock (s0 or s2)
    return max(s0, s2)

# Example usage:
print(maxProfit([1,2,3,0,2]))

```

• LC.ca

```

class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        f1, f2, f3 = -prices[0], 0, 0
        for price in prices[1:]:
            pf1, pf2, pf3 = f1, f2, f3
            f1 = max(pf1, pf3 - price)
            f2 = max(pf2, pf1 + price)
            f3 = max(pf3, pf2) # cooldown
        return f2

```

Round 8 · Low · Medium

p.3154

```
#####

class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        f, f0, f1 = 0, 0, -prices[0]
        for x in prices[1:]:
            f, f0, f1 = f0, max(f0, f1 + x), max(f1, f - x)
        return f0

#####
```

```
class Solution(object):
    def maxProfit(self, prices):
        """
        :type prices: List[int]
        :rtype: int
        """
        if len(prices) < 2:
            return 0
        buy = [0] * len(prices)
```

```
sell = [0] * len(prices)
buy[0] = -prices[0]
buy[1] = max(-prices[1], buy[0])
sell[0] = 0
sell[1] = max(prices[1] - prices[0], 0)
for i in range(2, len(prices)):
    buy[i] = max(sell[i - 2] - prices[i], buy[i - 1])
    sell[i] = max(prices[i] + buy[i - 1], sell[i - 1])
return max(sell)
```

◆ Quick Review

Key Insights

- Use Dynamic Programming to solve the problem.
- Maintain three states to represent:

- s0: the state where you are ready to buy (not holding any stock).
- s1: the state where you are holding a stock.
- s2: the cooldown state encountered right after selling a stock.
- Transition between states:
- Transition into s0 can come from staying in s0 or coming out from the cooldown state s2.
- To enter s1, you either continue holding your stock, or buy a stock using the profit available from s0.
- The only way to reach s2 is by selling the stock from s1.
- The final answer is determined by the maximum profit when you are not holding a stock (i.e., max(s0, s2)).

Space & Time

Time Complexity: $O(n)$, where n is the number of days.

Space Complexity: $O(1)$, since only a few variables are maintained for the states.

Solution

The solution involves updating three state variables for each day:

- s0 (ready to buy) is updated as the maximum of the previous s0 and the previous cooldown s2.
- s1 (holding a stock) is updated as the maximum of the previous s1 or buying today with the profit from s0 minus the current price.
- s2 (cooldown) is updated by selling the stock held in s1 (i.e., previous s1 plus the current price). At the end of the iteration, the maximum profit will be in state s0 (ready to buy) or s2 (cooldown), since remaining in s1 means you are holding a stock, which does not constitute a realized profit.

State Machine DP

☐ #714 Best Time to Buy and Sell Stock with Transaction Fee

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode Array · Dynamic Programming · Greedy Lists: AlgoMaster 600

Problem

You are given an array prices where prices[i] is the price of a given stock on the ith day, and an integer fee representing a transaction fee.

Find the maximum profit you can achieve. You may complete as many transactions as you like, but you need to pay the transaction fee for each transaction.

Note:

- You may not engage in multiple transactions simultaneously (i.e., you must sell the stock before you buy again).
- The transaction fee is only charged once for each stock purchase and sale.

Example 1:

Input: prices = [1,3,2,8,4,9], fee = 2
 Output: 8
 Explanation: The maximum profit can be achieved by:
 - Buying at prices[0] = 1
 - Selling at prices[3] = 8
 - Buying at prices[4] = 4
 - Selling at prices[5] = 9
 The total profit is $((8 - 1) - 2) + ((9 - 4) - 2) = 8$.

Example 2:

Input: prices = [1,3,7,5,10,3], fee = 3
 Output: 6

Constraints:

- $1 \leq \text{prices.length} \leq 5 * 10^4$
- $1 \leq \text{prices}[i] < 5 * 10^4$
- $0 \leq \text{fee} < 5 * 10^4$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxProfit(self, prices: List[int], fee: int) -> int:
        sell = 0
        hold = -math.inf

        for price in prices:
            sell = max(sell, hold + price)
            hold = max(hold, sell - price - fee)

        return sell
```

• LeetCode

```
class Solution:
    def maxProfit(self, prices: List[int], fee: int) -> int:
        @cache
        def dfs(i: int, j: int) -> int:
            if i >= len(prices):
                return 0
            ans = dfs(i + 1, j)
            if j:
                ans = max(ans, prices[i] + dfs(i + 1, 0) - fee)
            else:
                ans = max(ans, -prices[i] + dfs(i + 1, 1))
            return ans

        return dfs(0, 0)
```

```
class Solution:
    def maxProfit(self, prices: List[int], fee: int) -> int:
        n = len(prices)
        f = [[0] * 2 for _ in range(n)]
        f[0][1] = -prices[0]
        for i in range(1, n):
            f[i][0] = max(f[i - 1][0], f[i - 1][1] + prices[i] - fee)
            f[i][1] = max(f[i - 1][1], f[i - 1][0] - prices[i])
        return f[n - 1][0]
```

```
class Solution:
    def maxProfit(self, prices: List[int], fee: int) -> int:
        f0, f1 = 0, -prices[0]
        for x in prices[1:]:
            f0, f1 = max(f0, f1 + x - fee), max(f1, f0 - x)
        return f0
```

• SimplyLeet


```
# Define a function to calculate maximum profit.
def maxProfit(prices, fee):
    # Base case: if prices is empty, profit is 0
    if not prices:
        return 0

    # Initialize cash (profit without stock) and hold (profit with stock)
    cash = 0
    hold = -prices[0]

    # Loop through each day's price starting from the second day
    for price in prices[1:]:
        # Temporary variable to hold previous cash value
        prev_cash = cash
        # Update cash. Either you keep your previous cash or sell the stock
        # today.
        cash = max(cash, hold + price - fee)
        # Update hold. Either you keep holding or buy the stock today.
        hold = max(hold, prev_cash - price)

    # The maximum profit is when you are not holding any stock.

    return cash

# Example usage:
prices = [1, 3, 2, 8, 4, 9]
fee = 2
print(maxProfit(prices, fee)) # Output: 8
```

Round 8 · Low · Medium

p.3161

Maths / Geometry

Round 8 · Low · Medium · 11 questions

Round 8 · Low · Medium

p.3162

Maths / Geometry

#7 Reverse Integer

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode Math

Lists: Grind 169

Problem

Given a signed 32-bit integer x , return x with its digits reversed. If reversing x causes the value to go outside the signed 32-bit integer range $[-231, 231 - 1]$, then return 0.

Assume the environment does not allow you to store 64-bit integers (signed or unsigned).

Example 1:

Input: $x = 123$
Output: 321

Example 2:

Input: $x = -123$
Output: -321

Example 3:

Input: $x = 120$
Output: 21

Constraints:

Round 8 · Low · Medium

p.3163

- $-231 \leq x \leq 231 - 1$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def reverse(self, x: int) -> int:
        ans = 0
        sign = -1 if x < 0 else 1
        x *= sign

        while x:
            ans = ans * 10 + x % 10
            x //= 10

        return 0 if ans < -2**31 or ans > 2**31 - 1 else sign * ans
```

• LeetDoocs

```
class Solution:
    def reverse(self, x: int) -> int:
        ans = 0
        mi, mx = -(2**31), 2**31 - 1
        while x:
            if ans < mi // 10 + 1 or ans > mx // 10:
                return 0
            y = x % 10
            if x < 0 and y > 0:
                y -= 10
            ans = ans * 10 + y
            x = (x - y) // 10
        return ans
```

• SimplyLeet

Round 8 · Low · Medium

p.3164

```

# Function to reverse an integer
def reverse(x: int) -> int:
    # Define the maximum and minimum limits for a 32-bit signed integer
    INT_MAX = 2**31 - 1
    INT_MIN = -2**31

    reversed_num = 0 # This will store the reversed number

    # Process each digit until x becomes 0
    while x != 0:

        # Pop the last digit from x
        # Use modulo 10 to get the last digit
        # For negative numbers, modulo may yield negative remainder
        pop = x % 10 if x > 0 else x % -10
        # Remove the last digit from x by integer division
        x = x // 10

        # Check if appending the digit would cause overflow
        if reversed_num > INT_MAX // 10 or (reversed_num == INT_MAX // 10 and
pop > 7):
            return 0

        if reversed_num < INT_MIN // 10 or (reversed_num == INT_MIN // 10 and
pop < -8):
            return 0

        # Append the digit to the reversed number
        reversed_num = reversed_num * 10 + pop

    return reversed_num

# Example usage:
print(reverse(123)) # Output: 321

print(reverse(-123)) # Output: -321
print(reverse(120)) # Output: 21

```

☐ • LC.ca

Round 8 · Low · Medium

p.3165

```

class Solution:
    def reverse(self, x: int) -> int:
        ans = 0
        mi, mx = -(2**31), 2**31 - 1
        while x:
            if ans < mi // 10 + 1 or ans > mx // 10:
                return 0
            y = x % 10
            if x < 0 and y > 0:
                y -= 10

            ans = ans * 10 + y
            x = (x - y) // 10
        return ans

```

◆ Quick Review

Key Insights

- The problem requires reversing the digits while preserving the sign.
- Watch out for integer overflow due to the reversed number exceeding the 32-bit signed integer limits.
- The approach involves repeatedly extracting the last digit from the number and appending it to a new reversed integer.
- Overflow checks are done during each iteration before actually appending the digit.

Space & Time

Round 8 · Low · Medium

p.3166

Time Complexity: $O(n)$, where n is the number of digits in the integer.

Space Complexity: $O(1)$, only constant extra space is used.

Solution

The solution uses a while loop to extract the last digit from the original number using modulo operation ($x \% 10$) and then removes this digit using integer division ($x /= 10$). A reversed number is constructed by multiplying the current reversed number by 10 and adding the extracted digit. Before appending each digit, the algorithm checks if the new reversed number will overflow a 32-bit signed integer by comparing it with the thresholds derived from `INT_MAX` and `INT_MIN`. The algorithm handles negative values by working with x directly since the modulo and division operations work properly with negatives in most languages.

Maths / Geometry

□ #43 Multiply Strings

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode

Math · String · Simulation

Lists: NeetCode 150

Problem

Given two non-negative integers `num1` and `num2` represented as strings, return the product of `num1` and `num2`, also represented as a string.

Note: You must not use any built-in BigInteger library or convert the inputs to integer directly.

Example 1:

```
Input: num1 = "2", num2 = "3"
Output: "6"
```

Example 2:

```
Input: num1 = "123", num2 = "456"
Output: "56088"
```

Constraints:

- $1 \leq \text{num1.length}, \text{num2.length} \leq 200$
- `num1` and `num2` consist of digits only.
- Both `num1` and `num2` do not contain any leading zero, except the number 0 itself.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def multiply(self, num1: str, num2: str) -> str:
        s = [0] * (len(num1) + len(num2))

        for i in reversed(range(len(num1))):
            for j in reversed(range(len(num2))):
                mult = int(num1[i]) * int(num2[j])
                summ = mult + s[i + j + 1]
                s[i + j] += summ // 10
                s[i + j + 1] = summ % 10

        for i, c in enumerate(s):
            if c != 0:
                break

        return ''.join(map(str, s[i:]))
```

• LeetDoocs

```
class Solution:
    def multiply(self, num1: str, num2: str) -> str:
        if num1 == "0" or num2 == "0":
            return "0"

        m, n = len(num1), len(num2)
        arr = [0] * (m + n)

        for i in range(m - 1, -1, -1):
            a = int(num1[i])
            for j in range(n - 1, -1, -1):
                b = int(num2[j])

                arr[i + j + 1] += a * b
            for i in range(m + n - 1, 0, -1):
                arr[i - 1] += arr[i] // 10
                arr[i] %= 10
            i = 0 if arr[0] else 1
        return "".join(str(x) for x in arr[i:])
```

Round 8 · Low · Medium

p.3169

• SimplyLeet

```
# Python implementation with line-by-line comments

def multiply(num1, num2):
    # If either string is "0", the product is "0".
    if num1 == "0" or num2 == "0":
        return "0"

    n, m = len(num1), len(num2)
    # Initialize an array to store the product results.
    result = [0] * (n + m)

    # Multiply each digit of num1 with each digit of num2.
    for i in range(n - 1, -1, -1):
        for j in range(m - 1, -1, -1):
            # Multiply current digits.
            product = int(num1[i]) * int(num2[j])
            # Position in the result array where the product should be added.
            pos = i + j + 1
            total = product + result[pos]
            result[pos] = total % 10 # Place the digit at the current index.

            result[pos - 1] += total // 10 # Carry over the remaining value.

    # Convert the result array into a string, skipping any leading zero.
    result_str = ''.join(map(str, result)).lstrip('0')
    return result_str if result_str else "0"

# Example usage:
print(multiply("123", "456")) # Expected output: "56088"
```

Round 8 · Low · Medium

p.3170

Maths / Geometry

#172 Factorial Trailing Zeroes

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode Math

Lists: AlgoMaster 600

Problem

Given an integer n , return the number of trailing zeroes in $n!$.

Note that $n! = n * (n - 1) * (n - 2) * \dots * 3 * 2 * 1$.

Example 1:

Input: $n = 3$
Output: 0
Explanation: $3! = 6$, no trailing zero.

Example 2:

Input: $n = 5$
Output: 1
Explanation: $5! = 120$, one trailing zero.

Example 3:

Input: $n = 0$
Output: 0

Constraints:

- $0 \leq n \leq 104$

Round 8 · Low · Medium

p.3171

Follow up: Could you write a solution that works in logarithmic time complexity?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def trailingZeroes(self, n: int) -> int:
        return 0 if n == 0 else n // 5 + self.trailingZeroes(n // 5)
```

• LeetDoocs

```
class Solution:
    def trailingZeroes(self, n: int) -> int:
        ans = 0
        while n:
            n //= 5
            ans += n
        return ans
```

• SimplyLeet

```
# Python solution to count trailing zeroes in n!

def trailingZeroes(n):
    # Initialize count of trailing zeroes
    count = 0
    # Loop until n becomes zero
    while n > 0:
        # Divide n by 5 to count multiples of 5
        n //= 5
        # Add the result to the overall count

        count += n
    # Return the total count of trailing zeroes
    return count
```

```
# Example usage:
print(trailingZeroes(5)) # Expected output: 1
```

Round 8 · Low · Medium

p.3172

Maths / Geometry

#204 Count Primes

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Math · Enumeration · Number Theory
 Lists: AlgoMaster 600

Problem

Given an integer n , return the number of prime numbers that are strictly less than n .

Example 1:

Input: $n = 10$
 Output: 4
 Explanation: There are 4 prime numbers less than 10, they are 2, 3, 5, 7.

Example 2:

Input: $n = 0$
 Output: 0

Example 3:

Input: $n = 1$
 Output: 0

Constraints:

- $0 \leq n \leq 5 * 10^6$

Community Solutions (Python)

- WalkCC

Round 8 · Low · Medium

p.3173

```
class Solution:
    def countPrimes(self, n: int) -> int:
        if n <= 2:
            return 0
        return sum(self._sieveEratosthenes(n))

    def _sieveEratosthenes(self, n: int) -> list[bool]:
        isPrime = [True] * n
        isPrime[0] = False
        isPrime[1] = False

        for i in range(2, int(n**0.5) + 1):
            if isPrime[i]:
                for j in range(i * i, n, i):
                    isPrime[j] = False
        return isPrime
```

• LeetDoocs

```
class Solution:
    def countPrimes(self, n: int) -> int:
        primes = [True] * n
        ans = 0
        for i in range(2, n):
            if primes[i]:
                ans += 1
                for j in range(i + i, n, i):
                    primes[j] = False
        return ans
```

• SimplyLeet

```
def countPrimes(n):
    # If n is less than 2, there are no prime numbers
    if n < 2:
        return 0
    # Initialize a boolean array indicating prime status for each number
    is_prime = [True] * n
    # 0 and 1 are not prime numbers
    is_prime[0] = is_prime[1] = False
    # Iterate over each number up to the square root of n
    p = 2
```

Round 8 · Low · Medium

p.3174

```

while p * p < n:
    if is_prime[p]:
        # Mark multiples of p starting from p*p as non-prime
        for i in range(p * p, n, p):
            is_prime[i] = False
        p += 1
# Sum up the True values to get the total count of primes
return sum(is_prime)

```

Example usage:

```
print(countPrimes(10)) # Expected output: 4
```

Round 8 · Low · Medium

p.3175

Maths / Geometry

□ #264 Ugly Number II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Hash Table · Math · Dynamic Programming ·
 Heap (Priority Queue)
 Lists: AlgoMaster 600

Problem

An ugly number is a positive integer whose prime factors are limited to 2, 3, and 5.

Given an integer n , return the n th ugly number.

Example 1:

Input: $n = 10$
 Output: 12
 Explanation: [1, 2, 3, 4, 5, 6, 8, 9, 10, 12] is the sequence of the first 10 ugly numbers.

Example 2:

Input: $n = 1$
 Output: 1
 Explanation: 1 has no prime factors, therefore all of its prime factors are limited to 2, 3, and 5.

Constraints:

- $1 \leq n \leq 1690$

Community Solutions (Python)

Round 8 · Low · Medium

p.3176

• WalkCC

```
class Solution:
    def nthUglyNumber(self, n: int) -> int:
        nums = [1]
        i2 = 0
        i3 = 0
        i5 = 0

        while len(nums) < n:
            next2 = nums[i2] * 2
            next3 = nums[i3] * 3

            next5 = nums[i5] * 5
            next = min(next2, next3, next5)
            if next == next2:
                i2 += 1
            if next == next3:
                i3 += 1
            if next == next5:
                i5 += 1
            nums.append(next)

        return nums[-1]
```

• LeetCode

```
class Solution:
    def nthUglyNumber(self, n: int) -> int:
        h = [1]
        vis = {1}
        ans = 1
        for _ in range(n):
            ans = heappop(h)
            for v in [2, 3, 5]:
                nxt = ans * v
                if nxt not in vis:
                    vis.add(nxt)
                    heappush(h, nxt)
        return ans
```

Round 8 · Low · Medium

p.3177

```
class Solution:
    def nthUglyNumber(self, n: int) -> int:
        dp = [1] * n
        p2 = p3 = p5 = 0
        for i in range(1, n):
            next2, next3, next5 = dp[p2] * 2, dp[p3] * 3, dp[p5] * 5
            dp[i] = min(next2, next3, next5)
            if dp[i] == next2:
                p2 += 1
            if dp[i] == next3:
                p3 += 1
            if dp[i] == next5:
                p5 += 1
        return dp[-1]
```

• SimplyLeet

```
# Function to return the nth ugly number
def nthUglyNumber(n: int) -> int:
    # dp array to store ugly numbers, starting with 1 as the first ugly number
    ugly_numbers = [0] * n
    ugly_numbers[0] = 1

    # pointers for multiples of 2, 3, and 5
    i2 = i3 = i5 = 0

    # iterate to generate ugly numbers up to the nth

    for i in range(1, n):
        # Next possible ugly numbers from each prime factor
        next_multiple_of_2 = ugly_numbers[i2] * 2
        next_multiple_of_3 = ugly_numbers[i3] * 3
        next_multiple_of_5 = ugly_numbers[i5] * 5

        # Choose the smallest one as the next ugly number
        next_ugly = min(next_multiple_of_2, next_multiple_of_3, next_multiple_of_5)
        ugly_numbers[i] = next_ugly
```

Round 8 · Low · Medium

p.3178

```
# Move pointers if they were used
if next_ugly == next_multiple_of_2:
    i2 += 1
if next_ugly == next_multiple_of_3:
    i3 += 1
if next_ugly == next_multiple_of_5:
    i5 += 1
```

```
# The last element in the list is the nth ugly number
return ugly_numbers[-1]
```

```
# Example usage:
print(nthUglyNumber(10)) # Output: 12
```

Maths / Geometry

□ #287 Find the Duplicate Number

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Two Pointers · Binary Search · Bit
 Manipulation
 Lists: Grind 169

Problem

Given an array of integers `nums` containing $n + 1$ integers where each integer is in the range $[1, n]$ inclusive.

There is only one repeated number in `nums`, return this repeated number.

You must solve the problem without modifying the array `nums` and using only constant extra space.

Example 1:

Input: `nums = [1,3,4,2,2]`
 Output: 2

Example 2:

Input: `nums = [3,1,3,4,2]`
 Output: 3

Example 3:

Input: nums = [3,3,3,3,3]
Output: 3

Constraints:

- $1 \leq n \leq 105$
- `nums.length == n + 1`
- $1 \leq \text{nums}[i] \leq n$
- All the integers in `nums` appear only once except for precisely one integer which appears two or more times.

Follow up:

- How can we prove that at least one duplicate number must exist in `nums`?
- Can you solve the problem in linear runtime complexity?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findDuplicate(self, nums: list[int]) -> int:
        slow = nums[nums[0]]
        fast = nums[nums[nums[0]]]

        while slow != fast:
            slow = nums[slow]
            fast = nums[nums[fast]]

        slow = nums[0]
```

```
while slow != fast:
    slow = nums[slow]
    fast = nums[fast]

return slow
```

• LeetDoocs

```
class Solution:
    def findDuplicate(self, nums: List[int]) -> int:
        def f(x: int) -> bool:
            return sum(v <= x for v in nums) > x

        return bisect_left(range(len(nums)), True, key=f)
```

• SimplyLeet

```
# Function to find the duplicate number using Floyd's Tortoise and Hare algorithm.
def findDuplicate(nums):
    # Phase 1: Find the intersection point inside the cycle.
    tortoise = nums[0] # Initialize tortoise pointer.
    hare = nums[0] # Initialize hare pointer.
    while True:
        tortoise = nums[tortoise] # Move tortoise one step.
        hare = nums[nums[hare]] # Move hare two steps.
        if tortoise == hare: # If pointers meet, cycle is detected.
            break

    # Phase 2: Find the entrance to the cycle.
    pointer = nums[0] # Initialize a new pointer from the start.
    while pointer != tortoise:
        pointer = nums[pointer] # Move pointer one step.
        tortoise = nums[tortoise] # Move tortoise one step.
    return pointer # The duplicate number.

# Example usage:
if __name__ == "__main__":
```

```
sample = [1, 3, 4, 2, 2]
print(findDuplicate(sample)) # Expected output: 2
```

• LC.ca

```
class Solution:
    def findDuplicate(self, nums: List[int]) -> int:
        def f(x: int) -> bool:
            return sum(v <= x for v in nums) > x

        return bisect_left(range(len(nums)), True, key=f)
```

◆ Quick Review

Key Insights

- Since there are $n+1$ elements with values only from 1 to n , by the pigeonhole principle at least one number must be repeated.
- The problem can be approached by mapping the array values to indices, forming a cycle structure.
- Floyd's Tortoise and Hare algorithm (cycle detection) can be applied to find the duplicate number in linear time and constant space.
- There is no need for extra data structures like hash sets or alterations to the original array, satisfying the space and array-modification constraints.

Space & Time

Round 8 · Low · Medium

p.3183

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Solution

The solution leverages Floyd's Tortoise and Hare cycle detection algorithm. Think of each array element as a pointer to another index, forming a linked list with a cycle (the duplicate number creates the cycle). The algorithm works in two phases:

- Detect a meeting point within the cycle by having two pointers (tortoise and hare) move at different speeds.
 - Find the cycle entrance by resetting one pointer to the start and moving both pointers at the same speed. The meeting point reveals the duplicate number.
- This approach provides an elegant solution that satisfies both linear runtime and constant space requirements.

Round 8 · Low · Medium

p.3184

Maths / Geometry

#593 Valid Square

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · Geometry
Lists: AlgoMaster 600

Problem

Given the coordinates of four points in 2D space p_1 , p_2 , p_3 and p_4 , return true if the four points construct a square.

The coordinate of a point p_i is represented as $[x_i, y_i]$. The input is not given in any order.

A valid square has four equal sides with positive length and four equal angles (90-degree angles).

Example 1:

Input: $p_1 = [0,0]$, $p_2 = [1,1]$, $p_3 = [1,0]$, $p_4 = [0,1]$
Output: true

Example 2:

Input: $p_1 = [0,0]$, $p_2 = [1,1]$, $p_3 = [1,0]$, $p_4 = [0,12]$
Output: false

Example 3:

Round 8 · Low · Medium

p.3185

Input: $p_1 = [1,0]$, $p_2 = [-1,0]$, $p_3 = [0,1]$, $p_4 = [0,-1]$
Output: true

Constraints:

- $p_1.length == p_2.length == p_3.length == p_4.length == 2$
- $-104 \leq x_i, y_i \leq 104$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def validSquare(
        self,
        p1: list[int],
        p2: list[int],
        p3: list[int],
        p4: list[int],
    ) -> bool:
        def dist(p1: list[int], p2: list[int]) -> int:
            return (p1[0] - p2[0])**2 + (p1[1] - p2[1])**2

        distSet = set([dist(*pair)
                       for pair in list(
                           itertools.combinations([p1, p2, p3, p4], 2))]
                       )
        return 0 not in distSet and len(distSet) == 2
```

• LeetDoocs

Round 8 · Low · Medium

p.3186

```

class Solution:
    def validSquare(
        self, p1: List[int], p2: List[int], p3: List[int], p4: List[int]
    ) -> bool:
        def check(a, b, c):
            (x1, y1), (x2, y2), (x3, y3) = a, b, c
            d1 = (x1 - x2) * (x1 - x2) + (y1 - y2) * (y1 - y2)
            d2 = (x1 - x3) * (x1 - x3) + (y1 - y3) * (y1 - y3)
            d3 = (x2 - x3) * (x2 - x3) + (y2 - y3) * (y2 - y3)
            return any(

                [
                    d1 == d2 and d1 + d2 == d3 and d1,
                    d2 == d3 and d2 + d3 == d1 and d2,
                    d1 == d3 and d1 + d3 == d2 and d1,
                ]
            )

        return (
            check(p1, p2, p3)
            and check(p2, p3, p4)

            and check(p1, p3, p4)
            and check(p1, p2, p4)
        )

```

• SimplyLeet

```

# Define a function to calculate squared distance between two points
def squared_distance(p, q):
    # Calculate the squared difference in x and y coordinates
    return (p[0] - q[0]) ** 2 + (p[1] - q[1]) ** 2

def validSquare(p1, p2, p3, p4):
    # List of all points
    points = [p1, p2, p3, p4]
    distances = []

```

```

    # Compute squared distances for each unique pair among 4 points (6 pairs in
    total)
    for i in range(4):
        for j in range(i+1, 4):
            dist = squared_distance(points[i], points[j])
            if dist == 0:
                # If two points are the same, it's not a square
                return False
            distances.append(dist)

    # Use a dictionary to count frequency of each unique distance

    freq = {}
    for d in distances:
        freq[d] = freq.get(d, 0) + 1

    # There must be exactly 2 distinct distances
    if len(freq) != 2:
        return False

    # Identify side and diagonal distances based on frequency count
    side, diag = sorted(freq, key=lambda x: (freq[x], x))

    # For a valid square, the side should appear 4 times and diagonal 2 times
    return freq[side] == 4 and freq[diag] == 2

# Example usage:
print(validSquare([0,0], [1,1], [1,0], [0,1])) # Expected output: True

```

Maths / Geometry

#611 Valid Triangle Number

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Two Pointers · Binary Search · Greedy ·
 Sorting

Lists: AlgoMaster 600

Problem

Given an integer array `nums`, return the number of triplets chosen from the array that can make triangles if we take them as side lengths of a triangle.

Example 1:

Input: `nums = [2,2,3,4]`
 Output: 3
 Explanation: Valid combinations are:
 2,3,4 (using the first 2)
 2,3,4 (using the second 2)
 2,2,3

Example 2:

Input: `nums = [4,2,3,4]`
 Output: 4

Constraints:

- $1 \leq \text{nums.length} \leq 1000$
- $0 \leq \text{nums}[i] \leq 1000$

Round 8 · Low · Medium

p.3189

Community Solutions (Python)

• WalkCC

```
class Solution:
    def triangleNumber(self, nums: List[int]) -> int:
        ans = 0

        nums.sort()

        for k in range(len(nums) - 1, 1, -1):
            i = 0
            j = k - 1
            while i < j:
                if nums[i] + nums[j] > nums[k]:
                    ans += j - i
                    j -= 1
                else:
                    i += 1

        return ans
```

• LeetDoocs

```
class Solution:
    def triangleNumber(self, nums: List[int]) -> int:
        nums.sort()
        ans, n = 0, len(nums)
        for i in range(n - 2):
            for j in range(i + 1, n - 1):
                k = bisect_left(nums, nums[i] + nums[j], lo=j + 1) - 1
                ans += k - j
        return ans
```

• SimplyLeet

Round 8 · Low · Medium

p.3190

Python solution for Valid Triangle Number

```
def triangleNumber(nums):
    # First, sort the array to facilitate the two-pointer approach
    nums.sort()
    count = 0
    n = len(nums)

    # Iterate from the end of the array towards the beginning, selecting the
    # largest side at position k
    for k in range(n - 1, 1, -1):

        left = 0          # Start pointer for the smallest side
        right = k - 1     # End pointer for the middle side

        # While the left pointer is less than the right pointer
        while left < right:
            # Check if the sum of the two smaller sides is greater than the
            # largest side
            if nums[left] + nums[right] > nums[k]:
                # All indices from left to right-1 form a valid triangle with
                # nums[right] and nums[k]
                count += right - left
                # Move the right pointer one step left to find a new pair

                right -= 1
            else:
                # Move the left pointer one step right to try and increase the
                # sum
                left += 1

        return count

# Example usage:
# input_nums = [2,2,3,4]
# print(triangleNumber(input_nums)) # Expected output: 3
```

Maths / Geometry

□ #939 Minimum Area Rectangle

MED

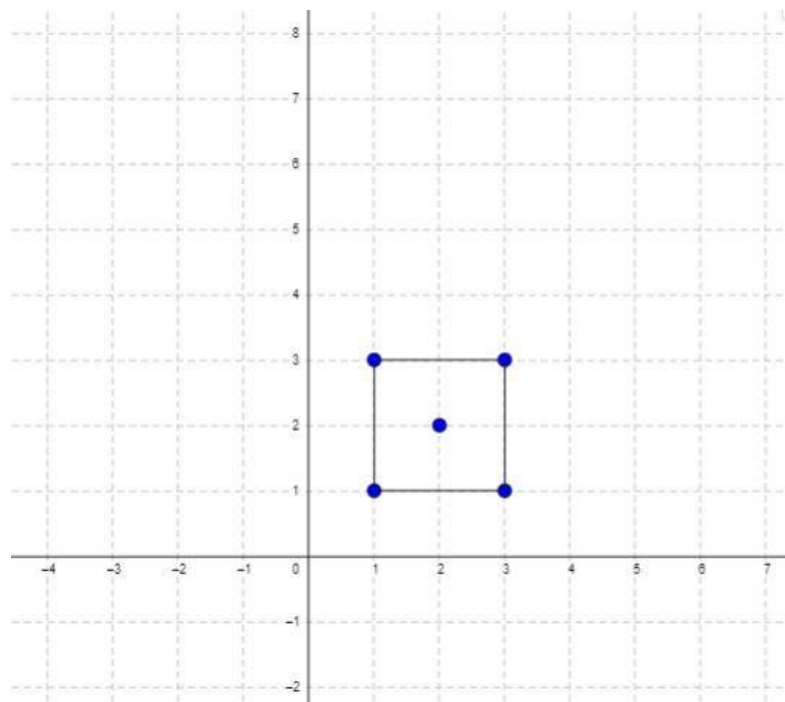
LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table · Math · Geometry · Sorting
 Lists: AlgoMaster 600

Problem

You are given an array of points in the X-Y plane where $\text{points}[i] = [x_i, y_i]$.

Return the minimum area of a rectangle formed from these points, with sides parallel to the X and Y axes. If there is not any such rectangle, return 0.

Example 1:

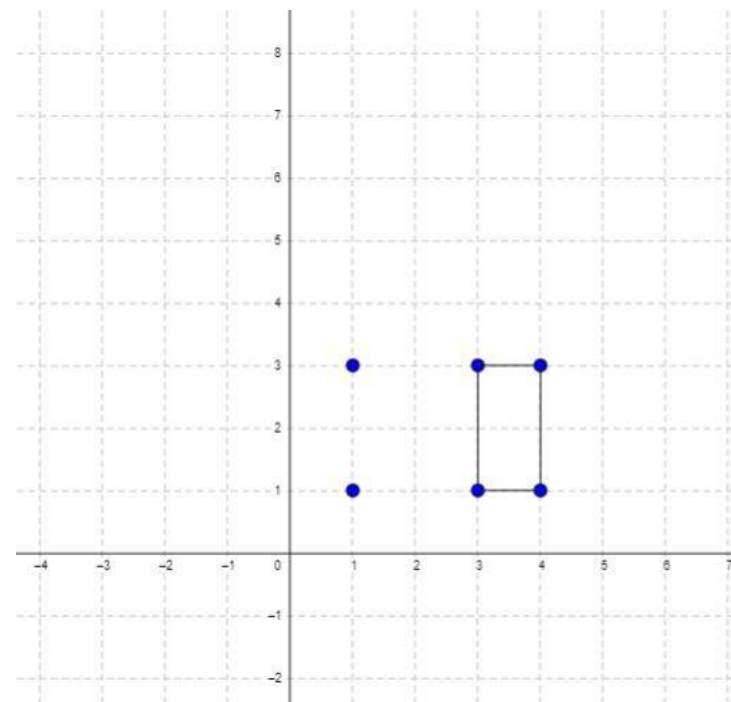


Input: points = [[1,1],[1,3],[3,1],[3,3],[2,2]]
Output: 4

Example 2:

Round 8 · Low · Medium

p.3193



Input: points = [[1,1],[1,3],[3,1],[3,3],[4,1],[4,3]]
Output: 2

Constraints:

- $1 \leq \text{points.length} \leq 500$
- $\text{points}[i].\text{length} == 2$
- $0 \leq x_i, y_i \leq 4 * 10^4$
- All the given points are unique.

Community Solutions (Python)

- WalkCC

Round 8 · Low · Medium

p.3194

```
class Solution:
    def minAreaRect(self, points: list[list[int]]) -> int:
        ans = math.inf
        xToYs = collections.defaultdict(set)

        for x, y in points:
            xToYs[x].add(y)

        for i in range(len(points)):
            for j in range(i):

                x1, y1 = points[i]
                x2, y2 = points[j]
                if x1 == x2 or y1 == y2:
                    continue
                if y2 in xToYs[x1] and y1 in xToYs[x2]:
                    ans = min(ans, abs(x1 - x2) * abs(y1 - y2))

        return ans if ans < math.inf else 0
```

• LeetCode

```
class Solution:
    def minAreaRect(self, points: List[List[int]]) -> int:
        d = defaultdict(list)
        for x, y in points:
            d[x].append(y)
        pos = {}
        ans = inf
        for x in sorted(d):
            ys = d[x]
            ys.sort()

            n = len(ys)
            for i, y1 in enumerate(ys):
                for y2 in ys[i + 1 :]:
                    if (y1, y2) in pos:
                        ans = min(ans, (x - pos[(y1, y2)]) * (y2 - y1))
                        pos[(y1, y2)] = x
        return 0 if ans == inf else ans
```

• SimplyLeet

Round 8 · Low · Medium

p.3195

```
# Python solution with step-by-step comments
class Solution:
    def minAreaRect(self, points: List[List[int]]) -> int:
        # Create a set of tuples for O(1) point lookup
        point_set = {(x, y) for x, y in points}
        n = len(points)
        min_area = float('inf')

        # Iterate through each pair of points to consider as potential
        # diagonals
        for i in range(n):
            x1, y1 = points[i]
            for j in range(i + 1, n):
                x2, y2 = points[j]
                # Check if the points are diagonally opposite (distinct x and
                # y)
                if x1 != x2 and y1 != y2:
                    # Check if the other two required corners exist
                    if (x1, y2) in point_set and (x2, y1) in point_set:
                        # Calculate the area of the rectangle
                        area = abs(x2 - x1) * abs(y2 - y1)
                        # Update the minimum area if this rectangle is smaller
                        min_area = min(min_area, area)

        # If no rectangle was found, return 0; otherwise, return the minimum
        # area
        return 0 if min_area == float('inf') else min_area
```

Round 8 · Low · Medium

p.3196

Maths / Geometry

#963 Minimum Area Rectangle II

MED

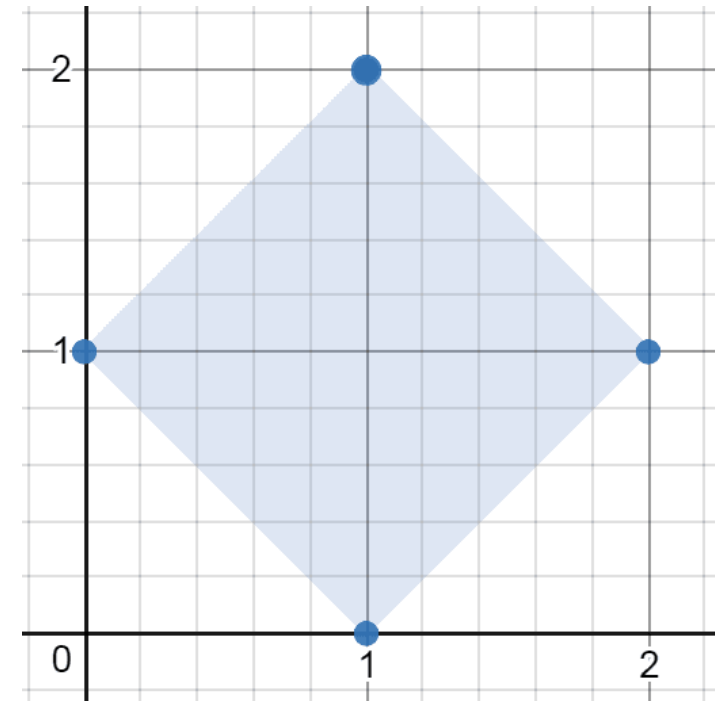
LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table · Math · Geometry
 Lists: AlgoMaster 600

Problem

You are given an array of points in the X-Y plane points[i] = [xi, yi].
 Return the minimum area of any rectangle formed from these points, with sides not necessarily parallel to the X and Y axes. If there is not any such rectangle, return 0.

Answers within 10^{-5} of the actual answer will be accepted.

Example 1:

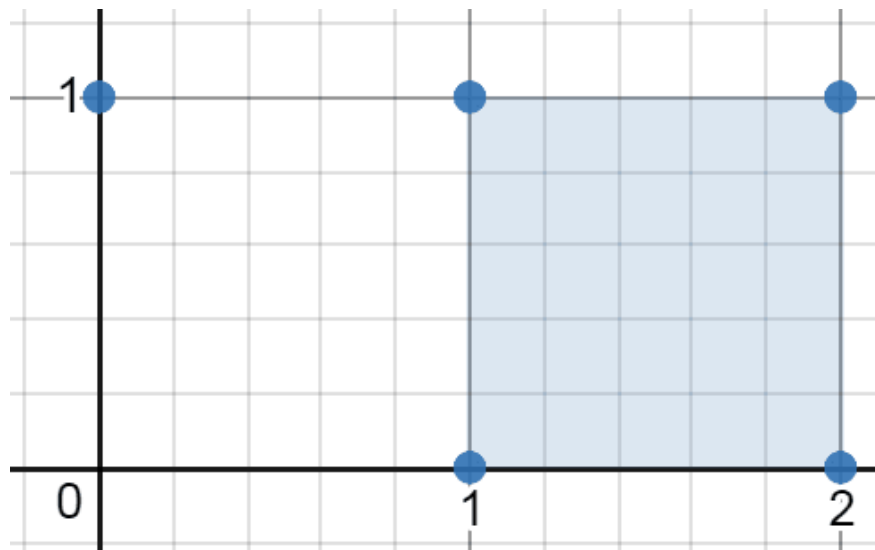


Input: points = [[1,2],[2,1],[1,0],[0,1]]

Output: 2.00000

Explanation: The minimum area rectangle occurs at [1,2],[2,1],[1,0],[0,1], with an area of 2.

Example 2:

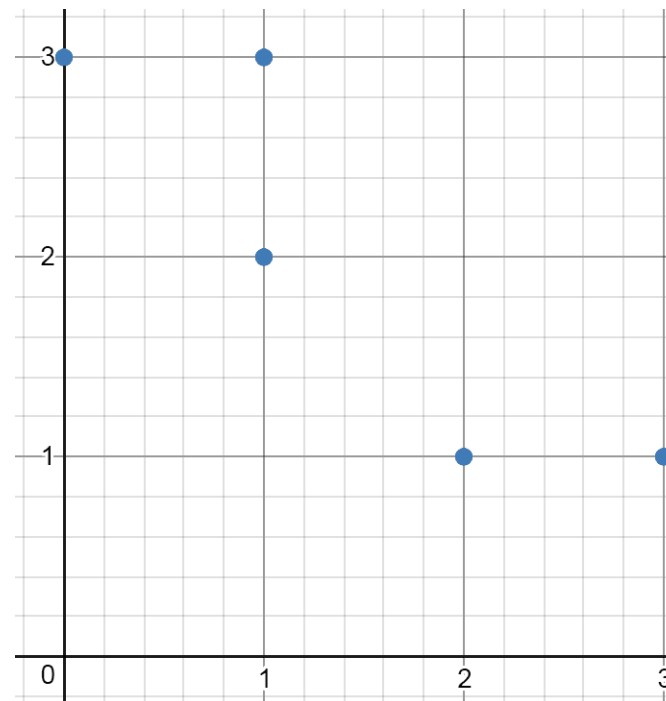


Input: points = [[0,1],[2,1],[1,1],[1,0],[2,0]]

Output: 1.00000

Explanation: The minimum area rectangle occurs at [1,0],[1,1],[2,1],[2,0], with an area of 1.

Example 3:



Input: points = [[0,3],[1,2],[3,1],[1,3],[2,1]]

Output: 0

Explanation: There is no possible rectangle to form from these points.

Constraints:

- $1 \leq \text{points.length} \leq 50$
- $\text{points}[i].\text{length} == 2$
- $0 \leq x_i, y_i \leq 4 * 10^4$
- All the given points are unique.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minAreaFreeRect(self, points: list[list[int]]) -> float:
        ans = math.inf
        # For each A, B pair points, {hash(A, B): (ax, ay, bx, by)}.
        centerToPoints = collections.defaultdict(list)

        for ax, ay in points:
            for bx, by in points:
                center = ((ax + bx) / 2, (ay + by) / 2)
                centerToPoints[center].append((ax, ay, bx, by))

        def dist(px: int, py: int, qx: int, qy: int) -> float:
            return (px - qx)**2 + (py - qy)**2

        # For all pair points "that share the same center".
        for points in centerToPoints.values():
            for ax, ay, _, _ in points:
                for cx, cy, dx, dy in points:
                    # AC is perpendicular to AD.
                    # AC dot AD = (cx - ax, cy - ay) dot (dx - ax, dy - ay) == 0.

                    if (cx - ax) * (dx - ax) + (cy - ay) * (dy - ay) == 0:
                        squaredArea = dist(ax, ay, cx, cy) * dist(ax, ay, dx, dy)
                        if squaredArea > 0:
                            ans = min(ans, squaredArea)

        return 0 if ans == math.inf else math.sqrt(ans)
```

• LeetCode

```
class Solution:
    def minAreaFreeRect(self, points: List[List[int]]) -> float:
        s = {(x, y) for x, y in points}
        n = len(points)
        ans = inf
        for i in range(n):
            x1, y1 = points[i]
            for j in range(n):
                if j != i:
                    x2, y2 = points[j]
```

Round 8 · Low · Medium

p.3201

```
        for k in range(j + 1, n):
            if k != i:
                x3, y3 = points[k]
                x4 = x2 - x1 + x3
                y4 = y2 - y1 + y3
                if (x4, y4) in s:
                    v21 = (x2 - x1, y2 - y1)
                    v31 = (x3 - x1, y3 - y1)
                    if v21[0] * v31[0] + v21[1] * v31[1] == 0:
                        w = sqrt(v21[0]**2 + v21[1]**2)

                        h = sqrt(v31[0]**2 + v31[1]**2)
                        ans = min(ans, w * h)

        return 0 if ans == inf else ans
```

• SimplyLeet

```
import math
from collections import defaultdict

def minAreaFreeRect(points):
    # Number of points
    n = len(points)
    # Dictionary to group pairs by (center, squared distance)
    diag_groups = defaultdict(list)

    # Iterate over each unique pair of points

    for i in range(n):
        for j in range(i + 1, n):
            # Compute the center as the sum of coordinates to avoid fractions
            center = (points[i][0] + points[j][0], points[i][1] + points[j][1])

            # Compute the squared distance between the two points (diagonal
            length squared)
            dx = points[i][0] - points[j][0]
            dy = points[i][1] - points[j][1]
            dist2 = dx*dx + dy*dy
            # Append the pair (points[i], points[j]) to the corresponding group
            key

            diag_groups[(center, dist2)].append((points[i], points[j]))
```

Round 8 · Low · Medium

p.3202

```

min_area = float('inf')
# For every group of pairs sharing same center and squared distance, they
form potential rectangles
for pairs in diag_groups.values():
    if len(pairs) < 2:
        continue
    # Compare every combination of two pairs from this group
    m = len(pairs)
    for i in range(m):
        for j in range(i + 1, m):

```

```

        p, q = pairs[i]
        r, s = pairs[j]
        # For rectangle, choose p as one vertex.
        # Determine vectors from p to r and p to s.
        vec_pr = (r[0] - p[0], r[1] - p[1])
        vec_ps = (s[0] - p[0], s[1] - p[1])
        # Check perpendicularity: (vec_pr) dot (vec_ps) should be zero.
        if vec_pr[0] * vec_ps[0] + vec_pr[1] * vec_ps[1] != 0:
            # If not perpendicular, try swapping r and s.
            vec_pr = (s[0] - p[0], s[1] - p[1])

```

```

        vec_ps = (r[0] - p[0], r[1] - p[1])
        if vec_pr[0] * vec_ps[0] + vec_pr[1] * vec_ps[1] != 0:
            continue # Not perpendicular; skip
        # Compute lengths of the two sides
        side1 = math.sqrt(vec_pr[0]*vec_pr[0] + vec_pr[1]*vec_pr[1])
        side2 = math.sqrt(vec_ps[0]*vec_ps[0] + vec_ps[1]*vec_ps[1])
        area = side1 * side2
        # Update the minimum area if a smaller valid rectangle is found
        if area < min_area:
            min_area = area

```

```

return 0 if min_area == float('inf') else min_area

```

```

# Example usage:
# print(minAreaFreeRect([[1,2],[2,1],[1,0],[0,1]]))

```

Maths / Geometry

□ #1922 Count Good Numbers

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode

Math · Recursion

Lists: AlgoMaster 600

Problem

A digit string is good if the digits (0-indexed) at even indices are even and the digits at odd indices are prime (2, 3, 5, or 7).

- For example, "2582" is good because the digits (2 and 8) at even positions are even and the digits (5 and 2) at odd positions are prime. However, "3245" is not good because 3 is at an even index but is not even.

Given an integer n , return the total number of good digit strings of length n . Since the answer may be large, return it modulo $10^9 + 7$.

A digit string is a string consisting of digits 0 through 9 that may contain leading zeros.

Example 1:

Input: n = 1
Output: 5
Explanation: The good numbers of length 1 are "0", "2", "4", "6", "8".

Example 2:

Input: n = 4
Output: 400

Example 3:

Input: n = 50
Output: 564908303

Constraints:

- $1 \leq n \leq 1015$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def countGoodNumbers(self, n: int) -> int:
        MOD = 1_000_000_007

        def modPow(x: int, n: int) -> int:
            if n == 0:
                return 1
            if n % 2 == 1:
                return x * modPow(x, n - 1) % MOD
            return modPow(x * x % MOD, n // 2)

        return modPow(4 * 5, n // 2) * (1 if n % 2 == 0 else 5) % MOD
```

• LeetCode

```
class Solution:
    def countGoodNumbers(self, n: int) -> int:
        mod = 10**9 + 7
        return pow(5, (n + 1) >> 1, mod) * pow(4, n >> 1, mod) % mod
```

• SimplyLeet

```
# Function to perform fast modular exponentiation
def mod_pow(base, exp, mod):
    result = 1
    while exp > 0:
        # If the current exponent is odd, multiply the result by base mod mod
        if exp % 2:
            result = (result * base) % mod
        # Square the base and reduce the exponent by half
        base = (base * base) % mod
        exp //= 2

    return result

def countGoodNumbers(n):
    mod = 10**9 + 7
    # Calculate the count of indices for even and odd positions
    even_count = (n + 1) // 2 # ceil(n/2)
    odd_count = n // 2 # floor(n/2)
    # Compute the result using modular exponentiation for both parts
    result = (mod_pow(5, even_count, mod) * mod_pow(4, odd_count, mod)) % mod
    return result

# For example: n = 4 should output 400
print(countGoodNumbers(4))
```

String Matching

Round 8 · Low · Medium · 2 questions

Round 8 · Low · Medium

p.3207

String Matching

□ #686 Repeated String Match

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · String Matching

Lists: AlgoMaster 600

Problem

Given two strings *a* and *b*, return the minimum number of times you should repeat string *a* so that string *b* is a substring of it. If it is impossible for *b* to be a substring of *a* after repeating it, return -1 .

Notice: string "abc" repeated 0 times is "", repeated 1 time is "abc" and repeated 2 times is "abcabc".

Example 1:

Input: *a* = "abcd", *b* = "cdabacdab"

Output: 3

Explanation: We return 3 because by repeating *a* three times "abcdabacdab", *b* is a substring of it.

Example 2:

Input: *a* = "a", *b* = "aa"

Output: 2

Constraints:

Round 8 · Low · Medium

p.3208

- $1 \leq a.length, b.length \leq 104$
- a and b consist of lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def repeatedStringMatch(self, a: str, b: str) -> int:
        n = math.ceil(len(b) / len(a))
        s = a * n
        if b in s:
            return n
        if b in s + a:
            return n + 1
        return -1
```

• LeetCode

```
class Solution:
    def repeatedStringMatch(self, a: str, b: str) -> int:
        m, n = len(a), len(b)
        ans = ceil(n / m)
        t = [a] * ans
        for _ in range(3):
            if b in ''.join(t):
                return ans
            ans += 1
            t.append(a)

        return -1
```

• SimplyLeet

```
# Function to determine the minimum number of times 'a' must be repeated
# so that 'b' is a substring of the repeated string.
def repeatedStringMatch(a, b):
    # Calculate the minimum repeats required using ceiling division
    repeat_count = -(-len(b) // len(a)) # This is equivalent to
    math.ceil(len(b) / len(a))
    # Build the repeated string
    repeated = a * repeat_count
    # Check if b is a substring of the repeated string
    if b in repeated:
        return repeat_count

    # Check with one additional repeat to handle potential overlap between
    # repeats
    if b in (repeated + a):
        return repeat_count + 1
    return -1

# Example usage:
print(repeatedStringMatch("abcd", "cdabcdab")) # Expected output: 3
```

String Matching

☐ #1698 Number of Distinct Substrings in a String MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode String · Trie · Rolling Hash · Suffix Array · Hash Function

Lists: AlgoMaster 600

Problem

Given a string s , return the number of distinct substrings of s .

A substring of a string is obtained by deleting any number of characters (possibly zero) from the front of the string and any number (possibly zero) from the back of the string.

Example 1:

Input: $s = \text{"aabbaba"}$

Output: 21

Explanation: The set of distinct strings is $[\text{"a"}, \text{"b"}, \text{"aa"}, \text{"bb"}, \text{"ab"}, \text{"ba"}, \text{"aab"}, \text{"abb"}, \text{"bab"}, \text{"bba"}, \text{"aba"}, \text{"aabb"}, \text{"abba"}, \text{"bbab"}, \text{"baba"}, \text{"aabbba"}, \text{"abbab"}, \text{"bbaba"}, \text{"aabbab"}, \text{"abbaba"}, \text{"aabbaba"}]$

Example 2:

Input: $s = \text{"abcdefg"}$

Output: 28

Constraints:

- $1 \leq s.length \leq 500$
- s consists of lowercase English letters.

Follow up: Can you solve this problem in $O(n)$ time complexity?

Community Solutions (Python)

☐ • WalkCC

```
class Solution:
    def countDistinct(self, s: str) -> int:
        BASE = 26
        HASH = 1_000_000_007

        n = len(s)
        ans = 0
        pow = [1] + [0] * n    # pow[i] := BASE^i
        hashes = [0] * (n + 1) # hashes[i] := the hash of s[0..i]

        def val(c: str) -> int:
            return ord(c) - ord('a')

        for i in range(1, n + 1):
            pow[i] = pow[i - 1] * BASE % HASH
            hashes[i] = (hashes[i - 1] * BASE + val(s[i - 1])) % HASH

        def getHash(l: int, r: int) -> int:
            """Returns the hash of s[l..r)."""
            hash = (hashes[r] - hashes[l] * pow[r - l]) % HASH

            return hash + HASH if hash < 0 else hash

        for length in range(1, n + 1):
            seen = set()
            for i in range(n - length + 1):
                seen.add(getHash(i, i + length))
            ans += len(seen)

        return ans
```

• LeetCode

```
class Solution:
    def countDistinct(self, s: str) -> int:
        n = len(s)
        return len({s[i:j] for i in range(n) for j in range(i + 1, n + 1)})
```

```
class Solution:
    def countDistinct(self, s: str) -> int:
        base = 131
        n = len(s)
        p = [0] * (n + 10)
        h = [0] * (n + 10)
        p[0] = 1
        for i, c in enumerate(s):
            p[i + 1] = p[i] * base
            h[i + 1] = h[i] * base + ord(c)
```

```
ss = set()
for i in range(1, n + 1):
    for j in range(i, n + 1):
        t = h[j] - h[i - 1] * p[j - i + 1]
        ss.add(t)
return len(ss)
```

• SimplyLeet

```
# Function to count distinct substrings in a string
def countDistinctSubstrings(s):
    # Initialize a set to store unique substrings
    unique_subs = set()
    n = len(s)
    # Loop over all possible starting indices
    for i in range(n):
        # Loop over all possible ending indices (non-inclusive)
        for j in range(i + 1, n + 1):
            # Add the current substring to the set
```

```
        unique_subs.add(s[i:j])
    # Return the count of unique substrings
    return len(unique_subs)

# Example usage:
s = "aabbaba"
print(countDistinctSubstrings(s)) # Expected output: 21
```

Binary Indexed Tree / Segment Tree

Round 8 · Low · Medium · 2 questions

Round 8 · Low · Medium

p.3215

Binary Indexed Tree / Segment Tree

□ #307 Range Sum Query – Mutable

MED

LeetCode · LeetCode · LeetCode · LeetCode
Array · Design · Binary Indexed Tree · Segment Tree

Lists: Denny Zhang

Problem

Given an integer array `nums`, handle multiple queries of the following types:

1. Update the value of an element in `nums`.
2. Calculate the sum of the elements of `nums` between indices `left` and `right` inclusive where `left ≤ right`.

Implement the `NumArray` class:

- `NumArray(int[] nums)` Initializes the object with the integer array `nums`.
- `void update(int index, int val)` Updates the value of `nums[index]` to be `val`.
- `int sumRange(int left, int right)` Returns the sum of the elements of `nums` between indices `left` and `right` inclusive (i.e. `nums[left] +`

Round 8 · Low · Medium

p.3216

`nums[left + 1] + ... + nums[right]]).`

Example 1:

Input
 ["NumArray", "sumRange", "update", "sumRange"]
 [[[1, 3, 5]], [0, 2], [1, 2], [0, 2]]
Output
 [null, 9, null, 8]

Explanation
 NumArray numArray = new NumArray([1, 3, 5]);

Constraints:

- $1 \leq \text{nums.length} \leq 3 * 10^4$
- $-100 \leq \text{nums}[i] \leq 100$
- $0 \leq \text{index} < \text{nums.length}$
- $-100 \leq \text{val} \leq 100$
- $0 \leq \text{left} \leq \text{right} < \text{nums.length}$
- At most $3 * 10^4$ calls will be made to update and sumRange.

Community Solutions (Python)

• WalkCC

```
class FenwickTree:
    def __init__(self, n: int):
        self.sums = [0] * (n + 1)

    def add(self, i: int, delta: int) -> None:
        while i < len(self.sums):
            self.sums[i] += delta
            i += FenwickTree.lowbit(i)

    def get(self, i: int) -> int:
```

```
summ = 0
while i > 0:
    summ += self.sums[i]
    i -= FenwickTree.lowbit(i)
return summ
```

```
@staticmethod
def lowbit(i: int) -> int:
    return i & -i
```

```
class NumArray:
    def __init__(self, nums: list[int]):
        self.nums = nums
        self.tree = FenwickTree(len(nums))
        for i, num in enumerate(nums):
            self.tree.add(i + 1, num)

    def update(self, index: int, val: int) -> None:
        self.tree.add(index + 1, val - self.nums[index])

    self.nums[index] = val

    def sumRange(self, left: int, right: int) -> int:
        return self.tree.get(right + 1) - self.tree.get(left)
```

• LeetDoocs

```
class BinaryIndexedTree:
    __slots__ = ["n", "c"]

    def __init__(self, n):
        self.n = n
        self.c = [0] * (n + 1)

    def update(self, x: int, delta: int):
        while x <= self.n:
            self.c[x] += delta
```

```

        x += x & -x

def query(self, x: int) -> int:
    s = 0
    while x > 0:
        s += self.c[x]
        x -= x & -x
    return s

class NumArray:
    __slots__ = ["tree"]

    def __init__(self, nums: List[int]):
        self.tree = BinaryIndexedTree(len(nums))
        for i, v in enumerate(nums, 1):
            self.tree.update(i, v)

    def update(self, index: int, val: int) -> None:
        prev = self.sumRange(index, index)

        self.tree.update(index + 1, val - prev)

    def sumRange(self, left: int, right: int) -> int:
        return self.tree.query(right + 1) - self.tree.query(left)

# Your NumArray object will be instantiated and called as such:
# obj = NumArray(nums)
# obj.update(index,val)
# param_2 = obj.sumRange(left,right)

```

• SimplyLeet

Round 8 · Low · Medium

p.3219

```

# Python implementation using a Binary Indexed Tree (Fenwick Tree)
class NumArray:
    def __init__(self, nums):
        # Store the original array
        self.nums = nums[:]
        self.n = len(nums)
        # Initialize the BIT with 0s, size is (n+1) because BIT is 1-indexed
        self.tree = [0] * (self.n + 1)
        # Build the BIT: update tree for each value in the nums array
        for i, num in enumerate(nums):
            self._update_tree(i, num)

    def _update_tree(self, i, num):
        # Update the BIT for index with delta change
        i = index + 1 # Convert to 1-indexed for BIT operations
        while i <= self.n:
            self.tree[i] += delta # Add delta to current tree node
            i += i & -i # Move to the next responsible node

    def update(self, index, val):
        # Calculate the difference between new value and current value
        delta = val - self.nums[index]
        self.nums[index] = val # Update the original array
        self._update_tree(index, delta) # Update the BIT to reflect this
        change

    def _prefix_sum(self, index):
        # Compute the prefix sum from start up to and including index
        result = 0
        i = index + 1 # Convert to 1-indexed
        while i:

```

Round 8 · Low · Medium

p.3220

```

        result += self.tree[i]
        i -= i & -i # Move to parent node
    return result

    def sumRange(self, left, right):
        # Return the sum of the range [left, right]
        return self._prefix_sum(right) - self._prefix_sum(left - 1) if left >
0 else self._prefix_sum(right)

# Example usage:
# numArray = NumArray([1, 3, 5])

# print(numArray.sumRange(0, 2))
# numArray.update(1, 2)
# print(numArray.sumRange(0, 2))

```

• LC.ca

```

class BinaryIndexedTree:
    def __init__(self, n):
        self.n = n
        self.c = [0] * (n + 1)

    @staticmethod
    def lowbit(x):
        return x & -x

    def update(self, x, delta):

        while x <= self.n:
            self.c[x] += delta
            x += BinaryIndexedTree.lowbit(x)

    def query(self, x):
        s = 0
        while x > 0:
            s += self.c[x]
            x -= BinaryIndexedTree.lowbit(x)
        return s

```

Round 8 · Low · Medium

p.3221

```

class NumArray:
    def __init__(self, nums: List[int]):
        self.tree = BinaryIndexedTree(len(nums))
        for i, v in enumerate(nums, 1):
            self.tree.update(i, v)

    def update(self, index: int, val: int) -> None:
        prev = self.sumRange(index, index)

        self.tree.update(index + 1, val - prev)

    def sumRange(self, left: int, right: int) -> int:
        return self.tree.query(right + 1) - self.tree.query(left)

# Your NumArray object will be instantiated and called as such:
# obj = NumArray(nums)
# obj.update(index,val)
# param_2 = obj.sumRange(left,right)

#####

# Segment tree node
class STNode(object):
    def __init__(self, start, end):
        self.start = start
        self.end = end
        self.total = 0
        self.left = None

        self.right = None

class SegmentedTree(object):
    def __init__(self, nums, start, end):
        self.root = self.buildTree(nums, start, end)

    def buildTree(self, nums, start, end):
        if start > end:
            return None

```

Round 8 · Low · Medium

p.3222

```

if start == end:
    node = STNode(start, end)
    node.total = nums[start]
    return node

```

```

mid = start + (end - start) / 2

```

```

root = STNode(start, end)
root.left = self.buildTree(nums, start, mid)

```

```

root.right = self.buildTree(nums, mid + 1, end)
root.total = root.left.total + root.right.total
return root

```

```

def updateVal(self, i, val):
    def updateVal(root, i, val):
        if root.start == root.end:
            root.total = val
            return val
        mid = root.start + (root.end - root.start) / 2

```

```

        if i <= mid:
            updateVal(root.left, i, val)
        else:
            updateVal(root.right, i, val)

        root.total = root.left.total + root.right.total
        return root.total

    return updateVal(self.root, i, val)

```

```

def sumRange(self, i, j):
    def rangeSum(root, start, end):
        if root.start == start and root.end == end:
            return root.total

        mid = root.start + (root.end - root.start) / 2
        if j <= mid:
            return rangeSum(root.left, start, end)
        elif i >= mid + 1:
            return rangeSum(root.right, start, end)

```

```

        else:
            return rangeSum(root.left, start, mid) + rangeSum(root.right, mid + 1,
end)

```

```

        return rangeSum(self.root, i, j)

```

```

class NumArray(object):
    def __init__(self, nums):
        """
        initialize your data structure here.

```

```

        :type nums: List[int]
        """
        self.stTree = SegmentedTree(nums, 0, len(nums) - 1)

```

```

    def update(self, i, val):
        """
        :type i: int
        :type val: int
        :rtype: int
        """

```

```

        return self.stTree.updateVal(i, val)

```

```

    def sumRange(self, i, j):
        """
        sum of elements nums[i..j], inclusive.
        :type i: int
        :type j: int
        :rtype: int
        """
        return self.stTree.sumRange(i, j)

```

```

# Your NumArray object will be instantiated and called as such:
# numArray = NumArray(nums)
# numArray.sumRange(0, 1)
# numArray.update(1, 10)
# numArray.sumRange(1, 2)

```

◆ Quick Review

Key Insights

- To efficiently support both point updates and range sum queries, a specialized data structure like a Binary Indexed Tree (Fenwick Tree) or a Segment Tree is ideal.
- Both data structures provide update and sum query operations in $O(\log n)$ time.
- The challenge is to maintain a mutable array while enabling fast sum computation over any specified range.

Space & Time

Time Complexity: $O(\log n)$ for both update and sumRange operations

Space Complexity: $O(n)$ to store the additional tree structure

Solution

We can use a Binary Indexed Tree (Fenwick Tree) to solve this problem. The Fenwick Tree allows for efficient prefix sum calculations and point updates, both in $O(\log n)$ time. The approach involves:

- Building the Fenwick Tree using the initial array.
- On an update, computing the difference between the new value and the

Round 8 · Low · Medium

p.3225

current value, then propagating the change appropriately in the tree. – For the sum range query, computing the prefix sum up to right and subtracting the prefix sum up to left-1.

Round 8 · Low · Medium

p.3226

Binary Indexed Tree / Segment Tree

#308 Range Sum Query 2D – Mutable

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Design · Binary Indexed Tree · Segment
Tree · Matrix

Lists: AlgoMaster 600

Problem

Given a 2D matrix `matrix`, handle multiple queries of the following types:

1. Update the value of a cell in matrix.
2. Calculate the sum of the elements of matrix inside the rectangle defined by its upper left corner (`row1`, `col1`) and lower right corner (`row2`, `col2`).

Implement the `NumMatrix` class:

- `NumMatrix(int[][] matrix)` Initializes the object with the integer matrix `matrix`.
- `void update(int row, int col, int val)` Updates the value of `matrix[row][col]` to be `val`.
- `int sumRegion(int row1, int col1, int row2, int col2)` Returns the sum of the elements of

Round 8 · Low · Medium

p.3227

matrix inside the rectangle defined by its upper left corner (`row1`, `col1`) and lower right corner (`row2`, `col2`).

Example 1:

3	0	1	4	2
5	6	3	2	1
1	2	0	1	5
4	1	0	1	7
1	0	3	0	5



3	0	1	4	2
5	6	3	2	1
1	2	0	1	5
4	1	2	1	7
1	0	3	0	5

Input
`[["NumMatrix", "sumRegion", "update", "sumRegion"],
 [[[3, 0, 1, 4, 2], [5, 6, 3, 2, 1], [1, 2, 0, 1, 5], [4, 1, 0, 1, 7], [1, 0, 3, 0, 5]]], [2, 1, 4, 3], [3, 2, 2], [2, 1, 4, 3]]`
Output
`[null, 8, null, 10]`

Explanation
`NumMatrix numMatrix = new NumMatrix([[3, 0, 1, 4, 2], [5, 6, 3, 2, 1], [1, 2, 0, 1, 5], [4, 1, 0, 1, 7], [1, 0, 3, 0, 5]]);`

Constraints:

- `m == matrix.length`
- `n == matrix[i].length`
- `1 <= m, n <= 200`
- `-1000 <= matrix[i][j] <= 1000`
- `0 <= row < m`

Round 8 · Low · Medium

p.3228

- $0 \leq \text{col} < n$
- $-1000 \leq \text{val} \leq 1000$
- $0 \leq \text{row1} \leq \text{row2} < m$
- $0 \leq \text{col1} \leq \text{col2} < n$
- At most 5000 calls will be made to `sumRegion` and `update`.

Community Solutions (Python)

• WalkCC

```
class FenwickTree:
    def __init__(self, m: int, n: int):
        self.sums = [[0] * (n + 1) for _ in range(m + 1)]

    def add(self, row: int, col: int, delta: int) -> None:
        i = row
        while i < len(self.sums):
            j = col
            while j < len(self.sums[0]):
                self.sums[i][j] += delta
                j += FenwickTree.lowbit(j)
            i += FenwickTree.lowbit(i)

    def get(self, row: int, col: int) -> int:
        summ = 0
        i = row
        while i > 0:
            j = col
            while j > 0:
                summ += self.sums[i][j]
                j += FenwickTree.lowbit(j)
            i += FenwickTree.lowbit(i)
```

```
j -= FenwickTree.lowbit(j)
i -= FenwickTree.lowbit(i)
return summ

@staticmethod
def lowbit(i: int) -> int:
    return i & -i

class NumMatrix:

    def __init__(self, matrix: list[list[int]]):
        self.matrix = matrix
        self.tree = FenwickTree(len(matrix), len(matrix[0]))

        for i in range(len(matrix)):
            for j, val in enumerate(matrix[i]):
                self.tree.add(i + 1, j + 1, val)

    def update(self, row: int, col: int, val: int) -> None:
        self.tree.add(row + 1, col + 1, val - self.matrix[row][col])

        self.matrix[row][col] = val

    def sumRegion(self, row1: int, col1: int, row2: int, col2: int) -> int:
        return (self.tree.get(row2 + 1, col2 + 1) - self.tree.get(row1, col2 + 1)
                - self.tree.get(row2 + 1, col1) + self.tree.get(row1, col1))
```

• LeetCode

```
class BinaryIndexedTree:
    def __init__(self, n):
        self.n = n
        self.c = [0] * (n + 1)

    @staticmethod
    def lowbit(x):
        return x & -x

    def update(self, x, delta):
```

```

while x <= self.n:
    self.c[x] += delta
    x += BinaryIndexedTree.lowbit(x)

def query(self, x):
    s = 0
    while x > 0:
        s += self.c[x]
        x -= BinaryIndexedTree.lowbit(x)
    return s

class NumMatrix:
    def __init__(self, matrix: List[List[int]]):
        self.trees = []
        n = len(matrix[0])
        for row in matrix:
            tree = BinaryIndexedTree(n)
            for j, v in enumerate(row):
                tree.update(j + 1, v)

        self.trees.append(tree)

    def update(self, row: int, col: int, val: int) -> None:
        tree = self.trees[row]
        prev = tree.query(col + 1) - tree.query(col)
        tree.update(col + 1, val - prev)

    def sumRegion(self, row1: int, col1: int, row2: int, col2: int) -> int:
        return sum(
            tree.query(col2 + 1) - tree.query(col1)
            for tree in self.trees[row1 : row2 + 1]
        )

# Your NumMatrix object will be instantiated and called as such:
# obj = NumMatrix(matrix)
# obj.update(row,col,val)
# param_2 = obj.sumRegion(row1,col1,row2,col2)

```

Round 8 · Low · Medium

p.3231

```

class Node:
    def __init__(self):
        self.l = 0
        self.r = 0
        self.v = 0

class SegmentTree:
    def __init__(self, nums):
        n = len(nums)

        self.nums = nums
        self.tr = [Node() for _ in range(4 * n)]
        self.build(1, 1, n)

    def build(self, u, l, r):
        self.tr[u].l = l
        self.tr[u].r = r
        if l == r:
            self.tr[u].v = self.nums[l - 1]
            return

        mid = (l + r) >> 1
        self.build(u << 1, l, mid)
        self.build(u << 1 | 1, mid + 1, r)
        self.pushup(u)

    def modify(self, u, x, v):
        if self.tr[u].l == x and self.tr[u].r == x:
            self.tr[u].v = v
            return
        mid = (self.tr[u].l + self.tr[u].r) >> 1
        if x <= mid:
            self.modify(u << 1, x, v)
        else:
            self.modify(u << 1 | 1, x, v)
        self.pushup(u)

    def query(self, u, l, r):
        if self.tr[u].l >= l and self.tr[u].r <= r:
            return self.tr[u].v
        mid = (self.tr[u].l + self.tr[u].r) >> 1

```

Round 8 · Low · Medium

p.3232

```

v = 0
if l <= mid:
    v += self.query(u << 1, l, r)
if r > mid:
    v += self.query(u << 1 | 1, l, r)
return v

def pushup(self, u):
    self.tr[u].v = self.tr[u << 1].v + self.tr[u << 1 | 1].v

```

```

class NumMatrix:
    def __init__(self, matrix: List[List[int]]):
        self.trees = [SegmentTree(row) for row in matrix]

    def update(self, row: int, col: int, val: int) -> None:
        tree = self.trees[row]
        tree.modify(1, col + 1, val)

    def sumRegion(self, row1: int, col1: int, row2: int, col2: int) -> int:

```

```

    return sum(
        self.trees[row].query(1, col1 + 1, col2 + 1)
        for row in range(row1, row2 + 1)
    )

```

```

# Your NumMatrix object will be instantiated and called as such:
# obj = NumMatrix(matrix)
# obj.update(row,col,val)
# param_2 = obj.sumRegion(row1,col1,row2,col2)

```

• SimplyLeet

Round 8 · Low · Medium

p.3233

```

class NumMatrix:
    def __init__(self, matrix):
        # initialize dimensions and store given matrix copy
        if not matrix or not matrix[0]:
            return
        self.m = len(matrix)
        self.n = len(matrix[0])
        self.matrix = [[0] * self.n for _ in range(self.m)]
        # BIT is 1-indexed so allocate with extra row and col
        self.bit = [[0] * (self.n + 1) for _ in range(self.m + 1)]

```

```

        # Initialize BIT with the matrix values
        for i in range(self.m):
            for j in range(self.n):
                self.update(i, j, matrix[i][j])

```

```

    def update(self, row, col, val):
        # find the difference from the current value
        if self.m == 0 or self.n == 0:
            return
        diff = val - self.matrix[row][col]

```

```

        self.matrix[row][col] = val
        # update BIT for all indexes that include cell (row, col)
        i = row + 1
        while i <= self.m:
            j = col + 1
            while j <= self.n:
                self.bit[i][j] += diff
                # j moves to next index responsible for including col
                j += (j & -j)
            i += (i & -i)

```

```

    def _bitSum(self, row, col):
        # Compute prefix sum from BIT for submatrix (0,0) to (row, col)
        s = 0
        i = row
        while i > 0:
            j = col
            while j > 0:
                s += self.bit[i][j]
                j -= (j & -j)

```

Round 8 · Low · Medium

p.3234

```

        i -= (i & -i)
    return s

def sumRegion(self, row1, col1, row2, col2):
    # Use inclusion-exclusion principle to get region sum
    sum_total = self._bitSum(row2 + 1, col2 + 1)
    sum_top = self._bitSum(row1, col2 + 1)
    sum_left = self._bitSum(row2 + 1, col1)
    sum_corner = self._bitSum(row1, col1)
    return sum_total - sum_top - sum_left + sum_corner

```

Round 8 · Low · Medium

p.3235

Round 9 · Low · Hard

Round 9

Low Priority · Hard

48 questions · 15 patterns

- ☐ ● Bucket Sort #220 ↗
- ☐ ● Eulerian Circuit #332 #753 #2097 ↗
- ☐ ● 1-D DP #1411 ↗
- ☐ ● 0/1 Knapsack #879 ↗
- ☐ ● Longest Increasing Subsequence (LIS) #354 ↗
- ☐ ● 2D Grid DP #85 #174 #312 #329 #403 #465 ↗
- ☐ ● #546 #741 #1235 #1335 #1547 #1931 #3651 ↗
- ☐ ● String DP #10 #44 #115 #132 #140 #664 #1092 ↗
- ☐ ● Tree / Graph DP #834 #968 ↗
- ☐ ● Bitmask DP #847 ↗
- ☐ ● Digit DP #233 #902 ↗
- ☐ ● State Machine DP #123 #188 ↗
- ☐ ● Maths / Geometry #149 ↗

Round 8 · Low · Medium

p.3236

- ☐ ● String Matching #214 #1044 #1392 ↗
- ☐ ● Binary Indexed Tree / Segment Tree #315 #699 ↗
- ☐ ● #2426 #3161 #3721 ↗
- ☐ ● Line Sweep #218 #391 #759 #850 #1851 ↗

Bucket Sort

Round 9 · Low · Hard · 1 question

Round 9 · Low · Hard

p.3237

Round 9 · Low · Hard

p.3238

Bucket Sort

□ #220 Contains Duplicate III

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Sliding Window · Sorting · Bucket Sort · Ordered Set

Lists: AlgoMaster 600

Problem

You are given an integer array `nums` and two integers `indexDiff` and `valueDiff`.

Find a pair of indices (i, j) such that:

- $i \neq j$,
- $\text{abs}(i - j) \leq \text{indexDiff}$.
- $\text{abs}(\text{nums}[i] - \text{nums}[j]) \leq \text{valueDiff}$, and

Return true if such pair exists or false otherwise.

Example 1:

Input: `nums = [1,2,3,1]`, `indexDiff = 3`, `valueDiff = 0`
Output: true
Explanation: We can choose $(i, j) = (0, 3)$.
We satisfy the three conditions:
 $i \neq j \rightarrow 0 \neq 3$
 $\text{abs}(i - j) \leq \text{indexDiff} \rightarrow \text{abs}(0 - 3) \leq 3$
 $\text{abs}(\text{nums}[i] - \text{nums}[j]) \leq \text{valueDiff} \rightarrow \text{abs}(1 - 1) \leq 0$

Example 2:

Input: `nums = [1,5,9,1,5,9]`, `indexDiff = 2`, `valueDiff = 3`

Output: false

Explanation: After trying all the possible pairs (i, j) , we cannot satisfy the three conditions, so we return false.

Constraints:

- $2 \leq \text{nums.length} \leq 105$
- $-109 \leq \text{nums}[i] \leq 109$
- $1 \leq \text{indexDiff} \leq \text{nums.length}$
- $0 \leq \text{valueDiff} \leq 109$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def containsNearbyAlmostDuplicate(
        self,
        nums: list[int],
        indexDiff: int,
        valueDiff: int,
    ) -> bool:
        if not nums or indexDiff <= 0 or valueDiff < 0:
            return False

        mn = min(nums)
        diff = valueDiff + 1 # In case that `valueDiff` equals 0.
        bucket = {}

        def getKey(num: int) -> int:
            return (num - mn) // diff

        for i, num in enumerate(nums):
            key = getKey(num)
            if key in bucket: # the current bucket
```



```

        return True
    # the left adjacent bucket
    if key - 1 in bucket and num - bucket[key - 1] < diff:
        return True
    # the right adjacent bucket
    if key + 1 in bucket and bucket[key + 1] - num < diff:
        return True
    bucket[key] = num
    if i >= indexDiff:
        del bucket[getKey(nums[i - indexDiff])]

    return False

```

• LeetCode

```

class Solution:
    def containsNearbyAlmostDuplicate(
        self, nums: List[int], indexDiff: int, valueDiff: int
    ) -> bool:
        s = SortedSet()
        for i, v in enumerate(nums):
            j = s.bisect_left(v - valueDiff)
            if j < len(s) and s[j] <= v + valueDiff:
                return True
            s.add(v)

        if i >= indexDiff:
            s.remove(nums[i - indexDiff])
        return False

```

• SimplyLeet

Round 9 · Low · Hard

p.3241

```

class Solution:
    def containsNearbyAlmostDuplicate(self, nums, indexDiff, valueDiff):
        # Early return; bucket size is valueDiff + 1 to avoid division by zero
        # when valueDiff is 0
        if valueDiff < 0:
            return False

        bucketSize = valueDiff + 1 # Bucket size ensures closer numbers fall
        # in same or adjacent buckets
        buckets = {} # Map bucket id to the number in that bucket

        for i, num in enumerate(nums):

            # Compute bucket id. For negative values, adjust bucket
            # calculation.
            bucketId = num // bucketSize if num >= 0 else ((num + 1) // bucket
            Size - 1)

            # Check if current bucket already contains an element
            if bucketId in buckets:
                return True

            # Check the left adjacent bucket
            if (bucketId - 1 in buckets and abs(num - buckets[bucketId - 1]) <
            bucketSize):
                return True

            # Check the right adjacent bucket
            if (bucketId + 1 in buckets and abs(num - buckets[bucketId + 1]) <
            bucketSize):
                return True

            # Insert the current number into its bucket
            buckets[bucketId] = num

            # Maintain the sliding window of size indexDiff
            if i >= indexDiff:

```

Round 9 · Low · Hard

p.3242

```
        # Compute bucket id for the element to be removed
        removeNum = nums[i - indexDiff]
        removeBucketId = removeNum // bucketSize if removeNum >= 0 else
e ((removeNum + 1) // bucketSize - 1)
        del buckets[removeBucketId]

    return False
```

Round 9 · Low · Hard**p.3243**

Eulerian Circuit

Round 9 · Low · Hard · 3 questions**Round 9 · Low · Hard****p.3244**

Eulerian Circuit

#332 Reconstruct Itinerary

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · String · Depth-First Search · Graph
 Theory · Sorting · Heap (Priority Queue) ·
 Eulerian Circuit

Lists: NeetCode 150

Problem

You are given a list of airline tickets where $\text{tickets}[i] = [\text{from}_i, \text{to}_i]$ represent the departure and the arrival airports of one flight.

Reconstruct the itinerary in order and return it.

All of the tickets belong to a man who departs from "JFK", thus, the itinerary must begin with "JFK". If there are multiple valid itineraries, you should return the itinerary that has the smallest lexical order when read as a single string.

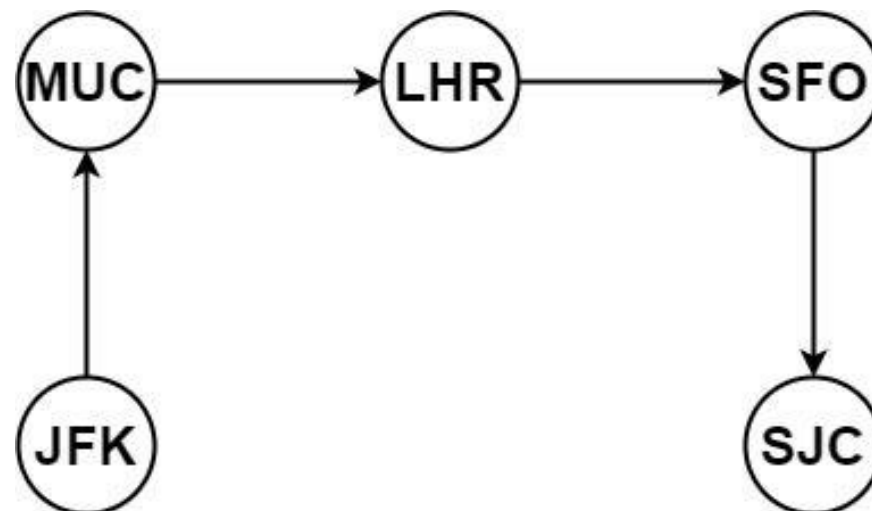
- For example, the itinerary ["JFK", "LGA"] has a smaller lexical order than ["JFK", "LGB"].

You may assume all tickets form at least one valid itinerary. You must use all the tickets once and only once.

Round 9 · Low · Hard

p.3245

Example 1:

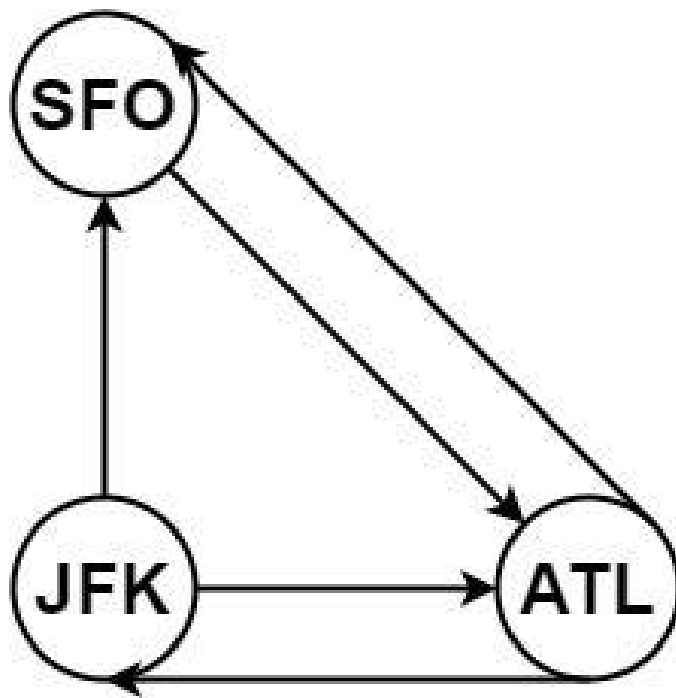


Input: tickets = [["MUC","LHR"],["JFK","MUC"],["SFO","SJC"],["LHR","SFO"]]
 Output: ["JFK","MUC","LHR","SFO","SJC"]

Example 2:

Round 9 · Low · Hard

p.3246



Input: tickets =
 [["JFK","SFO"],["JFK","ATL"],["SFO","ATL"],["ATL","JFK"],["ATL","SFO"]]
 Output: ["JFK","ATL","JFK","SFO","ATL","SFO"]
 Explanation: Another possible reconstruction is
 ["JFK","SFO","ATL","JFK","ATL","SFO"] but it is larger in lexical order.

Constraints:

- $1 \leq \text{tickets.length} \leq 300$
- $\text{tickets}[i].\text{length} == 2$
- $\text{fromi.length} == 3$
- $\text{toi.length} == 3$

Round 9 · Low · Hard

p.3247

- fromi and toi consist of uppercase English letters.
- fromi != toi

Community Solutions (Python)

• WalkCC

```

class Solution:
    def findItinerary(self, tickets: List[List[str]]) -> List[str]:
        ans = []
        graph = collections.defaultdict(list)

        for a, b in reversed(sorted(tickets)):
            graph[a].append(b)

        def dfs(u: str) -> None:
            while u in graph and graph[u]:
                dfs(graph[u].pop())
            ans.append(u)

        dfs('JFK')
        return ans[::-1]
  
```

• LeetCode

```

class Solution:
    def findItinerary(self, tickets: List[List[str]]) -> List[str]:
        def dfs(f: str):
            while g[f]:
                dfs(g[f].pop())
            ans.append(f)

        g = defaultdict(list)
        for f, t in sorted(tickets, reverse=True):
            g[f].append(t)

        dfs('JFK')
        return ans
  
```

Round 9 · Low · Hard

p.3248

```
ans = []
dfs("JFK")
return ans[::-1]
```

• SimplyLeet

```
def findItinerary(tickets):
    # Build the graph as an adjacency list.
    graph = {}
    for src, dst in tickets:
        if src not in graph:
            graph[src] = []
        graph[src].append(dst)
    # Sort the destination lists in reverse lexicographical order.
    for src in graph:
        graph[src].sort(reverse=True)

    itinerary = []

    def dfs(airport):
        # Explore all flights out of the current airport.
        while airport in graph and graph[airport]:
            next_dest = graph[airport].pop()
            dfs(next_dest)
            itinerary.append(airport)

    # Start DFS from 'JFK'
    dfs("JFK")
    # The itinerary is constructed in reverse.
    return itinerary[::-1]

# Example usage:
tickets = [
    ["JFK", "SFO"], ["JFK", "ATL"], ["SFO", "ATL"], ["ATL", "JFK"], ["ATL", "SFO"]
]
print(findItinerary(tickets))
```

Eulerian Circuit

□ #753 Cracking the Safe

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Depth-First Search · Graph Theory ·
Eulerian Circuit

Lists: AlgoMaster 600

Problem

There is a safe protected by a password. The password is a sequence of n digits where each digit can be in the range $[0, k - 1]$.

The safe has a peculiar way of checking the password. When you enter in a sequence, it checks the most recent n digits that were entered each time you type a digit.

- For example, the correct password is "345" and you enter in "012345": After typing 0, the most recent 3 digits is "0", which is incorrect.
- After typing 1, the most recent 3 digits is "01", which is incorrect.
- After typing 2, the most recent 3 digits is "012", which is incorrect.

- After typing 3, the most recent 3 digits is "123", which is incorrect.
- After typing 4, the most recent 3 digits is "234", which is incorrect.
- After typing 5, the most recent 3 digits is "345", which is correct and the safe unlocks.

Return any string of minimum length that will unlock the safe at some point of entering it.

Example 1:

Input: $n = 1, k = 2$
 Output: "10"
 Explanation: The password is a single digit, so enter each digit. "01" would also unlock the safe.

Example 2:

Input: $n = 2, k = 2$
 Output: "01100"
 Explanation: For each possible password:
 - "00" is typed in starting from the 4th digit.
 - "01" is typed in starting from the 1st digit.
 - "10" is typed in starting from the 3rd digit.
 - "11" is typed in starting from the 2nd digit.
 Thus "01100" will unlock the safe. "10011", and "11001" would also unlock the safe.

Constraints:

- $1 \leq n \leq 4$
- $1 \leq k \leq 10$
- $1 \leq kn \leq 4096$

Community Solutions (Python)

Round 9 · Low · Hard

p.3251

• WalkCC

```
class Solution:
    def crackSafe(self, n: int, k: int) -> str:
        passwordSize = k**n
        path = '0' * n
        seen = set()
        seen.add(path)

        def dfs(path: str) -> str:
            if len(seen) == passwordSize:
                return path

            for c in map(str, range(k)):
                node = path[-n + 1:] + c if n > 1 else c
                if node not in seen:
                    seen.add(node)
                    res = dfs(path + c)
                    if res:
                        return res
                    seen.remove(node)

            return dfs(path)
```

• LeetCode

```
class Solution:
    def crackSafe(self, n: int, k: int) -> str:
        def dfs(u):
            for x in range(k):
                e = u * 10 + x
                if e not in vis:
                    vis.add(e)
                    v = e % mod
                    dfs(v)
                    ans.append(str(x))

        vis = set()
        mod = 10**n
        ans = []
        dfs(0)
        return ''.join(ans)
```

Round 9 · Low · Hard

p.3252

```

mod = 10 ** (n - 1)
vis = set()
ans = []
dfs(0)
ans.append("0" * (n - 1))
return "".join(ans)

```

• SimplyLeet

```

# Python solution using DFS to generate a de Bruijn sequence
def crackSafe(n, k):
    # Initialize the set for visited combinations (edges)
    visited = set()
    result = []

    # Depth-first search helper function
    def dfs(node):
        # Try every digit in the alphabet [0, k-1]
        for digit in map(str, range(k)):
            # Form the new combination
            edge = node + digit
            # If the edge is not visited, continue DFS
            if edge not in visited:
                visited.add(edge)
                # Recursively visit the next node given by the last (n-1)
                # characters of edge
                dfs(edge[1:])
            # Append the digit to the result (backtracking)
            result.append(digit)

    # Start with an initial node of (n-1) zeros (or empty if n=1)
    start = "0" * (n - 1)
    dfs(start)
    # The final safe sequence is the starting node appended with the result
    # reversed
    return start + "".join(result[::-1])

# Example usage:
print(crackSafe(2, 2))

```

Round 9 · Low · Hard

p.3253

Eulerian Circuit

□ #2097 Valid Arrangement of Pairs

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Depth-First Search · Graph Theory ·
 Eulerian Circuit

Lists: AlgoMaster 600

Problem

You are given a 0-indexed 2D integer array pairs where pairs[i] = [starti, endi]. An arrangement of pairs is valid if for every index i where $1 \leq i < \text{pairs.length}$, we have $\text{endi-1} == \text{starti}$.

Return any valid arrangement of pairs.

Note: The inputs will be generated such that there exists a valid arrangement of pairs.

Example 1:

```

Input: pairs = [[5,1],[4,5],[11,9],[9,4]]
Output: [[11,9],[9,4],[4,5],[5,1]]
Explanation:
This is a valid arrangement since endi-1 always equals starti.
end0 = 9 == 9 = start1
end1 = 4 == 4 = start2
end2 = 5 == 5 = start3

```

Example 2:

Round 9 · Low · Hard

p.3254

Input: pairs = [[1,3],[3,2],[2,1]]
 Output: [[1,3],[3,2],[2,1]]
 Explanation:
 This is a valid arrangement since $\text{endi}-1$ always equals starti .
 $\text{end0} = 3 == 3 = \text{start1}$
 $\text{end1} = 2 == 2 = \text{start2}$
 The arrangements $[[2,1],[1,3],[3,2]]$ and $[[3,2],[2,1],[1,3]]$ are also valid.

Example 3:

Input: pairs = [[1,2],[1,3],[2,1]]
 Output: [[1,2],[2,1],[1,3]]
 Explanation:
 This is a valid arrangement since $\text{endi}-1$ always equals starti .
 $\text{end0} = 2 == 2 = \text{start1}$
 $\text{end1} = 1 == 1 = \text{start2}$

Constraints:

- $1 \leq \text{pairs.length} \leq 105$
- $\text{pairs}[i].\text{length} == 2$
- $0 \leq \text{starti}, \text{endi} \leq 109$
- $\text{starti} \neq \text{endi}$
- No two pairs are exactly the same.
- There exists a valid arrangement of pairs.

Community Solutions (Python)

- WalkCC

Round 9 · Low · Hard

p.3255

```
class Solution:
    def validArrangement(self, pairs: list[list[int]]) -> list[list[int]]:
        ans = []
        graph = collections.defaultdict(list)
        outDegree = collections.Counter()
        inDegrees = collections.Counter()

        for start, end in pairs:
            graph[start].append(end)
            outDegree[start] += 1

            inDegrees[end] += 1

        def getStartNode() -> int:
            for u in graph.keys():
                if outDegree[u] - inDegrees[u] == 1:
                    return u
            return pairs[0][0] # Arbitrarily choose a node.

        def euler(u: int) -> None:
            stack = graph[u]

            while stack:
                v = stack.pop()
                euler(v)
            ans.append([u, v])

        euler(getStartNode())
        return ans[::-1]
```

• SimplyLeet

```
# Build the graph, find the Eulerian path, and convert the path to the valid
# arrangement of pairs.
from collections import defaultdict, deque

def validArrangement(pairs):
    graph = defaultdict(list)
    in_degree = defaultdict(int)
    out_degree = defaultdict(int)

    # Build graph and degree count
    for start, end in pairs:
```

Round 9 · Low · Hard

p.3256


```

graph[start].append(end)
out_degree[start] += 1
in_degree[end] += 1

# Sort each list in reverse order for efficient pop
for start in graph:
    graph[start].sort(reverse=True)

# Find start node: node with out-degree = in-degree + 1, else any node with
outgoing edge.
start_node = pairs[0][0]

for node in out_degree:
    if out_degree[node] - in_degree[node] == 1:
        start_node = node
        break

path = []
def dfs(node):
    while graph[node]:
        next_node = graph[node].pop()
        dfs(next_node)

    path.append(node)

dfs(start_node)
path.reverse()

# Convert vertex path to pairs. Since path is a list of vertices, each
consecutive pair forms an edge.
arrangement = []
for i in range(1, len(path)):
    arrangement.append([path[i-1], path[i]])
return arrangement

# Example usage:
pairs_example = [[5,1],[4,5],[11,9],[9,4]]
print(validArrangement(pairs_example))

```

Round 9 · Low · Hard

p.3257

1-D DP

Round 9 · Low · Hard · 1 question

Round 9 · Low · Hard

p.3258

1-D DP

☐ #1411 Number of Ways to Paint N × 3 Grid

HAR

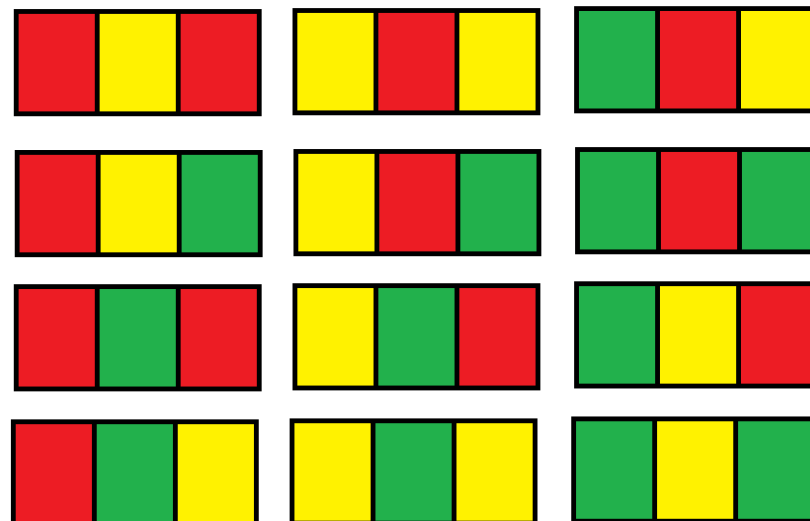
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Dynamic Programming
Lists: AlgoMaster 600

Problem

You have a grid of size $n \times 3$ and you want to paint each cell of the grid with exactly one of the three colors: Red, Yellow, or Green while making sure that no two adjacent cells have the same color (i.e., no two cells that share vertical or horizontal sides have the same color).

Given n the number of rows of the grid, return the number of ways you can paint this grid. As the answer may grow large, the answer must be computed modulo $10^9 + 7$.

Example 1:



Input: $n = 1$
Output: 12
Explanation: There are 12 possible way to paint the grid as shown.

Example 2:

Input: $n = 5000$
Output: 30228214

Constraints:

- $n == \text{grid.length}$
- $1 \leq n \leq 5000$

Community Solutions (Python)

- LeetDoocs

```
class Solution:
    def numOfWays(self, n: int) -> int:
        mod = 10**9 + 7
        f0 = f1 = 6
        for _ in range(n - 1):
            g0 = (3 * f0 + 2 * f1) % mod
            g1 = (2 * f0 + 2 * f1) % mod
            f0, f1 = g0, g1
        return (f0 + f1) % mod
```

```
class Solution:
    def numOfWays(self, n: int) -> int:
        def f1(x: int) -> bool:
            last = -1
            for _ in range(3):
                if x % 3 == last:
                    return False
                last = x % 3
                x //= 3
            return True
```

```
def f2(x: int, y: int) -> bool:
    for _ in range(3):
        if x % 3 == y % 3:
            return False
        x //= 3
        y //= 3
    return True
```

```
mod = 10**9 + 7
```

```
m = 27
valid = {i for i in range(m) if f1(i)}
d = defaultdict(list)
for i in valid:
    for j in valid:
        if f2(i, j):
            d[i].append(j)
f = [int(i in valid) for i in range(m)]
for _ in range(n - 1):
    g = [0] * m
```

Round 9 · Low · Hard

p.3261

```
for i in valid:
    for j in d[i]:
        g[j] = (g[j] + f[i]) % mod

f = g
return sum(f) % mod
```

• SimplyLeet

```
MOD = 10**9 + 7
```

```
def numOfWays(n: int) -> int:
    # Base case: for row 1, there are 6 patterns of Type A and 6 patterns of
    # Type B.
    dpA, dpB = 6, 6
    # Process rows 2 through n.
    for i in range(2, n + 1):
        nextA = (3 * dpA + 2 * dpB) % MOD # From Type A and Type B transitions
        # to Type A.
        nextB = (2 * dpA + 2 * dpB) % MOD # From Type A and Type B transitions
        # to Type B.
        dpA, dpB = nextA, nextB # Update current states.
```

```
return (dpA + dpB) % MOD
```

```
# Example usage:
print(numOfWays(1)) # Expected output: 12
print(numOfWays(5000)) # Outputs the computed answer modulo 10^9 + 7
```

Round 9 · Low · Hard

p.3262

0/1 Knapsack

Round 9 · Low · Hard · 1 question

Round 9 · Low · Hard

p.3263

0/1 Knapsack

☐ #879 Profitable Schemes

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: AlgoMaster 600

Problem

There is a group of n members, and a list of various crimes they could commit. The i th crime generates a profit $[i]$ and requires group $[i]$ members to participate in it. If a member participates in one crime, that member can't participate in another crime.

Let's call a profitable scheme any subset of these crimes that generates at least minProfit profit, and the total number of members participating in that subset of crimes is at most n .

Return the number of schemes that can be chosen. Since the answer may be very large, return it modulo $10^9 + 7$.

Example 1:

Round 9 · Low · Hard

p.3264

Input: $n = 5$, $\text{minProfit} = 3$, $\text{group} = [2,2]$, $\text{profit} = [2,3]$
 Output: 2
 Explanation: To make a profit of at least 3, the group could either commit crimes 0 and 1, or just crime 1.
 In total, there are 2 schemes.

Example 2:

Input: $n = 10$, $\text{minProfit} = 5$, $\text{group} = [2,3,5]$, $\text{profit} = [6,7,8]$
 Output: 7
 Explanation: To make a profit of at least 5, the group could commit any crimes, as long as they commit one.
 There are 7 possible schemes: (0), (1), (2), (0,1), (0,2), (1,2), and (0,1,2).

Constraints:

- $1 \leq n \leq 100$
- $0 \leq \text{minProfit} \leq 100$
- $1 \leq \text{group.length} \leq 100$
- $1 \leq \text{group}[i] \leq 100$
- $\text{profit.length} == \text{group.length}$
- $0 \leq \text{profit}[i] \leq 100$

Community Solutions (Python)

• LeetCode

```
class Solution:
    def profitableSchemes(
        self, n: int, minProfit: int, group: List[int], profit: List[int]
    ) -> int:
        @cache
        def dfs(i: int, j: int, k: int) -> int:
            if i >= len(group):
                return 1 if k == minProfit else 0
            ans = dfs(i + 1, j, k)
            if j + group[i] <= n:
```

```
        ans += dfs(i + 1, j + group[i], min(k + profit[i], minProfit))
        return ans % (10**9 + 7)

    return dfs(0, 0, 0)
```

• SimplyLeet

```
# Python solution
MOD = 10**9 + 7

def profitableSchemes(n, minProfit, group, profit):
    crimeCount = len(group)
    # dp[people][p] = number of schemes achieving profit 'p' with used members
    'people'
    dp = [[0] * (minProfit + 1) for _ in range(n + 1)]
    dp[0][0] = 1 # one way to achieve 0 profit with 0 members

    # Process each crime

    for idx in range(crimeCount):
        membersNeeded = group[idx]
        profitEarned = profit[idx]
        # iterate backwards for members to avoid reusing the same crime
        for people in range(n, membersNeeded - 1, -1):
            for p in range(minProfit, -1, -1):
                # New profit, not exceeding minProfit as cap
                newProfit = min(minProfit, p + profitEarned)
                # Update dp: add ways by including this crime
                dp[people][newProfit] = (dp[people][newProfit] + dp[people - m
                    embersNeeded][p]) % MOD

    # Sum over all ways where profit achieved is at least minProfit (which is
    dp[...][minProfit])
    totalSchemes = sum(dp[people][minProfit] for people in range(n + 1)) % MOD
    return totalSchemes

# Example usage:
print(profitableSchemes(5, 3, [2,2], [2,3])) # Output: 2
```

Longest Increasing Subsequence (LIS)

Round 9 · Low · Hard · 1 question

Round 9 · Low · Hard

p.3267

Longest Increasing Subsequence (LIS)

□ #354 Russian Doll Envelopes

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Dynamic Programming ·
Sorting

Lists: AlgoMaster 600

Problem

You are given a 2D array of integers envelopes where $\text{envelopes}[i] = [w_i, h_i]$ represents the width and the height of an envelope.

One envelope can fit into another if and only if both the width and height of one envelope are greater than the other envelope's width and height.

Return the maximum number of envelopes you can Russian doll (i.e., put one inside the other).

Note: You cannot rotate an envelope.

Example 1:

```
Input: envelopes = [[5,4],[6,4],[6,7],[2,3]]
Output: 3
Explanation: The maximum number of envelopes you can Russian doll is 3 ([2,3]
=> [5,4] => [6,7]).
```

Example 2:

Round 9 · Low · Hard

p.3268

Input: envelopes = [[1,1],[1,1],[1,1]]
Output: 1

Constraints:

- $1 \leq \text{envelopes.length} \leq 105$
- $\text{envelopes}[i].\text{length} == 2$
- $1 \leq w_i, h_i \leq 105$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxEnvelopes(self, envelopes: list[list[int]]) -> int:
        envelopes.sort(key=lambda x: (x[0], -x[1]))
        return self._lengthOfLIS(envelopes)

    def _lengthOfLIS(self, envelopes: list[list[int]]) -> int:
        # tails[i] := the minimum tails of all the increasing subsequences having
        # length i + 1
        tails = []

        for _, h in envelopes:
            if not tails or h > tails[-1]:
                tails.append(h)
            else:
                tails[bisect.bisect_left(tails, h)] = h

        return len(tails)
```

• LeetDoocs

```
class Solution:
    def maxEnvelopes(self, envelopes: List[List[int]]) -> int:
        envelopes.sort(key=lambda x: (x[0], -x[1]))
        d = [envelopes[0][1]]
        for _, h in envelopes[1:]:
            if h > d[-1]:
                d.append(h)
            else:
                idx = bisect_left(d, h)
                d[idx] = h

        return len(d)
```

• SimplyLeet

```
# Python solution with detailed comments
def maxEnvelopes(envelopes):
    # Sort envelopes primarily by width in ascending order and
    # by height in descending order if widths are equal.
    envelopes.sort(key=lambda x: (x[0], -x[1]))

    lis = [] # List to track the Longest Increasing Subsequence (LIS) of
    heights
    import bisect
    for _, height in envelopes:
        # Find the position where 'height' can be inserted to maintain lis in
        sorted order.

        pos = bisect.bisect_left(lis, height)
        if pos == len(lis):
            # 'height' is greater than all elements in lis; extend lis.
            lis.append(height)
        else:
            # Replace element to keep the potential of a longer increasing
            sequence.
            lis[pos] = height
    return len(lis)

# Example usage
print(maxEnvelopes([[5,4],[6,4],[6,7],[2,3]]))
```

2D Grid DP

Round 9 · Low · Hard · 13 questions

Round 9 · Low · Hard

p.3271

2D Grid DP

□ #85 Maximal Rectangle

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Stack · Matrix
· Monotonic Stack

Lists: AlgoMaster 600

Problem

Given a rows x cols binary matrix filled with 0's and 1's, find the largest rectangle containing only 1's and return its area.

Example 1:

Round 9 · Low · Hard

p.3272

1	0	1	0	0
1	0	1	1	1
1	1	1	1	1
1	0	0	1	0

Input: matrix = `[["1","0","1","0","0"],["1","0","1","1","1"],["1","1","1","1","1"],["1","0","0","1","0"]]`
 Output: 6
 Explanation: The maximal rectangle is shown in the above picture.

Example 2:

Input: matrix = `[["0"]]`
 Output: 0

Example 3:

Input: matrix = `[["1"]]`
 Output: 1

Constraints:

Round 9 · Low · Hard

p.3273

- `rows == matrix.length`
- `cols == matrix[i].length`
- `1 <= rows, cols <= 200`
- `matrix[i][j]` is '0' or '1'.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maximalRectangle(self, matrix: list[list[str]]) -> int:
        if not matrix:
            return 0

        ans = 0
        hist = [0] * len(matrix[0])

        def largestRectangleArea(heights: list[int]) -> int:
            ans = 0

            stack = []

            for i in range(len(heights) + 1):
                while stack and (i == len(heights) or heights[stack[-1]] > heights[i]):
                    h = heights[stack.pop()]
                    w = i - stack[-1] - 1 if stack else i
                    ans = max(ans, h * w)
                    stack.append(i)

            return ans

        for row in matrix:
            for i, num in enumerate(row):
                hist[i] = 0 if num == '0' else hist[i] + 1
            ans = max(ans, largestRectangleArea(hist))

        return ans
```

Round 9 · Low · Hard

p.3274

• LeetCode

```
class Solution:
    def maximalRectangle(self, matrix: List[List[str]]) -> int:
        heights = [0] * len(matrix[0])
        ans = 0
        for row in matrix:
            for j, v in enumerate(row):
                if v == "1":
                    heights[j] += 1
                else:
                    heights[j] = 0

            ans = max(ans, self.largestRectangleArea(heights))
        return ans

    def largestRectangleArea(self, heights: List[int]) -> int:
        n = len(heights)
        stk = []
        left = [-1] * n
        right = [n] * n
        for i, h in enumerate(heights):
            while stk and heights[stk[-1]] >= h:
                stk.pop()
            if stk:
                left[i] = stk[-1]
            stk.append(i)
        stk = []
        for i in range(n - 1, -1, -1):
            h = heights[i]
            while stk and heights[stk[-1]] >= h:
                stk.pop()
            if stk:
                right[i] = stk[-1]
            stk.append(i)
        return max(h * (right[i] - left[i] - 1) for i, h in enumerate(heights))
)
```

• SimplyLeet

Round 9 · Low · Hard

p.3275

```
# Python solution for Maximal Rectangle

class Solution:
    def maximalRectangle(self, matrix: List[List[str]]) -> int:
        if not matrix or not matrix[0]:
            return 0

        # Number of columns
        n = len(matrix[0])
        # Initialize histogram heights with 0's

        heights = [0] * n
        max_area = 0

        # Process each row in the matrix
        for row in matrix:
            for i in range(n):
                # Update histogram: increase height if '1' else reset to 0
                if row[i] == '1':
                    heights[i] += 1
                else:
                    heights[i] = 0

            # Calculate maximal area for the current histogram row
            max_area = max(max_area, self.largestRectangleArea(heights))

        return max_area

    def largestRectangleArea(self, heights: List[int]) -> int:
        # Append a zero at the end for flush-out
        heights.append(0)

        stack = []
        max_area = 0

        # Process every bar in the histogram
        for i, h in enumerate(heights):
            # Maintain stack in non-decreasing order of heights
            while stack and heights[stack[-1]] > h:
                height = heights[stack.pop()]
                # Determine width of the rectangle
                width = i if not stack else i - stack[-1] - 1
```

Round 9 · Low · Hard

p.3276

```
        max_area = max(max_area, height * width)
        stack.append(i)

# Remove the appended 0
heights.pop()
return max_area
```

Round 9 · Low · Hard

p.3277

2D Grid DP

☐ #174 Dungeon Game

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Matrix
Lists: AlgoMaster 600

Problem

The demons had captured the princess and imprisoned her in the bottom-right corner of a dungeon. The dungeon consists of $m \times n$ rooms laid out in a 2D grid. Our valiant knight was initially positioned in the top-left room and must fight his way through dungeon to rescue the princess.

The knight has an initial health point represented by a positive integer. If at any point his health point drops to 0 or below, he dies immediately.

Some of the rooms are guarded by demons (represented by negative integers), so the knight loses health upon entering these rooms; other rooms are either empty (represented as 0) or contain magic orbs that increase the knight's

Round 9 · Low · Hard

p.3278

health (represented by positive integers).
To reach the princess as quickly as possible, the knight decides to move only rightward or downward in each step.

Return the knight's minimum initial health so that he can rescue the princess.

Note that any room can contain threats or power-ups, even the first room the knight enters and the bottom-right room where the princess is imprisoned.

Example 1:

-2	-3	3
-5	-10	1
10	30	-5

Input: dungeon = [[-2,-3,3],[-5,-10,1],[10,30,-5]]

Output: 7

Explanation: The initial health of the knight must be at least 7 if he follows the optimal path: RIGHT-> RIGHT -> DOWN -> DOWN.

Example 2:

Input: dungeon = [[0]]

Output: 1

Constraints:

- $m == \text{dungeon.length}$
- $n == \text{dungeon}[i].\text{length}$
- $1 \leq m, n \leq 200$

- $-1000 \leq \text{dungeon}[i][j] \leq 1000$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def calculateMinimumHP(self, dungeon: List[List[int]]) -> int:
        m = len(dungeon)
        n = len(dungeon[0])
        dp = [math.inf] * (n + 1)
        dp[n - 1] = 1

        for i in reversed(range(m)):
            for j in reversed(range(n)):
                dp[j] = min(dp[j], dp[j + 1]) - dungeon[i][j]

            dp[j] = max(dp[j], 1)

        return dp[0]
```

• LeetDoocs

```
class Solution:
    def calculateMinimumHP(self, dungeon: List[List[int]]) -> int:
        m, n = len(dungeon), len(dungeon[0])
        dp = [[inf] * (n + 1) for _ in range(m + 1)]
        dp[m][n - 1] = dp[m - 1][n] = 1
        for i in range(m - 1, -1, -1):
            for j in range(n - 1, -1, -1):
                dp[i][j] = max(1, min(dp[i + 1][j], dp[i][j + 1]) - dungeon[i][j])

        return dp[0][0]
```

• SimplyLeet

```
def calculateMinimumHP(dungeon):
    # Determine the dimensions of the dungeon grid.
    m, n = len(dungeon), len(dungeon[0])

    # Create a DP table with an extra row and column for boundary conditions.
    dp = [[float('inf')] * (n + 1) for _ in range(m + 1)]

    # Set the cells adjacent to the princess's cell to 1.
    dp[m][n - 1] = 1
    dp[m - 1][n] = 1

    # Fill the DP table starting from the bottom-right corner moving backwards.
    for i in range(m - 1, -1, -1):
        for j in range(n - 1, -1, -1):
            # Min health required from right and bottom cells.
            min_next = min(dp[i + 1][j], dp[i][j + 1])
            # Calculate health needed at the current cell.
            dp[i][j] = max(min_next - dungeon[i][j], 1)

    return dp[0][0]

# Example usage:
dungeon = [[-2, -3, 3],
            [-5, -10, 1],
            [10, 30, -5]]
print(calculateMinimumHP(dungeon))
```

2D Grid DP

#312 Burst Balloons

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Dynamic Programming
 Lists: Denny Zhang

Problem

You are given n balloons, indexed from 0 to $n - 1$. Each balloon is painted with a number on it represented by an array `nums`. You are asked to burst all the balloons.

If you burst the i th balloon, you will get $\text{nums}[i - 1] * \text{nums}[i] * \text{nums}[i + 1]$ coins. If $i - 1$ or $i + 1$ goes out of bounds of the array, then treat it as if there is a balloon with a 1 painted on it.

Return the maximum coins you can collect by bursting the balloons wisely.

Example 1:

```
Input: nums = [3,1,5,8]
Output: 167
Explanation:
nums = [3,1,5,8] --> [3,5,8] --> [3,8] --> [8] --> []
coins = 3*1*5 + 3*5*8 + 1*3*8 + 1*8*1 = 167
```

Example 2:

Round 9 · Low · Hard

p.3283

```
Input: nums = [1,5]
Output: 10
```

Constraints:

- $n == \text{nums.length}$
- $1 \leq n \leq 300$
- $0 \leq \text{nums}[i] \leq 100$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxCoins(self, nums: List[int]) -> int:
        n = len(nums)
        nums = [1] + nums + [1]

        @functools.lru_cache(None)
        def dp(i: int, j: int) -> int:
            """Returns maxCoins(nums[i..j])."""
            if i > j:
                return 0

            return max(dp(i, k - 1) +
                       dp(k + 1, j) +
                       nums[i - 1] * nums[k] * nums[j + 1]
                       for k in range(i, j + 1))

        return dp(1, n)
```

Round 9 · Low · Hard

p.3284

```
class Solution:
    def maxCoins(self, nums: List[int]) -> int:
        n = len(nums)
        # dp[i][j] := maxCoins(nums[i..j])
        dp = [[0] * (n + 2) for _ in range(n + 2)]

        nums = [1] + nums + [1]

        for d in range(n):
            for i in range(1, n - d + 1):

                j = i + d
                for k in range(i, j + 1):
                    dp[i][j] = max(
                        dp[i][j],
                        dp[i][k - 1] +
                        dp[k + 1][j] +
                        nums[i - 1] * nums[k] * nums[j + 1])

        return dp[1][n]
```

• LeetCode

```
class Solution:
    def maxCoins(self, nums: List[int]) -> int:
        n = len(nums)
        arr = [1] + nums + [1]
        f = [[0] * (n + 2) for _ in range(n + 2)]
        for i in range(n - 1, -1, -1):
            for j in range(i + 2, n + 2):
                for k in range(i + 1, j):
                    f[i][j] = max(f[i][j], f[i][k] + f[k][j] + arr[i] * arr[k]
                                   * arr[j])
        return f[0][-1]
```

• SimplyLeet

Round 9 · Low · Hard

p.3285

```
# Define the function to compute maximum coins
def maxCoins(nums):
    # Add virtual balloons with value 1 at both boundaries
    n = len(nums)
    extended = [1] + nums + [1]
    # Initialize dp table with 0's; dimensions: (n+2) x (n+2)
    dp = [[0] * (n + 2) for _ in range(n + 2)]

    # Calculate dp values for intervals with length 2 to n+1
    for length in range(2, n + 2): # interval length from 2 to n+1

        for left in range(0, n + 2 - length):
            right = left + length
            # Iterate through all possible last burst balloons in the interval
            (left, right)
            for k in range(left + 1, right):
                # Coins when bursting balloon k last in the interval:
                # value from bursting: extended[left] * extended[k] *
            extended[right]
                # plus coins from subintervals [left, k] and [k, right]
                coins = extended[left] * extended[k] * extended[right] + dp[le
            ft][k] + dp[k][right]
                dp[left][right] = max(dp[left][right], coins)

    # The final answer is stored in dp[0][n+1] which represents bursting all
    real balloons.
    return dp[0][n+1]

# Example test cases
print(maxCoins([3,1,5,8])) # Expected output: 167
print(maxCoins([1,5])) # Expected output: 10
```

• LC.ca

Round 9 · Low · Hard

p.3286

```

class Solution:
    def maxCoins(self, nums: List[int]) -> int:
        nums = [1] + nums + [1]
        n = len(nums)
        dp = [[0] * n for _ in range(n)]
        for l in range(2, n):
            for i in range(n - l):
                j = i + l
                for k in range(i + 1, j):
                    dp[i][j] = max(
                        dp[i][j], dp[i][k] + dp[k][j] + nums[i] * nums[k] * nu
ms[j]
                    )
        return dp[0][-1]

```

◆ Quick Review

Key Insights

- Transform the problem by adding virtual balloons with value 1 at both ends to simplify edge handling.
- Use interval dynamic programming where $dp[i][j]$ represents the maximum coins that can be obtained by bursting all balloons between indices i and j (exclusive).
- Realize that the order of bursting balloons matters; choose the last balloon to burst in each interval to break dependencies into independent subproblems.

- The solution involves iterating over all possible intervals and determining the optimal burst order.

Space & Time

Time Complexity: $O(n^3)$ – Three nested loops: one for the interval length, one for the starting index, and one for the final balloon choice.

Space Complexity: $O(n^2)$ – A 2D dynamic programming table is used to store the maximum coins for each interval.

Solution

The problem is solved using interval dynamic programming. First, extend the input array by adding 1 at both ends to handle edge cases seamlessly. Define a dp table where $dp[i][j]$ represents the maximum coins that can be obtained by bursting all the balloons between i and j (not including i and j , as these are the virtual boundaries). For every interval $[i, j]$, iterate through the possible choices for the last balloon k to burst in that interval. The value for bursting balloon k will be determined by the coins from bursting it last (i.e., $nums[i] * nums[k] * nums[j]$) plus the maximum coins

from the two resulting subintervals: (i, k) and (k, j). By filling the dp table in increasing interval lengths, you build up to the solution for the overall interval that covers the entire (extended) array.

Round 9 · Low · Hard

p.3289

2D Grid DP

□ #329 Longest Increasing Path in a Matrix **HAR**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · DFS · BFS · Graph · Topological Sort ·
Memoization · Matrix

Lists: Grind 169

Problem

Given an $m \times n$ integers matrix, return the length of the longest increasing path in matrix. From each cell, you can either move in four directions: left, right, up, or down. You may not move diagonally or move outside the boundary (i.e., wrap-around is not allowed).

Example 1:

Round 9 · Low · Hard

p.3290

9	9	4
6	6	8
2	1	1

Input: matrix = [[9,9,4],[6,6,8],[2,1,1]]

Output: 4

Explanation: The longest increasing path is [1, 2, 6, 9].

Example 2:

Round 9 · Low · Hard

p.3291

3 → 4 → 5		
3	2	↓ 6
2	2	1

Input: matrix = [[3,4,5],[3,2,6],[2,2,1]]

Output: 4

Explanation: The longest increasing path is [3, 4, 5, 6]. Moving diagonally is not allowed.

Example 3:

Input: matrix = [[1]]

Output: 1

Constraints:

- $m == \text{matrix.length}$
- $n == \text{matrix}[i].\text{length}$
- $1 \leq m, n \leq 200$

Round 9 · Low · Hard

p.3292

- $0 \leq \text{matrix}[i][j] \leq 231 - 1$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def longestIncreasingPath(self, matrix: list[list[int]]) -> int:
        m = len(matrix)
        n = len(matrix[0])

        @functools.lru_cache(None)
        def dfs(i: int, j: int, prev: int) -> int:
            if i < 0 or i == m or j < 0 or j == n:
                return 0
            if matrix[i][j] <= prev:
                return 0

            return 0

        curr = matrix[i][j]
        return 1 + max(dfs(i + 1, j, curr),
                      dfs(i - 1, j, curr),
                      dfs(i, j + 1, curr),
                      dfs(i, j - 1, curr))

        return max(dfs(i, j, -math.inf) for i in range(m) for j in range(n))
```

• LeetCode

```
class Solution:
    def longestIncreasingPath(self, matrix: List[List[int]]) -> int:
        @cache
        def dfs(i: int, j: int) -> int:
            ans = 0
            for a, b in pairwise((-1, 0, 1, 0, -1)):
                x, y = i + a, j + b
                if 0 <= x < m and 0 <= y < n and matrix[x][y] > matrix[i][j]:
                    ans = max(ans, dfs(x, y))
            return ans + 1
```

```
m, n = len(matrix), len(matrix[0])
return max(dfs(i, j) for i in range(m) for j in range(n))
```

• SimplyLeet

Python implementation for Longest Increasing Path in a Matrix

```
class Solution:
    def longestIncreasingPath(self, matrix: List[List[int]]) -> int:
        # Return 0 if matrix is empty
        if not matrix or not matrix[0]:
            return 0

        # Dimensions of the matrix
        m, n = len(matrix), len(matrix[0])

        # Create a memoization table (same dimensions as the matrix)
        # initialized to 0
        memo = [[0] * n for _ in range(m)]

        # Define the DFS function to explore the matrix
        def dfs(i, j):
            # If result is already computed, return it
            if memo[i][j]:
                return memo[i][j]

            # Start with a path length of 1 (the cell itself)
            max_len = 1

            # Define directions: up, down, left, right
            directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

            # Explore each direction
            for dx, dy in directions:
                ni, nj = i + dx, j + dy
                # If neighbor is within bounds and its value is greater
                if 0 <= ni < m and 0 <= nj < n and matrix[ni][nj] > matrix[i][j]:
                    length = 1 + dfs(ni, nj)
```

```

        max_len = max(max_len, length)

    # Store computed longest length in memoization table
    memo[i][j] = max_len
    return max_len

# Compute the longest increasing path starting from each cell
longest_path = 0
for i in range(m):
    for j in range(n):

        longest_path = max(longest_path, dfs(i, j))

return longest_path

```

• LC.ca

```

class Solution:
    def longestIncreasingPath(self, matrix: List[List[int]]) -> int:
        @cache
        def dfs(i: int, j: int) -> int:
            ans = 0
            for a, b in pairwise((-1, 0, 1, 0, -1)):
                x, y = i + a, j + b
                if 0 <= x < m and 0 <= y < n and matrix[x][y] > matrix[i][j]:
                    ans = max(ans, dfs(x, y))
            return ans + 1

        m, n = len(matrix), len(matrix[0])
        return max(dfs(i, j) for i in range(m) for j in range(n))

```

◆ Quick Review

Key Insights

- The problem can be modeled as a graph where each cell is a node connected to its neighbors if the neighbor's value is greater.

Round 9 · Low · Hard

p.3295

- A Depth First Search (DFS) with memoization is effective to avoid recalculations.
- Dynamic Programming (DP) is used to cache the longest path found from each cell.
- Since each cell can be a start point, compute DFS for every cell and update the global maximum.

Space & Time

Time Complexity: $O(m * n)$ – each cell is computed once due to memoization.

Space Complexity: $O(m * n)$ – space is used for the memoization cache and recursion stack.

Solution

We solve the problem using a DFS approach combined with memoization to keep track of already computed longest paths from each cell. For each cell in the matrix, a DFS is initiated to explore all four valid directions (up, down, left, right). If a neighboring cell has a larger value, the DFS recurses from that neighbor. The result for each cell is cached in a 2D memoization table, ensuring that each cell is processed only once. After processing all cells, the maximum value in the memo table

Round 9 · Low · Hard

p.3296

represents the longest increasing path in the entire matrix.

Round 9 · Low · Hard

p.3297

2D Grid DP

#403 Frog Jump

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: AlgoMaster 600

Problem

A frog is crossing a river. The river is divided into some number of units, and at each unit, there may or may not exist a stone. The frog can jump on a stone, but it must not jump into the water.

Given a list of stones positions (in units) in sorted ascending order, determine if the frog can cross the river by landing on the last stone. Initially, the frog is on the first stone and assumes the first jump must be 1 unit.

If the frog's last jump was k units, its next jump must be either $k - 1$, k , or $k + 1$ units. The frog can only jump in the forward direction.

Example 1:

Round 9 · Low · Hard

p.3298

Input: stones = [0,1,3,5,6,8,12,17]

Output: true

Explanation: The frog can jump to the last stone by jumping 1 unit to the 2nd stone, then 2 units to the 3rd stone, then 2 units to the 4th stone, then 3 units to the 6th stone, 4 units to the 7th stone, and 5 units to the 8th stone.

Example 2:

Input: stones = [0,1,2,3,4,8,9,11]

Output: false

Explanation: There is no way to jump to the last stone as the gap between the 5th and 6th stone is too large.

Constraints:

- $2 \leq \text{stones.length} \leq 2000$
- $0 \leq \text{stones}[i] \leq 231 - 1$
- $\text{stones}[0] == 0$
- stones is sorted in a strictly increasing order.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def canCross(self, stones: list[int]) -> bool:
        n = len(stones)
        # dp[i][j] := True if a frog can make a size j jump to stones[i]
        dp = [[False] * (n + 1) for _ in range(n)]
        dp[0][0] = True

        for i in range(1, n):
            for j in range(i):
                k = stones[i] - stones[j]
```

```
if k > n:
    continue
for x in (k - 1, k, k + 1):
    if 0 <= x <= n:
        dp[i][k] |= dp[j][x]

return any(dp[-1])
```

• LeetDoocs

```
class Solution:
    def canCross(self, stones: List[int]) -> bool:
        @cache
        def dfs(i, k):
            if i == n - 1:
                return True
            for j in range(k - 1, k + 2):
                if j > 0 and stones[i] + j in pos and dfs(pos[stones[i] + j],
j):
                    return True
            return False
```

```
n = len(stones)
pos = {s: i for i, s in enumerate(stones)}
return dfs(0, 0)
```

```
class Solution:
    def canCross(self, stones: List[int]) -> bool:
        n = len(stones)
        f = [[False] * n for _ in range(n)]
        f[0][0] = True
        for i in range(1, n):
            for j in range(i - 1, -1, -1):
                k = stones[i] - stones[j]
                if k - 1 > j:
                    break

        f[i][k] = f[j][k - 1] or f[j][k] or f[j][k + 1]
        if i == n - 1 and f[i][k]:
            return True
        return False
```

• SimplyLeet

```
# Define the class with the canCross method.
class Solution:
    def canCross(self, stones):
        # Create a dictionary mapping each stone position to a set of jump
        # sizes that can reach that stone.
        stone_jumps = {stone: set() for stone in stones}
        # The frog starts at the first stone with a jump of 0 (this enables the
        # first jump of 1).
        stone_jumps[0].add(0)

        # Iterate over each stone.
        for stone in stones:

            # For each jump that reaches the current stone.
            for jump in stone_jumps[stone]:
                # Evaluate possible new jump sizes: k-1, k, k+1.
                for next_jump in (jump - 1, jump, jump + 1):
                    # The next jump must be positive.
                    if next_jump > 0:
                        next_stone = stone + next_jump
                        # If the calculated stone exists in the stones list.
                        if next_stone in stone_jumps:
                            stone_jumps[next_stone].add(next_jump)

            # If the last stone is reached, return True.
            if next_stone == stones[-1]:
                return True

        return False

# Example usage:
# sol = Solution()
# print(sol.canCross([0,1,3,5,6,8,12,17])) # Expected output: True
```

Round 9 · Low · Hard

p.3301

2D Grid DP

□ #465 Optimal Account Balancing

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Dynamic Programming · Backtracking ·
 Bit Manipulation · Bitmask
 Lists: AlgoMaster 600

Problem

You are given an array of transactions
 transactions where transactions[i] = [fromi, toi,
 amounti] indicates that the person with ID =
 fromi gave amounti \$ to the person with ID =
 toi.

Return the minimum number of transactions
 required to settle the debt.

Example 1:

```
Input: transactions = [[0,1,10],[2,0,5]]
Output: 2
Explanation:
Person #0 gave person #1 $10.
Person #2 gave person #0 $5.
Two transactions are needed. One way to settle the debt is person #1 pays
person #0 and #2 $5 each.
```

Example 2:

Round 9 · Low · Hard

p.3302

Input: transactions = [[0,1,10],[1,0,1],[1,2,5],[2,0,5]]

Output: 1

Explanation:

Person #0 gave person #1 \$10.

Person #1 gave person #0 \$1.

Person #1 gave person #2 \$5.

Person #2 gave person #0 \$5.

Therefore, person #1 only need to give person #0 \$4, and all debt is settled.

Constraints:

- $1 \leq \text{transactions.length} \leq 8$
- $\text{transactions}[i].\text{length} == 3$
- $0 \leq \text{from}_i, \text{to}_i < 12$
- $\text{from}_i \neq \text{to}_i$
- $1 \leq \text{amount}_i \leq 100$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minTransfers(self, transactions: list[list[int]]) -> int:
        balance = [0] * 21

        for u, v, amount in transactions:
            balance[u] -= amount
            balance[v] += amount

        debts = [b for b in balance if b]
```

```
def dfs(s: int) -> int:
    while s < len(debts) and not debts[s]:
        s += 1
    if s == len(debts):
        return 0

    ans = math.inf

    for i in range(s + 1, len(debts)):
        if debts[i] * debts[s] < 0:

            debts[i] += debts[s] # `debts[s]` is settled.
            ans = min(ans, 1 + dfs(s + 1))
            debts[i] -= debts[s] # Backtrack.

    return ans

return dfs(0)
```

• LeetDoocs

```
class Solution:
    def minTransfers(self, transactions: List[List[int]]) -> int:
        g = defaultdict(int)
        for f, t, x in transactions:
            g[f] -= x
            g[t] += x
        nums = [x for x in g.values() if x]
        m = len(nums)
        f = [inf] * (1 << m)
        f[0] = 0

        for i in range(1, 1 << m):
            s = 0
            for j, x in enumerate(nums):
                if i >> j & 1:
                    s += x
            if s == 0:
                f[i] = i.bit_count() - 1
                j = (i - 1) & i
                while j > 0:
                    f[i] = min(f[i], f[j] + f[i ^ j])
```



```

        j = (j - 1) & i
    return f[-1]

```

• SimplyLeet

Python solution for Optimal Account Balancing

```

def minTransfers(transactions):
    from collections import defaultdict
    # Compute net balance for each person.
    balance = defaultdict(int)
    for frm, to, amount in transactions:
        balance[frm] -= amount # person gives money
        balance[to] += amount # person receives money

```

```

# Filter out persons with zero balance.
debts = [amount for amount in balance.values() if amount != 0]

```

```

def dfs(start):
    # Skip settled debts.
    while start < len(debts) and debts[start] == 0:
        start += 1
    # All debts settled.
    if start == len(debts):
        return 0

```

```

min_trans = float('inf')
for i in range(start + 1, len(debts)):
    # Only attempt to settle if debts[start] and debts[i] are opposite
    in sign.
    if debts[start] * debts[i] < 0:
        # Settle debts[start] with debts[i].
        debts[i] += debts[start]
        # Recurse: 1 transaction is used.
        min_trans = min(min_trans, 1 + dfs(start + 1))
        # Backtrack: restore debts[i].
        debts[i] -= debts[start]

```

```

        # Optimization: if debts[i] becomes zero after settlement,
        # no need to try other similar values.
        if debts[i] + debts[start] == 0:
            break
        return min_trans

    return dfs(0)

# Example usage
transactions = [[0,1,10],[2,0,5]]

print(minTransfers(transactions)) # Output: 2

```

2D Grid DP

#546 Remove Boxes

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Memoization
Lists: AlgoMaster 600

Problem

You are given several boxes with different colors represented by different positive numbers.

You may experience several rounds to remove boxes until there is no box left. Each time you can choose some continuous boxes with the same color (i.e., composed of k boxes, $k \geq 1$), remove them and get $k * k$ points.

Return the maximum points you can get.

Example 1:

```
Input: boxes = [1,3,2,2,2,3,4,3,1]
Output: 23
Explanation:
[1, 3, 2, 2, 2, 3, 4, 3, 1]
----> [1, 3, 3, 4, 3, 1] (3*3=9 points)
----> [1, 3, 3, 3, 1] (1*1=1 points)
----> [1, 1] (3*3=9 points)
----> [] (2*2=4 points)
```

Example 2:

Round 9 · Low · Hard

p.3307

```
Input: boxes = [1,1,1]
Output: 9
```

Example 3:

```
Input: boxes = [1]
Output: 1
```

Constraints:

- $1 \leq \text{boxes.length} \leq 100$
- $1 \leq \text{boxes}[i] \leq 100$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def removeBoxes(self, boxes: list[int]) -> int:
        @functools.lru_cache(None)
        def dp(i: int, j: int, k: int) -> int:
            """
            Returns the maximum score of boxes[i..j] if k boxes equal to boxes[j].
            """
            if i > j:
                return 0

            r = j
            sameBoxes = k + 1
            while r > 0 and boxes[r - 1] == boxes[r]:
                r -= 1
                sameBoxes += 1
            res = dp(i, r - 1, 0) + sameBoxes * sameBoxes

            for p in range(i, r):
                if boxes[p] == boxes[r]:
                    res = max(res, dp(i, p, sameBoxes) + dp(p + 1, r - 1, 0))
```

Round 9 · Low · Hard

p.3308

```

    return res

    return dp(0, len(boxes) - 1, 0)

```

• LeetCode

```

class Solution:
    def removeBoxes(self, boxes: List[int]) -> int:
        @cache
        def dfs(i, j, k):
            if i > j:
                return 0
            while i < j and boxes[j] == boxes[j - 1]:
                j, k = j - 1, k + 1
            ans = dfs(i, j - 1, 0) + (k + 1) * (k + 1)
            for h in range(i, j):
                if boxes[h] == boxes[j]:
                    ans = max(ans, dfs(h + 1, j - 1, 0) + dfs(i, h, k + 1))
            return ans

        n = len(boxes)
        ans = dfs(0, n - 1, 0)
        dfs.cache_clear()
        return ans

```

• SimplyLeet

```

# Python solution using recursion with memoization
class Solution:
    def removeBoxes(self, boxes):
        # memo dictionary to store computed values for state (l, r, k)
        memo = {}

        def dp(l, r, k):
            # Base case: no boxes left
            if l > r:
                return 0

```

```

# Check memoized result
if (l, r, k) in memo:
    return memo[(l, r, k)]

# Increase l and k if consecutive boxes are the same as boxes[l]
orig_l, orig_k = l, k
while l + 1 <= r and boxes[l+1] == boxes[l]:
    l += 1
    k += 1

```

```

# Remove the boxes[l] block, get (k+1)^2 points, then solve for the
rest
result = (k+1)**2 + dp(l+1, r, 0)

# Try to merge non-adjacent boxes that are the same as boxes[l]
for m in range(l+1, r+1):
    if boxes[m] == boxes[l]:
        # Combine boxes[l] with boxes[m] after clearing boxes in
        between
        temp = dp(l+1, m-1, 0) + dp(m, r, k+1)
        if temp > result:
            result = temp

```

```

memo[(orig_l, r, orig_k)] = result
return result

return dp(0, len(boxes)-1, 0)

```

2D Grid DP

□ #741 Cherry Pickup

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Dynamic Programming · Matrix
 Lists: AlgoMaster 600

Problem

You are given an $n \times n$ grid representing a field of cherries, each cell is one of three possible integers.

- 0 means the cell is empty, so you can pass through,
- 1 means the cell contains a cherry that you can pick up and pass through, or
- -1 means the cell contains a thorn that blocks your way.

Return the maximum number of cherries you can collect by following the rules below:

- Starting at the position (0, 0) and reaching $(n - 1, n - 1)$ by moving right or down through valid path cells (cells with value 0 or 1).

- After reaching $(n - 1, n - 1)$, returning to (0, 0) by moving left or up through valid path cells.
- When passing through a path cell containing a cherry, you pick it up, and the cell becomes an empty cell 0.
- If there is no valid path between (0, 0) and $(n - 1, n - 1)$, then no cherries can be collected.

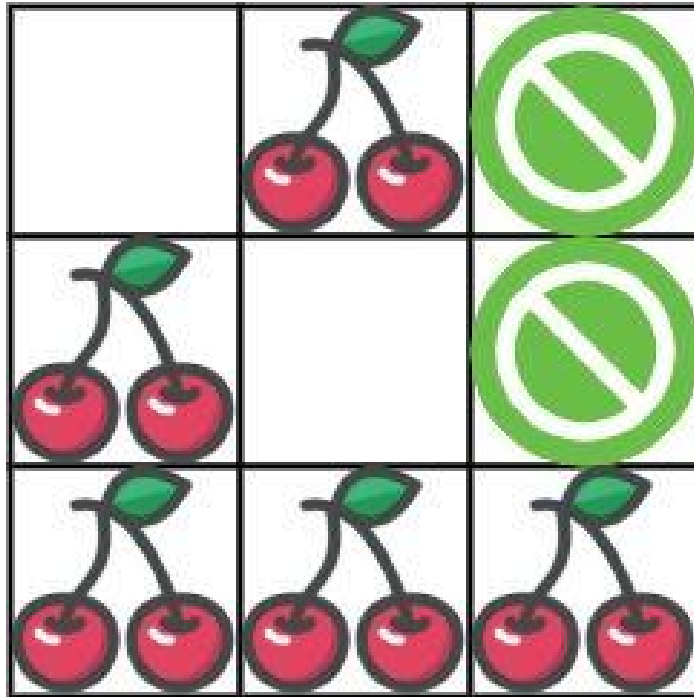
Example 1:

Round 9 · Low · Hard

p.3311

Round 9 · Low · Hard

p.3312



Input: grid = [[0,1,-1],[1,0,-1],[1,1,1]]

Output: 5

Explanation: The player started at (0, 0) and went down, down, right right to reach (2, 2).

4 cherries were picked up during this single trip, and the matrix becomes [[0,1,-1],[0,0,-1],[0,0,0]].

Then, the player went left, up, up, left to return home, picking up one more cherry.

The total number of cherries picked up is 5, and this is the maximum possible.

Example 2:

Input: grid = [[1,1,-1],[1,-1,1],[-1,1,1]]

Output: 0

Constraints:

- $n == \text{grid.length}$

Round 9 · Low · Hard

p.3313

- $n == \text{grid}[i].\text{length}$
- $1 \leq n \leq 50$
- $\text{grid}[i][j]$ is -1, 0, or 1.
- $\text{grid}[0][0] \neq -1$
- $\text{grid}[n - 1][n - 1] \neq -1$

Community Solutions (Python)

• LeetCode

```
class Solution:
    def cherryPickup(self, grid: List[List[int]]) -> int:
        n = len(grid)
        f = [[[-inf] * n for _ in range(n)] for _ in range((n << 1) - 1)]
        f[0][0][0] = grid[0][0]
        for k in range(1, (n << 1) - 1):
            for i1 in range(n):
                for i2 in range(n):
                    j1, j2 = k - i1, k - i2
                    if (
                        not 0 <= j1 < n
                        or not 0 <= j2 < n
                        or grid[i1][j1] == -1
                        or grid[i2][j2] == -1
                    ):
                        continue
                    t = grid[i1][j1]
                    if i1 != i2:
                        t += grid[i2][j2]
                    for x1 in range(i1 - 1, i1 + 1):
                        for x2 in range(i2 - 1, i2 + 1):
                            if x1 >= 0 and x2 >= 0:
                                f[k][i1][i2] = max(f[k][i1][i2], f[k - 1][x1][x2] + t)
        return max(0, f[-1][-1][-1])
```

Round 9 · Low · Hard

p.3314

• SimplyLeet

```
def cherryPickup(grid):
    n = len(grid)
    memo = {}

    def dp(r1, c1, r2):
        c2 = r1 + c1 - r2 # Because steps taken are the same: r1+c1 = r2+c2.
        # Check for out-of-bound indices or thorns.
        if r1 >= n or c1 >= n or r2 >= n or c2 >= n or grid[r1][c1] == -1 or grid[r2][c2] == -1:
            return float('-inf')
        # If reached destination.

        if r1 == n - 1 and c1 == n - 1:
            return grid[r1][c1]
        # Check memoization table.
        if (r1, c1, r2) in memo:
            return memo[(r1, c1, r2)]
        result = grid[r1][c1]
        # Avoid double counting if both agents are on the same cell.
        if r1 != r2 or c1 != c2:
            result += grid[r2][c2]
        # Explore next moves: down/down, right/right, down/right, right/down.

        temp = max(
            dp(r1 + 1, c1, r2 + 1),
            dp(r1, c1 + 1, r2),
            dp(r1 + 1, c1, r2),
            dp(r1, c1 + 1, r2 + 1)
        )
        result += temp
        memo[(r1, c1, r2)] = result
        return result

    ans = dp(0, 0, 0)
    return max(ans, 0)

# Example usage:
grid = [[0, 1, -1], [1, 0, -1], [1, 1, 1]]
print(cherryPickup(grid)) # Expected output: 5
```

Round 9 · Low · Hard

p.3315

2D Grid DP

□ #1235 Maximum Profit in Job Scheduling **HAR**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Dynamic Programming · Sorting

Lists: Grind 169

Problem

We have n jobs, where every job is scheduled to be done from $startTime[i]$ to $endTime[i]$, obtaining a profit of $profit[i]$.

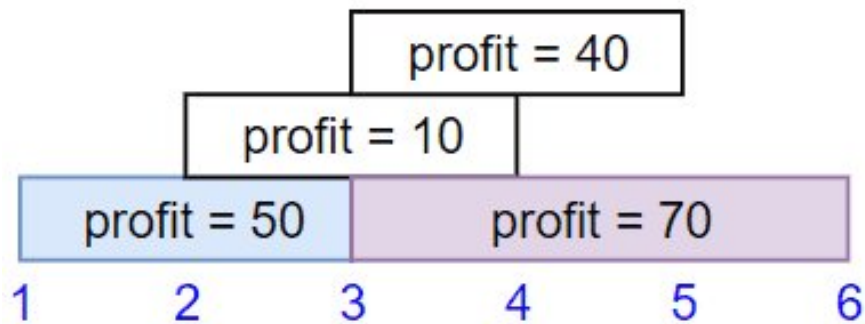
You're given the $startTime$, $endTime$ and $profit$ arrays, return the maximum profit you can take such that there are no two jobs in the subset with overlapping time range.

If you choose a job that ends at time X you will be able to start another job that starts at time X .

Example 1:

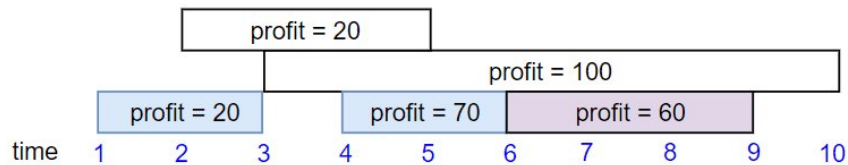
Round 9 · Low · Hard

p.3316



Input: startTime = [1,2,3,3], endTime = [3,4,5,6], profit = [50,10,40,70]
 Output: 120
 Explanation: The subset chosen is the first and fourth job.
 Time range [1-3]+[3-6], we get profit of 120 = 50 + 70.

Example 2:

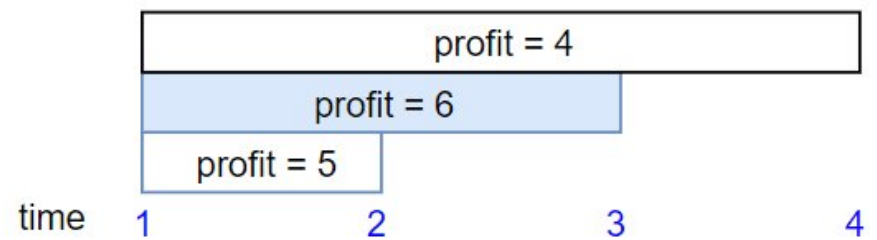


Input: startTime = [1,2,3,4,6], endTime = [3,5,10,6,9], profit = [20,20,100,70,60]
 Output: 150
 Explanation: The subset chosen is the first, fourth and fifth job.
 Profit obtained 150 = 20 + 70 + 60.

Example 3:

Round 9 · Low · Hard

p.3317



Input: startTime = [1,1,1], endTime = [2,3,4], profit = [5,6,4]
 Output: 6

Constraints:

- $1 \leq \text{startTime.length} == \text{endTime.length} == \text{profit.length} \leq 5 * 10^4$
- $1 \leq \text{startTime}[i] < \text{endTime}[i] \leq 10^9$
- $1 \leq \text{profit}[i] \leq 10^4$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def jobScheduling(
        self,
        startTime: list[int],
        endTime: list[int],
        profit: list[int],
    ) -> int:
        jobs = sorted([(s, e, p) for s, e, p in zip(startTime, endTime, profit)])

        # Will use binary search to find the first available startTime
```

Round 9 · Low · Hard

p.3318

```

for i in range(len(startTime)):
    startTime[i] = jobs[i][0]

@functools.lru_cache(None)
def dp(i: int) -> int:
    """Returns the maximum profit to schedule jobs[i..n)."""
    if i == len(startTime):
        return 0
    j = bisect.bisect_left(startTime, jobs[i][1])
    return max(jobs[i][2] + dp(j), dp(i + 1))

return dp(0)

```

• LeetDoocs

```

class Solution:
    def jobScheduling(
        self, startTime: List[int], endTime: List[int], profit: List[int]
    ) -> int:
        @cache
        def dfs(i):
            if i >= n:
                return 0
            _, e, p = jobs[i]
            j = bisect_left(jobs, e, lo=i + 1, key=lambda x: x[0])

            return max(dfs(i + 1), p + dfs(j))

        jobs = sorted(zip(startTime, endTime, profit))
        n = len(profit)
        return dfs(0)

```

• SimplyLeet

Round 9 · Low · Hard

p.3319

```

import bisect

def jobScheduling(startTime, endTime, profit):
    # Combine the jobs and sort them by end time.
    jobs = sorted(zip(startTime, endTime, profit), key=lambda x: x[1])

    # dp_end_times stores the end times and dp_profits stores the corresponding
    # maximum profit.
    dp_end_times = [0]
    dp_profits = [0]

    # Process each job in order.
    for s, e, p in jobs:
        # Find the index of the last job that finishes before or at the start
        # of the current job using binary search.
        index = bisect.bisect_right(dp_end_times, s)
        current_profit = dp_profits[index - 1] + p # Profit if we take current
        job.

        # If taking current job increases the overall profit, update the dp
        # arrays.
        if current_profit > dp_profits[-1]:
            dp_end_times.append(e)
            dp_profits.append(current_profit)

    return dp_profits[-1]

# Example usage:
print(jobScheduling([1,2,3,3], [3,4,5,6], [50,10,40,70]))

```

• LC.ca

Round 9 · Low · Hard

p.3320


```

class Solution:
    def jobScheduling(
        self, startTime: List[int], endTime: List[int], profit: List[int]
    ) -> int:
        @cache
        def dfs(i):
            if i >= n:
                return 0
            _, e, p = jobs[i]
            j = bisect_left(jobs, e, lo=i + 1, key=lambda x: x[0])

            return max(dfs(i + 1), p + dfs(j))

        jobs = sorted(zip(startTime, endTime, profit))
        n = len(profit)
        return dfs(0)

```

◆ Quick Review

Key Insights

- Sort jobs based on their end times to simplify finding non-conflicting jobs.
- Use dynamic programming where the state represents the maximum profit until a given point.
- For each job, perform a binary search to find the last job that ends before the current job's start time.
- The recurrence relation is: $dp[i] = \max(dp[i-1], profit[i] + dp[index])$ where $dp[index]$ is from the latest non-conflicting job.

Round 9 · Low · Hard

p.3321

Space & Time

Time Complexity: $O(n \log n)$ due to sorting and binary search for each job.

Space Complexity: $O(n)$ for the dp arrays.

Solution

The solution begins by sorting the jobs by their end times. For each job, a binary search is used to quickly locate the last job that does not conflict with the current job (i.e., whose end time is less than or equal to the current job's start time). Then, using dynamic programming, the maximum profit is updated by choosing whether to include the current job or skip it. Two parallel arrays (or lists) are maintained: one for the end times of the jobs considered so far, and another to store the corresponding maximum profit at those times. This approach efficiently computes the answer while keeping time complexity within acceptable bounds.

Round 9 · Low · Hard

p.3322

2D Grid DP

#1335 Minimum Difficulty of a Job Schedule

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Dynamic Programming
 Lists: AlgoMaster 600

Problem

You want to schedule a list of jobs in d days. Jobs are dependent (i.e To work on the i th job, you have to finish all the jobs j where $0 \leq j < i$).

You have to finish at least one task every day. The difficulty of a job schedule is the sum of difficulties of each day of the d days. The difficulty of a day is the maximum difficulty of a job done on that day.

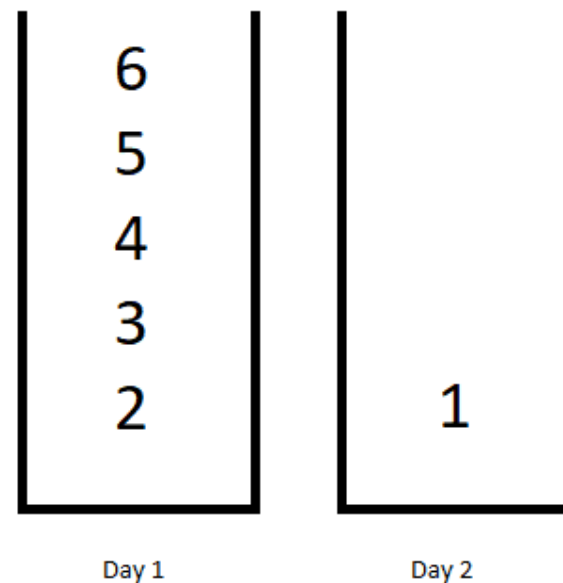
You are given an integer array `jobDifficulty` and an integer d . The difficulty of the i th job is `jobDifficulty[i]`.

Return the minimum difficulty of a job schedule. If you cannot find a schedule for the jobs return -1 .

Round 9 · Low · Hard

p.3323

Example 1:



Input: `jobDifficulty = [6,5,4,3,2,1]`, $d = 2$
 Output: 7
 Explanation: First day you can finish the first 5 jobs, total difficulty = 6.
 Second day you can finish the last job, total difficulty = 1.
 The difficulty of the schedule = $6 + 1 = 7$

Example 2:

Input: `jobDifficulty = [9,9,9]`, $d = 4$
 Output: -1
 Explanation: If you finish a job per day you will still have a free day. you cannot find a schedule for the given jobs.

Example 3:

Round 9 · Low · Hard

p.3324

Input: jobDifficulty = [1,1,1], d = 3

Output: 3

Explanation: The schedule is one job per day. total difficulty will be 3.

Constraints:

- $1 \leq \text{jobDifficulty.length} \leq 300$
- $0 \leq \text{jobDifficulty}[i] \leq 1000$
- $1 \leq d \leq 10$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minDifficulty(self, jobDifficulty: list[int], d: int) -> int:
        n = len(jobDifficulty)
        if d > n:
            return -1

        # dp[i][k] := the minimum difficulty to schedule the first i jobs in k days
        dp = [[math.inf] * (d + 1) for _ in range(n + 1)]
        dp[0][0] = 0

        for i in range(1, n + 1):
            for k in range(1, d + 1):
                maxDifficulty = 0 # max(job[j + 1..i])
                for j in range(i - 1, k - 2, -1): # 1-based
                    maxDifficulty = max(maxDifficulty, jobDifficulty[j]) # 0-based
                dp[i][k] = min(dp[i][k], dp[j][k - 1] + maxDifficulty)

        return dp[n][d]
```

• LeetCode

```
class Solution:
    def minDifficulty(self, jobDifficulty: List[int], d: int) -> int:
        n = len(jobDifficulty)
        f = [[inf] * (d + 1) for _ in range(n + 1)]
        f[0][0] = 0
        for i in range(1, n + 1):
            for j in range(1, min(d + 1, i + 1)):
                mx = 0
                for k in range(i, 0, -1):
                    mx = max(mx, jobDifficulty[k - 1])
                f[i][j] = min(f[i][j], f[k - 1][j - 1] + mx)
        return -1 if f[n][d] >= inf else f[n][d]
```

• SimplyLeet

```
def minDifficulty(jobDifficulty, d):
    n = len(jobDifficulty)
    # If there are fewer jobs than days, it's not possible to schedule
    if n < d:
        return -1

    # DP table: dp[i][day] represents the minimum difficulty for the first i
    # jobs in "day" days.
    dp = [[float('inf')] * (d + 1) for _ in range(n + 1)]
    dp[0][0] = 0

    # For each day, decide the partition of jobs.
    for day in range(1, d + 1):
        # i loops from "day" because at least one job must be done per day.
        for i in range(day, n + 1):
            max_difficulty = 0
            # Evaluate possible partitions by going backward.
            for j in range(i - 1, day - 2, -1):
                max_difficulty = max(max_difficulty, jobDifficulty[j])
                dp[i][day] = min(dp[i][day], dp[j][day - 1] + max_difficulty)

    return dp[n][d]

# Example usage:
print(minDifficulty([6,5,4,3,2,1], 2)) # Expected output: 7
```

2D Grid DP

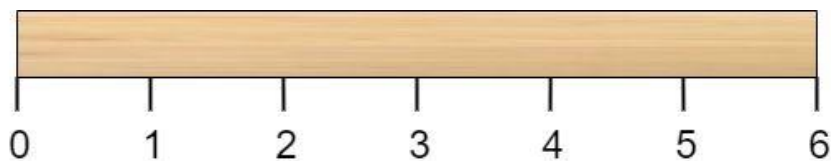
#1547 Minimum Cost to Cut a Stick

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Dynamic Programming · Sorting
 Lists: AlgoMaster 600

Problem

Given a wooden stick of length n units. The stick is labelled from 0 to n . For example, a stick of length 6 is labelled as follows:



Given an integer array `cuts` where `cuts[i]` denotes a position you should perform a cut at. You should perform the cuts in order, you can change the order of the cuts as you wish. The cost of one cut is the length of the stick to be cut, the total cost is the sum of costs of all cuts. When you cut a stick, it will be split into two smaller sticks (i.e. the sum of their lengths

Round 9 · Low · Hard

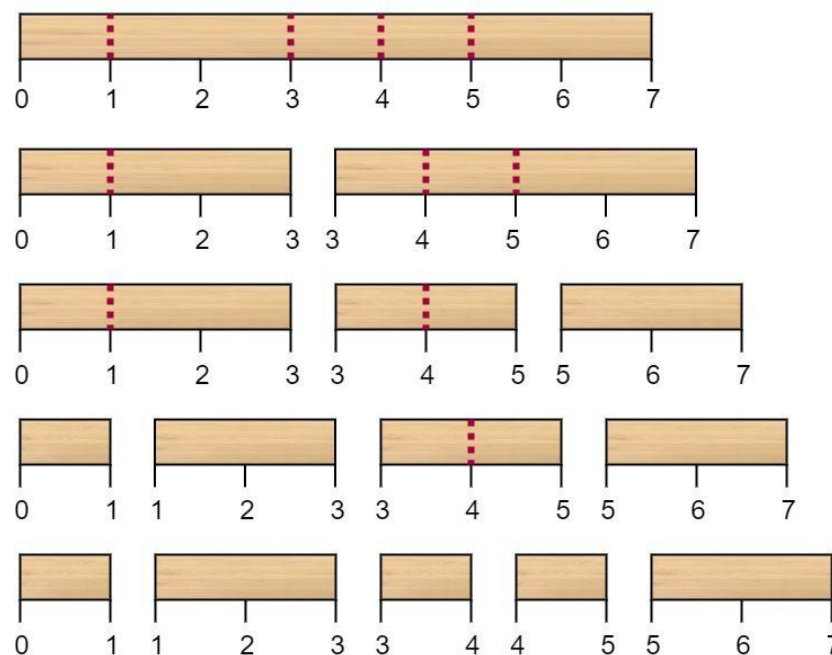
p.3327

is the length of the stick before the cut). Please refer to the first example for a better explanation.

Return the minimum total cost of the cuts.

Example 1:

`cuts = [3, 5, 1, 4]` (Optimal Ordering)



Round 9 · Low · Hard

p.3328

Input: n = 7, cuts = [1,3,4,5]

Output: 16

Explanation: Using cuts order = [1, 3, 4, 5] as in the input leads to the following scenario:

The first cut is done to a rod of length 7 so the cost is 7. The second cut is done to a rod of length 6 (i.e. the second part of the first cut), the third is done to a rod of length 4 and the last cut is to a rod of length 3. The total cost is $7 + 6 + 4 + 3 = 20$.

Rearranging the cuts to be [3, 5, 1, 4] for example will lead to a scenario with total cost = 16 (as shown in the example photo $7 + 4 + 3 + 2 = 16$).

Example 2:

Input: n = 9, cuts = [5,6,1,4,2]

Output: 22

Explanation: If you try the given cuts ordering the cost will be 25.

There are much ordering with total cost ≤ 25 , for example, the order [4, 6, 5, 2, 1] has total cost = 22 which is the minimum possible.

Constraints:

- $2 \leq n \leq 106$
- $1 \leq \text{cuts.length} \leq \min(n - 1, 100)$
- $1 \leq \text{cuts}[i] \leq n - 1$
- All the integers in cuts array are distinct.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minCost(self, n: int, cuts: list[int]) -> int:
        A = sorted([0] + cuts + [n])

        @functools.lru_cache(None)
        def dp(i, j):
            if j - i <= 1:
                return 0

            return min(A[j] - A[i] + dp(i, k) + dp(k, j) for k in range(i + 1, j))
```

Round 9 · Low · Hard

p.3329

```
return dp(0, len(A) - 1)
```

```
class Solution:
```

```
def minCost(self, n: int, cuts: list[int]) -> int:
    arr = sorted([0] + cuts + [n])
```

```
    dp = [[0] * len(arr) for _ in range(len(arr))]
```

```
    for d in range(2, len(arr)):
        for i in range(len(arr) - d):
            j = i + d
            dp[i][j] = math.inf
```

```
        for k in range(i + 1, j):
            dp[i][j] = min(dp[i][j], arr[j] - arr[i] + dp[i][k] + dp[k][j])
```

```
    return dp[0][len(arr) - 1]
```

• LeetCode

```
class Solution:
```

```
def minCost(self, n: int, cuts: List[int]) -> int:
```

```
    cuts.extend([0, n])
```

```
    cuts.sort()
```

```
    m = len(cuts)
```

```
    f = [[0] * m for _ in range(m)]
```

```
    for l in range(2, m):
```

```
        for i in range(m - l):
```

```
            j = i + l
```

```
            f[i][j] = inf
```

```
        for k in range(i + 1, j):
```

```
            f[i][j] = min(f[i][j], f[i][k] + f[k][j] + cuts[j] - cuts[i])
```

```
    return f[0][-1]
```

• SimplyLeet

Round 9 · Low · Hard

p.3330

```
# Python solution for Minimum Cost to Cut a Stick
```

```
def minCost(n, cuts):
```

```
    # Sort the cuts and add boundaries 0 and n.
```

```
    positions = [0] + sorted(cuts) + [n]
```

```
    m = len(positions)
```

```
    # Initialize dp table with 0's.
```

```
    dp = [[0] * m for _ in range(m)]
```

```
    # Build the dp table from smaller segments (gap) to larger ones.
```

```
    for gap in range(2, m): # gap is the window size between i and j.
```

```
        for i in range(m - gap):
```

```
            j = i + gap
```

```
            min_cost = float('inf')
```

```
            # Try every possible cut between positions[i] and positions[j]
```

```
            for k in range(i + 1, j):
```

```
                cost = dp[i][k] + dp[k][j] + positions[j] - positions[i]
```

```
                min_cost = min(min_cost, cost)
```

```
            dp[i][j] = min_cost
```

```
    return dp[0][m - 1]
```

```
# Example usage:
```

```
print(minCost(7, [1, 3, 4, 5])) # Output: 16
```

Round 9 · Low · Hard

p.3331

2D Grid DP

#1931 Painting a Grid With Three Different Colors

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Dynamic Programming

Lists: AlgoMaster 600

Problem

You are given two integers m and n . Consider an $m \times n$ grid where each cell is initially white. You can paint each cell red, green, or blue. All cells must be painted.

Return the number of ways to color the grid with no two adjacent cells having the same color. Since the answer can be very large, return it modulo $10^9 + 7$.

Example 1:

Input: $m = 1, n = 1$

Output: 3

Explanation: The three possible colorings are shown in the image above.

Example 2:

Input: $m = 1, n = 2$

Output: 6

Explanation: The six possible colorings are shown in the image above.

Example 3:

Round 9 · Low · Hard

p.3332

Input: m = 5, n = 5
Output: 580986

Constraints:

- $1 \leq m \leq 5$
- $1 \leq n \leq 1000$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def colorTheGrid(self, m: int, n: int) -> int:
        def getColor(mask: int, r: int) -> int:
            return mask >> r * 2 & 3

        def setColor(mask: int, r: int, color: int) -> int:
            return mask | color << r * 2

        MOD = 1_000_000_007

        @functools.lru_cache(None)
        def dp(r: int, c: int, prevColMask: int, currColMask: int) -> int:
            if c == n:
                return 1
            if r == m:
                return dp(0, c + 1, currColMask, 0)

            ans = 0

            # 1 := red, 2 := green, 3 := blue
```

```
for color in range(1, 4):
    if getColor(prevColMask, r) == color:
        continue
    if r > 0 and getColor(currColMask, r - 1) == color:
        continue
    ans += dp(r + 1, c, prevColMask, setColor(currColMask, r, color))
    ans %= MOD

return ans
```

```
return dp(0, 0, 0, 0)
```

• LeetDoocs

```
class Solution:
    def colorTheGrid(self, m: int, n: int) -> int:
        def f1(x: int) -> bool:
            last = -1
            for _ in range(m):
                if x % 3 == last:
                    return False
                last = x % 3
                x //= 3
            return True

        def f2(x: int, y: int) -> bool:
            for _ in range(m):
                if x % 3 == y % 3:
                    return False
                x, y = x // 3, y // 3
            return True

        mod = 10**9 + 7
        mx = 3**m
```

```

valid = {i for i in range(mx) if f1(i)}
d = defaultdict(list)
for x in valid:
    for y in valid:
        if f2(x, y):
            d[x].append(y)
f = [int(i in valid) for i in range(mx)]
for _ in range(n - 1):
    g = [0] * mx
    for i in valid:
        for j in d[i]:
            g[i] = (g[i] + f[j]) % mod
    f = g
return sum(f) % mod

```

• SimplyLeet

```

MOD = 10**9 + 7

# Recursively generate all valid states for one column
def generate_valid_states(m, colors=3):
    states = []
    def backtrack(row, current):
        if row == m:
            states.append(tuple(current))
            return
        for color in range(colors):
            if row > 0 and current[-1] == color:
                continue
            current.append(color)
            backtrack(row + 1, current)
            current.pop()
    backtrack(0, [])
    return states

# Check if two states are compatible (i.e., horizontal adjacent cells have
different colors)
def are_compatible(state1, state2):

```

Round 9 · Low · Hard

p.3335

```

return all(c1 != c2 for c1, c2 in zip(state1, state2))

def colorTheGrid(m, n):
    valid_states = generate_valid_states(m)
    state_count = len(valid_states)

    # Precompute valid transitions between states for adjacent columns
    transitions = [[False] * state_count for _ in range(state_count)]
    for i in range(state_count):
        for j in range(state_count):
            if are_compatible(valid_states[i], valid_states[j]):
                transitions[i][j] = True

    dp = [1] * state_count # Initialize dp for the first column
    for _ in range(1, n):
        new_dp = [0] * state_count
        for i in range(state_count):
            for j in range(state_count):
                if transitions[i][j]:
                    new_dp[j] = (new_dp[j] + dp[i]) % MOD

    dp = new_dp
    return sum(dp) % MOD

# Example usage:
print(colorTheGrid(1, 1)) # Expected Output: 3
print(colorTheGrid(1, 2)) # Expected Output: 6
print(colorTheGrid(5, 5)) # Expected Output: 580986

```

Round 9 · Low · Hard

p.3336

2D Grid DP

☐ #3651 Minimum Cost Path with Teleportations

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode Array · Dynamic Programming · Matrix Lists: AlgoMaster 600

Problem

You are given a $m \times n$ 2D integer array `grid` and an integer `k`. You start at the top-left cell $(0, 0)$ and your goal is to reach the bottom-right cell $(m - 1, n - 1)$.

There are two types of moves available:

- **Normal move:** You can move right or down from your current cell (i, j) , i.e. you can move to $(i, j + 1)$ (right) or $(i + 1, j)$ (down). The cost is the value of the destination cell.
- **Teleportation:** You can teleport from any cell (i, j) , to any cell (x, y) such that $\text{grid}[x][y] \leq \text{grid}[i][j]$; the cost of this move is 0. You may teleport at most `k` times.

Return the minimum total cost to reach cell $(m - 1, n - 1)$ from $(0, 0)$.

Round 9 · Low · Hard

p.3337

Example 1:

Input: `grid = [[1,3,3],[2,5,4],[4,3,5]]`, `k = 2`

Output: 7

Explanation:

Initially we are at $(0, 0)$ and cost is 0.

The minimum cost to reach bottom-right cell is 7.

Example 2:

Input: `grid = [[1,2],[2,3],[3,4]]`, `k = 1`

Output: 9

Explanation:

Initially we are at $(0, 0)$ and cost is 0.

The minimum cost to reach bottom-right cell is 9.

Constraints:

- $2 \leq m, n \leq 80$
- $m == \text{grid.length}$
- $n == \text{grid}[i].length$
- $0 \leq \text{grid}[i][j] \leq 104$
- $0 \leq k \leq 10$

Community Solutions (Python)

- ☐ LeetDoocs

Round 9 · Low · Hard

p.3338

```

class Solution:
    def minCost(self, grid: List[List[int]], k: int) -> int:
        m, n = len(grid), len(grid[0])
        f = [[[inf] * n for _ in range(m)] for _ in range(k + 1)]
        f[0][0][0] = 0
        for i in range(m):
            for j in range(n):
                if i:
                    f[0][i][j] = min(f[0][i][j], f[0][i - 1][j] + grid[i][j])
                if j:
                    f[0][i][j] = min(f[0][i][j], f[0][i][j - 1] + grid[i][j])
        g = defaultdict(list)
        for i, row in enumerate(grid):
            for j, x in enumerate(row):
                g[x].append((i, j))
        keys = sorted(g, reverse=True)
        for t in range(1, k + 1):
            mn = inf
            for key in keys:
                pos = g[key]
                for i, j in pos:
                    mn = min(mn, f[t - 1][i][j])
                for i, j in pos:
                    f[t][i][j] = mn
            for i in range(m):
                for j in range(n):
                    if i:
                        f[t][i][j] = min(f[t][i][j], f[t][i - 1][j] + grid[i][j])
                    if j:
                        f[t][i][j] = min(f[t][i][j], f[t][i][j - 1] + grid[i][j])

        return min(f[t][m - 1][n - 1] for t in range(k + 1))

```

Round 9 · Low · Hard

p.3339

String DP

Round 9 · Low · Hard · 7 questions

Round 9 · Low · Hard

p.3340

String DP

#10 Regular Expression Matching

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming · Recursion
Lists: Denny Zhang

Problem

Given an input string *s* and a pattern *p*, implement regular expression matching with support for '.' and '*' where:

- '.' Matches any single character.
- '*' Matches zero or more of the preceding element.

Return a boolean indicating whether the matching covers the entire input string (not partial).

Example 1:

Input: *s* = "aa", *p* = "a"
Output: false
Explanation: "a" does not match the entire string "aa".

Example 2:

Round 9 · Low · Hard

p.3341

Input: *s* = "aa", *p* = "a*"

Output: true

Explanation: '*' means zero or more of the preceding element, 'a'. Therefore, by repeating 'a' once, it becomes "aa".

Example 3:

Input: *s* = "ab", *p* = ".*"

Output: true

Explanation: ".*" means "zero or more (*) of any character (.)".

Constraints:

- $1 \leq s.length \leq 20$
- $1 \leq p.length \leq 20$
- *s* contains only lowercase English letters.
- *p* contains only lowercase English letters, '.', and '*'.
- It is guaranteed for each appearance of the character '*', there will be a previous valid character to match.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isMatch(self, s: str, p: str) -> bool:
        m = len(s)
        n = len(p)
        # dp[i][j] := True if s[0..i) matches p[0..j)
        dp = [[False] * (n + 1) for _ in range(m + 1)]
        dp[0][0] = True

        def isMatch(i: int, j: int) -> bool:
            return j >= 0 and p[j] == '.' or s[i] == p[j]
```

Round 9 · Low · Hard

p.3342

```

for j, c in enumerate(p):
    if c == '*' and dp[0][j - 1]:
        dp[0][j + 1] = True

for i in range(m):
    for j in range(n):
        if p[j] == '*':
            # The minimum index of '*' is 1.
            noRepeat = dp[i + 1][j - 1]

            doRepeat = isMatch(i, j - 1) and dp[i][j + 1]
            dp[i + 1][j + 1] = noRepeat or doRepeat
        elif isMatch(i, j):
            dp[i + 1][j + 1] = dp[i][j]

return dp[m][n]

```

• LeetDoocs

```

class Solution:
    def isMatch(self, s: str, p: str) -> bool:
        @cache
        def dfs(i, j):
            if j >= n:
                return i == m
            if j + 1 < n and p[j + 1] == '*':
                return dfs(i, j + 2) or (
                    i < m and (s[i] == p[j] or p[j] == '.') and dfs(i + 1, j)
                )

            return i < m and (s[i] == p[j] or p[j] == '.') and dfs(i + 1, j + 1)

        m, n = len(s), len(p)
        return dfs(0, 0)

```

Round 9 · Low · Hard

p.3343

```

class Solution:
    def isMatch(self, s: str, p: str) -> bool:
        m, n = len(s), len(p)
        f = [[False] * (n + 1) for _ in range(m + 1)]
        f[0][0] = True
        for i in range(m + 1):
            for j in range(1, n + 1):
                if p[j - 1] == '*':
                    f[i][j] = f[i][j - 2]
                    if i > 0 and (p[j - 2] == "." or s[i - 1] == p[j - 2]):
                        f[i][j] |= f[i - 1][j]
                elif i > 0 and (p[j - 1] == "." or s[i - 1] == p[j - 1]):
                    f[i][j] = f[i - 1][j - 1]
        return f[m][n]

```

• SimplyLeet

```

# Python solution using recursion with memoization
def isMatch(s: str, p: str) -> bool:
    memo = {}

    def dp(i: int, j: int) -> bool:
        # If result is already computed, return it.
        if (i, j) in memo:
            return memo[(i, j)]

        # If we have reached the end of the pattern, s must also be at the end.
        if j == len(p):
            ans = i == len(s)
        else:
            # Check if current characters match
            first_match = i < len(s) and (p[j] == s[i] or p[j] == '.')

            # If there is a '*' coming after the current pattern character
            if j + 1 < len(p) and p[j + 1] == '*':
                # Two cases:
                # 1. '*' stands for zero occurrence -> skip current pattern
                char and '*'
            else:
                # If first character matches, move to next character in both strings
                return first_match and dp(i + 1, j + 1)
            # If first character does not match, skip current pattern character
            return first_match and dp(i, j + 2)

        memo[(i, j)] = ans
        return ans

```

Round 9 · Low · Hard

p.3344

```

        # 2. first_match is true, so we try to consume one character in
        s and remain at same pattern index
        ans = dp(i, j+2) or (first_match and dp(i+1, j))
    else:
        # No '*' means we directly check next characters in both s and
        p.

        ans = first_match and dp(i+1, j+1)

    memo[(i, j)] = ans
    return ans

return dp(0, 0)

```

```

# Example test run
#print(isMatch("aa", "a*")) # Expected output: True

```

• LC.ca

```

class Solution:
    def isMatch(self, s: str, p: str) -> bool:
        m, n = len(s), len(p)
        f = [[False] * (n + 1) for _ in range(m + 1)]
        f[0][0] = True
        for i in range(m + 1):
            for j in range(1, n + 1):
                if p[j - 1] == "*":
                    f[i][j] = f[i][j - 2]
                    if i > 0 and (p[j - 2] == "." or s[i - 1] == p[j - 2]):
                        f[i][j] |= f[i - 1][j]
                elif i > 0 and (p[j - 1] == "." or s[i - 1] == p[j - 1]):
                    f[i][j] = f[i - 1][j - 1]
        return f[m][n]

```

◆ Quick Review

Key Insights

- Use recursion or dynamic programming to simulate matching as the '*' operator can affect the current and following characters.
- The '.' operator is a wildcard that matches any single character.
- When encountering a '*' in the pattern, consider both skipping the preceding element or consuming one character from the string if it matches.
- Using memoization helps to save intermediate results and efficiently process overlapping subproblems.

Space & Time

Time Complexity: $O(mn)$

where $m = \text{length}(s)$ and $n = \text{length}(p)$, due to exploring each subproblem combination.

Space Complexity: $O(mn)$

for storing the memoization table in the dynamic programming or recursive approach.

Solution

The solution uses a top-down recursive strategy with memoization to efficiently match the string against the pattern. For each

character position in *s* and *p*, we check if they match directly or if the pattern character is '.' to allow any match. When encountering '.' in the pattern, two possibilities are explored: either the '.' stands for zero occurrences of the preceding element, or if there is a valid match, it accounts for one occurrence and we proceed further in the string while maintaining the same pattern index. The memoization aspect prevents repeated computations of the same (i, j) state in the recursion, resulting in an improved overall runtime.

Round 9 · Low · Hard

p.3347

String DP

□ #44 Wildcard Matching

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming · Greedy · Recursion

Lists: AlgoMaster 600

Problem

Given an input string (*s*) and a pattern (*p*), implement wildcard pattern matching with support for '?' and '*' where:

- '?' Matches any single character.
- '*' Matches any sequence of characters (including the empty sequence).

The matching should cover the entire input string (not partial).

Example 1:

Input: *s* = "aa", *p* = "a"
Output: false
Explanation: "a" does not match the entire string "aa".

Example 2:

Input: *s* = "aa", *p* = "*"
Output: true
Explanation: '*' matches any sequence.

Round 9 · Low · Hard

p.3348

Example 3:

Input: s = "cb", p = "?a"

Output: false

Explanation: '?' matches 'c', but the second letter is 'a', which does not match 'b'.

Constraints:

- $0 \leq \text{s.length}, \text{p.length} \leq 2000$
- s contains only lowercase English letters.
- p contains only lowercase English letters, '?' or '*'.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isMatch(self, s: str, p: str) -> bool:
        m = len(s)
        n = len(p)
        # dp[i][j] := True if s[0..i) matches p[0..j)
        dp = [[False] * (n + 1) for _ in range(m + 1)]
        dp[0][0] = True

        def isMatch(i: int, j: int) -> bool:
            return i >= 0 and p[j] == '?' or s[i] == p[j]

        for j, c in enumerate(p):
            if c == '*':
                dp[0][j + 1] = dp[0][j]

        for i in range(m):
            for j in range(n):
                if p[j] == '*':
                    matchEmpty = dp[i + 1][j]
                    matchSome = dp[i][j + 1]
```

Round 9 · Low · Hard

p.3349

```
        dp[i + 1][j + 1] = matchEmpty or matchSome
    elif isMatch(i, j):
        dp[i + 1][j + 1] = dp[i][j]

    return dp[m][n]
```

• LeetDoocs

```
class Solution:
    def isMatch(self, s: str, p: str) -> bool:
        @cache
        def dfs(i: int, j: int) -> bool:
            if i >= len(s):
                return j >= len(p) or (p[j] == "*" and dfs(i, j + 1))
            if j >= len(p):
                return False
            if p[j] == "*":
                return dfs(i + 1, j) or dfs(i + 1, j + 1) or dfs(i, j + 1)

            return (p[j] == "?" or s[i] == p[j]) and dfs(i + 1, j + 1)

        return dfs(0, 0)

class Solution:
    def isMatch(self, s: str, p: str) -> bool:
        m, n = len(s), len(p)
        f = [[False] * (n + 1) for _ in range(m + 1)]
        f[0][0] = True
        for j in range(1, n + 1):
            if p[j - 1] == "*":
                f[0][j] = f[0][j - 1]
        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if p[j - 1] == "*":
                    f[i][j] = f[i - 1][j] or f[i][j - 1] or f[i - 1][j - 1]
                else:
                    f[i][j] = f[i - 1][j - 1] and (
                        p[j - 1] == "?" or s[i - 1] == p[j - 1]
                    )
        return f[m][n]
```

Round 9 · Low · Hard

p.3350

• SimplyLeet

```
def isMatch(s: str, p: str) -> bool:
    n, m = len(s), len(p)
    # Create a DP table with (n+1) x (m+1) initialized to False
    dp = [[False] * (m + 1) for _ in range(n + 1)]
    # Empty string matches empty pattern
    dp[0][0] = True

    # Initialize first row for patterns that have '*' which can match empty
    string
    for j in range(1, m + 1):
        if p[j - 1] == '*':
            dp[0][j] = dp[0][j - 1]

    # Build the DP table
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if p[j - 1] == '?' or p[j - 1] == s[i - 1]:
                # Characters match or pattern has '?' so inherit the previous
                state
                dp[i][j] = dp[i - 1][j - 1]
            elif p[j - 1] == '*':
                # '*' matches empty sequence: dp[i][j-1] or one/more
                characters: dp[i-1][j]
                dp[i][j] = dp[i][j - 1] or dp[i - 1][j]
            else:
                dp[i][j] = False

    return dp[n][m]

# Example usage:
print(isMatch("aa", "a"))    # False
print(isMatch("aa", "*"))    # True
print(isMatch("cb", "?a"))   # False
```

Round 9 · Low · Hard

p.3351

String DP

#115 Distinct Subsequences

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming
Lists: Denny Zhang

Problem

Given two strings s and t , return the number of distinct subsequences of s which equals t .

The test cases are generated so that the answer fits on a 32-bit signed integer.

Example 1:

```
Input: s = "rabbbit", t = "rabbit"
Output: 3
Explanation:
As shown below, there are 3 ways you can generate "rabbit" from s.
rabbbit
rabbbit
rabbbit
```

Example 2:

```
Input: s = "babgbag", t = "bag"
Output: 5
Explanation:
As shown below, there are 5 ways you can generate "bag" from s.
babgbag
babgbag
babgbag
babgbag
babgbag
```

Round 9 · Low · Hard

p.3352

Constraints:

- $1 \leq s.length, t.length \leq 1000$
- s and t consist of English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def numDistinct(self, s: str, t: str) -> int:
        m = len(s)
        n = len(t)
        dp = [[0] * (n + 1) for _ in range(m + 1)]

        for i in range(m + 1):
            dp[i][0] = 1

        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if s[i - 1] == t[j - 1]:
                    dp[i][j] = dp[i - 1][j - 1] + dp[i - 1][j]
                else:
                    dp[i][j] = dp[i - 1][j]

        return dp[m][n]
```

• LeetCode

```
class Solution:
    def numDistinct(self, s: str, t: str) -> int:
        m, n = len(s), len(t)
        f = [[0] * (n + 1) for _ in range(m + 1)]
        for i in range(m + 1):
            f[i][0] = 1
        for i, a in enumerate(s, 1):
            for j, b in enumerate(t, 1):
                f[i][j] = f[i - 1][j]
                if a == b:
```

Round 9 · Low · Hard

p.3353

```
f[i][j] += f[i - 1][j - 1]
return f[m][n]
```

```
class Solution:
    def numDistinct(self, s: str, t: str) -> int:
        n = len(t)
        f = [1] + [0] * n
        for a in s:
            for j in range(n, 0, -1):
                if a == t[j - 1]:
                    f[j] += f[j - 1]
        return f[n]
```

• SimplyLeet

```
# Python solution for Distinct Subsequences
def numDistinct(s: str, t: str) -> int:
    # Get lengths of the strings s and t
    m, n = len(s), len(t)
    # Initialize a dp table with (m+1) rows and (n+1) columns filled with 0
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    # Base case: An empty string t (j = 0) is a subsequence of any prefix of s,
    # so set dp[i][0] = 1
    for i in range(m + 1):
        dp[i][0] = 1

    # Fill the dp table
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            # If the current characters match, add the count of sequences by
            # either using or skipping s[i-1]
            if s[i - 1] == t[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + dp[i - 1][j]
            else:
                # If they do not match, skip s[i-1]
                dp[i][j] = dp[i - 1][j]
```

Round 9 · Low · Hard

p.3354

```
# The answer is the way to form all of t from all of s
return dp[m][n]
```

```
# Example usage:
print(numDistinct("rabbbit", "rabbit")) # Output: 3
```

• LC.ca

```
class Solution:
    def numDistinct(self, s: str, t: str) -> int:
        n = len(t)
        f = [1] + [0] * n
        for a in s:
            for j in range(n, 0, -1):
                if a == t[j - 1]:
                    f[j] += f[j - 1]
        return f[n]
```

◆ Quick Review

Key Insights

- Use dynamic programming to build the solution by comparing characters of s and t .
- Define a 2D dp table where $dp[i][j]$ represents the number of ways to form the first j characters of t using the first i characters of s .
- The recurrence relation:
- If $s[i-1]$ equals $t[j-1]$, then include both the cases of matching this character as well as skipping it: $dp[i][j] = dp[i-1][j-1] + dp[i-1][j]$.

Round 9 · Low · Hard

p.3355

- If they do not match, then simply skip the current character of s : $dp[i][j] = dp[i-1][j]$.
- Initialize $dp[i][0] = 1$ for all i since an empty t can always be formed.
- The final answer is $dp[s.length][t.length]$.

Space & Time

Time Complexity: $O(n * m)$, where n is the length of s and m is the length of t .

Space Complexity: $O(n * m)$, which can be optimized to $O(m)$ with careful row reuse.

Solution

We use a dynamic programming approach with a 2D table. The table has dimensions $(len(s) + 1) \times (len(t) + 1)$. Each cell $dp[i][j]$ stores the number of distinct subsequences from the first i characters of s that match the first j characters of t .

Steps:

- Initialize $dp[i][0] = 1$ for every i , because an empty t is a subsequence of any prefix of s .
- Iterate over each character of s (outer loop) and for each, iterate over t (inner loop).
- For every character in s , if it equals the current character in t , update $dp[i][j]$ as the sum of the

Round 9 · Low · Hard

p.3356

ways by using or skipping this character.
 Otherwise, carry over the previous count. –
 Return `dp[len(s)][len(t)]` as the result.

Round 9 · Low · Hard

p.3357

String DP

□ #132 Palindrome Partitioning II

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 String · Dynamic Programming
 Lists: Denny Zhang

Problem

Given a string `s`, partition `s` such that every substring of the partition is a palindrome.
 Return the minimum cuts needed for a palindrome partitioning of `s`.

Example 1:

Input: `s = "aab"`
 Output: 1
 Explanation: The palindrome partitioning `["aa","b"]` could be produced using 1 cut.

Example 2:

Input: `s = "a"`
 Output: 0

Example 3:

Input: `s = "ab"`
 Output: 1

Constraints:

- $1 \leq s.length \leq 2000$

Round 9 · Low · Hard

p.3358

- s consists of lowercase English letters only.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minCut(self, s: str) -> int:
        n = len(s)
        # isPalindrome[i][j] := True if s[i..j] is a palindrome
        isPalindrome = [[True] * n for _ in range(n)]
        # dp[i] := the minimum cuts needed for a palindrome partitioning of s[0..i]
        dp = [n] * n

        for l in range(2, n + 1):
            i = 0

            for j in range(l - 1, n):
                isPalindrome[i][j] = s[i] == s[j] and isPalindrome[i + 1][j - 1]
                i += 1

            for i in range(n):
                if isPalindrome[0][i]:
                    dp[i] = 0
                    continue

            # Try all the possible partitions.

            for j in range(i):
                if isPalindrome[j + 1][i]:
                    dp[i] = min(dp[i], dp[j] + 1)

        return dp[-1]
```

• LeetDoocs

```
class Solution:
    def minCut(self, s: str) -> int:
        n = len(s)
        g = [[True] * n for _ in range(n)]
        for i in range(n - 1, -1, -1):
            for j in range(i + 1, n):
                g[i][j] = s[i] == s[j] and g[i + 1][j - 1]
        f = list(range(n))
        for i in range(1, n):
            for j in range(i + 1):
                if g[j][i]:
                    f[i] = min(f[i], 1 + f[j - 1] if j else 0)
        return f[-1]
```

• SimplyLeet

```
def minCut(s: str) -> int:
    n = len(s)
    # Precompute palindrome table where isPal[i][j] is True if s[i:j+1] is a
    # palindrome.
    isPal = [[False] * n for _ in range(n)]
    # dp[i] will store the minimum cuts required for substring s[0...i]
    dp = [0] * n
    for i in range(n):
        min_cut = i # maximum cuts (cut every character)
        for j in range(i + 1):
            # Check if s[j...i] is a palindrome:
            if s[j] == s[i] and (i - j < 2 or isPal[j+1][i-1]):
                isPal[j][i] = True # Mark substring as palindrome.
                # If the palindrome starts at index 0, no cut is needed.
                min_cut = 0 if j == 0 else min(min_cut, dp[j-1] + 1)
        dp[i] = min_cut # Record minimum cuts needed for s[0...i]
    return dp[-1]

# Example usage:
# print(minCut("aab")) # Output: 1
```

• LC.ca

Round 9 · Low · Hard

p.3359

Round 9 · Low · Hard

p.3360

```

class Solution:
    def minCut(self, s: str) -> int:
        pal = [[False for j in range(0, len(s))] for i in range(0, len(s))]
        dp = [len(s) for _ in range(0, len(s) + 1)]
        for i in range(0, len(s)):
            for j in range(0, i + 1):
                # also ok if, (j + 1 >= i - 1)
                if (s[i] == s[j]) and ((j + 1 > i - 1) or (pal[i - 1][j + 1])):
                    pal[i][j] = True
                    dp[i + 1] = min(dp[i + 1], dp[j] + 1) if j != 0 else 0

        # 'if j != 0' to ensure from start to i is a palindrome
        return dp[-1]

#####

class Solution: # clear cache
    def minCut(self, s: str) -> int:
        @cache
        def dfs(i):
            if i >= n - 1:
                return 0
            ans = inf
            for j in range(i, n):
                if g[i][j]:
                    ans = min(ans, dfs(j + 1) + (j < n - 1))
            return ans

        n = len(s)
        g = [[True] * n for _ in range(n)]
        for i in range(n - 1, -1, -1):

```

Round 9 · Low · Hard

p.3361

```

        for j in range(i + 1, n):
            g[i][j] = s[i] == s[j] and g[i + 1][j - 1]
        ans = dfs(0)
        dfs.cache_clear() # nice, clear cache !!!
        return ans

#####

class Solution:
    def minCut(self, s: str) -> int:
        n = len(s)
        is_pal = [[False] * n for _ in range(n)]
        for i in range(n - 1, -1, -1):
            for j in range(i, n):
                is_pal[i][j] = s[i] == s[j] and (j - i < 3 or is_pal[i + 1][j - 1])

        min_cut = [i for i in range(n)]
        for i in range(n):
            for j in range(0, i + 1):
                if is_pal[j][i]:
                    min_cut[i] = min(min_cut[i], min_cut[j - 1] + 1) if j > 0
            else 0

        return min_cut[-1]

```

◆ Quick Review

Key Insights

- Precompute a 2D table (isPal) where isPal[i][j] indicates if the substring s[i...j] is a palindrome.
- Use dynamic programming: dp[i] represents the minimum cuts needed for the substring s[0...i].

Round 9 · Low · Hard

p.3362

- For every index i , iterate over all possible starting indices j . If $s[j...i]$ is a palindrome, update $dp[i]$ using $dp[j-1] + 1$ (or 0 if j is 0).
- Time complexity is dominated by the $O(n^2)$ palindrome checking and state transitions.

Space & Time

Time Complexity: $O(n^2)$

Space Complexity: $O(n^2)$ (due to the 2D table for palindrome checks, plus $O(n)$ for the dp array)

Solution

The solution relies on dynamic programming with a two-step approach. First, precompute a 2D table (`isPal`) where each cell indicates if a substring is a palindrome. This is done in $O(n^2)$ time by leveraging the fact that a substring $s[j...i]$ is a palindrome if the characters at the ends match and the inner substring $s[j+1...i-1]$ is also a palindrome (or if the length is less than 3). Then, use a dp array where $dp[i]$ holds the minimum number of cuts needed for the substring $s[0...i]$. For each i , check every index j ($0 \leq j \leq i$) and if $s[j...i]$ is a palindrome, update $dp[i]$ to be the minimum

Round 9 · Low · Hard

p.3363

of its current value and $dp[j-1] + 1$ (with a special case when j is 0, meaning no cut is needed). This two-phase approach efficiently breaks down the problem and computes the answer while avoiding redundant palindrome checks.

Round 9 · Low · Hard

p.3364

String DP

□ #140 Word Break II

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · String · Dynamic
Programming · Backtracking · Trie ·
Memoization

Lists: AlgoMaster 600

Problem

Given a string s and a dictionary of strings $wordDict$, add spaces in s to construct a sentence where each word is a valid dictionary word. Return all such possible sentences in any order.

Note that the same word in the dictionary may be reused multiple times in the segmentation.

Example 1:

Input: $s = \text{"catsanddog"}$, $wordDict = [\text{"cat"}, \text{"cats"}, \text{"and"}, \text{"sand"}, \text{"dog"}]$
Output: $[\text{"cats and dog"}, \text{"cat sand dog"}]$

Example 2:

Input: $s = \text{"pineapplepenapple"}$, $wordDict = [\text{"apple"}, \text{"pen"}, \text{"applepen"}, \text{"pine"}, \text{"pineapple"}]$
Output: $[\text{"pine apple pen apple"}, \text{"pineapple pen apple"}, \text{"pine applepen apple"}]$
Explanation: Note that you are allowed to reuse a dictionary word.

Round 9 · Low · Hard

p.3365

Example 3:

Input: $s = \text{"catsanddog"}$, $wordDict = [\text{"cats"}, \text{"dog"}, \text{"sand"}, \text{"and"}, \text{"cat"}]$
Output: $[\text{ }]$

Constraints:

- $1 \leq s.length \leq 20$
- $1 \leq wordDict.length \leq 1000$
- $1 \leq wordDict[i].length \leq 10$
- s and $wordDict[i]$ consist of only lowercase English letters.
- All the strings of $wordDict$ are unique.
- Input is generated in a way that the length of the answer doesn't exceed 105.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def wordBreak(self, s: str, wordDict: list[str]) -> list[str]:
        wordSet = set(wordDict)

        @functools.lru_cache(None)
        def wordBreak(s: str) -> list[str]:
            ans = []

            # 1 <= len(prefix) < len(s)
            for i in range(1, len(s)):
                prefix = s[:i]
                suffix = s[i:]
                if prefix in wordSet:
                    if suffix in wordSet:
                        ans.append(prefix + " " + suffix)
                    elif suffix in wordSet:
                        ans.append(suffix)
                    else:
                        ans.extend(wordBreak(suffix))
            return ans
```

Round 9 · Low · Hard

p.3366

```

    prefix = s[:i]
    suffix = s[i:]
    if prefix in wordSet:
        for word in wordBreak(suffix):
            ans.append(prefix + ' ' + word)

# `wordSet` contains the whole string s, so don't add any space.
if s in wordSet:
    ans.append(s)

return ans

return wordBreak(s)

```

• LeetCode

```

class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.is_end = False

    def insert(self, word):
        node = self
        for c in word:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
        node.is_end = True

    def search(self, word):
        node = self
        for c in word:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                return False

```

Round 9 · Low · Hard

p.3367

```

        node = node.children[idx]
        return node.is_end

class Solution:
    def wordBreak(self, s: str, wordDict: List[str]) -> List[str]:
        def dfs(s):
            if not s:
                return [[]]
            res = []

            for i in range(1, len(s) + 1):
                if trie.search(s[:i]):
                    for v in dfs(s[i:]):
                        res.append([s[:i]] + v)

            return res

        trie = Trie()
        for w in wordDict:
            trie.insert(w)
        ans = dfs(s)

        return [' '.join(v) for v in ans]

```

• SimplyLeet

```

# Python solution using memoized recursion (DFS)

def wordBreak(s, wordDict):
    word_set = set(wordDict) # Use a set for O(1) lookups
    memo = {} # dictionary to memoize results from each start index

    def dfs(start):
        # if reached the end of the string, return a list with empty string
        if start == len(s):
            return [""]

```

Round 9 · Low · Hard

p.3368


```

if start in memo:
    return memo[start]

sentences = []
for end in range(start + 1, len(s) + 1):
    word = s[start:end]
    if word in word_set:
        # recursive call to get sentences for the remaining substring
        sub_sentences = dfs(end)

        # build sentences with proper spacing
        for sub in sub_sentences:
            sentence = word + (" " + sub if sub else "")
            sentences.append(sentence)

memo[start] = sentences # cache result
return sentences

return dfs(0)

# Example usage:
if __name__ == "__main__":
    s = "catsanddog"
    wordDict = ["cat", "cats", "and", "sand", "dog"]
    result = wordBreak(s, wordDict)
    print(result) # Expected: ["cats and dog", "cat sand dog"]

```

String DP

□ #664 Strange Printer

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming
Lists: AlgoMaster 600

Problem

There is a strange printer with the following two special properties:

- The printer can only print a sequence of the same character each time.
- At each turn, the printer can print new characters starting from and ending at any place and will cover the original existing characters.

Given a string *s*, return the minimum number of turns the printer needed to print it.

Example 1:

Input: *s* = "aaabbb"
Output: 2
Explanation: Print "aaa" first and then print "bbb".

Example 2:

Input: s = "aba"
 Output: 2
 Explanation: Print "aaa" first and then print "b" from the second place of the string, which will cover the existing character 'a'.

Constraints:

- $1 \leq \text{s.length} \leq 100$
- s consists of lowercase English letters.

Community Solutions (Python)

• LeetCode

```
class Solution:
    def strangePrinter(self, s: str) -> int:
        n = len(s)
        f = [[inf] * n for _ in range(n)]
        for i in range(n - 1, -1, -1):
            f[i][i] = 1
            for j in range(i + 1, n):
                if s[i] == s[j]:
                    f[i][j] = f[i][j - 1]
                else:
                    for k in range(i, j):
                        f[i][j] = min(f[i][j], f[i][k] + f[k + 1][j])
        return f[0][n - 1]
```

• SimplyLeet

```
# Python solution for Strange Printer

def strangePrinter(s: str) -> int:
    n = len(s)
    # Initialize a 2D dp table with zeros.
    dp = [[0] * n for _ in range(n)]

    # Base case: each single character requires 1 turn.
    for i in range(n):
        dp[i][i] = 1
```

Round 9 · Low · Hard

p.3371

```
# Process all substrings of increasing lengths.
for length in range(2, n + 1):
    for i in range(n - length + 1):
        j = i + length - 1
        # Assume printing s[j] separately after s[i...j-1].
        dp[i][j] = dp[i][j - 1] + 1
        # Try merging operations where s[k] equals s[j].
        for k in range(i, j):
            if s[k] == s[j]:
                # When merging, cost is dp[i][k] plus dp[k+1][j-1], if any.
                cost = dp[i][k] + (dp[k + 1][j - 1] if k + 1 <= j - 1 else
                                0)
                dp[i][j] = min(dp[i][j], cost)
        return dp[0][n - 1]

# Example usage:
s = "aba"
print(strangePrinter(s)) # Output: 2
```

Round 9 · Low · Hard

p.3372

String DP

#1092 Shortest Common Supersequence **HAR**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming
Lists: AlgoMaster 600

Problem

Given two strings `str1` and `str2`, return the shortest string that has both `str1` and `str2` as subsequences. If there are multiple valid strings, return any of them.

A string `s` is a subsequence of string `t` if deleting some number of characters from `t` (possibly 0) results in the string `s`.

Example 1:

Input: `str1 = "abac", str2 = "cab"`
Output: `"cabac"`
Explanation:
`str1 = "abac"` is a subsequence of `"cabac"` because we can delete the first `"c"`.
`str2 = "cab"` is a subsequence of `"cabac"` because we can delete the last `"ac"`.
The answer provided is the shortest such string that satisfies these properties.

Example 2:

Input: `str1 = "aaaaaaa", str2 = "aaaaaaa"`
Output: `"aaaaaaa"`

Constraints:

- `1 <= str1.length, str2.length <= 1000`
- `str1` and `str2` consist of lowercase English letters.

Community Solutions (Python)

• LeetDoocs

```
class Solution:
    def shortestCommonSupersequence(self, str1: str, str2: str) -> str:
        m, n = len(str1), len(str2)
        f = [[0] * (n + 1) for _ in range(m + 1)]
        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if str1[i - 1] == str2[j - 1]:
                    f[i][j] = f[i - 1][j - 1] + 1
                else:
                    f[i][j] = max(f[i - 1][j], f[i][j - 1])

        ans = []
        i, j = m, n
        while i or j:
            if i == 0:
                j -= 1
                ans.append(str2[j])
            elif j == 0:
                i -= 1
                ans.append(str1[i])
            else:
                if f[i][j] == f[i - 1][j]:
                    i -= 1
                    ans.append(str1[i])
                elif f[i][j] == f[i][j - 1]:
                    j -= 1
                    ans.append(str2[j])
                else:
                    i, j = i - 1, j - 1
                    ans.append(str1[i])
        return ''.join(ans[::-1])
```

• SimplyLeet

Python solution using DP for LCS and reconstruction for SCS.

```
def shortestCommonSupersequence(str1: str, str2: str) -> str:
    m, n = len(str1), len(str2)
    # DP table initialization: dp[i][j] will hold LCS for str1[:i] and str2[:j]
    dp = [["" for _ in range(n + 1)] for _ in range(m + 1)]
```

```
    # Build the dp table for LCS string reconstruction.
    for i in range(1, m + 1):
        for j in range(1, n + 1):
```

```
            if str1[i - 1] == str2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + str1[i - 1]
            else:
                # Choose the longer LCS when characters do not match
                if len(dp[i - 1][j]) > len(dp[i][j - 1]):
                    dp[i][j] = dp[i - 1][j]
                else:
                    dp[i][j] = dp[i][j - 1]
```

```
    # Reconstruct the shortest common supersequence using the LCS.
```

```
    lcs = dp[m][n]
    i = j = 0
    scs = []
    # Append non-LCS characters from str1 and str2 until the LCS characters
    appear.
    for c in lcs:
        # Append all characters from str1 until we hit the LCS character.
        while str1[i] != c:
            scs.append(str1[i])
            i += 1
        # Append all characters from str2 until we hit the LCS character.
```

```
        while str2[j] != c:
            scs.append(str2[j])
            j += 1
        # Now append the LCS character.
        scs.append(c)
        i += 1
        j += 1
```

```
    # Append any remaining characters after processing the LCS.
    scs.extend(list(str1[i:]))
```

```
    scs.extend(list(str2[j:]))
    return "".join(scs)
```

Example test cases:

```
print(shortestCommonSupersequence("abac", "cab")) # Expected output: "cabac"
(or any valid SCS)
print(shortestCommonSupersequence("aaaaaaa", "aaaaaaa")) # Expected output:
"aaaaaaa"
```

Tree / Graph DP

Round 9 · Low · Hard · 2 questions

Round 9 · Low · Hard

p.3377

Tree / Graph DP

□ #834 Sum of Distances in Tree

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Dynamic Programming · Tree · Depth-First
Search · Graph Theory
Lists: AlgoMaster 600

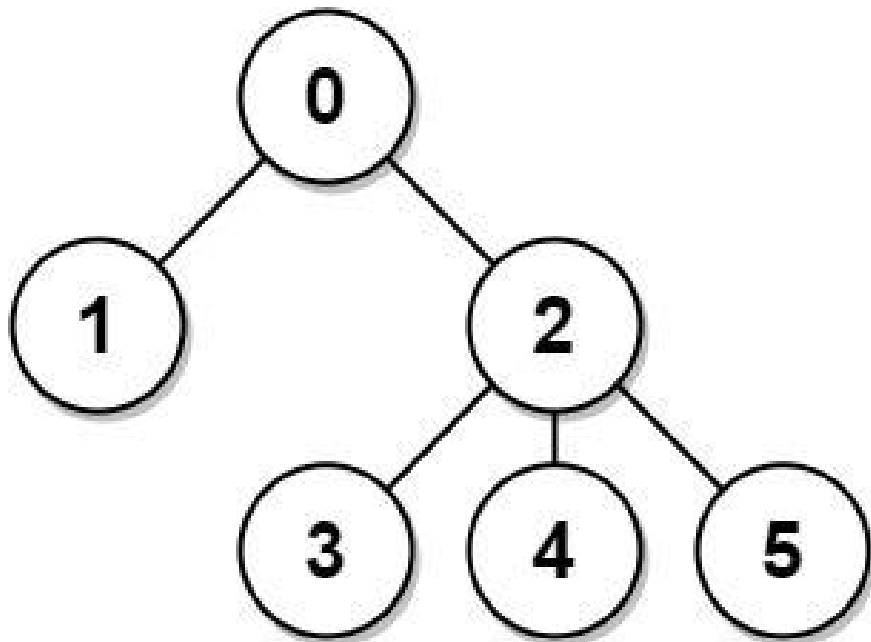
Problem

There is an undirected connected tree with n nodes labeled from 0 to $n - 1$ and $n - 1$ edges. You are given the integer n and the array `edges` where `edges[i] = [ai, bi]` indicates that there is an edge between nodes a_i and b_i in the tree. Return an array `answer` of length n where `answer[i]` is the sum of the distances between the i th node in the tree and all other nodes.

Example 1:

Round 9 · Low · Hard

p.3378



Input: $n = 6$, $edges = [[0,1],[0,2],[2,3],[2,4],[2,5]]$

Output: $[8,12,6,10,10,10]$

Explanation: The tree is shown above.

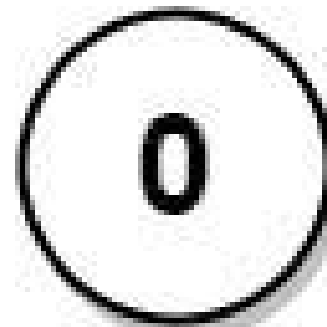
We can see that $dist(0,1) + dist(0,2) + dist(0,3) + dist(0,4) + dist(0,5)$ equals $1 + 1 + 2 + 2 + 2 = 8$.

Hence, $answer[0] = 8$, and so on.

Example 2:

Round 9 · Low · Hard

p.3379



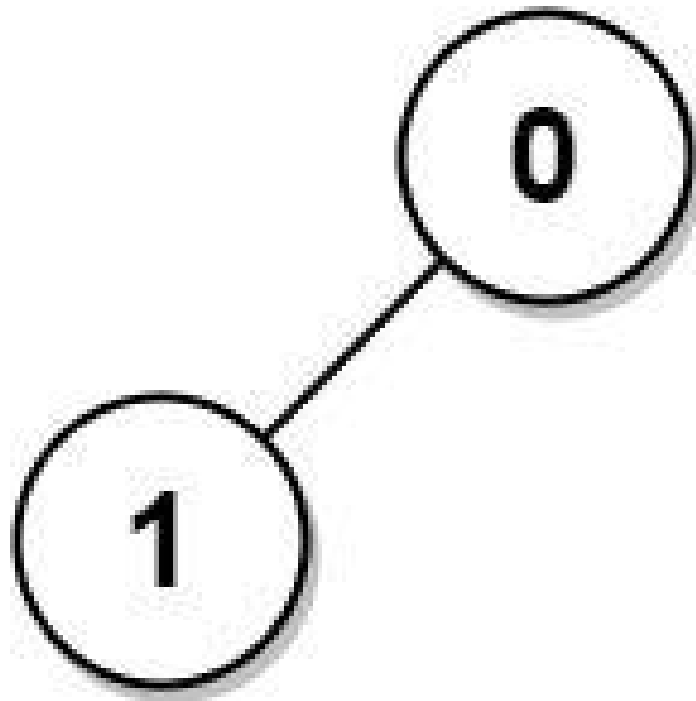
Input: $n = 1$, $edges = []$

Output: $[0]$

Example 3:

Round 9 · Low · Hard

p.3380



Input: $n = 2$, $edges = [[1,0]]$
 Output: $[1,1]$

Constraints:

- $1 \leq n \leq 3 * 10^4$
- $edges.length == n - 1$
- $edges[i].length == 2$
- $0 \leq a_i, b_i < n$
- $a_i \neq b_i$
- The given input represents a valid tree.

Round 9 · Low · Hard

p.3381

Community Solutions (Python)

• WalkCC

```

class Solution:
    def sumOfDistancesInTree(self, n: int, edges: list[list[int]]) -> list[int]:
        ans = [0] * n
        count = [1] * n
        tree = [set() for _ in range(n)]

        for u, v in edges:
            tree[u].add(v)
            tree[v].add(u)

        def postorder(u: int, prev: int) -> None:
            for v in tree[u]:
                if v == prev:
                    continue
                postorder(v, u)
                count[u] += count[v]
                ans[u] += ans[v] + count[v]

        def preorder(u: int, prev: int) -> None:
            for v in tree[u]:
                if v == prev:
                    continue
                # count[v] us are 1 step closer from v than prev.
                # (n - count[v]) us are 1 step farther from v than prev.
                ans[v] = ans[u] - count[v] + (n - count[v])
                preorder(v, u)

        postorder(0, -1)
        preorder(0, -1)
        return ans
  
```

• LeetCode

Round 9 · Low · Hard

p.3382

```

class Solution:
    def sumOfDistancesInTree(self, n: int, edges: List[List[int]]) -> List[int]:
        def dfs1(i: int, fa: int, d: int):
            ans[i] += d
            size[i] = 1
            for j in g[i]:
                if j != fa:
                    dfs1(j, i, d + 1)
            size[i] += size[j]

        def dfs2(i: int, fa: int, t: int):
            ans[i] = t
            for j in g[i]:
                if j != fa:
                    dfs2(j, i, t - size[j] + n - size[j])

        g = defaultdict(list)
        for a, b in edges:
            g[a].append(b)
            g[b].append(a)

        ans = [0] * n
        size = [0] * n
        dfs1(0, -1, 0)
        dfs2(0, -1, ans[0])
        return ans

```

• SimplyLeet

```

# Python solution
class Solution:
    def sumOfDistancesInTree(self, n: int, edges: List[List[int]]) -> List[int]:
        # Build tree as an adjacency list
        tree = [[] for _ in range(n)]
        for u, v in edges:
            tree[u].append(v)
            tree[v].append(u)

        # Initialize result and count arrays

```

Round 9 · Low · Hard

p.3383

```

res = [0] * n
count = [1] * n # each node counts as 1

# Postorder DFS: compute count and res for subtrees
def dfs(node: int, parent: int):
    for child in tree[node]:
        if child == parent:
            continue
        dfs(child, node)
        count[node] += count[child]

        res[node] += res[child] + count[child]
    dfs(0, -1)

# Preorder DFS: update results using parent's information
def dfs2(node: int, parent: int):
    for child in tree[node]:
        if child == parent:
            continue
        # Update result for child using parent's result formula
        res[child] = res[node] - count[child] + (n - count[child])

        dfs2(child, node)
    dfs2(0, -1)

return res

```

Round 9 · Low · Hard

p.3384

Tree / Graph DP

□ #968 Binary Tree Cameras

HAR

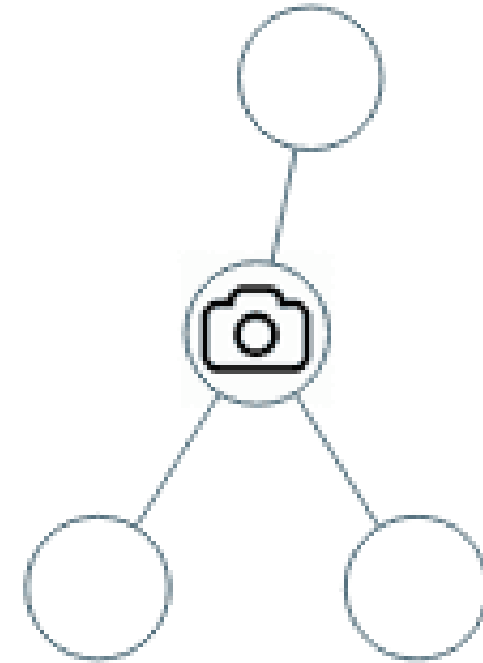
LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Dynamic Programming · Tree · Depth-First
 Search · Binary Tree
 Lists: AlgoMaster 600

Problem

You are given the root of a binary tree. We install cameras on the tree nodes where each camera at a node can monitor its parent, itself, and its immediate children.

Return the minimum number of cameras needed to monitor all nodes of the tree.

Example 1:



Input: root = [0,0,null,0,0]

Output: 1

Explanation: One camera is enough to monitor all nodes if placed as shown.

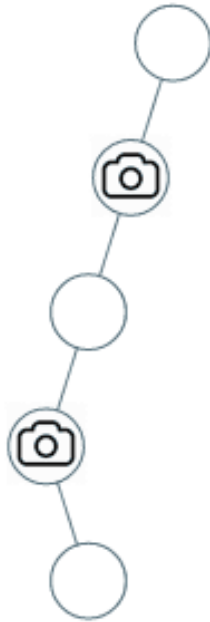
Example 2:

Round 9 · Low · Hard

p.3385

Round 9 · Low · Hard

p.3386



Input: root = [0,0,null,0,null,0,null,null,0]

Output: 2

Explanation: At least two cameras are needed to monitor all nodes of the tree. The above image shows one of the valid configurations of camera placement.

Constraints:

- The number of nodes in the tree is in the range [1, 1000].
- Node.val == 0

Community Solutions (Python)

- LeetCode

Round 9 · Low · Hard

p.3387

```
# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):
# self.val = val
# self.left = left
# self.right = right
class Solution:
    def minCameraCover(self, root: Optional[TreeNode]) -> int:
        def dfs(root):
            if root is None:
                return inf, 0, 0
            la, lb, lc = dfs(root.left)
            ra, rb, rc = dfs(root.right)
            a = min(la, lb, lc) + min(ra, rb, rc) + 1
            b = min(la + rb, lb + ra, la + ra)
            c = lb + rb
            return a, b, c

        a, b, _ = dfs(root)
        return min(a, b)
```

• SimplyLeet

```
# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):
# self.val = val
# self.left = left
# self.right = right
class Solution:
    def minCameraCover(self, root: TreeNode) -> int:
        # Define states:
```

Round 9 · Low · Hard

p.3388

```
# 0: Node has a camera.
# 1: Node is covered, but has no camera.
# 2: Node is not covered.
```

```
self.camera_count = 0
```

```
def dfs(node):
    if not node:
        # Null nodes are considered covered.
        return 1
```

```
    left = dfs(node.left)
    right = dfs(node.right)
```

```
    # If any child is not covered, current node must have a camera.
    if left == 2 or right == 2:
        self.camera_count += 1
        return 0 # Place camera at current node.
```

```
    # If any child has a camera, current node is covered.
```

```
    if left == 0 or right == 0:
        return 1 # Covered but no camera.
```

```
    # Else, node is not covered.
    return 2
```

```
if dfs(root) == 2:
    self.camera_count += 1
return self.camera_count
```

Round 9 · Low · Hard

p.3389

Bitmask DP

Round 9 · Low · Hard · 1 question

Round 9 · Low · Hard

p.3390

Bitmask DP

#847 Shortest Path Visiting All Nodes

HAR

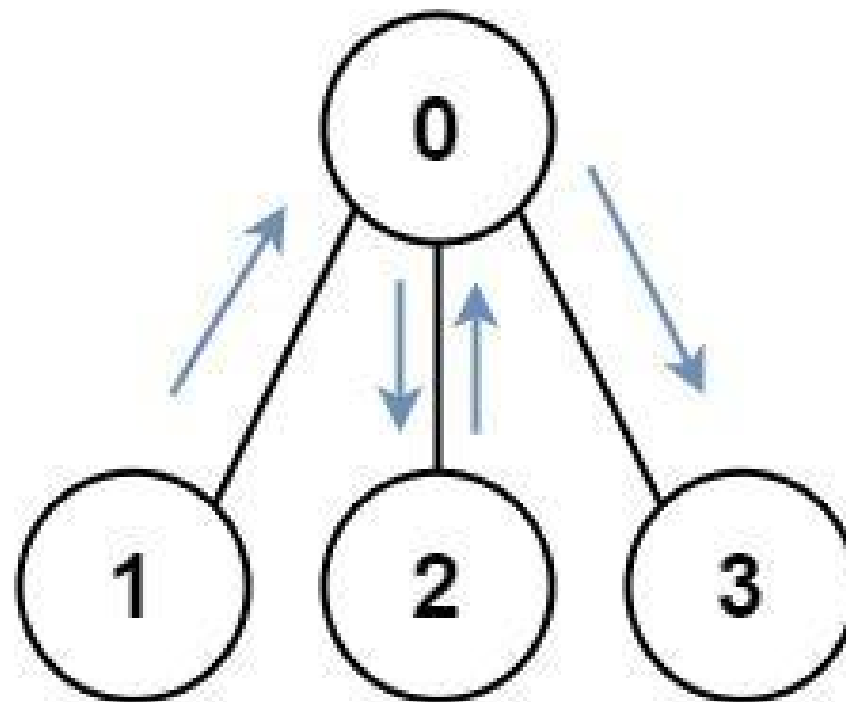
LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Dynamic Programming · Bit Manipulation ·
 Breadth-First Search · Graph Theory · Bitmask
 Lists: AlgoMaster 600

Problem

You have an undirected, connected graph of n nodes labeled from 0 to $n - 1$. You are given an array `graph` where `graph[i]` is a list of all the nodes connected with node i by an edge.

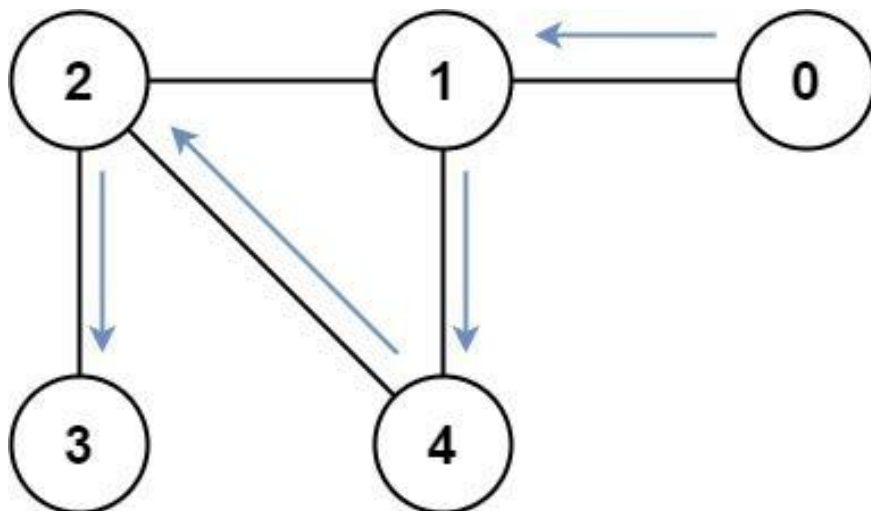
Return the length of the shortest path that visits every node. You may start and stop at any node, you may revisit nodes multiple times, and you may reuse edges.

Example 1:



Input: `graph = [[1,2,3],[0],[0],[0]]`
 Output: 4
 Explanation: One possible path is `[1,0,2,0,3]`

Example 2:



Input: graph = [[1],[0,2,4],[1,3,4],[2],[1,2]]

Output: 4

Explanation: One possible path is [0,1,4,2,3]

Constraints:

- $n == \text{graph.length}$
- $1 \leq n \leq 12$
- $0 \leq \text{graph}[i].\text{length} < n$
- $\text{graph}[i]$ does not contain i .
- If $\text{graph}[a]$ contains b , then $\text{graph}[b]$ contains a .
- The input graph is always connected.

Community Solutions (Python)

- WalkCC

Round 9 · Low · Hard

p.3393

```
class Solution:
    def shortestPathLength(self, graph: list[list[int]]) -> int:
        n = len(graph)
        goal = (1 << n) - 1
        q = collections.deque() # (u, state)
        seen = set()

        for i in range(n):
            q.append((i, 1 << i))

        step = 0
        while q:
            for _ in range(len(q)):
                u, state = q.popleft()
                if state == goal:
                    return step
                if (u, state) in seen:
                    continue
                seen.add((u, state))
                for v in graph[u]:
                    q.append((v, state | 1 << v))
            step += 1

        return -1
```

• LeetCode

```
class Solution:
    def shortestPathLength(self, graph: List[List[int]]) -> int:
        n = len(graph)
        q = deque()
        vis = set()
        for i in range(n):
            q.append((i, 1 << i))
            vis.add((i, 1 << i))
        ans = 0
        while 1:
```

Round 9 · Low · Hard

p.3394

```

    for _ in range(len(q)):
        i, st = q.popleft()
        if st == (1 << n) - 1:
            return ans
        for j in graph[i]:
            nst = st | 1 << j
            if (j, nst) not in vis:
                vis.add((j, nst))
                q.append((j, nst))
    ans += 1

```

• SimplyLeet

```

# Import required collections for the queue
from collections import deque

def shortestPathLength(graph):
    n = len(graph)
    # If there is only one node, we're already done
    if n == 1:
        return 0

    # All nodes visited bitmask (e.g., for n=3, all_visited = 0b111)

```

```

    all_visited = (1 << n) - 1

    # Creating a queue for BFS where each element is (current_node,
    visited_mask)
    queue = deque()
    # Use a set to record visited (node, mask) states
    seen = set()

    # Multi-source initialization: start from every node.
    for i in range(n):
        # visited_mask with only the i-th bit set

```

```

    mask = 1 << i
    queue.append((i, mask, 0)) # (node, visited_mask, distance)
    seen.add((i, mask))

```

```

# BFS loop
while queue:
    node, mask, dist = queue.popleft()
    # Check all adjacent nodes
    for neighbor in graph[node]:
        # Update the visited mask with the neighbor visited

```

```

        next_mask = mask | (1 << neighbor)
        # Return result if all nodes have been visited
        if next_mask == all_visited:
            return dist + 1
        # If this state is not seen, add it to the queue
        if (neighbor, next_mask) not in seen:
            seen.add((neighbor, next_mask))
            queue.append((neighbor, next_mask, dist + 1))

```

```

# Fallback (should not be reached since graph is connected)

```

```

    return -1

```

```

# Example usage:
# graph = [[1,2,3],[0],[0],[0]]
# print(shortestPathLength(graph))

```

Digit DP

Round 9 · Low · Hard · 2 questions

Round 9 · Low · Hard

p.3397

Digit DP

□ #233 Number of Digit One

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Math · Dynamic Programming · Recursion
 Lists: AlgoMaster 600

Problem

Given an integer n , count the total number of digit 1 appearing in all non-negative integers less than or equal to n .

Example 1:

Input: $n = 13$
 Output: 6

Example 2:

Input: $n = 0$
 Output: 0

Constraints:

- $0 \leq n \leq 10^9$

Community Solutions (Python)

- • WalkCC

Round 9 · Low · Hard

p.3398

```
class Solution:
    def countDigitOne(self, n: int) -> int:
        ans = 0

        pow10 = 1
        while pow10 <= n:
            divisor = pow10 * 10
            quotient = n // divisor
            remainder = n % divisor
            if quotient > 0:

                ans += quotient * pow10
            if remainder >= pow10:
                ans += min(remainder - pow10 + 1, pow10)
            pow10 *= 10

        return ans
```

• LeetDoocs

```
class Solution:
    def countDigitOne(self, n: int) -> int:
        @cache
        def dfs(i: int, cnt: int, limit: bool) -> int:
            if i >= len(s):
                return cnt
            up = int(s[i]) if limit else 9
            ans = 0
            for j in range(up + 1):
                ans += dfs(i + 1, cnt + (j == 1), limit and j == up)

            return ans

        s = str(n)
        return dfs(0, 0, True)
```

• SimplyLeet

Round 9 · Low · Hard

p.3399

```
# Python solution for counting the number of digit one in range [0, n]

def countDigitOne(n: int) -> int:
    count = 0
    i = 1 # digit position (1, 10, 100, ...)
    while i <= n:
        # Divide n into parts based on the current digit's place value.
        divider = i * 10
        high = n // divider # higher part of n
        current = (n // i) % 10 # digit at current position

        low = n % i # lower part of n

        # Accumulate count from high digits contributions.
        if current == 0:
            count += high * i
        elif current == 1:
            count += high * i + low + 1
        else:
            count += (high + 1) * i

        # Move to the next position.
        i *= 10

    return count

# Example usage:
print(countDigitOne(13)) # Output: 6
```

Round 9 · Low · Hard

p.3400

Digit DP

#902 Numbers At Most N Given Digit Set **HAR**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Math · String · Binary Search · Dynamic Programming

Lists: AlgoMaster 600

Problem

Given an array of digits which is sorted in non-decreasing order. You can write numbers using each `digits[i]` as many times as we want. For example, if `digits = ['1','3','5']`, we may write numbers such as '13', '551', and '1351315'.

Return the number of positive integers that can be generated that are less than or equal to a given integer `n`.

Example 1:

Input: `digits = ["1","3","5","7"]`, `n = 100`
Output: 20
Explanation:
The 20 numbers that can be written are:
1, 3, 5, 7, 11, 13, 15, 17, 31, 33, 35, 37, 51, 53, 55, 57, 71, 73, 75, 77.

Example 2:

Input: `digits = ["1","4","9"]`, `n = 1000000000`
Output: 29523
Explanation:
We can write 3 one digit numbers, 9 two digit numbers, 27 three digit numbers, 81 four digit numbers, 243 five digit numbers, 729 six digit numbers, 2187 seven digit numbers, 6561 eight digit numbers, and 19683 nine digit numbers.
In total, this is 29523 integers that can be written using the `digits` array.

Example 3:

Input: `digits = ["7"]`, `n = 8`
Output: 1

Constraints:

- `1 <= digits.length <= 9`
- `digits[i].length == 1`
- `digits[i]` is a digit from '1' to '9'.
- All the values in `digits` are unique.
- `digits` is sorted in non-decreasing order.
- `1 <= n <= 109`

Community Solutions (Python)

• WalkCC

```
class Solution:
    def atMostNGivenDigitSet(self, digits: list[str], n: int) -> int:
        ans = 0
        num = str(n)

        for i in range(1, len(num)):
            ans += pow(len(digits), i)

        for i, c in enumerate(num):
            dHasSameNum = False
```

```

for digit in digits:
    if digit[0] < c:
        ans += pow(len(digits), len(num) - i - 1)
    elif digit[0] == c:
        dHasSameNum = True
    if not dHasSameNum:
        return ans

return ans + 1

```

• LeetCode

```

class Solution:
    def atMostNGivenDigitSet(self, digits: List[str], n: int) -> int:
        @cache
        def dfs(i: int, lead: int, limit: bool) -> int:
            if i >= len(s):
                return lead ^ 1

            up = int(s[i]) if limit else 9
            ans = 0
            for j in range(up + 1):

                if j == 0 and lead:
                    ans += dfs(i + 1, 1, limit and j == up)
                elif j in nums:
                    ans += dfs(i + 1, 0, limit and j == up)
            return ans

        s = str(n)
        nums = {int(x) for x in digits}
        return dfs(0, 1, True)

```

• SimplyLeet

```

# Python Solution
def atMostNGivenDigitSet(digits, n):
    # Convert n to string for easy digit comparison
    s = str(n)
    n_len = len(s)
    k = len(digits)
    count = 0

    # Count numbers with length less than n
    for i in range(1, n_len):

        count += k ** i

    # Process numbers with length equal to n
    for i, ch in enumerate(s):
        # Count valid digits less than current digit ch in n
        smaller_count = 0
        for d in digits:
            if d < ch:
                smaller_count += 1
            else:
                break # since digits is sorted, no further digit is smaller

        # Options for the current digit position multiplied by choices for
        # remaining positions
        count += smaller_count * (k ** (n_len - i - 1))

        # If the current digit ch is not in digits, no need to continue further
        if ch not in digits:
            return count # early termination

    # If the loop completes, n itself can be formed using digits

    return count + 1

# Example usage:
#print(atMostNGivenDigitSet(["1", "3", "5", "7"], 100))

```

State Machine DP

Round 9 · Low · Hard · 2 questions

Round 9 · Low · Hard

p.3405

State Machine DP

□

#123 Best Time to Buy and Sell Stock III

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming

Lists: Denny Zhang

Problem

You are given an array prices where prices[i] is the price of a given stock on the ith day.

Find the maximum profit you can achieve. You may complete at most two transactions.

Note: You may not engage in multiple transactions simultaneously (i.e., you must sell the stock before you buy again).

Example 1:

Input: prices = [3,3,5,0,0,3,1,4]

Output: 6

Explanation: Buy on day 4 (price = 0) and sell on day 6 (price = 3), profit = 3-0 = 3.

Then buy on day 7 (price = 1) and sell on day 8 (price = 4), profit = 4-1 = 3.

Example 2:

Round 9 · Low · Hard

p.3406


```
# Backward pass: compute max profit from day i to end with one transaction.
max_price = prices[-1]
for i in range(n - 2, -1, -1):
    # Update max price seen from the right.
    max_price = max(max_price, prices[i])
    # Maximum profit possible starting from day i.
    right_profits[i] = max(right_profits[i + 1], max_price - prices[i])
```

```
# Combine the two profits to find the maximum total profit.
max_total_profit = 0
```

```
for i in range(n):
    max_total_profit = max(max_total_profit, left_profits[i] + right_profits[i])
```

```
return max_total_profit
```

```
# Example usage:
prices = [3,3,5,0,0,3,1,4]
print(max_profit(prices)) # Expected output: 6
```

• LC.ca

```
'''
- `f1`  []
- `f2`  []
- `f3`  []
- `f4`  []
'''
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        # []
        f1, f2, f3, f4 = -prices[0], 0, -prices[0], 0
```

```
for price in prices[1:]:
    f1 = max(f1, -price)
    f2 = max(f2, f1 + price)
    f3 = max(f3, f2 - price)
    f4 = max(f4, f3 + price)
return f4
```

```
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
```

```
if not prices:
    return 0
```

```
# 0 to i, max profit
left_max = [0] * len(prices)
```

```
tmp_min = prices[0]
for i in range(len(prices)):
    tmp_min = min(tmp_min, prices[i])
    left_max[i] = max(left_max[i-1], prices[i]-tmp_min) if i > 0 else 0
```

```
# i to end, max profit
right_max = [0] * len(prices)

tmp_max = prices[-1]
for i in range(len(prices)-1, -1, -1):
    tmp_max = max(tmp_max, prices[i])
    right_max[i] = max(right_max[i+1], tmp_max-prices[i]) if i < len(prices)-1 else 0

return max(left_max[i] + right_max[i] for i in range(len(prices)))
```

```
#####

class Solution(object):
    def maxProfit(self, prices):
        """
        :type prices: List[int]
        :rtype: int
        """
        buy1 = buy2 = float("-inf")

        sell1 = sell2 = 0

        for i in range(len(prices)):
            sell1 = max(prices[i] + buy1, sell1)
            buy1 = max(buy1, -prices[i])
            sell2 = max(sell2, prices[i] + buy2)
            buy2 = max(sell1 - prices[i], buy2)
        return max(sell1, sell2)
```

◆ Quick Review

Key Insights

- The problem requires the optimal selection of at most two buy-sell transactions.
- One effective strategy is to split the problem into two parts:
 - For each day i , calculate the maximum profit obtainable from day 0 to i with one transaction.
 - For each day i , calculate the maximum profit obtainable from day i to the end with one transaction.

Round 9 · Low · Hard

p.3411

- Finally, for each day, the sum of these two profits (left and right) gives a candidate answer.
- Alternatively, a state-based dynamic programming approach can track 4 states corresponding to the two transactions.

Space & Time

Time Complexity: $O(n)$ where n is the number of days

Space Complexity: $O(n)$ due to extra arrays used for forward and backward computations (this can be optimized to $O(1)$ with a state variable approach)

Solution

We use a two-pass dynamic programming approach. In the first pass (forward), we compute an array "leftProfits" where $\text{leftProfits}[i]$ is the maximum profit from day 0 to day i with one transaction. We do this by keeping track of the minimum price observed so far and calculating the profit if sold on day i . In the second pass (backward), we compute an array "rightProfits" where $\text{rightProfits}[i]$ is the maximum profit obtainable from day i to

Round 9 · Low · Hard

p.3412

the last day with one transaction by tracking the maximum price seen in reverse order. Finally, for every index i , the sum $\text{leftProfits}[i] + \text{rightProfits}[i]$ represents the maximum profit achievable by completing the first transaction on or before day i and the second transaction starting from day i . The answer is the maximum sum over all i .

State Machine DP

□ #188 Best Time to Buy and Sell Stock IV **HAR**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Dynamic Programming
 Lists: AlgoMaster 600

Problem

You are given an integer array `prices` where `prices[i]` is the price of a given stock on the i th day, and an integer k .

Find the maximum profit you can achieve. You may complete at most k transactions: i.e. you may buy at most k times and sell at most k times.

Note: You may not engage in multiple transactions simultaneously (i.e., you must sell the stock before you buy again).

Example 1:

Input: $k = 2$, `prices = [2,4,1]`
 Output: 2
 Explanation: Buy on day 1 (price = 2) and sell on day 2 (price = 4), profit = $4 - 2 = 2$.

Example 2:

Input: k = 2, prices = [3,2,6,5,0,3]

Output: 7

Explanation: Buy on day 2 (price = 2) and sell on day 3 (price = 6), profit = 6-2 = 4. Then buy on day 5 (price = 0) and sell on day 6 (price = 3), profit = 3-0 = 3.

Constraints:

- $1 \leq k \leq 100$
- $1 \leq \text{prices.length} \leq 1000$
- $0 \leq \text{prices}[i] \leq 1000$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxProfit(self, k: int, prices: list[int]) -> int:
        if k >= len(prices) // 2:
            sell = 0
            hold = -math.inf

            for price in prices:
                sell = max(sell, hold + price)
                hold = max(hold, sell - price)

            return sell

        sell = [0] * (k + 1)
        hold = [-math.inf] * (k + 1)

        for price in prices:
            for i in range(k, 0, -1):
                sell[i] = max(sell[i], hold[i] + price)
                hold[i] = max(hold[i], sell[i - 1] - price)

        return sell[k]
```

Round 9 · Low · Hard

p.3415

• LeetCode

```
class Solution:
    def maxProfit(self, k: int, prices: List[int]) -> int:
        @cache
        def dfs(i: int, j: int, k: int) -> int:
            if i >= len(prices):
                return 0
            ans = dfs(i + 1, j, k)
            if k:
                ans = max(ans, prices[i] + dfs(i + 1, j, 0))
            elif j:
                ans = max(ans, -prices[i] + dfs(i + 1, j - 1, 1))
            return ans

        return dfs(0, k, 0)
```

```
class Solution:
    def maxProfit(self, k: int, prices: List[int]) -> int:
        n = len(prices)
        f = [[0] * 2 for _ in range(k + 1)] for _ in range(n)
        for j in range(1, k + 1):
            f[0][j][1] = -prices[0]
        for i, x in enumerate(prices[1:], 1):
            for j in range(1, k + 1):
                f[i][j][0] = max(f[i - 1][j][1] + x, f[i - 1][j][0])
                f[i][j][1] = max(f[i - 1][j - 1][0] - x, f[i - 1][j][1])

        return f[n - 1][k][0]
```

```
class Solution:
    def maxProfit(self, k: int, prices: List[int]) -> int:
        f = [[0] * 2 for _ in range(k + 1)]
        for j in range(1, k + 1):
            f[j][1] = -prices[0]
        for x in prices[1:]:
            for j in range(k, 0, -1):
                f[j][0] = max(f[j][1] + x, f[j][0])
                f[j][1] = max(f[j - 1][0] - x, f[j][1])
        return f[k][0]
```

• SimplyLeet

Round 9 · Low · Hard

p.3416


```
# Python solution for Best Time to Buy and Sell Stock IV

def maxProfit(k, prices):
    # Edge case: if there are no prices, no profit can be made.
    n = len(prices)
    if n == 0 or k == 0:
        return 0

    # If k is large enough, perform as many transactions as possible.
    if k >= n // 2:
```

```
        total_profit = 0
        for i in range(1, n):
            # Add profit if today's price is higher than yesterday's.
            if prices[i] > prices[i - 1]:
                total_profit += prices[i] - prices[i - 1]
        return total_profit

    # Initialize DP table: dp[t][i] is the max profit for at most t
    transactions until day i.
    dp = [[0] * n for _ in range(k + 1)]
```

```
    # Fill the DP table for each transaction from 1 to k.
    for t in range(1, k + 1):
        maxDiff = -prices[0] # Initialize maxDiff as maximum value of
        dp[t-1][0] - prices[0].
        for i in range(1, n):
            # Either do not trade on day i or sell on day i using best buy
            option.
            dp[t][i] = max(dp[t][i - 1], prices[i] + maxDiff)
            # Update maxDiff to incorporate the profit from previous
            transaction.
            maxDiff = max(maxDiff, dp[t - 1][i] - prices[i])
    return dp[k][n - 1]
```

```
# Example usage:
print(maxProfit(2, [3,2,6,5,0,3])) # Expected output: 7
```

Round 9 · Low · Hard

p.3417

Maths / Geometry

Round 9 · Low · Hard · 1 question

Round 9 · Low · Hard

p.3418

Maths / Geometry

□ #149 Max Points on a Line

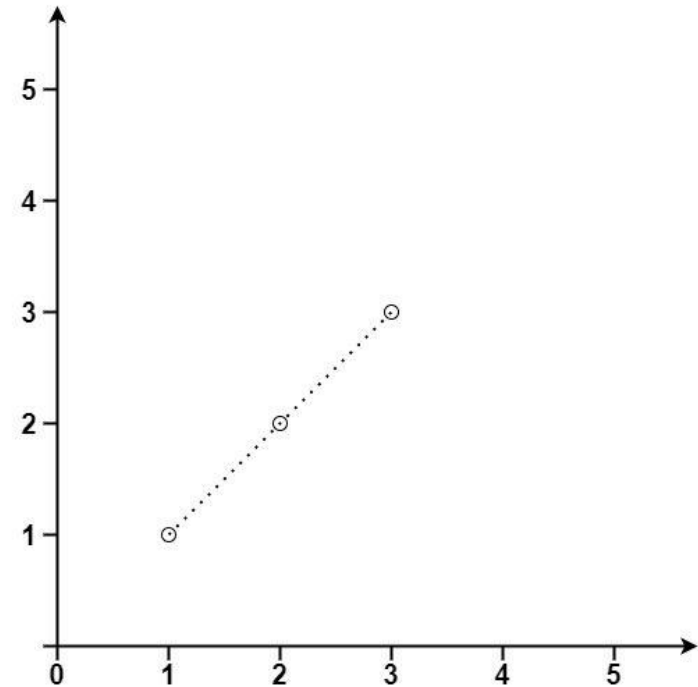
HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Math · Geometry
Lists: AlgoMaster 600

Problem

Given an array of points where $\text{points}[i] = [x_i, y_i]$ represents a point on the X-Y plane, return the maximum number of points that lie on the same straight line.

Example 1:



Input: points = [[1,1],[2,2],[3,3]]
Output: 3

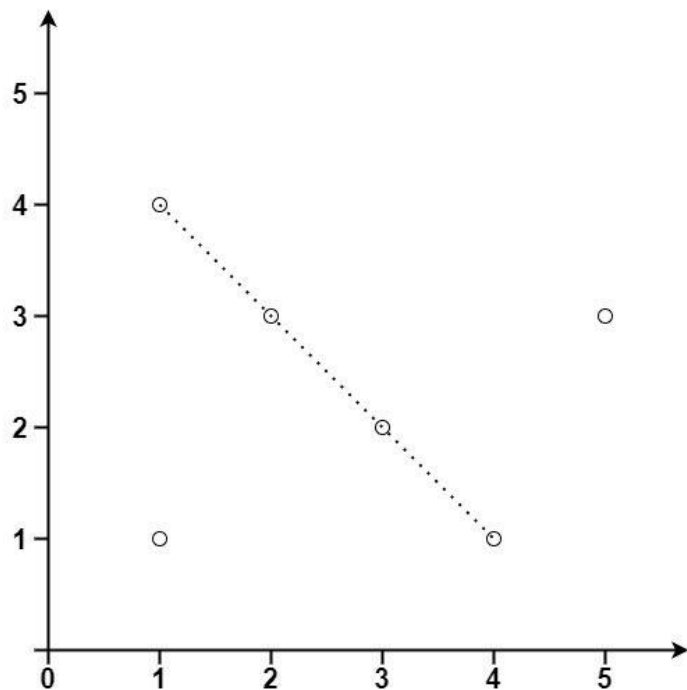
Example 2:

Round 9 · Low · Hard

p.3419

Round 9 · Low · Hard

p.3420



Input: points = [[1,1],[3,2],[5,3],[4,1],[2,3],[1,4]]
Output: 4

Constraints:

- $1 \leq \text{points.length} \leq 300$
- $\text{points}[i].\text{length} == 2$
- $-104 \leq x_i, y_i \leq 104$
- All the points are unique.

Community Solutions (Python)

- WalkCC

Round 9 · Low · Hard

p.3421

```
class Solution:
    def maxPoints(self, points: list[list[int]]) -> int:
        ans = 0

        def gcd(a: int, b: int) -> int:
            return a if b == 0 else gcd(b, a % b)

        def getSlope(p: list[int], q: list[int]) -> tuple[int, int]:
            dx = p[0] - q[0]
            dy = p[1] - q[1]

            if dx == 0:
                return (0, p[0])
            if dy == 0:
                return (p[1], 0)
            d = gcd(dx, dy)
            return (dx // d, dy // d)

        for i, p in enumerate(points):
            slopeCount = collections.defaultdict(int)
            samePoints = 1

            maxPoints = 0 # the maximum number of points with the same slope
            for j in range(i + 1, len(points)):
                q = points[j]
                if p == q:
                    samePoints += 1
                else:
                    slope = getSlope(p, q)
                    slopeCount[slope] += 1
                    maxPoints = max(maxPoints, slopeCount[slope])
            ans = max(ans, samePoints + maxPoints)

        return ans
```

- LeetDoocs

Round 9 · Low · Hard

p.3422

```
class Solution:
    def maxPoints(self, points: List[List[int]]) -> int:
        n = len(points)
        ans = 1
        for i in range(n):
            x1, y1 = points[i]
            for j in range(i + 1, n):
                x2, y2 = points[j]
                cnt = 2
                for k in range(j + 1, n):
                    x3, y3 = points[k]
                    a = (y2 - y1) * (x3 - x1)
                    b = (y3 - y1) * (x2 - x1)
                    cnt += a == b
                ans = max(ans, cnt)
        return ans
```

```
class Solution:
    def maxPoints(self, points: List[List[int]]) -> int:
        def gcd(a, b):
            return a if b == 0 else gcd(b, a % b)

        n = len(points)
        ans = 1
        for i in range(n):
            x1, y1 = points[i]
            cnt = Counter()

            for j in range(i + 1, n):
                x2, y2 = points[j]
                dx, dy = x2 - x1, y2 - y1
                g = gcd(dx, dy)
                k = (dx // g, dy // g)
                cnt[k] += 1
            ans = max(ans, cnt[k] + 1)
        return ans
```

• SimplyLeet

Round 9 · Low · Hard

p.3423

```
import math
from collections import defaultdict

class Solution:
    def maxPoints(self, points):
        # If there are less than 3 points, return the count directly
        if len(points) < 3:
            return len(points)

        max_points = 0

        # Iterate over each point treating it as an anchor
        for i in range(len(points)):
            slopes = defaultdict(int)
            same = 1 # Count the anchor point itself
            curr_max = 0

            # Compare with every other point after i
            for j in range(i + 1, len(points)):
                dx = points[j][0] - points[i][0]
                dy = points[j][1] - points[i][1]

                # Deal with the (theoretical) duplicate point case.
                if dx == 0 and dy == 0:
                    same += 1
                else:
                    # Handle vertical lines
                    if dx == 0:
                        slope = ('inf', 0)
                    else:
                        gcd = math.gcd(dx, dy)
                        dx_norm = dx // gcd
                        dy_norm = dy // gcd
                        # Normalize representation: ensure dx_norm is positive
                        if dx_norm < 0:
                            dx_norm *= -1
                            dy_norm *= -1
                        slope = (dy_norm, dx_norm)

                    slopes[slope] += 1
```

Round 9 · Low · Hard

p.3424

```
        curr_max = max(curr_max, slopes[slope])
    max_points = max(max_points, curr_max + same)
    return max_points
```

String Matching

Round 9 · Low · Hard · 3 questions

Round 9 · Low · Hard

p.3425

Round 9 · Low · Hard

p.3426

String Matching

#214 Shortest Palindrome

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Rolling Hash · String Matching · Hash
Function

Lists: AlgoMaster 600

Problem

You are given a string s . You can convert s to a palindrome by adding characters in front of it. Return the shortest palindrome you can find by performing this transformation.

Example 1:

Input: $s = \text{"aacecaaa"}$
Output: "aaacecaaa"

Example 2:

Input: $s = \text{"abcd"}$
Output: "dcbabcd"

Constraints:

- $0 \leq s.length \leq 5 * 10^4$
- s consists of lowercase English letters only.

Community Solutions (Python)

Round 9 · Low · Hard

p.3427

• WalkCC

```
class Solution:
    def shortestPalindrome(self, s: str) -> str:
        t = s[::-1]

        for i in range(len(t)):
            if s.startswith(t[i:]):
                return t[:i] + s

        return t + s
```

• LeetDoocs

```
class Solution:
    def shortestPalindrome(self, s: str) -> str:
        base = 131
        mod = 10**9 + 7
        n = len(s)
        prefix = suffix = 0
        mul = 1
        idx = 0
        for i, c in enumerate(s):
            prefix = (prefix * base + (ord(c) - ord('a') + 1)) % mod

            suffix = (suffix + (ord(c) - ord('a') + 1) * mul) % mod
            mul = (mul * base) % mod
            if prefix == suffix:
                idx = i + 1
        return s if idx == n else s[idx:][::-1] + s
```

```
typedef unsigned long long ull;
```

```
class Solution {
public:
    string shortestPalindrome(string s) {
        int base = 131;
        ull mul = 1;
        ull prefix = 0;
        ull suffix = 0;
        int idx = 0, n = s.size();
```

Round 9 · Low · Hard

p.3428

```

for (int i = 0; i < n; ++i) {
    int t = s[i] - 'a' + 1;
    prefix = prefix * base + t;
    suffix = suffix + mul * t;
    mul *= base;
    if (prefix == suffix) idx = i + 1;
}
if (idx == n) return s;
string x = s.substr(idx, n - idx);
reverse(x.begin(), x.end());

return x + s;
}
};

```

```

class Solution:
    def shortestPalindrome(self, s: str) -> str:
        t = s + "#" + s[::-1] + "$"
        n = len(t)
        next = [0] * n
        next[0] = -1
        i, j = 2, 0
        while i < n:
            if t[i - 1] == t[j]:
                j += 1
                next[i] = j
                i += 1
            elif j:
                j = next[j]
            else:
                next[i] = 0
                i += 1
        return s[::-1][:-next[-1]] + s

```

• SimplyLeet

Round 9 · Low · Hard

p.3429

```

# Python solution using KMP prefix function
def shortestPalindrome(s: str) -> str:
    if not s:
        return s
    # Generate combined string with a separator
    rev_s = s[::-1]
    combined = s + "#" + rev_s
    n = len(combined)

    # Build KMP table (lps) for combined string

    lps = [0] * n
    for i in range(1, n):
        j = lps[i - 1]
        while j > 0 and combined[i] != combined[j]:
            j = lps[j - 1]
        if combined[i] == combined[j]:
            j += 1
        lps[i] = j

    # lps[-1] gives length of longest prefix of s which is palindrome

    longest_pal_pref = lps[-1]
    add_on = rev_s[:len(s) - longest_pal_pref]
    return add_on + s

# Example usage:
print(shortestPalindrome("aacecaaa")) # Output: "aacecaaaa"
print(shortestPalindrome("abcd"))    # Output: "dcbabcd"

```

Round 9 · Low · Hard

p.3430

String Matching

#1044 Longest Duplicate Substring

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Binary Search · Sliding Window ·
Rolling Hash · Suffix Array · Hash Function
Lists: AlgoMaster 600

Problem

Given a string s , consider all duplicated substrings: (contiguous) substrings of s that occur 2 or more times. The occurrences may overlap.

Return any duplicated substring that has the longest possible length. If s does not have a duplicated substring, the answer is "".

Example 1:

Input: $s = \text{"banana"}$
Output: "ana"

Example 2:

Input: $s = \text{"abcd"}$
Output: ""

Constraints:

- $2 \leq s.length \leq 3 * 10^4$

Round 9 · Low · Hard

p.3431

- s consists of lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def longestDupSubstring(self, s: str) -> str:
        BASE = 26
        HASH = 1_000_000_007
        bestStart = -1
        l = 1
        r = len(s)

        def val(c: str) -> int:
            return ord(c) - ord('a')

        # k := the length of the substring to be hashed
        def getStart(k: int) -> int | None:
            maxPow = pow(BASE, k - 1, HASH)
            hashToStart = collections.defaultdict(list)
            h = 0

            # Compute the hash value of s[:k].
            for i in range(k):
                h = (h * BASE + val(s[i])) % HASH

            hashToStart[h].append(0)

            # Compute the rolling hash by Rabin Karp.
            for i in range(k, len(s)):
                startIndex = i - k + 1
                h = (h - maxPow * val(s[startIndex])) % HASH
                h = (h * BASE + val(s[i])) % HASH
                if h in hashToStart:
                    currSub = s[startIndex:startIndex + k]
                    for start in hashToStart[h]:
```

Round 9 · Low · Hard

p.3432


```

        if s[start:start + k] == currSub:
            return startIndex
        hashToStart[h].append(startIndex)

    while l < r:
        m = (l + r) // 2
        start: int | None = getStart(m)
        if start:
            bestStart = start
            l = m + 1

    else:
        r = m

    if bestStart == -1:
        return ''
    if getStart(l):
        return s[bestStart:bestStart + l]
    return s[bestStart:bestStart + l - 1]

```

• LeetCode

```

class Solution:
    def longestDupSubstring(self, s: str) -> str:
        def check(l):
            vis = set()
            for i in range(n - l + 1):
                t = s[i : i + l]
                if t in vis:
                    return t
                vis.add(t)
            return ''

        n = len(s)
        left, right = 0, n
        ans = ''
        while left < right:
            mid = (left + right + 1) >> 1
            t = check(mid)
            ans = t or ans
            if t:
                left = mid

```

Round 9 · Low · Hard

p.3433

```

        else:
            right = mid - 1
        return ans

typedef unsigned long long ULL;

class Solution {
public:
    ULL p[30010];
    ULL h[30010];
    string longestDupSubstring(string s) {
        int base = 131, n = s.size();
        p[0] = 1;
        for (int i = 0; i < n; ++i) {

            p[i + 1] = p[i] * base;
            h[i + 1] = h[i] * base + s[i];
        }
        int left = 0, right = n;
        string ans = "";
        while (left < right) {
            int mid = (left + right + 1) >> 1;
            string t = check(s, mid);
            if (t.empty())
                right = mid - 1;

            else {
                left = mid;
                ans = t;
            }
        }
        return ans;
    }

    string check(string& s, int len) {
        int n = s.size();

```

Round 9 · Low · Hard

p.3434

```
unordered_set<ULL> vis;
for (int i = 1; i + len - 1 <= n; ++i) {
    int j = i + len - 1;
    ULL t = h[j] - h[i - 1] * p[j - i + 1];
    if (vis.count(t)) return s.substr(i - 1, len);
    vis.insert(t);
}
return "";
};
```

• SimplyLeet

```
# Define the Solution class with the function longestDupSubstring
class Solution:
    def longestDupSubstring(self, s: str) -> str:
        n = len(s)
        # Convert characters to integers: 'a' -> 0, 'b' -> 1, ..., 'z' -> 25
        nums = [ord(c) - ord('a') for c in s]
        mod = 2**63 - 1 # a large modulus to minimize hash collisions
        base = 26 # base value for characters

        # search function returns the starting index of a duplicate substring
        # of length L if it exists, else -1
```

```
def search(L):
    h = 0
    # Compute the hash for the first substring of length L
    for i in range(L):
        h = (h * base + nums[i]) % mod
    seen = {h}
    # Precompute base^L % mod for use in rolling hash
    baseL = pow(base, L, mod)
    for i in range(1, n - L + 1):
        # Compute rolling hash in O(1) time
```

```
h = (h * base - nums[i - 1] * baseL + nums[i + L - 1]) % mod
if h in seen:
    return i # found duplicate, return starting index
seen.add(h)
return -1
```

```
left, right = 1, n
start = -1
# Binary search for the maximum length L that has a duplicate substring
while left <= right:
```

```
    mid = left + (right - left) // 2
    idx = search(mid)
    if idx != -1:
        left = mid + 1 # duplicate found, try longer substring
        start = idx
    else:
        right = mid - 1 # no duplicate found, decrease length
return s[start:start+left-1] if start != -1 else ""
```

String Matching

#1392 Longest Happy Prefix

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Rolling Hash · String Matching · Hash
Function

Lists: AlgoMaster 600

Problem

A string is called a happy prefix if is a non-empty prefix which is also a suffix (excluding itself).

Given a string s , return the longest happy prefix of s . Return an empty string "" if no such prefix exists.

Example 1:

Input: $s = \text{"level"}$
Output: "l"
Explanation: s contains 4 prefix excluding itself ("l", "le", "lev", "leve"), and suffix ("l", "el", "vel", "evel"). The largest prefix which is also suffix is given by "l".

Example 2:

Input: $s = \text{"ababab"}$
Output: "abab"
Explanation: "abab" is the largest prefix which is also suffix. They can overlap in the original string.

Constraints:

Round 9 · Low · Hard

p.3437

- $1 \leq s.length \leq 105$
- s contains only lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def longestPrefix(self, s: str) -> str:
        BASE = 26
        HASH = 8_417_508_174_513
        n = len(s)
        maxLength = 0
        pow = 1
        prefixHash = 0 # the hash of s[0..i]
        suffixHash = 0 # the hash of s[j..n)

        def val(c: str) -> int:
            return ord(c) - ord('a')

        j = n - 1
        for i in range(n - 1):
            prefixHash = (prefixHash * BASE + val(s[i])) % HASH
            suffixHash = (val(s[j]) * pow + suffixHash) % HASH
            pow = pow * BASE % HASH
            if prefixHash == suffixHash:
                maxLength = i + 1
                j -= 1

        return s[:maxLength]
```

• LeetDoocs

```
class Solution:
    def longestPrefix(self, s: str) -> str:
        for i in range(1, len(s)):
            if s[:-i] == s[i:]:
                return s[:i]
        return ''
```

Round 9 · Low · Hard

p.3438

```
typedef unsigned long long ULL;

class Solution {
public:
    string longestPrefix(string s) {
        int base = 131;
        int n = s.size();
        ULL p[n + 10];
        ULL h[n + 10];
        p[0] = 1;

        h[0] = 0;
        for (int i = 0; i < n; ++i) {
            p[i + 1] = p[i] * base;
            h[i + 1] = h[i] * base + s[i];
        }
        for (int l = n - 1; l > 0; --l) {
            ULL prefix = h[l];
            ULL suffix = h[n] - h[n - l] * p[l];
            if (prefix == suffix) return s.substr(0, l);
        }

        return "";
    }
};
```

```
class Solution:
    def longestPrefix(self, s: str) -> str:
        s += "#"
        n = len(s)
        next = [0] * n
        next[0] = -1
        i, j = 2, 0
        while i < n:
            if s[i - 1] == s[j]:
                j += 1
```

Round 9 · Low · Hard

p.3439

```
        next[i] = j
        i += 1
    elif j:
        j = next[j]
    else:
        next[i] = 0
        i += 1
    return s[: next[-1]]
```

• SimplyLeet

```
# Function to compute the longest happy prefix using KMP prefix function
def longestHappyPrefix(s: str) -> str:
    n = len(s)
    # pi[i] will hold the length of the longest prefix which is also suffix
    # ending at i
    pi = [0] * n
    # k: length of current longest prefix-suffix
    k = 0
    # iterate from second character to build the pi array
    for i in range(1, n):
        # if the current character doesn't match, fallback using pi array

        while k > 0 and s[i] != s[k]:
            k = pi[k - 1]
        # if there is a match, extend the prefix-suffix length
        if s[i] == s[k]:
            k += 1
            pi[i] = k
        else:
            pi[i] = 0
    # pi[n-1] gives the length of the longest happy prefix
    return s[:pi[-1]]
```

```
# Example usage:
print(longestHappyPrefix("level")) # Output: "l"
print(longestHappyPrefix("ababab")) # Output: "abab"
```

Round 9 · Low · Hard

p.3440

Binary Indexed Tree / Segment Tree

Round 9 · Low · Hard · 5 questions

Round 9 · Low · Hard

p.3441

Binary Indexed Tree / Segment Tree

□ #315 Count of Smaller Numbers After Self

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Binary Search · Divide and Conquer ·
 Binary Indexed Tree · Segment Tree · Merge
 Sort · Ordered Set

Lists: Denny Zhang

Problem

Given an integer array `nums`, return an integer array `counts` where `counts[i]` is the number of smaller elements to the right of `nums[i]`.

Example 1:

```
Input: nums = [5,2,6,1]
Output: [2,1,1,0]
Explanation:
To the right of 5 there are 2 smaller elements (2 and 1).
To the right of 2 there is only 1 smaller element (1).
To the right of 6 there is 1 smaller element (1).
To the right of 1 there is 0 smaller element.
```

Example 2:

```
Input: nums = [-1]
Output: [0]
```

Example 3:

Round 9 · Low · Hard

p.3442

Input: nums = [-1,-1]
Output: [0,0]

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $-104 \leq \text{nums}[i] \leq 104$

Community Solutions (Python)

• WalkCC

```
class FenwickTree:
    def __init__(self, n: int):
        self.sums = [0] * (n + 1)

    def add(self, i: int, delta: int) -> None:
        while i < len(self.sums):
            self.sums[i] += delta
            i += FenwickTree.lowbit(i)

    def get(self, i: int) -> int:
        summ = 0
        while i > 0:
            summ += self.sums[i]
            i -= FenwickTree.lowbit(i)
        return summ

    @staticmethod
    def lowbit(i: int) -> int:
        return i & -i
```

Round 9 · Low · Hard

p.3443

```
class Solution:
    def countSmaller(self, nums: list[int]) -> list[int]:
        ans = []
        ranks = self._getRanks(nums)
        tree = FenwickTree(len(ranks))

        for num in reversed(nums):
            ans.append(tree.get(ranks[num] - 1))
            tree.add(ranks[num], 1)

        return ans[::-1]

    def _getRanks(self, nums: list[int]) -> dict[int, int]:
        ranks = collections.Counter()
        rank = 0
        for num in sorted(set(nums)):
            rank += 1
            ranks[num] = rank
        return ranks
```

```
from dataclasses import dataclass
```

```
@dataclass
class Item:
    num: int = 0
    index: int = 0
```

```
class Solution:

    def countSmaller(self, nums: list[int]) -> list[int]:
        n = len(nums)
        ans = [0] * n
        items = [Item(num, i) for i, num in enumerate(nums)]

        self._mergeSort(items, 0, n - 1, ans)
        return ans

    def _mergeSort(
        self,
```

Round 9 · Low · Hard

p.3444

```

        items: list[Item],
        l: int,
        r: int,
        ans: list[int],
    ) -> None:
        if l >= r:
            return

        m = (l + r) // 2
        self._mergeSort(items, l, m, ans)

        self._mergeSort(items, m + 1, r, ans)
        self._merge(items, l, m, r, ans)

    def _merge(
        self,
        items: list[Item],
        l: int,
        m: int,
        r: int,
        ans: list[int],
    ) -> None:
        sorted = [Item()] * (r - l + 1)
        k = 0 # sorted's index
        i = l # left's index
        j = m + 1 # right's index
        rightCount = 0 # the number of numbers < items[i].num

        while i <= m and j <= r:
            if items[i].num > items[j].num:
                rightCount += 1

            sorted[k] = items[j]
            k += 1
            j += 1
        else:
            ans[items[i].index] += rightCount
            sorted[k] = items[i]
            k += 1
            i += 1

        # Put the possible remaining left part into the sorted array.

```

Round 9 · Low · Hard

p.3445

```

        while i <= m:
            ans[items[i].index] += rightCount
            sorted[k] = items[i]
            k += 1
            i += 1

        # Put the possible remaining right part into the sorted array.
        while j <= r:
            sorted[k] = items[j]
            k += 1
            j += 1

        items[l:l + len(sorted)] = sorted

```

• LeetCode

```

class BinaryIndexedTree:
    def __init__(self, n):
        self.n = n
        self.c = [0] * (n + 1)

    @staticmethod
    def lowbit(x):
        return x & -x

    def update(self, x, delta):

        while x <= self.n:
            self.c[x] += delta
            x += BinaryIndexedTree.lowbit(x)

    def query(self, x):
        s = 0
        while x > 0:
            s += self.c[x]
            x -= BinaryIndexedTree.lowbit(x)
        return s

```

Round 9 · Low · Hard

p.3446

```
class Solution:
    def countSmaller(self, nums: List[int]) -> List[int]:
        alls = sorted(set(nums))
        m = {v: i for i, v in enumerate(alls, 1)}
        tree = BinaryIndexedTree(len(m))
        ans = []
        for v in nums[::-1]:
            x = m[v]

            tree.update(x, 1)
            ans.append(tree.query(x - 1))
        return ans[::-1]
```

```
class Node:
    def __init__(self):
        self.l = 0
        self.r = 0
        self.v = 0
```

```
class SegmentTree:
    def __init__(self, n):
        self.tr = [Node() for _ in range(n << 2)]
```

```
        self.build(1, 1, n)
```

```
    def build(self, u, l, r):
        self.tr[u].l = l
        self.tr[u].r = r
        if l == r:
            return
        mid = (l + r) >> 1
        self.build(u << 1, l, mid)
        self.build(u << 1 | 1, mid + 1, r)
```

```
    def modify(self, u, x, v):
        if self.tr[u].l == x and self.tr[u].r == x:
            self.tr[u].v += v
            return
        mid = (self.tr[u].l + self.tr[u].r) >> 1
        if x <= mid:
            self.modify(u << 1, x, v)
        else:
            self.modify(u << 1 | 1, x, v)
```

```
        self.pushup(u)
```

```
    def query(self, u, l, r):
        if self.tr[u].l >= l and self.tr[u].r <= r:
            return self.tr[u].v
        mid = (self.tr[u].l + self.tr[u].r) >> 1
        v = 0
        if l <= mid:
            v += self.query(u << 1, l, r)
        if r > mid:
```

```
            v += self.query(u << 1 | 1, l, r)
        return v
```

```
    def pushup(self, u):
        self.tr[u].v = self.tr[u << 1].v + self.tr[u << 1 | 1].v
```

```
class Solution:
    def countSmaller(self, nums: List[int]) -> List[int]:
        s = sorted(set(nums))
```

```
        m = {v: i for i, v in enumerate(s, 1)}
        tree = SegmentTree(len(s))
        ans = []
        for v in nums[::-1]:
            x = m[v]
            ans.append(tree.query(1, 1, x - 1))
            tree.modify(1, x, 1)
        return ans[::-1]
```



```

class Solution:
    def countSmaller(self, nums: list[int]) -> list[int]:
        self.right_smaller_counts = [0] * len(nums)

        nums_indices = [(num, idx) for idx, num in enumerate(nums)]
        self.merge_sort(nums_indices)

        return self.right_smaller_counts

    def combine_arrays(
        self,
        left_nums_indices: list[tuple[int, int]],
        right_nums_indices: list[tuple[int, int]],
    ) -> list[tuple[int, int]]:
        merged_nums_indices: list[tuple[int, int]] = []
        left_idx, right_idx = 0, 0

        while left_idx < len(left_nums_indices) and right_idx < len(right_nums
_indices):
            if left_nums_indices[left_idx][0] <= right_nums_indices[right_idx]
[0]:
                # Iterated left side element finalizes its right smaller count.

                left_num_idx = left_nums_indices[left_idx][1]
                self.right_smaller_counts[left_num_idx] += right_idx

                merged_nums_indices.append(left_nums_indices[left_idx])
                left_idx += 1
                continue

            merged_nums_indices.append(right_nums_indices[right_idx])
            right_idx += 1

```

Round 9 · Low · Hard

p.3449

```

        while left_idx < len(left_nums_indices):
            # Iterated left side element finalizes its right smaller count.
            left_num_idx = left_nums_indices[left_idx][1]
            self.right_smaller_counts[left_num_idx] += len(right_nums_indices)

            merged_nums_indices.append(left_nums_indices[left_idx])
            left_idx += 1

        while right_idx < len(right_nums_indices):
            merged_nums_indices.append(right_nums_indices[right_idx])

            right_idx += 1

        return merged_nums_indices

    def merge_sort(self, nums_indices: list[tuple[int, int]]) -> list[tuple[in
t, int]]:
        if len(nums_indices) == 1:
            return nums_indices # Single element.

        split_idx = len(nums_indices) // 2

        left_nums_indices = self.merge_sort(nums_indices[:split_idx])
        right_nums_indices = self.merge_sort(nums_indices[split_idx:])

        return self.combine_arrays(left_nums_indices, right_nums_indices)

```

• SimplyLeet

```

# Python solution using merge sort approach

def countSmaller(nums):
    n = len(nums)
    # Initialize counts array for results
    counts = [0] * n
    # Pair each element with its original index
    enum = [(num, i) for i, num in enumerate(nums)]

    # Recursive function for merge sort with counting

```

Round 9 · Low · Hard

p.3450

```
def mergeSort(arr):
    if len(arr) <= 1:
        return arr
    # Split the array into two halves
    mid = len(arr) // 2
    left = mergeSort(arr[:mid])
    right = mergeSort(arr[mid:])
    merged = []
    l = r = 0
    # r_count counts number of elements taken from right array

    r_count = 0
    while l < len(left) and r < len(right):
        # If left element is less or equal, it means current right elements
        # are smaller
        if left[l][0] <= right[r][0]:
            # update count for original index
            counts[left[l][1]] += r_count
            merged.append(left[l])
            l += 1
        else:
            # right element is smaller, so increment r_count and add it to
            # merged
            merged.append(right[r])
            r += 1
            r_count += 1
        # Append remaining elements from left reduce count by accumulated right
        # elements
        while l < len(left):
            counts[left[l][1]] += r_count
            merged.append(left[l])
            l += 1
        # Append any remaining elements from right side
        while r < len(right):
```

Round 9 · Low · Hard

p.3451

```
        merged.append(right[r])
        r += 1
    return merged

# Start the merge sort process
mergeSort(enum)
return counts

# Example test
print(countSmaller([5, 2, 6, 1])) # Expected output: [2, 1, 1, 0]
```

• LC.ca

```
'''
bisect: maintaining a list in sorted order without having to sort the list
after each insertion.
https://docs.python.org/3/library/bisect.html

bisect.bisect_left()
Locate the insertion point for x in a to maintain sorted order.

bisect.bisect_right() or bisect.bisect()
Similar to bisect_left(), but returns an insertion point which comes after (to
the right of) any existing entries of x in a.

bisect.insort_left(a, x, lo=0, hi=len(a), *, key=None)
Insert x in a in sorted order.
Keep in mind that the O(log n) search is dominated by the slow O(n) insertion
step.

bisect.insort_right(a, x, lo=0, hi=len(a), *, key=None)
bisect.insort(a, x, lo=0, hi=len(a), *, key=None)
Similar to insort_left(), but inserting x in a after any existing entries of x.
```

Round 9 · Low · Hard

p.3452

```
>>> import bisect
>>> bisect.bisect_left([1,2,3], 2)
1
>>> bisect.bisect_right([1,2,3], 2)
2

>>> a = [1, 1, 1, 2, 3]
>>> bisect.insort_left(a, 1.0)
>>> a
```

```
[1.0, 1, 1, 1, 2, 3]

>>> a = [1, 1, 1, 2, 3]
>>> bisect.insort_right(a, 1.0)
>>> a
[1, 1, 1, 1.0, 2, 3]

>>> a = [1, 1, 1, 2, 3]
>>> bisect.insort(a, 1.0)
>>> a
```

```
[1, 1, 1, 1.0, 2, 3]
'''
```

```
import bisect
```

```
class Solution(object):
    def countSmaller(self, nums):
        """
        :type nums: List[int]
        :rtype: List[int]
```

```
        """
        ans = []
        bst = []
        for num in reversed(nums):
            idx = bisect.bisect_left(bst, num)
            ans.append(idx)
            bisect.insort(bst, num)
        return ans[::-1]
```

```
#####
```

Round 9 · Low · Hard

p.3453

```
class BinaryIndexedTree:
    def __init__(self, n):
        self.n = n
        self.c = [0] * (n + 1)

    @staticmethod
    def lowbit(x):
        return x & -x
```

```
    def update(self, x, delta):
        while x <= self.n:
            self.c[x] += delta
            x += BinaryIndexedTree.lowbit(x)
```

```
    def query(self, x):
        s = 0
        while x > 0:
            s += self.c[x]
            x -= BinaryIndexedTree.lowbit(x)
```

```
        return s
```

```
class Solution:
    def countSmaller(self, nums: List[int]) -> List[int]:
        alls = sorted(set(nums))
        m = {v: i for i, v in enumerate(alls, 1)}
        tree = BinaryIndexedTree(len(m))
        ans = []
        for v in nums[::-1]:
```

```
            x = m[v]
            tree.update(x, 1)
            ans.append(tree.query(x - 1))
        return ans[::-1]
```

◆ Quick Review

Key Insights

Round 9 · Low · Hard

p.3454

- The brute force solution takes $O(n^2)$ time, which is too slow for large input arrays.
- An efficient approach leverages the concept of Merge Sort to both sort the array and count the number of smaller elements by tracking the number of elements that "jump over" during the merge step.
- Other efficient methods include Binary Indexed Trees (Fenwick Tree) or Balanced Binary Search Trees.

Space & Time

Time Complexity: $O(n \log n)$ on average due to the merge sort based approach.

Space Complexity: $O(n)$ for auxiliary arrays used during sorting and counting.

Solution

We use a modified Merge Sort to sort the array while counting the number of smaller elements to the right. We keep track of the original indices by pairing each element with its index. During the merge process, when an element from the left subarray is placed into the temporary array, we add the count of elements from the right subarray that have

already been placed (since these represent smaller elements that were to its right). This efficient divide and conquer strategy guarantees an overall $O(n \log n)$ time complexity. Care must be taken to update the counts based on indices and during the merge step.

Binary Indexed Tree / Segment Tree

□ #699 Falling Squares

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Segment Tree · Ordered Set
 Lists: AlgoMaster 600

Problem

There are several squares being dropped onto the X-axis of a 2D plane.

You are given a 2D integer array `positions` where `positions[i] = [lefti, sideLengthi]` represents the *i*th square with a side length of `sideLengthi` that is dropped with its left edge aligned with X-coordinate `lefti`.

Each square is dropped one at a time from a height above any landed squares. It then falls downward (negative Y direction) until it either lands on the top side of another square or on the X-axis. A square brushing the left/right side of another square does not count as landing on it. Once it lands, it freezes in place and cannot be moved.

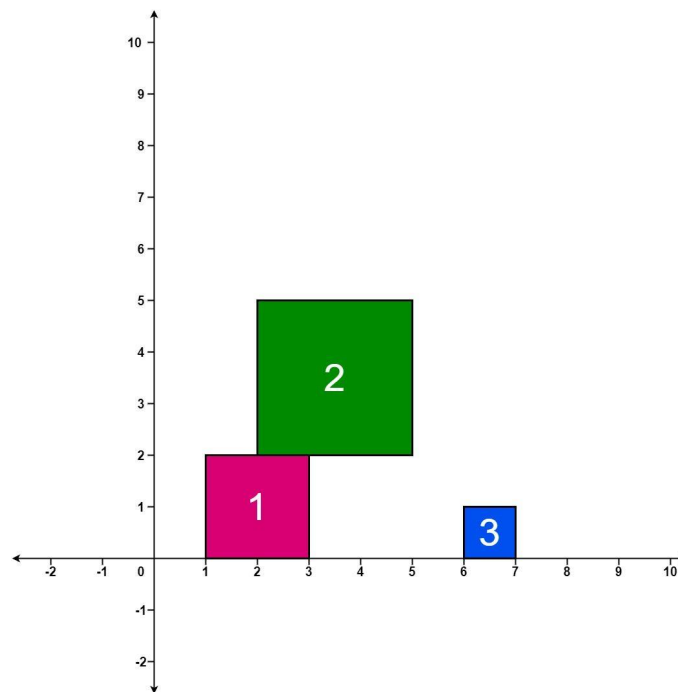
Round 9 · Low · Hard

p.3457

After each square is dropped, you must record the height of the current tallest stack of squares.

Return an integer array `ans` where `ans[i]` represents the height described above after dropping the *i*th square.

Example 1:



Round 9 · Low · Hard

p.3458

Input: positions = [[1,2],[2,3],[6,1]]
 Output: [2,5,5]
 Explanation:
 After the first drop, the tallest stack is square 1 with a height of 2.
 After the second drop, the tallest stack is squares 1 and 2 with a height of 5.
 After the third drop, the tallest stack is still squares 1 and 2 with a height of 5.
 Thus, we return an answer of [2, 5, 5].

Example 2:

Input: positions = [[100,100],[200,100]]
 Output: [100,100]
 Explanation:
 After the first drop, the tallest stack is square 1 with a height of 100.
 After the second drop, the tallest stack is either square 1 or square 2, both with heights of 100.
 Thus, we return an answer of [100, 100].
 Note that square 2 only brushes the right side of square 1, which does not count as landing on it.

Constraints:

- $1 \leq \text{positions.length} \leq 1000$
- $1 \leq \text{lefti} \leq 108$
- $1 \leq \text{sideLengthi} \leq 106$

Community Solutions (Python)

• LeetCode

```
class Node:
    def __init__(self, l, r):
        self.left = None
        self.right = None
        self.l = l
        self.r = r
        self.mid = (l + r) >> 1
        self.v = 0
        self.add = 0
```

```
class SegmentTree:
    def __init__(self):
        self.root = Node(1, int(1e9))

    def modify(self, l, r, v, node=None):
        if l > r:
            return
        if node is None:
            node = self.root

        if node.l >= l and node.r <= r:
            node.v = v
            node.add = v
            return
        self.pushdown(node)
        if l <= node.mid:
            self.modify(l, r, v, node.left)
        if r > node.mid:
            self.modify(l, r, v, node.right)
        self.pushup(node)

    def query(self, l, r, node=None):
        if l > r:
            return 0
        if node is None:
            node = self.root
        if node.l >= l and node.r <= r:
            return node.v
        self.pushdown(node)
        v = 0

        if l <= node.mid:
            v = max(v, self.query(l, r, node.left))
        if r > node.mid:
            v = max(v, self.query(l, r, node.right))
        return v

    def pushup(self, node):
        node.v = max(node.left.v, node.right.v)

    def pushdown(self, node):
```

```

if node.left is None:
    node.left = Node(node.l, node.mid)
if node.right is None:
    node.right = Node(node.mid + 1, node.r)
if node.add:
    node.left.v = node.add
    node.right.v = node.add
    node.left.add = node.add
    node.right.add = node.add
    node.add = 0

```

```

class Solution:
    def fallingSquares(self, positions: List[List[int]]) -> List[int]:
        ans = []
        mx = 0
        tree = SegmentTree()
        for l, w in positions:
            r = l + w - 1
            h = tree.query(l, r) + w

```

```

        mx = max(mx, h)
        ans.append(mx)
        tree.modify(l, r, h)
    return ans

```

• SimplyLeet

```

# Python implementation for Falling Squares
def fallingSquares(positions):
    # List to record the results after each drop
    ans = []
    # List to keep each square's (left, right, height)
    squares = []
    # The current global maximum height so far
    current_max = 0

    # Iterate through each position in the input list

```

```

for left, side in positions:
    new_left, new_right = left, left + side # calculate boundaries for
the new square
    base_height = 0 # start with height 0 for the landing base

    # Check all previous squares for overlapping intervals
    for prev_left, prev_right, prev_height in squares:
        # If the horizontal intervals overlap (not just touching)
        if not (new_right <= prev_left or new_left >= prev_right):
            # Update base_height to the maximum height this new square can
land on
            base_height = max(base_height, prev_height)

```

```

    # The height of the current square is the base height plus its side
length
    cur_square_height = base_height + side
    # Add the current square info to the squares list
    squares.append((new_left, new_right, cur_square_height))
    # Update the overall maximum height
    current_max = max(current_max, cur_square_height)
    # Append the current maximum height to the results list
    ans.append(current_max)

```

```

    # Return the results after dropping each square

```

```

    return ans

```

```

# Example usage:
#print(fallingSquares([[1,2],[2,3],[6,1]])) # Expected output: [2, 5, 5]

```

Binary Indexed Tree / Segment Tree

#2426 Number of Pairs Satisfying Inequality

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Binary Search · Divide and Conquer ·
 Binary Indexed Tree · Segment Tree · Merge
 Sort · Ordered Set

Lists: AlgoMaster 600

Problem

You are given two 0-indexed integer arrays `nums1` and `nums2`, each of size `n`, and an integer `diff`. Find the number of pairs (i, j) such that:

- $0 \leq i < j \leq n - 1$ and
- $\text{nums1}[i] - \text{nums1}[j] \leq \text{nums2}[i] - \text{nums2}[j] + \text{diff}$.

Return the number of pairs that satisfy the conditions.

Example 1:

Round 9 · Low · Hard

p.3463

Input: `nums1 = [3,2,5]`, `nums2 = [2,2,1]`, `diff = 1`

Output: 3

Explanation:

There are 3 pairs that satisfy the conditions:

1. $i = 0, j = 1$: $3 - 2 \leq 2 - 2 + 1$. Since $i < j$ and $1 \leq 1$, this pair satisfies the conditions.
 2. $i = 0, j = 2$: $3 - 5 \leq 2 - 1 + 1$. Since $i < j$ and $-2 \leq 2$, this pair satisfies the conditions.
 3. $i = 1, j = 2$: $2 - 5 \leq 2 - 1 + 1$. Since $i < j$ and $-3 \leq 2$, this pair satisfies the conditions.
- Therefore, we return 3.

Example 2:

Input: `nums1 = [3,-1]`, `nums2 = [-2,2]`, `diff = -1`

Output: 0

Explanation:

Since there does not exist any pair that satisfies the conditions, we return 0.

Constraints:

- $n == \text{nums1.length} == \text{nums2.length}$
- $2 \leq n \leq 105$
- $-104 \leq \text{nums1}[i], \text{nums2}[i] \leq 104$
- $-104 \leq \text{diff} \leq 104$

Community Solutions (Python)

• LeetDoocs

```
class BinaryIndexedTree:
    def __init__(self, n):
        self.n = n
        self.c = [0] * (n + 1)

    @staticmethod
    def lowbit(x):
        return x & -x

    def update(self, x, delta):
```

Round 9 · Low · Hard

p.3464


```

while x <= self.n:
    self.c[x] += delta
    x += BinaryIndexedTree.lowbit(x)

```

```

def query(self, x):
    s = 0
    while x:
        s += self.c[x]
        x -= BinaryIndexedTree.lowbit(x)
    return s

```

```

class Solution:
    def numberOfPairs(self, nums1: List[int], nums2: List[int], diff: int) ->
    int:
        tree = BinaryIndexedTree(10**5)
        ans = 0
        for a, b in zip(nums1, nums2):
            v = a - b
            ans += tree.query(v + diff + 40000)
            tree.update(v + 40000, 1)

```

```

return ans

```

• SimplyLeet

```

# Python solution using a Binary Indexed Tree (BIT)

class BIT:
    # Initialize BIT with given size
    def __init__(self, size):
        self.size = size
        self.tree = [0] * (size + 1)

    # Update BIT at index i (1-indexed) with value delta
    def update(self, i, delta):

```

```

while i <= self.size:
    self.tree[i] += delta
    i += i & -i

# Query BIT: get the prefix sum for index i (1-indexed)
def query(self, i):
    result = 0
    while i > 0:
        result += self.tree[i]
        i -= i & -i

```

```

return result

```

```

def numberOfPairs(nums1, nums2, diff):
    n = len(nums1)
    # Compute the difference array where each element is nums1[i] - nums2[i]
    diff_arr = [nums1[i] - nums2[i] for i in range(n)]

```

```

    # Prepare a sorted list of all values that we will query/update:
    # Include each diff_arr[j] and diff_arr[j] + diff for proper
    discretization.
    all_values = []

```

```

for val in diff_arr:
    all_values.append(val)
    all_values.append(val + diff)
sorted_values = sorted(set(all_values))

```

```

# Map each value to its discretized index (1-indexed for BIT)
value_to_index = {value: i+1 for i, value in enumerate(sorted_values)}

# Initialize BIT based on the number of unique values
bit = BIT(len(sorted_values))

```

```

result = 0

# Iterate over diff_arr elements
for val in diff_arr:
    # Query BIT: count number of previous indices where stored value <=
    current val + diff
    index_query = value_to_index[val + diff]
    result += bit.query(index_query)
    # Add current diff_arr value into BIT for later queries
    index_update = value_to_index[val]
    bit.update(index_update, 1)

return result

# Example usage:
nums1 = [3, 2, 5]
nums2 = [2, 2, 1]
diff = 1
print(numberOfPairs(nums1, nums2, diff)) # Output should be 3

```

Round 9 · Low · Hard

p.3467

Binary Indexed Tree / Segment Tree

□ #3161 Block Placement Queries

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Binary Search · Binary Indexed Tree ·
 Segment Tree · Ordered Set
 Lists: AlgoMaster 600

Problem

There exists an infinite number line, with its origin at 0 and extending towards the positive x-axis.

You are given a 2D array queries, which contains two types of queries:

1. For a query of type 1, queries[i] = [1, x]. Build an obstacle at distance x from the origin. It is guaranteed that there is no obstacle at distance x when the query is asked.
2. For a query of type 2, queries[i] = [2, x, sz]. Check if it is possible to place a block of size sz anywhere in the range [0, x] on the line, such that the block entirely lies in the range [0, x]. A block cannot be placed if it intersects with any obstacle, but it may touch it. Note

Round 9 · Low · Hard

p.3468

that you do not actually place the block.

Queries are separate.

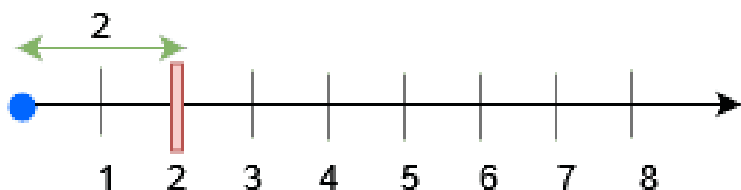
Return a boolean array `results`, where `results[i]` is true if you can place the block specified in the *i*th query of type 2, and false otherwise.

Example 1:

Input: `queries = [[1,2],[2,3,3],[2,3,1],[2,2,2]]`

Output: `[false,true,true]`

Explanation:



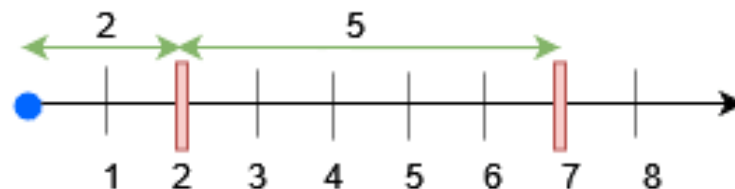
For query 0, place an obstacle at $x = 2$. A block of size at most 2 can be placed before $x = 3$.

Example 2:

Input: `queries = [[1,7],[2,7,6],[1,2],[2,7,5],[2,7,6]]`

Output: `[true,true,false]`

Explanation:



- Place an obstacle at $x = 7$ for query 0. A block of size at most 7 can be placed before $x = 7$.
- Place an obstacle at $x = 2$ for query 2. Now, a block of size at most 5 can be placed before $x = 7$, and a block of size at most 2 before $x = 2$.

Constraints:

- $1 \leq \text{queries.length} \leq 15 \cdot 10^4$
- $2 \leq \text{queries}[i].\text{length} \leq 3$
- $1 \leq \text{queries}[i][0] \leq 2$
- $1 \leq x, sz \leq \min(5 \cdot 10^4, 3 \cdot \text{queries.length})$
- The input is generated such that for queries of type 1, no obstacle exists at distance x when the query is asked.

- The input is generated such that there is at least one query of type 2.

Community Solutions (Python)

• WalkCC

```
from sortedcontainers import SortedList

class FenwickTree:
    def __init__(self, n: int):
        self.vals = [0] * (n + 1)

    def maximize(self, i: int, val: int) -> None:
        while i < len(self.vals):
            self.vals[i] = max(self.vals[i], val)
            i += FenwickTree.lowtree(i)

    def get(self, i: int) -> int:
        res = 0
        while i > 0:
            res = max(res, self.vals[i])
            i -= FenwickTree.lowtree(i)
        return res

    @staticmethod
    def lowtree(i: int) -> int:
        return i & -i

class Solution:
    def getResults(self, queries: list[list[int]]) -> list[bool]:
        n = min(50000, len(queries) * 3)
        ans = []
        tree = FenwickTree(n + 1)
        obstacles = SortedList([0, n]) # sentinel values
```

```
for query in queries:
    type = query[0]
    if type == 1:
        x = query[1]
        obstacles.add(x)

for x1, x2 in itertools.pairwise(obstacles):
    tree.maximize(x2, x2 - x1)
```

```
for query in reversed(queries):
    type = query[0]
    x = query[1]
    if type == 1:
        i = obstacles.index(x)
        next = obstacles[i + 1]
        prev = obstacles[i - 1]
        obstacles.remove(x)
        tree.maximize(next, next - prev)
    else:
        sz = query[2]
        i = obstacles.bisect_right(x)
        prev = obstacles[i - 1]
        ans.append(tree.get(prev) >= sz or x - prev >= sz)

return ans[::-1]
```

• SimplyLeet

```
# Python solution using Segment Tree

# A segment tree node which holds the computed free lengths.
class SegmentTreeNode:
    def __init__(self, start, end):
        self.start = start # left index of the segment
        self.end = end # right index of the segment
        self.total = end - start + 1 # total number of positions in the segment
        self.prefix = self.total # contiguous free length from the start
        self.suffix = self.total # contiguous free length up to the end
```

```

        self.best = self.total          # maximum contiguous free length in
this segment
        self.left = None                # left child
        self.right = None               # right child

# Segment Tree class that supports update and query.
class SegmentTree:
    def __init__(self, start, end):
        self.root = self.build(start, end)

    def build(self, start, end):

        node = SegmentTreeNode(start, end)
        if start == end:
            return node
        mid = (start + end) // 2
        node.left = self.build(start, mid)
        node.right = self.build(mid + 1, end)
        return node

    def update(self, node, idx, value):
        # Update the node corresponding to idx with value (1 for free, 0 for
        blocked)

        if node.start == node.end:
            node.prefix = value
            node.suffix = value
            node.best = value
            return
        if idx <= node.left.end:
            self.update(node.left, idx, value)
        else:
            self.update(node.right, idx, value)
        # Merge left and right children

```

Round 9 · Low · Hard

p.3473

```

        node.prefix = node.left.prefix
        if node.left.prefix == node.left.total:
            node.prefix = node.left.total + node.right.prefix
        node.suffix = node.right.suffix
        if node.right.suffix == node.right.total:
            node.suffix = node.right.total + node.left.suffix
        node.best = max(node.left.best, node.right.best, node.left.suffix + no
de.right.prefix)

    def query(self, node, l, r):
        # Return a tuple: (prefix, suffix, best, total) for the query interval
        [l, r]

        if node.start > r or node.end < l:
            return (0, 0, 0, 0)
        if l <= node.start and node.end <= r:
            return (node.prefix, node.suffix, node.best, node.total)
        left_res = self.query(node.left, l, r)
        right_res = self.query(node.right, l, r)
        total = left_res[3] + right_res[3]
        prefix = left_res[0]
        if left_res[0] == left_res[3]:
            prefix = left_res[3] + right_res[0]

        suffix = right_res[1]
        if right_res[1] == right_res[3]:
            suffix = right_res[3] + left_res[1]
        best = max(left_res[2], right_res[2], left_res[1] + right_res[0])
        return (prefix, suffix, best, total)

# Class to handle block placement queries.
class BlockPlacement:
    def __init__(self, max_x):
        self.max_x = max_x

```

Round 9 · Low · Hard

p.3474

```

self.segment_tree = SegmentTree(0, max_x)

def build_obstacle(self, x):
    # When an obstacle is built, mark the position as blocked (0)
    self.segment_tree.update(self.segment_tree.root, x, 0)

def can_place_block(self, x, sz):
    # Query the segment tree for range [0, x] and check if best free
    segment >= sz.
    res = self.segment_tree.query(self.segment_tree.root, 0, x)
    return res[2] >= sz

def blockPlacementQueries(queries):
    # Find maximum x in queries to initialize the segment tree.
    M = 0
    for q in queries:
        if q[0] == 1 or q[0] == 2:
            M = max(M, q[1])
    bp = BlockPlacement(M)
    results = []
    for q in queries:
        if q[0] == 1:
            bp.build_obstacle(q[1])
        else:
            results.append(bp.can_place_block(q[1], q[2]))
    return results

# Example:
# print(blockPlacementQueries([[1,2],[2,3,3],[2,3,1],[2,2,2]]))

```

Round 9 · Low · Hard

p.3475

Binary Indexed Tree / Segment Tree

□ #3721 Longest Balanced Subarray II

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table · Divide and Conquer ·
 Segment Tree · Prefix Sum
 Lists: AlgoMaster 600

Problem

You are given an integer array `nums`.

A subarray is called balanced if the number of distinct even numbers in the subarray is equal to the number of distinct odd numbers.

Return the length of the longest balanced subarray.

Example 1:

Input: `nums = [2,5,4,3]`

Output: 4

Explanation:

- The longest balanced subarray is [2, 5, 4, 3].
- It has 2 distinct even numbers [2, 4] and 2 distinct odd numbers [5, 3]. Thus, the answer is 4.

Round 9 · Low · Hard

p.3476

Example 2:**Input:** nums = [3,2,2,5,4]**Output:** 5**Explanation:**

- The longest balanced subarray is [3, 2, 2, 5, 4].
- It has 2 distinct even numbers [2, 4] and 2 distinct odd numbers [3, 5]. Thus, the answer is 5.

Example 3:**Input:** nums = [1,2,3,2]**Output:** 3**Explanation:**

- The longest balanced subarray is [2, 3, 2].
- It has 1 distinct even number [2] and 1 distinct odd number [3]. Thus, the answer is 3.

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $1 \leq \text{nums}[i] \leq 105$

Community Solutions (Python)

- LeetDoocs

Round 9 · Low · Hard

p.3477

```

class Solution:
    def longestBalanced(self, nums: List[int]) -> int:
        n = len(nums)

        class Node:
            __slots__ = ("l", "r", "mn", "mx", "lazy")
            def __init__(self):
                self.l = self.r = 0
                self.mn = self.mx = 0
                self.lazy = 0

        tr = [Node() for _ in range((n + 1) * 4)]

        def build(u: int, l: int, r: int):
            tr[u].l, tr[u].r = l, r
            tr[u].mn = tr[u].mx = tr[u].lazy = 0
            if l == r:
                return
            mid = (l + r) >> 1
            build(u << 1, l, mid)

            build(u << 1 | 1, mid + 1, r)

        def apply(u: int, v: int):
            tr[u].mn += v
            tr[u].mx += v
            tr[u].lazy += v

        def pushdown(u: int):
            if tr[u].lazy:
                apply(u << 1, tr[u].lazy)

            apply(u << 1 | 1, tr[u].lazy)
            tr[u].lazy = 0

        def pushup(u: int):
            tr[u].mn = min(tr[u << 1].mn, tr[u << 1 | 1].mn)
            tr[u].mx = max(tr[u << 1].mx, tr[u << 1 | 1].mx)

        def modify(u: int, l: int, r: int, v: int):
            if tr[u].l >= l and tr[u].r <= r:
                apply(u, v)

```

Round 9 · Low · Hard

p.3478

```

        return
    pushdown(u)
    mid = (tr[u].l + tr[u].r) >> 1
    if l <= mid:
        modify(u << 1, l, r, v)
    if r > mid:
        modify(u << 1 | 1, l, r, v)
    pushup(u)

def query(u: int, target: int) -> int:

    if tr[u].l == tr[u].r:
        return tr[u].l
    pushdown(u)
    if tr[u << 1].mn <= target <= tr[u << 1].mx:
        return query(u << 1, target)
    return query(u << 1 | 1, target)

build(1, 0, n)

last = {}

now = ans = 0

for i, x in enumerate(nums, start=1):
    det = 1 if (x & 1) else -1
    if x in last:
        modify(1, last[x], n, -det)
        now -= det
    last[x] = i
    modify(1, i, n, det)
    now += det

pos = query(1, now)
ans = max(ans, i - pos)

return ans

```

Round 9 · Low · Hard

p.3479

Line Sweep

Round 9 · Low · Hard · 5 questions

Round 9 · Low · Hard

p.3480

Line Sweep

□ #218 The Skyline Problem

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode Array · Divide and Conquer · Binary Indexed Tree · Segment Tree · Line Sweep · Heap · Ordered Set

Lists: Denny Zhang

Problem

A city's skyline is the outer contour of the silhouette formed by all the buildings in that city when viewed from a distance. Given the locations and heights of all the buildings, return the skyline formed by these buildings collectively.

The geometric information of each building is given in the array `buildings` where `buildings[i] = [lefti, righti, heighti]`:

- `lefti` is the x coordinate of the left edge of the *i*th building.
- `righti` is the x coordinate of the right edge of the *i*th building.

Round 9 · Low · Hard

p.3481

- `heighti` is the height of the *i*th building.

You may assume all buildings are perfect rectangles grounded on an absolutely flat surface at height 0.

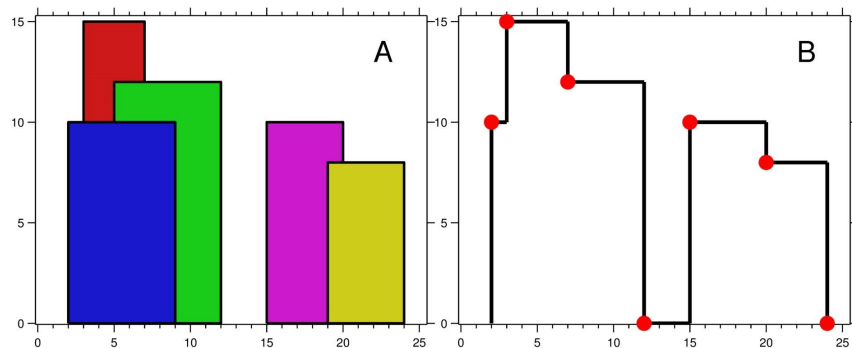
The skyline should be represented as a list of "key points" sorted by their x-coordinate in the form `[[x1,y1],[x2,y2],...]`. Each key point is the left endpoint of some horizontal segment in the skyline except the last point in the list, which always has a y-coordinate 0 and is used to mark the skyline's termination where the rightmost building ends. Any ground between the leftmost and rightmost buildings should be part of the skyline's contour.

Note: There must be no consecutive horizontal lines of equal height in the output skyline. For instance, `[[...],[2 3],[4 5],[7 5],[11 5],[12 7],...]` is not acceptable; the three lines of height 5 should be merged into one in the final output as such: `[[...],[2 3],[4 5],[12 7],...]`

Example 1:

Round 9 · Low · Hard

p.3482



Input: buildings = [[2,9,10],[3,7,15],[5,12,12],[15,20,10],[19,24,8]]

Output: [[2,10],[3,15],[7,12],[12,0],[15,10],[20,8],[24,0]]

Explanation:

Figure A shows the buildings of the input.

Figure B shows the skyline formed by those buildings. The red points in figure B represent the key points in the output list.

Example 2:

Input: buildings = [[0,2,3],[2,5,3]]

Output: [[0,3],[5,0]]

Constraints:

- $1 \leq \text{buildings.length} \leq 104$
- $0 \leq \text{left}_i < \text{right}_i \leq 231 - 1$
- $1 \leq \text{height}_i \leq 231 - 1$
- buildings is sorted by lefti in non-decreasing order.

Community Solutions (Python)

- WalkCC

```
class Solution:
    def getSkyline(self, buildings: list[list[int]]) -> list[list[int]]:
        n = len(buildings)
        if n == 0:
            return []
        if n == 1:
            left, right, height = buildings[0]
            return [[left, height], [right, 0]]

        left = self.getSkyline(buildings[:n // 2])

        right = self.getSkyline(buildings[n // 2:])
        return self._merge(left, right)

    def _merge(self, left: list[list[int]],
               right: list[list[int]]) -> list[list[int]]:
        ans = []
        i = 0 # left's index
        j = 0 # right's index
        leftY = 0
        rightY = 0

        while i < len(left) and j < len(right):
            # Choose the point with smaller x
            if left[i][0] < right[j][0]:
                leftY = left[i][1] # Update the ongoing `leftY`.
                self._addPoint(ans, left[i][0], max(left[i][1], rightY))
                i += 1
            else:
                rightY = right[j][1] # Update the ongoing `rightY`.
                self._addPoint(ans, right[j][0], max(right[j][1], leftY))

            j += 1

        while i < len(left):
            self._addPoint(ans, left[i][0], left[i][1])
            i += 1

        while j < len(right):
            self._addPoint(ans, right[j][0], right[j][1])
            j += 1
```

```

return ans

def _addPoint(self, ans: list[list[int]], x: int, y: int) -> None:
    if ans and ans[-1][0] == x:
        ans[-1][1] = y
        return
    if ans and ans[-1][1] == y:
        return
    ans.append([x, y])

```

• SimplyLeet

```

import heapq

def getSkyline(buildings):
    # List to store all events: (x, height, right)
    events = []
    # Create events for all buildings: left edge (start) and right edge (end)
    for left, right, height in buildings:
        # For the start point, height is negative for proper sorting (start
        # events come before end events)
        events.append((left, -height, right))
        # For the end point, height is 0 to indicate removal of building from
        # active heap
        events.append((right, 0, 0))

    # Sort events by x coordinate.
    events.sort()

    # Max heap to store active buildings (using negative heights)
    result = []
    # Initialize the heap with a dummy building of height 0 that extends
    # infinitely
    heap = [(0, float('inf'))]

```

```

for x, neg_height, right in events:
    # Remove buildings from the heap that have ended by current x.
    while heap and heap[0][1] <= x:
        heapq.heappop(heap)

    # If event is a start event (neg_height != 0), push it into the heap.
    if neg_height != 0:
        heapq.heappush(heap, (neg_height, right))

    # Current highest building height is at the top of the heap

    current_height = -heap[0][0]
    # If the result is empty or current height differs from last recorded
    # height, add a new key point.
    if not result or current_height != result[-1][1]:
        result.append([x, current_height])

    return result

# Example usage:
buildings = [[2,9,10],[3,7,15],[5,12,12],[15,20,10],[19,24,8]]
print(getSkyline(buildings)) # Expected output:
[[2,10],[3,15],[7,12],[12,0],[15,10],[20,8],[24,0]]

```

• LC.ca

```

...
>>> a = []
>>> a.append((3, 5))
>>> a.append((3, -5))
>>> a.append((2, -10))
>>> a.append((2, 10))
>>> a
[(3, 5), (3, -5), (2, -10), (2, 10)]

>>> a.sort()

```


After processing each point, the maximum height is obtained from the heap, and if it is different from the previous maximum height, the current point is added to the skyline.

Finally, the skyline is returned, excluding the initial point (0, 0) that was added as a starting point.

Note: The solution assumes that the input buildings is a list of tuples (left, right, height), where left and right represent the x-coordinates of the building's left and right edges, and height represents the height of the building.

```
import heapq
```

```
class Solution:
    def getSkyline(self, buildings):
        # Create a list to store the critical points and heights
        points = []
        for left, right, height in buildings:
            points.append((left, -height)) # Start of building, negative height
            points.append((right, height)) # End of building, positive height

        # Sort the points in ascending order based on x-coordinate
        # If two points have the same x-coordinate, the one with larger height comes first
        points.sort()

        # Create a heap to store the heights in descending order
        heights = [0] # Initialize the heap with a height of 0
        skyline = [(0, 0)] # Initialize the skyline with a point (0, 0)

        for x, h in points:
            if h < 0:
                heapq.heappush(heights, h) # Add the negative height to the heap
            else:
                heights.remove(-h) # Remove the corresponding negative height from the heap
                heapq.heapify(heights) # Reorganize the heap

            # The current maximum height is the first element in the heap
            max_height = -heights[0]

            # If the maximum height has changed, add the current point to the skyline
            if max_height != skyline[-1][1]:
                skyline.append((x, max_height))

        return skyline[1:] # Exclude the initial point (0, 0)
```

Round 9 · Low · Hard

p.3489

```
                heights.remove(-h) # Remove the corresponding negative height from the heap
                heapq.heapify(heights) # Reorganize the heap

            # The current maximum height is the first element in the heap
            max_height = -heights[0]

            # If the maximum height has changed, add the current point to the skyline
            if max_height != skyline[-1][1]:
                skyline.append((x, max_height))

        return skyline[1:] # Exclude the initial point (0, 0)
```

◆ Quick Review

Key Insights

- Use a line sweep approach to process building start and end events in sorted order.
- Differentiate between start events (which add to the skyline) and end events (which remove from the skyline).
- Employ a max heap (or a priority queue) to efficiently track the current highest building at each sweep line position.
- When the current maximum height changes, record the corresponding x-coordinate and new height as a key point.
- Take care to remove expired building heights from the heap as the sweep line

Round 9 · Low · Hard

p.3490

moves past the building's end.

Space & Time

Time Complexity: $O(n \log n)$ due to sorting events and heap operations for each building event.

Space Complexity: $O(n)$ for the event list and the heap used for processing active buildings.

Solution

The solution uses the line sweep algorithm with a max heap (or priority queue) to maintain the heights of active buildings. Each building contributes two events: one when the building starts (adding its height) and one when it ends (removing its height). These events are sorted primarily by their x -coordinate. For start events, we use negative height values to ensure they are processed before end events at the same x -coordinate. During the sweep, the heap is updated by adding new building heights and removing buildings that have ended. Whenever there is a change in the top of the heap (i.e., the current maximum height), a key point is added to the result. This key point signifies a vertical

change in the skyline.

Line Sweep

□ #391 Perfect Rectangle

HAR

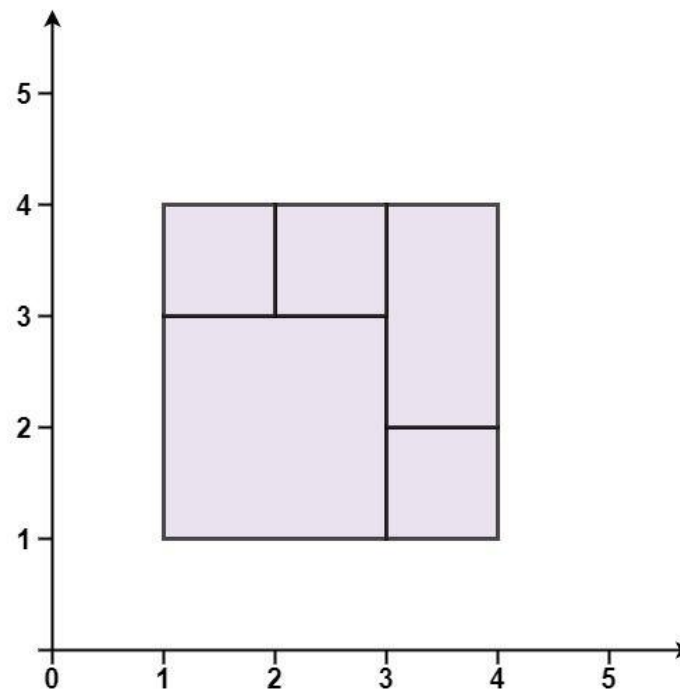
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Math · Geometry · Sweep Line

Lists: AlgoMaster 600

Problem

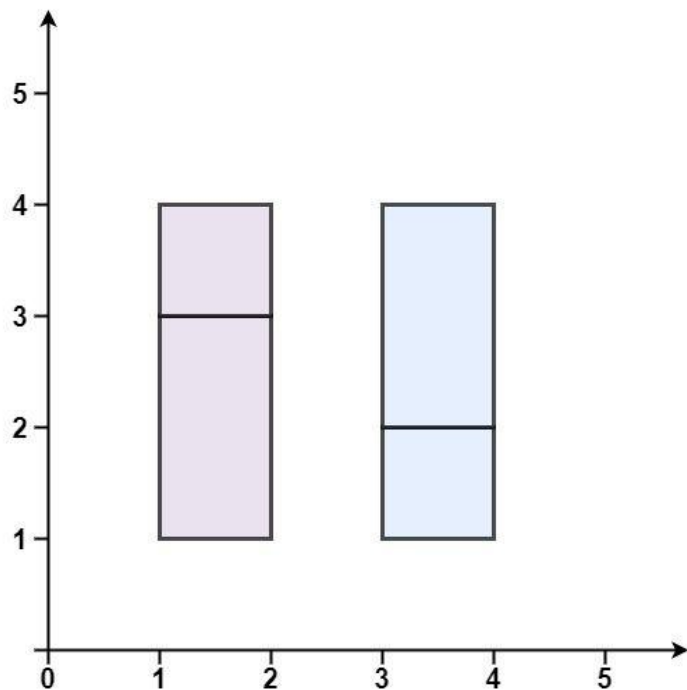
Given an array rectangles where rectangles[i] = [xi, yi, ai, bi] represents an axis-aligned rectangle. The bottom-left point of the rectangle is (xi, yi) and the top-right point of it is (ai, bi). Return true if all the rectangles together form an exact cover of a rectangular region.

Example 1:



Input: rectangles = [[1,1,3,3],[3,1,4,2],[3,2,4,4],[1,3,2,4],[2,3,3,4]]
Output: true
Explanation: All 5 rectangles together form an exact cover of a rectangular region.

Example 2:

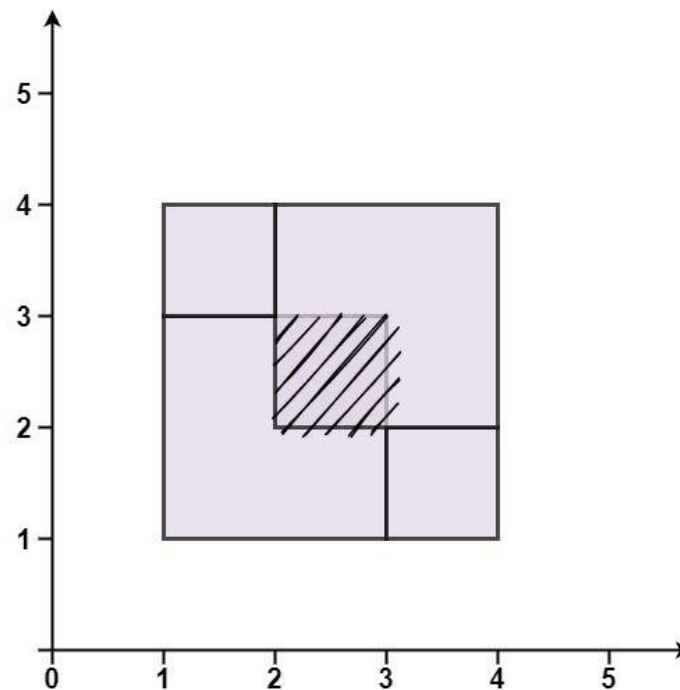


Input: rectangles = [[1,1,2,3],[1,3,2,4],[3,1,4,2],[3,2,4,4]]
 Output: false
 Explanation: Because there is a gap between the two rectangular regions.

Example 3:

Round 9 · Low · Hard

p.3495



Input: rectangles = [[1,1,3,3],[3,1,4,2],[1,3,2,4],[2,2,4,4]]
 Output: false
 Explanation: Because two of the rectangles overlap with each other.

Constraints:

- $1 \leq \text{rectangles.length} \leq 2 * 10^4$
- $\text{rectangles}[i].\text{length} == 4$
- $-105 \leq x_i < a_i \leq 105$
- $-105 \leq y_i < b_i \leq 105$

Community Solutions (Python)

Round 9 · Low · Hard

p.3496

• WalkCC

```
class Solution:
    def isRectangleCover(self, rectangles: list[list[int]]) -> bool:
        area = 0
        x1 = math.inf
        y1 = math.inf
        x2 = -math.inf
        y2 = -math.inf
        corners: set[tuple[int, int]] = set()

        for x, y, a, b in rectangles:

            area += (a - x) * (b - y)
            x1 = min(x1, x)
            y1 = min(y1, y)
            x2 = max(x2, a)
            y2 = max(y2, b)

            # the four points of the current rectangle
            for point in [(x, y), (x, b), (a, y), (a, b)]:
                if point in corners:
                    corners.remove(point)

            else:
                corners.add(point)

        if len(corners) != 4:
            return False
        if ((x1, y1) not in corners or
            (x1, y2) not in corners or
            (x2, y1) not in corners or
            (x2, y2) not in corners):
            return False

        return area == (x2 - x1) * (y2 - y1)
```

• LeetCode

Round 9 · Low · Hard

p.3497

```
class Solution:
    def isRectangleCover(self, rectangles: List[List[int]]) -> bool:
        area = 0
        minX, minY = rectangles[0][0], rectangles[0][1]
        maxX, maxY = rectangles[0][2], rectangles[0][3]
        cnt = defaultdict(int)

        for r in rectangles:
            area += (r[2] - r[0]) * (r[3] - r[1])

            minX = min(minX, r[0])
            minY = min(minY, r[1])
            maxX = max(maxX, r[2])
            maxY = max(maxY, r[3])

            cnt[(r[0], r[1])] += 1
            cnt[(r[0], r[3])] += 1
            cnt[(r[2], r[3])] += 1
            cnt[(r[2], r[1])] += 1

        if (
            area != (maxX - minX) * (maxY - minY)
            or cnt[(minX, minY)] != 1
            or cnt[(minX, maxY)] != 1
            or cnt[(maxX, maxY)] != 1
            or cnt[(maxX, minY)] != 1
        ):
            return False

        del cnt[(minX, minY)], cnt[(minX, maxY)], cnt[(maxX, maxY)], cnt[(maxX,
, minY)]

        return all(c == 2 or c == 4 for c in cnt.values())
```

• SimplyLeet

Round 9 · Low · Hard

p.3498

```
def isRectangleCover(rectangles):
    # Initialize bounding rectangle coordinates
    min_x = float('inf')
    min_y = float('inf')
    max_x = float('-inf')
    max_y = float('-inf')
    total_area = 0
    # Set to track all corners
    corners = set()

    for rect in rectangles:
        x, y, a, b = rect[0], rect[1], rect[2], rect[3]
        # Update overall bounding box
        min_x = min(min_x, x)
        min_y = min(min_y, y)
        max_x = max(max_x, a)
        max_y = max(max_y, b)
        # Calculate area of the current rectangle and add to total area
        total_area += (a - x) * (b - y)

        # List of current rectangle's corners
        current_corners = [(x, y), (x, b), (a, y), (a, b)]
        # Add or remove corners from the set
        for point in current_corners:
            if point in corners:
                corners.remove(point)
            else:
                corners.add(point)

    # Check if total area matches the area of the overall bounding rectangle
```

Round 9 · Low · Hard

p.3499

```
expected_area = (max_x - min_x) * (max_y - min_y)
if total_area != expected_area:
    return False

# The set must contain exactly the four corners of the overall rectangle
expected_corners = {(min_x, min_y), (min_x, max_y), (max_x, min_y), (max_x
, max_y)}
return corners == expected_corners

# Example usage:
rectangles = [[1,1,3,3],[3,1,4,2],[3,2,4,4],[1,3,2,4],[2,3,3,4]]

print(isRectangleCover(rectangles)) # Output: True
```

Round 9 · Low · Hard

p.3500

Line Sweep

☐ #759 Employee Free Time

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode

Array · Sorting · Heap

Lists: Grind 169

Problem

We are given a list `schedule` of employees, which represents the working time for each employee. Each employee has a list of non-overlapping Intervals, and these intervals are in sorted order. Return the list of finite intervals representing common, positive-length free time for all employees, also in sorted order.

(Even though we are representing Intervals in the form `[x, y]`, the objects inside are `Intervals`, not lists or arrays. For example, `schedule[0][0].start = 1`, `schedule[0][0].end = 2`, and `schedule[0][0][0]` is not defined). Also, we wouldn't include intervals like `[5, 5]` in our answer, as they have zero length.

Example 1:

Round 9 · Low · Hard

p.3501

Input: `schedule = [[[1,2],[5,6]],[[1,3]],[[4,10]]]`

Output: `[[3,4]]`

Explanation: There are a total of three employees, and all common free time intervals would be `[-inf, 1]`, `[3, 4]`, `[10, inf]`. We discard any intervals that contain `inf` as they aren't finite.

Example 2:

Input: `schedule = [[[1,3],[6,7]],[[2,4]],[[2,5],[9,12]]]`

Output: `[[5,6],[7,9]]`

Constraints:

- `1 <= schedule.length` , `schedule[i].length <= 50`
- `0 <= schedule[i].start < schedule[i].end <= 10^8`

Community Solutions (Python)

☐ • WalkCC

```
class Solution:
    def employeeFreeTime(self, schedule: '[[Interval]]') -> '[Interval]':
        ans = []
        intervals = []

        for s in schedule:
            intervals.extend(s)

        intervals.sort(key=lambda x: x.start)
```

Round 9 · Low · Hard

p.3502

```

prevEnd = intervals[0].end

for interval in intervals:
    if interval.start > prevEnd:
        ans.append(Interval(prevEnd, interval.start))
        prevEnd = max(prevEnd, interval.end)

return ans

```

• LeetCode

```

"""
# Definition for an Interval.
class Interval:
    def __init__(self, start: int = None, end: int = None):
        self.start = start
        self.end = end
"""

class Solution:

```

```

    def employeeFreeTime(self, schedule: "[[Interval]]") -> "[Interval]":
        intervals = []
        for e in schedule:
            intervals.extend(e)
        intervals.sort(key=lambda x: (x.start, x.end))
        merged = [intervals[0]]
        for x in intervals[1:]:
            if merged[-1].end < x.start:
                merged.append(x)
            else:

```

```

                merged[-1].end = max(merged[-1].end, x.end)
        ans = []
        for a, b in pairwise(merged):
            ans.append(Interval(a.end, b.start))
        return ans

```

• SimplyLeet

Round 9 · Low · Hard

p.3503

```

# Class definition for Interval
class Interval:
    def __init__(self, start, end):
        self.start = start # Start time of the interval
        self.end = end # End time of the interval

def employeeFreeTime(schedule):
    # Flatten the schedule: extract all intervals from each employee
    allIntervals = []
    for employee in schedule:
        for interval in employee:
            allIntervals.append(interval)

    # Sort intervals by start time
    allIntervals.sort(key=lambda interval: interval.start)

    # Merge overlapping intervals
    merged = []
    prev = allIntervals[0]
    for interval in allIntervals[1:]:
        if interval.start <= prev.end:
            # Expand the previous interval if there is an overlap
            prev.end = max(prev.end, interval.end)
        else:
            merged.append(prev)
            prev = interval
    merged.append(prev)

    # Collect free times from the gaps between merged intervals
    freeTimes = []

    for i in range(1, len(merged)):
        # Only intervals with positive length are added
        if merged[i].start > merged[i-1].end:
            freeTimes.append(Interval(merged[i-1].end, merged[i].start))

    return freeTimes

# Example usage:
# schedule = [
#     [Interval(1,2), Interval(5,6)],

```

Round 9 · Low · Hard

p.3504

```
# [Interval(1,3)],
# [Interval(4,10)]
# ]
# freeTimes = employeeFreeTime(schedule)
# for interval in freeTimes:
# print(f"[{interval.start}, {interval.end}]")
```

• LC.ca

```
"""
# Definition for an Interval.
class Interval:
    def __init__(self, start: int = None, end: int = None):
        self.start = start
        self.end = end
"""

class Solution:

    def employeeFreeTime(self, schedule: "[[Interval]]") -> "[Interval]":
        intervals = []
        for e in schedule:
            intervals.extend(e)
        intervals.sort(key=lambda x: (x.start, x.end))
        merged = [intervals[0]]
        for x in intervals[1:]:
            if merged[-1].end < x.start:
                merged.append(x)
            else:
                merged[-1].end = max(merged[-1].end, x.end)

        ans = []
        for a, b in pairwise(merged):
            ans.append(Interval(a.end, b.start))
        return ans
```

◆ Quick Review

Key Insights

Round 9 · Low · Hard

p.3505

- Flatten the schedule to combine all employees' intervals.
- Sort the flattened intervals by start time.
- Merge overlapping intervals to form a consolidated view of busy times.
- Identify gaps between merged intervals as the common free time.
- Ensure free intervals have strictly positive length.

Space & Time

Time Complexity: $O(N \log N)$, where N is the total number of intervals (due to sorting).

Space Complexity: $O(N)$, used for the flattened list and merged intervals.

Solution

The solution leverages interval merging. First, all intervals from every employee are collected and sorted by start time. By iterating through the sorted intervals, overlapping intervals are merged into one consolidated busy period. Finally, the gaps between consecutive merged intervals are identified as free periods available to all employees. This approach efficiently handles the intervals with a simple

Round 9 · Low · Hard

p.3506

sort and a linear scan.

Round 9 · Low · Hard

p.3507

Line Sweep

□ #850 Rectangle Area II

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Segment Tree · Sweep Line · Ordered
Set

Lists: AlgoMaster 600

Problem

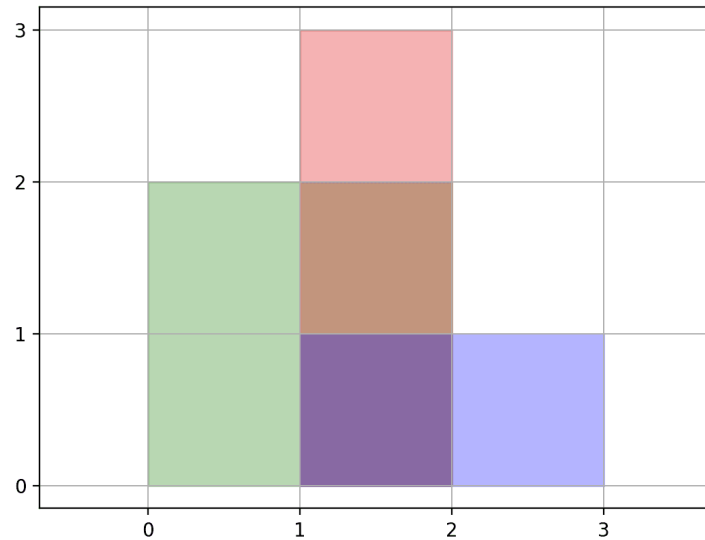
You are given a 2D array of axis-aligned rectangles. Each $\text{rectangle}[i] = [x_{i1}, y_{i1}, x_{i2}, y_{i2}]$ denotes the i th rectangle where (x_{i1}, y_{i1}) are the coordinates of the bottom-left corner, and (x_{i2}, y_{i2}) are the coordinates of the top-right corner. Calculate the total area covered by all rectangles in the plane. Any area covered by two or more rectangles should only be counted once.

Return the total area. Since the answer may be too large, return it modulo $10^9 + 7$.

Example 1:

Round 9 · Low · Hard

p.3508



Input: rectangles = [[0,0,2,2],[1,0,2,3],[1,0,3,1]]

Output: 6

Explanation: A total area of 6 is covered by all three rectangles, as illustrated in the picture.

From (1,1) to (2,2), the green and red rectangles overlap.

From (1,0) to (2,3), all three rectangles overlap.

Example 2:

Input: rectangles = [[0,0,1000000000,1000000000]]

Output: 49

Explanation: The answer is 1018 modulo (109 + 7), which is 49.

Constraints:

- $1 \leq \text{rectangles.length} \leq 200$
- $\text{rectangles}[i].\text{length} == 4$

- $0 \leq x_{i1}, y_{i1}, x_{i2}, y_{i2} \leq 109$
- $x_{i1} \leq x_{i2}$
- $y_{i1} \leq y_{i2}$
- All rectangles have non zero area.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def rectangleArea(self, rectangles: list[list[int]]) -> int:
        events = []

        for x1, y1, x2, y2 in rectangles:
            events.append((x1, y1, y2, 's'))
            events.append((x2, y1, y2, 'e'))

        events.sort(key=lambda x: x[0])

        ans = 0
        prevX = 0
        yPairs = []

        def getHeight(yPairs: list[tuple[int, int]]) -> int:
            height = 0
            prevY = 0

            for y1, y2 in yPairs:
                prevY = max(prevY, y1)

            return height
```

```

        if y2 > prevY:
            height += y2 - prevY
            prevY = y2

    return height

for currX, y1, y2, type in events:
    if currX > prevX:
        width = currX - prevX
        ans += width * getHeight(yPairs)

    prevX = currX
    if type == 's':
        yPairs.append((y1, y2))
        yPairs.sort()
    else: # type == 'e'
        yPairs.remove((y1, y2))

return ans % (10**9 + 7)

```

• LeetCode

```

class Node:
    def __init__(self):
        self.l = self.r = 0
        self.cnt = self.length = 0

class SegmentTree:
    def __init__(self, nums):
        n = len(nums) - 1
        self.nums = nums

        self.tr = [Node() for _ in range(n << 2)]
        self.build(1, 0, n - 1)

    def build(self, u, l, r):
        self.tr[u].l, self.tr[u].r = l, r
        if l == r:
            mid = (l + r) >> 1
            self.build(u << 1, l, mid)
            self.build(u << 1 | 1, mid + 1, r)

```

Round 9 · Low · Hard

p.3511

```

def modify(self, u, l, r, k):
    if self.tr[u].l >= l and self.tr[u].r <= r:
        self.tr[u].cnt += k
    else:
        mid = (self.tr[u].l + self.tr[u].r) >> 1
        if l <= mid:
            self.modify(u << 1, l, r, k)
        if r > mid:
            self.modify(u << 1 | 1, l, r, k)
        self.pushup(u)

def pushup(self, u):
    if self.tr[u].cnt:
        self.tr[u].length = self.nums[self.tr[u].r + 1] - self.nums[self.t
r[u].l]
    elif self.tr[u].l == self.tr[u].r:
        self.tr[u].length = 0
    else:
        self.tr[u].length = self.tr[u << 1].length + self.tr[u << 1 | 1].l
        ength

@property

def length(self):
    return self.tr[1].length

class Solution:
    def rectangleArea(self, rectangles: List[List[int]]) -> int:
        segs = []
        alls = set()
        for x1, y1, x2, y2 in rectangles:
            segs.append((x1, y1, y2, 1))

```

Round 9 · Low · Hard

p.3512


```

        segs.append((x2, y1, y2, -1))
        alls.update([y1, y2])

    segs.sort()
    alls = sorted(alls)
    tree = SegmentTree(alls)
    m = {v: i for i, v in enumerate(alls)}
    ans = 0
    for i, (x, y1, y2, k) in enumerate(segs):
        if i:
            ans += tree.length * (x - segs[i - 1][0])
            tree.modify(1, m[y1], m[y2] - 1, k)
    ans %= int(1e9 + 7)
    return ans

```

• SimplyLeet

Python solution using segment tree with coordinate compression

MOD = 10**9 + 7

Helper function to build and update the segment tree

```

class SegmentTree:
    def __init__(self, ys):
        self.ys = ys # sorted unique y values
        self.n = len(ys) - 1 # intervals count
        # count: how many times a segment is covered

        # length: the total covered length in that segment
        self.count = [0] * (4 * self.n)
        self.length = [0] * (4 * self.n)

    def update(self, idx, left, right, ql, qr, val):
        if ql >= right or qr <= left: # no overlap
            return
        if ql <= left and right <= qr: # total overlap
            self.count[idx] += val
        else:

```

```

        mid = (left + right) // 2
        self.update(idx * 2, left, mid, ql, qr, val)
        self.update(idx * 2 + 1, mid, right, ql, qr, val)

        if self.count[idx] > 0:
            # If count > 0, entire interval is covered
            self.length[idx] = self.ys[right] - self.ys[left]
        else:
            if right - left == 1:
                self.length[idx] = 0
            else:
                self.length[idx] = self.length[idx * 2] + self.length[idx * 2
+ 1]

    def rectangleArea(self, rectangles):
        events = []
        ys = set()
        # Prepare events and collect y values for compression
        for x1, y1, x2, y2 in rectangles:
            events.append((x1, y1, y2, 1)) # entering event
            events.append((x2, y1, y2, -1)) # leaving event

        ys.add(y1)
        ys.add(y2)

        # Sort unique y coordinates and events by x-coordinate
        ys = sorted(ys)
        events.sort()

        st = SegmentTree(ys)
        prev_x = events[0][0]
        area = 0

```

```

for x, y1, y2, typ in events:
    dx = x - prev_x
    # st.length[1] holds the total covered length in the y direction
    area = (area + dx * st.length[1]) % MOD
    # Find indices for y1 and y2 in the compressed list
    left = ys.index(y1)
    right = ys.index(y2)
    st.update(1, 0, st.n, left, right, typ)
    prev_x = x
return area

```

```

# Example usage:
# print(rectangleArea([[0,0,2,2],[1,0,2,3],[1,0,3,1]]))

```

Round 9 · Low · Hard

p.3515

Line Sweep

#1851 Minimum Interval to Include Each Query HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Binary Search · Sweep Line · Sorting ·
 Heap (Priority Queue)

Lists: NeetCode 150

Problem

You are given a 2D integer array `intervals`, where `intervals[i] = [lefti, righti]` describes the *i*th interval starting at `lefti` and ending at `righti` (inclusive). The size of an interval is defined as the number of integers it contains, or more formally `righti - lefti + 1`.

You are also given an integer array `queries`. The answer to the *j*th query is the size of the smallest interval *i* such that `lefti ≤ queries[j] ≤ righti`. If no such interval exists, the answer is `-1`.

Return an array containing the answers to the queries.

Example 1:

Round 9 · Low · Hard

p.3516

Input: intervals = [[1,4],[2,4],[3,6],[4,4]], queries = [2,3,4,5]

Output: [3,3,1,4]

Explanation: The queries are processed as follows:

- Query = 2: The interval [2,4] is the smallest interval containing 2. The answer is $4 - 2 + 1 = 3$.
- Query = 3: The interval [2,4] is the smallest interval containing 3. The answer is $4 - 2 + 1 = 3$.
- Query = 4: The interval [4,4] is the smallest interval containing 4. The answer is $4 - 4 + 1 = 1$.
- Query = 5: The interval [3,6] is the smallest interval containing 5. The answer is $6 - 3 + 1 = 4$.

Example 2:

Input: intervals = [[2,3],[2,5],[1,8],[20,25]], queries = [2,19,5,22]

Output: [2,-1,4,6]

Explanation: The queries are processed as follows:

- Query = 2: The interval [2,3] is the smallest interval containing 2. The answer is $3 - 2 + 1 = 2$.
- Query = 19: None of the intervals contain 19. The answer is -1.
- Query = 5: The interval [2,5] is the smallest interval containing 5. The answer is $5 - 2 + 1 = 4$.
- Query = 22: The interval [20,25] is the smallest interval containing 22. The answer is $25 - 20 + 1 = 6$.

Constraints:

- $1 \leq \text{intervals.length} \leq 105$
- $1 \leq \text{queries.length} \leq 105$
- $\text{intervals}[i].\text{length} == 2$
- $1 \leq \text{lefti} \leq \text{righti} \leq 107$
- $1 \leq \text{queries}[j] \leq 107$

Community Solutions (Python)

• LeetCode

```
class Solution:
    def minInterval(self, intervals: List[List[int]], queries: List[int]) -> List[int]:
        n, m = len(intervals), len(queries)
        intervals.sort()
        queries = sorted((x, i) for i, x in enumerate(queries))
        ans = [-1] * m
        pq = []
        i = 0
        for x, j in queries:
            while i < n and intervals[i][0] <= x:
                a, b = intervals[i]
                heappush(pq, (b - a + 1, b))
                i += 1
            while pq and pq[0][1] < x:
                heappop(pq)
            if pq:
                ans[j] = pq[0][0]
        return ans
```

• SimplyLeet

```
import heapq

def minInterval(intervals, queries):
    # Sort intervals by their starting point.
    intervals.sort(key=lambda itv: itv[0])

    # Prepare queries sorted by their value, keeping track of original index.
    sorted_queries = sorted([(q, i) for i, q in enumerate(queries)])

    result = [-1] * len(queries)
```

```
min_heap = []
i = 0 # for intervals

# Process each query in ascending order.
for q, qi in sorted_queries:
    # Add all intervals that start <= current query.
    while i < len(intervals) and intervals[i][0] <= q:
        start, end = intervals[i]
        # Compute the size of the interval.
        size = end - start + 1

        # Push tuple: (size, end) into the min-heap.
        heapq.heappush(min_heap, (size, end))
        i += 1

    # Remove intervals that do not cover the current query.
    while min_heap and min_heap[0][1] < q:
        heapq.heappop(min_heap)

    # The top of the heap (if any) is the smallest interval covering the
    query.
    if min_heap:
        result[qi] = min_heap[0][0]
    return result

# Example usage
intervals = [[1,4],[2,4],[3,6],[4,4]]
queries = [2,3,4,5]
print(minInterval(intervals, queries)) # Output: [3, 3, 1, 4]
```